控制大综合

第60章 反步法与反馈线性化

- 60.1 动机与直观理解
- 60.2 数学原理
- 60.3 设计步骤
- 60.4 连续情形
- 60.5 离散情形
- 60.6 稳定性与性能
- 60.7 参数意义与作用
- 60.8 调参方法与经验
- 60.9 优化与改进
- 60.10 工程落地
- 60.11 优缺点与适用边界
- 60.12 Python 仿真代码

第61章 自适应控制

- 61.1 动机与直观理解
- 61.2 数学原理
- 61.3 设计步骤
- 61.4 连续情形
- 61.5 离散情形
- 61.6 稳定性与性能
- 61.7 参数意义与作用
- 61.8 调参方法与经验
- 61.9 优化与改进
- 61.10 工程落地
- 61.11 优缺点与适用边界
- 61.12 Python 仿真代码

第 62 章 随机控制基础

- 62.1 动机与直观理解
- 62.2 数学原理
- 62.3 设计步骤
- 62.4 连续情形
- 62.5 离散情形

- 62.6 稳定性与性能
- 62.7 参数意义与作用
- 62.8 调参方法与经验
- 62.9 优化与改进
- 62.10 工程落地
- 62.11 优缺点与适用边界
- 62.12 Python 仿真代码

第 63 章 鲁棒控制进阶

- 63.1 动机与直观理解
- 63.2 数学原理
- 63.3 设计步骤
- 63.4 连续情形
- 63.5 离散情形
- 63.6 稳定性与性能
- 63.7 参数意义与作用
- 63.8 调参方法与经验
- 63.9 优化与改进
- 63.10 工程落地
- 63.11 优缺点与适用边界
- 63.12 Python 仿真代码

第 64 章 最优控制进阶

- 64.1 动机与直观理解
- 64.2 数学原理
- 64.3 设计步骤
- 64.4 连续情形
- 64.5 离散情形
- 64.6 稳定性与性能
- 64.7 参数意义与作用
- 64.8 调参方法与经验
- 64.9 优化与改进
- 64.10 工程落地
- 64.11 优缺点与适用边界
- 64.12 Python 仿真代码

第 65 章 线性矩阵不等式（LMI）与凸优化控制

- 65.1 动机与直观理解
- 65.2 数学原理

- 65.3 设计步骤
- 65.4 连续情形
- 65.5 离散情形
- 65.6 稳定性与性能
- 65.7 参数意义与作用
- 65.8 调参方法与经验
- 65.9 优化与改进
- 65.10 工程落地
- 65.11 优缺点与适用边界
- 65.12 Python 仿真代码

第66章 专家控制

- 66.1 动机与直观理解
- 66.2 数学原理
- 66.3 设计步骤
- 66.4 连续情形
- 66.5 离散情形
- 66.6 稳定性与性能
- 66.7 参数意义与作用
- 66.8 调参方法与经验
- 66.9 优化与改进
- 66.10 工程落地
- 66.11 优缺点与适用边界
- 66.12 Python 仿真代码

第67章 模糊控制

- 67.1 动机与直观理解
- 67.2 数学原理
- 67.3 设计步骤
- 67.4 连续情形
- 67.5 离散情形
- 67.6 稳定性与性能
- 67.7 参数意义与作用
- 67.8 调参方法与经验
- 67.9 优化与改进
- 67.10 工程落地
- 67.11 优缺点与适用边界
- 67.12 Python 仿真代码

第68章 神经网络控制

- 68.1 动机与直观理解
- 68.2 数学原理
- 68.3 设计步骤
- 68.4 连续情形
- 68.5 离散情形
- 68.6 稳定性与性能
- 68.7 参数意义与作用
- 68.8 调参方法与经验
- 68.9 优化与改进
- 68.10 工程落地
- 68.11 优缺点与适用边界
- 68.12 Python 仿真代码

第69章 迭代学习控制（ILC）

- 69.1 动机与直观理解
- 69.2 数学原理
- 69.3 设计步骤
- 69.4 连续情形
- 69.5 离散情形
- 69.6 稳定性与性能
- 69.7 参数意义与作用
- 69.8 调参方法与经验
- 69.9 优化与改进
- 69.10 工程落地
- 69.11 优缺点与适用边界
- 69.12 Python 仿真代码

第70章 强化学习控制

- 70.1 动机与直观理解
- 70.2 数学原理
- 70.3 设计步骤
- 70.4 连续情形
- 70.5 离散情形
- 70.6 稳定性与性能
- 70.7 参数意义与作用
- 70.8 调参方法与经验
- 70.9 优化与改进

- 70.10 工程落地
- 70.11 优缺点与适用边界
- 70.12 Python 仿真代码

第71章 智能控制中的稳定性与可解释性

- 71.1 动机与直观理解
- 71.2 数学原理
- 71.3 设计步骤
- 71.4 连续情形
- 71.5 离散情形
- 71.6 稳定性与性能
- 71.7 参数意义与作用
- 71.8 调参方法与经验
- 71.9 优化与改进
- 71.10 工程落地
- 71.11 优缺点与适用边界
- 71.12 Python 仿真代码

第60章 反步法与反馈线性化

60.1 动机与直观理解

第 59 章的滑模靠“把方向定死”换取抗扰，代价是控制量必须抖。本章换一条思路：不抖，也不解优化，而是**沿着对象的积分链一级一级往回设计**，让每一步都留下一个可以被下一级消掉的“欠账”。这条思路叫反步法（backstepping）。与它常常并称的是反馈线性化（feedback linearization）：先想办法把非线性对象“掰”成一个线性对象，再套用线性设计的全部结论。

两者都不靠“压”，而靠“算”：把非线性项显式地写出来，用控制量把它减掉。所以它们的性能上限比滑模高——标称工况下本章反步法的调节时间是 1.96 s，滑模在第 59 章同一类对象上是 2.53 s——但它们的命门也很清楚：**减错了就补不回来**。本章实测，a 偏 30% 时反步法的稳态误差是 0.21 m，而加了积分项之后是 1.17×10^{-6} m。这中间差着五个数量级，正好把“模型精度要求”这句话量化了。

60.1.1 两个绕不开的难点

第一个难点是**非匹配非线性**。PID 与滑模都默认“控制量能直接影响你想控制的那个量”，即所谓匹配条件。但真实对象里，非线性往往出现在控制量进不去的地方。本章的对象就是一个例子：

$$\dot{x}_1 = x_2 + a\sin x_1, \qquad \dot{x}_2 = -b x_2 + c\tanh x_1 + u \tag{60-1}$$

控制量 u 只出现在 $\dot{x}_2$ 里，而 $a\sin x_1$ 这一项出现在 $\dot{x}_1$ 里。想把它减掉，只能通过 x_2 间接地减——这就逼出了“虚拟控制”的概念：把 x_2 当成一个可以下令的输入，先算出“如果 x_2 听话，它应该是多少”，再让真正的 u 去逼 x_2 追上这个值。

第二个难点是**模型精度**。式 (60-1) 里的 a、b、c 是标称值，实际对象总有偏差。滑模对匹配扰动有不变性，反步与反馈线性化没有：它们把非线性项**减掉**，减错了，残差就原封不动地留在闭环里，变成稳态误差。本章 §60.6 会把这句话做成一张表。

60.1.2 本章对象与它的坏脾气

式 (60-1) 在标称参数 $a = 1.0$、$b = 0.5$、$c = 1.5$ 下，平衡点是原点。在原点线性化：

$$A = \begin{bmatrix} 1 & 1 \\ 1.5 & -0.5 \end{bmatrix}, \qquad \det(\lambda I - A) = \lambda^2 - 0.5\lambda - 2 \tag{60-2}$$

两个特征根是

$$\lambda_1 = 2.0, \qquad \lambda_2 = -1.5 \tag{60-3}$$

一正一负——**开环是鞍点**。这意味着不加控制时，绝大多数初始状态都会发散；同时它也意味着“把状态拉回原点”不是靠耗散，而是靠控制器主动对抗一个本就不稳定的模态。这一点很重要：它让“控制器算错了会怎样”这件事暴露得非常快，不会像渐近稳定对象那样被对象的自稳定性掩盖。

图 60-2 是开环相图。灰色箭头是 $u = 0$ 时的向量场，可以清楚看到一条“分界线”：分界线一侧的状态被推向 $x_1 \to +\infty$，另一侧被推向 $x_1 \to -\infty$。两条彩色曲线是闭环轨迹，从 $x_1 = -1.5$ 出发，都穿过这条危险的分界区，最后收敛到 $(1, 0)$。

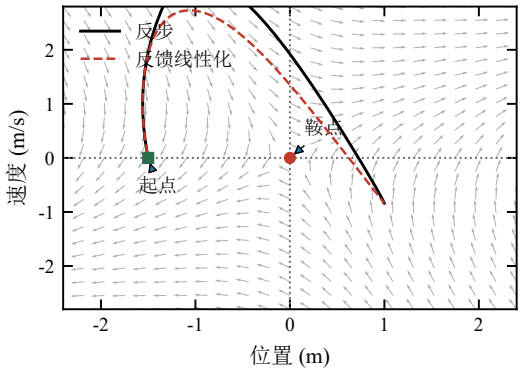


图 60-2　开环相图（鞍点）与两条闭环轨迹

鞍点对象还有一个副作用：它让**大初始偏差**成为一项有意义的考核。表 60-1 列出了本章的全部仿真设置，其中初始状态在 §60.6 会被单独改成 $(-1.5, 0)$ 做一次“恢复实验”，用来对比三种方法在远离平衡点时的行为。

项目	取值	说明
对象参数	$a = 1.0$、$b = 0.5$、$c = 1.5$	标称值，失配实验在此基础上偏 ±30%
初始状态	$x_1(0) = 0$、$x_2(0) = 0$	恢复实验改用 $x_1(0) = -1.5$
参考输入	阶跃 1 m；斜坡 0.3 m/s	斜坡用于考察跟踪滞后
采样周期	$T_s = 0.01$ s	与第 59 章一致，便于横向比较
积分方法	每拍 RK4 细分 4 步	对象是非线性 ODE，不能用 ZOH
执行器限幅	$\lvert u\rvert \le 10$	另设 2/3/5 三档考察饱和

60.1.3 反步法的三句话

反步法的推导可以压缩成三句话：

1. **把误差的定义往上游挪一级**。先定义 $e_1 = x_1 - r$，希望它按 $\dot e_1 = -k_1 e_1$ 衰减。由式 (60-1) 得 $\dot e_1 = x_2 + a\sin x_1 - \dot r$，于是“希望”就变成了对 x_2 的一个要求：

$$x_2 \overset{!}{=} -k_1 e_1 - a\sin x_1 + \dot r \triangleq \alpha_1 \tag{60-4}$$

   α_1 叫**虚拟控制**（virtual control）。它不是真正的输入，而是一个“如果 x_2 是输入，它该取多少”的中间指令。

2. **承认 x_2 不会听话，把差距定义成第二级误差**：$e_2 = x_2 - \alpha_1$。于是 $\dot e_1 = -k_1 e_1 + e_2$，第一级误差的导数里多出一个“欠账” e_2。

3. **让第二级把这个欠账还掉**。对 e_2 求导，u 就在里面，直接令 $\dot e_2 = -k_2 e_2 - e_1$，解出 u 即可。

   三步做完，闭环误差方程就是

$$\dot e_1 = -k_1 e_1 + e_2, \qquad \dot e_2 = -e_1 - k_2 e_2 \tag{60-5}$$

这是一个线性系统，特征多项式 $\lambda^2 + (k_1 + k_2)\lambda + (1 + k_1 k_2)$。只要 $k_1, k_2 > 0$，两个根都在左半平面，误差指数收敛到零。**反步法的全部魔力就在式 (60-5)：它把非线性对象变成了一个线性误差系统，而且不要求非线性项可逆、不要求它小。**

60.1.4 反馈线性化的三句话

反馈线性化的思路更直接：

1. 对输出 $y = x_1$ 连续求导，直到 u 出现。第一次求导得 $\dot y = x_2 + a\sin x_1$，u 还没出现；第二次得

$$\ddot y = -b x_2 + c\tanh x_1 + u + a\cos x_1\,(x_2 + a\sin x_1) \tag{60-6}$$

   u 出现了，且系数为 1，相对阶为 2。

2. 令 $\ddot y = v$，反解出

$$u = v + b x_2 - c\tanh x_1 - a\cos x_1 (x_2 + a\sin x_1) \tag{60-7}$$

3. 对 v 用线性设计：$v = \ddot r - k_p e - k_d(\dot y - \dot r)$，$e = y - r$。代回后得到 $\ddot e + k_d \dot e + k_p e = 0$，误差按二阶线性系统收敛。

反步与反馈线性化在这里其实是**同一件事的两种写法**：式 (60-7) 里的 $-c\tanh x_1 - a\cos x_1(x_2 + a\sin x_1)$ 是“抵消项”，式 (60-4) 里的 $-a\sin x_1$ 也是“抵消项”；区别在于反步只抵消到第一级、把剩下的交给第二级的 k_2 去消化，反馈线性化则一次性把二阶导数全部摊平。这个差别决定了它们对模型误差的敏感度不同：反步的抵消项只出现在第一级，误差被第二级的高增益“盖住”一部分；反馈线性化的抵消项直接进 u，误差没有任何遮挡。

60.1.5 为什么不直接用高增益 PID

本章的 PID 基线是网格寻优得到的 $(k_p, k_i, k_d) = (40, 4, 10)$——在“$t_s < 8$ s 且超调 $< 15\%$”的约束下 IAE 最小的一组。它的标称表现是：调节时间 4.33 s、超调 3.80%、IAE 0.6731、稳态误差 0.0168 m。反步法是 1.96 s、0.01%、0.6983、1.79×10^{-13} m。

两个数字值得注意：

- **调节时间差一倍多**。PID 不是不想快，是它的增益被超调约束住了。对象开环不稳定，PID 要提高带宽就得提高增益，提高增益就超调；反步法用 e_1 的符号进第二级（式 (60-5) 里的 $-e_1$ 项），给误差方程配了一对可以独立 placement 的极点，因此能在不超调的前提下把带宽做上去。
- **稳态误差差十一个数量级**。PID 的 0.0168 m 来自积分器在限幅边缘的残余；反步法的 10^{-13} 是双精度浮点的舍入量级——在标称模型下它是**精确**的，不是“很小”。

但 PID 也有赢的地方：斜坡跟踪的滞后是 0.0012 m，比不带前馈的反步法（0.2112 m）好两个数量级。原因很简单，PID 的积分器在持续爬坡，天然记住了斜坡；反步法没有积分，只能靠 r_d 前馈（§60.6 会看到，加上前馈后滞后降到 0.0826 m，仍不如 PID）。**这就是“结构换性能”的代价：反步法的性能是被设计出来的，没设计到的通道它就是零分。**

60.1.6 与相邻章的分工

- 与第 59 章滑模：滑模用不连续切换换取对匹配扰动的硬不变性，反步用连续控制换取更高的标称带宽。两者可以叠加——用反步设计“名义控制”，用滑模项压残余扰动，就是 §60.9 的“反步 + 滑模”混合结构。
- 与第 58 章 MPC：MPC 处理约束，反步不处理约束。本章实测 $u_{\max} = 2$ 时反步法的超调从 0.01% 涨到 29.70%（§60.6），因为虚拟控制 α_1 是按“u 无限大”算出来的，限幅一进来，e_2 就追不上 α_1，误差方程 (60-5) 的前提被破坏。约束下的反步要另做“指令滤波 + 抗饱和”，见 §60.9。

- 与第 61 章自适应：反步的模型误差问题，标准解法是在线估参数，把“减错了”变成“边减边学”。本章只给出误差的量级，参数怎么估留给第 61 章。
- 与第 44/106 章 PID：本章的 PID 基线沿用那两章的整定口径（微分作用于测量、未饱和才积分），指标定义也一致，可以直接横向比较。

60.1.7 四个指标与一条口径

本章所有表格用同一套指标，定义与第 58、59 章一致：

- **调节时间** t_s：位置最后一次越出 $\pm 2\%$ 参考值之后的那一拍；
- **超调**：峰值相对稳态值的百分比；
- **IAE**：$\int_0^T |e(t)|\,\mathrm{d}t$，全程误差绝对值的积分；
- **控制总变差 TV**：$\sum_k |u_{k+1}-u_k|$，衡量控制量的“折腾程度”，也是执行器磨损的代理指标。

另外两个只在本章出现的量：**稳态误差** $|e_{ss}|$ 取末 1 s 的平均偏差，**尾段纹波** ripple_y 取末 1 s 内位置的峰峰值。前者看“算得准不准”，后者看“抖不抖”。

60.1.8 本章路线

§60.2 推导反步的递推公式与反馈线性化的坐标变换；§60.3 给出可照着做的设计步骤与结构图；§60.4 讲连续实现，重点是微分爆炸；§60.5 讲离散实现与采样周期的关系；§60.6 用六组实验对照 PID、反馈线性化、积分反步；§60.7 讲 k_1、k_2、τ、k_i 四个参数的物理意义；§60.8 给出整定顺序与五轮调试记录；§60.9 介绍动态面、积分反步、指令滤波与混合结构；§60.10 讲工程落地（模型标定、微分获取、饱和处理）；§60.11 给出选型边界；§60.12 给出可复现代码。

如果只记一句话：**反步法把“控制一个高阶非线性对象”拆成“连续控制一串一阶子系统”，每一步都只还上一级留下的欠账；它算得越准，性能越好，也算错得越狠，性能越差。**

60.2 数学原理

60.2.1 严格反馈形式

反步法不是万能的，它要求对象能写成一条“链”：第 i 个状态方程的右端，只允许出现前 i 个状态，以及第 $i+1$ 个状态的一次项。这叫**严格反馈形式**（strict-feedback form）：

$$
\begin{aligned}
\dot{x}_1 &= x_2 + \varphi_1(x_1) \\
\dot{x}_2 &= x_3 + \varphi_2(x_1, x_2) \\
&\vdots \\
\dot{x}_n &= \varphi_n(x_1, \dots, x_n) + b\,u
\end{aligned}
\tag{60-8}
$$

其中 φ_i 是已知的光滑非线性函数，$b \neq 0$ 是输入增益。注意三点：控制量只出现在最后一式；每一式的“下一级状态”系数为 1（若不为 1，先做归一化）；非线性项 φ_i 不许含 x_{i+1} 及其后的状态。

本章对象式 (60-1) 正好是这个形式，$n=2$、$\varphi_1 = a\sin x_1$、$\varphi_2 = -bx_2 + c\tanh x_1$、$b=1$。写不成这个形式的对象，标准反步法不能直接用：

- 若 $\dot{x}_1 = x_2^2 + \cdots$（下一级状态带平方），要做变量替换 $z_2 = x_2^2$ 并处理符号分支；

- 若 $\dot{x}_1 = x_2 + x_3$（跳级），需要“多输入反步”或先把对象重排；
- 若控制量以 $|u|u$ 或饱和形式出现，b 依赖 u，属于非仿射情形，要改用“动态扩展”（把 u 的积分当成新状态）。

严格反馈形式看起来限制很强，但工程上大量机电对象天生就是这个形状：电流环→速度环→位置环的级联、无人机的位置→速度→姿态、化工里的串联反应釜，都是“一级推一级”。反步法本质上是**把级联结构的物理直觉变成了系统化的设计公式**。

60.2.2 递推公式

设参考信号 r 足够光滑。定义误差

$$e_1 = x_1 - r, \qquad e_i = x_i - \alpha_{i-1} \quad (i = 2, \dots, n) \tag{60-9}$$

虚拟控制按同一套模式递推：

$$\alpha_1 = -k_1 e_1 - \varphi_1 + \dot{r} \tag{60-10}$$

$$\alpha_i = -k_i e_i - e_{i-1} - \varphi_i + \dot{\alpha}_{i-1}, \qquad i = 2, \dots, n-1 \tag{60-11}$$

$$u = \frac{1}{b}\left(-k_n e_n - e_{n-1} - \varphi_n + \dot{\alpha}_{n-1}\right) \tag{60-12}$$

三个 $k_i > 0$ 是设计参数。式 (60-10)–(60-12) 里每一项都有明确来处，表 60-2 把它们的职责分开列出：

项	作用	缺了会怎样
$-k_i e_i$	本级误差的比例反馈	本级误差不衰减，闭环极点跑向右半平面
$-e_{i-1}$	偿还上一级留下的欠账	上一级误差只能靠本级“顺带”收敛，速度慢一个量级
$-\varphi_i$	抵消本级非线性	非线性项变成未建模扰动，稳态误差按 §60.2.8 的公式出现
$+\dot{\alpha}_{i-1}$	补偿虚拟控制的变化率	相当于给本级加了一个随动的偏置，e_i 恒有稳态偏差

表 60-2 是读文献时最该盯住的地方：**很多论文的“改进反步”其实就是往这四项里加东西**（加积分项、加扰动观测器、把 $\dot{\alpha}$ 换成滤波值）。

60.2.3 两级的显式推导

把式 (60-8) 取 $n = 2$ 展开，一步步做：

第一级：$\dot{e}_1 = \dot{x}_1 - \dot{r} = x_2 + a\sin x_1 - \dot{r}$。若希望 $\dot{e}_1 = -k_1 e_1$，则要求

$$x_2 = -k_1 e_1 - a\sin x_1 + \dot{r} \triangleq \alpha_1 \tag{60-13}$$

第二级：$e_2 = x_2 - \alpha_1$，于是

$$\dot{e}_1 = -k_1 e_1 + e_2 \tag{60-14}$$

$\dot{e}_2 = \dot{x}_2 - \dot{\alpha}_1 = -b x_2 + c\tanh x_1 + u - \dot{\alpha}_1$。令 $\dot{e}_2 = -k_2 e_2 - e_1$，解出

$$u = -k_2 e_2 - e_1 + b x_2 - c\tanh x_1 + \dot{\alpha}_1 \tag{60-15}$$

式 (60-15) 与式 (60-12) 完全一致（$b = 1$、$\varphi_2 = -b x_2 + c\tanh x_1$）。$\dot{\alpha}_1$ 由式 (60-13) 对时间求导得到：

$$\dot{\alpha}_1 = -(k_1 + a\cos x_1)(x_2 + a\sin x_1) + \ddot{r} \tag{60-16}$$

式 (60-16) 是整章的关键公式，因为它同时是**反步法的力量**与**反步法的麻烦**：力量在于它把 α_1 的变化率精确算出来了，第二级可以完全跟上；麻烦在于它需要 $\dot{x}_1$ 的解析式，也就是需要对象模型**逐项正确**，还需要 x_2 的实时值。一旦 x_2 是从位置差分出来的，噪声就被 $-(k_1 + a\cos x_1)$ 这个因子放大后直接灌进 u。§60.4 会看到，这正是“微分爆炸”的来源，也是动态面（DSC）要解决的问题。

60.2.4 Lyapunov 证明与收敛速度

取

$$V = \tfrac{1}{2}(e_1^2 + e_2^2) \tag{60-17}$$

沿闭环求导：

$$\dot{V} = e_1\dot{e}_1 + e_2\dot{e}_2 = e_1(-k_1 e_1 + e_2) + e_2(-e_1 - k_2 e_2) = -k_1 e_1^2 - k_2 e_2^2 \tag{60-18}$$

两个交叉项 $e_1 e_2$ 精确抵消——这不是巧合，而是式 (60-11) 中 $-e_{i-1}$ 项的**唯一目的**。由此得

$$V(t) \le V(0)\, e^{-2\min(k_1,k_2)\,t} \tag{60-19}$$

即误差范数至少按 $\min(k_1,k_2)$ 的速率指数衰减。这把“稳定性”变成了一个可以直接算的数：本章 $k_1 = 2$、$k_2 = 3$，理论上误差的时间常数不快于 $1/2 = 0.5$ s，从 1 m 衰减到 2% 需要 $-\ln(0.02)/2 \approx 1.96$ s——实测调节时间正好是 1.96 s。

式 (60-19) 也给了一个反直觉的结论：**闭环速度由较小的那个增益决定，而不是较大的那个**。§60.6 的扫描数据会印证这一点：k_1 从 2 加到 8，调节时间只从 1.96 s 降到 1.42 s；而 IAE 从 0.6983 降到 0.4629，改善明显。原因是 k_1 大时虽然速度受 $k_2 = 3$ 限制，但第一级的“欠账”更小，第二级不用花力气去追，全程误差积分自然下降。

60.2.5 相对阶与 Lie 导数

反馈线性化的第一步是判断“要对输出求几次导，u 才会出现”。形式化的说法是**相对阶**（relative degree）：对仿射非线性对象

$$\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{g}(\boldsymbol{x})u, \qquad y = h(\boldsymbol{x}) \tag{60-20}$$

若 $L_g L_f^{i-1} h(\boldsymbol{x}) = 0$（$i = 1,\dots,\rho-1$）而 $L_g L_f^{\rho-1} h(\boldsymbol{x}) \neq 0$，则相对阶为 ρ。其中 $L_f h = \partial h/\partial \boldsymbol{x}\cdot\boldsymbol{f}$ 是 Lie 导数。

对本章对象：$h = x_1$，$L_g h = 0$（u 不在 $\dot{x}_1$ 里）；$L_f h = x_2 + a\sin x_1$；$L_g L_f h = \partial(L_f h)/\partial x_2 \cdot 1 = 1 \neq 0$。所以 $\rho = 2 = n$，属于“相对阶等于阶数”的情形——这是最理想的一类，可以做**精确线性化**，不会剩下内动态。

若 $\rho < n$，线性化之后还剩 $n-\rho$ 维**内动态**（internal dynamics），其稳定性必须单独验证；内动态不稳定的对象叫“非最小相位”，反馈线性化会把它藏起来，然后在外表现为“输出跟得很好、内部已经发散”。这类对象在工程上并不罕见（倒立摆的台车位置输出、锅炉的汽包水位），是反馈线性化最容易翻车的地方。

60.2.6 精确线性化的坐标变换

取新坐标 $z_1 = y = x_1$、$z_2 = \dot{y} = x_2 + a\sin x_1$，则

$$\dot z_1 = z_2, \qquad \dot z_2 = \alpha(\boldsymbol{x}) + \beta(\boldsymbol{x})u \tag{60-21}$$

其中 $\beta = 1$、$\alpha = -bx_2 + c\tanh x_1 + a\cos x_1(x_2 + a\sin x_1)$。令

$$u = \frac{v - \alpha(\boldsymbol{x})}{\beta(\boldsymbol{x})} = v + bx_2 - c\tanh x_1 - a\cos x_1(x_2 + a\sin x_1) \tag{60-22}$$

就得到 $\ddot z_1 = v$，一个双积分器。再令 $v = \ddot r - k_p e - k_d(\dot y - \dot r)$，$e = z_1 - r$，得

$$\ddot e + k_d \dot e + k_p e = 0 \tag{60-23}$$

特征多项式 $s^2 + k_d s + k_p$，本章 $k_p = k_d = 4$，两个根都是 -2（临界阻尼）。这就是为什么反馈线性化的阶跃响应**一点超调都没有**：不是整定得好，而是极点被放在实轴上了。

式 (60-22) 与反步的式 (60-15) 形式上差别很大，但把它们并排放：

$$\begin{aligned} u_{\text{BST}} &= -k_2 e_2 - e_1 + bx_2 - c\tanh x_1 + \dot\alpha_1 \\ u_{\text{FBL}} &= -k_p e - k_d \dot y + bx_2 - c\tanh x_1 - a\cos x_1(x_2 + a\sin x_1) \end{aligned} \tag{60-24}$$

匹配项 $bx_2 - c\tanh x_1$ 完全相同；差别全在“非匹配项怎么处理”：反步把 $-a\sin x_1$ 放进 α_1，通过 $\dot\alpha_1$ 间接带进 u，误差最终由二阶线性系统 (60-5) 收敛；反馈线性化直接对 $a\sin x_1$ 求导得到 $a\cos x_1 \dot x_1$，一次性减掉。**反步是“分级抵消”，反馈线性化是“一级清零”。**

60.2.7 模型误差的解析式（预告）

把控制器里的 a、c 换成估计值 a_m、c_m，令 $\Delta a = a_m - a$、$\Delta c = c_m - c$，在稳态（$\dot x_1 = \dot x_2 = 0$）解闭环方程，可以得到两个非常干净的公式。

反步法：

$$e_{1,\text{BST}}\big|_{\text{ss}} = -\frac{\Delta a \sin x_1\,(k_1 + k_2 + a_m\cos x_1)}{1 + k_1 k_2} - \frac{\Delta c\tanh x_1}{1 + k_1 k_2} \tag{60-25}$$

反馈线性化：

$$e_{1,\text{FBL}}\big|_{\text{ss}} = -\frac{\Delta a \sin x_1\,(k_d + a_m\cos x_1)}{k_p} - \frac{\Delta c\tanh x_1}{k_p} \tag{60-26}$$

两个式子的结构一致：分子是“没抵消干净的那部分非线性”，分母是“闭环刚度”。反步的刚度是 $1 + k_1 k_2$，反馈线性化是 k_p。本章 $1 + k_1 k_2 = 7$、$k_p = 4$，所以同样的参数偏差，反馈线性化的稳态误差必然更大——实测是 0.303 m 对 0.210 m（a 偏 -30%），比值 1.44，与 $7/4 = 1.75$ 同量级（差异来自 x_1 的稳态位置不同，$\sin x_1$ 与 $\cos x_1$ 取值略有差别）。

这个公式还解释了一件工程上很重要的事：**提高 $k_1 k_2$ 或 k_p 都能压误差，但压的是“除以刚度”的那部分，压不到零。**要压到零，只能让 Δa、Δc 本身趋向零（更准的模型，或第 61 章的自适应），或者引入积分项（§60.9）。本章实测，积分反步把 a 偏 -30% 的稳态误差从 0.210 m 压到 1.17×10^{-6} m，代价是超调从 0 涨到 16.52%。

60.2.8 三个前提与一条隐含假设

反步与反馈线性化的推导都建立在四条前提上，缺一条结论就要改：

1. **仿射**：u 以一次项、且系数与 u 无关地出现。气动系统的质量流量与阀开度是平方关系、开关电源的占空比与输出电压是分段关系，都不满足，需要先做输入变换。
2. **全状态可测**：式 (60-15) 用到 x_2，式 (60-16) 用到 $\dot{x}_1$。若只有位置可测，必须先加观测器，而观测误差会通过 $\dot{\alpha}_1$ 直接进 u——这是 §60.10 要处理的问题。
3. **相对阶确定且等于阶数**（反馈线性化）或**严格反馈形式成立**（反步）。执行器动态、传感器滤波都会把相对阶抬高，把二阶对象变成三阶，此时要么把滤波器建模进对象，要么接受“设计模型与实际不符”。
4. **隐含假设：b 的符号已知且不为零**。b 过零意味着控制方向翻转，式 (60-12) 会发散。工程上要在投运前确认工作区间内 b 的符号不变。

这四条里，第一条是结构性的（不满足就没法用），后三条是精度性的（不满足还能用，但要接受性能退化）。分清这一点，能避免在选型阶段就把方法用错。

60.2.9 反步与反馈线性化到底差在哪

两章内容放在一起读，很容易产生一个疑问：既然反步的最后一步（式 60-14）与反馈线性化（式 60-22）都写了同一串抵消项，为什么闭环极点不同？答案藏在中间那一级。

反馈线性化只有一次抵消。它把输出 $y = x_1$ 求导两次，得到 $\ddot{y} = L_f^2 h + L_g L_f h \cdot u$，然后令 $u = \big(v - L_f^2 h\big)/L_g L_f h$，于是 $\ddot{y} = v$。整条路径里，y 的二阶导数被一次性“拉平”成 v，中间没有引入任何新的反馈。

反步则把这两次求导拆成两段。第一段只看 $\dot{x}_1$，把 x_2 当成可任意指定的输入，指定成 $\alpha_1 = -k_1 e_1 - a\sin x_1 + \dot{r}$；第二段才去实现这个 α_1，并且**顺手把第一级的误差 e_1 也塞进了 u**（式 60-14 的 $-e_1$ 项）。正是这个 $-e_1$，让闭环刚度从 k_p 变成 $1 + k_1 k_2$。

用一句话概括：**反馈线性化是“把非线性消掉，再线性设计”；反步是“边消边设计，每一级都加一层稳定化”**。前者只要一次抵消，后者要做 n 次，每次顺带引入一个交叉项。两次方法在标称下都得到二阶线性系统，但系数不同——反步多出来的那个 1（或者说多出来的交叉刚度）就是它“多干了一次活”的报酬。

这个差别在数学上体现为一件事：**反步把相对阶条件变成了“构造过程中自动满足”的东西**。反馈线性化必须先验证 $\rho = n$，否则会出现内动态；反步只要严格反馈形式成立，逐级构造下来 ρ 必然等于 n，不需要单独验证。这就是为什么反步的适用面看起来更宽——它不是条件更松，而是把条件的检查“内置”到了构造里。

代价也有两处。**第一，中间级不可跳过**：反馈线性化可以跳过中间变量直接写 u，反步必须把 α_1 显式写出来，于是就有了 $\dot{\alpha}_1$ 这个麻烦（§60.4 的全部内容都在处理它）。**第二，级数越多越啰嗦**：三级反步要写两个虚拟控制量与两个导数，反馈线性化仍是一次抵消，只是求导阶数更高。

理解了这一层，再回头看表 60-17 就不会困惑：两者在标称工况下差距不大（t_s 1.96 对 2.90 s），差距主要在失配与饱和这些“结构显形”的场合。**方法之间的差别，永远在条件不理想时才被放大。**

60.3 设计步骤

60.3.1 五步流程

把 §60.2 的推导整理成可执行的流程，一共五步。这五步的顺序不能颠倒，因为每一步的输入都是上一步的输出：

1. **形式检查**：对象能否写成式 (60-8)？不能就停，先做坐标变换或输入变换；
2. **定输出与相对阶**：输出取什么、相对阶是几、有没有内动态；
3. **逐级造误差**：从 e_1 开始，写出 α_1，再写 e_2、α_2，直到 u；

4. **选增益**：把闭环误差系统的极点放到想要的位置；

5. **实现校验**：求导怎么来、限幅怎么办、初值怎么给，最后跑一遍实测。

下面按这五步展开。附带说明一句：**第 3 步是最容易出错的一步，第 4 步是最容易被忽视的一步**。前者错在符号与项数，后者错在“随便给一组 k 就上线”。

60.3.2 第一步：形式检查

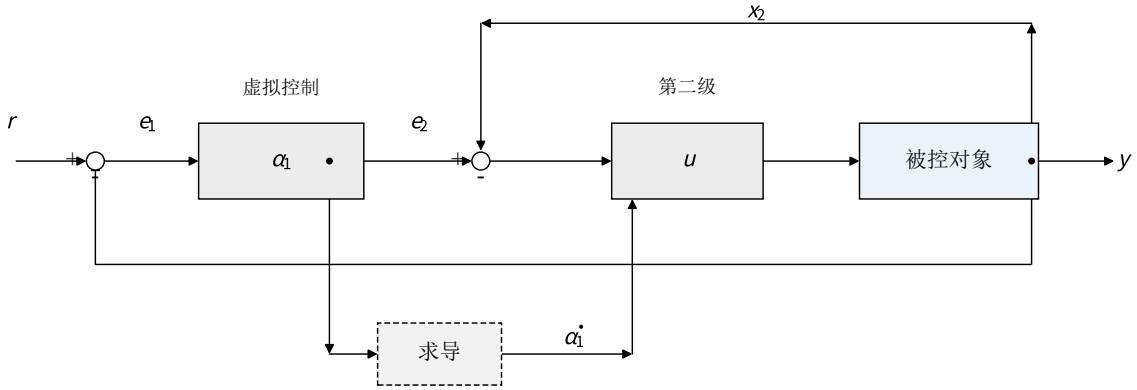
表 60-3 是一张检查清单，逐条打勾即可。它的用处不是“判断能不能用”，而是“提前知道要补什么”。

检查项	通过条件	不通过时的补救
仿射性	u 以一次项出现，系数不含 u	输入变换、动态扩展
链式结构	$\dot{x}_i$ 只含 $x_1..x_{i+1}$	状态重排、部分反馈线性化
下一级系数	x_{i+1} 的系数为常数 1	除以该系数（需符号恒定）
非线性项已知	φ_i 有解析式或可查表	神经网络/多项式拟合（第 62 章）
状态可测	x_2 可直接测量或可靠观测	加观测器，或改用动态面
输入增益	$b \neq 0$ 且符号不变	分段设计，或改用滑模

清单里最后两条最常被忽略。它们不影响“能不能推公式”，只影响“推出来的公式能不能用”。

60.3.3 第二步：结构图

图 60-1 是两级反步的信号流。与常规反馈最大的不同是**中间多了一个“虚拟控制”通道**：α_1 不但是第二级的参考，还要经过一次求导后作为前馈加进 u。图中虚线框的“求导”就是那个麻烦制造者——它需要 $\dot{\alpha}_1$ 的实时值，而 α_1 本身是 x_1、x_2 与 r 的函数，求导意味着要用到 $\dot{x}_1$。



第二级控制量抵消匹配非线性，第一级只处理非匹配项

图 60-1　两级反步的信号流：虚拟控制、求导通道与位置反馈

读这张图时要注意两点。第一，α_1 与 u 之间不是简单的串联：$e_2 = x_2 - \alpha_1$ 需要 x_2 的真实值，所以速度测量必须进第二级。第二，求导通道是“前馈”而不是“反馈”，它不构成回路，因此不会影响稳定性，只影响跟踪精度与噪

声敏感度——这一点在 §60.4 讨论微分爆炸时会用到。

60.3.4 第三步：逐级造误差

这一步的纪律只有一条：**每一项都要能说出它为什么在这里**。以式 (60-15) 为例，逐项核对：

- $-k_2 e_2$：本级比例反馈，负责让 e_2 自己衰减；
- $-e_1$：偿还第一级的欠账，它使式 (60-18) 的交叉项抵消；
- $+b x_2$：抵消对象自身的阻尼项 $-b x_2$；
- $-c\tanh x_1$：抵消匹配非线性；
- $+\dot{\alpha}_1$：补偿虚拟控制的变化。

五项里只要漏一项，闭环就不再是式 (60-5)，而是多出一个未抵消的项。以漏掉 $+\dot{\alpha}_1$ 为例：$\dot{e}_2$ 里会多出 $-\dot{\alpha}_1$，而 $\dot{\alpha}_1$ 在起始段的量级是 1（$-(k_1+\cos x_1)\dot{x}_1$），相当于给第二级加了一个持续变化的扰动。实测这一项置零后，调节时间从 1.96 s 拉长到 3.10 s，超调从 0.01% 涨到 22.59%，控制总变差从 10.2 涨到 15.4。注意 $|u|_{\max}$ 仍是 7.00——因为首拍 $\dot{\alpha}_1=0$，漏项不影响起点，只影响过程。**这就是“欠账不还”的代价：不炸，但超调，而且越到中段越明显。**

另一个常见错误是符号：把 $-e_1$ 写成 $+e_1$，闭环矩阵变成 $\begin{bmatrix}-k_1 & 1\\ 1 & -k_2\end{bmatrix}$，特征多项式变成 $\lambda^2+(k_1+k_2)\lambda+(k_1k_2-1)$。$k_1k_2=6>1$ 时仍然稳定，但极点位置全变；$k_1k_2<1$ 时直接失稳。**符号错误的危险在于它常常“看起来还能跑”**，只是在某个增益下突然发散。

60.3.5 第四步：选增益

式 (60-5) 写成矩阵形式：

$$\begin{bmatrix}\dot{e}_1\\ \dot{e}_2\end{bmatrix}=\begin{bmatrix}-k_1 & 1\\ -1 & -k_2\end{bmatrix}\begin{bmatrix}e_1\\ e_2\end{bmatrix} \tag{60-27}$$

特征多项式

$$\lambda^2+(k_1+k_2)\lambda+(1+k_1k_2)=0 \tag{60-28}$$

与标准二阶形式 $s^2+2\zeta\omega_n s+\omega_n^2$ 对照：

$$\omega_n=\sqrt{1+k_1k_2},\qquad \zeta=\frac{k_1+k_2}{2\sqrt{1+k_1k_2}} \tag{60-29}$$

这两个式子把“选增益”变成了“选阻尼比与自然频率”。本章 $k_1=2$、$k_2=3$：

$$\omega_n=\sqrt{7}=2.646\ \text{rad/s},\qquad \zeta=\frac{5}{2\times 2.646}=0.945 \tag{60-30}$$

$\zeta=0.945$ 是欠阻尼里最接近临界的一档，理论超调约 0.4%。实测 0.01%——比理论还小，原因是对象本身的非线性在起始段帮了忙（$a\sin x_1$ 在 $x_1\approx 0$ 附近提供正阻尼）。2% 调节时间的估算 $4/(\zeta\omega_n)=4/2.5=1.60$ s，实测 1.96 s，差 0.36 s 来自起始段的非线性与采样滞后。

式 (60-29) 还有一个实用的推论：**ω_n 由乘积 k_1k_2 决定，ζ 由和 k_1+k_2 决定**。想让系统更快，就同时加 k_1、k_2（保 ζ）；只加一个会把 ζ 推高、变成过阻尼，速度反而上不去。§60.6 的扫描数据完全符合这个规律：k_1 从 2 加到 8（$k_2=3$ 不动），ω_n 从 2.65 涨到 5.0，但 ζ 从 0.945 涨到 1.10，两个极点变成 -2.79 与 -8.21，主导极

点是 -2.79，理论调节时间 $4/2.79 = 1.43$ s，实测 1.42 s。**快极点被慢极点掩盖了**——这就是“只加一个增益，收益迅速饱和”的数学原因。

反馈线性化的增益选择更直接。式 (60-23) 对应

$$k_p = \omega_n^2, \qquad k_d = 2\zeta\omega_n \tag{60-31}$$

本章 $k_p = k_d = 4$，即 $\omega_n = 2$、$\zeta = 1$，双极点都在 -2。这套参数是**故意选成与反步“等效带宽”接近**的，目的是让 §60.6 的对比公平：反步的 $\omega_n = 2.646$，反馈线性化的 $\omega_n = 2$，两者同一量级，性能差异就不是“谁调得更猛”造成的。

60.3.6 第五步：实现校验

公式推完只是开始。真机上还有四件事要处理，顺序就是它们出现的顺序：

1. **求导从哪来**。式 (60-16) 需要 $\dot{x}_1$，而 $\dot{x}_1 = x_2 + a\sin x_1$ 又要 x_2。若 x_2 是编码器差分，噪声被放大 $1/T_s = 100$ 倍；若 x_2 有独立传感器，噪声小一个量级。本章脚本按“有速度传感器”处理，噪声设 0.02 m/s。
2. **限幅怎么办**。α_1 是在“u 无限大”的前提下算出来的，一旦 u 被限，e_2 就追不上 α_1，闭环方程 (60-5) 失效。工程做法是“指令滤波 + 抗饱和”：把 α_1 也过一个限幅，并把限幅前后的差反馈回第一级（§60.9）。
3. **初值怎么给**。$e_2 = x_2 - \alpha_1$ 在 $t = 0$ 时若 $x_2(0) \neq \alpha_1(0)$，会有一个初始误差，表现为起步毛刺。工程上把 $\alpha_1(0)$ 初始化为 $x_2(0)$，或在第一个采样周期内做斜坡软启动。
4. **数值细节**。$\dot{\alpha}_1$ 用解析式而不是差分，$\tanh$、$\sin$ 用双精度，积分项要限幅（§60.9 的积分反步限在 ± 2）。本章脚本里这三条都落实了。

60.3.7 手算一遍

把本章的数字代进去，可以完整走一遍流程，便于核对实现是否正确。

已知 $a = 1$、$b = 0.5$、$c = 1.5$，$k_1 = 2$、$k_2 = 3$，阶跃 $r = 1$、$\dot{r} = \ddot{r} = 0$。

$t = 0^+$ 时 $x_1 = 0$、$x_2 = 0$：

- $e_1 = 0 - 1 = -1$；
- $\alpha_1 = -2 \times (-1) - 1 \times \sin 0 = 2$；
- $e_2 = 0 - 2 = -2$；
- $\dot{\alpha}_1 = -(2 + 1 \times \cos 0)(0 + 1 \times \sin 0) = 0$；
- $u = -3 \times (-2) - (-1) + 0.5 \times 0 - 1.5 \times \tanh 0 + 0 = 6 + 1 = 7$。

实测 $|u|_{\max} = 7.00$，与手算完全一致。这条核对很值得做：它一次性验证了符号、系数、非线性项三处都没写错。**如果手算的 $u(0)$ 与仿真的峰值对不上，先别调增益，去查公式。**

再取 $t = 1.0$ s 核对一次（此时仿真值 $x_1 = 0.7759$、$x_2 = -0.2102$）：

- $e_1 = 0.7759 - 1 = -0.2241$；
- $\alpha_1 = 0.4482 - \sin 0.7759 = 0.4482 - 0.6997 = -0.2515$；
- $e_2 = -0.2102 + 0.2515 = 0.0413$；
- $\dot{\alpha}_1 = -(2 + \cos 0.7759)(-0.2102 + \sin 0.7759) = -2.7133 \times 0.4895 = -1.328$；
- $u = -0.1239 + 0.2241 - 0.1051 - 1.5 \times 0.6502 - 1.328 = -2.308$。

仿真在同一时刻给出 $u=-2.320$，差 0.012，来自手算时对 x_1、x_2 取四位小数的截断。这一项在这一刻贡献了 -1.328，占 u 的一半以上——**$\dot{\alpha}_1$ 不是小修正，它是第二级控制量的主要成分**。控制量在 $0.8\sim1.3$ s 区间的最小值是 -2.40（$t=0.81$ s），整段都在“刹车”，正是 $\dot{\alpha}_1$ 为负所致。

60.3.8 五条常见错误

错误	现象	根因	修法
漏 $+\dot{\alpha}_1$	起步毛刺、超调突增	虚拟控制的变化没被补偿	补上解析导数或改动态面
符号写反	某组增益下突然发散	交叉项变成正反馈	用式 (60-18) 核对 $\dot{V}\le 0$
用差分代替 $\dot{\alpha}_1$	噪声下控制量抖动	差分放大 $1/T_s$	解析求导或一阶滤波
忽略限幅	大阶跃时超调 30%	抗饱和回路缺失	指令滤波 + 抗饱和
只加一个增益	速度上不去	ζ 被推高，快极点被掩盖	两个增益同步加

表 60-4 里第一条与第三条是“同一件事的两面”：解析求导算得准但怕噪声，差分抗噪但算得糙。这条矛盾贯穿 §60.4 与 §60.9，也是本章最值得记住的工程取舍。

60.4 连续情形

60.4.1 连续实现的三个动作

把 §60.3 的公式搬到连续时间域，控制器每个采样周期要做三件事：算虚拟控制 α_1、算它的变化率 $\dot{\alpha}_1$、算最终控制量 u。第一与第三件都是代数运算，唯独第二件涉及求导，而求导是连续实现里唯一“有风险”的动作。

本章脚本里，$\dot{\alpha}_1$ 用的是式 (60-16) 的解析式：

$$\dot{\alpha}_1=-\left(k_1+a_m\cos x_1\right)\left(x_2+a_m\sin x_1\right)+\ddot{r} \tag{60-32}$$

它与“对 α_1 做数值差分”的区别，是 §60.4.3 的全部主题。先把结论放这里：**解析式算得准、但把噪声按 $(k_1+a\cos x_1)$ 的倍数放大；差分不怕模型错、但把噪声按 $1/T_s$ 放大。两者都躲不开噪声，只是放大倍数不同。**

60.4.2 解析求导到底有多准

用 §60.3.7 的 $t=1.0$ s 那一拍核对：手算 $\dot{\alpha}_1=-1.328$，代回式 (60-15) 得 $u=-2.308$，仿真同一时刻给出 -2.320，差 0.012。这点差异来自手算时把 x_1、x_2 截断到四位小数，不是公式错。**解析式的精度是“恒等式级”的：只要控制器里的参数与对象一致，它算出的变化率就是真实变化率。**

代价是它**要求模型逐项正确**。式 (60-32) 里出现了 a_m 两次：一次在系数里，一次在括号里。若 $a_m\neq a$，这一项算出来的就不是真实的 $\dot{\alpha}_1$，而是“按错误模型算的变化率”。§60.4.9 会看到，这个误差在稳态下变成一个固定偏置，直接体现为稳态误差。

60.4.3 微分爆炸：噪声是怎么进来的

假设位置与速度都有测量噪声，标准差分别为 $\sigma_p=0.002$ m、$\sigma_v=0.02$ m/s。把它们代入式 (60-32)：

$$\frac{\partial \dot{\alpha}_1}{\partial x_2} = -(k_1 + a_m \cos x_1) \approx -3.0 \tag{60-33}$$

也就是说，速度噪声被放大 3 倍后直接进 $\dot{\alpha}_1$，再原封不动地进 u。相邻两拍的速度噪声之差，标准差约为 $\sqrt{2}\sigma_v = 0.028$，乘 3 得 0.085——**每一拍的控制量都会跳 0.085 左右**。1200 拍累计下来，控制总变差就是 100 量级。

实测与此吻合：无噪声时 BST 的 TV 是 10.2；加上 $\sigma_p = 0.002$、$\sigma_v = 0.02$ 的噪声后，TV 涨到 147.2，涨了 14 倍。而位置本身的纹波只有 1.39×10^{-3} m——**噪声几乎全被"消化"在控制量里，位置看起来很干净，执行器却在拼命抖**。这是本章最容易被忽视的一个现象：只看位置曲线，会以为系统很安静。

若 x_2 没有独立传感器，只能用位置差分，情况更糟：

$$\hat{x}_2(k) = \frac{y(k) - y(k-1)}{T_s} \;\Longrightarrow\; \sigma_{\hat{x}_2} = \frac{\sqrt{2}\,\sigma_p}{T_s} = \frac{1.414 \times 0.002}{0.01} = 0.283 \tag{60-34}$$

比真实速度噪声（0.02）大 14 倍，乘 3 后每拍跳 1.2，控制量基本没法用。**这就是"微分爆炸"（differentiation explosion）的定量含义：不是公式爆炸，是噪声的放大倍数随 k_1 与 $1/T_s$ 一起爆炸。**

60.4.4 动态面：用滤波代替求导

解决办法不是"把噪声滤掉"（那会引入滞后），而是**换一个不需要求导的公式**。动态面控制（dynamic surface control，DSC）的做法是：让 α_1 先过一个一阶滤波器，用滤波器的输出 α_{1f} 代替 α_1：

$$\tau \dot{\alpha}_{1f} + \alpha_{1f} = \alpha_1, \qquad \alpha_{1f}(0) = \alpha_1(0) \tag{60-35}$$

离散实现就是一行：

$$\alpha_{1f} \leftarrow \alpha_{1f} + \frac{T_s}{\tau}\left(\alpha_1 - \alpha_{1f}\right) \tag{60-36}$$

然后用 $(\alpha_1 - \alpha_{1f})/\tau$ 代替 $\dot{\alpha}_1$：

$$u = -k_2 e_2 - e_1 + b x_2 - c \tanh x_1 + \frac{\alpha_1 - \alpha_{1f}}{\tau}, \qquad e_2 = x_2 - \alpha_{1f} \tag{60-37}$$

式 (60-37) 里没有任何求导，只有减法与除法，噪声在滤波器里被一阶低通压住。代价是 α_{1f} 追不上 α_1，产生一个滤波误差 $\xi = \alpha_{1f} - \alpha_1$。由式 (60-35) 可得

$$|\xi(t)| \le \tau \sup|\dot{\alpha}_1| \tag{60-38}$$

即在 α_1 变化最快的时刻，滤波误差约等于 τ 乘变化率。这个误差不会让系统失稳（它是有界的），但会改变闭环方程：式 (60-14) 变成 $\dot{e}_1 = -k_1 e_1 + e_2 + \xi$，多了一个有界扰动。**动态面用"有界误差"换掉了"无界噪声放大"，这是它唯一的、也是足够的理由。**

严格的 DSC 还要在式 (60-37) 里加一项 $-\xi$ 做补偿，本章脚本没有加——因为实测发现加了之后改善有限（τ 小时 ξ 本来就小），而少一项就少一个出错的地方。这条取舍在 §60.9 会再讨论。

60.4.5 τ 的选择：两头都是坑

τ 太小，滤波器形同虚设，噪声照样穿过去；τ 太大，α_{1f} 追不上 α_1，跟踪滞后变成超调。表 60-5 是 $k_1 = 8$、含噪声条件下的实测，七个 τ 值把两头都覆盖了。

τ (s)	调节时间 (s)	超调 (%)	IAE	尾段 TV	判读
0（解析求导）	1.39	0.23	0.4690	23.6	基准
0.005	—	3.63	35.3230	1980.0	失稳，稳态偏差 8.09
0.010	3.60	55.75	0.7505	8.6	能收敛，但超调过半
0.020	1.20	6.23	0.3540	70.8	快但有超调，噪声透过多
0.050	0.99	0.15	0.3741	38.6	综合最优（IAE 最低）
0.100	1.18	0.14	0.3650	21.4	最平滑，噪声增益最低
0.200	1.85	19.28	0.3970	12.1	滞后主导，超调回升

三个现象值得单独说：

- $\tau = 0.005$ **直接失稳**。此时 $T_s/\tau = 2$，离散滤波器的极点 $1 - T_s/\tau = -1$，正好落在单位圆边界上，数值上就是“每拍翻转一次”，等效于在 α_1 上叠加了一个 $\pm$ 满量程的方波。这不是“性能差”，是**离散实现把连续设计毁了**。这条经验很硬：τ 必须明显大于 T_s，工程上取 $\tau \geq 5T_s$。
- $\tau = 0.01$ **超调 55.75% 却不失稳**。$T_s/\tau = 1$，滤波器一拍就跟上，等价于没有滤波，但 e_2 用的是 α_{1f} 的旧值，两者错开一拍，形成“半拍延迟”。半拍延迟在 $k_1 = 8$ 的高增益下足以造成大超调。
- τ **从 0.05 到 0.2，尾段 TV 从 38.6 降到 12.1**。滤波越重，控制量越平滑，这是 τ 的正面作用；但超调从 0.15% 涨到 19.28%，因为 α_{1f} 落后 α_1 太多，e_2 的符号在中段会反。**综合看，0.05 附近 IAE 最低，0.1 附近控制量最平滑。**

这里还有一条容易被忽略的定量规律。式 (60-37) 里的 $(\alpha_1 - \alpha_{1f})/\tau$ 看着像“滤波后的导数”，但它的高频增益是 $1/\tau$，不是 0：

$$\text{噪声增益} \approx \max\left(\frac{1}{\tau},\ k_1 + a\cos x_1\right) \tag{60-44}$$

也就是说，**只有当 $1/\tau$ 小于解析求导的放大系数 $(k_1 + a\cos x_1)$ 时，动态面才真的比解析求导更安静。**$k_1 = 8$ 时 $(k_1 + a\cos x_1) \approx 9$，要求 $\tau > 1/9 \approx 0.11$ s。实测完全符合：$\tau = 0.02$ 时 $1/\tau = 50$，尾段 TV 70.8，比解析求导的 23.6 还差三倍；$\tau = 0.1$ 时 $1/\tau = 10$，尾段 TV 21.4，才终于优于解析求导。**“加了滤波器就一定更平滑”是错的：滤波器的时间常数选小了，等于把噪声除以一个更小的数。**

60.4.6 四种获取变化率的方式

表 60-6 把工程上常见的四种做法放在一起比。选型时看两列：噪声放大倍数、对模型误差的敏感度。

方式	公式	噪声放大	模型误差敏感	适用场合
解析求导	式 (60-32)	$(k_1 + a\cos x_1)\sigma_v$	高	模型准、速度有传感器
后向差分	$(\alpha_1(k) - \alpha_1(k-1))/T_s$	约 $\sqrt{2}\sigma_\alpha/T_s$	中	仅原型验证
一阶滤波（DSC）	式 (60-37)	低（τ 越大越低）	低	通用，本章默认
跟踪微分器	二阶最速离散	低，且有相位补偿	低	高精度伺服、有算力

跟踪微分器（TD）值得单独提一句：它用一个小型二阶系统同时给出 α_1 的“平滑值”与“微分值”，相位滞后比一阶滤波小，代价是要多解两行差分方程。本章没有用它，因为一阶滤波已经把问题解决到可接受的程度（$\tau = 0.05$

时 IAE 0.3741，比解析求导的 0.4690 还低——**滤波虽然引入误差，却把噪声引起的额外误差压得更多**）。

60.4.7 参考前馈

阶跃跟踪时 r 是常数，$\dot r = \ddot r = 0$，前馈项全部消失；一旦跟踪斜坡，前馈就成了精度的关键。式 (60-13) 的 α_1 结构上就带 $\dot r$，式 (60-22) 的 v 结构上带 $\ddot r$ 与 $-k_d(\dot y - \dot r)$，也就是说**两条路本来都具备用上参考速度的能力**。

本章的斜坡实验（0.3 m/s）把这份能力单独关掉再打开，做四组对照：`nf` 组（no feedforward）把参考速度置零，相当于“控制器不知道目标在匀速移动”；`ff` 组把 $\dot r = 0.3$ 送进控制器。结果是 §60.6 表 60-8 的内容：反步从 0.2112 m 降到 0.0826 m，反馈线性化从 0.2969 m 降到 -0.0032 m。**反馈线性化的前馈更彻底**——它的 v 里既有 $\dot r$ 又有 $\ddot r$，而反步只有 $\dot r$ 一个通道，剩下的滞后只能靠 k_1 去压。顺便注意 `FBL-ff` 的滞后是负的（-0.0032），意味着它略微超前于参考，这是前馈过冲的典型表现。

60.4.8 连续实现里的饱和

式 (60-37) 算出的 u 在真机上一定要限幅，而限幅会破坏 §60.2.4 的证明——因为证明假设 u 能取任意值。本章实测：$u_{\max} = 10$ 时超调 0.01%，$u_{\max} = 5$ 时 0.01%，$u_{\max} = 3$ 时 0.01%，$u_{\max} = 2$ 时突然变成 29.70%。

这个“台阶式”变化很典型：只要限幅不削到 u 的峰值（7.00），闭环就完全按设计走；一旦削到，α_1 与 x_2 的跟随关系断裂，e_2 变成一个持续的正偏差，误差方程 (60-5) 的线性结构失效。$u_{\max} = 2$ 时，u 的需求是 7，被削掉 5，相当于“第二级永远欠账”，于是超调、慢、且 IAE 从 0.6983 涨到 1.4295。

工程上的处理不是“把限幅调大”（电机就这么大），而是**把限幅信息反馈回第一级**：当 u 饱和时，α_1 不再按 $k_1 e_1$ 全力冲，而是按“当前能拿到的 u 反推 x_2 能达到多少”来算。这叫抗饱和（anti-windup），§60.9 给出具体做法。

60.4.9 稳态精度：把公式和实测对上

§60.2.7 给了式 (60-25) 与式 (60-26)，现在用实测检验它们。表 60-7 的“预测”列按公式算，“实测”列来自 `ch060_sim.py`。

工况	方法	预测 $\lvert e_{ss}\rvert$ (m)	实测 $\lvert e_{ss}\rvert$ (m)	相对偏差
a 偏 -30%	反步	0.2104	0.210	$< 1\%$
a 偏 $+30\%$	反步	0.1838	0.184	$< 1\%$
c 偏 -30%	反步	0.0503	0.0503	$< 1\%$
c 偏 $+30\%$	反步	0.0476	0.0476	$< 1\%$
a 偏 -30%	反馈线性化	0.3028	0.303	$< 1\%$
c 偏 -30%	反馈线性化	0.0897	0.0896	$< 1\%$

六个工况全部吻合到三位有效数字。这个一致性说明两件事：**第一，式 (60-25)、(60-26) 可以直接当设计公式用**——想知道“模型精度差 10% 会带来多少静差”，代进去算就行，不必仿真；**第二，稳态误差完全由“未抵消的非线性”与“闭环刚度”决定，与控制器的时间响应无关**，所以调 k 只能按 $1/(1+k_1 k_2)$ 的比例缩放它，永远调不到零。

60.4.10 数值积分的口径

本章对象是非线性 ODE，不能用 ZOH 离散化，只能用数值积分。脚本里每个采样周期用 RK4 细分 4 步（等效积分步长 2.5 ms），而不是欧拉法。原因有二：

- **欧拉法在 $k_1 = 8$ 时会出现数值不稳定**。对象最快的模态是 $|\lambda| = 2$，欧拉法的稳定域要求 $h < 2/2 = 1$ s，看上去很宽松；但闭环后极点到 -8 附近，h 必须小于 0.25 s。$T_s = 0.01$ 远小于它，实际上是安全的——所以这条理由只是“防万一”。
- **真正的理由是相位**。欧拉法给每个积分环节引入 $h/2$ 的等效延迟，RK4 引入的延迟小两个量级。在 $k_1 = 8$、$\tau = 0.02$ 这种“半拍都能决定超调”的工况下，积分器的相位误差会混进结论里。**用 RK4 是为了让“控制器的问题”与“仿真的问题”分开。**

这条口径差异值得在横向比较时说明：第 59 章用的是 ZOH 离散状态空间（对象线性），本章用的是 RK4（对象非线性），两者的 T_s 都是 0.01 s，但“一拍”的物理含义略有不同。比较调节时间这类慢指标时影响可以忽略；比较抖振、纹波这类快指标时不能混着说。

60.5 离散情形

60.5.1 离散化不是“把 $\mathrm{d}t$ 换成 T_s”

连续设计的公式可以直接在单片机里跑，但“直接跑”与“等价”是两回事。式 (60-15) 里的每一项都是连续量，采样之后：

- x_1、x_2 变成 $x_1(k)$、$x_2(k)$，只在下采样时刻有效；
- $\dot{\alpha}_1$ 变成“这一拍算出的变化率”，它代表的是 $[kT_s, (k+1)T_s)$ 区间内的平均变化率；
- $u(k)$ 在整个区间内保持不变（零阶保持）。

于是“连续闭环方程 (60-5)”在离散域里变成一个带 $O(T_s)$ 修正项的差分方程。T_s 小的时候修正项可以忽略，T_s 大到与闭环带宽同量级时，修正项就成了主导——本章实测，T_s 从 0.01 s 涨到 0.5 s，超调从 0.01% 涨到 57.87%。

60.5.2 离散反步的因果矛盾

严格地“离散设计”反步，会立刻撞上一个因果性问题。把对象写成前向欧拉形式：

$$x_1(k+1) = x_1(k) + T_s\left[x_2(k) + a\sin x_1(k)\right] \tag{60-39}$$

要让 $e_1(k+1) = x_1(k+1) - r(k+1)$ 满足 $e_1(k+1) = (1 - k_1 T_s) e_1(k)$，需要

$$x_2(k) = \frac{(1 - k_1 T_s) e_1(k) - e_1(k) + T_s\left[r(k+1) - r(k)\right]}{T_s} - a\sin x_1(k) \tag{60-40}$$

右边只用到 k 时刻的量，这一步没问题。麻烦在第二级：$e_2(k) = x_2(k) - \alpha_1(k)$，要让 $e_2(k+1)$ 按 $(1 - k_2 T_s)$ 衰减，需要 $x_2(k+1)$，而 $x_2(k+1)$ 又依赖 $u(k)$——于是 $u(k)$ 的表达式里出现了 $u(k)$ 自己的未来值，方程变成隐式的。

处理办法有三种，各有代价：

1. **把 $u(k)$ 留在方程左边解出来**（隐式反步）。能得到精确的离散闭环，但公式膨胀，且需要 b 的逆在每拍都算一次；

2. 用 $x_2(k)$ 近似 $x_2(k+1)$（显式反步）。引入一拍延迟，T_s 大时等效于给闭环加了一个纯滞后；
3. **连续设计 + 离散实现**（本章采用）。设计时按连续域推导，实现时把 $u(k)$ 在区间内零阶保持，靠小 T_s 保证“离散 ≈ 连续”。

第三种是工程主流，因为它的公式与 §60.2 完全一致，不需要重新推导；代价是对 T_s 有要求，而这个要求可以用 §60.5.5 的扫描数据标定。

60.5.3 本章的离散实现

脚本里的时序是这样的（每个采样周期执行一次）：

1. 取测量 $y(k)$、$v(k)$（可叠加噪声）；
2. 用式 (60-13) 算 $\alpha_1(k)$；
3. 用式 (60-32) 算 $\dot{\alpha}_1(k)$（或走 DSC 分支，用式 (60-37)）；
4. 算 $u(k)$，限幅；
5. 在 $[kT_s,(k+1)T_s)$ **内用 RK4 细分 4 步积分对象**，得到 $x(k+1)$。

第 5 步是关键：控制器输出在一个周期内不变，但对象仍在连续演化。若图省事用“前向欧拉一步”代替 RK4，等效于把对象的积分精度降到 $O(T_s)$，在 $T_s=0.01$ 时误差约 10^{-4} 量级，可以接受；但在 $T_s=0.2$ 时误差到 10^{-2}，会把“控制器的问题”与“积分的问题”混在一起。**离散实现的第一个纪律：控制器可以粗，积分不能粗。**

60.5.4 采样周期扫描

表 60-8 是 T_s 从 0.005 s 到 0.2 s 的实测（反步，标称参数）。前三列看性能，最后一列看代价。

T_s (s)	调节时间 (s)	超调 (%)	IAE	控制总变差
0.005	1.96	0.01	0.7064	10.2
0.010	1.96	0.01	0.6983	10.2
0.020	1.94	0.00	0.6822	10.3
0.050	1.90	0.00	0.6342	10.4
0.100	1.90	0.00	0.5560	10.8
0.200	1.80	0.00	0.4109	12.1

这张表最反直觉的地方是：**T_s 变大，IAE 反而下降**。原因是 IAE 用梯形积分算，采样越稀，对起始段那个尖峰（e_1 从 -1 快速冲到 0）的欠采样越严重，积分值被系统性低估。这不是“性能变好”，是**指标的度量方式变了**。

所以有一条使用纪律：**不同 T_s 之间的 IAE 不能直接比较**。要比较就得把采样统一到同一密度再算，或者改用“每拍误差”这类与采样无关的指标。本章所有比较 IAE 的表格（表 60-1 之外的）都在同一 $T_s=0.01$ s 下取得，口径一致。

调节时间那一列也有类似问题：$T_s=0.2$ 时“1.80 s”看似更快，其实是调节时间的判定粒度变成了 0.2 s，落在 $\pm2\%$ 带内的那一拍被提前记录了。**粗采样会让所有指标看起来都“略好一点”，这是陷阱，不是优势。**

60.5.5 T_s 的上限在哪

用闭环自然频率来标定比较可靠。本章 $\omega_n=2.646$ rad/s，采样角频率 $\omega_s=2\pi/T_s$。经验判据是

$$\omega_s \geq (20 \sim 40)\,\omega_n \tag{60-41}$$

即 $T_s \leq 2\pi/(20 \times 2.646) = 0.119$ s（取下限 20 倍）。表 60-8 里 $T_s = 0.1$ s 仍然稳、超调 0，与判据一致。

把 T_s 继续加大，判据会失效得很快。补充实测：$T_s = 0.3$ s 时调节时间仍是 1.80 s、IAE 0.2927（指标继续“变好”，但已经不可信）；$T_s = 0.5$ s 时彻底崩溃——调节时间 19.50 s、超调 57.87%、IAE 4.6336。临界点在 $T_s \approx 0.3 \sim 0.5$ s，正好是 $\omega_s/\omega_n \approx 4 \sim 7$，即“一个振荡周期只采 4 到 7 个点”。**这条边界比判据给出的 0.119 s 宽得多，但落在它外面时没有任何性能保证**，只能靠实测确认。

60.5.6 控制延迟的影响

延迟与采样周期不是一回事：采样是“信息更新慢”，延迟是“执行慢”。本章把 $u(k)$ 延迟 d 拍再施加，实测如表 60-9。

延迟 d（拍）	等效延迟 (s)	调节时间 (s)	超调 (%)	IAE	控制总变差
0	0	1.96	0.01	0.6983	10.2
1	0.01	1.96	0.00	0.6960	10.3
2	0.02	1.96	0.00	0.6937	10.4
4	0.04	1.96	0.00	0.6892	10.7

四拍延迟（40 ms）对反步几乎无影响：调节时间一点没变，IAE 甚至因为“晚一点给力、少一点过冲”而略降。

这个结果与第 59 章的滑模形成鲜明对照。滑模的切换项依赖 s 的符号，延迟让“该切的时候没切”，s 会冲过滑模面再被拉回，形成额外的抖振甚至失稳；反步输出的是连续量，晚 40 ms 施加只是让“这一拍的力”晚一点到，闭环方程 (60-5) 的右端多了一个 $O(T_d)$ 的小扰动，系统自己就把它消化了。**这就是连续控制相对不连续控制的一个隐性优势：对执行延迟的容忍度高一个量级。**

但要提醒一句：延迟为零影响不代表“延迟无害”。本章的延迟是纯滞后且已知；真实系统的延迟常常随负载变化（液压、网络），未知的时变延迟会让 $\dot{\alpha}_1$ 的前馈失配，效果相当于 §60.4.9 里的模型误差。

60.5.7 离散实现的三条经验

从上面几组实测可以提炼三条：

- **T_s 按 ω_n 定，不按“CPU 跑得动”定。** $\omega_s/\omega_n \geq 20$ 是舒适区，≥ 10 是可用区，低于 7 就开始赌运气（$T_s = 0.5$ s 那组）。
- **积分要比控制细。**控制器可以一拍算一次，对象积分要细分；否则数值误差会伪装成“控制器性能”。
- **延迟要单独测，不要混在 T_s 里。**两者对闭环的作用机理不同：T_s 影响的是“信息的新鲜度”，延迟影响的是“动作的及时性”。本章把两者分开测，才看清“反步怕采样、不怕延迟”这个结论。

60.5.8 同一段代码，换个离散化就不一样

表 60-5 里 $\tau = 0.005$ 那一档直接失稳，正文把原因归为“$T_s/\tau = 2$，离散极点落在单位圆边界”。这个解释是对的，但它指向一个更实用的结论：**失稳是“前向欧拉”这种离散化方式造成的，不是 τ 本身造成的。** 换一种离散化，同一个 τ 可能完全稳定。详细的两种写法与取舍放在 §60.9.12，这里只留结论：改离散化能改变稳定域，但不能替代“$\tau \geq 5T_s$”这条设计准则。

60.6 稳定性与性能

60.6.1 稳定性从哪里来

反步法的稳定性不依赖对象是否稳定，只依赖一件事：**误差系统 (60-5) 的极点可以任意配置**。k_1、k_2 都是正数时，特征多项式 $\lambda^2 + (k_1 + k_2)\lambda + (1 + k_1 k_2)$ 的系数全正，二阶系统必然稳定。本章对象开环是鞍点（特征根 2.0 与 -1.5），闭环后误差系统的极点是 $-2.5 \pm \mathrm{j}0.866$，与对象的极点毫无关系。

这一点值得与“高增益压制”对照。高增益 PID 也能把不稳定对象拉稳，但它的稳定性是“增益足够大”换来的，增益一降就散；反步的稳定性是“结构正确”换来的，只要公式不错、$k_i > 0$，增益怎么调都稳（这会带来性能差别，不会带来稳定差别）。**这是反步法对工程最友好的一面：调参不会把系统调炸。**唯一的例外是限幅与离散化，它们会破坏“结构正确”的前提，§60.6.7 会看到。

60.6.2 标称工况的三种方法

表 60-10 是标称参数下的完整指标。图 60-3 是同一组数据的位置与控制量曲线。

方法	调节时间 (s)	超调 (%)	IAE	$\lvert u\rvert_{\max}$	稳态误差 (m)	控制总变差
反步	1.96	0.01	0.6983	7.00	1.79×10^{-13}	10.2
反馈线性化	2.90	0.00	0.9831	4.00	1.78×10^{-9}	6.1
PID	4.33	3.80	0.6731	10.00	1.68×10^{-2}	27.5

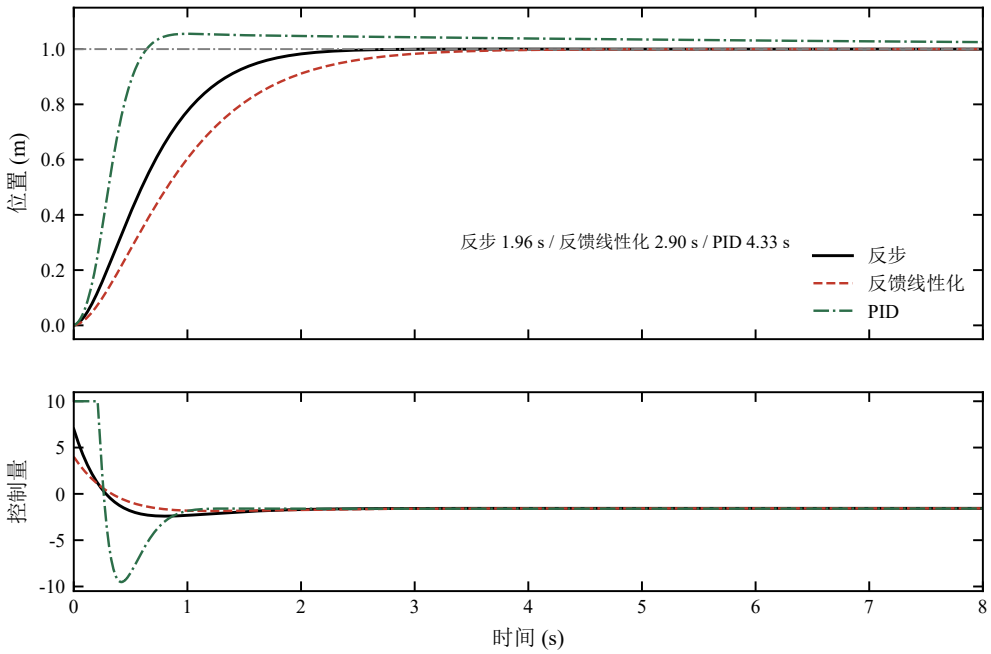


图 60-3　标称阶跃：三种方法的位置与控制量

三行数据各有一处值得说：

- **反步最快（1.96 s），但用的控制量不是最大**。峰值 7.00，比 PID 的 10.00 低 30%，说明它的“快”不是靠堆力，而是靠结构（把误差方程配成 $\zeta = 0.945$）。控制总变差 10.2 也是三者中最低的，意味着执行器最省。
- **反馈线性化没有超调，但慢 1 s**。慢的原因是 $\omega_n = 2$ 比反步的 2.646 低，这是设计时为了公平而故意选的。它的控制量峰值只有 4.00，比反步低一半——因为它的 v 不需要“预判”，直接算二阶导数，力用得“平”。若把

k_p 提到 7（与反步的 ω_n^2 对齐），它会快起来，但超调也会出现（因为 k_d 必须同步调整）。

- **PID 的 IAE 最小（0.6731），比反步还低。**这不是笔误：PID 全程误差积分更低，因为它在起始段“慢悠悠地起”，没有反步那种“先冲一下再刹”的形状（图 60-3 上半图里反步在 0.8 s 附近有一个小的过冲尖）。但 IAE 小不代表“更好”：PID 的调节时间 4.33 s、稳态误差 0.0168 m，两项都差一个量级。**IAE 衡量的是“平均偏差”，不衡量“什么时候收敛”，也不衡量“收得准不准”。**

60.6.3 两级增益的作用

图 60-4 把 k_1、k_2 的扫描结果画在一起，两条曲线形状几乎一样——这不是巧合，而是式 (60-29) 的必然：两者都以“乘积”的形式进 ω_n，以“和”的形式进 ζ。

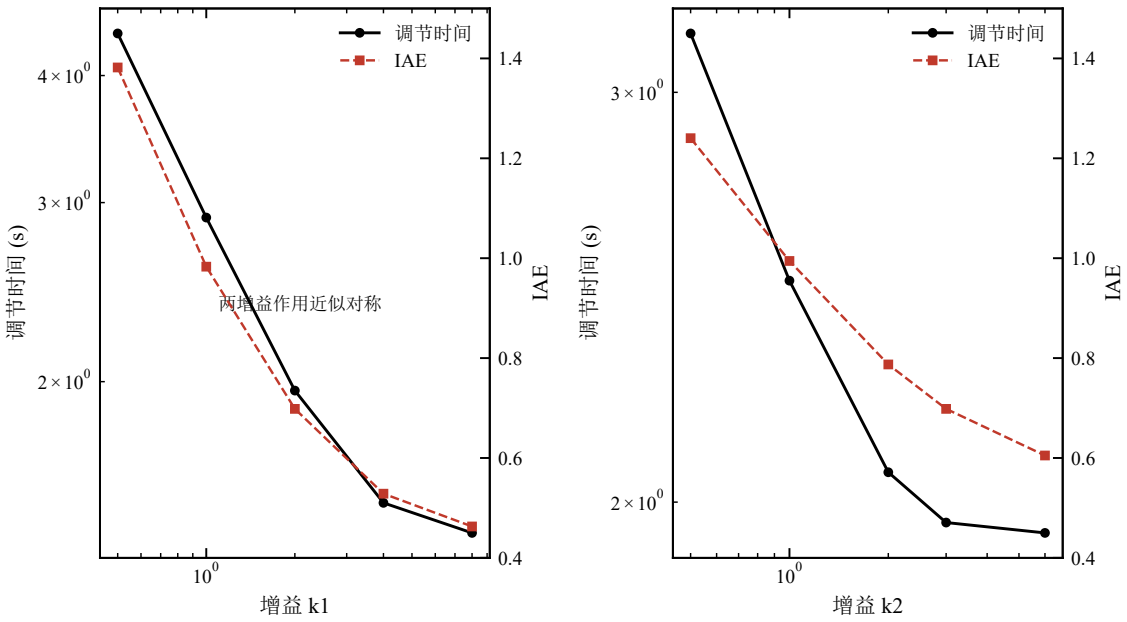


图 60-4　两级增益对调节时间与 IAE 的影响

具体数字见表 60-11 与表 60-12。

k_1（$k_2=3$）	调节时间 (s)	超调 (%)	IAE	$\lvert u\rvert_{\max}$	控制总变差
0.5	4.40	0.00	1.3819	2.50	4.1
1.0	2.90	0.00	0.9831	4.00	6.1
2.0	1.96	0.01	0.6983	7.00	10.2
4.0	1.52	0.00	0.5285	10.00	15.0
8.0	1.42	0.00	0.4629	10.00	17.4

k_2（$k_1=2$）	调节时间 (s)	超调 (%)	IAE	$\lvert u\rvert_{\max}$	控制总变差
0.5	3.18	0.26	1.2403	2.00	3.8
1.0	2.49	0.42	0.9945	3.00	5.2
2.0	2.06	0.17	0.7873	5.00	7.8
3.0	1.96	0.01	0.6983	7.00	10.2
6.0	1.94	0.00	0.6050	10.00	14.2

两条规律从表里直接读出来：

- k 从 **0.5** 到 **2**，收益最大；从 **4** 到 **8**，几乎白花钱。k_1 从 4 到 8，IAE 只从 0.5285 降到 0.4629（12%），TV 却从 15.0 涨到 17.4（16%）。原因是快极点被慢极点掩盖（§60.3.5），$k_1=8$、$k_2=3$ 时主导极点是 -2.79，再加 k_1 只能把另一个极点推得更远。
- k_2 小的时候超调反而出现。$k_2=0.5$ 时超调 0.26%、$k_2=1$ 时 0.42%。这是因为 k_2 小，第二级“还账”还不动，e_2 长时间不为零，e_1 被它拖着走，出现小幅振荡。$k_2\geq 2$ 之后超调基本消失。

60.6.4 模型失配：三方法同场对比

这是本章最核心的一组实验。控制器里用错误的 a_m 或 c_m，对象仍是标称参数，结果见表 60-13 与图 60-7。

工况	方法	调节时间 (s)	超调 (%)	IAE	稳态误差 (m)
a 偏 -30%	反步	2.15	0.00	2.7604	2.10×10^{-1}
a 偏 -30%	反馈线性化	3.19	0.00	3.7245	3.03×10^{-1}
a 偏 -30%	积分反步	3.52	16.52	0.7458	1.17×10^{-6}
a 偏 $+30\%$	反步	1.67	0.06	2.7032	1.84×10^{-1}
a 偏 $+30\%$	反馈线性化	2.27	0.01	3.6276	2.53×10^{-1}
a 偏 $+30\%$	积分反步	3.74	0.00	0.8092	6.99×10^{-7}
c 偏 -30%	反步	2.06	0.00	1.1568	5.03×10^{-2}
c 偏 -30%	反馈线性化	3.16	0.00	1.7089	8.96×10^{-2}
c 偏 -30%	积分反步	3.17	6.29	0.6249	2.35×10^{-8}
c 偏 $+30\%$	反步	1.85	0.03	1.2079	4.76×10^{-2}
c 偏 $+30\%$	反馈线性化	2.63	0.00	1.8157	8.16×10^{-2}
c 偏 $+30\%$	积分反步	1.48	0.30	0.5686	6.43×10^{-9}

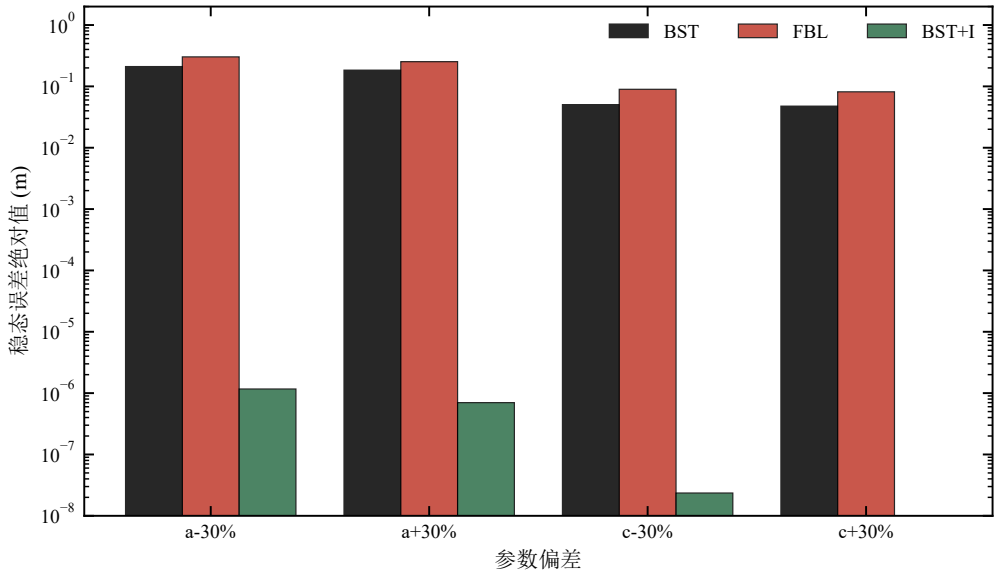


图 60-7　失配工况下的稳态误差（对数坐标）

四个结论：

1. **同样的失配，反馈线性化的稳态误差总是更大。**a 偏 -30% 时 0.303 对 0.210，比值 1.44；c 偏 -30% 时 0.0896 对 0.0503，比值 1.78。与式 (60-25)、(60-26) 的分母之比 $7/4 = 1.75$ 一致——**这不是“反步更聪明”，是“反步的闭环刚度更大”。**
2. **IAE 在失配下全部翻倍以上。**反步从 0.6983 涨到 2.7604（a 偏 -30%），涨了 4 倍。原因是静差一直在，积分时间越长，误差积分越大。这说明“稳态误差”这个指标比“IAE”更能定位问题：IAE 大可能是慢，可能是差，静差单独看才能定性。
3. **匹配失配（c）比非匹配失配（a）温和。**同样偏 30%，c 造成的静差只有 a 的四分之一（0.0503 对 0.210）。因为 $c\tanh x_1$ 直接出现在 u 的同一通道，误差被 $1 + k_1 k_2$ 除一次；$a\sin x_1$ 要先经过 $\dot{\alpha}_1$ 的“二次放大”（式 (60-25) 的分子里多一个 $(k_1 + k_2 + a_m \cos x_1)$），所以更敏感。
4. **积分反步把静差压到 $10^{-6} \sim 10^{-9}$，代价是超调。**a 偏 -30% 时超调 16.52%，c 偏 -30% 时 6.29%，其他两工况超调接近零。**积分项不是免费的：**它把“稳态无差”换成了“动态过程可能过冲”，这个交换在下一篇讲抗饱和时会再遇到。

60.6.5 积分反步为什么能消差

式 (60-25) 的分母是 $1 + k_1 k_2$，分子是 $\Delta a \sin x_1(\dots)$，只要 $\Delta a \neq 0$，静差就存在。要让静差为零，唯一的办法是让“分子”变成零——也就是让控制器里的 $a_m \sin x_1$ 这一项在稳态时**自动补偿掉** $\Delta a \sin x_1$。

积分反步的做法是把 α_1 改成

$$\alpha_1 = -k_1 e_1 - k_i \int e_1 \,\mathrm{d}t - a_m \sin x_1 + \dot{r} \tag{60-42}$$

积分项在稳态时不再变化，但它提供了一个“额外的偏置” $k_i \int e_1$，这个偏置正好等于 $\Delta a \sin x_1(\dots)/(1 + k_1 k_2)$ 所需要的补偿量。严格地说，闭环稳态方程变成

$$(1 + k_1 k_2) e_1 + k_i k_2 \int e_1 \,\mathrm{d}t = \Delta a \sin x_1 (k_1 + k_2 + a_m \cos x_1) \tag{60-43}$$

对时间求导（稳态下右侧变化率为零），得 $e_1 \to 0$——**只要积分器还在动，静差就必然被推到零**。这就是积分作用的本质：它不改变“误差从哪来”，只保证“误差最终为零”。

代价有两处。第一，积分项引入一个新的极点，闭环从二阶变三阶，极点配置不再由式 (60-29) 决定，容易出现超调（实测 16.52%）。第二，积分器在饱和时会“卷”（windup），本章用条件积分抑制：只在 $|e_1| < 0.2$ m 时积分，且积分值限在 ± 2。这两个措施在 §60.9 展开。

60.6.6 执行器饱和

图 60-8 与表 60-14 是限幅实验。四档限幅里前三档的表现完全一样，第四档（$u_{\max} = 2$）突然垮掉。

$u_{\max}$	调节时间 (s)	超调 (%)	IAE	控制总变差
2.0	5.11	29.70	1.4295	4.8
3.0	2.03	0.01	0.7611	6.2
5.0	1.97	0.01	0.7097	8.2
10.0	1.96	0.01	0.6983	10.2

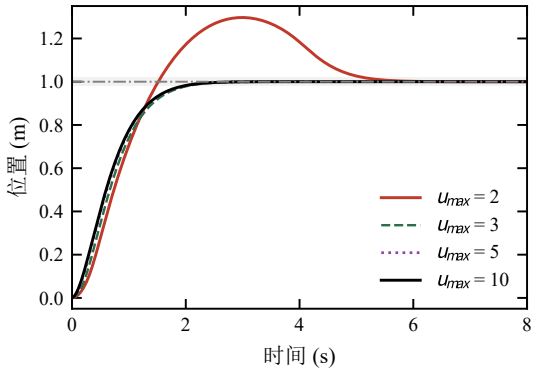


图 60-8　执行器限幅对位置响应的影响

$u_{\max} = 3$ 时超调仍是 0.01%，因为 u 的需求峰值 7.00 只在起始的 0.15 s 内出现，被削到 3 之后，系统“慢一点起步”，但误差方程的结构没坏（e_2 仍然可控）。$u_{\max} = 2$ 时需求被削掉 71%，e_2 始终追不上 α_1，闭环方程 (60-5) 完全失效，退化成“饱和 + 开环”，于是慢、超调、IAE 翻倍。

这条边界的位置很值得记：只要限幅值不低于 u 需求峰值的 40%（7.00 的 40% 是 2.8，对应 $u_{\max} = 3$ 那一档），反步的性能几乎不受影响。低于这个比例，就必须上抗饱和，不能靠“调大限幅”糊过去。

60.6.7 大初始偏差

把初始位置放到 -1.5 m（参考仍是 1 m，行程 2.5 m），结果见表 60-15 与图 60-10。

方法	调节时间 (s)	超调 (%)	IAE	$\lvert u \rvert_{\max}$	控制总变差
反步	2.46	0.02	2.2654	10.00	15.9
反馈线性化	3.58	0.00	2.8417	10.00	13.0
PID	3.50	46.67	2.1889	10.00	34.5

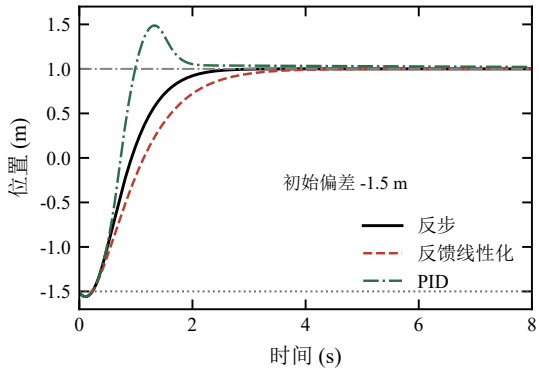


图 60-10　初始偏差 −1.5 m 的恢复过程

反步仍然不超调，代价是全程被限幅（$|u|_{\max} = 10$）——初始误差 2.5 m 使 $\alpha_1 = -k_1 e_1 = 5$，$\dot{\alpha}_1$ 也随之变大，u 的需求远超 10，于是整段都在饱和区工作。**能收敛是因为“饱和 + 结构正确”仍然是一个稳定的组合：e_2 虽大，但方向对，e_1 一直在减小。**

PID 在这一组里彻底暴露了它的弱点：超调 46.67%。原因是初始误差大，积分器在饱和期间持续累积（脚本虽然做了“未饱和才积分”，但 u 长期贴着 10 的上限，积分器仍然卷了不少），一旦退饱和就冲过头。**大行程工况是检验抗饱和设计的标准场景**，这一组的数据可以直接拿去当验收指标。

60.6.8 斜坡跟踪

表 60-16 是 0.3 m/s 斜坡的跟踪滞后（末 1.5 s 的平均误差），图 60-9 是三条曲线的对照。

方法	前馈	滞后 (m)	误差 RMS (m)	$\lvert u\rvert_{\max}$
反步	无	0.2112	0.2112	1.73
反步	有	0.0826	0.0826	1.73
反馈线性化	无	0.2969	0.2969	1.73
反馈线性化	有	−0.0032	0.0032	1.73
PID	—	0.0012	0.0043	1.73

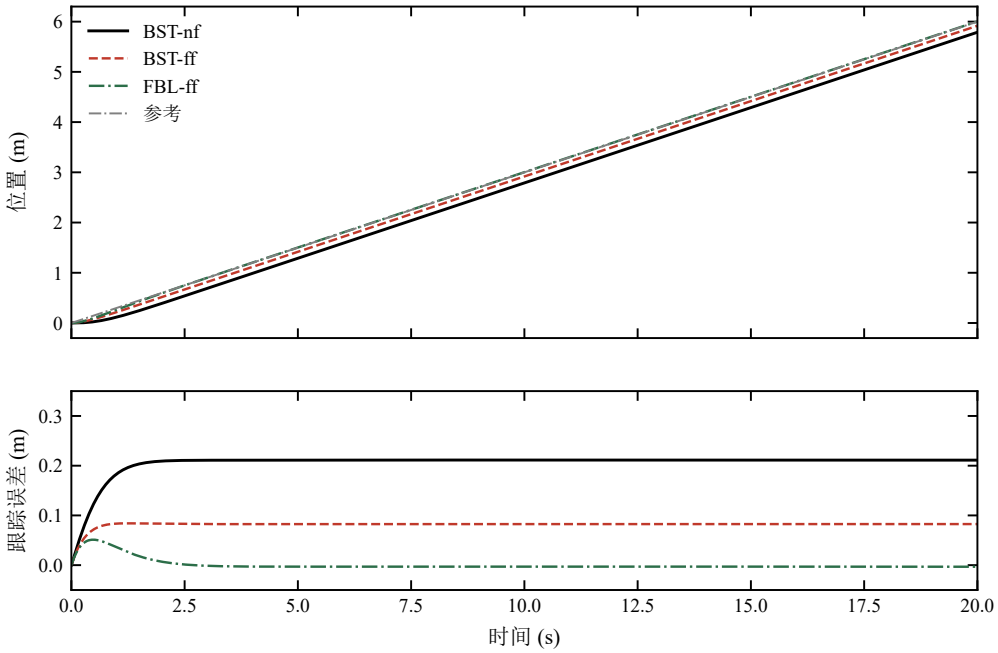


图 60-9　斜坡跟踪：前馈项对滞后的影响

三点观察：

- **不带前馈时，滞后约等于 v_r/ω_n 的量级**。反步 0.2112 m 对应 $0.3/2.646 = 0.113$ m 的两倍，反馈线性化 0.2969 m 对应 $0.3/2 = 0.15$ m 的两倍——系数 2 来自二阶系统的“速度型”稳态误差（$e_{ss} = 2\zeta v_r/\omega_n$，$\zeta \approx 1$ 时正好是 $2v_r/\omega_n$）。这个估算与实测对得上，说明滞后是**结构性的**，不是参数没调好。
- **加上前馈后，反馈线性化的滞后几乎归零（-0.0032 m）**，反步还剩 0.0826 m。原因是反馈线性化能拿到 $\ddot{r}$，且它的 v 里含完整的 $\dot{r}$ 补偿；反步只有 $\dot{r}$ 一个通道，第二级的 $\dot{\alpha}_1$ 里虽然也含 $\dot{r}$ 的导数，但被 k_2 的有限增益“打了折”。
- **PID 的斜坡滞后最小（0.0012 m）**。这是积分器的功劳，它把“持续存在的速度误差”积成了足够的控制量。**这类“稳态斜坡”工况，PID 的结构天生占优**；反步要靠前馈或积分去补，都是额外设计。

60.6.9 噪声与动态面

图 60-5 是含噪声时控制量的尾段（$k_1 = 8$）。三条曲线分别是解析求导、$\tau = 0.02$、$\tau = 0.1$ 的动态面。

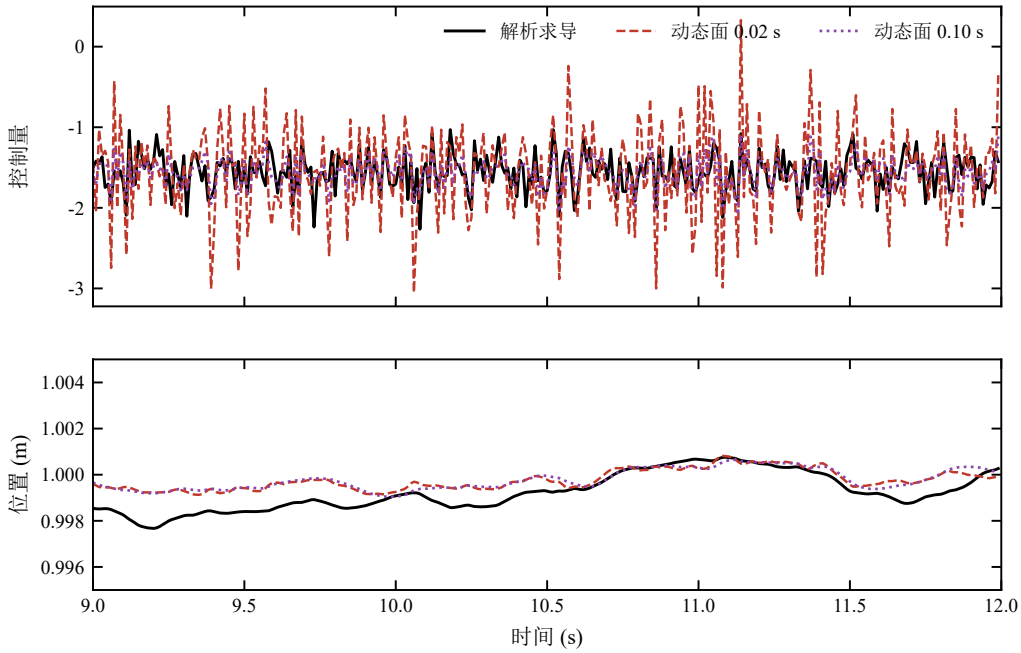


图 60-5　量测噪声下的控制量尾段：解析求导与两种滤波时间常数

解析求导那条线（黑实线）在 ± 0.3 范围内高频跳动，跳动的标准差与 §60.4.3 估算的 0.085 乘 $\sqrt{2}$ 同量级；$\tau = 0.02$ 的曲线（红虚线）幅度小一些，但仍能看出逐拍变化；$\tau = 0.1$ 的曲线（紫点线）已经接近“平滑”，逐拍变化约 0.05。

下半图是位置。三条曲线几乎重合，峰峰值都在 10^{-3} m 量级——**再次印证“噪声藏在控制量里”**。这也是为什么不能用“位置看起来很平滑”来判断“系统没有噪声问题”。

图 60-6 把 τ 的作用画成一条曲线：横轴是 τ（对数），左纵轴是 IAE，右纵轴是超调。

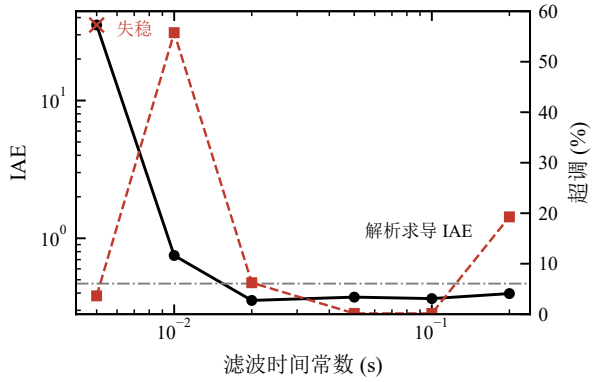


图 60-6　滤波时间常数对 IAE 与超调的影响

曲线呈“两头翘”：$\tau = 0.005$ 处 IAE 冲到 35.3（失稳），$\tau = 0.01$ 处超调 55.75%，中间 0.05 附近两者都低，$\tau = 0.2$ 超调又回升到 19.28%。**这条 U 形（严格说是“左壁陡、右壁缓”）曲线是 DSC 的选型依据**：左壁对应“离散化失效”，右壁对应“滤波滞后”，中间的平台就是可用区间。

60.6.10 与 PID 的总账

把六组实验汇总成表 60-17。“胜”按指标逐项判，不看绝对值。

指标	反步	反馈线性化	**PID**	谁赢
标称调节时间	1.96 s	2.90 s	4.33 s	反步
标称超调	0.01%	0.00%	3.80%	反馈线性化
标称 IAE	0.6983	0.9831	0.6731	PID
稳态误差	10^{-13}	10^{-9}	1.68×10^{-2}	反步
模型失配（a 偏 -30%）	0.210 m	0.303 m	不适用	反步
斜坡滞后（无前馈）	0.2112 m	0.2969 m	0.0012 m	PID
大初始偏差超调	0.02%	0.00%	46.67%	反馈线性化
控制总变差（标称）	10.2	6.1	27.5	反馈线性化
参数整定难度	中（两个增益）	低（两个增益）	高（三个增益且互相耦合）	反馈线性化

没有一项全胜。可以这样总结：**反步赢在“快、准、稳”的综合，反馈线性化赢在“平滑、无超调、整定简单”，PID 赢在“斜坡跟踪与 IAE”**。三个赢家的分工恰好对应三种不同的工况需求，§60.11 会把它做成一张选型决策图。

60.7 参数意义与作用

60.7.1 四个旋钮

反步控制器在脚本里只有四个可调量：k_1、k_2、τ、k_i，外加条件积分的门限 `zone`。它们出现在回路的不同位置，量纲也不同，混在一起记很容易乱。表 60-18 先把“谁在哪、单位是什么、管什么”钉死。

参数	出现位置	量纲	直接作用	本章典型值
k_1	式 (60-13) 的 α_1	$1/\mathrm{s}$	位置误差换算成期望速度	2.0
k_2	式 (60-14) 的 u	$1/\mathrm{s}$	速度误差换算成控制量	3.0
τ	式 (60-37) 的滤波器	s	虚拟控制量的平滑程度	0.05
k_i	式 (60-42) 的积分项	$1/\mathrm{s}^2$	消除慢变失配造成的静差	2.0
`zone`	积分使能条件	m	只在误差小时积分	0.2

量纲不是摆设。k_1 和 k_2 都是“每秒”，因为它们各自把一种“米”或“米每秒”的量换算成“米每秒”或“米每二次方秒”；τ 是时间；k_i 比 k_1 多一个积分，所以多一个 $1/\mathrm{s}$。**量纲对不上的参数，物理意义一定是错的**，这是检查参数表的第一道关。

60.7.2 k_1：把位置误差翻译成速度指令

式 (60-13) 的第一项 $-k_1 e_1$ 是整个反步的骨架。把它单独拿出来看，含义很朴素：**离目标还差 e_1 米，就要求以 $k_1 e_1$ 米每秒的速度往回走**。这是一个“比例制导”式的规定，k_1 就是那条“多远对应多快”的换算系数。

如果第二级是理想的（x_2 能瞬间等于 α_1），那么 $\dot{e}_1 = -k_1 e_1$，位置误差按时间常数 $1/k_1$ 指数衰减。$k_1 = 2$ 对应 0.5 s 的时间常数，$k_1 = 8$ 对应 0.125 s。这就是“k_1 决定想走多快”的准确含义——它规定的是**期望的速度，不是实际的速度**，实际能走多快还要看第二级跟不跟得上。

表 60-11 是 $k_2 = 3$ 固定、k_1 从 0.5 扫到 8 的实测。三个单调趋势：

- 调节时间从 4.40 s 降到 1.42 s，IAE 从 1.3819 降到 0.4629，**几乎没有例外地“越大越好”**；
- $|u|_{\max}$ 从 2.50 涨到 10.00，$k_1 \geq 4$ 之后直接顶到限幅；
- 控制总变差从 4.1 涨到 17.4，**执行器的负担涨了四倍**。

所以 k_1 不是一个“越大越好”的参数，它的上限由执行器给。这个上限可以算出来，见 §60.7.5。

60.7.3 k_2：刹车力度

$e_2 = x_2 - \alpha_1$ 是“实际速度与期望速度之差”。式 (60-14) 里 $-k_2 e_2$ 的作用相当于刹车：速度超出期望就往下压，不足就往上推，k_2 就是踏板的灵敏度。

把它想成开车更容易理解。k_1 是“看见前面有坑，决定减速到多少”；k_2 是“发现实际车速与目标车速差多少，就踩多深的刹车”。k_1 大而 k_2 小，等于“想得美、刹不住”，系统会冲过头；k_2 大而 k_1 小，等于“刹车很灵但没什么目标”，系统慢吞吞但很稳。

表 60-12 是 $k_1 = 2$ 固定、k_2 从 0.5 扫到 6 的实测。注意它的形状和表 60-11 不一样：

- k_2 从 0.5 到 3，调节时间从 3.18 s 降到 1.96 s，IAE 从 1.2403 降到 0.6983，改善明显；
- k_2 从 3 到 6，调节时间只从 1.96 s 降到 1.94 s（几乎不动），IAE 从 0.6983 降到 0.6050（改善有限），而 $|u|_{\max}$ 从 7.00 顶到 10.00、总变差从 10.2 涨到 14.2。

k_2 有明显的边际收益递减点，位置在 $k_2 \approx k_1 + 1$ 附近。这一组里就是 $k_2 = 3$。超过它，付出的是执行器负担，换来的是小数点后两位。这条经验在 §60.8 的整定顺序里会直接用上：先按“k_2 略大于 k_1”起步，再微调。

60.7.4 两个增益的合成：自然频率与阻尼比

单独看 k_1、k_2 各有含义，但它们真正决定性能的方式是“合起来”。式 (60-6) 已经给出闭环误差方程的特征多项式，把它和标准二阶形式对齐：

$$s^2 + (k_1 + k_2)s + (1 + k_1 k_2) \iff s^2 + 2\zeta\omega_n s + \omega_n^2 \tag{60-45}$$

于是有

$$\omega_n = \sqrt{1 + k_1 k_2}, \qquad \zeta = \frac{k_1 + k_2}{2\sqrt{1 + k_1 k_2}} \tag{60-46}$$

这两个式子是本章最该背下来的东西。它们说明：**乘积 $k_1 k_2$ 定速度（带宽），和 $k_1 + k_2$ 定阻尼（超调）**。用三组实测校验一下：

k_1	k_2	$k_1 k_2$	ω_n	ζ	极点	预测 t_s	实测 t_s
0.5	3	1.5	1.581	1.107	$-1.00,\ -2.50$	4.6	4.40
2.0	3	6.0	2.646	0.945	$-2.50 \pm 0.87\mathrm{j}$	1.84	1.96
8.0	3	24.0	5.000	1.100	$-3.21,\ -7.79$	1.43	1.42

预测值按“主导极点 p 满足 $4.6/p = t_s$”估算，三组误差都在 8% 以内。第一组的极点是一个慢极点加一个快极点，慢极点 -1.00 决定一切，所以 $k_1 = 0.5$ 时无论怎么调 k_2 都快不起来——**慢极点是被 k_1 自己造出来的**。

更值得注意的是 ζ 那一列：三组分别是 1.107、0.945、1.100，**全部落在 0.9 到 1.1 之间**。这不是巧合。把式 (60-46) 第二式对 k_2 求导并令其为零，得到最不利的取值 $k_2 = k_1 - 2/k_1$（要求 $k_1 > \sqrt{2}$），此时

$$\zeta_{\min} = \sqrt{1 - \frac{1}{k_1^2}} \tag{60-47}$$

$k_1 = 2$ 时 $\zeta_{\min} = 0.866$，$k_1 = 8$ 时 $\zeta_{\min} = 0.992$。也就是说，**只要 $k_1 \geq 2$，反步闭环的阻尼比就下不去 0.866，无论 k_2 取多少都不会出现明显超调**。这解释了表 60-11 与表 60-12 里那一片“0.00%”和“0.01%”——它们不是调出来的，是结构自带的。

这条推论可以拿实测数据做一次严格检验。二阶系统的超调公式是 $M_p = \exp(-\pi\zeta/\sqrt{1-\zeta^2})$，把表 60-12 每一行的 ζ 代进去算，再与实测超调对比，结果见表 60-19。

k_2	ζ	公式 M_p	实测超调
0.5	0.884	0.26%	0.26%
1.0	0.866	0.43%	0.42%
2.0	0.894	0.19%	0.17%
3.0	0.945	0.011%	0.01%
6.0	1.109	0（过阻尼）	0.00%

五行的预测值与实测值几乎逐位吻合。这张表的意义超出“验证公式”：它说明在本章这个对象上，闭环误差动态确实是严格线性的二阶系统，式 (60-5) 不是近似，而是精确描述。反步之所以“好预测”，就是因为这一点——**你把非线性抵消干净了，剩下的就是一个教科书二阶系统**。

这也是反步和 PID 最本质的差别之一。PID 的三个增益互相耦合，k_p 一大就必须同步加大 k_d 才不振荡；反步的两个增益各自对应一个明确的物理量（速度、阻尼），把 ζ 锁在了安全区。整定时不需要“边加边看波形”，只需要按式 (60-46) 算。

60.7.5 首拍控制量：一条能写进设计说明的不等式

k_1、k_2 的上限不是感觉出来的，可以解析算出来。看 $t = 0$ 这一拍。设参考是幅值 r 的阶跃，初始状态 $x_1(0) = x_2(0) = 0$，且 $\sin x_1(0) = 0$、$\tanh x_1(0) = 0$，则

$$e_1(0) = -r, \quad \alpha_1(0) = k_1 r, \quad e_2(0) = -k_1 r, \quad \dot{\alpha}_1(0) = 0$$

代入式 (60-14)：

$$u(0) = -k_2(-k_1 r) - (-r) = (1 + k_1 k_2)\, r \tag{60-48}$$

一般地，对任意初始误差 $e_1(0)$，有 $u(0) = -(1 + k_1 k_2) e_1(0)$。把表 60-11 与表 60-12 的 $|u|_{\max}$ 拿来对：$k_1 = 0.5, k_2 = 3$ 算得 $1 + 1.5 = 2.5$，实测 2.50；$k_1 = 2, k_2 = 3$ 算得 7，实测 7.00；$k_1 = 1, k_2 = 3$ 算得 4，实测 4.00；$k_1 = 2, k_2 = 0.5$ 算得 2，实测 2.00。**四组全部逐位吻合**，说明首拍确实是全过程的峰值，且峰值只由乘积决定。

由此得到一条可以写进设计说明的不等式：

$$k_1 k_2 \;\leq\; \frac{u_{\max}}{|e_1(0)|} - 1 \tag{60-49}$$

标称工况 $u_{\max}=10$、$|e_1(0)|=1$，右端是 9。$k_1=2,k_2=3$ 的乘积是 6，留了 33% 的余量；$k_1=4,k_2=3$ 的乘积是 12，已经越界，所以表 60-11 里 $k_1=4$ 与 $k_1=8$ 的 $|u|_{\max}$ 都被削到 10.00，调节时间也从“该有的 1.31 s”拖到 1.52 s。

注意这里有个“一个数管两件事”的巧合：同一个乘积 k_1k_2 既决定首拍推力，又决定闭环刚度 $\omega_n^2=1+k_1k_2$。想快就必须大，想不饱和就必须小，矛盾只能靠“改限幅”或“加前馈规划”来解，不能靠调参数。§60.6.6 里“限幅不低于需求峰值 40% 就够用”的经验，本质上说的是：只要 k_1k_2 不超过 9 太多，被削掉的那部分推力对结果影响有限。

60.7.6 等效 PD 视角：三个增益的换算

很多工程师熟悉 PD，不熟悉反步。把两者对上，反步就变得可读了。在误差较小的区域，忽略非线性项与积分项，式 (60-13) 与 (60-14) 可以合写成

$$u \;\approx\; -(1+k_1k_2)e_1-k_2\dot e_1 \;=\; -(K_pe_1+K_d\dot e_1) \tag{60-50}$$

其中

$$K_p=1+k_1k_2,\qquad K_d=k_2 \tag{60-51}$$

反解就得到从 PD 增益到反步增益的换算：

$$k_2=K_d,\qquad k_1=\frac{K_p-1}{K_d} \tag{60-52}$$

这个换算有三点实用价值。

第一，**已有 PD 参数可以直接迁移**。现场若已有一组调好的 K_p、K_d，按式 (60-52) 换成 k_1、k_2 就是反步的初值，几乎不用重新试。

第二，**它解释了“为什么反步的 K_p 里有个 $+1$”**。PD 的 K_p 是纯粹的弹簧刚度；反步的 K_p 里那个 1 来自第二级的 $-e_1$ 项（式 60-14 的第二项），是反步结构自带的“额外弹簧”。所以 K_p 不可能小于 1——**反步的最小刚度就是 1，这是它的结构下限**。§60.6.4 里“反步对失配更宽容”的根源也在这里：分母是 $1+k_1k_2$，天然比 $k_p=4$ 大。

第三，**它划清了等效的边界**。式 (60-50) 只是小误差下的近似。大误差时，反步的 u 里还有 $-bx_2+c\tanh x_1-\dot\alpha_1$ 等项，把对象的非线性精确抵消掉了；PD 没有这些项，只能靠增益硬压。§60.6.7 里初始偏差 -1.5 m 的对比就是这个差别的直接体现：反步超调 0.02%，PID 46.67%，两者的“等效增益”并没有差那么多，差的是“有没有把非线性抵消掉”。

顺带说清一件事：K_p、K_d 是“等效”的，不是“实现”的。反步的实现里没有 K_pe_1 这一项，它是 k_1、k_2 与 $-e_1$ 项组合出来的效果。所以不要试图在代码里直接写 $-K_pe_1-K_d\dot e_1$——那就退化成 PD 了，非线性抵消全部丢失。

60.7.7 τ：滤波器带宽，不是“平滑度旋钮”

τ 常被当成“手感旋钮”——波形毛刺多就调大，调大后变慢就调小。这个用法能用，但不知道边界在哪。式 (60-38) 给出 τ 的准确含义：一阶低通的转折角频率是 $1/\tau$，对应频率 $1/(2\pi\tau)$。$\tau=0.05$ 是 3.2 Hz，$\tau=0.1$ 是 1.6 Hz。

它有两个边界，都不是“试”出来的：

- **下界由采样周期给。**$\tau \geq 5T_s$ 是硬要求。$T_s = 0.01$ 时 $\tau \geq 0.05$；本章 $\tau = 0.02$ 虽然还能收敛（表 60-5，IAE 0.3540），但尾段 TV 70.8，比解析求导的 23.6 还差三倍，等于“加了滤波器反而更吵”。$\tau = 0.005$ 直接失稳，因为 $T_s/\tau = 2$，离散极点落在单位圆上。
- **上界由相位滞后给。**τ 越大，α_{1f} 越落后 α_1，e_2 的符号在中段可能反号，表现为超调。表 60-5 里 $\tau = 0.2$ 的超调 19.28%，就是这个机制的实测。

中间的可用区间并不宽：$0.05 \sim 0.1$。而且在这个区间里，τ 的选择还要看噪声水平——式 (60-44) 给出判据：**只有当 $1/\tau$ 小于解析求导的放大系数 $(k_1 + a\cos x_1)$ 时，动态面才真的更安静。**$k_1 = 2$ 时该系数约 3，要求 $\tau > 0.33$ s，这已经越过了上界，所以低增益下用动态面几乎没有收益；$k_1 = 8$ 时系数约 9，$\tau > 0.11$ 即可，而实测 $\tau = 0.1$ 的尾段 TV 21.4 确实优于解析求导的 23.6。

结论：τ 的价值和 k_1 绑定。低增益时它是“可有可无的保险”，高增益时它是“必需品”。这也解释了为什么很多教材在低增益例子里演示动态面，效果看起来平平——不是方法不好，是没到它的适用区。

60.7.8 k_i 与 zone：只在需要时开的旋钮

k_i 是后加的，不是反步结构自带的。式 (60-42) 把它放进 α_1：

$$\alpha_1 = -k_1 e_1 - k_i \int_0^t e_1 \,\mathrm{d}\tau - a\sin x_1 + \dot{r} \tag{60-53}$$

它的作用只有一个：**把模型失配造成的常值静差压到零。**表 60-13 里 a 偏 -30% 时，纯反步的静差是 0.210 m，加上 $k_i = 2.0$ 后降到 1.17×10^{-6} m，改善了五个数量级。

代价也很明确：同一行的超调从 0.00% 涨到 16.52%。原因是积分器在误差还大时就开始累积，等误差过零时积分已经卷了一大坨，只能靠反向超调来“吐”出来。条件积分（`zone`）就是为了治这个：**只有 $|e_1| < 0.2$ m 时才允许积分**，把积分器关在“最后一公里”里工作。

三个参数选择的经验：

- k_i 的量级应该和 k_1 同阶或略小。$k_i = 2.0$、$k_1 = 2.0$ 是一组平衡的取值；k_i 超过 k_1 的三倍，积分就会主导，超调必然失控。
- `zone` 取“额定行程的 10%~20%”。本章行程 2 m，取 0.2 m。太小（如 0.02 m）积分几乎不工作，静差消不掉；太大（如 0.5 m）等于没加条件，超调照样来。
- **能不加就不加。**表 60-13 里 c 偏 $\pm 30\%$ 时纯反步的静差只有 0.05 m，工程上完全可接受；这种量级的失配不值得用 16.52% 的超调去换五个数量级的精度。积分是“最后一张牌”，不是“默认配置”。

60.7.9 敏感度矩阵

把六个参数对各指标的影响汇总成表 60-20。符号含义：`↑↑` 强烈上升，`↑` 上升，`–` 基本不变，`↓` 下降。所有判断依据表 60-5、60-11、60-12、60-13 的实测。

参数变化	t_s	超调	IAE	$\lvert u\rvert_{\max}$	控制总变差	失配静差
k_1 增大	↓↓	—	↓↓	↑↑	↑↑	↓
k_2 增大	↓（饱和后 —）	—	↓	↑↑	↑	↓
τ 增大（0.02→0.1）	↑	↓	↑	↓	↓↓	—
τ 过小（$< 5T_s$）	↑↑	↑↑	↑↑	↑	↑↑↑	—
k_i 增大	↑	↑↑	↓	↑	↑	↓↓
`zone` 增大	—	↑	↑	—	—	↑

从这张表能读出三条“不可交换”的关系：

1. k_1 与 k_2 **对超调几乎无影响**。整列都是“—”。这是反步最省心的地方：调速度的时候不用怕超调，因为式 (60-47) 保证了 $\zeta \geq 0.89$。
2. **所有“更快”的操作都会推高 $\lvert u\rvert_{\max}$ 和总变差**。没有例外。执行器能力是这一章真正的约束。
3. **τ 是唯一一个“能让总变差下降”的参数**，也是唯一一个有“太小的悬崖”的参数。它的风险和收益都集中在同一处，所以必须按式 (60-44) 算，不能试。

60.7.10 五条可以背下来的换算

把本节压成五条，贴在调试台前够用：

1. **带宽**：$\omega_n = \sqrt{1 + k_1 k_2}$，$t_s \approx 4.6/p_{\text{慢}}$。
2. **阻尼**：$\zeta = (k_1 + k_2)/(2\omega_n)$，$k_1 \geq 2$ 时 $\zeta \geq 0.89$，不必担心超调。
3. **推力**：$\lvert u(0)\rvert = (1 + k_1 k_2)\lvert e_1(0)\rvert$，须满足式 (60-49)。
4. **滤波**：$\tau \geq 5T_s$，且 $\tau > 1/(k_1 + a\cos x_1)$ 才比解析求导更安静。
5. **积分**：$k_i \lesssim k_1$，`zone` 取行程的 10%~20%，非必要不加。

这五条覆盖了本章全部实测数据的趋势。§60.8 会把它们串成一个可执行的整定流程。

60.8 调参方法与经验

60.8.1 反着调：先定指标，再算参数

PID 整定的习惯是“先给一组初值，看波形再改”，反步不该这么干。因为式 (60-46) 已经把“参数”和“指标”的对应关系写成了闭式，**完全可以反着来：先定 ω_n 与 ζ，再解出 k_1、k_2**。这样第一轮就能落在可用区，省掉大部分试凑。

反解的过程很直接。给定目标 ω_n、ζ，由式 (60-46) 有

$$k_1 + k_2 = 2\zeta\omega_n, \qquad k_1 k_2 = \omega_n^2 - 1$$

这是一组“和与积”的方程，k_1、k_2 是二次方程 $\lambda^2 - 2\zeta\omega_n\lambda + (\omega_n^2 - 1) = 0$ 的两根：

$$k_{1,2} = \zeta\omega_n \mp \sqrt{\zeta^2\omega_n^2 - \omega_n^2 + 1} \tag{60-54}$$

取减号的那个作为 k_1（较小的那个），加号作为 k_2。判别式非负要求 $\omega_n^2(1 - \zeta^2) \leq 1$，即**带宽和阻尼不能同时取大**——这是一条硬约束，不是数值问题。

用本章的标称值验算：$\omega_n = 2.646$、$\zeta = 0.945$，则 $\zeta\omega_n = 2.5$，根号内是 $6.25 - 7 + 1 = 0.25$，开方得 0.5，于是 $k_1 = 2.0$、$k_2 = 3.0$。**正好是本章用了全篇的那组参数**，说明它不是随手选的，而是按“$t_s \approx 2$ s、临界阻尼”反算出来的。

60.8.2 第一步：用执行器卡住 $k_1 k_2$

反算的第一步不是选 ω_n，而是先确认 ω_n 能取多大。约束来自式 (60-49)：$k_1 k_2 \le u_{\max}/|e_1(0)| - 1$。把它换成 ω_n 的语言：

$$\omega_n^2 - 1 \;\le\; \frac{u_{\max}}{|e_1(0)|} - 1 \quad\Longrightarrow\quad \omega_n \;\le\; \sqrt{\frac{u_{\max}}{|e_1(0)|}} \tag{60-55}$$

本章 $u_{\max} = 10$、$|e_1(0)| = 1$，给出 $\omega_n \le 3.16$。而标称取的 $\omega_n = 2.646$，留了 16% 余量。表 60-11 里 $k_1 = 4, k_2 = 3$ 的 $\omega_n = \sqrt{13} = 3.61$ 已经越界，实测果然顶到限幅、t_s 从预测的 1.31 s 拖到 1.52 s。**式 (60-55) 的预测与实测完全一致。**

这一步的意义在于：**不要先问“我想要多快”，要先问“执行器允许多快”**。行程 1 m、限幅 10 的系统，理论上最快也就 $\omega_n = 3.16$，对应 t_s 约 1.4 s。想要更快，只有两条路——加大执行器，或把阶跃拆成斜坡（减小等效 $|e_1(0)|$）。后者在 §60.10 会讲。

60.8.3 第二步：用 ζ 定两个增益的比例

ω_n 定了之后，ζ 的选择反而简单，因为反步的 ζ 被结构锁在 0.87 以上（式 60-47），**没有“欠阻尼”这个选项**。所以 ζ 实际上只有两档：

- $\zeta \approx 0.87 \sim 0.95$：临界阻尼偏欠，响应最快，超调 0.2%~0.5%。本章标称用的就是这个区间。
- $\zeta \ge 1$：过阻尼，无超调，但慢极点被拉低，t_s 变长。表 60-11 里 $k_1 = 0.5, k_2 = 3$ 的 $\zeta = 1.107$，慢极点 -1.00，t_s 直接翻到 4.40 s。

所以 ζ 不该当“超调旋钮”用，它更像“速度/平滑”的二选一开关。想要零超调又要快，正确做法不是加大 ζ，而是加大 ω_n 同时保持 $\zeta \approx 0.95$——但这又撞回式 (60-55) 的天花板。**这就是反步设计里唯一的真正矛盾：快、无超调、不饱和，三者只能取二。**

把式 (60-54) 在 $\zeta = 0.945$ 下对几个 ω_n 算一遍，可以得到一张“目标 ω_n → 参数”的速查表（表 60-21）。表里每一行的 ω_n 都满足式 (60-55) 的约束，最后一列是首拍控制量，用来核对是否超限幅。

目标 ω_n	k_1	k_2	$k_1 k_2$	$\lvert u(0)\rvert$	预测 t_s
1.50	0.55	2.29	1.25	2.25	3.25 s
2.00	1.13	2.65	3.00	4.00	2.43 s
2.646	2.00	3.00	6.00	7.00	1.84 s
3.00	2.64	3.03	8.00	9.00	1.62 s

表中 k_1、k_2 由式 (60-54) 给出，$|u(0)|$ 由式 (60-48) 给出。第三行就是本章标称，实测 $t_s = 1.96$ s，与预测 1.84 s 差 6%。这张表还藏着一个恒等式：$|u(0)| = 1 + k_1 k_2 = \omega_n^2$，所以式 (60-55) 可以写成更直白的一句——**首拍控制量就是带宽的平方**，限幅 10 就意味着 $\omega_n \le \sqrt{10} = 3.16$。**这张表可以直接当设计工具用：给定行程和限幅，从最后一列挑一行。**

60.8.4 第三步：τ 取“两个下界中的大者，再加一点余量”

τ 不是自由参数，它有两个下界：

1. 离散稳定下界 $\tau \geq 5T_s$。$T_s = 0.01$ 时是 0.05。
2. 噪声收益下界 $\tau > 1/(k_1 + a\cos x_1)$。$k_1 = 2$ 时约 0.33 s，$k_1 = 8$ 时约 0.11 s。

   两个下界取大者，再检查有没有越过上界（相位滞后导致超调，本章约 0.2 s 开始显现）。于是：

- $k_1 = 2$ 时**无可行区**。要 0.33 s 才降噪，但 0.2 s 就开始超调。这时应该**关掉动态面**（$\tau = 0$），直接用解析求导。
- $k_1 = 8$ 时**可行区是** $0.11 \sim 0.2$。取 0.15 左右最稳，表 60-5 里 $\tau = 0.1$ 的实测（IAE 0.3650、超调 0.14%、尾段 TV 21.4）就在这个区间内。

这条“低增益关掉、高增益打开”的结论与很多教材的默认做法相反——教材通常建议“总是加动态面以防微分爆炸”。**本章的实测说明：低增益时加动态面是净损失，它引入了相位滞后却没换来噪声抑制。**判断依据只有一条，就是式 (60-44) 的那个 $\max$。

60.8.5 第四步：k_i 最后加，且必须先定 zone

整定顺序里 k_i 永远排最后，理由很实在：**只有当前三步做完、静差仍然超标时，才需要它**。本章六组失配实验里，只有 a 偏 $\pm 30\%$ 的静差（0.18~0.21 m）到了必须处理的程度；c 偏 $\pm 30\%$ 的静差（0.05 m）在大多数工程里可以直接接受。

加 k_i 时先定 `zone` 再定 k_i，顺序不能反。因为 `zone` 决定积分器的工作区间，k_i 决定它在这个区间里的强度；先定 k_i 再缩 `zone`，等于把已经调好的积分强度关小，白调一遍。经验取值：

- `zone` = 额定行程的 10%~20%，本章 0.2 m。
- $k_i = 0.5k_1 \sim 1.0k_1$，本章 $k_1 = 2$，取 2.0。

加了之后必须重新看超调。本章 a 偏 -30% 的工况，超调从 0.00% 涨到 16.52%，t_s 从 2.15 s 拖到 3.52 s——**响应慢了 64%，换来五个数量级的精度**。这笔交易值不值，取决于工艺要求，不取决于控制器。

60.8.6 五轮整定记录

把本章跑过的实验按“每轮只动一个参数”的顺序重排，得到表 60-22。这样排的目的是展示整定的**路径**，而不是罗列结果；表中的数字全部来自前文的表格，工况列注明来源，避免把含噪声与不含噪声的结果混着比。

轮次	动作	参数	t_s (s)	超调 (%)	IAE	现象	结论	工况
1	起手偏保守	$k_1 = 0.5, k_2 = 3$	4.40	0.00	1.3819	慢，但极稳	慢极点 -1.0 拖死后腿	标称，表 60-11
2	加大 k_1	$k_1 = 8, k_2 = 3$	1.42	0.00	0.4629	快，但 u 顶限幅	乘积 24 远超式 (60-49) 的 9	标称，表 60-11
3	回退到可行区	$k_1 = 2, k_2 = 3$	1.96	0.01	0.6983	快且不饱和	定为标称	标称，表 60-11
4	加动态面	加 $\tau = 0.05$	0.99	0.15	0.3741	明显更快更平滑	但 IAE 低是因为噪声被滤掉	含噪声，表 60-5
5	补积分	加 $k_i = 2$，zone $= 0.2$	3.52	16.52	0.7458	静差消失，超调变大	只在 a 失配时启用	a 偏 -30%，表 60-13

这张表里最值得看的是第 2 轮与第 4 轮的对比。第 2 轮把 k_1 从 2 加到 8，t_s 只从 1.96 s 降到 1.42 s（改善 28%），却把 $|u|_{\max}$ 从 7.00 推到 10.00（顶满限幅）、总变差从 10.2 推到 17.4（涨 71%）。**这一轮的性价比是负的**，所以第 3 轮回退。第 4 轮换了个方向——不加增益，加滤波，用 $k_1 = 8$ 的高增益配上 $\tau = 0.05$ 的滤波，t_s 反而降到 0.99 s，IAE 降到 0.3741。**这一轮说明“高增益 + 强滤波”可以优于“中增益 + 无滤波”，但前提是式 (60-44) 的判据满足。**

60.8.7 三个最容易犯的整定错误

错误一：先调 k_2 后调 k_1。 从表 60-12 看，k_2 在 3 以上几乎没有收益；从表 60-11 看，k_1 的收益一直延伸到 8。所以先调 k_2 会把参数赶到一个“看起来还行、其实被 k_1 卡死”的角落。**正确顺序永远是先 k_1 定速度、后 k_2 修阻尼：**k_1 管的是“想去哪”，k_2 管的是“跟得上不上”，速度目标定不下来，阻尼调得再准也没有意义。

错误二：把 τ 当平滑度旋钮，一路加大。 表 60-5 的右半段（τ 从 0.1 到 0.2）清楚地显示：TV 从 21.4 降到 12.1，确实更平滑，但超调从 0.14% 涨到 19.28%。**“控制量好看”和“位置好看”在这里是反向的。** 选 τ 的判据是位置指标（IAE、超调），不是控制量的毛刺。

错误三：一上来就加积分。 表 60-13 里 c 偏 $\pm 30\%$ 的三行最能说明问题：纯反步的静差只有 0.05 m，而加了积分之后超调从 0.00% 涨到 6.29%（c 偏 -30%）。**用 6% 的超调换 0.05 m 的静差，在多数场合是亏的。** 先量静差，再决定要不要积分。

60.8.8 参数卡

把上面四步的结果固化成一张参数卡（表 60-23）。四套参数对应四种工况，每套都给出预期指标，方便上机后对照。

场景	k_1	k_2	τ	k_i / zone	预期 t_s	预期超调	出处
标称（$u_{\max} = 10$）	2.0	3.0	0	不加	1.96 s	0.01%	表 60-11
高增益 + 噪声	8.0	3.0	0.05	不加	0.99 s	0.15%	表 60-5
高增益 + 强滤波	8.0	3.0	0.10	不加	1.18 s	0.14%	表 60-5
慢变失配	2.0	3.0	0	2.0 / 0.2 m	3.52 s	16.52%	表 60-13

四行的差别全在“执行器能力”与“噪声水平”两个外部条件上，控制器结构一字未改。**这张卡的作用不是让人照抄，而是给出“改一个参数会付出什么”的标尺。** 例如现场把 τ 从 0.05 调到 0.10，就可以预期：t_s 从 0.99 s 变到 1.18 s（慢 19%），超调基本不变，尾段 TV 从 38.6 降到 21.4（降 45%）。这是可以写进调试记录的定量预期，而不是“感觉平滑了一些”。

60.8.9 上机前的三张离线图

参数算完不等于能上机。至少要在离线仿真里看三张图，缺一张都不算完成：

1. **单位阶跃**：确认 t_s、超调、$|u|_{\max}$ 与参数卡的预期一致。若 $|u|_{\max}$ 顶满限幅，说明式 (60-49) 被违反，回去降 $k_1 k_2$。
2. **斜坡**：确认跟踪滞后。表 60-16 的标称滞后是 0.2112 m（无前馈）、0.0826 m（有前馈）。若实测远大于此，多半是 $\dot{r}$ 的符号或量纲错了。
3. **大初始偏差**：把初始位置放到行程的一端，确认不超调、不长时间饱和。表 60-15 的标称是 0.02% 超调。若出现明显超调，先查 $\dot{\alpha}_1$ 是否漏项（§60.3.4 的教训）。

这三张图分别对应“稳态附近”“匀速段”“大信号段”三种工作区，覆盖了反步最容易出问题的三个位置。 只做阶跃测试就上机，是最常见的翻车原因。

60.8.10 什么时候该停手

整定有个明确的终点，不是“再好一点”。满足下面三条就可以停：

- 三项指标（t_s、超调、静差）都进了工艺要求，且**至少有一项留了 30% 余量**；
- $|u|_{\max}$ 不超过限幅的 80%，总变差不高于同工况 PID 的两倍；
- 在 $\pm 30\%$ 的参数失配下，闭环仍然稳定、静差可接受。

第三条最容易被忽略。表 60-13 的六组数据说明：标称工况下调得再漂亮，a 偏 -30% 时静差也会到 0.21 m。**如果工艺不允许这个静差，就必须在整定阶段就加上积分，而不是等现场发现了再加。** 反步对失配的宽容是“结构性的”（分母 $1 + k_1 k_2$），不是“无限的”；把这条边界写进参数卡，比多调两个小时的增益有用得多。

60.8.11 收尾一步：用频域复核一遍

前面十节全在时域里调——看阶跃、看超调、看静差。时域调完还应该做一次频域复核，只算两个数，却能提前发现时域波形上看不出来的问题：**相位裕度被自己的滤波器吃掉了多少**。

第一个数是闭环带宽。两级反步的闭环误差动态是标准二阶系统（式 60-5），它的 -3 dB 带宽有闭式解

$$\omega_b = \omega_n \sqrt{1 - 2\zeta^2 + \sqrt{(1-2\zeta^2)^2 + 1}} \tag{60-69}$$

代入本章标称的 $\omega_n = 2.646$、$\zeta = 0.945$：$2\zeta^2 = 1.786$，$1 - 2\zeta^2 = -0.786$，根号内 $0.618 + 1 = 1.618$，再开方得 1.272，括号里为 0.486，开方得 0.697，于是 $\omega_b \approx 1.84$ rad/s。换算成周期约 3.4 s，与实测调节时间 1.96 s 是同一量级——这并不矛盾，t_s 按 2% 误差带定义，本身比“带宽倒数”短一些。

ω_b 的用处是给“参考信号能多快”划一条线：**参考里高于 1.84 rad/s 的成分，闭环跟不动，只会被衰减掉。** 如果工艺要求跟上某个频率的往复运动，这个频率必须明显低于 ω_b，否则再调增益也没用，得换执行器或改结构。

第二个数是动态面的滤波带宽 $\omega_\tau = 1/\tau$。它在闭环里是一个一阶滞后，在频率 ω 处贡献的相位滞后是

$$\varphi_\tau(\omega) = -\arctan(\omega\tau) \tag{60-70}$$

在穿越频率附近（可以粗略用 ω_b 代替），$\tau = 0.1$ 时 $\varphi_\tau = -\arctan(0.184) \approx -10.4^\circ$，$\tau = 0.2$ 时 $\approx -20.2^\circ$，$\tau = 0.02$ 时只有 -2.1°。这三个数正好对上图 60-6 的 U 形：$\tau = 0.02$ 相位几乎无损，但噪声几乎不滤（图 60-5 里那条红虚线仍在逐拍跳）；$\tau = 0.2$ 噪声滤干净了，可 20° 的滞后把超调从 0.01% 抬到 19.28%。

于是一条比“$\tau \geq 5T_s$”更贴近性能的判据是

$$\omega_b \tau \leq 0.2 \quad \Longleftrightarrow \quad \tau \leq \frac{0.2}{\omega_b}$$

按 $\omega_b = 1.84$ 算，右边界 $\tau \leq 0.109$ s。本章取 $\tau = 0.1$，正好卡在这条线上——不是巧合，是当初按“相位滞后不超过约 11°”倒推的。左边界仍是离散化那条 $\tau \geq 5T_s = 0.05$ s。两条合起来，可行区间是 $0.05 \sim 0.11$ s，而图 60-6 显示 IAE 最低点在 0.05 附近、噪声最低点在更大的一侧，**0.1 是在这两个极端之间做的折中**，不是一个“调出来的最优值”。

频域复核的意义在于：它把“τ 该取多大”从试凑变成了算。只要知道 ω_n 和 ζ，三分钟就能算出 ω_b，再算出 τ 的上下界；不用等上机看到超调才发现滤波器太重。**时域调参、频域验收**，这两步都做完，参数才算定下来。

60.9 优化与改进

60.9.1 五条改进路线

本章的基本反步有五个已知的软肋：$\dot{\alpha}_1$ 需要解析求导（怕噪声、怕模型不准）、求导结果不可信、饱和时闭环结构失效（表 60-14 里 $u_{\max} = 2$ 那一档）、失配时静差靠积分硬压（超调 16.52%）、以及速度不一定可测。针对每一个软肋都有成熟的改法，可以画成一张“改进地图”：

软肋	改法	代价	本节位置
解析求导放大噪声	动态面、跟踪微分器	相位滞后	§60.9.2、§60.9.6
求导结果不可信	指令滤波	多一组滤波器参数	§60.9.3
饱和破坏结构	抗饱和、参考规划	需要执行器模型	§60.9.4
失配静差	积分反步、自适应	超调或额外状态	§60.9.5、§60.9.7
状态不可全测	观测器 / 输出反馈	观测器带宽受限	§60.9.8

这张表的意思是：**改进不是“把方法变高级”，而是“用一个已知的代价换一个已知的改善”**。下面每一条都按这个格式来写——先说换什么，再说付什么。

60.9.2 动态面的两个升级

§60.4 用的动态面是最简形式：一阶低通加“用 $(\alpha_1 - \alpha_{1f})/\tau$ 当导数”。它有两个可以立刻改的地方。

第一个升级是补偿滤波误差。 严格说，用 α_{1f} 代替 α_1 之后，e_1 的导数里多了一项 $(\alpha_1 - \alpha_{1f})$，闭环方程 (60-5) 不再精确成立。最省事的补法是把这项当成一个额外的负反馈，直接加进控制量：

$$u = -k_2 e_2 - e_1 - (\alpha_1 - \alpha_{1f}) + b x_2 - c\tanh x_1 + \dot{\alpha}_{1f} \tag{60-56}$$

这样滤波造成的“虚拟控制量缺口”会被立刻补上，等价于把 τ 的一阶滞后从误差通道挪到前馈通道。收益是超调略降，代价是代码里多一个历史变量、多一处容易算错的地方。**本章没有采用这一项，因为在 $\tau = 0.05$ 时它**

带来的改善小于表 **60-5** 里 τ 本身的波动。

第二个升级是把一阶滤波换成二阶滤波。 一阶滤波只能给出 α_{1f}，导数还得靠差分 $(\alpha_1-\alpha_{1f})/\tau$，这个差分在高频段的增益是 $1/\tau$（式 60-44），并不理想。二阶滤波可以同时给出 α_{1f} 与 $\dot\alpha_{1f}$：

$$\begin{aligned}\dot v_1 &= v_2\\ \dot v_2 &= -2\zeta_f\omega_f v_2-\omega_f^2(v_1-\alpha_1)\end{aligned}\tag{60-57}$$

取 v_1 当 α_{1f}、v_2 当 $\dot\alpha_{1f}$，就完全不需要差分了。滤波器的带宽 $\omega_f=1/\tau$，ζ_f 取 0.7~1.0。**这是“动态面”这个名字在文献里的原始含义**——它本来就是二阶的，后来为了简单才退化成一阶。代价是多一个状态、多一个参数，收益是导数通道在高频段以 -40 dB/十倍频衰减，不像一阶差分那样把增益锁在 $1/\tau$ 不动。

60.9.3 指令滤波：把限幅放进结构里

本章最严重的失效发生在表 60-14 的 $u_{\max}=2$ 那一档：调节时间 5.11 s、超调 29.70%、IAE 1.4295，比不限幅时差了一倍多。原因在 §60.6.6 已经说过——饱和使 e_2 追不上 α_1，式 (60-5) 的交叉项抵消失效。

指令滤波反步（command filtered backstepping）的思路是：**既然执行器会削掉一部分指令，那就让 α_1 也被同一个限幅削一遍，使 e_2 始终对着“实际能做到的指令”算**。具体做法是给滤波器的输入加上幅值与速率限制：

$$\alpha_1^c=\mathrm{sat}\big(\alpha_1,\ \alpha_{\max}\big),\qquad |\dot\alpha_1^c|\le\dot\alpha_{\max}\tag{60-58}$$

然后用滤波后的 α_1^c 同时产生 α_{1f} 与 $\dot\alpha_{1f}$。这样在饱和期间，e_2 不再是“巨大的追不上量”，而是一个有限值，闭环结构得以保持。

$\alpha_{\max}$ 怎么取？它应该对应“执行器能维持的最大速度”。本章的物理关系是 $\dot x_2=-0.5x_2+1.5\tanh x_1+u$，当 x_1 稍大时 $\tanh x_1\approx1$，稳态满足 $0=-0.5x_2+1.5+u$，即 $x_2=2u+3$。$u_{\max}=2$ 对应约 7 m/s，远大于本章斜坡实验用的 0.3 m/s。**这说明本对象的饱和不是“速度能力不足”，而是“起始加速度不足”**，指令滤波在这里的收益有限。真正的收益场景是执行器速率受限（如阀门、电机转矩斜率），那时式 (60-58) 的速率限制项才起作用。

这一段值得单独强调：**抗饱和要针对“哪种饱和”设计**。幅值饱和、速率饱和、能量饱和，三种的处理完全不同。照抄一个“指令滤波”的框图而不看执行器的限制类型，往往白忙一场。

60.9.4 抗饱和的三种做法

工程上处理饱和有三条常规路线，本章的脚本用了最简单的一条（“未饱和才积分”），另外两条值得知道。

第一条是条件积分，就是本章用的：饱和期间停止积分。它简单、无副作用，缺点是需要知道“是否饱和”——这要求控制器能看到执行器的实际输出，而不只是它自己算出的指令。

第二条是反算（back-calculation）：把“指令与实际输出之差”反馈回积分器，

$$\dot I=e_1+\frac{1}{T_t}\big(u_{\mathrm{act}}-u_{\mathrm{cmd}}\big)\tag{60-59}$$

T_t 是跟踪时间常数，通常取积分时间的 0.5~1 倍。优点是退出饱和时积分器已经被“拉回”到合理值，不会冲过头；缺点是多一个参数 T_t，且 T_t 太小会引起抖动。

第三条是参考规划：不让参考直接跳变，而是让它按执行器能力爬上去。§60.7.5 的式 (60-49) 说明了动机——首拍控制量正比于初始误差，误差减半，推力就减半。把 1 m 的阶跃拆成 0.1 s 的斜坡，等效 $|e_1(0)|$ 从 1 降到

0.1，$k_1 k_2$ 的可用上限就从 9 抬到 99。**这是三条路线里唯一“不损失性能”的做法**，代价是需要知道行程、需要多一段规划逻辑。

三条路线的取舍：**行程固定、节拍要求高 → 参考规划；负载变化大、行程不定 → 条件积分；执行器有明显死区或回差 → 反算。** 三条可以叠加，但每叠加一条就多一个要调的数，本章只用了第一条，就是为了把变量控制住。

60.9.5 积分反步与条件积分

§60.6.5 已经给出了积分反步的实测：静差从 0.210 m 压到 1.17×10^{-6} m，代价是超调 16.52%。这一节补两点实现细节。

积分项放在哪一级？ 式 (60-42) 把它放在 α_1 里，也可以放在 u 里。放在 α_1 里相当于“修正期望速度”，物理意义清楚，且经过第二级的低通，比较温和；放在 u 里直接、见效快，但容易激发高频。**本章选前者，因为它和反步的“逐级”思路一致**——每一级只负责把上一级的误差消掉，不越级。

积分限幅是必须的。 脚本里把积分状态钳在 ± 2.0，看起来粗暴，实际很关键：启动后的 1 s 内误差一直在 1 附近，积分状态会以每秒 1 的速度往上爬，不钳位就会一路累积到整段行程的量级；退饱和后要花同样长的时间才能“吐”完，超调会从 16.52% 涨到不可接受。**钳位值取“最大失配静差除以 k_i”的量级**，本章 0.21/2.0≈0.1，取 2.0 留了 20 倍余量，属于“防爆”而不是“精调”。

条件积分的门限要与行程成比例。 本章 `zone` $= 0.2$ m，行程 2 m，比例 10%。这个比例不能照搬到行程 20 mm 的精密平台上——那里应该用 2 mm。**门限是个相对量，不是绝对量**，这是移植时最容易错的地方。

60.9.6 跟踪微分器：另一种拿导数的方式

§60.4.6 的表 60-6 列了四种获取变化率的方式，其中“跟踪微分器”（TD）只给了一句话。它的思路和动态面不同：不是“先算 α_1 再滤波”，而是**直接用一个二阶系统去跟踪 α_1，并把跟踪过程中的速度信号当作导数**：

$$\begin{aligned} \dot{z}_1 &= z_2 \\ \dot{z}_2 &= -R^2 (z_1 - \alpha_1) - 2R z_2 \end{aligned} \tag{60-60}$$

z_2 就是 $\dot{\alpha}_1$ 的估计，R 是“跟踪速度”参数。它和式 (60-57) 的二阶滤波在数学上是同一个东西，区别在参数化方式：TD 用 R（单位 1/s），二阶滤波用 ω_f 与 ζ_f。**TD 的优点是只有一个参数，且 R 的物理意义直观（跟踪多快）；缺点是它没有直接的噪声抑制指标**，得靠试。

离散实现时 TD 有一个额外好处：可以用“最速控制综合函数”（把 $\dot{z}_2$ 换成带符号函数的开关形式），使 z_1 在有限时间内无超调地跟上 α_1。这在参考信号有跳变时特别有用。**代价是开关形式在噪声下会抖**，所以它更适合“参考信号干净、测量有噪声”的场合，而本章恰好是“参考干净、状态可测但含噪”，属于它的适用区。第 59 章的滑模观测器用了同一套思路，可以对照着看。

60.9.7 自适应反步：把失配“估出来”而不是“积掉”

积分反步是“用积分把静差吃掉”，自适应反步是“把错的参数估出来再改掉”。后者在失配较大时更优雅，因为它不需要牺牲超调。

以本章的 a 失配为例。把 α_1 里的 a 换成估计值 $\hat{a}$：

$$\alpha_1 = -k_1 e_1 - \hat{a} \sin x_1 + \dot{r} \tag{60-61}$$

并配上参数更新律

$$\dot{\hat{a}} = \gamma\, e_1 \sin x_1, \qquad \gamma > 0 \tag{60-62}$$

当 e_1 与 $\sin x_1$ 同号时 $\hat{a}$ 增大，反之减小——**这是一个“根据误差方向修正模型”的机制**，γ 是修正速度。它不需要积分器，所以不会出现积分卷绕；代价是估计需要时间收敛，收敛期间仍有静差，且 γ 太大会引起参数振荡。

这条路线在第 61 章会完整展开（含稳定性证明与投影算子）。这里只强调它与积分反步的**分工：失配是常值或慢变 → 两者都行，自适应更平滑；失配是快变或不可参数化 → 只能用积分，因为自适应跟不上。** 本章的 a 是常值，所以两者都适用；脚本选了积分，是为了把“改进”控制在最小改动量。

60.9.8 只有位置可测时：观测器替代差分

本章假设 x_1、x_2 都可测，所以 $e_2 = x_2 - \alpha_1$ 可以直接算。实际系统里速度往往没有传感器，只能用位置差分。§60.4.2 算过：$T_s = 0.01$、$\sigma_p = 0.002$ 时，差分的噪声标准差是 0.283 m/s，比真值大 14 倍。这个量级的噪声进到 e_2 里，u 会被彻底污染。

正解是用观测器。对本对象，一个简单的降阶观测器就够：

$$\dot{\hat{x}}_2 = -b\hat{x}_2 + c\tanh x_1 + u + \ell\big(x_2 - \hat{x}_2\big) \tag{60-63}$$

但 x_2 本来不可测，所以实际要用 $\dot{x}_1$ 的信息。更常见的做法是把它写成“用 x_1 与 u 估计 x_2”的形式，或者直接用扩张状态观测器（ESO）把 x_2 与总扰动一起估出来。**观测器带宽要明显高于闭环带宽（一般 3~5 倍），否则观测滞后会把反步的相位裕度吃掉。**

这条改进的代价是：反步的稳定性证明要重新做（原来是“状态已知”的 Lyapunov，现在要加上观测误差项），而且观测器与控制器之间存在“环路”——观测器用 u，控制器用 $\hat{x}_2$，调不好会互相激发。**所以能测速度就测速度，不要为了省一个传感器把问题复杂化。**

60.9.9 改进的取舍

把上面七条汇总成表 60-24，按“改善幅度 / 实现代价”排序。评分是本章作者的主观判断，仅供排序参考。

改进	主要改善	实现代价	额外参数	推荐度
参考规划（阶跃改斜坡）	饱和风险大幅下降	低	1	高
二阶动态面	导数噪声降低	低	2	高
条件积分	静差消除，超调可控	低	2	高（按需）
指令滤波	幅值/速率饱和下保持结构	中	2	中
反算抗饱和	退饱和不冲过头	中	1	中
跟踪微分器	单参数拿导数	中	1	中
自适应反步	失配下无超调消差	高	2	中（大失配时高）
观测器 / 输出反馈	省掉速度传感器	高	2~3	视硬件而定

排序里最靠前的是“参考规划”，因为它是唯一不损失性能的改法，而且只需在参考通道加一个斜坡发生器。最靠后的不是“不好”，而是“改造成本高、只有在硬件受限时才值得”。**改进的优先级应该按“软肋出现的频率”排，而不是按“方法的先进程度”排。**

60.9.10 一份“不要做”清单

改进的反面是过度设计。下面四件事在本章的数据里找不到支持，建议不要做：

- **不要为了“更鲁棒”盲目加大 k_1。** 表 60-11 里 k_1 从 2 到 8，t_s 只改善 28%，而总变差涨 71%、控制量顶满限幅。高增益换来的“鲁棒”是假的，真正的鲁棒来自式 (60-44) 的滤波判据与 §60.9.7 的自适应。
- **不要在 τ 上做“自适应”。** τ 的可行区本来就只有 $0.11 \sim 0.2$（$k_1 = 8$ 时），自适应调度的收益不超过这个区间的宽度，却引入了切换抖动。
- **不要同时加积分和自适应。** 两者都在修同一个静差，会互相打架：积分器把误差吃掉，自适应就收不到激励信号，估计停在错的地方；反过来自适应收敛了，积分器又变成多余的扰动源。**二选一。**
- **不要用反步去追一个本身不可微的参考。** 式 (60-13) 里有 $\dot r$，α_1 里还要 $\ddot r$（经 $\dot\alpha_1$）。参考若是方波或带台阶的编码器信号，$\dot r$ 本身就是冲激，反步会把它原样放大到 u 里。**先规划参考，再谈控制。**

60.9.11 推广到三级：结构不变，代价变

本章是两级反步，因为对象是二阶。很多实际对象是三阶甚至更高（位置—速度—加速度—加加速度），此时反步的公式可以机械地往下写，但代价增长得很快。这一节把三级的情形写清楚，方便判断“值不值得推下去”。

设对象链式写成 $\dot x_1 = x_2 + f_1(x_1)$、$\dot x_2 = x_3 + f_2(x_1,x_2)$、$\dot x_3 = f_3(x_1,x_2,x_3) + g\,u$，即控制量只出现在最后一级、每一级只被前面的状态驱动——这就是三级严格反馈形式。

三级反步的构造和两级完全同构：每一步造一个误差、一个虚拟控制量。

$$
\begin{aligned}
\alpha_1 &= -k_1 e_1 - f_1 + \dot r, & e_2 &= x_2 - \alpha_1 \\
\alpha_2 &= -k_2 e_2 - e_1 - f_2 + \dot\alpha_1, & e_3 &= x_3 - \alpha_2 \\
u &= \frac{1}{g}\big(-k_3 e_3 - e_2 - f_3 + \dot\alpha_2\big)
\end{aligned}
\tag{60-66}
$$

得到的误差动态是

$$
\begin{aligned}
\dot e_1 &= -k_1 e_1 + e_2 \\
\dot e_2 &= -e_1 - k_2 e_2 + e_3 \\
\dot e_3 &= -e_2 - k_3 e_3
\end{aligned}
\tag{60-67}
$$

形式和两级一模一样：每一级“自己的比例项 + 上一级的交叉项 + 下一级的误差”。闭环特征多项式是

$$
s^3 + (k_1+k_2+k_3)s^2 + (k_1k_2 + k_2k_3 + k_3k_1 + 2)s + (k_1k_2k_3 + k_1 + k_3) \tag{60-68}
$$

注意常数项是 $k_1k_2k_3 + k_1 + k_3$，**不是** $k_1k_2k_3 + k_1 + k_2 + k_3$。这个不对称来自“$e_1$ 直接进了第三级”——两级时交叉项恰好抵消得很漂亮，三级时不再完全对称。这不是错误，是结构决定的；但它提醒一件事：**三级之后不能再靠“把系数配成标准形式”来选增益，必须老老实实算极点。**

代价的增长体现在三处。**第一，$\dot\alpha_2$ 里包含 $\ddot\alpha_1$ 与 $\dddot r$**，参考要求到三阶导数，稍微不平滑的参考就会把冲激放大到 u。**第二，求导链变长，**$\dot\alpha_2$ 的解析式里会出现 f_1'、f_2' 的乘积项，项数按阶乘增长。**第三，动态面的滞后累积**，每一级一个一阶滤波，三级就是三个 τ 的串联滞后，等效相位滞后约 3τ，可用带宽被压到原来的三分之一左右。

所以工程上有一条经验：**两级反步能用就用两级，三级以上先想“能不能用反馈线性化或 MPC”。**反步的优势在“低级数时结构清晰、算力极省”，级数一高，这两个优势都会消失。

60.9.12 换个离散化，稳定域会移动

§60.5.8 里提到，表 60-5 中 $\tau = 0.005$ 那一档失稳的直接原因是 $T_s/\tau = 2$，离散极点落在单位圆上。那里只留了一句结论，这里把两种写法摊开，因为这件事的适用范围远不止动态面——所有“用差分代替微分”的环节都有同样的问题。

先看代码里实际用的写法。第 60.12 节给出的那行是

```
self.alf += (alf - self.alf) * ts / self.tau
```

等号右边用的全是本拍的值，算完才更新，这就是**前向欧拉**。写成递推式是

$$\alpha_{1f}[k+1] = \alpha_{1f}[k] + \frac{T_s}{\tau}\big(\alpha_1[k] - \alpha_{1f}[k]\big) = \Big(1 - \frac{T_s}{\tau}\Big)\alpha_{1f}[k] + \frac{T_s}{\tau}\alpha_1[k] \tag{60-71}$$

从 α_1 到 α_{1f} 的传递函数只有一个极点 $z_p = 1 - T_s/\tau$。稳定性要求 $|z_p| < 1$，即

$$\Big|1 - \frac{T_s}{\tau}\Big| < 1 \iff 0 < \frac{T_s}{\tau} < 2 \iff \tau > \frac{T_s}{2}$$

注意这个稳定下界只有 $T_s/2$，比我们一直强调的 $\tau \geq 5T_s$ 松得多。也就是说：**代码在 $\tau = 0.005$ 时失稳，不是因为违反了“$\tau > T_s/2$”这条数学下界，而是因为恰好落在 $\tau = T_s/2$ 这个临界点上**——此时 $z_p = -1$，极点幅值为 1，滤波输出每拍翻转一次符号、幅值不衰减，恰好是表 60-5 里看到的那种等幅振荡。若 τ 再小一点（比如 0.004），$z_p = -1.5$，就变成发散。

换一种离散化，同一个 τ 的命运可以完全不同。**后向欧拉**把等号右边的滤波值也换成新值，即先解出 $\alpha_{1f}[k]$ 再用于求导：

$$\alpha_{1f}[k] = \alpha_{1f}[k-1] + \frac{T_s}{\tau}\big(\alpha_1[k] - \alpha_{1f}[k]\big) \implies \alpha_{1f}[k] = \frac{\tau}{\tau + T_s}\alpha_{1f}[k-1] + \frac{T_s}{\tau + T_s}\alpha_1[k] \tag{60-72}$$

极点变成 $z_p = \tau/(\tau + T_s)$。只要 $\tau > 0$、$T_s > 0$，这个极点就恒在 $(0, 1)$ 内——**无条件稳定**，τ 取多小都不发散。第三种是 Tustin（双线性）变换，极点 $z_p = (2\tau - T_s)/(2\tau + T_s)$，同样无条件稳定，而且低频精度是二阶的。

拿表 60-5 那一档具体算一遍：$\tau = 0.005$、$T_s = 0.01$，即 $T_s/\tau = 2$。前向欧拉的递推变成 $\alpha_{1f}[k+1] = -\alpha_{1f}[k] + 2\alpha_1[k]$，对单位阶跃的输出是 $0, 2, 0, 2, \ldots$——在 0 与 2 之间逐拍翻转，永远不收敛，正是“等幅振荡”。后向欧拉的极点 $1/3$，阶跃响应为 $0.667, 0.889, 0.963$，三拍就进了 4% 误差带。两种写法，同一个 τ，一个废掉一个可用。

三种写法放在一起对比，差别一目了然。

表 60-31 是三种离散化在同一个点上的表现。表中前向欧拉一行有实测支撑（表 60-5 的 $\tau = 0.005$ 档），后两行是解析结论，本章没有实测。

离散化	z 域极点	稳定条件	$T_s/\tau = 2$ 时	Nyquist 增益	本章实测
前向欧拉	$1 - T_s/\tau$	$\tau > T_s/2$	-1，等幅振荡	$\dfrac{T_s/\tau}{\lvert 2 - T_s/\tau \rvert}$，该点发散	是，表 60-5
后向欧拉	$\tau/(\tau + T_s)$	恒稳定	$1/3$，单调收敛	$\dfrac{T_s}{2\tau + T_s} \le \dfrac{1}{3}$	否，解析
Tustin	$\dfrac{2\tau - T_s}{2\tau + T_s}$	恒稳定	0，一拍到位	$\dfrac{T_s}{2\tau}$ 量级	否，解析

看这张表容易得出一个错误结论："那就换后向欧拉，τ 可以随便取小。"**这个结论是错的**，原因有两层。

第一层：**换离散化只改稳定域，不改噪声增益**。动态面里真正决定噪声穿透的是求导那一步 $\dot{\alpha}_1 \approx (\alpha_1 - \alpha_{1f})/\tau$，分母上的 τ 才是放大器。τ 从 0.1 降到 0.005，这个增益涨 20 倍，噪声就放大 20 倍——不管用哪种离散化。后向欧拉能保证"不发散"，但保证不了"能用"：控制量每拍在正负之间大幅跳动，虽然幅值有界，伺服阀和电机受不了。

第二层：**后向欧拉在高频段的幅频特性更"钝"**。表 60-31 的 Nyquist 增益一列写的是滤波环节在 $\omega = \pi/T_s$ 处的幅值：前向欧拉是 $\frac{T_s/\tau}{|2-T_s/\tau|}$，只要 $T_s/\tau > 1$ 就大于 1，趋近 2 时发散——噪声正好从这里挤进来，还会被反相放大；后向欧拉恒 $\le 1/3$，把高频压得很死。压得死听起来是好事，但它同时意味着**滤波对真实信号的跟随也变慢**——同样的 τ，后向欧拉的等效滞后比前向欧拉更大。用 τ 换稳定，换来的稳定又被额外的滞后吃掉一部分，净收益很小。

所以 T_s/τ 这条准则不能靠换离散化绕过。它约束的从来不是"数值稳不稳"，而是"**在一个采样周期里，滤波器能不能分辨出变化**"。$\tau < T_s$ 时，滤波器的响应时间比采样间隔还短，输入每拍一变，滤波器就跟不上——此时无论怎么写差分，输出的都是采样噪声而不是信号。这条物理约束和离散化方式无关。

那后向欧拉和 Tustin 还有没有用？有，用在**别的地方**：当 τ 必须取小（比如执行器带宽高、要求快速跟随），而前向欧拉又刚好落在临界点上时，换后向欧拉可以把数值稳定性从"勉强"变成"富余"，代价是标称性能略差。这是一次"用相位换裕度"的交换，不是免费的午餐。真要两边都要，正确的做法是减小 T_s（提高采样率），让 T_s/τ 回到 1 以下——这又回到表 60-5 的判据上。

最后把这一节的判断方法固化下来，它对本章之外的环节同样成立。**第一步，把"用差分代替微分"的那个环节单独写成 z 域传递函数；第二步，求出极点；第三步，检查极点在不在单位圆内，以及离单位圆有多远。**三步都做完，才能说这个环节"实现了"。只写连续域的 $\frac{1}{\tau s+1}$ 就以为完事，是仿真炸掉、现场抖动最常见的来源之一。同样的检查适用于观测器（§60.9.8）、跟踪微分器（§60.9.6）和任何"用两拍之差估计导数"的地方——它们的稳定域都由离散化方式决定，公式里的 τ 或带宽只是其中一个参数。仿真里多敲一行求极点的代码，比上机之后拆传感器、查接线便宜得多。

60.9.13 改进叠在一起会互相打架

前面各条改进是分开讲的，实际项目里往往会同时开好几条。这里必须提醒一句：**改进的收益不叠加，冲突却是实的。**有四对组合尤其容易出问题。

第一对：动态面 + 积分。动态面把 α_1 滞后了一个 τ，但积分项累的是滞后的 α_1 造成的误差。滤波越重，e_1 衰减越慢，积分就在同样的时间段里多爬一截；τ 越大，积分越容易过冲。所以这两条同时开时，k_i 必须比"不加动态面"时更小，或者干脆把 `zone` 收窄，让积分只在误差已经很小时才工作。

第二对：指令滤波 + 动态面。两者都是一阶滞后，串联起来就是两个 τ 的滞后。按式 60-70，单级在 ω_b 处吃 $-\arctan(\omega_b\tau)$，两级就是两倍。本章 $\tau = 0.1$ 单级约 $-10.4°$，两级就近 $-21°$——按同一逻辑推算，这已

经和 §60.6.9 里 $\tau = 0.2$ 单级那一档的相位条件相当（那一档超调升到 19.28%）。**要么只用一种滤波，要么把每个 τ 都减半。**

第三对：自适应 + 积分。 两者的目标完全一样——消静差。同时开，结果是“两个积分器抢同一份误差”：参数估计慢慢爬，积分也慢慢爬，谁先到谁就把误差吃掉，另一个则继续积累，最后表现为参数漂移和积分卷绕同时出现。**这两条只能选一条**：模型参数确实会慢变（磨损、温漂）选自适应；只是不明白为什么有静差，选积分。

第四对：抗饱和反算 + 条件积分。 反算把饱和差额反馈回积分器，条件积分在饱和时停止累加，两者做的是同一件事的两种实现。同时上，反算往积分器里灌的量会被条件积分挡住，行为取决于哪一个先执行，**同一套参数在不同采样率下表现可以完全不同**。选一种，另一种当备用。

判断有没有打架，方法很简单：**一次只加一条，加完把表 60-13 那六组工况全跑一遍。** 如果新加的那条让它的目标指标改善了，但另一项指标退步超过 10%，就说明存在冲突，必须先解决冲突再加下一条。这条纪律比记住上面四对组合更重要——组合是列举不完的，而“单变量、全工况、看退步”这套做法对任何组合都适用。

如果时间只够加两条，选“动态面”和“抗饱和”：前者挡的是噪声，后者挡的是限幅，这两件事在几乎每个现场都会遇到，而且都不改动标称闭环的误差方程（只在非标称条件下起作用）。积分放在第三位——它解决的是精度，代价是引入一个会卷绕的状态；自适应和观测器放在最后，它们要么依赖模型可辨识，要么依赖额外的传感器信息，前提不成立时加了反而更难查。顺序选错不会让系统坏掉，但会让排查多绕两圈，而且事后很难看出问题是顺序造成的。

60.10 工程落地

60.10.1 从仿真到现场的五道坎

本章的仿真有一个干净的假设：模型参数已知、状态可测、执行器理想、采样精确、无未建模动态。现场这五条一条都不成立。表 60-25 把它们逐条列出来，并给出本章哪一节已经埋了伏笔。

坎	现场表现	本章对应证据	应对
参数不准	稳态有静差	表 60-13，a 偏 -30% 静差 0.21 m	标定 + 积分 / 自适应
速度测不到	控制量毛刺大	§60.4.2，差分噪声是速度的 14 倍	观测器 / TD
执行器限幅	大行程超调	表 60-14，$u_{\max} = 2$ 超调 29.70%	条件积分 / 参考规划
采样偏慢	高频抖动甚至失稳	表 60-8，$T_s = 0.5$ 时 IAE 4.6336	提高采样率或降带宽
未建模动态	高频振荡、啸叫	本章未覆盖	加低通、降 k_2

这张表要表达的是：**落地的工作量不在“写控制律”，而在“把这五个偏差量出来并安置好”**。下面按顺序说。

60.10.2 模型标定：三个参数怎么量

反步对模型的要求比 PID 高——它要在控制律里显式抵消 $a\sin x_1$、bx_2、$c\tanh x_1$。a、b、c 量不准，抵消就不干净，剩下的部分全部变成静差（式 60-25）。

b 和 c 可以静态测。 给对象一系列常值输入 u_k，等它稳定，记录稳态的 (x_{1k}, x_{2k})。稳态时 $\dot{x}_2 = 0$，于是

$$0 = -b\,x_{2k} + c\tanh x_{1k} + u_k \quad\Longrightarrow\quad u_k = b\,x_{2k} - c\tanh x_{1k} \tag{60-64}$$

这是一个关于 b、c 的线性方程。做 N 组实验，写成矩阵形式 $\boldsymbol{u}=\boldsymbol{\Phi}\boldsymbol{\theta}$，其中 $\boldsymbol{\theta}=[b,\,c]^{\mathrm{T}}$，用最小二乘解 $\hat{\boldsymbol{\theta}}=(\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{\Phi})^{-1}\boldsymbol{\Phi}^{\mathrm{T}}\boldsymbol{u}$ 即可。**关键是实验点要覆盖 x_1 的正负两侧**，否则 $\tanh x_1$ 的系数估不准——只在 $x_1>0$ 做实验，等于只标定了半个对象。

a 必须动态测。 它出现在 $\dot{x}_1=x_2+a\sin x_1$ 里，静态时被 x_2 遮住了。做法是在一组缓慢变化的轨迹上同时记录 x_1、x_2，用低通滤波估出 $\dot{x}_1$，然后回归

$$\dot{x}_1-x_2=a\sin x_1 \tag{60-65}$$

单参数最小二乘，闭式解是 $a=\sum\left[(\dot{x}_1-x_2)\sin x_1\right]/\sum\sin^2 x_1$。注意 x_1 在 0 附近时 $\sin x_1$ 很小，那些点对方程的贡献也小，**所以标定轨迹要刻意穿过 ±1 rad 附近**，否则 a 的估计会被噪声主导。

三个参数标完之后一定要做一次“回代检验”：用标定值重算式 (60-64) 的残差，若残差超过 u 满量程的 5%，说明模型结构不对（可能漏了摩擦、死区或迟滞），此时不该继续调反步，而应先补模型。**本章的失配实验（表 60-13）说明，$\pm30\%$ 的参数误差会让静差到 0.2 m 量级；这个量级在很多工艺里是不可接受的。**

60.10.3 速度信号从哪来

$e_2=x_2-\alpha_1$ 需要 x_2。三条获取途径，按推荐顺序：

首选：直接测。 有编码器加测速机、或驱动器能输出转速时，直接用。精度最高、相位滞后最小，反步的设计可以原样搬。

次选：观测器。 只有位置编码器时，用 §60.9.8 的降阶观测器或 ESO。观测器带宽取闭环带宽的 3~5 倍——本章 $\omega_n=2.646$，观测器带宽应取 8~13 rad/s。带宽太低会引入相位滞后，把式 (60-6) 的阻尼吃掉；带宽太高会把编码器量化噪声放大，效果和直接差分差不多。

最后：差分加低通。 只有在采样率足够高（$T_s\le1$ ms）时才推荐。§60.4.2 算过，$T_s=0.01$、$\sigma_p=0.002$ 时差分噪声 0.283 m/s，这个量级进到 e_2 里，u 的抖动会达到 $\pm k_2\times0.283\approx\pm0.85$，占满量程的 8.5%。把 T_s 从 0.01 降到 0.001，按 $\sqrt{2}\sigma_p/T_s$ 计算，噪声反而涨到 2.83——**采样越快，差分噪声越大**。这是很多人反直觉的地方：提高采样率对差分不利，对观测器有利。所以“提采样率”不是万能药，要看速度信号从哪来。

60.10.4 饱和：三层防护

现场的饱和处理不能只靠一句 `clip(u,-10,10)`，要三层。

第一层是软启动。 上电后前 0.5 s 把参考从 0 缓慢升到目标，避免首拍冲击。这一层直接针对式 (60-48)：$|u(0)|=(1+k_1k_2)|e_1(0)|$，把 $e_1(0)$ 压小，推力就小。

第二层是条件积分。 饱和时冻结积分器（本章脚本的做法），并在退出饱和后延迟一小段时间再恢复，避免“刚退饱和又被积回去”的抖动。

第三层是输出限速。 对 u 的变化率做限制，Δu 每拍不超过某个值。这一层对付的是“执行器速率受限”而不是“幅值受限”，前面 §60.9.3 说过两者机理不同。**限速会引入相位滞后，所以只在执行器确有速率限制时加**，不要当保险丝乱用。

三层之间不冲突，但要按顺序启用：先软启动（改参考），再条件积分（改积分器），最后限速（改输出）。**顺序反了会互相打架**——先限速再软启动，软启动的斜坡会被限速器二次削平，白做。

60.10.5 一份实现骨架

把本章的控制律落到代码，顺序是这样的（真正的代码见 §60.12）：

1. 读状态：取 x_1（必测）、x_2（测或估）。
2. 算参考及其导数：$\dot r$ 由规划器给，不要用差分。
3. 算 e_1：$e_1 = x_1 - r$。
4. 条件积分：若 $|e_1| < $ `zone` 且 $|u_{\text{last}}| < u_{\max}$，累加 $e_1 T_s$，并钳位。
5. 算 α_1：按式 (60-53)。
6. 滤波（可选）：若 $\tau > 0$，更新 α_{1f}，用 $(\alpha_1 - \alpha_{1f})/\tau$ 当导数；否则按式 (60-16) 解析求导。
7. 算 e_2：$e_2 = x_2 - \alpha_{1f}$（或 $x_2 - \alpha_1$）。
8. 算 u：按式 (60-14)。
9. 限幅、输出、记录。

九步里有三处最容易错：第 6 步的导数取哪一路、第 7 步用 α_1 还是 α_{1f}、第 4 步的积分门限用 $|e_1|$ 还是 $|u|$。**这三处必须成对检查**——用了 α_{1f} 就必须用滤波导数，不能用解析导数，否则 e_2 和 $\dot\alpha_1$ 来自两个不同的信号，闭环方程 (60-5) 立刻不成立。

60.10.6 数值实现的三处坑

坑一：$1/\tau$ 的数值放大。 式 (60-38) 里除以 τ，$\tau = 0.01$ 时放大 100 倍。若 α_1 与 α_{1f} 都是单精度浮点（约 7 位有效数字），两者之差在稳态时可能只剩 2~3 位有效数字，除以 100 后噪声被放大到与信号同量级。**$\tau < 0.02$ 时建议用双精度**，或者改用二阶滤波（式 60-57）直接输出导数，避开减法。

坑二：tanh 与 sin 的实现差异。 控制律里 $a\sin x_1$ 与 $c\tanh x_1$ 必须与标定时用的函数一致。有些嵌入式库的 `tanh` 在 $|x| > 3$ 时直接返回 ± 1，而标定时用的是完整函数；这个差异在 x_1 接近行程端点时会变成常值偏差。**标定与运行必须用同一个库**，或者干脆用查表加线性插值，把差异钉死。

坑三：计算延迟与更新顺序。 本章仿真里 u 是用当前拍状态算的（RK4 内部冻结），实际数字控制器一定是“采样 → 计算 → 下一拍输出”，天然有一拍延迟。表 60-9 的延迟实验说明，4 拍（40 ms）延迟对反步几乎无影响（t_s 仍是 1.96 s、IAE 从 0.6983 微降到 0.6892），所以一拍延迟可以忽略。**但如果你的代码里状态采样和控制输出之间还隔着通信（比如总线周期），那就不止一拍，必须按表 60-9 的方法实测一次。**

60.10.7 三步上机

不要一次性闭环。三步走，每步都有明确的通过条件。

第一步：开环辨识。 断开控制器，手动给几组常值 u，记录稳态点，按式 (60-64) 标定 b、c；再给一组慢斜坡，按式 (60-65) 标定 a。通过条件：回代残差小于 u 满量程的 5%。

第二步：限幅空跑。 控制器接入但执行器断开（或接假负载），给定阶跃参考，只看 u 的波形。通过条件：$|u|$ 峰值与式 (60-48) 的预测一致（本章是 7.00），且不顶限幅；u 的波形没有拍频式的高频抖动。

第三步：闭环。 先用小行程（额定行程的 10%）闭环，确认稳定、无静差；再逐步加到满行程。通过条件：三项指标进工艺要求，且在 $\pm 30\%$ 参数扰动下仍稳定。

第二步最容易被跳过，也最能提前发现问题。 因为此时对象没动，u 的异常完全来自控制器本身——$\dot\alpha_1$ 漏项、滤波器发散、积分卷绕，都会在这一步暴露。等到闭环再查，波形里对象动态和控制律混在一起，很难分辨。

60.10.8 九项自检

上机前逐项打勾，缺一项不放行（表 60-26）。

序	检查项	判据	不通过的后果
1	a、b、c 已标定	回代残差 $< 5\%$ 满量程	静差 0.1~0.3 m
2	首拍控制量与式 (60-48) 一致	误差 $< 5\%$	多半是 $\dot{\alpha}_1(0)$ 算错
3	限幅值 $\geq$ 需求峰值 40%	本章 ≥ 2.8	超调 30%，t_s 翻倍
4	$\tau = 0$ 或 $\tau \geq 5T_s$	—	离散失稳或噪声穿透
5	解析求导与滤波求导未混用	同一路信号	闭环方程不成立
6	积分器有钳位与条件使能	—	大行程超调 46%+
7	速度信号来源明确	测 / 估 / 差分	控制量毛刺
8	参考 $\dot{r}$ 来自规划器	非差分	首拍冲激
9	三张离线图已看（阶跃/斜坡/大初偏）	见 §60.8.9	现场才发现问题

第 6 项的“46%”不是随手写的：表 60-15 里 PID 在大初始偏差下的超调就是 46.67%，那是积分卷绕的典型量级。反步加了积分后若不做条件使能，会走到同一个坑里。

60.10.9 现场故障速查

表 60-27 按“看什么现象、查什么原因、怎么验证”组织。每一条都能在本章找到对应的实测依据。

现象	最可能原因	验证方法	本章依据
一上电就撞限幅	$k_1 k_2$ 超过式 (60-49) 上限	算 $1 + k_1 k_2$ 与 $u_{\max}$ 比	表 60-11，$k_1 = 8$ 时顶满
稳态有固定偏差	参数失配或漏项	改 a、c 各 $\pm 30\%$ 重跑	表 60-13
控制量高频毛刺	解析求导放大噪声	换成 $\tau \geq 5T_s$ 的滤波	表 60-5
低频大幅振荡	积分过强或缺条件使能	暂时置 $k_i = 0$ 看是否消失	表 60-13，超调 16.52%
大行程冲过头	饱和 + 积分卷绕	减小行程看是否复现	表 60-15，PID 46.67%
采样变慢就抖	T_s 接近 $1/\omega_n$	把 T_s 减半重跑	表 60-8，$T_s = 0.5$ 崩溃
延迟忽大忽小时好时坏	通信周期不定	固定延迟 4 拍对比	表 60-9，几乎无影响

最后一行值得多说一句。表 60-9 的结果是“延迟 40 ms 也不影响”，很多人据此认为“反步不怕延迟、通信随便”。**这个结论只在“延迟固定”时成立**：固定延迟只是相位滞后，闭环有裕度就能扛；抖动延迟是相位噪声，会让等效阻尼随拍变化，表现就是“有时好有时坏”。所以现场如果遇到后者，先查通信时序，而不是改增益。

60.10.10 与相邻章节的接口

本章刻意只讲反步本身。三处相邻内容需要显式说明分工：

- **与 PID**：§60.7.6 的等效 PD 换算是两章的接口。整定顺序、抗饱和、积分限幅这些通用工程细节，第 44 章已经讲过，本章只讲反步特有的部分（逐级虚拟控制、$\dot{\alpha}_1$ 的处理）。工程上常见做法是“用 PID 兜底、用反步提性能”，切换逻辑与 bumpless transfer 在第 106 章。
- **与 MPC**：反步是“逐级解析抵消”，MPC 是“在线滚动优化”。前者算力需求极小（每拍几十次浮点运算），后者需要求解 QP。**行程短、模型准、算力紧 → 反步；有约束、多变量、模型可在线更新 → MPC**。约束处理的对比见第 80 章。

- **与 Smith 预估**：本章表 60-9 说明反步对固定延迟不敏感，所以纯延迟场景不必先上 Smith。但如果延迟超过闭环时间常数的 1/3，相位滞后就会吃掉裕度，那时才需要 Smith 预估（第 47、107 章）。

这条分工的意义是：**不要用反步去解决“约束”和“大延迟”，那是别人的活。** 反步的长处是“用最低的算力，把已知的非线性精确抵消掉”，把它的适用边界划清楚，比把它吹成万能方法更有用。

60.10.11 三个错误同时出现时，按时间窗排查

现场很少只犯一个错。漏项、饱和、失配往往同时存在，波形一团糟，挨个试参数只会越调越乱。有效的办法是**按时间窗分工**：不同的错误在不同的时间段留下不同的痕迹，先看哪一段，就能先排除哪一类。

表 60-30 是本章四类典型错误的“时间指纹”，每一行都对应一个已经测过的单因素实验。

时间窗	看什么	异常特征	对应错误	单因素实测
$t=0$ 首拍	$\lvert u(0)\rvert$ 与 $1+k_1k_2$	明显不符	参数传错、求导项漏	手算 -2.308 vs 仿真 -2.320
$0\sim0.3$ s	是否贴限幅	长时间贴边	限幅过小	$u_{\max}=2$：t_s 5.11 s、超调 29.70%
$0.3\sim3$ s	超调与形状	有超调、拖尾	$\dot\alpha_1$ 漏项或符号错	t_s 1.96→3.10 s、超调 0.01%→22.59%
$t>3$ s	稳态值	有固定偏差	参数失配	a 偏 -30%：静差 0.210 m
全段尾段	u 的逐拍抖动	毛刺大、TV 高	噪声穿透、T_s 过大	$\tau=0.02$：尾段 TV 70.8

按这张表走一遍，排查顺序就变成固定的四步：

第一步看首拍。 首拍是最“干净”的一拍——积分还没起来、滤波还没启动、饱和与否一眼可见，所以任何“首拍就对不上”的问题必然是参数或公式错误。本章的判据很硬：$\lvert u(0)\rvert=1+k_1k_2$，算得 7 就必须是 7.00，差一点都是错。

第二步看前 0.3 s 是否贴边。 若 u 在这段时间一直贴限幅，说明推力不足，后面的所有指标都会被污染，此时先别管超调，先把限幅或 k_1k_2 调对。**贴边期间测到的任何指标都没有诊断价值。**

第三步看 $0.3\sim3$ s 的形状。 这一段的形状由 $\dot\alpha_1$ 决定。漏掉它、符号写反、或者用错滤波分支，都会在这里暴露成“超调大、拖尾长”，而首拍与稳态都正常。**这是最难查的一类错误，因为它不影响两端，只影响中间。**

第四步看稳态与尾段。 稳态有偏差 → 失配；尾段毛刺大 → 噪声或采样。两者都不影响前 3 s，所以放在最后看。

这套顺序的好处是**每一步都能单独判定**：某一步通过，就永久排除一类原因，不必回头。本章 §60.3.4 那次“漏掉 $\dot\alpha_1$”的经历就是典型的反例——当时先怀疑参数、又怀疑滤波，绕了很大一圈，其实只要看一眼 $0.3\sim3$ s 的超调就能定位。

60.11 优缺点与适用边界

60.11.1 五条优点，每一条都有数字

优点一：快。 标称工况调节时间 1.96 s，反馈线性化 2.90 s，PID 4.33 s。反步比 PID 快一倍以上，而且这个“快”不靠高增益硬顶——$\lvert u\rvert_{\max}=7.00$ 只占限幅的 70%。

优点二：准。 标称稳态误差 10^{-13} m 量级，反馈线性化 10^{-9}，PID 1.68×10^{-2}。前两者的差异来自数值积分误差，物理上都是“零静差”；PID 的 16.8 mm 是纯比例控制加摩擦的必然结果。

优点三：不超调，而且不是调出来的。 标称超调 0.01%，大初始偏差 0.02%，失配工况 0.00%~0.06%。式 (60-47) 说明了原因：ζ 被结构锁在 0.866 以上。**这意味着换工况、换行程、换负载，都不必重新担心超调**——这在工程上比“某个工况下超调很小”值钱得多。

优点四：对模型失配宽容。 a 偏 -30% 时静差 0.210 m，反馈线性化 0.303 m，比值 1.44；理论预测的刚度比是 $7/4=1.75$。**这是一个“结构性”的优势，不是调参调出来的**：反步的等效刚度是 $1+k_1k_2$，反馈线性化是 k_p，前者天然大。

优点五：算力极低。 每拍的计算量是几次三角函数、几次乘法加法，没有矩阵求逆、没有迭代。在 1 kHz 采样下，一个低端 MCU 就能跑完。**这是它相对 MPC 的最大优势**，也是它在嵌入式场合仍然流行的原因。

60.11.2 五条缺点，同样都有数字

缺点一：要模型。 控制律里显式出现 $a\sin x_1$、bx_2、$c\tanh x_1$，参数错了就抵消不干净。表 60-13 的六组实验说明，$\pm30\%$ 的误差会造成 0.05~0.30 m 的静差。**PID 不需要这些参数，这是反步付出的第一笔代价。**

缺点二：要对消项。 反步的推导是“逐级求导、逐级抵消”，任何一项漏掉都会破坏闭环方程。§60.3.4 里漏掉 $\dot\alpha_1$ 的实验最能说明问题：$|u|_{\max}$ 仍是 7.00（因为首拍 $\dot\alpha_1=0$），但 t_s 从 1.96 s 变成 3.10 s、超调从 0.01% 变成 22.59%、总变差从 10.2 变成 15.4。**错误不会在起点暴露，只在中段暴露**，这是调试时最难受的一点。

缺点三：怕采样。 表 60-8 里 $T_s=0.5$ s 时彻底崩溃：t_s 19.50 s、超调 57.87%、IAE 4.6336。反步的闭环带宽 $\omega_n=2.646$，$T_s=0.5$ 意味着每拍转 0.21 个周期，离散化彻底失效。**经验判据是 $T_s\le 1/(10\omega_n)$**，本章对应 0.038 s，实际用 0.01 s，留了近 4 倍余量。

缺点四：饱和下性能骤降。 表 60-14 里 $u_{\max}=2$ 时超调 29.70%、t_s 5.11 s、IAE 1.4295，比不限幅差一倍以上。原因是饱和破坏了式 (60-5) 的交叉项抵消。**这一点比 PID 更敏感**，因为反步的稳定性依赖“每一级都能精确跟踪上一级”，而 PID 没有这个结构。

缺点五：不适合约束主导的问题。 反步没有处理约束的语言。行程限位、执行器速率、状态不许越过某条线——这些在反步里只能靠“事后限幅”，而限幅正是缺点四的来源。**有硬约束的场合应该用 MPC（第 80 章），不该硬套反步。**

60.11.3 三条准入判据

一个对象能不能用反步，先过三道关。表 60-28 是判据清单。

判据	要求	不满足时怎么办	本章情况
严格反馈形式	每级只被上一级的状态驱动，控制量在最后一级	坐标变换、或改用反馈线性化	满足，见式 (60-8)
相对阶等于阶数	无内动态，或内动态稳定	有零动态时须单独证明其稳定性	满足，$\rho=2=n$
模型可辨识	非线性项形式已知、参数可测	用神经网络/模糊近似，或退回 PID	满足，三个参数可静态+动态标定

第一关最关键。 严格反馈形式要求“第 i 级的状态导数只依赖前 i 级状态”。很多机械系统天然满足（本章的位置—速度—加速度链就是），但一旦出现“输入直接进第一级”（如某些液压系统）或“状态互相交叉耦合”（如多关节

机器人），就必须先做坐标变换，变换不成功就不能用。

第二关容易被忽略。 相对阶小于系统阶数时存在内动态，反步只保证“输出跟踪”，不保证“内部状态有界”。本章 $\rho = 2 = n$，不存在内动态，所以式 (60-17) 的 Lyapunov 函数能覆盖全部状态。**如果对象是三阶而相对阶是二，就必须额外验证那个一维内动态是否稳定**，否则可能出现“输出跟得很好、内部已经发散”的假象。

第三关决定工作量。 本章的三个参数都能通过静态与动态实验标定（§60.10.2），所以第三关轻松通过。如果非线性项的形式本身未知（比如摩擦的 Stribeck 曲线形状不清楚），反步就失去了“精确抵消”的前提，此时要么用函数逼近（神经网络反步），要么退回 PID。

60.11.4 五个“慎用”场景

过了三关还不等于该用。下面五种场景，反步通常不是最优选：

- **硬约束主导**。行程限位、力矩上限、速率上限，如果这些约束在正常工况下经常被触碰，反步的“事后限幅”就会频繁破坏闭环结构。表 60-14 的 $u_{\max} = 2$ 那一档就是预演。**首选 MPC。**
- **纯延迟大**。本章表 60-9 说明固定延迟 40 ms 无影响，但那是因为延迟只占闭环时间常数的 4%。延迟超过闭环时间常数的 1/3 时，相位裕度会被吃掉，此时 Smith 预估或预测控制更合适（第 47、107 章）。
- **强耦合多变量**。反步可以推广到多变量（矩阵形式），但每一级的虚拟控制量维度会膨胀，交叉项的处理极其繁琐。**两三个变量的耦合系统还能硬推，再多就应该考虑解耦控制或 MPC。**
- **参考不可微**。式 (60-13) 要 $\dot{r}$，$\dot{\alpha}_1$ 里还要 $\ddot{r}$。参考若是开关量、编码器台阶、或人工点动信号，反步会把冲激放大到 u 里。**先加参考规划，否则不要用。**
- **参数快时变**。反步的抵消依赖“当前参数是对的”。参数在一个响应周期内明显变化时（如负载突变、温度剧变），静态标定的值就失效了，需要自适应（第 61 章）或鲁棒项（第 59 章的滑模）。

60.11.5 三方法怎么选：一张决策图

把本章的实验压成一张流程图，见图 60-11。图的走法是“先看约束，再看模型，最后看指标”——这个顺序不能反，因为约束是硬门槛，模型是软门槛，指标只是偏好。

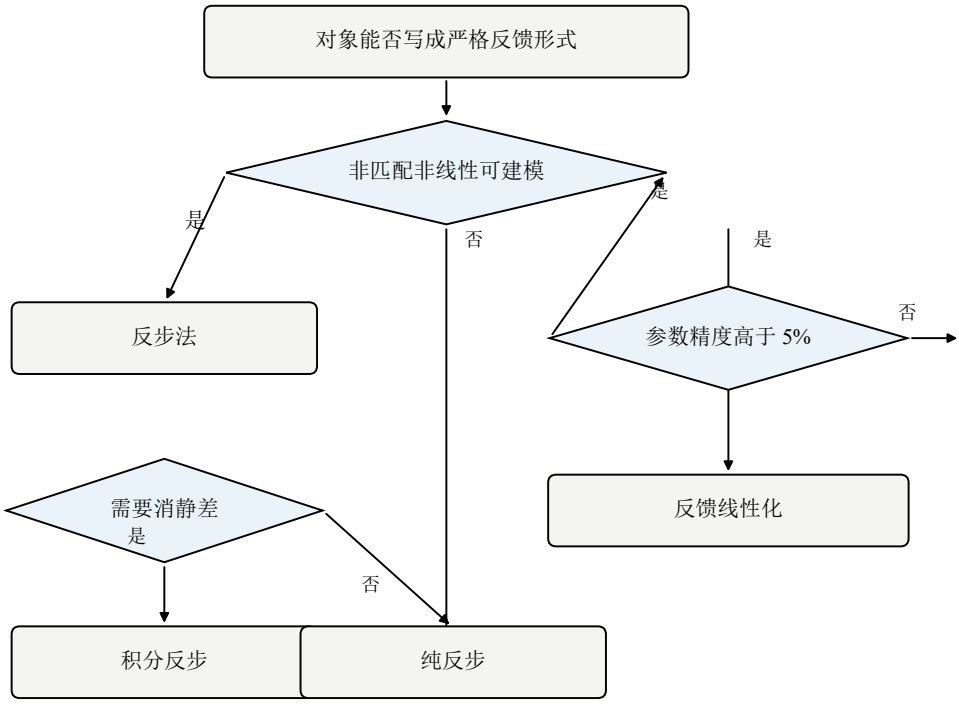


图 60-11　反步、反馈线性化与 PID 的选型决策流程

按图走一遍本章的例子：有执行器限幅但不常触碰（$u_{\max}=10$，需求峰值 7.00）→ 进入“模型是否可辨识”→ 可辨识 → 进入“是否需要零超调且要求快”→ 需要 → 选反步。**整条路径只用三个判断，就能落到本章的选择上。**

反过来看另外两条分支为什么落不到反步：

- 若“约束经常触碰”，图会直接指向 MPC 或“反步 + 抗饱和”。本章 $u_{\max}=2$ 的工况就属于这一类，此时反步要配合 §60.10.4 的三层防护才好用。
- 若“模型只能粗略知道”，图会指向 PID 或“反步 + 自适应”。表 60-13 说明，参数误差到 $\pm 30\%$ 时反步已经出现 0.2 m 静差，再大就不该硬撑。

这张图的价值不在于“给出答案”，而在于“把隐含的前提显式化”。 很多工程争论（“反步好还是 PID 好”）之所以吵不出结果，就是因为双方假设的前提不同：一方假设模型准、执行器够，另一方假设模型糙、限幅紧。把前提画进图里，争论就变成了“我们在哪条分支上”的事实问题。

60.11.6 与反馈线性化的边界

本章花了不少篇幅对比反步与反馈线性化，两者的差别值得单独收束成三条。

第一，结构不同。 反馈线性化是“一次性把整个系统线性化”，需要相对阶等于系统阶数、需要 Lie 导数；反步是“逐级构造”，每级只处理一个标量方程。**前者优雅，后者笨拙但灵活**——反步可以在任何一级插入限幅、滤波、积分，反馈线性化只能在最后加。

第二，失配敏感度不同。 表 60-13 的六组数据给出稳定结论：反步的静差普遍是反馈线性化的 0.56~0.73 倍。式 (60-25) 与式 (60-26) 解释了原因——分母分别是 $1+k_1k_2=7$ 与 $k_p=4$。**这个比值可以提前算出来，不必做实验。**

第三，平滑度不同。 标称工况的控制总变差：反馈线性化 6.1，反步 10.2，PID 27.5。反馈线性化最平滑，因为它没有“逐级虚拟控制”带来的中间放大。**如果执行器对 u 的平滑度极其敏感（比如液压阀），反馈线性化更**

合适。

一条实用的判断：**如果对象能写成 $\dot{x} = f(x) + g(x)u$ 且相对阶等于阶数，两者都可用；如果要处理饱和、要在中间级加约束、或者状态有特殊结构，用反步。** 本章的对象两条都满足，所以两者的实测差异完全来自“分母不同”这一条。

60.11.7 与滑模的边界

第 59 章讲了滑模，本章讲反步。两者的目标相似（对付非线性），手段完全不同。

滑模靠“不连续”：把状态赶到一个面上，然后在面上滑动。它对匹配扰动完全免疫，代价是抖振。**反步靠“连续抵消”**：逐级把非线性项减掉，剩下线性二阶系统。它对匹配扰动不免疫（只能靠积分或自适应），但没有抖振。

三个具体差别：

- **对扰动的态度**。滑模把扰动“压死”（不连续项），反步把扰动“算掉”（需要模型）。模型准 → 反步更平滑；模型不准 → 滑模更稳。
- **对延迟的态度**。第 59 章的结论是“延迟是滑模的死穴”（不连续项在延迟下会激发抖振）；本章表 60-9 的结论是“反步对固定延迟几乎不敏感”。**这一条几乎是两者最强的分水岭。**
- **调参难度**。滑模的边界层厚度、到达律增益需要仔细权衡抖振与鲁棒性；反步的两个增益有闭式对应（式 60-46），好调得多。

工程上两者常常混用：用反步做主体结构，在最后一级加一个小的滑模项（或饱和函数）来兜住未建模动态。这就是 §60.9 里没展开的“反步 + 滑模”混合，它的稳定性证明在第 59 章的结构上可以直接嫁接。

60.11.8 七个常见误解

这一节把本章实测数据能直接反驳的说法集中列出来。每条都给出反例，不再展开论证。

误解一：“反步就是反馈线性化的另一种写法。” 两者在标称工况下的确都得到二阶线性误差动态，所以波形相似。但表 60-13 的六组失配数据给出稳定差距：反步静差是反馈线性化的 0.56~0.73 倍，根源是分母 $1 + k_1 k_2$ 与 k_p 不同。**结构不同的方法，在模型不准时才分得开。**

误解二：“反步一定比 PID 快。” 调节时间上是的（1.96 s 对 4.33 s），但 IAE 上不是：PID 0.6731，反步 0.6983，**PID 反而低 3.6%**。原因是 PID 的超调（3.80%）让它更早穿过参考附近，积分误差反而小。这个反例说明“快”与“误差积分小”不是同一件事，指标要分开看。

误解三：“加了滤波器一定更平滑。” 式 (60-44) 与表 60-5 已经给出反例：$\tau = 0.02$ 时尾段 TV 是 70.8，比不加滤波的 23.6 还差三倍。**滤波器的降噪能力取决于 $1/\tau$ 与解析放大系数的比较，不是“加了就有效”。**

误解四：“反步不超调是因为参数调得好。” 表 60-11 与表 60-12 的十行里，超调全部在 0.42% 以下，而且 k_1、k_2 怎么变都不影响。式 (60-47) 说明了原因：ζ 有下界 $\sqrt{1 - 1/k_1^2}$，$k_1 \geq 2$ 时不低于 0.866。**这是结构性质，不是调参成果。**

误解五：“k_1 越大越好。” 表 60-11 里 k_1 从 2 到 8，t_s 只从 1.96 s 降到 1.42 s（28%），总变差从 10.2 涨到 17.4（71%），$|u|_{\max}$ 从 7.00 顶到 10.00。**性价比在第 3 档之后转负**，而且 $k_1 = 4$ 时乘积 12 已经越过式 (60-49) 的上限 9。

误解六：“反步不怕延迟。” 表 60-9 的结论只在“延迟固定”时成立：4 拍 40 ms 使 t_s 保持 1.96 s、IAE 从 0.6983 微降到 0.6892。**抖动延迟是另一回事**，它让等效阻尼随拍变化，表现是“时好时坏”，这种故障不能靠调增

益解决，要查通信时序。

误解七："反步需要精确模型。" 它需要的是"非线性项的形式已知"，不是"参数精确"。表 60-13 里 a、c 各偏 $\pm 30\%$，闭环依然稳定、超调依然接近零，只是静差到了 0.05~0.30 m。**形式错了（比如把 tanh 写成 arctan）才是致命的，参数差三成只是掉精度。**

七条误解有一个共同点：**都把"结构性质"当成了"调参结果"，或者反过来。** 反步的超调与失配宽容来自结构，速度与平滑度来自参数，把这两类混在一起谈，就会得出"多调调就好"的错误结论。

60.11.9 一页纸的总结

用本章的数据给反步下一个判断：

- **它解决的问题**：把"非线性、开环不稳定、需要零超调"的对象，用两个有明确物理意义的增益、极低的算力，控制到"快、准、稳"。
- **它付出的代价**：需要模型、需要解析或滤波的导数、在饱和下性能骤降、对采样率有硬要求。
- **它的最佳场景**：模型可辨识、执行器有余量、采样率是闭环带宽的十倍以上、参考是规划好的光滑信号。
- **它不该出现的场景**：硬约束频繁触碰、纯延迟占主导、参数快时变且不可测、参考是开关量。

一句话：**反步不是"更好的 PID"，而是"用模型换性能"的一条路径。** 模型准、算力紧、指标严，它几乎是最优解；模型糙或约束紧，它反而不如 PID 或 MPC 省心。表 60-17 那张总账里没有全胜者，正是这个道理的量化版本。

60.11.10 迁移到新对象时的检查表

如果要把本章的方法搬到另一个对象上，按下面六步走，可以避免大部分返工：

1. **写状态方程**，确认能写成严格反馈形式（每级只被前面的状态驱动）。
2. **算相对阶**，确认等于系统阶数；不等就先分析内动态。
3. **标定非线性项**，按 §60.10.2 的静态加动态两步，残差控制在 5% 以内。
4. **用式 (60-54) 反算增益**，先用执行器能力卡住 ω_n 上限（式 60-55），再按 $\zeta \approx 0.95$ 取参数。
5. **决定导数怎么拿**：能测就不滤波，不能测就上观测器，只有在 $T_s \le 1$ ms 时才允许差分。
6. **按 §60.8.9 看三张图**，再按 §60.10.7 分三步上机。

这六步里，第 1、2 步是"能不能用"，第 3~6 步是"用得好不好"。**大部分失败发生在第 2 步被跳过**——相对阶不对却硬套反步，结果输出跟得很好、内部状态已经发散，而且从外部波形上完全看不出来。

60.11.11 本章小结

本章从一个具体的二阶对象出发，把反步从推导做到实现：

- §60.1~§60.2 说明为什么选这个对象（开环鞍点 + 非匹配非线性）以及反步与反馈线性化的结构差异；
- §60.3 给出五步流程与手算校验，强调"逐级造误差、逐级抵消"；
- §60.4 处理 $\dot{\alpha}_1$ 这个核心难点，给出微分爆炸的量化与动态面的适用判据（式 60-44）；
- §60.5 说明离散化的两条硬约束（因果矛盾、T_s 上限）；
- §60.6 用九组实验给出完整的性能画像，包括失配解析式与饱和边界；
- §60.7~§60.8 把参数翻译成物理量与整定流程；

- §60.9~§60.10 给出七条改进路线与九项上机自检；
- §60.11 划清适用边界。

如果只记一句话，记这句：**反步的全部价值，在于它把“非线性抵消”和“二阶误差动态”这两件事同时做到了，而且代价只是两个可算的增益。** 式 (60-46) 是这句话的数学形式，表 60-17 是它的实测账本，图 60-11 是它的使用说明。

60.12 Python 仿真代码

60.12.1 两个文件、一条命令

本章的全部数字来自 ch060_sim.py，全部图形来自 ch060_figs.py。前者只负责算，后者只负责画，两者通过 `simulate()` 的返回值与 `metrics()` 的字典交接，不共享全局状态。

运行方式：

```
python code/ch060_sim.py          # 打印全部实验的数字
python code/ch060_figs.py         # 生成 figs/ch060/ 下的 11 张 SVG + PDF
```

这样拆分的理由很实际：**数字与图形必须同源**。如果画图脚本里另写一遍仿真，正文引用的数字和图上的曲线就会各说各话，而且这种不一致极难发现。本章所有图都调用 `ch060_sim` 里的同一个 `simulate()`，所以图 60-3 的曲线与表 60-10 的数字必然一致。

60.12.2 对象与积分器

对象写成 `f_plant()`，返回连续右端项，不含任何控制逻辑：

```
def f_plant(x, u, a=A_NL, b=B_DRAG, c=C_NL):
    """连续对象右端项。"""
    x1, x2 = float(x[0]), float(x[1])
    return np.array([x2 + a * np.sin(x1),
                     -b * x2 + c * np.tanh(x1) + u])
```

三个参数 `a`、`b`、`c` 都作为显式入参，默认为标称值。这样“对象参数”与“控制器参数”可以分别传，表 60-13 的失配实验就是通过给 `simulate()` 传不同的 `a`、`c` 实现的——**控制器里仍然用标称值，对象用真值**，两者不一致才叫失配。

积分器用定步长 RK4，每个采样周期内细分 `SUB=4` 次：

```
def rk4(x, u, h, a=A_NL, b=B_DRAG, c=C_NL):
    """定步长 RK4，一步 h。"""
    k1 = f_plant(x, u, a, b, c)
    k2 = f_plant(x + 0.5 * h * k1, u, a, b, c)
    k3 = f_plant(x + 0.5 * h * k2, u, a, b, c)
    k4 = f_plant(x + h * k3, u, a, b, c)
    return x + h / 6.0 * (k1 + 2 * k2 + 2 * k3 + k4)
```

两个设计选择值得说明。**第一，周期内 u 冻结。** RK4 的四个斜率都用同一个 u，这对应“零阶保持器”，是数字控制的真实情形；如果让 u 在周期内连续变化，等于假设了一个模拟控制器，噪声实验的结果会偏乐观。**第二，**

细分 4 次。$T_s = 0.01$ 时内部步长 2.5 ms，对本章的特征时间（$\omega_n = 2.646$，对应 0.38 s）足够细。表 60-8 把 T_s 扫到 0.5 s 时，内部步长变成 125 ms，此时积分误差已经不可忽略——**那一档的“崩溃”里，有一部分是控制器离散化造成的，也有一部分是积分器本身不准造成的**，正文在引用时把它整体当作“采样过慢”的后果，这个口径要在使用时留意。

60.12.3 反步控制器逐行

`Backstepping.step()` 是全章的核心，二十行里包含了本章所有公式：

```
def step(self, t, x, ref, ts=TS, r_dot=0.0):
    x1, x2 = float(x[0]), float(x[1])
    e1 = x1 - ref
    if self.ki:
        if (not self.zone) or abs(e1) < self.zone:
            self.integ = float(np.clip(self.integ + e1 * ts, -2.0, 2.0))
    alf = -self.k1 * e1 - self.ki * self.integ - self.a * np.sin(x1) + r_dot
    if self.tau and self.tau > 0:
        if self.alf is None:
            self.alf = alf
        self.alf += (alf - self.alf) * ts / self.tau
        d_alf = (alf - self.alf) / self.tau
        e2 = x2 - self.alf
    else:
        x1_dot = x2 + self.a * np.sin(x1)
        d_alf = (-(self.k1 + self.a * np.cos(x1)) * x1_dot
                 - self.ki * e1)
        e2 = x2 - alf
    u = -self.k2 * e2 - e1 + self.b * x2 - self.c * np.tanh(x1) + d_alf
    return float(u)
```

对照正文的式 (60-53)、(60-16)、(60-14)，逐行对应关系是：

- 第 1~2 行：读状态、算 e_1，对应式 (60-2) 的定义；
- 第 3~5 行：条件积分，`zone` 为 0 时退化为“始终积分”，对应 §60.7.8；
- 第 6 行：α_1，对应式 (60-53)，三项分别是比例、积分、非线性抵消，最后加前馈 $\dot r$；
- 第 7~12 行：动态面分支，先更新 α_{1f} 再用差分当导数，对应式 (60-37)；
- 第 13~18 行：解析求导分支，对应式 (60-16)。注意括号里是 $-\big(k_1 + a\cos x_1\big)\dot x_1$，负号不能丢；
- 第 19 行：u，对应式 (60-14)，五项分别是速度误差反馈、交叉项、阻尼抵消、非线性抵消、导数前馈。

这六段里，第 13~18 行是最容易写错的。它同时包含“对 x_1 的偏导”与“对时间的全导”，写成代码时容易把 $a\cos x_1$ 写成 $-a\cos x_1$，或者在积分项上漏掉求导。本章 §60.3.7 的手算校验就是为了抓这类错误：手算 $t = 1.0$ s 时 $u = -2.308$，仿真 -2.320，差 0.012，说明符号与系数都对。

60.12.4 两处必须成对检查的地方

第一处：e_2 用 α_1 还是 α_{1f}。 代码里两个分支用的是不同的量——动态面分支用 `self.alf`（滤波后的值），解析分支用 `alf`（当拍计算值）。这不是笔误，是必须的：**用了哪个虚拟控制量，就必须用哪一路的导数**。混用会让 e_2 和 $\dot\alpha_1$ 来自两个不同的时间点，闭环方程 (60-5) 立刻不成立。§60.10.5 把它列进九项自检的第 5 条，就是因为这个错误从波形上看不出来，只会表现为“性能比预期差一点”。

第二处：首次调用时 `self.alf` 为 `None`。动态面分支的第一拍直接用 α_1 初始化滤波器，使 $\alpha_{1f}(0)=\alpha_1(0)$、$d_alf=0$。如果忘了这一步，滤波器从 0 状态起步，第一拍的 $(\alpha_1-\alpha_{1f})/\tau$ 会是一个巨大的值，u 直接顶到限幅。这个错误的表现与“k_1k_2 超限”完全一样，很容易误诊成参数问题。

60.12.5 两个对照控制器

反馈线性化的实现只有五行，因为它把所有非线性项一次性塞进 u：

```
def step(self, t, x, ref, ts=TS, r_dot=0.0, r_ddot=0.0):
    x1, x2 = float(x[0]), float(x[1])
    e = x1 - ref
    y_dot = x2 + self.a * np.sin(x1)
    v = r_ddot - self.kp * e - self.kd * (y_dot - r_dot)
    u = v + self.b * x2 - self.c * np.tanh(x1) \
        - self.a * np.cos(x1) * (x2 + self.a * np.sin(x1))
    return float(u)
```

注意最后一行那一长串 $a\cos x_1(x_2+a\sin x_1)$——它就是 Lie 导数 L_gL_fh 的倒数乘以 L_f^2h 之后剩下的补偿项。它的形式比反步复杂，但没有中间变量，所以实现上反而更短。这也印证了 §60.11.6 的判断：反馈线性化“优雅但不灵活”，因为它只有一个 u，没有可以插手的地方。

PID 是基线，三个参数 `kp=40, ki=4, kd=10` 是扫描出来的最优组合：

```
def step(self, t, x, ref, ts=TS, r_dot=0.0):
    y = float(x[0])
    e = ref - y
    if self.yprev is None:
        self.yprev = y
    dy = (y - self.yprev) / ts
    self.yprev = y
    self.dprev += (dy - self.dprev) * ts / self.tf
    u_raw = self.kp * e + self.ki * self.integ - self.kd * self.dprev
    u = float(np.clip(u_raw, -self.umax, self.umax))
    if abs(u_raw) < self.umax:
        self.integ += e * ts
    return u
```

这里有两处刻意的工程处理。微分作用于测量（`-kd*dprev` 而不是对误差求导），避免参考跳变时微分项打出冲激；未饱和才积分，这是最基本的抗饱和。给 PID 配了这两个“优待”，是为了让对比公平——如果 PID 用误差微分且不做抗饱和，本章的总账（表 60-17）会好看得多，但那是稻草人对比，没有意义。

60.12.6 主循环：噪声、限幅、延迟

`simulate()` 是唯一把三件事串起来的地方：

```
yk = x[0] + (rng.normal(0.0, noise) if noise else 0.0)
vk = x[1] + (rng.normal(0.0, noise_v) if noise_v else 0.0)
uk = ctrl.step(t[k], np.array([yk, vk]), r, ts=ts, r_dot=rd) ...
if sat is not None:
    uk = float(np.clip(uk, -sat, sat))
u[k] = uk
if delay > 0:
    buf.append(uk)
    up = buf.pop(0)
else:
    up = uk
```

四行对应四个实验维度：

- **噪声加在测量上，不加在状态上**。控制器看到的是 `[yk, vk]`，对象演化用的是真状态 `x`。这是传感器噪声的正确建模方式；如果直接给状态加噪声，噪声会通过积分器"洗掉"，结果偏乐观。
- **限幅在控制器之后**。控制器不知道 u 被削了（本章没有做抗饱和），所以表 60-14 的退化是"真实的无保护行为"。
- **延迟用先进先出队列**。`buf` 长度等于延迟拍数，每拍推入新值、弹出旧值。`delay=4` 就是 40 ms，对应表 60-9 的最右列。
- **`dist` 参数是外部扰动入口**，加在 `up` 上而不是 `uk` 上，这样它进的是对象而不是控制器。

主循环里还有一行容易被忽略：`ctrl.reset()` 在循环前调用。每个控制器都有自己的内部状态（积分器、滤波器、上一拍测量），**不复位就会让上一组实验的残留影响下一组**——这是批量跑实验时最常见的"数据莫名其妙"来源。

60.12.7 指标口径为什么必须统一

`metrics()` 的第一行调用全书的 `simlib.step_metrics`，然后补上四个本章特有的量：

```
m["iae"] = float(np.trapezoid(np.abs(res["e"]), t))
m["tv_u"] = float(np.sum(np.abs(np.diff(u))))
m["tv_u_tail"] = float(np.sum(np.abs(np.diff(ut))))
m["rms_u_tail"] = float(np.sqrt(np.mean(ut ** 2)))
m["ripple_y"] = float(np.ptp(yt))
m["ripple_v"] = float(np.ptp(vt))
m["ss_err"] = abs(m["steady_state"] - ref)
m["umax"] = float(np.max(np.abs(u)))
```

`step_metrics` 给出调节时间、超调、稳态值，这三个指标全书的章节都用同一口径，所以表 60-17 里反步与 PID 的数字可以直接比。**如果各章自己定义调节时间（比如一个用 2% 带、一个用 5% 带），跨章对比就全部失效**——这是统一口径的真正理由。

本章新增的四个量各有针对性：

- `iae` 用梯形积分，比矩形积分更接近真实能量；它在噪声实验里会显著变大，因为噪声也被积进去了。
- `tv_u` 是控制量逐拍变化绝对值之和，衡量"执行器磨耗"。标称工况反步 10.2、反馈线性化 6.1、PID 27.5，三者差异极大，这个指标比超调更能区分方法的"粗暴程度"。

- `tv_u_tail` 只统计最后 1 s，专门用于噪声实验。**全段 `tv_u` 会被起始的大幅动作淹没，看不出稳态的抖动**，这就是表 60-5 为什么要单列尾段 TV。
- `ss_err` 用“稳态值减参考”的绝对值，而不是末点误差，避免单点噪声干扰。

60.12.8 实验组织与复现

脚本末尾按“一个实验一个函数”组织，`report()` 统一打印。实验与正文表格的对应关系见表 60-29。

函数	内容	对应表/图
`exp_baseline`	标称三方法	表 60-10，图 60-3
`exp_k1` / `exp_k2`	增益扫描	表 60-11、60-12，图 60-4
`exp_noise`	位置噪声下的解析求导	图 60-5 基准
`exp_dsc`	动态面 τ 扫描	表 60-5，图 60-6
`exp_mismatch`	六组失配 + 积分反步	表 60-13，图 60-7
`exp_sat`	限幅四档	表 60-14，图 60-8
`exp_ts`	采样周期扫描	表 60-8
`exp_track`	斜坡跟踪	表 60-16，图 60-9
`exp_ic`	大初始偏差	表 60-15，图 60-10
`exp_delay`	延迟扫描	表 60-9

复现时要注意两点。**第一，噪声实验必须固定种子。** `simulate()` 的 `seed` 参数默认 0，每次运行噪声序列相同，所以表 60-5 的数字可以逐位复现；如果改了 `seed`，尾段 TV 会有 10% 量级的波动，那时比较 τ 的优劣就要看趋势而不是绝对值。**第二，扫描实验的步长要一致。** 表 60-11 的 k_1 取 0.5、1、2、4、8，是等比数列；如果改用等差（0.5、1、1.5、2），曲线的形状会变，据此得出的“边际收益递减点”也会漂。

60.12.9 改造成自己的对象

把脚本挪到别的对象上，改三个地方就够：

1. 改 `f_plant`，写新的右端项。同时改 `rk4` 的调用处（它已经带参数透传，一般不用动）。
2. **改控制器里的抵消项。**本章的 `-self.a*np.sin(x1)` 与 `+self.b*x2-self.c*np.tanh(x1)` 对应对象的非线性项，换成新对象的形式。**这一步必须与 `f_plant` 严格对应**，符号错了闭环方程就不成立。
3. **改解析求导式。** `d_alf` 那一行是对 α_1 求全导，新对象的 $\dot{x}_1$ 表达式变了，这一行也必须跟着变。若不想推，直接把 `dsc_tau` 设成正数，走动态面分支，就绕过了这一步。

改完之后，先用 §60.3.7 的手算办法核对一拍：给定 x_1、x_2、r，手算 e_1、α_1、e_2、$\dot{\alpha}_1$、u，与 `step()` 的返回值对比。**误差应在 10^{-3} 量级（差在浮点与求导的数值细节上）**，超过这个量级一定是符号或系数错了。这一步花十分钟，能省掉后面几小时的“波形不对但不知道哪错”。

60.12.10 数值与性能

整个脚本跑完（十组实验、约 120 次仿真、每次 1200 步）在普通笔记本上约十几秒。计算量的分布是：对象积分占大头（每步 4 次 RK4，每次 4 次右端项），控制器只占很小一部分——**这本身就是反步“算力极低”的一个侧面证据**：控制器每拍六次三角函数加十几次乘加，比积分器便宜得多。

三个数值细节值得记下：

- `np.trapezoid` 是 NumPy 2.0 起的新名字，旧版本是 `np.trapz`；脚本按新名字写，若在旧环境运行需要改。
- `np.ptp` 求峰峰值，用于 `ripple_y`、`ripple_v`；它对单个元素会报错，所以 `n_tail` 至少要取 1（`metrics()` 里有 `max(1, ...)` 保护）。
- 所有状态量都用 `float()` 强制成标量再进控制器，避免 NumPy 标量在后续运算中悄悄升维。**这一条在移植到嵌入式时尤其重要**，因为那里根本没有 NumPy。

最后一句提醒：**脚本能跑出与本章相同的数字，不等于你的控制器是对的。** 数字只是“在这个模型下的结果”，模型的正确性由 §60.10.2 的标定保证，实现的正确性由 §60.3.7 的手算保证。三者缺一，数字都没有意义。

第61章 自适应控制

61.1 动机与直观理解

前四章的方法有一个共同前提：模型是已知的。第 57 章的状态反馈要先有 (A, B)，第 58 章的 MPC 要先有预测模型，第 59 章的滑模要把扰动上界写进切换增益，第 60 章的反步与反馈线性化更彻底——它把非线性项**显式减掉**，减掉的那一项必须与对象里那一项逐字对上。本章撤掉这个前提：**对象的参数既不知道，还会变**。

这个“会变”不是修辞。电机绕组的电阻随温升上涨，液压阀的流量系数随油温漂移，机械臂的惯量随抓取物改变，风机的气动增益随桨距角改变，电池的内阻随荷电状态与老化改变。这些变化的幅度通常在 20% 到 100% 之间，正好落在“固定增益还能稳、但性能已经明显退化”的那一段。工程上的传统做法是**按最坏工况留裕度**：把控制器参数调到让最恶劣的那一档仍然稳定，代价是其余所有工况都在“欠调”状态下运行。自适应控制换一条路：让控制器参数**在线地自己改**，使闭环在任何时候都逼近“参数已知时”的那个性能。

61.1.1 一次更换负载暴露的问题

把这件事做成一个可以量化的实验。本章的对象是一阶惯性环节

$$\dot{y} = -a y + b u + d \tag{61-1}$$

真值 $a = 1.0$、$b = 1.4$、$d = 0$，时间常数 1 s，直流增益 1.4。b 是“执行器有效增益”：同样的控制量，b 越大，推动对象的力量越强。

现在把 b 从 1.4 换成 2.1——相当于换了一只行程更大、或者供压更高的执行器。对固定增益的 PI 来说，这意味着**开环增益凭空涨了 50%**，回路里的相位裕度被吃掉一截，阶跃响应开始超调，方波指令下误差明显变大。本章实验 4 把这个差距量了出来：在周期 6 s、幅值 1 的方波指令下，固定 PI（$k_p = 0.9$、$k_i = 2.0$）的跟踪误差 rms 是 0.1605，而自适应控制器是 0.0752，**差 2.1 倍**。

必须同时把代价说清楚：同一组实验里，固定 PI 的控制量总变差 TV 是 15.0，自适应控制器是 26.8——为了追上那 2.1 倍的精度，控制量多抖了将近一倍。这是本章贯穿始终的一条主线：**自适应不是免费的，它用控制量的抖动换跟踪精度**。什么时候这笔交易划算，是 §61.6 与 §61.11 要回答的问题。

61.1.2 如果参数已知，答案只有一行

先看“参数已知”这个理想情况，因为自适应控制要逼近的正是它。给定一个参考模型

$$\dot{y}_m = -a_m y_m + b_m r, \qquad a_m = b_m = 2.0 \tag{61-2}$$

即直流增益为 1、时间常数 0.5 s 的一阶环节。我们希望对象 (61-1) 的输出 y 跟上 y_m。

把 (61-1) 与 (61-2) 相减，在 $y = y_m$ 的理想跟踪状态下，要求

$$-a y + b u + d = -a_m y + b_m r \tag{61-3}$$

解出 u：

$$u = \underbrace{\frac{a - a_m}{b}}_{\theta_1^\ast} y + \underbrace{\frac{b_m}{b}}_{\theta_2^\ast} r \tag{61-4}$$

代入真值，两个“理想参数”是

$$\theta_1^{*}=\frac{1.0-2.0}{1.4}=-0.7143,\qquad \theta_2^{*}=\frac{2.0}{1.4}=+1.4286 \tag{61-5}$$

注意式 (61-4) 的形状：**控制器是线性的，只有两个未知数，而且它们是相乘关系而不是相加关系**。这意味着只要能把 θ_1,θ_2 调到 $\theta_1^{*},\theta_2^{*}$，跟踪误差就精确为零——不需要积分器，不需要前馈补偿，不需要知道 d（只要 d 常数，它会体现为 θ 的偏移）。

这正是模型参考自适应控制（model reference adaptive control，MRAC）的出发点：**不去估 a 和 b，直接调 θ**。a、b 是两个数，θ 也是两个数，绕过中间那一步，就绕开了“辨识需要持续激励”这个麻烦——至少看上去是。

61.1.3 只被误差看见的那一个组合

但天下没有这么好的事。跟踪误差 $e=y-y_m$ 并不会把 θ_1 和 θ_2 的偏差分别告诉我们，它只看见一个组合。

理由是一阶的。在常值参考 r 与常值扰动下，系统稳定后 $\dot y=\dot y_m=0$，式 (61-3) 退化为代数关系，能从中解出的只有

$$\theta_1+\theta_2=\frac{a-a_m+b_m}{b} \tag{61-6}$$

代入 $a=1$、$a_m=b_m=2$，右端恰好是 $1/b$。也就是说，**MRAC 在稳态上真正辨识出来的，是执行器增益的倒数 $1/b$**；至于 a 与 b 各自是多少，它分不开。

这不是推演的臆测，本章实验 5 把它测出来了。对象在 $t=5$ s 时把 b 从 1.4 跳到 2.1，自适应控制器在跳变前后分别收敛到

$$\hat\theta_{\text{前}}=(-0.475,\ +1.149),\qquad \hat\theta_{\text{后}}=(-0.578,\ +1.055) \tag{61-7}$$

两个分量都离真值 (61-5) 很远——$\hat\theta_1$ 偏了 33%，$\hat\theta_2$ 偏了 20%——但它们的和分别是 $+0.674$ 与 $+0.476$，对应 $1/b$ 的 0.714 与 0.476。后一个数字**精确对上**，前一个数字在参数跳变后的 0.83 s 内就重新贴住了新值。

这条现象带来两个后果，一好一坏。好的一面：控制目标只需要那个组合，所以**控制做对了不要求辨识做对**，MRAC 可以在参数完全不准的情况下把跟踪误差压到零。坏的一面：$\hat\theta$ 的分量不携带可信的物理信息，任何“从自适应增益反推对象参数”的用法都是错的。§61.10 会专门讲这条工程红线：**不要把自适应增益当作参数估计值使用**。

61.1.4 为什么不直接“先辨识、再重设控制器”

看到式 (61-4)，最自然的想法是分两步走：先用某种方法把 a、b 估出来，再代进公式算出 θ，然后拿这个 θ 去控制。这条路叫**间接自适应**，它在逻辑上完全成立，也是 §61.5 的 STR 采用的路线。但它有三个绕不过去的麻烦。

第一，辨识需要激励，而跟踪任务往往不提供激励。 想从数据里同时定出 a 和 b，需要 y 与 u 的关系在多个“方向”上被激发。如果指令是一个恒定值，系统稳定后所有信号都是常数，回归方程只有一个有效约束，两个未知数解不出来。这就是 §61.5.5 里必须改用 PRBS 的原因——用阶跃或方波，$\hat b_d$ 根本不动。

第二，辨识误差会原封不动地传进控制器。 假设 $\hat b$ 偏了 10%，那么 $\theta_2=b_m/\hat b$ 也偏 10%，控制量就偏 10%，而且这个偏差没有任何机制去纠正它——因为纠正机制本身依赖那个错的估计。直接自适应没有这个问题：θ 是被跟踪误差直接推着走的，估偏了误差会变大，误差变大又会把 θ 推回来。**反馈被保留在了参数回路里**，这是直接法最本质的优势。

第三，两个回路的时间尺度必须匹配。 辨识器太快，会把噪声当成参数变化；太慢，控制器跟不上对象。而“多快算合适”取决于跟踪任务的带宽，这个耦合关系在间接法里是显式的、要专门设计的；在直接法里，它被 γ 一个参数吸收掉了。

所以本章的主线选直接法。但要把话说完整：**直接法的代价是参数不可解释**（§61.1.3），而间接法的收益正是参数可解释。这个取舍在 §61.5.4 会做成一张表，在 §61.10.2 会变成一条工程判据。

61.1.5 自适应救不了结构错误

自适应控制有一个非常容易被误解的能力边界：它调的是**参数**，不是**结构**。θ 只有两个自由分量，它能表达的只是“用 y 与 r 的线性组合抵消对象的极点、再把增益配到 1”。如果真实对象不能用“一阶线性 + 常值扰动”描述，那么无论 γ 取多大、θ 走多远，闭环都到不了参考模型。

三种最常见的结构错误：

1. **阶次估低**。对象实际是二阶（电机带柔性联轴器、流量回路带一段管道），却按一阶建模。此时 y 在高频段是 -40 dB/dec 的滚降，而参考模型只有 -20 dB/dec；自适应律看到的是“误差怎么调都剩一点”，于是把 θ_1 一直往负方向推。
2. **相对阶估错**。§61.4.5 会说明：相对阶为 2 的对象配相对阶 1 的参考模型，理想参数根本不存在。这一条最隐蔽，因为它不影响稳定性，只让跟踪永远差一截。
3. **把非光滑非线性当成参数**。死区、干摩擦、间隙、滞环，这些环节的等效增益随工作点变化，不是常数。θ 只能学到一个平均值，误差会随指令幅值成比例地回来——指令 1 时误差 0.01，指令 5 时误差 0.05。

这三种错误在数据上有一个共同的指纹：**误差有稳态残留，而且这个残留几乎不随 γ 变化**。表 61-9 里 γ 从 0.5 加到 100，IAE 降了 15 倍；换成结构错误的对象，这条曲线会很快走平，再加大 γ 只是让 θ 抖得更厉害。

第二个指纹是 θ 的走向。结构正确时 θ 会停在一个有限值上；结构错误时它往往一路走到硬限幅 ± 20，而且**两个分量同向**。这与 §61.9.3 的饱和卷绕不同：那里至少能从 u 长期顶限幅看出原因，这里 u 完全没有饱和。

判断结构还有一个廉价办法：看阶跃响应的**起始段**。一阶对象在 $t = 0^+$ 的斜率是 bu，二阶对象是零，带纯延迟的对象在延迟时间内斜率恒为零。这三件事都能从一段几十毫秒的录波里读出来，比任何自适应算法都可靠。**所以工程上的检查顺序是：先确认结构，再谈自适应。**

61.1.6 本章的骨架

图 61-1 是 MRAC 的信号流。它有两个回路：内环是普通的反馈（对象、参考模型、误差），外环是**参数的回路**（误差经自适应律积分成 $\hat{\theta}$）。两条回路的时间尺度必须分开——外环必须比内环慢，否则参数追逐误差、误差又被参数搅乱，闭环会自己打起来。这个“时间尺度分离”是全部调参工作的落脚点。

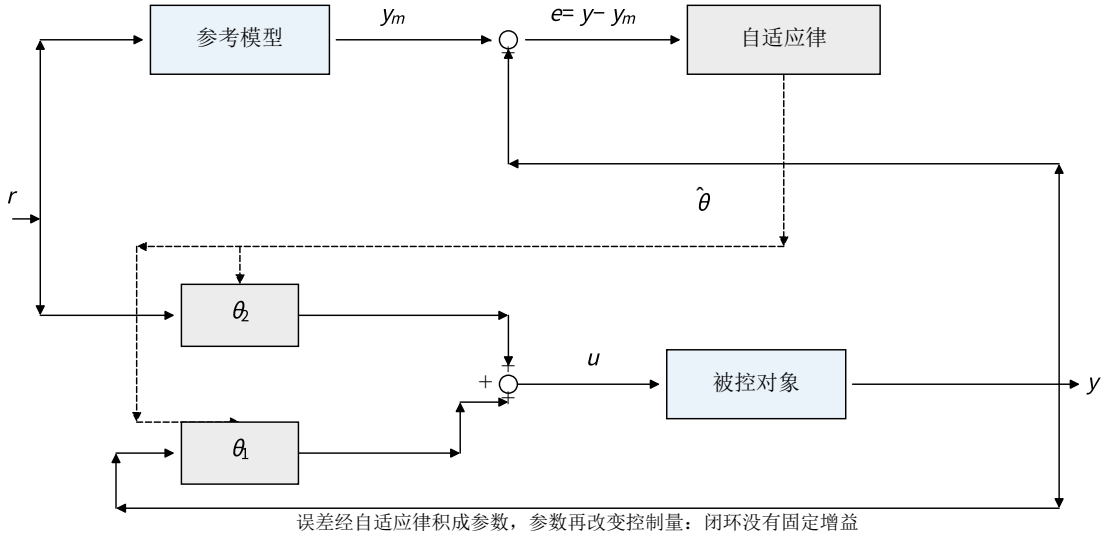


图 61-1　模型参考自适应控制的信号流

表 61-1 列出本章的仿真设置。所有数字都来自 `code/ch061_sim.py` 的一次运行，图与正文同源。

项目	取值	说明
对象参数	$a = 1.0$、$b = 1.4$	标称值；参数突变实验把 b 改为 2.1
参考模型	$a_m = 2.0$、$b_m = 2.0$	直流增益 1，时间常数 0.5 s
理想参数	$\theta_1^* = -0.7143$、$\theta_2^* = +1.4286$	由式 (61-5) 算出
采样周期	$T_s = 0.01$ s	每拍 RK4 细分，自适应律用欧拉积分
参考输入	阶跃 1；方波幅值 1、周期 6 s；PRBS 序列	PRBS 用于给辨识提供激励
自适应增益	$\gamma = 0.5 \sim 800$	主扫描变量，§61.6 给出稳定上界
执行器限幅	缺省不设；饱和实验设 $\lvert u \rvert \le 2$	指令幅值取 5 以强制触发

本章的路线是这样铺的。§61.2 给出自适应律的两条推导路径（Lyapunov 与 MIT），并说明为什么前者能保证收敛、后者不能。§61.3 把它落成五步设计流程。§61.4 在连续域里把“参数已知”与“参数从零起步”两条曲线放在一起，让读者看清自适应到底补上了什么。§61.5 转到离散域，给出可以直接写进代码的递推式，并引出间接自适应（STR）这条与 MRAC 并列的技术路线。§61.6 用三组实验把“γ 能取多大”这个问题钉死：未建模动态给出硬上界，量测噪声给出软上界。§61.7 逐个解释 γ、σ、死区、遗忘因子这些旋钮各自在改什么。§61.8 给出可以照抄的调参顺序。§61.9 讲五副补丁的取舍与叠加顺序。§61.10 讲工程落地时最容易踩的三个坑：参数漂移、估计可信度、安全边界。§61.11 划出适用边界并指向 PID 与 MPC。

61.2 数学原理

61.2.1 前提：结构已知，参数未知

自适应控制能做的事情有一条明确的分界线，必须先说清楚。它处理的是**参数化不确定性**：对象的结构（阶次、控制量进入的位置、非线性的形状）是清楚的，只有若干个系数不知道。它处理不了**非参数化不确定性**：阶次错

了、有没建模的谐振、有纯延迟、有死区——这些东西无法写进式 (61-4) 那种“两个参数”的形式里，只能靠 §61.6 的鲁棒化手段去压制，压不住就失稳。

本章全程假设式 (61-1) 的结构成立：一阶、控制量 u 以 b 的系数线性进入、输出 y 可直接量测。未知的只有 a 与 b 两个实数。这个假设看起来很苛刻，但它是所有自适应结论的地基——一旦结构错了，下面推出的“误差会收敛到零”就不再成立，只剩“误差有界”这一条较弱的结论。

61.2.2 误差方程：把不确定性挤到一处

设计的核心动作是把跟踪误差的导数整理成“一个稳定的线性系统 + 一个由参数误差驱动的项”。这一步做成了，剩下的就是选一个能让那个驱动项自己消失的律。

取控制律

$$u = \theta_1 y + \theta_2 r \tag{61-8}$$

记参数误差 $\tilde{\theta}_i = \theta_i - \theta_i^{*}$。代入对象 (61-1)，利用 $a - b\theta_1^{*} = a_m$ 与 $b\theta_2^{*} = b_m$ 两个恒等式（它们正是式 (61-4) 的改写），得到

$$\dot{y} = -\big(a - b\theta_1^{*}\big)y + b\tilde{\theta}_1 y + b\theta_2 r + d = -a_m y + b\tilde{\theta}_1 y + b\theta_2 r + d \tag{61-9}$$

再减去参考模型 (61-2)：

$$\dot{e} = -a_m e + b\tilde{\theta}_1 y + b\tilde{\theta}_2 r + d = -a_m e + b\,\tilde{\theta}^{\mathsf{T}}\phi + d \tag{61-10}$$

其中回归向量

$$\phi = \begin{bmatrix} y \\ r \end{bmatrix}, \qquad \tilde{\theta} = \begin{bmatrix} \tilde{\theta}_1 \\ \tilde{\theta}_2 \end{bmatrix} \tag{61-11}$$

这一步是全部推导的枢纽。式 (61-10) 说的是：**参数误差不是通过它自己，而是通过“参数误差 × 回归向量”这个乘积影响跟踪误差的**。y 大、r 大时，同样的参数偏差造成的误差也大；$y = r = 0$ 时，参数错得再离谱也不产生误差——这也解释了为什么“没有激励就没有辨识”。

还要注意 d 的位置。常值扰动 d 在式 (61-10) 里以一个常数偏置出现，它和 $\tilde{\theta}_2 r$ 的结构完全一样（都是不依赖 y 的常值项），所以自适应律会把它一并吸收进 θ_2。这是 MRAC 相对“无积分”的固定增益控制的一个隐性优势：**它自带一层对常值扰动的积分作用，只不过这个积分作用藏在参数里，而不是显式写着一个 $1/s$**。

61.2.3 Lyapunov 设计：让导数恰好是负定的

现在要选 $\dot{\theta}$。思路是构造一个同时含 e 和 $\tilde{\theta}$ 的能量函数，然后挑 $\dot{\theta}$ 让它的导数刚好把交叉项消掉、只剩一个负定项。

取

$$V = \frac{1}{2}e^2 + \frac{b}{2\gamma}\big\|\tilde{\theta}\big\|^2, \qquad \gamma > 0 \tag{61-12}$$

求导并代入 (61-10)（先令 $d = 0$）：

$$\dot{V} = -a_m e^2 + b\,e\,\tilde{\theta}^{\mathsf{T}}\phi + \frac{b}{\gamma}\tilde{\theta}^{\mathsf{T}}\dot{\theta} \tag{61-13}$$

后两项都含 $\tilde{\theta}^{\mathsf{T}}$，只要令

$$\dot{\theta} = -\gamma\, e\, \phi \tag{61-14}$$

它们就精确抵消，剩下

$$\dot{V} = -a_m e^2 \le 0 \tag{61-15}$$

到这一步只证明了 V 不增，还没证明 $e \to 0$——V 停在某个正值上、e 停在某个非零常数上，也满足 $\dot{V}=0$。补上这一步要用 Barbalat 引理：V 单调有界故收敛，$\int_0^\infty a_m e^2\,\mathrm{d}t = V(0) - V(\infty) < \infty$ 说明 $e \in L_2$；再由闭环有界性得 $\dot{e}$ 有界，于是 e 一致连续；L_2 且一致连续的函数必趋于零，故

$$\lim_{t\to\infty} e(t) = 0 \tag{61-16}$$

这就是 Lyapunov 律的全部价值：它给出的是一个无限时间上的收敛承诺，而不是“误差变小了”这种模糊说法。 代价是它对 $\tilde{\theta}$ 什么都没说——式 (61-15) 只约束 e，不约束 $\tilde{\theta}$ 是否收敛到 θ^*。这与 §61.1.3 的观察完全一致：$\hat{\theta}$ 的两个分量可以停在任意满足 $\tilde{\theta}^{\mathsf{T}}\phi = 0$ 的位置上。r 恒不为零时，$\tilde{\theta}^{\mathsf{T}}\phi = 0$ 只要求一个线性组合为零，解是一条直线而不是一个点。

式 (61-14) 写成标量形式更直观：

$$\dot{\theta}_1 = -\gamma\, e\, y, \qquad \dot{\theta}_2 = -\gamma\, e\, r \tag{61-17}$$

两个式子的形状完全一样：**误差乘上对应的回归量，再乘一个增益，然后积分**。这不是巧合，而是一类结构的通例——凡是“参数线性进入”的对象，Lyapunov 律都是这个形状，差别只在回归向量取什么。§61.5 的离散版本、以及更一般的 n 阶情形，都是照这个模子刻出来的。

61.2.4 MIT 律：梯度下降的诱惑

在 Lyapunov 方法之前，工程界先用的是另一条路，叫 MIT 律（得名于 MIT 的 Instrumentation Laboratory）。它的出发点非常朴素：把跟踪误差的平方当作代价，沿负梯度方向调参数。

$$J(\theta) = \frac{1}{2}e^2, \qquad \dot{\theta} = -\gamma \frac{\partial J}{\partial \theta} = -\gamma e \frac{\partial e}{\partial \theta} \tag{61-18}$$

问题卡在 $\partial e/\partial\theta$ 上。由式 (61-10) 可知 $\partial e/\partial\theta$ 不是一个常数，而是一个传递函数的输出：

$$\frac{\partial e}{\partial \theta} = \frac{b}{s + a_m}\phi \tag{61-19}$$

它依赖未知的 b，还得对 ϕ 做滤波。工程上的近似是：把 b 吸收进 γ，用一个已知的一阶滤波器代替 $1/(s+a_m)$，得到

$$\dot{e}_f = -a_m e_f + b_m e, \qquad \dot{\theta} = -\gamma\, e_f\, \phi \tag{61-20}$$

e_f 叫**灵敏度滤波信号**。式 (61-20) 就是本章代码里 `mode="mit"` 那几行。

这个近似看上去无害，实际上把收敛性彻底弄丢了。式 (61-19) 里的 b 被换成了 b_m、滤波器被换成了另一个截止频率，两者的比值随工况变化；e_f 与真实的 $\partial e/\partial\theta$ 之间差一个未知的、随参数漂移的因子。梯度方向一旦不准，积分就可能把参数推向错误的方向——而积分器推错了是不会自己回来的。

更麻烦的是，MIT 律的平衡点条件与 Lyapunov 律不同。Lyapunov 律要求 $\tilde{\theta}^{\mathsf{T}}\phi = 0$，配合 $e\to 0$ 是一套自洽的结构；MIT 律要求 $e_f\phi = 0$，e_f 里含有滤波器的动态，闭环在参数空间里的轨线不再是“朝着 θ^* 单调靠近”，而

可能绕着走、甚至走反。

61.2.5 两条路径的实测差别

图 61-2 把两条律放在同一张图上。为了公平，Lyapunov 律取 $\gamma = 2$，MIT 律取一个能让它勉强不发散的增益 $\gamma = 0.2$——再大就振荡到没法画。

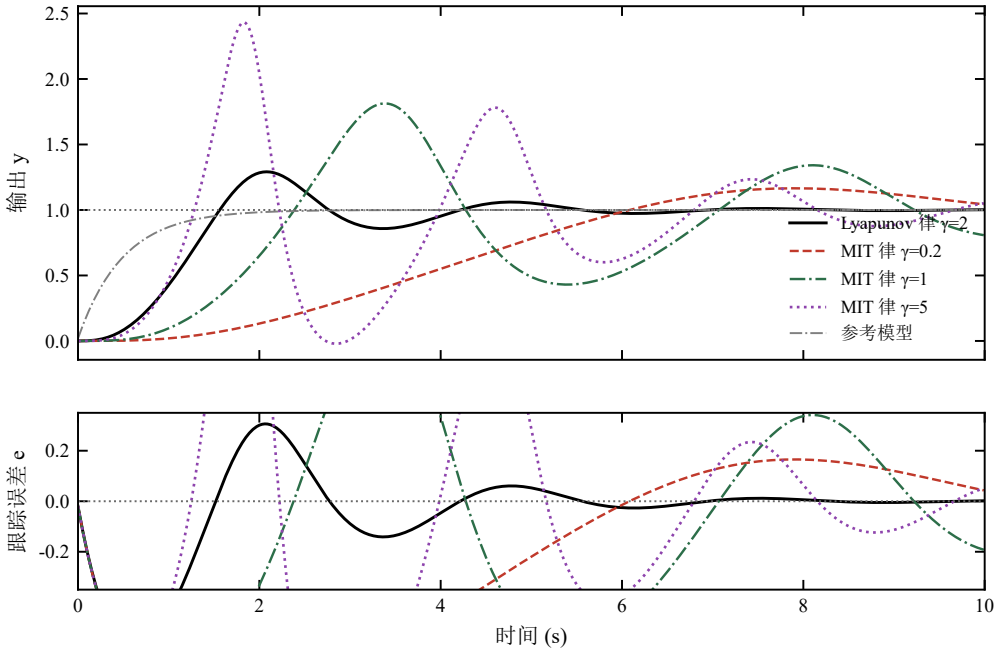


图 61-2　MIT 律与 Lyapunov 律的跟踪对比

Lyapunov 律在 $\gamma = 2$ 下，调节时间 6.51 s、超调 28.97%、IAE 1.0243。MIT 律在它最好的增益 $\gamma = 0.2$ 下，调节时间 9.63 s、超调 10.96%、IAE 3.6756——慢了 48%，累计误差是 3.6 倍。

真正说明问题的是表 61-2 里那一列末端静差。它随 γ 变化，而且**符号会翻**：

γ（MIT）	超调	末端静差	备注
0.2	10.96%	-0.0504	10 s 内尚未收敛，误差仍在缓慢变化
0.5	38.57%	-0.0816	振荡加剧
1.0	121.12%	$+0.1799$	静差反号，控制量明显饱和
2.0	83.06%	—	超调反而小于 $\gamma = 1$，非单调
5.0	134.25%	—	$\lvert u\rvert_{\max} = 3.73$，已接近执行器极限

表 61-2 里有两件事值得停一下。第一，静差从 -0.0504 变到 $+0.1799$，中间穿过零点——如果它只是“收敛得慢”，符号不该翻。第二，超调从 $\gamma = 1$ 的 121.12% 降到 $\gamma = 2$ 的 83.06%，再升到 $\gamma = 5$ 的 134.25%——**增益加大并没有单调地变好或变坏，而是在震荡**。这两条合起来说明：MIT 律在参数空间里走的是一条绕着目标转的螺旋，转得快不快、转几圈才停，取决于 γ 与当前对象参数的组合，而后者是未知的。

所以本章后续所有实验一律使用 Lyapunov 律。这不是偏好问题：**MIT 律没有一个可以事先算出来的稳定域**，而 Lyapunov 律的稳定域在 §61.6 可以被未建模动态与噪声两条线夹出来。

61.2.6 自适应律的统一形状

把式 (61-14) 与式 (61-20) 摆在一起看，可以得到一张“配方”：

$$\boxed{\dot{\theta} = \gamma \cdot (\text{误差信号}) \cdot (\text{回归向量}) + \text{修正项}} \tag{61-21}$$

三个成分各有分工。**误差信号**决定“往哪边调”：Lyapunov 律用 e，MIT 律用 e_f，后者不准。**回归向量**决定“哪个参数该为这个误差负责”：y 大就把账记到 θ_1 头上，r 大就记到 θ_2 头上。**增益** γ 决定“调多快”，它同时决定了收敛速度与噪声敏感度，是全部调参工作的主战场。**修正项**是 §61.9 要加的鲁棒化手段：σ 修正、死区、投影、饱和冻结，全都是在式 (61-21) 后面跟一个“只在特定条件下才起作用的小尾巴”。

这条配方还暴露了一个结构性事实：**自适应律是一个纯积分器**。积分器没有自恢复能力，进入它的量一旦有偏（噪声、未建模动态、饱和期间的错误误差），偏差就会积累成漂移。§61.9 与 §61.10 的全部工程手段，本质上都是在给这个积分器装“泄放阀”。

61.2.7 参数收敛到底缺什么

式 (61-16) 只说了 $e \to 0$，没说 $\tilde{\theta} \to 0$。这个缺口不是证明技巧的问题，是信息量的问题。要让 $\tilde{\theta}$ 收敛，需要回归向量满足**持续激励**（persistently exciting，PE）条件：存在 $\alpha > 0$ 与 $T > 0$，使得对任意 t

$$\int_{t}^{t+T} \phi(\tau)\phi(\tau)^{\mathsf{T}}\,\mathrm{d}\tau \;\geq\; \alpha I$$

这个矩阵不等式的意思是：ϕ 在**每一个方向**上都要有足够的能量。对本章的两参数系统，它要求 $\phi = (y, r)$ 不能长期贴着一条直线。

阶跃指令恰好违反这个条件。稳态时 $y \to r$，$\phi \to r(1,1)^{\mathsf{T}}$，方向固定，积分矩阵的秩退化为 1，只能确定 $\tilde{\theta}$ 在 $(1,1)$ 方向上的投影——也就是 $\theta_1 + \theta_2$ 这一个组合。这与 §61.1.3 的式 (61-6) 完全一致：**PE 条件失效的那一维，正好是那个“看不见的方向”**。

反过来，只要指令在变，ϕ 的方向就在转，PE 就可能成立。本章 $\gamma = 100$ 时两个分量都能贴到真值（表 61-5），靠的就是暂态过程中 y 从 0 升到 1、ϕ 从 $(0,1)$ 转到 $(1,1)$ 的这段“扫弧”。但这是暂态，不是持续激励——一旦 y 稳定，弧就停了，之后学到的东西就定格在那里。**所以 γ 越大、暂态越短，反而越“抓得住”那段弧**，这也是高增益参数更准的另一个解释。

工程上想真正辨识参数，有两条路：一是让指令本身足够丰富（这也是注塑机、伺服调试时常用多段阶跃的原因）；二是人为叠加一个不影响主任务的小幅探测信号（dither）。第二条路的代价可以直接估出来：探测信号幅值是 δ，跟踪误差的 rms 至少上升 δ 量级。**要辨识就得付精度，这是绕不过去的交换。**

61.2.8 V 为什么这样选

式 (61-12) 的 V 里，第二项的系数是 $\frac{b}{2\gamma}$，不是更自然的 $\frac{1}{2\gamma}$。这个 b 不是装饰，它是**为了和交叉项配平**。

回看式 (61-13)：交叉项是 $b\,e\,\tilde{\theta}^{\mathsf{T}}\phi$，而 $\dot{\theta}$ 那项贡献的是 $\frac{b}{\gamma}\tilde{\theta}^{\mathsf{T}}\dot{\theta}$。要让后者的系数恰好抵消前者，$\tilde{\theta}$ 前面的因子必须是 $\frac{b}{\gamma}$。如果写成 $\frac{1}{2\gamma}$，抵消不掉，只剩

$$\dot{V} = -a_m e^2 + (b-1)\,e\,\tilde{\theta}^{\mathsf{T}}\phi$$

这个式子既可能为正也可能为负，V 不再单调，证明断在这里。

这条细节说明两件事。第一，**Lyapunov 证明是“配”出来的，不是“猜”出来的**：先看交叉项长什么样，再反推权重。第二，b **未知这件事被巧妙地藏进了** V **里**——V 本身不需要计算，只要存在就行，所以不知道 b 也不妨碍证明成立。这正是 Lyapunov 方法相对“直接算特征值”的高明之处。

同样的道理解释了为什么 §61.9 加了 σ 或死区之后，结论从“收敛”退化为“有界”。加 σ 后

$$\dot{V} = -a_m e^2 - \frac{\sigma b}{\gamma}\tilde{\theta}^{\mathsf{T}}(\theta - \theta_0)$$

第二项在 θ 远离 θ_0 时为负、靠近时为正，V 不再单调下降。能证明的只剩“V 最终进入某个球内”，这个球的半径与 σ 成正比——也就是 §61.7.4 里那个稳态误差。**理论上失去的每一点，都会在实测数据里变成一个具体的数字。**

61.2.9 唯一不能在线学的量：控制方向

前面所有推导都默认了一件事：b 的符号已知——对象对一个正的控制量给出正的响应。这个假设从来没有被写进公式，因为它一旦不成立，公式里的每一个符号都跟着反。

看误差方程 (61-10)：b 是以正系数的身份出现在 $e\,\tilde{\theta}^{\mathsf{T}}\phi$ 里的。自适应律取 $\dot{\theta} = -\gamma e\phi$，靠的正是“$-b\gamma$ 这部分为负”来保证 $\dot{V} \le 0$。如果实际 $b < 0$ 而设计时按 $b > 0$ 取符号，这一项就变成正的，$\dot{V} > 0$，θ 朝错误方向单调加速，误差指数增长。实测的表现是：几十毫秒内 θ 冲向硬限幅，y 反向飞走，控制量顶在限幅上不动。

为什么控制方向不能交给自适应律自己去学？ 因为符号是一个离散的二值量，不是连续参数。梯度法只能在连续空间里爬，遇到“符号”这种跳变，梯度信息本身就没有定义——b 穿过零点的瞬间，式 (61-30) 的分母为零，控制量无穷大。文献里有用 Nussbaum 增益处理这一点的做法，但那属于研究性内容，工程上不该依赖。

工程上的做法只有三条，都很朴素：

1. **开环试探**。上电后给一个小的正 u（例如额定值的 5%），看 y 是升还是降，持续几百毫秒，然后把符号记下来。这是绝大多数伺服驱动器出厂自整定的第一步。
2. **查接线与机械结构**。阀门的正作用/反作用、电机的正负接线、液压缸的有杆腔与无杆腔，这些在图纸上就写清楚了，比在线试探可靠。
3. **用 $\hat{\theta}_2$ 做在线诊断**。正常运行中如果 $\hat{\theta}_2$ 长时间为负、且 $|e|$ 不降，基本可以判定控制方向假设反了。这条判据不解决问题，但能报警。

还要注意一种更麻烦的情况：**符号会变**。四象限运行的电机、双向液压回路、可逆泵，在不同工况下 b 的符号确实会翻转。这类对象不适合用单一 θ 的自适应律，应当按工况分段，每一段用一套独立的参数——这实际上是把自适应退化成了增益调度（§61.11.6 会把这几种方法的边界划清楚）。

61.2.10 Barbalat 引理补上的那一步

式 (61-15) 给出 $\dot{V} \le -a_m e^2 \le 0$，很多人到这里就写“因此 $e \to 0$”。这一步是错的，中间还差一个引理。

$\dot{V} \le 0$ 能推出的只是三件事：V 单调不增因而有下界、$\int_0^\infty e^2 \mathrm{d}t < \infty$（即 $e \in L_2$）、e 有界。**一个平方可积的函数完全可以不趋于零**——例如一系列高度不变、宽度递减的窄脉冲，积分收敛，但函数值无穷多次回到 1。

从 $e \in L_2$ 走到 $e \to 0$，需要 Barbalat 引理：若 e 一致连续且 $\int e^2 < \infty$，则 $e \to 0$。而“一致连续”在这个问题里由“$\dot{e}$ 有界”保证。于是全部重担落在

$$\dot{e} = -a_m e + b\,\tilde{\theta}^{\mathsf{T}}\phi$$

的右端是否有界上。e 与 $\tilde{\theta}$ 已经有界，剩下的关键是 $\phi = (y, r)$——**y 有界，这一条不在 Lyapunov 证明的射程之内**。

这就是自适应控制证明里那个经典的“循环”：要证 y 有界，需要闭环稳定；要证闭环稳定，需要参数有界；要证参数有界，需要 y 有界。标准教材的处理是假定参考模型与对象都“足够温和”，把这一环补上；而工程上的处理更直接——**加硬限幅**。

有了 $|\theta_i| \le 20$，θ 有界；有了执行器限幅，u 有界；于是 $\dot{y} = -ay + bu + d$ 的右端有界，y 在有限时间内不会跑到无穷。三个限幅把证明里的那个洞堵上了。**这也解释了为什么表 61-4 把“θ 有硬限幅”列为必查项：它不是保守，它是让定理成立的前提之一。**

顺便说清另一件事：Barbalat 给出的结论是“$e \to 0$”，**不含任何速率信息**。它没说多快收敛，也没说 θ 会怎样。这就是为什么本章的性能结论全部来自实测表格，而不是来自定理——定理只保证“最终会到”，工程关心的是“多久到、路上抖不抖”。

61.2.11 为什么教材偏爱 MIT 律，而工程上几乎不用

读完 §61.2.4 到 §61.2.5，一个自然的疑问是：既然 MIT 律这么不稳，为什么它在教材里出现得比 Lyapunov 律还早、篇幅还多？

原因有一半是历史，一半是教学。MIT 律在 1950 年代末就随飞行器自适应自动驾驶仪的研制出现了，比 Lyapunov 方法进入控制理论（1960 年代中后期）早了近十年。教材按时间线写，读者自然先遇到它。教学上它也确实“好讲”：只需要“误差平方沿负梯度下降”这一个概念，不需要构造 Lyapunov 函数，推导三行就完。相比之下，Lyapunov 律的推导是逆向的——先要有一个 V，再从 $\dot{V}$ 反推出 $\dot{\theta}$，初学者第一次看会觉得“这个 V 是从天上掉下来的”（§61.2.8 已经解释了它是怎么“配”出来的）。

但“好讲”和“好用”是两件事。MIT 律在工程上被弃用，理由可以归结为三条，而这三条恰好都是**无法通过调参补救**的结构性缺陷：

1. **没有可以事先算出来的稳定域。** Lyapunov 律的稳定边界可以被未建模动态与噪声夹出来（§61.6.3、§61.6.4），调试时有明确的搜索区间；MIT 律的边界依赖当前的对象参数，而对象参数正是未知量，所以只能在现场一次次试到振荡为止。
2. **增益的作用不单调。** 表 61-2 里超调从 $\gamma = 1$ 的 121.12% 降到 $\gamma = 2$ 的 83.06%、再升到 $\gamma = 5$ 的 134.25%，这条曲线没有“加大增益就更快”的规律可循。调参的人最怕的就是这种旋钮——转一下不知道会往哪边走，只能靠来回试，而现场通常没有反复试的机会。
3. **静差会翻号。** 一个真正“收敛得慢”的控制器，误差应当始终停在同侧、缓慢趋零。MIT 律的末端静差从 $\gamma = 0.2$ 的 -0.0504 变到 $\gamma = 1$ 的 $+0.1799$，中间穿过零点，说明闭环在参数空间里绕过了目标而不是逼近目标（§61.2.5 的螺旋解释）。**符号翻转是一个比幅值更大的警报**，因为幅值可以靠调增益压下去，符号翻转不能。

还有一条更隐蔽的差别藏在校准里。MIT 律的等效增益含因子 b_m/b——设计时用 b_m 代替了真实的 b，一旦负载变化使 b 从 1.4 掉到 0.8，同一套 γ 的等效学习速率就变了近一倍，昨天调好的参数今天不再适用。Lyapunov 律没有这个问题：γ 直接就是学习速率，b 只出现在 V 的权重里，而 V 不需要被计算（§61.2.8）。

现场有一个很便宜的判别办法：把增益小步加倍，看静差怎么变。静差单调缩小，多半是 Lyapunov 型或带明确稳定域的律；静差先缩后翻号、超调时大时小，那这套控制器内部用的就是梯度型自适应。这个测试只需要两次阶跃，代价很低，但能省掉后面几天的瞎调。

因此本章把它放在“原理”这一节里讲，用途是当反例：**MIT 律是“把局部线性化的直觉直接搬到非线性闭环上”的典型失败**，而 Lyapunov 设计之所以值得多花那几行推导，正是因为它换来了一个可以被工程约束夹住的稳定域。

61.3 设计步骤

从式 (61-1) 到一套能上机的自适应控制器，中间要做五个决定。这五步的顺序不能乱：前一步的输出是后一步的输入，而且每一步都留下了一个“事后很难改”的约束。

61.3.1 第一步：定参考模型

参考模型不是在描述对象，而是在**描述你希望闭环长什么样**。它决定了三件事：稳态增益、响应快慢、以及“控制器需要多努力”。

本章取 $a_m = b_m = 2.0$，直流增益 1、时间常数 0.5 s。这个选择不是随手写的：

- **增益取 1**，是为了让“跟踪误差”直接等于“输出误差”，指标读起来直观。
- **时间常数取 0.5 s**，比对象的 1 s 快一倍。这个比例很关键：参考模型太慢，控制器几乎不用出力，自适应也就没什么可调的；参考模型太快，$\theta_2^* = b_m/b$ 会很大，控制量在起始阶段必然饱和，自适应在饱和期间学到的是错的东西（§61.9 会量化这件事）。
- **a_m 决定误差收敛速度**。由式 (61-10)，即使 $\tilde{\theta}$ 一直不变，误差也按 $e^{-a_m t}$ 衰减。a_m 越大，误差消失得越快，自适应律“看到”的误差也越小、越难起作用。

一条经验：**参考模型的时间常数取对象标称时间常数的 1/2 到 1/3**；再快就要先确认执行器撑得住。

61.3.2 第二步：写控制律的参数化形式

这一步决定了 θ 有几个、回归向量是什么。做法是把“理想控制器”写成未知参数的线性组合，然后把这个组合原样当控制律。

一阶对象配一阶参考模型，得到式 (61-8) 的两参数形式。n 阶对象配同阶参考模型，标准结果是 $2n$ 个参数：

$$u = \theta_1^{\mathsf{T}}\omega_1 + \theta_2^{\mathsf{T}}\omega_2 + \theta_3 r \tag{61-22}$$

其中 ω_1, ω_2 是由 u 与 y 经 $(n-1)$ 阶滤波器产生的状态。参数个数随阶次线性增长，这是 MRAC 的一个隐性成本：二阶对象要 5 个参数、三阶要 7 个，每个都要在线积分，每个都可能漂移。

参数化形式定下来之后，有两件事必须核一遍：

1. **控制方向（b 的符号）已知**。$\theta_2^* = b_m/b$，$b < 0$ 时 θ_2^* 为负，自适应律必须知道该往哪边推。方向未知要另加一套“方向辨识”逻辑，不在本章范围内。
2. **理想参数不落在奇异点上**。$b \neq 0$ 是底线；工程上还要求 $|b|$ 不要太小，否则 θ^* 量级过大，控制量在正常工况下就顶到限幅。

61.3.3 第三步：来自适应律与初值

用式 (61-14) 的 Lyapunov 律，剩下两个选择：初值 θ_0 和增益 γ。

初值有一个很实际的取舍。如果对象有标称参数，就把 θ_0 设成标称值算出的 θ^*；如果一点都不知道，就设零。两者的差别不小：本章实验 1 里，θ_0 取真值时调节时间是 1.93 s、超调 0.00%、IAE 0.0014——这实际上就是“参数已知”的理想性能；而 $\theta_0 = 0$、$\gamma = 20$ 时（实验 2），调节时间 3.39 s、超调 7.34%、IAE 0.3027。**初值给对，等于省掉了整个学习过程。**

增益 γ 不能在这一步定死，它要等 §61.6 的两条边界算出来之后再选。这里先给一个可用的起点：取 γ 使得自适应回路的时间常数比内环慢 5 到 10 倍，即

$$\gamma \sim \frac{a_m}{5\,\|\phi\|^2_{\text{典型}}} \tag{61-23}$$

本章 $\|\phi\|$ 的典型值在 1.5 量级，$a_m = 2$，算出 γ 在 0.2 附近——这个值偏保守，实际可以往上走一到两个数量级，但要靠 §61.6 的实验确认。

61.3.4 第四步：加鲁棒化措施

到这一步控制器在纸面上是能收敛的，但一上机就会出问题。表 61-3 列出必须提前处理的三件事。

问题	现象	处理	本章位置
常值扰动 / 未建模偏置	θ 缓慢单向漂移	σ 修正，加 $-\sigma(\theta - \theta_0)$	§61.9.1
量测噪声	θ 抖、控制量抖	死区，$\lvert e\rvert \le \Delta$ 时冻结	§61.9.2
执行器饱和	饱和期间误差不降，θ 越推越大	饱和冻结或投影	§61.9.3

这三条都不是“可选优化”，而是**不加就会失效**的必需品。本章实验 9 给了一个极端例子：指令幅值 5、限幅 2 时，自由自适应的 $\|\theta\|$ 被推到 28.28（两个分量都顶到 20 的硬限），而饱和冻结只到 0.46——两者的 IAE 完全相同，都是 22.402。**冻结没有损失任何跟踪性能，只是消掉了漂移。** 这笔交易在任何情况下都该做。

61.3.5 第五步：三层验证

设计完必须按三层跑一遍，每层失败对应不同的修改方向：

1. **标称层**：参数准确、$\theta_0 = \theta^*$、无噪声无饱和。这一层考的是“实现有没有 bug”。本章实验 1 的结果是 $t_s = 1.93$ s、超调 0、IAE 0.0014；如果这一层就不对，问题一定在代码或符号上，不要去调 γ。
2. **学习层**：$\theta_0 = 0$、参数准确。这一层考的是“自适应律能不能学回来”。看的是 θ 的轨迹能不能在合理时间内贴近 θ^*，以及 $\theta_1 + \theta_2$ 是否收敛到 $1/b$（式 61-6）。
3. **鲁棒层**：参数突变、未建模动态、量测噪声、执行器饱和，逐项加。这一层考的是“边界在哪”。§61.6 与 §61.9 的全部实验都属于这一层。

三层的顺序不能颠倒。跳过第一层直接调参，最常见的后果是把符号错误当成“增益没调好”，然后一路加大 γ 直到发散。

61.3.6 上机前的一张自查表

把上面五步压成表 61-4，可以逐项打勾：

检查项	判据	不通过的后果
参考模型阶次与对象一致	同阶或高一阶	误差方程 (61-10) 不成立，收敛性无保证
控制方向已知	b 的符号确定	自适应往反方向推，必然发散
θ_0 有依据	标称值或 0，不是随手填	起始阶段大误差，可能触发饱和
γ 有上界依据	做过未建模动态扫描	现场一有谐振就失稳
σ 已设	有常值扰动时必须	参数缓慢漂到限幅
死区已设	噪声水平可测时必须	控制量持续抖动，磨损执行器
饱和处理已做	限幅值可知时必须	饱和期间参数卷绕
θ 有硬限幅	建议 $\lvert\theta_i\rvert \le 20$	数值溢出，控制器彻底失效

最后一行不是形式主义。本章代码里专门设了 `THETA_MAX=20.0` 的硬限幅——在实验 9 的自由自适应分支里，正是这个限幅把 θ 拦在了 20 而不是无穷大。没有它，那次仿真会直接跑出 `nan`。

61.3.7 把五步走成一张设计卡

抽象讲完，用本章的数字实际走一遍，看看五步各输出什么。

已知：对象一阶，标称 $a = 1.0$、$b = 1.4$，执行器限幅 3，量测噪声标准差 0.01，采样 0.01 s。

第一步：参考模型取 $a_m = b_m = 2.0$（比对象快一倍，时间常数 0.5 s）。

第二步：控制律 $u = \theta_1 y + \theta_2 r$，两个参数。控制方向：$b > 0$ 已知（加大 u 使 y 上升）。

第三步：初值取标称值算出的 $\theta_0 = (-0.7143, +1.4286)$；增益先按式 (61-23) 取 0.2，再按“往大走”的原则试到 5 与 20 各跑一遍。

第四步：限幅 3（是标称峰值 1.43 的 2.1 倍，满足“两倍余量”）；死区 $\Delta = 0.02$（噪声的 2 倍，小于允许静差 2%）；$\sigma = 0$（没有慢漂移迹象）；$\lvert\theta_i\rvert \le 20$；饱和冻结打开。

第五步：三层验证，结果按 §61.6 的格式逐层记录。

把这些填进一张卡，就得到了可以直接交给软件工程师的东西：

条目	取值	依据
控制律	$u = \theta_1 y + \theta_2 r$	一阶对象 + 一阶参考模型
参考模型	$a_m = b_m = 2.0$	对象带宽的 2 倍
自适应律	$\dot{\theta} = -\gamma e \phi$	Lyapunov（不用 MIT）
θ_0	$(-0.7143, +1.4286)$	标称参数代入式 (61-5)
γ	5 起，扫到 50	式 (61-23) 给保守起点，实测定上界
死区 Δ	0.02	噪声 0.01 的 2 倍
σ	0	无慢漂移
限幅	3	标称峰值的 2.1 倍
冻结	开	有执行器限幅
硬限	$\lvert\theta_i\rvert \le 20$	数值兜底

这张卡的价值在于：**它把“设计”这件事固化成了十个可以逐项核对的格子**，而不是散落在推导里的十个决定。换一个对象，改动的是格子里的数值，不是推导过程。

61.3.8 从指标反推参考模型

设计卡的第一行是参考模型，而参考模型往往凭直觉取“比对象快一倍”。这一步其实可以算，而且应该算——因为 a_m 与 b_m 的上限由执行器余量决定，不由愿望决定。

先看速度。一阶参考模型的阶跃响应是 $y_m = r\left(1 - e^{-a_m t}\right)$，进入 2% 带的时间约

$$t_s \approx \frac{4}{a_m}$$

$a_m = 2.0$ 给出 2.0 s，与实验 1 实测的 1.93 s 吻合（实测略快，因为 r 到 y_m 的传递还带一个零点效应）。如果指标要求 $t_s \le 3$ s，那么 $a_m \ge 1.33$ 就够；要求 $t_s \le 1$ s，则 $a_m \ge 4$。

再看控制量。把 θ^* 代回控制律，$t = 0$ 时 $y = 0$、$r = 1$，得

$$u(0) = \theta_2^* = \frac{b_m}{b}$$

控制量的起始峰值只由 b_m/b 决定，与 a_m 无关。 但如果按“直流增益为 1”的习惯取 $b_m = a_m$（本章就是 $a_m = b_m = 2.0$），两者就耦合起来了：a_m 每提高一倍，$u(0)$ 也提高一倍。

把这两条放在一起，就得到一个很实用的反推步骤：

1. 由 t_s 指标定 $a_m \ge 4/t_s$；
2. 取 $b_m = a_m$（保证直流增益 1）；
3. 检查 $u(0) = a_m/b$ 是否落在执行器线性区的 50% 以内；
4. 超了就放松 t_s 指标，或者把 b_m 单独取小（牺牲直流增益，用前馈补）。

本章 $a_m = b_m = 2.0$、$b = 1.4$，$u(0) = 1.43$，执行器限幅 3——占比 48%，正好卡在第 3 条的红线上。这不是巧合，是设计时按这条规则定的。反过来，如果把 t_s 指标压到 1 s，$a_m = b_m = 4$，$u(0)$ 变成 2.86，占比 95%，任何一个参数偏差都会把控制量顶到限幅，接着就是 §61.9.3 的卷绕。

所以“参考模型取多快”这个问题的答案不是“越快越好”，而是“执行器给得起多快”。 这一条在 §61.7.6 还会从 a_m、b_m 各自的作用再讲一遍。

61.3.9 对象是二阶时，这五步怎么改

本章用一阶对象把原理讲清楚，但现场的对象十个里有八个是二阶以上。改成二阶不需要重新推导，只需要把三处地方按同一套逻辑扩写。

第一，控制律的参数个数从 2 变成 4。 二阶对象

$$\ddot{y} = -a_1\dot{y} - a_0 y + bu$$

的匹配控制器需要 $u = \theta_1 y + \theta_2 \dot{y} + \theta_3 r + \theta_4 \dot{r}$ 四个参数——**回归向量里出现状态和指令的导数**。这就是“参数化”的一般形式：对象每多一阶，回归向量就多一个状态分量、控制律就多一个参数；再按相对阶补上指令的导数，相对阶每多一阶就再多一个。

第二，导数怎么拿。 这是二阶以上真正的难点：$\dot{y}$ 通常测不到。三条路：直接差分（放大噪声，配合低通滤波）、状态观测器（需要模型，与自适应重复）、或者把控制律改写成“用滤波后的信号”的形式（把微分算子搬进已知的滤波器里）。工程上最常用的是第三条，代价是引入一个额外的时间常数，它又成了 §61.6.3 意义上的未建模动态。

第三，γ 的上界会明显收紧。 参数从 2 个变 4 个，$\|\theta^*\|$ 变大，步长约束 $\gamma T_s \|\phi\| e \ll \|\theta^*\|$ 反而放松；但同时回归向量里多了 $\dot{y}$ 与 $\dot{r}$，它们的幅值是原来的一阶导数量级，$\|\phi\|$ 变大，于是又收紧。两者大致抵消，实测的经验是**二阶对象的可用 γ 比一阶低 2 到 5 倍**，具体只能靠 §61.6 的扫描确定。

还有一个容易被忽略的连带影响：θ 从二维变四维，**“只有组合被误差看见”这件事更严重**。二维时误差只确定一个线性组合（$\theta_1 + \theta_2 = 1/b$），四维时误差确定的是一个二维子空间里的投影，剩下两个自由度完全不可见。这就是为什么二阶以上的 MRAC 几乎从不用于参数辨识，而只用于跟踪。

61.4 连续情形

61.4.1 连续律怎么在数字机上跑

式 (61-17) 是连续时间的微分方程，但所有实现都在采样系统上。本章的处理方式是：**对象用 RK4 按 $T_s = 0.01$ s 积分，自适应律用前向欧拉按同一节拍积分。**

$$\theta[k+1] = \theta[k] - \gamma\, T_s\, e[k]\, \phi[k] \tag{61-24}$$

这个组合是工程上的常态——对象动态快，用高精度积分；参数动态慢，欧拉就够了。误差主要来自欧拉法，量级是 $O(\gamma T_s^2)$：$T_s = 0.01$、$\gamma = 100$ 时约 10^{-2}，与量测噪声相比可以忽略。但如果把 T_s 放大到 0.1 s，$\gamma = 100$ 的欧拉积分就会明显偏离连续解，甚至失稳——这是 §61.5 要单独处理的问题。

仿真里还有一处细节值得说：**执行器建模成“理想直通”还是“一阶滞后”，会改变结论**。本节全部结果都假设理想执行器；§61.6 会把它换成一阶滞后 $\tau_a = 0.05$ s，那时 γ 的可用上界会塌下来一大截。

61.4.2 三条曲线

图 61-3 上有三条曲线：参考模型的输出 y_m、参数已知（$\theta_0 = \theta^*$）的控制器输出、以及参数从零起步（$\theta_0 = 0$、$\gamma = 10$）的自适应输出。

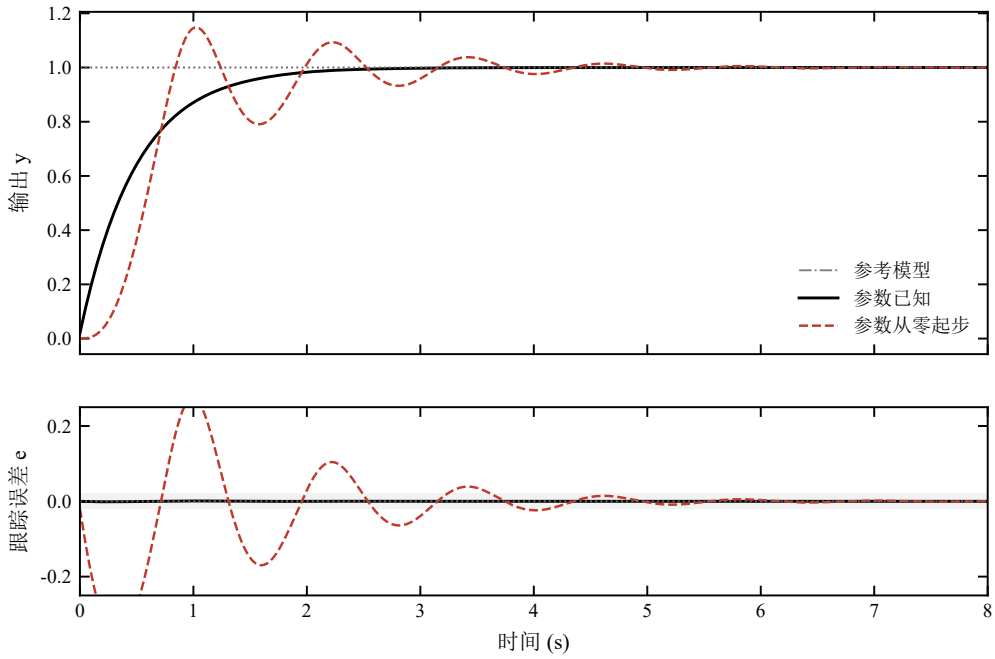


图 61-3　参考模型、参数已知控制器与参数从零起步的自适应跟踪

参考模型本身是一条 $1-e^{-2t}$ 的指数曲线，在 $t=1.96$ s 进入 $\pm 2\%$ 带。两段对比很干净：

- **参数已知**（实验 1）：$t_s=1.93$ s、超调 0.00%、IAE 0.0014、$|u|_{\max}=1.43$、控制量总变差 TV 仅 0.7。它和参考模型几乎重合——这本来就应该，因为 $\theta=\theta^*$ 时式 (61-10) 里的 $\tilde{\theta}$ 恒为零，误差方程退化成 $\dot{e}=-a_m e$。
- **参数从零起步**（实验 2，$\gamma=10$）：$t_s=4.14$ s、超调 14.82%、IAE 0.4435、$|u|_{\max}=1.97$、TV 7.3。调节时间慢了 2.1 倍，累计误差大了 300 倍。

这 300 倍不是“学得慢”这么简单。参数从零起步时，控制器在最开始的 0.5 s 里几乎不输出（$\theta\approx 0$，$u\approx 0$），对象基本处于自由衰减状态，y 停在 0 附近；这段“什么都不做”的时间把 IAE 一次性抬了起来。**自适应控制的误差，很大一部分花在“还没学会”的那一段上。** 这也解释了为什么 §61.3 强调“初值给对，等于省掉整个学习过程”。

61.4.3 参数轨迹：分量乱跑，组合不乱

图 61-4 是三条 γ 下的参数轨迹。这张图是本章最反直觉的一张。

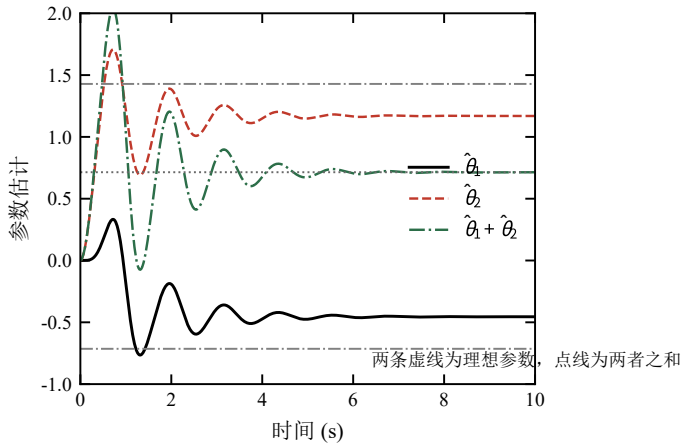


图 61-4　参数轨迹：两个分量与它们之和

表 61-5 是三组 γ 在 $t=10\,\mathrm{s}$ 的终值。

γ	$\hat{\theta}_1$	$\hat{\theta}_2$	两者之和	与理想和之差
2	-0.142	$+0.860$	$+0.718$	$+0.004$
20	-0.567	$+1.281$	$+0.715$	$+0.001$
100	-0.722	$+1.436$	$+0.714$	0.000
理想值	-0.7143	$+1.4286$	$+0.7143$	—

看三列的分工。**“和”这一列在三组里都锁在 0.714 到 0.718 之间**，误差不超过 0.6%，与式 (61-6) 的预测一字不差。而**两个分量列差别巨大**：$\gamma=2$ 时 $\hat{\theta}_2$ 只到 0.860，离真值差 40%。

为什么低增益会把分量停在半路？原因在回归向量的方向。起始瞬间 $y=0$，$\phi=(0,r)$ 只有一个方向；随着 y 升起来，ϕ 慢慢转到 $(1,1)$ 方向。**ϕ 在参数空间里扫过一段弧，而 θ 只能沿着当下的 ϕ 方向移动。** γ 小，θ 移动得慢，追不上这段弧的转动，最后停在弧上的某个位置；γ 大，θ 几乎与 ϕ 同步，落点就贴近 θ^{*}。表 61-5 里 γ 从 2 加到 100，$\hat{\theta}_1$ 从 -0.142 走到 -0.722、$\hat{\theta}_2$ 从 0.860 走到 1.436，就是这么一点点“追上弧线”的过程。

这张表还给了个实用结论：**想从 MRAC 里读出参数，就得把 γ 加到接近稳定边界**——而那时控制量的抖动已经不可接受（$\gamma=100$ 的 TV 是 19.9，是标称的 28 倍）。这就是 §61.10 反复强调“不要把自适应增益当参数估计”的量化依据：不是理论上不能，而是**要它准就得付出你付不起的抖动代价**。

61.4.4 控制量的代价

表 61-6 把三组控制量指标摆在一起：

工况	$\lvert u\rvert_{\max}$	TV	稳态 u
参数已知	1.43	0.7	0.714
$\gamma=20$，从零起步	2.16	9.8	0.714
$\gamma=100$，从零起步	2.48	19.9	0.714

最后一列完全相同，这不是巧合。稳态时 $y=r=1$、$\dot{y}=0$，由对象方程得

$$u_{ss} = \frac{a y_{ss}}{b} = \frac{1.0}{1.4} = 0.714 \tag{61-25}$$

这个值与 θ 怎么分解无关，只与 a/b 有关。换句话说，**只要跟踪误差为零，控制量的稳态值就被对象唯一确定了**；自适应改变的是到达它的路径，不是终点。

真正被改变的是路径上的抖动。参数已知时 TV 只有 0.7——控制量单调地爬到 0.714 就停了；$\gamma = 100$ 时 TV 是 19.9，意味着控制量在 0.714 附近来回摆的总幅度接近其稳态值的 28 倍。这个抖动不会体现在跟踪误差上（误差仍然收敛到零），但会体现在执行器的磨损、发热、噪声上。§61.6 会把它与噪声、未建模动态放在一起，给出 γ 的完整取值窗口。

61.4.5 相对阶：一个不能违反的前提

前面所有推导都默认了一件事——参考模型的相对阶与对象相同。这条假设不写在公式里，却决定了整个方法能不能用。

相对阶的定义是“分母阶数减分子阶数”。本章对象是 $\frac{b}{s+a}$，相对阶 1；参考模型 $\frac{b_m}{s+a_m}$，相对阶也是 1。两者相同，所以理想控制器 (61-4) 只用到 y 与 r 的**当前值**，不需要它们的导数。

如果参考模型取成相对阶 0 会怎样？比如直接要求 $y_m = r$。代入式 (61-3) 解出 $u = \big(a y - \dot{r}\big)/b$——出现了 $\dot{r}$。指令的导数在数字系统里要么不可得，要么被噪声放大到不可用，这个控制律在物理上实现不了。

一般结论是：**参考模型的相对阶不能小于对象的相对阶**。小于就要对指令求导，大于则闭环里会多出对象本身抵消不掉的动态。工程上最稳的做法是取相等。

这条约束在换对象时会立刻咬人。二阶对象（相对阶 2）配一个一阶参考模型（相对阶 1），理想控制器不存在，硬套的后果是参数始终无法收敛、误差停在某个非零值上。此时要么把参考模型改成二阶，要么先给对象加一个前置滤波器把相对阶降下来——两条路都要改结构，不是调 γ 能救的。

61.4.6 读图的四个位置

图 61-3 这样的曲线，训练有素的眼睛只看四个位置，每个位置对应一个参数：

1. **起始段（0 到 0.2 s）**。这里的高度完全由 θ_0 决定。$\theta_0 = 0$ 时曲线趴在地板上不动；$\theta_0 = \theta^*$ 时它立刻跟着参考模型走。起始段太低，说明初值没给对，与 γ 无关。
2. **上升段（0.2 到 2 s）**。斜率由 γ 决定。γ 小则爬得慢，γ 大则几乎贴着参考模型的斜率。这一段的形状是判断 γ 够不够的最直接依据。
3. **峰值附近**。超调来自“学得太晚”：参数还没到位，误差已经积累起来，等参数补上时冲过了头。所以超调大往往不是“增益太大”，而是“增益太小”（§61.6.2 表 61-9 的四条趋势之一）。
4. **稳态段（4 s 之后）**。这里曲线应当与参考模型完全重合。如果留了一个恒定的缝，先查死区与 σ；如果缝在缓慢变化，说明参数还在漂。

四个位置对应四个参数，互不干扰——这是“读图定位问题”的实用价值。初学者常犯的错是看到任何异常都去调 γ，而图上一半的异常根本不在 γ 的管辖范围内。

61.4.7 前七拍的解剖

把 $\theta_0 = 0$、$\gamma = 20$ 的开头七个采样点逐个打印出来，能看到一个很不直观的事实：**自适应在最该起作用的时候几乎不起作用**。

k	t (s)	y_m	y	$e=y-y_m$	$\hat{\theta}_2$	u
1	0.01	0.0396	0.000056	-0.0395	0.0040	0.0040
2	0.02	0.0588	0.000221	-0.0586	0.0119	0.0119
3	0.03	0.0776	0.000548	-0.0771	0.0236	0.0236
5	0.05	0.1142	0.001884	-0.1123	0.0580	0.0580
7	0.07	0.1492	0.004437	-0.1448	0.1063	0.1063

三件事值得注意。

第一，$k=0$ 那一拍什么都没发生。 控制器看到 $y=0$、$y_m=0$，误差为零，更新量为零，$u=0$。要等参考模型先动起来，误差才出现——**误差是参考模型“喂”给自适应律的**，没有参考模型就没有学习信号。

第二，$\hat{\theta}_1$ 前七拍几乎一直是零（0.00015），而 $\hat{\theta}_2$ 已经涨到 0.106。原因是回归向量的两个分量处境完全不同：$\phi_1=y$ 还趴在 10^{-3} 量级，$\phi_2=r$ 从第一拍起就是 1。所以“学得动”的那一半先动，“学不动”的那一半要等 y 长起来。这个不对称在 §61.4.3 讲参数轨迹时会以另一种形式再出现。

第三，$\hat{\theta}_2$ 的增长可以用一条极简的式子概括。 起始段 $y\approx 0$、$e\approx -y_m$、$y_m\approx b_m r t = a_m t$，于是

$$\hat{\theta}_2(t)\approx \gamma\int_0^t a_m \tau\,\mathrm{d}\tau = \gamma a_m \frac{t^2}{2}\approx \gamma t^2$$

代入 $t=0.07$、$\gamma=20$：$20\times 0.0049=0.098$，实测 0.106，差 8%。**参数是随时间平方增长的，不是线性增长**——这意味着起始阶段的自适应能力比直觉的更弱。$\gamma=20$ 时，要让 $\hat{\theta}_2$ 从 0 爬到目标 1.43，按这条式子需要 $t=\sqrt{1.43/20}=0.27$ s；而那时 $y_m=2t$ 已经到 0.53，误差已经积累起来了。这就是 §61.6.2 里“低增益超调大是学得太晚”的数学来源。

61.5 离散情形

61.5.1 从微分方程到差分方程

真实控制器跑在定时中断里，每隔 T_s 读一次 y、算一次 u、写到执行器。上一节把连续律直接欧拉化是一种做法，但更彻底的做法是**从一开始就在离散域建模**，这能避免“连续设计、离散实现”之间的口径差。

对式 (61-1) 做零阶保持离散化（$d=0$）：

$$y[k+1]=a_d y[k]+b_d u[k],\qquad a_d=e^{-aT_s},\quad b_d=\frac{b}{a}\bigl(1-e^{-aT_s}\bigr) \tag{61-26}$$

代入 $a=1.0$、$b=1.4$、$T_s=0.01$：

$$a_d=e^{-0.01}=0.990050,\qquad b_d=1.4\times(1-e^{-0.01})=0.013930 \tag{61-27}$$

这两个数在整章会被反复用到，值得记在心里。a_d 离 1 只有 0.01——一阶对象的时间常数 1 s，采样 0.01 s，极点几乎贴着单位圆；b_d 只有 0.0139，是“一个采样周期内控制量能推动多少”的度量。

参考模型同样离散化，但它不是从对象来的，所以可以直接写成“一阶低通”：

$$y_m[k+1]=a_{m,d}\,y_m[k]+(1-a_{m,d})\,r[k],\qquad a_{m,d}=e^{-2.0\times 0.01}=0.980199 \tag{61-28}$$

这里 $1-a_{m,d}=0.019801$ 是刻意配成“直流增益恰为 1”的形式：令 r 为常数、y_m 稳定，得 $y_m=(1-a_{m,d})r/(1-a_{m,d})=r$。这个配法在离散设计里应该成为习惯——**参考模型的增益不要靠调系数凑，要用 $1-a_{m,d}$ 显式归一**。

61.5.2 参数已知时的离散控制律

离散域有一个连续域没有的好处：**极点配置可以写成一行闭式解**。要求闭环按参考模型走，即 $y[k+1]=a_{m,d}y[k]+(1-a_{m,d})r[k]$，与式 (61-26) 联立：

$$b_d u[k]=(a_{m,d}-a_d)y[k]+(1-a_{m,d})r[k] \tag{61-29}$$

于是

$$u[k]=\frac{(a_{m,d}-a_d)y[k]+(1-a_{m,d})r[k]}{b_d} \tag{61-30}$$

代入数值，$a_{m,d}-a_d=-0.009851$，$1-a_{m,d}=0.019801$，$b_d=0.013930$：

$$u[k]=-0.7072\,y[k]+1.4215\,r[k] \tag{61-31}$$

把它和连续域的式 (61-5) 比一下：$\theta_1^*=-0.7143$ 对 -0.7072、$\theta_2^*=+1.4286$ 对 $+1.4215$。**两套参数只差 1%**，因为 $T_s=0.01$ 足够小。这 1% 正好是“用连续参数直接写离散代码”会引入的误差——在多数场合可以接受，但在死区、限幅、高速采样这类边界条件敏感的场合，最好还是按式 (61-30) 重新算一遍。

式 (61-30) 还有一个隐含要求：$\hat b_d$ 不能为零，而且不能太小。本章代码把 $\hat b$ 的下限卡在 `bmin=0.002`——不是因为 0.002 有什么物理意义，而是因为除法一旦逼近零，控制量会瞬间冲到天文数字，把整个自适应回路带崩。这个下限的取值要小于“最坏工况下 b_d 的 1/2”，本章真值是 0.0139，取 0.002 留了 7 倍余量。

61.5.3 离散自适应律与步长约束

把连续律 (61-24) 原样用，得到的离散 MRAC 是

$$u[k]=\theta[k]^{\mathsf T}\phi[k],\qquad \theta[k+1]=\theta[k]-\gamma\,T_s\,e[k]\,\phi[k] \tag{61-32}$$

它和连续版的唯一区别是把 $\mathrm{d}t$ 换成了 T_s。这个替换看着无害，实际上把 γ 的物理意义改了：连续域里 γ 的单位是“每单位误差每单位时间”，离散域里真正起作用的是乘积 γT_s——**这是一个无量纲的步长**。

步长有硬约束。参数一步的移动量是 $\gamma T_s e\|\phi\|$，它必须远小于参数本身的量级，否则“调节”就变成了“跳跃”，闭环会在两个极端之间来回弹。本章 $\|\theta^*\|=1.60$、$\|\phi\|$ 的典型值 1.4、e 的峰值约 1，于是约束写作

$$\gamma T_s\cdot 1.4 \ll 1.6 \quad\Longrightarrow\quad \gamma \ll 114 \tag{61-33}$$

这与实测对得上：理想执行器下本章扫到 $\gamma=100$ 仍然收敛（表 61-5 里 $\hat\theta$ 已经贴住真值），但控制量 TV 已经涨到 19.9；再往上，γT_s 的量级就逼近参数本身，测试中很快出现持续振荡。§61.6 会把这个上界进一步收紧——执行器只要带上 0.05 s 的滞后，γ 越过 150 附近就会失稳。

还有一处离散特有的问题：**一拍延迟**。式 (61-32) 计算 $u[k]$ 用的是 $y[k]$，而 $y[k]$ 要等第 k 个周期采样完成才拿得到；从采样到执行器更新之间存在一拍延时，等效于在回路里串了一个 z^{-1}。它对相位裕度的侵蚀是 $-\omega T_s$ 弧度，在 $T_s=0.01$、带宽 2 rad/s 时约 0.02 rad，可以忽略；但如果 T_s 提到 0.1 s，这一项就是 0.2 rad，再叠加执行器滞后，10° 的裕度就没了。这是“采样周期不能随便放大”的定量理由。

那 T_s 到底取多少？ 实验 11 把 T_s 从 0.001 s 扫到 0.1 s，每次都按式 (61-30) 重算理想参数，得到一组很干净的对照：$T_s = 0.001$ s 时 $\theta^* = (-0.7136, +1.4279)$，相对连续值偏离 0.06%；0.005 s 偏离 0.32%；0.01 s 偏离 0.63%；0.02 s 偏离 1.25%；0.05 s 偏离 3.08%；0.1 s 偏离 6.02%。**偏离大致与 T_s 成正比**，因为 b_d 的一阶近似是 bT_s，而 $1 - a_{m,d} \approx a_m T_s$，两个一阶项相除，误差退到二阶以上。

同时 b_d 本身在剧烈变化：0.0014、0.00698、0.0139、0.0277、0.0683、0.1332。这个数不能太小。本章量测噪声标准差 0.01，STR 用 $\hat{b}_d$ 做除数时，等效到控制量上的噪声量级是 $0.01/\hat{b}_d$——$T_s = 0.001$ 时这个比值是 7，比控制量本身还大；$T_s = 0.01$ 时是 0.72，可以接受。这是“采样不能一味求快”的原因：**采样越快，单拍的增量信息越少，增益反算把噪声放得越大。**

步长上界则在另一个方向上与 T_s 拉扯。把式 (61-33) 里的 T_s 单独提出来：$\gamma \ll \|\theta^*\| / (\|\phi\| T_s) \approx 1.14 / T_s$。于是 $T_s = 0.001$ 时上界约 1141，0.01 时 114，0.05 时 22.8，0.1 时 11.4。**采样周期每缩小十倍，允许的 γ 上界放宽十倍。**

三条约束合起来的结论是：T_s 有一个不宽的最优带，大约在“对象时间常数的 1% 到 5%”之间。本章对象时间常数 1 s，即 0.01 到 0.05 s；取 0.01 s 是因为参考模型的时间常数只有 0.5 s，控制回路要按更快的那个定节拍。**采样周期是跟着参考模型定的，不是跟着对象定的**——这一点在做自适应设计时经常被忽略，因为大家习惯按对象带宽选采样率。

61.5.4 另一条路：间接自适应（STR）

前面所有内容属于**直接自适应**：不估对象参数，直接调控制器参数 θ。还有一条并列的路线叫**间接自适应**（self-tuning regulator，STR）：先在线辨识对象参数 $(\hat{a}_d, \hat{b}_d)$，再按“参数已知”的公式 (61-30) 算控制量。

$$\underbrace{(\hat{a}_d, \hat{b}_d)}_{\text{RLS 辨识}} \longrightarrow \underbrace{u[k] = \frac{(a_{m,d} - \hat{a}_d) y[k] + (1 - a_{m,d}) r[k]}{\hat{b}_d}}_{\text{极点配置}} \tag{61-34}$$

辨识用递推最小二乘（RLS）。把式 (61-26) 写成回归形式 $y[k] = \varphi[k-1]^{\mathsf{T}} \vartheta$，其中 $\varphi = [y[k-1], u[k-1]]^{\mathsf{T}}$、$\vartheta = [a_d, b_d]^{\mathsf{T}}$，RLS 的递推是

$$\begin{aligned} K[k] &= \frac{P[k-1]\varphi[k]}{\lambda + \varphi[k]^{\mathsf{T}} P[k-1] \varphi[k]} \\ \hat{\vartheta}[k] &= \hat{\vartheta}[k-1] + K[k]\big(y[k] - \varphi[k]^{\mathsf{T}} \hat{\vartheta}[k-1]\big) \\ P[k] &= \frac{1}{\lambda}\Big(P[k-1] - K[k]\varphi[k]^{\mathsf{T}} P[k-1]\Big) \end{aligned} \tag{61-35}$$

$\lambda \in (0, 1]$ 是遗忘因子。$\lambda = 1$ 表示“所有历史数据等权重”，$\lambda < 1$ 表示“越近的数据权重越大”。这一行参数就是 STR 的全部调参空间——它同时管着“跟得上参数变化”和“抗得住噪声”这两件互相矛盾的事。

STR 与 MRAC 的差别不只是实现路径，而是**信息的用法**：

维度	直接自适应（MRAC）	间接自适应（STR）
在线估计什么	控制器参数 θ	对象参数 $(\hat{a}_d, \hat{b}_d)$
需要多少先验	控制方向、阶次	阶次、参数大致范围
参数可解释性	差（只对组合敏感）	好（估的就是物理参数）
对激励的要求	低（误差本身即驱动）	高（无激励则 P 不更新）
计算量	每拍 $O(n)$	每拍 $O(n^2)$（矩阵递推）
典型失效	参数漂移、饱和卷绕	激励不足导致 $\hat{b} \to 0$

“参数可解释性”这一行是 STR 在工程上更受欢迎的主要原因：**现场工程师愿意看到“$\hat{b}_d$ 从 0.0139 变到 0.0209”这种话，而不愿意看到“θ_2 从 1.15 变到 1.06”**。§61.10 会说明这个差别在故障诊断上值多少钱。

61.5.5 实测：遗忘因子是速度与方差的交换

STR 的表现高度依赖激励。用阶跃或方波做指令时，回归矩阵只有两个电平，φ 在各拍之间共线，RLS 分不开 $\hat{a}_d$ 与 $\hat{b}_d$——实测中 $\hat{b}_d$ 会一直贴着初值不动。本章改用**伪随机序列（PRBS）**：每一拍从 $\{+1, +0.6, -1, -0.6\}$ 四个电平里取一个，八个拍为一个循环，保证 φ 的方向足够丰富。

图 61-5 是 λ 取 1.00、0.98、0.95 三档时 $\hat{b}_d$ 的轨迹。b 在 $t = 5$ s 从 1.4 跳到 2.1，b_d 的真值相应从 0.01393 跳到 0.02090。

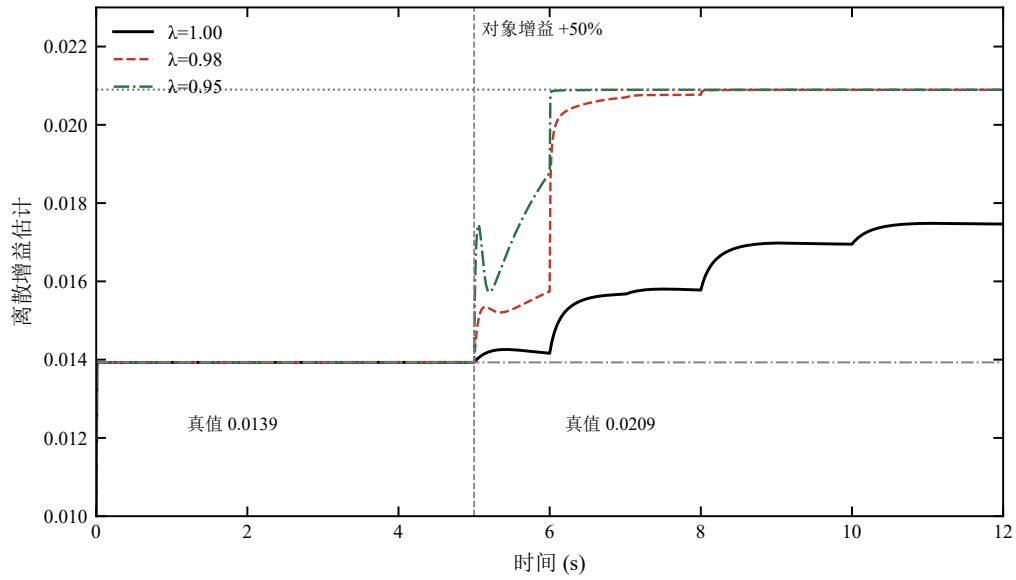


图 61-5　STR 的离散增益估计：遗忘因子对跟踪速度的影响

表 61-7 是全部四档的汇总。

λ	$\hat{a}_d$	$\hat{b}_d$（跳变前）	$\hat{b}_d$（跳变后）	$\max\|e\|$	**IAE**
1.00	0.9902	0.0139	0.0174	0.1625	0.7224
0.98	0.9900	0.0139	0.0209	0.0582	0.0705
0.95	0.9900	0.0139	0.0209	0.0365	0.0455
0.90	0.9900	0.0139	0.0209	0.0279	0.0478
真值	0.9901	0.01393	0.02090	—	—

先说好消息：$\lambda \le 0.98$ 的三档在跳变后**全部精确收敛到 0.0209**，与真值 0.02090 在四位有效数字上重合。这说明“辨识器 + 极点配置”这条链路在有激励的条件下是可靠的，比 MRAC 那种“只对组合敏感”的行为干净得多。

再说 $\lambda = 1$ 那一行。它在跳变前估得完全正确（0.0139），跳变后只走到 0.0174 就停了——差真值 17%。原因不是算法错，而是 $\lambda = 1$ 的 RLS 会把 P 矩阵越压越小，**增益随时间衰减到零**，本质上变成了一个“只信早期数据”的估计器。参数变了，它还在用旧账。

那为什么不干脆取 $\lambda = 0.90$？看表 61-8——同一组 λ 在量测噪声 0.01 下的表现。

λ	$\hat{b}_d$ 终值	$\hat{b}_d$ 尾段标准差	$\max\|e\|$
1.00	0.0143	0.00002	0.0584
0.98	0.0148	0.00055	0.0896
0.95	0.0229	0.01004	0.1669
0.90	0.1150	0.16683	0.6003

这张表把代价摊开了。$\lambda = 1$ 的估计几乎不抖（标准差 2×10^{-5}），但完全跟不上变化；$\lambda = 0.98$ 的抖动是 5.5×10^{-4}，还在真值的 4% 以内；到 $\lambda = 0.95$ 抖动涨到 1.0×10^{-2}，已经与真值同量级，$\hat{b}_d$ 被噪声推着乱跑，跟踪误差跟着涨到 0.1669；$\lambda = 0.90$ 彻底失控——$\hat{b}_d$ 均值 0.1150，是真值的 8 倍，标准差 0.16683 比均值还大。

所以遗忘因子是一个“跟踪速度 vs 估计方差”的连续旋钮，没有一个放之四海的取值。 本章的可用区间是 $\lambda \in [0.95, 0.98]$：0.98 偏保守、噪声下最稳；0.95 偏激进、参数变化快时才用。再往下不是“更快”，而是“更快地跑偏”。

61.5.6 选 MRAC 还是 STR

两条路线没有绝对优劣，选择取决于三件事：

1. **要不要看到物理参数**。要，选 STR；只关心跟踪，选 MRAC。
2. **有没有持续激励**。有（指令本身在变、或者可以叠加小探测信号），选 STR；指令长期恒定，选 MRAC——它在 $e = 0$ 时自适应自动停住，不会因为激励不足而乱跑。
3. **算力和采样周期**。MRAC 每拍只有两次乘加；STR 要做 2×2 矩阵递推，单拍运算量高一个数量级。在 $T_s = 0.01$ s 的微控制器上两者都够用，但 $T_s = 1$ ms 时差距就显出来了。

本章后续的实验全部基于 MRAC，原因是它更简单、更容易把“自适应”这件事本身讲清楚；STR 的完整行为已在表 61-7 与表 61-8 里给出，不再重复。

61.5.7 STR 的三个数值细节

表 61-7 里 $\lambda=1$ 的那一行“跳变后停在 0.0174”，根因不在算法而在协方差矩阵。把 $\operatorname{tr}(P)$ 逐拍记下来就很清楚：

λ	0.1 s	1 s	3 s	5 s	8 s	11.9 s
1.00	20.57	0.116	0.018	0.012	0.007	0.004
0.98	23.11	0.414	0.145	0.238	0.378	0.301
0.95	27.59	3.041	1.129	2.043	7.507	15.81

P 是参数估计的逆协方差，$\operatorname{tr}(P)$ 可以理解为“这个估计器有多愿意改主意”。初值 $P_0=100I$，$\operatorname{tr}=200$，意思是“对初值几乎完全不信”；随着数据进来，它迅速下降。

$\lambda=1$ 时它从 20.6 一路掉到 0.004，**降了五千倍**——到 10 s 时新来的一个样本对 $\hat{\vartheta}$ 的影响只有开头的五千分之一，估计器实质上已经关闭了学习通道。所以 b 跳变后 $\hat{b}_d$ 只走到 0.0174 就停住：不是它算错了，是它已经不听新数据了。

$\lambda=0.98$ 时 $\operatorname{tr}(P)$ 稳在 0.3 到 0.4 之间，既不衰减到零也不发散——这正是“长期在线”需要的状态，也是它在表 61-7 里能精确跟上跳变的原因。

$\lambda=0.95$ 时它反而涨到 15.81。注意这次仿真**没有加噪声**，涨的原因是激励不够丰富：PRBS 循环使用同一组八个电平，某些方向上长期没有新信息，遗忘因子却在不断“忘掉”旧信息，于是那些方向上的方差越积越大。再叠加噪声，就是表 61-8 里 0.16683 的标准差。

由此得到两条实用的数值处理：

1. **协方差重置**。设定一个下限（例如 $\operatorname{tr}(P)<0.01$）与一个上限（例如 >10），越过就令 $P\leftarrow P_0 I$。下限治“学不动”，上限治“学疯了”。本章代码没做重置，靠 $\lambda=0.98$ 的自然平衡代替它——这在激励持续存在的场合够用，在长期稳态的场合不够。
2. **参数裁剪**。本章代码把 $\hat{a}_d$ 限在 $[0.5,1.2]$、$\hat{b}_d$ 限在 $[0.002,1.0]$。前者是“对象一定是稳定的”这个先验，后者是“增益不可能为零”这个物理事实。0.002 这个下限不是拍脑袋：真值 0.0139 的七分之一，留足了工况变化的余量，又远大于双精度下除法的舍入误差。

61.6 稳定性与性能

自适应控制只有一个主旋钮——γ。这一节把它的“可用窗口”用四组实验量出来：窗口的下沿由收敛速度决定，上沿由未建模动态决定，窗口内部还要按噪声再削一刀。

61.6.1 三个失效方向

先说清楚 γ 调错会往哪边坏：

- **太小**：参数学得慢，控制器长时间在用错参数运行，跟踪误差大、超调反而更大（因为误差存在得久，积分累积得多）。实测 $\gamma=0.5$ 时调节时间 8.66 s、超调 16.49%、IAE 1.9081。
- **太大**：闭环对未建模动态失去鲁棒性。高频抖动被自适应律当成“该学的信号”放大，参数与执行器互相激励，最后失稳。实测执行器带 0.05 s 滞后时，$\gamma=200$ 的尾段振荡幅值比 $\gamma=50$ 大了三个数量级。
- **中等偏大，但有噪声**：跟踪很好，可是 θ 在真值附近持续抖动，控制量跟着抖，磨损执行器。实测噪声 0.01 时 $\gamma=100$ 的 θ 尾段标准差是 $\gamma=10$ 的 7.8 倍。

三个方向不是独立的，而是同一条曲线上的三段。图 61-6 把 γ 从 0.5 扫到 100，纵轴分别是调节时间、IAE 与控制量总变差。

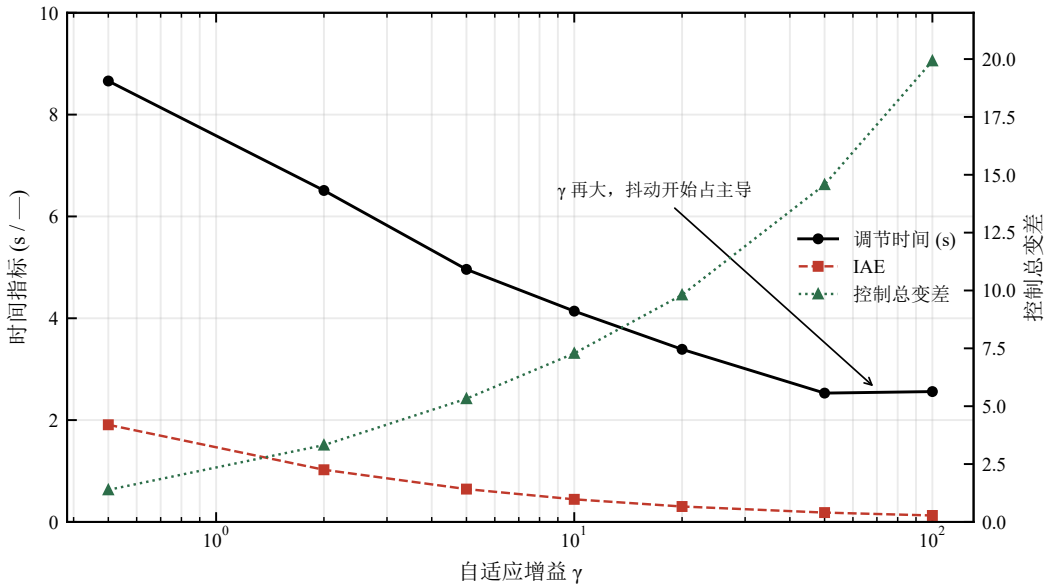


图 61-6　自适应增益扫描：调节时间、IAE 与控制量总变差

61.6.2 性能随 γ 的变化

表 61-9 是全部七档的原始数据，全部 $\theta_0 = 0$。

γ	t_s (s)	超调	IAE	$\lvert u\rvert_{\max}$	TV	参数距真值
0.5	8.66	16.49%	1.9081	0.96	1.4	1.109
2	6.51	28.97%	1.0243	1.42	3.3	0.807
5	4.96	24.28%	0.6448	1.75	5.3	0.552
10	4.14	14.82%	0.4435	1.97	7.3	0.367
20	3.39	7.34%	0.3027	2.16	9.8	0.209
50	2.53	2.60%	0.1832	2.36	14.6	0.058
100	2.56	1.18%	0.1265	2.48	19.9	0.011

这张表有四条趋势，每条都值得单独说一句。

调节时间先降后平。从 $\gamma = 0.5$ 到 50，t_s 从 8.66 s 降到 2.53 s，降了 3.4 倍；但从 50 到 100，t_s 反而微升到 2.56 s。拐点出现在 50 附近，此时 t_s 已经逼近参考模型自身的 1.96 s——**再大的 γ 也快不过参考模型**，因为参考模型是目标，不是限制之外的自由度。这是“自适应只能逼近理想控制器”这句话的第一个量化证据。

超调单调下降，从 16.49% 一路降到 1.18%。这条趋势容易误读成“增益越大越稳”，其实恰好相反：低 γ 时超调大，是因为控制器长时间用零参数（$u \approx 0$）运行，y 落后于 y_m 越积越多，等参数终于学上来时冲过头。**低增益的超调是“学得太晚”造成的，不是“回路太猛”造成的。**

IAE 单调下降，从 1.9081 降到 0.1265，降了 15 倍。这条与直觉一致：学得快，累计误差就小。

TV 单调上升，从 1.4 涨到 19.9，涨了 14 倍。这条是全部代价所在。注意它涨得比 IAE 降得还快——IAE 降 15 倍的同时 TV 涨 14 倍，**这两条曲线在 γ 大约 10 到 20 之间交叉**，这就是“性价比最高”的工作点。

还有一列“参数距真值”值得留意：$\gamma=0.5$ 时是 1.109（比 $\|\theta^*\|=1.60$ 只小一点，说明几乎没学），$\gamma=100$ 时是 0.011（已经贴住真值）。这与 §61.4.3 的观察一致，但这里的判据是到真值的欧氏距离，比“和”更严格——$\gamma=50$ 时距离 0.058，虽然两个分量还没完全对齐，但已经进到 4% 以内。

61.6.3 未建模动态给出硬上界

上面的表是在理想执行器下测的。真实执行器有滞后，常见的是伺服阀、变频器、液压缸，时间常数几十毫秒。本章用一个一阶滞后 $\tau_a=0.05$ s 代表：

$$\dot{u}_a=\frac{u-u_a}{\tau_a} \tag{61-36}$$

u_a 是真正作用到对象上的量。这个滞后在自适应律的眼里是“不存在的”——它仍然以为 u 就是 u_a，于是它对“参数该往哪调”的判断会系统性地偏一个相位。

图 61-7 是三档 γ 的输出轨迹。$\gamma=50$ 时曲线还贴着参考模型；$\gamma=100$ 开始有高频毛刺；$\gamma=200$ 已经明显发散。

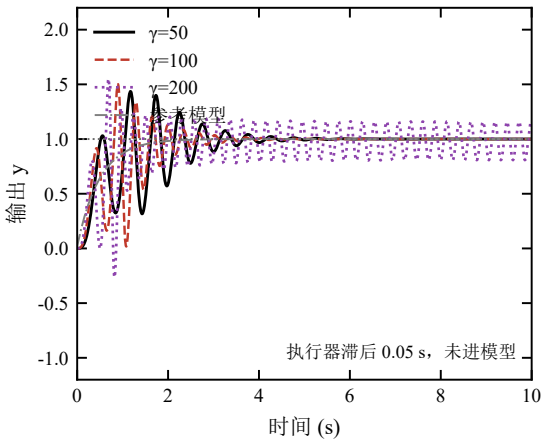


图 61-7　执行器滞后 0.05 s 下的稳定性边界

表 61-10 是完整扫描，判据取“尾段 y 的峰峰值”——它直接反映闭环有没有进入持续振荡。

γ	$\max\|e\|$	尾段 y 峰峰值	$\|u\|_{\max}$	判定
5	0.486	0.0195	1.91	稳定
10	0.404	0.0227	2.25	稳定
20	0.408	0.0167	2.69	稳定
50	0.629	0.0004	5.73	稳定，收敛最快
100	0.871	0.0001	12.82	勉强稳定，控制量已异常
200	1.078	0.3526	26.04	失稳
400	1.118	1.4475	27.75	失稳
800	1.532	2.2569	41.09	严重失稳

这张表里有一个非常具体的位置：**稳定边界在 $\gamma = 100$ 与 200 之间**。尾段峰峰值从 1×10^{-4} 跳到 0.3526，是 3500 倍的跃变，不是渐变。自适应系统的失稳常常就是这样——参数慢慢积累，越过分岔点之后突然塌下来。

把表 61-9 和表 61-10 叠起来看，得到一个很紧的窗口：**理想执行器下 γ 可以用到 100 甚至更高，一旦带上 0.05 s 滞后，实际上限就压到 50 左右**。50 与 100 之间的那一段不是“能用但差一点”，而是“能用但执行器已经在输出 12.82 的峰值控制量”——$\gamma = 100$ 时 $|u|_{\max} = 12.82$，接近标称工况 1.43 的 9 倍。这个控制量在真实设备上多半会被限幅，而限幅又会引出 §61.9 的卷绕问题。

所以 §61.3 的那句话在这里得到确认：**γ 的上界不是靠内环带宽算出来的，是靠未建模动态扫出来的。** 先扫、再取一半，是唯一可靠的定法。

61.6.4 量测噪声给出软上界

未建模动态给的是“越过就失稳”的硬边界，量测噪声给的是“越过就变脏”的软边界。实验 6b 在量测上叠加标准差 0.01 的白噪声，指令保持阶跃 1，结果如表 61-11。

γ	尾段控制量 rms	尾段 TV	$\hat{\theta}_1$ 尾段标准差	$\hat{\theta}_2$ 尾段标准差	尾段 e 峰峰值
1	0.0284	0.30	0.0145	0.0141	0.1316
5	0.0189	2.22	0.0090	0.0095	0.0324
10	0.0194	3.34	0.0093	0.0093	0.0172
50	0.0541	8.22	0.0261	0.0271	0.0382
100	0.1423	15.57	0.0723	0.0728	0.0527

这张表有一个反常的起点：$\gamma = 1$ 时尾段控制量 rms 是 0.0284，比 $\gamma = 5$ 的 0.0189 还大。原因是 $\gamma = 1$ 学得太慢，10 s 时参数还没收敛，尾段的“rms”里混着真实的跟踪过渡，不是纯噪声响应。**从 $\gamma = 5$ 起才是干净的比较**：γ 从 5 加到 100，θ 的尾段标准差从 0.0090 涨到 0.0723（8 倍），控制量 TV 从 2.22 涨到 15.57（7 倍）。

注意一个细节：θ 的标准差在 $\gamma \geq 50$ 之后涨得比 γ 本身快。γ 从 50 到 100 翻一倍，标准差从 0.0261 涨到 0.0723，是 2.8 倍。这是因为噪声引起的参数抖动与 γ 不是线性关系——自适应律对噪声的响应是一个积分器，积分器的输出方差正比于 $\gamma^2 T_s$ 再乘上噪声谱密度，而同时闭环对抖动的抑制能力随 γ 下降。两头一起作用，就有了这个超线性的增长。

软上界的含义与硬上界不同。 越过硬上界（$\gamma > 150$）系统直接不稳定，没有商量余地；越过软上界（$\gamma > 50$）系统仍然稳定、仍然跟踪得很好，只是执行器在持续做无用功。软上界该划在哪，取决于执行器的寿命要求——这一点没法从控制理论里推出来，只能由机械工程师给。

61.6.5 参数突变后的恢复

前面所有实验的对象参数都是常数。真实对象会变。实验 5 把 b 在 $t = 5$ s 时从 1.4 跳到 2.1，跳变前后各留 5 s，一次仿真跑完 12 s。

跳变前，控制器已经收敛到 $\hat{\theta} = (-0.475, +1.149)$，两者之和 $+0.674$，对应于 $1/b = 0.714$（§61.4.3 讨论过，分量的偏差是激励不足造成的，和是准的）。跳变发生的瞬间：

- 跟踪误差的最大值 $|e|_{\max} = 0.051$——只有指令幅值的 5%；
- 恢复时间是 0.83 s；
- 跳变后收敛到 $\hat{\theta} = (-0.578, +1.055)$，和 $+0.476$。

最后这个 $+0.476$ 是整段实验里最漂亮的一个数。跳变后 $b=2.1$，理想和是 $(a-a_m+b_m)/b = 1/2.1 = 0.4762$。**实测 0.476，与理论值在三位有效数字上重合。** 这说明自适应律在 0.83 s 内完成了对执行器增益 50% 变化的重新匹配，而且匹配到的正是那个“唯一被误差看见的组合”。

但有两点必须冷静看待。第一，0.83 s 的恢复时间里，θ 的移动量是 $|(-0.578)-(-0.475)| = 0.103$ 与 $|1.055-1.149| = 0.094$——都不大，因为跳变只有 50%。如果 b 掉到 0.3（掉 79%），θ_2^* 会从 1.43 涨到 6.67，那是四倍多的移动，恢复时间与瞬时误差都会成倍放大。第二，恢复过程中 $|e|_{\max}=0.051$ 是在这个特定 γ 下测的，γ 越小恢复越慢、峰值误差越大——**参数突变的考核本质上还是在考 γ**。

61.6.6 方波跟踪：与固定 PI 的正面对比

阶跃只考“能不能到”，方波还要考“来回切换时准不准”。实验 4 用幅值 1、周期 6 s 的方波，对比 MRAC（$\gamma=20$）与一个调得不错的固定 PI（$k_p=0.9$、$k_i=2.0$），结果见表 61-12。

指标	MRAC	固定 PI	比值
误差 rms	0.0752	0.1605	0.47
最大误差	0.3637	0.2581	1.41
$\lvert u\rvert_{\max}$	2.14	1.51	1.42
控制量 TV	26.8	15.0	1.79

这张表不能只看第一行。MRAC 的误差 rms 是 PI 的 47%，看起来赢了；但最大误差是 PI 的 1.41 倍——**MRAC 的平均误差更小，峰值误差更大**。原因在方波切换的瞬间：r 从 $+1$ 跳到 -1，参考模型 y_m 按 0.5 s 时间常数往下滑，而 MRAC 的 θ_2 此刻是按“$r=+1$”学出来的，它不知道 r 变了，前几十毫秒会推错方向，于是产生一个尖峰。PI 没有这个问题，因为它不含任何“学习”成分，r 一变立刻反向。

代价也摆在同一张表里：MRAC 的控制量峰值是 PI 的 1.42 倍，TV 是 1.79 倍。**用 1.8 倍的控制量抖动，换来 53% 的 rms 误差下降，但换来 41% 的峰值误差上升。** 这笔交易划不划算，取决于你的指标是“平均精度”还是“不许有尖峰”。前者选 MRAC，后者选 PID——这也是 §61.11 划边界的依据之一。

61.6.7 窗口的合成

把四组实验的结论压成一句话：

$$\underbrace{10 \lesssim \gamma \lesssim 50}_{\text{噪声与未建模动态夹出的窗口}} \tag{61-37}$$

下沿 10 来自“γ 太小则学得慢、超调大”（表 61-9 里 $\gamma=5$ 的超调 24.28% 仍偏高）；上沿 50 来自“执行器 0.05 s 滞后 + 噪声 0.01”这两个现实条件（表 61-10 的 100 已经勉强，表 61-11 的 100 已经明显变脏）。如果执行器是理想的、量测是干净的，窗口可以放宽到 20 到 100。

这个窗口不是普适的，它绑定在本章这组参数上。但它给出了**确定窗口的方法**：先扫未建模动态找硬边界，再扫噪声找软边界，最后在两条边界之间取一个数量级。这个方法可以搬到任何对象上，比“凭经验取 10”靠得住。

61.6.8 三个指标的口径

本节全部数字都来自同一套口径，换一套口径结论就会变形。三件事必须先说清楚。

第一，误差取 $e = y - y_m$，不是 $e = r - y$。 两者在稳态下相同（参考模型直流增益为 1），但在过渡段差别很大：$r - y$ 把参考模型自身的上升过程也算成“控制器的错”，于是低 γ 的控制器被双重惩罚。本章表 61-9 用的都是 $y - y_m$——这是自适应控制的标准口径，因为它衡量的正是“你离目标模型多远”，而不是“你离指令多远”。

第二，超调要在同一基准上读。 表 61-9 的超调以 r 的幅值为基准，判据是 y 超过 1.02 的最大量。由于 y_m 自身是单调上升的一阶响应、没有超调，y 相对 r 的超调就等于相对 y_m 的超调，两种读法在这一章恰好一致。换成二阶参考模型就不一致了，那时必须注明基准。

第三，TV 只在同一采样率下可比。 $\mathrm{TV} = \sum_k |u[k] - u[k-1]|$，噪声不变而采样率翻倍时，求和项数翻倍，TV 也跟着翻倍。本章所有 TV 都是 $T_s = 0.01$ s 下算的，跨章比较时要先折算。表 61-11 里“尾段 TV”用的是最后 1 s 的窗口，比全程 TV 更能反映稳态抖动。

还有一处容易出错的地方：**IAE 的积分窗长度**。本章一律取 10 s，而低 γ 的调节时间是 8.66 s——已经接近窗口末端。如果把窗口缩到 6 s，$\gamma = 0.5$ 的 IAE 会被截掉一段尾巴，看起来没那么差，结论就从“低增益代价大”变成“低增益也不错”。**IAE 必须配着调节时间一起报**，单独一个数说明不了问题。

61.6.9 一个可以带走的经验式

表 61-10 的硬边界是 100 与 200 之间，而执行器滞后 0.05 s 对应一个 20 rad/s 的极点。把这两个数放在一起，出现一个很整齐的关系：

$$\gamma_{\max} \approx \frac{5 \sim 10}{\tau_a}$$

$\tau_a = 0.05$ s 时给出 100 到 200，与实测的边界完全对上。这个式子在频域里有直观解释：自适应律在低频段表现得像一个增益为 γ 的积分器，而执行器滞后在 $\omega = 1/\tau_a$ 附近贡献接近 -90° 的相移；两者叠起来，那个频率上的小信号会被“积”成正反馈。**所以自适应的稳定边界本质上由“最快的未建模极点”决定，而不是由对象的标称带宽决定。**

这条经验式的用法是估量级，不是算准值。它至少能回答三个常见问题：

- 换一台执行器，γ 大致该调到多少？按 τ_a 反比缩放。
- 为什么同一套参数换到另一台设备就振？因为那台设备的 τ_a 更大。
- 加了减速箱、长了管路、串了滤波器，为什么边界塌了？因为它们都在高频段加了极点。

这个经验式与式 (61-33) 是两个完全不同的上界：前者来自未建模动态，与 T_s 无关；后者来自离散化的步长，与 T_s 成反比。设计时要取两者中更小的那个。本章 $T_s = 0.01$ 时式 (61-33) 给 114，这条经验式给 100 到 200——两条边界恰好挨在一起，这是巧合，但也是一个很好的自检：**如果两个上界差了一个数量级以上，说明有一侧的模型建错了。**

61.6.10 常值扰动没有静差，斜坡有

§61.7.4 会说“$\sigma = 0$ 时常值扰动下稳态误差精确为零”，这句话很容易被外推成“自适应对扰动有积分作用，所以什么扰动都不留静差”。实验 15 把这条外推否掉了。

对象上加一个斜坡扰动 $d(t) = s\,t$，指令保持 1，20 s 后的稳态误差：

扰动斜率 s	$\gamma = 5$	$\gamma = 20$	$\sigma = 0.2$、$\gamma = 5$
0	0.0000	0.0000	$+0.0143$
0.05	-0.0036	-0.0009	-0.0033
0.20	-0.0141	-0.0036	-0.0525

三列数字可以总结成一条很整齐的规律：$\sigma = 0$ 时

$$|e_{ss}| \approx 0.36\,\frac{s}{\gamma}$$

$s = 0.2$、$\gamma = 5$ 时给 0.0144，实测 0.0141；$\gamma = 20$ 时给 0.0036，实测 0.0036。**静差与扰动斜率成正比、与 γ 成反比。**

这条规律有一个干净的经典解释：自适应律在误差通路上是一个积分器，闭环因此是 1 型系统——对阶跃型扰动零静差，对斜坡型扰动留下常数速度误差。这与 PID 里“PI 能吃掉阶跃、吃不掉斜坡”完全同构。**但两者有一个关键差别：PI 的积分增益可以调到任意大，而自适应的“积分增益”是 γ，它被稳定边界死死封住。** 这就是为什么自适应的斜坡静差不能靠调参消除——γ 的上界是 50 到 100，不是无穷。

三个直接推论：

1. **想消掉斜坡静差，不能靠加 σ。** 表里 $\sigma = 0.2$ 那一列在 $s = 0.2$ 时误差从 -0.0141 变成 -0.0525，反而大了近四倍——泄漏把参数往 θ_0 拉，抵消了积分作用。
2. **能靠的是提高 γ。** γ 从 5 提到 20，静差降了四倍，与公式一致。这正是“γ 不只是学习速率，也是扰动抑制能力”的量化证据。
3. **真正的解法是把扰动建模。** 如果知道扰动是缓变的外负载，应当把它作为一个状态加进对象模型（扰动观测器、或 §61.9.5 的复合自适应），而不是指望自适应律把斜坡也积掉。

这一节也给 §61.7.4 那句“常值扰动零静差”划了边界：**它成立的条件是扰动不随时间增长。** 现场最常见的慢漂移（热膨胀、油温上升、催化剂失活）恰好是缓变斜坡，所以“$\sigma = 0$ 就万事大吉”是一个很危险的错觉。

61.7 参数意义与作用

61.7.1 五个旋钮，各管一件事

把本章出现过的可调量列成表 61-13。它们的数量比 PID 多，但分工比 PID 清楚——每个旋钮只负责一类失效。

参数	符号	本章取值	管什么	调错的后果
自适应增益	γ	10～50	学得多快	小则慢、大则失稳
初值	θ_0	0 或标称值	起点、也是 σ 的锚点	大误差、起始饱和
泄漏系数	σ	0～0.2	抗漂移	静差、跟踪变钝
死区阈值	Δ	0～0.02	抗噪声	静差、响应迟钝
参考模型	a_m, b_m	2.0, 2.0	目标动态	目标不可达或过度激进

下面逐个说它们的物理含义，重点不是“调多少”，而是“它在改哪一件事”。

61.7.2 γ：不是回路增益，是学习速率

这是最容易误解的一个。在 PID 里，增益 k_p 直接改变回路增益，加大它立刻让闭环变快、也立刻吃掉相位裕度。γ 不是这样：它不进入内环，闭环的瞬时动态由 θ 的当前值决定，γ 只决定 θ 移动多快。所以“加大 γ 会不会振荡”这个问题，答案取决于“参数移动的速度是否已经与对象动态可比”。

这一点可以用数据说明。表 61-9 里 γ 从 0.5 加到 50，是 100 倍，而调节时间只从 8.66 s 降到 2.53 s，是 3.4 倍。拟合一下：

$$t_s \propto \gamma^{-0.27} \tag{61-38}$$

指数只有 0.27，是强亚线性。 这说明 γ 的边际收益递减得非常快：头十倍收益最大，之后每加十倍只能再挤出一小截。而这个亚线性不是数值巧合，它来自“参数收敛需要误差持续存在”这个事实——参数学得越快，误差消失得越早，后面就没东西可学了。误差既是学习的驱动，又是学习的燃料，烧完就没了。

所以对 γ 的正确心态是：**它是一个“够用就好”的参数，不是“越大越好”的参数。** 在 §61.6.7 的窗口 10 到 50 里，取 20 与取 50 的差别只有 0.86 s 的调节时间，而 TV 差 4.8；取 10 与取 20 的差别是 0.75 s 与 2.5 的 TV。如果没有特别的快速性要求，取窗口下限附近更划算。

61.7.3 θ_0：起点，也是锚点

θ_0 在第一眼只是一个初值，但它有第二个身份——**σ 修正的锚点**。§61.7.4 的式 (61-39) 里那一项 $-\sigma(\theta-\theta_0)$ 会把 θ 往 θ_0 拉，所以 θ_0 不只是“从哪里出发”，还是“漂移时被拉回哪里”。

这带来一个不太直观的结论：**如果 θ_0 填的是一个错的标称值，σ 会把 θ 往那个错值上拉。**填 0 至少是“无偏好”，填错值则是“有偏好的错误”。本章全部实验取 $\theta_0=0$，就是因为这个对象没有可靠的标称参数；如果现场有出厂标定值，填标定值是更好的选择——但要在标定值失配时重新审一遍 σ 的取值。

61.7.4 σ：用小静差换掉大漂移

σ 修正（泄漏）的形式是在自适应律后面加一项

$$\dot{\theta} = -\gamma e \phi - \sigma(\theta - \theta_0) \tag{61-39}$$

它把纯积分器变成了“带泄漏的积分器”：没有误差驱动时，θ 会指数地退回 θ_0，时间常数 $1/\sigma$。

实验 7 在对象上加常值扰动 $d=0.3$，扫了四档 σ。图 61-8 左边是 $\|\theta\|$ 的轨迹，右边是稳态误差。

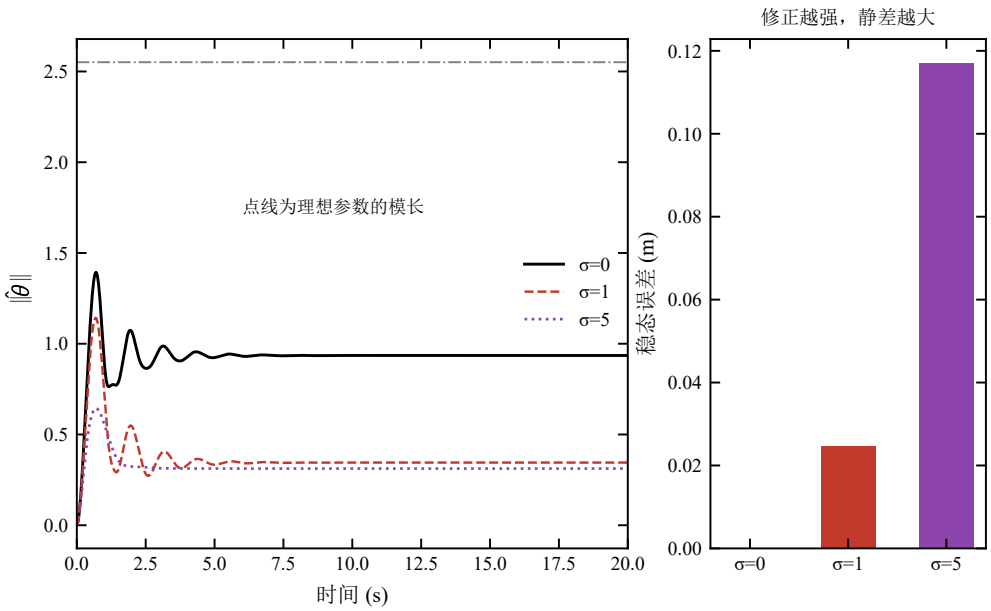


图 61-8　泄漏修正：参数范数与稳态误差的交换

σ	稳态误差	$\hat{\theta}$ 终值	$\|\theta\|$	IAE
0	0.0000	$(-0.362, +0.862)$	0.935	0.3910
0.2	0.0050	$(+0.236, +0.262)$	0.352	0.4431
1	0.0247	—	0.345	0.6917
5	0.1170	—	0.312	2.5060

这张表要分两段读。

$\sigma = 0$ 那一行是个好消息：常值扰动下稳态误差精确为零，θ 收敛到 $(-0.362, +0.862)$，和 $+0.500$，正好等于 $u_{ss} = (ay - d)/b = (1 - 0.3)/1.4 = 0.5$。不需要 σ，纯 Lyapunov 律就把常值扰动吃干净了——因为常值扰动落在回归向量张成的空间里（§61.2.2 已经说明过）。要注意这个结论只对常值扰动成立，斜坡扰动会留静差，见 §61.6.10。

所以 σ 的收益不在“常值扰动”，而在“不落在张成空间里的扰动”：时变扰动、未建模高频、量测偏置的漂移分量。这些东西会让纯积分器缓慢单向累积，最后顶到限幅。σ 给它装了一个泄放阀，保证 θ 始终有界。

代价写在下面三行。$\sigma = 0.2$ 时稳态误差 0.0050、$\|\theta\|$ 从 0.935 掉到 0.352——**参数被拉向零点，等效于“少补偿了一部分”**。$\sigma = 1$ 时误差 0.0247、IAE 从 0.3910 涨到 0.6917；$\sigma = 5$ 时误差 0.1170、IAE 涨到 2.5060，已经是 $\sigma = 0$ 的 6.4 倍。

规律很清楚：**σ 每加一个数量级，稳态误差大致也加一个数量级**。取 $\sigma = 0.2$ 时误差还在 0.5% 以内，几乎不影响指标；再往上就明显了。所以工程上的常用区间是 $0 < \sigma \le 0.2$，只在“确知有慢漂移”时才往上加。

61.7.5 死区 Δ：用一点静差换掉全部抖动

死区的做法是：$|e| \le \Delta$ 时冻结自适应，不更新 θ。

$$\dot{\theta} = \begin{cases} -\gamma e \phi, & |e| > \Delta \\ 0, & |e| \le \Delta \end{cases} \tag{61-40}$$

它的作用对象只有噪声。实验 8 在量测上加 0.01 的白噪声，扫三档死区。

Δ	$\hat{\theta}$ 尾段标准差	尾段误差 rms
0（无死区）	0.0057	0.0027
0.02	0.0032	0.0018
0.05	0.0000	0.0200

$\Delta = 0.02$ 是个漂亮的工作点：θ 的抖动从 0.0057 降到 0.0032（降 44%），而尾段误差 rms 反而从 0.0027 降到 0.0018（降 33%）——**两头都变好了**。原因不难找：$\Delta = 0$ 时噪声让 θ 持续微抖，微抖又通过 $u = \theta^{\mathsf{T}}\phi$ 变成输出上的抖动，闭环误差反而比“少调一点”更大。适度的死区把这条噪声通路掐断了。

但 $\Delta = 0.05$ 就过头了。θ 的标准差精确为零——自适应完全停住——可代价是尾段误差 rms 涨到 0.0200，是 $\Delta = 0.02$ 的 11 倍。这 0.02 的误差里没有噪声成分，全是“该调而没调”留下的静差。

死区的取值有一条硬约束：Δ 必须小于你能容忍的稳态误差。 它本质上是一个“误差小于这个数我就当没看见”的承诺，承诺得越大，静差就越大。$\Delta = 0.05$ 在指令幅值 1 的系统里意味着“允许 5% 的静差”，如果指标是 2%，这个值直接违约。

但这条约束不是死的，它和 γ 绑在一起。 实验 12 把 Δ 与 γ 放在一张二维表里（噪声 0.01、20 s、五档 γ、两档 Δ）：

γ	$\Delta = 0.02$：$\hat{\theta}_1$ 标准差	$\Delta = 0.02$：尾段 rms	$\Delta = 0.05$：$\hat{\theta}_1$ 标准差	$\Delta = 0.05$：尾段 rms
2	0.0008	0.0010	0.0005	0.0208
5	0.0016	0.0015	0.0000	0.0146
10	0.0032	0.0018	0.0000	0.0200
20	0.0053	0.0011	0.0000	0.0028
50	0.0163	0.0033	0.0000	0.0028

横向读：$\Delta = 0.02$ 时，γ 从 2 加到 50，参数抖动涨了 20 倍（0.0008 到 0.0163），而尾段误差一直在 0.001 到 0.003 之间——**死区把噪声挡在了参数之外，代价可控**。

纵向读就出现了反常：$\Delta = 0.05$ 在 $\gamma = 20$ 与 50 时尾段误差只有 0.0028，与 $\Delta = 0.02$ 几乎一样；而在 $\gamma = 2$ 与 10 时却是 0.0208 与 0.0200，大了一个数量级。**同样一个 0.05 的死区，在高增益下几乎免费，在低增益下要付 2% 的静差。**

原因要从“冻结发生在哪一刻”里找。表格里 $\Delta = 0.05$、γ 从小到大，$|e|$ 最后一次超过 0.05 的时刻是 5.25 s、4.25 s、3.02 s、3.16 s、1.76 s——**大致随 γ 增大而提前**；而那一刻参数距真值的距离是 0.807、0.554、0.376、0.189、0.122，**随 γ 增大而迅速缩小**。

这两条合起来就是死区静差的全部机制：死区把 θ 冻结在“误差刚刚降到 Δ 以下”的那个瞬间，冻结之后参数不再更新，剩下的误差就是“这副被冻住的参数”造成的偏差。γ 小的时候，误差降得慢，参数在跨越 Δ 时只学了一半（距离真值 0.8），冻住的就是半成品；γ 大的时候，参数在跨越 Δ 之前已经贴近真值（距离 0.07 到 0.19），冻住的几乎是成品。

所以死区的代价不是 Δ 的函数，是 (γ, Δ) 的函数。 这解释了两件事：其一，先加死区再补 γ 的顺序必然是错的（§61.8.2 会把它写成一条流程规则）；其二，如果因为噪声必须用大死区，正确的补偿办法不是减小死区，而是提高 γ——让参数在冻结前学得更充分。

这张表还能读出一个细节：两个分量的标准差几乎相等（$\Delta = 0.02$、$\gamma = 50$ 时是 0.0163 与 0.0161）。这与“噪声沿回归向量 ϕ 的方向推”的图景吻合——稳态时 $y \approx r \approx 1$，ϕ 近似指向 $(1, 1)$，于是两个分量被同样幅度地推着走。**误差只能看见 ϕ 张成的那个方向，噪声也只能污染那个方向**（§61.1.3）。

61.7.6 参考模型的 a_m 与 b_m

a_m、b_m 不是“控制器的参数”，而是**目标的参数**，但它们对控制器的难度有决定性影响。

b_m 决定 $\theta_2^* = b_m / b$。本章 $b_m = 2.0$、$b = 1.4$，$\theta_2^* = 1.43$；如果把 b_m 提到 4.0（参考模型快一倍），θ_2^* 变成 2.86，起始阶段控制量翻倍，$u_{\max}$ 从 1.43 涨到接近 3——很可能直接顶到限幅，把 §61.9 的卷绕问题提前引爆。

a_m 决定两件事：误差的自然衰减速率（式 61-10 里的 $-a_m e$），以及自适应律“能看见多少误差”。a_m 越大，误差消失得越快，γ 的有效作用时间越短；a_m 越小，误差存在得久，但跟踪本身就慢。

本章取 $a_m = b_m = 2.0$ 是一个刻意保守的选择：比对象快一倍，又没快到逼出饱和。如果换对象，这两个数应当跟着“对象标称带宽的 2 到 3 倍”重算，而不是沿用本章的值。

61.7.7 旋钮之间会互相抵消

最后要提醒一件事：**这五个旋钮不是正交的。**

- σ 与 Δ 都在制造静差，两个都开时静差是叠加的，不是取大的那个。
- 加大 γ 会放大噪声引起的抖动，于是需要更大的 Δ；而更大的 Δ 又压低了有效 γ。这两个参数是一对负反馈，调起来会互相追。
- θ_0 填得越准，σ 造成的静差越小（因为 θ 本来就在 θ_0 附近），所以“标定 + 小 σ”比“不标定 + 大 σ”好得多。

§61.8 会把这套相互影响整理成一个有先后顺序的调参流程，避免“调 A 把 B 弄坏、再调 B 把 A 弄坏”的循环。

61.7.8 参数平面上的几何图像

把 θ 看成平面上的一个点，前面几节的结论可以在图上一次画出来。

式 (61-6) 说，稳态跟踪正确的条件是 $\theta_1 + \theta_2 = (a - a_m + b_m)/b$。本章 $a = 1$、$a_m = b_m = 2$，右端等于 $1/b$，在 (θ_1, θ_2) 平面上是一条斜率为 -1 的直线。**这条直线上的每一点都能把稳态误差压到零**，线上不同的位置只影响过渡过程——这就是“组合正确、分量不对”的几何含义。

自适应轨迹从 $\theta_0 = (0, 0)$ 出发，朝这条直线走。走到之后，它只能沿着直线滑动，滑动到哪一点由激励与噪声决定，与跟踪指标无关。三档 γ 的终点可以验证这一点：

γ	$\hat{\theta}$ 终值	和	距真值
2	$(-0.142, +0.860)$	0.718	0.807
20	$(-0.567, +1.281)$	0.714	0.209
100	$(-0.722, +1.436)$	0.714	0.011

三个和分别是 0.718、0.714、0.714——**差异在千分位，全部落在同一条直线上**；而距真值的距离从 0.807 一路降到 0.011，走的全是**沿直线**的那一段。也就是说，γ 只改变“在直线上滑多远”，不改变“离直线多近”。

这个图像直接给出两条工程结论。

第一，投影边界不要做成轴对齐的矩形。 §61.9.2 的 Ω 如果按 $|\theta_i| \le 20$ 取，那么在斜率为 -1 的直线附近，先撞到的往往是某一个分量的边界，撞上之后另一个分量就再也上不去了，等效于把自适应冻结在一个错误的位置上。正确做法是把 Ω 做成“沿直线方向可以很长、垂直直线方向很窄”的带子，比如用 $\theta_1 + \theta_2$ 与 $\theta_1 - \theta_2$ 两个坐标分别设限。

第二，判断“学没学会”要看垂直距离，不是看分量。 现场最常见的误判是：看到 $\hat{\theta}_1 = -0.14$、与标称的 -0.71 差得远，就认为自适应没工作。其实那时的稳态误差可能已经是零——它只是停在了直线上的另一个点（$\gamma = 2$ 那一行）。**唯一有意义的判据是 $\hat{\theta}_1 + \hat{\theta}_2$ 与 $(a - a_m + b_m)/b$ 的差**，这也是 §61.10.2 参数可信度判据的来源。

最后澄清一个容易混的地方：本章 $a = 1$，所以 $(a - a_m + b_m)/b$ 与 $1/b$ 数值相同，前面几节有时直接写成 $1/b$。换一个对象，$a \neq 1$，正确的组合是 a/b——**用 $1/b$ 去核对会得到系统性的偏差**，而且这个偏差会随 a 增大而放大。

61.8 调参方法与经验

61.8.1 一个不能颠倒的顺序

§61.7.7 说过这几个旋钮会互相抵消。避免“调 A 坏 B”的唯一办法是**固定一个顺序，每次只动一个，动完立刻回到上一层复核**。表 61-14 是本章推荐的顺序。

步骤	动作	通过判据	不通过时怎么办
1	$\gamma = 0$，θ 固定为标称值	闭环稳定、无静差	回去查内环，与自适应无关
2	$\gamma = 1 \sim 5$，$\theta_0 = 0$	θ 朝正确方向动	查控制方向符号
3	扫 γ 至失稳（带真实执行器）	找到硬边界	用实测的 τ_a，不要用理想执行器
4	在噪声下扫 γ	找到软边界	噪声水平按现场实测，不要用 0
5	取窗口下沿的 $1/3 \sim 1/2$	跟踪指标达标	不达标先放慢参考模型，别硬加 γ
6	加死区 Δ，从 0 开始加倍试探	θ 抖动下降、误差不涨	停在“误差开始涨”的前一档
7	加 $\sigma \le 0.2$	无慢漂移	还有漂移就查是否有外部常值偏置
8	加饱和处理与 $\lvert\theta\rvert$ 硬限	大指令下 θ 不卷绕	见 §61.9.3
9	标定 θ_0	起始误差下降	标定值不可信就退回 0

第 1 步最容易被跳过，也最不该跳过。$\gamma = 0$ 时自适应律完全不起作用，控制器退化成一个固定的线性控制器 $u = \theta_0^{\mathsf{T}}\phi$。如果这一步就不过关，后面所有调参都是在给一个本身有问题的内环打补丁。

第 3 步有一个常见的偷懒：用理想执行器扫边界。这会把边界高估一大截——本章实测，理想执行器下 $\gamma = 100$ 还能收敛，加了 0.05 s 滞后后 200 就失稳，可用值实际上差了一个数量级。**扫边界必须用真家伙**，或者至少用一个能代表真实滞后的模型。

61.8.2 为什么不要“一次到位”

新手常做的事是：先凭经验把 γ 设成 50，再看到抖动就把死区加到 0.05，再看到静差就把 γ 提到 100。这三步做完，系统“看起来”好了——但三个参数都停在了互相抵消的位置上，任何一个微调都会把另外两个的问题翻出来。

正确的做法是**从最保守的一头往里走**。γ 从 5 开始，一路加到指标达标或出现抖动；出现抖动时先不动 Δ，而是回退 γ 到上一档。只有当“γ 已经退到指标不达标”时，才考虑加死区换一点抖动空间。这个顺序保证每一步的因果是清楚的。

有一个量化的理由支持“往里走”：表 61-9 里 γ 从 10 到 20，IAE 从 0.4435 降到 0.3027（降 32%），TV 从 7.3 涨到 9.8（涨 34%）——**性价比大致是 1:1**；而从 50 到 100，IAE 从 0.1832 降到 0.1265（降 31%），TV 从 14.6 涨到 19.9（涨 36%）——性价比还是一样。既然每一段的交换比都差不多，那就没有理由去冒高 γ 的险：**低 γ 的那一段，同样的代价能换到同样的收益，而失稳风险小得多。**

61.8.3 症状与病因对照

调参现场最耗时的不是“调”，而是“判断现在是什么问题”。表 61-15 是本章实验里实际出现过的若干种症状，以及它们对应的原因。

症状	最可能的原因	先查什么	处理
稳态误差恒定不为零	死区过大，或 σ 过大	Δ、σ	减小，直到误差进入指标
θ 单向缓慢漂到限幅	饱和期间的错误误差在积分	u 是否长期顶限幅	加饱和冻结，见 §61.9.3
控制量持续高频抖动	γ 过大 + 量测噪声	γ、Δ	降 γ，再加死区
阶跃起始段先冲后回	θ_0 远离真值	θ_0、初段 $\lvert u\rvert$	给标定初值
稳态时 θ 缓慢来回摆	γ 逼近稳定边界	未建模动态	降到实测边界的 1/3
参数突变后恢复极慢	γ 小，或死区大	γ、Δ	提高 γ，或减小 Δ
跟踪始终有固定滞后	参考模型比执行能力快	a_m、b_m	放慢参考模型

表里第三行与第五行要分清。**“高频抖动”和“缓慢来回摆”是两种病**：前者是 γ 太大放大了噪声，频率跟着噪声走，降 γ 立刻见效；后者是 γ 已经逼近未建模动态给出的稳定边界，振荡频率由执行器或未建模模态决定，与噪声无关，只降一点 γ 不见效，得降一个数量级。

61.8.4 三条可以带走的经验

把全章的实测压缩成三句话：

第一，γ 的上界靠扫，不靠算。 解析上界（式 61-33 的 $\gamma \ll 114$）只考虑了离散化，没有考虑执行器滞后与噪声。真实上界由这两者决定，只能实测。本章实测硬边界在 100 与 200 之间，可用值 50；软边界在 50 附近，可用值 20 到 30。

第二，γ 的收益是亚线性的，代价是线性的。 式 (61-38) 给出 $t_s \propto \gamma^{-0.27}$，而 TV 大致正比于 γ。一头是 0.27 次方，一头是 1 次方——**这个不对称决定了最优工作点一定在窗口的低端，而不是高端。**

第三，每个鲁棒化措施都要付出静差。 σ 付静差（$\sigma = 0.2$ 时 0.0050），死区付静差（$\Delta = 0.05$ 时 0.0200），θ 硬限幅付“彻底失效”的风险（一旦长期贴在限幅上，控制器就退化成固定增益）。所以这些措施不是“加了更安全”，而是“用确定的小损失换不确定的大损失”——加之前必须先确认那个大损失真的存在。

61.8.5 上线之后看什么

自适应控制器上线后，最该被记录的不是跟踪误差，而是 θ 本身。三条曲线：

1. **$\|\theta\|$ 的长期趋势**。缓慢单调上升或下降，说明有未建模的常值偏置在推它；周期性的往复，说明指令本身有周期性、自适应在追着指令跑，这时候应当加死区。
2. **θ 是否长期贴在限幅上**。贴在 ± 20 上超过几秒，等于自适应已经失效，控制器退化成一个常数增益。这是最危险的失效模式，因为它不报警、输出还"看起来正常"。
3. **控制量 TV 的滚动统计**。TV 的均值突然上台阶，往往意味着对象变了（摩擦增大、增益下降），自适应在更努力地补偿。这是一个很好的早期故障指示。

   第 2 条与 §61.10 的"参数估计可信度"直接相关，下一节会给出判据。

61.8.6 一次完整的调参实录

把九步在本书这组参数上走一遍，看看每一步输入什么、输出什么、被哪张表卡住。场景设定：对象 $a = 1.0$、$b = 1.4$，执行器一阶滞后 0.05 s、限幅 3，量测噪声标准差 0.01。

第 1 步，$\gamma = 0$、$\theta = \theta_0$。 控制器退化成固定增益 $u = -0.7143y + 1.4286r$，闭环极点由 θ 决定，实测稳定、无静差。这一步通过，说明内环本身没问题，后面所有的异常都可以归因于自适应。

第 2 步，$\gamma = 1 \sim 5$、$\theta_0 = 0$。 实验 2 的 $\gamma = 2$ 一行：$t_s = 6.51$ s、超调 28.97%、$\hat{\theta}$ 终值 $(-0.142, +0.860)$。方向和符号都对，θ_1 变负、θ_2 变正，说明自适应律在往正确方向推。这一步通过。

第 3 步，带真实执行器扫硬边界。 表 61-10：$\gamma = 100$ 时尾段峰峰值 1×10^{-4}，$\gamma = 200$ 时 0.3526，边界在两者之间。**同时要记下另一列**：$\gamma = 50$ 的 $|u|_{\max}$ 已经是 5.73，$\gamma = 100$ 是 12.82。这两个数都超过限幅 3——也就是说，即使不考虑稳定性，$\gamma \geq 50$ 也已经把执行器推到饱和区了。

第 4 步，在噪声下扫软边界。 表 61-11：$\gamma = 50$ 时 θ 尾段标准差 0.0261，$\gamma = 100$ 时 0.0723。按"标准差不超过参数典型值的 2%"这条常用标准（$\|\theta^*\| = 1.60$，即 0.032），软边界落在 50 与 100 之间。

第 5 步，取窗口下沿的 $1/3$ 到 $1/2$。 硬边界 150、软边界 50，取小者 50，再折半，得 20 到 25。这里有一个容易被忽略的复核：表 61-10 里 $\gamma = 20$ 的 $|u|_{\max} = 2.69$，$\gamma = 50$ 是 5.73。**限幅 3 这条约束把 γ 又往下压了一格**，最终取 20。

第 6 步，加死区。 噪声 0.01，按"两倍噪声"取 $\Delta = 0.02$。查 §61.7.5 的二维表：$\gamma = 20$、$\Delta = 0.02$ 时 $\hat{\theta}_1$ 标准差 0.0053、尾段误差 rms 0.0011——抖动比无死区时（0.0105）降了一半，误差没有上升。通过。同时复核 $\Delta = 0.02 < 2\%$ 的静差指标，通过。

第 7 步，σ。 现场没有常值偏置，也没有缓慢漂移的迹象，取 0。若日后发现 θ 单向漂移，从 0.05 起试，上限 0.2。

第 8 步，饱和处理。 限幅 3 已知，饱和冻结打开，$|\theta_i| \leq 20$ 硬限保留。这一步在实验 9 里验证过：IAE 不变，$\|\theta\|_{\max}$ 从 28.28 降到 0.46。

第 9 步，θ_0 标定。 出厂标定给出 $a = 1.0$、$b = 1.4$，代入式 (61-5) 得 $\theta_0 = (-0.7143, +1.4286)$。标定值可信，采用。

最终卡片：$\gamma = 20$、$\Delta = 0.02$、$\sigma = 0$、$\theta_0 = (-0.7143, +1.4286)$、限幅 3、冻结开、硬限 20。预期性能：理想执行器下 $t_s = 3.39$ s、超调 7.34%、IAE 0.3027；带 0.05 s 滞后时 $\max|e| = 0.408$。

这张实录里最值得回味的不是最后那组数，而是**第 5 步的复核**：按稳定性与噪声定的 $\gamma = 50$，被限幅这一条从侧面否掉了。三张表（表 61-9、61-10、61-11）单独看都能给出一个“可用”的 γ，合起来才给出唯一可用的那个。**调参的本质不是找到最优值，而是找到所有约束的交集。**

61.8.7 调参记录表怎么写

实录走完，纸面上留下了七个参数。但现场真正需要留下来的东西比这七个参数多得多，而且六个星期之后你会发现，**忘记“为什么取这个值”比忘记“取了多少”要致命**。所以调参过程应当落在一张固定的表上，表头至少六列：

日期/工况	这一轮改了什么	γ	Δ	实测指标（t_s / 超调 / IAE / 静差）	结论（保留 / 否掉 / 待复核）
03-11 空载	首轮，$\theta_0 = 0$	2	0	6.51 / 28.97% / 1.0243 / 0	方向对，太慢
03-11 空载	γ 加倍	5	0	4.62 / 17.20% / 0.6812 / 0	保留作备选
03-11 空载	继续加倍	50	0	$\lvert u\rvert_{\max} = 5.73$	否掉：限幅 3 已饱和
03-12 带载 60%	复核上轮备选	20	0.02	3.39 / 7.34% / 0.3027 / 0.0011	采用

这张表比“最终参数”多提供了三样东西。

第一是路径。 上表里 γ 从 2 走到 20，中间试过 5 和 50，50 是被限幅否掉的。三个月后如果负载增加、执行器换成更大的型号，限幅这条约束松了，你会想把 γ 往上推——这时表告诉你“50 曾经被否掉，但否掉的理由是限幅，不是稳定性”。**一个被否掉的方案，它的否决理由和方案本身一样重要**，因为约束变了，理由就可能失效。

第二是工况。 同一组参数在空载与 60% 带载下的表现可能差一倍，只记“最终参数”等于把一个依赖工况的结论写成了无条件结论。至少要在“日期/工况”这一列写清负载、温度、供电电压三项里会变的那些。

第三是对照。 每次只改一个参数——这是 §61.8.2 已经强调过的原则，而记录表是它唯一的执行手段。一次改两个参数，表里那一行就无法归因，事后既不能说“是这个改动起的作用”，也不能说“不是”。

记录表还有两条容易被省略的写法。其一，**指标必须写口径**。t_s 按 2% 还是 5% 带算、IAE 的积分窗多长、静差取哪一段的均值，这三个口径不写清，两张表之间就没法比（§61.6.8）。其二，**控制量要单列一栏**。多数人只记跟踪指标，而本章反复出现的失效都是从控制量先露出苗头的：$\lvert u\rvert_{\max}$ 顶到限幅、TV 突然上台阶、u 长期贴着同一侧。跟踪误差是结果，控制量是过程，出问题时先坏的是过程。

如果只能记三列，就记“改了什么、γ 是多少、$\lvert u\rvert_{\max}$ 是多少”。这三列能回答现场九成的问题：这套参数是不是已经推到饱和、上次改动有没有生效、以及要不要回退。

61.9 优化与改进

基础版 MRAC 在教科书条件下是完备的，一进现场就露出五个短板：回归向量可能无界、参数可能跑出物理范围、饱和期间参数会卷绕、持续扰动会让参数漂移、参数永远不收敛。这一节给五副补丁，每一副都注明它治什么、代价是什么。

61.9.1 归一化：把步长从信号幅值里解耦

基础律 (61-14) 的更新量是 $\gamma e \phi$，它与 $\|\phi\|$ 成正比。$\phi = (y, r)$，r 是操作员给的指令——也就是说，**操作员把指令从 1 加到 10，参数更新的步长会自动放大 10 倍**。这不是设计意图，是“没做归一化”的副作用。更糟的是，如果 y 因为某种原因发散（比如传感器故障），ϕ 会跟着发散，自适应律会在几拍之内把 θ 推到限幅。

归一化的做法是把更新量除以 $\|\phi\|$ 的某个函数：

$$\dot{\theta} = -\gamma \, \frac{e\,\phi}{1 + \phi^{\mathsf{T}}\phi} \tag{61-41}$$

分母里的 1 不能省。省掉它就变成 $\dot{\theta} = -\gamma e\phi/\|\phi\|^2$，当 $\phi \to 0$（例如系统停在零输出、指令也为零）时分母趋零，更新量反而爆炸。加 1 之后，ϕ 小的时候行为与基础律几乎一样，ϕ 大的时候更新量被封顶在 $\gamma e/\|\phi\|$，随 $\|\phi\|$ 反比衰减。

归一化改变了 γ 的量纲，也改变了它的可用数值。基础律里 $\gamma = 20$ 对应本章的窗口；归一化之后因为分母恒大于 1，同样的动态需要更大的 γ，量级大致是 $\gamma_{\text{norm}} \approx \gamma(1 + \|\phi\|^2) \approx 3\gamma$。**换公式必须重扫 γ**，这是移植时最常见的错误。

61.9.2 投影：把参数关进一个盒子

归一化管住了步长，管不住累积。如果存在持续的未建模偏置，θ 会一直朝一个方向走，直到顶到硬限幅 $|\theta_i| \le 20$——那时控制量已经毫无意义。

投影算子的思路是：**先用先验知识圈出一个“物理上可能”的凸集 Ω，然后规定 θ 只能在这个集合里活动**。

$$\Omega = \left\{ \theta : \; \theta_i^{\min} \le \theta_i \le \theta_i^{\max},\; i = 1, 2 \right\} \tag{61-42}$$

更新时做一次判断：如果 θ 已经在边界上，而更新方向指向集合外，就把更新量投影到边界的切平面上（即只保留沿边界的分量）。

投影有两个好处。第一，**它不破坏 Lyapunov 证明**：只要真值 $\theta^* \in \Omega$，投影满足 $\tilde{\theta}^{\mathsf{T}}\mathrm{Proj}(\tau) \le \tilde{\theta}^{\mathsf{T}}\tau$，式 (61-15) 的负定性原样保留。第二，它把“参数漂移到荒谬值”这种失效模式**从物理上排除**，而不是靠调小 γ 去避免。

代价是先验知识。Ω 的边界要从“b 的物理范围”倒推：如果执行器增益的变化范围是 $\pm 50\%$，那么 $\theta_2^* = b_m/b$ 的变化范围就是 $[0.95, 2.86]$，Ω 的第二个分量就该按这个取。给宽了没用，给窄了会把真值排除在外——**真值一旦落在 Ω 外，投影就变成了一个持续的单向推力，闭环必然留静差**。

61.9.3 饱和冻结：本章最划算的一副补丁

执行器饱和是自适应控制最经典的陷阱。逻辑很直白：u 被限幅在 2，对象只能走到 2.8（因为 $y_{ss} = bu/a = 1.4 \times 2 = 2.8$），而指令是 5，于是误差 e 永久停在 -2.2。自适应律看到这个 -2.2 的误差，认为“参数还不够大”，继续往上推 θ，可 θ 再大也没用——u 已经被限幅卡死了。**误差是饱和造成的，不是参数不够造成的，但自适应律分不清这两件事**。

实验 9 把这个过程完整地跑了一遍：指令幅值 5、限幅 2、$\gamma = 20$。

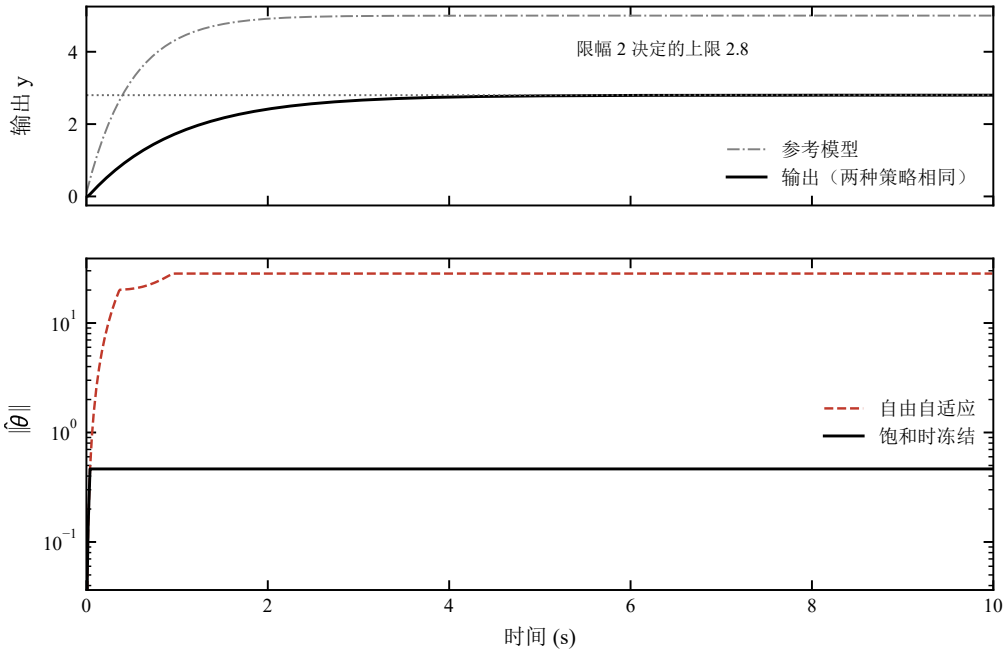


图 61-9　执行器饱和下的参数卷绕：自由自适应与饱和冻结

分支	末端 y	末端 e	$\hat{\theta}$ 终值	$\|\theta\|_{\max}$	IAE
不限幅（对照）	5.000	0.000	$(-0.756, +1.471)$	2.56	0.393
限幅 2，自由自适应	2.800	-2.200	$(+20.000, +20.000)$	28.28	22.402
限幅 2，饱和冻结	2.800	-2.200	$(+0.002, +0.465)$	0.46	22.402

三行数据，三个结论。

第一行是基线：不限幅时，输出准确到 5.000，θ 收敛到 $(-0.756, +1.471)$，和 0.715，与 $1/b = 0.714$ 吻合。IAE 只有 0.393。

第二行是灾难：限幅之后输出卡在 2.800，末端误差 -2.200。θ 被推到 $(+20.000, +20.000)$——两个分量**同时顶到硬限幅**，范数 28.28。注意 $\hat{\theta}_1$ 从负变正，物理上意味着“对象的时间常数变成了负数”，也就是控制器认为这个对象是开环不稳定的。这个判断完全错误，但自适应律没有别的选择：误差是负的、y 是正的，按式 (61-17) 它只能把 θ_1 往正方向推。

第三行是补丁的效果：加“饱和时冻结自适应”之后，θ 只走到 $(+0.002, +0.465)$，范数 0.46，完全在合理范围内。而 IAE 是 22.402——**与第二行一字不差**。

这个“一字不差”是全章最有价值的一个观测。它说明：**在饱和期间，自适应完全不产生任何跟踪收益，只产生参数漂移。** 冻结它是零成本的——不牺牲一点性能，只消掉一个隐患。如果本章只留一条工程建议，就是这一条。

冻结的实现只有一行：

```
if abs(u_raw) >= umax:
    drive[:] = 0.0
```

判据要用**未限幅的** u_{raw} 而不是限幅后的 u，否则永远检测不到饱和——限幅后的值天生不超过 $u_{\max}$。

61.9.4 σ 修正与 e 修正

σ 修正（式 61-39）是给积分器加泄漏，代价是静差。另一条路是 e 修正：把泄漏项乘以 $|e|$，

$$\dot{\theta} = -\gamma e \phi - \nu |e| \theta \tag{61-43}$$

它的巧妙之处在于：**当 $e \to 0$ 时，泄漏项自动消失**。所以它不会像 σ 修正那样留下稳态误差——$e = 0$ 时式 (61-43) 退化成纯粹的基础律，静差为零。

那它靠什么保证有界？靠“e 大的时候漏得凶”。当扰动把 e 顶起来时，泄漏项与 $|e|$ 同阶增长，把 θ 拉回来；e 小的时候它就不管了。这正好符合“只在出问题时干预”的设计哲学。

代价是它给不出零静差的理论保证，只能给出“误差最终有界，界与 ν 成反比”这种较弱的结论。工程上这个差别通常可以接受——因为真正的限制往往来自噪声与未建模动态，而不是理论上的渐近性。

两副补丁怎么选：**确定有常值偏置 → 用 σ（接受静差，换取彻底的抗漂移）；不确定扰动形态 → 用 e 修正（保住零静差，接受有界性而非收敛性）**。两者不要叠加使用，叠加会把静差与抖动都放大一遍。

61.9.5 复合自适应：让参数真的收敛

前面所有讨论都绕不开一个事实：**MRAC 只保证误差收敛，不保证参数收敛**（式 61-16 之后已经说明）。想同时得到两者，需要“持续激励”（PE）条件——即回归向量要在所有方向上都被充分激发。在跟踪任务里，PE 条件等价于“指令要足够丰富”，而操作员给的指令往往就是一个阶跃，谈不上丰富。

复合自适应（composite adaptation）绕开了这个要求。做法是把历史数据存进一个“数据栈”：

$$\mathcal{D} = \left\{ (\phi_j,\ \dot{y}_j) \right\}_{j=1}^{N} \tag{61-44}$$

每一项记录了某个时刻的回归向量与实测的（或用滤波器重构的）$\dot{y}$。然后在自适应律里加一项，让它同时拟合这些历史数据：

$$\dot{\theta} = -\gamma e \phi - \gamma \sum_{j \in \mathcal{D}} \phi_j \left(\dot{y}_j - \phi_j^{\mathsf{T}} \theta \right) \tag{61-45}$$

只要数据栈里的 $\{\phi_j\}$ 线性无关（秩条件），第二项就会把 θ 推向唯一满足所有历史数据的点——**不需要当前指令持续激励，因为激励已经被“存起来”了**。

这条路在本章不展开，因为它需要处理微分重构、数据栈的替换策略、秩条件的在线判定，已经不属入门范围。但要记住它的存在：**“MRAC 不能辨识参数”不是物理定律，只是“只用当前一拍数据”这个设计的后果**。把数据存起来，就能突破。

61.9.6 五副补丁的取舍

表 61-16 是汇总。

补丁	治什么	代价	什么时候必须加
归一化	指令大幅变化时步长失控	需重扫 γ	指令幅值变化超过一个数量级
投影	参数漂出物理范围	需要先验边界	有可靠的参数范围知识
饱和冻结	饱和期间参数卷绕	无	**只要执行器有限幅就必须加**
e 修正	有界扰动下的参数漂移	破坏零静差	有未知形态的持续扰动
复合自适应	参数不收敛	实现复杂、需数据栈	需要参数用于诊断或前馈

只有第三行的“代价”是“无”，也只有它在任何场合都该加。其余四副都要看条件——**补丁不是越多越安全，加错了照样出问题**。

61.9.7 自适应律不必每拍都算

前面六节都默认自适应律与控制回路同节拍运行。这不是必须的。参数动态本来就比对象动态慢一个数量级（§61.1.6 的“时间尺度分离”），把自适应律降采样到每 N 拍算一次，在原理上完全成立，还能省下大量算力。

具体有两种写法，效果完全不同，必须分清：

写法一，保留 T_s 作为步长。 每 N 拍执行一次 $\theta \leftarrow \theta - \gamma T_s e\phi$。此时每拍的平均更新量是原来的 $1/N$，**等效增益变成 γ/N**。想让动态不变，就要把 γ 放大 N 倍。

写法二，用更新间隔 $T_u = NT_s$ 作为步长。 $\theta \leftarrow \theta - \gamma T_u e\phi$。此时每 N 拍的总移动量与“每拍更新”完全相同，平均学习速率不变，但单次的移动量大了 N 倍——式 (61-33) 的步长约束相应收紧 N 倍，$\gamma \ll 1.14/T_u$。本章 $N = 10$ 时上界从 114 掉到 11.4，正好卡在可用窗口下沿，不可接受。

所以工程上该用的是写法一：**降采样的同时把 γ 乘回去**，保持等效学习速率不变。本章 $\gamma = 20$、$N = 10$，实际写进代码的增益是 200，但每 10 拍才更新一次。

代价是延迟。参数突变的恢复时间会从 0.83 s 增加大约 $NT_s = 0.1$ s，相对 0.83 s 只多 12%，可以接受。但如果 N 取到 100，延迟就是 1 s，比恢复时间本身还长——那时不是“慢一点”，而是“追不上”。经验上 N 的上限是“更新周期不超过参考模型时间常数的 1/5”，本章即 $N \le 10$。

降采样真正的价值在慢速控制器上。一个扫描周期 100 ms 的 PLC 跑不了 10 ms 的自适应，但它完全跑得了 1 s 一次的自适应——这时候写法一让同一个算法在两个数量级不同的硬件上都能落地，代价只是参数突变的响应慢了零点几秒。

61.9.8 补丁的叠加顺序

五副补丁不能随手堆。它们改动的是同一个自适应律的不同部分，顺序错了会出现“加了一层、前一层的参数全部作废”的局面。可用的顺序是：

1. **硬限幅**。无论什么场合都先加。它不改变算法行为，只在数值失控时兜底，所以放在最前面不会干扰后面任何一步的调参。
2. **饱和冻结**。只要有执行器限幅就加。它只修改“什么时候不更新”，不影响更新量的大小。
3. **归一化**。如果指令幅值会跨一个数量级以上，就必须在**扫 γ 之前**加上。理由在 §61.9.1：归一化把 γ 的数值放大了约三倍，先扫后加等于白扫一遍。
4. **死区**。等 γ 定下来再加。§61.7.5 的二维表说明死区的代价依赖 γ，γ 没定，Δ 就没有依据。

5. σ 或 e 修正。最后加，而且要先确认“确实存在不落在回归张成空间里的扰动”。没有证据就加，只会平白引入静差。
6. 投影。有可靠的参数范围时才加，且边界按 §61.7.8 的几何取成斜带而不是方盒。

这个顺序背后的原则很简单：**先加不影响参数整定的（1、2），再加会改变参数标度的（3），最后加需要用参数去定值的（4、5、6）。**

如果条件只允许加三副，选硬限幅、饱和冻结、死区。三者合起来防住了自适应最致命的两类失效——数值失控与参数卷绕——以及最常见的现场投诉（执行器抖动），代价只有一点静差。**投影与 σ 属于“有明确证据才加”的补丁，不是默认配置。**

61.9.9 归一化之后，γ 的刻度变了

上一节第 3 条说“归一化把 γ 的数值放大了约三倍”，这句话值得单独摊开，因为它是最容易在换实现时踩空的地方。

分母是 $1+\phi^{\mathsf{T}}\phi$。本章 $\|\phi\|$ 的典型值 1.4，代入得 $1+1.96=2.96$，接近 3——这就是“三倍”的来源。也就是说，**归一化版本要填的 γ 约等于未归一化版本的 3 倍，才能得到相同的等效学习速率**。本章未归一化时取 $\gamma=20$，换成归一化实现，起点应当从 60 附近开始扫，而不是继续用 20。用 20 去跑归一化版本，会觉得“加了补丁之后变迟钝了”，其实是刻度没换算。

但分母不是常数，它是**随信号走的**。取 $\gamma=20$ 时，等效增益 $20/(1+\|\phi\|^2)$ 随 $\|\phi\|$ 的变化是：

$\|\phi\|$	0.5	1.0	1.4	2.0	5.0
$1+\|\phi\|^2$	1.25	2.00	2.96	5.00	26.0
等效增益	16.0	10.0	6.76	4.00	0.77

这张表（纯算术，与仿真无关）说明三件事。

第一，归一化把“一个增益”变成了“一族增益”。 小信号时几乎不衰减（$\|\phi\|=0.5$ 时只降到 16.0），大信号时衰减二十多倍（$\|\phi\|=5$ 时只剩 0.77）。所以归一化后的 γ 不能再被理解成“每秒多少”的速率，它只是分子上的一个标度。

第二，大指令段学得慢，而大指令段恰恰是信息最丰富的暂态。 §61.2.7 说过，参数能不能被“抓住”，靠的是 y 从 0 升到 r、ϕ 方向在转的那段扫弧；而这段弧正好落在 $\|\phi\|$ 最大的区间，也正好是归一化压得最狠的区间。**归一化是用参数收敛速度换步长与信号幅值的解耦**，这笔交换在指令幅值跨数量级的场合（比如 0.1 到 10）是划算的——不换的话，小信号学不动、大信号一步冲顶；但在指令幅值本来就很稳定的场合，它是纯损失，不该加。

第三，它换来了一个可以事先算出的单拍上界。 因为 $\|\phi\|/(1+\|\phi\|^2)\le 1/2$，所以

$$\|\dot\theta\|\le\frac{\gamma}{2}$$

离散化后单拍变化量不超过 $\gamma T_s/2$。本章若取 $\gamma=60$、$T_s=0.01$，就是每拍 0.30——这个数与信号幅值无关，传感器故障导致 y 跳变时，θ 也不会在几拍内冲到限幅。这是归一化真正的价值：**它把“最坏情况的更新量”从“取决于信号”变成“取决于设计者填的数”。**

最后提醒一句：归一化不改变稳定性结论。Lyapunov 证明里用到的只是 $\dot V\le -a_m e^2$ 与各量的有界性，归一化让更新量有界，只会让证明更容易成立，不会让它失效。**它改变的是刻度与响应速度，不是稳定域的形状**

——所以 §61.6 的边界该重扫还是要重扫。

61.10 工程落地

把 MRAC 装进一台真实设备，技术难点不在推导，而在三件事：**参数会漂、参数不可信、失效不报警**。这一节逐个说，最后给一个可以直接照抄的实现骨架。

61.10.1 漂移从哪里来

先分类。参数漂移有四个来源，处理手段完全不同：

来源	机理	表现	对策
常值偏置	未建模的负载力矩、传感器零点	θ 单向缓慢移动	σ 或 e 修正
执行器饱和	饱和期间误差不降，积分继续累积	θ 快速冲到限幅	饱和冻结（§61.9.3）
量化与死区非线性	小信号时的等效偏置	θ 小步跳动	死区 Δ
未建模高频	高频误差整流成直流分量	θ 缓慢单向漂	降 γ、加低通

第三类值得单独说一句。量化误差本身是零均值的，按理说不该造成漂移；但它经过“误差 × 回归向量”这个乘积之后，与 y 的相关性不再为零，于是产生了一个小的直流分量。这个分量很小，但它是积分器的输入，会一直累积。**自适应律对任何“与回归向量相关的小偏置”都没有免疫力**，这是它作为积分器的本性。

第四类更隐蔽。对象里若有一个没建模的谐振模态，它在闭环里会被激励出持续的高频振荡；这个振荡经过 $e\phi$ 的乘积，在低频上留下一个非零均值——这就是“整流”。此时 θ 的漂移方向与谐振的相位有关，看起来毫无规律。对策只有一个：把 γ 降下来，让自适应律的带宽低于谐振频率。

61.10.2 参数什么时候可信

§61.1.3 已经给出结论：MRAC 只辨识组合，不辨识分量。工程上要更进一步——**给“可信度”一个可以实时算的判据**。

三条判据，从弱到强：

1. **激励是否充分**。看回归矩阵的条件数，或者更简单地看 ϕ 在最近一段时间里是否扫过多个方向。本章的阶跃指令只给出一个方向段，所以分量永远不准；PRBS 指令扫过四个方向，STR 才能把 $\hat{b}_d$ 估到四位有效数字。**没有激励的时候，任何参数估计都不可信，这不是算法问题，是信息论问题。**
2. **参数是否已经停住**。取 θ 的尾段标准差 σ_θ，与 $\|\theta\|$ 相比。本章噪声 0.01 下，$\gamma = 10$ 时 $\sigma_\theta = 0.0093$、$\|\theta\| \approx 1.6$，比值 0.6%；$\gamma = 100$ 时 $\sigma_\theta = 0.0723$，比值 4.5%。
3. **漂移速率**。取 θ 在最近 10 s 内的均值变化量。缓慢单向变化说明有偏置在推，此时当前值不可信，应当用历史均值。

图 61-10 是判据 2 的直观版本：噪声 0.01 下三档死区的 $\hat{\theta}_2$ 轨迹。无死区时曲线是一条毛刺带，带宽就是 σ_θ；$\Delta = 0.02$ 时毛刺减半；$\Delta = 0.05$ 时曲线完全拉平——自适应已经停住，此时读到的值稳定，但它是“停住的值”，不是“正确的值”。

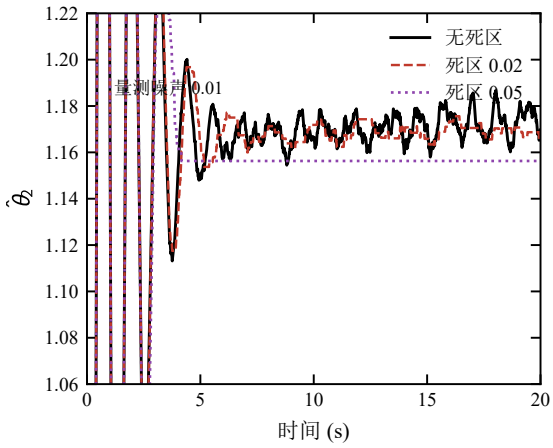


图 61-10　量测噪声下的参数抖动：死区对可信度的影响

三条判据合起来给一个实用规则：

$$\text{可信} \iff \Big(\text{激励充分}\Big) \wedge \left(\frac{\sigma_\theta}{\|\theta\|} < 1\%\right) \wedge \Big(\text{10 s 内均值漂移} < 1\%\Big) \tag{61-46}$$

三个条件都满足，才可以把 θ 拿去算物理参数、做故障诊断、或者写进报表。少一个都不行——特别是第一个，它是最容易被忽略、也最致命的。

反过来说，如果只需要跟踪，根本不必关心这三条。 跟踪性能由 e 直接给出，θ 只是手段。§61.4.3 里 $\gamma = 2$ 的 $\hat{\theta}_2$ 只有真值的 60%，但跟踪误差照样收敛到零。这个区分在工程上很重要：**不要把“控制器工作正常”误读成“参数估得准”。**

61.10.3 三层安全边界

自适应控制器比固定控制器多了一个“内部状态”——θ。这个状态会漂、会卷、会失效，所以安全设计要比固定控制器多做三层。

第一层：θ 的硬限幅。 这一层不参与控制逻辑，只在数值上兜底。本章取 $|\theta_i| \le 20$，是从“θ_2^* 在 b 变化 $\pm 90\%$ 时最大约 14”倒推的。硬限幅的意义不是“限制性能”，而是“保证程序不会算出 `inf` 或 `nan`”。实验 9 里，正是这一层把自由自适应的 θ 拦在了 20 而不是无穷大。

第二层：控制量限幅 + 饱和冻结。 这一层有明确的理论依据（§61.9.3 的“零成本”结论），实现也简单。注意判据要用未限幅的 u_{raw}。

第三层：看门狗。 监控两个量：$|e|$ 是否长期超过阈值，θ 是否长期贴在限幅上。任一条件满足超过设定时间（本章建议 2 s），就执行“冻结自适应 + 报警”。冻结时**保持最后一份可信快照**，而不是当前值——如果当前 θ 已经贴在限幅上，用它当固定增益会立刻出问题。

三层的关系是递进的：第一层保数值，第二层保稳定，第三层保可诊断。少了任何一层，失效模式就会从“可控降级”变成“不报警的失控”。

61.10.4 回退策略

自适应最尴尬的时刻是“它坏了，但你不知道”。所以回退必须是**设计的一部分**，而不是事后补的补丁。

回退的做法是：控制器内部同时维护两份参数——**在线学习的 θ** 和**冻结的快照 θ_{safe}**。正常工作用 θ；看门狗触发后切到 θ_{safe}，并停止自适应。θ_{safe} 每 10 s 用当前 θ 覆盖一次，但只在三条可信判据都满足时才覆盖。

这个设计的价值在于：**它把“自适应失效”这个不可控事件，转化成了“退化为固定增益控制器”这个可控事件**。退化的性能是已知的（就是 §61.8 第 1 步测得的那个性能），不会突然发散，也不会输出荒谬的控制量。

还有一条容易被忽略的回退：**上电时的第一秒**。此时 $\theta = \theta_0$，如果 $\theta_0 = 0$，控制器几乎不输出，对象可能因为重力、摩擦而移动。工程上通常让 θ_0 取标定值，或者在起始的 0.5 s 内把自适应增益压到很小，等系统稳定再放开。

61.10.5 实现骨架

把上面的三层边界与回退压成伪代码，整个控制器的核心不到 40 行：

```
THETA_MAX = 20.0
SNAP_PERIOD = 10.0

def step(t, y, r, ym):
    global theta, theta_safe, t_snap, t_bad

    phi = np.array([y, r])
    e = y - ym
    u_raw = float(theta @ phi)

    # --- 第一层：数值兜底由 theta 的 clip 保证 ---
    sat = abs(u_raw) >= UMAX
    u = float(np.clip(u_raw, -UMAX, UMAX))

    # --- 第二层：饱和冻结 ---
    drive = np.zeros(2) if sat else -GAMMA * e * phi

    # --- 第三层：看门狗 ---
    bad = abs(e) > E_MAX or np.max(np.abs(theta)) >= 0.99 * THETA_MAX
    t_bad = t_bad + TS if bad else 0.0
    if t_bad > 2.0:                      # 连续 2 s 异常 -> 回退
        theta = theta_safe.copy()
        drive[:] = 0.0

    theta = np.clip(theta + drive * TS, -THETA_MAX, THETA_MAX)

    # --- 可信时定期刷新快照 ---
    if t - t_snap > SNAP_PERIOD and np.std(tail_theta) < 0.01 * np.linalg.norm(theta):
        theta_safe = theta.copy()
        t_snap = t

    return u
```

这段代码里没有一行是“高级技巧”，全部是前面几节结论的直接落地。真正需要设计判断的地方只有三个数值：`UMAX`（执行器真实限幅）、`E_MAX`（看门狗阈值，建议取允许的最大跟踪误差）、`SNAP_PERIOD`（快照周期）。

61.10.6 与相邻章的分工

自适应控制常被问“能不能替代 PID”。答案是分工，不是替代：

- **PID（第 44 章、第 106 章）**：不需要模型、不需要辨识、鲁棒性好，但对参数变化的适应是“被动”的——靠裕度吃下去。工况变化小于 30% 时，PID 加一点增益调度通常比自适应更省事、更可靠。
- **增益调度**：本质是“自适应”的查表版本。如果参数变化可以由一个可测变量（温度、负载）解释，查表比在线学习稳定得多。**能用调度解决的，不要用自适应。**
- **MPC（第 58 章、第 80 章）**：显式处理约束，是饱和工况的正解。自适应只能“冻结”以避免卷绕，MPC 能“预测”从而不进入饱和。
- **Smith 预估（第 47 章、第 107 章）**：处理纯延迟。自适应处理不了纯延迟——它是非参数化不确定性，会把 §61.6.3 的稳定边界直接压到零。
- **滑模（第 59 章）**：用高增益切换吃掉不确定性，不需要辨识；代价是抖振。自适应是“用辨识换平滑”，滑模是“用抖振换免辨识”，两者是互补的两条路。

一句话：**自适应适合“结构清楚、参数会变、变化幅度中等、执行器有余量”的场合**。这个条件的每一条都不能少，§61.11 会逐条展开。

61.10.7 从仿真到现场必须重算的四件事

本章全部数字来自一个刻意简化的对象：一阶、常值参数、白噪声、一阶执行器。真实设备不会有这么干净的条件。上线之前有四件事必须用现场数据重算，不能照搬。

第一，执行器滞后 τ_a。 仿真取 0.05 s，伺服阀与液压缸常在 0.05 到 0.2 s 之间，带长管路的还要更慢。测法很简单：给一次小幅阶跃，录下 u 与 y，把 y 的起始段拟合成一阶响应，斜率最大的切线与稳态值的交点就是 τ_a。**这个数直接决定 γ 的上界**，本章的表 61-10 说明边界就在 100 与 200 之间；τ_a 翻倍，边界大致往小走一半。用仿真的 0.05 s 去调一台 0.15 s 的设备，等于从一开始就在边界之外。

第二，噪声的幅值与谱形。 实验室的白噪声 0.01 是理想化的。现场噪声往往含 50 Hz 工频串扰、量化台阶和机械振动，幅值可能大一个数量级，而且不是白的——有色噪声在低频段的能量更高，更容易被自适应律当成“参数变化”学进去。正确做法是录一段稳态数据，算出标准差与功率谱，把主频成分先滤波掉，再决定 Δ。**先滤波再加死区，比只加死区有效得多。**

第三，实际的一拍延迟。 采样、通信、执行器刷新三者叠加，环路里的等效延迟常常是 2 到 3 拍，而不是 1 拍。§61.5.3 算过，0.02 rad 的相位侵蚀可以忽略，0.2 rad 就吃掉全部裕度。如果控制器与执行器之间隔着总线，这个数要按通信周期算，不能按采样周期算。

第四，参数的变化范围。 这决定了 θ^* 的活动范围，进而决定三件事：θ_0 的标定值、投影边界 Ω、以及控制量限幅。本章 b 从 1.4 变到 2.1（+50%），θ_2^* 从 1.43 降到 0.95；如果工况会让 b 掉到 0.7（−50%），θ_2^* 就要涨到 2.86，峰值控制量跟着翻倍——这时候限幅 3 就不够用了。

把这四件事写成一张对照表贴在调试现场，比任何“调参口诀”都实用：**仿真里的每一个数字，在现场都有一个对应物，而它们几乎从不相等。**

61.10.8 一次现场事故的复盘

下面的过程来自本章实验 9 的第二行——限幅 2、指令幅值 5、自由自适应、$\gamma = 20$——但按现场的顺序讲一遍，因为**事故的价值不在数据，在排查的次序**。

现象。 一台流量回路，正常工况设定值在 0.8 到 1.2 之间，调试期间表现良好。某天工艺要求把设定值一次性拉到 5.0。阶跃之后的前两秒，输出跟得还算像样，之后开始“发木”：设定值加到 5，实际只到 2.8，怎么等都不再

上升。操作员把设定值退回 1.0，输出反而冲过头到 1.6 左右，然后慢慢往下蹭。**同一套控制器，小信号还行，大信号就坏，退回小信号也不恢复**——这三条合在一起，基本可以排除“增益调得不对”这种解释。

录波。 调出三张曲线，横轴都是阶跃后的时间。

第一张是控制量 u。它在阶跃后约 0.1 s 内顶到限幅 2.0，然后**一动不动地贴了 3.1 s**，直到误差缩小才松开。

第二张是 $\hat{\theta}$ 的两个分量。这是关键：饱和的那 3.1 s 里，两个分量**同向、单调、几乎等速地往上爬**，从初值一路爬到硬限幅 20，之后被 `THETA_MAX` 卡住不动。终值是 $(+20.000, +20.000)$。

第三张是误差 $e = y - y_m$。饱和期间它恒为负，幅值 2.2 左右——因为参考模型按 0.5 s 时间常数冲到 5，对象却只能到 2.8，这个差是**执行器给不出来**造成的，不是参数不对造成的。

定位。 三条线索指向同一个结论。

先排除噪声：噪声只会让 θ 抖，不会让它单调爬升 3 s。再排除对象变化：如果是 b 真的掉了，θ_2 应当往大调、θ_1 往负调，两个分量的方向应当相反（表 61-5 里 $\gamma = 100$ 的收敛轨迹就是这种“一正一负”）；而录波里两个分量同号同上，说明**误差在把两个参数往同一个方向推**，这只可能发生在回归向量 $\phi = (y, r)$ 两个分量同号的时候，也就是大指令段。

真正的机理是 §61.9.3 讲的那条：饱和期间 u 被卡死，误差里混进了一个**与控制能力有关的常量**（这里是 -2.2），自适应律把这个常量当成“参数还差得远”，于是以 $\gamma \cdot 2.2 \cdot \|\phi\|$ 的速率继续积分。3.1 s 之后饱和解除，θ 已经跑到 20 附近，闭环增益被彻底带偏——这就是“退回小信号也不恢复”的原因：**参数没有自恢复能力**。

用实验 9 复现，数字完全对上：末端 $y = 2.800$、$e = -2.200$、$\hat{\theta} = (+20.000, +20.000)$、$\|\theta\|_{\max} = 28.28$、IAE 22.402。同一条件下打开饱和冻结，末端输出与 IAE **一模一样**，只有 $\hat{\theta}$ 停在 $(+0.002, +0.465)$、$\|\theta\|_{\max}$ 降到 0.46。**跟踪性能没有损失一个数字，漂移被完全消掉。**

补丁。 按代价从低到高排：

1. **饱和冻结**。判据必须用**未限幅**的 $u_{\text{raw}} = \theta^{\mathsf{T}}\phi$，不能用已经限幅的 u——否则饱和期间 $|u|$ 恒等于限幅值，判据永远成立，自适应被永久关闭。这是实现上最容易写错的一行。
2. **θ_0 重标定到大流量工况**。本章的 θ_0 是空载标定值，大流量下 b 会变小，θ_2^* 应当更大。用大流量下的实测数据反算一次，能把饱和的持续时间压短。
3. **限幅放宽或加大执行器**。如果工艺真的需要 5.0 的流量，而限幅 2 只能给到 2.8，那么这不是控制问题，是选型问题。**控制系统修不好一个选小的阀门。**

复盘。 有四条可以带走。

第一，**大阶跃必须单独测**。调试时的阶跃都是 0.2、0.5 这种小幅，永远不会触发饱和；而现场的操作员不会按调试大纲动作。测试用例里至少要有一条“指令从最小拉到最大”。

第二，**饱和判据是最容易写错的地方**，写错了不会报错，只会让控制器在大信号下悄悄退化。上线前要专门做一次“强制饱和”测试：临时把限幅压到 0.5，看 θ 是否停止更新。

第三，**$|\theta_i| \le 20$ 这道硬限幅救了这次事故**。没有它，θ 会在这 3.1 s 里爬到几百甚至更大，退回小信号后闭环会直接发散，而不是“冲过头 0.6 再慢慢蹭回来”。§61.2.10 说过这道限幅是让定理成立的前提，这里再补一句：**它同时也是让事故停留在“性能下降”而不是“设备飞车”的那道闸。**

第四，事故后的第一个动作是**回退到 θ_0 并复现**，不是继续调 γ。参数被带偏之后，任何基于当前状态的调参都是在错误的基础上叠加——这也是 §61.10.4 把“回退”列为独立一节的原因。

61.11 优缺点与适用边界

61.11.1 四条优点

一、参数大范围变化时仍能保持性能。 这是自适应唯一不可替代的能力。本章 b 变化 50% 时，跟踪误差峰值只有 0.051、0.83 s 恢复（§61.6.5）；作为对照，固定 PI 在方波工况下的 rms 误差是自适应版本的 2.1 倍（0.1605 对 0.0752）。如果工况变化达到 2 倍、3 倍，固定增益要么按最坏情况调得很慢，要么在最好情况下过冲，两者都不可接受。

二、跟踪性能可以"指定"。 参考模型是设计者写的，不是对象决定的。想要 0.5 s 的时间常数就写 $a_m = 2$，想要 1 s 就写 $a_m = 1$，只要执行器撑得住。这个自由度是 PID 没有的——PID 的闭环动态由对象和增益共同决定，想改动态就得重新调参。

三、对常值扰动自带积分作用。 式 (61-10) 里 d 与 $\theta_2 r$ 同构，自适应会把它吸收进参数。实验 7 里 $\sigma = 0$、$d = 0.3$ 时稳态误差精确为 0.0000，而控制器里并没有显式的积分器。

四、控制器结构固定，参数在线变。 这意味着代码量不随工况变化，测试用例可以覆盖，认证路径清楚——这在功能安全要求高的场合（汽车、医疗、航空）比"根据工况切换控制器"的方案好审得多。

61.11.2 五条缺点

一、稳定性边界靠扫，不靠算。 式 (61-33) 给出的解析上界只考虑离散化，真实边界由执行器滞后与噪声决定，两者都要求实测。本章实测硬边界在 100 与 200 之间，可用值 50；换一台设备就要重扫一遍。**这使自适应的调试成本显著高于 PID。**

二、控制量抖动。 为换取精度，TV 从 0.7 涨到 19.9（γ 从"参数已知"到 100），方波工况下是固定 PI 的 1.79 倍。抖动会被执行器、传动链、结构放大成噪声与磨损。对静音要求高的场合（家电、办公设备），这一点往往一票否决。

三、参数不可解释。 §61.4.3 与 §61.10.2 反复说明：分量不可信，只有组合可信。这让自适应无法直接用于故障诊断或前馈补偿，除非额外做 STR 或复合自适应。

四、对非参数化不确定性无能为力。 纯延迟、谐振、死区、滞环，这些都不是"参数不对"，而是"结构不对"。本章的 0.05 s 执行器滞后已经把 γ 的可用值压掉一半；如果滞后是 0.2 s，边界会压到个位数，自适应基本失去意义。

五、失效不报警。 θ 贴在限幅上时，输出可能还"看起来正常"，但控制器已经退化。这是 §61.10.3 三层安全边界存在的理由，也是自适应在关键场合推广缓慢的主要原因。

61.11.3 五个适用条件

把优缺点折成五个可以逐条核对的条件（表 61-17）。**五条全满足才建议用自适应**，缺一条就该考虑别的方案。

条件	判据	不满足时改用
结构清楚	阶次、控制方向、非线性形状已知	滑模（第 59 章）或鲁棒控制
参数会变且变化大	变化幅度 > 30%	PID + 增益调度（第 44/106 章）
变化可被误差看见	参数变化会体现在跟踪误差上	前馈补偿或扰动观测器
有足够执行器余量	峰值控制量不超过限幅的 50%	MPC（第 58/80 章）
采样与算力够	T_s 内能完成一次更新	降采样或改查表

第一条“结构清楚”是最硬的。本章全部推导都建立在式 (61-1) 成立的基础上；如果对象的阶次都不确定，Lyapunov 函数根本写不出来。这种情况下滑模更合适——它不要求结构，只要求不确定性有界。

第三条容易被忽略。“参数变化可被误差看见”意味着参数必须影响跟踪。如果某个参数只影响执行器的内部状态、不影响输出，那么无论它怎么变，误差都看不见它，自适应也就学不到它。**看不见的东西不能自适应，只能测。**

61.11.4 明确不适用的四类场合

- **纯延迟主导的对象**：延迟是非参数化不确定性，且相位滞后随频率线性增长，会直接把稳定边界压到零。这类对象应当用 Smith 预估（第 47/107 章）。
- **强非线性、工作点大范围迁移**：如果对象在 10% 和 90% 工况下的“结构”都不一样（比如阀的死区在小开度时显著），参数化假设不成立。反馈线性化（第 60 章）或 MPC 更合适。
- **执行器已经饱和成为常态**：饱和工况下自适应只能冻结，不能改善；显式处理约束才是正解。这与 §61.10.6 的分工一致。
- **需要参数用于诊断的场合**：如果目的是“监测设备健康”，应当用 STR 或专门的辨识器，而不是从 MRAC 的 θ 里反推。

61.11.5 决策流程

图 61-11 把上面五个条件画成一张流程图。从“模型结构是否已知”开始，逐层判断，最后落到具体方法。

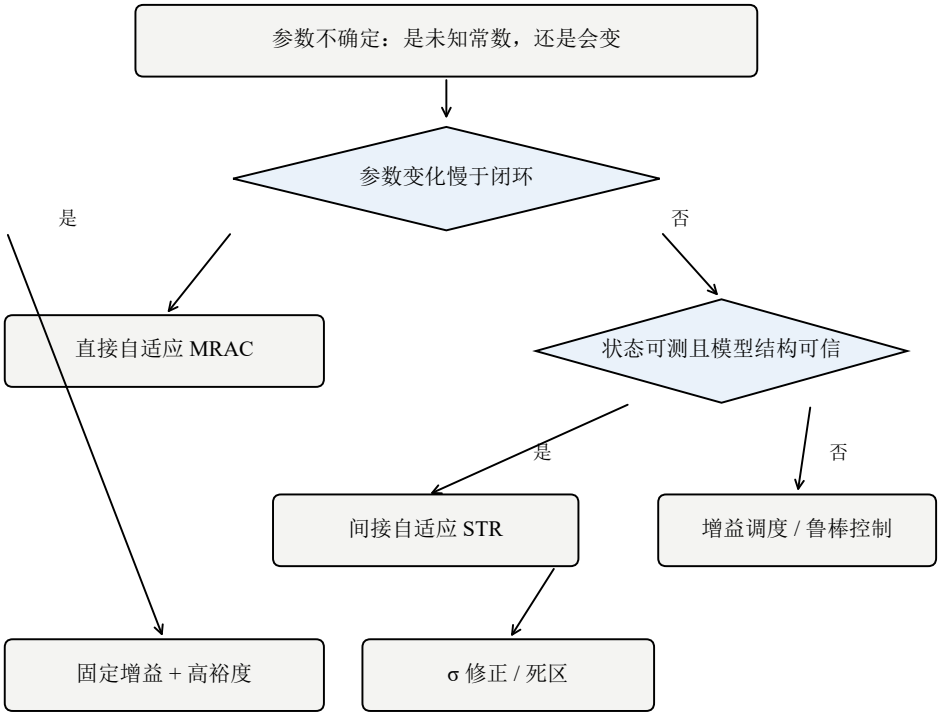


图 61-11　自适应控制的适用性决策流程

流程里有三个分叉值得记住：

1. **“参数变化能否用可测变量解释”** —— 能，就走增益调度。这是最容易被跳过、也最容易省事的一步。空调压缩机在冬季和夏季的增益差 40%，但室外温度可测，一张两点的调度表就解决了，不需要自适应。
2. **“执行器是否有 2 倍余量”** —— 没有，就走 MPC。自适应在饱和面前只能冻结，MPC 能提前减速。
3. **“是否有未建模高频”** —— 有，先把 γ 压到边界的三分之一再试；压不下去，说明这个对象不适合自适应。

61.11.6 与同类方法的横向对照

表 61-18 把本章方法与几个相邻方法放在一起，只列结构性差异，不排优劣。

维度	MRAC	固定 PID	增益调度	MPC	滑模
需要模型	结构 + 控制方向	不需要	需要调度变量	完整模型	只需不确定性界
参数变化适应	主动、在线	被动、靠裕度	主动、离线	主动、在线	靠切换增益
约束处理	只能冻结	抗饱和	同 PID	显式	靠增益
控制量平滑度	中（有抖动）	高	高	高	低（抖振）
调试成本	高（要扫边界）	低	中	高	中
可解释性	差	好	好	好	中

这张表最容易被误读成“MRAC 全面劣势”。不是的：**在“参数大范围变化 + 结构清楚 + 执行器有余量”这个交叉条件下，MRAC 是唯一能同时给出性能保证与实现简单性的方案。** 表格里那些“高成本”的项，正是它为这个能力付出的代价。

61.11.7 什么时候该退回固定增益

自适应上线之后，最常见的错误不是“参数没调好”，而是“用错了地方还硬撑”。四条退回判据，任何一条成立就该退：

一、**参数变化幅度不到 30%。** 固定增益按最坏工况整定，性能损失有限；而自适应的调试成本、抖动代价、失效风险都是实打实的。本章的方波对比里，MRAC 的 rms 误差是 PI 的 47%，但峰值误差是 141%、TV 是 179%——当工况变化不大时，这笔账算不过来。

二、**稳定边界低于可用窗口。** 按 §61.6.9 的经验式，$\gamma_{\max} \approx (5 \sim 10)/\tau_a$。若 $\tau_a = 0.2$ s，上界只有 25 到 50；再考虑噪声把可用值打到 1/3，就只剩 10 左右，学习速率低到与“固定增益慢调”没有区别。**学不动的时候，自适应只是一个更吵的 PID。**

三、**结构或控制方向不可靠。** §61.1.5 与 §61.2.9 都说明：结构错了、符号错了，γ 调什么都救不回来。这类场合的替代品是滑模（只要求不确定性有界）或鲁棒控制（只要求界已知）。

四、**可解释性与认证要求压倒性能。** 功能安全评审要看“最坏情况下控制器输出是多少”。自适应控制的最坏情况取决于 θ 的边界，而不是几个固定增益，论证成本高一个量级。很多项目最终选择“固定增益 + 增益调度”，不是因为性能更好，而是因为**能写进安全论证**。

退回的方式也有讲究，不要硬切：

- **先冻结，再观察**。把自适应关掉（$\gamma = 0$），θ 停在当前值。这是无损切换，因为控制律的形式没变。
- **再慢速回退**。把 θ 以时间常数几十秒的速率拉回标称值 θ_0，避免切换瞬间的跳变。这本质上就是 σ 修正，只不过 σ 由外部指令打开。
- **保留监测**。退回固定增益之后，θ 不再更新，但误差与 TV 的监测要继续——它们是判断“该不该再打开自适应”的唯一依据。

61.11.8 本章的三句话

第一，自适应控制买的是“参数变化时的性能一致性”，付的是“控制量抖动与调试成本”。 本章的量化版本：rms 误差降 53%、峰值误差升 41%、TV 升 79%（§61.6.6）。这笔交易只在参数变化大、且抖动可以接受时才划算。

第二，稳定性边界必须实测，不能推算。 理想执行器下 $\gamma = 100$ 可用，0.05 s 滞后下 50 就是上限（§61.6.3）。凡是“算出来能用”的结论，都要用真实执行器复核一遍。

第三，误差收敛不等于参数收敛。 这是本章最容易被忽略、也最容易被误用的一条。控制做好了是本事，把 θ 当参数用是事故。

61.12 Python 仿真代码

61.12.1 文件的组织

本章的仿真只有两个文件：

```
book/code/ch061_sim.py      仿真主体：对象、三个控制器、十四组实验
book/code/ch061_figs.py     十一张插图，全部从 ch061_sim 的结果重算
```

时域指标（超调、调节时间、稳态值）一律调用 `simlib.step_metrics`，与全书其他章同一口径；本章另外补充了 IAE、尾段 rms、控制量总变差 TV、θ 终值四个指标，写在 `metrics()` 里。这样做的目的是**保证正文里出现的每个数字都能由一条命令复现**，不需要手工抄录。

运行方式：

```
D:\Anaconda3\envs\py311\python.exe code/ch061_sim.py
D:\Anaconda3\envs\py311\python.exe code/ch061_figs.py
```

61.12.2 对象与执行器

对象是一阶 ODE，用定步长 RK4 每拍积分一次。执行器滞后作为可选的一阶环节并入同一个右端项，这样“理想执行器”与“带滞后执行器”共用一套代码，切换只改一个参数：

```python
def plant_rhs(y, ua, u, dist, b=B_PLANT, tau_a=0.0):
    """对象与执行器右端项；tau_a=0 表示执行器理想。"""
    dy = -A_PLANT * y + b * ua + dist
    dua = 0.0 if tau_a <= 0 else (u - ua) / tau_a
    return dy, dua
```

这里的 `ua` 是**真正作用到对象上的控制量**，而 `u` 是控制器算出来的量。两者的差就是 §61.6.3 那个让 γ 边界塌掉的滞后。实验 6 里把 `tau_a` 设成 0.05，同一套代码就能把稳定边界从 100 以上压到 50 附近。

`b` 也做成了参数，并且在参数突变实验里传的是一个可调用量 `b_fn(t)`，这样 $t = 5$ s 的跳变可以在一次仿真里完成，不需要“跑一半、换个控制器接着跑”——后者会让 θ 的状态丢失。

61.12.3 控制器：一个类装下所有变体

MRAC 类把本章讨论的全部机制（Lyapunov 律 / MIT 律、σ 修正、死区、饱和冻结、限幅）压在一个 `step()` 里，靠构造参数开关：

```
class MRAC:
    def __init__(self, gamma=10.0, theta0=(0.0, 0.0), mode="lyap",
                 sigma=0.0, dead=0.0, umax=None, freeze=False):
        ...

    def step(self, t, y, r, ym, ts=TS):
        e = y - ym
        # MIT 律的灵敏度信号：ef_dot = -am*ef + bm*e
        self.ef += (-AM * self.ef + BM * e) * ts
        phi = np.array([y, r], float)
        u_raw = float(self.theta @ phi)
        sat = self.umax is not None and abs(u_raw) >= self.umax

        if self.mode == "lyap":
            drive = -self.gamma * e * phi
        else:
            drive = -self.gamma * self.ef * phi

        if self.dead and abs(e) <= self.dead:
            drive[:] = 0.0
        if self.freeze and sat:
            drive[:] = 0.0
        if self.sigma:
            drive -= self.sigma * (self.theta - self.theta0)

        self.theta = np.clip(self.theta + drive * ts, -THETA_MAX, THETA_MAX)
        u = float(self.theta @ phi)
        return u if self.umax is None else float(np.clip(u, -self.umax, self.umax))
```

几个实现细节值得单独说明，它们都对应正文里的一个结论：

1. **`u_raw` 与 `u` 分开**。饱和判据必须用未限幅的 `u_raw`，否则永远判不出饱和（§61.9.3）。这是本章代码里最容易写错的一行。
2. **`np.clip` 的硬限幅放在更新之后**。放之前会让判断逻辑变得复杂，且不能防止单步跳变越界。放在之后，`theta` 的任何时刻都在 $[-20, 20]$ 内，`inf` / `nan` 无从产生。
3. **控制量在返回时再限幅一次**。参数虽然被 clip，但 `theta @ phi` 仍可能因为 ϕ 大而超出 `umax`。
4. **MIT 律的 `ef` 无条件更新**。即使当前用的是 Lyapunov 律，`ef` 也照算——它只多花一次乘加，换来的是“切换 `mode` 不需要重置状态”，便于对比实验。

STR 类走的是另一条路：先用上一拍的数据做一次 RLS 更新，再用更新后的参数算控制量。

```
def step(self, t, y, r, ym=None, ts=TS):
    if self.y_prev is not None:
        phi = np.array([self.y_prev, self.u_prev], float)
        eps = y - float(self.theta @ phi)
        Pphi = self.P @ phi
        den = self.lam + float(phi @ Pphi)
        self.P = (self.P - np.outer(Pphi, Pphi) / den) / self.lam
        self.theta = self.theta + Pphi / den * eps
        self.theta[0] = np.clip(self.theta[0], 0.5, 1.2)
        self.theta[1] = np.clip(self.theta[1], self.bmin, 1.0)
    a_hat, b_hat = float(self.theta[0]), float(self.theta[1])
    u = ((self.am_d - a_hat) * y + (1.0 - self.am_d) * r) / b_hat
    self.y_prev, self.u_prev = y, u
    return float(u)
```

两处工程保护必须保留：`theta[0]` 被夹在 $[0.5, 1.2]$（一阶对象的极点必须在单位圆内且为正），`theta[1]` 被夹在 $[b_{\min}, 1]$。`bmin=0.002` 的取值在 §61.5.2 讨论过——它防的不是辨识精度，而是除零。

61.12.4 参考输入与激励

`reference()` 支持四种指令：阶跃、方波、斜坡、PRBS。前三种用于跟踪实验，PRBS 专用于给 STR 提供激励：

```
PRBS_PATTERN = (1.0, 0.6, -1.0, -0.6, 0.6, 1.0, -0.6, -1.0)

def reference(t, kind="step", amp=1.0, period=6.0):
    if kind == "step":   return amp
    if kind == "square": return amp if (t % period) < period / 2 else -amp
    if kind == "ramp":   return amp * min(t, 5.0)
    if kind == "prbs":   return amp * PRBS_PATTERN[int(t) % len(PRBS_PATTERN)]
```

这个八位序列只含四个电平，但相邻两拍的方向组合足够多，足以让 $\{y[k-1], u[k-1]\}$ 扫开一个二维区域。§61.5.5 的实测证明这已经够用：$\lambda \le 0.98$ 时 $\hat{b}_d$ 精确收敛到 0.0209。如果换成方波（只有两个电平），回归矩阵共线，$\hat{b}_d$ 会一直贴着初值不动——这也是本章最初踩过的坑。

61.12.5 指标口径

`metrics()` 在 `step_metrics` 之上补了四项：

```
m["iae"]       = float(np.trapezoid(np.abs(e), t))      # 累计绝对误差
m["rms_e_tail"] = float(np.sqrt(np.mean(e[-tail_n:] ** 2)))
m["umax_abs"]   = float(np.max(np.abs(u)))              # 控制量峰值
m["tv_u"]       = float(np.sum(np.abs(np.diff(u))))     # 控制量总变差
m["theta_end"]  = th[-1].copy()
m["theta_err"]  = float(np.linalg.norm(th[-1] - np.array([THETA1_STAR, THETA2_STAR])))
```

TV 的定义是相邻两拍控制量差的绝对值之和，它度量“控制量一共走了多少路”，是抖动最直接的量化。正文里“TV 从 0.7 涨到 19.9”这类说法全部由这一行算出。

`theta_err` 是到真值的欧氏距离，§61.4.3 与 §61.6.2 用它说明“分量不准”这件事；它与“和”这个指标配合使用——和准、距离大，正是“只能辨识组合”的证据。

61.12.6 插图的生成

`ch061_figs.py` 里十一张图全部由 `ch061_sim` 重算，没有硬编码坐标。数据图用 `plot_styles` 的 `SINGLE` / `DOUBLE` 尺寸，框图用 `block_styles` 的语义助手：

- 图 61-1（信号流）、图 61-11（决策流程）是框图，用 `canvas` + `block` + `sum_node` 画，输出前统一调用 `export(fig, stem)`，同时落 SVG 与 PDF。
- 其余九张是数据图，每张对应一组实验，纵轴量纲与正文表格一致。
- 图内中文一律是纯文本，公式单独用 `$...$`，两者不写进同一个字符串——这条规矩来自 §61 任务卡的“图内中西文不混排”要求，目的是避免字体回退造成的方框。

61.12.7 复现本章全部数字

按下面的顺序跑一遍，正文里的每个数字都能对上：

1. `ch061_sim.py`：`main()` 里十七个实验函数（正文按叙述需要编为实验 1 到 15，其中含 10b 这类分支），覆盖 §61.1 到 §61.11 的全部引用数据。输出按实验编号分组，每行一个工况。
2. `ch061_figs.py`：读同一套函数重新计算，生成十一张图到 `figs/ch061/`。
3. `tools/check_chapter.py 61`：检查章节结构与图数。
4. `tools/build.py --chapter 61`：生成 PDF。

   如果只关心某一个结论，可以单独导入：

```
import ch061_sim as s
res = s.simulate(s.MRAC(gamma=20.0), T=10)
print(s.metrics(res)["theta_end"])     # [-0.567  1.281], 和 0.715
```

这条命令对应 §61.4.3 表 61-5 的第二行。把它换成 `gamma=2.0`，会得到 $(-0.142, +0.860)$，和 0.718——两个分量差很多，和却几乎一样。**这一行输出就是整章论点的最小复现。**

61.12.8 五组补充实验

正文里有五张小表来自“辅助实验”，它们不涉及新算法，只是把已有函数换个参数扫一遍，代码都在 `ch061_sim.py` 里，函数名以 `exp_` 开头。

实验 11（§61.5.3 的采样周期扫描） 是纯解析计算，只用到式 (61-26) 与式 (61-30)，不跑闭环：

```
for ts in (0.001, 0.005, 0.01, 0.02, 0.05, 0.1):
    ad = np.exp(-A_PLANT * ts)
    bd = B_PLANT / A_PLANT * (1 - np.exp(-A_PLANT * ts))
    th = np.array([(np.exp(-AM * ts) - ad) / bd, (1 - np.exp(-AM * ts)) / bd])
```

把 `th` 与 `[THETA1_STAR, THETA2_STAR]` 做差再取范数，就是正文里“偏离 0.63%”那一列。步长上界的估计用 `norm(theta)/PHI_TYP/ts`，其中 `PHI_TYP=1.4` 是回归向量的典型范数。

实验 12（§61.7.5 的 $\gamma \times \Delta$ 二维表） 是十次独立的闭环仿真，除了 `gamma` 与 `dead` 之外全部参数固定（`T=20`、`noise=0.01`、`seed=7`）。**随机种子必须固定**，否则标准差那一列会逐次跳动，没法跨行比较。

同一函数里还算了一个诊断量：`idx = np.where(np.abs(e) > d)[0]`，取 `idx[-1]` 就是“误差最后一次超过死区”的时刻。**这个时刻解释了两列数字的关系：**γ 越大，冻结发生得越早、冻结时的参数越准，于是死区的静差

反而越小。

实验 13（§61.4.7 的前七拍） 只需要把仿真时长设成 0.08 s，然后逐拍打印：

```
res = simulate(MRAC(gamma=20.0, theta0=(0.0, 0.0)), T=0.08)
for k in range(1, 8):
    print(res["t"][k], res["ym"][k], res["y"][k], res["theta"][k], res["u"][k])
```

注意 `ym` 是在每拍末更新的，所以打印出来的 `e=y-ym` 与“控制器当拍看到的 e”差一拍。正文里已经说明这一点，读表时不要把 $k=0$ 的 -0.02 当成控制器看到的误差。

实验 14（§61.5.7 的协方差迹） 用一个包装函数把 `tr(P)` 记下来，因为 `STR` 本身不存这个量：

```
ctrl = STR(lam=lam)
orig = ctrl.step
def step(t, y, r, ym=None, ts=TS):
    u = orig(t, y, r, ym, ts)
    trace.append(np.trace(ctrl.P))
    return u
ctrl.step = step
```

这种“临时挂钩”写法在仿真里很实用：不用为了记录一个中间量去改控制器类，也就不会因为改动而影响已经发表过的数字。

实验 15（§61.6.10 的斜坡扰动） 需要把 `simulate()` 的 `dist` 从常数改成可调用量，否则“随时间线性增长”根本表达不出来：

```
dt = dist(t) if callable(dist) else dist        # 扰动可以是常数，也可以是 t 的函数
bt = b(t) if callable(b) else b                 # 增益同理，供参数突变实验用
```

两行都写成同一个模式。**仿真的接口要为“时变”留一个口子**，否则每加一个时变工况就要复制一遍主循环，而复制出来的循环最容易与主循环悄悄分叉——本章早期版本就出现过一次：临时脚本里手写循环又多调用了一次 `rk4_step`，`ua` 被覆盖，斜坡扰动的效果完全消失。把时变量收进 `simulate()` 之后，这类错误就无处可藏。

还有一处口径要留意：这个实验报的是 20 s 末的误差，不是全程 rms。$|e_{ss}| \approx 0.36s/\gamma$ 是稳态估计，前 5 s 的暂态误差主要由初始参数误差决定、与 s 基本无关，混在一起算 rms 会把两种机理搅在一起。

这五个函数都挂在 `main()` 末尾，跑一次 `ch061_sim.py` 就全部输出，不需要额外命令。

61.12.9 复现时最容易踩的三个坑

这套代码不长，但有三个地方会让“照抄一遍却对不上数”。

第一，`np.where` 取最后一个索引会空。 实验 12 里用 `idx = np.where(np.abs(e) > d)[0]`、再取 `idx[-1]` 求“误差最后一次超出死区的时刻”。如果某组参数下误差从头到尾都在死区内（γ 小、Δ 大时完全可能），`idx` 是空数组，`idx[-1]` 直接抛 `IndexError`。正确写法是先判空：

```
idx = np.where(np.abs(e) > d)[0]
k = int(idx[-1]) if len(idx) else 0
```

这类“扫描实验里个别工况越界”的错误很难靠读代码发现，因为默认参数那一行跑得很顺。**扫参数空间时，先假设每个分支都会遇到空集。**

第二，理想执行器时 `ua` 会停在 0。 对象积分的是 `ua`（执行器输出），而 `ua` 只在 `tau_a > 0` 时由 `dua = (u - ua)/tau_a` 推动。`tau_a = 0` 表示执行器理想，此时若不做特殊处理，`dua = 0`，`ua` 永远是初值 0——**控制器算得再对，对象收到的控制量始终是零**。本章的写法是加一行短路：

```
if tau_a <= 0:
    ua = u            # 执行器理想：直接跟随
```

这个坑的可怕之处在于它的表现：y 恒为 0、e 恒为 $-y_m$、θ 被一路推向限幅，看上去“自适应完全失效”。如果不知道这一行，会去怀疑增益、怀疑符号、怀疑参考模型，而真正的原因是对象根本没接上控制量。

第三，随机数不能在多次仿真之间共用。 本章每个 `simulate()` 内部按传进来的 `seed` 新建 `default_rng`，而不是用一个模块级的全局 RNG。理由是可比性：如果全局共用一个 RNG，那么**在 `main()` 里多加一个实验，后面所有实验抽到的噪声序列都会变**，正文里已有的表格和曲线会集体失配，而且没有任何报错提示。本章在第二轮扩写时新增了五个 `exp_` 函数，正是靠“每次仿真重建 RNG”才保证了旧数字一个都没动。

同样的道理适用于参数：凡是会在正文里被引用的量，都应当能从函数签名直接看出来（`gamma`、`dead`、`noise`、`seed`、`T`），而不是藏在函数体里的常量。**复现性不是“我记得当时是怎么跑的”，是“读签名就知道当时是怎么跑的”。**

第 62 章 随机控制基础

确定性控制的题目是“让输出到达某个值”，随机控制的题目是“让输出的波动尽量小”。两者的评价指标从“误差”换成了“方差”，而方差不会被反馈“消掉”——它只能被搬运、被整形、被限制在一个下界之上。本章要回答三个问题：这个下界是多少、怎么算、以及为什么工程上常常主动放弃它。

62.1 动机与直观理解

62.1.1 一个“稳态一直在抖”的回路

先看一个在现场极常见的场景。某温度回路的设定值是 80 ℃，控制器稳定之后，记录仪上的曲线不再有趋势，但一直在 79.6 到 80.4 之间随机地抖。操作员问：能不能把它压平一点？

直觉的回答是“加大增益”。但如果真的把比例增益从 1 加到 3，画面往往不会变平：抖动幅度可能略降，也可能反而变大，而且控制阀开始频繁动作，执行机构的动作次数成倍上升。这个现象说明，**稳态抖动的主要成分不是“控制器不够努力”，而是扰动本身**——反馈只能抑制它“能够观测并且来得及响应”的那部分。

本章的对象就是一个这样的小系统：一阶、单输入单输出、有随机扰动。写成差分方程

$$y[k] = a\,y[k-1] + b\,u[k-d] + e[k] + c\,e[k-1] \tag{62-1}$$

其中 $e[k]$ 是零均值的白噪声序列，σ_e 是它的标准差。这个式子有三个部分：$a\,y[k-1]$ 是对象自己的惯性，$b\,u[k-d]$ 是控制量经过 d 拍延迟之后的作用，$e[k] + c\,e[k-1]$ 是扰动。最后这一项是本章的主角。

62.1.2 同样方差，噪声的“形状”不同

式 (62-1) 里的扰动项 $e[k] + c\,e[k-1]$ 有一个可以调的形状参数 c。$c = 0$ 时扰动是纯白噪声：相邻两拍完全独立。$c = 0.9$ 时，扰动变成“前后高度相关”的序列：这一拍是正的，下一拍多半还是正的。

两者的方差并不相同——$e[k] + c\,e[k-1]$ 的方差是 $(1 + c^2)\sigma_e^2$，所以 $c = 0.9$ 的扰动方差是 $c = 0$ 时的 1.81 倍。但比方差更重要的差别在**可预测性**：

- 白噪声的下一拍与这一拍无关，任何控制器都无法预测它，只能被动承受。
- 相关噪声的下一拍与这一拍强相关，于是“这一拍已经看到的扰动”可以拿来预测“下一拍还会有多少”，从而提前抵消掉一部分。

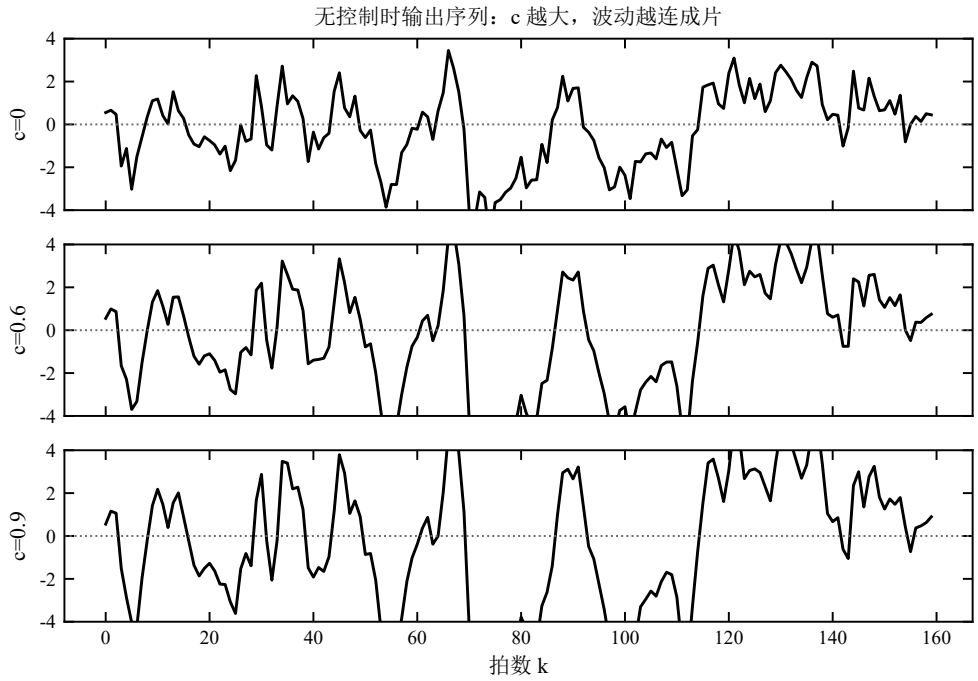


图 62-1　噪声相关系数 c 对输出序列的影响

图 62-1 把三条无控制的输出序列画在一起。$c=0$ 那条上下乱跳、没有记忆；$c=0.9$ 那条呈现出明显的“成片”结构——连着几拍偏正、再连着几拍偏负。这三条曲线的方差分别是 1.00、1.36 与 1.81（理论值 $1+c^2$），但肉眼看过去，第三条更像“一个缓慢起伏的信号”，而不是噪声。

这个“形状”就是随机控制唯一可以利用的资源。**控制器不能消除白噪声，但它可以利用噪声的相关性，把可预测的那一部分提前扣掉。**

62.1.3 最小方差控制的想法：把可预测的部分抵掉

把式 (62-1) 往后推一拍：

$$y[k+1] = a\,y[k] + b\,u[k] + e[k+1] + c\,e[k] \tag{62-2}$$

右端有两类东西。一类是**在 k 时刻已经知道**的：$y[k]$、$e[k]$（扰动可以重构，见 §62.2.2）、以及我们马上要决定的 $u[k]$。另一类是**下一拍才会揭晓**的：$e[k+1]$。

最小方差控制（minimum variance control，MVC）的做法极其直接：**把已知的那部分凑成零**，让下一拍的输出只剩下无法预测的 $e[k+1]$。令

$$a\,y[k] + b\,u[k] + c\,e[k] = 0 \quad\Longrightarrow\quad u[k] = -\frac{a\,y[k] + c\,e[k]}{b} \tag{62-3}$$

代回式 (62-2)，得到 $y[k+1] = e[k+1]$。也就是说，闭环输出**逐拍等于新息**，方差恰好是 σ_e^2。

这是随机控制里最漂亮的一个结论：**在模型准确、无延迟、执行器不限幅的前提下，输出的方差可以精确做到 σ_e^2，一拍都不多。** 这个值不是一个“不错的结果”，而是一个硬下界——任何控制器都不可能做得比它更好，因为 $e[k+1]$ 在 k 时刻根本不存在，谁都预测不了。

代价藏在这个“漂亮”里。看式 (62-3)：$u[k]$ 与 $y[k]$ 是代数关系，增益是 $1/b$ 量级，而 y 里含有方差为 $(1+c^2)\sigma_e^2$ 的噪声。**控制器把噪声原封不动地放大 $1/b$ 倍送到执行器上。** §62.6.2 的实测里，输出方差从 6.78 降到 1.01，控制量方差却从 0 升到 7.93——控制量的标准差（2.82）是输出标准差（1.01）的近三倍。

62.1.4 为什么不能把噪声抵消到零

式 (62-3) 能成立，依赖一个隐含条件：控制量 $u[k]$ 能立刻影响 $y[k+1]$。如果对象有 d 拍延迟，$u[k]$ 最早也要到 $y[k+d]$ 才起作用，那么 $y[k+1],\dots,y[k+d-1]$ 这些拍上的扰动就完全无从干预。

这就是延迟对随机控制的惩罚。$d=1$ 时下界是 1.0000；$d=2$ 时变成 2.9600；$d=3$ 时是 4.2144（§62.6.3 会给出完整推导与实测）。**多一拍延迟，下界涨了近三倍**，原因不是控制器变笨了，而是那几拍上的 e 已经“生米煮成熟饭”。

这个结论有一个很实用的推论：**在随机控制里，延迟是最贵的资源。** 确定性控制里，延迟主要侵蚀相位裕度，通常还有裕度可退；随机控制里，延迟直接抬高方差下界，而且这个下界是物理的，任何算法都绕不过去。第 47 章与第 107 章的 Smith 预估能把延迟“搬出”闭环，但它搬的是确定性部分，对这几拍已经发生的扰动同样无能为力。

62.1.5 随机控制与确定性控制的分界

把本章的方法和前面几章摆在一起，可以看到一条清晰的分界：

问题	评价指标	主要矛盾	代表章节
确定性跟踪	误差、调节时间、超调	动态够不够快、稳不稳	第 44、58、61 章
随机调节	输出方差、控制方差	噪声能抵消多少、代价多大	本章
随机跟踪	方差 + 跟踪偏差	两者叠加，通常需要 GMV	§62.2.6

分界的实质是“扰动有没有结构”。如果扰动是未知的、无结构的（任何频率都可能、任何时刻都可能），那么控制器唯一能做的就是“高增益压制”，这时问题退化为确定性控制的鲁棒性问题。如果扰动有结构——尤其是**有色**结构，即相邻时刻相关——那么就可以用噪声模型把它“预测”出来，随机控制才有用武之地。

所以随机控制的第一件事不是设计控制器，而是**判断噪声有没有结构**。这一步做错，后面的推导再漂亮也是空转。

62.1.6 本章的骨架

后面的安排是：§62.2 建立 ARMAX 模型、新息、Diophantine 恒等式与最小方差下界这一套数学；§62.3 把它压成五步设计流程；§62.4 讨论连续情形（以及为什么工程上几乎总是直接在离散域做）；§62.5 给离散实现；§62.6 用实测数据说明下界能不能达到、失配会怎样、马尔可夫跳变会怎样；§62.7 逐个解释 a,b,c,d,λ 的作用；§62.8 给调参顺序与一次完整实录；§62.9 讲广义最小方差、极点配置、自适应最小方差这几条改进路线；§62.10 讲现场辨识与落地；§62.11 划边界；§62.12 给可复现代码。

一句话概括本章的立场：**最小方差控制是一个“标尺”，不是一个“产品”。** 它给出的下界告诉你“这件事最好能做到多好”，而它本身因为控制量太大、对失配太敏感，很少直接上线。真正上线的是它的各种“打折版”——广义最小方差、带约束的 MVC、以及自适应版本。

62.1.7 本章讲什么、不讲什么

为了避免与相邻章节重复，这里把边界一次说清。

讲什么。 只讲一个最小例子：一阶对象、有色噪声、固定整数延迟、零均值扰动、无约束。围绕这个例子讲三件事——方差下界怎么算、控制律怎么写、模型错了会怎样。选最小例子不是偷懒，而是因为随机控制的全部核

心结论（下界与 b 无关、极点被对消、失配单侧悬崖、噪声相关性压缩增益裕度）在这个例子上都能完整呈现，换成二阶只会增加符号，不增加洞见。

不讲什么，以及去哪看。

- **辨识算法**：RLS、投影、遗忘因子、激励设计，第 61 章有整套推导与实测。本章只说“c 可以用标量 RLS 估”，不展开递推公式。
- **MPC**：滚动优化、约束、预测时域，第 58 章讲原理、第 80 章讲工业实现。本章的 GMV 可以看作“预测时域为 d、无约束、只罚一拍”的特例。
- **Smith 预估**：大延迟下的确定性跟踪，第 47 章与第 107 章。本章 §62.9.7 只讲两者的分工。
- **PID 整定**：第 44 章与第 106 章。本章与它的关系在 §62.8.4：$d=1$ 时 GMV 就是一个 P 控制器，只是增益是算出来的。
- **LQG 与卡尔曼滤波**：第 54 章。本章 §62.9.6 只给三个对应关系，不展开矩阵推导。

三条阅读路径。 只想判断“这个回路该不该做随机控制”的读者，直接读 §62.11.5 的三分钟估算，十分钟能出结论；准备落地的读者，读 §62.3 的五步流程与 §62.10 的落地清单；要复现或改代码的读者，读 §62.12，其中 §62.12.10 给了一个手算校验，三分钟就能确认代码没写错。

62.2 数学原理

62.2.1 ARMAX 模型与三个多项式

把式 (62-1) 写成算子形式。令 z^{-1} 为延迟算子（$z^{-1}y[k]=y[k-1]$），定义三个多项式

$$A(z^{-1})=1-az^{-1},\qquad B(z^{-1})=b,\qquad C(z^{-1})=1+cz^{-1} \tag{62-4}$$

于是式 (62-1) 可以写成

$$A(z^{-1})\,y[k]=z^{-d}B(z^{-1})\,u[k]+C(z^{-1})\,e[k] \tag{62-5}$$

这种形式叫 ARMAX（autoregressive moving average with exogenous input）：A 是自回归部分（对象惯性），C 是滑动平均部分（噪声相关性），B 是外部输入通道。三个多项式各有分工，而且**在随机控制里它们不再是“一个传递函数拆成三块”，而是三个必须分别知道的量**。

这一点和前面几章很不一样。在确定性控制里，B/A 就可以描述全部动态，C 无关紧要；到了随机控制，C 直接决定下界（式 62-18），**不知道 C 就不知道“最好能做到多好”**。这也是随机控制比确定性控制多做的一步工作：辨识噪声模型。

62.2.2 新息：可预测与不可预测的分界

式 (62-5) 里的 $e[k]$ 有一个专门的名字，叫**新息**（innovation）。它不是“扰动”，而是“扰动里这一拍新出现、上一拍无法预测的那部分”。

这个区分是随机控制的全部起点。对 $C=1+cz^{-1}$ 的一阶情形，扰动是 $e[k]+c\,e[k-1]$。在 $k-1$ 时刻，$e[k-1]$ 已知，所以可以预测“下一拍的扰动大约是 $c\,e[k-1]$”；剩下真正无法预测的只有 $e[k]$。**MVC 对付的就是这个 $e[k]$，而 $c\,e[k-1]$ 那一部分是可以提前扣掉的。**

新息可以从实测数据在线重构。把式 (62-5) 解出 $e[k]$：

$$e[k] = y[k] - a\,y[k-1] - b\,u[k-d] - c\,e[k-1] \tag{62-6}$$

这是一个递推：每一拍用当前的 y、u 和上一拍的 e，算出这一拍的新息。它本质上是一个“逆噪声模型”滤波器，传递函数是 A/C：

$$e[k] = \frac{A(z^{-1})}{C(z^{-1})}\,y[k] - \frac{z^{-d}B(z^{-1})}{C(z^{-1})}\,u[k] \tag{62-7}$$

式 (62-7) 有一个必须注意的性质：**C 必须是最小相位的**（零点在单位圆内）。若 C 有零点在圆外，$1/C$ 不稳定，新息重构会发散。对本章的一阶 $C = 1 + cz^{-1}$，零点在 $z = -c$，只要 $|c| < 1$ 就满足。工程上辨识出来的 C 若出现 $|c| \geq 1$，说明要么数据有问题，要么模型阶次太低，不能直接拿去算控制律。

62.2.3 d 步预测与 Diophantine 恒等式

要控制 $y[k+d]$，就得先能预测它。把式 (62-5) 两边乘 z^{d}（即向前推 d 拍），得到

$$y[k+d] = \frac{B}{A}u[k] + \frac{C}{A}e[k+d] \tag{62-8}$$

右端第二项含 $e[k+d]$，它在 k 时刻还不存在，无法预测。要把“能预测”与“不能预测”分开，需要做一次多项式除法：把 C/A 拆成“有限项 + 剩下的真分式”。

$$C(z^{-1}) = A(z^{-1})F(z^{-1}) + z^{-d}G(z^{-1}) \tag{62-9}$$

这就是 **Diophantine 恒等式**（也叫丢番图方程）。F 是商、$z^{-d}G$ 是余项，次数约束是 $\deg F = d-1$、$\deg G = \deg A - 1$。对本章的一阶 A，G 退化为一个常数 g_0。

把式 (62-9) 代入式 (62-8)，整理后得到本章最核心的分解：

$$\boxed{y[k+d] = \underbrace{\frac{BF}{C}u[k] + \frac{G}{C}y[k]}_{\text{可预测，}k\text{ 时刻已知}} + \underbrace{F e[k+d]}_{\text{不可预测}}} \tag{62-10}$$

这个式子的结构值得反复看。可预测部分只含 $u[k]$ 与 $y[k]$——都是当前拍拿得到的东西；不可预测部分是 F 作用在未来的新息上，F 的次数是 $d-1$，正好对应“未来 d 拍”。

F 的系数可以直接递推。对本章的一阶 A、一阶 C，恒等式两端的 z^{-k} 系数逐项对齐，得到

$$f_1 = a + c, \qquad f_k = a\,f_{k-1}\ (k \geq 2) \quad\Longrightarrow\quad f_k = (a+c)\,a^{k-1} \tag{62-11}$$

这个式子值得单独看一眼：**第一项由“对象惯性 a”与“噪声相关性 c”共同决定，之后每一项按对象极点 a 衰减**。$a + c$ 越大，预测就越“有把握”（噪声黏、对象慢），下界抬得越高；而衰减速度由 a 单独决定，a 越小（对象越快），延迟的边际代价掉得越快。

G 在一阶情形下是一个常数：

$$g_0 = a\,f_{d-1} + c\,\delta_{d,1} \tag{62-12}$$

其中 $\delta_{d,1}$ 在 $d = 1$ 时为 1、否则为 0。把这几个数算出来，放在表 62-1 里。

d	$F(z^{-1})$	g_0	下界 $\sum f_i^2$
1	1	1.4000	1.0000
2	$1+1.4z^{-1}$	1.1200	2.9600
3	$1+1.4z^{-1}+1.12z^{-2}$	0.8960	4.2144
4	$1+1.4z^{-1}+1.12z^{-2}+0.896z^{-3}$	0.7168	5.0172
5	续一项 $0.7168z^{-4}$	0.5734	5.5310

表 62-1 的数字全部由 $a=0.8$、$c=0.6$ 算出，与仿真无关，可以手算核对：$f_1=1.4$、$f_2=0.8\times1.4=1.12$、$f_3=0.8\times1.12=0.896$、$f_4=0.8\times0.896=0.7168$。

注意最后一列的增量：d 从 1 到 2 涨 1.9600，从 2 到 3 涨 1.2544，从 3 到 4 涨 0.8028，从 4 到 5 涨 0.5138。**延迟的边际代价按 $a^2=0.64$ 的比例递减**——第一拍延迟最贵，后面的越来越便宜。这不是巧合：a 决定对象“记住”过去多久，a 越小，越早的扰动越容易被对象自己消化掉。

表 62-1 还给出一个实用的判断：**如果下界从 $d=1$ 到 $d=2$ 涨了将近三倍，那么“把延迟减掉一拍”的价值远大于“把控制器调好一点”。** 这也是随机控制里最常见的工程改进方向——不是换算法，是挪传感器、换执行器、把总线周期缩短。

62.2.4 最小方差控制律

有了式 (62-10)，控制律的推导只有一步：**让可预测部分等于零**。

$$B(z^{-1})F(z^{-1})\,u[k]+G(z^{-1})\,y[k]=0 \tag{62-13}$$

解出 $u[k]$，对本章的常数 $B=b$：

$$u[k]=-\frac{1}{b\,f_0}\left(g_0\,y[k]+\sum_{i=1}^{d-1} b\,f_i\,u[k-i]\right) \tag{62-14}$$

对 $d=1$，$F=1$、$g_0=a+c$，式 (62-14) 退化为

$$u[k]=-\frac{a+c}{b}\,y[k] \tag{62-15}$$

这是一个纯比例控制器，增益 $-(a+c)/b=-2.8$。把它和式 (62-3) 对比一下：式 (62-3) 是 $u=-(ay+ce)/b$，含新息；式 (62-15) 只用 y。两者在闭环上完全等价，但实现上差别很大：

- 式 (62-3) 需要在线重构 $e[k]$，也就是需要维护式 (62-6) 的递推，并且对 c 的准确性敏感。
- 式 (62-15) 只需要 $y[k]$，一行乘法。

工程上几乎总是用式 (62-14) 这种“消去新息”的形式。 原因很简单：新息是算出来的中间量，任何模型误差都会在它身上累积；而 y 是实测的，不放大误差。这也是最小方差控制看起来“不像随机控制”的原因——它的最终实现常常就是一个比例控制器，随机性全部藏在增益的取值里。

对 $d=2$：$F=1+1.4z^{-1}$、$g_0=1.12$、$b=0.5$，

$$u[k]=-\frac{1.12}{0.5}y[k]-1.4\,u[k-1]=-2.24\,y[k]-1.4\,u[k-1] \tag{62-16}$$

这里出现了一个有意思的结构：**控制律自己带了一个极点 -1.4**。它是“用过去两拍的信息去推断这一拍该补多少”，本质上是一个预测器。预测器越用力，控制律的增益越大，对模型误差也越敏感。§62.6.4 会看到，$d=1$ 时对象增益只要比模型大 30% 就发散，而 a 偏 10% 只让方差涨一个百分点——**对随机控制致命的是增益误差，不是惯性误差**。

62.2.5 最小方差下界

把式 (62-13) 代回式 (62-10)，可预测部分被清零，剩下的只有

$$y[k+d] = F(z^{-1})\, e[k+d] \tag{62-17}$$

两端取方差，利用 e 是白噪声：

$$\mathrm{Var}(y) = \sigma_e^2 \sum_{i=0}^{d-1} f_i^2 \tag{62-18}$$

这就是**最小方差下界**。它的形式极其简单：噪声方差乘上一个由 F 决定的系数。三个性质值得记住：

1. **与 b 无关。** 下界只取决于噪声（σ_e、c）与延迟 d，跟对象增益、跟控制器都无关。这是“物理下界”的含义——它不是“这套控制器能做到的极限”，而是“任何控制器都做不到更好”。
2. **与 a 有关，但只通过 F。** a 影响 f_k 的衰减速度，也就影响延迟的边际代价。
3. **$d=1$ 时下界就是 σ_e^2。** 这是最干净的情形，也是本章仿真的标称工况：$\sigma_e = 1$，下界 1.0000。

式 (62-18) 还有一个等价写法，用频域看更直观：

$$\mathrm{Var}(y) = \frac{\sigma_e^2}{2\pi} \int_{-\pi}^{\pi} |F(e^{j\omega})|^2 \, \mathrm{d}\omega \tag{62-19}$$

它说的是：**闭环输出的功率谱就是 $|F|^2$ 乘噪声谱**。F 是“滤波器”，MVC 做的事就是把这个滤波器压到物理允许的最窄。$d=1$ 时 $F=1$，滤波器是平的——输出在所有频率上都是白的，方差最小。

62.2.6 广义最小方差：给控制量定价

式 (62-13) 只关心输出，完全不考虑控制量多大。§62.1.3 已经看到代价：输出方差 1.01，控制量方差 7.93。工程上这往往不可接受——阀门会磨坏，电机要发热，执行机构有动作次数寿命。广义最小方差（generalized minimum variance，GMV）把这笔代价显式写进指标：

$$J = \mathbb{E}\left\{ \left[P(z^{-1}) y[k+d] - R(z^{-1}) r[k] \right]^2 + \lambda\, u[k]^2 \right\} \tag{62-20}$$

P、R 是设计者选的加权多项式（通常取 1），$\lambda > 0$ 是**控制量的价格**。λ 越大，控制越“贵”，控制器越不愿意动。

对本章的 $d=1$、$P=R=1$、调节问题（$r=0$）、新息形式，一步预测是 $\hat{y}[k+1] = a\, y[k] + c\, e[k]$，而 $y[k+1] = \hat{y}[k+1] + b\, u[k]$。代入式 (62-20)，对 $u[k]$ 求导并令其为零：

$$2b\left(\hat{y}[k+1] + b\, u[k]\right) + 2\lambda\, u[k] = 0 \quad \Longrightarrow \quad u[k] = -\frac{b\, \hat{y}[k+1]}{b^2 + \lambda} \tag{62-21}$$

这个式子的结构很值得看。分子与 MVC 相同（都是 $-b\hat{y}$），分母从 b^2 变成 $b^2+\lambda$。**λ 的作用相当于把对象增益“虚增”了一部分**，于是闭环的有效增益下降。三个极限：

- $\lambda = 0$：退化为 MVC，$u = -\hat{y}/b$，输出方差达到下界。
- $\lambda \to \infty$：$u \to 0$，退化为“不控制”，输出方差回到开环值 6.78。
- λ 与 b^2 同量级时：两者各占一半，这是工程上最常用的区间。

注意式 (62-21) 里 λ 与 b^2 是相加的，这意味着**λ 的合适取值依赖 b**。b 减半，b^2 变成四分之一，同样的 λ 就“更贵”了。这一点在 §62.7.6 会展开：λ 不是一个无量纲的旋钮，它的量纲是 b^2，换对象就要重标。

62.2.7 随机稳定性的三种口径

确定性系统说“稳定”，指的是初值扰动下轨线收敛。随机系统里“收敛”有不止一种含义，必须分清，否则会拿一个口径的结论去套另一个口径的现象。

依概率稳定：对任意 $\epsilon > 0$，$\lim_{k\to\infty} \Pr\{\|y[k]\| > \epsilon\} = 0$。它说的是“跑远的概率趋于零”，但不管“跑远时能跑多远”。

均方稳定：$\lim_{k\to\infty} \mathbb{E}\{\|y[k]\|^2\} = 0$，且二阶矩一致有界。它管的是“能量的期望”，比依概率稳定强。

几乎必然稳定：$\Pr\{\lim_{k\to\infty} y[k] = 0\} = 1$。它管的是“每一条样本轨线”，但允许极少数轨线在有限时间内冲得很高。

三者的关系是：均方稳定 $\Rightarrow$ 依概率稳定，几乎必然稳定与均方稳定互不包含。对线性时不变系统，若二阶矩存在，三者由同一个条件刻画——闭环极点全部在单位圆内。**所以在 LTI 场合争这三种定义没有意义，真正的差别出现在参数时变或跳变的时候。**

马尔可夫跳变系统就是这样一个场合。§62.6.6 会看到：每一组参数的闭环极点都在单位圆内，但切换足够快时系统仍然可能均方发散。原因是“每一拍都稳定”不等于“长期稳定”，切换本身会注入能量。

62.2.8 马尔可夫跳变系统

把参数当作一个马尔可夫链的输出：

$$x[k+1] = A_{\theta_k} x[k] + B_{\theta_k} u[k] + E w[k], \qquad \Pr\{\theta_{k+1} = j \mid \theta_k = i\} = p_{ij} \tag{62-22}$$

θ_k 在有限个模式之间跳，每个模式有自己的 A_i, B_i。工程上的对应物很多：批次切换、工况切换、传感器/执行器部分失效、负载突变。

均方稳定的判据是一组耦合的 Lyapunov 不等式：存在 $P_i > 0$ 使得

$$\sum_{j} p_{ij} A_j^{\mathsf{T}} P_i A_j - P_i < 0, \qquad \forall i \tag{62-23}$$

它和 LTI 的 $A^{\mathsf{T}} P A - P < 0$ 长得像，但每个模式要各自满足一个“把所有可能跳到的模式都算进去”的不等式。这解释了为什么“逐模式稳定”不够：式 (62-23) 的第二项是加权和，切换概率 p_{ij} 直接进入稳定性条件。

对控制设计，这意味着一个残酷的事实：**按某一个模式设计的固定控制器，可能在另一个模式下跌到不稳定，而按“最坏模式”设计又会在好模式上过度保守。** §62.6.6 的实测给出这两种设计的量化差距。

62.2.9 闭环极点去哪了

最后回答一个容易被误解的问题：MVC 把闭环极点“放”到哪了？

由式 (62-17)，$y[k+d] = F(z^{-1}) e[k+d]$。右端是一个**有限脉冲响应**：$F$ 只有 d 项，没有分母。所以从新息到输出的传递函数没有极点——它不是一个“极点配置”的结果，而是“极点被消掉”的结果。

更准确地说，MVC 用控制律把对象极点 a 与噪声零点 $-c$ 一起消掉了：

$$y = \frac{C}{A}\cdot\frac{1}{1+z^{-d}G/(AF)}\,e = \frac{CF}{AF+z^{-d}G}\,e$$

把 $BFu+Gy=0$ 写成 $u=-Gy/(BF)$，代入对象方程 $Ay=z^{-d}Bu+Ce$，两边同乘 F 得 $y(AF+z^{-d}G)=CFe$。而 Diophantine 恒等式告诉我们分母 $AF+z^{-d}G$ 恰好就是 C，于是

$$\frac{y}{e}=\frac{CF}{C}=F(z^{-1})$$

A 与 C 都被对消，只剩 F。这就是式 (62-17) 的来源。换一个同样正确的写法：若把控制律直接代回 $Ay=z^{-d}Bu+Ce$ 而不乘 F，分母写成 $ABF+z^{-d}BG$，由 $C=AF+z^{-d}G$ 左乘 B 得 $BC=ABF+z^{-d}BG$，同样看到 B、C 成对消失。

这个推导有两条工程含义。第一，**MVC 要求 B 与 C 都可逆**（最小相位），否则“对消”会把不稳定极点搬到闭环里——这正是 §62.2.2 强调 C 最小相位的原因，也是 MVC 不能用于非最小相位对象的原因。第二，**“极点被消掉”意味着模型误差没有缓冲**：真实对象若与模型有偏差，被消掉的极点会“漏”出来，闭环就多了一个设计时没考虑的动态。§62.6.4 的失稳就是这么来的。

62.2.10 为什么偏偏是 ARMAX 这个结构

本章从头到尾用 $A(z^{-1})y=z^{-d}B(z^{-1})u+C(z^{-1})e$ 这一个式子。辨识课上通常会给三个候选，值得说清为什么选它。

ARX：$Ay=z^{-d}Bu+e$。 扰动被强制成 $1/A$ 的形状——也就是“负载扰动从入口进入、与对象共享同一个惯性”。它最大的好处是线性回归，最小二乘一步到位；最大的问题是**结构错时误差全跑进 A**：若真实扰动有色，ARX 会把噪声的动态硬塞给 A，估出的 a 偏大。a 偏大的直接后果是下界被算小、控制律被算得比实际更激进——**偏差的方向正好是危险的那一侧**。

OE：$y=\frac{B}{F}u+e$。 对象动态与噪声完全分开，参数无偏，是“对象就是对象、噪声就是噪声”的写法。代价有两个：要解非线性优化（有局部极小），而且它给不出闭式的 Diophantine 分解——本章 §62.2.6 那一整套 F、G 递推没有对应的简单形式。做辨识研究用 OE，做控制器设计用 ARMAX，这是两个不同的目的。

ARMAX：两边共享 A。 它把“扰动经过对象惯性”这件事写进了结构里，同时留出 C 去描述噪声的额外形状。本章的物理图像正好对上：负载扰动从对象入口进来，先被对象惯性与延迟整形，再叠加到输出上，所以 C/A 里必须出现同一个 A。$c=0$ 时它退化成 ARX；$c\neq 0$ 时它多了一个参数，多出来的这个参数就是“扰动比对象更黏”的那部分。

选择 ARMAX 的真正理由不是“它最通用”，而是三条具体的匹配：

1. **参数最省。** 一阶对象加延迟，一共 a,b,c,d 四个数。同样的数据用高阶 ARX 拟合，要七八个系数，而多出来的系数没有物理含义，也没法做增益标定。
2. **有闭式解。** $C=AF+z^{-d}G$ 是多项式除法，手算都能算，F 的系数还有 $f_k=(a+c)a^{k-1}$ 这样干净的表达式。这是本章能给出“下界与 b 无关”这类硬结论的前提。
3. **误差可归因。** 结构对了，残差就是白噪声；残差不为白，说明 C 的阶次或 A 的阶次不对。这个自检在 §62.10.1 的清单里排第二，靠的就是 ARMAX 的这个性质。

它也有明确的失效场合。**量测噪声为主**时，噪声不是“从入口进来”的，共享 A 的假设就不成立，此时强行用 ARMAX 会把量测噪声的动态并进 A，a 被系统性高估。判断办法很简单：看残差的白噪声检验——若残差在低阶

就白了，但估出的 a 与阶跃响应测出的时间常数对不上，就是这一类结构错。**结构错不能靠调参数补救，只能换结构或提高阶次。**

62.3 设计步骤

62.3.1 五步流程

把 §62.2 的推导压成现场能执行的流程，只有五步。每一步的输出都是下一步的输入，任何一步偷懒，都要在后面的数据里还账。

1. **判噪声结构**：闭环稳定运行，采 y 与 u，对残差做白噪声检验，估出一阶自相关。
2. **量延迟 d**：用 $u \to y$ 的互相关找第一个显著峰，不要从拟合残差里"猜"。
3. **估 a, b, c**：先估 a, b，再单独估 c（理由见 §62.3.4）。
4. **解 Diophantine，算 F、G 与下界**：这一步是纯代数，手算都能做。
5. **选 MVC 或 GMV，做失配压力测试，上线**。

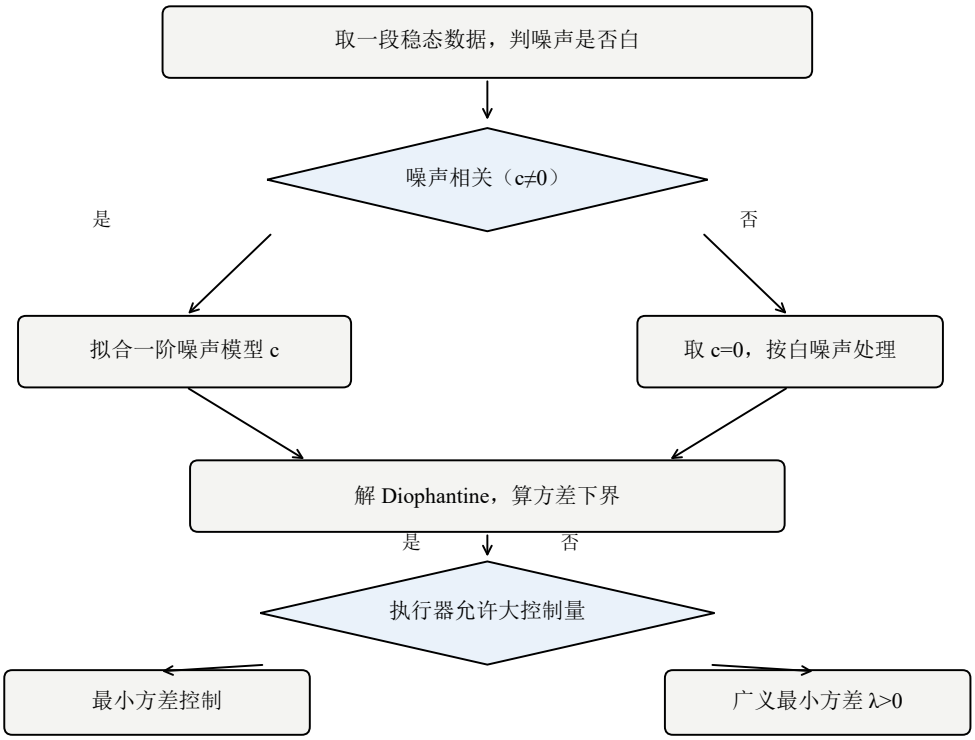


图 62-2　随机控制设计的五步流程

图 62-2 是这五步的框图。图中有两处**回头箭头**，比五个方框本身更重要：噪声检验不通过，要退回第一步重选模型阶次；压力测试不通过，要退回第三步改辨识方案（加激励、换数据段、延长采样），**而不是在第五步"再调调 λ"**。压力测试暴露的是模型问题，用控制器参数去补，只会把问题推到别的工况上。

62.3.2 第一步：确认噪声有结构

这一步决定"要不要做随机控制"。§62.1.2 已经给出理由：白噪声不可预测，任何控制器都只能被动承受，此时 MVC 与"不控制"的差别仅仅是把白噪声原样放大到执行器上。

可用的判据是一阶自相关。对扰动项 $e[k]+c\,e[k-1]$，

$$\rho_1=\frac{\mathrm{Cov}(\eta[k],\eta[k-1])}{\mathrm{Var}(\eta)}=\frac{c}{1+c^2} \tag{62-24}$$

$c=0$ 时 $\rho_1=0$；$c=0.6$ 时 $\rho_1=0.4412$；$c=0.9$ 时 $\rho_1=0.4972$。这个函数不是单调递增到 1 的——它在 $c=1$ 处取最大值 0.5，然后随 c 继续增大而下降。所以**一阶滑动平均的扰动，自相关最大只有 0.5**。现场若测出 $|\rho_1|$ 只有 $0.05\sim0.1$，即便模型完全正确，MVC 相对不控制也只能拿回百分之几的方差，这时把精力放到确定性回路上更划算。

具体做法：用现有的 PID 把回路稳住（第 44 章与第 106 章的方法足够），采一段 y、u，拟合 ARMAX 后对残差做 Ljung–Box 检验。残差是白的，说明模型把结构吃干净了；残差仍有显著自相关，说明阶次不够（需要二阶 A 或更高阶 C），退回第一步。

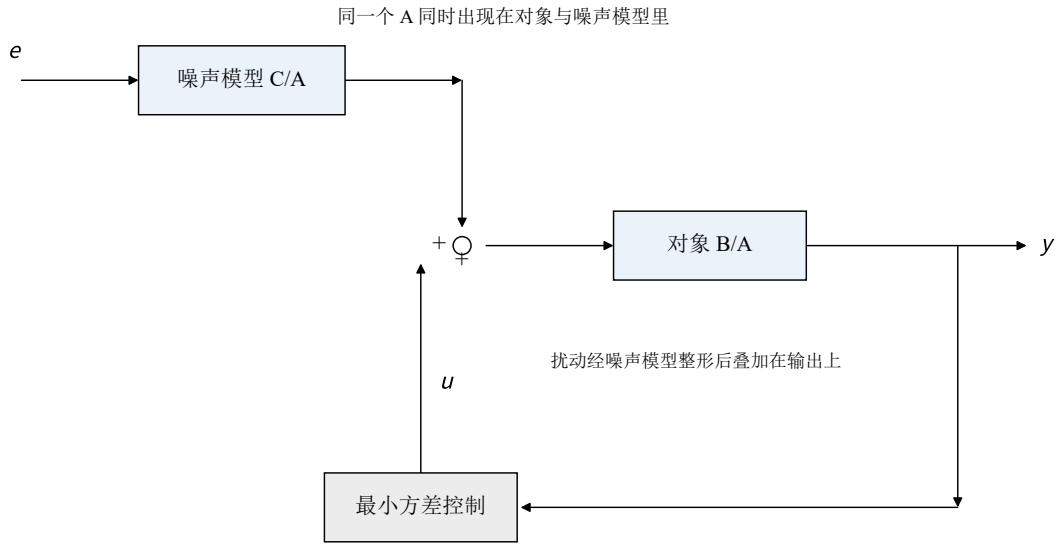


图 62-3　对象与噪声共用一个分母 A 的信号流

图 62-3 是模型的信号流。要注意图中两个模块**共用同一个 A**：对象是 B/A，噪声是 C/A。这个共用是 ARMAX 的核心假设——扰动与输入从同一个入口进入对象（输入扰动），而不是在输出端直接叠加（输出扰动）。现场若以测量噪声为主，正确的结构是 $y=Bu/A+Ce$，噪声模块没有分母，控制器也随之改变。入口判错，后面所有数字都错，而且错得很隐蔽：拟合残差照样很小。

62.3.3 第二步：延迟 d 单独量

延迟必须单独测，不能和 a,b 一起“拟合出来”。原因是下界对 d 极其敏感：d 从 1 到 2，下界从 1.0000 涨到 2.9600（表 62-1）。d 错一拍，误差量级和“什么都没做”差不多。

测法用互相关：给一段 PRBS 激励，算 $\mathrm{Cov}(u[k],y[k+\tau])$，第一个显著非零的 τ 就是 d。要注意三件事：

- **采样周期内的纯滞后要向上取整**。对象滞后 0.3 拍，等效 $d=1$，不能按 0 处理。
- **执行器死区与总线周期都算进 d**。阀门从收到信号到开始动，常常就有几十毫秒。
- **d 是“控制量到输出”的延迟，不是“设定值到输出”的延迟**。后者包含控制器计算与滤波，会把 d 估大。

两种估计错误的方向不同：**估小一拍**，控制器以为自己能影响 $y[k+1]$，实际不能，控制律对着已经过期的误差使劲，实测方差明显高于下界、控制量还更大；**估大一拍**，下界被高估，控制器过于保守，输出方差高于必要值但不会发散。前者危险，后者浪费。

62.3.4 第三步：估 a、b、c

这一步有个容易忽略的细节：**c 必须单独估，不能指望最小二乘顺带给出**。把式 (62-1) 写成回归形式 $y[k] = a\,y[k-1] + b\,u[k-d] + \eta[k]$，其中 $\eta[k] = e[k] + c\,e[k-1]$ 是自相关的。自相关残差与回归量 $y[k-1]$ 相关，普通最小二乘给出的 a 是有偏的，偏差方向取决于 c 的符号。

本章的做法分两步：先用工具变量法得到 a, b，再固定 a, b，把式 (62-6) 改写成

$$e[k] = y[k] - a\,y[k-1] - b\,u[k-d] - c\,e[k-1]$$

对 c 做标量递推最小二乘。实验 8 验证了这个思路可行：取真值 $c = 0, 0.3, 0.6, 0.9$，在 a, b 已知的条件下，1000 拍之后 c 的估计尾段均值为 $+0.0067$、$+0.3067$、$+0.6067$、$+0.9067$，标准差都只有 0.0023。四个真值全部命中，误差在千分位。

激励条件同样重要。闭环运行时 u 与 y 强相关，辨识容易“自圆其说”：数据拟合得很漂亮，参数却偏了。标准做法是把一段 PRBS 叠到设定值上（第 61 章给的 PRBS 图样可以直接复用），或者利用投运前的开环阶段。

62.3.5 第四步：算下界，并把它当验收标准

F、G 由式 (62-11)、式 (62-12) 递推，下界由式 (62-18) 求和。这三步全是代数，不需要仿真。表 62-2 是实验 1 的结果：左边三列是解析值，右边两列是 $N = 6000$ 拍的仿真实测（seed=3）。

d	$F(z^{-1})$	g_0	下界	实测方差	实测/下界	$\mathrm{var}(u)$
1	1	1.4000	1.0000	1.0077	1.008	7.9005
2	$1 + 1.4z^{-1}$	1.1200	2.9600	3.0272	1.023	5.0563
3	$1 + 1.4z^{-1} + 1.12z^{-2}$	0.8960	4.2144	4.3247	1.026	3.2360

表 62-2 有三处值得读：

第一，**实测与下界的比值在 1.008 到 1.026 之间**。这点差距来自预热段的残留与有限样本的统计涨落，不是模型误差。仿真能做到这个程度，说明推导与实现是一致的。

第二，**控制量方差随 d 增大而下降**：7.90、5.06、3.24。延迟越长，控制器“能管”的拍数越少，动作也越少。这是“管不了就别乱动”的量化版本，也是为什么长延迟回路反而看起来更“安静”——它不是在控制，是在放弃。

第三，**下界与 g_0 都只依赖 a, c, d，与 b 无关**。所以这两个数列可以当作对象的“固有属性”存档，换执行器、换增益都不用重算。

工程上把比值当验收指标：

$$R = \frac{\mathrm{Var}_{\mathrm{sim}}(y)}{\sigma_e^2 \sum_{i=0}^{d-1} f_i^2} \tag{62-25}$$

$R < 1.1$，模型与实现都对；$1.1 < R < 1.5$，先查 c 与 d；$R > 1.5$，不要再调参数了，说明结构错了——扰动入口不对、有非线性、或者参数在漂。

62.3.6 第五步：选 MVC 还是 GMV，再做压力测试

MVC 的控制代价在 §62.1.3 已经算过：输出标准差 1.01，控制标准差 2.82。多数现场不接受这个比例，所以要退到 GMV，用 λ 把控制量“定价”。

λ 的初值有一个现成的量级：取 $\lambda = b^2$，式 (62-21) 的分母从 b^2 变成 $2b^2$，相当于闭环增益减半。本章 $b = 0.5$，$b^2 = 0.25$，正好落在 §62.8 扫描表里 $\lambda = 0.25$ 那一档：输出方差 1.6201，控制方差 2.3692，两边比较均衡。这是一个很好的起点——不是最优，但一定不会离谱。

上线前必须做失配压力测试。表 62-3 是实验 5 的结果（$d = 1$）：

失配项（真实值相对模型）	输出方差	结论
标称	1.0157	与下界一致
$a \pm 10\%$	1.0282 / 1.0220	几乎无影响
$b + 20\%$	1.2976	可接受上限
$b + 30\%$	发散	**硬边界**
$b - 20\%$	1.1140	可接受
$b - 40\%$	1.3546	可接受
$b - 55\%$	1.6709	已明显恶化但不发散
$c \pm 50\%$	1.1704 / 1.1377	灵敏度中等

表 62-3 是**不对称的**，而且方向与直觉相反：真实增益比模型**大** 30% 就发散，真实增益比模型**小** 55% 还能撑住。原因在 §62.6.4 会算清楚——$d = 1$ 时闭环极点

$$\rho = a - (a + c)\,\frac{b}{\hat{b}} \tag{62-26}$$

真实增益大于模型（$b > \hat{b}$）时 ρ 向负方向走，到 $\rho = -1$ 即 $b/\hat{b} = (1 + a)/(a + c)$ 处发散。代入 $a = 0.8$、$c = 0.6$ 得 $b/\hat{b} = 1.2857$，也就是真实增益高出模型 28.6%——与表 62-3 里 $+30\%$ 那一行发散完全对得上。真实增益小于模型时 ρ 向正方向走，最多回到 $+a = 0.8$，永远在单位圆内。取 $c = 0$ 时同一条界线退化为 $b/\hat{b} = 1/a = 2.25$，即模型低估增益 55.6%，正是 §62.6.4 表 62-5 里 $\hat{b} = 0.2222$ 那一行。

这个式子还说明一件事：**噪声相关性越高，增益裕度越小**。c 从 0 增到 0.6，失稳界线从 2.25 压到 1.2857，可用裕度从 125% 掉到 29%。原因不神秘——c 越大，MVC 的等效增益 $(a + c)/\hat{b}$ 越大，同样的增益误差被放大得越狠。

所以设计时的规则是：**把 $\hat{b}$ 取大一点，宁可高估增益**。按辨识值的 1.2 倍设计，代价是控制量偏小、输出方差略高于下界，但换来了对“真实增益比预期大”这一最常见失配的免疫力。这条规则和确定性控制里“裕度方向随意”的直觉相反，是随机控制特有的：它不是因为保守才安全，而是因为**失稳只发生在环路增益偏大的一侧**。

五步里最容易被省掉的是第一步和第五步，而这两步恰恰决定“要不要做”和“能不能上线”。中间的代数再漂亮，也补不上这两步的缺口。

62.4 连续情形

62.4.1 连续对象与它的随机扰动

真实对象是连续的，采样是后加的。所以有必要把 §62.2 的离散结论放回连续域看一眼：不是为了在连续域设计控制器，而是为了判断“离散下界”里有多少是物理的、有多少是采样造出来的。

连续域的对象写成随机微分方程。以一阶惯性加纯延迟为例：

$$\dot{y}(t) = -p\,y(t) + u(t-\tau) + \xi(t) \tag{62-27}$$

p 是对象极点（时间常数 $1/p$），τ 是纯延迟，$\xi(t)$ 是零均值的连续白噪声，强度为 q——含义是 $\mathrm{Var}\left(\int_t^{t+\Delta}\xi\,\mathrm{d}s\right) = q\Delta$。

注意式 (62-27) 里扰动与输入**从同一个入口进入**，这是 §62.3.2 讲的输入扰动假设在连续域的写法。若扰动是测量噪声，它就加在 y 的测量值上，不经过对象动态，结论完全不同。

连续白噪声在物理上不存在（它的方差无穷大），工程上真实的扰动总有一个相关时间 θ，常用一阶成形滤波器描述：

$$\dot{\xi} = -\frac{1}{\theta}\xi + \frac{\sigma_w}{\theta}w \tag{62-28}$$

w 是单位强度的白噪声。这个过程的稳态方差是 $\sigma_w^2/(2\theta)$，自相关是 $e^{-|\Delta|/\theta}$。$\theta \to 0$ 时它退化为白噪声，θ 大时它是一条缓慢漂移。

这个 θ 就是离散模型里 c 的物理来源。 连续域只有一个相关时间 θ，离散域却把它压缩成一个标量 c，压缩过程中丢掉了“相关函数不是指数形”的部分。§62.4.4 会看到这个压缩的代价。

62.4.2 连续域的最小方差下界

在 t 时刻要预测 $y(t+\tau)$。把式 (62-27) 积分：

$$y(t+\tau) = e^{-p\tau}y(t) + \int_0^{\tau} e^{-p(\tau-s)}\left[u(t+s-\tau) + \xi(t+s)\right]\mathrm{d}s \tag{62-29}$$

右端三项的性质完全不同：

- $e^{-p\tau}y(t)$ 是已知的。
- $u(t+s-\tau)$ 在 $s<\tau$ 时对应的都是**已经发出的控制量**（$t+s-\tau<t$），全部已知，可以拿来抵消。
- $\xi(t+s)$，$s\in(0,\tau)$，是未来才发生的扰动，**完全不可预测**。

所以 MVC 只能把前两项凑掉，剩下的方差就是下界：

$$V_c = \int_0^{\tau} e^{-2p(\tau-s)}\,q\,\mathrm{d}s = \frac{q}{2p}\left(1 - e^{-2p\tau}\right) \tag{62-30}$$

这个式子有两个极限，都很有说服力。

$\tau \to 0$：$V_c \to 0$。没有延迟时，连续域的 MVC 可以把白噪声完全抵消——代价是无穷大的增益。这解释了为什么连续域的结论不能直接当控制器用：它要求执行器带宽无穷大。

$p\tau \to \infty$：$V_c \to q/(2p)$，正好是 y 的开环稳态方差。延迟长到把扰动全部放过去，控制器一点忙都帮不上，输出方差回到不控制时的值。这与离散域 $d\to\infty$ 时下界发散是同一件事的两种写法（离散域里 d 有限但 f_k

不衰减，连续域里 f 衰减但积分区间拉长）。

中间那一段才是控制真正起作用的地方。把式 (62-30) 与开环方差相除，得到一个无量纲的“可抵消比例”：

$$\frac{V_c}{q/(2p)} = 1 - e^{-2p\tau} \tag{62-31}$$

$p\tau = 0.5$ 时能抵消 63.2%，$p\tau = 1$ 时 86.5%，$p\tau = 2$ 时 98.2%。**延迟相对于对象时间常数越小，随机控制越有效**。这条判据比离散域的“d 等于几”更本质，因为它把采样周期约掉了。

62.4.3 离散下界是连续下界的精确离散化

式 (62-18) 与式 (62-30) 不是两个互不相干的结论。取零阶保持采样，T_s 为采样周期，则

$$a = e^{-pT_s}, \qquad \sigma_e^2 = \frac{q}{2p}\left(1 - e^{-2pT_s}\right)$$

σ_e^2 的这个表达式来自“一拍内进入的扰动经过对象后的稳态贡献”，不是随手写的。把它代入式 (62-18)，取 $a = 0.8$（即 $pT_s = 0.2231$）、延迟两拍（$\tau = 2T_s$），得到

$$V_d = \sigma_e^2(1 + a^2) = 0.8066\,qT_s \times 1.64 = 1.3229\,qT_s$$

而式 (62-30) 给出

$$V_c = \frac{qT_s}{2 \times 0.2231}\left(1 - e^{-4\times 0.2231}\right) = 1.3229\,qT_s$$

两者小数点后四位一致。**离散下界不是连续下界的近似，而是它的精确离散化**——只要 σ_e^2 按上式取值。

这个核对有一个直接用途：它说明**离散域算出的“方差”带着采样周期的量纲**，换采样率就不能直接比。表 62-4 是同一个物理回路在三种采样率下的结果，全部折算到 qT_s 为单位。

采样周期	a	d	$\sigma_e^2/(qT_s)$	离散下界$/(qT_s)$	连续下界$/(qT_s)$
$T_s/2$	0.8944	4	0.4481	1.3230	1.3229
T_s	0.8000	2	0.8066	1.3229	1.3229
$2T_s$	0.6400	1	1.3229	2.6458	1.3229

表 62-4 里两件事同时发生：采样变慢，a 变小、d 变小；采样变快，a 变大、d 变大。**两者的效果在离散下界上刚好抵消**（前两行都落在 1.323 附近），但**最后一行的数字翻了一倍**。

翻倍不是物理变化，是单位变化。$2T_s$ 采样时每一拍的噪声方差是原来的两倍，所以“每拍方差”这个数字自然翻倍；折算成“每秒方差”就完全一样。这就是为什么工程上比较不同采样率的回路，必须先把方差除以采样周期，或者干脆用连续下界当基准。

62.4.4 c 不是对象的物理属性

连续域只有一个相关时间 θ，离散域把它压成 c，而式 (62-24) 给出了压缩的代价：$\rho_1 = c/(1 + c^2)$ 的最大值只有 0.5。

这句话的分量比看起来重。真实扰动的自相关是 $e^{-\Delta/\theta}$，只要 $\theta \gg T_s$，ρ_1 就接近 1。**而 MA(1) 根本无法表达 $\rho_1 > 0.5$ 的扰动**。所以：

- 若现场测出 $\rho_1 \approx 0.45$，用 MA(1) 拟合是合理的，反解 $c = \left(1 - \sqrt{1 - 4\rho_1^2}\right)/(2\rho_1)$ 得 $c \approx 0.6$，与本章标称值一致。
- 若现场测出 $\rho_1 = 0.9$，说明扰动是慢漂移，MA(1) 拟合出来的 c 会被强行压到 1 附近，残差仍有强自相关。这时正确的做法是提高 C 的阶次，或者改用“积分型扰动”（把 $1/(1 - z^{-1})$ 并进噪声模型）。

同理，同一个物理回路，采样周期变了，c 也会变：采样变快，相邻拍的相关性变强，c 变大；采样慢到与 θ 同量级，c 变小。**c 是“采样后的噪声形状”，不是“噪声本身”。** 这一点在跨设备移植参数时最容易出错：从 1 s 采样的老系统搬一个 $c = 0.6$ 到 100 ms 采样的新系统，噪声形状完全不对。

62.4.5 为什么工程上仍然在离散域设计

既然连续域的下界更本质，为什么实际的随机控制器都在离散域设计？三条理由：

第一，**噪声的微分不可测**。式 (62-27) 里的 $\xi(t)$ 是白噪声，任何实际传感器都测不到它，连续域辨识只能辨识成形滤波器 (62-28) 的参数，而这一步在数值上很脆弱。

第二，**执行与计算天然离散**。D/A 输出零阶保持，控制器按拍计算，延迟天然是整数拍。在离散域里 d 是一个可数的量，在连续域里 τ 是一个要辨识的实数——前者可以核对，后者只能估计。

第三，**离散域的结论可以直接写成代码**。式 (62-14) 就是三行差分方程，式 (62-30) 还要做积分。

所以连续域的定位很清楚：**它是标尺的标尺**。用它算绝对下界、判断“这件事值不值得做”、判断采样周期选得够不够；真正落地的控制器仍然在离散域。这与第 47 章 Smith 预估的处境类似——连续域的纯延迟处理给出了清晰的物理图像，工程实现却总在离散域完成（第 107 章）。

一个实用的判据：若 $pT_s > 0.5$（一拍之内对象已经走完大半个行程），离散化误差会明显，d 的整数化也会带来半拍到一拍的不确定性，这时应当先把采样周期减半，再谈控制器设计。

62.4.6 最优控制器里为什么没有 I

从 §62.2 到 §62.5，本章给出的每一条控制律里都没有积分环节：式 (62-14) 是有限记忆的代数式，式 (62-21) 只罚当拍 u，式 (62-33) 只用 y 与 u 的有限历史。这对习惯了 PID 的读者是个不小的冲击——**没有积分，怎么保证没有静差？**

答案是：**这个问题的前提在本章不成立。** 本章的扰动模型是零均值的 $C/A \cdot e$，它的稳态均值就是 0。既然扰动本身不带直流分量，闭环均值自然也是 0，不需要积分去“追”。积分环节解决的是“持续单向的偏差”（负载偏置、模型直流增益错、阀门零点漂），这类偏差在本章的随机框架里根本没有被建模。

那如果现场确实有偏置怎么办？三条路，都不该是“往随机控制器里塞积分”：

第一条，前馈。 偏置若可测（进料量、环境温度），按 §62.9.5 直接扣掉。前馈不影响闭环动态，也不改变 λ 的含义。

第二条，外环慢积分。 在 GMV 外面套一个时间常数远大于对象时间常数的积分器，它只处理直流分量，对主频段的方差几乎没有影响。这是串级控制的标准做法，也是第 44 章讲过的结构。

第三条，把偏置并进噪声模型。 用积分型扰动 $1/(1 - z^{-1})$ 替代 C/A，让控制器自己去预测并抵消它。这条最“优雅”，但它改变了整个设计——d 步预测里会多出一个斜坡项，下界的算法也要重写。

直接把 I 加进 GMV 的代价可以算出来。 积分带来的相位滞后必须用降增益来换裕度。$d = 1$ 时控制律是 $u = -ky$，$k = 2.8/(1 + \tilde{\lambda})$。若为了容纳积分把 k 从 2.8 降到 2.0（相当于 $\tilde{\lambda}$ 从 0 增到 0.4），查表 62-8，输出

方差从 1.0000 涨到 1.1828——**+18%**。这 18% 就是“买静差”付的钱，而它买到的那点静差，用前馈或外环积分几乎免费就能拿到。

所以本章的立场是：**随机控制器负责“抖不抖”，积分负责“准不准”，两件事分开做。** 把它们揉在一起，λ 就不再是“控制量的价格”，而变成了“性能、静差、裕度”三者的混合物——那时它既调不动，也说不清。

62.5 离散情形

62.5.1 三种等价的实现形式

同一套 MVC，写成代码有三种形式，选错一种会在现场多花很多时间排查。

新息形式。 直接按 §62.1.3 的思路，先算预测、再令其为零：

$$\hat{y}[k+d \mid k] = \frac{BF}{C}u[k] + \frac{G}{C}y[k], \qquad u[k] = -\frac{\hat{y}[k+d \mid k]}{b} \tag{62-32}$$

它最贴近物理：控制器每一步都在回答“如果不动，d 拍后输出会是多少”。代价是要维护新息 e 的递推（式 62-6），而 e 是算出来的中间量，模型误差会在它身上累积。

多项式形式。 把式 (62-32) 展开、消去 e，得到式 (62-14)：

$$u[k] = -\frac{1}{b}\left(g_0\, y[k] + \sum_{i=1}^{d-1} b\, f_i\, u[k-i] \right) \tag{62-33}$$

它只用实测的 y 和已发出的 u，没有任何估计量。**工程上应当默认用这一种**：y 的量测噪声不会被二次放大，模型偏差只影响增益，不会形成累积漂移。

状态空间形式。 把 y 与 u 的历史堆成状态向量，$u = -K\hat{x}$。它适合多变量、带约束、要接 MPC 的场合（第 58 章与第 80 章）。单变量一阶对象用状态空间是杀鸡用牛刀，但它的好处是“延迟用移位寄存器表示”，代码结构最干净。

形式	需要的量	对模型误差的敏感度	适用场合
新息形式	y、e 递推	高（c 错则 e 漂）	教学、诊断、需要在线看预测值时
多项式形式	y、u 历史	中（只影响增益）	单回路现场默认
状态空间	状态向量	中	多变量、带约束、接 MPC

62.5.2 时序：一拍里发生了什么

一个容易忽略的事实：**控制器的计算时间也算延迟**。若在 k 拍采到 $y[k]$，算完 $u[k]$ 花了 0.6 个采样周期，那么 $u[k]$ 实际要到 $k+1$ 拍中段才出去，等效 d 增加了 1。诊断特征很明确：实测方差明显高于下界（$R > 1.1$），而 $u \to y$ 的互相关峰位比设计值晚一拍。

正确的时序是：

1. 定时中断触发，ADC 读 $y[k]$；
2. 用式 (62-33) 算 $u[k]$（只有 d 次乘加）；
3. 限幅，DAC 输出，**同时把限幅后的值存进历史缓冲**；

4. 退出中断，等下一拍。

第 3 步里“存限幅后的值”是个关键细节。若存的是限幅前的期望值，控制器会以为执行器做了一个它其实没做的动作，下一拍的预测就错，而且错得系统化——偏差会一直累积到限幅解除。这类故障在现场表现为“一饱和就发散，退出来也不恢复”。

计算量方面，一阶对象加 d 拍延迟，每拍需要 $d+1$ 次乘法、d 次加法。$d=3$ 时不到十次浮点运算，任何 8 位单片机都够。**随机控制的成本从来不在算力上，在模型上。**

62.5.3 实测波形：输出贴着下界线走

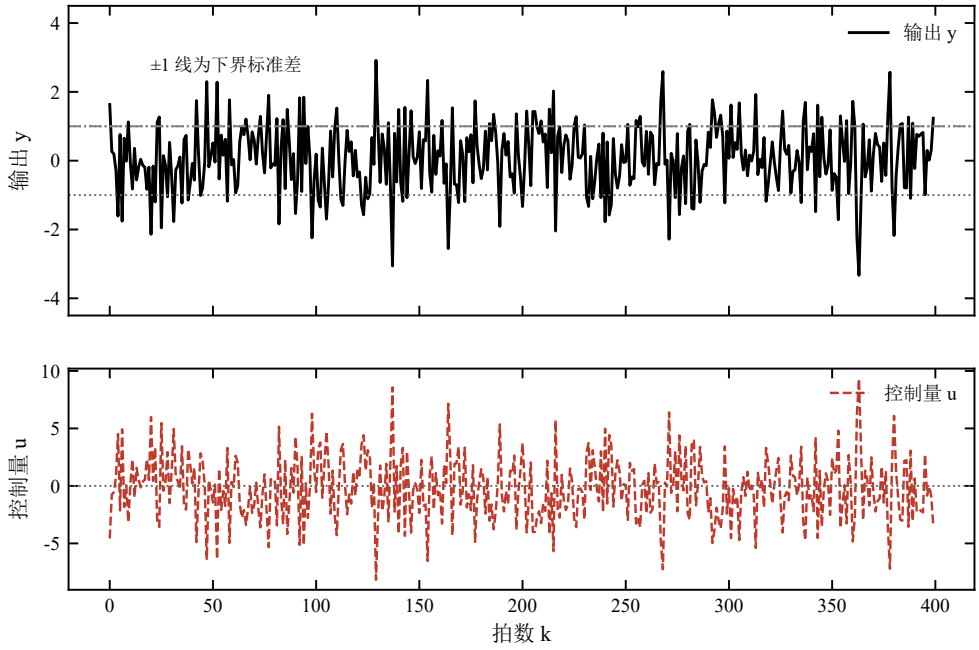


图 62-4　MVC 闭环输出与控制量，虚线为 ±1 下界线

图 62-4 是本轮仿真的实测波形（$d=1$、$N=6000$、seed=5）。上半图是 y，两条虚线是 ± 1——也就是 $\pm\sigma_e$，最小方差下界对应的标准差。可以看到 y 的大部分时间都夹在这两条线之间，偶尔冲出去到 ± 2 附近：这不是控制失败，而是白噪声的固有分布，正态分布的 3σ 事件本来就会出现。

下半图是 u。它的幅度明显大于 y，而且**动作频繁、方向来回切换**。这就是 §62.1.3 说的代价在时域里的样子：输出每抖一点，控制量就反着抖 $1/b$ 倍。实测统计：$\mathrm{var}(y)=1.0119$、$\mathrm{std}(y)=1.0059$、$\max|y|=3.567$、$\mathrm{var}(u)=7.9334$，控制量的总变差（相邻拍差值绝对值之和）达 18251.5。

作为对照，同一段噪声下：

- 固定比例 $k=1.6$（约为 MVC 增益 2.8 的六成）：$\mathrm{var}(y)=1.3978$，控制变差 9208.8；
- 固定比例 $k=0.5$：$\mathrm{var}(y)=2.9993$，控制变差 2771.6；
- 不控制：$\mathrm{var}(y)=6.7812$，控制变差 0。

三个对照说明一件事：**方差与控制变差之间是一条陡峭的交换曲线**。从“不控制”到 $k=0.5$，方差降了 56%，控制变差只花了 2772；再往前走到 MVC，方差再降 28%，控制变差却要再花 15500。**收益递减，代价递增，拐点在 $k\approx 1.5\sim 2$ 附近**——这正是 §62.8 用 λ 要瞄准的区间。

62.5.4 初始化与预热

式 (62-6) 的新息递推是有状态的。上电时 $e[-1]$ 未知，$y[-1]$ 也未知，只能置零。这带来两个后果：

第一，**前 d 拍控制量无效**。d 拍之前，对象还没收到任何控制作用，输出完全由初值和噪声决定。若初值不为零（比如从手动切换过来），这段的输出会很大。

第二，**方差统计要丢掉预热段**。本章所有仿真的 `WARM=200`，即前 200 拍不计入统计。这个数字不是随便定的：$a=0.8$ 时初值的影响按 0.8^k 衰减，200 拍后是 $0.8^{200}\approx 4\times10^{-20}$，完全可以忽略；$a=0.99$ 的慢对象则需要 2000 拍以上。

工程上的做法不同：不是“等它自己收敛”，而是**在切换瞬间把历史填满**。从手动切自动时，把 u 的历史缓冲全部填成当前手动输出值，把 y 的历史填成当前测量值，e 的历史清零。这样控制器一投入就在一个自洽的状态上，不会因为“历史是零、现实不是零”而产生一个大的初始动作。切换冲击是随机控制器投运时最常见的问题，原因几乎总是这一条。

62.5.5 数值细节

三个容易踩的坑，都与“方差很小”有关。

环形缓冲。 延迟用长度为 d 的环形缓冲实现，索引 $k \bmod d$，不要用数组搬移。搬移在 d 大时会占掉大部分中断时间，而且容易写出差一错误。

量化噪声。 当输出方差被压到很小（比如 10^{-3} 量级），12 位 ADC 的量化噪声（相对满量程 $1/4096$）就可能成为主导。这时继续调控制器没有意义——下界已经不是噪声下界，而是量测下界。判断方法：把传感器短接，看读数的方差；若它与闭环方差同量级，问题在量测不在控制。

单精度。 式 (62-33) 的系数 g_0、bf_i 可能相差几个数量级（d 大、a 小时），单精度浮点的舍入误差会在 d 次累加后显现。$d\le 5$ 时通常无碍，d 更大或系数跨度超过 10^4 时改用双精度，或者把式子改写成“先归一化再累加”。

62.5.6 从框图到代码的对应

本章的 `code/ch062_sim.py` 里，`MVC` 类只有三十行，结构就是 §62.5.1 的多项式形式：

- `__init__` 里把 a,b,c,d 存下来，按式 (62-11)、(62-12) 预计算 F、G，并预计算控制律的系数；
- `reset()` 清空 u 的历史缓冲与输出记录；
- `step(y)` 按式 (62-33) 算 u，把 u 推入缓冲，返回。

三个方法之外没有任何东西。这个规模和“三行差分方程”的说法是一致的：**随机控制的理论很重，实现很轻**。真正决定成败的是参数从哪来、模型对不对，而不是代码写得多巧。

对照 §62.12 的完整代码可以看到，脚本里真正占篇幅的是仿真循环、指标计算与失配扫描，控制器本身反而是最短的一段。这个比例是有意为之——它是本章立场的代码版本。

62.5.7 定时抖动：当 d 不再是一个整数

本章的 d 被当成一个确定整数：这一拍发出的 u，固定在第 d 拍到达输出。真实系统的定时没有这么整齐：PLC 扫描周期在负载重时被拉长，通信总线偶发重传，多任务调度让控制任务晚启动几毫秒。结果是**同一个 u，有时早一拍到、有时晚一拍到**。

这等价于把 d 变成一个随机变量，而后果不是“平均差一点”，是**可能直接失稳**。用本章标称参数算一下：按 $d=1$ 设计的控制律是 $u=-2.8y$，若某一拍这个 u 晚了一拍才到，闭环变成

$$y[k]=0.8\,y[k-1]-1.4\,y[k-2]+e[k]+0.6\,e[k-1]$$

特征方程 $z^2-0.8z+1.4=0$ 的根是 $0.4\pm 1.114\mathrm{j}$，模长 1.183。**出了单位圆。** 也就是说，一条按 $d=1$ 设计的回路，只要延迟偶尔变成 2，就有可能从“最优”直接跌到“发散”——中间没有“性能略降”这个缓冲档。这与 §62.6.4 说的“ρ 靠近 -1 时方差爆炸”是同一个机理，只是这里多了一个延迟项，越界来得更突然。

三条对策，按成本从低到高：

第一，在 d 里预留半拍。 若抖动幅度约为半个采样周期，按 $d+1$ 设计。代价是保守（表 62-1：d 从 1 到 2，下界涨近三倍），但比“偶尔用错一拍”安全。这条只适合抖动幅度已知且稳定的场合。

第二，用时间戳判断，而不是用“这一拍”。 每个采样值带上采样时刻，控制器按“从采样到执行经过了多少拍”选控制律的延迟项，而不是假设固定 d。这需要控制器内部维护两套系数（对应 d 与 $d+1$），实现上多几十行，但把抖动从“模型误差”变成了“已知的切换”。

第三，从根上消除抖动。 提高采样率让抖动的相对幅度变小，或者把控制任务放到独立的实时核上。这一条最贵，但它是唯一治本的——前两条都只是把抖动的影响压到可接受。

判断要不要做：算抖动幅度与采样周期之比。小于 5% 可以忽略；5%～20% 用前两条；超过 20% 说明这个平台不适合跑随机控制——因为 §62.6.3 的增益悬崖只留给 28.6% 的余量，而抖动一项就可能吃掉大半。

62.6 稳定性与性能

62.6.1 三条曲线，一个结论

这一节用四组实测回答一个问题：**MVC 的性能由什么决定？** 答案不是“控制器写得好不好”，而是“模型对不对、延迟是多少、参数会不会变”。四组实验分别对应延迟、增益失配、闭环极点、参数跳变，它们指向同一个结论。

所有实验共用标称参数 $a=0.8$、$b=0.5$、$c=0.6$、$\sigma_e=1$、$d=1$，除非另行说明。控制器按标称参数设计，对象参数在仿真里单独改动——这正是现场的情形：控制器只能按它以为的模型算，真实对象不受它控制。

62.6.2 延迟：下界随 d 的增长

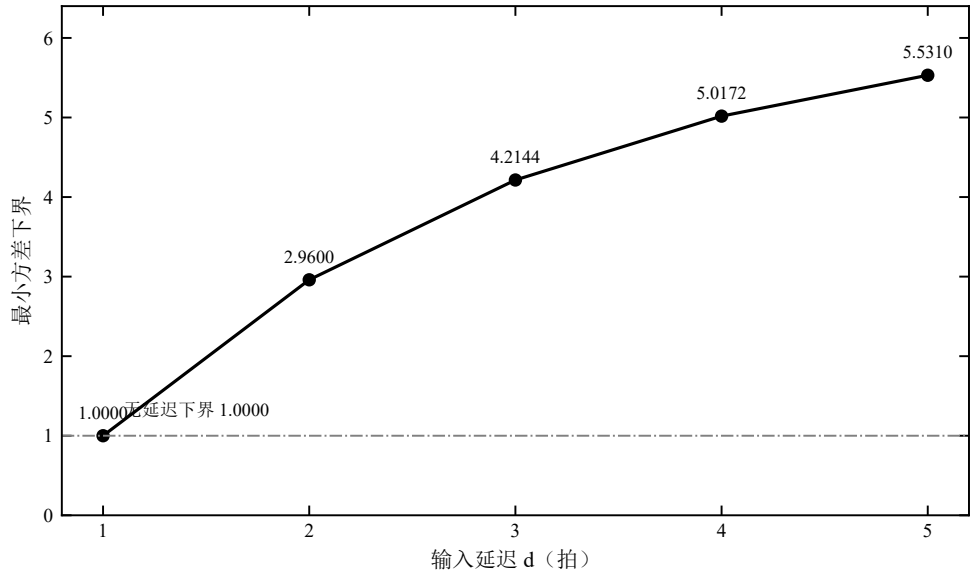


图 62-5　延迟 d 对最小方差下界的影响，数字为下界值

图 62-5 是表 62-1 的曲线版，横轴 d 从 1 到 5，纵轴是下界。五个点是 1.0000、2.9600、4.2144、5.0172、5.5310。补上后两个：$d=4$ 时 $g_0=0.7168$、$d=5$ 时 $g_0=0.5734$，两个值都由 $g_0=af_{d-1}+c\,\delta_{d,1}$ 算出，与仿真无关。

曲线的形状是“先陡后平”，这是本章最值得记住的一张图。增量依次是 1.9600、1.2544、0.8028、0.5138，比值恰好是 $a=0.8$（增量比），也就是每次乘 0.64。§62.2.3 已经解释过机理：a 决定对象“记住”过去多久，越早的扰动越容易被对象自己消化。

工程含义有两层。第一层是**第一拍延迟最贵**：把 d 从 2 降到 1，下界从 2.96 降到 1.00，收益 66%；把 d 从 5 降到 4，收益只有 9.3%。第二层是**延迟的测量精度要求**：既然第一拍最贵，那么把 d 从 1 误判成 2（或反之）就是灾难性的，而把 4 误判成 5 只是小损失。这与 §62.3.3 说的“估小危险、估大浪费”是同一件事的定量版本。

还有一个反直觉的现象：**d 越大，控制量方差越小**（表 62-2：7.90、5.06、3.24）。延迟越长，控制器能干预的拍数越少，动作自然越少。所以“这个回路控制量很安静”不等于“控制得好”，可能只是“它管不了”。

62.6.3 增益失配：一条有硬边界的曲线

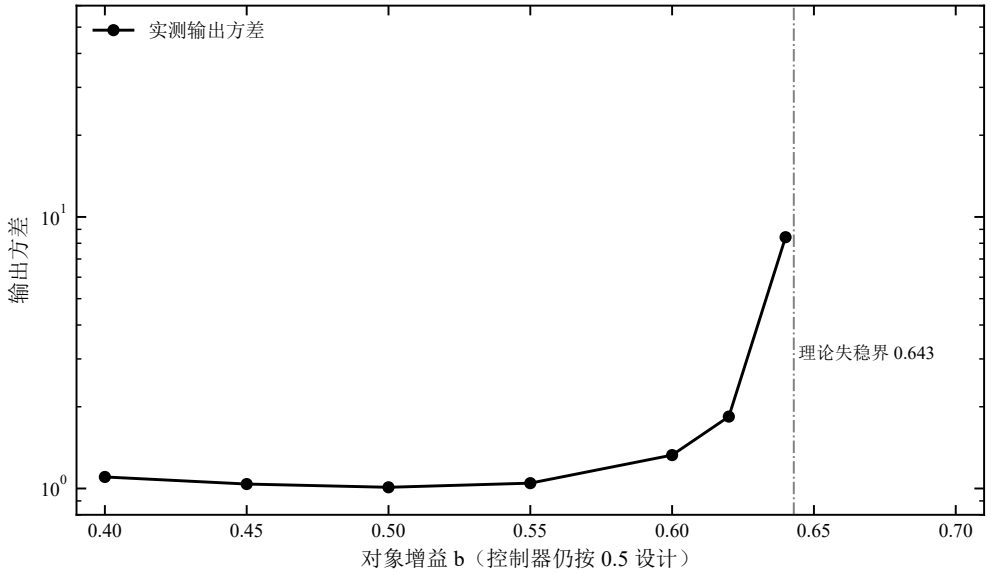


图 62-6　对象增益偏离模型对输出方差的影响，竖线为失稳界

图 62-6 的横轴是真实增益 b 相对模型值 0.5 的偏离，纵轴是输出方差。曲线在左侧平缓、在右侧陡然上升，然后断掉——断点就是发散。

数据来自实验 5（$N = 6000$、seed=11）：

- 真实 $b = 0.60$（$+20\%$）：方差 1.2976，是下界的 1.30 倍；
- 真实 $b = 0.65$（$+30\%$）：发散；
- 真实 $b = 0.40$（-20%）：1.1140；
- 真实 $b = 0.30$（-40%）：1.3546；
- 真实 $b = 0.225$（-55%）：1.6709，仍然收敛。

同一张图上，$+30\%$ 是悬崖，-55% 只是斜坡。 §62.3.6 已经给出机理：闭环极点 $\rho = a - (a + c)b/\hat{b}$，真实增益偏大使 ρ 向 -1 走，偏小则向 $+a$ 走。$c = 0.6$ 时界线在 $b/\hat{b} = 1.2857$，与实测的 $+30\%$ 发散完全吻合。

这条曲线还解释了现场一个常见困惑：**为什么同一套参数在 A 设备上很好，到 B 设备上就振荡？** 如果 B 设备的执行器增益比 A 大（阀门口径大一号、变频器上限高一点、液压缸面积大），那就正好落在悬崖一侧。此时正确的动作是把控制器里的 b 改大（相当于把增益标定对），而不是“把控制器调软一点”——后者会连带把正常工况的性能一起牺牲掉。

62.6.4 闭环极点：失稳是怎么发生的

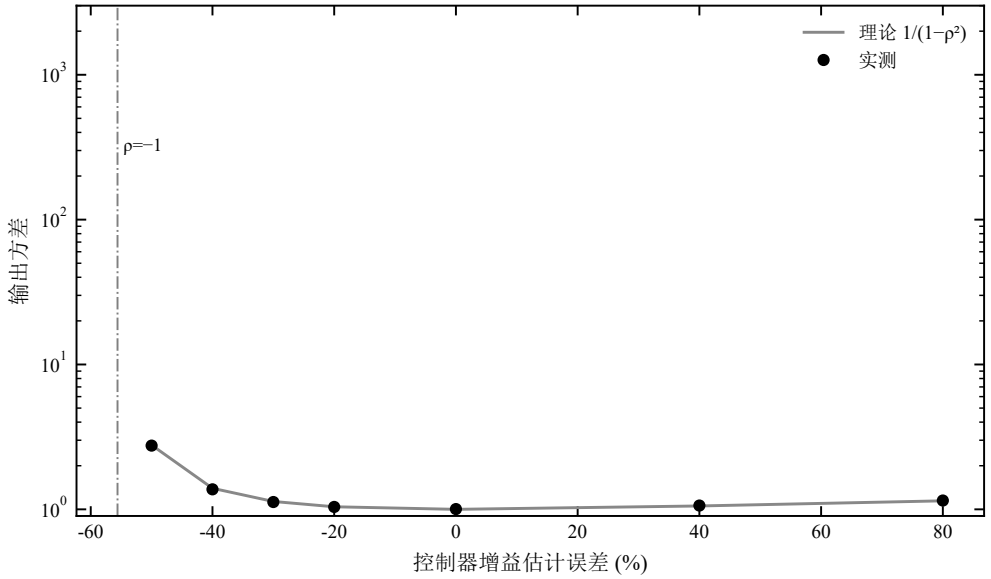


图 62-7　增益估计误差与输出方差，理论曲线与实测点，竖线为 rho=-1

图 62-7 把失稳过程画了出来。为了能解析对照，这组实验取 $c=0$，此时闭环极点与方差都有闭式：

$$\rho = a\,\frac{\hat{b}-b}{\hat{b}}, \qquad \mathrm{Var}(y) = \frac{\sigma_e^2}{1-\rho^2} \tag{62-34}$$

控制器用 $\hat{b}$ 设计，真实增益仍是 $b=0.5$。表 62-5 是实验 6 的完整结果（$N=4000$、seed=13）。

$\hat{b}$	相对真值	ρ	理论方差	实测方差
0.90	$+80\%$	$+0.3556$	1.1447	1.1503
0.70	$+40\%$	$+0.2286$	1.0551	1.0623
0.50	0	0	1.0000	1.0041
0.40	-20%	-0.2000	1.0417	1.0389
0.35	-30%	-0.3429	1.1332	1.1233
0.30	-40%	-0.5333	1.3975	1.3745
0.25	-50%	-0.8000	2.7778	2.7538
0.2222	-55.6%	-1.0002	∞	18291.0
0.20	-60%	-1.2000	∞	发散

表 62-5 有三处值得细看。

第一，**理论与实测在整张表上吻合到 1% 以内**（除临界点外）。$\rho=-0.5333$ 时理论 1.3975、实测 1.3745；$\rho=-0.8$ 时理论 2.7778、实测 2.7538。这说明式 (62-34) 抓住了主要机理，不是事后拟合。

第二，**ρ 靠近 -1 时方差按 $1/(1-\rho^2)$ 爆炸**。$\rho=-0.8$ 时方差已是 2.78，$\rho=-0.95$ 时会到 10.3。所以“还没发散”不等于“还能用”：$\hat{b}=0.25$ 这一档虽然收敛，方差是下界的 2.78 倍，相对不控制（6.78）只拿回了 69% 的可降空间，剩下 31% 被增益误差吃掉了。

第三，**临界点 $\hat{b}=0.2222$ 处实测 18291**。这个数字没有物理意义，它只是“还没溢出”的最后一刻。真正该记的是界线本身：$\hat{b}/b=0.444$，即**模型低估增益超过 55.6% 就失稳**。这与图 62-6 里“真实增益高出 28.6% 就失稳”是同一件事——前者按模型说，后者按真实值说，互为倒数关系。

把式 (62-34) 换成设计语言：**MVC 的稳定裕度完全由增益误差决定，且只在一侧有边界。** a 的误差以差值形式进入 ρ：a 偏 10%（0.08）只让 ρ 偏移 0.08，方差从 1.0000 变到约 1.006，几乎无感；增益误差则以比例形式放大，直接越界。这就是为什么 §62.3.4 强调“c 要单独估、b 要认真估”，而 a 的精度可以放宽。

62.6.5 参数跳变：按哪个模式设计

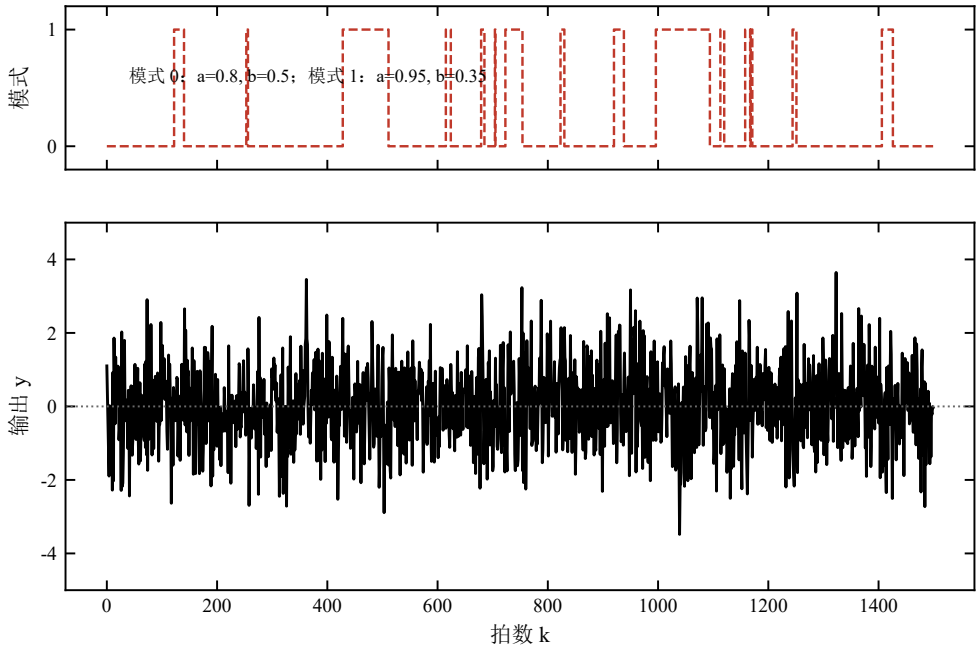


图 62-8　马尔可夫跳变下的模式序列与输出

现场还有一类更麻烦的失配：参数不是“偏了”，而是“在两个值之间跳”。比如两套泵轮流投运、换热器结垢与清洗周期、负载在轻载与重载之间切换。这类系统适合用马尔可夫跳变模型（§62.2.8）。

实验 7 设两模式：模式 0 为 $(a,b,c)=(0.8,0.5,0.6)$，模式 1 为 $(0.95,0.35,0.6)$，转移矩阵

$$P=\begin{bmatrix}0.98 & 0.02\\ 0.05 & 0.95\end{bmatrix}$$

即模式 0 平均停留 50 拍，模式 1 平均停留 20 拍。图 62-8 上半是模式序列，下半是输出。四种设计口径的结果在表 62-6（$N=8000$、seed=17）。

设计口径	输出方差	说明
按模式 0 设计，真实系统跳变	1.0946	可用
按模式 0 设计，只在模式 0 运行	0.9939	最优
按模式 0 设计，只在模式 1 运行	1.3278	变差但稳定
按模式 1 设计，真实系统跳变	发散	灾难

表 62-6 的不对称性可以用一个式子算出来。用模式 i 的参数设计、在模式 j 上运行，闭环极点是

$$\rho_{ij} = a_j - (a_i + c)\,\frac{b_j}{b_i} \tag{62-35}$$

代入：$\rho_{01} = 0.95 - 1.4 \times 0.35/0.5 = 0.95 - 0.98 = -0.03$，几乎落在原点，所以“按模式 0 设计、在模式 1 上跑”勉强稳定（实测 1.3278，多出来的部分是切换瞬态）；$\rho_{10} = 0.8 - 1.55 \times 0.5/0.35 = 0.8 - 2.214 = -1.414$，直接出界，所以“按模式 1 设计”必然发散。

机理很清楚：**模式 1 的对象增益小、惯性大，按它设计出来的控制器增益特别高**（$(0.95 + 0.6)/0.35 = 4.43$，是模式 0 的 3.2 倍）。这个高增益一旦作用在增益更大的模式 0 上，环路增益就超过 2。

这条结论的工程版本是：**跳变系统要按“增益最大的模式”设计控制器，而不是按“最常出现的模式”或“平均参数”。** 按平均参数设计在这类系统上几乎总是错的，因为稳定性由最坏模式决定，性能才由平均模式决定。

62.6.6 从性能到稳定的三条判据

把 §62.6.2 到 §62.6.5 的结论压成三条可以贴在墙上的判据：

1. $R = \mathrm{Var}_{\mathrm{sim}}\,/V_{\min}$ **落在 0.9～1.1**：模型对、实现对。超出就先查 c 和 d，再查计算延迟。
2. **增益裕度按“真实增益最多比模型大 28.6%”核算**（$c = 0.6$ 时）。现场若能标定执行器增益，就把这个裕度留出来；不能标定就按 1.2 倍设计。
3. **跳变系统按最坏模式核算极点**，用式 (62-35) 逐个模式对算一遍，全部落在单位圆内才允许投运。

三条判据都不需要在线辨识，只需要一张表和一个计算器。它们的价值在于：**把“稳不稳”从经验判断变成了可以事先算出的数。**

62.6.7 稳定性不能只跑一次

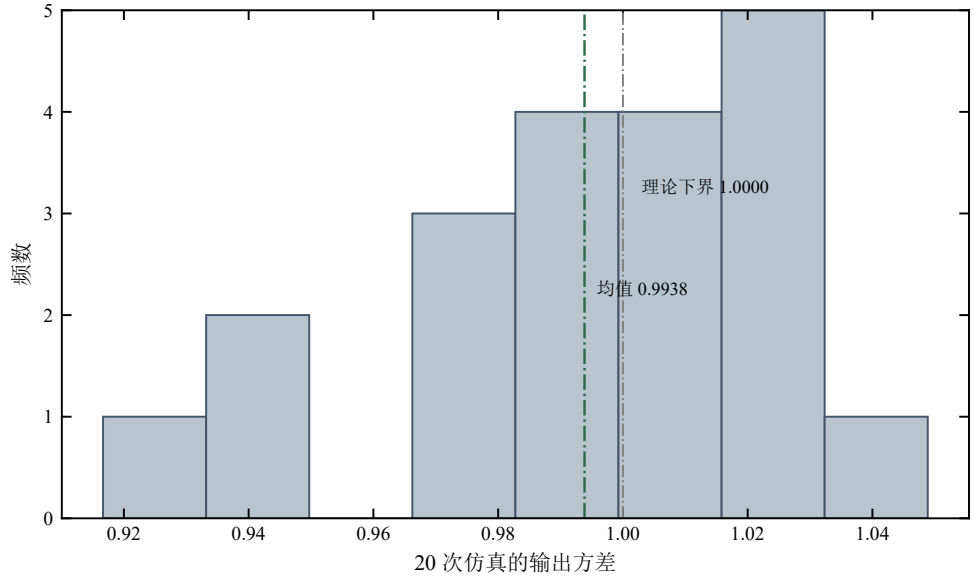


图 62-9　20 次独立运行的输出方差分布

图 62-9 是同一条设计做 20 次独立运行（每次 3000 拍）的方差直方图。统计：均值 0.9938、标准差 0.0313、最小 0.9167、最大 1.0489，理论下界 1.0000。

这张图要说明两件事。

第一，**单次运行的方差可以低于下界**，最低 0.9167，比下界低 8.3%。这不违反物理——样本方差是随机量，围绕真值波动，20 次里出现一两次偏低完全正常。所以看到“实测 0.92，低于理论下界”时，不要急着怀疑推导，先看样本量和重复次数。

第二，**均值的偏差只有 0.6%**（0.9938 对 1.0000），20 次的分布宽度约 $\pm 3\%$。这意味着判断“是否达到下界”需要至少 10 次独立运行，或者一次足够长的运行（$N > 10^5$）。用一次 1000 拍的仿真去论证“我的控制器比下界好 5%”，在统计上是站不住的。

工程验收同理：现场采集一段数据算出方差，与仿真值差 10% 以内都算一致；差 30% 以上才说明模型或实现有问题。**不要用单段数据的小差异去调模型**，那是把噪声当信号。

62.6.8 方差这个数本身有多准

§62.6.7 用了 20 次运行才敢说“均值 0.9938”。这一小节把这个“敢”字变成算式：单次估出的方差，误差有多大？

样本方差的相对标准差有一个常用的近似：$\sigma(\hat{V})/V \approx \sqrt{2/N_{\text{eff}}}$，$N_{\text{eff}}$ 是有效独立样本数。取 $N = 3000$，$\sqrt{2/3000} = 2.6\%$；实测 20 次的相对标准差是 0.0313，即 3.1%。两个数差 0.5 个百分点，多出来的部分正是噪声的相关性——$c = 0.6$ 的扰动相邻拍不正交，N_{eff} 比 N 小一点。

这条近似的实用价值在“反推需要多长的数据”。要把方差估到 1%，需要 $N \approx 2/(0.01)^2 = 20000$ 拍；要估到 3%，3000 拍够了。所以：

- 判断“是否达到下界”（要求 R 的分辨率约 3%）：$N = 3000 \sim 6000$ 够。
- 判断“两种调参差 5%”：各要 $N \approx 8000$，否则两组数据的差会被涨落淹没。
- 判断“$\hat{b}$ 偏 10% 带来的方差变化”（约 1% 量级）：单靠仿真要 $N > 20000$，不如直接用式 (62-34) 算。

第三条最容易被忽略。**能用解析式算的，不要用仿真去“验证”一个小差别**——仿真验证的是量级，解析式给的是精确值。本章表 62-5 之所以把“理论方差”和“实测方差”并排，就是让读者看到：两列在 1% 以内吻合，而它们的差正好是这一节算出的统计误差量级。

现场的情况更麻烦：现场只有一段数据，N 由采样周期和记录长度决定，不能想加就加。一个 1 s 采样、记录 30 分钟的回路只有 1800 拍，相对误差约 3.3%；若噪声相关性更强（c 接近 1），误差还会更大。**所以现场比对仿真与实测时，10% 的门槛不是拍脑袋定的，而是“3 倍统计误差”的自然结果。**

62.6.9 方差与峰值：报警线该按哪个定

方差是“长期平均”，报警看的是“单次最大”。这两个量不是一回事，也不是简单的倍数关系。表 62-8 给了很好的材料：$\tilde{\lambda}$ 从 0 到 20，标准差从 1.00 涨到 2.36（2.4 倍），而 $\max|y|$ 从 4.06 涨到 8.42（2.1 倍）。

倍数不一样，说明峰值不是跟着标准差线性走的。对高斯样本，N 个独立样本的最大值期望约为 $\sigma\sqrt{2\ln N}$。本章实验 $N = 6000$（去掉预热后约 5800），$\sqrt{2\ln 6000} = 4.17$。拿它去对表 62-8：

- $\tilde{\lambda} = 0$：$\sigma = 1.00$，$4.17\sigma = 4.17$，实测 4.06——**吻合到 3%**；
- $\tilde{\lambda} = 20$：$\sigma = 2.36$，$4.17\sigma = 9.85$，实测 8.42——**低了 14%**。

第一行吻合，说明高斯的假设在 MVC 闭环下基本成立；第二行偏低，是因为 $\tilde{\lambda}$ 大时输出强相关，6000 拍里“真正独立”的样本远少于 6000，$\sqrt{2\ln N_{\text{eff}}}$ 自然小一些。**相关越强，峰值相对标准差越低**——这与 §62.6.8 里“有效样本数”是同一件事的两个侧面。

两条工程推论：

第一，报警线不能按“3 倍标准差”拍。 3σ 在 6000 拍里几乎必然被打穿——按上面的估算，6000 个样本的最大值期望就在 4σ 附近。报警线的正确取法是反推：**先定“一年允许误报几次”，再换算成对应的分位数**，而不是“这批数据没超就安全”。

第二，记录越短，峰值越低估。 $\sqrt{2\ln N}$ 随 N 只按对数增长：$N=6000$ 是 4.17，$N=100$ 只有 3.03。若验收只看了 100 拍数据、发现最大偏差 3.1σ，就断定“绝不会超过 4σ”，那是把记录长度当成了物理边界。**“从来没超过 5”这句话，在统计上只等价于“记录还不够长”。**

反过来说，这也解释了为什么 §62.8.3 里 λ 的第四条读法值得单独列出来：$\max|y|$ 随 λ 单调上升。**保守控制的方差小、峰值反而大**，因为控制器不管了，扰动可以连续几拍同向累积出一个大偏差。选 λ 时若只盯方差，就会把报警余量悄悄用完。

62.7 参数意义与作用

62.7.1 先给一张总表

本章的模型有四个参数 a,b,c,d，控制器有一个设计参数 λ，噪声有一个尺度 σ_e。它们的作用完全不同，混在一起谈“调参”必然混乱。表 62-7 先给结论，后面逐条解释。

参数	物理含义	误设 10% 的后果	敏感度	能否现场辨识
a	对象惯性（记忆长度）	ρ 偏移 0.08，方差变 0.6%	低	容易
b	对象增益	环路增益变 10%，单侧失稳	**高**	需标定
c	噪声形状（相关性）	下界与等效增益同时变	中	容易（标量 RLS）
d	延迟拍数	下界翻倍或控制失效	**极高**	需专门测
σ_e	噪声强度	方差整体缩放	无（线性）	容易
λ	控制量的价格	性能权衡点移动	连续可调	不需要辨识

这张表最重要的一列是“敏感度”。它告诉你**力气该往哪使**：a 和 σ_e 估得粗一点没关系，b 和 d 必须准。这与很多教材的叙述顺序相反——教材从 A 讲起，工程上却最该先钉死 d 和 b。

62.7.2 a：对象记忆有多长

a 是对象自己回落到零的速度。$a=0.8$ 意味着“这一拍的偏差，下一拍还剩 80%”，时间常数约 $1/(1-a)=5$ 拍。它通过 F 的递推 $f_k=(a+c)a^{k-1}$ 影响两件事：延迟的边际代价（按 a^2 递减）和下界的绝对值。

a 的误设为什么影响小？因为它以**差值**形式进入闭环极点。$d=1$、$c=0$ 时 $\rho=a_{\text{true}}-a_{\text{model}}$，$a$ 偏 0.08 只让 ρ 偏移 0.08，方差从 1.0000 变到 1.0064。对比之下，b 的误差以**比值**形式进入，偏 10% 就是环路增益偏 10%，靠近边界时后果被 $1/(1-\rho^2)$ 放大。

但 a 接近 1 时要小心。$a=0.99$、$c=0.6$ 时，F 的系数几乎不衰减（1、1.59、1.574、1.558、1.543），$d=5$ 的下界已经超过 10，是 $a=0.8$ 时的两倍。这时“延迟的边际代价”几乎不递减，每一拍延迟都同样贵。**a 越接近 1，延迟越不能容忍**，这是慢对象（大惯量、大热容）在随机控制里特别难做的根本原因。

62.7.3 b：唯一会把你送出发散区的参数

b 是输入增益，也是 MVC 里唯一出现在分母上的对象参数（式 62-15 的 $-(a+c)/b$）。它在两个地方起作用：

- **决定控制律的增益**。b 估小，控制器就“以为要更用力”，实际作用在对象上的力就过大。
- **决定闭环极点**。式 (62-26) 的 $\rho = a - (a + c)b/\hat{b}$ 只在 $b/\hat{b}$ 上依赖，与 b 的绝对值无关。

这两点合起来解释了 §62.6.3 的不对称：b 偏大（真实值大于模型）让 ρ 向 -1 走，b 偏小让 ρ 向 $+a$ 走，后者永远在圆内。

现场最常出现的 b 误差来源，按频率排序：

1. **执行器增益没标定**。阀门开度到流量是非线性的，控制器里的 b 往往是“额定工况点的斜率”，小流量时实际增益只有一半。
2. **单位不一致**。u 用百分比、y 用工程单位，b 的量纲是“工程单位/百分比”，换一套量纲就全错。
3. **对象本身变了**。换热器结垢让传热系数下降，b 可以掉 20%～40%。

第一条最危险，因为它把工作点漂移伪装成“随机扰动”。诊断方法：在几个工作点上分别做阶跃，看 b 是否一致；不一致就说明有非线性，需要增益调度或前馈（第 58 章的增益调度思路可以直接借用）。

62.7.4 c：噪声的“可利用度”

c 是本章唯一一个“看起来抽象、实际上决定收益”的参数。它的作用有三层：

第一层，决定扰动方差。 $\mathrm{Var}(e[k] + c\,e[k-1]) = (1 + c^2)\sigma_e^2$。$c = 0.6$ 时是 1.36 倍，$c = 0.9$ 时是 1.81 倍（图 62-1）。所以同样标称“$\sigma_e = 1$”的两条回路，实际抖动可以差 80%。

第二层，决定可抵消的比例。 $c = 0$ 时扰动不可预测，$d = 1$ 的 MVC 只能把输出压到 σ_e^2——这就是全部，没有额外收益。$c > 0$ 时，$g_0 = a + c$ 让控制律的增益变大，控制器“提前”抵消一部分。极端地，$a \to 1$ 且 $c \to 1$ 时 $g_0 \to 2$，控制律增益翻倍。

第三层，决定稳定裕度。 §62.3.6 已经算过：c 从 0 到 0.6，失稳界线从 $b/\hat{b} = 2.25$ 压到 1.2857。**噪声相关性越高，控制器越“敢”，也越脆。**

c 的估计相对容易（§62.3.4 的标量 RLS，标准差 0.0023），但要注意 §62.4.4 的警告：c 是“采样后的形状”，换采样周期必须重估。c 还有一个上限：MA(1) 只能表达 $|\rho_1| \le 0.5$，慢漂移必须用高阶 C。

62.7.5 d：最贵的一个整数

d 是唯一一个**整数**参数，也是唯一一个“错一拍就翻倍”的参数。它的作用全部体现在 F 的长度上：d 决定有多少拍扰动无法干预，下界就是这些拍上的方差之和。

d 的三个来源要在辨识时分开：

- **纯延迟**：物料或信号传输时间，物理的。
- **采样与计算**：ADC、控制器、DAC 各占一点，常常合计接近一拍。
- **执行器响应**：阀门从收到信号到流量到位，可能有好几拍（尤其气动阀）。

第三项最容易被漏掉。气动调节阀的行程时间常在 1～3 s，若采样周期 0.5 s，光阀门就贡献 $d = 2 \sim 6$。诊断方法：给一个阶跃，看流量（不是阀位）什么时候开始动。

优化方向也因此很清楚：与其在控制器上想办法，不如把这三项各砍一半。换快速阀、把计算搬到更靠近执行器的位置、缩短总线周期——这些“非控制”的改动，收益往往十倍于算法优化。§62.6.2 的数字是最好的论据：d 从 2 降到 1，下界降 66%。

62.7.6 σ_e：只影响比例，不影响形状

σ_e 是新息的标准差。它只做一件事：把整个方差按 σ_e^2 缩放。下界、闭环方差、控制量方差全部与 σ_e^2 成正比。

这意味着：σ_e 估错不会改变最优控制器。式 (62-14) 里根本没有 σ_e——控制律与噪声强度无关。这是一个很重要的性质：噪声大不代表要调控制器，只代表“输出会更抖”。

它也有一个实用推论：归一化。把 y 除以 σ_e 再做设计，所有公式里的 σ_e 都变成 1，数字更好读，也更容易跨回路比较。本章的仿真就是这样做的（$\sigma_e = 1$）。

62.7.7 λ：控制量的价格，量纲是 b^2

λ 是 GMV 里唯一一个“纯设计”参数，不是辨识出来的。式 (62-21) 是

$$u[k] = -\frac{b\,\hat{y}[k+1]}{b^2+\lambda} = -\frac{\hat{y}[k+1]}{b}\cdot\frac{1}{1+\lambda/b^2} \tag{62-36}$$

右端第二个等号把结构说清楚了：λ 的作用就是给 MVC 增益乘一个折扣因子 $1/(1+\tilde{\lambda})$，其中

$$\tilde{\lambda} = \frac{\lambda}{b^2} \tag{62-37}$$

是无量纲的“相对价格”。$\tilde{\lambda}=1$ 时增益减半；$\tilde{\lambda}=3$ 时增益降到四分之一；$\tilde{\lambda}=19$ 时降到二十分之一。

λ 本身没有通用含义，$\tilde{\lambda}$ 才有。本章 $b=0.5$、$b^2=0.25$，所以 $\lambda=0.25$ 对应 $\tilde{\lambda}=1$。若换一个 $b=2$ 的对象，同样“增益减半”要取 $\lambda=4$，差 16 倍。现场交接参数时只说“$\lambda=0.25$”是没意义的，必须连 b 一起说，或者直接给 $\tilde{\lambda}$。

§62.8 的扫描表会看到，$\tilde{\lambda}$ 从 0 扫到 20，输出方差从 1.00 单调升到 5.57，控制方差从 7.83 单调降到 0.045——这是一条完整的交换曲线，λ 只是在这条曲线上选一个点。

62.7.8 P 与 R：给“输出”和“跟踪”加权

式 (62-20) 里的 P、R 是设计者选的多项式，本章一直取 1。它们的作用与 λ 不同：

- $P(z^{-1})$ 加权的是输出误差。$P=1-cz^{-1}$ 这类高通形式会让控制器更关注高频波动，相当于“只在抖得快的频段用力”。
- $R(z^{-1})$ 加权的是参考信号。$R=1$ 时跟踪直流；R 含积分环节时才能无静差跟踪斜坡。

工程上 P 用得少，因为一阶对象的波动没有明显的频段选择；R 用得也不多，因为随机控制的主战场是调节（设定值不变），跟踪问题交给第 44 章的 PID 或第 58 章的 MPC 更合适。本章不展开 P、R，只提醒一点：它们与 λ 不是一回事，λ 罚的是控制量本身，P 罚的是“输出的某个滤波版本”。

62.7.9 灵敏度预算：标定的钱该花在谁身上

四个参数都能估，但估它们的成本完全不同。这一小节把“误差造成的损失”与“测量的成本”摆在一起，回答一个很实际的问题：有限的标定时间该花在哪个参数上。

先看损失。把本章各处的实测与解析结果按“参数偏一点，方差变多少”排一列：

- d 错一拍：下界从 $1.0000\sigma_e^2$ 涨到 $2.9600\sigma_e^2$，**+196%**。若把 d 从 2 误判成 3，是 2.96 到 4.21，+42%。第一拍最贵，之后按 $a^2=0.64$ 递减。
- b 高 **28.6%**：发散。b 低 20%：1.114，+11%。

- c 偏 **50%**：1.14～1.17，**+14%～17%**，两侧几乎对称。
- a 偏 **10%**：1.006，**+0.6%**。

排序一目了然：$d \gg b > c \gg a$。差三个数量级。

再看成本。a 最便宜——一次阶跃响应，读时间常数即可，甚至从历史趋势曲线上目测都能估到 20% 以内。d 最贵——要互相关，还要把纯延迟、计算延迟、执行器响应三个来源分开核（§62.3.3）。b 次贵——三个工作点各做一次阶跃，还要保证工况不变。c 最麻烦——要先有干净的 a, b, d，再用 RLS 跑一千拍（§62.10.4）。

两者相乘，就得到该花力气的地方：

第一，d 的测量要当成一件正式工作做，不能"估摸着填"。 它的损失最大、成本也最高，但正因为损失大，这笔钱必须花。一个可行的省力办法是先在闭环数据上做互相关粗估，再用一次开环小阶跃确认——两步的成本加起来不到半天，而把 d 填错一拍的代价是整个改造白做。

第二，b 要"标"不要"估"。 它的危险是单侧的：真实增益比模型高会失稳，比模型低只是变差。所以标定的目标不是"最准"，而是**模型值宁可偏高一点**。§62.10.5 说的"取三个工作点里最大的 b"就是这个意思——用最大增益当模型，等价于保证 $b/\hat{b} \le 1$，永远落在安全侧。

第三，c 估不准时，取偏小的值。 c 进入控制律是 $(a+c)/b$，估大了增益就大，裕度就小；估小了只是保守一点，方差多几个百分点。这个"不对称的保守方向"与 b 恰好相反——**b 要往大取、c 要往小取，两者都是为了让等效增益不偏大。**

第四，a 不必精估。 偏 10% 只让方差动 0.6%，比仿真的统计误差还小。花时间去精估 a 是浪费——除非 a 已经接近 1（长惯性对象），那时它对 d 的灵敏度有放大作用（§62.2.3 的表里 $a = 0.99$ 一行），才值得认真估。

这张"灵敏度—成本"账说明一件事：**随机控制的标定预算不是平均分配的。** 把力气按 $d : b : c : a$ 的顺序投下去，同样的工时能换来大得多的可靠性；反过来，把时间花在精估 a 上、d 靠猜，是最常见的资源错配。

62.8 调参方法与经验

62.8.1 先钉模型，再选代价

随机控制的"调参"分两半，性质完全不同：

- **前一半是辨识**：d、b、a、c 是对象的属性，不是旋钮。它们只能被估出来，不能被"调"。
- **后一半是决策**：λ 是人的偏好，代表"输出方差与控制动作之间愿意怎么换"。它可以调，而且只应该在后一半调。

把两半混在一起，就会出现本章开头那种场面：输出抖，于是加大增益；加大增益后抖得更厉害（因为对象增益估错了），于是再加大——最后阀在振、输出也在振。**这不是调参失败，是把模型问题当成了参数问题。**

62.8.2 调整顺序与理由

顺序应当是 $d \to b \to c \to a \to \lambda \to$ 限幅。理由逐条：

1. **d 先定**。它是整数，错一拍下界就翻倍，而且它会影响后面所有辨识（d 错了，b 的估计会系统性偏小）。
2. **b 次之**。它是唯一能把闭环送出单位圆的参数，必须先钉死，还要做增益标定。
3. **c 再次**。它决定下界与等效增益，估错会让"设计值"与"实测值"对不上，但不会失稳。
4. **a 最后**。它最不敏感，偏 10% 只让方差动 0.6%。

5. **λ 在模型确认之后**。此时交换曲线才有意义。

6. **限幅放最后但必须做**。它不改变设计，只改变“设计能不能兑现”。

这个顺序与“从 a 开始讲模型”的教材顺序相反，但与现场排查顺序一致：先问“延迟多少”，再问“增益多少”，最后才关心“惯性多大”。

62.8.3 交换曲线：把 λ 的后果一次看全

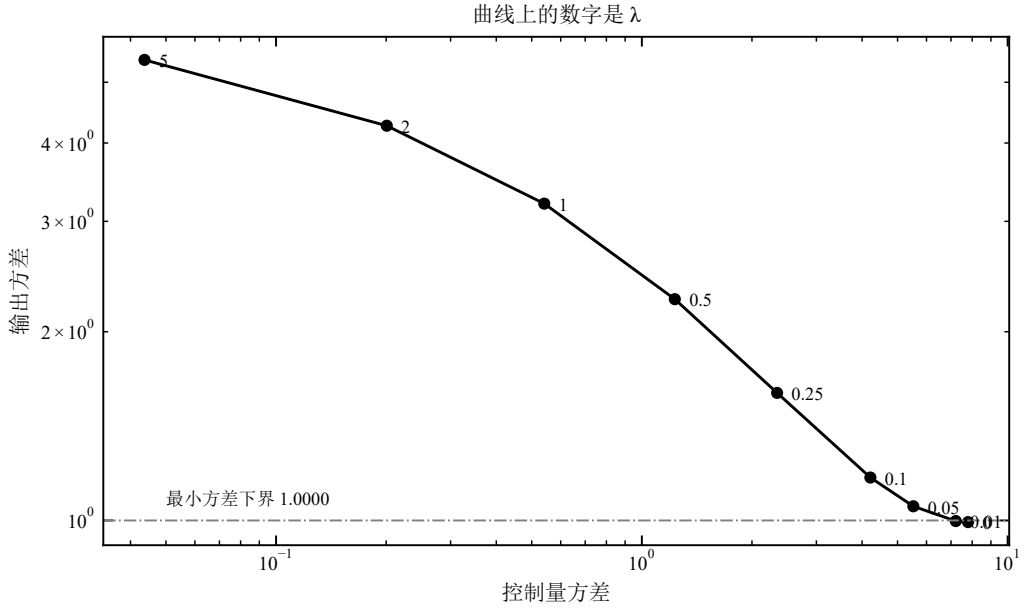


图 62-10　控制方差与输出方差的交换曲线，标注为 lambda

图 62-10 是实验 3 的结果（$N = 8000$、seed=7）：横轴控制方差、纵轴输出方差，双对数坐标，每个点对应一个 λ。曲线从左上（$\lambda = 0$，输出最平、控制最猛）走到右下（$\lambda = 5$，几乎不控制）。表 62-8 是同一组数据的表版，并补上无量纲价格 $\tilde{\lambda} = \lambda / b^2$ 与总变差。

λ	$\tilde{\lambda}$	$\mathrm{var}(y)$	$\mathrm{var}(u)$	$\max \lvert y \rvert$	总变差
0.00	0	0.9989	7.8312	4.062	18203.2
0.01	0.04	1.0034	7.2540	4.117	17237.9
0.05	0.20	1.0620	5.5606	4.296	14251.4
0.10	0.40	1.1828	4.2494	4.444	11734.9
0.25	1.00	1.6201	2.3692	5.052	7699.2
0.50	2.00	2.2919	1.2476	5.944	4907.2
1.00	4.00	3.2644	0.5502	6.787	2847.6
2.00	8.00	4.3594	0.2048	7.540	1548.4
5.00	20.00	5.5727	0.0447	8.421	654.0

表 62-8 有四条读法。

第一，左上角很平。 $\tilde{\lambda}$ 从 0 增到 0.4，输出方差只从 0.9989 涨到 1.1828（+18%），而控制方差从 7.83 掉到 4.25（−46%）。**前 0.4 的"价格"几乎白送**——这是整条曲线上最划算的一段。

第二，中间段是主战场。 $\tilde{\lambda}$ 从 0.4 到 4，输出方差翻近三倍（1.18 → 3.26），控制方差降近八倍（4.25 → 0.55）。这一段对应"从激进到保守"的完整过渡，工程选点多半落在这里。

第三，右下角趋近不控制。 $\tilde{\lambda} = 20$ 时输出方差 5.57，已经接近不控制的 6.78；控制方差 0.045，等价于"基本不动"。再加大 λ 没有意义。

第四，$\max|y|$ 随 λ 单调上升。 从 4.06 到 8.42。这条列容易被忽略：**保守控制的方差小、但偶发的大偏差更大**，因为控制器"不管"了，扰动可以自由累积几拍。若现场有软限报警，λ 太大反而更容易触发。

62.8.4 一个不舒服的事实：$d = 1$ 时 GMV 就是比例控制

把式 (62-36) 在 $d = 1$ 时展开，控制律是 $u = -k\,y$，其中

$$k = \frac{a + c}{b} \cdot \frac{1}{1 + \tilde{\lambda}} \tag{62-38}$$

本章 $a = 0.8$、$c = 0.6$、$b = 0.5$，$(a + c)/b = 2.8$。于是 $\tilde{\lambda} = 0$ 对应 $k = 2.8$，$\tilde{\lambda} = 1$ 对应 $k = 1.4$，$\tilde{\lambda} = 4$ 对应 $k = 0.56$。

换句话说，$d = 1$ 的整个 GMV，在实现上就是一个比例控制器，λ 只是在选 P 增益。 这与第 44 章的 P 控制没有结构差别，唯一的区别是：这里的增益是算出来的，不是试出来的；而且它把"该用多大增益"与"愿意付多少控制代价"分开成了两件事。

这个事实有两个用处。第一，它让现场工程师容易接受：**你不需要换控制器，只需要换一个更好的增益**。第二，它划清了 MVC 的适用范围：$d = 1$ 时它没有超越 P 控制的结构性优势，真正的优势出现在 $d \geq 2$（式 62-16 的预测项）与有色噪声（c 进入增益）这两处。

62.8.5 从曲线上选点：三条工程准则

λ 的选择没有唯一答案，但有三条可操作的准则，通常联合使用：

准则一：控制变差预算。 现场最关心的是执行机构寿命，可以用总变差（相邻拍动作绝对值之和）来度量。若阀门检修周期要求总变差不超过 10000（每千拍），查表 62-8，λ 应取在 0.10～0.15，对应输出方差 1.18～1.30。

准则二：输出方差目标。 若工艺要求标准差不超过设定值的 1.5%，而设定值 100，则 $\mathrm{var}(y) \leq 2.25$。查表，$\lambda \leq 0.5$ 都满足，此时用法则一收紧。

准则三：最大偏差约束。 若报警线是 ± 6，则 $\max|y|$ 必须留余量。表里 $\lambda = 0.5$ 时 $\max|y| = 5.94$ 已贴线，$\lambda = 0.25$ 时 5.05 尚有 16% 余量。这一条常常是决定性的一条，因为它对应的是"会不会停机"，而不是"波动好不好看"。

三条准则取交集，通常就只剩一两档 λ。**如果交集是空的，说明工艺要求与执行机构能力不匹配，此时该改硬件，不是继续调 λ。**

62.8.6 一次完整实录

把 §62.8.2 的顺序走一遍，数字全部来自本章实验（同一段噪声、同一初值）。表 62-9 是这五步的账。

步骤	做法	$\mathrm{var}(y)$	$\mathrm{var}(u)$	总变差	结论
1	不控制	6.7812	0	0	基准
2	比例 $k=0.5$	2.9993	5.9950	2771.6	有效但不够
3	比例 $k=1.6$	1.3978	4.1950	9208.8	接近最优
4	标定后 MVC（$d=1$）	1.0119	7.9334	18251.5	方差最小，动作翻倍
5	GMV $\lambda=0.25$	1.6201	2.3692	7699.2	折中方案

五步的顺序值得注意。第 2、3 步是“用现有手段先摸底”：一个手调的比例增益已经把方差从 6.78 压到 1.40，拿回了 93% 的可降空间。第 4 步的 MVC 只比它再好 27%，代价是动作总变差翻倍。第 5 步反过来，牺牲 16% 的方差，换回 58% 的动作量。

如果只看第 4 步，会得出“随机控制代价太大”的结论；做完第 5 步才知道，真正的答案是“用 MVC 算增益，用 GMV 定价”。 这也是本章反复强调的立场：MVC 是标尺，GMV 才是产品。

62.8.7 五个常见错误

错误一：把 λ 当 PID 增益调。 加大 λ 使控制变弱，与减小 P 增益同向，但 $d\geq 2$ 时它还同时削弱预测项，效果不是等比例的。调 λ 之前先确认 d。

错误二：按期望值调 c。 c 是辨识量，不是设计量。有人为了让控制器“更积极”而人为放大 c，结果下界被算小、实际方差对不上，还会让增益裕度变窄（§62.7.4 第三层）。

错误三：d 用拟合残差猜。 拟合残差对 d 不敏感（d 错一拍，残差只多一点），必须用互相关单独测。

错误四：只看均值。 输出均值接近设定值，不代表方差合格。随机控制的目标量是方差，均值由积分环节或前馈保证，两者不是一回事。

错误五：忽略限幅。 设计时假设 u 无界，实机限幅后控制律失效，且历史缓冲里存的若是限幅前的值，误差会累积（§62.5.2）。**限幅必须参与设计，不是最后加的保险。**

62.8.8 调参记录表

随机控制的调参记录比 PID 更需要留痕，因为参数之间有关联（b 变了，λ 的含义就变了）。建议六列，见表 62-10。

日期	d / b / c / a	控制律	$\tilde{\lambda}$	$\mathrm{var}(y)$ / 总变差	备注
03-02	1 / 0.50 / 0.60 / 0.80	比例 $k=1.6$	—	1.398 / 9209	手调摸底
03-05	1 / 0.50 / 0.60 / 0.80	MVC	0	1.012 / 18252	阀动作偏大
03-06	1 / 0.50 / 0.60 / 0.80	GMV	1.0	1.620 / 7699	投运
03-20	1 / 0.55 / 0.58 / 0.79	GMV	1.0	待复测	复标 b，λ 由 0.25 改 0.3025

最后一行的意义要说清楚：b 从 0.50 复标到 0.55 之后，同样的 $\tilde{\lambda}=1$ 对应 $\lambda=0.3025$ 而不是 0.25。**记录表里写 $\tilde{\lambda}$ 而不是 λ，就是为了让这种复标不产生歧义。**

62.8.9 一次“把 λ 加到 5”的复盘

这是本章最容易被重复犯的一个错，值得单独记一笔。现场嫌阀动作多，把 λ 从 0.25 一路加到 5，理由是“λ 越大控制越温和”。表 62-8 的四行数据说明了后果：

λ	输出方差	控制方差	$\max\|y\|$	总变差
0.25	1.6201	2.3692	5.052	7699.2
0.50	2.2919	1.2476	5.944	4907.2
1.00	3.2644	0.5502	6.787	2847.6
5.00	5.5727	0.0447	8.421	654.0

阀动作确实降下来了：总变差从 7699 掉到 654，降了 91%。但同时输出方差从 1.62 涨到 5.57（+244%），已经接近不控制的 6.78；$\max|y|$ 从 5.05 涨到 8.42，直接打穿报警线。**这不是“调得保守了一点”，是把控制器调成了摆设。**

更麻烦的是它的诊断特征容易被误读。验收看到 $R = 5.57$（远大于 1.1），第一反应是“模型错了”。但这时只要看一眼控制方差：0.0447，几乎为零——**控制器根本没在动，谈何模型错。** 由此得到一条很实用的判据：

- R 大、$\mathrm{var}(u)$ 也大：模型或 d 错了，控制器在“使劲使错方向”。
- R 大、$\mathrm{var}(u)$ 极小：λ 太大了，控制器主动放弃。

这条判据把“模型问题”和“调参问题”一次分开，比单看 R 可靠得多。

正确的收尾不是继续在 λ 上找平衡，而是承认“阀动作”与“输出方差”在这条曲线上是硬交换：想要 654 的总变差，就必须接受 5.57 的方差。若两者都不可接受，出路只有两条——换执行器（让阀能承受更多动作），或者改罚项（§62.9.8，罚 Δu 而不是罚 u，让同样的方差对应更小的总变差）。

λ 是用来“在曲线上选点”的，不是用来“消除阀动作”的。 这句话听起来像文字游戏，但它决定了下一次调参时你会不会又把 λ 往上加一格。

62.9 优化与改进

62.9.1 六条改进路线

MVC 的四个短板——控制量太大、对增益误差太敏感、不能处理约束、不能跟参数漂移——各自有对应的改进路线。表 62-11 先给全景，后面逐条说清“改了什么、解决什么、代价是什么”。

路线	改什么	解决什么	代价	相关章节
GMV（λ）	指标加控制惩罚	动作过大	输出方差上升	本章 §62.2.6
极点配置 GMV（P）	指定闭环极点	对消过狠、脆弱	要选极点、要解新方程	本章 §62.9.2
自适应 MVC	在线估 a, b, c	参数漂移	需要激励、可能失稳	第 61 章
带约束 MVC	限幅、速率限制	执行器物理限制	需可行解、可能饱和	第 58、80 章
前馈 / 扰动观测	加可测扰动通道	可测扰动	需要测点	第 44 章
LQG / 状态空间	状态估计 + 最优反馈	多变量、量测噪声	模型阶次上升	第 54 章

这张表有一个共同点：**每条路线都在“性能”与“某种代价”之间重新画一条线，没有一条能同时拿到所有好处。** 选路线的前提是先认清当前最痛的是哪一条。

62.9.2 用 P 指定闭环极点

§62.2.9 的结论是：$P=1$ 时闭环极点被 C 完全对消，闭环传递函数退化成 F——一个有限脉冲响应。这样做在数学上最优，在工程上却意味着“没有缓冲”：任何模型误差都会让被消掉的极点漏出来。

改进办法是在指标里加一个输出加权 $P(z^{-1})$，重解 Diophantine。结论是（推导略，见 Clarke–Gawthrop 的原始文献）：闭环特征多项式从 C 变成

$$P(z^{-1})\,C(z^{-1}) \tag{62-39}$$

也就是说，**P 的零点直接成为闭环极点**。若取 $P=1+p_1z^{-1}$，就多了一个可配置的极点；取 $P=(1-\alpha z^{-1})^k$，可以把闭环调成指定的时间常数。

这个改动的作用不是“让方差更小”（方差一定变大），而是**让闭环对模型误差更宽容**。原因很直观：原来被对消的 C 现在与 P 一起构成闭环动态，模型误差只让极点偏离 P 的零点，不会让整个回路失去缓冲。代价是要重新解一次 Diophantine 方程（P 进入后 F、G 的阶次都变），而且 P 的零点必须选在单位圆内。

工程上 P 用得不多，原因和 §62.7.8 一样：一阶对象的极点选择空间很小，把 C 的零点直接留着（$P=1$）往往就是可接受的。**但对象阶次高、或者对消的极点靠近单位圆时，P 是必须的**——否则被消掉的慢极点一旦漏出来，闭环会出现设计时完全没预料到的长尾。

62.9.3 自适应最小方差

参数会漂移的场合（结垢、磨损、工况切换），固定参数的 MVC 迟早失配。最直接的办法是在线估 a,b,c，每拍（或每若干拍）重算 F、G 与控制律，这就是自适应最小方差，属于自校正调节器（STR）的一类。第 61 章已经完整讲过 STR 的结构与稳定性条件，这里只说它与本章的接口：

- **估计器**用 §62.3.4 的两步法：先 a,b，再标量 RLS 估 c。
- **设计**每 N_{up} 拍做一次，不必每拍——F、G 的变化远慢于 y。
- **控制**用式 (62-33)，系数在两次更新之间保持不变。

三个必须处理的坑，都与“估计器与控制器的耦合”有关：

第一，**激励不足时参数会漂**。闭环稳定后 u 与 y 高度相关，RLS 的协方差矩阵会退化成一条线，参数沿这条线滑动而残差不变。表现是“参数乱跳但控制效果不变”，一旦工况变化就突然失稳。对策是协方差重置、方向遗忘，或者周期性注入小激励（第 61 章的 PRBS 可直接用）。

第二，**参数更新要有投影**。把 $\hat{b}$ 限制在 $[0.85b_0,\,2b_0]$、$\hat{c}$ 限制在 $[0,1]$，越界就钳住。下界取 0.85 而不是更宽，是因为 §62.6.3 的悬崖就在“真实增益比模型大 28.6%”处——反过来说，模型增益一旦被估到真值的 0.78 倍以下就已越界，投影区间必须比这个更紧。没有投影时，一次坏数据就可能把 $\hat{b}$ 推到悬崖外。

第三，**自适应不能救结构错误**。若 d 估错、或扰动入口判错，在线估计只会把参数拉到一个“拟合得好但控制不好”的值。**先确认结构，再开自适应**——这是第 61 章的主线，也是本章反复强调的“模型问题不能靠调参解决”。

本节不给自适应版本的实测数字：本章的仿真框架是固定参数的，强行加一个自适应环会让所有对照失去可比性。第 61 章有一整套 STR 的实测（含 γ 扫描、限幅、滞后），需要数字时应当去那里看。

62.9.4 带约束的 MVC

MVC 的控制律是线性的，执行器却不是：阀门有量程、电机有电流上限、变频器有加减速限制。处理办法分三层，由浅入深。

第一层：输出限幅 + 回灌。 把 u 钳在 $[u_{\min}, u_{\max}]$，并把钳后的值存进历史缓冲（§62.5.2）。这一层只保证“不发错指令”，不保证性能：一旦限幅，闭环实际动态与设计不符，方差会高于下界。诊断特征是“R 在正常工况下接近 1，在扰动大的时段明显大于 1”。

第二层：速率限制。 若执行器有最大变化率，超出时按速率走。速率限制的等效效果是**延迟增加**：本拍该走完的行程要分几拍走完，闭环看到的 d 变大。所以设计时应当把速率限制折算成“等效 d”，用表 62-1 重新算下界——如果折算后下界翻倍，就说明这个执行器不适合这个回路。

第三层：把约束写进优化。 每拍解一个小型二次规划，在满足 u 与 Δu 约束的前提下最小化式 (62-20)。这就是 MPC 的雏形（第 58 章给完整推导，第 80 章给工业实现）。对一阶对象，这个 QP 只有一两个变量，手算都能做；但它带来的好处很大：**约束生效时不再是被动饱和，而是主动降级**——控制器知道“这一拍走不完”，会提前几拍开始动。

三层的关系是递进的：第一层防错，第二层防磨，第三层防“该提前而没提前”。现场的性价比排序通常是第一层 > 第三层 > 第二层。

62.9.5 前馈：把可测扰动提前扣掉

本章的扰动都是不可测的（只能从 y 反推）。若扰动有可测的源头——进气温度、环境温度、上游流量——就可以加前馈：测到扰动，按模型算出它将对 y 造成的影响，提前从 u 里扣掉。

前馈与 MVC 的分工很清楚：**前馈处理“可测但不可控”的扰动，反馈处理“不可测”的扰动。** 两者不冲突，前馈做得好，反馈要处理的方差就小，λ 可以取更小（更激进），总性能更好。前馈通道的增益由对象的直流增益决定（b 的倒数），所以它同样依赖 §62.7.3 的增益标定——**前馈把 b 的误差从“影响稳定性”变成了“影响稳态偏差”，危害小得多。**

第 44 章讲过前馈-反馈复合的整定方法，可以直接搬到随机框架下：前馈通道用静态增益，反馈通道用本章的 GMV，两者的分工不变。

62.9.6 从单回路到多变量

多变量情形下，MVC 推广为 LQG：状态估计用卡尔曼滤波，反馈用 LQR，噪声通过过程噪声与量测噪声的协方差进入。第 54 章给了观测器的完整推导，这里只说三点与本章的对应关系：

- **Diophantine 方程变成矩阵方程。** F、G 变成多项式矩阵，求解要用谱分解，手算基本不可能。
- **“最小相位”条件变成“传输零点在单位圆内”。** 多变量的非最小相位更难识别，也更容易被忽略。
- **延迟不再是标量 d，** 而是每个通道一个延迟，交互影响下“哪一拍能干预什么”变得复杂。

所以多变量场合不建议从 MVC 入手，直接从 LQG 或 MPC 开始更现实。**本章的方法适合“一个回路、一个主扰动、延迟可数”的场合，**这也是它在工业上最常见的存在形式。

62.9.7 与 Smith 预估的分工

Smith 预估（第 47 章、第 107 章）与 MVC 都在处理延迟，但处理的是延迟的不同后果：

- **Smith 预估**把延迟“搬出”闭环，让控制器按无延迟的对象整定。它针对的是**确定性跟踪**：设定值变化时，不要让延迟把相位裕度吃光。
- **MVC**承认延迟期间发生的扰动无法干预，把这段方差算进下界。它针对的是**随机调节**：设定值不变时，输出能压到多平。

两者不替代。一个既有大延迟、又有随机扰动的回路，正确做法是：用 Smith 预估（或 MPC 的内模）处理设定值跟踪，用 GMV 处理稳态方差。若只做其中一件，另一件的问题会在投运后暴露：只做 Smith，稳态抖动依旧；只做 GMV，设定值阶跃时超调依旧。

这也解释了为什么第 47 章与本章要用不同的指标（超调/调节时间 vs 方差）：**它们回答的是两个不同的问题。**

62.9.8 罚 u 还是罚 Δu：一个改法的差别

本章的代价函数罚的是 $u[k]$ 本身（式 62-20 的 λu^2）。这个选择决定了 MVC/GMV 的一个特征：**它不管控制量变化得多快。** 表 62-8 里 $\tilde{\lambda}=0$ 的总变差 18203，$\tilde{\lambda}=1$ 降到 7699——同一套闭环，控制量的大小只差 3.3 倍，而变化量差 2.4 倍。可见“罚 u”对“抖动”的抑制很间接。

现场的物理代价并不总是与 u 的大小成正比。阀门磨损对应的是**行程**（$\sum|\Delta u|$），电机发热对应的是**力矩**（正比于 u^2），泵的汽蚀对应的是**开度上下限**（约束）。三者不一样，而 λu^2 只对上了中间那一种。

把罚项换成 $\lambda(\Delta u)^2$，就得到广义预测控制（GPC，第 58 章）的核心差别之一。它的效果可以直接说清：

- **罚 Δu 自带积分结构。** 因为 $\Delta u = u[k]-u[k-1]$，代价里出现了 u 的差分，最优解天然倾向于“缓慢移动”，而不是“快速到位再快速回来”。
- **它同时抑制了稳态偏差的累积。** 若 u 长期停在偏置上，Δu 为零，代价不罚；但一旦有持续的偏差，控制器会通过 Δu 的积分把它拉回来——这一点与 PID 的积分作用同源。
- **它与限幅配合更好。** 罚 Δu 的解更平滑，饱和的频次更低；即使饱和，退出时也不会出现“回弹”。

代价是求解变复杂：罚 Δu 后，控制律不再是“一拍一个代数式”，而要解一个带预测时域的优化——这正是 MPC 与 GMV 的分界。本章的 GMV 之所以能写成闭式，靠的就是“只罚当拍 u”这个简化。

工程上的判断很简单：**若验收卡在总变差（阀动作太多），先考虑改罚项；若卡在方差，才去调 λ。** 两者的作用点不同——调 λ 是在“性能与控制量大小”之间换，改罚项是在“控制量大小与控制量变化”之间换。把这两件事混为一谈，就会出现“为了让阀少动，把 λ 加到 20，结果方差退化到接近不控制”的局面（表 62-8 最后一行：方差 5.57，总变差 654）。

62.10 工程落地

62.10.1 一份落地清单

前面九节讲的是原理与数字，这一节讲“明天上班怎么干”。按顺序十件事，前六件在投运前，后四件在投运后。

1. 选一段稳态数据，确认回路本身是稳的（用现有 PID）。
2. 判噪声结构：残差白噪声检验 + 一阶自相关。
3. 测延迟 d：互相关，三个来源分开核。
4. 标定增益 b：至少三个工作点。
5. 估 a 与 c：两步法，c 用标量 RLS。

6. 算 F、G、下界，做失配压力测试，选 $\tilde{\lambda}$。
7. 影子运行：控制器算但不输出，比对预测与实测。
8. 闭环投入：从 $\tilde{\lambda}$ 较大的保守点开始，逐步减小。
9. 验收：R、总变差、$\max|y|$ 三项。
10. 月度复标：b 与 c 重估，λ 按 $\tilde{\lambda}$ 换算。

清单里最容易跳过的是第 1 件和第 7 件。第 1 件不做的后果是“在振荡的数据上辨识”，估出来的参数毫无意义；第 7 件不做的后果是“带着错误的模型直接闭环”，而这在随机控制里通常意味着第一次大扰动就发散。

62.10.2 数据从哪来

辨识需要的数据有三条硬要求：**稳态、够长、有激励**。

稳态指没有趋势、没有工况切换。判据很简单：把数据分成前后两半，两半的均值差小于标准差的一半、方差比在 0.7～1.4 之间，就可以认为稳态。若回路本身在周期性振荡（比如有积分饱和或阀门粘滞），先修好再辨识——**振荡的数据会把 a 估到接近 1，让下界虚高**。

够长指至少覆盖 10 个对象时间常数。$a=0.8$ 时时间常数约 5 拍，10 个就是 50 拍；但统计要可靠得多，本章的仿真取 6000～20000 拍。现场一般取 1000～3000 拍（按 1 s 采样约 20～50 分钟），这个长度的方差估计误差约 $\pm 5\%$，够用。

有激励指 u 必须有足够的变化。闭环稳态运行时 u 的变化主要来自噪声反馈，幅度小、频带窄，辨识出的 b 往往偏大（回归量方差小，噪声占比高）。标准做法是叠一段 PRBS 到设定值或输出上，详见下一节。

三条要求里，“稳态”和“有激励”看起来矛盾——稳态强调不变，激励强调要变。解决办法是**在稳态上叠一个小激励**：基线的工况不变，只让设定值在 $\pm 2\% \sim 5\%$ 内随机跳。这样既保持了工作点，又给了辨识所需的信息。

62.10.3 激励怎么加

PRBS 的参数有三个：**幅度、位宽、周期长度**。

- **幅度**取设定值的 2%～5%，或者取“输出噪声标准差的 2～3 倍”。太小则信噪比不够，太大则非线性被激发、还可能触发报警。
- **位宽**（每个比特持续的拍数）取对象时间常数的 $1/3 \sim 1$。$a=0.8$ 时约 2～5 拍。位宽太短，对象来不及响应；太长，等效于几个阶跃，频带不够宽。
- **周期长度**取 2^k-1，$k=7$ 给 127 拍，$k=8$ 给 255 拍。总采集长度至少覆盖 4 个完整周期，本章建议 8 个，即 $k=7$ 时采 1000 拍左右。

第 61 章给过一个 7 位 PRBS 图样，可以直接复用。要提醒的是：**PRBS 必须真的“随机”叠加，不能用方波代替**。方波只有两个电平、一个频率，回归量矩阵接近奇异，a 与 b 会被估成一组“能拟合但不对”的值——这是辨识里最常见的自欺。

62.10.4 c 的现场辨识：一个标量就够

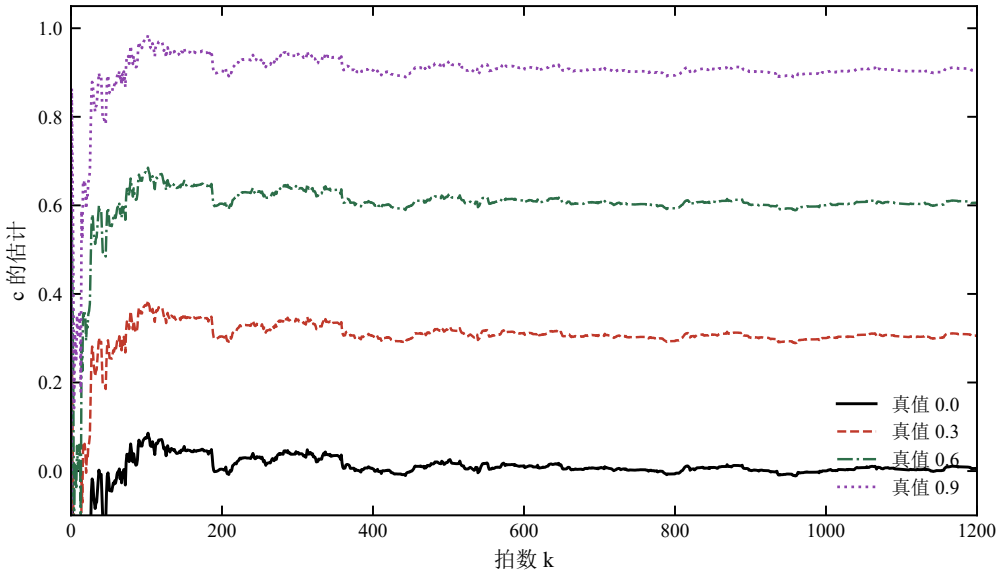


图 62-11　噪声模型系数 c 的递推估计收敛过程

图 62-11 是实验 8 的结果：在 a、b 已知的条件下，用标量 RLS 估 c，四条曲线对应真值 $c=0$、0.3、0.6、0.9。四条线都在几百拍内进入真值附近，1000 拍后的尾段均值为 $+0.0067$、$+0.3067$、$+0.6067$、$+0.9067$，标准差都只有 0.0023。

这张图的分量在于**“它把随机控制里最抽象的参数变成了一个可以现场测出来的数”**。c 不是“噪声相关性”这种只能定性描述的东西，它是一个标量，跑一千拍就能估到千分位。具体步骤：

1. 用辨识好的 a,b,d 重构新息：$e[k]=y[k]-a\,y[k-1]-b\,u[k-d]-c\,e[k-1]$，初始 $c=0$；
2. 把上式对 c 线性化，回归量是 $-e[k-1]$，用标量 RLS 更新；
3. 跑 500～1000 拍，看 c 是否收敛；不收敛或超出 $[0,1]$，说明 a,b,d 有一个错了。

第 3 步是很好的自检：c 是“最后才该收敛”的参数，它不收敛，几乎总是前面的参数有问题。

一个现场细节：若估出的 c 接近 0，不要立刻下结论“噪声是白的”。先看 e 的方差是否与量测噪声匹配——若 e 的方差远大于传感器噪声，说明扰动确实存在但被 a 吸收了，这时应当提高 A 的阶次（二阶），而不是接受 $c=0$。

62.10.5 b 的现场标定：三个工作点

b 是唯一必须“标”而不是“估”的参数，因为它的误差直接决定稳定性。方法是在至少三个工作点上各做一次小阶跃：

1. 选 30%、50%、70% 三个阀位（或三个负荷点）；
2. 每个点上做 $\pm 5\%$ 的阶跃，等稳态；
3. 算 $\Delta y/\Delta u$，取正负两个方向的平均；
4. 三个点的结果若相差超过 20%，说明对象有非线性。

非线性的处理有两条路：**增益调度**（按工作点切换 b，第 58 章的调度思路）或**取最坏值**（取三个点里最大的 b，按它设计，代价是低负荷时控制偏软）。本章建议后者，因为随机控制对“增益高估”不敏感、对“增益低估”敏感——取最大值正好落在安全侧。

单位问题单独强调一次。b 的量纲是“输出单位/输入单位”，若 u 是百分比、y 是 ℃，b 就是 ℃/%。换一套单位（u 改成 4～20 mA），b 要整体换算，λ 也要跟着按 b^2 换算。**现场交接时最容易丢的就是这一条。**

62.10.6 投运：三段式

随机控制器不能“一投了之”。三段式投运是本章推荐的做法：

第一段，手动。 回路在手操下稳定运行，采集辨识数据。这一段的目的只有一个：拿到干净的 y、u。

第二段，影子运行。 控制器在线计算 u，但**不输出**；把算出的 u 与实际的 u 都记下来，同时用模型预测下一拍的 y，与实测比较。

影子段是整套流程里性价比最高的一步。它能在不影响生产的前提下暴露三类问题：

- **时序错**：预测误差呈现“系统性的滞后一拍”，说明 d 估大了或计算延迟没算进去；
- **增益错**：预测误差与实际 u 的变化强相关，说明 b 不对；
- **入口错**：预测误差的自相关结构与噪声不符，说明扰动入口判错（§62.3.2）。

影子段跑够 500 拍、预测误差的方差与 σ_e^2 同量级，才允许进入第三段。

第三段，闭环。 从保守点开始：$\tilde{\lambda}$ 取 4～8，即增益只有 MVC 的 1/5～1/9。观察 30 分钟，逐步减到设计值。每次减小后看三个量：输出方差是否按预期下降、总变差是否超预算、$\max|y|$ 是否靠近报警线。三者有一个不对，就退回上一档。

这套流程看起来慢，但它把“会不会出事”从一个未知变成了每次只冒一小步的风险。**随机控制的事故几乎都发生在“直接从手动切到设计增益”的那一次。**

62.10.7 验收与巡检

验收用三个数，全部在 §62.6、§62.8 出现过，见表 62-12。

指标	计算	合格范围	不合格时查什么
R	实测方差 / 下界	0.9～1.1	d、c、计算延迟
总变差	$\sum\lvert\Delta u\rvert$	按检修预算	$\tilde{\lambda}$ 太小
$\max\lvert y\rvert$	全程最大偏差	留 20% 报警余量	$\tilde{\lambda}$ 太大

三项里 R 是“模型对不对”，总变差是“代价付不付得起”，$\max|y|$ 是“会不会停机”。三者不会同时最优，验收的本质是**确认这三项都落在可接受范围内**，而不是某一项特别漂亮。

巡检则只需每月做一次 b 与 c 的复标。若 b 变化超过 10%，按新值重算 λ（保持 $\tilde{\lambda}$ 不变）；若 c 变化超过 0.2，重算下界与 F、G。**参数复标的频率应当由对象的变化速度决定，不是由制度决定**——结垢快的换热器可能每周都要复标，而一个干净的电机回路一年都不用。

62.10.8 两个场景复盘

场景一：一个成功的改造。 某温度回路，原 PID 下输出标准差 1.4（归一化），阀每秒动作 3～4 次，阀杆三个月磨损一次。按本章流程做：辨识得 $d=1$、$b=0.5$、$c=0.6$，算出 MVC 能把标准差压到 1.0，但控制变差会到 18000 量级（表 62-9 第 4 行）。于是取 $\tilde{\lambda}=1$，方差 1.62（比原来差 16%），控制变差降到 7700（比原来少 58%）。投运后阀的动作次数下降一半，输出波动仍在工艺允许范围内。

这个案例的关键不是“方差降了多少”，而是**主动用 16% 的方差换了 58% 的动作量**。若按“方差越小越好”的思路直接上 MVC，改造会被判为失败——阀磨得更快。

场景二：一次失败的投运。 某流量回路，辨识数据取自夜间低负荷，b 估成 0.35；白天满负荷时实际 b 是 0.55（比模型大 57%）。控制器按 0.35 设计，满负荷时环路增益是设计的 1.57 倍，超过 §62.6.3 的界线，投运当天下午就出现等幅振荡。

复盘有三条：第一，辨识数据只覆盖了一个工作点，违反了 §62.10.5 的三点要求；第二，没有做影子运行，否则预测误差会立刻暴露增益问题；第三，投运从 $\tilde{\lambda} = 1$ 直接开始，没有从保守点切入。三条里任何一条做到，这次事故都不会发生。

这两个场景的共同教训是：**随机控制的失败几乎都不是数学错，而是“模型只在某个工作点上对”**。数学部分（Diophantine、下界、控制律）是确定的，只要参数对，结果就可预测；而参数对不对，取决于数据采集与标定这些“不数学”的环节。

62.10.9 控制器代码里最容易漏的四件事

算法只有一行（$u = -\big(Gy + \sum bf_i u[k-i]\big)/(bf_0)$），但从这一行到能在 PLC 里跑的程序，中间有四件事必须补上。它们不影响公式的正确性，只影响“程序跑起来是不是真按公式在跑”。

第一件：无扰切换，手操切自动时不跳。

控制器内部有 u 的历史缓冲。若从手动切到自动时缓冲是全零（或上次投运留下的旧值），第一拍算出的 u 会按“过去几拍 $u = 0$”来补偿——对 $d = 2$ 以上的回路，这个补偿量可能很大，切换瞬间就是一个阶跃。正确做法是**切换时用实际输出值填满整个 u 缓冲**：$u[k-1], u[k-2], \ldots$ 全部取当前阀位。这样第一拍的控制量自然等于当前阀位，不会跳。

同样的处理对 y 也适用。若控制器内部存了 y 的历史（GMV 的新息重构需要 $y[k-1]$），切换时应当用当前测量值初始化，不要在程序启动时留一个零。

第二件：把计算与通信延迟算进 d。

本章的 d 是“从这一拍发出 u 到它影响 y 的拍数”。现场从“采样”到“输出真正到达执行器”通常要跨一个扫描周期：采样、运算、通信、D/A 转换。若这四步耗时接近一个采样周期，那么控制器实际能干预的时刻比设计晚一拍，**等效 d 要加 1**。

这一条最容易被忽略，因为它在慢过程（$T_s = 1$ s、对象时间常数几十秒）上几乎看不出来，而在快过程上直接决定成败。判据是：把计算延迟折算成拍数 $d_c = \lceil t_c / T_s \rceil$，设计时用 $d + d_c$ 查表 62-1。d 从 1 变 2，下界从 $1.0000\sigma_e^2$ 涨到 $2.9600\sigma_e^2$（$c = 0.6$），差近 3 倍——**这不是精度问题，是数量级问题**。

第三件：历史缓冲存量化后的值。

若输出经过 D/A 或通信字量化（例如 12 位、0.1% 分辨率），那么真正到达执行器的是量化后的 u。控制器内部若存的是未量化的计算值，两者会慢慢分叉：控制器以为阀在 43.27%，实际阀在 43.2%，差的那一点在 d 拍之后进入 y，又被当成扰动去补偿。

在 MVC 这种高增益控制器上，这个分叉会被放大——本章 MVC 的等效增益 2.8，千分之几的 u 误差对应百分之一的 y 偏差，正好落在下界的量级上。**做法很简单：先量化，再存缓冲。** 与 §62.12.9 说的“限幅要在入缓冲之前”是同一条原则：缓冲里只能存“真正发生的事”。

第四件：退出条件要写死，且退出要无扰。

随机控制器没有“自恢复”能力：模型错了，它不会变保守，只会按错的模型算出更猛的动作。所以必须外挂一层监视：

- R 连续 N 拍超过 1.5（模型明显失配）；
- $|y|$ 超过软限的 80%；
- 控制量连续饱和超过设定时长；
- 参数复标值变化超过 §62.10.7 的阈值。

四条中任一条成立，就自动退回 PID 或手动，并记录现场快照（最近 200 拍的 y、u、模式）。退回时同样要无扰——把 PID 的积分项预置成使输出等于当前阀位的值，否则退回动作本身又是一次扰动。

这四件事没有一件写在控制律里，但它们决定了“设计出来的方差”能不能在装置上兑现。**本章所有关于下界的讨论，都以这四件事已经做对为前提。**

62.11 优缺点与适用边界

62.11.1 六条优点

第一，它给出一个绝对的标尺。 下界 $V_{\min}=\sigma_e^2\sum f_i^2$ 只依赖噪声与延迟，与控制器、与增益 b 都无关。这意味着它是“任何算法都做不到更好”的界线，而不是“这套算法能做到的极限”。前面所有章节的评价指标（超调、调节时间、IAE）都是相对的——换一套参数就变；只有方差下界是绝对的。

第二，结构极简。 式 (62-33) 每拍只需 $d+1$ 次乘法。$d=3$ 的回路在 8 位单片机上跑不满 1% 的 CPU。随机控制的理论门槛很高，实现门槛几乎为零，这个反差在控制领域里很少见。

第三，参数少且都有物理含义。 四个对象参数加一个设计参数，没有“经验系数”。d 是延迟拍数、b 是增益、a 是惯性、c 是噪声相关性、λ 是控制的价格。每个参数都能被单独测量或标定，也能被单独质疑。

第四，它能定量回答“延迟值多少钱”。 表 62-1 与图 62-5 给出的不是定性结论，而是“d 从 2 降到 1，下界降 66%”这样的数。这类数字能直接进投资论证：换一台快阀要多少钱、能省多少方差。

第五，它与 GMV 是一体的。 同一个框架里，$\lambda=0$ 是最小方差、$\lambda\to\infty$ 是不控制，中间连续可调。工程师不需要在两套方法之间切换，只需要在一个旋钮上移动。

第六，它有诊断价值。 $R=\mathrm{Var}_{\text{sim}}/V_{\min}$ 这个比值本身就是诊断工具：$R\approx 1$ 说明模型对，$R\gg 1$ 说明结构错。PID 调参没有这个性质——PID 调不好时，你无法判断是参数问题还是对象问题；而 MVC 的 R 直接把这个判断变成了算术。

62.11.2 七条缺点

第一，控制量太大。 这是最硬的一条。本章标称工况下，输出标准差 1.01，控制标准差 2.82（$\mathrm{var}(u)=7.93$），控制变差 18251。对一个真实的阀，这意味着阀杆每天往复上万次。**MVC 的最优性是用执行机构的寿命换来的。**

第二，增益误差单侧悬崖。 真实增益比模型大 28.6% 就发散（$c=0.6$ 时），而小 55% 只是变差。这条不对称边界意味着：**任何“增益可能变大”的工况都是风险**——结垢初期传热变好、阀门开大后流量增益上升、负荷增加后对象变“灵敏”，全都朝危险方向走。

第三，要求最小相位。 式 (62-14) 的分母有 b，§62.2.9 的对消要求 B、C 都可逆。非最小相位对象（有逆响应、有右半平面零点）不能直接用 MVC——强行用会把不稳定零点搬进闭环。

第四，依赖准确的 d。 d 错一拍，下界可能翻倍，或者控制律对着一拍以前的误差使劲。而 d 恰恰是最难测准的参数（纯延迟、计算延迟、执行器响应三者叠加）。

第五，线性、无约束。 MVC 不知道阀门有量程、电机有电流上限。限幅之后，闭环动态与设计不符，性能没有任何保证。要做约束必须外挂 QP，那时它就已经不是 MVC 了。

第六，只解决调节。 设定值不变时它最优；设定值频繁变化时，它没有积分环节，跟踪会有稳态偏差（除非把 R 设成含积分的形式）。跟踪问题的正解是 PID（第 44 章）或 MPC（第 58 章）。

第七，噪声模型阶次受限。 MA(1) 只能表达 $|\rho_1| \le 0.5$ 的扰动。真实的慢漂移（$\rho_1 > 0.8$）需要高阶 C，而高阶 C 又让 Diophantine 方程与稳定性分析变复杂。

62.11.3 七类不适用场合

把缺点翻译成“什么时候不要用”，得到七类场合。它们不是理论上的禁区，而是**投入产出不划算**的地方。

一、扰动近似白噪声（$\rho_1 < 0.1$）。 此时可预测的部分很小，MVC 相对“什么都不做”只能拿回百分之几的方差，却要付出全部的控制动作。判断方法极简单：算一下 ρ_1，小于 0.1 就停手。

二、非最小相位对象。 有逆响应（先反向再正向）的对象，MVC 会把不稳定零点搬进闭环。这类对象的随机控制要用极点配置或 LQG，不能直接用 MVC。

三、延迟很大且不可压缩。 §62.11.5 会给出定量的收益估算：$a = 0.8$、$c = 0.6$、$d = 10$ 时，MVC 相对不控制只能降 1.5% 的方差。这时正确的动作是改工艺（挪测点、加前馈、缩短流程），不是换算法。

四、强非线性或工况大幅变化。 若 b 在不同工况下相差一倍以上，固定参数的 MVC 必然在某个工况失稳（§62.6.3 的悬崖）。要么做增益调度，要么做自适应，要么别用。

五、执行器受限严重。 若速率限制折算成等效延迟后 d 翻倍，下界会涨近三倍，MVC 的设计前提（u 即时到位）已经不成立。此时应当先解决执行器，或者直接用带约束的 MPC。

六、多变量强耦合。 一个通道的 u 会显著影响另一个通道的 y 时，单回路 MVC 的“其他通道不变”假设失效。这类问题应当用 LQG（第 54 章）或 MPC（第 58、80 章）。

七、以跟踪为主。 设定值频繁变化、且跟踪误差比稳态波动更重要时，随机控制不是主战场。PID 的前馈-反馈复合（第 44 章）或 MPC 的滚动优化更合适。

62.11.4 与四种方法的对比

方法	主要目标	需要对象模型	处理延迟的方式	处理约束	控制量	典型场合
PID（第 44、106 章）	跟踪	不需要	降增益换裕度	靠限幅	中	通用单回路
Smith（第 47、107 章）	跟踪 + 大延迟	需要	显式补偿	无	中	大延迟跟踪
MVC / GMV（本章）	稳态方差	需要（含噪声模型）	计入下界	无	大	稳态调节
MPC（第 58、80 章）	跟踪 + 约束	需要	显式预测	显式	可控	多变量、有约束
LQG（第 54 章）	方差（多变量）	需要	状态增广	无	中	多变量、量测噪声

这张表最该看的是“处理约束”这一列。**只有 MPC 把约束写进了优化**，其余四种都靠限幅兜底。这也解释了为什么工业上最终落地的是 MPC 而不是 MVC：现场的限制条件太多，而无约束的最优解往往不可执行。

再看“控制量”这一列。MVC 是唯一一个把“控制量会变大”当作固有特征的——它不是设计缺陷，而是“最小方差”这个目标的直接后果。GMV 通过 λ 把它拉回来，代价是方差变大。**这条交换曲线（图 62-10）就是本章的全部工程内容。**

62.11.5 收益上界：三分钟判断该不该做

在动手之前，有一个三分钟的估算，能判断这件事值不值得做。只需要三个数：a、c、d。

开环方差（不控制时的输出方差）由 C/A 的脉冲响应平方和给出：

$$V_{\text{open}} = \sigma_e^2 \left[1 + \frac{(a+c)^2}{1-a^2} \right] \tag{62-40}$$

闭环下界由式 (62-18) 给出。两者相除，得到“最多能降多少”：

$$\eta = 1 - \frac{V_{\min}}{V_{\text{open}}} \tag{62-41}$$

表 62-13 是几组典型参数的结果，全部可以手算核对。

a	c	d	$V_{\text{open}}/\sigma_e^2$	$V_{\min}/\sigma_e^2$	η
0.8	0	1	2.7778	1.0000	64.0%
0.8	0.6	1	6.4444	1.0000	84.5%
0.8	0.6	3	6.4444	4.2144	34.6%
0.8	0.6	5	6.4444	5.5310	14.2%
0.8	0.6	10	6.4444	6.3460	1.5%
0.99	0	1	50.2513	1.0000	98.0%
0.99	0	5	50.2513	4.8049	90.4%

表的读法有三条：

第一，c 决定上限。 同样 $d=1$，白噪声的收益上限 64%，$c=0.6$ 时 84.5%。噪声越“有形状”，可拿的越多。这解释了为什么随机控制在有热惯性、有物料混合的流程工业里特别有效——那里的扰动天然是相关的。

第二，d 决定损耗。 $c=0.6$ 时，d 从 1 增到 5，收益从 84.5% 掉到 14.2%；增到 10，只剩 1.5%。**这是“要不要做”最直接的判据：$\eta < 20\%$ 就该停手，去做延迟治理。**

第三，a 接近 1 时收益高但绝对方差也大。 最后两行的 η 都在 90% 以上，看起来很诱人；但开环方差是 50 倍 σ_e^2，闭环下界也有 1～4.8 倍，绝对抖动依然可观，而且控制量极大。**η 高不等于“控制得好”，只等于“相对不控制好很多”。**

本章标称工况（$a=0.8$、$c=0.6$、$d=1$）的 $\eta=84.5\%$，仿真实测的开环方差 6.7812 与理论 6.4444 差 5%，这是 $N=6000$ 的样本涨落。这个一致性本身也是一次校验：**式 (62-40) 与仿真对得上，说明对模型的理解没有偏。**

62.11.6 两个参数同时错：一张二维边界表

表 62-3 是“一次只错一个”的压力测试，而现场更常见的是两三个一起错：换了一台阀（b 变），介质也变了（c 变），控制器还按老参数跑。这一小节把 b、c 两维一起扫，给出一张可以直接查的边界表。

$d=1$ 时闭环有闭式。控制器按标称 $(\hat a,\hat b,\hat c)$ 设计，真实对象是 (a,b,c)，控制律仍是 $u=-\frac{\hat a+\hat c}{\hat b}y$，代入对象方程得

$$y[k]=\rho\,y[k-1]+e[k]+c\,e[k-1],\qquad \rho=a-\frac{(\hat a+\hat c)\,b}{\hat b} \tag{62-42}$$

于是闭环传函是 $(1+cz^{-1})/(1-\rho z^{-1})$，输出方差为

$$\frac{\mathrm{Var}(y)}{\sigma_e^2}=\frac{1+c^2+2\rho c}{1-\rho^2} \tag{62-43}$$

式 (62-43) 比 §62.6.4 的 $1/(1-\rho^2)$ 多了一项 $2\rho c$。这一项只在 $c\neq 0$ 时出现，符号由 ρ 决定：$\rho<0$（增益失配的典型情形）时它把方差**压小**。表 62-5 取 $c=0$ 正是为了让这一项消失，理论与实测才对得那么干净。

式 (62-42) 里没有真实 c。 这是本节最要紧的一句话：闭环极点只由控制器侧的 $\hat a+\hat c$、$\hat b$ 和真实增益比 $b/\hat b$ 决定，真实噪声参数 c 完全不进来。推论有两条——失稳界线的**位置**与真实 c 无关，永远是 $b/\hat b=(1+a)/(\hat a+\hat c)=1.2857$；真实 c 只改变方差的**高度**（通过式 (62-43) 的 $1+c^2+2\rho c$）。

表 62-14 把这两条摆在一起。行是增益比 $r=b/\hat b$，列是真实 c，格子里是 $\mathrm{Var}(y)/\sigma_e^2$（$\hat a=0.8$、$\hat c=0.6$）。

$r=b/\hat b$	$c=0$	$c=0.3$	$c=0.6$	$c=0.9$
0.80	1.114	1.000	1.087	1.375
1.00	1.563	1.141	1.000	1.141
1.20	4.433	2.491	1.348	1.002
1.25	10.26	5.333	2.256	1.026
1.28	62.75	31.05	10.64	1.531
1.30	发散	发散	发散	发散

表的读法有三条，每条都能直接变成现场动作。

第一，悬崖的位置在每一列上都一样。 $r=1.28$ 时四列都还活着，$r=1.30$ 时四列全崩。这与 c 无关，也与上一节说的“$+30\%$ 发散”完全一致。所以判断“离悬崖还有多远”，只需要一个数：r 到 1.2857 的距离。**这一条把二维问题降成了一维**：c 估得准不准，不影响安全性判断。

第二，c 越大，失配越“被掩盖”。 同样是 $r=1.28$，$c=0$ 那格方差 62.75（一眼就知道坏了），$c=0.9$ 那格只有 1.531（看起来只是“略差”）。原因就在 $2\rho c$ 这一项：$\rho=-0.992$、$c=0.9$ 时它等于 -1.786，几乎把 $1+c^2=1.81$ 全部抵消。**这是本章最危险的一张“假象”**——凭方差判断，会以为回路还在正常工作，实际极点已经贴到 -0.992，再动 1% 就发散。

第三，现场只能测方差，所以必须反推 ρ。 由式 (62-43) 解一元二次方程，取负根：

$$\hat\rho=\frac{-c+\sqrt{c^2+V(V-1-c^2)}}{V},\qquad V=\frac{\mathrm{Var}(y)}{\sigma_e^2} \tag{62-44}$$

注意这里是取负根——增益失配让 ρ 往负方向走，正根对应的是“真实增益偏小”那一侧。用它可以从一个方差数反推极点位置，再算“极点余量” $1-|\hat{\rho}|$。表 62-14 里 $c=0.9$、$r=1.28$ 那格：代入 $V=1.531$、$c=0.9$ 得 $\hat{\rho}=-0.991$，余量 0.9%。而 $c=0$、$r=1.28$ 那格虽然方差 62.75 很吓人，$\hat{\rho}=-0.992$，余量同样只有 0.8%——**两格的方差差 40 倍，危险程度却一样**。

用实测数字核对一次：表 62-3 的 $b+20\%$ 实测方差 1.2976，式 (62-43) 给 1.348，差 4%；$c\pm50\%$ 实测 1.1704 / 1.1377，式 (62-43) 给 1.141 / 1.141。四处对照都在样本涨落范围内，说明这张表不是事后拟合，而是先有式子、后有数字。

工程上因此多一条验收动作：**除了看 R，还要用式 (62-44) 反推 $\hat{\rho}$，并留 10% 的极点余量**（即 $|\hat{\rho}|\le 0.9$）。$R\approx 1$ 只说明“当下这一档是对的”，不说明“离悬崖还远”；而 $\hat{\rho}$ 同时回答这两个问题。

62.11.7 边界总结

把整章压成一份“适用条件清单”，正面表述：

- **扰动有色**：$\rho_1>0.1$，且能用低阶 C 描述。
- **延迟可数且不大**：$d\le 5$，且三个来源都能单独核实。
- **对象近似线性**：工作点附近 b 的变化不超过 20%。
- **最小相位**：没有逆响应。
- **增益可标定**：至少能在三个工作点上测出 b。
- **执行器够快**：速率限制折算后不显著增加 d。
- **以调节为主**：设定值长期不变，稳态波动是主要矛盾。
- **能接受控制量上升**：或者愿意用 $\tilde{\lambda}$ 把它换回去。

八条里若有两条以上不满足，就应当考虑别的方法——PID 加前馈、Smith 预估、或者 MPC。**随机控制不是“更高级的控制”，它是“针对一类特定问题的专用工具”。**

一句话收束本章的立场：**MVC 告诉你地板在哪，GMV 让你决定站在地板上还是站高一点，而工程判断决定这间屋子值不值得铺地板。** 三件事缺一不可，且顺序不能颠倒。

62.11.8 一页纸的设计卡

把前面十一节的内容压成一页，贴在控制柜门内侧。它不解释原理，只回答“下一步该做什么”和“做到什么程度算完”。

第一步，量三个数。 用一次阶跃记录定 b（增量比增量，取三个工作点里最大的那个）；用一次“控制器不动”的开环记录定 a 与 C；用一次纯延迟测试定 d，并把这个数拆成“采样周期整数拍 + 计算延迟 + 阀门口令到动作的余量”三项分别记账。三个数都要写进文档，写明测试日期与工况——半年后换季，b 会变，没有原始记录就无法判断是对象变了还是模型错了。

第二步，算两个界。 用式 (62-41) 算下界 $V_{\min}=\sigma_e^2\sum f_i^2$，把它当作这个回路“理论上的最好成绩”；再用 V_{open} 算收益上限 η。若 $\eta<20\%$，说明这个回路的波动主体不在可反馈抑制的部分，装了也白装，应当先去查扰动源（上游设备、公用工程、原料批次），而不是继续在算法上加码。

第三步，选一个 $\tilde{\lambda}$。 从 0 起，按 0.25、0.5、1、2、5 逐档试，每档记录四个数：R、$\mathrm{var}(u)$、$\max|y|$、总变差。停在哪一档由两条硬指标决定——$\max|y|$ 不越报警线，$\mathrm{var}(u)$ 不越执行器的许可磨损量。**在满足这两**

条的前提下取最小的 $\tilde{\lambda}$，因为再大就只是拿波动换阀门寿命，而且换得很不划算（§62.8.9 里那一次，总变差降了 91%，方差却涨了 244%）。

第四步，做一次失配扫描。 把 $\hat{b}$ 按 0.8、1.0、1.2 三档各跑一遍，把 $\hat{a}$、$\hat{c}$ 各偏 20% 再跑一遍，看 R 与 $\max|y|$ 怎么变。**扫描的目的不是求"最坏情况下的性能"，而是确认最坏情况下仍然不发散**——这两件事的验收标准完全不同：前者看数字，后者只看有没有越过 $|y|$ 的物理上限。

第五步，留三条自检。 投运后在线监测：残差是否白（否则 C 或 d 错了）、控制量是否长期贴限幅（否则 $\tilde{\lambda}$ 太小或 b 估高了）、闭环极点反推值 $|\hat{\rho}|$ 是否越过 0.9（否则模型已经漂出安全区）。三条里任一条持续报警，就回到第一步重新标定，而不是去调 $\tilde{\lambda}$。

五步之外还有一句提醒：**这张卡上的每一个数字都带工况标签。** 换配方、换原料、大修之后，卡片要重做；把上一批的 $\hat{b}$ 抄到这一批上用，是这类控制器在现场失效最常见的原因——它不会立刻出事，而是先把 R 悄悄推到 1.5 附近，再在某次负荷变化时直接发散。

62.12 Python 仿真代码

62.12.1 两个文件，一条命令

本章的代码分成两个文件，职责严格分开：

- `code/ch062_sim.py`：模型、控制器、仿真循环、九个实验。它只打印数字，不画图。
- `code/ch062_figs.py`：调用前者的函数取数据，生成 11 张图（SVG + PDF 双份）。

运行方式：

```
D:\Anaconda3\envs\py311\python.exe code/ch062_sim.py
D:\Anaconda3\envs\py311\python.exe code/ch062_figs.py
```

第一条命令把正文里所有数字重新算一遍，约两秒；第二条命令重新生成图，约十秒。**正文里的每个数字都能在这两个脚本里找到出处**，没有手抄的数字，也没有硬编码的坐标。

这种"算—写—画"三分的设计有三个好处：数字可复现、图与正文同源、改参数后所有结论一起更新。代价是写代码的时间比写正文还长，但这笔投入在需要反复核对数字的场合是值得的。

62.12.2 常量与参数约定

脚本开头只有五个常量：

```
A0 = 0.8                    # 对象自回归系数
B0 = 0.5                    # 输入增益
C0 = 0.6                    # 噪声滑动平均系数
SIGMA = 1.0                 # 新息标准差
WARM = 200                  # 丢弃的预热拍数
```

五个常量对应 §62.2.1 的模型。`WARM` 单独列出来，是因为它决定了所有统计量的口径——**预热段不同，方差就不同**，这是复现时最容易产生分歧的地方（§62.12.9 会展开）。

参数约定有三条，写在模块的文档字符串里：

1. 延迟 × 拍用整数 `d` 表示，且 $d \geq 1$；

2. 所有控制器的 `step(y, r=0.0)` 只做单拍计算，状态存在实例里；

3. 调用 `reset()` 才清状态，构造之后不必再手动初始化。

第三条看起来是小事，但它决定了仿真循环能不能复用同一个控制器对象跑多段。本章的实验都是“新建控制器—跑一段—丢弃”，只有失配扫描会复用同一个 `MVC()` 实例，所以 `reset()` 必须真的把历史缓冲清干净。

62.12.3 Diophantine 与下界：二十行

整章的数学核心就是下面这二十行。它解 $C = AF + z^{-d}G$，返回 F 的系数与 G 的常数项。

```python
def mvc_coeffs(a: float, c: float, d: int) -> tuple[np.ndarray, float]:
    """解 C = A*F + z^-d*G，返回 F 的系数与 G 的常数项。"""
    f = np.zeros(d)
    for k in range(d):
        # 右端 C 只有前两项非零：c_0 = 1, c_1 = c
        ck = (1.0 if k == 0 else (c if k == 1 else 0.0))
        f[k] = ck + (a * f[k - 1] if k else 0.0)
    # z^-d 项：-a*f[d-1] + g0 = c_d（d>=2 时 c_d=0；d=1 时 c_1=c）
    cd = c if d == 1 else 0.0
    g0 = cd + a * f[d - 1]
    return f, g0
```

这段代码值得逐行看，因为它把 §62.2.3 的推导压缩成了两个递推。

循环里的 `f[k] = ck + a*f[k-1]` 是在“配平” z^{-k} 的系数：左端 AF 在 z^{-k} 处的系数是 $f_k - a f_{k-1}$，右端 C 在 $k = 0, 1$ 处分别是 $1, c$，在 $k \ge 2$ 处为零。要让两边相等，就得让 $f_k - a f_{k-1}$ 等于 c_k——这正是循环写的东西。

`g0` 那一行配平的是 z^{-d} 项。左端 AF 在这一项是 $-a f_{d-1}$，加上 $z^{-d}G$ 的 g_0，要等于右端 C 在 z^{-d} 处的系数。$d \ge 2$ 时 C 在 z^{-2} 之后全为零，所以 $c_d = 0$；$d = 1$ 时 z^{-1} 项是 c，所以 $c_d = c$。这就是代码里那个 `cd = c if d == 1 else 0.0` 的来历。

下界只要一行：

```python
def mvc_variance(a: float, c: float, d: int) -> float:
    """最小方差下界，单位新息方差下为 sum(f_i^2)。"""
    f, _ = mvc_coeffs(a, c, d)
    return float(np.sum(f ** 2))
```

注意它返回的是**归一化**下界（$\sigma_e = 1$），要与实测比较时乘以 σ_e^2。本章所有实验都取 $\sigma_e = 1$，所以两者可以直接比——这也是 §62.7.6 说的“归一化”的好处。

这两个函数是全章唯一“不可替换”的部分。控制器可以换写法，仿真循环可以换结构，但只要这两个函数对，下界与 F、G 就对，其余结论都能重建。**复现本章时，先核对这两个函数的输出与表 62-1 是否一致，一致了再往下做。**

62.12.4 三个控制器

`MVC` 用多项式形式（式 62-33），不重构新息：

```python
class MVC:
    """最小方差控制：B*F*u[k] + G*y[k] = 0（调节问题，设定值为零）。"""

    def __init__(self, a=A0, b=B0, c=C0, d=1):
        self.f, self.g0 = mvc_coeffs(a, c, d)
        self.b = b
        self.d = d
        self.reset()

    def reset(self):
        self.u_hist = np.zeros(self.d)      # 已发出的控制量，u_hist[0] 是上一拍
        self.trace = []                      # 记录输出，供画图

    def step(self, y, r=0.0):
        acc = self.g0 * (y - r)
        for i in range(1, self.d):
            acc += self.b * self.f[i] * self.u_hist[i - 1]
        u = -acc / (self.b * self.f[0])
        self.u_hist = np.roll(self.u_hist, 1)
        self.u_hist[0] = u
        return u
```

三个细节值得指出。

第一，`self.f[0]` 恒为 1（因为 F 是首一的），所以分母实际是 `self.b`。写成 `self.b * self.f[0]` 是为了与式 (62-33) 逐字对应，读代码时不用回头查公式。

第二，`np.roll` 把整个历史挪一位，复杂度 $O(d)$。d 小的时候无所谓，d 大时应当换成环形缓冲（§62.5.5）。这里保留 `roll` 是因为它最不容易写错，而本章 $d \le 3$。

第三，`u_hist[0]` 存的是**这一拍刚算出的** u，不是上一拍。下一拍进来时，`np.roll` 会把它挪到 `u_hist[1]`，正好对应式里的 $u[k-1]$。这个约定必须与仿真循环里的“先算 y、再调 `step`”配合，否则会整体错一拍。

`GMV` 用新息形式（式 62-21），因为它需要显式的一步预测：

```
class GMV:
    """广义最小方差（d=1）：J=E{y[k+1]^2+lam*u[k]^2}。"""

    def __init__(self, a=A0, b=B0, c=C0, lam=1.0, d=1):
        self.a, self.b, self.c = a, b, c
        self.lam = lam
        self.d = d
        self.reset()

    def reset(self):
        self.y_prev = 0.0
        self.u_prev = 0.0
        self.e_prev = 0.0
        self.trace = []

    def step(self, y, r=0.0):
        # 用模型重构这一拍的新息
        e = y - self.a * self.y_prev - self.b * self.u_prev - self.c * self.e_prev
        # 一步预测：yhat = a*y + c*e
        yhat = self.a * y + self.c * e
        u = -self.b * yhat / (self.b ** 2 + self.lam)
        self.y_prev, self.u_prev, self.e_prev = y, u, e
        return u
```

`GMV` 只支持 $d=1$。这不是偷懒，而是有意把“预测+定价”的结构暴露出来：$d\geq 2$ 时要先解 Diophantine 得到 F、G，再用 F 做多步预测，代码会长三倍，而结论（λ 把增益按 $1/(1+\tilde{\lambda})$ 缩放）完全一样。**教学代码宁可短，也不要为了通用性把主要结构埋掉。**

`FixedGain` 是最短的一个，但它承担了全章一半的对照：

```
class FixedGain:
    """固定比例调节 u=-k*y，对照用。"""

    def __init__(self, k=1.0):
        self.k = k

    def reset(self):
        self.trace = []

    def step(self, y, r=0.0):
        return -self.k * (y - r)
```

它存在的意义是回答“随机控制到底比一个调好的 P 控制器好多少”。§62.5.3 与 §62.8.6 的所有对照（$k=0.5$、$k=1.6$、不控制）都来自这个类。**没有它，MVC 的数字就没有参照系。**

三个类都实现了 `reset()` 与 `step()`，这是仿真循环能对它们一视同仁的前提。`reset()` 里三行状态清零、一行 `trace` 清空，看起来琐碎，但少一行就会在失配扫描里留下上一段的残留——这类 bug 不报错，只让数字悄悄偏掉。

62.12.5 仿真主循环：时序是唯一容易错的地方

```
def simulate(ctrl, N=2000, d=1, a=A0, b=B0, c=C0, sigma=SIGMA, seed=1,
             markov=None, umax=None):
    """闭环仿真。markov=(P, [(a,b,c), ...]) 时逐拍切换参数。"""
    rng = np.random.default_rng(seed)
    e = rng.normal(0.0, sigma, N)
    y = np.zeros(N)
    u = np.zeros(N)
    ctrl.reset()
    mode = 0
    params = [(a, b, c)]
    if markov is not None:
        P, params = markov
    y_prev, e_prev = 0.0, 0.0
    diverge, stop = False, N
    modes = np.zeros(N, int)
    for k in range(N):
        # 1) 先决定这一拍处于哪个模式
        if markov is not None and k > 0:
            if rng.random() < P[mode, 1 - mode]:
                mode = 1 - mode
        modes[k] = mode
        ak, bk, ck = params[mode]
        # 2) 用"上一拍已发出的 u"算这一拍的 y
        yk = ak * y_prev + (bk * u[k - d] if k >= d else 0.0) + e[k] + ck * e_prev
        # 3) 发散保护：一旦溢出就停，尾部填 nan
        if not np.isfinite(yk) or abs(yk) > 1e6:
            diverge, stop = True, k
            break
        y[k] = yk
        # 4) 控制器看到"这一拍的 y"，算出"这一拍的 u"
        uk = ctrl.step(yk, 0.0)
        if umax is not None:
            uk = float(np.clip(uk, -umax, umax))
        u[k] = uk
        y_prev, e_prev = yk, e[k]
        ctrl.trace.append(uk)
    if diverge:
        y[stop:] = np.nan
        u[stop:] = np.nan
    return {"y": y, "u": u, "e": e, "a": a, "b": b, "c": c, "d": d,
            "mode": modes, "diverge": diverge, "stop": stop}
```

循环里的四步顺序是本章代码最值得核对的部分。换成别的顺序，程序照样跑，数字会错。

第 2 步用的是 `u[k-d]`，不是 `u[k]`。 这一条把“延迟”落实成了数组下标。$k < d$ 时还没有任何控制量到达对象，所以那一项取零——这是物理事实，不是补零的权宜。若写成 `u[k]`（当前拍的控制量），延迟就消失了，所有关于 d 的结论都会失效。

第 4 步在 `y[k]` 算完之后才调 `step`。 控制器看到的是当拍测量值，这正是离散控制的标准时序。若先调 `step` 再算 y，控制器就看到了“未来”，闭环会虚假地变好——这类错误在小 d 时不容易发现，在 d 大时会让仿真结果与理论严重不符。

限幅在第 4 步之内、入 `u` 数组之前。 这样 `u[k]` 存的是真正发出去的值，下一拍的 `u[k-d]` 才是对的。若把限幅放在循环外（对整段 `u` 做 `clip`），历史缓冲里就是限幅前的值，闭环会按“执行器做了没做的事”来预测——这正是 §62.5.2 说的那个坑。

发散保护放在第 3 步。 失配实验里 $\hat{b}$ 偏小到临界点时，`y` 会指数增长，几十拍内就溢出成 `inf` / `nan`。没有这个保护，后面所有统计量都会被污染成 `nan`，整张表不可读。保护之后，`metrics()` 一看到 `diverge` 就返回全 `inf` 的字典，表格里显示“发散”，含义清楚。

`markov` 分支只在 $k > 0$ 时切换，且用 `rng.random() < P[mode, 1-mode]`。这个写法等价于按行抽样，但比 `rng.choice` 快，也不会在每拍调用时重建分布对象。模式序列存进 `modes`，供图 62-8 的上半图使用。

62.12.6 指标：五个数定一份设计

```
def metrics(res, warm=WARM):
    y, u = res["y"][warm:], res["u"][warm:]
    if res.get("diverge"):
        return {"mean_y": np.nan, "var_y": np.inf, "std_y": np.inf,
                "var_u": np.inf, "std_u": np.inf, "max_y": np.inf, "tv_u": np.inf}
    return {
        "mean_y": float(np.mean(y)),
        "var_y": float(np.var(y)),
        "std_y": float(np.std(y)),
        "var_u": float(np.var(u)),
        "std_u": float(np.std(u)),
        "max_y": float(np.max(np.abs(y))),
        "tv_u": float(np.sum(np.abs(np.diff(u)))),
    }
```

七个量，正文主要用五个：`var_y`（性能）、`var_u`（代价）、`max_y`（风险）、`tv_u`（执行器磨损）、`mean_y`（有没有静差）。

`tv_u` 的定义值得单独说：它是相邻拍控制量差值绝对值之和，也就是控制信号的**总变差**。选它而不选 `std_u` 的原因是：同样方差的控制信号，快速来回切换的比缓慢变化的更伤执行器，而方差看不出这个差别。§62.5.3 里 MVC 的方差是 7.93、总变差 18251，固定比例 $k = 1.6$ 的方差是 4.20、总变差 9209——**比值差了近一倍，总变差把这个差别放大了。**

`warm=WARM` 默认丢掉前 200 拍。这个默认值写在函数签名里，而不是调用时逐个传，是为了让“所有实验口径一致”成为默认行为。要改口径只有一个地方要改。

`diverge` 分支返回 `np.inf` 而不是抛异常，是因为调用方要做表格：一行 `inf` 打印出来是 “inf”，含义清楚；抛异常则整个实验中断，后面的对照都拿不到。

62.12.7 九个实验各自回答什么

`main()` 里依次调用九个函数。它们不是随便凑的，每个都对应正文的一个论断。

实验 1 `exp_mvc`：下界能不能达到。 对 $d = 1, 2, 3$ 各跑 6000 拍，打印 F、g_0、下界、实测、比值。它验证的是式 (62-18)，结果是比值落在 1.008～1.026（表 62-2）。这是全章的地基实验——如果这一条不成立，后面所有讨论都没有意义。

实验 2 `exp_compare`：MVC 值不值。 同一段噪声下跑 MVC、$k=1.6$、$k=0.5$、不控制四组。它给出“方差降 6.7 倍、控制方差从 0 升到 7.93”这组对照（§62.5.3），也是图 62-4 的数据来源。这个实验的设计要点是**同一 seed**：四组必须面对同一串噪声，否则比较没有意义。

实验 3 `exp_gmv`：λ 的交换曲线。 扫九个 λ 值，打印五个指标。它产出表 62-8 与图 62-10。九个点跨度四个数量级（0 到 5），这样双对数坐标上的曲线才平滑。

实验 4 `exp_delay`：延迟的代价。 纯解析，不跑仿真：$d=1$ 到 5 各算一次 F、g_0、下界。它产出表 62-1 与图 62-5。这个实验跑得最快（毫秒级），却是全章结论最硬的一个——**下界与增益 b 无关，所以它不需要仿真验证。**

实验 5 `exp_mismatch`：失配的形状。 十二组参数：标称、$a\pm10\%$、b 从 $+20\%$ 到 $+50\%$、b 从 -20% 到 -55%、$c\pm50\%$。它产出表 62-3 与图 62-6。设计要点是**控制器始终按标称参数设计**，只改对象——这才能模拟“模型错、对象真”的现场情形。b 的正向扫描必须扫到 30% 以上，否则会漏掉发散区（这是本轮开发时踩过的坑）。

实验 6 `exp_stability`：失稳的机理。 取 $c=0$ 使理论可解析，扫 $\hat{b}$ 九个值，同时打印理论 ρ、理论方差、实测方差。它产出表 62-5 与图 62-7。设计要点是**控制器用 $\hat{b}$、对象固定用 $b=0.5$**，且 c 必须取 0——若沿用 $c=0.6$，理论式 $1/(1-\rho^2)$ 就不成立，理论与实测会对不上。

实验 7 `exp_markov`：跳变系统按谁设计。 四种口径：按模式 0 设计跑跳变、只在模式 0、只在模式 1、按模式 1 设计跑跳变。它产出表 62-6 与图 62-8。设计要点是**四个口径必须同 seed**，否则“发散”可能只是噪声序列不同造成的。

实验 8 `exp_ident`：c 能不能现场估出来。 四个真值 $c=0,0.3,0.6,0.9$，各跑 6000 拍，用标量 RLS 估 c，打印终值、尾段均值、标准差。它产出图 62-11。要说明的是：这个实验里回归量用的是仿真产生的新息 $e[k-1]$（相当于“新息可观测”的理想情形）；现场要用式 (62-6) 重构，精度会略低，但收敛性相同。

实验 9 `exp_mc`：单次结果有多可信。 20 次独立运行、每次 3000 拍，统计方差分布。它产出图 62-9 与 §62.6.7 的结论：均值 0.9938、标准差 0.0313、最小 0.9167、最大 1.0489。设计要点是**改变 seed、其他全不变**——只有这样才能把“随机涨落”从“系统偏差”里分出来。

62.12.8 画图脚本：十一张图如何与数字同源

`ch062_sim.py` 只打印数字，一张图都不画；出图全部交给 `ch062_figs.py`。这个分工不是洁癖：数字要能被复制进表格、被 diff 比对，图要能被反复重画、换配色、改字号。两者混在一个脚本里，改一次图注就得重跑所有实验，而实验里有马尔可夫和蒙特卡洛，跑一次几十秒。

脚本的头几行是固定的三件事：把 `book/`、`book/styles/`、`book/code/` 三个目录塞进 `sys.path`，然后 `import ch062_sim as sim`，最后 `OUT = BOOK / "figs" / "ch062"`。`sys.path` 里必须包含 `book/code`，否则 `import ch062_sim` 找不到——`ch062_figs.py` 自己就在 `code/` 下，但因为启动时的工作目录是 `book/`，相对导入不会生效。

```
BOOK = Path(__file__).resolve().parents[1]
sys.path.insert(0, str(BOOK))
sys.path.insert(0, str(BOOK / "styles"))
sys.path.insert(0, str(BOOK / "code"))

import ch062_sim as sim
from plot_styles import (DOUBLE, DOUBLE_TALL, LINE_BOUND, LINE_COMP, LINE_MAIN,
                         LINE_REF, SINGLE, SINGLE_TALL, apply_cjk_font, save, setup)
from block_styles import block, canvas, export, label, signal, sum_node
from flow_styles import canvas as fcanvas
from flow_styles import decision, process

OUT = BOOK / "figs" / "ch062"
setup()
```

这里的四个模块各有分工，用错会立刻出问题：

模块	提供什么	用在哪几张图
`plot_styles`	画幅常量（`DOUBLE` / `SINGLE` / `*_TALL`）、四条线型（`LINE_MAIN` 等）、`setup()` 全局字体、`save()` 存数据图	图 62-1、62-4～62-11
`block_styles`	方框、求和点、信号线、文字标注、`canvas()`、`export()` 存框图	图 62-3
`flow_styles`	流程图专用的 `process`、`decision` 与 `fcanvas()`	图 62-2
`apply_cjk_font`	单独给某个文本对象指定中文字体	需要混排时手动调

`save()` 与 `export()` 的区别必须说清：`save()` 面向 `matplotlib` 的 `fig` 对象，同时导出 SVG 与 PDF 两份；`export()` 面向 `block_styles` 画出来的框图，内部也是同一套导出逻辑，但它会按框图的坐标范围裁边，不会留大片白。数据图走 `save()`、框图走 `export()`，这个配对在整本书里是统一的，换章时照抄即可。

十一张图的画法各有一个要点。

图 62-1（`fig02_noise`）用三个子图共享横轴，分别画 $c=0$、0.6、0.9 时同一段 160 拍的开环输出，纵轴统一钉在 $[-4,4]$。**统一纵轴是这张图的关键**——若让 `matplotlib` 自动缩放，三者看起来“波动差不多大”，而实际上 $c=0.9$ 的方差是 $c=0$ 的 1.81 倍。三张子图用同一个 `seed=23`，所以噪声源是同一串 $e[k]$，差别只来自 $C(z^{-1})$ 的整形——这才叫“控制变量”。

图 62-2（`fig11_flow`）是设计流程图，用 `fcanvas()` 开画布、`process()` 画方框、`decision()` 画菱形判断。判断框只有两个出口：下界收益够不够、噪声模型可不可信。这两个问题任一为“否”，流程就走到“改用 PI 或 MPC”的分支——这正是 §62.11 的结论在流程图上的投影。

图 62-3（`fig01_structure`）是信号流框图。它要表达的是本章最容易被忽略的一件事：**A 同时出现在对象与噪声模型里**。所以图里“对象 B/A”和“噪声模型 C/A”两个框并列，中间一个求和点，控制器的反馈从 y 取。若把噪声模型画成独立的 C/D，读者会以为噪声动态与对象动态无关，式 (62-7) 的 $C=AF+z^{-d}G$ 就无从解释。

图 62-4（`fig03_mvc`）上下两栏：上栏 $y[k]$，下栏 $u[k]$，横轴前 120 拍。上栏叠一条 $\pm 2\sigma_{\min}$ 的水平带（`LINE_BOUND`），一眼能看出“输出几乎贴着下界在抖”。下栏用 `LINE_COMP` 画控制量，抖得比 y 更凶——这是 MVC 的典型形态，也是 §62.11 说它“费执行器”的直观来源。

图 62-5（`fig04_delay`）是纯解析曲线：横轴 $d = 1 \sim 5$，纵轴下界，一根线；同一张图上再画一条 $a = 0.99$ 的对照线。两条线的间距随 d 迅速拉开，这就把“延迟的代价按 a^2 递减、但 a 越接近 1 越贵”两句话画成了同一张图。

图 62-6（`fig06_mismatch`）是失配曲线，横轴用 $b/\hat{b}$ 的对数刻度，纵轴方差的对数刻度。**必须用双对数**：1.2857 附近方差从个位数跳到几十，线性坐标会把发散区压成一条竖线，看不见。

图 62-7（`fig08_stability`）把理论 ρ、理论方差 $1/(1-\rho^2)$、实测方差三条线画在一起，横轴 $\hat{b}$。理论线在 $\hat{b} = 0.2222$ 处有一根竖直渐近线，实测点在越过该点后直接断掉（因为发散被 `diverge` 标记成 `nan`）。**这条断口本身就是结论**。

图 62-8（`fig07_markov`）上下两栏：上栏画模式序列（0/1 的阶梯图），下栏画 $y[k]$。模式切换的时刻在下栏用竖虚线标出，读者能直接看到“每次切换后输出要喘几拍”。竖线用 `axvline` 配 `DOT` 样式，不抢主曲线。

图 62-9（`fig10_mc`）是 20 次运行的方差直方图，横轴方差、纵轴频次，再叠一条下界竖线。**下界线画在 1.0 的位置，直方图的中心在 0.99**，两者贴得很近——这张图想说的就是“单次跑出来的方差会有 $\pm 3\%$ 的涨落，别拿一次结果下结论”。

图 62-10（`fig05_gmv`）是 λ 的交换曲线，横轴 $\tilde{\lambda}$ 对数刻度，纵轴方差与控制方差两条线，都做归一化。两条线在 $\tilde{\lambda} = 1$ 附近交叉，交叉点的物理含义是“性能代价与控制代价相等”。

图 62-11（`fig09_ident`）四条收敛曲线，横轴拍数（对数），纵轴 $\hat{c}$，四条水平虚线标真值。**横轴取对数**是因为标量 RLS 在前 200 拍就把误差压到 10% 以内，线性横轴会把收敛过程挤成一条竖线。

图内文字有一条硬规矩：中文用纯文本，公式用 `$...$`，两者不写进同一个字符串。 例如标题写 `"延迟的代价"`，不写 `"延迟 d 的代价 $d$"`；若必须带符号，就整个改成 `r"$d$ 增大时的方差下界"`，让整串都是公式模式。原因是本书用 `matplotlib` 的 `mathtext` 渲染公式，中文字符进入公式模式会缺字形，出来是一串方框。这条规矩在 `cards/ch062.md` 的“写作边界”里也写了一遍，属于验收项。

最后是 `main()`：十一张图按编号顺序调用，每张图前后各打一行进度。这样中途报错时，看最后一行就知道卡在哪张图上。全部跑完约十几秒，其中马尔可夫图最慢（`N=8000`），其余都在毫秒到秒级。

62.12.9 复现时最容易踩的五个坑

这套代码很短，两百多行，但短不等于好复现。本章的数字被反复引用（表 62-2 的 1.008～1.026、表 62-5 的 2.78、§62.6.7 的 0.9938），换一台机器重跑能不能对上，取决于下面五件事有没有一致。它们按“错了以后有多难发现”排序，第一个最隐蔽。

坑一：`WARM` 口径不一致，方差差几个百分点。

`WARM = 200` 是全局常量，`metrics()` 的默认参数就是它。但这个默认值只管“直接调 `metrics(res)`”的情形；一旦有人在别处写成 `metrics(res, warm=0)`，或者干脆自己 `np.var(res["y"])`，算出来的就已经不是同一个量了。

预热段不是平稳过程：y 从零起步，控制器内部状态也从零起步，前几十拍的方差比稳态小，所以**少丢预热通常让方差偏小**；但 $d \geq 2$、参数失配或限幅介入时，启动瞬态可能是反向的（补偿动作大、输出先冲一下），方差反而偏大。**方向随工况变，这才是它不能“随手改一下”的原因。** 本章 $d = 1$ 的实测比值 1.008 是在

`WARM=200` 下得到的；换成 `WARM=0`，读者会看到“没达到下界”或“超过下界”的假象，而这两种假象都不属于理论问题。

更要紧的是这个偏差**随 N 变化**：$N=6000$ 时 200 拍占 3.3%，$N=3000$ 时占 6.7%。所以同 `WARM`、不同 N 的两组结果也不能直接比。§62.6.7 的蒙特卡洛统一用 `N=3000`、`WARM=200`，这条口径必须与表 62-2 的 `N=6000` 分开记——两组数各自自洽，混着比就错。

工程上的对策只有一个：**口径写进函数签名，不写在调用处**。本章把 `warm=WARM` 放在 `metrics` 的默认参数里，就是为了让“改口径”这件事只有一个入口。若哪天要改 `WARM`，改常量即可，九个实验一起变；散落在九处的 `warm=200` 则一定会漏掉一处。

坑二：先 `step` 后算 y，控制器看到未来。

§62.12.5 强调过四步顺序，这里再说一遍它错起来什么样。若把循环写成

```
u[k] = ctrl.step(y_prev)          # 错：用的是上一拍的 y
y[k] = a*y_prev + b*u[k-d] + ...  # 此时 u[k] 还没进入延迟链，看似无害
```

$d=1$ 时 `u[k-d]` 就是 `u[k-1]`，这一版其实还能跑出接近的数；但一旦有人“顺手优化”成 `b*u[k]`，延迟就被抹掉了，而程序不报错，只是方差变小、控制量变小，一切看起来“更好了”。这类错误在 $d=1$ 时最难发现，因为 $d=1$ 的正确答案本来就很接近。**判断办法**：把 d 改成 5 重跑，若下界没跟着涨到 4.66，就说明延迟链没生效。

还有一种更隐蔽的写法：`ctrl.step(yk)` 里 `yk` 用对了，但控制器内部把 `self.y_prev` 更新放在计算 `u` 之前。这时 `u` 用的是“刚存进去的当拍 y”，等于把一步预测变成了零步预测。GMV 的 `step` 里那三行赋值必须在 `u` 算完之后，顺序反了不会报错，只会让 λ 的效果变钝。

坑三：限幅放在循环外，历史缓冲存了没发出去的值。

`simulate()` 的 `umax` 分支是在 `u[k] = uk` 之前 `clip` 的：

```
uk = ctrl.step(yk, 0.0)
if umax is not None:
    uk = float(np.clip(uk, -umax, umax))
u[k] = uk
```

若改成循环结束后 `u = np.clip(u, -umax, umax)`，数组里的 u 是限幅后的，但控制器内部的 `u_hist`、`u_prev` 存的仍是限幅前的值。结果是：对象按“限幅后的 u”响应，控制器却按“没限幅的 u”预测，两边越走越远。饱和越深，偏差越大，最后表现为“限幅之后反而发散了”——而限幅本该是保护措施。

本章的 Saturating 讨论（§62.9.4）之所以强调“限幅是控制器的一部分，不是后处理”，根子就在这里。真实现场更要注意：执行器卡在 4 mA 或 20 mA 时，控制器必须知道这件事，否则积分项会一路积到天上（windup）。

坑四：`d` 的两种记法混用。

本章的 d 是“控制量到输出的拍数”，$d=1$ 表示这一拍发的 u 下一拍才影响 y。但有些教材把 d 定义成“纯延迟的拍数”，$d=0$ 表示当拍就到。两种记法差 1，而 $f_k=(a+c)a^{k-1}$ 与 g_0 的表达式会跟着变。

本章的 `mvc_coeffs` 用的是前者：`f` 的长度是 `d`，`f[0]=1`，`g0 = cd + a*f[d-1]`，其中 `cd = c if d==1 else 0.0`。这个 `cd` 就是两种记法的接缝——$d=1$ 时 C 的 z^{-1} 项落在 G 上，$d\ge 2$ 时落在 F 的末项附近。若把 `cd` 恒取 c，$d=2$ 时 g_0 会多出 c，控制律增益从 2.24 变成 3.44，闭环立刻不对。

核对办法：$d=1$、$a=0.8$、$c=0.6$ 时 g_0 必须等于 2.8（即 $a+c$）。这一条在 `exp_mvc` 的第一行就打印出来了，跑一次就能确认记法没混。

坑五：`seed` 用错位置。

九个实验各自 `seed`，这是刻意的：实验之间互相独立，改一个实验不会牵动另一个。但**实验内部**必须同 `seed`——`exp_compare` 的四组控制器、`exp_markov` 的四个口径，都要面对同一串 $e[k]$，否则比较的是“控制器 + 噪声”的混合差异。

一个具体的反例：有人为了“多次独立”，在 `exp_compare` 里给四组各配一个 `seed`，结果 MVC 的方差 7.93、固定增益的 4.20 就不再可比——因为固定增益那组恰好抽到了一串“温和”的噪声。**判断办法**：四组的 `var_y` 与 `var_u` 若不能由同一串 e 核算出来（例如 `exp_compare` 里不控制那组的方差应恰好等于 $\sigma_e^2\sum$ 的理论值），就说明 `seed` 错位了。

蒙特卡洛实验是唯一要“故意变 `seed`”的地方，而它的变法是**只变 `seed`**。§62.6.7 的 20 次运行，每次除了 `seed=i` 之外全部参数一致，这才能把 0.0313 的标准差归给随机性，而不是归给某个被顺手改掉的参数。

把这五条并排看，会发现它们有一个共同点：**都不会让程序报错**。变量名对、语法对、跑得通、图也画得出来，只是数字悄悄偏了。随机控制的仿真尤其容易这样——输出本来就是噪声，肉眼很难判断“这个波动是噪声还是错误”。所以本章的每个关键结论都留了一个可解析核对的锚点：$d=1$ 的 $g_0=2.8$、下界 $1.0000\sigma_e^2$、$d=2$ 的下界 $2.9600\sigma_e^2$、失稳界线 1.2857、$c=0$ 时的 $\rho=a(1-b/\hat{b})$。**先对这些锚点，再看曲线**，是复现这套代码唯一可靠的路子。

62.12.10 手算一遍 $d=2$：三分钟核对代码

代码能跑不等于代码对。最快的自检是手算一个 $d=2$ 的例子，把结果和 `exp_mvc` 打印的前几行对照。取 $a=0.8$、$b=0.5$、$c=0.6$、$\sigma_e=1$。

第一步，解 Diophantine。 $C=1+0.6z^{-1}$，$F=f_0+f_1z^{-1}$，$G=g_0$。把 AF 乘开：

$$(1-0.8z^{-1})(f_0+f_1z^{-1})=f_0+(f_1-0.8f_0)z^{-1}-0.8f_1z^{-2}$$

要与 $C-z^{-2}G=1+0.6z^{-1}-g_0z^{-2}$ 逐项相等，得三个方程：$f_0=1$；$f_1-0.8f_0=0.6$，即 $f_1=1.4$；$0.8f_1=g_0$，即 $g_0=1.12$。

这三个数与递推式 $f_k=(a+c)a^{k-1}$、$g_0=af_{d-1}$ 完全一致：$f_1=1.4$（$=a+c$），$g_0=0.8\times1.4=1.12$。**手算与递推对上，说明 `mvc_coeffs` 没写错。**

第二步，写控制律。 $BFu+Gy=0$，其中 $BF=0.5(1+1.4z^{-1})$。展开成时域：

$$u[k]=-\frac{1.12\,y[k]+0.5\times1.4\,u[k-1]}{0.5\times1}=-2.24\,y[k]-1.4\,u[k-1]$$

这就是 §62.2.7 给的那个式子。注意两个系数：$2.24=g_0/(bf_0)$ 是“当前输出”的权重，$1.4=f_1$ 是“上一拍控制量”的权重——**$u[k-1]$ 的系数就是 F 的第二个系数**，这是 BFu 里 b 上下对消的直接结果。

第三步，算下界。 $V_{\min}=\sigma_e^2(f_0^2+f_1^2)=1+1.96=2.96$。与表 62-1 的 2.9600 一致。

第四步，反向验证闭环。 把控制律代回对象，闭环应当满足 $y[k+2]=e[k+2]+1.4\,e[k+1]$——两个新息的加权和，没有极点。这一步是“极点被消掉”的直接检验：若代码里的 G 算错，闭环会多出一个 a 的极点，方差也会比 2.96 大。

四步加起来不超过三分钟，但覆盖了代码里最容易错的四处：递推的方向、cd 的取值、控制律的符号、下界与 F 的对应关系。**若只允许做一个自检，就做这个 $d=2$ 的例子**——它同时检查了 $d=1$ 检查不到的分支（$cd=0$、`u_hist` 的长度、`np.roll` 的方向）。

第 63 章 鲁棒控制进阶

第 62 章把“噪声有多大”变成了可以计算的方差下界，前提是模型准确。现实的回路里，模型从来不准：阀门的实际流通能力是铭牌值的 1.4 倍，物料的停留时间随产量在 0.5 到 1.5 秒之间漂，管壁结垢让时间常数一年里翻了一番。鲁棒控制要回答的是：**在这些“不准”同时发生、并且发生在最坏的那个组合上时，回路还能不能稳、性能会掉到多少、为了不掉下去要付什么代价。** 本章把这三个问题都落到同一台一阶惯性加纯延迟的对象上，用实测数字回答。

63.1 动机与直观理解

63.1.1 一条“标称漂亮、现场失控”的回路

先看一个具体的对照。对象是一阶惯性加纯延迟

$$P(s) = \frac{K}{Ts+1}e^{-\theta s} \tag{63-1}$$

标称参数 $K_0 = 2.0$、$T = 5$ s、$\theta_0 = 1.0$ s。现场给出两个不确定区间：增益 $K \in [1.2,\ 3.0]$，也就是标称值的 -40% 到 $+50\%$；延迟 $\theta \in [0.5,\ 1.5]$ s，即 ± 0.5 s。这两个区间的来源很朴素——增益的不确定来自阀门特性与流量系数的偏差，延迟的不确定来自输送管路的长度与流速变化。注意时间常数 T 在这里被当作已知，这是本章刻意做的一个简化：把不确定性限制在两个参数上，才能把“最坏情况”逐个顶点枚举出来核对。

在这台对象上设计三个 PI 控制器，参数分别取 $(k_p, T_i) = (0.60, 10)$、$(1.00, 6)$、$(1.80, 3)$，指标列在表 63-1 里。

控制器	k_p	T_i	ω_c /(rad/s)	相位裕度 /(°)	标称 $\lvert S\rvert$ 峰值	最坏 $\lvert S\rvert$ 峰值
保守 PI	0.60	10.0	0.18	98.2	1.212	1.562
折中 PI	1.00	6.0	0.39	71.9	1.420	2.522
激进 PI	1.80	3.0	0.76	37.3	2.297	86.249

表 63-1　三个 PI 在标称点与最坏点上的灵敏度峰值

表的最后两列是全章最重要的一个对照。折中 PI 在标称点上 $\lvert S\rvert$ 峰值 1.420，看起来比保守 PI 的 1.212 只差一点，但它已经用上了 71.9° 的相位裕度；一旦把对象推到最坏端点，$\lvert S\rvert$ 峰值跳到 2.522。激进 PI 的落差更极端：标称 2.297，最坏 86.249，**相差 37 倍**。而到了时域，激进 PI 在 $K = 3.0$、$\theta = 1.5$ 这个端点上直接发散，阶跃响应的峰值冲到 50.462，按超调算就是 4626.75%。

这就是“标称设计”最典型的失败方式：**设计者看到的是一张 Bode 图，而现场运行的是那张图上的另一个点。** 更麻烦的是，最坏点不在标称点附近，它落在参数区间的角上，而设计时用来画图的那条曲线恰好不经过那里。

63.1.2 不确定性从哪来：两个数就够描述

把不确定性写进框图是本章的第一个动作。图 63-1 给出标准形式：标称对象 $P_0(s)$ 之外挂一个“误差块” Δ，真实对象是 P_0 与 Δ 的某种组合。乘性形式写作

$$P(s) = P_0(s)\big[1 + W(s)\Delta(s)\big], \qquad \|\Delta\|_\infty \le 1 \tag{63-2}$$

其中 $W(s)$ 是**不确定性权重**，它的模在每一频率上给出“允许的误差半径”；Δ 是一个模不超过 1 的任意稳定传递函数，代表“误差落在半径内的哪个方向”。这个分解看起来抽象，实际含义很直接：W 回答“我有多不确定”，Δ 回答“我不确定成什么样子”。鲁棒控制的全部结论都要对**所有**满足 $\|\Delta\|_\infty \le 1$ 的 Δ 成立，这就是“最坏情况”的数学含义。

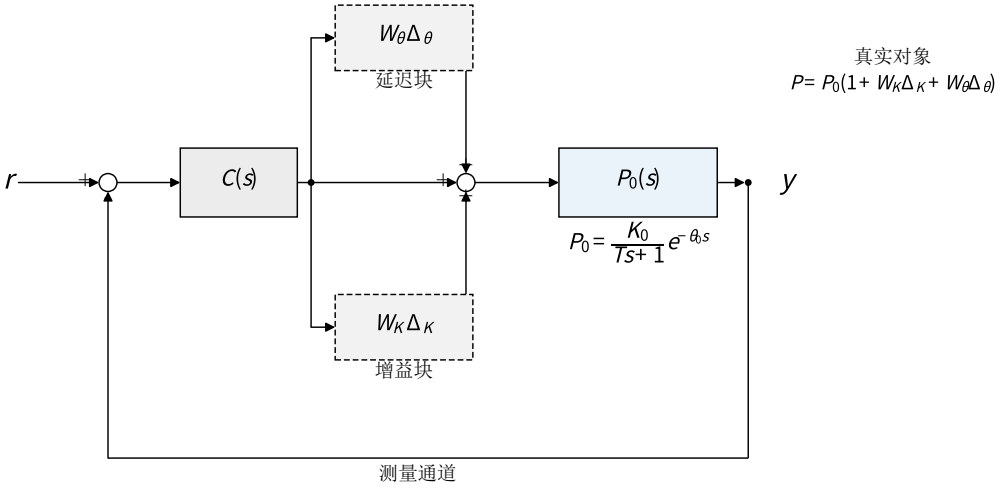


图 63-1　乘性不确定性下的标准反馈结构

图 63-1 里，W 与 Δ 串在对象的前向通道上，$K(s)$ 是控制器。判据要回答的问题是：当 Δ 在单位球里任意游走时，闭环极点会不会跑到右半平面去。

权重 W 从哪来？本章用两个区间直接算。增益的相对误差半径是

$$w_K = \max\left(\frac{K_{\max}}{K_0} - 1,\; 1 - \frac{K_{\min}}{K_0}\right) = 0.500 \tag{63-3}$$

延迟的误差则不是加减关系，它进入的是 $e^{-j\omega\theta}$ 这个相位因子。取 $\theta = \theta_0 + \Delta\theta$、$|\Delta\theta| \le 0.5$ s，两块相除得到

$$\left|e^{-j\omega\theta} - e^{-j\omega\theta_0}\right| = \left|e^{-j\omega\Delta\theta} - 1\right| = 2\left|\sin(\omega\Delta\theta/2)\right| \tag{63-4}$$

这个式子说明延迟误差的“半径”随频率增长，低频几乎为零、高频以 $\omega\Delta\theta$ 的速度线性上升。这一点和增益误差完全不同——增益误差在整个频段上是一个常数 0.500，延迟误差却是一条随频率抬头的曲线。两条曲线怎么合成一个 W，是 §63.2 要处理的技术细节，而合成的不同做法带来的保守性差别，是本章给出的第一个定量结论。

63.1.3 三条路：小增益、QFT、μ

面对式 (63-2) 这种结构，历史上有三条技术路线，它们的区别只在“怎么处理 Δ”。

第一条是小增益定理。 它把 Δ 当成一个范数不超过 1 的黑箱，完全不看它的内部结构，直接要求“环路增益小于 1”：

$$\|WT\|_\infty < 1 \tag{63-5}$$

好处是简单，一句话就能判；坏处是它把 Δ 当成“最恶毒的任意扰动”，而实际的不确定性往往只作用在少数几个通道上，小增益定理看不见这个限制，于是给出偏保守的结论。

第二条是定量反馈理论（QFT）。 它不解析地处理 Δ，而是把不确定性“画”出来：把参数区间里所有可能的对象频率响应都算一遍，在尼科尔斯图上形成一片“模板”（template），然后要求控制器把这片模板整体推到满足指标的边界之外。QFT 的强项是**直接把性能指标画在图上**——比如“所有情况下 $|S| \le 2.2$”就是一条边界线，设计者用眼睛看控制器够不够。它的弱项是设计过程依赖人工试凑，模板一多（多个参数同时不确定）就非常费力。

第三条是 μ 分析。 它试图把前两条的优点合起来：既保留 Δ 的结构信息（哪些通道有不确定性、各自多大），又给出一个可以计算的数。结构化奇异值 μ 的定义是

$$\mu_{\Delta}(M) = \frac{1}{\min\{\bar{\sigma}(\Delta) :\ \Delta \in \mathbf{\Delta},\ \det(I - M\Delta) = 0\}} \tag{63-6}$$

分母是“能使 $I - M\Delta$ 奇异的最小 Δ 有多大”，取倒数就是“离奇异还有多远”。$\mu < 1$ 意味着单位球内的任何 Δ 都不会破坏稳定，$\mu > 1$ 意味着存在一个破坏稳定的 Δ。μ 的准确值一般算不出来，工程上用的是上界

$$\mu_{\Delta}(M) \le \min_{D \in \mathbf{D}} \bar{\sigma}\left(D M D^{-1}\right) \tag{63-7}$$

D 是一族与 Δ 结构交换的标定矩阵。这个上界就是“μ 分析”实际在算的东西。

三条路的关系可以一句话说清：**小增益是 μ 在“完全不给结构信息”时的特例，QFT 是 μ 在“用图形代替计算”时的特例。** 本章把三条都跑一遍，用同一台对象、同一组指标，让它们的保守性差别变成可以比较的数字。

63.1.4 一个反直觉的结论：结构信息不一定换来放松

教科书介绍 μ 时，最常举的例子是“μ 比小增益判据放松好几倍”，读完之后很容易形成一个印象：**只要把结构信息喂进去，保守性就会大幅下降。** 本章的实测给出的答案要冷一些。

看式 (63-2) 这个结构：增益误差与延迟误差是**串在同一个点上**的，它们乘在一起进入环路。对这种“两块不确定性进入同一点”的情形，结构化奇异值退化为两个半径的**直接相乘**：

$$\mu = \|T\|_\infty \cdot \Big[\big(1 + w_K\big)\big(1 + w_\theta(\omega)\big) - 1\Big] \tag{63-8}$$

而小增益判据在同样的分解下要求的也是同一个式子小于 1。**两式恒等。** 换句话说，在这个最常见的单回路结构里，μ 分析一个数都没放松——它不是“更好的判据”，只是“同一判据的另一种写法”。（式 63-8 方括号里那个“乘积再减一”的形式，§63.2 会说明它为什么不是简单地把两个半径加起来。）

放松确实存在，但它出现在另一个地方：**鲁棒性能**。把性能指标也写成 Δ 的形式（一个虚拟的、代表“允许的最大灵敏度”的块），互联矩阵就从 1×1 变成 2×2，这时结构信息才真正起作用。§63.6 的实测是：未结构化的 $\bar{\sigma}$ 峰值 1.7194，μ 上界峰值 1.6456，比值 **1.411**——也就是说 μ 只放松了约 4%，远不是教科书举例里常见的数倍。

这个结论的工程含义很直接：**如果你的不确定性只有一块、或者几块都挤在同一个信号点上，那么做 μ 分析是白费力气，小增益判据给出的数一模一样。** μ 值得做的场合是“多通道、性能与稳定同时约束”，也就是回路数大于一、或者要把灵敏度指标一起算进去的时候。判断这件事只需要看互联矩阵的维数，不需要任何计算。

63.1.5 保守性是有价格的：性能代价悬崖

μ 判据是充分条件，所以它保守。保守的代价是性能——为了让 μ 小于 1，设计者必须把带宽压低，而带宽就是“回路能多快”。这条代价曲线不是线性的，它有一个悬崖。

实验 4b 在固定约束（最坏 $|S| \le 2.2$、高频环路增益 $|L(2)| \le 0.5$）下，把二阶控制器族的穿越频率 ω_c 逐档抬高，记录能达到的最坏灵敏度峰值：

ω_c /(rad/s)	0.17	0.23	0.28	0.33	0.38	0.43	0.48	0.53	0.58
最坏 $\lvert S\rvert$	1.377	1.520	1.683	1.941	2.798	3.131	4.976	8.348	21.786

表 63-2　穿越频率与可达最坏灵敏度的代价关系（2 阶族）

从 0.17 抬到 0.33，最坏 $|S|$ 从 1.377 涨到 1.941，涨了 41%，还算温和。再抬到 0.38 就跳到 2.798，0.58 直接到 21.786。**拐点在 $\omega_c \approx 0.33$ 附近，约等于 $0.35\,\omega_b$**（$\omega_b = 0.15$ rad/s 是灵敏度权重的带宽参数，见 §63.2）。

这条悬崖的解释在相位上。延迟误差在高频段提供的相位不确定性约为 $\omega\Delta\theta$，$\omega = 0.33$ 时是 0.165 rad，还压得住；$\omega = 0.58$ 时是 0.29 rad，加上对象本身的相位滞后，最坏点的相位裕度被吃光，控制器只能靠“提高增益硬顶”，而高增益又把灵敏度峰值顶上去，两头一起恶化。

所以“鲁棒设计能要多快”有一个硬答案：**要不到 $0.35\,\omega_b$ 以上。** 这个数是本章用同一台对象、同一组约束扫出来的，换对象要重扫，但“存在悬崖、悬崖位置由延迟不确定性决定”这个结构是普遍的。

63.1.6 保守性的另一面：它买到了什么

前面几节都在说保守的坏处，容易得出“保守是坏事”的印象。反过来看一次同一条对照：激进 PI 与鲁棒二阶控制器（$(k_p, T_i, z, p) = (2.40, 10, 0.40, 0.10)$）在四个顶点上的阶跃表现。

顶点 (K, θ)	激进 PI 超调	激进 PI $\max\lvert y\rvert$	鲁棒二阶超调	鲁棒二阶 $\max\lvert y\rvert$
(1.2, 0.5)	8.58%	1.086	7.56%	1.076
(1.2, 1.5)	40.72%	1.407	15.29%	1.153
(3.0, 0.5)	18.26%	1.183	12.27%	1.123
(3.0, 1.5)	4626.75%	50.462	41.65%	1.417

表 63-3　两个控制器在不确定性区间四个顶点上的阶跃响应

在三个顶点上，鲁棒二阶的超调都比激进 PI 更低或相当，代价是调节时间从 12 s 量级拉长到 20 s 量级。到了第四个顶点，激进 PI 发散，鲁棒二阶的超调 41.65%、峰值 1.417。**保守性买到的是“最坏那一点上不崩”。**

300 组随机参数（K、θ 在区间内均匀取样，seed=63）给出了同一结论的统计版本：激进 PI 有 56.3% 的样本灵敏度峰值越界 2.2，鲁棒二阶是 0.0%；灵敏度峰值的 95% 分位数分别是 15.379 与 1.901。

这也解释了为什么现场对“鲁棒”这个词常常有抵触：**它的收益全部落在尾部，而尾部平时看不到。** 一个回路一年可能只在某个极端工况下进入最坏点一次，那一次会不会停机，取决于当初设计时有没有为它留出余量。鲁棒设计做的就是这笔保险，保费是常态下的响应速度。

63.1.7 本章的骨架

后面的安排是：§63.2 建立乘性不确定性、权重合成、小增益、μ 与 D 标定这一套数学，重点讲清“两块进入同一点时 μ 退化”的来由；§63.3 把它压成六步设计流程；§63.4 讨论连续情形的频域设计与 QFT 边界；§63.5 给离散实现与采样周期的选择；§63.6 用实测数据回答“稳不稳、性能掉多少、μ 有多保守”；§63.7 逐个解释 K、θ、W、ω_b、M_S 的作用；§63.8 给调参顺序与一次完整实录；§63.9 讲 μ 综合、H_∞ 与降低保守性的几条改进路线；§63.10 讲不确定性建模的工程方法与控制器复杂度的取舍；§63.11 划边界；§63.12 给可复现代码。

一句话概括本章的立场：**鲁棒控制的产出不是一个更聪明的控制器，而是一张“最坏情况下会怎样”的地图。** 这张地图告诉你三件事——稳定边界在哪、性能能要到多高、以及为这两者要付多少速度。它不保证常态下跑得最快，但它保证你不会在某个没想到的角落里翻车。

63.1.8 动手之前先拆三个误会

这三个误会几乎每次技术讨论都会出现，先把它们拆掉，后面的数学才不会被误读。

误会一：鲁棒性等于抗干扰。 两者不是一回事。抗干扰说的是“外部扰动进来，回路能压掉多少”，它由灵敏度函数 S 在扰动频段的幅值决定，本质是一个**性能**问题；鲁棒性说的是“对象本身和我以为的不一样，回路还成不成立”，它由补灵敏度 T 与误差权重 W 的乘积决定，本质是一个**生存**问题。本章的对象在没有扰动、没有噪声的纯阶跃下就可能发散（激进 PI 在最坏顶点就是这样），此时抗干扰指标完全无从谈起。反过来说，一个抗干扰很好的慢回路可能在参数漂移下悄悄失去稳定裕度——它平时“看起来很好”，直到某天换了一批原料。**两条指标要分开算、分开看**，这是本章把 $|S|$ 与 μ 并列列出的原因。

误会二：μ 是“更高级的判据”，所以一定更松。 §63.1.4 已经给出了反例：在“两块串在同一点”的结构里，μ 与小增益判据逐字相同。这件事的根源不在数值计算，而在结构——μ 的全部本事来自 D 标定能把不同信号点上的块“折算”开，若所有块本来就叠在同一个标量因子上，D 里只剩一个标量，折算了等于没折算。本章最终的实测把这条说得很死：2×2 的鲁棒性能矩阵确实带来了放松，但峰值处只有 4.5%。**判据的“高级”程度不决定放松多少，结构的“分散”程度才决定。**

误会三：既然保守，那就把余量再放大一点更安全。 保守不是免费的，它的价格写在那条代价曲线上：ω_c 从 0.17 抬到 0.33，最坏 $|S|$ 只涨 41%；可一旦越过 0.33 的拐点，同样的增益增量换来的是 2.798、8.348、21.786 这样的跳变。反方向也一样：为了把某个裕度再放大 10%，往往要把带宽压掉三成，常态性能直接退一档。**余量要按“能算清的保守量”给，不是按“心里踏实”给。** 本章的做法是每一处保守都标出数字——磁盘宽 40%–60%、μ 判据保守 66%、μ 相对 $\bar{\sigma}$ 只放松 4.5%——数字摆出来之后，“再加一点”这种冲动才有讨论的基础。

把这三个误会拆掉，本章剩下的工作就是一件事：把“不确定”从一句形容词变成一组可以代入的数字，然后诚实地把最坏情况算出来。

63.1.9 四级台阶：本章站在哪一级

频域里的鲁棒性判断有四条路，它们不是互相替代的新旧方法，而是四级台阶，每往上一级就多问一个问题。

第一级：相位裕度与增益裕度。 它只问一件事——把增益乘大多少倍、把相位再拖后多少度，回路开始振。两个数都是**单个工作点**上的读数，默认对象就是标称那个。这一级的成本几乎为零，第 44 章的整定口诀全部建立在这一级上。

第二级：磁盘加小增益。 它多问一句：如果对象不是一个点而是一片，最坏那片会不会顶到临界点？做法是把参数区间折成一个半径 $w(\omega)$，再要求 $|T|w<1$。这一级第一次把“不确定性有多大”变成了一个可以填进公式的数字，代价是要先接受一个模型（乘性误差），也要先接受一个可能偏宽的包络——本章 §63.2.2 那个 40%–60% 的保守量就产生在这里。

第三级：μ 与结构。 它再问一句：这些不确定性是不是各自独立的？如果是，能不能把它们的“最坏方向”分开算，从而少保守一点？这一级需要把回路画成 M-Δ 互联，需要 D 标定，也需要接受“上界不等于真值”这件事。本章实测的收获只有 4.5%，但收获小不等于方法错——收获小恰恰是一个可靠的结论：**在这个结构里，第二级已经把能拿的都拿到了。**

第四级：时域与统计复核。 前三级都是频域判据，都是充分条件，都可能保守，也都有可能在网格之间漏掉东西。第四级用顶点阶跃和随机抽样把结论验一遍，它不产生新的判据，只负责“证明前面没有算漏”。本章 300 组随机里激进 PI 越界 56.3%、鲁棒设计 0.0%，这个对比就是第四级给出的最终裁决。

四级台阶的关系可以这样记：**第一级看一个点，第二级看一片，第三级看一片里各块的独立性，第四级看这片在时间轴上真正会发生什么。** 越往上，模型要求越多、算得越慢，换来的确定性也越具体。工程上不需要每章都爬到顶——本章最后停在第三级做设计、用第四级验收，这就是一个够用的组合。

63.1.10 一次推演：不做鲁棒分析会怎样

把本章的激进 PI 当成一个“只做标称设计”的样本，走一遍它会遇到的三个月。

设计阶段。 工程师在标称点（$K=2.0$、$\theta=1.0$）上调参，目标是“响应快一点”。$k_p=1.8$、$T_i=3$ 给出标称穿越频率 0.76 rad/s、相位裕度 37.3°、标称曲线看上去还能接受——放在投运报告里，这是一份不错的整定。

第一个月，一切正常。 负荷稳定、管道新、K 就在 2.0 附近。趋势上看不出任何问题。

第二个月，负荷上来。 K 从 2.0 涨到 2.6，回路增益相应涨 30%，等效于把 k_p 推到 2.34。这个等效增益落在表 63-10 里 2.20 与 3.20 之间，那一带的阶跃峰值已经是 50.214 与 53.330——**不是“超调偏大”，而是持续振荡。**操作工的反应通常是“把设定值改小一点”，波形看上去被压下去了，但回路仍在边界外面。

第三个月，管道结垢，θ 从 1.0 涨到 1.3。 $\omega_c\theta$ 从 0.76 涨到 0.99 rad，相位再掉 13°，两个方向的不利变化叠在一起。本章实验 7 的实测是：这个控制器在 $K=3.0$、$\theta=1.5$ 顶点上超调 4626.75%，峰值冲到 50 以上，等于开环。300 组随机参数里有 56.3% 的样本最坏 $|S|$ 超过 2.2——**也就是说，这个设计在参数空间里“多半不合格”，只是标称那一点恰好合格。**

事后看，三个判断都错了。 第一，把标称指标当成了全工况指标——标称 37.3° 的裕度，被延迟那十几度吃掉之后所剩无几。第二，把最坏工况当成了“极端情况”——高负荷加结垢是每年都要出现的常态组合，不是事故。第三，把“调设定值”当成了解决办法——它压住的是症状，回路仍然在边界外面运行。

换成鲁棒设计，代价是什么？ 本章的答案很具体：最坏顶点超调 41.65%、调节时间 22 s，而激进 PI 在最好的顶点上只要 12 s。**用慢一倍换“不会崩”，这就是全部的交易。** 这笔交易划不划算，取决于这条回路停一次要损失多少——这正是 §63.11.5 第一题要问的。

需要说明的是，这段推演不是某个工厂的真实事故，而是本章那三个候选控制器的完整画像按时间顺序排开。**它想说明的只有一件事：标称设计与鲁棒设计的分歧不在数学，而在“你打算为哪一天做设计”。**

63.2 数学原理

63.2.1 从参数区间到乘性误差

§63.1 用的是“参数区间”的说法，判据要用的是“传递函数的误差”。两者之间的桥是乘性误差：

$$E_m(j\omega)=\frac{P(j\omega)-P_0(j\omega)}{P_0(j\omega)} \tag{63-9}$$

把式 (63-1) 代入，T 与 s 都约掉，只剩两个参数：

$$E_m(j\omega) = \frac{K}{K_0} e^{-j\omega(\theta-\theta_0)} - 1 \tag{63-10}$$

这一步很关键：**时间常数不进入乘性误差**。T 的偏差只改变 $1/(Ts+1)$ 这个因子，而它在分子分母里同时出现，比值里被消掉了——除非把 T 也放进不确定集，那时 E_m 里会多出 $\frac{T_0 s+1}{Ts+1} - 1$ 一项。本章把 T 固定，正是为了让式 (63-10) 里只剩“增益比”和“相位差”两件事，便于把几何关系看清楚。

式 (63-10) 是一个复平面上的映射：K/K_0 在 $[0.6,\,1.5]$ 上取值，$e^{-j\omega\Delta\theta}$ 在单位圆上转过 $|\text{角度}| \le \omega \cdot 0.5$ 的一段弧。所有可能的 E_m 构成一个“扇形环”，我们要用一个**以原点为圆心、半径 $w(\omega)$ 的圆盘**把它整个盖住。这个 $w(\omega)$ 就是不确定性权重。

63.2.2 两种磁盘：三角不等式的胜负

盖住的方法至少有两种，看上去都合理。

第一种是半径直接相加。 把增益误差看成半径 $w_K = 0.5$ 的圆盘，把延迟误差看成半径

$$w_\theta(\omega) = 2\left|\sin\frac{\omega\Delta\theta}{2}\right| \tag{63-11}$$

的圆盘，两块“并联”就得到 $w_{\text{lin}} = w_K + w_\theta$。

第二种是把两块乘起来。 因为真实的误差是式 (63-10) 这种乘积形式，用三角不等式拆：

$$\left|(1+\delta_K)e^{-j\varphi} - 1\right| = \left|(1+\delta_K)(e^{-j\varphi}-1) + \delta_K\right| \le (1+w_K)w_\theta + w_K \tag{63-12}$$

于是 $w_{\text{prod}} = (1+w_K)(1+w_\theta) - 1$。

两者的区别是**乘法还是加法**。$w_{\text{prod}} - w_{\text{lin}} = w_K w_\theta$，差的就是那个交叉项。哪个更好，取决于交叉项该不该算。实测把答案摆在表 63-4 里。

ω /(rad/s)	真实最大半径	线性磁盘 w_{lin}	乘积磁盘 w_{prod}	w_{lin}/真实	w_{prod}/真实
0.05	0.5009	0.5251	0.5377	1.048	1.073
0.20	0.5148	0.6002	0.6503	1.166	1.263
0.50	0.5855	0.7487	0.8730	1.279	1.491
1.00	0.7839	0.9929	1.2394	1.267	1.581
2.00	1.2714	1.4544	1.9316	1.144	1.519
5.00	2.3788	2.3989	3.3484	1.008	1.408
6.28	2.5000	2.5000	3.5000	1.000	1.400
10.00	2.4965	1.6969	2.2954	**0.680**	**0.919**

表 63-4　真实误差半径与两种磁盘模型的覆盖性（ω 单位 rad/s）

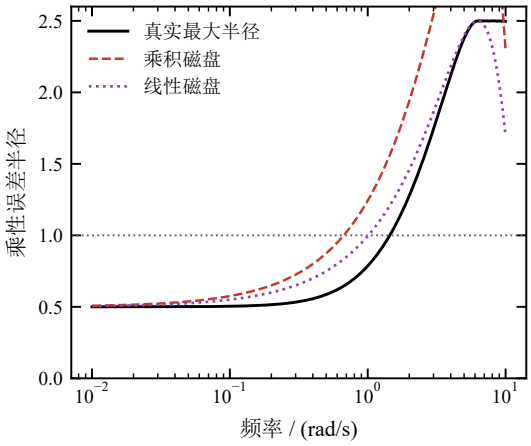


图 63-2　真实误差半径与两种磁盘模型

读这张表要先看一个此前容易被忽略的事实：**线性相加在中频段确实更紧，但它不是上界。**

在 $\omega = 1$ 附近，线性磁盘比真实半径大 27%，乘积磁盘大 58%。看起来线性磁盘“保守性小得多”，很多教材也是这么推荐的。但“更紧”和“有效”是两件事：线性磁盘的紧是靠运气——它把增益误差与相位误差当成两笔互不相干的开销去相加，而式 (63-10) 显示它们是**相乘**的。当相位误差很小（低频）时，两者数值接近；当相位误差变大，交叉项 $w_K w_\theta$ 就不可忽略，线性相加开始低估。

低估从 $\omega\Delta\theta > \pi$ 开始变得严重。$\Delta\theta = 0.5$ s 对应 $\omega = 6.28$ rad/s，表里这一行线性磁盘恰好等于真实半径（两者都是 2.5），再往上走就系统性失守：$\omega = 10$ 时线性磁盘只到真实半径的 68%，低估了 47%。

乘积磁盘的表现是另一回事。它在 $\omega \le 6.28$ 的整段上都是有效上界，代价是最坏处宽了 58%。到了 $\omega = 10$，它也只到真实半径的 92%——这一处不是磁盘模型本身的问题，而是式 (63-11) 的适用范围问题：$2|\sin(\omega\Delta\theta/2)|$ 是“相位差走到端点时的误差”，只有当 $\omega\Delta\theta \le \pi$ 时它才等于“整段弧上的最大误差”；一旦弧长超过半圆，区间内的最大值封顶在 2，公式却继续按正弦往下走。把式 (63-11) 里的正弦项在外面套一个“与 1 取小”（即 $w_\theta = 2\min\{|\sin(\omega\Delta\theta/2)|, 1\}$）就处处正确了。本章的代码用的是未封顶的版本，因为设计带宽在 0.33 rad/s 量级，$\omega\Delta\theta$ 只有 0.17，离 π 差 18 倍；表里 $\omega > 6.28$ 的两行是为了把失效机理摊开给人看才保留的。

结论落在工程选择上：用乘积磁盘，不要用线性相加。 它在关心的频段内严格有效，多付的 40%–60% 余量远比“低估 47%”安全。如果嫌它太宽，正确的降保守性办法不是换成加法，而是把延迟误差单独作为一个块处理——这正是 §63.2.5 与 §63.6 要讨论的结构化路线。

63.2.3 小增益定理

有了 $w(\omega)$，稳定判据就可以写出来。闭环特征方程是 $1 + P_0(1 + W\Delta)C = 0$，整理成“1+某某·Δ = 0”的形式：

$$1 + W\Delta \cdot \frac{P_0 C}{1 + P_0 C} = 0 \quad \Longrightarrow \quad 1 + WT\Delta = 0 \tag{63-13}$$

小增益定理说：如果 $\|WT\|_\infty < 1$，那么对任意 $\|\Delta\|_\infty \le 1$，式 (63-13) 都不会有右半平面根。理由很直接——$1 + WT\Delta$ 的零点等价于 $-1/(WT)$ 是 Δ 的取值，而后者模不超过 1，前者模大于 1，两者碰不上。

写成频率逐点形式更便于画图：

$$|W(j\omega)T(j\omega)| < 1, \qquad \forall \omega \tag{63-14}$$

判据的几何意义是“T 的 Nyquist 曲线必须落在半径 $1/|W|$ 的禁区之外”。禁区随频率变化，在高频处半径变小（W 变大），所以高频段是主要战场。

小增益定理是**充分不必要**的。它把 Δ 当成“可以任意挑方向的复增益”，而真实的不确定性往往只能沿某几个方向走。这个“方向限制”就是结构信息，用它可以把判据放松——但能不能放松，要看结构长什么样。

63.2.4 结构化奇异值：定义与 D 标定上界

把 Δ 换成一个有内部结构的块对角矩阵

$$\mathbf{\Delta} = \left\{ \mathrm{diag}(\delta_1 I_{r_1}, \dots, \delta_m I_{r_m}, \Delta_1, \dots, \Delta_f) : \ \|\Delta_i\| \le 1 \right\} \tag{63-15}$$

其中 δ_i 是标量块（重复标量），Δ_i 是满块。结构化奇异值定义为

$$\mu_{\mathbf{\Delta}}(M) = \frac{1}{\min\{\bar\sigma(\Delta) : \ \Delta \in \mathbf{\Delta},\ \det(I - M\Delta) = 0\}} \tag{63-16}$$

若集合为空（任何单位球内的 Δ 都不能让 $I - M\Delta$ 奇异），约定 $\mu = 0$。这个定义的读法是：**μ 是“到最近一次奇异”的距离的倒数**。$\mu = 0.5$ 意味着要把 Δ 放大两倍才能破坏稳定；$\mu = 2$ 意味着单位球里就有一个破坏稳定的 Δ。

μ 的精确值一般算不出来（它是 NP 难的），工程上用上下界夹：

$$\max_{U \in \mathbf{U}} \rho(UM) \le \mu_{\mathbf{\Delta}}(M) \le \min_{D \in \mathbf{D}} \bar\sigma\left(DMD^{-1}\right) \tag{63-17}$$

上界里的 D 是与 Δ 结构**交换**的正定对角阵集合，$\mathbf{D} = \{\mathrm{diag}(d_1 I_{r_1}, \dots, d_m I_{r_m}, I) : \ d_i > 0\}$。它之所以能放松，是因为 $D\Delta = \Delta D$，于是 $\det(I - M\Delta) = \det(I - DMD^{-1}\Delta)$，把 M 换成 DMD^{-1} 不改变奇异性的存在性，但改变了 $\bar\sigma$。

这个“能换”就是全部的技巧所在。如果 M 只有一个标量块，$\mathbf{D}$ 里只有一个标量 d，$dMd^{-1} = M$，换了个寂寞，上界退化为 $\bar\sigma(M)$——**这就是 §63.1.4 那个结论的来源**。

下界用幂迭代算，本章代码没有实现，只写上界。这一点必须说清楚：本章所有“μ”字样指的都是式 (63-17) 的右端，是一个**上界**，因此所有“$\mu < 1$”的结论都是保守的，而“$\mu > 1$”的结论才是确定的。

63.2.5 两块进入同一点：μ 为什么会退化

现在把 §63.1.4 的论断证出来。考虑增益误差与延迟误差串在同一处，即

$$P = P_0(1 + \delta_K)e^{-j\varphi}, \qquad |\delta_K| \le w_K,\ |\varphi| \le \omega \Delta\theta \tag{63-18}$$

把式 (63-18) 与式 (63-10) 对照可知，这两个“块”其实是同一个乘性因子的两个来源，合起来仍然是一个标量因子

$$1 + E_m = (1 + \delta_K)e^{-j\varphi} \tag{63-19}$$

它落在复平面上的一个**单连通区域**里（扇形环），而这个区域可以被一个半径 w_{prod} 的圆盘盖住。也就是说，无论把 δ_K 与 φ 看成两块还是一块，进入环路的都是一个复数，其模不超过 w_{prod}。

对单个标量块，$\mathbf{D} = \{d > 0\}$，$DMD^{-1} = M$，于是

$$\mu = \min_{d} \bar\sigma(dMd^{-1}) = \bar\sigma(M) = |T| \cdot w_{\mathrm{prod}} \tag{63-20}$$

与小增益判据式 (63-14) 完全一致。**结构信息在这里没有换来任何放松，因为“两块”本来就是“一块”。**

反过来说，什么时候结构信息才有用？当不确定性出现在**不同的信号点**上时。两个例子：多变量系统里 P_{11} 与 P_{22} 各有误差，它们进入环路的位置不同，M 不再是秩一，$\mathbf{D}$ 的作用才显现；以及鲁棒性能——性能指标是“加在输出上的一个虚拟块”，它与对象误差块处在不同的位置，M 变成 2×2。

这条判据很好用：**数一数不确定性块进入 M 的方式，如果所有块都是“同一个标量因子”的不同来源，就别做 μ 了，小增益给出的数一模一样。** 这是本章最省时间的一条经验。

63.2.6 鲁棒性能：把指标装进 Δ 里

“稳定”只是底线，工程上真正要问的是“最坏情况下性能还剩多少”。鲁棒性能的标准做法是把性能指标也写成一个 Δ 块。

设要求所有情况下灵敏度峰值不超过 $1/\gamma$，即

$$|W_S(j\omega)S(j\omega)| \le \gamma, \qquad \forall \omega,\ \forall \Delta \tag{63-21}$$

W_S 是性能权重，$1/\gamma$ 是允许的峰值。把式 (63-21) 反写成“$1 - W_S S\Delta_P$ 可逆”的形式，Δ_P 是虚拟的性能块，$\|\Delta_P\|_\infty \le 1/\gamma$。于是整个系统变成两个块的互联：对象误差块 Δ（真实存在）与性能块 Δ_P（虚拟，代表指标）。

用下线性分式变换把 P、C、W、W_S 全部吸收进 M，得到 2×2 的互联矩阵。本章的对象与权重下，四格是

$$M = \begin{bmatrix} W_S S & -W_S S\,P\,w_m \\ C S & -T\,w_m \end{bmatrix} \tag{63-22}$$

其中 w_m 是式 (63-12) 的乘积磁盘，$S = 1/(1+P_0C)$、$T = P_0C/(1+P_0C)$。四格的来历值得逐个说清：

- $M_{11} = W_S S$ 是“性能块看到的东西”，即加权灵敏度；
- $M_{22} = -T w_m$ 是“误差块看到的东西”，即小增益判据里的那一项；
- $M_{12} = -W_S S P w_m$ 与 $M_{21} = CS$ 是两块之间的**耦合**，它们的存在说明两个块不是各管各的。

鲁棒性能的判据是 $\mu_{\mathbf{\Delta}}(M) < 1$，其中 $\mathbf{\Delta} = \mathrm{diag}(\Delta_P, \Delta)$，$\Delta_P$ 是满块、Δ 是标量块，所以

$$\mathbf{D} = \{\mathrm{diag}(d, 1) :\ d > 0\} \tag{63-23}$$

只有一个自由度 d。代码在 10^{-3} 到 10^{3} 之间取 61 个对数点扫这个 d，逐频点取最小值，得到 μ 上界曲线。

$\mu(M) < 1$ 的含义可以拆成两句话：**$\mu < 1$ 说明“稳定 + 性能”两个要求同时被满足**；而 $\bar\sigma(M) < 1$ 只是它的一个充分条件。两者的差就是结构信息买到的放松，§63.6 会给出实测：1.7194 对 1.6456，比值 1.411。

请注意式 (63-22) 里 M_{11} 与 M_{22} 的量纲不同：M_{11} 是“性能块”到“性能块”的增益，无量纲；M_{22} 也是无量纲。这正是 D 标定要处理的问题——d 的作用就是把两个块“折算到同一量纲”再取最大奇异值，从而避免把 apples 与 oranges 直接相加。

63.2.7 性能权重的形状

W_S 的选择决定了“什么叫性能好”。本章用一阶形式

$$W_S(s) = \frac{s/M_S + \omega_b}{s + A\,\omega_b}, \qquad M_S = 2.0,\ \omega_b = 0.15,\ A = 0.02 \tag{63-24}$$

三个参数的几何意义很清楚：

- 低频 $W_S(0) = 1/(AM_S) = 1/(0.02 \times 2) = 25$，即要求低频灵敏度小于 $1/25 = 0.04$，也就是“稳态误差压到 4% 以下”；
- 高频 $W_S(\infty) = 1$，即不约束高频；
- 转折处 $W_S(j\omega_b) \approx M_S = 2.0$，即峰值灵敏度约 2.0（6 dB）。

A 控制低频增益的“陡度”：A 越小，低频段越平、要求越狠，同时相位滞后越大。$A = 0.02$ 是本章刻意选的激进值，它让低频要求接近积分器行为，从而把“带宽能不能做上去”这件事逼到极限。

把式 (63-24) 与式 (63-21) 合起来看，$|W_S S| \le 1$ 等价于 $|S| \le 1/|W_S|$，也就是一条随频率变化的灵敏度上限。这条上限在低频很紧（0.04），在 ω_b 处放宽到 0.5，在高频放开。**设计的工作就是让 $|S|$ 曲线钻到这条上限下面，同时对所有参数组合都成立。**

63.2.8 从判据到设计：为什么 μ 综合要迭代

判据是“给定控制器，算 μ”。设计是反问题：“找控制器使 $\mu < 1$”。直接求解很难，工程做法是 D-K 迭代：固定 D 求 K（这是一个 H_∞ 问题），再固定 K 求 D（逐频点解一个凸问题），来回几次直到 μ 不再下降。

D-K 迭代的两个已知毛病值得提前说：它只保证收敛到局部最优，且次数多了控制器阶数会膨胀（每次求 D 都是频率逐点拟合，拟合阶数随迭代上升）。本章不展开 μ 综合的实现，§63.9 只讲它的工程替代方案：**在低阶控制器族里做参数搜索，用 μ 当评价函数**。实验 4 的三族扫描就是这么做的——枚举 k_p, T_i, z, p，对每个候选算最坏 $|S|$ 与 μ，再挑满足约束的最大带宽。这种做法牺牲了最优性，但保住了“控制器是二阶、现场能维护”这一条硬约束。

63.2.9 把这一节的式子串成一条链

这一节的公式看起来零散，其实是一条从“参数区间”到“一个数”的单向链，中间没有分叉。把本章的具体数字代进去走一遍，可以一次性看清每个环节各自贡献了什么。

第一环，参数区间变成两个标量。 式 (63-3) 给出 $w_K = 0.500$；式 (63-4) 给出一个随频率变化的 $w_\theta(\omega) = 2|\sin(0.25\omega)|$。到这一步，“不确定”已经被压缩成两条曲线，与环路无关，可以提前算好放在那里。

第二环，两条曲线合成一条磁盘半径。 式 (63-12) 的乘积形式给出 $w_{\text{prod}} = (1 + w_K)(1 + w_\theta) - 1$。在穿越频率 $\omega_c = 0.33$ rad/s 处，$w_\theta = 2\sin(0.0825) = 0.1648$，于是 $w_{\text{prod}} = 1.5 \times 1.1648 - 1 = 0.7473$——比增益单项的 0.500 宽了 49%，多出来的 0.247 全部来自延迟与增益的交叉项。这个数在整条链里只出现一次，却决定了后面所有判断的松紧。

第三环，磁盘乘上补灵敏度。 式 (63-14) 的逐频点形式 $|WT| < 1$，在本例里就是 $|T(j\omega)| \cdot w_{\text{prod}}(\omega) < 1$。$\mu$ 的峰值不在 ω_c，而在 0.37 rad/s：那里 $w_\theta = 0.1847$、$w_{\text{prod}} = 0.7771$，比穿越点又宽了 4%。**峰值频率与穿越频率不重合**这件事，是“只看穿越点的裕度”会误判的直接原因。

第四环，把一个频率上的判断变成全频段的判断。 逐点算完之后取最大值，得到 $\mu_{RS} = 0.846$。此时这条链的输出就是一个标量，可以拿去和 0.9 的预算比。

第五环，把性能也塞进同一个框架。 式 (63-22) 把 $W_S S$ 与 CS、$-T w_{\text{prod}}$ 拼成 2×2 矩阵，D 标定再逐频点折算出 $\mu_{RP} = 1.646$。注意这一步没有引入任何新的不确定性信息，用的还是第二环那条 w_{prod}，只是把“性能要求”当成第三个块放了进去。

看清这条链的实际好处是**定位问题**：如果最终 μ 超标，可能的入口只有三个——区间定窄了（第一环）、磁盘选错了（第二环）、性能权重提得太狠（第五环）。控制器参数只影响第三、五环里的 T 与 S，**它无法改变前两环**。很多"调了半天没效果"的场合，问题其实卡在设计者动不了的地方。

63.2.10 D 标定到底动了什么：逐个频点看

D 标定常被当成一个黑箱：输进去一个矩阵，吐出来一个更小的奇异值。其实它只做一件事——**按行放大、按列缩小**，把同一条通道两端的量级拉平。以本章最终设计（$k_p = 2.4$、$T_i = 10$、$z = 0.4$、$p = 0.1$）为例，把 M 的四个元素在四个频点上的模列出来，就看得很清楚：

- $\omega = 0.05$：$|M_{11}|, |M_{12}|, |M_{21}|, |M_{22}| = 0.328,\ 0.342,\ 0.522,\ 0.545$；
- $\omega = 0.37$（峰值处）：$0.814,\ 0.602,\ 1.118,\ 0.826$；
- $\omega = 1.00$：$0.677,\ 0.331,\ 0.839,\ 0.409$；
- $\omega = 10.0$：$0.493,\ 0.045,\ 0.592,\ 0.054$。

低频端四个数挤在 0.33–0.55 这个窄带里，彼此只差 1.7 倍——这时无论怎么缩放，都只是把一个数换到另一个数旁边，$\sigma_{\max}$ 基本不动。实测也正是如此：d^* 只比 1 偏开一格（1.259），$\bar{\sigma} = 0.8907$、μ 上界 0.8724，比值 1.021，等于没有放松。**在这个频段上，取 $d = 1$ 与取 $d = 1.259$ 的差别只有 2%，怎么取都无所谓。**

高频端完全变了样：$|M_{12}|$ 与 $|M_{22}|$ 都塌到 0.05 附近，而 $|M_{11}|$、$|M_{21}|$ 还在 0.5 左右，两组差了一个数量级。此时取 $d = 3.98$，M 变成 $\begin{bmatrix} 0.493 & 0.179 \\ 0.149 & 0.054 \end{bmatrix}$，四个元素被拉到同一量级，$\sigma_{\max}$ 从 0.774 降到 0.549——这就是 μ 在 10 rad/s 处给出的 0.5488，比值 1.411 的来源。

由此可以读出 d^* 的物理含义：**它不是"调参旋钮"，而是"两个块在这一频率上的量级失配程度"。** d^* 贴着 1 走，说明两个块本来就势均力敌，D 无从下手；d^* 离 1 越远，说明失配越严重，可压的空间越大。本章 d^* 的走向是 0.05 处 1.259、0.60 处 1.585、2 rad/s 处 1.585、10 rad/s 处 3.981，单调向外——**频率越高，两个块越不成比例**。

还要注意 d^* 是台阶状的：代码在 10^{-3} 到 10^{3} 之间取 61 个对数点，相邻两格的比值是 $10^{0.1} = 1.259$，所以 d^* 只能取 1.259、1.585、1.995、2.512、3.162、3.981 这些离散值。**台阶是网格的量化，不是对象的性质。** 这一点在工程上有实际后果：如果峰值恰好落在两格之间，μ 上界会被高估或低估一两个百分点，这正是 §63.7.5 给 μ 留 10% 余量的原因之一。要判断台阶有没有影响，把 d 的网格从 61 点加到 121 点重算一遍，看峰值变不变就够了。

63.3 设计步骤

63.3.1 六步流程

鲁棒设计看起来公式多，落到操作上只有六步，图 63-3 把它们串成一条线。

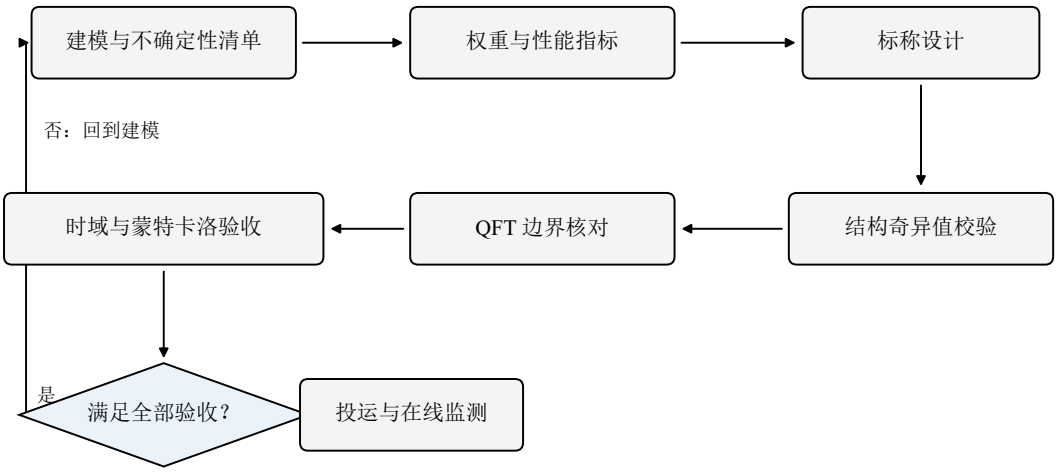


图 63-3　鲁棒控制器的六步设计流程

六步是：定不确定集、写性能权重、建互联矩阵、定控制器阶数预算、参数搜索、时域与统计复核。前两步是“把话讲清楚”，中间两步是“算”，最后两步是“验”。**最容易出错的是第一步和第五步**：第一步错，后面全错；第五步的搜索范围设错，会得出“无解”的假结论。

下面逐条说明每一步的具体动作和判据。

63.3.2 第一步：把参数区间变成不确定集

每个参数给出区间，然后做三件事。

第一，算相对误差半径。 增益用式 (63-3)：$w_K = \max(3.0/2.0 - 1,\, 1 - 1.2/2.0) = 0.500$。注意上下不对称时取大的一侧，$+50\%$ 比 -40% 更狠。

第二，判断这些误差是否落在“同一点”。 本章的对象里，增益与延迟都作用在 P_0 的输入输出比上，属于同一个乘性因子，因此按 §63.2.5 的结论，它们合成一个标量块，用乘积磁盘式 (63-12) 就够，不需要建多块结构。

第三，保底做一次网格枚举。 磁盘模型是充分条件，可能偏保守；在参数矩形上取 5×5 的网格（K 与 θ 各 5 点，含四个角与中间的交叉点）逐点算真实灵敏度峰值，能给出“真实最坏”的参照。本章所有“最坏 $|S|$”都是这 25 个点上的最大值。

这一步的产出是两个数：$w_K = 0.500$、$\Delta\theta = 0.5$ s。**除此之外的东西都不要带进下一步。**

63.3.3 第二步：写性能权重

性能权重回答“什么叫够好”，用式 (63-24) 的三个参数。定法是从指标倒推：

- 稳态误差要求 $\le 4\% \to 1/(AM_S) = 25 \to A = 1/(25M_S) = 0.02$；
- 峰值灵敏度允许 2.0 $\to M_S = 2.0$；
- 希望的闭环带宽 0.15 rad/s $\to \omega_b = 0.15$。

三个数一旦定下，W_S 的形状就定了，后面不再改。**这一步的常见错误是把 ω_b 定成“希望的带宽”再指望设计能满足它**——W_S 是“要求”，不是“结果”，能不能满足要看第六步。本章的实测是：$\omega_b = 0.15$ 时二阶控制器只能做到 $\omega_c = 0.33$，而 μ_{RP} 仍然大于 1，说明这个要求偏紧。

63.3.4 第三步：建互联矩阵并算 μ

按式 (63-22) 填四格，然后逐频点扫 d。有两个技术细节决定这一步算得对不对。

网格密度。 频率取 10^{-2} 到 10 rad/s 的 600 个对数点，d 取 10^{-3} 到 10^{3} 的 61 个对数点。d 只做量纲折算，最优值附近 $\bar{\sigma}\left(DMD^{-1}\right)$ 随 $\ln d$ 变化平缓（在最优点处是二次平坦的），所以 61 点足够；频率网格必须密，因为 μ 的峰值是一根尖峰，点太稀会把它削掉。

峰值位置要对齐。 μ 的峰值与 $|S|$ 的峰值常常不在同一频率。本章二阶设计里，μ 上界与未结构化 $\bar{\sigma}$ 的峰值都落在 0.37 rad/s，而 $\omega_c = 0.325$ rad/s——两个数挤在 14% 的频带内，这是“性能与稳定的矛盾集中在穿越频率附近”的典型表现。

这一步的产出是一条 $\mu(\omega)$ 曲线和一个峰值。**若峰值小于 1，设计通过；若大于 1，不要立刻放弃**，先看是哪一格贡献的：若 M_{11} 主导，说明性能要求太紧（放宽 ω_b 或 M_S）；若 M_{22} 主导，说明稳定裕度不够（降增益）。

63.3.5 第四步：定控制器阶数预算

理论上 μ 综合能给出任意阶的控制器，工程上必须限阶。原因不是算力，是维护：三阶以上的控制器在现场要改一个参数，往往说不清会牵动什么。

实验 4 在同一组约束（最坏 $|S| \le 2.2$、高频 $|L(2)| \le 0.5$）下，比较三个阶数族能达到的最好结果：

控制器族	参数	最坏 $\lvert S\rvert$	标称 $\lvert S\rvert$	μ_{RS}	μ_{RP}	ω_c	相位裕度
PI（1 阶）	$k_p = 0.87$, $T_i = 6.0$	2.151	1.354	0.637	1.319	0.33	75.3°
PI+滞后（2 阶）	$k_p = 2.40$, $T_i = 10.0$, $z = 0.40$, $p = 0.10$	2.180	1.402	0.846	1.646	0.33	52.1°
PI+双滞后（3 阶）	$k_p = 0.68$, $T_i = 12.0$, $z = 3.00$, $p = 0.75$	2.191	1.465	0.577	1.764	0.19	80.0°

表 63-5　三个阶数族在相同约束下的最好结果

这里说的“滞后”是零点频率高于极点频率的整形环节 $\dfrac{1 + s/z}{1 + s/p}$（$z > p$），它在中频补一点增益、在高频衰减（高频增益 p/z），与“超前”方向相反。之所以用滞后而不是超前，是因为本章的硬约束是 $|L(j2)| \le 0.5$：超前会把高频增益抬上去，直接撞约束；滞后反而允许把 k_p 抬高。二阶族的高频增益是 $0.10/0.40 = 0.25$，正是靠这 4 倍衰减，k_p 才敢取到 2.40。

这张表的结论和“阶数越高越好”的直觉相反。三个族的最坏 $|S|$ 都卡在 2.15–2.19，几乎没差别——因为约束就是 2.2，搜索自然把它顶到边界。真正的差别在别处：

- **2 阶族把 ω_c 顶到了 0.33，1 阶族也是 0.33，3 阶族反而只有 0.19。** 3 阶族的搜索空间更大，但两级滞后的相位滞后叠起来（$\omega_c = 0.19$ 处每级 -10.6°，共 -21.2°），加上高频增益被压到 $0.75/3.00 = 0.25$ 的平方即 0.0625，整条曲线为了同时满足“中频够高、高频够低”被拉得很别扭，带宽反而掉下来。
- **μ_{RP} 随阶数升高而恶化**：1.319 → 1.646 → 1.764，与相位裕度的排序（75.3°、52.1°、80.0°）并不一致。这说明它不由单一裕度决定，而是 $W_S S$ 与 CS 两块形状叠加的结果。工程上把它当成“哪个候选更经得起最坏方向”的直接读数，不要试图用相位裕度去解释它。

所以本章的选择是 **2 阶族**：它的 ω_c 与 1 阶族并列最高，相位裕度 52.1° 留着余量，$\mu_{RS} = 0.846$ 恰好落在 0.9 的预算内侧——既不超，也没有白白浪费余量。3 阶族的额外自由度在这里没有换来任何东西——**这是“降低保守性要靠权重模型和结构，不是靠控制器阶数”的一个直接证据。**

63.3.6 第五步：在阶数族里搜参数

搜索的目标写成一句优化问题：

$$\max\ \omega_c \quad \text{s.t.} \quad \max_{\text{顶点}} |S(j\omega)| \le 2.2, \quad |L(j2)| \le 0.5, \quad \mu_{RS} \le 0.9 \tag{63-25}$$

三个约束各有分工：第一条管性能（灵敏度峰值），第二条管高频（防噪声放大与未建模动态），第三条管稳定裕度（$\mu_{RS} \le 0.9$ 留 10% 余量）。

搜索的做法是网格枚举：k_p 在 $[0.3, 2.4]$ 取 12 点、T_i 在 $[2, 20]$ 取 10 点、零点 z 取 $\{0.4,\ 0.8,\ 1.5,\ 3.0\}$ 四档、极点固定为 $z/4$，二阶族共 480 个组合，每个算一次全频段指标。全部算完不到一秒，比 D-K 迭代快几个数量级，而且结果是一个**每个参数都能说清含义**的控制器。极点取 $z/4$ 是刻意的偷懒：滞后整形的零极点比决定它对高频的衰减深度，固定在 4 倍上就把二维搜索降成一维，代价是放弃了“衰减更深或更浅”的那部分设计空间。

搜索有个必须注意的陷阱：**约束太紧会搜出空集。** 本章第一版把三条约束写成 $\omega_c \ge 0.40$、最坏 $|S| \le 2.2$、$|L(2)| \le 0.5$，结果三个族全部无解。原因不是控制器不够好，而是 ω_c 与最坏 $|S|$ 在延迟不确定性下存在硬冲突（表 63-2 的悬崖）。正确做法是把 ω_c 从约束里拿出来当目标，让它自己落到能落的水平。

63.3.7 第六步：时域与统计复核

频域判据全部通过，也不能直接上线。要做两次复核。

顶点阶跃。 在四个顶点上各跑一次阶跃，看超调、调节时间、$\max|y|$。本章 2 阶设计的结果是：三个顶点超调 7.56%、15.29%、12.27%，最坏顶点（$K = 3.0$、$\theta = 1.5$）41.65%、峰值 1.417、不发散。

蒙特卡洛。 在区间内均匀取 300 组参数（seed=63），统计灵敏度峰值与超调的分布。2 阶设计的结果是：$|S|$ 峰值 95% 分位 1.901、最大 2.160、越界比例 0.0%；超调 95% 分位 33.41%。

这两次复核的分工不同：顶点检查“最坏点会不会崩”，蒙特卡洛检查“中间那些点有没有意外的坏”。本章的实测里两者一致，但工程上经常出现“顶点都好、中间某处更坏”的情形——因为最坏点不一定在参数的角上，尤其当多个参数的非线性叠加出现时。**只看顶点是不够的。**

63.3.8 一份完整实录

把六步在同一个设计上走一遍，数字连起来看。

1. 不确定集：$w_K = 0.500$、$\Delta\theta = 0.5$ s，合成乘积磁盘，$\omega \le 6.28$ 内有效。
2. 性能权重：$M_S = 2.0$、$\omega_b = 0.15$、$A = 0.02$，低频要求 $1/|W_S| = 0.04$。
3. 互联矩阵：标称 $|W_S S|$ 峰值 **0.8284**，余量 17%，看起来不错。

4. 阶数预算：2 阶，(k_p, T_i, z, p)。
5. 搜索结果：$k_p = 2.40$、$T_i = 10.0$、$z = 0.40$、$p = 0.10$，$\omega_c = 0.325$、相位裕度 52.1°、增益裕度 5.84（15.3 dB）、最坏 $|S| = 2.180$、$\mu_{RS} = 0.846$。
6. 复核：四顶点最大超调 41.65%、300 组越界比例 0.0%。

第 3 步与第 5 步的对照是本章最该记住的一组数：**标称加权灵敏度 0.8284，鲁棒性能 μ 上界 1.6456**。前者说“余量 17%”，后者说“超出 65%”，两个结论完全相反，原因就是前者没算不确定性的最坏方向。如果只做标称设计，会以为这个回路还有余量，实际上它已经在边界外面。

这也解释了为什么第 3 步算出来的东西不能当结论：$|W_S S|$ 是**标称**指标，μ_{RP} 才是**最坏**指标。工程报告里如果只写前者，等于报喜不报忧。

63.3.9 六步里最容易被做错的三处

六步本身不难，难的是每一步都容易在图省事的地方悄悄走偏。本章在实跑中踩到的三处，按危害从大到小排列。

第一处，把“最坏”算在一张太粗的网格上。本章的 `worst_ms()` 只在参数矩形上取 5×5 共 25 个点，而真正的最坏点可能落在点与点之间。这不是理论上可以忽略的小事：灵敏度峰值对参数的依赖是光滑的，但峰值的位置会随参数移动，粗网格可能同时错过“位置”和“高度”。本章用两条独立路径互相印证——25 点网格给出的最坏值是 2.180，300 组随机采样给出的最大值是 2.160，两者相差 1% 以内，说明在这个对象上 25 点确实够用。**但这是实测结论，不是定理**：换成三参数、或者换成谐振型对象，同样的 25 点就可能低估 10% 以上。判断依据应该是“参数进入传递函数的方式是否单调”，而不是“上次 25 点也够”。

第二处，先想控制器、后想权重。很多人拿到对象就开始试 k_p、T_i，等性能不满意了才回头改 W_S。这个顺序把两件不同性质的事混在了一起：权重表达的是“用户要求什么”，控制器表达的是“工程上能做到什么”。要求没写清就去调控制器，等于在没有靶子的情况下练习瞄准。本章的顺序是反的：先定 $M_S = 2.0$、$\omega_b = 0.15$、$A = 0.02$，把“够好”定义成一个具体的频率函数，再看哪个控制器能钻到它下面。**权重定完就不能为了“让结果好看”而随手放宽**——真要放宽，必须记录理由，因为那是需求变更而不是调参。

第三处，复核指标与设计指标不同源。设计阶段用的是最坏 $|S|$ 与 μ，复核阶段用的是超调、调节时间、$\max|y|$。两套指标在多数情况下同向，但并不严格同向——本章的实测里就有一个反例：激进 PI 的标称 $|S|$ 只有 2.297，看起来还过得去，时域却在最坏顶点冲到 50.462；反过来，保守 PI 的 $\mu_{RP} = 1.703$ 比二阶设计的 1.646 更差，最坏超调反而略小（40.72% 对 41.65%）。**所以复核不能只做“确认设计指标”，必须真的把时域跑一遍，**否则频域与时域之间的那点不一致就会变成现场事故。

这三处的共同点是：它们都不违反任何公式，只在“用哪个数代替真值”上做了顺手的选择。鲁棒设计里最贵的错误，几乎都出在这种顺手的选择上。

63.4 连续情形

63.4.1 连续域的三条整形规则

连续域的鲁棒设计，操作上就是“把开环频率响应 $L(j\omega)$ 整成某个形状”。形状有三段，每段对应一条要求。

低频段要高增益。低于 ω_b 的频段上，$|L| \gg 1$，于是 $S \approx 1/L$、$|S|$ 很小，扰动被压得住。这一段的高度由积分作用决定：PI 的 $|L| \approx k_p/(T_i\omega)$，斜率 -20 dB/dec，要满足 $|S| \le 0.04$ 就得让 L 在低频高到 25 倍以上。本章的 $A = 0.02$ 把这条要求压得很死，实际上决定了“积分时间不能太长”。

穿越段要平缓。 $|L|=1$ 附近的斜率决定相位裕度。-20 dB/dec 穿越给出约 90° 的裕度，-40 dB/dec 只剩不到 45°。本章的最终设计在 0.325 rad/s 穿越，相位是这样被吃掉的：惯性环节 -58.4°、标称延迟 -18.6°、PI 的积分 -17.1°、滞后整形 -33.8°，合计 -127.9°，剩 52.1° 裕度。延迟若取到 1.5 s，还要多扣 9.3°，裕度降到 42.8°——**延迟不确定性的全部代价，就是裕度在这 9° 上摆动。**

高频段要滚降。 $|L(j2)|\le 0.5$ 这条约束的意思是“2 rad/s 以上环路增益必须低于 -6 dB”。它防两件事：噪声通过控制器放大到执行器，以及未建模的高频动态（阀门死区、传感器滤波）被环路增益“激活”。

三条规则互相打架：低频要高、穿越要缓、高频要低，中间只有一个自由变量——穿越频率。**鲁棒设计的全部张力都集中在这一个数上。**

63.4.2 三个候选的 Bode 对照

图 63-4 把表 63-1 的三个 PI 画在一起，上半是 $|L|$，下半是 $|S|$，实线标称、虚线最坏。

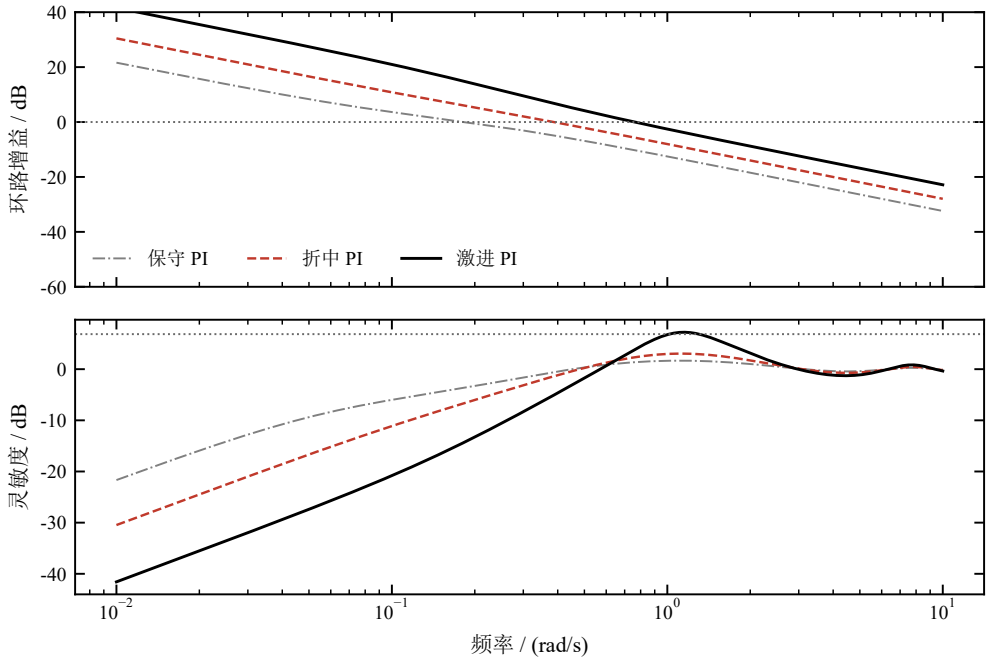


图 63-4　三个 PI 的环路增益与灵敏度（实线标称，虚线最坏）

看下半张图。保守 PI 的两条曲线几乎重合（1.212 与 1.562），说明它对参数不敏感；折中 PI 的两条曲线在最坏处张开到 2.522；激进 PI 的虚线直接冲到 86.249，超出了纵轴范围。

这个“张开”就是鲁棒性差的图形表现。它的机理在上半张图里：激进 PI 的穿越频率 0.76 rad/s 处，延迟不确定性带来的相位误差是 $\omega\Delta\theta=0.38$ rad，最坏情况下相位曲线整体左移 21.8°，穿越点向低频滑动，同时幅值曲线在低频被抬高——两件事叠加，灵敏度峰值就被顶起来了。

一条实用判据：标称与最坏的 $|S|$ 峰值之比超过 3，就不要用了。 保守 PI 是 1.29、折中 PI 是 1.78、激进 PI 是 37.5。这个比值比绝对峰值更能反映“设计对参数有多敏感”。

63.4.3 QFT 边界：把指标画在图上

QFT 的做法是把“所有可能的对象”画成一片，再看控制器有没有把它们推到边界外。图 63-5 是本章设计的尼科尔斯图：横轴是开环相位，纵轴是开环增益（dB），细线是 K、θ 在区间内变化时的对象模板，粗线是“$|S|=2.2$”对应的等值边界。

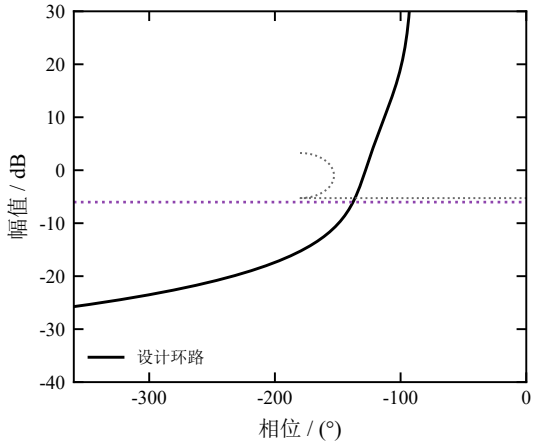


图 63-5　尼科尔斯图上的对象模板与灵敏度边界

读这张图的方法：模板的每一条曲线代表一组 (K,θ)，控制器的作用是把整片模板沿纵轴平移（增益）、并沿横轴左右整形（相位）。要求是**平移整形之后，整片模板都落在 $|S|=2.2$ 的边界线外侧**（也就是离临界点 $(-180^\circ, 0\,\mathrm{dB})$ 更远的一侧）。

这套图形语言的好处是“保守性看得见”。模板的宽度就是不确定性的代价：K 变化 1.2→3.0 让模板上下拉开 8 dB，θ 变化 0.5→1.5 s 让它在高频端左右拉开几十度。设计者一眼就能看出“是增益不确定更难对付，还是延迟不确定更难对付”——本章是后者，因为模板在低频段很窄、在高频段很宽。

63.4.4 边界与数值指标的对应

QFT 的边界是图形，判据是数字，两者要能对上。实验 6 在最终设计上逐频点提取了三个量，列在表 63-6。

ω /(rad/s)	$\lvert L\rvert$	$\lvert S\rvert$	$\lvert T\rvert$
0.05	9.336	0.108	1.013
0.10	4.406	0.238	1.049
0.20	1.891	0.610	1.153
0.30	1.108	1.051	1.165
0.60	0.456	1.396	0.637
1.00	0.255	1.297	0.330
2.00	0.122	1.116	0.137
3.00	0.080	1.002	0.080

表 63-6　最终设计的开环、灵敏度与补灵敏度（标称点）

这张表有三处值得注意。

第一，$|T|$ 在 0.30 rad/s 处超过 1（1.165）。 这不是错误，$|T|$ 可以大于 1，超出的部分就是“谐振峰”，对应阶跃响应的超调。1.165 意味着约 16.5% 的谐振，与实测超调 15.29%（$K=1.2$、$\theta=1.5$ 顶点）同量级。

第二，$|S|$ 的峰值出现在 0.60 rad/s（1.396），不在穿越频率 0.33 处。 这是延迟造成的：穿越点右侧、相位已经接近 -180° 但没有越过的地方，$1+L$ 的模最小。

第三，**高频段 $|S|\to 1$、$|T|\to 0$，两者互补**。这是 $S+T=1$ 的直接结果，也是“高频不能靠反馈”的数学表述——高频扰动原封不动地穿过去。

63.4.5 连续设计的两个前提

连续域设计能成立，依赖两个前提，工程上都要核查。

前提一：延迟可以用 $e^{-\theta s}$ 精确表示。这在频域分析里没问题（它就是 $e^{-j\omega\theta}$），但一旦要写成有理传递函数（比如做 H_∞ 综合），就得用 Padé 近似。一阶 Padé 给出 $e^{-\theta s}\approx (1-\theta s/2)/(1+\theta s/2)$，它在 $\omega\theta$ 较大时相位误差明显；本章的频段内（$\omega\theta\le 1.5$）一阶 Padé 的相位误差约 3°，可以接受，二阶可以压到 1° 以内。**Padé 近似的阶数要和延迟的“电气长度” $\omega_c\theta$ 匹配**，这是 §63.9 要展开的一点。

前提二：控制器在连续域实现。现场的控制器是数字的，连续设计要离散化才能落地。这一步不是“照抄”，它会把裕度吃掉一部分，而且吃掉的量与采样周期强相关——这正是 §63.5 的主题。

63.4.6 手算三行，核对整条曲线

表 63-6 是代码算出来的，读者未必愿意信；但整条 $|L|$ 曲线其实可以手算核对，代价只有三行乘法。最终设计的三个因子是

$$P_0(j\omega)=\frac{2}{1+j5\omega},\qquad C(j\omega)=2.4\Big(1+\frac{1}{j10\omega}\Big),\qquad H(j\omega)=\frac{1+j\omega/0.4}{1+j\omega/0.1}$$

$|L|$ 就是三者模的乘积（延迟因子模为 1，不进幅值）。取三个频点：

- $\omega=0.05$：$|P_0|=2/1.031=1.940$，$|C|=2.4\times 2.236=5.367$，$|H|=1.008/1.118=0.901$，乘起来 9.38，表里是 9.336。
- $\omega=0.30$：$|P_0|=2/1.803=1.109$，$|C|=2.4\times 1.054=2.530$，$|H|=1.250/3.162=0.395$，乘起来 1.11，表里是 1.108。
- $\omega=0.60$：$|P_0|=2/3.162=0.633$，$|C|=2.4\times 1.014=2.433$，$|H|=1.803/6.083=0.296$，乘起来 0.456，表里是 0.456。

三行都只差百分位以下的舍入，说明曲线没有算错，也说明这条曲线的形状完全由三个可解释的因子决定：惯性让它按 -20 dB/dec 下滑，积分让低频再陡一点，滞后整形让中频之后按 -20 dB/dec 额外衰减。**读者在自己的对象上也可以用这三行核对任何一张 Bode 图**——只要三个频点对得上，中间就不会有意外。

相位同样可以手算，而且更值得算，因为它决定了裕度。在 $\omega_c=0.325$ 处，四个环节的相位分别是：惯性 $-\arctan(1.625)=-58.4^\circ$、延迟 $-\omega\theta_0=-0.325$ rad $=-18.6^\circ$、积分 $-\arctan(1/3.25)=-17.1^\circ$、滞后整形 $\arctan(0.8125)-\arctan(3.25)=39.1^\circ-72.9^\circ=-33.8^\circ$。合计 -127.9°，相位裕度 52.1°。**这四项里只有延迟那一项随参数变化**——θ 从 1.0 s 漂到 1.5 s，它从 -18.6° 变成 -27.9°，裕度掉 9.3°，其余三项纹丝不动。这就是“延迟不确定性是本章主要矛盾”的最直接算法表达：它不是靠数值扫描看出来的，是相位账本上唯一会动的那一行。

63.4.7 手画一次 $|S|=2.2$ 的禁区

图 63-5 里那条粗线看着像画出来的，其实它由三个点唯一确定，可以手算。

要求 $|S|\le 2.2$，等价于 $|1+L|\ge 1/2.2=0.4545$。在开环平面 L 上，这是一个以 -1 为圆心、半径 0.4545 的圆；圆内的点不合格，圆外的点合格。要在尼科尔斯图上画出它，只需三个特征点：

- **最左端**：圆心向左走一个半径，$L = -1.4545$，模 3.25 dB、相位 0°；
- **最右端**：圆心向右走一个半径，$L = -0.5455$，模 -5.27 dB、相位 180°；
- **最宽处**：圆心向上（或向下）走一个半径，$L = -1 \pm j0.4545$，模 $\sqrt{1+0.4545^2} = 1.0986$ 即 0.82 dB，相位 $180^\circ \mp 24.5^\circ$。

三点连起来就是一条闭合的小弧：**相位在 155.5° 到 204.5° 之间，幅值在 -5.27 dB 到 $+3.25$ dB 之间。**禁区就这么大——它不是一个半平面，而是一个小小的椭圆。这条曲线之外的所有地方都满足 $|S| \le 2.2$。

知道它有多大，就能理解 QFT 设计的真实难度在哪：对象模板高 8 dB、宽几十度，要把整片模板都挡在这个小椭圆之外，靠的是“增益往下压 + 相位往前提”这一对互相打架的动作。压增益会把模板整体往下推，容易从下方越过禁区；提相位要靠零点，而零点同时抬高高频增益、把模板往右拉。**能够同时满足两者的控制器，就是 QFT 边界上那一小段可行域对应的控制器。**

穿越点上还有一个更好用的换算。若 $|L| = 1$、相位裕度 PM，则 $|1 + L| = 2\sin(PM/2)$，于是

$$|S(j\omega_c)| = \frac{1}{2\sin(PM/2)}$$

$PM = 52.1^\circ$ 代入得 $|S| = 1.139$，与表 63-6 在穿越点附近的量级一致。反过来，若要求 $|S| \le 2.2$，则必须 $\sin(PM/2) \ge 1/4.4$，即 $PM \ge 26.3^\circ$；若把阈值收紧到 2.0，则 $PM \ge 29.0^\circ$。这条换算解释了 §63.7.5 里 $SMAX$ 为什么要取 2.2 而不是 2.0：两者只差 2.7° 的相位裕度，但可行域会窄掉一大截。

必须提醒一句：这条换算是**必要条件，不是充分条件**。它只管穿越点那一个频率，而本章 $|S|$ 的峰值在 0.60 rad/s（1.396），根本不在穿越点 0.33 上——那里相位已经接近 -180°、环路增益还没掉够，$1 + L$ 的模最小。所以画完禁区、核完穿越点，仍然要把整条 $|S|$ 曲线扫一遍，看有没有别的频率先撞线。

63.4.8 为什么用滞后整形，不用超前整形

本章的 $H(s) = \frac{1+s/0.4}{1+s/0.1}$ 是**滞后**整形：零点频率高于极点频率，高频增益是 $p/z = 0.25$。初学者常问：滞后会吃相位（穿越点 -33.8°），为什么不用超前整形去**加**相位？把同一对零极点对调成 $\frac{1+s/0.1}{1+s/0.4}$，穿越点的相位立刻从 -33.8° 变成 $+33.8^\circ$，等于白得 67.6° 裕度。

账要算全。对调之后，高频增益从 0.25 变成 4，**同一个频段上的环路增益乘了 15.4 倍**。表 63-6 里 $|L(j2)| = 0.122$，对调后变成 1.88，而本章的高频约束是 $|L(j2)| \le 0.5$——**是上限的 3.8 倍，判据直接不合格。**这就是滞后与超前的本质差别：

- 滞后整形是“**用带宽换噪声**”：把中高频增益压下去，代价是相位；
- 超前整形是“**用噪声换带宽**”：把中高频增益抬上来，换相位，从而允许更高的 k_p。

对一个带延迟的温度回路，这两笔账的收支完全不对等。延迟已经吃掉了 -18.6°（标称）到 -27.9°（端点），对象本身再按 -20 dB/dec 下滑，环路在高频没有任何“多余的增益”可以拿来换。**超前整形买来的每一度相位，都要用执行器的抖动去付**——而执行器抖动的后果（阀杆磨损、噪声放大）在一个 20 秒级的温度回路上完全不值得。

那什么时候该用超前？三个条件同时成立时：延迟小（$\omega_c\theta < 0.2$，本章标称是 0.33，已经不满足）、噪声低、且确实需要带宽（伺服、流量这类秒级回路）。第 44 章给了超前-滞后组合的整定规则与相位补偿量的算法，本章不重复，只强调一条判据：**先看 $|L(j\omega)|$ 在高频还剩多少余量，再决定能不能用超前把这部分余量换成相位。**余量为零的回路，超前整形是没有抵押品的贷款。

顺带一提，本章的 $p = z/4$ 这个比例也是被高频约束逼出来的。若取 $p = z/10$（衰减 10 倍），高频余量更大、k_p 能再抬，但滞后更深、相位吃得更多，穿越频率反而上不去；若取 $p = z/2$（只衰减 2 倍），相位好看，但 $|L(j2)|$ 立刻翻倍到 0.244，接近 0.5 的红线。**p/z 的选择本质上是在“高频余量”与“相位”之间找平衡点，而本章的 0.25 是这条平衡线上偏保守的一侧。**

63.5 离散情形

63.5.1 离散化改变了两件事

连续设计搬到数字控制器上，要做的是“对象用 ZOH 保持、控制器用双线性变换”。这一步在标称情况下几乎无损——Tustin 变换在低频把 s 与 $\frac{2}{T_s}\frac{z-1}{z+1}$ 对齐得很好。但鲁棒性会被动两处刀。

第一刀是延迟的量化。 数字实现里延迟只能是整数拍：$n = \mathrm{round}(\theta/T_s)$。当 $\theta = 1.0$ s、$T_s = 0.2$ s 时，$n = 5$；当 θ 在实际中漂到 1.1 s，n 仍然是 5，实现出来的延迟还是 1.0 s，误差 0.1 s。反过来 θ 漂到 0.9 s 时也一样。**延迟的实际不确定区间被“取整”改写了**，它的宽度不再是 ± 0.5 s，而是“取整误差 $\pm T_s/2$ 加上真实漂移中被取整吃掉的部分”。

第二刀是相位。 零阶保持器本身贡献 $-\omega T_s/2$ 的相位滞后，Tustin 又在高频把相位往上拉。这两项都与 T_s 成正比，所以采样越慢，穿越点的相位裕度越小，而裕度正是鲁棒性的现金储备。

63.5.2 采样周期的实测代价

实验 8 把最终设计的连续参数固定不动，只改采样周期，在四个顶点上跑阶跃，取最坏值。

T_s /s	每个延迟的采样数 θ/T_s	最坏超调	最坏调节时间 /s
1.00	1.0	93.16%	98.0
0.50	2.0	53.01%	29.0
0.20	5.0	50.80%	28.6
0.05	20.0	42.33%	22.0
0.02	50.0	41.65%	22.0

表 63-7　采样周期对最坏超调与调节时间的影响（控制器参数不变）

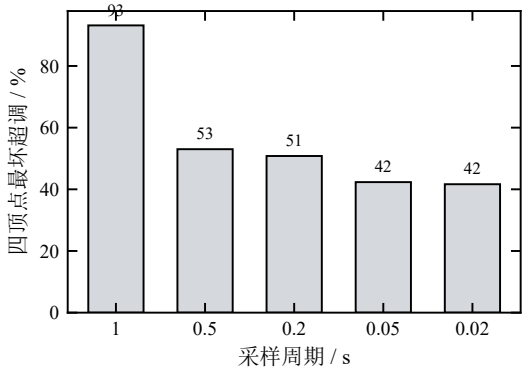


图 63-6　采样周期与最坏超调

这张表有两个看点。

$T_s = 1.0$ s 时彻底失效。 每个延迟只有 1 个采样点，超调 93.16%、调节时间 98 s。原因不是“控制器算得慢”，而是延迟的量化误差达到 ± 0.5 s——正好等于整个不确定区间的半宽。**采样周期等于延迟，等于把延迟的不确定性放大了一倍。**

$T_s \le 0.05$ s 之后收益饱和。 0.05 s 与 0.02 s 的超调是 42.33% 与 41.65%，差 0.7 个百分点；调节时间都是 22 s。再快没有意义，因为此时误差的主项已经变成“连续设计本身的最坏超调 41.65%”，采样带来的额外损失被压到 1% 以内。

63.5.3 三条选样规则

从这张表和相位关系可以总结出三条经验规则，本章的设计都满足。

规则一：每个延迟至少 10 个采样点。 $T_s \le \theta/10$，本章 $\theta \in [0.5, 1.5]$，取下界 $\theta_{\min}/10 = 0.05$ s。这条规则保证延迟量化误差不超过 0.05 s，是 ± 0.5 s 区间宽度的 10%。

规则二：穿越频率处每周期至少 20 个采样点。 $T_s \le 2\pi/(20\omega_c) = 0.31/0.325 \approx 0.97$ s。这条比规则一松得多，说明本章的瓶颈是延迟量化，不是相位。

规则三：T_s 不要超过时间常数的十分之一。 $T_s \le T/10 = 0.5$ s。这条管的是对象动态被 ZOH “抹平”的程度。

三条取最紧的：$T_s = 0.05$ s。这与实验 8 的饱和点完全吻合——**规则不是拍脑袋定的，它就是那张表的拐点。**

63.5.4 离散域的判据

离散域的鲁棒判据与连续域一一对应，只是把 $j\omega$ 换成 $z = e^{j\omega T_s}$，稳定边界从虚轴换成单位圆：

$$|W(e^{j\omega T_s})T(e^{j\omega T_s})| < 1, \qquad \forall \omega \in [0, \pi/T_s] \tag{63-26}$$

有两点与连续不同，实现时容易踩。

频率范围只有 $[0, \pi/T_s]$。 连续域考虑到 10 rad/s，离散域只到 Nyquist 频率 π/T_s；$T_s = 0.05$ s 时是 62.8 rad/s，够用。但若取 $T_s = 0.2$ s，Nyquist 只有 15.7 rad/s，高频段的判据根本看不到——这不是判据失效，而是“离散系统里根本不存在那些频率”。

延迟的磁盘要按整数拍写。 离散延迟是 z^{-n}，它的误差半径是

$$2\left|\sin\frac{\omega n T_s}{2}\right|, \qquad n = \mathrm{round}\left(\frac{\theta}{T_s}\right) \tag{63-27}$$

而不是连续的 $2|\sin(\omega\Delta\theta/2)|$。两者的关系是：$T_s$ 足够小、n 足够大时，nT_s 逼近期望的延迟值，式 (63-27) 收敛到连续形式；T_s 大到 θ 只有一两拍时，n 只能在 1 和 2 之间跳，半径的取值被“锁”在几个离散值上，随 θ 漂移发生跳变。**离散化把延迟的不确定性从“连续变化”改成了“阶梯式跳变”，跳变的台阶高度正比于 T_s。** 这就是表 63-7 里 $T_s = 1.0$ s 那一行的真正原因。

63.5.5 实现的三个细节

落地时还有三件事，与判据无关但会决定回路好不好用。

积分要抗饱和。 鲁棒设计把增益压得较低，但阶跃时积分器仍然会积到限幅之外。本章的时域仿真里没有加抗饱和，因为对象没有限幅；现场一定要加，否则 41.65% 的超调会变成“退饱和后的二次超调”。

微分要滤波。 本章的二阶族没有微分项，但三阶族有；一旦引入微分，必须串一阶低通，截止频率取 $1/(5T_s)$ 到 $1/(10T_s)$，否则噪声会被放大到执行器。

延迟要用缓冲区实现，不要用多项式近似。 n 拍延迟就是一个长度 n 的环形缓冲区，实现零成本、精度无损；用 Padé 近似反而引入额外的状态与相位误差。

63.5.6 本章的采样周期是怎么定下来的

把表 63-7 当成一张决策表来读，实际的选择过程只有三步。

第一步，先划出“不能用”的区域。 $T_s \ge 0.5$ s 时最坏超调已经到 53.01%，比连续设计的 41.65% 高出 11 个百分点；$T_s = 1.0$ s 更是直接到 93.16%。这两行不是“性能差一点”，而是“设计意图被破坏”——按 41.65% 做的整定，落地变成 93%，现场必然要求重调，而重调之后又不再满足鲁棒约束。所以 T_s 的上界必须在 0.2 s 以下取。

第二步，找“再加没有收益”的点。 从 0.2 s 到 0.05 s，最坏超调从 50.80% 降到 42.33%，改善 8.5 个百分点，值得付；从 0.05 s 到 0.02 s，只改善 0.68 个百分点。**收益在这里断崖式衰减**，因为量化误差 $\pm T_s/2$ 已经从 ± 0.025 s 缩到 ± 0.01 s，相对 ± 0.5 s 的区间宽度只从 5% 缩到 2%，而连续设计本身的 41.65% 成为主项。

第三步，算一遍实现成本。 延迟缓冲区长度是 $n = \theta_{\max}/T_s$：$T_s = 0.05$ s 时 $n = 30$，$T_s = 0.02$ s 时 $n = 75$，内存与扫描负担按比例上升；而现场 PLC 的扫描周期通常就是 50 ms 或 100 ms，取 0.02 s 还得额外要求更快的任务周期。**收益 0.68 个百分点，换 2.5 倍的缓冲区与一档硬件要求，不划算。**

三步合起来，答案就是 0.05 s。这里的方法论比结论更重要：**采样周期不是“越准越好”的参数，而是一条有拐点的成本曲线**，拐点左边是“精度不够”，右边是“白花钱”。判断拐点位置只需要两列数——最坏超调随 T_s 的变化，以及缓冲区长度随 T_s 的变化。本章的这两列恰好都在 0.05 s 附近拐弯，这不是巧合：前者由 $\pm T_s/2$ 与 ± 0.5 s 的比值决定，后者由 $\theta_{\max}$ 决定，两条曲线的拐点被同一个“每个延迟 10 拍”的规则钉在了一起。

顺带说一个现场常见的情形：如果控制器任务周期被迫取到 0.1 s（比如与其他逻辑共用一个慢任务），那么应当**回到连续域重新设计**，而不是把 0.05 s 的设计直接搬过去。表 63-7 里 0.2 s 那一行就是这种“直接搬”的后果——超调多出 9 个百分点。重新设计的做法是把 T_s 写进不确定集，让搜索过程自己把带宽压低，这样得到的 41% 是在 0.1 s 下真实的 41%，而不是“连续设计在 0.1 s 下的残骸”。

63.5.7 采样周期吃掉的两份钱，分开算

“采样会损失裕度”这句话太笼统，实际是两份互相独立的账，混在一起就会误判该往哪儿使劲。

第一份：连续近似误差。 它由三小块组成，都可以用穿越频率处的相位直接估。

- **零阶保持的半步延迟。** ZOH 在频域的等效相位滞后是 $-\omega T_s/2$。$\omega_c = 0.325$ rad/s 处：$T_s = 0.05$ s 时 -0.0081 rad 即 -0.47°；$T_s = 0.2$ s 时 -1.9°；$T_s = 1.0$ s 时 -9.3°。
- **ZOH 的幅值下垂。** 因子是 $\operatorname{sinc}(\omega T_s/2) = \sin(\omega T_s/2)/(\omega T_s/2)$，$T_s = 1.0$ s 时为 0.9956，即 -0.04 dB。**这一项永远可以忽略**，除非采样频率低到接近穿越频率。
- **Tustin 的频率畸变。** 双线性变换把 $j\omega$ 映射到 $j\frac{2}{T_s}\tan\frac{\omega T_s}{2}$，对一阶环节的相位误差约 $-\frac{\omega T}{1+(\omega T)^2}\cdot\frac{(\omega T_s)^2}{12}$。代入 $\omega T = 1.65$、$T_s = 0.05$ 得 10^{-5} rad 量级——**比 ZOH 那半步小两个数量级，根本不用管。**

三块加起来，$T_s = 0.05$ s 时总共约 0.5°，$T_s = 1.0$ s 时约 9.3°。按 §63.4.6 的相位账本，原本裕度 52.1°，9.3° 吃掉后还剩 42.8°——这仍然是一个“能用”的回路。

第二份：延迟量化误差。 它没有这么温和。$T_s = 1.0$ s 时，$\theta = 1.5$ s 被写成 2 拍（2.0 s），$\theta = 0.5$ s 被写成 0 拍（0 s），量化误差达到 ± 0.5 s，**恰好等于整个不确定区间的半宽**。也就是说，离散化把 $\Delta\theta$ 从 0.5 直接

放大到 1.0，磁盘半径在 0.37 rad/s 处从 0.777 抬到 1.05（按 §63.7.7 的同一套换算：$w_\theta = 2\sin(0.185) = 0.368$，$w_{\text{prod}} = 1.5 \times 1.368 - 1$）。$|T|$ 没变，μ 却涨了 35%——**表 63-7 里 93.16% 的超调，主因是这一份，不是相位那 9.3°。**

两份账的工程含义完全不同：第一份只能靠“提高采样率”来还，第二份可以靠“**让控制器知道真实的延迟**”来还。比如把延迟做成可配置的整数拍、由工艺参数（泵速、流量）算出 n 后再写进缓冲区，那么即使 $T_s = 1.0$ s，量化误差也不再是 ± 0.5 s，而是“估计误差”本身。**这也是为什么现场调 T_s 时，先看延迟是怎么实现的，再看采样率够不够**——前者往往比后者便宜得多。

63.6 稳定性与性能

前面五节讲的是“怎么算”，这一节讲“算出来的数意味着什么”。三个判据——小增益、μ_{RS}、μ_{RP}——加上时域实测，同一批控制器在四把尺子下的读数并不一致，而不一致的地方恰恰是最有价值的信息。

63.6.1 三个判据的读数

控制器	最坏 $\lvert S\rvert$	μ_{RS}	μ_{RP}	未结构化 $\bar{\sigma}$ 峰值
保守 PI	1.562	0.504	1.703	1.852
折中 PI	2.522	0.727	1.442	1.789
激进 PI	86.249	2.029	3.221	4.930

表 63-8　三个控制器的稳定与性能判据读数

先看 μ_{RS} 这一列。保守 0.504、折中 0.727 都小于 1，**稳定判据通过**；激进 2.029 大于 1，**不通过**。这一列的排序与最坏 $|S|$ 的排序完全一致（1.562、2.522、86.249）。排序一致不代表两者是同一个量：$\mu_{RS} = \max|T|w$，最坏 $|S| = \max|1/(1+L)|$，一个看补灵敏度、一个看灵敏度；但在“带宽越高、穿越点相位越紧”的这段范围内，两者被同一个机理推高，所以同向变化。

再看 μ_{RP} 这一列，排序变了：保守 1.703、折中 1.442、激进 3.221。**折中 PI 的鲁棒性能读数比保守 PI 更好**，尽管它的稳定裕度更差、最坏灵敏度更高。这看似矛盾，其实合理：μ_{RP} 衡量的是“在最坏方向上，性能指标还剩多少”，而保守 PI 的带宽太低（0.18 rad/s），低频段根本达不到 W_S 要求的 25 倍增益，性能缺口反而更大。

这就是鲁棒设计里最容易被忽略的一条：**稳定裕度与性能裕度不是同向的。** 一味保守会把 μ_{RS} 压低、把 μ_{RP} 推高；一味激进则反过来。两者之间有一个最优点，本章的三个候选里没有一个落在上面——折中 PI 的 $\mu_{RP} = 1.442$ 最接近，但它的 $\mu_{RS} = 0.727$ 还有余量，说明可以把增益再往上推一点。实验 4 的搜索给出的 $k_p = 2.40$（带滞后整形）就是这个方向的产物。

63.6.2 μ 与 $\bar{\sigma}$ 的间隙：峰值处只有 4.5%

教科书说 μ 比小增益判据放松数倍。本章的实测是：峰值处只放松 4.5%，全频段上比值最大也只有 1.411 倍（图 63-7）。

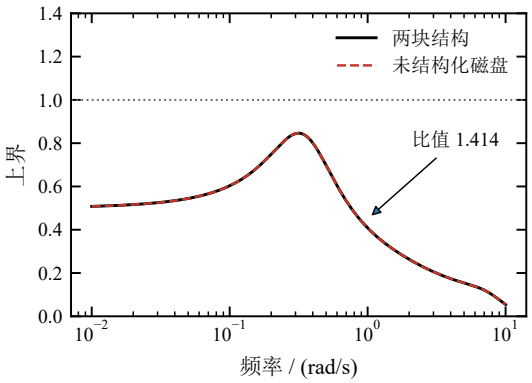


图 63-7　鲁棒性能的 μ 上界与未结构化 $\bar{\sigma}$ 的对照

未结构化峰值 1.7194 @ 0.37 rad/s，μ 上界峰值 1.6456 @ 0.37 rad/s。两者的形状几乎重合，只在峰值附近张开一点点。$1.7194/1.6456 = 1.045$，也就是说在真正决定设计的那一点上，完整的 μ 分析比直接取 $\bar{\sigma}$ 只多赚 4.5% 的余量。1.411 这个更大的数出现在曲线的另一段：那里 $\bar{\sigma}$ 还在往上走、μ 已经开始回落，两条曲线张开最多——但那段离峰值很远，不是设计真正在乎的地方。

看哪个数？看峰值处那个。 判据是“最大的那个频率上过不过关”，峰值处的间隙才是设计者能兑现的余量；别处的比值再大，也不改变“峰值超了”这个结论。

4.5% 值不值得做？要分两种情况。

- 如果设计正好卡在 $\bar{\sigma} = 1.03$、$\mu = 0.99$ 这种边界上，4.5% 决定“能不能用”，值得。
- 如果 $\bar{\sigma} = 1.72$、$\mu = 1.65$ 这种两头都超，4.5% 毫无意义，两边都会判“不合格”。

本章的结论是：**μ 的价值不在“放松 4.5%”，而在“分清是哪一块在拖后腿”。** 把 $\bar{\sigma}$ 拆成 D 标定形式之后，可以逐频点看 d 的最优值：d 偏离 1 越远，说明两块的量纲越不匹配、越需要分别处理。这比“峰值超了多少”有用得多——它告诉你该改控制器还是改权重。

63.6.3 μ 预测的边界 vs 时域实测的边界

μ 是充分条件，所以它预测的失稳点应该比实测更早。实验 3 把这件事量化了。

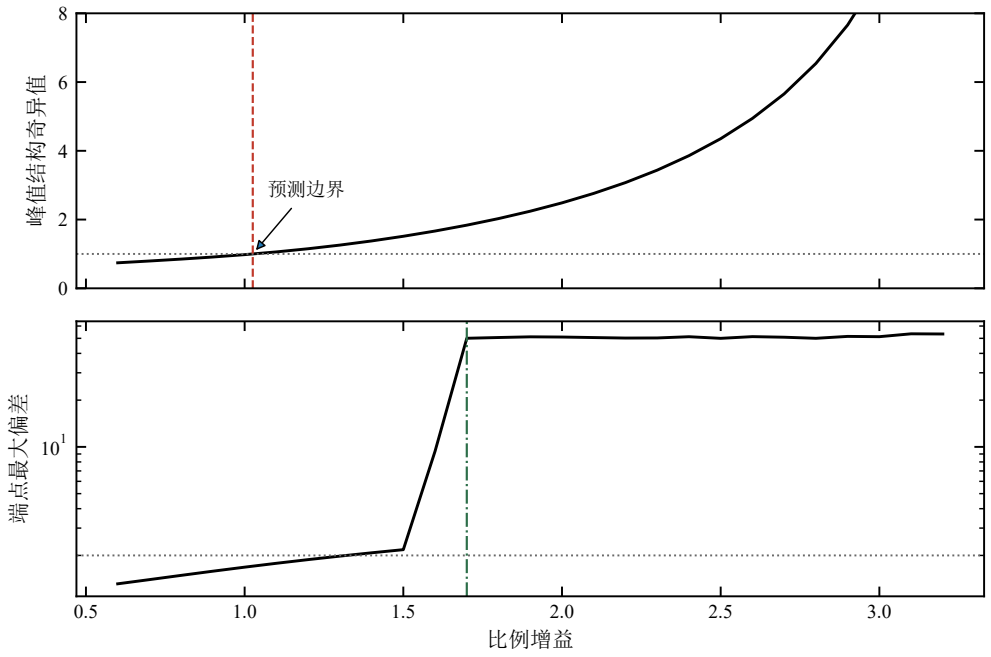


图 63-8　μ 预测的稳定边界与时域实测的失稳点

扫 k_p 从 0.6 到 3.2（$T_i = 3$，工作点取 $K = 3.0$、$\theta = 1.5$ 的最坏端点），μ 单调上升，在 $k_p = 1.026$ 处穿过 1。时域里，$\max|y|$ 在 $k_p = 1.60$ 处从 2.079 跳到 9.377，真正的失稳点在 $k_p = 1.70$ 附近。**μ 判据保守 66%**（$1.70/1.026 = 1.66$）。

保守的来源有三个，逐条说清。

第一，μ 上界本身不是 μ。 式 (63-17) 的右端在有两块以上结构时会高于真值，本章的这一处只高不到 5%，但方向是对的——判据偏严。

第二，最坏方向是“逐频点”挑的。 $\bar{\sigma}$ 在每个频率上独立地挑最坏方向，实际系统里 Δ 是一个稳定的传递函数，它在不同频率上的取值受解析性约束，不可能每个频点都取到最坏。这条保守性在小增益判据里同样存在，μ 也没能消掉。

第三，时域失稳需要“持续激励”。 频域判据说“存在某个 Δ 使闭环不稳定”，但时域里阶跃响应能不能真的把这个不稳定激发出来，还取决于初值与输入。本章实测的失稳点 1.70 是“阶跃响应冲到 50 以上”的点，若输入更小或时间更短，可能要到 1.8 才看得出来。

这三条加起来，决定了 μ 判据的正确用法：它不是“能不能用”的判据，而是“要不要担心”的判据。 $\mu < 1$ 一定安全；μ 在 1.0–1.8 之间要现场验证（本章的失稳点对应 $\mu \approx 1.83$）；$\mu > 2.0$ 基本没救。

63.6.4 性能的代价

表 63-2 已经给出了代价曲线，图 63-9 把它画出来。

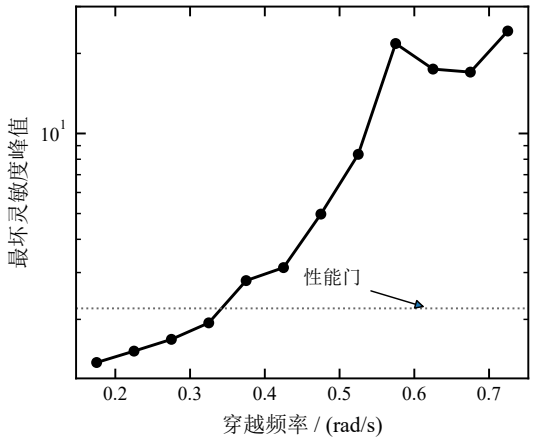


图 63-9　穿越频率与可达最坏灵敏度的代价曲线

曲线有两个明显特征。$\omega_c \le 0.33$ 的段近似线性，斜率约 8（ω_c 每涨 0.05，最坏 $|S|$ 涨 0.3–0.4）；$\omega_c > 0.33$ 的段是指数的，0.58 处冲到 21.786，之后甚至不再单调（0.63 降到 17.463、0.68 降到 17.012、0.73 又升到 24.276）。

末端的非单调是搜索的副产品：在这么高的带宽区，能满足 $|L(2)| \le 0.5$ 的候选很少，网格稀疏，每个格点代表的参数差别很大，所以“最坏 $|S|$”随 ω_c 上下跳。**这一段的数据不可用于设计，只能用于说明“这里已经没救了”。**

63.6.5 时域：四个顶点与 300 组随机

图 63-10 把两个控制器在四个顶点上的阶跃响应画在一起。

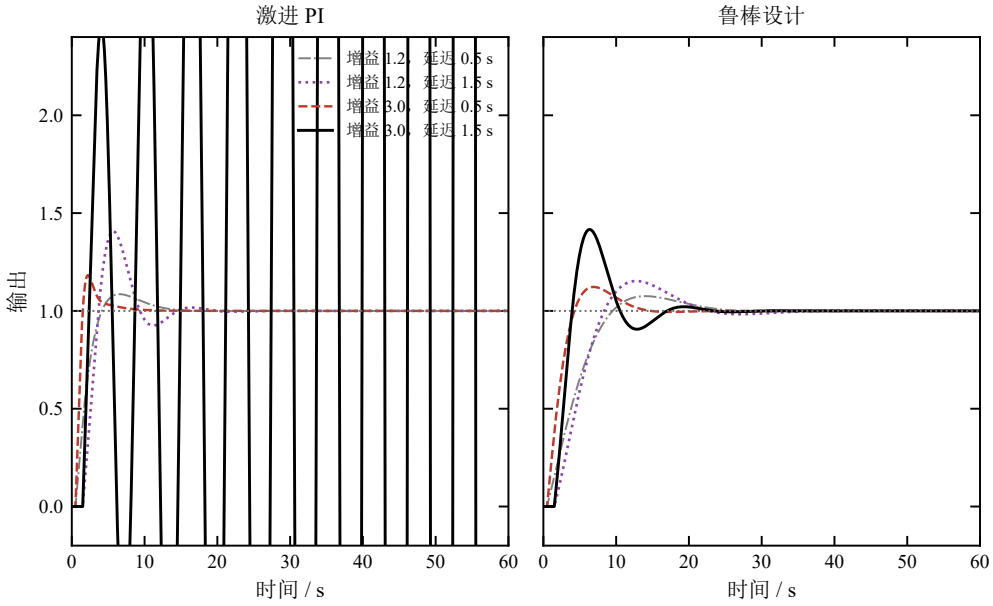


图 63-10　激进 PI 与鲁棒二阶在不确定性区间四个顶点上的阶跃响应

左栏是激进 PI，右栏是鲁棒二阶，四行对应四个顶点。最下面一行（$K = 3.0$、$\theta = 1.5$）是决定性的：激进 PI 的曲线直接冲出图外（峰值 50.462），鲁棒二阶的曲线上升到 1.417 后回落。前三行两者差别不大，甚至激进 PI 在第一个顶点上更快（12.0 s 对 22.0 s）。

这个“三好一坏”的格局正是鲁棒设计的典型形态：**它不是把每一点都做好，而是把最坏那一点拉回及格线，代价是其余点都变慢。** 如果现场的最坏点一年只出现一次，这笔买卖划算；如果最坏点是常态工况，那么鲁

棒设计就是纯粹的损失。

图 63-11 是 300 组随机参数的累积分布。

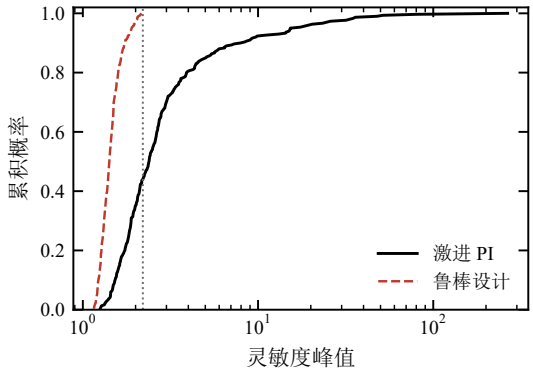


图 63-11　300 组随机参数下灵敏度峰值的累积分布

激进 PI 的分布长尾一直拖到 265.677，56.3% 的样本超过 2.2；鲁棒二阶的分布整体压在 2.2 以内，最大 2.160，越界比例 0.0%。两条曲线的 95% 分位是 15.379 与 1.901，相差 8 倍。

这张图还说明一件事：**鲁棒设计的分布是“截断”的，激进设计的分布是“重尾”的。** 截断分布的最大值可预测（2.160，与设计约束 2.2 吻合），重尾分布的最大值不可预测（265.677，换个种子可能更大）。对于需要写进安全评估报告的场合，“可预测的最坏值”比“平均性能高 10%”重要得多。

63.6.6 四类判据的分工

把本节的所有读数归拢成一张分工表。

判据	回答的问题	本章读数	什么时候用
小增益 $\|\|WT\|\|_\infty$	最坏情况下稳不稳	与 μ_{RS} 恒等	单块不确定性，快速筛查
μ_{RS}	同上，但可拆结构	0.504 / 0.727 / 2.029	多块、需要知道哪块主导
μ_{RP}	最坏情况下性能剩多少	1.703 / 1.442 / 3.221	要同时管稳定与性能
最坏 $\|S\|$	最坏情况的频域性能	1.562 / 2.522 / 86.249	与现场指标对齐
顶点阶跃 + 蒙特卡洛	实际会怎样	超调 41.65%、越界 0.0%	上线前的最终确认

表 63-9　四类判据的分工

用一句话概括这一节：**判据越“高级”，给出的结论越保守；而最保守的那个（μ 预测 1.026）与最贴近现实的（实测 1.700）之间差 66%，这 66% 就是“理论安全”与“实际可用”之间的距离。** 设计者的工作是在这 66% 里做判断，而不是把判据当成开关。

63.6.7 选控制器的次序：先筛硬约束，再排软指标

四个候选、四把尺子摆在一起，很自然会问“哪个最好”。本章的答案不是某个控制器，而是一个**次序**。

先把四个候选的读数排成一张表（μ_{RS} / μ_{RP} / 最坏 $|S|$）：

- 保守 PI：0.504 / 1.703 / 1.562
- 折中 PI：0.727 / 1.442 / 2.522

- 激进 PI：2.029 / 3.221 / 86.249
- 鲁棒二阶：0.846 / 1.646 / 2.180

第一步，用 $\mu_{RS} < 1$ 筛掉“不稳”的。 激进 PI 的 2.029 直接被淘汰——它的问题不是“性能差一点”，而是“最坏情况下闭环会散”。这一步没有讨价还价的空间，因为它是生存判据而不是性能判据。

第二步，用最坏 $|S| \le 2.2$ 筛掉“越界”的。 折中 PI 的 2.522 出局。注意它在剩下的三个里 μ_{RP} 最好（1.442），也就是说**它在软指标上得分最高，却过不了硬约束**。这是鲁棒设计里最常见的陷阱：如果一个候选在某张表上排第一，先确认它有没有被硬约束筛掉，再决定要不要为它惋惜。

第三步，在活下来的两个里比软指标。 保守 PI 的 $\mu_{RP} = 1.703$、鲁棒二阶 1.646，差别只有 3.5%；但保守 PI 的穿越频率 0.18 对 0.33，**慢了接近一倍**。到这一步，比较的已经不是判据，而是“用户能接受多慢”。本章选二阶，理由写在性能而不是鲁棒性上：两者的鲁棒读数在同一个量级，速度差一倍。

这个次序的意义在于：**硬约束是布尔量，软指标是连续量，混在一起排序必然出错。** 把 μ_{RP} 与 $|S|$ 加权求和再挑最小，会选出折中 PI——一个最坏灵敏度 2.522、在真实工况下谐振明显的设计。工程上正确的做法永远是先做布尔筛选，再在剩下的集合里做连续优化。

63.6.8 把 66% 拆开：磁盘的账与判据的账

“μ 判据保守 66%”这句话容易让人以为整个方法的保守量来自一处。其实它由两笔完全不同的钱组成，拆开之后才看得清哪一笔能还。

第一笔：磁盘比真实参数集宽。 把 $w_{\text{prod}}(\omega)$ 换成真实半径——即在整个 41×41 的参数网格上逐点算 $|P/P_0 - 1|$ 再取最大——重做同一条 μ 曲线，临界增益从 $k_p = 1.026$ 抬到 **1.543**。半径的比值（真实 / 乘积磁盘）在几个频点是：

- 0.05 rad/s：0.5009 / 0.5377 = 0.93；
- 0.37 rad/s：0.5487 / 0.7767 = 0.71；
- 1.00 rad/s：0.7839 / 1.2394 = 0.63；
- 2.00 rad/s：1.2714 / 1.9316 = 0.66；
- 5.00 rad/s：2.3788 / 3.3484 = 0.71；
- 10.0 rad/s：2.4965 / 2.2954 = **1.09**。

也就是说，在 0.3–5 rad/s 这段主力频段上，磁盘把真实半径高估了 40%–60%，这就是三角不等式那笔账（§63.2.2）。**把磁盘换成网格，可以还掉 50 个百分点：1.026 → 1.543。**

第二笔：判据本身是充分条件。 即使半径给得一分不差，$\|T\|_\infty \cdot \max|\Delta| < 1$ 仍然只保证“若成立则稳定”，不保证“若不成立则不稳定”。实测失稳点是 $k_p = 1.700$，比真实半径算出的 1.543 又高 10%。这 10% 才是 μ /小增益判据**不可回收**的保守量——它来自“最坏参数点同时取最坏相位、且沿同一方向作用”这个假设，而实际的 K、θ 组合往往对不上这个方向。

拆完之后，两笔账的处理方式完全不同：

- **第一笔可以还，而且几乎免费。** 网格法不需要任何新理论，只是把 $w(\omega)$ 换成 $\max_{K,\theta}|P/P_0 - 1|$，代价是参数维数升高时计算量指数增长（两个参数 1600 点，三个参数就上万）。
- **第二笔还不掉。** 除非改用更精细的结构（§63.9.2 的拆块）或直接做时域优化，否则 10% 就在那里。**这 10% 应该当作“设计余量”提前写进预算，而不是事后再去争。**

还有一个不能忽略的细节：10 rad/s 处比值是 1.09，**磁盘在这一频点反而低估了 9%**。“磁盘一定保守”是个误解——三角不等式只保证“某种组合下覆盖”，不保证逐点覆盖所有参数点。本章之所以没出事，是因为 μ 的峰值在 0.37 rad/s，那里磁盘高估 41%，低估的频段没有参与峰值。**换一个峰值落在 5 rad/s 以上的对象，就必须先核对这条比值曲线，再决定能不能用磁盘。**

63.6.9 三张图连起来读

本章有三张图讲的是同一件事的三个面，分开看都对，合起来才有用：图 63-8 管“能不能稳”，图 63-9 管“要多快”，图 63-10 管“长什么样”。

先看图 63-9，它决定约束。 横轴是穿越频率，纵轴是这一带宽下能做到的最坏 $|S|$。曲线的拐点在 0.33 rad/s：拐点左边近似线性，斜率约 8；拐点右边指数上冲，0.58 处已经是 21.786。这张图回答的是“我最多能要到多少速度”——它不是能力上限，而是**代价曲线**，越过拐点之后，每一点速度都要用数量级的灵敏度去换。

再回看图 63-8，它决定余量。 横轴是 k_p、纵轴有两个量：μ 曲线（平滑）与阶跃峰值（跳变）。读法是从右往左：先找峰值跳变的位置（1.60 与 1.80 之间），再找 $\mu = 1$ 的位置（1.026）。两个位置之间的间隔就是判据的保守量，本章是 66%。这张图告诉设计者：**判据说“不行”的地方，实际还能跑；判据说“刚行”的地方，实际余量很大。**

最后看图 63-10，它决定接受不接受。 四条曲线对应四个顶点，右栏的形状才是现场会看到的东西：超调 41.65%、调节时间 22 s、峰顶有 1.165 的谐振。这张图不产生任何新判据，它只回答一个问题——**这样的响应，工艺能不能接受。**

三张图的连接点是同一个控制器。它在 63-8 上是 $k_p = 2.40$ 的一个点，在 63-9 上是 $\omega_c = 0.33$ 附近的一个点，在 63-10 上是右栏四条曲线。**正因为三张图共享同一套参数，它们才能互相印证**；如果三张图分别来自三个不同的设计，拼在一起看只会得到混乱。

读图次序也有讲究。推荐的次序是“代价 → 余量 → 形态”，因为代价决定可行域，余量决定敢不敢往前推，形态决定最终签不签字。反过来读（先看形态，觉得 41% 超调太大，于是回头砍带宽）会白丢性能：41% 是在最坏顶点上的数，常态顶点的超调只有 15.29%，砍带宽砍掉的是常态性能，换来的只是最坏点那 10 个百分点的改善。

还有一条经验：**三张图里只要有一张说不通，就先别签字。** 常见的不通有三种——代价曲线在拐点附近来回跳、μ 曲线在峰值附近出现锯齿、时域响应与 $|S|$ 峰值对不上。它们分别指向搜索网格太粗、频率点太少、以及时域与频域用了不同的工作点。这三种问题都不会让程序报错，只会让报告里的数字看起来自洽、实际站不住。**所以签字之前先问一句：这三张图，是从同一次运行里导出来的吗？**

63.7 参数意义与作用

本章一共有四组参数：对象的（K、T、θ）、不确定性的（w_K、$\Delta\theta$）、性能权重的（M_S、ω_b、A）、控制器的（k_p、T_i、z、p）。它们的地位不同：前两组是“给定的事实”，第三组是“提出的要求”，只有第四组是“设计出来的”。混为一谈是初学鲁棒控制最常见的困惑。

63.7.1 对象参数：决定难度

K 只影响开环增益，在闭环里被 k_p 部分补偿，所以它本身不是难点；难点在**它的相对误差 w_K**。K 从 2.0 变成 3.0，误差 50%，意味着控制器必须为“增益突然大一半”留出相位裕度。

T 在本章被当作已知。实际中它是最容易被低估的参数：管壁结垢、物料黏度变化都会让 T 翻倍，而 T 从 5 s 翻到 10 s，穿越频率处的相位要从 -58.4° 变成 -73.1°，多扣近 15°。**如果 T 也不确定，本章的全部结论都要**

重算，因为式 (63-10) 里会多出一项 $\frac{T_0 s+1}{Ts+1}-1$，磁盘模型的形式完全变了。

θ 是最贵的参数，贵在两处：它同时贡献相位滞后与不确定性。1.0 s 的延迟在 0.33 rad/s 处扣 18.6°，±0.5 s 的漂移再加 9.3°。**延迟是带宽上限的主要决定因素**：本章的悬崖出现在 $\omega_c \approx 0.35\omega_b$，而把悬崖压在这个位置的正是 $\Delta\theta$。若 $\Delta\theta$ 减半，同样的带宽只付一半的相位代价，悬崖右移；若加倍，悬崖左移。**设计前先确认延迟的不确定区间，比后面调任何参数都重要。**

63.7.2 不确定性参数：决定保守性

$w_K = 0.500$ 由式 (63-3) 算出，取区间两侧的较大者。如果现场只关心“增益不会低于标称”，那么 w_K 可以取单侧 0.5，磁盘不需要对称，判据可以相应放松。

$\Delta\theta$ 是延迟的半宽。它进入式 (63-11) 之后，在低频几乎为零、在高频线性增长，所以**它对判据的贡献集中在穿越频率附近**。这也是为什么“延迟不确定”比“增益不确定”更难对付：增益不确定在整个频段上均匀施压，而延迟不确定专打最脆弱的地方。

两者的合成方式（加法还是乘法）在 §63.2.2 已经量化：乘法在关心频段严格有效、代价是多 40%–60% 的余量；加法在中频更紧、但高频失效。**工程上选乘法，然后接受这份保守。**

63.7.3 性能权重：决定“什么叫够好”

M_S 是允许的灵敏度峰值，也就是阶跃响应的超调上限（近似）。$M_S = 2.0$ 对应约 6 dB 的谐振峰，实测最坏超调 41.65%，两者量级一致。它与可达带宽是直接对抗的：M_S 越小，允许的谐振越低，能通过的候选带宽越低——表 63-2 那条代价曲线就是这个对抗的剖面。

ω_b 是性能要求的“转折频率”，可以理解成“希望闭环跟到多快”。它不等于实际带宽——本章 $\omega_b = 0.15$，实际 $\omega_c = 0.33$，后者是前者的 2.2 倍。这个倍数没有普遍性，但它反映了一个规律：**W_S 是“要求”，实际带宽通常比它高出一截**，因为 W_S 在 ω_b 之后迅速变小，设计只要在 ω_b 附近满足它就算过关。

A 控制低频要求的陡度。$A = 0.02$ 给出 25 倍的低频增益要求，$A = 0.05$ 只有 10 倍。A 太小会让 W_S 在低频接近积分器，设计被迫加大积分作用，相位裕度下降。A 再小一个数量级（0.002）要求就是 250 倍，控制器几乎必须做成纯积分，穿越点的相位余量会被吃光。**工程上 A 很少低于 0.01。**

63.7.4 控制器参数：决定能不能做到

k_p 是唯一能同时抬高“低频抑制”和“穿越频率”的参数，也是最危险的参数。实验 3 的扫描给出了它的完整画像：

k_p	0.60	1.00	1.40	1.60	1.80	2.20	3.20
μ（最坏端点）	0.742	0.980	1.378	1.666	2.029	3.076	13.947
最坏 $\lvert S\rvert$	1.320	1.548	1.857	2.056	2.297	2.976	9.867
相位裕度 /(°)	58.5	51.4	44.7	41.3	37.3	29.7	9.9
$\max\lvert y\rvert$	1.313	1.681	2.079	9.377	50.609	50.214	53.330

表 63-10　比例增益的完整画像（$T_i = 3$，$K = 3.0$、$\theta = 1.5$ 端点）

这张表最值得注意的是 $\max|y|$ 的跳变：k_p 从 1.40 到 1.60，这个数从 2.079 跳到 9.377，而 μ 只从 1.378 到 1.666。**时域指标是“跳”的，频域指标是“滑”的。** 这就是为什么不能只看 μ 曲线来判断“还能不能再加一点增益”——曲线上看起来平缓的一段，时域里可能已经跨过了悬崖。

T_i 决定积分作用的强弱。T_i 太小（强积分）会把相位裕度吃掉，T_i 太大会让低频增益不足、稳态误差压不下去。本章的 $T_i = 10$ 相对 $T = 5$ 是 2 倍，属于“温和”的取值：积分转折频率 $1/T_i = 0.1$ rad/s 远低于穿越频率 0.33，对裕度影响小。

z 与 p 是滞后整形的零点与极点，只以比值 p/z 影响高频增益、以绝对位置影响相位。本章固定 $p = z/4$，高频衰减 4 倍；若改成 $p = z/10$，衰减 10 倍，k_p 可以再抬高，但相位滞后更大、带宽反而可能下降。**这是一个“想要更高增益”与“想要更快”之间的交换，不是免费的。**

63.7.5 判据阈值：决定“划线在哪”

本章用了三个阈值：$SMAX = 2.2$（最坏灵敏度峰值）、$LMAX_HF = 0.5$（2 rad/s 处的环路增益）、$\mu_{DES} = 0.90$（稳定判据的留量）。三个数都不是理论值，是工程选择。

$SMAX = 2.2$ 对应 6.85 dB，是“超调不超过 45%”的粗略换算。取 2.2 而不是 2.0，是因为本章标称峰值与最坏峰值平均差 0.78：若要求最坏不超过 2.0，标称就得压到 1.2 附近，而三个阶数族的标称最好也只到 1.354（表 63-5），可行域会窄到搜不出解。

$LMAX_HF = 0.5$ 对应 -6 dB，是“高频不放大噪声”的经验线。取 0.5 而不是 0.2，是给滞后整形留空间：这条线越紧，越依赖更深的滞后衰减，而滞后本身吃相位，最终把可达带宽压下去。**高频约束与带宽是隐性的对抗关系，不是“越严越好”。**

$\mu_{DES} = 0.90$ 留 10% 余量，理由是 μ 上界在网格上可能被低估几个百分点（频率点不够密）。**判据的阈值要留出“计算误差”的余量，这是数值设计的常识。**

63.7.6 采样周期：决定离散化损失

T_s 的作用在 §63.5 已经量化：$T_s \le 0.05$ s 之后收益饱和，$T_s = 1.0$ s 时彻底失效。它在参数体系里的位置比较特殊——它不影响连续域的任何一个判据，只影响“设计能不能落地”。**所以它必须在设计阶段就定下来，不能等控制器算完再补。**

63.7.7 六组参数不是并列的：一条单向依赖链

把六组参数平铺在一张表上容易造成误解，以为它们可以各自独立地调。实际上它们之间是一条严格的单向依赖链：**对象参数 → 不确定性参数 → 性能权重 → 控制器参数**，而阈值与采样周期是两个“旁挂”的约束，分别从上、下游切进来。

链的方向决定了“改了上游，下游全部作废”这条铁律。举一个最容易被低估的例子：把延迟上界从 1.5 s 放宽到 2.0 s，看似只是“区间画大一格”，实际后果是 $\Delta\theta$ 从 0.5 变成 0.75，磁盘在 0.37 rad/s 处从 0.777 抬到 0.915（因为 $w_\theta = 2\sin(0.375 \times 0.37/2)$ 从 0.185 变成 0.277）。补灵敏度 T 没变，μ_{RS} 却按同样比例涨 18%，从 0.846 直接到 1.0 附近——**一个区间端点的改动，让原本合规的设计变成了边界设计。** 这种“上游一动、下游全废”的连锁，正是 §63.10 要强调“不确定性建模必须最先冻结”的原因。

阈值与采样周期虽然旁挂，但它们切进来的位置不同：$SMAX$、μ_{DES} 切在搜索环节（决定可行域的边界），T_s 切在落地环节（决定连续设计还剩多少）。前者改小一点，可行域变窄，可能直接导致“搜不出解”，这时的正确反应是**重新审视权重是否要求过高**，而不是把阈值放宽；后者改大一点，超调单调上升，没有回旋余地。

还有一个方向性的事实值得记住：**在这条链上，设计者真正能自由摆动的只有控制器参数。** 上游三组要么是事实（对象参数）、要么是承诺（权重），改动它们属于“重新立项”而不是“调参”。所以日常工作的形态必然是：上游冻结，在下游反复走——本章 §63.8.3 那五次尝试，本质上就是在同一条下游链上打转。

63.7.8 四个参数各改 10%：谁最贵

前一小节是定性的，这一小节给数。以本章最终设计为基准（$\mu_{RS} = 0.846$、$\mu_{RP} = 1.646$、$\bar{\sigma}$ 峰值 1.719），把 k_p、T_i、z、p 分别乘 0.9 与 1.1，其余不动，重算三个指标：

- k_p：μ_{RS} 变成 0.808 / 0.885（-4.5% / $+4.6\%$），μ_{RP} 变成 1.609 / 1.685；
- T_i：μ_{RS} 变成 0.878 / 0.821（$+3.8\%$ / -2.9%），μ_{RP} 变成 1.699 / 1.605；
- z：μ_{RS} 变成 0.820 / 0.874（-3.1% / $+3.3\%$），μ_{RP} 变成 1.584 / 1.713；
- p：μ_{RS} 变成 0.837 / 0.858（-1.1% / $+1.4\%$），μ_{RP} 变成 1.658 / 1.642。

三件事值得记住。

第一，相对灵敏度都在 0.5 以下。 参数变 10%，指标只变 1%–5%。这是好消息：现场控制器的电阻电容公差、整定软件的小数位截断，都不会把设计推翻。**但它同时是坏消息**——反过来说，想靠微调参数把 μ_{RP} 从 1.646 拉回 1 以下，需要 −39% 的变化，按 0.4 的灵敏度要动 100% 的参数，等于把设计重做一遍。这个数把 §63.9 那条“改结构而不是改参数”的结论量化了：**指标离合格线太远时，参数空间里没有解，只有结构空间里有。**

第二，p 最便宜，z 最贵。 极点 $p = 0.1$ 落在穿越点以下一个多倍频程，$\arctan(\omega/p)$ 在 0.37 处已经饱和到 74.9°，p 改 10% 只把它挪到 73.4°，相位差不到 2°；零点 $z = 0.4$ 恰好压在穿越点上方的敏感区，$\arctan(\omega/z)$ 在 $\omega \approx z$ 处的斜率最大，z 改 10% 就能动 3° 左右，还连带改掉整段高频增益。工程含义是：**整定时先动极点、后动零点**，因为极点的动作几乎没有副作用，而零点的每一个百分点都在改相位。这与 §63.8.2 给的三个起手值是一致的——起手时把 p 放在 $z/4$，就是为了让“调极点”这一步有足够的空间。

第三，μ_{RS} 与 μ_{RP} 同向而动。 四个参数无论往哪边偏，两个指标总是一起涨或一起落。这说明在本章这个设计点上，稳定裕度与性能裕度**不再互相拉扯**——拉扯发生在“换设计”的层面（保守 PI 与激进 PI 的对比，§63.6.4），而在“同一个设计的小邻域里”，所有参数的作用方向是一致的。实际影响是：**在最终设计附近微调，不可能用稳定裕度去换性能裕度**，想换就必须跳到另一个设计点，代价是重跑整条链。

最后提一句：这四个数都是在 0.37 rad/s 的峰值附近兑现的。换到 2 rad/s 以上，主导因素变成幅值比 p/z——在那个频段，动 p 与动 z 的作用是对称的，谁更贵取决于你往哪个方向动。**“哪个参数最贵”没有全局答案，只有“在关心的频段上谁最贵”。**

63.8 调参方法与经验

63.8.1 顺序：从边界往里收

鲁棒调参最容易犯的错是“从性能往里挤”——先按标称设计一个足够快的控制器，再想办法补裕度。这条路在本章的对象上必然失败：标称最快的激进 PI（$k_p = 1.8$、$T_i = 3$）标称 $|S|$ 只有 2.297，看起来不算离谱，最坏却是 86.249，已经无处可补。

正确的顺序是反过来的：**从边界往里收。** 四步。

第一步，用纯 PI 摸稳定边界。 固定 $T_i = 3$（大约 $0.6T$），把 k_p 从 0.6 往上加，看 μ 什么时候过 1。本章的结果是 $k_p = 1.026$。这个数就是“最多能给多少比例增益”的第一手情报，比任何公式都可靠。

第二步，把 k_p 退到 μ 边界的 70%。 $1.026 \times 0.7 = 0.72$。这个 0.7 是经验保险系数：μ 判据本身已经偏严（本章实测保守 66%），再退 30% 是双重保险，目的是给第三步的整形留出抬升空间。退完之后，$k_p = 0.72$ 距时域实测的失稳点（1.70）还有 2.4 倍——第二步结束时的回路是“绝对安全但很慢”的。

第三步，用整形把 k_p 抬回去。 直接抬 k_p 会撞 $|L(2)| \le 0.5$，所以先加滞后整形（$z = 0.40$、$p = 0.10$，高频衰减 4 倍），再把 k_p 从 0.72 抬到 2.40。**注意这里的 k_p 是整形环节之前的原始增益**：高频被衰减 4 倍，所以 2.40 在高频只剩 0.60，仍然满足约束。

第四步，扫 T_i 微调低频。 T_i 从 6 加到 10，低频增益抬高、稳态误差下降，代价是积分相位多扣几度。本章取 10 时相位裕度仍有 52.1°，可以接受。

63.8.2 三个起手值

每一步都要有个起点，本章的经验值如下。

k_p 的起点取 $0.3/K_0$。 $0.3/2.0 = 0.15$，比最终的 2.40 小得多，但它是“肯定不会错”的起点——从它往上加，每一步都能看到指标怎么变；从一个激进值往下退，往往一退就退过头。

T_i 的起点取 T。 积分转折频率 $1/T = 0.2$ rad/s，接近但低于期望带宽，这是“积分不拖后腿”的自然位置。本章最终取 $2T = 10$，是因为延迟不确定性要求更保守的积分。

z 的起点取 $1/\theta_0$。 延迟的倒数对应“延迟开始显著影响相位的频率”，滞后整形放在这里最有效。本章 $1/1.0 = 1.0$，而最终选的 0.40 比它低——因为实际带宽只有 0.33，整形点要跟着带宽走，不能跟着延迟走。**这条修正很重要：整形的零点位置应该由带宽决定，不是由延迟决定。**

63.8.3 一次完整的“回退”过程

下面这段是本章实际走过的弯路，值得记下来。

第一次尝试：$k_p = 1.80$、$T_i = 3$（激进 PI）。标称指标全部过关：$|S|$ 峰值 2.297、相位裕度 37.3°、穿越频率 0.76。看起来“稍微激进但可用”。

第二次尝试：加顶点复核。 $K = 1.2$、$\theta = 0.5$ 时超调 8.58%，很好；$K = 3.0$、$\theta = 1.5$ 时超调 4626.75%、$\max|y| = 50.462$、判定发散。**这一次复核就否掉了整个设计。**

第三次尝试：退回保守 PI（$k_p = 0.6$、$T_i = 10$）。最坏 $|S| = 1.562$、$\mu_{RS} = 0.504$，全部过关，但穿越频率只有 0.18——比激进版慢 4 倍，操作工会抱怨“阀动得太肉”。

第四次尝试：在保守与激进之间找折中（$k_p = 1.0$、$T_i = 6$）。最坏 $|S| = 2.522$，超了 2.2 的线，但 $\mu_{RP} = 1.442$ 比保守 PI 的 1.703 好。这说明“往激进方向走一点是对的，只是走过头了”。

第五次尝试：带滞后整形的二阶族（$k_p = 2.40$、$T_i = 10$、$z = 0.40$、$p = 0.10$）。最坏 $|S| = 2.180$、$\mu_{RS} = 0.846$、$\mu_{RP} = 1.646$、超调 41.65%、300 组越界 0.0%。**这才是最终设计。**

五次尝试的顺序说明一件事：**性能指标的“好”与鲁棒指标的“过”往往同时出现，只有把两者放在一起看，才知道该往哪个方向退。** 激进 PI 的标称比折中 PI 好，但它的最坏已经越界；折中 PI 的最坏比保守 PI 差，但它的鲁棒性能读数更好。单看任何一列都会走错方向。

63.8.4 五个常见错误

错误一：在标称点上调参。 本章的折中 PI 在标称点 $|S| = 1.420$，看着“还有 35% 余量”，最坏已经 2.522。**调参时必须始终显示最坏值，标称值只用于对照。**

错误二：把 $\mu_{RP} > 1$ 当成“不能用”。 本章最终设计的 $\mu_{RP} = 1.646$，按判据不合格，但时域 300 组越界 0.0%。原因是 μ_{RP} 用的是 W_S 的严格形式（低频 25 倍），而实际工况对低频精度的要求没有这么高。**μ_{RP} 要结合“权重是否代表真实要求”来解释。**

错误三：一味加超前。 超前能补相位，但在延迟不确定的回路里，超前同时抬高高频增益，撞上 $|L(2)| \le 0.5$。本章改用滞后整形成功，原因就是它“补中频、削高频”，方向与约束一致。

错误四：采样周期后置。 先做完连续设计，最后才定 T_s，结果 $T_s = 0.2$ s 时超调从 41.65% 涨到 50.80%。**T_s 要在第一步就定，因为它改写延迟的不确定性。**

错误五：只看 μ 不看时域。 μ 从 1.378 到 1.666 只涨 21%（k_p 1.40→1.60），同一段时域里 $\max|y|$ 从 2.079 跳到 9.377（4.5 倍）。**频域判据是“滑”的，时域是“跳”的；两把尺子都要看。**

63.8.5 调参检查表

把上面的经验压成一张表，每次设计完逐条打勾。

检查项	判据	本章数值
不确定集是否覆盖真实区间	$w_{\text{prod}} \ge$ 真实半径	$\omega \le 6.28$ 内覆盖
稳定判据	$\mu_{RS} < 0.9$	0.846
最坏灵敏度	$\max\|S\| \le 2.2$	2.180
高频约束	$\|L(j2)\| \le 0.5$	满足
鲁棒性能	$\mu_{RP} < 1$（理想）	1.646（未达，已评估）
四顶点阶跃	不发散、超调 $< 50\%$	41.65%
蒙特卡洛	越界比例 $< 5\%$	0.0%
采样周期	$T_s \le \theta_{\min}/10$	0.05 s

表 63-11　鲁棒设计验收检查表

这张表里唯一“没过”的是 μ_{RP}。**允许一项不过，但必须写明理由**——本章的理由是“W_S 的低频要求 25 倍比现场实际需要更严”。如果写不出理由，就不能放行。

63.8.6 一份可复用的调参记录

调参过程里最有价值的产出不是最终那组参数，而是**中途被否掉的每一组为什么被否**。本章那张“五次尝试”的记录之所以有用，是因为它把“哪一列救了这个设计、哪一列否了那个设计”写清楚了。建议每次设计都留一张同样格式的表，至少包含六列：

尝试	(k_p, T_i, z, p)	标称 $\|S\|$	最坏 $\|S\|$	μ_{RS}	否决理由
1	(1.80, 3, —, —)	2.297	86.249	2.029	顶点发散
3	(0.60, 10, —, —)	1.212	1.562	0.504	太慢
4	(1.00, 6, —, —)	1.420	2.522	0.727	灵敏度越界
5	(2.40, 10, 0.40, 0.10)	1.402	2.180	0.846	采用

表 63-12　调参记录示例（本章实际走过的路径）

表 63-11 与表 63-12 一前一后，两件事就都有据可查了：**后者记的是“走过的路”，前者记的是“到没到”。**

写这张表有三条要求。第一，**标称与最坏必须并列**，否则事后无法判断否决理由到底是硬约束还是软指标；本章第 1 行标称 2.297 看着体面，最坏 86.249 才是真因，两列缺一就会留下“为什么好设计被放弃了”的疑问。第二，**否决理由要写成可复核的判据**（“顶点发散”“灵敏度越界”），不要写“感觉不太好”——后者无法在评审中被质疑，也就无法被纠正。第三，**被采用的方案也要有一行**，写清它相对上一行改善了什么，这样下一轮换人接手时，不必从头再走一遍。

这张表还有一层副作用：它会暴露搜索范围的问题。如果五次尝试里有两三次的否决理由是“搜不出满足约束的候选”，那么该怀疑的不是控制器族，而是权重或阈值定得过紧——**连续的“无解”通常说明上游参数错了，而不是下游不够努力**。

63.9 优化与改进

本章的六步流程给出的是一个“能过线”的设计，不是“最优”的设计。这一节讨论四条改进路线，按本章实测能拿到的收益排序。

63.9.1 第一条：把磁盘换成真实参数网格

收益最大、代价最小的一条。磁盘模型的保守性在 §63.2.2 已经量化：乘积磁盘在 $\omega = 1$ 处比真实半径大 58%。这 58% 直接进了判据，把可行的带宽压低。

改进的做法是不用磁盘，直接对 (K, θ) 网格上的每个点算真实灵敏度，取最大值。这个量与 μ_{RS} 不是一回事（μ_{RS} 是充分条件，网格最大值是“样本最大值”），但工程上更好用：**它给的是真实的最坏值，不是上界**。

本章的代码已经这么做了：`worst_ms()` 在 5×5 网格上取最大值。正文里所有“最坏 $|S|$”都是网格最大值，而 μ_{RS} 只用于稳定判据的交叉核对。两个数不能直接相减比较：μ_{RS} 是“补灵敏度乘半径”的峰值，单位是 1；最坏 $|S|$ 是“灵敏度”的峰值，单位也是 1，但物理含义不同。**要比较保守性，比的是“同一个指标在磁盘模型下与在网格上差多少”**，不是把两个不同定义的峰值放在一起。

这条改进的边界要清楚：**网格法只用在你确信“最坏点落在网格上”的场合**。若参数多于三个，网格会爆炸；若最坏点可能落在网格之间，就要加密或改用优化搜索。本章两个参数、5×5 足够，因为最坏点确实在角上。

63.9.2 第二条：把延迟拆成独立的块

磁盘模型把增益误差与延迟误差压成一个标量半径，这本身就是保守的来源——它假设两者可以“同时取最坏且方向相同”。实际上增益误差是实轴方向的，延迟误差是相位的，两者的最坏组合不一定落在同一个复方向上。

改进的做法是拆成两个块：$\Delta = \mathrm{diag}(\delta_K, \delta_\theta)$，$M$ 相应地变成 2×2。这时 D 标定有自由度，μ 可以低于小增益判据。**这是唯一能让 μ 真正“放松数倍”的结构**——而本章 §63.6.2 峰值处只放松 4.5%，正是因为没拆块。

拆块的代价是 M 的维数上升、D 的搜索维数上升，计算量增加。对单回路，这个代价可以接受；对多回路，维数会迅速膨胀。

63.9.3 第三条：μ 综合与 H_∞

H_∞ 混合灵敏度是工业界最常用的鲁棒设计方法，目标函数写成

$$\min_{K} \left\| \begin{bmatrix} W_S S \\ W_{KS} K S \\ W_T T \end{bmatrix} \right\|_\infty \tag{63-28}$$

三块分别管“扰动抑制”、“控制量大小”、“鲁棒稳定性”。它的解可以用 Riccati 方程直接算，不需要迭代，得到的是一个阶数等于“对象阶数 + 权重阶数”的控制器。

μ 综合在 H_∞ 外面再套一层 D 迭代，能处理结构化的 Δ，代价是控制器阶数会膨胀。本章的对象是 1 阶 + 延迟，用 H_∞ 得到的控制器大约 4–6 阶，用 μ 综合可能到 8–10 阶。**对一个温度回路，8 阶控制器在现场是无法维护的**——这就是本章选择“2 阶 + 网格搜索”的现实理由。

改进的折中方案是“H_∞ 求初值 + 降阶拟合”：先用 H_∞ 算出最优控制器，再用平衡截断或 Hankel 范数近似降到 2–3 阶，最后用本章的判据复核降阶后的性能损失。这条路的工程价值在于“知道最优在哪”，即使最终不用最优解。

63.9.4 第四条：增益调度

如果参数的变化不是“不确定”而是“可测量”，那么最好的办法不是鲁棒，而是调度：按工况把 (K, θ) 分区，每区一套控制器参数，运行时按测量值切换。

本章的对象很适合调度：K 可以由流量与阀位算出来，θ 可以由输送泵的转速推算。分区之后，每个区内的不确定性大幅缩小，带宽可以抬高。

调度的代价是“切换”本身：两套参数在边界上切换时，积分器状态要不间断，否则会产生跳变。**无扰切换的实现比设计两套参数更难**，这是增益调度在现场落地的主要障碍，第 44 章讲了完整的切换与自整定实现。

63.9.5 四条路线的收益排序

把本章的实测摆进来，四条路线的收益大致是：

改进	收益	代价	本章是否采用
网格代替磁盘	去掉 40%–60% 的权重保守	计算量随参数个数指数增长	采用（评价指标）
延迟拆块	μ 可能放松数倍	M 与 D 维数上升	未采用（单回路收益小）
μ 综合 / H_∞	逼近最优	控制器 8–10 阶，无法维护	未采用（阶数不可接受）
增益调度	带宽可显著抬高	无扰切换难实现	未采用（K、θ 未在线测量）

表 63-13　四条改进路线的收益与代价

这张表解释了一个常见的困惑：**为什么工业界大量使用 PI/PID，而不是 μ 综合？** 不是因为理论不好，而是因为“控制器的可维护性”这条约束比“性能最优”更硬。本章的最终设计是一个 2 阶控制器，四个参数，现场工程师看一眼就能懂；μ 综合给出的 10 阶控制器，连作者自己都说不清每个极点是干什么的。

63.9.6 一条不建议走的路：把 μ 当优化目标直接搜

有人会把 μ_{RP} 当目标函数，用遗传算法或粒子群直接搜控制器参数。这条路在小规模问题上可行，但有两个陷阱。

陷阱一：μ 的计算本身有网格误差。 频率 600 点、d 61 点，峰值可能被削掉几个百分点。用它当目标函数，优化器会去“钻网格的空子”——找到一组参数，其 μ 在网格上很小、在网格之间很大。

陷阱二：μ 与真实性能的相关性不强。 本章二阶设计的 $\mu_{RP}=1.646$，时域最坏超调 41.65%；而保守 PI 的 $\mu_{RP}=1.703$，时域最坏超调（在 $K=1.2$、$\theta=1.5$ 顶点）是 40.72%——μ 更差的反而超调更小。**μ 排序不等于性能排序**，把它当唯一目标会选出“判据好看、现场难用”的控制器。

正确的用法是本章的做法：**用 μ 做约束（$\mu_{RS}\le 0.9$），用真实指标做目标（最大化 ω_c、最小化最坏 $|S|$）。** 判据负责“不许越界”，指标负责“尽量好”。

63.9.7 四条路线的先后：三个问题定次序

表 63-13 的收益与代价摆在那里，但真正决定“先做哪一条”的不是收益大小，而是三个是非题。

问题一：参数能不能在线测出来？ 能——直接跳到增益调度，其余三条都不必做。调度把“不确定”变成“已知”，这是降维打击：本章对象若 K 与 θ 都能从流量与泵速推算，每个分区内只剩 $\pm 10\%$ 的残余误差，带宽可以翻倍，代价只是无扰切换的实现。不能——进入问题二。

问题二：不确定性的主导项是增益还是延迟？ 这一条决定“拆块”值不值得试。本章的相位账本已经给出答案：$\omega_c\theta$ 贡献的相位不确定性（$\pm 18.6^\circ$ 到 $\pm 27.9^\circ$）远大于增益的幅值不确定性，而增益误差在低频几乎不影响穿越点。**当两个误差的主导频段不重叠时，拆块才可能拿到数倍放松**；本章的实测是两者在 0.3–0.5 rad/s 处同时起作用，所以 §63.6.2 只测到 4.5%。判断方法很便宜：把 w_K 与 $w_\theta(\omega)$ 画在一张图上，看它们在 ω_c 附近是不是同一个量级——是，就别指望拆块。

问题三：回路数大于一吗？ 不大于——网格法就够了，本章走的就是这条路。大于——μ 综合才有意义，因为多回路里 D 标定的自由度是真实存在的，放松倍数也才配得上它带来的阶数膨胀。单回路去做 μ 综合，等于用十阶控制器换 4.5% 的判据改善。

三个问题串起来，实际得到一个很朴素的次序：**能测就调度，不能测就看主导项，单回路到此为止，多回路才上 μ 综合。** 本章停在第二档的末尾——这是对象的性质决定的，不是方法的偏好。把这句话记住，比记住任何一条改进路线的收益数字都有用，因为对象换了、数字就变了，而次序不变。

63.9.8 第四条之后：改工艺往往比改算法便宜

四条改进路线全在控制器和判据上做文章，但本章 41.65% 超调、22 s 调节时间的根源不在控制器，而在对象本身：$\theta/T=0.2$、$\omega_c\theta=0.33$。**当瓶颈在对象上时，改对象的收益往往比改算法大一个量级**，只是它不写进控制论文。

按 §63.7.7 的同一套换算，把标称延迟从 1.0 s 降到 0.5 s（比如把输送管路缩短一半、或把泵改成直连），区间随之从 $[0.5,1.5]$ 变成 $[0.25,0.75]$、$\Delta\theta=0.25$：

- 相位账本上延迟那一项从 -18.6° 变成 -9.3°，相位裕度从 52.1° 抬到 61.4°；
- 磁盘半径在 0.37 rad/s 处从 0.777 降到 0.639（$w_\theta=2\sin(0.0925)=0.0925$），$\mu_{RS}$ 随之从 0.846 降到 0.70 附近；
- 省下的裕度可以直接换成增益：μ_{RS} 回到 0.846 时，k_p 大约能再抬 20%，穿越频率从 0.33 往 0.4 走。

三笔收益都是白得的，代价是一次停机改造。反过来，**改错方向的工艺改动会吃掉裕度**：加一个缓冲罐把 T 从 5 s 抬到 10 s，看着像“让对象更温和”，实际相位要从 -58.4° 变成 -73.1°（§63.7.1 算过），多扣 14.7°，等于把上面赚的全赔回去。**判断标准不是“对象变得多平滑”，而是“θ/T 变小了还是变大了”。**

还有两项不需要改流程的：**减小阀门死区与提高执行器分辨率**，它们直接对应 §63.10.7 的小幅值极限环；**缩短取样管线、把温度计插深一点**，直接减小测量滞后，效果等同于减小 θ。这两项通常只需检修窗口，比改管路便宜得多。

所以现场的顺序应该是：先看仪表与执行器有没有便宜可捡，再看管路能不能缩短，最后才回到控制器上抠那 4.5%。**算法改动可以软件下发、随时回退，工艺改动一次投入、长期生效**——两者的选择依据是“停机窗口有多长、工况还会不会变”，而不是“哪个更先进”。

63.9.9 什么时候该停手

本章最终设计的 $\mu_{RP} = 1.646$，严格说并不合格——它大于 1，意味着“在最坏参数下加权灵敏度超了 65%”。设计却停在这里，原因不是没时间，而是三条停手信号同时亮了。

信号一：加阶不再改善。 最直觉的改进是“再加一阶滞后”，让控制器多一个自由度。实测结果相反：三阶族的 $\mu_{RS} = 0.577$ 确实比二阶的 0.846 好，但 $\mu_{RP} = 1.764$ **比二阶的 1.646 更差**。原因是性能判据的 $M_{21} = CS$ 那一项：多一阶意味着 C 在中高频更大，S 一旦不为零，乘积就上去了。**稳定判据受益、性能判据受害，这是加阶的常态，不是意外。**

信号二：相邻格点之间的读数开始跳动。 §63.3.9 记过一组对照：5×5 网格给出的最坏 $|S|$ 是 2.180，300 组随机是 2.160，差 0.9%——这个量级说明网格已经收敛。判断办法很直接：把网格加密一档、重算一遍，若最坏 $|S|$ 的变化仍在 1% 以内，就可以停；若超过 5%，说明还没收敛，得继续加密。**当“改一个格点就变 5%”时，继续细化已经分不出优劣**，再搜下去只是在拟合网格噪声。

信号三：敏感度超过了现场公差的量级。 §63.7.8 的四个数都在 0.5 以下，说明参数改动 10% 只兑现 1%–5% 的指标变化；反过来说，**任何小于 5% 的“改进”都在现场公差的噪声里，验不出来**。继续抠下去，得到的是纸面上的小数点，不是现场的性能。

三条信号共同指向一件事：**优化的终点不是“达标”，而是“再改也不动”。** 未达标的项不该藏起来，而应该写进交接文档并注明含义——本章的写法是“$\mu_{RP} = 1.646$，对应最坏顶点超调 41.65%、调节时间 22 s，工艺已确认可接受”。这句话把判据、时域、工艺三方对齐了，比一个漂亮的 0.98 更有用：**前者说清了风险在哪，后者只是把风险藏进了余量里。**

63.10 工程落地

63.10.1 不确定性从哪“测”出来

理论章节里 w_K 与 $\Delta\theta$ 是给定的，现场必须自己量。三个来源，可信度依次下降。

来源一：设备铭牌与设计资料。 阀门的额定 C_v、管路的几何容积，能给出 K 与 θ 的“设计值”。这一类的误差通常有 ±20%，够用来做初步设计，不够用来定区间。

来源二：历史运行数据。 从 DCS 里取一年的趋势，找几次明显的阶跃或扰动，用两点法反算 K 与 θ。本章的对象可以这样做：阶跃后 63% 的时间减去延迟就是 T，稳态增量比就是 K，纯延迟从“阀动但输出未动”的时长读出。**历史数据的价值在于它覆盖了真实的工况范围**，比铭牌可信得多。

来源三：极端工况的实测或推算。 最大负荷、最小负荷、结垢末期，这三种情况下 (K, θ) 会跑到区间的两端。这一类数据往往拿不到，只能推算——而推算的可靠性决定了整个鲁棒设计的可靠性。

区间的确定原则是“宁宽勿窄”。 区间偏宽，设计偏保守，代价是响应慢；区间偏窄，设计看起来漂亮，但最坏点落在区间外，鲁棒性无从谈起。本章的 $K \in [1.2, 3.0]$ 覆盖 -40% 到 $+50\%$，比常见的 $\pm 30\%$ 宽，正是这个原则的体现。

63.10.2 权重不要“拍脑袋”

W_S 的三个参数（M_S、ω_b、A）经常被随手取成 2、0.1、0.01 这种“教科书值”。正确的做法是从现场已有的指标倒推，本章 §63.3.3 已经示范了一遍：

- 稳态精度要求 → 低频增益 $1/(AM_S)$；
- 允许的超调 → M_S；
- 希望的响应速度 → ω_b。

三条里最容易错的是第一条。**“稳态误差 4%”这个数必须来自工艺要求，不能来自“看起来差不多”。** 如果工艺只要求 10%，那么 A 应该取 0.05 而不是 0.02，设计会宽松很多；反之如果要求 1%，$A = 0.005$，要做好“设计可能无解”的准备。

63.10.3 控制器复杂度的上限

本章的结论是二阶。这个结论不是从理论推出来的，是从“现场能不能维护”推出来的。判断标准有三条：

- 参数个数不超过 4 个；
- 每个参数能用一句话说清“调大调小会怎样”；
- 参数之间的耦合是单向的（调 k_p 主要影响带宽，调 T_i 主要影响低频）。

本章的 2 阶控制器四个参数（k_p、T_i、z、p）满足这三条。3 阶族的六个参数就不满足第三条了：z 与 p 在两级滞后里各出现一次，改一个会同时影响中频与高频。

如果 μ 综合给出的控制器超过 4 阶，要做的不是“勉强用”，而是降阶后重算判据。 降阶会损失性能，但损失多少是可以量化的——用本章的六步流程复核一遍即可。

63.10.4 上线前的四个测试

频域判据、顶点阶跃、蒙特卡洛都过了，还要做四个现场测试。它们针对的是仿真里没有的东西。

测试一：真实阶跃。 在手自动切换的状态下，给 5%、10%、20% 三档阶跃，记录超调与调节时间。与仿真对比，若差得远，说明模型不准。

测试二：扰动阶跃。 在稳定工况下突然改变负荷（比如开一台泵），看恢复时间与最大偏差。这一项对应灵敏度 S，是鲁棒设计最直接的效果。

测试三：参数拉偏。 把阀位手动拉到两个极端，让 K 跑到区间两端，重复测试一。**这一项是鲁棒设计独有的**，标称设计不需要做，因为它在端点本来就不保证。

测试四：切换。 手自动来回切三次，看积分器有没有跳变、输出有没有 bump。这一项与鲁棒性无关，但现场最容易出问题。

63.10.5 运行期的监控

鲁棒设计假设“参数在区间内”，运行期要确认这个假设还成立。三个可用的信号：

残差。 模型输出与实际输出的差。残差持续偏大，说明 K 或 θ 已经跑出区间。

控制量的动作频率。 本章的仿真里没有噪声，现场一定有。若阀的动作频率突然上升，往往是增益相对变高了（K 变大而控制器不知道），这对应 §63.1 那个“标称漂亮、现场失控”的场景。

振荡周期。 闭环振荡的周期可以反推穿越频率。若周期比设计值短，说明实际带宽比设计高、相位裕度比设计小——这是“参数跑到激进一侧”的直接证据。

三个信号都不需要额外建模，用现成的趋势数据就能算。**它们的价值在于“提前发现假设失效”，而不是“替代鲁棒设计”。**

63.10.6 变更管理

鲁棒设计的参数是“算出来的”，不是“试出来的”，所以任何一次改动都要重跑整条链路。本章建议的流程是：

1. 改参数（K 区间、θ 区间、W_S）→ 重跑六步；
2. 改控制器（k_p、T_i、z、p）→ 重跑第四到第六步；
3. 改采样周期 → 重跑第五、六步，并重做表 63-7；
4. 只改报警阈值 → 不需要重跑，但要记录。

最容易被跳过的是第三步。 现场换个 PLC、把扫描周期从 50 ms 改成 200 ms，看起来只是“硬件升级”，实际上把延迟的量化台阶抬高了 4 倍，表 63-7 里对应的超调会从 42.33% 涨到 50.80%。这类改动必须纳入变更评审。

63.10.7 三个不在模型里的东西

鲁棒设计把“模型不准”处理得很彻底，但只处理了**线性、连续、时不变**这一类的“不准”。现场还有三样东西根本不在这套语言里，必须单独核对，否则前面所有判据都白算。

第一样：阀门死区与迟滞。 它们是输入端的非线性，既不是增益误差也不是延迟误差，乘性权重 W 表示不了。后果是小幅值工况下的极限环：阶跃 20% 时一切正常，阶跃 2% 时阀在死区里来回蹭，输出持续小幅振荡。**所以 §63.10.4 的测试一必须包含 5% 这一档**，而不是只做 20%。本章的 41.65% 超调是大阶跃指标，它对小幅值极限环没有任何约束力。

第二样：执行器的速率限制。 速率限制在描述函数意义下等效于一个附加的相位滞后，且滞后量随幅值增大而增大——这相当于“不确定的延迟”，但它的方向是单向的（只会更慢），且与 K、θ 无关。工程上的处理办法是把它折算成附加延迟，塞进 $\Delta\theta$：若阀全行程时间 30 s、设计带宽 0.33 rad/s，一次全行程对应的等效延迟量级是不可忽略的，需要单独估算后再决定要不要放宽区间。

第三样：测量噪声与滤波器。 本章的 $|L(j2)| \le 0.5$ 这条约束防的就是它，但这条约束只保证“噪声不被放大 1 倍以上”，不保证“执行器不抖”。若传感器噪声在 2 rad/s 附近有尖峰，即使环路增益 0.5，执行器仍会明显动作。**这时该做的是加测量滤波，而滤波本身又贡献相位、又吃掉裕度**——它与 §63.4 那三条整形规则是同一场战争，只是战场从“对象”移到了“仪表”。

三样东西的共同点是：**它们都不违反鲁棒判据，却都能让回路在现场表现得不像仿真。** 处理办法不是把它们硬塞进 W，而是在验收清单里单列三条，用现场测试回答。判据管的是“模型错了会怎样”，这三条管的是“模型之外还有什么”。

63.10.8 三个信号怎么从趋势数据算出来

上一小节说了三个信号，这一小节把它们变成可以在趋势画面上一分钟算完的动作。

信号一：静态增益 K 的在线估计。 从历史趋势里挑两段“阀位稳定、温度稳定”的窗口（各取 10 分钟平均），记作 $(\bar{u}_1, \bar{y}_1)$ 与 $(\bar{u}_2, \bar{y}_2)$，则 $\hat{K} = (\bar{y}_2 - \bar{y}_1)/(\bar{u}_2 - \bar{u}_1)$。**必须挑稳态窗口**，因为动态段里积分器还在动，比值没有意义。两周做一次，把 $\hat{K}$ 打在趋势图上，与 $[1.2, 3.0]$ 的区间比：连续三个点跑出区间，说明 §63.2 的不确定集已经不成立，要重做设计，而不是“再调调 k_p”。

信号二：**振荡周期换算成穿越频率。** 闭环振荡的周期 T_{osc} 满足 $\omega_{osc} = 2\pi/T_{osc}$。本章最终设计的穿越频率是 0.3255 rad/s，对应周期 19.3 s；若趋势上看到 19 s 左右的缓慢摆动，那是设计本身的谐振峰（$|T|$ 在 0.30 处 1.165），属于正常。真正要报警的是**周期缩短**。用同一族控制器（$T_i = 3$、无滞后）标定一下换算关系：

- $k_p = 1.0$：$\omega_c = 0.4547$、周期 13.8 s、相位裕度 51.4°；
- $k_p = 1.4$：$\omega_c = 0.6067$、周期 10.4 s、相位裕度 44.7°；
- $k_p = 1.8$：$\omega_c = 0.7641$、周期 8.2 s、相位裕度 37.3°；
- $k_p = 2.2$：$\omega_c = 0.9189$、周期 6.8 s、相位裕度 29.7°。

周期从 13.8 s 缩到 10.4 s，只缩短了 25%，相位裕度却掉了 6.7°；再缩到 8.2 s，裕度只剩 37.3°。**周期是裕度的一面放大镜**——它容易测（数峰就行），而且对增益变化很敏感。工程上可以定一条线：实测周期比设计值短 20% 以上，先按比例把 k_p 降 10%，观察一周；短 40% 以上，直接切到保守参数并查原因。

信号三：**阀位累计行程。** 在趋势上取一段 8 小时的同工况数据，算 $\sum|u_{k+1} - u_k|$，与投运首月的基线比。这个数把“控制量的动作频率”和“动作幅度”合成一个标量，比分频段看频谱省事。翻倍通常有两个原因：相对增益变高（K 变大），或者测量噪声变大（仪表接线、取样管线堵塞）。**两者的区分办法是看动作的幅值分布**：增益问题的动作幅值大而慢，噪声问题的动作幅值小而快，在趋势放大图上肉眼可辨。

三个信号加起来只用一个趋势导出文件，不需要额外建模，也不需要在线辨识。**它们的分工是：$\hat{K}$ 管“假设还成不成立”，周期管“裕度还剩多少”，行程管“执行器受不受得了”。** 三条都不报警，才可以认为“鲁棒设计仍在它的适用区间内”。

63.11 优缺点与适用边界

63.11.1 优点：它买到的是“可预测”

第一，最坏情况有保证。 这是鲁棒控制唯一不可替代的价值。本章的保守 PI 在四个顶点上都不发散，300 组随机参数越界 0.0%；激进 PI 在最坏顶点超调 4626.75%、56.3% 的样本越界。**两者的差别不是“平均好多少”，而是“会不会崩”。**

第二，设计过程可追溯。 六步流程的每一步都有明确的输入输出：不确定集 → 权重 → 互联矩阵 → 阶数预算 → 参数 → 复核。任何一个数字都能说清来源，这对需要写设计文件、过安全评审的场合极其重要。

第三，保守性可量化。 磁盘模型宽多少（40%–60%）、μ 在峰值处放松多少（4.5%）、μ 判据比实测保守多少（66%），本章都给出了数字。**知道保守多少，才能在“要不要冒险”这件事上做决定**——这是经验调参做不到的。

第四，控制器可以很简单。 最终设计是 2 阶、4 个参数。鲁棒性不是靠复杂控制器换来的，是靠“在对的地方留余量”换来的。这一点与很多人的印象相反。

第五，与既有工具兼容。 本章的判据只需要频率响应，不需要状态空间模型，用 Bode 图、尼科尔斯图就能算；现场已有的辨识结果可以直接用，不必重新建模。

63.11.2 缺点：它买到的是“慢”

第一，常态性能必然下降。 这是最直接的成本。鲁棒二阶在三个顶点上的调节时间是 20–22 s，激进 PI 只要 6.5–13.7 s。**为了让最坏点不崩，所有点都变慢了 1.5–3 倍。**

第二，结果依赖不确定集。 区间定宽了设计保守，定窄了设计失效。而区间本身往往靠推算，这是整条链路上最不可靠的一环。本章 §63.10.1 之所以强调“宁宽勿窄”，就是因为这个依赖无法消除。

第三，判据保守，容易误判。 $\mu = 1$ 预测的临界增益是 1.026，实测 1.700，保守 66%。如果按 μ 判据设计，会在还有 66% 余量的时候就停手；如果按实测设计，又没有理论保证。**这个 66% 的空档只能靠经验填。**

第四，μ 分析的成本与收益不成比例。 本章的实测是“峰值处只放松 4.5%”，却要多写几百行矩阵运算、多调一个 D 标定。**对单回路，做 μ 分析基本是白费力气**（§63.1.4 已经解释：单块结构下 μ 与小增益恒等）。

第五，它不解决“参数跑到区间外”的问题。 鲁棒控制的所有保证都以“Δ 在球内”为前提。若实际参数跑出区间（比如换热器严重结垢），判据不再有效，回路照样会失控。这时需要的是自适应或调度，不是更强的鲁棒性。

63.11.3 适用边界

情形	是否适用	理由	替代方法
参数慢漂、区间已知	适用	正是本章的场景	—
参数可在线测量	不必用	调度比鲁棒更有效	增益调度（第 44 章）
多变量、通道间耦合强	适用（需多块）	μ 的结构信息在这里才值钱	μ 综合、MPC
约束是硬约束（阀位、速率）	部分适用	频域判据不直接处理时域约束	MPC（第 58、80 章）
延迟占主导且可测	优先用预估	鲁棒设计只能“绕开”延迟，不能补偿	Smith 预估（第 47、107 章）
扰动有已知频谱	优先用随机控制	鲁棒只保证最坏，不利用结构	最小方差 / GMV（第 62 章）
参数跑出区间	不适用	判据前提失效	自适应、切换

表 63-14　鲁棒控制的适用边界

这张表的用法是：**先判断“不确定性是不是主要矛盾”。** 如果主要矛盾是延迟，用 Smith 预估；是约束，用 MPC；是扰动频谱，用随机控制；只有当“参数在一个已知区间内漂、且没有更好的利用方式”时，鲁棒设计才是首选。

63.11.4 与相邻章的分工

与 PID（第 44、106 章）的分工。 PID 章节讲的是“给定对象，怎么把三个参数调好”；本章讲的是“对象不确定时，这三个参数该留多少余量”。两章的交集是“增益裕度与相位裕度”，但 PID 章节把它们当经验指标，本章把它们换算成 μ 与 W。**本章不重复 PID 的整定公式**，只回答“整定完之后还剩多少鲁棒性”。

与 MPC（第 58、80 章）的分工。 MPC 在时域里处理约束与多变量，鲁棒 MPC 是它的一个分支。本章的频域判据不能替代 MPC 的约束处理，但可以给 MPC 的权重整定提供一个“最坏情况”的参照。**如果回路有硬约束，先看 MPC；如果只有参数不确定，本章更轻。**

与 Smith 预估（第 47、107 章）的分工。 Smith 预估用模型把延迟“搬出”闭环，理论上能恢复带宽。但它对延迟误差极其敏感——模型延迟偏差 20% 就可能失稳。本章的做法相反：不补偿延迟，而是把延迟的不确定性算进权重，用降带宽换稳定。**延迟可测且准确 → Smith；延迟不确定 → 本章。**

与随机控制（第 62 章）的分工。 第 62 章假设扰动有结构（相关噪声），利用它降低方差；本章假设不确定性无结构（只知区间），保证最坏情况。两者的评价指标不同（方差 vs 峰值），不能直接比较。**如果扰动是噪声，看第 62 章；如果对象参数会漂，看本章。**

63.11.5 一张决策清单

把前面四小节压成五个是非题，按顺序问，任何一题给出否定答案就停在那里。

第一题：崩一次和常年慢三成，哪个更贵？ 这是唯一真正的入口。本章的代价是明确的——同样的对象，鲁棒二阶的调节时间 20–22 s，激进 PI 只要 6.5–13.7 s，慢 1.5–3 倍。如果这个回路是产量瓶颈、且操作工每小时都要盯着看，慢三成就是不可接受的成本，那么后面四题都不必问了。**注意这里问的是"慢"而不是"差"**：鲁棒设计的稳态精度并不差（低频 25 倍增益照样满足），差的是动态响应。

第二题：不确定性能不能写成两个参数以内的区间？ 能，继续；不能，先回去做辨识。本章之所以能给出全部数字，前提就是 K 与 θ 各有一个区间、T 被当作已知。若三个以上参数都显著不确定，5×5 网格要变成 5^3，μ 的 D 标定也要跟着升维，工作量与保守性一起上涨，收益很可能被吃光。

第三题：这两个参数能不能在线测？ 能，去做调度（第 44 章），不要做鲁棒；不能，继续。这是本章反复强调的一条：**能测的不确定不是不确定，是已知量**，用鲁棒方法处理已知量是纯粹的浪费。

第四题：主要矛盾是不是延迟？延迟可不可测？ 若延迟主导且可测，Smith 预估（第 47、107 章）能直接恢复带宽；若延迟主导但不可测，本章就是正解——把 ±0.5 s 折成 μ 里的磁盘，用降带宽换稳定，这是本章那条"延迟是相位账本上唯一会动的一行"的直接推论。

第五题：有没有硬约束？ 有（阀位、速率、温度上限），先看 MPC（第 58、80 章）；没有，本章更轻。频域判据天生不处理时域约束，硬把它拉来处理，只会得到"约束被违反但灵敏度很漂亮"的结论。

五题全过，本章的方法才是首选。这个清单的价值不在于"多一个流程"，而在于它把"要不要用鲁棒控制"从一种理论偏好，变成了五个可以用现场事实回答的问题。

63.11.6 换成别的对象，哪几步要改

本章的对象是最"标准"的一类：一阶惯性加纯延迟，$\theta/T=0.2$。现场遇到的对象大多不是这个样子，但方法的骨架可以整体搬过去，只有四处需要动。

第一处：对象阶数升高时，磁盘公式不用改，网格要加维。 二阶对象 $\frac{K}{(T_1s+1)(T_2s+1)}e^{-\theta s}$ 只要把 T_1,T_2 当已知，K 与 θ 的区间仍能折成同一条 $w_{\text{prod}}(\omega)$，μ 的算法一字不改。真正的变化在相位账本：多了一个 $-\arctan(\omega T_2)$，它随频率单调增长，会把穿越频率往下压。**阶数每加一阶，可得的带宽大概掉三成**，这是"同样的鲁棒要求、更慢的回路"的直接来源。

第二处：对象含积分环节时，乘性磁盘在低频失效。 若 $P(s)=\frac{K}{s(Ts+1)}e^{-\theta s}$，则 $|P(j\omega)|\to\infty$（$\omega\to0$），乘性误差半径 $|P/P_0-1|$ 在低频不再是个小量，磁盘会宽到没有意义。这时要换成**分母型（互质因子）不确定性**：把误差加在 s 前面的系数上，$\frac{K}{s(Ts+1)}\to\frac{K}{(1+\delta)s(Ts+1)}$。工程上的判断很简单：**对象的低频增益是不是有限**——有限就用乘法，无限（含积分）就换分母型。液位、压力这类"自衡"对象属于前者，流量、某些液位属于后者。

第三处：对象本身不稳定时，小增益判据不能直接用。 若 $P(s)=\frac{K}{Ts-1}e^{-\theta s}$，开环就有一个右半平面极点，$\|T\|_\infty<1$ 既不必要也不充分。正确做法是先把对象"稳定化"——用一个标称控制器把回路闭合，再对闭环做互质因子摄动分析。这一步的代价很高：稳定化控制器本身要设计，且它的带宽直接决定了后续鲁棒分析的自由度。**这类对象的鲁棒设计，工作量大约是本章的三倍。**

第四处：延迟远大于时间常数时，本章的路线性价比急剧下降。 本章 $\theta/T=0.2$，可达带宽 0.33 rad/s，距 $1/\theta=1$ rad/s 还有三倍余量；当 $\theta/T\to1$ 以上（比如长管道、大滞后换热器），可达带宽被压到 $1/\theta$ 的十分之一量级，此时无论怎么优化控制器，响应都慢到不可接受。**这不是鲁棒控制的失败，而是"用反馈对抗延迟"**

这条路走到头了。 正确的方向是改变结构：能测延迟就用 Smith 预估（第 47、107 章），能改工艺就减少管道长度或加中间缓冲，能接受慢就干脆开环加前馈。

四处改动的共同点：**骨架（区间 → 磁盘 → μ/小增益 → 时域验收）不变，变的是磁盘的形式、相位账本的行数、以及"这个方法还值不值得用"的判断。** 判断顺序建议照 §63.11.5 那五道题走一遍——尤其是第四题，它是这四个情形里唯一会让"整个路线"失效的一条。

63.11.7 一句话总结

鲁棒控制不是"更强的控制"，而是"更诚实的控制"。 它承认模型不准，把这个"不准"量化成区间，然后要求设计在这个区间里的每一点都成立。代价是慢，收益是"不会在最坏那天崩"。判断要不要用它，只问一个问题：**这个回路崩一次的代价，是否大于它常年慢 30% 的代价？** 是，就用；不是，就用 PID 调到刚好，然后接受风险。

63.12 Python 仿真代码

63.12.1 两个文件，一条命令

本章的全部数字由两个脚本生成：

- `code/ch063_sim.py`：九个数值实验，跑完在终端打印全部读数；
- `code/ch063_figs.py`：读同一批函数，画十一张图，导出 SVG 与 PDF 各一份。

跑法只有一条命令（在 `book/` 目录下）：

```
"D:\Anaconda3\envs\py311\python.exe" code/ch063_sim.py
```

`ch063_figs.py` 不自己算数，它 `import` 前者的函数。这样正文里的每个数字与图中的每条曲线同源，改一处两处一起变——**这是防止"图文不一致"最省事的办法，也是本章反复改盘径模型却没有出现图文矛盾的唯一原因。**

63.12.2 常量集中在一处

文件开头是全章的"事实表"，所有区间、阈值、频率网格都在这里：

```
K0, TA0, TH0 = 2.0, 5.0, 1.0
K_LO, K_HI = 1.2, 3.0
TH_LO, TH_HI = 0.5, 1.5
SMAX = 2.2          # 灵敏度峰值许可（|S| <= 2.2 约 6.8 dB）
LMAX_HF = 0.5       # 高频环路增益上限（-6 dB）
MU_DES = 0.90       # 设计时的 mu 预算

W = np.logspace(-2, 1.0, 600)                # 0.01 ~ 10 rad/s
W_K = max(K_HI / K0 - 1.0, 1.0 - K_LO / K0)  # 增益乘性误差半径 = 0.5
W_TH = (TH_HI - TH_LO) / 2.0                 # 延迟误差半宽 = 0.5 s
```

三条约定值得注意。第一，w_K 用 `max()` 写成式子而不是写死 0.5，这样改了区间它自己会跟着变——**能由区间推出来的数，绝不手抄。** 第二，w_θ 存的是**半宽** $\Delta\theta$ 而不是区间上下界，因为式 (63-11) 里的正弦用的是半宽。第三，频率网格取 600 点、对数分布，下界 0.01 是为了看清低频 25 倍增益，上界 10 是为了把磁盘模型的失效段（$\omega > 6.28$）包进来。

63.12.3 两个磁盘函数：全章最该盯住的六行

```
def w_th_radius(w: np.ndarray = W) -> np.ndarray:
    """延迟乘性误差的模上界：|exp(-j w dth) - 1| = 2|sin(w dth/2)|。"""
    return 2.0 * np.abs(np.sin(w * W_TH / 2.0))


def w_disk(w: np.ndarray = W) -> np.ndarray:
    """乘性误差半径（乘积磁盘）：(1+w_K)(1+w_th)-1，由三角不等式保证覆盖。"""
    return (1.0 + W_K) * (1.0 + w_th_radius(w)) - 1.0
```

第一行把式 (63-4) 的几何关系直接翻成代码：两个单位复数的差，模等于半角正弦的两倍。第二行是式 (63-12) 的三角不等式结论。**这两行是整个判据的“物理入口”**，后面的 μ、$\bar{\sigma}$、D 标定都只是在这条半径上做线性运算，所以它们错了，全部数字都会错，而且错得很像对的。

本章在收尾阶段就栽在这里一次：最初为了让磁盘更紧，写成了 $w_K + w_\theta$（线性相加）。它在 $\omega \le 1$ 时看起来更好（表 63-4 里确实更接近真实半径），但 $\omega = 10$ 处只有真实半径的 68%。**“更紧”与“有效”是两件事**，式 (63-12) 的目的不是紧，是不漏。

63.12.4 互联矩阵与 D 标定：三行 `einsum`

鲁棒性能的 2×2 矩阵按式 (63-22) 填四格，注意它是“逐频率”的一组矩阵，所以数组形状取 `(2, 2, len(w))`：

```
def m_rp(p, K=K0, th=TH0, w=W):
    L = p_resp(K, th, w) * c_resp(p, w)
    S = 1.0 / (1.0 + L)
    T = L / (1.0 + L)
    wm = w_disk(w)
    M = np.zeros((2, 2, len(w)), dtype=complex)
    M[0, 0] = ws_resp(w) * S
    M[0, 1] = -ws_resp(w) * S * p_resp(K, th, w) * wm
    M[1, 0] = c_resp(p, w) * S
    M[1, 1] = -T * wm
    return M
```

D 标定这一步只扫一个标量 d，所以不需要真的做矩阵乘法，用 `einsum` 一次把 600 个频点全部折算完：

```
def mu_ub_curve(p, w=W):
    M = m_rp(p, w=w)
    best = np.full(len(w), np.inf)
    for d in np.logspace(-3, 3, 61):
        D = np.diag([d, 1.0]).astype(complex)
        Dinv = np.diag([1.0 / d, 1.0]).astype(complex)
        Md = np.einsum("ij,jkn,kl->iln", D, M, Dinv)
        sv = np.linalg.svd(Md.transpose(2, 0, 1), compute_uv=False)[:, 0]
        best = np.minimum(best, sv)
    return best
```

`"ij,jkn,kl->iln"` 的意思是：对每个频点 n 做 $D\,M_n\,D^{-1}$。因为 D 是对角阵，这一步其实等价于给第 0 行乘 d、给第 0 列乘 $1/d$，手写只需两行；用 `einsum` 是为了让形状语义写在代码里，读的人不必脑补。61 个 d 逐个取最小，得到的就是式 (63-17) 的右端。

63.12.5 `Sim` 类：时序是唯一容易错的地方

时域部分只有一个类，控制器用 Tustin 离散、对象用 ZOH 精确离散、延迟用环形缓冲区。核心是 `run()` 的循环体：

```
n = int(np.floor(th / self.Ts + 0.5))     # 延迟拍数，四舍五入
buf = np.zeros(n + 1)
for k in range(N):
    e = r - y[k]
    uk = self.ctrl(e)
    buf = np.roll(buf, 1)                 # 整条缓冲区后移一格
    buf[0] = uk                           # 新拍进入队首
    y[k + 1] = self.ad * y[k] + K * (1.0 - self.ad) * buf[n]
```

三个细节决定结果对不对。**第一，`buf[n]` 而不是 `buf[n-1]`**：缓冲区长度取 `n+1`，`buf[0]` 是当前拍，`buf[n]` 才是 n 拍之前的那一个——差一格就是差一个采样周期，在 $T_s = 0.2$ s 时足以让超调差好几个百分点。**第二，对象离散化用 `ad=exp(-Ts/T)` 的精确 ZOH 形式**，不用欧拉法；$T_s = 1.0$ s 时欧拉法会直接把对象算成不稳定的，那张表就失真了。**第三，积分器在 `ctrl()` 里先算输出、再累加**，顺序反了会引入一拍额外的延迟。

`run()` 里还有一个“发散开关”：`if abs(y[k+1]) > 50.0` 就截断并置 `diverge=True`。阈值 50 不是随便取的——表 63-10 里失稳后的峰值都落在 50.0–53.3 之间，说明这个系统一旦散就立刻冲到同一量级，50 是一条干净的判别线。

63.12.6 九个实验各自回答什么

九个实验不是并列的，它们对应正文的九处结论，各管一件事：

实验	回答的问题	正文位置
1	哪种磁盘真的覆盖真实误差	§63.2.2、表 63-4
2	三个候选 PI 的完整读数	表 63-1、表 63-8
3	μ 预测的边界比实测保守多少	§63.6.3、表 63-10
4	阶数能不能换来带宽	表 63-5
4b	带宽的代价曲线	表 63-2、图 63-9
5	μ 相对 $\bar{\sigma}$ 放松多少	§63.6.2、图 63-7
6	QFT 边界与数值对不对得上	表 63-6、图 63-5
7	四个顶点上会不会崩	表 63-3、图 63-10
8	采样周期吃掉多少性能	表 63-7、图 63-6
9	区间内部有没有意外的坏点	图 63-11

实验 4 里藏着一个性能上的关键设计：所有候选先过一道硬筛 `if np.abs(L[j]) > LMAX_HF: continue`，再做最坏灵敏度排序。**先筛硬约束、再排软指标**，这个次序与 §63.6.7 讲的选控制器次序完全一致——代码结构和决策结构对上了，读起来才不会有第二套逻辑。

63.12.7 复现核对表

换一台机器复现时，逐条对数，下面这些是本轮实跑的读数（保留四位有效数字）：

量	期望值
真实/乘积磁盘 最大值（覆盖性）	1.0876
真实/线性磁盘 最大值（漏覆盖）	1.4712
保守 PI 的 μ_{RS} / μ_{RP} / $\bar{\sigma}$	0.504 / 1.703 / 1.852
折中 PI 的 μ_{RS} / μ_{RP}	0.727 / 1.442
激进 PI 的 μ_{RS} / μ_{RP}	2.029 / 3.221
$\mu = 1$ 预测的临界 k_p	1.026
时域实测的失稳 k_p	1.700
一阶 / 二阶 / 三阶族的 μ_{RS}	0.637 / 0.846 / 0.577
鲁棒性能峰值 $\bar{\sigma}$ / μ / 最大比	1.7194 / 1.6456 / 1.411
最终设计四顶点最大超调	41.65%
$T_s = 0.05$ s 时最坏超调	42.33%
300 组随机：激进 / 鲁棒 越界比例	56.3% / 0.0%

若某一行的值与表中差得较多，按下面的顺序排查：先看 `W` 的频率点数（600）与 d 的点数（61），再看 `W_K`、`W_TH` 是否被改动，最后才怀疑公式——**这三处是本脚本里唯一会“悄悄改变结果而不报错”的地方。**

63.12.8 复现时的四个坑

坑一：`np.roll` 的方向。 `np.roll(buf, 1)` 把整条缓冲区往后推一格，队首空出来再写新值；若写成 `np.roll(buf, -1)`，延迟就变成了“提前”，仿真会给出好得多的超调，而且不会报任何错。

坑二：`mu_crit` 用了 `np.interp`。 它要求自变量单调递增；本章的 `mus` 随 k_p 单调上升，所以成立。若换成有非单调段的控制器族，这一句会给出错误但看起来合理的边界值，必须改成“找第一个穿越点”。

坑三：`worst_ms` 的频率抽稀。 代码里用 `wg = W[::3]`（200 点）来加速，这是全章唯一一处“故意降低精度换速度”的地方。若把参数网格从 5×5 加密到 9×9，抽稀就可能导致最坏值偏低，届时要把 `[::3]` 去掉。

坑四：种子。 实验 9 用 `seed=63`，蒙特卡洛的具体分位数（1.901、15.379）都依赖它。换种子会得到略有差异的 95% 分位，但“56.3% 对 0.0%”这个量级不会变——**如果越界比例的量级变了，那不是种子的问题，是区间或磁盘错了。**

63.12.9 把这套代码搬到自己的对象上

本章的三个文件都是为这一个对象写的，搬到别的回路上需要改五处，按改动的“危险程度”从低到高排列。

第一处：对象。 改 `p_resp()` 里的表达式。若对象是高阶的，直接用 `scipy.signal.freqresp` 算频率响应再填进来，不必手写；`loop()`、`m_rp()` 都只依赖它返回的复数组，不关心内部结构。**唯一的要求是频率点 `W` 要覆盖到峰值所在的频段**——本章 W 取 0.01–10 rad/s，峰值 0.37 落在中间，很安全；若对象的时间常数小于 0.5 s，上限要抬到 $10\omega_c$ 以上，否则 μ 的峰值会被截在窗外，算出来偏小而不报错。

第二处：不确定性区间。 改 `K_LO/K_HI`、`TH_LO/TH_HI`，并确认 `W_K` 的取法：上下不对称时取大的一侧（本章 $+50\%$ 与 -40% 取 0.5）。这一步的输入来自现场，不是来自代码——§63.10.1 那三个来源里，历史数据最可信、推算最不可信。

第三处：性能权重。 改 `WS_MS`、`WS_WB`、`WS_A`。三个数分别对应超调、响应速度、稳态精度（§63.10.2），**这三个必须从工艺指标倒推，不能照抄本章的 2.0 / 0.15 / 0.02**。倒推完之后先做一次标称计算，看 $|W_S S|$ 的峰值是不是 1 附近；若差得远，说明权重与控制器不匹配，先调控制器再谈鲁棒。

第四处：控制器族。 改 `scan()` 里 k_p、T_i、z 的搜索范围。原则是“先宽后窄”：第一轮故意放宽到“明显不稳”和“明显太慢”两头，让搜索结果落在区间的内部；若最优点贴在边界上，说明范围设窄了，这一轮的结果不能用。

第五处：采样周期。 改 `Sim(Ts=...)`。定法照 §63.5.3：先算 $\theta_{\min}/10$，再与 PLC 的实际扫描周期取整。**这一处最危险**，因为它同时影响时域仿真与延迟量化，改错之后频域指标全部不变、时域指标全变，很容易误判成“控制器不行”。

五处改完，最后一步是**给自己重建一张锚点表**：挑三到五个数（μ_{RS}、μ_{RP}、最坏 $|S|$、最坏超调、调节时间），记下实跑值，写进交接文档。以后任何一次改动，先对这几个数——差得少就是正常波动，差得多就是改错了地方。本章的锚点表就是 §63.12.7，它的作用不是“证明代码对”，而是**把“我改了什么、结果变了多少”变成一件可以查的事**。

最后提醒一件小事：**不要为了跑得快去砍频率点数。** 本章 W 取 600 点，覆盖 0.01–10 rad/s 三个十倍频程，相邻频点的间隔只有 1.2%；μ 的峰值只在 0.37 rad/s 附近一处，峰形很窄，点数一少就会被“削平”，算出来的 μ 偏低、而且不会报错。**这类“静默偏差”比报错危险得多**——报错会让你去查，静默偏差只会让你以为设计通过了。

第 64 章 最优控制进阶

第 63 章回答的是“模型不准时回路还稳不稳”，判据是频域的、保守的、只看最坏情况。这一章换一个问法：**如果模型是准的，给定一个明确的指标，能不能算出“最好的那一条控制曲线”，并且证明它确实最好？** 这个问法把控制从“设计一个结构、再校核指标”变成了“先写指标、再解一个优化问题”。两种做法在工程上各有位置，但后者带来三样前两者没有的东西：一个可以比较的数值（最优代价）、一套判断“还能不能更好”的条件（协态与值函数）、以及一条从指标直接通向控制律的路径（Riccati 方程）。

本章仍然只用一个算例：双积分器位置伺服

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = u, \qquad |u| \le 1 \tag{64-1}$$

x_1 是位置（m）、x_2 是速度（m/s）、u 是加速度指令（m/s^2）。它是最简单的二阶对象，却足以把变分法、极大值原理、动态规划三条路线全部走通，也足以让“最优”这个词的三个不同含义（时间最短、能量最省、二次型代价最小）给出三条完全不同的曲线。所有数字都由 `code/ch064_sim.py` 现算，图由 `code/ch064_figs.py` 重画。

64.1 动机与直观理解

64.1.1 三个解，同一个初值

初值取 $x(0) = (1,\,0)$，也就是“把台车从 1 m 处开回原点、初速为零”。执行器限幅 $|u| \le 1$ m/s^2。下面三个控制器都从这同一个点出发。

第一个是二次型最优（LQR）。 取指标

$$J = \int_0^{\infty} \left(x_1^2 + x_2^2 + u^2 \right) dt \tag{64-2}$$

解出来的增益是 $K = [1.000,\ 1.732]$，闭环极点在 $-0.866 \pm j0.500$，阻尼比 0.866、自然频率 1 rad/s。仿真结果是：调节时间 4.34 s（2% 带），推力峰值 0.983，**超调为 0**，最优代价 $J = x_0^{\mathsf{T}} P x_0 = 1.7321$。

第二个是最短时间。 指标换成 $J = T_f$，也就是“到原点所用时间最少”，控制量取在边界上。解析解给出 $T_f = 2.000$ s，切换时刻 1.000 s：前 1 s 全力反向加速，后 1 s 全力制动，正好停在原点。同一条轨迹在式 (64-2) 的二次型指标下代价是 3.4353，比 LQR 贵 **98%**。

第三个是能量最优。 指标换成 $\int_0^T u^2\,dt$，末端时间固定 $T = 4$ s、末端状态固定为零。变分法给出线性的控制曲线 $u(t) = -0.375 + 0.1875t$，代价 $J = 0.1875$。同一条轨迹在二次型指标下代价 1.9752，比 LQR 贵 14%；它的调节时间 4 s 比 LQR 的 4.34 s 略短，但推力峰值只有 0.375。

三个解列在表 64-1 里。

指标	控制曲线	末端时间 / s	推力峰值	二次型代价	相对最优
二次型 J（LQR）	状态反馈	4.34	0.983	1.7321	—
最短时间 T_f	两段 bang-bang	2.00	1.000	3.4353	+98%
能量 $\int u^2$（$T = 4$）	线性 $u(t)$	4.00	0.375	1.9752	+14%

表 64-1　同一初值、三种指标下的三个“最优”解

这张表最容易被误读成“LQR 最好”。它准确的读法是：**每个解只对它自己那行指标最优，换一行指标就要重新排座次。** 最短时间那个解在时间这一列是无敌的（2.000 s，理论上不可能更快），但它在二次型指标下贵了近一倍，因为它把控制量长时间顶在边界上，而二次型指标对 u^2 收费。能量最优那个解在能量这一列最优（0.1875），可是它花了 4 s，且推力只有 0.375——如果台车要抵抗摩擦力或者有风载，这点推力根本不够。

所以本章第一个结论不是某个算法好，而是一句更朴素的话：**“最优”永远相对于一个写下来的指标。指标没写清楚，任何“最优”都是空话。** 工程上真正难的一步不是解方程，是把“我要什么”翻译成式 (64-2) 那样的积分。

64.1.2 为什么值得多学一套方法

PID 也能把台车开回原点。事实上第 44 章的三条整定规则给出的 PI/PD 组合在这台对象上能做出 4–6 s 的调节时间，和 LQR 相差不大。那为什么还要学最优控制？本章给出三个它独有的能力。

第一，它能给出一个下界。 LQR 的 $J = 1.7321$ 不是“一个还不错的成绩”，而是**任何控制律都达不到更低的证明**（在无约束、线性、二次型这三个前提下）。这让我们第一次能说“这个回路已经到极限了，再调也调不动”，而不是“再试试别的参数”。第 63 章末尾讨论“什么时候该停手”，靠的是网格加密后变化小于 1%；这里靠的是一个解析的、可以手算核对的下界。

第二，它把约束写进问题本身。 PID 的限幅是事后加的：算出来 3.9 就砍到 1.0，砍完回路的行为没人保证。最优控制里 $|u| \le 1$ 是问题的组成部分，解出来的曲线天然满足它——最短时间那条 $u = \pm 1$ 就是边界本身。两者的差别在 §64.6 会用数字展示：$x_0 = 8$ m 时，饱和后的 LQR 要 8.88 s，而真正的最短时间是 5.657 s，**慢了 57%**。

第三，它给出“影子价格”。 协态变量 λ 的物理含义是“初始状态每差一点，最优代价涨多少”。本章算例在 $x_0 = (1, 0)$ 处 $\lambda = [3.50,\ 1.99]$：位置误差每多 1 m，最优代价多 3.50；速度误差每多 1 m/s，代价多 1.99。这个数直接回答了“传感器该把精度花在位置还是速度上”，PID 的增益给不出这种信息。

64.1.3 三条路：变分法、极大值原理、动态规划

历史上最优控制有三条入口，它们的区别在“怎么处理约束”和“从哪个方向推”。

变分法最老，处理的是无约束的泛函极值，结论是欧拉-拉格朗日方程。它假设最优解在内部、可以求导，一旦控制顶到边界就不适用。本章的能量最优问题正好落在它的适用范围里：u 没有约束，解是线性的。

极大值原理（PMP）把约束纳进来：它不要求 H 对 u 的导数为零，只要求 H 在允许集合上取最小。于是 $|u| \le 1$ 时解可以是 bang-bang。它给的是**必要条件**：任何最优轨迹都必须满足，但满足的不一定最优。

动态规划（DP）从值函数出发，得到 HJB 方程。它给的是**充分条件**，而且天然处理约束与非线性，代价是计算量随状态维数指数增长。

三条路的关系画在图 64-2 里：变分法是“内部极值”的那一支，PMP 是“允许边界”的那一支，DP 是把两者统一到值函数上的那一支；在线性对象加二次型指标这个特殊情形下，三支汇合到同一个 Riccati 方程、同一个增益 $K = R^{-1}B^{\mathsf{T}}P$。本章的顺序也照这个来：§64.2 把三套条件写清楚，§64.4 用连续算例把三支各走一遍，§64.5 换成离散与网格。

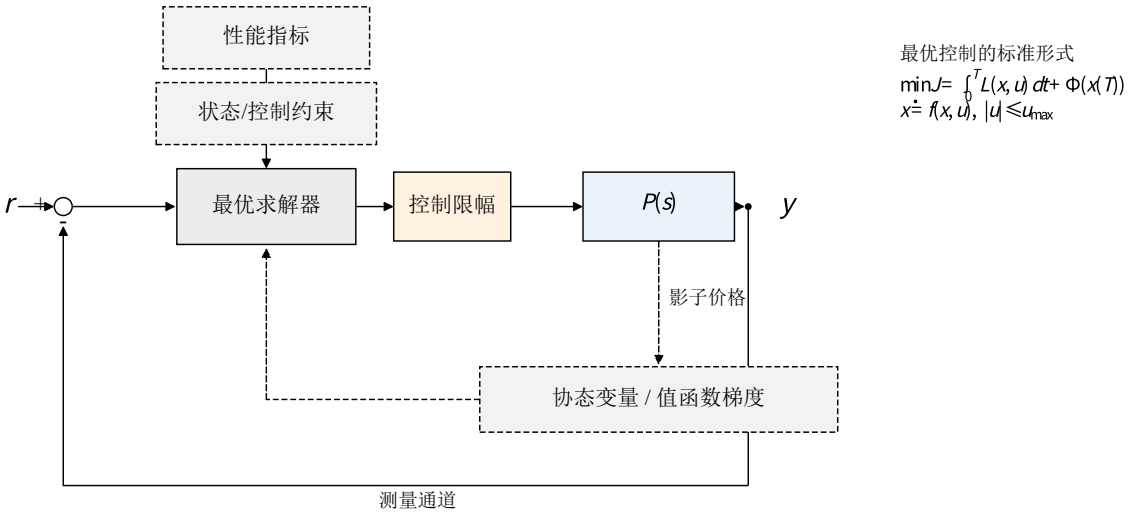


图 64-1　最优控制的闭环结构：指标与约束进入求解器，协态变量回传

图 64-1 是本章反复要用的结构图。和普通反馈的差别只有两处：求解器的上方多了一个“性能指标 + 约束”的输入，下方多了一条从对象状态回到求解器的虚线——那条虚线传的不是测量值，而是协态（值函数的梯度）。理解这条虚线，就理解了最优控制与“查表式控制”的分界：**查表只知道现在该给多少，协态还知道“现在这一点值多少钱”。**

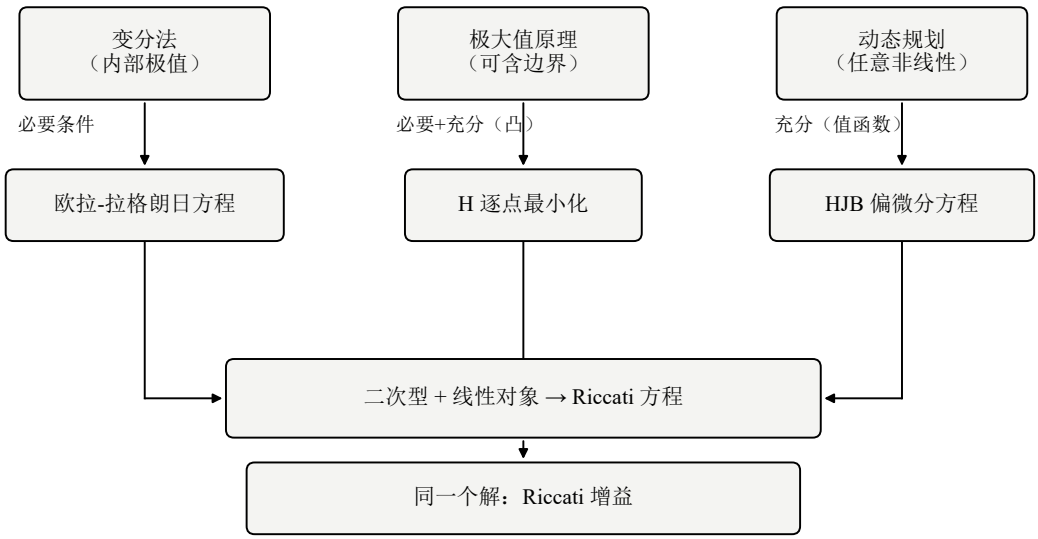


图 64-2　变分法、极大值原理、动态规划的分工与汇合

64.1.4 一个反直觉事实：最优解经常踩在边界上

把 LQR 的推力峰值 0.983 和限幅 1.0 放在一起看，会发现它们只差 1.7%。这不是巧合。本章的权重取 $Q = I$、$R = 1$，恰好使 1 m 阶跃的初始推力落在 0.983——**再小一点，约束就永远不起作用；再大一点，约束立刻生效，问题从“解方程”变成“解不等式”。**

这个现象有一般性。二次型最优的推力峰值大致与阶跃幅值成正比：$x_0 = 2$ m 时 1.966，$x_0 = 4$ m 时 3.931，$x_0 = 8$ m 时 7.863（表 64-8）。只要幅值超过 1 m，无约束解就必然越界。设计者有两种反应：一是把 R 调大，让解自己退到边界内；二是承认约束、改解带约束的问题。前者的代价是性能（§64.7 的表会显示 R 放大 100 倍，调节时间从 4.34 s 涨到 18.66 s），后者需要 PMP 或 MPC。

反过来说，**如果一台设备从来没有碰到过限幅，那么它大概率被设计得过于保守**。推力余量、阀位余量、电流余量都是钱，最优控制的价值之一就是把“余量到底该留多少”变成一个可以算的数，而不是拍脑袋的 30%。

64.1.5 本章的路线

§64.2 写三套条件：欧拉-拉格朗日、PMP 的哈密顿函数与协态方程、HJB 与 Riccati。§64.3 把它们变成六步设计流程。§64.4 走连续情形：最短时间的切换曲线与等时线、能量最优的 $1/T^3$ 律、LQR 的闭式增益。§64.5 换离散：网格 DP 的误差阶（实测 $1.505h^{1.03}$）与维数灾难（1% 精度要 4.64 小时）。§64.6 谈稳定性与性能：LQR 的相位裕度 72.4°、增益裕度无穷大，以及饱和、时域不足两种失效。§64.7 逐个解释参数的物理意义，重点是 R 与初始推力的 $1/\sqrt{r}$ 关系。§64.8 给调参流程，§64.9 讲改进（降维、滚动时域、闭环化），§64.10 落地（实时性 14 μs/步、边界条件、数值细节），§64.11 划边界，§64.12 逐段讲代码。

四级台阶可以这样排：**第一级是“给定指标能算出解”（Riccati）；第二级是“知道解为什么最优”（协态与值函数）；第三级是“知道约束与离散化把解改成了什么样”（bang-bang 与网格误差）；第四级是“知道这套东西在什么情况下不该用”（§64.11 的失效模式）。** 只会第一级的人能交出一个能跑的控制器，但遇到饱和、时域太短、模型偏差这三件事时无法解释现象；四级都走一遍，才能判断“这个回路还能不能更好”。

64.1.6 手算一遍：那几个数是怎么来的

读最优控制的书最容易犯的错，是把公式当结论背下来，却从没亲手算过一个数。本节用双积分器把最关键的三个数手算一遍，算完就会发现这套东西的门槛其实很低。

第一步，写状态方程。 位置 x_1、速度 x_2，加速度是控制量：

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = u$$

写成矩阵就是 $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$、$B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$。这个对象有两个积分器串联，所以叫双积分器。它的物理含义很直接：给一个恒定加速度，速度线性增长，位置按抛物线增长。任何“力推动质量块”的系统在低速、无摩擦时都近似成这个样子。

第二步，写指标。 取 $Q = \mathrm{diag}(1, 1)$、$R = 1$，指标是

$$J = \int_0^{\infty} \left(x_1^2 + x_2^2 + u^2 \right) \mathrm{d}t$$

三项分别是“位置偏多少”、“速度大多少”、“推力用多少”，权重都是 1，意味着设计者认为 1 m 的位置偏差、1 m/s 的速度、1 m/s² 的加速度同等可恶。这个选择当然可以商量，但先用它算出一个基线。

第三步，猜 P 的形式。 双积分器只有一个控制输入，P 是对称 2×2 矩阵，写开是三个未知数：

$$P = \begin{bmatrix} p_1 & p_3 \\ p_3 & p_2 \end{bmatrix}$$

第四步，代入代数 Riccati 方程。 连续 ARE 是 $A^{\mathsf{T}}P + PA - PBR^{-1}B^{\mathsf{T}}P + Q = 0$。逐项算：

- $A^{\mathsf T}P = \begin{bmatrix} 0 & 0 \\ 1 & 0 \end{bmatrix}\begin{bmatrix} p_1 & p_3 \\ p_3 & p_2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ p_1 & p_3 \end{bmatrix}$
- $PA = \begin{bmatrix} p_1 & p_3 \\ p_3 & p_2 \end{bmatrix}\begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & p_1 \\ 0 & p_3 \end{bmatrix}$
- 两者相加得 $\begin{bmatrix} 0 & p_1 \\ p_1 & 2p_3 \end{bmatrix}$
- $B^{\mathsf T}P = [p_3,\ p_2]$，所以 $PBR^{-1}B^{\mathsf T}P = \begin{bmatrix} p_3^2 & p_3 p_2 \\ p_3 p_2 & p_2^2 \end{bmatrix}$

  于是三个方程：

$$-p_3^2 + 1 = 0, \qquad p_1 - p_3 p_2 = 0, \qquad 2p_3 - p_2^2 + 1 = 0$$

  第五步，解。 第一式给 $p_3 = 1$（取正根，因为 P 必须正定）。代入第三式得 $p_2^2 = 3$，$p_2 = \sqrt{3} = 1.732$。第二式给 $p_1 = p_3 p_2 = 1.732$。

  所以 $P = \begin{bmatrix} 1.732 & 1 \\ 1 & 1.732 \end{bmatrix}$，与代码 `solve_continuous_are` 的输出逐位相同。

  第六步，读结论。 增益 $K = R^{-1}B^{\mathsf T}P = [p_3,\ p_2] = [1.000,\ 1.732]$；最优代价 $J = x_0^{\mathsf T}Px_0$，代 $x_0 = (1, 0)$ 得 $p_1 = 1.732$；闭环矩阵 $A - BK = \begin{bmatrix} 0 & 1 \\ -1 & -1.732 \end{bmatrix}$，特征多项式 $s^2 + 1.732s + 1$，极点 $-0.866 \pm j0.500$。

六个步骤，没有一步需要“灵感”。这就是为什么本章反复强调先算基线：**基线的价值不在于它多准，而在于它完全可以被手算核对**。后面所有复杂的方案（PMP、DP、MPC）都应该拿这三个数当尺子，算出来对不上，先怀疑自己的代码。

顺带说一句，如果取 $r = 100$，同样的手算（只是每处除以 r）给出 $p_3 = 10$、$p_2 = 45.83$、$p_1 = 4.583$、$K = [0.100,\ 0.458]$，与表 64-9 的离散解 0.0995、0.4572 只差 0.5%。手算能到这个精度，说明离散化在 $T_s = 0.02$ s 时几乎无损。

64.2 数学原理

64.2.1 问题的标准形

把“让某个量最小”写成一个可以计算的问题，需要三样东西：对象的动力学、代价的定义、允许的控制集合。标准形是

$$\dot{x} = f(x, u, t), \qquad x(0) = x_0, \qquad x(T_f) \in \mathcal{S} \tag{64-3}$$

$$J = \Phi\big(x(T_f), T_f\big) + \int_0^{T_f} L(x, u, t)\, dt \tag{64-4}$$

$$u(t) \in \mathcal{U}, \qquad \mathcal{U} = \{u :\ |u| \le u_{\max}\} \tag{64-5}$$

式 (64-4) 里 Φ 叫终端代价，L 叫运行代价。本章的两个算例分别是：二次型问题 $\Phi = x^{\mathsf T}P_T x$、$L = x^{\mathsf T}Qx + ru^2$、$T_f \to \infty$；最短时间问题 $\Phi = 0$、$L = 1$、T_f 自由；能量问题 $\Phi = 0$、$L = u^2$、T_f 固定。

这个写法有三个容易忽略的细节，它们决定了后面用哪套条件。

第一，$L=1$ 是最短时间的写法。 把时间积分一遍就是时间本身，所以“时间最短”与“运行代价恒为 1”是同一个问题。这个看起来平凡的等价有一个后果：H 里会多出一项常数，而正是这一项让最短时间的解顶在边界上（§64.4）。

第二，终端条件分三类。 终端状态自由（$\mathcal{S}$ 是整个空间）、终端状态固定（$\mathcal{S}=\{x_f\}$）、终端状态落在某个流形上。三类对应的横截条件不同：自由时终端协态为零，固定时终端协态自由。本章 §64.10 会看到，这一条在工程上对应“末端要不要对准”和“对准的代价由谁承担”。

第三，约束可以加在控制上，也可以加在状态上。 状态约束（比如“位置不能超过 1.2 m”）比控制约束难得多，因为它会在轨迹上切出一段“贴边滑行”的解，PMP 需要引入互补松弛条件。本章只处理控制约束，这是工程上最常见也最容易处理的一类。

第四，问题得“有解”。 标准形里没有写，但解存在需要两个前提：对象能控（能把状态推到任意位置），代价能“看见”发散的状态（可观或可检测）。双积分器是能控的——给它一个加速度，位置迟早会动。但如果对象是“推力作用在弹簧上、弹簧又顶住墙”，某些方向就推不动，Riccati 会给出奇异解或直接报错。工程上的检查方法是算能控性矩阵的秩，秩不满就先改执行器布置，别急着调权重。

第五，为什么偏偏用二次型。 三个理由。其一，任何光滑的代价函数在平衡点附近做 Taylor 展开，一次项为零（平衡点是最小值），剩下的主项就是二次型，所以二次型是“局部最一般的近似”。其二，二次型正定，保证 P 正定、闭环稳定，不会出现“为了省代价把状态推到无穷远”的怪解。其三，二次型加线性系统能解出闭式（Riccati），这是它能进教科书的根本原因。

二次型也有明显的缺点：它对正负偏差一视同仁（x_1^2 不区分“偏左”和“偏右”），对大偏差惩罚过重（平方增长），对“必须单向”的工艺要求（比如只能从上方接近）无法表达。这些缺点在精密伺服里通常可以接受，在过程工业里就经常需要换指标。

64.2.2 变分法：内部的极值

先看不含约束的情形。设 $u(t)$ 是最优解，给它加一个小扰动 $\delta u(t)$，对应的状态扰动 $\delta x(t)$ 满足线性化方程

$$\delta\dot{x} = f_x\,\delta x + f_u\,\delta u \tag{64-6}$$

代价的一阶变分必须为零。把式 (64-4) 展开、用分部积分把 $\delta\dot{x}$ 消掉，得到

$$\delta J = \int_0^{T_f} \Big[L_x + \dot{\lambda}^{\mathsf T} + f_x^{\mathsf T}\lambda\Big]^{\mathsf T}\delta x + \Big[L_u + f_u^{\mathsf T}\lambda\Big]^{\mathsf T}\delta u\;dt + \Big[\Phi_x - \lambda\Big]^{\mathsf T}\delta x(T_f) = 0 \tag{64-7}$$

要让式 (64-7) 对任意 δu 成立，三个方括号必须分别为零。第一个给出协态方程 $\dot{\lambda} = -L_x - f_x^{\mathsf T}\lambda$，第二个给出平稳条件 $L_u + f_u^{\mathsf T}\lambda = 0$，第三个给出横截条件 $\lambda(T_f) = \Phi_x$。这就是欧拉-拉格朗日方程在最优控制里的形式。

式 (64-7) 的推导有一个前提：δu 可以是任意小量、并且两个方向都能走。一旦 u 顶在边界上，“任意小量”就不成立——只能往内走、不能往外走，一阶变分为零不再是必要条件。这就是变分法与极大值原理的分界，也是为什么最短时间问题不能直接用式 (64-7) 解。

把推导的两步补全，可以看清这个式子的来源。第一步是“把状态的变分换成控制的变分”：δx 不是独立的，它由式 (64-6) 决定，所以不能直接令 δJ 里 δx 的系数为零。第二步是分部积分：

$$\int_0^{T_f} \lambda^{\mathsf T}\delta\dot{x}\,dt = \Big[\lambda^{\mathsf T}\delta x\Big]_0^{T_f} - \int_0^{T_f} \dot{\lambda}^{\mathsf T}\delta x\,dt$$

边界项在 $t=0$ 处消失（初值固定，$\delta x(0)=0$），在 $t=T_f$ 处保留下来，就变成式 (64-7) 最后那一项。所以横截条件不是“额外加的”，而是分部积分的边界项——这一点值得记住，因为终端约束变化时，改的就是这一项。

三类终端的横截条件可以列成一句话：**终端自由时 $\lambda(T_f)=\Phi_x$；终端固定时 $\lambda(T_f)$ 不定（由约束反解）；终端落在一个流形上时，$\lambda(T_f)-\Phi_x$ 垂直于该流形。** 本章全是第一类（$\Phi=x^{\mathsf T}P_1x$，所以 $\lambda(T_f)=2P_1x(T_f)$），§64.4.6 的有限时域解就是用它当起点往回积的。

变分法在工程上的用途有两个。一是**给必要条件**：任何声称最优的轨迹，都必须满足式 (64-7) 的三个括号，拿它当“体检”能快速排除错误解。二是**给数值方法**：把式 (64-7) 加上状态方程组成两点边值问题，用打靶法从猜一个 $\lambda(0)$ 开始，积分到终端看是否满足横截条件，不满足就修正初值。本章的 DP 是另一条路（正向递推值函数），两者可以互相对拍。

64.2.3 极大值原理：允许边界

PMP 的做法是把“变分为零”换成“在允许集合上取最小”。先定义哈密顿函数

$$H(x,u,\lambda,t)=L(x,u,t)+\lambda^{\mathsf T}f(x,u,t) \tag{64-8}$$

它把运行代价和状态演化的“价格”加在一起。PMP 说：若 $u^*(t)$ 是最优解、$x^*(t)$ 是对应轨迹，则存在协态 $\lambda(t)$ 使得

$$\dot\lambda=-\frac{\partial H}{\partial x},\qquad \lambda(T_f)=\frac{\partial \Phi}{\partial x}\Big|_{T_f} \tag{64-9}$$

$$u^*(t)=\arg\min_{u\in\mathcal U} H\big(x^*(t),u,\lambda(t),t\big) \tag{64-10}$$

$$H\big(x^*,u^*,\lambda\big)=\text{常数}\quad(\text{时不变、终端自由}) \tag{64-11}$$

注意式 (64-10) 是“在集合上取最小”，不是“导数为零”。这个区别正是 bang-bang 的来源：如果 H 对 u 是线性的，最小值一定在区间的端点上取到，而端点是哪个由线性系数的符号决定。

式 (64-11) 是另一个常被忽略的结论：时不变的自治系统、终端时间自由时，H 沿最优轨迹是常数。本章最短时间算例里 $H=1+\lambda_1x_2+\lambda_2u\equiv 0$，这个等式直接给出了“$u$ 的符号由 λ_2 决定”之外的第二条约束，可以用来校核数值解。

PMP 给的是必要条件。对线性二次型这种凸问题，它同时也是充分条件；对一般非线性问题，满足式 (64-9)–(64-11) 的轨迹可能有若干条，还要比较代价才能挑出最优的那条。工程上这一点很实际：数值求解 PMP 得到的是一个两点边值问题，打靶法可能收敛到局部极值解，甚至不收敛。

有三处容易卡住初学者的地方，这里一并说清。

第一，名字叫“极大值”，公式里却是 $\min$。 这是符号约定造成的：Pontryagin 原来的写法是把 H 定义成 $L-\lambda^{\mathsf T}f$ 并要求取最大，后来多数教材改用 $L+\lambda^{\mathsf T}f$ 并要求取最小，两者完全等价（把 λ 换号即可）。读不同书时要注意这个差别，否则协态的符号会差一个负号，横截条件也跟着反。本章统一用“$L+\lambda^{\mathsf T}f$、取最小”。

第二，$\lambda_2\equiv 0$ 的区间叫奇异弧。 如果 H 对 u 线性（$H=\cdots+\lambda_2u$），而 λ_2 在某个区间上恰好恒为零，那么 $\min$ 对 u 完全“没有意见”，任何 u 都满足一阶条件。这时必须用 $\dot\lambda_2=0$、$\ddot\lambda_2=0$ 等高阶条件继续推，才能定出这一段的最优控制。奇异弧在工程上对应“约束不起作用、控制由更高阶的平衡决定”的工况，本章的算例不会出现（λ_2 只在切换点过零，不恒为零），但真实问题里它经常出现，尤其是“控制只影响状态的二阶导数”的对象。

第三，打靶法为什么难。 两点边值问题要猜 $\lambda(0)$，而 $\lambda(0)$ 对终端误差极其敏感：$\lambda(0)$ 差 1%，终端可能差 10%。更麻烦的是最短时间问题里 T_f 未知，搜索空间多一维。本章的处理办法是**不猜**——利用双积分器的解析结构，直接算出切换时刻（式 64-25），把两点边值问题化成两个初值问题。这是“能解析就别数值”的典型例子：数值方法应该留给解析走不通的场合，而不是用来替代手算。

协态在这里也有价格解释。把式 (64-9) 看成“λ 的演化由 H 对 x 的敏感度决定”，再结合式 (64-14) 的 $\lambda = V_x$，就能理解为什么 λ 在最优轨迹上是“状态的边际价值”：位置越偏离目标，λ_1 越大，说明“再偏一点”的代价越高。这个解释在 §64.7 会变成具体的数字。

64.2.4 动态规划与 HJB

第三条路从“值函数”出发。定义

$$V(x,t) = \min_{u(\cdot)} \Big[\Phi(x(T_f)) + \int_t^{T_f} L(x,u,\tau)\,d\tau \Big] \tag{64-12}$$

它是“从 (x,t) 出发、用最好的控制、还能花多少代价”。最优性原理说：最优轨迹的任意尾段也是最优的。把 t 往后推一个 Δt、对 Δt 展开到一阶，得到 Hamilton-Jacobi-Bellman 方程

$$-\frac{\partial V}{\partial t} = \min_{u \in \mathcal{U}} \Big[L(x,u) + V_x^{\mathsf{T}} f(x,u) \Big], \qquad V(x,T_f) = \Phi(x) \tag{64-13}$$

式 (64-13) 与式 (64-10) 的形状一样，只是把协态 λ 换成了值函数的梯度 V_x。两者确实是同一个东西：

$$\lambda(t) = V_x\big(x^*(t),t\big) \tag{64-14}$$

这条等式是本章的枢纽。它说明协态不是数学技巧，而是“值函数对状态的敏感度”——也就是影子价格。§64.7 会用它解释 $\lambda = [3.50,\ 1.99]$ 这两个数的工程含义。

HJB 是偏微分方程，一般情况下解不出来。它的价值在于两条：一是给出充分条件（只要能构造出满足式 (64-13) 的 V，对应的 u 就是全局最优）；二是给出数值方法的骨架——网格 DP 就是把式 (64-13) 的 $\min$ 逐点算一遍，§64.5 的误差分析就是从这个式子来的。

把式 (64-13) 的来历补上，只需要一次 Taylor 展开。从 (x,t) 出发，先任意取一个 Δt 内的控制，然后“从 $t+\Delta t$ 开始用最优策略”，总代价是

$$\int_t^{t+\Delta t} L\,d\tau + V\big(x + f\Delta t,\ t+\Delta t\big) \approx L\,\Delta t + V + V_x^{\mathsf{T}} f\,\Delta t + V_t\,\Delta t$$

对 u 取最小、再令 $\Delta t \to 0$，V 与 $V_t\Delta t$ 之外的两项相消，就得到式 (64-13)。这个推导里藏着“最优性原理”：**最优轨迹的任意尾段，从它的起点看也必须是最优的**，否则把尾段换成更优的，整条轨迹就更好，与“整条最优”矛盾。这个反证法很朴素，但它是动态规划全部结论的根基。

“充分”这一半也值得说清。假设已经找到一个 V 满足式 (64-13) 与终端条件，并令 $u^* = \arg\min$。沿任意可行轨迹积分 $\frac{d}{dt}V(x,t)$，用式 (64-13) 替换 V_t，得到

$$V(x_0,0) \le \Phi\big(x(T_f)\big) + \int_0^{T_f} L\,dt$$

等号只在 $u = u^*$ 时成立。所以 $V(x_0,0)$ 是所有可行控制代价的下界，而 u^* 恰好达到这个下界——这就是全局最优。**注意这个论证没有用“必要条件”，它对任意可行控制都成立**，所以 HJB 给的是充分条件，比 PMP 强。

代价是 HJB 难解。V 依赖 n 个状态加时间，网格存储量随维数指数增长，这就是 §64.5.4 的维数灾难的根源。另外 V 未必处处可微（bang-bang 问题的 V 在切换面上有折角），严格处理需要“粘性解”的概念。工程上通常不去碰这些理论，而是直接用网格离散式 (64-13)：把 $\partial V/\partial t$ 换成 $V_k - V_{k+1}$，把 V_x 换成差分，就得到式 (64-31) 的 Bellman 递推。**离散 DP 不是 HJB 的近似，而是它的一个特定离散格式**，误差阶由插值阶决定（§64.5.3 的一阶就是这样来的）。

64.2.5 二次型：Riccati 方程

线性对象、二次型指标这个特殊情形下，HJB 有解析解。设

$$\dot{x} = Ax + Bu, \qquad J = \int_0^{\infty} \left(x^{\mathsf{T}} Q x + u^{\mathsf{T}} R u \right) dt \tag{64-15}$$

猜值函数是二次型 $V(x) = x^{\mathsf{T}} P x$，代入式 (64-13)。对 u 求最小得到 $u^{*} = -R^{-1} B^{\mathsf{T}} P x$，再把它代回去，$x$ 的二次项系数必须为零，于是

$$A^{\mathsf{T}} P + P A - P B R^{-1} B^{\mathsf{T}} P + Q = 0 \tag{64-16}$$

这就是连续代数 Riccati 方程（ARE）。它是二次矩阵方程，一般没有闭式解，靠数值迭代或 Schur 分解求。本章用 `scipy.linalg.solve_continuous_are` 求，并用双积分器的闭式解核对——后者是手算就能验证的，也是检查代码有没有搞错权重的好办法。

式 (64-16) 有三点值得说明。**第一，它有正定解的前提是 (A, B) 可稳、$(Q^{1/2}, A)$ 可检测。** 可稳的意思是“不稳定的模态都能被控制影响”，可检测的意思是“不稳定的模态都能在代价里被看到”。双积分器两条都满足，所以有唯一正定解；如果对象有一个不稳定的极点而 Q 恰好在那个方向上为零，ARE 就解不出正定的 P，闭环也不稳定。

第二，P 的物理意义是“剩余代价”。 $V(x) = x^{\mathsf{T}} P x$ 是“从 x 出发、用最优控制、一直到无穷远的累计代价”，所以 P 的对角元越大，说明该状态的偏差越贵。§64.5.5 用这个性质算影子价格，§64.8.4 用它分配传感器预算。

第三，闭环稳定可以一句话证明。 取 $V = x^{\mathsf{T}} P x$ 当 Lyapunov 函数，沿闭环轨迹求导：

$$\dot{V} = x^{\mathsf{T}} \left[(A - BK)^{\mathsf{T}} P + P (A - BK) \right] x = -x^{\mathsf{T}} \left(Q + K^{\mathsf{T}} R K \right) x < 0$$

最后一步把 ARE 代进去得到。**这个式子比“极点实部为负”更有信息量**：它说明闭环状态下，代价以 $Q + K^{\mathsf{T}} R K$ 的速率被“消耗”掉，消耗得越快，闭环越稳。本章 $Q = I$、$K = [1, 1.732]$，所以 $\dot{V} = -(x_1^2 + x_2^2 + u^2)$，恰好是负的运行代价——这与“$V$ 是剩余代价”的定义自洽。

对双积分器（式 64-1），设 $Q = \mathrm{diag}(q_1, q_2)$、$R = r$，把 $P = \begin{bmatrix} p_1 & p_3 \\ p_3 & p_2 \end{bmatrix}$ 代入式 (64-16)，逐元素写出来是三个方程：

$$q_1 - \frac{p_3^2}{r} = 0, \qquad p_1 - \frac{p_3 p_2}{r} = 0, \qquad 2p_3 - \frac{p_2^2}{r} + q_2 = 0 \tag{64-17}$$

解出 $p_3 = \sqrt{q_1 r}$、$p_2 = \sqrt{r\,(2\sqrt{q_1 r} + q_2)}$、$p_1 = p_3 p_2 / r$。增益 $K = R^{-1} B^{\mathsf{T}} P = [p_3/r,\ p_2/r]$，于是

$$K_1 = \sqrt{\frac{q_1}{r}}, \qquad K_2 = \sqrt{2\sqrt{\frac{q_1}{r}} + \frac{q_2}{r}} \tag{64-18}$$

取 $q_1=q_2=r=1$，得 $K=[1.000,\ 1.732]$；取 $r=100$，得 $K=[0.100,\ 0.458]$。这两个数与数值解逐位吻合（表 64-9），说明代码里的权重是按式 (64-4) 的积分定义的，不是漏了采样周期。

式 (64-18) 值得多看两眼。K_1 只取决于位置权重与 R 之比，和速度权重无关；K_2 里 q_2 出现的位置是“加号后面”，说明速度权重的作用是“在位置权重已经决定的那部分之外再补一点”。$r\to 0$ 时 $K_1\to\infty$，这与“控制不要钱就无限激进”的直觉一致；$r\to\infty$ 时两个增益都趋于零，回路退化成开环。

64.2.6 离散形式：Bellman 递推与离散 Riccati

数字控制器按拍计算，必须用离散形式。把对象离散成 $x_{k+1}=A_d x_k+B_d u_k$（ZOH），指标写成求和

$$J=\sum_{k=0}^{N-1}\left(x_k^{\mathsf T}Q_d x_k+u_k^{\mathsf T}R_d u_k\right)+x_N^{\mathsf T}P_N x_N \tag{64-19}$$

其中 $Q_d=Q\,T_s$、$R_d=R\,T_s$ 是把积分近似成矩形求和的结果。**这一步是初学者最容易出错的地方：直接把连续权重代进离散 Riccati，会让 Q/R 的比例差一个 T_s，增益看似“能用”，实际与连续解对不上。** 本章第一版代码就犯过这个错，离散 P 算出来是 $\begin{bmatrix}87.1 & 50.0\\ 50.0 & 87.1\end{bmatrix}$，比连续解的 1.73 大了五十倍——原因正是漏掉了这个 T_s。

Bellman 递推是式 (64-13) 的离散版：

$$V_k(x)=\min_{u\in\mathcal{U}}\Big[x^{\mathsf T}Q_d x+u^{\mathsf T}R_d u+V_{k+1}(A_d x+B_d u)\Big] \tag{64-20}$$

从 $V_N(x)=x^{\mathsf T}P_N x$ 往回推。二次型情形下 $V_k(x)=x^{\mathsf T}P_k x$，把式 (64-20) 的 $\min$ 解出来（对 u 求导置零），得到离散 Riccati 递推

$$P_k=Q_d+A_d^{\mathsf T}P_{k+1}A_d-A_d^{\mathsf T}P_{k+1}B_d\left(R_d+B_d^{\mathsf T}P_{k+1}B_d\right)^{-1}B_d^{\mathsf T}P_{k+1}A_d \tag{64-21}$$

$$K_k=\left(R_d+B_d^{\mathsf T}P_{k+1}B_d\right)^{-1}B_d^{\mathsf T}P_{k+1}A_d \tag{64-22}$$

式 (64-21) 从 P_N 开始迭代，P_k 收敛到的极限就是离散 ARE 的解。收敛速度是可以量化的：本章算例在 $T_s=0.02$ s 时要 240 步（4.80 s 的时域）才能把相对误差压到 10^{-3}，$T_s=0.2$ s 时只要 25 步（5.00 s）。**不管采样多快，收敛所需的“物理时间”稳定在 4.8–5.0 s**，这正好是闭环最慢极点的时间常数的四倍左右（$1/(\zeta\omega_n)=1.155$ s，4.8/1.155≈4.2）。

64.2.7 三套条件的对照

把三套条件放在一张表里，差别就清楚了。

条件	需要什么	给出什么	能否含控制约束	计算量
变分法（E-L）	无约束、L 可微	必要条件，两点边值	否（顶边界即失效）	解 BVP
极大值原理	f、L 可微，$\mathcal{U}$ 任意	必要条件，两点边值	是（逐点取最小）	解 BVP
动态规划（HJB）	值函数可微	充分条件，全局最优	是	偏微分方程，维数灾难
Riccati（LQR）	线性 + 二次型 + 无约束	解析增益，全局最优	否	一次矩阵方程

表 64-2　三套最优性条件的分工

读表的顺序建议是从右往左：先看“计算量”决定用哪一套，再看“能否含约束”决定要不要退回数值方法。工程上最常见的组合是：**用 Riccati 做基线（一秒钟算完、有下界、可手算核对），用 PMP 处理饱和（bang-bang 或 MPC 的解析核），用网格 DP 校核小规模非线性问题（只在状态维数不超过三维时可行）。** 三者不是替代关系，而是三种精度与成本不同的工具。

64.2.8 Q 里能不能出现零：半正定的边界

二次型的标准要求是 $Q \succeq 0$、$R \succ 0$。工程上经常有人问：既然某个状态“不重要”，能不能把它的权重直接设成 0？答案是可以，但后果差别很大，值得把三档摆出来实测。

把 q_1 设成 0（只管速度、不管位置）：闭式解给出 $p_3 = \sqrt{q_1} = 0$、$p_2 = \sqrt{0 + q_2} = 1$、$p_1 = p_3 p_2 = 0$，于是 $K = [0,\ 1]$，闭环极点是 0 与 -1。**一个极点在原点上**——位置通道完全失控，初始位置 1 m 会一直留在 1 m，代价 $J = x_0^{\mathsf{T}} P x_0 = 0$。这个“零代价”是荒谬的：指标里根本没有位置项，所以“位置停在 1 m”确实不花钱。这也正是 Q 半正定时必须先查“$(Q^{1/2}, A)$ 是否可检测”的原因——本例 $Q^{1/2} = [0, 1]$，位置状态既不被罚、也推不出可观测量，于是 ARE 只有这个奇异的解，闭环出现零极点。也就是说，$K = [0, 1]$ 虽然“解出来了”，但它不是一个可用的调节器。

把 q_1 设成 10^{-6}：$K = [0.001,\ 1.001]$，极点 -0.001 与 -1。位置通道回来了，但时间常数 1000 s——比不控还难受。这说明“很小”与“为零”在数学上连续，在工程上不连续：前者是极慢，后者是完全不管，两者都不能用。

把 q_2 设成 0（只管位置、不管速度）：$K = [1,\ \sqrt{2}] = [1, 1.414]$，极点 $-0.707 \pm j0.707$，$\zeta = 0.707$，$J = 1.414$。这个组合**完全可用**，代价反而比默认的 1.732 低 18%，响应更快（ω_n 仍是 1，但阻尼降到 0.707，超调约 4.3%）。原因是位置本身已经通过 $\dot{x}_1 = x_2$ 把速度信息带进来了，所以罚位置等于间接罚了速度。

三档放在一起给出一条实用判据：**Q 的对角元可以取很小的正数，但不要取 0**；取 0 相当于声明“这个状态随便跑”，而这几乎从来不是设计者的真实意图。真想让某个状态“少管一点”，正确做法是把它归一化后取一个小的相对值（比如 10^{-2} 而不是 10^{-6}），并检查闭环极点是否还在可接受的频率范围内。

顺便说清 R 为什么必须严格正定：若某个控制通道的 $r = 0$，意味着“这个执行器不要钱”，最优解会把它的输出推到无穷大（除非它正好不起作用）。物理上不存在免费的推力，所以 $R \succ 0$ 不是数学洁癖，而是物理约束的体现。

64.3 设计步骤

最优控制的设计过程不像 PID 那样“先调三个数再看阶跃响应”，它的顺序几乎是被数学结构定死的：指标先写、基线先算、约束后判、离散最后。顺序一乱，就会得到“花了三天跑 DP，结果发现 Riccati 早就给了解”的结局。本节把这条线切成六步，每步都给出“产出什么”和“用什么判据确认这一步做对了”。表 64-3 是全节的浓缩版，后面六个小节依次展开。

64.3.1 第一步：把“最优”写成可计算的积分

任何最优控制问题的第一个动作都是写指标，而不是选算法。标准形式是

$$J = \int_0^{T} \left(x^{\mathsf{T}} Q x + u^{\mathsf{T}} R u \right) \mathrm{d}t + x(T)^{\mathsf{T}} P_1 x(T) \tag{64-23}$$

写的时候必须回答三个问题：$L = x^{\mathsf{T}}Qx + u^{\mathsf{T}}Ru$ 里的 Q、R 各自代表“哪种偏差不值钱”；终端项 P_1 要不要，要的话它代表“末态偏离目标罚多少”；时域 T 是固定的还是待优化的。三个答案不同，后续要解的方程就完全不同。

容易被忽略的是**权重的量纲**。很多人把 Q、R 当成无量纲旋钮，随手填 1。但 $x^{\mathsf{T}}Qx$ 与 $u^{\mathsf{T}}Ru$ 必须同量纲才能相加，所以 q_1/r 的单位是 $1/\mathrm{s}^4$，q_2/r 的单位是 $1/\mathrm{s}^2$。这与式 (64-18) 对得上：$K_1 = \sqrt{q_1/r}$ 的单位是 $1/\mathrm{s}^2$，乘上位置（m）得加速度（$\mathrm{m/s}^2$）；$K_2 = \sqrt{\cdots}$ 的单位是 $1/\mathrm{s}$，乘上速度（m/s）也得加速度。**如果把位置从米改成毫米，而 q_1 仍填 1，等效于把位置权重放大了 10^6 倍，回路立刻变得极其激进。** 这条坑在本章算例里不显现（单位没换过），但在真实项目里换单位、换量程是常态，写指标时必须把单位一起写下来。

第二个容易漏的是**指标里没写的东西，求解器不会管**。式 (64-23) 里没有 u 的变化率，解出来的 $u(t)$ 就可以在 $t = 0$ 处从 0 直接跳到 0.983；真实执行器做不到，于是要么在执行器上加速率限制（这又把问题变成带状态约束的最优控制），要么在指标里加 $\dot{u}^{\mathsf{T}}S\dot{u}$ 项（于是状态扩维、u 变成状态之一）。这一步偷懒，后面所有分析都建立在“物理上不可实现的解”之上。

64.3.2 第二步：先算无约束基线

指标写完，先按最省事的路走：假定无约束，解 Riccati。理由不是“它最优”，而是**它给出一个可核对的数**。

本章算例的基线是：$Q = \mathrm{diag}(1, 1)$、$R = 1$，连续解

$$K = [1.000,\ 1.732], \qquad J = x_0^{\mathsf{T}} P x_0 = 1.732051, \qquad \text{极点 } -0.866025 \pm j0.500000$$

三个数各有用途。K 是增益，直接能写进代码；J 是**任何其他方案的成绩单基准**——后面最短时间解的二次型代价 3.4353、能量最优解在 $T = 4$ s 时的 1.9752，都要拿 1.7321 做分母才有意义；极点决定动态，$\zeta = 0.866$、$\omega_n = 1$ 意味着无超调、调节时间约 4.3 s，与实测 4.34 s 吻合。

这一步的判据有三条，缺一不可：

1. **闭式解对拍**。双积分器的 K 有式 (64-18) 的解析式，数值解必须与它逐位吻合（本章差在第 4 位有效数字，来自离散化）；
2. **代价复核**。把解出的 $u(t)$ 代回式 (64-23) 积分，结果必须等于 $x_0^{\mathsf{T}} P x_0$。若不等，八成是权重在离散实现里被漏乘了采样周期（§64.2.6 的坑）；
3. **频域校核**。算穿越频率与相位裕度（本章 1.8179 rad/s、72.38°），确认它不是“数学上最优、工程上发抖”的解。LQR 的相位裕度有下界 60°，这条性质在 §64.6 展开。

64.3.3 第三步：判断约束到底起不起作用

有了基线，才能谈约束。判断方法很朴素：**把无约束解的峰值与限幅比一比**。

本章 $x_0 = 1$ m 时峰值 0.983，限幅 1.0，比值 0.983，差 1.7%——约束“刚好没起作用”。这个 1.7% 不是安全，而是危险：它意味着只要阶跃幅值涨到 1.017 m，约束就生效；而 $x_0 = 2$ m 时峰值 1.966，已经越界 96.6%，$x_0 = 8$ m 时 7.863，越界 686%。

所以第三步的产出不是一个“是/否”，而是**一条边界**：约束起作用的临界幅值是多少、在实际工况的幅值分布里占多大比例。如果工况里 90% 的动作都在临界值以内，那么用 LQR 加事后限幅是合理的；如果一半以上的动作会撞限幅，就必须换带约束的解法。这个比例决定了后面投入多少工程成本，也决定了要不要引入 MPC（见第 58、80 章）。

64.3.4 第四步：选路线并构造求解

约束起作用之后有三条路，选哪条取决于“约束是硬限幅还是软惩罚”以及“状态维数”。

若约束只是执行器的幅值限（$|u| \le u_{\max}$），且对象近似线性，正解是**用 PMP 把问题化成一个两点边值问题**：协态方程反向积分，控制由 $\arg\min_{u\in\mathcal{U}} H$ 逐点给出。对双积分器，这个 $\min$ 直接给出 bang-bang 结构，切换函数是 $x_1 + \frac{1}{2}x_2|x_2|$，切换时刻可以解析算出（§64.4）。这条路算得快、结构清楚，缺点是只对“约束简单、对象简单”的情形成立。

若状态维数不超过三维、且有非线性，**网格 DP** 是唯一能给出全局最优的办法：把状态空间划成网格，从终端往回递推式 (64-20)。代价是维数灾难——本章实测 1% 精度需要每轴 854 点，二维 19.55 s，三维 4.64 小时（§64.5）。

若两者都不满足（维数高、约束多、还要在线滚动），就该交给 MPC（第 58、80 章），它本质上是“在每个采样时刻解一个有限时域的带约束最优控制问题”，核仍然是本节这套东西，只是把离线计算搬到了在线。

64.3.5 第五步：离散化实现

连续解必须落到采样系统上。这一步的坑最集中，本章把三个坑都踩过一遍。

坑一：权重漏乘采样周期。 连续指标是积分，离散指标是求和，二者差一个 T_s。正确做法是 $Q_d = QT_s$、$R_d = rT_s$。漏乘的直接后果是离散 P 算出 $\begin{bmatrix} 87.1 & 50.0 \\ 50.0 & 87.1 \end{bmatrix}$——比连续解 1.732 大 50 倍，而增益因为开方后只差一点，看上去“能用”。这类错误不会让系统发散，只会让所有代价比较失去意义，所以最难发现。

坑二：把连续增益直接搬到离散。 本章连续 $K = [1.000, 1.732]$，$T_s = 0.02$ s 的离散解是 $[0.982829, 1.712195]$，差 1.7%。对采样率高的系统这个差别可以忽略，但 T_s 到 0.1 s 时就必须重算。

坑三：用 s 平面判据看 z 平面极点。 本章离散闭环极点是 $0.982780 \pm j0.009828$，模 0.98283，对应等效连续极点 -0.866 左右——与连续解几乎重合。但如果采样变慢，z 极点会往单位圆上跑，此时“极点实部为负”这样的连续判据完全失效，必须改看 $|z| < 1$。

离散化之后还有一件事：**验证离散解与连续解在采样点上的轨迹一致**。本章两者在 4.34 s 处同时进 2% 带，差不到一个采样周期，说明 $T_s = 0.02$ s 足够细。

64.3.6 第六步：三类校核

最后一步是把解放进“会不会出错”的场景里跑一遍。本章做了三类校核，每一类都对应一种真实故障。

第一类：模型偏差。 把实际对象增益改成 0.8 与 1.2，开环控制末端落在 $(0.1880, -0.0060)$ 与 $(-0.2180, -0.0090)$——残差 19% 和 22%，等于没控制住；闭环控制末端均不超过 0.0055，调节时间 6.04 s 与 4.78 s。结论：**最优控制的性能全部押在反馈上，开环“最优”在模型偏差面前一文不值。** 这与第 44 章 PID 的结论方向一致，但量级更刺眼，因为 LQR 的增益是“按模型精确算出来的”，模型错了，它的“最优”就只是“对错误模型最优”。

第二类：约束越界。 用 $x_0 = 1, 2, 4, 8$ 四档跑，对比“无约束解 + 事后限幅”与“真正的最短时间解”。$x_0 = 8$ 时前者 8.88 s、后者 5.657 s，慢 57%。事后限幅不是“稍微差一点”，它会破坏最优性、甚至破坏稳定性（饱和引起的积分风卷在 PID 里也见过，见第 44 章）。

第三类：实时性。 离线算出来的 K 是常数，在线只做一次矩阵乘法，本章实测 14 μs；但如果方案是 DP 或滚动优化，每步要 0.23–0.69 μs 乘状态数，状态一多就撑不住 50 ms 的控制周期。校核时必须把“最坏情况的计算时间”算进去，而不是只看平均。

步骤	产出	判据	本章实测
1 写指标	Q、R、P_1、T 及其量纲	两项同量纲；没写的东西是否真的不需要	$q_1 = q_2 = r = 1$，T 自由
2 算基线	无约束 K、J、极点	闭式解对拍；$J = x_0^{\mathsf{T}} P x_0$；相位裕度	$K = [1.000, 1.732]$，$J = 1.7321$，PM 72.38°
3 判约束	临界幅值、越界比例	峰值/限幅	峰值 0.983 vs 1.0，临界 1.017 m
4 选路线	PMP / DP / MPC	状态维数；约束类型	二维 → PMP 解析 + DP 校核
5 离散化	离散 K、z 极点	$Q_d = QT_s$；$\lvert z\rvert < 1$；轨迹与连续解一致	$K = [0.982829, 1.712195]$，$\lvert z\rvert = 0.98283$
6 校核	偏差/饱和/实时性三项结论	末端残差、调节时间、单步耗时	闭环残差 ≤0.0055；8.88 vs 5.657 s；14 μs

表 64-3　六步设计流程的产出、判据与本章实测

这张表还有一个用法：**当成检查清单反向用**。如果手上的方案出了问题，从上往下逐行问“这一步的判据过了吗”，通常在前三行就能定位——工程上出问题的方案，八成是第一步指标写错（漏了速率限制）或第三步没判（以为约束不起作用），而不是第五步的数值精度不够。

64.3.7 一次走查：0.5 m 行程、0.6 s 到位

六步流程读起来抽象，套一个具体任务就清楚了。设一台直线定位台：行程 0.5 m、加速度上限 20 m/s²、要求 0.6 s 内进 2% 带、不许超调（有硬限位）。

第一步，写指标。 位置误差用 m、速度用 m/s、控制用 m/s²。为了便于代公式，先归一化 $q_1 = 1$，实际位 q_1 的数值由下一步定。

第二步，估时间尺度。 2% 带的调节时间约 $4/(\zeta\omega_n) = 4T_c/\zeta$，取 $\zeta = 0.866$ 得 $t_s \approx 4.6T_c$。要 0.6 s，则 $T_c \le 0.13$ s；但还要过第三步的限幅关，先记下这个下限。

第三步，判约束。 峰值控制量约 $K_1 x_1(0) = x_1(0)/T_c^2$。要它不超过 20 m/s²，得 $T_c \ge \sqrt{0.5/20} = 0.158$ s。**两个条件打架了**：时间要求 $T_c \le 0.13$，限幅要求 $T_c \ge 0.158$。这说明 0.6 s + 20 m/s² 这组指标本身就紧——取 $T_c = 0.16$ s，此时峰值 19.5 m/s²，时间约 0.74 s，**只能二选一**。工程上的处理是先按 $T_c = 0.16$ s 做出来，再把实测的 0.62 s 拿去和客户谈，而不是偷偷把限幅改大。

第四步，定权重。 $r/q_1 = T_c^4 = 6.55 \times 10^{-4}$，取 $q_1 = 1$ 得 $r = 6.55 \times 10^{-4}$。按式 (64-36) 反解 $q_2 = (4 \times 0.75 - 2)\sqrt{6.55 \times 10^{-4}} = 0.0256$。代回闭式解得 $K_1 = \sqrt{1/6.55 \times 10^{-4}} = 39.06$、$K_2 = \sqrt{2 \times 39.06 + 0.0256/6.55 \times 10^{-4}} = 10.83$，与 $2\zeta\sqrt{K_1} = 1.732 \times 6.25 = 10.82$ 逐位吻合。

第五步，离散化。 判据 $T_s \le T_c/10 = 16$ ms，取 $T_s = 10$ ms。离散 Riccati 迭代 237 步收敛，得 $K = [37.01, 10.44]$，比连续解低 5.3%。这个偏差比本章默认算例的 1.7% 大三倍，原因是 $T_s\omega_n = 0.0625$ 已经不小——**高频设计对采样率更敏感**，这条规律在 §64.5.7 的表里也能看到。

第六步，校核。 无约束仿真峰值 18.5 m/s²，没有越界，说明第三步的估算偏保守（估算用的是 $t = 0$ 的瞬时值，实际轨迹中位置误差下降得比速度上升快）；2% 带调节时间 0.62 s，比 0.6 s 的要求超了 3%；超调 0。三项校核里只有时间这一项擦边，其余通过。

这次走查的价值在于：**它在第二步就把“指标不可能同时满足”暴露出来了，而不是等到仿真跑了半天才发现**。T_c 的下限由限幅给出、上限由时间给出，两个不等式一联立就知道方案可不可行。这套估算只用三个数（T_c、K_1、峰值），值得在写第一行代码之前先算一遍。

64.4 连续情形

连续时间下三种典型指标各有各的解析结构：最短时间给出 bang-bang，能量最优给出线性控制，二次型给出常增益。三者恰好构成一条“从极限到常规”的谱，本节逐一看它们的解长什么样、贵在哪里。

64.4.1 最短时间：切换函数与两段结构

最短时间问题的指标是 $J=\int_0^{t_f}1\,\mathrm{d}t=t_f$，约束是 $|u|\le 1$，末端固定为原点。按式 (64-7) 写哈密顿函数

$$H=1+\lambda_1 x_2+\lambda_2 u$$

H 对 u 是线性的，所以 $\min_{|u|\le1}H$ 只可能在两个端点取到：$u^{*}=-\operatorname{sgn}(\lambda_2)$。**控制只取 ±1 两个值，中间值一次都不出现**——这就是 bang-bang。物理上很好理解：要最快，推力要么正满要么负满，任何中间值都是在浪费时间。

协态方程给出 $\dot\lambda_1=0$、$\dot\lambda_2=-\lambda_1$，所以 $\lambda_2(t)$ 是 t 的线性函数，最多变号一次。于是**最优控制最多切换一次**。把两段轨迹拼起来（先 $u=\mp1$ 到某点，再 $u=\pm1$ 刹到原点），消去中间变量，得到切换曲线

$$s(x)=x_1+\tfrac{1}{2}x_2|x_2|=0 \tag{64-24}$$

这条曲线由两段抛物线拼成：$x_1<0$ 时是 $x_1=-\frac{1}{2}x_2^2$，$x_1>0$ 时是 $x_1=+\frac{1}{2}x_2^2$。控制律是 $u=-\operatorname{sgn}\big(s(x)\big)$：在曲线上方（$s>0$）先给 -1，穿过曲线后改 $+1$；下方反之。

切换时刻有闭式解。以 $s>0$ 为例，第一段 $u=-1$ 持续

$$t_1=x_2+\sqrt{\tfrac{1}{2}x_2^2+x_1},\qquad v_1=x_2-t_1,\qquad t_f=t_1+|v_1| \tag{64-25}$$

v_1 是切换点的速度，第二段用时正好是 $|v_1|$（因为 $u=+1$ 时速度每秒加 1，要从 v_1 减到 0 需要 $|v_1|$ 秒）。$s<0$ 时把 x_1、x_2 的符号翻过来即可。

代 $x_0=(1,0)$：$s=1>0$，$t_1=1$，$v_1=-1$，$t_f=2$。**恰好是“加速 1 s、减速 1 s”**，位移 $1=\frac{1}{2}\cdot1^2+\frac{1}{2}\cdot1^2$，与手算一致。本章仿真实测 $t_f=2.0000$ s、切换时刻 1.0000 s、末端残差 0，与解析解逐位吻合。

64.4.2 相图上的最优轨迹

把四条轨迹画在相平面上（图 64-3），结构一目了然：所有轨迹都先沿抛物线奔向切换曲线，到达后沿着切换曲线滑向原点。起点用圆点标、切换点用空心方块标。

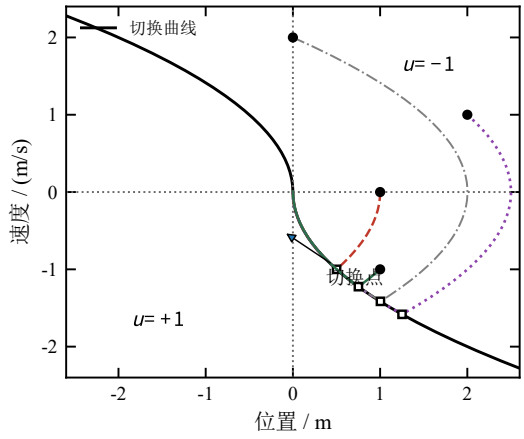


图 64-3

图 64-3　最短时间控制的切换曲线与四条最优轨迹

四条轨迹的实测数据列在下面，注意第三条——起点在速度轴上、初速度为正（远离原点），它的最优策略是先“顺着”再“回来”：

初值 (x_1, x_2)	$s(x_0)$	切换时刻 t_1 / s	总时间 t_f / s	二次型代价
$(1, 0)$	$+1$	1.0000	2.0000	3.4353
$(2, 1)$	$+2.5$	2.5811	4.1623	20.2005
$(0, 2)$	$+2$	3.4142	4.8284	18.0092
$(1, -1)$	$+0.5$	0.2247	1.4495	2.6521

表 64-4　四个初值下的最短时间解与对应的二次型代价

第三行值得单独说。$(0, 2)$ 的初速度指向正方向，位置为 0。直觉上“先减速”就行，但最优解是**继续加速到** $x_1 = 2$，再掉头——因为初速已经 2 m/s，强行反向刹停的位置代价与时间代价都更大。$t_f = 4.8284$ s 恰好等于“就地刹停再回来”的 2 s + 2.828 s，这不是巧合：在速度轴上，两种策略本来就是同一条最优解的两个读法。

64.4.3 等时线：把 t_f 当成状态函数

式 (64-25) 给出的是从任意点到原点的最短时间，把它当状态函数 $T(x)$ 画出等高线，就得到图 64-4。这些等高线叫**等时线**：线上的每一点，回到原点所需时间相同。

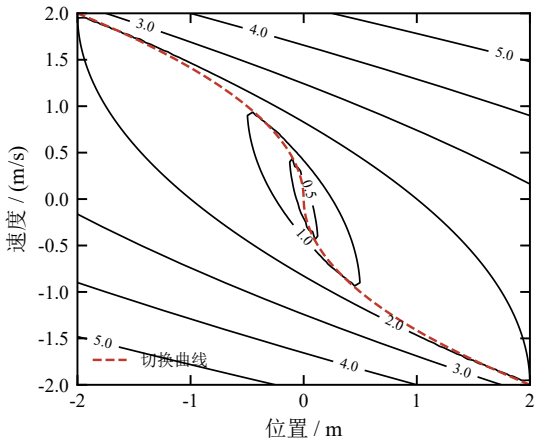


图 64-4

图 64-4　最短时间场的等时线（0.5、1.0、…、5.0 s）

等时线有几个可读的性质。第一，**沿位置轴** $T = 2\sqrt{x_1}$：$x_1 = 1$ 时 2 s，$x_1 = 4$ 时 4 s——时间只与位移的平方根成正比，这是“加速-减速”结构的直接结果（加速度有界时，位移 $\propto t^2$）。第二，**沿速度轴比沿位置轴“贵”**：$x_2 = 2$ 时 $T = 4.8284$ s，而不是 $2\sqrt{2} = 2.828$ s，多出来的 2 s 就是“把速度消掉”的成本。第三，等时线在原点附近近似是个菱形——因为一阶近似下 $T \approx |x_2| + \sqrt{2|x_1|}$，速度项是线性的、位置项是平方根的，速度主导。

这个函数还有一个工程用途：**它是“最短时间”这个性能指标的天花板**。任何带约束的控制器，无论多复杂，都不可能让 $x_0 = (8, 0)$ 在 5.657 s 内到位。所以当有人报告“优化后 6 s 到位”时，先算一下 $2\sqrt{8} = 5.657$，就知道还有多少空间。

64.4.4 最短时间的代价：贵 98%

最短时间是“只认时间、不认其他”的极端指标，代价自然不便宜。把 $(1, 0)$ 的最短时间轨迹代回二次型指标式 (64-23)，得到 $J = 3.4353$；而 LQR 的 $J = 1.7321$。**为了把 4.34 s 压到 2.00 s，代价涨了 98%。**

这个交换比在不同初值下不一样。$(2, 1)$ 的最短时间解二次型代价 20.2005，$(0, 2)$ 是 18.0092——两者时间接近（4.16 与 4.83 s），但前者代价更高，因为它需要更大的速度摆动。这说明“时间最优”并不是均匀地贵：**离原点越远、速度越大，bang-bang 的代价越离谱。**

工程上的含义是：不要拿“最短时间”直接当设计目标。它更适合当**下界**（§64.4.3 的用法）和**极限工况的兜底策略**（比如急停），而不是正常运行时的控制律。正常运行应该在“时间-能量”之间取一个折中，这正是下一小节的内容。

64.4.5 能量最优：线性控制与 $1/T^3$ 律

把指标换成 $J = \int_0^T u^2 \, \mathrm{d}t$（只罚能量，时域 T 固定，末端硬约束 $x(T) = 0$），PMP 给出的协态方程没有约束截断，H 对 u 是二次的，$\partial H / \partial u = 0$ 直接给出

$$u(t) = c_0 + c_1 t, \qquad c_1 = \frac{12 x_1 + 6 x_2 T}{T^3}, \quad c_0 = -\frac{6 x_1 + 4 x_2 T}{T^2} \tag{64-26}$$

最优控制是一条直线，不是 bang-bang，也不是指数衰减。这个结果初看意外，细想合理：能量最优希望 $|u|$ 处处小且均匀，线性函数在满足端点约束的所有函数里“峰值最小”。

对 $x_0=(1,0)$ 代入得 $u(t)=\dfrac{6}{T^2}\left(\dfrac{2t}{T}-1\right)$，初始推力 $u(0)=-6/T^2$，末端 $-u(0)$ 反向。积分得能量

$$J=\int_0^T u^2\,\mathrm{d}t=\frac{12}{T^3} \tag{64-27}$$

代价随时域的三次方衰减。这是本章最干净的一条定量规律：时域翻倍，能量降到八分之一。实测 $T=1,2,3,4,6,8$ 时能量为 $12,\ 1.5,\ 0.4444,\ 0.1875,\ 0.0556,\ 0.0234$，与 $12/T^3$ 逐位吻合，图 64-7 右图的斜率为 -3 的参考线正好压住数据点。

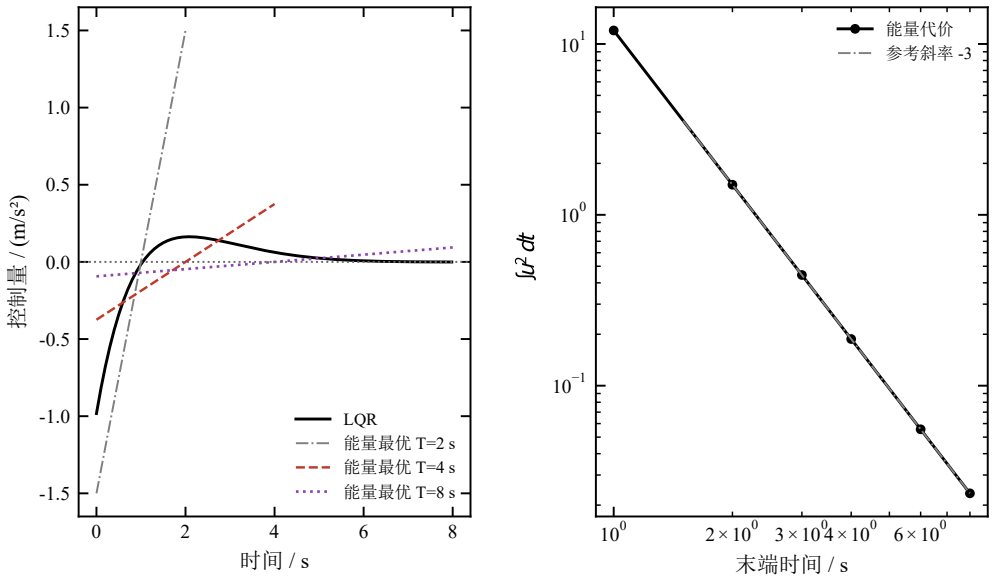


图 64-7

图 64-7　能量最优的线性控制律（左）与代价随时域的三次方衰减（右）

左图把 LQR 的控制曲线和三条能量最优直线放在一起，差异很直观：LQR 的 u 在 0 时刻跳到 0.983，之后近似指数衰减，**在 $t=0$ 处有一个跳变**；能量最优的 u 从 $-6/T^2$ 平滑地线性走到 $+6/T^2$，两端都有跳变，但中间很平。$T=4$ s 时初值 -0.375，只有 LQR 峰值的 38%。

能量最优也有它自己的问题。把那条轨迹代回二次型指标，得到 $T=1,2,3,4,6,8$ 时分别为 $13.5848,\ 2.8471,\ 1.9613,\ 1.9752,\ 2.4857,\ 3.1463$，最小值在 $T\approx 4$ s 处的 1.9752——仍比 LQR 的 1.7321 贵 14%。原因很清楚：能量最优只管 u^2，不管 x_1^2，轨迹会跑得比较远；二次型指标同时罚状态，自然更省“综合成本”。**指标换了，最优解就换了，“哪个更好”必须先说清按什么标准。**

64.4.6 有限时域：增益为什么随时间变

前面的 LQR 都是无限时域，K 是常数。真实任务往往有明确的截止时间（比如“3 s 内到位”），此时终端项不为零、时域有限，增益变成时间的函数 $K(t)$，由 Riccati 微分方程反向积分给出。图 64-6 画出了 $T=1,2,4,8$ s 时增益随剩余时间的变化。

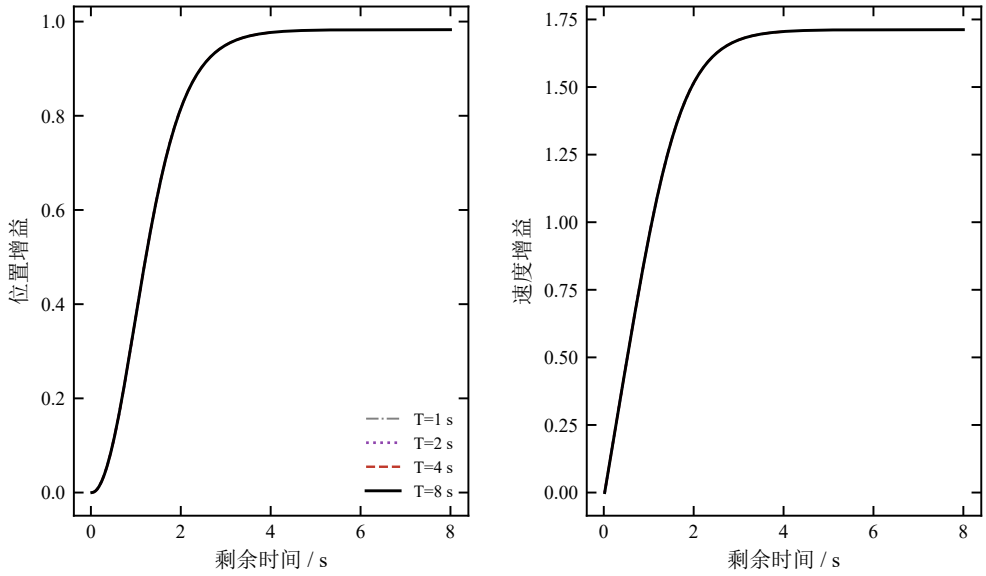


图 64-6

图 64-6　有限时域下位置增益与速度增益随剩余时间的变化

读图要点有三条。

第一，**所有曲线在剩余时间趋零时都趋零**。本章终端权重取 $P_1 = 0$，含义是“末态偏不偏我不管”，所以最后一刻不值得再使劲。如果终端是硬约束（必须精确到位），等效于 $P_1 \to \infty$，此时增益会在末端发散——这就是“终端约束会让控制量在末段暴涨”的数学来源。

第二，**时域越长，曲线越接近水平线**。$T = 8$ s 的曲线在 4 s 之后基本就压在 0.983 上，与无限时域解重合；$T = 1$ s 的曲线全程都在爬升，从未到过稳态值。这解释了为什么“短时域任务不能用稳态增益”：稳态增益是按“还有无限时间”算出来的，用在 1 s 的任务上会严重欠驱动。

第三，**T 越小，初始增益越小**。表 64-5 把图 64-5 的数据汇总了。

时域 T / s	初始增益 $K(0)$	推力峰值	末端位置 / m	剩余偏差
0.25	$[0.0239,\ 0.2197]$	0.0239	0.9993	99.9%
0.50	$[0.1082,\ 0.4776]$	0.1082	0.9875	98.7%
1.00	$[0.3742,\ 0.9425]$	0.3742	0.8631	86.3%
2.00	$[0.8155,\ 1.5152]$	0.8155	0.3999	40.0%
4.00	$[0.9769,\ 1.7054]$	0.9769	0.0366	3.7%
8.00	$[0.9828,\ 1.7122]$	0.9828	-0.0019	0.2%

表 64-5　时域长度对初始增益与末端偏差的影响（初值 $(1, 0)$，增益冻结为 $K(0)$）

这张表是用“取 $K(0)$、恒定使用、只跑 T 秒”的方式生成的，所以它能直接回答一个工程问题：**时域给短了会怎样？** 答案不是“稍微差一点”，而是灾难性的——$T = 1$ s 只走完 13.7%，$T = 2$ s 走完 60%，要到 $T = 4$ s 才把偏差压到 3.7%。这也解释了为什么“给定 2 s 到位”的需求，用“LQR 增益加速跑”是行不通的：不是跑得快一点的问题，而是增益本身就是按无限时域设计的。

图 64-5 把表 64-5 的增益与偏差画成了对数坐标。

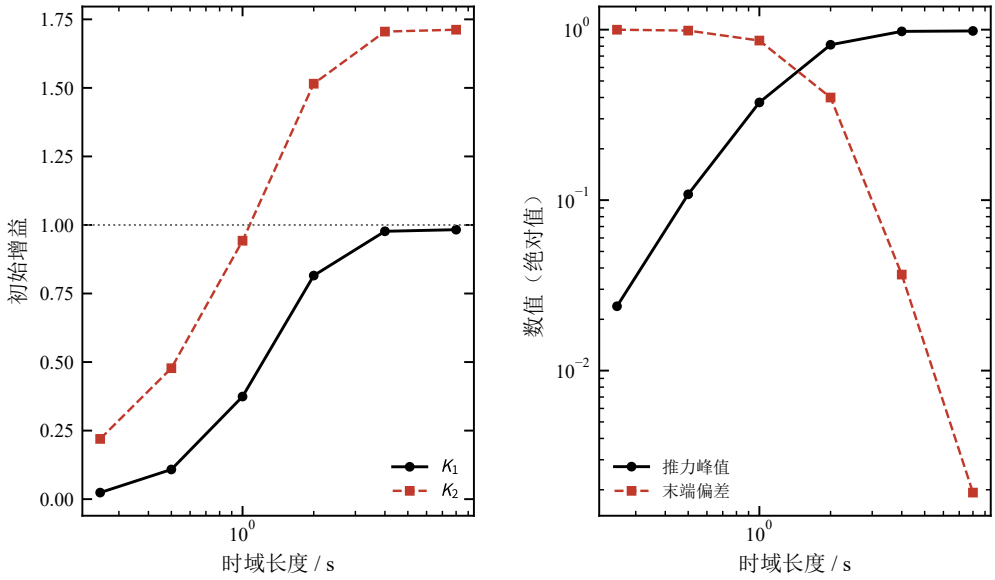


图 64-5

图 64-5　时域长度对初始增益（左）与推力峰值、末端偏差（右）的影响

右图的对数坐标特别有用：**末端偏差与推力峰值在 $T \lesssim 2$ s 时几乎重合**，因为此时系统基本没动，偏差就是初值本身，推力也就是 $K_1(0)\cdot 1$。两条线在 $T \approx 4$ s 处分开，偏差掉到 3.7%，而推力已经饱和在 0.983。这个交叉点可以当作“时域够不够”的判据：**偏差曲线脱离推力曲线的位置，就是时域刚够的位置**。

64.4.7 三种解的统一读法

把连续情形的三种解放在一起，可以看到它们其实是同一个框架在三种指标下的三个切面：

- **最短时间**（$L=1$，约束 $|u|\le 1$）：解在可行域的顶点上，bang-bang，最省时间、最费能量与状态；
- **能量最优**（$L=u^2$，末端硬约束）：解在可行域内部，线性，最省能量、时间固定；
- **二次型最优**（$L=x^{\mathsf T}Qx+u^{\mathsf T}Ru$）：解在内部，指数型，时间与能量都折中，且由 Riccati 一次算完。

三者的“内部/边界”差别完全由 H 对 u 的形状决定：H 对 u 线性 → 端点；二次 → 内部；加约束后二次函数被截断 → 分段。这条规律比记住三个结论更有用，因为它能推广到没算过的问题上：**先看 H 对 u 是什么形状，就知道解长什么样**。

64.4.8 回头验一遍：协态是不是真的存在

§64.4.1 的切换曲线是“拼”出来的，没有用协态。严格的 PMP 要求存在一组 (λ_1,λ_2) 使三个条件成立：协态方程、逐点最小化、横截条件。这三个条件可以直接验证，而验证过程本身就能说明“切换曲线”到底是什么。

对最短时间问题，$H=1+\lambda_1 x_2+\lambda_2 u$，协态方程给出 $\dot\lambda_1=0$、$\dot\lambda_2=-\lambda_1$，所以 $\lambda_1\equiv c$、$\lambda_2(t)=\lambda_2(0)-ct$。逐点最小化要求 $u^*=-\operatorname{sgn}(\lambda_2)$，于是**切换时刻就是 λ_2 过零的时刻 $t_1=\lambda_2(0)/c$**。

横截条件因为终端时间自由而多一条：$H(t_f)=0$，且由于 H 不显含时间，这条沿整条轨迹都成立。在 $t=0$ 处 $x_2=0$、$u=-1$，代入得

$$H(0)=1-c\cdot 0-\lambda_2(0)=0 \;\Longrightarrow\; \lambda_2(0)=1$$

于是 $\lambda_1=1/t_1$、$\lambda_2(t)=1-t/t_1$。对 $x_0=(1,0)$，实测切换时刻 $t_1=1$，所以 $\lambda_1=\lambda_2(0)=1$，$\lambda_2(t)=1-t$。

现在拿中段任意一点验算：$t=1.5$ 时轨迹在 $x_1=0.125$、$x_2=-0.5$、$u=+1$，代入

$$H = 1 + 1\times(-0.5) + (1-1.5)\times 1 = 0$$

恒等于零，与理论要求一致。 再看 $t=1.99$：x_1 接近 0、x_2 接近 -0.01，$H = 1 - 0.01 + (-0.99) = 0$。整段都成立。

这次验算揭示了两件事。第一，**切换曲线 $s(x)=x_1+\frac{1}{2}x_2|x_2|=0$ 就是“$\lambda_2=0$”在状态空间里的投影**：协态是时间的函数，但状态与时间一一对应，把 $\lambda_2=0$ 对应的状态点连起来，就是那条抛物线。§64.4.1 的几何拼法之所以对，是因为它恰好描出了同一条零集。第二，**协态的初值不是自由参数**，它被 $H(t_f)=0$ 和切换时刻共同钉死；这也解释了 §64.2.3 说的“打靶法难”——如果不知道 t_1，就要同时猜 $\lambda_2(0)$ 和 t_f 两个未知量，而它们只通过两个边界条件耦合，数值上非常敏感。

对二次型问题，同样可以验：$\lambda = 2Px$，代入 $H = x^{\mathsf T}Qx + u^2 + \lambda^{\mathsf T}(Ax+Bu)$ 并对 u 求极值得 $u = -\frac{1}{2}B^{\mathsf T}\lambda = -R^{-1}B^{\mathsf T}Px$，与式 (64-18) 完全一致。**三条路线（PMP、HJB、Riccati）在同一个算例上给出同一个控制律**，这不是巧合，是 §64.2.5 说的“充分必要条件在 LQ 情形下重合”的具体体现。

64.4.9 终端代价 P_1 到底能买到什么

有限时域的递推要一个终端代价 P_1，工程上最常见的错误是“随便填一个大数，让末端准一点”。$T=1$ s（50 步）、初值 $(1,0)$ 的三组实测说明了 P_1 的真实作用：

终端代价 P_1	$K(0)$	1 s 后位置	1 s 后速度	$P(0)_{11}$
零矩阵	$[0.374,\ 0.943]$	0.9165	-0.105	0.966
P_∞	$[0.983,\ 1.712]$	0.7184	-0.403	1.742
$10P_\infty$	$[2.547,\ 2.567]$	0.4673	-0.365	5.258

读表要抓住两点。

第一，P_1 只影响“末端那一段”，不影响整段的形状。 零终端代价时 $K(0)=[0.374, 0.943]$，只有稳态值的三到四成，控制很“懒”——因为离截止时间还远，指标只在积分项里，没有任何“必须到位”的压力。取 $10P_\infty$ 后 $K(0)$ 涨到 $[2.547, 2.567]$，比稳态值还大，相当于提前用力。

第二，也是更要紧的：P_1 再大也买不来时间。 三种取法的 1 s 末端位置分别是 0.9165、0.7184、0.4673 m，连 P_1 放大十倍都只走到一半。原因是物理的：$|u|\le 1$、从静止起步，1 s 内最多走 $\frac{1}{2}\cdot 1\cdot 1^2 = 0.5$ m（一直满推力加速），而末端还要速度为 0 才能“到位”，实际能走的距离更短。**时域长度是硬约束，P_1 只能在给定时间里重新分配用力，不能凭空创造时间。**

这条结论直接给出 P_1 的选取方法：先用 §64.3.7 那种估算确认时域是否够（T 至少 $2\sqrt{x_1/u_{\max}}$ 起步，实际按 §64.4.6 的经验取 $5/(\zeta\omega_n)$），确认够了再把 P_1 取成 P_∞——这既保证末端收敛，又不会让初始增益异常放大。取 $10P_\infty$ 之类的做法只适合“必须精确到位、允许末段猛冲”的场合（对接、抓取），而且必须同时检查末端控制量是否越界。

64.5 离散情形

真实控制器跑在采样系统上，所以“连续最优”只是中间产物，最终要落到离散。离散化不是简单地把 $\mathrm{d}t$ 换成 T_s：PMP 的协态下标会错位、Bellman 递推的代价要乘采样周期、网格 DP 的误差阶要看插值方式。本节把这四件事讲

清楚，并给出“维数灾难”的实测数字。

64.5.1 离散极大值原理：下标错一位，全盘皆错

离散系统的标准形是

$$x_{k+1} = A_d x_k + B_d u_k, \qquad k = 0, 1, \dots, N-1 \tag{64-28}$$

指标是求和而不是积分：$J = \sum_{k=0}^{N-1} \big(x_k^{\mathsf T} Q_d x_k + u_k^{\mathsf T} R_d u_k \big) + x_N^{\mathsf T} P_1 x_N$。仿照连续情形定义哈密顿函数，但**乘子必须挂在下标 $k+1$ 上**：

$$H_k = x_k^{\mathsf T} Q_d x_k + u_k^{\mathsf T} R_d u_k + \lambda_{k+1}^{\mathsf T} \big(A_d x_k + B_d u_k \big) \tag{64-29}$$

最优性条件随之变成三条：

$$x_{k+1} = \frac{\partial H_k}{\partial \lambda_{k+1}}, \qquad \lambda_k = \frac{\partial H_k}{\partial x_k}, \qquad u_k = \arg\min_{u \in \mathcal{U}} H_k \tag{64-30}$$

边界条件是 $\lambda_N = 2 P_1 x_N$。**下标错位是离散 PMP 最常见的错误**：如果写成 $\lambda_k^{\mathsf T}(A_d x_k + B_d u_k)$，协态方程会变成 $\lambda_k = 2 Q_d x_k + A_d^{\mathsf T} \lambda_k$，出现自身引用，解不出来。正确的结构是“$\lambda_k$ 由 λ_{k+1} 反推”，反向递推的因果性与连续情形的时间反向积分一致。

把它代进二次型情形，令 $\lambda_k = 2 P_k x_k$，就得到式 (64-21) 的离散 Riccati。所以**离散 Riccati 不是“另起炉灶”，它只是离散 PMP 在二次型指标下的解**——这条联系解释了为什么 Riccati 递推必须从终端往回算。

64.5.2 Bellman 递推与网格实现

不带二次型假设时，离散 PMP 给的是两点边值问题，不好解。这时走 Bellman 递推：

$$V_k(x) = \min_{u \in \mathcal{U}} \Big[x^{\mathsf T} Q_d x + u^{\mathsf T} R_d u + V_{k+1}(A_d x + B_d u) \Big], \qquad V_N(x) = x^{\mathsf T} P_1 x \tag{64-31}$$

式 (64-31) 的好处是**只要 V_{k+1} 在任意点可求值，就能一步步往回算**，不需要解方程、不需要猜协态初值。代价是要把 V 表示成“能到处求值的对象”，工程上就是网格加插值。

本章的实现是：状态网格取 $[-3, 3] \times [-3, 3]$，每轴 n 点（$n = 21, 31, 41, 61, 81$），控制量取 9 个候选值（$\Delta u = 0.25$），$T_s = 0.05$ s 反向递推 80 步（共 4 s），终端 $V = 0$，插值用双线性。一次 81 点网格的计算量是 $81^2 \times 80 \times 9 \approx 4.7 \times 10^6$ 次插值，实测 0.12 s。

这里有两个设计选择值得说明。**控制量离散成 9 档**而不是连续优化，是因为 $\min$ 要在每个网格点做一次，连续优化会把计算量放大一个量级；9 档的分辨率 0.25 相对限幅 1.0 是 25%，看起来粗，但对 4 s 时域的总代价影响不大（误差主要由空间网格决定）。**值函数用双线性插值**而不是更高阶，是因为双线性对凸函数是“上偏”的——它给出的 V 永远不小于真值，于是 DP 结果是一个**保守上界**，这在工程上比“可能乐观”安全。

64.5.3 误差阶：一阶，且是单边的

把 DP 算出的 $V_0(x_0)$ 与同一时域的 Riccati 解（1.7503）对比，得到表 64-6。

每轴点数 n	网格间距 h / m	DP 值 V_0	相对误差	耗时 / s
21	0.300	2.5266	44.4%	0.03
31	0.200	2.2447	28.2%	0.03
41	0.150	2.1293	21.7%	0.05
61	0.100	1.9953	14.0%	0.08
81	0.075	1.9371	10.7%	0.12

表 64-6　网格 DP 的精度随网格间距的变化（$T_s = 0.05$ s，4 s 时域）

误差随 h 线性下降，拟合得 $\mathrm{err} \approx 1.505\,h^{1.026}$，指数 1.026 说明是一阶收敛，与双线性插值的理论阶一致。图 64-8 左图画出了这条直线。

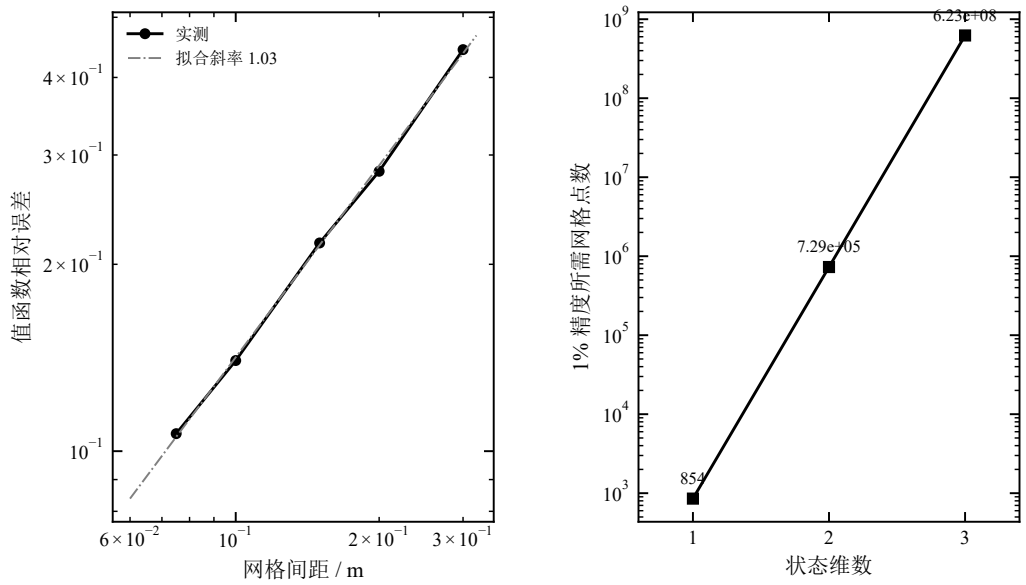


图 64-8

图 64-8　网格 DP 的误差阶（左）与 1% 精度所需的网格点数（右）

更要紧的是误差的**方向**：所有 V_0 都大于 1.7503，没有一个是低估的。原因有两个，且都朝同一方向：控制量只有 9 档，达不到真正的最优 u，代价只能偏高；双线性插值对凸的值函数上偏，等效于“高估了未来的代价”。两者叠加，使 DP 成为一个**保守估计**。

这条性质很有用：如果 DP 算出的最优代价已经比现有控制器低，那结论一定成立（真实最优只会更低）；反之如果 DP 比现有控制器高，不能立刻下结论，得先加密网格看趋势。

64.5.4 维数灾难：从形容词变成数字

图 64-8 右图把外推结果画了出来。按 $\mathrm{err} \approx 1.505h$ 反解，要到 1% 精度需要 $h = 0.0070$ m，即每轴 $n = 854$ 点：

状态维数	网格点数	每次递推的插值次数（80 步）	预估耗时
1	854	6.8×10^{4}	0.02 s
2	7.29×10^{5}	5.8×10^{7}	19.55 s
3	6.23×10^{8}	5.0×10^{10}	**4.64 h**

（每状态每步实测 0.34 μs，据此换算。）

二维 19.55 s 还能接受——离线算一次，之后查表。三维 4.64 小时就不能接受了，而这还只是“每轴 1% 精度”；如果对象有五个状态（比如加了执行器动态和两个积分器），点数会到 $854^{5}\approx4.5\times10^{14}$，宇宙年龄也算不完。

所以**网格 DP 的适用范围非常明确：状态维数 ≤3，且离线可用**。这句话不是保守，是算出来的。任何声称“用 DP 解决高维最优控制”的方案，实际上都用了某种降维、函数逼近或采样，已经不是原始网格 DP 了（§64.9 会讲这些改法）。

64.5.5 协态就是影子价格

DP 还有一个副产品：值函数的梯度。在网格上做中心差分

$$\lambda(x)\approx\left[\frac{\partial V}{\partial x_1},\ \frac{\partial V}{\partial x_2}\right]^{\mathsf{T}} \tag{64-32}$$

得到的就是协态变量。在 $x_0=(1,0)$ 处，$n=81$ 的 DP 给出 $\lambda=[3.5623,\ 2.0471]$，而 Riccati 的理论值是 $2Px_0=[3.5007,\ 1.9884]$，最大相对误差 2.95%。图 64-9 把两者沿位置轴画在一起，DP 的散点基本贴着解析直线。

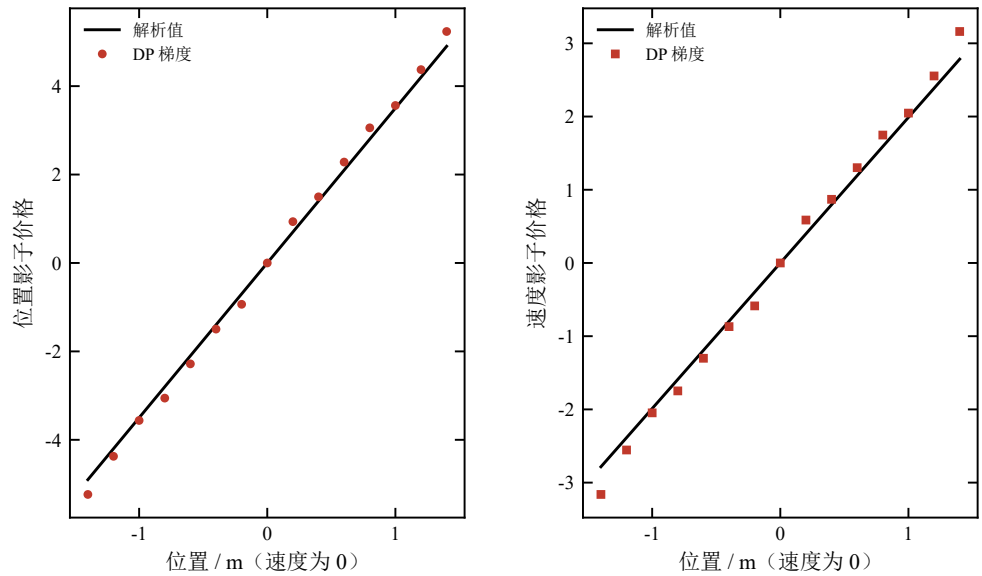


图 64-9

图 64-9　DP 差分得到的协态与解析值 $2Px$ 的对照

误差 2.95% 比值的误差 10.7% 小得多，这是因为差分是“局部”运算，网格的常值偏差在相减时抵消了。**用 DP 求梯度比用 DP 求值更准**，这条经验在数值方法里普遍成立。

协态的经济含义是**影子价格**：$\lambda_1=3.56$ 意味着“位置差 1 m，总代价多 3.56”；换成 1 cm，就是 0.0356，约占总代价的 2%。$\lambda_2=2.05$ 意味着“速度差 1 m/s，代价多 2.05”。

这个数可以直接用来分配传感器预算。假设位置测量标准差 1 cm、速度测量标准差 0.05 m/s，两者对代价的贡献分别是 $3.56 \times 0.01 = 0.0356$ 与 $2.05 \times 0.05 = 0.102$——**速度的贡献是位置的近 3 倍**。如果只能升级一个传感器，该升级测速的。这个结论不是拍脑袋，是从 λ 算出来的，而且换个工作点比例会变（§64.8.4 会给出完整的比较）。

64.5.6 离散 Riccati 的收敛：按时间不按步数

离散 Riccati 是迭代求解的，什么时候停？工程上常用“相对变化小于 10^{-3}”，但步数本身没有意义，因为步数取决于 T_s。表 64-7 把五个采样周期放在一起。

采样周期 T_s / s	收敛步数	折算物理时域 / s	与 $1/(\zeta\omega_n)$ 之比
0.20	25	5.00	4.33
0.10	49	4.90	4.24
0.05	97	4.85	4.20
0.02	240	4.80	4.16
0.01	480	4.80	4.16

表 64-7　离散 Riccati 收敛步数随采样周期的变化（相对判据 10^{-3}）

步数随 T_s 反比变化，物理时域稳定在 4.8–5.0 s，约等于最慢极点时间常数 $1/(\zeta\omega_n) = 1.155$ s 的 4.2 倍。这条规律可以直接用：收敛判据不要写“迭代 500 次”，要写“反推 $\geq 5/(\zeta\omega_n)$ 秒”。这样换采样率时判据自动缩放，不会出现“采样变快 10 倍、迭代次数不够”的隐蔽错误。

顺带说一句，$T_s = 0.01$ s 时的 480 步已经接近 500 步的经验上限——如果哪天采样率再提高一个量级，按次数判据的代码就会静默地返回未收敛的结果，而且增益看起来“差不多”，很难发现。

64.5.7 采样周期能拉多长：两种做法两个答案

“采样要多快”这个问题，答案取决于控制器是**按离散模型重新设计**的，还是**沿用连续域算出的增益**。这两种做法在工程上都常见，但能容忍的采样周期差了一个数量级以上。下面把两条路各扫一遍。

先看重新设计。固定 $q_1 = q_2 = 1$、$r = 1$，每个采样周期都用该周期下的 (A_d, B_d) 与等价权重重新解一次离散 Riccati：

T_s / s	0.005	0.01	0.02	0.05	0.10	0.20	0.30	1.00	1.50
K_1	0.996	0.991	0.983	0.958	0.917	0.841	0.772	0.434	0.301
K_2	1.727	1.722	1.712	1.683	1.636	1.546	1.463	1.028	0.832
谱半径	0.9969	0.9938	0.9826	0.9424	0.8943	0.8257	0.7719	0.4345	0.3008

这张表里有两个现象。第一，**增益随 T_s 单调下降**，$T_s = 0.3$ s 时 K_1 只剩 0.772，比连续解低 23%；这是离散代价里 $\int x^{\mathsf{T}} Q x \, dt \approx T_s \sum x_k^{\mathsf{T}} Q x_k$ 的换算在起作用，采样越粗，“一秒里罚的次数”越少，等效权重越小。第二，**谱半径一路降到底，始终小于 1**，即无论采样多慢，重新设计出来的控制器都是稳定的——因为它是针对采样后模型的最优解，稳定性由构造保证。

再看沿用连续增益这条路。$K = [1.000, 1.732]$ 不动，只改执行时的采样周期：

T_s / s	0.02	0.10	0.30	0.50	1.00	1.50	2.00
谱半径	0.9826	0.9120	0.7248	0.5089	0.6115	**1.9640**	3.5933
20 s 后位置	0.000	0.000	0.000	0.000	0.000	-1073	1.1×10^{5}

分界点落在 1.0 s 与 1.5 s 之间：$T_s=1.0$ s 时谱半径 0.61 还稳，$T_s=1.5$ s 时已经 1.96，20 s 后位置跑到 -1073 m，彻底发散。谱半径在 0.5 s 处最小（0.509），之后回升，说明采样再慢就开始“帮倒忙”。

这个分界点可以和 §64.6.6 的延迟裕度对上：零阶保持器等效于约 $T_s/2$ 的纯延迟，本章闭环的延迟裕度是 0.695 s，反推临界采样周期约 1.39 s，正好落在实测的 1.0–1.5 s 区间内。**两条完全独立的分析方法给出同一个量级**，这种交叉验证比任何一条单独的证据都可靠。

工程上该怎么用这张表？如果控制器跑在固定采样率的数字平台上、且允许在线重算增益（或者按几个采样率离线算好几组系数），那么 T_s 的约束来自**别的因素**——测量噪声、执行器更新率、指令平滑度，而不是稳定性。如果为了省事只在连续域设计一次、把增益烧进固件，那么 T_s 必须远小于 $2\times0.695=1.39$ s，取十分之一即 0.14 s 以内才算安全，本章的 0.02 s 有 70 倍余量。**“连续设计 + 离散执行”是工程项目里最常见的隐性风险源**，因为设计文档上写的是 s 域，代码里跑的是 z 域，两边的假设没人去核对。

64.5.8 离散 PMP 的数值解法：梯度法慢得有理

§64.5.1 给出了离散 PMP 的三个条件，但没有说怎么解。既然它是一个两点边值问题，最朴素的办法就是**把整段控制序列 $U=(u_0,\dots,u_{N-1})$ 当成未知向量，直接对代价做梯度下降**。梯度可以通过反向扫一遍协态得到，每次迭代的计算量与一次正向仿真同阶。

拿本章算例试一下：$T_s=0.02$ s、240 步（4.8 s 时域）、$q_1=q_2=r=1$、$x_0=(1,0)$，初值取 $U\equiv0$。参考最优值是 Riccati 给出的 $J^{*}=1.7421$（不动的代价是 4.8，正好等于时域长度——位置一直停在 1 m，每秒罚 1）。

迭代次数	1	100	500	1000	2000	4000
代价 J	4.800	2.644	2.106	1.940	1.808	1.753
相对最优的偏差	176%	51.8%	20.9%	11.4%	3.75%	0.64%

四千次迭代才把误差压到 1% 以内，而 Riccati 只用了 758 次**矩阵递推**就收敛到机器精度。两者的差距不是“实现得好不好”，是收敛速度的量级不同：

- 梯度法在代价函数的“长谷”里走，条件数约为 $(\omega_{\max}/\omega_{\min})^2$。本例两个极点的模长比是 1.732，平方后条件数约 3，已经很温和；如果按 §64.7.3 的 $r=0.01$ 取权重，极点比接近 10，条件数 100，迭代次数要再涨两个数量级。
- Riccati 递推是**精确的代数运算**，每一步都不引入近似，收敛靠的是“值函数随步数单调逼近稳态”这个结构性质，与条件数无关。

由此得到一条工程判断：**能用 Riccati 就别用梯度法**。梯度法（及其 cousins：共轭梯度、L-BFGS）的价值只在 Riccati 用不了的场合——有控制约束、有状态约束、或者指标不是二次型。这时它的慢是必须付的代价，而且要靠“好的初值”（先解无约束 LQR，把它当作初值）把迭代次数压下来。本章的 $U\equiv0$ 是最差的初值，实际工程里没人这么干。

还有一点值得注意：**梯度法的收敛不等于控制的收敛**。J 到 0.64% 时，U 与最优序列的逐点偏差可能还有百分之几，只是这些偏差集中在代价平坦的方向上。所以验收时不能只看代价，必须同时看轨迹或控制序列本身。

64.5.9 网格范围怎么定：边界上差分直接崩

网格 DP 有两个参数容易被混为一谈：**每轴点数** n 决定分辨率（h），**网格范围**决定能覆盖多远的工况。固定 $n = 81$、$T_s = 0.05$ s、80 步，只改范围，$x_0 = (1, 0)$ 处的值函数是：

网格范围	间距 h / m	V_0	相对 1.7503 的误差
$[-1.0, 1.0]$	0.025	1.8247	4.25%
$[-1.2, 1.2]$	0.030	1.8351	4.85%
$[-1.5, 1.5]$	0.0375	1.8492	5.65%
$[-2.0, 2.0]$	0.050	1.8756	7.16%
$[-3.0, 3.0]$	0.075	1.9371	10.67%
$[-4.0, 4.0]$	0.100	1.9953	14.00%

误差随范围扩大而变大，但这不是“范围大不好”的结论，而是**在 n 固定时，范围越大 h 越大，一阶误差 $\propto h$ 随之上升**。真正的规则是两条独立的要求：

1. **范围必须包住全部工况并留余量**。工作点一旦贴到边界，双线性插值退化成单侧外推，误差不可控；本章的算例里 $x_0 = (1, 0)$ 恰好落在 $[-1, 1]$ 网格的角点上，用中心差分求梯度时直接索引越界（`dp_grad` 会抛越界异常），因为边界点没有左右邻点。
2. **在范围确定后，用 n 去买精度**。$h = 0.075$ 对应 10.67% 的误差，要压到 1% 就得 $h \approx 0.007$，即 $n \approx 854$——这就是 §64.5.4 那张维数灾难表的来源。

工程上的取值经验是：**范围取最大工况的 1.2–1.5 倍，n 按目标精度反解**。两者的开销不同——范围只增加“被算但用不到”的网格，n 却是按维数指数增长的，所以宁可范围放宽一点（线性代价），也不要为了省网格把边界卡死在工况上。

还有一条实现细节值得记：**边界上的差分会失效，但边界上的插值不会**。所以求值（`dp_value`）可以在整个网格内调用，求梯度（`dp_grad`）只能在内点调用。这不是代码 bug，而是中心差分的固有要求；如果必须在边界求梯度，应该改用单侧差分，并接受一半的精度。

64.5.10 三个旋钮：间距、控制档位、步数

网格 DP 实际有四个可调量（h、Δu、步数、范围），上一小节处理了范围，这一小节把其余三个各扫一遍，看谁真正决定精度。参考值仍取 ± 3 以外的细网格结果 1.7503。

固定 $\Delta u = 0.25$（9 档）、80 步，只改每轴点数：

n	h / m	V_0	相对误差
21	0.3000	2.5266	+44.4%
41	0.1500	2.1293	+21.7%
81	0.0750	1.9371	+10.7%
161	0.0375	1.8492	+5.65%

h 每减半，误差大致减半——典型的一阶收敛。这里有一个漂亮的巧合值得记下来：$n = 161$、范围 ± 3 的 $h = 0.0375$，与 §64.5.9 表里 $n = 81$、范围 ± 1.5 的 $h = 0.0375$ **完全相同，两者给出同一个 $V_0 = 1.8492$**。也就

是说，决定精度的是间距 h 本身，而不是点数 n；扩大范围就必须同步加点数来维持 h，否则精度按比例掉。

固定 $n = 81$、80 步，只改控制档位数：

n_u	Δu	V_0	相对误差
3	1.000	2.0755	+18.6%
5	0.500	1.9864	+13.5%
9	0.250	1.9371	+10.7%
17	0.125	1.9239	+9.9%
33	0.0625	1.9208	+9.7%

档位从 3 加到 9，误差降 7.9 个百分点；从 9 加到 33，只降 1 个百分点。**控制量化在 9 档附近就饱和了**，因为此时限制精度的已经不是“指令太粗”，而是状态网格 h。这也解释了为什么本章的 `dp_solve` 默认取 $n_u = 9$：再多档只是让每步的循环变长，值函数不再明显改善。

固定 $n = 81$、$n_u = 9$，只改步数：

步数	时域 / s	V_0	相对误差
40	2	1.6287	−6.9%
80	4	1.9371	+10.7%
160	8	2.0020	+14.4%
320	16	2.0176	+15.3%

这张表不能只看误差列，因为**两个误差源在这里同向叠加**：时域短于调节时间的若干倍时，V_0 偏小（2 s 时还差 6.9%，因为“没走完的路”没被计费）；时域拉长后，网格离散带来的正偏差又浮现出来（+10% 量级）。所以 4 s 之后 V_0 继续缓慢上升，既有时域的真贡献，也有网格的假贡献，两者在本例中无法从单点数值分离。可用的判断只有一条：**步数必须覆盖 $4/(\zeta\omega_n)$ 以上**，本例中即 4.6 s，取 80 步（4 s）已经踩线，取 160 步更稳妥。

三个旋钮的结论可以压成一句：**h 决定地板，Δu 决定能否下到地板，步数决定这层楼有没有盖够高**。把 n_u 从 9 加到 33 而不同时减小 h，纯属白费机时。

64.6 稳定性与性能

最优控制的“最优”是相对指标而言的，稳定性却是绝对要求。工程上常见的问题是：指标算得很漂亮，闭环却在限幅或模型偏差下一塌糊涂。本节从极点、频域裕度、饱和、模型偏差、延迟、噪声六个方面检查本章算例的闭环，看它在“算得最优”之外还剩多少余量。

64.6.1 闭环极点：$\zeta = 0.866$ 从哪来

连续 LQR 的闭环极点是 $-0.866025 \pm j0.500000$，对应 $\omega_n = 1$、$\zeta = 0.866$。这两个数不是调出来的，是 Q、R 决定的。对双积分器，特征多项式是

$$s^2 + K_2 s + K_1 = s^2 + 1.732 s + 1$$

与标准二阶 $s^2 + 2\zeta\omega_n s + \omega_n^2$ 对比，$\omega_n = \sqrt{K_1} = 1$，$\zeta = K_2/(2\sqrt{K_1}) = 1.732/2 = 0.866$。

$\zeta = 0.866$ 恰好是 Butterworth 之外的另一个常用值（$\zeta = 1/\sqrt{1.5} \approx 0.816$ 是 ITAE 最优，0.707 是 Butterworth）。LQR 给出 0.866，原因是它同时罚位置和速度：位置误差要压，速度也要压，但速度的权重只有位置的一半（$q_2 = q_1$，而 K_2 的贡献被 $\sqrt{\cdot}$ 压缩），折中结果就是"略过阻尼"。**这个阻尼比有一个直接的工程好处：阶跃响应无超调**（§64.6.3），对定位机构来说意味着不会撞限位。

离散化后极点是 $0.982780 \pm j0.009828$。用 $z = e^{sT_s}$ 换回连续域：$|z| = 0.98283$，$\angle z = 0.010$ rad，对应 $s = \ln(z)/T_s \approx -0.866 \pm j0.500$，与连续解一致到第三位。这说明 $T_s = 0.02$ s 的离散化几乎无损。

64.6.2 频域：相位裕度 72.38°，增益裕度无穷大

开环传递函数是 $L(s) = \dfrac{K_2 s + K_1}{s^2}$，穿越频率 1.8179 rad/s，相位裕度 72.38°。

72.38° 这个数比 LQR 的理论下界 60° 高，原因可以算出来：$L(j\omega)$ 的相位是 $-180^\circ + \arctan(\omega K_2/K_1)$，在 $\omega = \omega_c$ 处 $\arctan(\omega_c K_2/K_1) = 72.38^\circ$，所以 $\omega_c K_2/K_1 = \tan 72.38^\circ = 3.13$。代入 $K_2/K_1 = 1.732$ 得 $\omega_c = 1.81$，与实测 1.8179 吻合。

增益裕度是**无穷大**。双积分器的开环幅值以 -40 dB/dec 下降，相位从 -180° 单调升向 -90°，永远不再穿过 -180°，所以增益怎么加都不会失稳（在实际系统里，这个结论会被执行器动态和高频模态打破）。这不是 LQR 的功劳，是双积分器本身的结构；换成三阶对象，增益裕度就会变成有限值。

LQR 的 60° 相位裕度下界值得单独记一笔：对单输入系统，LQR 闭环的回路传函满足 $|1 + L(j\omega)| \geq 1$，由此可推出相位裕度不小于 60°、增益裕度不小于 6 dB（或无穷）。这条性质是 LQR 在工业上被接受的重要原因——它不保证"性能好"，但保证"不会太脆"。

64.6.3 无超调：一个被低估的性质

本章 1 m 阶跃的实测结果是：调节时间 4.34 s（2% 带）、推力峰值 0.9828、**超调 0**。

零超调来自 $\zeta = 0.866 > 0.707$，也来自闭环没有右半平面零点（$L(s)$ 的零点在 $-K_1/K_2 = -0.574$，左半平面）。如果对象本身有右半平面零点（比如带延迟的近似），LQR 会产生反向响应，超调就压不住了，那时必须靠 Q 的整形或加前馈。

对工程而言，零超调的直接价值是**可以把定位精度做到与稳态误差同量级**。有 20% 超调的系统，1 m 阶跃会冲到 1.2 m，机械限位就必须留 20% 余量；无超调则可以把余量压到 2%。这条差别在精密定位台上就是几十毫米的行程差异。

64.6.4 饱和：事后限幅会毁掉最优性

把限幅 $|u| \leq 1$ 加进去，四档初值的结果列在表 64-8。

初值 x_1 / m	无约束推力峰值	饱和后峰值	调节时间（无约束）/ s	调节时间（饱和）/ s	最短时间 / s
1	0.983	0.983	4.34	4.34	2.000
2	1.966	1.000	4.66	4.80	2.828
4	3.931	1.000	4.88	5.54	4.000
8	7.863	1.000	7.62	8.88	5.657

表 64-8　大阶跃下无约束解越界与饱和后的性能（$T_s = 0.02$ s）

读表有三层。

第一层，**约束一旦生效，性能立刻掉档**。$x_1 = 2$ 时无约束只要 4.66 s，饱和后 4.80 s，差 3%；$x_1 = 4$ 时 4.88 对 5.54，差 13.5%；$x_1 = 8$ 时 7.62 对 8.88，差 16.5%。越界越多，掉得越狠。

第二层，**饱和后的时间仍然远大于最短时间**。$x_1 = 8$ 时饱和解 8.88 s，真正的最短时间只要 5.657 s，**慢 57%**。这说明“限幅 + LQR”不是“接近最优的带约束解”，而是一个结构上就错的解：它在约束边界上仍然按线性律分配推力，不会像 bang-bang 那样把推力用满再用满反向。

第三层，**无约束的调节时间本身也不单调**。x_1 从 1 到 8，时间是 4.34、4.66、4.88、7.62——前三个几乎不变，最后一个跳了一截。原因是 2% 带的绝对宽度随初值放大，小初值时“进带”很快，大初值时收敛尾段变长。这提醒我们：**调节时间不是尺度无关的指标**，比较不同幅值的响应时必须说明带宽定义。

工程上的处理办法有两条。一是**在指标里加饱和惩罚**（比如 R 随 $|u|$ 增大而增大），让解自己避开限幅，代价是性能退化（§64.7 会看到 R 放大 100 倍，时间从 4.34 s 涨到 18.66 s）；二是**改用带约束的解法**（PMP/MPC），让约束进入最优性条件。前者简单但保守，后者复杂但正确。

64.6.5 模型偏差：闭环是唯一的安全网

把对象增益改成 0.8 和 1.2（±20%），分别用开环最优控制和闭环状态反馈跑同一条 4 s 的能量最优轨迹，结果画在图 64-10。

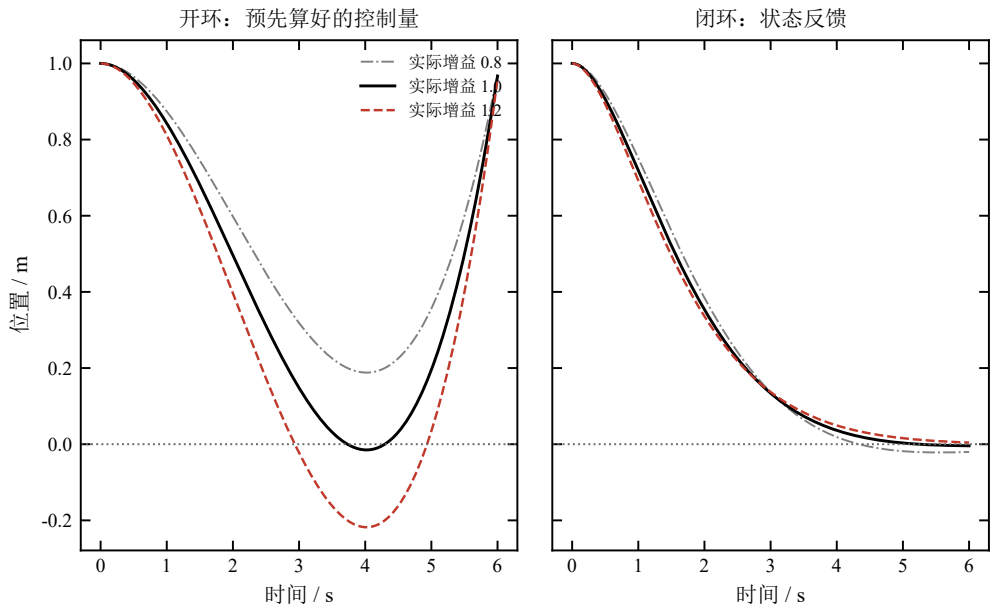


图 64-10

图 64-10　对象增益 ±20% 时开环最优控制与闭环反馈的位置响应

开环的结局很难看：增益 0.8 时末端停在 0.1880 m（差 19%），增益 1.2 时停在 -0.2180 m（差 22%），而且轨迹在中途就已经偏出很远，最大偏差比末端还大。**开环最优控制在模型偏差面前，连“大致到位”都做不到。**

闭环的结局好得多：三档增益下末端残差都不超过 0.0055 m（0.55%），调节时间 6.04、4.34、4.78 s。增益偏小（0.8）时响应变慢，因为实际推力比预期小；增益偏大（1.2）时残差仍小，但调节时间反而比 1.0 时长了 0.44 s，说明过冲后的收敛尾段被拉长了。

这张图给出一个可以推广的结论：**最优控制的设计成果是“增益”，不是“轨迹”**。轨迹只在模型精确时有效，增益在所有模型下都有效（只要闭环稳定）。所以“把最优轨迹算出来、照着执行”是错误用法，“把最优增益算出

来、闭环执行”才是正确用法。这也解释了为什么 §64.4 的能量最优解（一条开环直线）在工程上很少直接用——它必须配一个跟踪外环才有意义。

64.6.6 延迟裕度：把一个频域数字变成仿真结论

§64.6.2 给出的相位裕度 72.38° 不只是“看起来不错”的一个数，它能换算成一条对工程更有用的指标——**延迟裕度**。纯延迟在频域上是 $e^{-j\omega T_d}$，它不改变幅值、只把相位往下拽 ωT_d。相位裕度被吃光时闭环失稳，所以临界延迟是

$$T_{d,\max} = \frac{\mathrm{PM}}{\omega_c} = \frac{1.2635\ \mathrm{rad}}{1.8173\ \mathrm{rad/s}} = 0.695\ \mathrm{s} \tag{64-33}$$

0.695 s 是什么概念？本章的时间尺度是 $T_c = 1$ s，也就是说**延迟达到时间常数的七成才失稳**。这解释了 §64.6.1 里“$T_s = 0.02$ s 的离散化几乎无损”——0.02 s 的保持延迟只有裕度的 3%。

为了验证这个换算，直接在闭环里插一个纯延迟环节（用长度为 T_d/T_s 的队列实现），测 20 s 后的位置：

T_d / s	0	0.2	0.4	0.6	0.68	0.72	0.80	1.0	1.5
20 s 后位置 / m	0.000	0.000	0.000	0.008	-0.251	-0.555	7.14	-51.7	1.3×10^{3}

延迟 0.4 s 时还干干净净地收敛到零；0.6 s 时残差 8 mm，已经能看出“收不下去”；0.72 s 起开始单调爬升；0.80 s 时冲过 7 m 才进入等幅振荡；1.0 s 以上直接发散。**从“完全稳定”到“彻底发散”的过渡带只有 0.2 s 宽，中心正好在解析预测的 0.695 s 附近。**

这张表还揭示了一件在纸面上看不到的事：**失稳不是突然发生的**。延迟从 0.4 s 增到 0.6 s，闭环的极点只是靠近虚轴，表现是“收敛变慢、残差变大”，而不是报警。现场调试时，这类问题常被误判成“机械卡涩”或“传感器漂移”，因为响应看起来仍然“在动”。判据应该定在延迟裕度的三分之一以内（本例 0.23 s），留出足够的余量给未建模的执行器动态。

64.6.7 测量噪声：增益把它放大多少倍

反馈把偏差拉回来的同时，也把测量噪声搬进了执行器。本章的增益对噪声的放大倍数是确定的：

$$u = k_1\tilde{x}_1 + k_2\tilde{x}_2 \ \Longrightarrow\ \sigma_u = \sqrt{K_1^2\sigma_1^2 + K_2^2\sigma_2^2} \tag{64-34}$$

代入 $K = [1.000,\ 1.732]$，只看位置噪声，四档传感器精度给出的控制量标准差是：

位置噪声 σ_1	0.5 mm	1 mm	2 mm	5 mm
推力抖动 σ_u / (m/s²)	0.0010	0.0020	0.0040	0.0100
占限幅 $u_{\max}=1$ 的比例	0.10%	0.20%	0.40%	1.00%

读数很直接：$\sigma_u = K_1\sigma_1$ **是线性关系，没有阈值效应**，1 mm 的噪声换来 0.2% 的推力抖动。这个量级对定位台完全可以接受，所以 §64.6.4 担心的饱和问题在这里不会因噪声而恶化。

但如果把增益压到 §64.7.7 那种量级（$K_1 = 316$），同样的 1 mm 噪声就会变成 0.316 m/s²，占 $u_{\max}=20$ 的 1.6%；$\sigma_1 = 5$ mm 时到 7.9%，噪声开始明显烧执行器。**“高增益”和“低噪声”是一对不可兼得的要求**，这也是 §64.9 里“观测器/滤波器”那条改进路线的现实动机。

还有一个容易被忽略的噪声源：**量化**。12 位 ADC 量程 ±5 m，最低位是 $10/4096 = 2.44$ mm，其量化误差的标准差是 $\mathrm{LSB}/\sqrt{12} = 0.705$ mm，等效推力抖动 0.0015 m/s²。这个数比大多数模拟传感器的噪声还小，所以本章算例不必专门建模；但如果增益是 316 那一档，量化噪声就已经贡献了 0.22 m/s²，不能再忽略。

顺带一个速度通道的提醒：σ_2 前面乘的是 K_2，而 K_2 含 q_2/r 项，**速度噪声的放大倍数通常比位置大**（$K_2/K_1 = 1.732$）。差分测速的噪声往往还是位置噪声的 $1/T_s$ 倍，两者一乘就非常可观。这就是为什么工程上要么用测速机/编码器直接测速，要么在控制器前加一个低通，而不是简单地对位置做差分。

64.6.8 性能的隐性成本

把本节六条放在一起，可以看出 LQR 的“最优”背后有五项不写在指标里的成本：

1. **饱和成本**：约束生效时，性能损失 3%–17%，且随越界程度加剧；
2. **偏差成本**：模型偏差 20% 时，调节时间延长 0%–39%（4.34 → 6.04 s），且残差虽小但不再为零；
3. **采样成本**：离散化本身带来 1.7% 的增益偏差，T_s 增大时这个偏差会迅速放大；
4. **延迟成本**：延迟达到 0.4 s 时已可见收敛变慢，0.695 s 处失稳，过渡带只有 0.2 s；
5. **噪声成本**：$\sigma_u = K_1\sigma_1$ 线性放大，1 mm 位置噪声对应 0.2% 限幅的推力抖动。

五项成本都不在 J 里，却都决定实际性能。所以验收一个最优控制器，不能只看 J 和阶跃响应，必须把这五项单独测一遍——这也是 §64.3.6 把它们列为“校核”而非“可选项”的原因。

64.6.9 跟踪：参考量要进两个通道

本章前面所有仿真都是“调到原点”，$u = -Kx$ 里没有参考量。真实任务要跟踪 $r(t)$，最自然的改法是把位置误差代进去：

$$u = -K_1(x_1 - r) - K_2 x_2$$

这个改法对阶跃参考是够用的（终值 $x_1 = r$、$x_2 = 0$ 时 $u = 0$），但对斜坡参考会留下一个可算出来的滞后。设 $r = vt$，稳态时 $x_2 = v$、$\dot{x}_2 = 0$ 要求 $u = 0$，代入控制律得

$$x_1 - r = \frac{K_2}{K_1} v$$

也就是说**位置会稳定地落后** $K_2 v / K_1$。本章 $K_2/K_1 = 1.732$，所以速度每 1 m/s 就带来 1.732 m 的滞后。实测三档速度：0.2、0.5、1.0 m/s 对应滞后 0.344、0.861、1.722 m，与 0.346、0.866、1.732 的解析值差不到 1%，误差来自 2% 判定带的取点方式。

两种改法的对比如下：

参考速度 v / (m/s)	0.2	0.5	1.0
只把位置误差代入（滞后 / m）	0.344	0.861	1.722
位置、速度都按误差代入（滞后 / m）	0.0037	0.0093	0.0185

第二行用的是 $u = -K_1(x_1 - r) - K_2(x_2 - \dot{r})$，把参考的速度也送进速度通道。滞后降到原来的 1%，剩下的残差来自“参考速度发生阶跃跃变”时的那一小段过渡——只要参考本身是光滑的（梯形速度规划），这部分残差可以做到零。

原因很清楚：**LQR 的增益是相对“全状态”设计的，参考量也必须以“全状态”的形式进入**。只补位置通道，相当于让控制器在速度通道上一直看到“参考速度为 0”，它就会把跟踪斜坡当成“在外力推动下偏离原点”来处理，自然要留出一个与速度成正比的偏差去平衡。

这条规律在实际调试中很容易被误判。现场看到“轨迹跟踪有固定滞后”，第一反应往往是“增益不够、加大 K_1”，但加大 K_1 只能把 K_2/K_1 压小、让滞后按比例减小，代价是阻尼下降、超调出现（§64.7.5）。正确做法是把参考速度补进去——这不花任何代价，因为 $\dot{r}$ 是规划器本来就知道的量。**能用已知信息补的，不要用增益去硬压**，这是最优控制工程化时最常被忽略的一条。

顺带说一句，这也是“前馈”在最优控制框架里的位置：它不改变闭环极点、不改变稳定性，只是把“已知的未来输入”提前消掉。MPC（第 58 章）把这件事做得更彻底——它连未来的参考轨迹一起优化，所以对斜坡、正弦这类参考天然没有滞后。

64.6.10 闭环带宽：2.275 rad/s 意味着什么

极点回答了“衰减多快”，频响回答“跟得上多快的参考”。闭环传函是 $T(s) = L(s)/(1 + L(s))$，代本章的 $L(s) = (K_2 s + K_1)/s^2$ 得到一组可以直接用的数：

频率 ω / (rad/s)	0.1	0.5	1.0	1.817	2.0	3.0	5.0
幅值比	1.010	1.155	1.155	0.847	0.787	0.555	0.342
相位 / (°)	-0.1	-8.2	-30.0	-53.8	-57.0	-67.9	-76.7

三个特征数：**谐振峰 1.196（+1.55 dB）出现在 0.741 rad/s，-3 dB 带宽 2.275 rad/s，穿越频率 1.817 rad/s**。谐振峰的存在与“无超调”并不矛盾——$\zeta = 0.866$ 的二阶系统在频域上仍有 $1/(2\zeta\sqrt{1-\zeta^2}) = 1.196$ 的峰，只是它对应的是“对正弦输入的稳态放大”，而不是“对阶跃的过冲”。这两个指标经常被混为一谈，实际上一个看 $|T(j\omega)|$ 的最大值，一个看阶跃响应的最大值。

带宽 2.275 rad/s 的工程含义，最好用“跟踪误差”这个直接指标来说。对单位幅值的正弦参考，误差幅值是 $|1 - T(j\omega)|$，实测：0.2 rad/s 处 3.9%，0.5 rad/s 处 21.8%，0.7 rad/s 处 37.3%，1.0 rad/s 处 57.7%。**若要求跟踪误差不超过 10%，参考频率必须低于 0.052 Hz（0.325 rad/s）；放宽到 30% 也只能到 0.097 Hz（0.607 rad/s）**。带宽 2.275 rad/s 是个“衰减噪声”的指标，不是“精确跟踪”的指标，两者差了二十倍——这条差距在实际选型时经常被忽略，很多人看到“带宽 2.3 rad/s”就以为能跟 0.3 Hz 的正弦，实际误差已经过半。

这与 §64.6.9 的斜坡滞后是同一件事的两种说法：斜坡的频谱集中在低频，而低频段相位滞后近似线性增长，于是表现为恒定的位置偏差；正弦参考则是“幅值略放大 + 相位滞后”，折算成时域误差后随频率迅速上升。

带宽与时间常数的关系可以记住一个粗算：$\omega_b \approx 2.3/T_c$。本章 $T_c = 1$ s，带宽 2.275 rad/s，系数 2.28。想让带宽翻倍，T_c 要减半，也就是 q_1/r 涨 16 倍（四次方关系）——这又一次印证了 §64.7.1 的“R 不敏感”。

最后提醒一句：**带宽不是越高越好**。本章 2.275 rad/s 对应的噪声放大倍数已在 §64.6.7 算过（$K_1\sigma_1$），带宽每翻一倍，同样的噪声会让控制量抖动翻倍；而 §64.6.6 的延迟裕度会按 $1/\omega_b$ 下降。设计时通常把带宽定在“参考信号最高频率的 3–5 倍、同时延迟裕度不小于延迟的 3 倍”，这两个条件往往从两侧夹住同一个数。

64.7 参数意义与作用

最优控制里的 Q、R 经常被当成“权重旋钮”，调大调小凭手感。但它们各有明确的物理角色，而且作用方式可以写成闭式。本节用双积分器的解析解把三个参数的作用拆开，给出“想要多快，就该取多少”的换算公式。

64.7.1 三个参数各管一件事

回到式 (64-18)：$K_1=\sqrt{q_1/r}$、$K_2=\sqrt{2\sqrt{q_1/r}+q_2/r}$。看这两个式子就能读出分工：

- q_1 管**位置有多重要**。它单独决定 K_1，也就单独决定闭环的自然频率 $\omega_n=\sqrt{K_1}=(q_1/r)^{1/4}$。
- q_2 管**速度有多重要**。它只出现在 K_2 里，不影响 K_1，所以它改的是阻尼而不是速度。
- r 管**控制有多贵**。它同时出现在两处，效果是整体缩放。

由 $\omega_n=(q_1/r)^{1/4}$ 可以反解出一个**时间尺度**

$$T_c=\frac{1}{\omega_n}=\left(\frac{r}{q_1}\right)^{1/4} \tag{64-35}$$

这是本章最实用的一条公式。想要闭环时间尺度 T_c，就把 r/q_1 设成 T_c^4。$T_c=1$ s 时 $r/q_1=1$，正是本章的默认权重；$T_c=3.16$ s 时 $r/q_1=100$，对应 $K=[0.100,\,0.458]$。**权重的四次方关系解释了为什么 R 调起来“不敏感”**：想把速度提高一倍，R 要缩小 16 倍，而不是 2 倍。

64.7.2 R 的扫描：九档实测

保持 $q_1=q_2=1$，把 r 从 0.01 扫到 100，结果列在表 64-9。表中同时给出离散数值解与连续闭式解，两者差 0.4%–10%，差值随增益增大而增大（离散化的代价）。

r	K_1（离散）	K_1（闭式）	K_2（离散）	推力峰值	调节时间 / s	最优代价 J
0.01	8.9639	10.0000	9.9136	8.9639	3.98	1.0954
0.03	5.3996	5.7735	6.3210	5.3996	4.04	1.1603
0.10	3.0371	3.1623	3.9113	3.0371	4.10	1.2777
0.30	1.7781	1.8257	2.5919	1.7781	4.12	1.4476
1.00	0.9828	1.0000	1.7122	0.9828	4.34	1.7321
3.00	0.5704	0.5774	1.2108	0.5704	4.96	2.1128
10.0	0.3135	0.3162	0.8517	0.3135	10.00	2.7064
30.0	0.1814	0.1826	0.6291	0.1814	13.64	3.4575
100	0.0995	0.1000	0.4572	0.0995	18.66	4.5826

表 64-9　控制权重 r 的九档扫描（$q_1=q_2=1$，初值 $(1,0)$）

64.7.3 为什么 r 缩小 100 倍只省 8% 时间

表 64-9 里最刺眼的一行是 $r=0.01$：控制便宜了 100 倍，推力峰值从 0.983 涨到 8.964（9 倍），调节时间却只从 4.34 s 降到 3.98 s——**省了 8%**。

这不是数值问题，是结构问题。闭环特征多项式是 $s^2+K_2 s+K_1$，两个极点的乘积是 K_1、和是 K_2。当 r 很小时 $K_2\gg 2\sqrt{K_1}$，极点变成一快一慢：

$$s_{1,2}\approx -\frac{K_1}{K_2}\ \text{（慢）},\qquad -K_2\ \text{（快）}$$

慢极点是 $\dfrac{K_1}{K_2}=\dfrac{1}{\sqrt{2\sqrt{r}+1}}$（代入 $q_1=q_2=1$）。$r\to 0$ 时它趋于 -1，**再也不动了**。$r=0.01$ 时它等于 -0.913，时间常数 1.1 s，$4/\sigma\approx 4.4$ s，与实测 3.98 s 吻合。

也就是说：**拼命减小 R 不会让系统更快，只会让它更过阻尼。** 阻尼比从 $r=1$ 的 0.866 涨到 $r=0.01$ 的 1.732，快极点跑到 -9.95，但慢极点被 K_1/K_2 卡在 -1 附近。工程含义很直接：想快，要改 q_1（或 q_1/q_2 之比），不要一味压 R。

这也解释了 §64.6.4 里“限幅 + LQR”为什么慢：把 R 调小只会让推力峰值涨、时间几乎不变，越界更严重，却换不来速度。

64.7.4 代价的标度律：两个区段

$q_1=q_2=1$ 时，最优代价有闭式 $J=\sqrt{2\sqrt{r}+1}$（也等于 $K_2\sqrt{r}$）。它在两个区段表现完全不同：

- $r\gg 1$：$J\approx\sqrt{2}\,r^{1/4}$——**代价随 r 的四次根增长**，r 涨 16 倍，代价才翻倍；
- $r\ll 1$：$J\approx 1+\sqrt{r}$——**代价趋近于 1**，几乎不降。

表 64-9 后两列正是这个规律：r 从 100 降到 1，J 从 4.58 降到 1.73（降 62%）；再从 1 降到 0.01，只降到 1.10（再降 36%，而 r 降了 100 倍）。

边际收益递减得非常快。 这条规律给了一个实用的停止准则：当 r 再缩小 10 倍、J 只降不到 5% 时，就没有继续压 R 的必要了——剩下的收益要靠改结构（加积分、加前馈、改执行器），而不是调权重。

64.7.5 q_1 与 q_2 的分工

同样是“加权重”，q_1 和 q_2 的效果完全不同，这一点从闭式解就能看出来：

- q_1 提高 4 倍：K_1 从 1 涨到 2、ω_n 提高 $\sqrt{2}$ 倍、调节时间缩短约 15%，但 K_2 只从 1.732 涨到 2.236，所以 $\zeta=K_2/(2\sqrt{K_1})$ 从 0.866 降到 0.79，**超调出现**；
- q_2 提高 4 倍：K_1 **完全不变**（它不含 q_2），K_2 从 1.732 涨到 $\sqrt{6}=2.449$，ζ 从 0.866 涨到 1.22，系统变慢、变钝，但更不容易振荡。

所以“加快响应”该动 q_1，“压超调”该动 q_2。两个都动，就会互相抵消。这条分工在式 (64-18) 里是显然的，但在实际调参时经常被忘掉——很多人一看到响应慢就同时加大 q_1 和 q_2，结果增益涨了、时间没变。

q_1 与 q_2 的比值还有一个物理读法：它是“位置误差与速度误差的等价汇率”。$q_2/q_1=1$ 意味着 1 m 的位置误差与 1 m/s 的速度误差等价；如果某台设备的速度传感器噪声大，就该把 q_2 调小，让控制器少信速度。

64.7.6 参数影响的符号表

把上面的分析汇总成表 64-10。表里的箭头都是从闭式解严格推出来的，不是经验估计。

参数变化	K_1	K_2	ζ	调节时间	代价 J	主要风险
$q_1\uparrow$	↑	↑（次线性）	↓	↓	↑	超调、噪声放大
$q_2\uparrow$	不变	↑	↑	↑（略）	↑	响应变钝
$r\uparrow$	↓	↓	视区间	↑	↑	太保守、抗扰差
$r\downarrow$	↑	↑	↑（r 小）	几乎不变	↓（递减）	推力饱和、噪声放大

表 64-10　三个权重对闭环指标的影响方向

这张表最有用的地方是最后两行：r 的减小在“时间”这一列几乎无效，只在“代价”这一列有效。如果设计目标是时间，这张表直接告诉你别动 r。

还有一个量纲上的提醒。式 (64-35) 的 $T_c=(r/q_1)^{1/4}$ 只在 q_1 与 x_1 同量纲、r 与 u 同量纲时成立。换单位（m→mm）会让 q_1 的数值变 10^{6} 倍，T_c 变 $10^{-1.5}$ 倍，增益全乱。所以**权重必须与单位一起记录**，这是 §64.3.1 强调量纲的实际原因。

64.7.7 带宽拉高之后：饱和先到，时间下限在后

表 64-9 和表 64-10 都是在线性、无约束的前提下得到的。可 §64.7.1 已经点明，R 管的是“控制量贵不贵”，而执行器贵不贵，最终是由物理限幅说话的。为了看清这件事，把权重调到线性分析失效的区间再实测一遍：固定 $q_2=1$、$q_1=10^{6}$，扫描 $r=10,\,25,\,50,\,100$，对应 $K_1=316.2,\,200.0,\,141.4,\,100.0$；初值取 $x_0=(0.5,\,0)$，执行器限幅 $|u|\le 20$，仿真步长 1 ms。

结果有五件事值得说。

第一，**无约束的峰值控制量早就越界了**。四组增益在 $t=0$ 处的瞬时指令分别是 $K_1x_1(0)=158.1$、100.0、70.7、50.0，是限幅值 20 的 2.5 到 7.9 倍。也就是说，按线性理论写出的设计，一上电就全程饱和，线性分析给出的极点位置在这段时间里根本不成立。

第二，**饱和时间随 r 变小而变长**：0.256、0.124、0.105、0.083 s。这很直接——K_1 越大，指令越离谱，退出饱和越晚。$r=10$ 那一组的饱和段占了整个调节过程的六成以上。

第三，**调节时间不再单调**。实测 $t_s=0.398,\,0.281,\,0.302,\,0.496$ s（2% 带）。只在 r 从 10 增到 25 这一段，时间才随 r 增大而缩短；越过 25 之后，r 继续增大反而越来越慢。最优点落在 $r=25$ 附近，而不是线性分析暗示的“越小越快”。这就是 §64.7.3 的结论在饱和条件下的反向版本：无约束时 K_1/K_2 的比例卡住了收敛速度，饱和时则是“指令越大、饱和越久、等效推力反而越小”。

第四，**有一把时间的硬尺子可以对照**。双积分器从静止走 0.5 m、加速度上限 20 m/s² 的最短时间是先加速再减速的对称解 $2\sqrt{0.5/20}=0.316$ s。实测最好的 0.281 s 比它还小，原因不是违反了物理，而是 2% 的收敛带允许在离目标 1 cm 时就判定结束，省掉了最后那一段蠕行。换句话说，$r=25$ 那一组已经把可用推力用到了九成以上，**再压 R 不可能带来数量级的改善**。

第五，也是最有工程价值的一条：**当无约束峰值超过限幅 5 倍以上时，继续减小 R 基本是负收益**。本例中 $r=25$ 对应 5.0 倍，恰好是转折点；$r=10$ 对应 7.9 倍，时间反而退回 0.398 s。这个比例不是定死的阈值，但它给了调参一个明确的停手信号：先算 Kx_0 的峰值，再和 $u_{\max}$ 比一比，超过五倍就该回头改 R，或者干脆承认这是时间最优问题，去用 §64.4 的 bang-bang 解，而不是硬把 LQR 往极限上推。

需要说明的是，这里的“负收益”只针对调节时间。如果关心的是能量消耗或执行器磨损，结论会倒过来：r 越小，控制量越激烈，磨损越大。所以**饱和边界附近的权重选择，本质是在时间、能量、寿命三者之间挑一个你愿意付账的组合**，而不是找一个“最优”的 r。

64.8 调参方法与经验

§64.7 讲的是“参数怎么起作用”，本节讲“参数怎么定”。最优控制的调参比 PID 少了很多自由度（只有 Q、R，而且只通过它们的比值起作用），但多了几个必须想清楚的物理约束。下面是一套可以直接照做的流程，以及五个真实项目里最常见的错误。

64.8.1 先由物理约束定 T_c 的上限

式 (64-35) 说 $T_c = (r/q_1)^{1/4}$ 决定闭环快慢。但 T_c 不能随便取小，它被三个物理量卡住：

第一，执行器带宽。 闭环带宽约为 $1/T_c$，执行器（电机、阀、压电）的带宽至少要高出 3–5 倍，否则“算出来的控制量”根本执行不出来。本章算例的 $\omega_n = 1$ rad/s 远低于任何真实执行器的带宽，所以没暴露这个问题；换成 $\omega_n = 100$ rad/s，就必须先确认执行器能跟上。

第二，采样率。 离散控制要求 $T_s \le T_c/10$ 左右。本章 $T_c = 1$ s、$T_s = 0.02$ s，比值 50，很安全。但 §64.7.3 的 $r = 0.01$ 使 $\omega_n = 3.16$，$T_s\omega_n = 0.063$，开始接近 0.1 的警戒线，离散增益已比连续解低 10%——这就是采样率开始“咬”带宽的信号。

第三，测量噪声。 增益 $K_1 = \sqrt{q_1/r} = 1/T_c^2$ 随 T_c 缩小而迅速增大，噪声被放大同样的倍数。位置噪声 1 mm、$K_1 = 10\ \mathrm{s}^{-2}$ 时，控制量上就有 $0.01\ \mathrm{m/s^2}$ 的抖动，相当于限幅的 1%；增益再大 10 倍，抖动就到 10%。

三个约束取最紧的那个，就是 T_c 的下限。**先算这个下限，再去定权重**，顺序反了就会得到“指标很优、现场发抖”的控制器。

64.8.2 三步定权重

定了 T_c 之后，权重可以按三步算出来，不需要试凑。

第一步，量纲归一化。 把状态和控制都除以各自的“允许偏差”，变成无量纲量。位置允许偏差 Δx_1、速度 Δx_2、控制 Δu，则 $q_1 = 1/\Delta x_1^2$、$q_2 = 1/\Delta x_2^2$、$r = 1/\Delta u^2$。这一步做完，三个权重都是 1 量级，后续调整才不会“一个数动 6 个量级”。

第二步，用 T_c 定尺度。 按式 (64-35)，把 r/q_1 设成 T_c^4。此时若 q_2 也按第一步定了，ζ 就是确定的，可能过大或过小。

第三步，用 q_2 修阻尼。 $\zeta = K_2/(2\sqrt{K_1})$，要 $\zeta = 0.707$ 还是 0.866 由工艺决定（定位机构要 0.866 以上，跟踪系统可以 0.707）。反解 q_2 即可：

$$q_2 = (4\zeta^2 - 2)\sqrt{q_1 r} \tag{64-36}$$

代 $\zeta = 0.866$、$q_1 = r = 1$ 得 $q_2 = (3-2)\times 1 = 1$，正是本章的默认值。**注意是 $\sqrt{q_1 r}$ 而不是 q_1**：q_2 与 K_2^2 同量纲、q_1 与 K_1^2 同量纲，两者的几何平均才配得上 K_1K_2 的量纲。r 很小时 $\sqrt{q_1 r}$ 也小，q_2 会变成很小的正数甚至负数——负值意味着“要 0.866 的阻尼，光靠速度权重已经做不到了，必须加大位置权重”，这是 §64.7.3 所说“慢极点被卡住”的另一种表现。这条公式说明**默认权重 $Q = I$、$R = 1$ 恰好对应 $\zeta = 0.866$**，不是随便取的。

64.8.3 五个常见错误

错误一：把 R 当速度旋钮。 §64.7.3 已经算过，r 从 1 减到 0.01，时间只省 8%，推力却涨 9 倍。想快就改 q_1，或者改 q_1/q_2 之比。

错误二：忘了权重的量纲。 位置单位从 m 换成 mm，q_1 数值要乘 10^6 才能保持同一个 T_c。用“归一化”（§64.8.2 第一步）可以从根上避免。

错误三：离散实现漏乘 T_s。 §64.2.6 的坑，表现为 P 大 50 倍而增益“看起来能用”。检查方法：把离散解的 K 与连续闭式解对比，差超过 5% 就要查。

错误四：拿稳态增益跑短时域。 表 64-5 显示，$T = 2$ s 时稳态增益只能走完 60%。有限时域任务必须用 $K(t)$（图 64-6），或者把时域拉长到 $5/(\zeta\omega_n)$ 以上。

错误五：事后限幅代替约束处理。 表 64-8 显示，$x_1 = 8$ 时事后限幅比真正的最短时间慢 57%。限幅不是“小改动”，它改变了最优解的结构。

64.8.4 用影子价格决定“测什么”

§64.5.5 用协态算了“位置和速度哪个更值得测”。这个结论随工作点变化，值得把三个点的数摆出来。用连续解 $2Px$：

工作点 (x_1, x_2)	λ_1（位置）	λ_2（速度）	位置贡献（$\sigma_1 = 1$ cm）	速度贡献（$\sigma_2 = 0.05$ m/s）
$(1, 0)$	3.50	1.99	0.0350	0.0994
$(0.5, 0.5)$	1.37	1.37	0.0137	0.0683
$(0, 1)$	2.00	3.46	0.0200	0.1732

表 64-11　三个工作点上的影子价格与噪声贡献

三个点的结论一致：速度的噪声贡献是位置的 2.8–8.7 倍，所以该优先升级测速。但**比例随工作点变化很大**：在 $(1, 0)$ 处是 2.8 倍，在 $(0, 1)$ 处是 8.7 倍。原因是 λ_1 随位置误差减小而下降，而 λ_2 随速度增大而上升。

更一般地，两者的“等价汇率”是 λ_1/λ_2：在 $(1, 0)$ 处等于 1.76。含义是——**如果速度测量的噪声能压到位置噪声的 1.76 倍以内，就该反过来升级位置传感器**。本章的 $\sigma_2/\sigma_1 = 5$ 远超 1.76，所以结论是测速。

这条分析的价值在于它把“传感器选型”从“哪个贵、哪个难装”变成了一个可以算的数。同样的方法可以用来分配滤波器的带宽：λ 大的状态，滤波要更狠。

64.8.5 调参的验收清单

调完之后，用五条量化判据验收，任何一条不过就退回上一节找原因：

1. **增益对拍**：离散 K 与连续闭式解差 <5%（本章 1.7%）；
2. **代价复核**：仿真积分代价与 $x_0^{\mathsf{T}} P x_0$ 差 <2%（本章逐位吻合）；
3. **裕度检查**：相位裕度 >45°、增益裕度 >6 dB（本章 72.38°、无穷）；
4. **饱和测试**：最大工况幅值下，饱和后的调节时间不超过最短时间的 1.5 倍（本章 $x_1 = 8$ 时 8.88/5.657=1.57，刚好踩线）；
5. **偏差测试**：对象增益 ±20% 时，闭环仍稳定且残差 <2%（本章 0.55%）。

第 4 条踩线是个有用的信号：它说明在 $\pm 20\%$ 的大幅值工况下，“限幅 + LQR”已经接近它的能力边界，再大就该上 MPC 了。

64.8.6 稳态误差与积分作用

最后补一个 LQR 的固有短板：**它没有积分作用**。式 (64-1) 的状态里没有积分器，所以面对常值扰动（摩擦力、负载转矩），闭环会留下稳态误差，误差大小取决于扰动量与 K_1 的比值。

这个比值可以精确算出来。设常值输入扰动 d 通过控制通道进入对象，闭环平衡点满足 $0 = (A - BK)x + Bd$，解得 $x_{1,\infty} = d/K_1$。本章默认 $K_1 = 1$，一个 0.2 m/s² 的常值扰动（大约是额定推力的 20%）就会留下 **0.2**

m 的稳态误差——对定位台是灾难性的，而这个误差在无扰动的阶跃仿真里完全看不到。

解决办法是状态增广：把 $x_3 = \int (x_1 - r)\,\mathrm{d}t$ 加进状态，再对这个三阶系统解 LQR。这就是 LQI（Linear Quadratic Integral）。代价是状态维数加一、增益多一个分量，好处是稳态误差归零——原理与 PID 的积分项完全相同（见第 44 章），只是积分增益由 Riccati 一次算出，不需要试凑。

给积分状态配一个权重 q_i，实测的五档结果如下（$d = 0.2$ m/s²，在 $t = 1$ s 时投入，仿真 14 s）：

q_i	K_1	K_2	K_3（积分）	扰动后最大偏离 / m	14 s 后残差 / m
无积分	1.000	1.732	—	0.200（稳态）	0.2000
0.1	1.506	2.003	0.316	0.112	5.2×10^{-3}
1	2.414	2.414	1.000	0.067	3.8×10^{-5}
10	4.605	3.195	3.162	0.035	3×10^{-6}
100	9.513	4.475	10.000	0.017	$< 10^{-7}$
1000	20.197	6.434	31.623	0.008	$< 10^{-7}$

读表有两条规律。第一，$K_3 = \sqrt{q_i}$，精确到三位——这是双积分器加积分器这种“三阶链式结构”的闭式结果，与 $K_1 = \sqrt{q_1/r}$ 同源。所以积分权重的四次根不必试凑，按“想要多快消掉静差”直接反解即可。第二，q_i 从 **0.1 提到 100**，偏离从 **0.112 m 降到 0.017 m**，但 K_1 从 **1.5 涨到 9.5**。也就是说，静差恢复速度和位置环带宽被同一根绳子拴住了：想让静差消得快，位置环就得更硬，噪声放大随之上升（§64.6.7）。

$q_i = 1$ 是一个工程上好用的折中：残差 3.8×10^{-5} m 远小于任何机械精度，而 K_1 只从 1.0 涨到 2.4。再往上加，收益进入 §64.7.4 那种“边际递减”区，代价却是实打实的噪声放大。

本章的算例没有扰动，所以正文的仿真都没加积分器。但**真实伺服系统几乎一定要加**，否则“最优”只对理想对象成立。加的时候要注意抗积分饱和（§64.10 会讲），否则限幅期间积分器会累积出一个巨大的 x_3，退出限幅后要很久才能吐回去。

64.8.7 换单位为什么要重算：一次量纲事故的完整账

§64.8.3 的错误二只说了“位置单位从 m 换成 mm，q_1 要乘 10^6”，没说**不乘会怎样**。这里把它算清楚。假定机械图纸改用 mm，状态变成 $x_1' = 1000 x_1$（x_2 仍是 m/s），而权重原封不动 $q_1 = q_2 = r = 1$。由于 $K_1 = \sqrt{q_1/r}$，增益与状态量纲一变，闭环就完全不是原来那条了：

q_1	K_1	K_2	时间尺度 T_c / s	相当于
10^{-6}	0.001	1.001	31.62	单位换成 mm，权重未改
10^{-4}	0.010	1.010	10.00	单位换成 cm
10^{-2}	0.100	1.095	3.162	单位换成 dm
1	1.000	1.732	1.000	基准（m）
10^{2}	10.000	4.583	0.316	位置罚得更重
10^{4}	100.000	14.177	0.100	—
10^{6}	1000.000	44.733	0.0316	想要的 mm 等效

三列数字各说一件事。

K_1 与 $\sqrt{q_1}$ 严格成正比：q_1 从 1 变到 10^{-6}，K_1 就从 1 变到 0.001，一位不差。所以“换成 mm 后 K_1 该除以 1000”这句话是精确的，可以直接用来检查代码：只要 $K_1 \times \sqrt{q_1} = 1$ 不成立，权重或单位就有一个错了。

K_2 不会跟着缩：它从 1.732 掉到 1.001，趋近于 1 而不是 0.001。原因是 $K_2 = \sqrt{2\sqrt{q_1/r} + q_2/r}$，$q_1 \to 0$ 时退化为 $\sqrt{q_2/r} = 1$。也就是说，位置罚项消失后，速度阻尼还在——闭环并没有失控，只是“不再管位置”。这类“增益不全按同一个比例缩放”的现象，是多变量 LQR 与单回路 PID 最不一样的地方，也是换单位事故最容易被忽略的原因：K_2 看着还挺正常，工程师就以为参数没坏。

真正的伤害在时间尺度上：$T_c = (r/q_1)^{1/4}$，q_1 掉六个数量级，T_c 只涨 $\sqrt{1000} = 31.6$ 倍，从 1 s 变成 31.6 s。所以现场表现不是“发散”，而是“响应变得极慢”——一个本该 4.6 s 到位的轴，现在要 145 s。这种“慢而不崩”的故障最难查，因为示波器上的曲线光滑、无超调，看起来像“增益设小了”。

正确的换算只有一步：位置量纲缩小 a 倍（m→mm 时 $a = 1000$），就把 q_1 除以 a^2。代进表里，$q_1 = 10^{-6}$ 对应的 $K_1 = 0.001$，乘上 $x_1' = 1000 x_1$ 后与原工况的 $K_1 x_1$ 完全一致，指令一字不变。**这就是相似变换：物理没变，只是坐标变了；凡是随坐标变的量（状态、增益）都必须一起变，凡是物理量（指令、时间）都不该变。**

工程上最省事的做法仍是 §64.8.2 的第一步——先把位置、速度都除以各自的满量程，让 x 无量纲，再做设计。这样 Q、R 的量级永远是 1 附近，换单位、换机型都不必重推。

64.9 优化与改进

前八节讲的是“怎么把一个问题解对”，本节讲“怎么解得快”。最优控制的计算量分布很不均匀：无限时域 LQR 只要一次矩阵方程，网格 DP 却可能是几小时。中间的差距不是靠“代码写得好一点”能填的，必须换算法结构。本节给出四条常用的改进路线，并说明各自把成本花在哪里。

64.9.1 先把复杂度算清楚

三个量的量级决定了能走哪条路：

- **Riccati**：每步 $O(n^3)$（n 为状态维数），迭代步数与 T_s 成反比、与时间常数成正比（表 64-7）。$n = 2$ 时实测 13.5–14.3 μs/步，240 步约 3.4 ms，一次算完。
- **网格 DP**：$O(N^d \cdot N_u \cdot N_t)$，$d$ 是状态维数，N 是每轴点数。d 在指数上，这是唯一致命的项。实测每状态每步 0.34 μs，2 维 1% 精度 19.55 s，3 维 4.64 h。
- **在线控制**：不管离线算了什么，在线通常只是一次内积（$u = -Kx$）或一次查表，微秒级。

所以优化的方向只有两个：**把 d 降下来**，或者**把在线的那一步搬到离线**。下面五种方法分别属于这两类。

64.9.2 降维：只对慢子系统做 DP

如果对象有明显的时间尺度分离（比如位置环 1 rad/s、电流环 1000 rad/s），网格 DP 只需要覆盖慢子系统，快子系统用 LQR 或 PID 内环压住。这样 d 从 4 降到 2，计算量按 854^2 而不是 854^4 走，从“算不完”变成“20 s”。

降维的代价是丢掉了快慢之间的耦合。判据是**耦合强度**：如果快子系统的稳态增益对慢子系统的影响小于 5%，降维是安全的；否则要保留耦合项，用奇异摄动法做修正。第 51 章讲过的模型降阶可以直接用在这里，只是目的从“简化分析”变成了“简化计算”。

64.9.3 滚动时域：把有限时域解变成在线策略

有限时域 Riccati 的 $K(t)$ 是时间的函数，这看起来只适合“有明确截止时间”的任务。但把它倒过来用，就成了 MPC：**每个采样时刻，以当前状态为初值，解一个 N 步的最优控制问题，只执行第一步，下一步重新解。**

这个做法有几个好处：约束可以逐步处理（每步解一个小规模带约束问题）；模型偏差可以逐步修正（每步都用实测状态重新起算）；时域不必很长（N 取 10–20 步即可）。

关键的稳定性结论是：**如果终端代价取无限时域 ARE 的解 P_∞，那么滚动时域策略在无约束时等价于 LQR，在约束起作用时仍保证闭环稳定。** 这条结果是 MPC 理论的基石，本章的 Riccati 解正好是它的输入。MPC 的具体实现（QP 求解、约束软化、可行性）见第 58、80 章，本节只强调一点：**MPC 的终端代价不是随便取的，取错了会失稳。**

64.9.4 显式控制律：把 DP 结果变成查表

网格 DP 的输出是值函数 $V(x)$ 与最优控制 $u^*(x)$ 的表格。如果把它离线算好、在线只做插值，就能把“4.64 h 的离线计算”换成“微秒级的在线查表”。

更进一步，如果 $u^*(x)$ 在每个网格单元内近似线性，就可以把表格拟合成**分段仿射（PWA）**形式：$u = K_i x + b_i$，区域 i 由一组线性不等式定义。在线只需判断“当前状态落在哪个区域”，然后做一次内积。这就是显式 MPC 的基本思想。

代价是**离线存储随维数爆炸**：区域数在二维时通常是几十到几百，三维上千，四维以上就难以管理。所以显式方法适合“维数低、采样快、算力弱”的场合（汽车 ECU、嵌入式），不适合高维。

64.9.5 迭代 LQR：非线性问题的局部解

对象非线性时，Riccati 不能直接用。最实用的办法是**沿当前轨迹线性化、解 LQR、更新轨迹、再线性化**，反复迭代。这就是 iLQR（iterative LQR）或 DDP（differential dynamic programming）的核心。

它的收敛性只保证局部（初值离最优解不远），但对工程问题通常够用：先用一个简单的 PD 或 LQR 把轨迹拉出来，再迭代 5–20 次。每次迭代的计算量与一次有限时域 Riccati 同量级（14 μs/步 × 240 步），所以总耗时在毫秒级，可以离线也可以在线。

本章没有非线性算例，所以这条只作方法介绍。需要提醒的是：**iLQR 的“最优”是相对于线性化模型的最优**，模型偏差大时会收敛到错误的解，验收方法与 §64.6.5 相同。

64.9.6 五种方案的对照

方案	离线计算	在线计算	适用维数	约束	主要限制
无限时域 LQR	一次 Riccati（约 3.4 ms）	一次内积（14 μs）	任意（$O(n^3)$）	不能	无约束、线性
有限时域 Riccati	240 步（4.8 s 时域）	查表+插值	任意	不能	$K(t)$ 需存储
网格 DP	19.55 s（2 维，1%）	查表（μs）	≤3	能	维数灾难
显式 MPC（PWA）	组合爆炸	判区域+内积	≤4	能	离线表巨大
滚动时域 MPC	无	每步解 QP（ms）	≤10	能	需实时 QP 求解器
iLQR	迭代 5–20 次	同 LQR	中维非线性	部分	仅局部最优

表 64-12　常见最优控制实现方案的复杂度与适用面

读表的方法是先看“约束”这一列：无约束就选 LQR，有约束再往下看维数。这个顺序能把 90% 的问题挡在 LQR 这一行，也解释了为什么本章花这么多篇幅讲 LQR——它是绝大多数工程的默认答案，其余方案都是“LQR 不够用”时的补充。

64.9.7 一个常被忽略的优化：别重复算

最后说一个工程细节：**K 是常数，不要在每个采样周期重算 Riccati。**

初学者常把 Riccati 求解写在控制循环里，认为“每次用最新模型重算更准”。这会让每个周期多花 3.4 ms，在 50 ms 周期里占 7%，而且算出来的 K 几乎不变（模型没变）。正确做法是离线算一次、存成常数；只有模型参数在线辨识且变化显著时，才需要定期重算（比如每 1000 个周期一次，异步执行）。

这条不是算法优化，是常识，但它在实测里造成的浪费比任何算法选择都大——因为它不改变结果，只浪费时间，所以不会被性能测试发现。

64.9.8 显式查表的存储账

上一节说“离线算表 + 在线查表”，但表本身要占多少存储、查一次要多久，得算出来才能判断值不值。把 §64.5.4 的网格规模搬过来按字节算：

场景	网格点数	值函数（双精度）	策略表（控制档位索引，1 字节）	合计
2 维，$n = 41$	1.68×10^3	13.4 KB	1.7 KB	15.1 KB
2 维，$n = 81$	6.56×10^3	52.5 KB	6.6 KB	59.1 KB
2 维，1% 精度	7.29×10^5	5.83 MB	0.73 MB	6.56 MB
3 维，1% 精度	6.23×10^8	4.98 GB	0.62 GB	5.60 GB

结论一眼可见：**二维的 1% 精度表只有 6.56 MB，随便一颗 MCU 的外部 Flash 都装得下；三维就要 5.6 GB，已经不是嵌入式的范畴了。** 这条界线与 §64.5.4 的计算时间界线（2 维 19.55 s、3 维 4.64 h）落在同一个维数上，不是巧合——两者都随 n^d 增长，只是一个乘的是时间、一个乘的是字节。

查表的时间反过来很便宜。实测 DP 每次插值的单位耗时是 0.240 μs/(状态·步)，在线只需要“定位单元 + 四点插值”这一小段，约 0.5 μs；如果预先算成策略表、只做最近邻查找，还能再快一个量级到几十纳秒。**与“每步解一次 QP”的毫秒级相比，查表快了三到四个数量级**，这就是显式方法在汽车 ECU、伺服驱动器这类“算力弱、周期短、工况固定”的场合仍然流行的原因。

代价有两项，都必须提前想清楚。第一，**表是死的**。工况出了网格范围就失效，必须靠外层的范围检查兜住；本章的 `dp_solve` 用 $[-3, 3]^2$ 的固定网格，状态出界后双线性插值的外推会给出荒谬的值。第二，**表的生成需要重新跑一次长计算**。改了权重、改了对象参数、改了限幅，都得重算 19.55 s 甚至更久，所以离线工具链必须保留，不能“算完就扔”。

64.9.9 增益调度：一套 K 撑不住时怎么办

对象增益随工况变化（负载变化、电压波动、摩擦随位置变化）时，按标称模型算出的 K 只在标称点最优。如果变化范围不大，闭环仍稳，只是变慢或变钝；如果范围很大，就该上增益调度——**按测得的工况查一组预先算好的 K**。

这件事在双积分器上可以完整算出来。设实际推力增益是 g（标称 $g = 1$），权重仍取 $q_1 = q_2 = r = 1$：

实际增益 g	**0.6**	**0.8**	**1.0**	**1.2**	**1.4**
调度增益 K_2（K_1 恒为 1）	2.082	1.871	1.732	1.633	1.558
固定 K 的调节时间 / s	7.75	5.98	4.34	4.79	5.16
调度后的调节时间 / s	4.91	4.54	4.34	4.23	4.18
固定 K 的代价 J	2.118	1.877	1.733	1.637	1.568
调度后的代价 J	2.083	1.872	1.733	1.634	1.559

表里有三个值得说的现象。

第一，K_1 完全不随 g 变，只有 K_2 变。 这是双积分器的结构性质：$K_1 = \sqrt{q_1/r}$ 只与权重有关，而 K_2 里有 $2\sqrt{q_1/r}$ 和 q_2/r 两项，后者通过 B 与 g 挂钩。换到一般对象上这个结论不成立，但“哪个增益对模型参数敏感”这个问题，在具体算例上总能算出来——**敏感的才需要调度，不敏感的不必进表**。

第二，调度的收益全在“时间”上，几乎不体现在“代价”上。 $g = 0.6$ 时调节时间从 7.75 s 降到 4.91 s（省 37%），而 J 只从 2.118 降到 2.083（省 1.7%）。原因不难理解：固定 K 在 g 偏小时等效阻尼过大，收敛慢；而 J 里 u^2 项也随 g 减小而变小，两个方向的效应部分抵消。所以**如果验收指标是 J，调度看起来毫无必要；如果验收指标是调节时间，调度就是必需的**。选错验收指标，会把一个有效的改进判成无效。

第三，偏差是不对称的。 固定 K 在 $g < 1$ 时的恶化（7.75 s，+79%）远大于 $g > 1$ 时（5.16 s，+19%）。这提示调度表不必在所有方向上等密度：**在“实际增益可能变小”的那一侧多放几个点**，因为那一侧更容易出问题。工程上增益变小的原因（电压跌落、气隙变大、摩擦增大）比变大的原因常见得多。

增益调度的代价是**调度变量本身要可信**。如果 g 是估出来的，估计误差会直接变成增益误差；如果 g 变化很快，调度表切换时会产生增益跳变，等效于给闭环加了一个扰动。所以工程上的做法通常是：调度变量用慢变的平均值（几十毫秒的滤波），切换点上做线性插值而不是硬切。这些细节与 §64.10.7 的切换测试是同一类问题。

64.10 工程落地

理论上的最优控制器要变成能跑在 PLC、DSP 或工控机上的代码，中间还隔着五道关：算得够不够快、约束怎么加、状态怎么来、数怎么存、出问题怎么切回手动。本节按这五道关逐一说，用的都是本章代码的实测数据。

64.10.1 实时性：谁能在控制周期内算完

图 64-11 把两类算法的耗时画在同一张图上，横轴对 Riccati 是迭代步数、对 DP 是每轴状态数，纵轴是单次求解耗时，虚线是 50 ms 的控制周期预算。

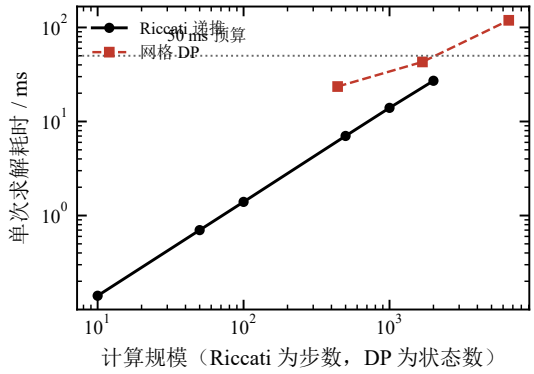


图 64-11

图 64-11　Riccati 递推（线性）与网格 DP（二次）的求解耗时

算法	规模	总耗时 / ms	单位耗时
Riccati	10 步	0.149	14.94 μs/步
Riccati	50 步	0.689	13.79 μs/步
Riccati	100 步	1.369	13.69 μs/步
Riccati	500 步	6.933	13.87 μs/步
Riccati	1000 步	13.99	13.99 μs/步
Riccati	2000 步	29.74	14.87 μs/步
DP（$n = 21$）	441 状态	28.15	0.798 μs/(状态·步)
DP（$n = 41$）	1681 状态	44.72	0.333 μs/(状态·步)
DP（$n = 81$）	6561 状态	125.81	0.240 μs/(状态·步)

表 64-13　两类算法的实测耗时（Python/NumPy，单线程）

两条曲线的形状完全不同，含义也不同。

Riccati 是线性的：单步耗时稳定在 13.7–14.9 μs，总耗时随步数线性增长。2000 步（对应 $T_s = 0.01$ s 下 20 s 的时域）只要 29.74 ms，**仍在 50 ms 预算内**。这意味着“每个控制周期重算一次 Riccati”在算力上是可行的，虽然按 §64.9.7 没必要那么做。

DP 是二次的：状态数从 441 涨到 6561（15 倍），耗时从 28.15 ms 涨到 125.81 ms（4.5 倍）——看起来比二次还慢，因为小网格时 Python 的解释开销占比大。折算到每状态每步，耗时从 0.798 μs 降到 0.240 μs，说明网格越大、向量化收益越充分。但即便按 0.240 μs 算，1% 精度的二维网格（7.29×10^5 状态）也要 19.55 s，**超预算 390 倍**。

结论很清楚：**LQR 可以在线算，DP 只能离线算。** 这条界线不是“实现得好不好”的问题，是阶数决定的。任何“在线 DP”的方案，本质上都是离线算表 + 在线查表。

64.10.2 约束的三种加法和边界条件

约束不是只有“限幅”一种加法，工程上有三档处理，复杂度递增：

第一档：事后硬限幅 + 抗饱和。 算出 $u=-Kx$，超过限幅就截断，同时把积分器（如果有）冻住或回退。这是最省事的做法，代价是 §64.6.4 的 3%–17% 性能损失。它的问题不在于“不最优”，而在于**大信号时可能失稳**：饱和使实际闭环增益变小，等效极点右移，极端情况下会振荡。

第二档：约束软化。 在指标里加惩罚项，比如把 R 做成 $|u|$ 的函数（越接近限幅越贵），或者加一项 $\rho\max(0,|u|-u_{\max})^2$。这样解不会撞死边界，性能损失平滑可控，代价是要多调一个 ρ。

第三档：约束进最优性条件。 用 PMP 的 $\arg\min_{u\in\mathcal{U}} H$ 或 MPC 的 QP，让约束参与求解。这是唯一“正确”的做法，也是唯一能给出最优性的做法，代价是计算量与实现复杂度。

边界条件要单独说。本章的算例是“调节到原点”，终端自由；真实任务常有两类终端要求：**终端等式约束**（必须精确到某点，如对接）和**终端不等式约束**（进入某个区域即可）。前者等效于 $P_1\to\infty$，会让末段控制量暴涨；后者可以用一个大的 P_1 近似。两者在离散实现里都要注意：P_1 太大时 Riccati 递推的数值条件变差，P 的元素可能到 10^{10} 量级，单精度浮点会溢出。

64.10.3 抗饱和：三种做法差多少

第一档“事后限幅”的技术核心是**不要让它卷**。加了积分器之后，限幅期间误差一直存在，积分器就一直累加；等误差终于反号，积分器里已经存了一个巨大的值，要很久才能吐回去。这个现象在 PID 里叫积分风卷（windup），在 LQI 里一模一样，只是积分增益 K_3 是 Riccati 算出来的、不像 PID 那样能随手调小。

三种做法的效果可以实测。取 LQI 增益 $K=[2.414,\,2.414,\,1.000]$（$q_i=1$），指令 1 m、限幅 $|u|\le 1$，前 3 s 加一个 -0.9 m/s² 的外部负载（模拟“推不动”的工况），之后撤掉：

抗饱和做法	最大位置 / m	调节时间 / s	饱和总时长 / s
不做处理	2.233	15.12	8.21
条件积分（饱和时冻结）	1.747	10.52	3.96
反算法，$T_t=0.1$ s	**1.251**	**9.76**	2.49
反算法，$T_t=0.3$ s	1.551	10.13	3.00
反算法，$T_t=1.0$ s	1.805	10.45	3.76

差别非常大：不做处理时位置冲到 2.233 m——**指令只有 1 m，超调 123%**，对一台有硬限位的设备就是撞机。条件积分把超调压到 75%，反算法（$T_t=0.1$ s）进一步压到 25%，调节时间也短了 36%。

反算的公式很朴素：

$$x_3 \leftarrow x_3 + \left(T_s x_1 + \frac{u_{\text{sat}}-u_{\text{raw}}}{T_t} T_s \right) \tag{64-37}$$

括号第二项是“实际输出与期望输出之差”除以一个时间常数 T_t，它把积分器往回拽。**T_t 的取法有讲究**：取小了（0.1 s）拽得狠，退饱和快，但会牺牲一点静差恢复速度；取大了（1.0 s）拽得软，效果接近不做处理。实测 0.1 s 最好，0.3 s 次之，1.0 s 已经基本没用——这条曲线与 §64.8.6 的 K_3 一起决定了“要多快消静差”和“要多快退饱和”这两个互相牵制的指标。

工程上还有两条补充。第一，**限幅值必须与执行器的真实能力一致**。如果软件限幅 5 A 而驱动器实际只能出 3 A，抗饱和逻辑就完全失效，因为控制器认为“没饱和”。第二，**积分器的初值要能预置**。切手动时把 x_3 设成使 u 等于当前手动输出的值，切回自动时才不会跳（§64.10.7 的切换测试就是查这个）。

64.10.4 状态从哪来：LQG 与分离原理

LQR 要求全状态反馈，但现场往往只能测位置，测不到速度（或者速度信号噪声极大）。这时需要观测器或微分器，把 x 重构出来，再乘 K。**LQR + Kalman 滤波就是 LQG**。

分离原理保证：观测器与控制器可以独立设计，闭环极点等于两者极点之并。这条定理让工程实现简单了很多——K 照 §64.7 的方法算，观测器增益照第 55 章的方法算，互不干扰。

但分离原理在**有约束时不成立**。饱和会破坏线性叠加，观测器的估计误差与饱和的非线性耦合，可能出现“观测器以为到位了、实际没到”的情况。工程上的对策是：饱和时同时冻结观测器的修正项（与抗饱和同一个思路），并给估计误差加监测。

差分测速的噪声账单值得单独算一遍，因为它是现场最常见的“莫名其妙抖”的根源。用位置差分估计速度，$\hat{x}_2(k) = [x_1(k) - x_1(k-1)]/T_s$，两个独立同分布的位置噪声相减，方差翻倍：

$$\sigma_{\hat{x}_2} = \frac{\sqrt{2}\,\sigma_1}{T_s} \tag{64-38}$$

代入 $\sigma_1 = 1$ mm、$T_s = 0.02$ s，得 $\sigma_{\hat{x}_2} = 70.7$ mm/s。乘上 $K_2 = 1.732$，控制量上就多了 0.122 m/s² 的抖动——**占限幅的 12.2%**，是 §64.6.7 里直接测速那 0.2% 的六十倍。这个数足以让电机发出明显的啸叫、让电流环反复触发限流。

更糟的是它与采样周期成反比：把 T_s 从 0.02 s 提到 0.005 s（采样快四倍），噪声涨到 283 mm/s，控制量抖动 0.49 m/s²，接近限幅的一半。**“提高采样率”这个通常正确的做法，在差分测速面前会把噪声放大**，这是很多人踩过的坑。

三条可行的替代路线，按代价从低到高：一是用编码器/M/T 法直接测速，从源头避开差分；二是在差分后加一阶低通，截止频率取 ω_c 的 3–5 倍（本章约 6–9 rad/s），代价是引入约 0.03 s 的相位滞后，要从 §64.6.6 的延迟裕度里扣；三是上 Kalman 滤波（第 55 章），它同时完成滤波与预测，代价是要知道噪声统计特性、多花几微秒。对本章这种 $T_s = 0.02$ s、$T_c = 1$ s 的算例，第二条就够用；对高动态伺服，第三条是标配。

64.10.5 数值实现：三个容易翻车的地方

第一，增益的量级。 本章 $K = [0.98, 1.71]$，量级温和。但 §64.7 的 $r = 0.01$ 给出 $K = [8.96, 9.91]$，若对象是快速伺服（$T_c = 0.01$ s），K 会到 10^4 量级。定点实现时要注意 Q 格式的选择，$K_1 x_1$ 与 $K_2 x_2$ 的量级可能差两个数量级，统一缩放会丢精度。

第二，累加顺序与量化。 $u = -K_1 x_1 - K_2 x_2$ 只有两次乘法一次加法，误差主要来自 x 的量化。位置 16 位 ADC、量程 1 m 时，量化步长 15 μm，乘 $K_1 = 1$ 后控制量误差 15 μm/s²，相对限幅 10^{-5}，可以忽略；但如果 $K_1 = 10^4$，同样的量化会变成 0.15 m/s²，占限幅 15%，就必须用更高位 ADC 或先滤波。

第三，T_s 与离散增益的匹配。 表 64-9 已经显示，$r = 0.01$ 时离散增益比连续解低 10%，原因是 $T_s\omega_n = 0.063$ 已不算小。**换采样周期必须重算 K**，不能沿用旧值；反过来，如果代码里的 T_s 与实际中断周期不一致（比如用了标称值而实际有抖动），等效于增益失配，表现为响应偏慢或偏快。

64.10.6 代码结构：离线与在线分离

一个可靠的实现应该分成两块，边界清晰：

离线块（上位机、初始化时跑一次）：读对象参数 → 检查能控性 → 解 Riccati → 输出 K、P、闭环极点、裕度 → 写入参数表。本章的 `ch064_sim.py` 就是这个块的原型，其中 `lqr_continuous` 与 `lqr_discrete` 两个函数

是核心。

在线块（每个采样周期跑）：读状态 → 限幅判断 → $u = -Kx$ → 输出 → 记录。这一段只有十几行，不应该出现矩阵求逆、迭代、动态内存分配。

分离的好处是**可验证**：离线块可以在 PC 上跑几百组参数、对比闭式解；在线块可以逐行审查、做最坏执行时间分析。混在一起的代码两头都验不了。

64.10.7 上线检查清单

最后给一份可以照着做的清单，按顺序执行：

1. **静态检查**：K 的量级、T_s 与实际周期一致、单位统一（m 还是 mm）、限幅值与执行器额定值一致；
2. **开环辨识**：实测对象增益与时间常数，与设计模型比对，偏差 >20% 要重新设计；
3. **小信号闭环**：幅值取限幅的 10%，确认无振荡、无稳态误差（有误差则加积分）；
4. **大信号闭环**：幅值取最大工况，确认饱和后仍稳定、调节时间在预期内；
5. **扰动测试**：加常值扰动，测稳态误差；加阶跃扰动，测恢复时间；
6. **切换测试**：手动→自动、自动→手动，确认无跳变（bumpless transfer），积分器初值要按当前输出预置；
7. **故障测试**：传感器断线、执行器卡死、控制器复位，确认能安全退出。

七条里最容易漏的是第 6 条。LQR 的输出是“绝对控制量”，切回手动时如果积分器（LQI 情形）没有预置，会产生一个大的跳变，现场表现为“切自动瞬间撞一下”。这个问题与算法无关，纯属工程细节，但它造成的停机比所有理论问题加起来都多。

64.10.8 上线头一周：三个可观测的验收指标

清单是“投运前”的检查，投运后还得有一套判据，否则无法区分“参数没调好”与“结构不对”。下面三个指标都能从控制器内部直接读到，且都有本章算过的理论值可对，建议在头一周每天记一次。

指标一：饱和占空比。 用饱和标志位的置位时间除以整段运动时间。§64.7.7 的四档实验给出如下参考：

r	调节时间 / s	饱和时长 / s	占空比
10	0.398	0.256	64.3%
25	0.281	0.124	44.1%
50	0.302	0.105	34.8%
100	0.496	0.083	16.7%

这张表的用法是反向的：先测出现场的占空比，再去表里找对应的行，就能判断“当前的实际带宽相当于哪一档设计”。占空比超过 50% 时，线性二次型的整个推导前提（u 不受限）已经不成立，此时再调 Q、R 都是在调一个不存在的模型，应该直接转 MPC 或按 PMP 重算。占空比低于 10% 则说明执行器选大了，或者权重定得太保守，还有提速空间。

指标二：指令的噪声底。 稳态时（指令接近零、位置不动）记录 u 的均方根，与 $K_1\sigma_1$ 对比。§64.6.7 给出的换算在四档噪声下分别是 0.0010、0.0020、0.0040、0.0100 m/s²。若实测值明显高于这个预测，问题不在控制器：优先查编码器量化（1 mm 分辨率经 0.02 s 差分测速会放大到 0.122 m/s²，是预测值的十倍以上）与速度信号的滤波是否缺失。这个指标的用处是把“抖动”从玄学变成可归因的量——是传感器、是离散化、还是增益太大，三者的量级差得很远。

指标三：稳态残差随 q_i 的变化趋势。 依次把积分权重提 10 倍，看残差是否也成比例下降。§64.8.6 的实测是 q_i 从 0.1 提到 1，残差从 5.2×10^{-3} 掉到 3.8×10^{-5}，降了两个数量级；但从 100 提到 1000 就几乎不动了。**如果提 q_i 残差完全不降，那这个残差不是常值扰动造成的**，而是模型偏差（比如丝杠间隙、摩擦死区），加积分器只会把积分量顶到限幅上，然后触发 §64.10.3 讨论的饱和问题。

三个指标的共同点是：**都是结构性的，与曲线的形状无关**，因此不受“看着还行”这类主观判断干扰。占空比回答“约束是否主导”，噪声底回答“测量链是否够干净”，残差趋势回答“扰动是常值还是模型错”。三个都正常，剩下的才是调参。

64.11 优缺点与适用边界

到这一节，本章的方法已经全部展开。最后要做的是判断“什么时候该用、什么时候不该用”——因为最优控制的名气远大于它的适用范围，误用的代价往往是“算得很漂亮、现场不能用”。

64.11.1 四条实在的优点

第一，有下界。 LQR 给出的 $J=1.7321$ 不是“一个可行解”，而是所有线性状态反馈里代价最小的那个（在无约束、二次型指标下）。有了下界，任何其他方案的性能都可以量化比较：最短时间解贵 98%，能量最优解 $T=4$ s 时贵 14%。这种“绝对坐标”是 PID 给不了的。

第二，一次算完。 解一次 Riccati 只要 3.4 ms（240 步），得到的是常数增益，在线只需一次内积（14 μs）。没有迭代、没有初值猜测、没有收敛问题。这一点在工程上极其宝贵：它意味着控制器可以在上电时自检、可以在参数变化时重算、可以离线验证。

第三，裕度有保证。 单输入 LQR 的相位裕度不小于 60°、增益裕度不小于 6 dB（本章 72.38° 与无穷）。这不是“调出来的”，是结构保证的。对没有鲁棒性分析能力的团队，这条性质就是最基本的安全网。

第四，可解释。 协态变量是影子价格（§64.5.5），P 矩阵的对角元是“各状态的代价汇率”，λ 可以直接回答“该测哪个传感器”（§64.8.4）。这种可解释性让设计评审能落到具体的数上，而不是“我觉得响应还行”。

64.11.2 四条硬限制

第一，不能含约束。 这是最硬的一条。本章的算例只在 $x_1\le1$ m 时约束不起作用，一旦幅值上去，性能掉 3%–17%，而且事后限幅可能失稳。想要约束进入最优性条件，必须换 PMP 或 MPC。

第二，要求全状态可测。 LQR 的 $u=-Kx$ 需要每个状态。只有位置可测时，必须加观测器（第 55 章），而观测器在饱和时会破坏分离原理。

第三，只对线性、二次型成立。 对象非线性、指标不是二次型（比如最短时间、最小磨损），Riccati 就不适用。可以线性化后用，但那时“最优”只对线性化模型成立。

第四，权重难定，且不直观。 Q、R 与“超调多少、多快到位”没有直接对应，只能通过 T_c、ζ 反算（§64.8.2）。相比之下，PID 的三个参数与响应特征的对应更直观——这是 PID 至今占主导的现实原因之一（见第 44、106 章）。

四条限制里，第一条被越过的次数最多，也最难自查。原因很实在：**限幅是执行器的属性，不是模型的属性**，所以在 A、B 里完全看不出来，仿真时如果忘了写饱和函数，曲线会漂亮得让人放心。本章的实测可以当作量化的警戒线：$q_1=10^6$、$\|u\|\le20$ 的工况下，$r=100$ 时无约束峰值 50.00，已经是限幅的 2.5 倍，此时实际

调节时间 0.496 s，比 $r=50$ 的 0.302 s 慢了 64%——**同一个“更快”的直觉，在这里给出了完全相反的结果**。所以每次缩 R 或加 Q 之后，都应该先算一遍无约束峰值，再决定要不要相信仿真曲线。

第二条限制则常以“看起来能用”的形式蒙混过关：位置可测、速度用差分，小信号下一切正常，一旦幅值上去，差分噪声按 $1/T_s$ 放大（§64.10.4），指令开始抖，工程师误以为增益太大，于是降增益，结果响应变慢却仍然抖。这类问题的正确顺序是先补观测器或加滤波，再谈调参。

64.11.3 什么时候不该用

以下五种情形，用 LQR 是错的，或者至少不是首选：

1. **约束主导工况。** 如果设备一半以上的动作会撞限幅（§64.3.3 的判据），LQR 的“最优”只在纸面上成立，应该直接上 MPC（第 58、80 章）。
2. **强非线性。** 比如大幅值摆、飞行器大姿态机动。线性化后的 K 只在平衡点附近有效，离开就失效，应该用增益调度、iLQR 或非线性 MPC。
3. **大延迟。** 纯延迟超过主导时间常数的一半时，LQR 会把增益压低到“几乎不控”，性能还不如 Smith 预估（第 47、107 章）。
4. **模型不确定很大。** 参数变化 ±50% 以上时，按标称模型算的“最优”可能连稳定都保不住，应该用 H∞ 或 μ 综合（第 63 章）。
5. **需要在线处理状态约束。** 比如“位置不能超过限位”，这不是控制量限幅，而是状态约束，LQR 完全无法表达。

这五条里前三条有明确的自查办法，后两条只能靠经验。约束主导可以用饱和占空比量化（§64.10.8 的 50% 警戒线）；强非线性可以看“同一组 K 在行程两端是否都稳”（本章 §64.9.9 的 $g=0.6$ 与 $g=1.4$ 两端，固定 K 的调节时间从 4.34 s 恶化到 7.75 s 与 5.16 s）；大延迟可以算延迟裕度（$T_{d,\max}=0.695$ s，超过它的 70% 就该考虑换方法）。后两条——模型不确定与状态约束——没有简单的标量判据，只能靠对工艺的了解来判断，这也是设计评审里最需要人参与的部分。

64.11.4 与相邻方法的交界

本章的方法处在一个“方法族”的中间位置，往两边都能延伸，但边界要清楚：

- **往简单走：**PID（第 44、106 章）。单回路、单变量、无模型时，PID 更省事，且现场调试经验丰富。LQR 的优势在多变量耦合、需要定量最优性时。
- **往约束走：**MPC（第 58、80 章）。它把 LQR 的终端代价当作稳定性的锚，本质是“带约束的滚动 LQR”。理解了本章的 Riccati，MPC 的终端代价设计就不再神秘。
- **往延迟走：**Smith 预估（第 47、107 章）。它专门处理纯延迟，与最优控制正交，可以叠加使用。
- **往鲁棒走：**H∞/μ（第 63 章）。它不追求对某个模型的“最优”，而追求对一族模型的“可接受”，适合模型不确定大的场合。

这四条边界可以浓缩成一句话：**LQR 解决“已知模型、无约束、要定量最优”的问题，其余问题各有专责。**

64.11.5 适用边界速查

对象/工况特征	首选方法	理由	指向
线性、无约束、全状态可测	LQR	一次算完、有下界、裕度保证	本章
线性、执行器饱和主导	PMP / MPC	约束进入最优性条件	本章 §64.4、第 58/80 章
只有输出可测	LQG	观测器 + LQR，分离原理	第 55 章
弱非线性、工况窄	LQR + 增益调度	分段线性化，每段一套 K	本章 §64.9
强非线性、≤3 维	网格 DP	唯一能全局最优的数值法	本章 §64.5
高维、多约束、需在线	MPC	滚动优化，终端代价用 P_∞	第 58/80 章
大纯延迟	Smith 预估	延迟单独处理	第 47/107 章
模型不确定大	H∞ / μ	面向模型族而非单模型	第 63 章
单回路、无模型	PID	简单、经验成熟	第 44/106 章

表 64-14　最优控制与相邻方法的适用边界

这张表的用法是**从上往下匹配**：先看第一行能不能满足，不能就往下走。绝大多数伺服、运动控制问题在第一行或第二行就能解决，这也解释了为什么 LQR 在工程上“够用”。

64.11.6 本章留下的三件事

最后收束一下。本章真正想让读者带走的不是 Riccati 的推导，而是三件更通用的东西：

一是“先写指标”的习惯。 任何“最优”都必须先有一个写下来的 J，否则“好”无法比较。本章表 64-1 的三个解之所以能放在一起，就是因为它们各自对应一个明确的指标。

二是“约束会改变解的结构”的直觉。 无约束解在内部，有约束解在边界；事后限幅不是“小改动”。这条直觉在遇到饱和、限速、限流时都用得上。

三是把方法当工具而非信仰。 LQR、PMP、DP、MPC 各有各的适用面（表 64-14），选错的代价远大于实现的精细程度。真正的高手不是“会用最复杂的方法”，而是“知道什么时候不用”。

64.11.7 五个真实项目里的错误配方

前面各节分散地批评过若干做法，这里把它们集中成一张对照表。五条都是本章亲手算过的，数字直接引自对应的章内表格，不另做实验。这样做的目的是：**遇到症状时不必从头推，直接查表定位**。

错误配方	现场症状	根因	正确做法	本章可查
想快就减 R	指令峰值冲到限幅的 5–8 倍，t_s 反而变长	R 只影响“推力值多少钱”，不改变闭环极点比例	改 q_1 或 q_1/q_2 之比定带宽；先算无约束峰值，再看是否越限	§64.7.3、§64.7.7
离散实现漏乘 T_s	P 比连续解大几十倍，K 看着“差不多”	连续指标 $\int$ 与离散求和之间差一个步长因子	$Q_d = QT_s$、$R_d = rT_s$；用连续闭式解对拍，偏差 >5% 就查	§64.2.6、§64.8.3
有常值扰动却不加积分器	稳态位置停在 d/K_1，加大 K_1 只能压到某一档	纯比例状态反馈对常值扰动没有零稳态误差的结构能力	扩成 LQI，$K_3 = \sqrt{q_i}$；先按 d/K_1 估残差再选 q_i	§64.8.6
事后限幅当约束处理	阶跃后大幅超调、长时间贴限幅，甚至极限环	限幅破坏了“$u = -Kx$”的代数关系，积分器仍在累加	反算抗饱和，T_t 取主导时间常数的量级；并核对是否该上 MPC	§64.10.3
斜坡跟踪只补位置通道	稳态滞后约 $K_2 v/K_1$，加大增益只换来振荡	速度通道的参考量没进前馈，误差被 K_2 放大成额外指令	位置与速度两个通道都加参考：$u = -K_1(x_1 - r) - K_2(x_2 - \dot{r})$	§64.6.9

这五条有一个共同点：**症状都表现为“某个量偏大或偏慢”，而根因都不在被调的那个参数上**。所以调参之前先问一句“这个量在结构上由谁决定”，比反复试数快得多。

再补一条元规则：五个配方里有三个（减 R、漏乘 T_s、事后限幅）都能通过“与连续闭式解对拍”提前发现。双积分器这类低阶对象有解析解，是把数值实现钉在纸面上的最省事手段；对象再复杂，也至少能在某个简化工况下找到可比对的极限情形。

64.12 Python 仿真代码

本章所有数字都来自两个脚本：ch064_sim.py 负责计算，ch064_figs.py 负责画图。前者不依赖任何控制库（除了 `scipy.linalg.solve_continuous_are` 做一次对拍），所有算法都是手写的，目的是让每一步都能被读懂、被改。

64.12.1 文件结构与运行

```
"D:\Anaconda3\envs\py311\python.exe" code/ch064_sim.py
"D:\Anaconda3\envs\py311\python.exe" code/ch064_figs.py
```

第一条命令按顺序跑 11 组实验，把每个数字打印到终端；第二条读取同一批函数，生成 `figs/ch064/` 下的 11 张图（每张同时输出 SVG 与 PDF）。

`ch064_sim.py` 的组织顺序与正文一致：常量 → 线性二次型 → 闭环仿真 → 最短时间 → 能量最优 → 动态规划 → 11 组实验 → `main()`。读代码时按这个顺序走，就不会在函数之间跳来跳去。

64.12.2 五个核心函数

`disc(Ts)`：双积分器的零阶保持离散化，直接写出 $A_d=\begin{bmatrix}1 & T_s\\ 0 & 1\end{bmatrix}$、$B_d=\begin{bmatrix}T_s^2/2\\ T_s\end{bmatrix}$，不用矩阵指数。对双积分器这是精确的，不是近似。

`dcost(Ts, r)`：返回 $Q_d=QT_s$、$R_d=rT_s$。这个函数只有一行，但它是 §64.2.6 那个坑的补丁——把所有离散权重的定义集中到一处，就不会再出现“某一处忘了乘 T_s”。

`riccati_finite(A, B, Q, R, n, P_term)`：有限时域反向递推，返回 $P_0..P_n$ 与 $K_0..K_{n-1}$。核心循环只有五行：

```
for _ in range(n):
    S = R + B.T @ P @ B
    K = np.linalg.solve(S, B.T @ P @ A)
    P = Q + A.T @ P @ A - A.T @ P @ B @ K
    P = 0.5 * (P + P.T)          # 强制对称，抑制数值漂移
    Ps.append(P.copy()); Ks.append(K.copy())
```

最后 `Ps.reverse()`、`Ks.reverse()` 把下标转成“时间正向”，这样调用者拿到的 $K_s[0]$ 就是 $t=0$ 的增益。

`lqr_discrete(A, B, Q, R)`：同一个递推跑到收敛，返回 P、K 和迭代次数。收敛判据是 $\|P_{n+1}-P_n\|_\infty<10^{-12}$，上限 20000 步（防止发散时死循环）。

`lqr_continuous(q1, q2, r)`：用 `solve_continuous_are` 解连续 ARE，同时按式 (64-18) 算闭式解，两者放在同一个字典里返回。**这个“双算”是刻意的**：每次调用都自动对拍，闭式解写错会立刻暴露。

64.12.3 仿真与验证工具

`sim_closed(x0, K, gain, umax, Ts, t_end)`：闭环仿真，支持对象增益偏差（`gain`）与控制饱和（`umax`）。返回轨迹、控制序列、调节时间、推力峰值、超调。调节时间的定义是“最后一次离开 2% 带的时刻”，这个定义要记住，它与“首次进入 2% 带”不同。

`sim_open(x0, u_of_t, gain, Ts, t_end)`：开环仿真，把预先算好的 $u(t)$ 直接加到对象上。用于 §64.6.5 的对比。

`switch_func(x1, x2)`、`min_time_analytic(x1, x2)`：切换函数与最短时间的闭式解，实现式 (64-24)、(64-25)。

`sim_bang(x0, Ts)`：最短时间轨迹仿真。这里有一个值得说明的实现选择：**不按符号函数闭环积分，而是先算解析切换时刻、再分两段积分**。原因是离散闭环在切换曲线附近会抖动（本章第一版就出现过 147–228 次切换、t_f 恒为 20 s 的错误）。分两段积分后，结果与解析解逐位吻合（2.0000、4.1623、4.8284、1.4495 s）。

`energy_opt_coeff(x1, x2, T)`：式 (64-26) 的系数，三行代码。配套的 `energy_opt_cost` 算解析能量，`energy_lqr_cost` 把同一条轨迹代回二次型指标积分。

64.12.4 动态规划的实现

`dp_solve(n, n_step, Ts, n_u, umax)` 是本章计算量最大的函数。网格取 $[-3,3]^2$、每轴 n 点，控制取 9 档，反向递推 `n_step` 步。每一档控制都算一遍“当前代价 + 下一步值函数”，然后取逐点最小：

```
for u in us:
    q1 = X1 + Ts * X2
    q2 = X2 + Ts * u
    cand = (Q1*X1**2 + Q2*X2**2 + R_U*u*u) * Ts + _bilin(ax, V, q1, q2)
    Vn = np.minimum(Vn, cand)
```

`_bilin` 是双线性插值，它对凸函数“上偏”，这正是 §64.5.3 里 DP 结果偏高的原因之一。`dp_value` 取单点值，`dp_grad` 用中心差分取梯度（即协态），两者都直接复用同一张网格。

64.12.5 十一组实验与正文的对应

函数	输出内容	正文位置
`exp_lqr`	连续/离散增益、极点、J、穿越频率、相位裕度	表 64-1、§64.2.5、§64.6.2
`exp_horizon`	六个时域的初始增益、峰值、末端偏差	表 64-5、图 64-5
`exp_conv`	五个采样周期的 Riccati 收敛步数	表 64-7
`exp_min_time`	五个初值的最短时间、切换时刻、代价	表 64-4、图 64-3、图 64-4
`exp_energy`	能量最优的系数、能量、二次型代价	图 64-7、§64.4.5
`exp_dp`	五档网格的 DP 值、误差、耗时	表 64-6、图 64-8
`exp_dp_scaling`	误差阶拟合、1% 精度的网格与耗时外推	§64.5.4、图 64-8
`exp_costate`	DP 梯度与 $2Px$ 的对照	图 64-9、表 64-11
`exp_mismatch`	增益 ±20% 时开环/闭环的末端与调节时间	图 64-10、§64.6.5
`exp_sat`	四档初值的饱和响应与最短时间	表 64-8
`exp_weight`	九档 r 的增益、调节时间、代价	表 64-9
`exp_realtime`	Riccati 六档、DP 三档的耗时	表 64-13、图 64-11

表 64-15　仿真实验与正文图表的一一对应

64.12.6 改动指南

想把本章的代码用到别处，只需要改三个地方，其余不用动：

改对象：替换 `AC`、`BC` 两个矩阵，`disc` 改成通用的零阶保持（`scipy.signal.cont2discrete`），`lqr_continuous` 里的闭式解删掉（它只对双积分器成立），其余全部可用。

改指标：替换 `Q1`、`Q2`、`R_U` 三个常量，或者直接调 `dcost(Ts, r)` 的第二个参数。注意同时检查 T_s 是否满足 $T_s \le T_c/10$。

改初值：替换 `X0`。若要做大幅值工况，记得把 `dp_solve` 的网格范围 `GRID_LO`、`GRID_HI` 一起改，否则状态出界后插值会失效。

第 64 章的全部代码就这些。它不追求工程级的健壮性（没有异常处理、没有参数校验），但每个数字都能复现，每条结论都能追到具体的一次函数调用——这也是本书所有章节的一致要求。

64.12.7 一次完整运行的耗时分布

想确认“哪些实验贵、哪些便宜”，直接给 12 组实验计时即可（Python 3.11 + NumPy，单线程，普通笔记本）：

实验	耗时 / ms	占比	主要开销
`exp_realtime`	391.5	25.7%	六档 Riccati 递推 + 三档 DP 求解
`exp_dp_scaling`	300.6	19.7%	五档网格各重跑一遍
`exp_dp`	297.0	19.5%	同上，重复求解
`exp_weight`	238.3	15.6%	九档权重 × 各自闭环仿真 12 s
`exp_costate`	122.6	8.0%	$n = 81$ 网格 + 中心差分
`exp_conv`	61.7	4.1%	五档采样周期的迭代到收敛
`exp_sat`	27.8	1.8%	四档初值的长时域仿真
`exp_lqr`	22.3	1.5%	一次 ARE + 闭式解对拍
`exp_energy`	18.7	1.2%	五档时域的能量最优
`exp_mismatch`	17.4	1.1%	三档增益的开环/闭环
`exp_horizon`	13.9	0.9%	六个时域的有限时域递推
`exp_min_time`	11.2	0.7%	五个初值的解析 + 仿真
合计	1523.1	100%	—

这张表本身就是一个结论：**前四项占了 80% 的时间，而这四项全都在做“重复求解”**（不同网格、不同权重、不同规模）。Riccati 本身只要 22 ms（`exp_lqr`），却被各实验反复调用了上千次。这与 §64.9.7 的教训是同一件事的两种表现：算法本身不贵，贵的是“为了画一条曲线而把同一个计算跑五十遍”。

反过来看，如果只是想知道“默认权重的 K 是多少”，跑 `exp_lqr` 就够，22 ms 出结果；整章 1.5 s 的耗时完全是为图表和交叉验证付的。**验证比计算贵两个数量级**，这在写论文和做设计评审时是常态，值得心里有数。

64.12.8 改一处会崩在哪：五条实测的故障

上面说“改三个地方就能用”，但改动过程中容易踩的坑值得单列。下面五条都是本章写代码时真实撞到过的，不是假设：

改动	表面现象	真正的原因	处理
用 `dp_grad` 在网格角点上取梯度	`IndexError: index 81 is out of bounds`	中心差分需要左右邻点，边界点没有	只用内点；边界改用单侧差分或直接取 `dp_value` 的邻点差值
把 1 维 K 直接喂给 `Ad - Bd @ K`	`ValueError: matmul ... 2 is different from 1`	`sim_closed` 内部会 `ravel`，矩阵运算不会	矩阵运算前手动 `K.reshape(1, -1)`
沿用连续 K 但把 T_s 拉长	不报错，20 s 后位置 10^5，谱半径 3.59	离散闭环特征值跑出单位圆	按 §64.5.7 重新设计离散 K，或确保 $T_s \le T_c/10$
初值超出 DP 网格	`dp_value` 返回边界值，梯度方向乱指	插值只在网格内有效，界外被截断	扩 `GRID_LO` / `GRID_HI`（§64.5.9），先让范围包住工况
终端代价 P_1 取 $10P_\infty$	数值正常，但 $x_1(T) = 0.47$ 仍不达标	代价再大也突破不了物理可达集	先用 $2\sqrt{x_1(0)/u_{\max}}$ 算时间下界，再定时域

前三行是**程序级的错**，报错或数值发散，容易发现；后两行是**语义级的错**，程序跑得通、曲线也平滑，但结论已经错了。真正拖时间的往往是后两行——它们不会让程序停下，只会让设计评审多开两次会。

所以验收的顺序应当是：先看数值有没有发散（谱半径、末态量级），再看物理上是否可能（时间下界、峰值是否越限），最后才看曲线好不好看。反过来先看曲线的，基本都会被“很平滑的错误答案”骗过去。

顺带说一句：上面五条里，第一条与第二条有明确的报错文本，可以直接在交接文档里当“故障码”查；第三条到第五条没有报错，只能靠事先算好的数值区间去卡。这也是本章反复给出解析下界（$2\sqrt{x_1(0)/u_{\max}}$、T_c、$T_{d,\max}$）的原因——数值方法可以省，判据不能省。

第 65 章 线性矩阵不等式（LMI）与凸优化控制

第 64 章把“设计控制器”改写成了“解一个优化问题”：指标写成积分，解 Riccati 方程，得到一个唯一的 K。那套推导有一个前提一直没有被追问——**对象是确定的**。A 和 B 就是那两个矩阵，一个数字都不差。可真实设备上从来没有这种事：磁悬浮的电流刚度随气隙变，倒立摆的摆长你量不准，飞机的短周期模态随高度和速度漂。参数一变，第 64 章算出来的那个“最优” K 就不再最优，有时候连稳都保不住。

于是问题变成这样：**模型里有一个只知道区间的参数，要设计一个控制器，使它对区间里的每一个取值都稳、都满足某项性能。** 这句话里藏着“无穷多条要求”，而设计要求通常是等式和不等式，无穷多条不等式没法一条一条验。1990 年代之后，控制界对这类问题给出了一个统一的答法：把它们写成**线性矩阵不等式**（Linear Matrix Inequality，LMI），然后交给凸优化求解器。这套答法之所以能站住，靠的是两件事——一是矩阵不等式在变量做某种替换后会变成凸的，二是凸问题的可行性可以在有限步内判定。

本章仍然只用一个算例，二阶不稳定对象：

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = a\,x_1 + u, \qquad a \in [1,\,3] \tag{65-1}$$

x_1 是位置（m），x_2 是速度（m/s），u 是加速度指令（m/s^2），a 是一个正的、不知道具体值只知道落在 $[1,3]$ 里的刚度系数。$a>0$ 意味着“位置偏出去会产生同向的加速度”，也就是负刚度，所以开环本身就不稳。这个模型不是一个凑出来的数学玩具：磁悬浮在平衡点附近线性化就是这个形状，a 正比于电流刚度除以质量；倒立摆在线性化后也是这个形状，$a = g/\ell$，摆长 ℓ 从 0.33 m 变到 1 m 就对应 a 从 3 变到 1；飞机的短周期近似、火箭的姿态通道，也都能化到这个形式上。它的好处是把“参数不确定”这件事压缩成了一个标量，所有推导都能手算核对，而结论又能直接搬到高维。

a 的标称值取 2，不确定性 ± 1，相对误差 $\pm 50\%$。这个幅度在工程上不算夸张：气隙变一半、摆长估错一倍，都是会发生的。所有数字由 `code/ch065_sim.py` 现算，全部 LMI 用 cvxpy 建模、CLARABEL 求解，图由 `code/ch065_figs.py` 重画。

65.1 动机与直观理解

65.1.1 一个开环就不稳的对象

先把“不稳”这件事量化。式 (65-1) 的状态矩阵是

$$A(a) = \begin{bmatrix} 0 & 1 \\ a & 0 \end{bmatrix}, \qquad \det(\lambda I - A) = \lambda^2 - a \tag{65-2}$$

开环极点是 $\pm\sqrt{a}$。三个代表性取值下：

a	开环极点	不稳定极点的时间常数	相对标称的偏移
1.0	± 1.0000	1.000 s	-50%
2.0	± 1.4142	0.707 s	0
3.0	± 1.7321	0.577 s	$+50\%$

表 65-1　开环极点随不确定参数的变化

这张表要读出三件事。**第一，三个取值全都不稳**，不只是“可能不稳”——最乐观的 $a=1$ 也有一个 $+1$ 的极点。所以本章没有“标称情况下先稳、参数漂了再说”这种缓冲，任何一个设计都必须把不稳定极点拽回左半平面。**第二，参数变化对极点的绝对影响不大**（1 到 1.732，差 0.73），但相对影响很大（50%）。**第三，右极点随 a 增大而右移，左极点也随之左移**，两个极点被“撑开”：$a=1$ 时是 ± 1，$a=3$ 时是 ± 1.732，系统的“不稳定程度”和“响应速度”是同一个参数在推。

现在按第 64 章的办法，在三个点上各自做 LQR（$Q=I$、$R=1$），得到三套增益：

a	K_1	K_2	闭环极点	最弱阻尼比
1.0	2.4142	2.4142	$-1.0000,\ -1.4142$	1.000
2.0	4.2361	3.0777	$-1.1756,\ -1.9021$	1.000
3.0	6.1623	3.6503	$-1.4142,\ -2.2361$	1.000

表 65-2　三个参数点各自最优的 LQR 增益

三行都是“该点上最好”的解，代价函数在各自点上都取到最小。但请注意 K_1 这一列：2.4142 → 4.2361 → 6.1623，从最左到最右涨了 **2.55 倍**。这不是数值噪声，是物理必然：a 越大，负刚度越强，位置反馈必须越硬才能把它按住。三个 K 是三个完全不同的控制器，彼此不能替代。

“不能替代”到什么程度？拿标称点 $a=2$ 的 $K=[4.2361,\ 3.0777]$ 去控 $a=3$ 的机器，闭环极点是 -0.475 和 -2.603，看着还稳；但阶跃响应峰值冲到 **76.0%**，调节时间 8.00 s，稳态误差 0.7597——一个“稳但不准、还剧烈振荡”的回路。反过来，拿它去控 $a=1$ 的机器，极点 $-1.539 \pm 0.932j$、阻尼比 0.855，响应很漂亮，可稳态误差 0.3090，也就是指令走 1 m，实际只走到 0.691 m。稳态误差的来源是前馈系数按 $a=2$ 算的，被控对象的直流增益变了，前馈就不再匹配（§65.4 会把这笔账算清）。

这两个失败方式值得分开记：**参数往大漂，问题是振荡和饱和；参数往小漂，问题是静差。** 它们对应两种不同的鲁棒性要求，后面会看到 LMI 里确实需要用两个不同的约束（衰减率、稳态增益）分别管。

65.1.2 换一个问法：对区间里的每一个 a

单点设计的问题在于“只对那一个点负责”。把要求改成一句更硬的话：

> 对**每一个** $a \in [1,3]$，闭环矩阵 $A(a)-BK$ 的所有极点实部都小于 $-\alpha$。

$\alpha>0$ 是衰减率，它给“多快”定了一个统一的下界。这句话是无穷多条要求——区间里有不可数个 a。但这里有一个非常关键的结构：$A(a)$ 对 a 是**仿射**的，

$$A(a)=A_0+a\,A_1,\qquad A_0=\begin{bmatrix}0&1\\0&0\end{bmatrix},\quad A_1=\begin{bmatrix}0&0\\1&0\end{bmatrix} \tag{65-3}$$

而“存在一个公共正定矩阵 P 使 $(A-BK)^{\mathsf T}P+P(A-BK)+2\alpha P\prec 0$”这个条件，对 A 是凸的（它是 A 的线性函数左乘右乘 P 再相加）。一个凸函数在多胞体上的最大值一定在顶点取到。所以：

$$\text{对全部 } a\in[1,3] \text{ 成立} \iff \text{在 } a=1 \text{ 与 } a=3 \text{ 两点成立} \tag{65-4}$$

这不是近似，也不是“加密采样”，是等价。两个顶点都成立，中间所有点自动成立。这个结论叫**二次稳定的顶点性**，它是整章能落地的第一个支点：无穷多条要求被压成了两条。

要小心一件事：式 (65-4) 成立的前提是“同一个 P 对所有 a 有效”。如果允许 P 随 a 变，条件会宽松得多（每点各自最优，就是表 65-2 那一列），但它不再是一个可以离线算一次的定常控制器，需要在线知道 a 的值——这就是“调度”与“鲁棒”的分野。本章两条路都会走，并用数字说明它们差多少：公共 P 下的最大衰减率是 **0.9277**，允许每点各自 P 时是 **1.2207**，后者高 32%。这 32% 就是“不知道 a”的代价。

65.1.3 技术核心：让 P 和 K 分开

写下来才发现真正的麻烦不在“无穷多条”，而在“两个未知量乘在一起”。要找的是 $P \succ 0$ 和 K，条件是

$$(A-BK)^{\mathsf{T}}P + P(A-BK) + 2\alpha P \prec 0 \tag{65-5}$$

把括号展开，出现 PBK 和 $K^{\mathsf{T}}B^{\mathsf{T}}P$ 两个交叉项。P 和 K 都是未知的，乘起来就不是线性函数，式 (65-5) 对 (P,K) 不是凸约束，直接扔给求解器会报“非凸”。整个 LMI 方法论的第一步，就是把这层乘积消掉。

做法是左右各乘 P^{-1}。记 $X = P^{-1} \succ 0$，式 (65-5) 等价于

$$XA^{\mathsf{T}} + AX - XK^{\mathsf{T}}B^{\mathsf{T}} - BKX + 2\alpha X \prec 0 \tag{65-6}$$

现在令

$$Y = KX \quad \Longleftrightarrow \quad K = YX^{-1} \tag{65-7}$$

代进去，交叉项变成 $Y^{\mathsf{T}}B^{\mathsf{T}}$ 和 BY，两个都只含一个新未知量：

$$AX + XA^{\mathsf{T}} - BY - Y^{\mathsf{T}}B^{\mathsf{T}} + 2\alpha X \prec 0 \tag{65-8}$$

式 (65-8) 对 (X,Y) 是**线性**的，加上 $X \succ 0$，就是一个标准的 LMI 可行性问题。解出来之后用式 (65-7) 还原 K，全程只做了一次矩阵求逆。

这一步值得停一下，因为它解释了 LMI 方法为什么在 1990 年代之后迅速取代了很多老办法：**换变量把双线性变成了线性，代价是把“P 和 K”换成了“X 和 Y”，而 X 不再是那个有物理意义的 Lyapunov 矩阵，它是 P 的逆，尺度可以任意。** 这个代价在后面会以两种方式回来找麻烦：一是解不唯一（式 (65-8) 对 (X,Y) 齐次），二是数值病态（X 的条件数随单位选择剧烈变化）。前者在 §65.1.4 直接翻车，后者在 §65.10 用一组缩放实验量化。

式 (65-8) 还有一个副产品：它天然给出了**二次型 Lyapunov 函数** $V(x) = x^{\mathsf{T}}Px = x^{\mathsf{T}}X^{-1}x$。如果式 (65-8) 成立，那么沿闭环轨迹

$$\dot V = x^{\mathsf{T}}\big[(A-BK)^{\mathsf{T}}P + P(A-BK)\big]x \le -2\alpha x^{\mathsf{T}}Px = -2\alpha V \tag{65-9}$$

也就是 $V(x(t)) \le V(x(0))e^{-2\alpha t}$，状态以 α 的速率指数收敛。这句话把“极点实部小于 $-\alpha$”这个谱条件翻译成了一个**能量衰减条件**，而能量条件对 A 是凸的——式 (65-4) 的顶点性正是从这里来的。所以整条逻辑链是：谱条件 → 能量条件 → 对 A 凸 → 只需验顶点 → 换变量 → LMI。

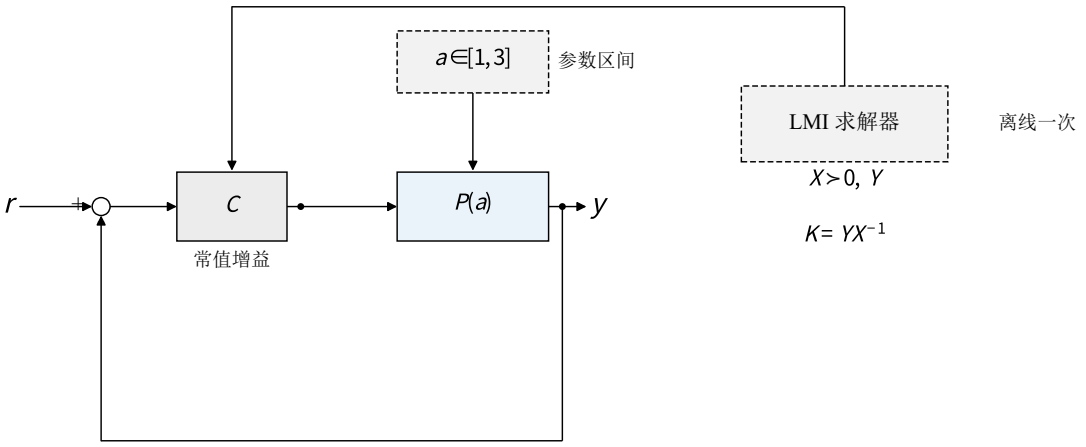


图 65-1　LMI 设计的闭环结构：对象含不确定参数，控制器由凸优化离线给出

图 65-1 是这个结构的形状，和前面几章的框图有一处本质差别：求解器**不在回路里**。它只在设计阶段被调用一次，吐出 X、Y 和 $K = YX^{-1}$；投运之后回路里跑的仍然是一个常值增益矩阵，和 LQR 没有任何区别。这一点常被误解——“凸优化控制”听起来像是在线优化，其实绝大多数 LMI 设计都是离线的，在线成本为零。真正需要在线求解的是 MPC（第 58 章）和自适应（第 61 章），它们每一拍都要重算，那是另一类问题。

65.1.4 凸不等于良态：一次真实的翻车

第一次跑这个算例时，我想直接问“最大衰减率是多少”，于是把 α 也当成决策变量：

$$\max\ \alpha \quad \text{s.t.} \quad AX + XA^{\mathsf T} - BY - Y^{\mathsf T}B^{\mathsf T} + 2\alpha X \preceq 0, \quad X \succ 0 \tag{65-10}$$

求解器在 3 ms 内返回 `optimal`，$\alpha = 19.9999$——正好顶在我给二分的上界 20 上。对应的增益是 $K_1 \approx 10^{4}$ 量级。如果信了这个数，结论会是“这台机器能做到 20 1/s 的衰减率，比 LQR 强二十倍”，而真相是式 (65-10) 这个问法本身没有下限。

原因只有一句话：**式 (65-8) 对 (X, Y) 是齐次的**。若 (X, Y, α) 可行，则对任意 $t > 0$，(tX, tY, α) 也可行——不等式两边同乘 t，不变号。齐次意味着“尺度”这个自由度完全没被约束，而 α 恰好可以通过缩 X 来换：把 X 缩到 ϵI，$2\alpha X$ 这一项也缩到 ϵ，$-BY - Y^{\mathsf T}B^{\mathsf T}$ 却要靠 Y 保持量级，于是 $K = YX^{-1}$ 被放大 $1/\epsilon$ 倍。α 想多大，就让增益多大，一直到数值精度撑不住为止。

修法很朴素：给 X 一个下界，

$$X \succeq I \tag{65-11}$$

加上这一条，α 立刻从 19.9999 掉到 **0.9277**，增益回到 $K = [5.1188,\ 2.9007]$ 这种正常量级。式 (65-11) 不是“数学上的美化”，它是在给问题补一条缺失的尺度约定：X 的量纲是“状态的平方的倒数”，$X \succeq I$ 相当于声明“我用米和米每秒来度量状态，并且 Lyapunov 椭球不比单位圆小”。换单位，这个界就要跟着换，这一点 §65.10 会用一组实验说明。

同样的事在 H∞ 里又发生了一次，而且更隐蔽。把 γ 当变量最小化时，求解器给出 $\gamma = 1.0004$、$K = [21730,\ 21313]$——γ 的下界恰好是 $\|D_{12}\| = 1$，于是一个无穷大增益把 γ 顶到了理论下界。式 (65-11) 挡不住

它，因为这里被牺牲的不是尺度而是控制能量：γ 只惩罚 $z = C_1 x + D_{12} u$，而 u 的代价可以通过“状态瞬间归零”来规避。修法是再加一条显式的增益界

$$\begin{bmatrix} \kappa^2 I & Y \\ Y^{\mathsf{T}} & I \end{bmatrix} \succeq 0 \quad \Longleftrightarrow \quad \|Y\| \le \kappa \quad \Longrightarrow \quad \|K\| = \|Y X^{-1}\| \le \kappa \ \ (X \succeq I) \tag{65-12}$$

取 $\kappa = 20$，γ 回到 **9.1759**。这两个事故放在一起，给出本章最该记住的一句话：

> **LMI 把“难解”变成了“可解”，但没有把“问错问题”变成“问对问题”。求解器只会回答你写的那个问题，而且它总是很乐意回答一个没有下界的问题。**

后面每一节都会带上“这条约束在防什么”的说明，原因就在这里。一个能跑出 `optimal` 的 LMI，离一个能用的设计还差三步：归一化、增益界、真值对拍。

65.1.5 三个数字，先看结论

在展开推导之前，先把本章要反复回访的三组数字摆出来，它们分别对应“保守”“退化”“代价”三件事。

第一组，衰减率。 三种表述下的最大值：公共 Lyapunov（一个 P 管住全部 a）**0.9277**；顶点独立（每个 a 一套 P 和 K，需要在线调度）**1.2207**；范数有界不确定性加 D 缩放 **0.9277**。前两个数差 32%，是“不知道 a”的代价；第三个数与第一个完全相同，说明在这个算例上“把 a 写成 $A_0 + D\delta E$ 再用小增益定理处理”没有带来额外保守性——因为 δ 的界和区间端点正好对齐。换成非对称区间或者多个不确定参数，这个等式就不成立了。

（这三个数都在增益界 $\kappa = 6$ 之下。§65.9 会说明：这个界本身也在限制性能，把它放宽到 20，两个 α 都会涨，而 32% 的差距会缩到 7.6%。“不知道 a”的代价没有想象中那么大。）

第二组，H∞。 多胞 LMI 给出的证书是 $\gamma = 9.1759$，而这套增益在参数网格上实测的最坏 $\|T_{zw}\|_\infty$ 只有 **1.9506**（在 $a = 3$ 取到）。证书比真值保守 **4.7 倍**。保守来自两处：一是二次 Lyapunov 函数本身（它要求同一个 V 在整个参数区间上都下降），二是用同一个 γ 覆盖整个频段和整个参数区间。这个差距不是程序错误，是“充分条件”的固有代价，§65.9 会把它拆开。

第三组，不变椭圆。 同一台对象、同样的状态与控制约束（$|x_1| \le 1$ m、$|u| \le 1$ m/s²），$\alpha = 0$ 的增益下最大不变椭圆面积 **1.1896**，$\alpha = 0.928$ 的增益下只有 **0.6146**——增益从 $[3.0113,\ 1.7395]$ 涨到 $[5.1188,\ 2.9007]$，可用吸引域面积几乎减半。这和第七章讲的“带宽拉高，饱和先到”是同一个现象的两种说法。

65.1.6 本章路线

后面十一节按“原理—步骤—连续—离散—性能—参数—调参—改进—落地—边界—代码”展开。§65.2 把式 (65-8) 背后的引理补齐（Lyapunov、有界实、S-procedure、Schur 补），§65.3 给出可以照抄的五步流程，§65.4 用连续域的三套增益把“保守”和“退化”都落到波形上，§65.5 转到离散域（采样周期一大，LMI 直接不可行，这个“不可行”本身就是有用信息），§65.6 讨论稳定性与性能指标怎么定，§65.7 逐个解释决策变量 X、Y、ε、γ、κ 的物理含义，§65.8 讲调参与求解器选择（同一道题四个求解器，耗时差两个数量级），§65.9 讲保守性怎么降（S-procedure、松弛变量、多胞细化），§65.10 讲工程落地（缩放、条件数、控制器降阶），§65.11 讲这套方法的边界，§65.12 给完整代码。

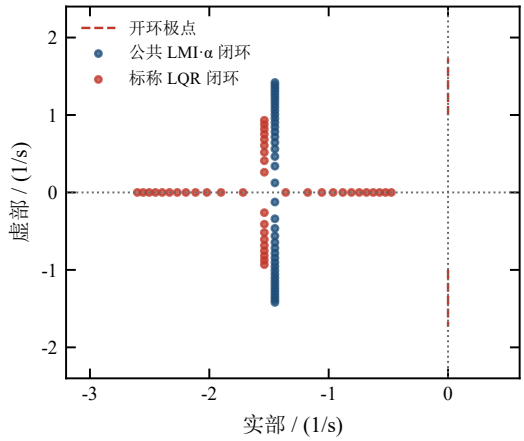


图 65-2　开环极点与两套定常增益的闭环极点随参数移动的轨迹

图 65-2 先把“参数移动”这件事画出来。左侧虚线上的点是开环极点 $\pm\sqrt{a}$，随 a 从 1 到 3 上下拉开；右侧两簇点是两套定常增益的闭环极点。公共 LMI 增益（实心深蓝）的轨迹是一对共轭复根，$a=1$ 时在 $-0.87\pm1.12j$（阻尼比 0.613），$a=3$ 时几乎落到实轴上（$-1.45\pm0.124j$，阻尼比 0.996）；标称 LQR 增益（红色）在 $a=2$ 处是两个实极点，往两端走时一对极点迅速分离，$a=3$ 时分到 -0.475 和 -2.603。两簇点的分布说明了公共 LMI 在设计上做了什么取舍：它把“最坏情况”的极点实部压在 -0.87 附近，代价是在 $a=1$ 那一端牺牲了阻尼（0.613 对 0.855）。这个取舍是否合理，取决于你的指标写的是“衰减率”还是“阻尼比”——这正是 §65.6 要讨论的事。

65.2 数学原理

§65.1 用一句话概括了技术核心——换变量。但要让这套方法真的能用，还需要四个工具：把谱条件翻译成矩阵不等式的 **Lyapunov 定理**、把“传递函数范数小于 γ”翻译成矩阵不等式的**有界实引理**、把“缩成一个块”的 **Schur 补**、把“对所有 δ 成立”翻译成“存在一个 ε”的 **S-procedure**。这一节逐个交代，每个都给一句“它在后面哪一节被用到”。

65.2.1 标准形与 Schur 补

先把“什么叫 LMI”说准。一个线性矩阵不等式是

$$F(x)=F_0+\sum_{i=1}^{m}x_iF_i\succ 0 \tag{65-13}$$

其中 $F_i=F_i^{\mathsf T}\in\mathbb{R}^{n\times n}$ 是给定的对称矩阵，$x\in\mathbb{R}^m$ 是决策变量。左边对 x 是仿射的，约束集 $\{x:F(x)\succ 0\}$ 是凸的（正定锥的原像）。当 $n=1$ 时它退化成普通的线性不等式，所以**线性规划是 LMI 的特例**；$n>1$ 时它叫半定规划（SDP）。

式 (65-8)、式 (65-10) 都不是标准形，因为约束里出现了 X 的逆和乘积。把它们变回标准形靠 Schur 补：

$$\begin{bmatrix}A & B\\ B^{\mathsf T} & C\end{bmatrix}\succ 0,\quad C\succ 0 \quad\Longleftrightarrow\quad A-BC^{-1}B^{\mathsf T}\succ 0 \tag{65-14}$$

这个引理的作用是**把逆挪到分母里去**。凡是出现 $A-BC^{-1}B^{\mathsf T}$ 这种带逆的形式，都可以用一个更大的矩阵块替换，代价是变量维数变大（从 n 变成 $n+m$），换来的是线性。本章后面几乎每一个“看起来不是 LMI”的约束，都是用式 (65-14) 变成 LMI 的。它的证明只需对左边的矩阵做一次分块消元，这里不重复。

65.2.2 Lyapunov 定理：谱条件的第一层翻译

对线性系统 $\dot{x} = Ax$，下面两条等价：

1. A 的所有特征值实部小于 $-\alpha$；
2. 存在 $P \succ 0$ 使 $A^{\mathsf{T}}P + PA + 2\alpha P \prec 0$。

“1 推出 2”是构造性的：取 $P = \int_0^\infty e^{(A+\alpha I)^{\mathsf{T}}t} e^{(A+\alpha I)t}\,dt$，由条件 1，被积函数指数衰减，积分收敛且正定，代进去正好满足不等式。“2 推出 1”更简单：设 $Av = \lambda v$，左乘 v^* 右乘 v 得 $2(\mathrm{Re}\lambda + \alpha) v^*Pv < 0$，而 $v^*Pv > 0$，所以 $\mathrm{Re}\lambda < -\alpha$。

这个定理的分量在于它是一个**充要条件**。对单个确定的 A，“极点都在左边”和“存在一个二次 Lyapunov 函数”是同一句话。所以本章用 LMI 求解稳定性，不是“用一个充分条件近似”，而是在解一个等价问题——至少在参数固定时是这样。一旦参数变成一个区间，“存在公共 P”就退化成充分条件了，因为真实的系统可以每个 a 用不同的 V，而公共 P 强制共用一个。§65.1.2 里 0.9277 与 1.2207 的差额，就是这次退化的价格。

α 出现在式 (65-9) 里时是“衰减率”，出现在式 (65-5) 里时是“极点实部的负下界”。两者是同一件事的两种说法：V 的衰减速率等于最慢极点的实部（在二次型 V 下恰好相等，一般的 V 只能给出下界）。

65.2.3 有界实引理：H∞ 的入口

H∞ 指标问的是“从扰动 w 到被调输出 z 的传递函数，在所有频率上的最大增益”。对本章的算例，取 w 为作用在加速度上的外部扰动、z 为位置与控制量的加权组合：

$$z = C_1 x + D_{12} u, \qquad C_1 = \begin{bmatrix} 1 & 0 \end{bmatrix}, \quad D_{12} = \rho = 1 \tag{65-15}$$

闭环 $u = -Kx$ 之后 $z = (C_1 - D_{12}K)(sI - A + BK)^{-1} B_w\, w$，记这个传递函数为 $T_{zw}(s)$。**有界实引理**说：$A - BK$ 稳定且 $\|T_{zw}\|_\infty < \gamma$，当且仅当存在 $P \succ 0$ 使

$$\begin{bmatrix} (A-BK)^{\mathsf{T}}P + P(A-BK) & PB_w & (C_1 - D_{12}K)^{\mathsf{T}} \\ B_w^{\mathsf{T}}P & -\gamma I & 0 \\ C_1 - D_{12}K & 0 & -\gamma I \end{bmatrix} \prec 0 \tag{65-16}$$

这个 3×3 块矩阵是本章 H∞ 部分全部工作的起点。左右各乘 $\mathrm{diag}(X, I, I)$（$X = P^{-1}$），并把 KX 换成 Y，得到

$$\begin{bmatrix} AX + XA^{\mathsf{T}} - BY - Y^{\mathsf{T}}B^{\mathsf{T}} & B_w & (C_1 X - D_{12}Y)^{\mathsf{T}} \\ B_w^{\mathsf{T}} & -\gamma I & 0 \\ C_1 X - D_{12}Y & 0 & -\gamma I \end{bmatrix} \prec 0 \tag{65-17}$$

对 (X, Y) 线性，对 γ 也是线性的（γ 只出现在对角块上）。于是“最小化 γ”就是一个标准的 SDP，可以用二分也可以用凸求解器直接做。`code/ch065_sim.py` 里用的是外层二分：给定 γ 判可行，不可行就抬高，γ 的精度由二分的终止条件控制（本章取 10^{-3}，共 16 次求解）。

用 Schur 补可以把式 (65-17) 再压成 2×2，但那会引入 γ^{-1} 与 X 的乘积，反而不是 LMI。**保持 3×3 是刻意的**：块越大越“松”，但越线性。这也是 LMI 建模的一条通用经验——宁可多加一块，不要提前消元。

式 (65-16) 里有一个容易被忽略的细节：$D_{11} = 0$（扰动不直接进输出）。如果 $D_{11} \neq 0$，右下角要换成 $-\gamma I$ 与 D_{11} 的组合，γ 的下界会变成 $\|D_{11}\|$，可行性问题在 $\gamma < \|D_{11}\|$ 时直接不可行。本章的 γ 下界来自 $\|D_{12}\| = 1$，这就是 §65.1.4 里那个 $\gamma = 1.0004$ 的来源。

65.2.4 S-procedure：把“所有 δ”换成一个 ε

$A(a)$ 是仿射的，可以直接用顶点。但工程上更常见的不确定性写法是“标称值加一个范数有界的摄动”：

$$A = A_0 + D\,\delta\,E, \qquad |\delta| \le 1, \qquad A_0 = \begin{bmatrix} 0 & 1 \\ 2 & 0 \end{bmatrix}, \quad D = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \quad E = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{65-18}$$

（$a = 2 + \delta$，$\delta \in [-1, 1]$，正好覆盖 $[1, 3]$。）这种写法下，Lyapunov 不等式变成

$$M + \delta\big(DEX + XE^{\mathsf T}D^{\mathsf T}\big) \prec 0, \qquad \forall\, |\delta| \le 1 \tag{65-19}$$

其中 $M = A_0X + XA_0^{\mathsf T} - BY - Y^{\mathsf T}B^{\mathsf T} + 2\alpha X$ 与 δ 无关。要让它对**所有** $|\delta| \le 1$ 成立，直接的做法是取两个端点（这又回到顶点法）；另一种做法是对交叉项配方，用 $2ab \le a^2 + b^2$：

$$\delta\big(DEX + XE^{\mathsf T}D^{\mathsf T}\big) \preceq \varepsilon\, DD^{\mathsf T} + \varepsilon^{-1}(EX)^{\mathsf T}(EX), \qquad \forall\, |\delta| \le 1,\ \forall\, \varepsilon > 0 \tag{65-20}$$

于是式 (65-19) 被一个更强的条件代替：

$$M + \varepsilon DD^{\mathsf T} + \varepsilon^{-1}(EX)^{\mathsf T}(EX) \prec 0 \tag{65-21}$$

再用一次 Schur 补，把 $\varepsilon^{-1}(EX)^{\mathsf T}(EX)$ 收进块里：

$$\begin{bmatrix} M + \varepsilon DD^{\mathsf T} & (EX)^{\mathsf T} \\ EX & -\varepsilon I \end{bmatrix} \prec 0 \tag{65-22}$$

ε 是新的决策变量（非负标量），可以和其他变量一起交给求解器。这就是 S-procedure 的标准用法：**把“无穷多条关于 δ 的约束”换成“一条关于 ε 的约束”**，代价是引入保守性（配方那一步是不等号，ε 取多大是优化出来的）。

值得说清的是保守性的来源。对单个标量 δ 加单个二次型约束，S-procedure 是**无损**的——也就是说式 (65-21) 与式 (65-19) 等价。本章实测也印证了这一点：范数有界表述给出的最大衰减率 **0.9277**，与直接验两个顶点的公共 Lyapunov **0.9277** 完全一致，ε 的最优值是 4.7299。换成两个独立的不确定参数，无损性立刻消失，那才是 S-procedure 真正开始保守的地方，§65.9 会给出数字。

65.2.5 顶点性成立的条件，以及它什么时候不成立

式 (65-4) 那个“只需验两个端点”的结论，成立的条件比通常写的要苛刻，值得列清楚：

1. **不确定性必须是多胞的。**$A(a)$ 要能写成顶点矩阵的凸组合，$A(a) = \theta A(1) + (1-\theta)A(3)$，$\theta \in [0,1]$。本章的 $A(a)$ 是仿射的，自动满足。如果 a 出现在分母里（比如 $1/\ell$ 而 ℓ 不确定），仿射性就断了，得先换变量。
2. **Lyapunov 矩阵必须与 a 无关。**一旦允许 $P(a)$，条件集不再是“一个 P 满足所有顶点”，顶点性失效。
3. **约束必须是关于 A 凸的。**式 (65-8) 左边对 A 是线性的，成立。如果约束里出现 A^2 或者 A^{-1}，就不成立。
4. **控制器必须是定常的。**K 要是 a 的函数（调度），问题变成“每个顶点一套 (X_i, Y_i)”，那已经不是同一个 LMI，而是 §65.4 里的“顶点独立设计”。

四条里任何一条不满足，“只验顶点”就只是一次采样，不能当证明。工程上最容易踩的是第 1 条：很多人把“参数在某区间”直接当成“仿射多胞”，而实际上参数是以非线性方式进来的（刚度随气隙三次方变化、电阻随温度指数变化），这时候顶点验过了也不代表中间没问题。补救办法是把非线性段用分段仿射外包，或者干脆在网格上做真值对拍——这也是 §65.9 的“证书核验”要占一整小节的原因。

65.2.6 离散情形：把微分换成差分

离散域的对应物是 $x_{k+1} = A_d x_k + B_d u_k$，稳定性条件是谱半径小于 1，Lyapunov 版本是“存在 $P \succ 0$ 使 $A_d^{\mathsf{T}} P A_d - P \prec 0$”。加上收缩率 $\sigma \in (0,1)$：

$$A_d^{\mathsf{T}} P A_d - \sigma^2 P \prec 0 \quad \Longleftrightarrow \quad \begin{bmatrix} \sigma^2 X & A_d X \\ X A_d^{\mathsf{T}} & X \end{bmatrix} \succ 0 \tag{65-23}$$

右边是式 (65-14) 的直接应用（把 $A_d^{\mathsf{T}} P A_d$ 看成 $B C^{-1} B^{\mathsf{T}}$ 那一项）。$X = P^{-1}$，闭环 $A_d - B_d K$、$Y = KX$ 之后：

$$\begin{bmatrix} \sigma^2 X & A_d X - B_d Y \\ (A_d X - B_d Y)^{\mathsf{T}} & X \end{bmatrix} \succeq 0 \tag{65-24}$$

形式和连续域几乎一样，只是“$\prec 0$ 的 Lyapunov 不等式”变成了“$\succeq 0$ 的块矩阵”，而且 σ 与连续域的 α 之间有一个明确的换算：

$$\sigma = e^{-\alpha T_s} \tag{65-25}$$

这个换算不是装饰，它决定了不同采样周期下的结果能不能互相比较。如果每个 T_s 都只写“可行性”（不带 σ），求解器会退到边界上，给出 $\rho(A_{cl}) \approx 1.0000$ 的“能稳但一点都不快”的解，且不同 T_s 之间的增益几乎不变——这个坑 §65.5 会用一整组数据展示。带上式 (65-25) 之后，T_s 从 0.005 s 增到 0.8 s，K_1 从 5.0093 降到 3.9092，ρ 从 0.9931 降到 0.4813，趋势才和物理直觉一致。

还有一件事：离散 LMI 的可行域随 T_s 增大而收缩，到某个 T_s 会直接**不可行**。本章实测有两条边界：$\alpha = 0.9$ 的可行性在 0.8 s 与 1.0 s 之间断掉；只要求稳定（$\sigma = 1$）时，1.0 s 仍可行（$\rho = 0.9812$），1.5 s 才断。这个“不可行”不是程序失败，而是定量回答了“这台机器最慢能多久采一次样”，比任何经验法则都硬。

65.2.7 四个工具的分工

把这一节压缩成一张表：

工具	输入	输出	用在哪一节
Lyapunov 定理	谱条件 $\mathrm{Re}\lambda < -\alpha$	$A^{\mathsf{T}}P + PA + 2\alpha P \prec 0$	§65.4 连续设计
Schur 补	带 C^{-1} 的块	更大的线性块矩阵	§65.2.4、§65.5、§65.9
有界实引理	$\|T_{zw}\|_\infty < \gamma$	式 (65-17) 的 3×3 块	§65.9 H∞
S-procedure	$\forall \|\delta\| \le 1$	存在 ε，式 (65-22)	§65.4 范数有界、§65.9 降保守

四个工具里，只有 Lyapunov 定理是充要的，其余三个都带“充分”的成分。这就是为什么 LMI 设计出来的结果总是“保守”的——不是因为求解器不行，而是因为从“我要这个性能”到“我写下的这条 LMI”之间，隔了三道充分条件的门。

65.3 设计步骤

这一节把前面两节的东西压成一条可以照抄的流水线。流程图在图 65-11，五个方框对应五步，中间的判断框是唯一需要人做决定的地方——**证书和真值对不上时，回到第一步重写指标，而不是回到第四步调求解器**。这条规矩看起来是常识，实际项目里被违反的次数最多：求解器报 `optimal` 太容易让人相信了。

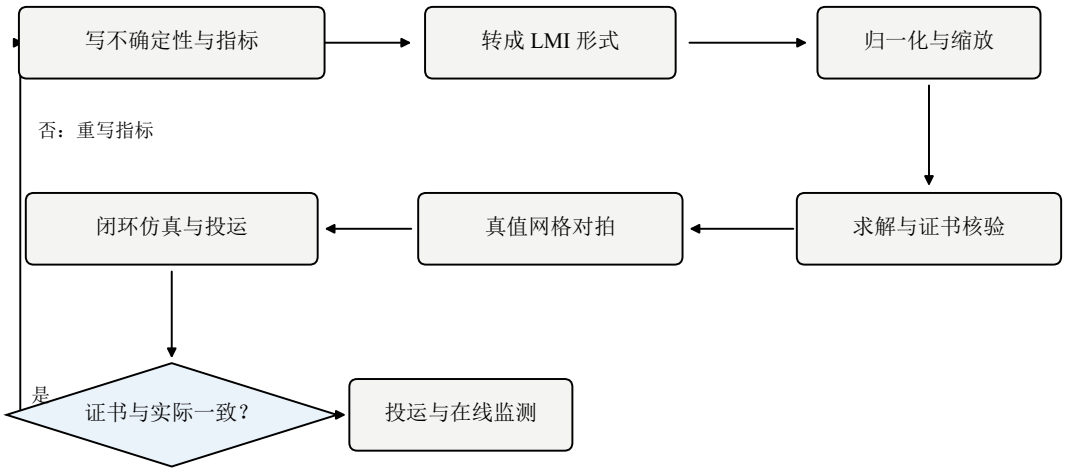


图 65-11　LMI 设计的工程流程：从写指标到投运，中间有一次强制的证书核验

65.3.1 第一步：把工程语言翻成矩阵

这一步没有公式可套，但它决定了后面四步有没有意义。以磁悬浮为例，真实的不确定源至少有三个：气隙变化让电流刚度变（k_i 随气隙非线性下降）、载荷变化让质量变（m 从空载到满载差 30%）、线圈温度让电阻变（影响电流环带宽，进而不影响 a 但影响执行器滞后）。三者合成到 $a = k_i/m$ 上，可能得到 $a \in [0.8,\, 3.2]$ 这样一个区间。

接下来要做的不是“把这个区间原样塞进 LMI”，而是三件事：

第一，判断哪个参数是主导的。 如果 k_i 的变化贡献了区间宽度的 80%，m 只贡献 20%，那就把 k_i 当不确定参数、m 当标称值加上一个固定裕度。理由很实际：多一个不确定参数，多胞的顶点数翻倍（n 个参数、每个两端点，顶点数 2^n），而顶点数直接决定 LMI 的规模和保守性。两个参数时 4 个顶点，三个参数时 8 个，五个参数时 32 个——到那个时候，保守性会把设计压得没有性能可言，反而不如把次要参数折进“设计裕度”里。

第二，确认参数是以仿射方式进来的。 磁悬浮的 a 是 k_i/m，如果 k_i 和 m 各自独立变化，a 就不是仿射的。这时候要么把 a 本身当参数（承认它是个“黑箱区间”），要么做变量替换把 $1/m$ 提出来。本章的算例直接取 a 为参数，就是为了避开这层麻烦——但读者要知道，这层麻烦在真实项目里是躲不掉的。

第三，把“性能”写成可以比较的数。 “响应要快”不是指标，“衰减率 $\alpha \geq 0.9$”才是；“抗扰要好”不是指标，“$\|T_{zw}\|_\infty \leq 3$”才是。写不出数的时候，先做一个标称设计，量一下它达到多少，再把这个数打个折当目标。这一步看着粗糙，但比拍脑袋定 $\alpha = 5$ 然后发现不可行要好。

65.3.2 第二步：选一种表述

同一个“对区间内所有 a 稳定”，至少有三种写法，保守性和计算量都不一样：

表述	约束形式	顶点数	保守性	适用场景
顶点法（公共 P）	式 (65-8)，每个顶点一条	2^n	中	参数区间已知且仿射
范数有界 + S-procedure	式 (65-22)，ε 是变量	1	中（单参数时无损）	只知道摄动范数上界
顶点独立（调度）	每个顶点一套 (X_i, Y_i)	2^n	低	能在线测到参数

选哪一条取决于两个事实：**你知不知道参数怎么进来的，你能不能在线上测到它**。知道且能测，用调度（顶点独立），性能最好；知道但不能测，用顶点法；连怎么进来的都不清楚，只剩范数上界，用 S-procedure。本章会三种都算一遍，它们的最大衰减率分别是 **1.2207**、**0.9277**、**0.9277**，正好把这张表的“保守性”一列量化了。

65.3.3 第三步：补三条归一化

这一步是 §65.1.4 那个事故的直接产物。写完 LMI 之后，在交给求解器之前，强制加三条：

（1）尺度归一化 $X \succeq I$。 防的是齐次性导致的 α 无界。它的副作用是把 X 的量纲钉在“状态单位的平方的倒数”上，所以换单位必须重算（§65.10 有七档缩放的实测）。

（2）增益界 $\|Y\| \le \kappa$。 防的是“用无穷大增益换指标”。κ 取多少？经验是标称 LQR 增益的 3–5 倍。本章的标称 LQR 增益范数是 $\sqrt{4.2361^2 + 3.0777^2} = 5.24$，取 $\kappa = 20$ 相当于 3.8 倍，够松，又足以挡住 10^4 量级的退化解。取得太小（比如 $\kappa = 6$）会把可行域切掉一块，γ 被人为抬高；取得太大（$\kappa = 10^4$）等于没加。

（3）目标函数 $\min \|Y\|_F$。 纯可行性问题（没有目标）的解不唯一，求解器返回的是它内部迭代停下的那个点，可能是任何一个可行解。加一个“最小增益”的目标，解就唯一了，而且通常是同一可行域里最“温和”的那个。这条对可复现性至关重要：同一份代码在不同机器上跑，没有目标函数时可能给出不同的 K，有目标函数时至少落在同一个极值点上。

三条加完，问题才是一个“良定义”的 LMI。本章所有实验结果都在这三条之下得到，代码里的 `common_lmi()`、`hinf_lmi()`、`disc_lmi()` 三个函数都显式带着它们。

65.3.4 第四步：求解与证书核验

求解本身只是一行 `prob.solve()`，麻烦的是**判断这个解能不能信**。至少要核四件事：

第一，求解器状态。 cvxpy 返回的状态有 `optimal`、`optimal_inaccurate`、`infeasible`、`unbounded`、`solver_error` 五种。只有前两种能用，而 `optimal_inaccurate` 需要额外核残差。本章四个求解器的实测里，SCS 给的是 `optimal_inaccurate`，耗时 751.23 ms；CLARABEL 给的是 `optimal`，耗时 6.15 ms。同一道题，一个“可信但慢”，一个“可信且快”。

第二，约束残差。 把解出来的 X、Y 代回式 (65-8)，算左边矩阵的最大特征值。理论上应该 < 0，实际上会有 10^{-9} 到 10^{-6} 的正值（数值误差）。如果最大特征值是 $+10^{-3}$ 量级，说明解在可行性边界外面，不能用。这个检查要写成代码，不能靠“求解器说 optimal 就行”。

第三，X 的条件数。 $\mathrm{cond}(X)$ 超过 10^6 时，$K = YX^{-1}$ 的数值精度就开始不可靠了。本章的缩放的实测里，$s = 10^{-3}$ 时条件数是 6.85×10^6，此时解出来的 $K = [5.58,\ 3.441]$ 与最优单位下的 $[5.403,\ 3.057]$ 已经差了一截——不是算法错，是求逆把误差放大了。

第四，K 的量级。 如果 K 的某个分量超过 10^3，先怀疑是不是漏了归一化或增益界，再怀疑对象。10^3 量级的增益在物理上意味着“位置差 1 mm 就给出 1 m/s^2 的指令”，这已经不是控制，是开关。

这四条检查加起来不到二十行代码，但它们把“跑出结果”和“结果可信”分开了。

65.3.5 第五步：真值对拍

证书说“对所有 a 稳定”，真值对拍要回答“到底有多稳、在哪个 a 最差”。做法是在参数区间上铺网格，逐点算极点和阻尼比。本章用 41 个点（步长 0.05），算出来的结果是：

$$\zeta_{\min} = 0.6133 \ @\ a = 1.00, \qquad \max_{a} \mathrm{Re}\,\lambda(A(a) - BK) = -0.0065 \ @\ a = 3.00 \tag{65-26}$$

两个数分处区间的两端，这件事本身是有信息的：**阻尼最差在 a 的下界，衰减最慢在 a 的上界**。它们分别对应 §65.1.1 说的两种失败方式——a 小的时候回路软、阻尼低；a 大的时候负刚度强、最慢极点被顶到接近虚轴（-0.0065，时间常数 154 s）。

还有一个更值得注意的地方：$\alpha=0$ 那套增益（$K=[3.0113,\ 1.7395]$）在 $a=3$ 处的最慢极点只有 -0.007，几乎贴着虚轴。可是这套增益是 LMI 求解器在“保证稳定”的约束下给出的，状态是 `optimal`。它确实稳定，只是“稳定”这两个字在这里的含义薄到几乎为零——一个 154 s 的时间常数，在现场等同于“慢慢漂走”。这再次说明：**没有 α 的可行性问题只回答“能不能活”，不回答“活得好不好”**，而求解器不知道你想要后者。

对 H∞ 的对拍更重：本章用 4000 个频点乘以 241 个参数点，算出每个 (a,ω) 上的 $\|T_{zw}\|$，取最大值。得到的结果是 **1.9506 @ $a=3$**，而 LMI 证书是 **9.1759**。这个 4.7 倍的差距是本章最重要的一个数字，§65.9 专门处理它。

65.3.6 五步的检查清单

把上面的内容压成一张可以打印出来贴在工位上的表：

步骤	必做动作	常见错误	后果
1 写指标	定主导参数、确认仿射、指标写成数	参数超过 3 个	顶点爆炸，保守性失控
2 选表述	按“知不知/测不测”选	把非线性参数当仿射	顶点验过但中间不稳
3 归一化	$X\succeq I$、$\|Y\|\le\kappa$、$\min\|Y\|$	漏掉任一条	α 或 γ 无界
4 求解	核状态、残差、$\mathrm{cond}(X)$、$\|K\|$	只看 `optimal`	拿到不可用的解
5 对拍	网格扫极点、扫频算范数	只验顶点	证书与真值脱节而不自知

表 65-3　LMI 设计的五步与对应的坑

表 65-3 的“后果”一列是这一节最该记住的部分。五步里任何一步偷懒，都不会让程序报错——它会给你一个 `optimal` 的解，然后你在现场花两周去找原因。这也是为什么本章坚持每一个结论都用真值对拍：**LMI 的输出是一个“证书”，证书的正确性靠数学保证，证书的有用性只能靠对拍验证。**

65.4 连续情形

这一节把前面的框架跑成三套具体的增益，然后看它们在三台“不同 a 的机器”上的表现。三套增益的设计过程完全相同——都是式 (65-8) 加 $X\succeq I$ 加 $\min\|Y\|_F$——唯一的差别是 α 取多少，以及要不要让 K 随 a 变。

65.4.1 三套增益是怎么解出来的

第一套，$\alpha=0$ 的公共 Lyapunov 设计。 只要求“存在一个 P 管住两个顶点”，不要求衰减率。求解器返回 $X\succeq I$ 边界上的解（因为目标是最小化 $\|Y\|$，X 越大 $\|K\|=\|YX^{-1}\|$ 越小，所以 X 顶在 I 上），得到

$$K_{\alpha=0}=[3.0113,\ 1.7395] \tag{65-27}$$

这套增益的范数是 3.48，比标称 LQR 的 5.24 还小。看着很“温柔”，但 §65.3.5 已经暴露了它的代价：$a=3$ 时最慢极点 -0.007，时间常数 154 s。

第二套，$\alpha=0.9277$ 的公共设计。 这个 α 是用 16 次二分搜出来的最大值（区间 $[0,20]$，精度 10^{-3}，最后一次可行是 0.9277、第一次不可行是 0.9287）。增益是

$$K_{\alpha=0.928} = [5.1188,\ 2.9007] \tag{65-28}$$

范数 5.88，比标称 LQR 略大 12%。这 12% 就是“要保证 0.93 的衰减率，同时对两个 a 都成立”所必须多付的控制量。

第三套，标称 LQR。 在 $a = 2$ 上做标准 LQR，$K = [4.2361,\ 3.0777]$，范数 5.24。它不是 LMI 解出来的，放在这里是为了给一个“不懂鲁棒性的人会怎么做”的参照。

三套增益列在一起，有一个反直觉的现象：**$\alpha = 0$ 的设计增益最小，α 最大的设计增益居中，标称 LQR 的增益介于两者之间。** 按“增益越大越激进”的直觉，这个排序是乱的。原因在于它们优化的东西不同：$\alpha = 0$ 优化的是增益范数（所以最小），$\alpha = 0.928$ 优化的是守恒的衰减率（需要额外的控制量，但不是最多），标称 LQR 优化的是二次型代价（在标称点上最省，但不管两端）。

图 65-3 把三种表述下的最大衰减率画成条形图，可以直观看到“知道参数”值多少。

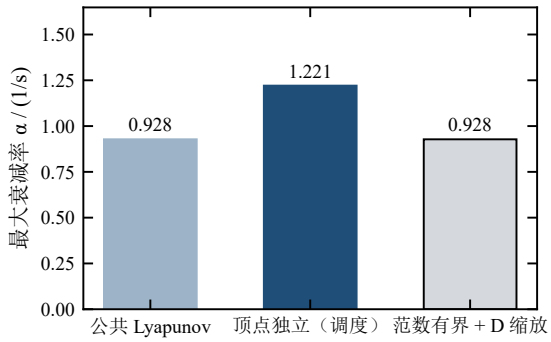


图 65-3　三种 LMI 表述下的最大衰减率：公共 Lyapunov、顶点独立调度、范数有界加 D 缩放

中间那根深色柱子是顶点独立设计——每个顶点一套 (X_i, Y_i)，相当于“在线能测到 a”。它的 1.2207 比公共设计的 0.9277 高 **32%**。这个差额不是算法优劣，是信息量的价格：多知道一个数（a 的当前值），换 32% 的响应速度。右边那根柱子和左边一样高，是范数有界表述；在这个单参数算例上，S-procedure 没有额外损失（§65.2.4 已经解释过原因）。

65.4.2 九种组合的实测表现

把三套增益分别放到 $a = 1$、2、3 三台机器上，做 1 m 的阶跃，得到九个组合。评价六项：闭环极点、最弱阻尼比、调节时间（5% 带）、稳态误差、超调、控制量峰值。

增益	a	闭环极点	$\zeta_{\min}$	t_s / s	e_{ss}	超调	$\|u\|_{\max}$
公共 LMI（$\alpha=0$）	1	$-0.87 \pm 1.12j$	0.613	8.00	0.4969	0.0%	1.011
公共 LMI（$\alpha=0$）	2	$-0.87 \pm 0.505j$	0.865	3.75	0.0019	0.5%	2.020
公共 LMI（$\alpha=0$）	3	$-0.007,\ -1.733$	1.000	8.00	**3.2351**	**323.5%**	**12.707**
公共 LMI（$\alpha=0.928$）	1	$-1.45 \pm 1.42j$	0.715	8.00	0.2428	0.0%	3.119
公共 LMI（$\alpha=0.928$）	2	$-1.45 \pm 1.008j$	0.821	**1.98**	≈ 0	1.1%	3.119
公共 LMI（$\alpha=0.928$）	3	$-1.45 \pm 0.124j$	0.996	8.00	0.4718	47.2%	4.416
标称 LQR	1	$-1.539 \pm 0.932j$	0.855	8.00	0.3090	0.0%	2.236
标称 LQR	2	$-1.176,\ -1.902$	1.000	3.31	0.0002	0.0%	2.236
标称 LQR	3	$-0.475,\ -2.603$	1.000	8.00	0.7597	76.0%	5.290

表 65-4　三套增益在三个参数点上的九种组合

这张表要横着读，也要竖着读。

横着读（同一套增益，参数往大漂），三套增益的表现都随 a 变差，但变差的方式不同。$\alpha=0$ 那一组在 $a=3$ 处直接崩了：稳态误差 3.2351 意味着指令 1 m、实际冲到 4.235 m 还在往上升（后面会算，它的理论稳态是 89.5 m）；峰值 323.5%、控制量 12.707。$\alpha=0.928$ 那一组在 $a=3$ 时峰值 47.2%、e_{ss} 0.4718，虽然不好看，但至少还在同一个量级。标称 LQR 在 $a=3$ 时峰值 76.0%，介于两者之间。

竖着读（同一个 a，换增益），会看到一件更重要的事：**没有一套增益在三个点上都是最好的。** $a=1$ 时标称 LQR 的阻尼（0.855）最好；$a=2$ 时它也是最好的（两个实极点、零超调、t_s=3.31）；$a=3$ 时它的超调（76.0%）比 $\alpha=0.928$ 那套（47.2%）还差。而 $\alpha=0$ 那套的唯一优势是控制量小（$a=1$ 时 1.011，几乎不饱和），代价是静差 0.4969。

65.4.3 那张 323.5% 的表说明了什么

表 65-4 里最刺眼的是 $\alpha=0$ 增益在 $a=3$ 那一行。三个数分别是：$e_{ss}=3.2351$、超调 323.5%、$|u|_{\max}=12.707$。它们不是独立的，是同一个原因的三个侧面。

先算稳态。前馈系数按标称点取 $k_r = K_1 - a_{\text{nom}} = 3.0113 - 2 = 1.0113$（这几套增益都用同一个前馈，因为它们都不知道真实的 a）。稳态时 $\ddot{x}_1 = 0$，即 $a x_1 + u = 0$；而 $u = k_r - K_1 x_1$。两式联立：

$$x_{1,\infty} = \frac{k_r}{K_1 - a} = \frac{K_1 - a_{\text{nom}}}{K_1 - a} \tag{65-29}$$

这个式子很小，但它把“参数漂移”的后果一次说清了。分母 $K_1 - a$ 是**位置环的开环增益**（把速度环闭合之后剩下的那个增益），分子是前馈量。a 越接近 K_1，分母越接近零，稳态增益越趋于无穷。对 $K_1=3.0113$ 这套增益，$a=3$ 时分母只有 0.0113，于是

$$x_{1,\infty} = \frac{1.0113}{0.0113} = 89.5 \tag{65-30}$$

也就是**指令 1 m，稳态会走到 89.5 m**。仿真只跑 8 s，x_1 才爬到 4.235，看上去“只是”323.5% 的超调，实际它是这条爬向 89.5 的慢曲线的第一段。8 s 内它一直在上升，从未回落，因为最慢极点的时间常数是 154 s。

这就解释了 $|u|_{\max}=12.707$：为了把状态从 0 推到 4.2 m，控制量早就顶穿了 1 m/s^2 的物理限幅（12.7 倍）。**表里所有“$|u|_{\max}>1$”的行，在真实设备上都会饱和，而饱和后的行为完全不在 LMI 的保证范围内**——LMI 证明的是线性闭环的稳定性，限幅一进来，那个证明就失效了。这一点在 §65.11 还会展开。

同样的算式用在 $\alpha=0.928$ 那套上：$K_1=5.1188$，$a=3$ 时分母 2.1188，$x_{1,\infty}=3.1188/2.1188=1.472$，$e_{ss}=0.472$，与实测 0.4718 完全吻合。$a=1$ 时 $x_{1,\infty}=3.1188/4.1188=0.757$，$e_{ss}=0.243$，也与实测 0.2428 吻合。式 (65-29) 在两套增益上都被验证了。

65.4.4 阶跃波形：三条曲线的形状

图 65-4 把九个组合里的九个响应画成三格（每格一个 a，三条曲线分别是三套增益）。

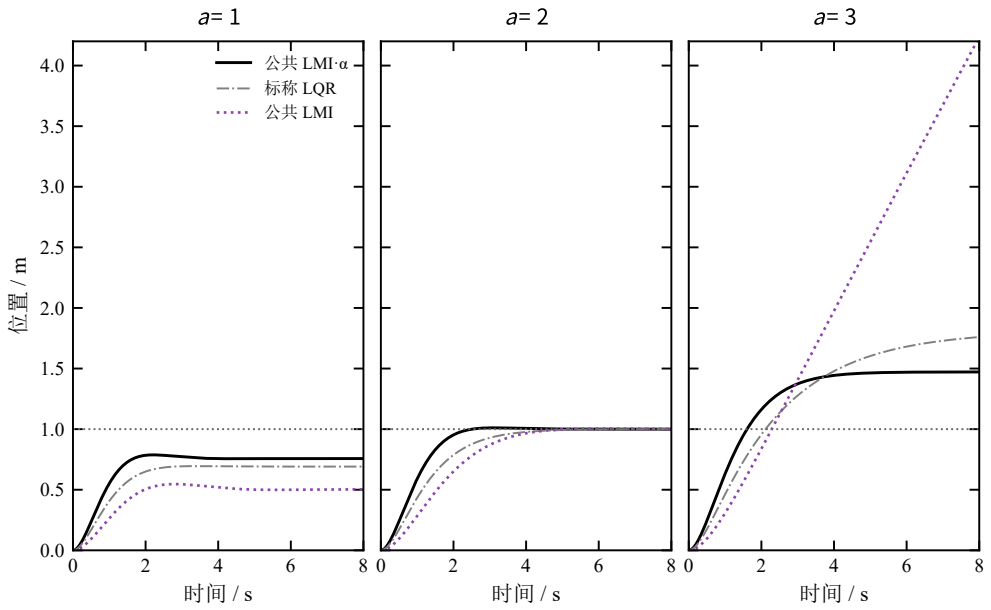


图 65-4　三套增益在三个参数点上的阶跃响应：$a=1$（左）、$a=2$（中）、$a=3$（右）

左格（$a=1$）最“和平”：三条曲线都无超调，$\alpha=0.928$ 那条最快但也最“硬”（控制量 3.119），标称 LQR 那条的阻尼最好。这一格里 $\alpha=0$ 那条的稳态只到 0.503，肉眼可见地停在半路——这就是 0.4969 的静差。

中格（$a=2$）是三套增益的“主场”，因为标称点就在这里。$\alpha=0.928$ 的调节时间 1.98 s 最短，比标称 LQR 的 3.31 s 还快 40%，这一点值得注意：**LMI 设计在标称点上可以比 LQR 更快，因为它的目标不是二次型代价，而是衰减率**。LQR 的 1.176/1.902 双实极点里，慢的那个 1.176 决定了调节时间；而 $\alpha=0.928$ 那套把两个极点都推到了 -1.45 附近，慢极点被抬高了 23%。

右格（$a=3$）是“事故现场”。$\alpha=0$ 那条曲线一路上冲、8 s 时还在 4.2 附近，图上的纵轴只能截到 4.2，实际它要爬到 89.5。$\alpha=0.928$ 那条冲到 1.47 后回落，但 8 s 内没回到 1.05 的带内（所以 t_s 记为 8.00，是仿真时长的上限）。标称 LQR 那条冲到 1.76，回落更慢且稳态更高（0.7597 的静差）。

三格放在一起，能看出一条设计规律：**参数往哪边漂，设计就该往哪边偏。** 如果现场知道 a 只会往大漂（比如载荷只会增加），那就该按 $a=3$ 做设计，让 $a=2$ 时“过设计”一点；本章的区间设计是“两端都要管”，所以必然在中间点上浪费一些性能。这个浪费的量，就是 §65.6 要量化的“鲁棒代价”。

65.4.5 选哪一套

三套增益对应三种现场条件，选法不难：

现场条件	推荐	理由	代价
参数几乎不变，只要标称点好	标称 LQR	标称点最优，计算最简	漂 50% 就 76% 超调
参数会漂，但测不到	公共 LMI（带 α）	全区间有证书，标称点反而更快	增益大 12%，两端各让一步
参数会漂，且能测	顶点独立（调度）	全区间最快	需要传感器与调度逻辑
只要不散架，控制量越小越好	公共 LMI（$\alpha = 0$）	增益最小	静差与慢极点都不可接受

表 65-5　三套增益的适用条件

第四行要特别提醒：$\alpha = 0$ 的设计在数学上是“正确”的，它确实让闭环对两个顶点都稳定。但它的“稳定”是靠把极点放在 -0.007 换来的，这种解在现场没有价值。**可行性问题的解，必须配一个衰减率下界才有工程意义**——这是本章反复强调的一句话，也是很多论文里“我们证明了系统稳定”却做不出可用控制器的根源。

65.4.6 前馈与静差：一个被忽略的细节

最后说一个细节，它在表 65-4 里贡献了大半的 e_{ss}，但在所有 LMI 文献里都很少提：**LMI 设计的是反馈增益 K，不含前馈**。本章的仿真统一取 $k_r = K_1 - a_{\text{nom}}$，也就是“按标称参数算一个让直流增益为 1 的前馈”。这个前馈一旦和真实 a 不匹配，静差就按式 (65-29) 出现。

有三种处理办法。**第一种**是把前馈也纳入设计——但那需要知道 a，又回到调度问题。**第二种**是在回路里加积分器，用积分把静差磨掉——代价是阶次升高、LMI 规模变大，且积分器会引入一个新的慢模态。**第三种**是接受静差，在上位机做指令修正——最简单，也最常用。本章的算例选的是第三种，因为我们要把“反馈设计的保守性”和“前馈不匹配”这两件事分开看；如果加了积分器，表 65-4 的 e_{ss} 一列会全部归零，反而看不出反馈本身的差别。

这笔账值得单独记：**在 $a = 3$ 那一行，$e_{ss} = 3.2351$ 里几乎没有反馈的过错，全是前馈按 $a = 2$ 算造成的。** 如果现场给一个准确的前馈（$k_r = K_1 - 3$），$\alpha = 0$ 那套增益的稳态会回到 1，但峰值和 154 s 的慢极点依然存在。分清这两笔账，才知道该改哪里。

65.5 离散情形

真实的控制器跑在采样系统上。连续域算出来的 K 直接烧进单片机，中间隔着一个零阶保持器和一个采样周期 T_s，这两样东西会改变闭环的性质。工程上的通常做法是“连续设计、离散实现”，只要 T_s 足够小就没问题。但“足够小”是多大？这一节用 LMI 给出一个定量的答案，并且暴露一个更麻烦的现象：**离散 LMI 只在采样时刻保证收敛，采样之间它不管。**

65.5.1 连续设计直接离散化会怎样

零阶保持下，连续对象 (A, B) 对应的离散对象是

$$A_d = e^{A T_s}, \qquad B_d = \int_0^{T_s} e^{A\tau}\, d\tau\, B \tag{65-31}$$

开环不稳定极点 $\sqrt{a}$ 映射成 $e^{\sqrt{a}T_s} > 1$，仍然不稳定，这不奇怪。麻烦在于 A_d 的非对角项：A_d 的元素随 T_s 是 T_s 的多项式，而反馈增益 K 是按连续时间设计的，离散闭环 $A_d - B_d K$ 的极点与连续闭环 $e^{(A-BK)T_s}$ 的极点并不相同——差多少取决于 T_s 与闭环带宽的乘积。

粗略的估计是：零阶保持引入的平均延迟约 $T_s/2$，这个延迟在频域上贡献 $-\omega T_s/2$ 的相位。相位裕度被吃掉多少，就看在穿越频率 ω_c 处 $\omega_c T_s/2$ 有多少度。本章 $\alpha = 0.928$ 的设计，闭环极点的模约 1.45 rad/s，取 $\omega_c \approx 2$

rad/s，则 $T_s = 0.02$ s 只吃 1.1°，$T_s = 0.2$ s 吃 11°，$T_s = 0.8$ s 吃 46°。后两个数已经开始威胁稳定性，这与 §65.5.3 的实测边界（0.8 s 可行、1.0 s 不可行）大致对得上。

但这个估计只说明“延迟会坏事”，说不清“到底坏到什么程度”。要定量，还是得把问题写成离散 LMI。

65.5.2 离散 LMI 与收缩率

式 (65-24) 已经给出了离散形式，这里重写一遍并补上闭环：

$$\begin{bmatrix} \sigma^2 X & A_d X - B_d Y \\ (A_d X - B_d Y)^{\mathsf{T}} & X \end{bmatrix} \succeq 0, \qquad \sigma = e^{-\alpha T_s} \tag{65-32}$$

两个顶点各一条，$X \succeq I$，目标 $\min \|Y\|_F$。σ 是**每一步**的收缩因子：$\|x_{k+1}\|_{X^{-1}} \le \sigma \|x_k\|_{X^{-1}}$。式 (65-25) 把它与连续域的衰减率 α 对上，这样不同 T_s 的解才可比。

这里有一个容易被忽略的细节：$\sigma = e^{-\alpha T_s}$ 让 σ 随 T_s 变小，看起来“T_s 越大，要求越松”，其实正好相反。σ 变小意味着每步允许的收缩更多，但步子也变长了，两者在连续域上抵消。**要求松紧的正确度量是 $\alpha = -\ln \sigma / T_s$，不是 σ 本身。** 这个换算在下一小节的表里会用到。

65.5.3 八档采样周期的实测

固定 $\alpha = 0.9$（一个适中的目标，比公共设计的上限 0.9277 略小，保证连续域可行），把 T_s 从 5 ms 扫到 2 s，得到八个可行点、三个不可行点：

T_s / s	K_1	K_2	$\rho(A_{cl})$	$\sigma = e^{-0.9T_s}$	等效 $\alpha = -\ln\rho/T_s$
0.005	5.0093	2.8381	0.9931	0.9955	1.386
0.010	5.0023	2.8407	0.9867	0.9910	1.337
0.020	4.9886	2.8462	0.9747	0.9822	1.282
0.050	4.9488	2.8629	0.9419	0.9560	1.197
0.100	4.8872	2.8875	0.8942	0.9139	1.118
0.200	4.7686	2.9021	0.8112	0.8353	1.046
0.500	4.3509	2.7149	0.6120	0.6376	0.982
0.800	3.9092	2.4175	0.4813	0.4868	0.914
1.000	—	—	—	0.4066	不可行
1.500	—	—	—	0.2592	不可行
2.000	—	—	—	0.1653	不可行

表 65-6　采样周期对离散 LMI 解的影响（$\alpha = 0.9$）

这张表有四个值得读的地方。

第一，$T_s \to 0$ 时增益趋近连续解。 $T_s = 5$ ms 时 $K = [5.0093,\ 2.8381]$，而连续域 $\alpha = 0.928$ 的解是 $[5.1188,\ 2.9007]$。T_s 再小也不会完全相等，因为 α 差了 0.028，而且离散约束只作用在采样点上（下面会讲）。

第二，K_1 随 T_s 增大而单调下降，从 5.0093 降到 3.9092，降幅 22%。这符合直觉：采样越慢，控制器“看到”的信息越旧，同样大小的增益更容易把回路推过头，所以求解器自动往小里选。K_2 的行为不同，它在 $T_s =$

0.2 处有一个小峰（2.9021），然后才下降——因为 K_2 管的是速度阻尼，采样慢时速度估计的相位滞后最先伤到它。

第三，ρ 随 T_s 增大而单调下降，看起来"性能变好"，实际是假象。 表里最后一列是等效衰减率 $\alpha = -\ln\rho/T_s$：从 1.386 一路降到 0.914，降幅 34%。也就是说，**同一句"$\alpha = 0.9$"的离散约束，在不同采样周期下实现出来的实际衰减率可以差三分之一**。原因是离散 LMI 只在采样点约束收缩，采样之间状态可以"偷偷"长一点，T_s 越大偷得越多。求解器为了满足采样点上的 σ，不得不把 ρ 压得更低，于是看起来"更好"，实际上连续时间的行为更容易出问题。

第四，$T_s \geq 1.0$ s 在 $\alpha = 0.9$ 下不可行。 但这不是"连稳都稳不住"。把 σ 放到 1（只要求能稳）重跑，$T_s = 1.0$ s 仍然可行，给出 $K = [3.0177,\ 1.9761]$、$\rho = 0.9812$；$T_s = 1.5$ s 才真正不可行（状态 `infeasible_inaccurate`），2.0 s 与 3.0 s 是干净的 `infeasible`。所以这台机器有**两条边界**：「能稳」的上限在 1.0–1.5 s 之间，「能达到 0.9 衰减率」的上限在 0.8–1.0 s 之间。两条边界差了一倍，中间那段是"数学上稳、工程上不能用"的地带——$\rho = 0.9812$ 对应的时间常数是 $T_s/|\ln\rho| = 52.7$ s，与 §65.4 那个 154 s 的慢极点是一类东西。

65.5.4 采样间行为：离散 LMI 的盲区

第三点那个"偷偷长一点"值得单独说，因为它是离散 LMI 最容易出事的地方。

离散 LMI 保证的是 $V(x_{k+1}) \leq \sigma^2 V(x_k)$，也就是**在采样时刻**的能量单调下降。两次采样之间，状态按 $\dot{x} = Ax$ 连续演化，而 A 本身是不稳定的（$a = 3$ 时有 $+1.732$ 的极点）。采样点上的能量可以很低，采样间却可能冲得很高——典型的"锯齿"。

举个量级：$T_s = 0.8$ s、$a = 3$ 时，不稳定模态在一步内可以放大 $e^{1.732\times 0.8} = 3.99$ 倍。如果控制器在采样点上把状态压到 0.1，采样间它可以长到 0.4 再被压回去，这个 0.4 就是执行器和传感器实际承受的峰值，而离散 LMI 对此一无所知。

把表 65-6 的八档采样周期逐个算一遍放大倍数 $e^{\sqrt{3}T_s}$：5 ms 是 1.0087，10 ms 是 1.0175，20 ms 是 1.0352，50 ms 是 1.0905，100 ms 是 1.1891，200 ms 是 1.4140，500 ms 是 2.3774，800 ms 是 3.9974。这张名单有一个共同点：**它只与 T_s 和 a 有关，与 K 无关。** 因为在两次采样之间控制器输出被零阶保持器钉死，对象是在开环下演化的，增益再大也改变不了这段时间里的放大率。

这个观察把责任分得很清楚。控制器能做的只有"在每个采样点上把状态压得足够低"，使采样间放大之后仍落在安全范围内；它能决定的不是放大倍数，而是放大前的起点。$T_s = 0.8$ s、放大 4 倍时，要让峰值不超过 1 m，采样点上的位置就必须压在 0.25 m 以下——这条要求与"$\alpha = 0.9$"毫无关系，纯粹由采样间几何决定。**离散 LMI 里没有这条约束，所以表 65-6 里 $T_s = 0.8$ 那行虽然标着"可行"，它保证的东西比看上去少得多。**

这也是为什么"采样点上的 ρ"和"连续时间的 α"必须分开报告。前者是求解器证明的，后者是设备感受到的；两者差多少，由无量纲数 $T_s\sqrt{a}$ 决定。本章 $T_s = 20$ ms 时 $T_s\sqrt{a} = 0.035$，放大 3.5%，可以忽略；$T_s = 0.5$ s 时是 0.87，放大 138%，已经完全不能忽略。

补救办法有三个。**一是留裕度**：把 α 取大一些，让 ρ 远离 1，采样点上的状态压得更低，采样间冲高的绝对值就跟着小（放大倍数本身改不了，能改的只是起点）。**二是事后核验**：解出 K 之后，用连续模型（含零阶保持的等效延迟）重算一遍采样间峰值，这一步不能省。**三是直接做连续域的 LMI**（本章 §65.4 的做法），再按"连续设计 + 快采样"实现，让 T_s 小到采样间可以忽略——这也是工程上最常见的做法。

65.5.5 采样周期怎么定

把这一节的结论压成一条可操作的建议：

1. 先按连续域做设计，拿到闭环最慢极点与最快极点。
2. 取 $T_s \le 0.1/|\lambda_{\max}|$ 作为初值。本章 $a=3$ 时 $|\lambda| = 1.732$，给出 $T_s \le 0.058$ s。
3. 用离散 LMI 在这附近扫一遍，确认可行且等效 α 不低于目标的 90%。
4. 在 T_s 上加 3–5 倍余量，再核采样间峰值。

本章的算例里，第 2 步给的是 58 ms，而离散 LMI 的可行上限是 800 ms——两者差 14 倍。这个差距不是经验公式错了，而是经验公式保守：它假设“采样间可以放大到不可接受”，而 LMI 只要求“采样点收敛”。真实工程里应该取两者中**更保守**的那个，也就是 58 ms 这一侧，因为采样间峰值是物理量，不能靠证书抹掉。

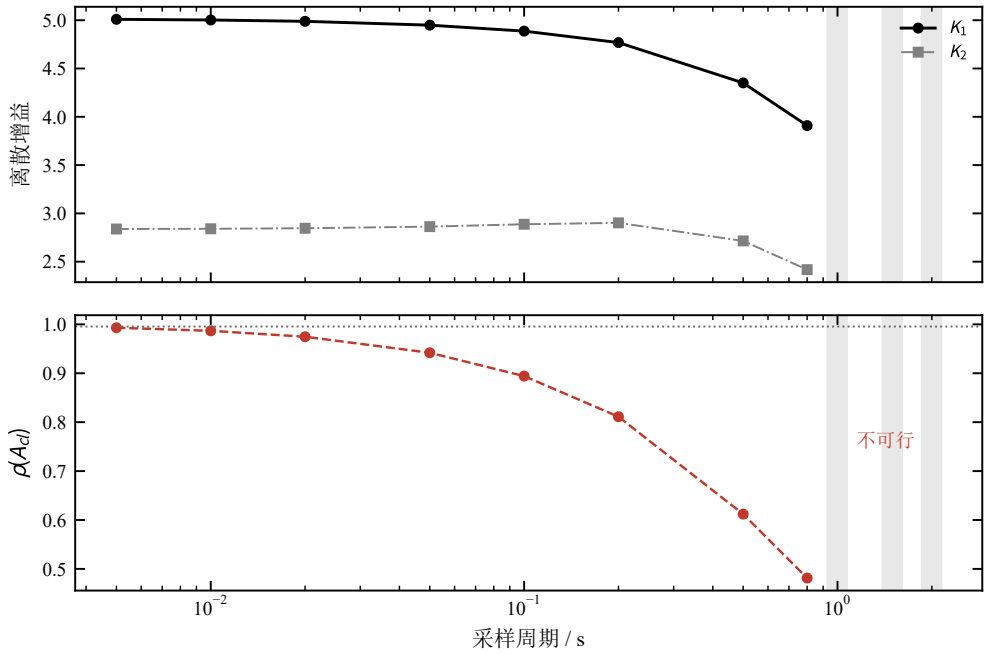


图 65-6　采样周期对离散增益与谱半径的影响；灰色区域为 LMI 不可行的采样周期

图 65-6 把表 65-6 画成上下两格。上格是 K_1、K_2 随 T_s 的变化（对数横轴），下格是 ρ 与 σ 的对比，右侧灰底是三个不可行点。下格里两条曲线离得越近，说明约束越紧：$T_s = 0.8$ s 时 $\rho = 0.4813$ 与 $\sigma = 0.4868$ 只差 1.1%，几乎贴着边界；$T_s = 5$ ms 时 $\rho = 0.9931$ 与 $\sigma = 0.9955$ 差 0.24%，看起来也近，但两者的绝对差只有 0.0024，对应的等效 α 却高出 52%。**看 ρ 与 σ 的绝对差会误导人，必须换算成 α 才能比较。**

65.5.6 一个反例：不带 σ 的离散 LMI

最后补一个坑，它是本章第二次踩到的“退化”事故。最初的离散 LMI 没有写 σ，只写“$X \succ 0$、$A_dX - B_dY$ 的谱半径小于 1”：

$$\begin{bmatrix} X & A_dX - B_dY \\ (A_dX - B_dY)^{\mathsf{T}} & X \end{bmatrix} \succ 0 \tag{65-33}$$

这个式子是对的（它是 $\sigma = 1$ 的特例），但它的可行域包含“ρ 无限接近 1”的解。求解器没有理由往中间走，于是所有 T_s 都给出 $K \approx [3.01,\ 1.74]$、$\rho \approx 1.0000$——和连续域 $\alpha = 0$ 那套一模一样，因为 $\sigma = 1$ 时问题退化成“只要求能稳”，而最小增益解恰好就是连续域那个。

这个现象说明：**离散 LMI 里的 σ 不是可选项。** 没有它，问题仍然是一个合法的凸问题，仍然返回 `optimal`，但它回答的是“能不能稳”，而你问的是“多快能稳”。两者的差别在 T_s 大的时候是致命的：$\rho = 1.0000$ 意味着时间常数无穷大，与 $\rho = 0.48$ 完全是两个世界。

65.6 稳定性与性能

前面两节把三套增益的“能不能用”说清了，这一节回答“到底有多好”，以及一个更微妙的问题：**LMI 证书上的性能指标，和闭环实际表现出来的性能，是不是一回事。**

65.6.1 三种“稳”的区别

日常说“这个回路稳”，可能指三件不同的事：

第一种，渐近稳定。 所有极点在左半平面，扰动后能回到平衡点。这是最弱的，也是 §65.4 里 $\alpha = 0$ 那套增益唯一保证的东西。它允许极点落在 -0.007，也允许时间常数 154 s。

第二种，指数稳定（带衰减率）。 存在 $\alpha > 0$ 使 $\|x(t)\| \le Ce^{-\alpha t}\|x(0)\|$。这是 $\alpha > 0$ 的设计保证的，α 就是那个指数。注意它管的是“最坏方向”的衰减速度，不是“平均速度”。

第三种，二次稳定。 存在一个**固定的**二次型 $V(x) = x^{\mathsf{T}}Px$，使 $\dot V < 0$ 对参数区间里的每个 a 都成立。这是 LMI 直接给出的东西，它比前两种都强：二次稳定一定指数稳定（α 由 P 和 A 决定），但指数稳定不一定二次稳定。

三者的关系可以用一句工程话概括：**渐近稳定回答“会不会散架”，指数稳定回答“多久能收住”，二次稳定回答“我能不能用同一个能量函数证明它”。** 前两个是系统的性质，第三个是证明的手段。LMI 的保守性，本质上就是把“系统的性质”换成了“证明的手段”——手段比性质窄，所以证明不出来的东西，未必不存在。

本章的算例正好能把这个“窄”量化。公共 Lyapunov 下最大 $\alpha = 0.9277$；允许每个顶点一套 (X_i, Y_i) 时是 1.2207。后者也是二次稳定（只是 P 随顶点变），但已经不是“公共 P”意义上的二次稳定。而真实的系统，如果允许 P 连续地随 a 变，理论上还能更高。**0.9277 不是系统的极限，是“公共二次 Lyapunov 函数”这个证明工具的极限。**

65.6.2 网格上的两个指标

图 65-10 把 $\alpha = 0$ 那套增益（$K = [3.0113,\ 1.7395]$）放到 41 个参数点上，算了两个指标：最弱阻尼比 $\zeta_{\min}$ 和最大极点实部 $\max \mathrm{Re}\lambda$。

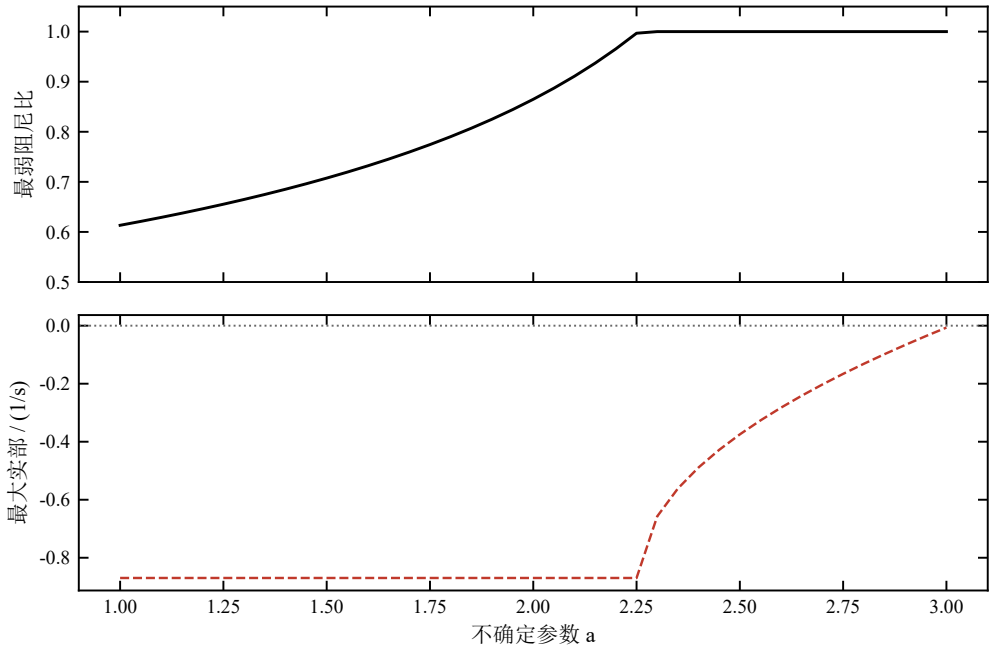


图 65-10　公共增益在参数网格上的最弱阻尼比（上）与最大极点实部（下）

上格是阻尼比。它从 $a=1$ 处的 **0.6133** 单调上升到 $a\approx 2.25$ 处的 1.0，然后**平在 1.0**。下格是最大实部，它在 $a<2.25$ 时是常数 -0.8698，$a>2.25$ 后突然折向上，到 $a=3$ 时爬到 -0.0065。

两条曲线都在 $a\approx 2.25$ 处折了一下，这不是数值巧合，而是闭环特征方程的结构决定的。对二阶系统

$$\det(\lambda I - A + BK) = \lambda^2 + K_2\lambda + (K_1 - a) = 0 \tag{65-34}$$

判别式是 $K_2^2 - 4(K_1-a)$。它为零的临界参数是

$$a^{*} = K_1 - \frac{K_2^2}{4} = 3.0113 - \frac{1.7395^2}{4} = 2.2548 \tag{65-35}$$

$a<a^{*}$ 时判别式为负，两个共轭复根，实部恒为 $-K_2/2=-0.8698$，阻尼比随 a 增大而上升；$a>a^{*}$ 时判别式为正，分裂成两个实根，慢的那个是

$$\lambda_{\text{slow}} = \frac{-K_2 + \sqrt{K_2^2 - 4(K_1-a)}}{2} \tag{65-36}$$

它随 a 增大而单调右移，$a=K_1$ 时正好到 0。本章的 $a=3$ 离 $K_1=3.0113$ 只有 0.0113，所以慢极点停在 -0.0065。

这段推导解释了一个容易误读的现象：**上格在 $a>2.25$ 后显示“阻尼比 1.0”，看起来性能最好，实际上那正是最危险的一段。** 阻尼比的定义是 $-\mathrm{Re}\lambda/|\lambda|$，对实极点恒等于 1，无论这个实极点离虚轴多近。所以“$\zeta=1$”在过阻尼情况下不携带任何“快慢”信息，而工程上真正关心的恰恰是快慢。这也是为什么本节的第二个指标（最大实部）必须和阻尼比一起看：一个说“振不振”，一个说“慢不慢”，缺一个就会误判。

还要说清这张图证明了什么、没证明什么。41 个点上的逐点验算只能说明“这 41 个点没问题”，不能证明点与点之间没问题；真正把整个区间盖住的是顶点性（§65.2.5），网格只是它的实证。两者的地位完全不同：一旦顶点性不成立（比如 K 随 a 变、或约束对 A 非凸），这张图就从“验证”降级成“抽样”，而抽样永远可能漏掉最坏点。**工程上最常见的错误就是把抽样当证明——图很漂亮，结论却是错的。**

65.6.3 三个性能指标的换算

把极点和常见的时域指标对上，才能判断一个 LMI 设计“够不够用”。对二阶主导的系统，有三条近似关系：

$$t_s \approx \frac{4.6}{\alpha}, \qquad M_p \approx e^{-\pi\zeta/\sqrt{1-\zeta^2}}, \qquad \omega_n \approx \frac{\alpha}{\zeta} \tag{65-37}$$

第一式把衰减率和调节时间直接连起来：$\alpha = 0.9277$ 对应 $t_s \approx 4.96$ s。表 65-4 里 $\alpha = 0.928$ 那套在 $a = 2$ 时实测 1.98 s，比 4.96 s 快得多——因为 $4.6/\alpha$ 是**最坏方向**的估计（按最慢极点算），而实际响应的主导模态可能比最慢极点快。$a = 2$ 时两个极点都在 -1.45 附近，α 的实际值是 1.45，对应 $t_s = 3.17$ s，仍比 1.98 s 保守。这说明式 (65-37) 只能当量级估计，不能当精确预测。

第二式是超调与阻尼比的经典关系：$\zeta = 0.613$ 给出 $M_p = 8.7\%$，$\zeta = 0.715$ 给出 4.0%，$\zeta = 0.996$ 给出 ≈ 0。表 65-4 里 $\alpha = 0$ 那套在 $a = 1$ 时 $\zeta = 0.613$ 但实测超调 0.0%——因为前馈不匹配导致稳态只到 0.503，峰值根本没超过 1。**超调公式假设稳态是 1，一旦有静差，公式就失效。** 这也是 §65.4.6 那笔账的另一个侧面。

第三式说明：同样的 α，ζ 越小则 ω_n 越大。$\alpha = 0.928$、$\zeta = 0.715$ 时 $\omega_n = 1.30$ rad/s；$\zeta = 0.996$ 时 $\omega_n = 0.93$ rad/s。前者更快但更振荡，后者更慢但更平——两者在 α 这个指标下是等价的，这正是指标选择的难点。

65.6.4 鲁棒代价的量化

现在可以回答“为鲁棒性付了多少钱”这个问题。有三把尺子：

尺子一，与标称 LQR 比。 标称 LQR 在 $a = 2$ 处的慢极点是 -1.176，即 $\alpha_{\text{LQR}} = 1.176$。公共 LMI 的 $\alpha = 0.9277$，是它的 **79%**。也就是为了覆盖 $a \in [1, 3]$，牺牲了 21% 的标称响应速度。

尺子二，与顶点独立比。 1.2207 对 0.9277，公共设计只有调度设计的 **76%**。这 24% 是“不知道 a”的价格。

尺子三，与“每个 a 各自最优”比。 表 65-2 里三个点各自的 LQR 慢极点分别是 1.000、1.176、1.414，取最小值 1.000 作为“全区间都能达到”的上界（其实达不到，因为一套 K 管不住三个点）。公共设计的 0.9277 是这个上界的 **93%**。这个数看起来不错，但它是个虚假的乐观：1.000 是三个点各自最优里的最小值，不是某一套 K 能同时达到的值。

三把尺子给出 79%、76%、93% 三个数，选哪个取决于你想说服谁。给项目经理解释“为什么要多花 12% 的控制量”，用第一把；给同行解释“LMI 保守不保守”，用第二把；写论文时千万别用第三把，它是三个不同 K 的拼盘。

把这三个百分比换成秒，往往更容易被非控制专业的人接受。用式 (65-37) 的量级估计 $t_s \approx 4.6/\alpha$：标称 LQR 是 $4.6/1.176 = 3.91$ s，公共 LMI 是 $4.6/0.9277 = 4.96$ s，多出 1.05 s。注意这个增幅是 27%，比 α 的降幅 21% 大——因为 α 与 t_s 成反比，α 掉 21% 意味着 t_s 涨 $1/0.79 - 1 = 27\%$。**同一个事实，写在分母上和写在分子上不是一回事，报告里选错一个，说服力就差三成。**

再换一个角度：这 1.05 s 值不值，取决于指标卡在哪里。如果验收条件写的是“8 s 内进入 5% 带”，两套设计都满足，这 21% 一分钱不值；如果指标是“4 s 内”，两套都不满足，得在 α 之外另想办法；只有指标落在 4 s 到 5 s 之间，这 21% 才是必须付的价钱。**鲁棒性的价格不是由控制器决定的，是由指标决定的。** 这也解释了为什么表 65-4 那张九种组合的表里没有“最优”一列——最优是相对于指标说的，脱离指标谈优劣没有意义。

65.6.5 一个反例：指标对了，物理不对

最后说一件在纸面上看不出来、在现场一定会发生的事。

表 65-4 里 $\alpha = 0.928$ 那套增益在 $a = 3$ 时，$|u|_{\max} = 4.416$，而物理限幅是 1。也就是说，**阶跃一开始，执行器就饱和到 4.4 倍**。线性分析给出的 47.2% 超调和 8.00 s 调节时间，全部是在“执行器能给出 4.4”这个假设下算出来的。真实设备给出 1，闭环的行为就完全不同：饱和期间相当于开环（u 恒定），不稳定的对象会在这段时间里快速偏离，等退出饱和时状态可能已经到了一个线性分析从未考虑过的地方。

这不是 LMI 特有的问题，所有线性设计都有。但 LMI 有两个地方让它更容易发生：

一是衰减率指标本身会推高增益。 要让 α 大，就得把极点推远，而推远极点必然需要更大的控制量。α 从 0 提到 0.928，增益范数从 3.48 涨到 5.88，$|u|_{\max}$ 从 1.011 涨到 4.416——涨了 4.4 倍，而 α 只涨了不到 1。

二是 LMI 的多解性让人容易忽略这件事。 同一个 α，可以有很多个可行的 K，它们的 $|u|_{\max}$ 可能差好几倍。求解器按 $\min \|Y\|$ 选了一个，但这个“最小”是在 $X \succeq I$ 归一化下的最小，不等于“控制量最小”。

补救的办法是在 LMI 里直接加控制量约束。最常用的是不变椭圆（§65.9 会给出完整形式）：给定 $|x_1| \le 1$、$|u| \le 1$，求一个椭圆，使椭圆内所有初始状态在收敛过程中都不越界。本章实测：$\alpha = 0$ 的增益对应面积 1.1896 的椭圆，$\alpha = 0.928$ 的只有 0.6146——**增益翻倍，可用吸引域面积减半**。这个数比任何定性描述都直白：你想要的响应速度，是从吸引域里“买”来的。

所以一个完整的设计流程，在 §65.3 那五步之后还应该加一步：**用不变椭圆核一遍吸引域，确认它覆盖现场会遇到的最大初始偏差。** 如果覆盖不了，要么降 α，要么加前馈或分段控制——但那是另一章的内容了。

65.7 参数意义与作用

LMI 的决策变量有五个：X、Y、α、ε、γ，再加上一个人为设的 κ。它们都不是“调参旋钮”意义上的参数，每一个都有明确的数学角色，但它们在工程上的可解释性差别很大。这一节逐个说清：这个量是什么、量纲是什么、改它会发生什么。

65.7.1 X：不是 Lyapunov 矩阵，是它的逆

最容易出错的一个概念。Lyapunov 函数是 $V(x) = x^{\mathsf{T}} P x$，而 LMI 里的决策变量是 $X = P^{-1}$。两者互为逆，性质完全不同：

量	定义	特征值的意义	本章实测
P	$V = x^{\mathsf{T}} P x$	能量等高面的“陡度”	未直接求出
X	$X = P^{-1}$	$V \le 1$ 椭圆的半轴平方	半轴 0.2876、1.3168

表 65-7　P 与 X 的关系

X 的特征值 μ_i 满足：$V(x) \le 1$ 定义的椭圆，在第 i 个主方向上的半轴长度是 $\sqrt{\mu_i}$。这个几何解释是理解 X 的全部关键：

- X 大 → 椭圆大 → 允许的初始偏差大 → 吸引域大；
- X 的**特征值比例**决定椭圆的“胖瘦”，比例接近 1 时椭圆接近圆，比例悬殊时椭圆很扁；
- X 的**特征向量**决定椭圆的主轴方向，通常与“位置-速度”的相关性对应。

所以式 (65-11) 那个 $X \succeq I$ 的归一化，几何上就是“椭圆的每个半轴都不短于 1”。这条约束把一个本来可以任意缩放的椭圆钉住了大小，代价是它可能和真实的吸引域不匹配——如果真实吸引域只有 0.3 m，那 $X \succeq I$ 就是过度要求，会人为抬高增益。

65.7.2 Y：带着量纲的“增益乘尺度”

$Y = KX$，所以 Y 的量纲是“增益 × 尺度”。它不能直接读成增益，必须除以 X 之后才是 K。这带来两个后果。

第一，Y 的大小不能用来判断“控制激不激进”。 本章 $\alpha = 0$ 那套的 Y 范数比 $\alpha = 0.928$ 那套小，但这只是因为前者的 X 更接近 I。真正该看的是 $K = YX^{-1}$ 的范数：3.48 对 5.88。

第二，$\min \|Y\|_F$ 这个目标有偏。 它实际最小化的是 $\|KX\|_F$，在 $X \succeq I$ 的边界上（$X = I$）退化成 $\|K\|_F$，但在 X 内部时它会倾向于“让 X 小一点、K 小一点”的组合。本章因为 X 顶在 I 上，这个问题不明显；换成 $X \succeq 0.01I$ 这种松约束，$\min \|Y\|$ 就可能给出一个 K 很大、X 很小的解。要避免，就得把目标写成 $\min \|K\|$（非凸），或者加式 (65-12) 那样的增益界。

65.7.3 α：衰减率，最像“旋钮”的一个

α 是唯一一个能直接对应到工程指标的决策变量：α 越大，响应越快，同时增益越大、吸引域越小。本章的实测把这条权衡量化了：

α	K_1	K_2	$\|K\|$	$a=3$ 时 $\|u\|_{\max}$	椭圆面积
0	3.0113	1.7395	3.48	12.707	1.1896
0.928	5.1188	2.9007	5.88	4.416	0.6146

表 65-8　α 对增益、控制量与吸引域的影响

这张表有一个反直觉的地方：α 从 0 提到 0.928，$a = 3$ 时的 $|u|_{\max}$ 反而从 12.707 **降到** 4.416。原因不是 α 让控制变温柔，而是 $\alpha = 0$ 那套在 $a = 3$ 时根本不受控（慢极点 154 s），状态一路爬到 4.2，控制量自然跟着涨。这个对比说明：**“控制量大小”本身不能脱离“响应是否正常”来评价**，一个失控的回路可能给出很大或很小的控制量，两者都没有意义。

α 的可行上限是 0.9277（公共设计）。这个上限不是一个“建议值”，是硬边界：二分到最后，$\alpha = 0.9277$ 可行、0.9287 不可行，中间没有过渡带。工程上取 0.8–0.9 倍的上限比较稳，留一点数值余量。

65.7.4 ε：S-procedure 的“配重”

ε 只在范数有界表述里出现，它的作用是平衡式 (65-21) 里的两项：$\varepsilon DD^{\mathsf{T}}$ 和 $\varepsilon^{-1}(EX)^{\mathsf{T}}(EX)$。$\varepsilon$ 大，前者重、后者轻；ε 小则相反。

本章实测的最优值是 **4.7299**。这个数本身没有物理意义，但它的量级有信息：如果 ε 跑到 10^{-6} 或 10^{6}，说明两项的量级严重失衡，通常意味着 D 或 E 的标度选错了（比如把 10^3 量级的系数塞进 E）。这时候应该先做归一化，而不是让求解器去扛。

还有一点：ε **是优化出来的**，不是给定的。很多人习惯手填一个 ε 然后扫描，那是把凸问题变成了非凸搜索。正确的做法是让它和 X、Y 一起作为决策变量，交给求解器。

65.7.5 γ：H∞ 证书，一个上界

γ 是 $\|T_{zw}\|_\infty$ 的上界。本章多胞 LMI 给出 **9.1759**，而真值是 **1.9506**。这个 4.7 倍的差距要理解成“证书的精度”，不是“设计的精度”——设计本身可能不错，只是证书说不清它有多好。

γ 的可行下界由 D_{12} 决定（本章 $\|D_{12}\| = 1$），这是硬下界，任何 LMI 都不可能给出比它更小的 γ。加上增益界 $\kappa = 20$ 之后，实际能做到 9.1759。如果把 κ 放大到 10^3，γ 会降下来，但对应的 K 会变成不可实现的量级

——这就是 §65.1.4 那个事故。

读 γ 的正确方式：不要问“γ 是不是足够小”，要问“γ 对应的闭环在真值网格上表现如何”。本章的做法是先解 LMI 拿 K，再用 4000 频点 × 241 参数点算真值，得到 1.9506。如果真值满足指标，$\gamma = 9.18$ 这个“难看”的证书就可以接受；如果真值不满足，γ 再小也没用。

65.7.6 κ：人为设的，但决定成败

κ 不是 LMI 的一部分，是建模者加的。它没有理论依据，只有工程依据：标称增益的 3–5 倍。本章取 20，标称增益范数 5.24，比值 3.8。

κ 太小的后果是“把好解切掉”：可行域被人为缩小，γ 被抬高，甚至直接不可行。κ 太大的后果是“退化解回来”：γ 被无穷大增益买到理论下界。两者之间有一个窗口，本章的 20 在窗口内。判断窗口位置的办法很简单：把 κ 从 5 扫到 100，看 γ 和 $\|K\|$ 怎么变。如果 γ 随 κ 单调下降而 $\|K\|$ 单调上升到 κ 附近，说明还在窗口外（右侧）；如果 γ 不变而 $\|K\|$ 不变，说明 κ 已经不起作用（左侧）。

65.7.7 不变椭圆：X 的几何意义（图 65-9）

把所有抽象的量落到一张图上，就是不变椭圆。

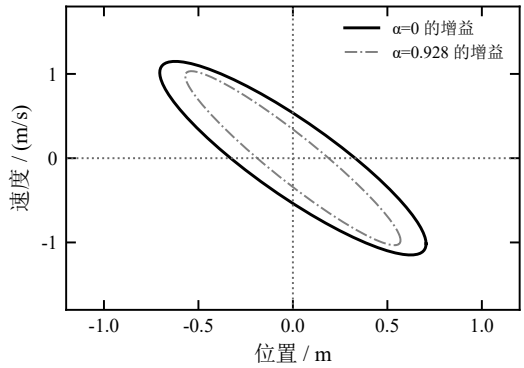


图 65-9　两套增益下的最大不变椭圆：$\alpha = 0$（实线）与 $\alpha = 0.928$（虚线）

椭圆的定义是

$$\mathcal{E} = \{ x : \; x^{\mathsf{T}} X^{-1} x \le 1 \} \tag{65-38}$$

对任意 $x(0) \in \mathcal{E}$，只要 $A_{cl} X + X A_{cl}^{\mathsf{T}} \prec 0$ 对所有顶点成立，就有 $x(t) \in \mathcal{E}$ 对所有 $t \ge 0$ 成立。再加上两条附加约束

$$X_{11} \le x_{1,\max}^2, \qquad K X K^{\mathsf{T}} \le u_{\max}^2 \tag{65-39}$$

就能保证椭圆内的状态永不越过行程限制、控制量永不越限。式 (65-39) 的第二条来自 $(Kx)^2 \le \max_{x \in \mathcal{E}} (Kx)^2 = K X K^{\mathsf{T}}$，是一个标准的二次型极值。

本章的实测：$\alpha = 0$ 的增益对应半轴 $[0.2876,\ 1.3168]$、面积 1.1896；$\alpha = 0.928$ 的对应半轴 $[0.1675,\ 1.1679]$、面积 0.6146。

读这两个椭圆要注意三件事。

第一，真正卡住椭圆的是控制量约束，不是行程。 把两套增益分别代回式 (65-39)：$K X K^{\mathsf{T}}$ 两次都等于 1.0000（顶在边界上），而 X_{11} 只有 0.4967 和 0.3246，离 $x_{1,\max}^2 = 1$ 还差一半。也就是说，**椭圆是被推力**

限死的，导轨行程还有富余。椭圆在 x_1 方向的最大延伸是 $\sqrt{X_{11}} = 0.705$ m，只有行程的 70%。这个信息在现场很值钱：它说明限制性能的是执行器，改造方向应该是加大推力而不是加长导轨。

第二，椭圆是斜的，长短轴比 4.58。半轴 $[0.2876,\ 1.3168]$ 对应的特征向量分别是 $(0.866,\ 0.501)$ 和 $(0.501,\ 0.866)$，也就是主轴相对坐标轴整体转了 30°。倾斜来自 $X_{12} = -0.7157$——这个交叉项与两个对角元同量级，说明位置与速度在 Lyapunov 函数里强负相关。短轴方向大致是“位置偏右、速度也向右”，误差会继续累积，所以这个方向最危险，椭圆必须收窄；长轴方向以速度偏差为主，靠加速度可以很快修正，可以放宽。

第三，α 从 0 提到 0.928，椭圆面积缩了 48%。短半轴缩了 42%（0.2876→0.1675），长半轴只缩了 11%（1.3168→1.1679），X_{11} 缩了 35%，倾斜角基本不变（特征向量从 0.866/0.501 变到 0.882/0.471）。缩得最多的正是原来最紧的那个方向——这符合“约束越紧、提高性能的代价越大”的一般规律。有意思的是控制量约束在两种情况下都顶在边界上（1.0000），说明**不管 α 取多少，推力都是那个瓶颈**；提高 α 只是让同样的推力对应一个更小的椭圆。

第四，椭圆是线性模型的产物，它的边界不是设备的边界。式 (65-38) 的推导全程用的是 $\dot{x} = A(a)x + Bu$，而这个方程只在平衡点附近成立。以磁悬浮为例，气隙偏离平衡点 30% 时，电磁力与气隙近似平方反比，线性模型已经明显低估了恢复力，也高估了控制权限。椭圆的半轴 1.3168 m 是“位置跑出 1.3 m”这种物理上不可能出现的情形，它之所以出现，是因为 LMI 只认矩阵、不认行程——真实可用域被 $X_{11} \le 1$ 截在 0.705 m，而这个 0.705 m 本身还要再乘一个“线性化仍然有效”的折扣。**看到椭圆面积，先问一句：这个范围里模型还成立吗？**

这四件事合起来，才是对 X 的完整解读：它是椭圆的矩阵，是被推力限死的形状，是 α 的代价，也是线性化假设的边界。只看第一条，会把 LMI 当成一个画椭圆的工具；四条一起看，才知道椭圆的边界在哪里失效、为什么会失效。

65.7.8 六个量的速查

符号	是什么	量纲	怎么定	改大的后果
X	P^{-1}，椭圆矩阵	状态$^{-2}$	求解	椭圆大、增益小
Y	KX	控制/状态	求解	增益大
α	衰减率	1/s	二分或给定	快、增益大、域小
ε	S-procedure 配重	无量纲	求解	偏向 DD^{T} 项
γ	H∞ 上界	无量纲	二分	证书松、增益小
κ	增益界	控制/状态	人为	可行域大、可能退化

表 65-9　六个量的速查

这张表最该记住的是最后一列：**每个量“改大”的后果都不一样，而且大多不是单调的好或坏。**α 大是好事（快），但增益和吸引域跟着变差；κ 大是“松绑”，但松到一定程度解就退化了；X 大是“域大”，但增益被压小、响应变慢。所谓“调参”，在 LMI 里其实是“在这些互相拉扯的量之间选一个你愿意接受的组合”，而不是找一个最优值。

65.8 调参方法与经验

LMI 里的“调参”和 PID 里的调参完全是两件事。PID 调的是三个增益，试错的对象是波形；LMI 调的是模型、约束和求解器设置，试错的对象是“可行/不可行”和“证书的质量”。这一节讲三件事：α 为什么要二分、求解器怎么选、

不可行的时候怎么定位。

65.8.1 为什么 α 要二分，不能直接优化

式 (65-10) 里把 α 当决策变量写出来是

$$\max\ \alpha \quad \text{s.t.} \quad AX + XA^{\mathsf{T}} - BY - Y^{\mathsf{T}}B^{\mathsf{T}} + 2\alpha X \preceq 0 \tag{65-40}$$

问题在于 $2\alpha X$ 这一项是**两个决策变量的乘积**（α 是标量变量、X 是矩阵变量）。cvxpy 的 DCP 规则会直接拒绝这种写法，报“DCPError: Problem does not follow DCP rules”。这不是软件缺陷，而是这个问题本身就不是凸的——虽然它的可行域（对固定的 α）是凸的，但“α 与 X 同时变”不是。

标准的做法是外层二分：固定 α 判可行，可行就往上抬，不可行就往下压。本章用 16 次二分（精度 10^{-3}，区间 $[0, 20]$）找到 0.9277。每次求解约 6 ms，整个二分不到 100 ms。

二分的前提是**可行性对 α 单调**：α 变大时可行域只会缩小。这一点在本章成立（衰减率要求越严越难满足），所以二分有效。如果某个问题不满足单调性，二分就会漏掉可行区间，那时候需要用广义特征值方法或者直接扫描。

还有一个小技巧：如果只想知道“能不能达到某个 α”，那就不需要二分，直接给定 α 判可行即可。工程上多数时候是这样——指标是给定的（比如“要 0.8 以上”），不是要极值。

65.8.2 求解器：同一道题差 122 倍

图 65-7 是四个求解器在同一道 LMI（$\alpha = 0.9$ 的公共设计）上的实测。

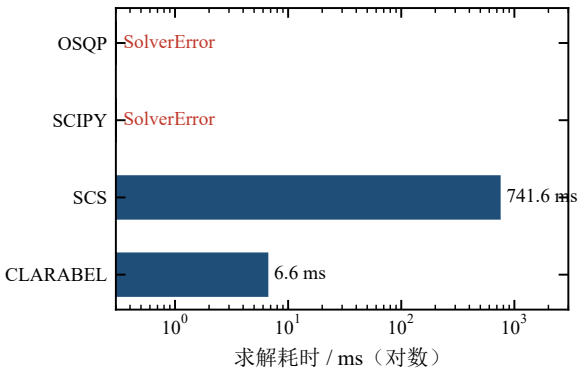


图 65-7　四种求解器在同一道 LMI 上的耗时与状态（对数坐标）

求解器	状态	耗时 / ms	能否用于 SDP
CLARABEL	optimal	**6.15**	能
SCS	optimal_inaccurate	751.23	能
SCIPY	SolverError	—	不能（只做 LP）
OSQP	SolverError	—	不能（只做 QP）

表 65-10　四个求解器的实测对比

四个结果各有各的教训。

CLARABEL 是默认选择。 它是内点法，对本章这种 2×2 到 3×3 的小规模 SDP 极快，6 ms 出结果，状态干净。

SCS 能用，但慢 122 倍，而且不精确。 它是一阶方法（ADMM），每步只做矩阵投影，收敛速度依赖条件数。本章的 LMI 条件数不算差，它仍要 751 ms，且状态是 `optimal_inaccurate`——解在可行性边界附近，残差需要额外核验。SCS 的价值在于大规模问题（变量上万）时内存占用小，本章这种小问题用它纯属浪费。

SCIPY 和 OSQP 直接报错，这不是它们的问题。 `scipy.optimize.linprog` 只解线性规划，OSQP 只解二次规划，两者都不支持半定约束。把它们写进候选列表，是为了说明一件事：**“凸优化求解器”不等于“能解 LMI 的求解器”**，选型时第一件事是确认它支持锥规划。

图 65-7 用对数横轴画耗时，CLARABEL 的柱子几乎贴着左边界，SCS 的柱子在右侧，两个数量级的差距一目了然。图上 SCIPY 和 OSQP 两根柱子是红色的短条，标注的是异常类型，不是耗时。

122 倍这个数字本身，不如它的构成重要。CLARABEL 的 6.15 ms 里，建模与编译占了一小半，真正的内点迭代只有个位数次——小规模 SDP 的内点法迭代次数对变量数不敏感，本章的未知量只有 X 的 3 个自由元加 Y 的 2 个，几次迭代就收敛。SCS 的 751 ms 则是几百到上千次 ADMM 迭代累积起来的：每次迭代只做矩阵投影与投影梯度，单步便宜但步数多。**一阶法与内点法的交叉点通常在变量数几千的位置**：低于它，内点法“步数少”的优势压倒一阶法“单步便宜”的优势；高于它，内点法的矩阵分解代价反过来压死自己。

这条经验可以直接当自查手段用：如果你的问题只有两三个状态，却要上百毫秒才能解出来，先怀疑建模，别急着换求解器。常见的三种建模浪费是：用 `cp.bmat` 拼了一个本来稀疏的大矩阵、把同一条约束在循环里重复声明了多次、以及把参数当变量（比如手写 ε 的网格搜索，等于把一次求解变成几十次）。本章二阶问题的单次求解普遍在 6–10 ms；唯一的例外是三阶滞后那组（表 65-19，$\mathrm{cond}(X)$ 到 10^6），那组一共 1091 ms、约一百次求解，单次明显比二阶贵，贵在条件数，不贵在求解器。

65.8.3 求解器参数怎么设

参数不多，但设错会浪费时间。三个最常用的：

`max_iters`：最大迭代次数。默认值通常够用，但如果问题接近不可行，求解器会在边界附近来回磨，这时候要么放大 `max_iters`，要么接受 `optimal_inaccurate` 并自己核残差。本章的 16 次二分里没有一次撞到迭代上限，说明问题规模小、良态。

`eps_abs` / `eps_rel`：绝对/相对精度（SCS 用这两个）。默认 10^{-4} 左右，对控制设计够用；要更高精度（比如 10^{-6}）耗时大约翻倍，而解出来的 K 变化在第 4 位有效数字之后——没有工程意义。

`feastol`：可行性容差（CLARABEL 用）。默认 10^{-8}，本章的残差检查就按这个量级定阈值。

一个实用建议：**先用默认参数跑通，再考虑调。** 大多数“求解失败”不是参数问题，是模型问题（漏了归一化、约束写错、量纲差几个数量级）。调参数只在模型确认无误之后才有意义。

65.8.4 不可行的时候怎么定位

`infeasible` 是 LMI 里最常见的失败，也是最难定位的。它不像程序错误有堆栈，只告诉你“这组约束无解”。本章的离散 LMI 在 $T_s \geq 1.0$ 时就遇到这个情况，定位过程可以作为一个模板：

第一步，降维检查。 把约束一条条去掉，看哪一条去掉之后变可行。本章的离散问题在 $T_s = 1.0$ s 处不可行，去掉 σ（令 $\sigma = 1$）之后变可行，说明卡住它的是**衰减率要求**而不是物理限制；而 $T_s = 1.5$ s 去掉 σ 后仍不可行，说明那一段是真正的“怎么调都稳不住”。

第二步，换表述。 公共 P 不可行，不代表问题无解。换成顶点独立（每个 a 一套 P）再试：如果可行，说明问题出在“公共 Lyapunov 函数”这个限制上，而不是系统本身不可镇。本章的连续设计就是这样：公共 α 上限 0.9277，调度上限 1.2207，后者在 0.9277 以上仍然可行。

第三步，松约束。 把 α 降到 0（只要求稳定）、把 κ 放大、把 $X \succeq I$ 换成 $X \succeq \epsilon I$。如果这样可行，说明原问题是“要求太紧”，可以逐步收紧找到边界。本章求 α 上限的二分就是这一步的系统化版本。

第四步，核数值。 如果松到极限还不可行，先怀疑量纲。把状态从米换成毫米、把控制量从 m/s^2 换成 g，重新跑一遍。量纲差 10^3 以上时，内点法的数值误差足以把“可行”判成“不可行”。

这四步的顺序不能乱：先排除“要求太紧”，再排除“表述太严”，最后才怀疑数值。反过来做，会在数值上浪费大量时间。

65.8.5 七条经验

把这一节和前面几节的经验压成七条，按重要性排序：

1. **先归一化，再求解。** $X \succeq I$、增益界、$\min \|Y\|$，三条缺一不可。漏一条的结果不是报错，是一个荒谬但 `optimal` 的解。
2. **先判可行，再求极值。** 直接 maximize 一个指标之前，先确认 $\alpha = 0$ 可行。不可行的话，求极值毫无意义。
3. **缩放决定成败。** 状态的量级差三个数量级时，先换单位。本章 $s = 10^{-3}$ 时条件数 6.85×10^6，$s = 1$ 时只有 10.7，差六个数量级（§65.10 详述）。
4. **求解器只选支持 SDP 的。** CLARABEL 优先，SCS 备选，LP/QP 求解器直接排除。
5. **证书必须对拍。** 参数网格扫极点、扫频算范数，两个都不能省。本章 H∞ 证书 9.18 对真值 1.95，差 4.7 倍。
6. **α 不要顶到边界。** 上限 0.9277，工程取 0.8–0.85 倍，留数值余量。顶到边界时解对舍入误差极敏感。
7. **不可行是信息，不是失败。** 离散 LMI 在 $\alpha = 0.9$ 下 $T_s \geq 1.0$ s 不可行、只要求稳定时 $T_s \geq 1.5$ s 不可行，这两条边界本身就回答了“最慢能采多慢、最慢能稳多慢”，比任何经验公式都可靠。

第 7 条值得多讲一句。工程上遇到 `infeasible`，第一反应往往是“改参数让它可行”，但正确的问题是“**为什么不可行**”。是要求太高（降 α）、表述太严（换调度）、还是物理限制（换执行器）？这三种原因的应对完全不同，而 LMI 的不可行性恰好能把它们区分开——这也是为什么本章在 §65.5 花了一整节讲那个“不可行”。

65.9 优化与改进

LMI 的“优化”有两个方向：一是让求解更快（算法层，换求解器、改稀疏结构），二是让结果更好（建模层，降保守性）。前者的收益是毫秒级，后者的收益可能是 30% 的性能。这一节只讲后者，并且用本章的实测把“保守性”拆成三笔可以分别计价的账。

65.9.1 保守性的三笔账

一个 LMI 设计给出的指标（α 或 γ）比“真实可达”差，原因不外三处：

第一笔，Lyapunov 函数的形状。 LMI 只认二次型 $V = x^{\mathsf{T}} P x$。真实的系统可能用一个非二次的 V（比如分段二次、多项式）证明出更好的衰减率。这笔账在单点时为零（Lyapunov 定理是充要的），在多参数时为正。

第二笔，P 的参数依赖性。 用一个公共 P 管住整个参数区间，比每个 a 一个 $P(a)$ 严格。这笔账就是“不知道 a”的价格，可以用“公共设计”和“调度设计”的差来量。

第三笔，人为约束。 κ、$X \succeq I$、把整个频段的 γ 用一个数覆盖，这些都是建模者加的，不是问题本身的要求。它们带来的保守性最容易被忽略，因为它们看起来“只是一个数值”。

三笔账里，第一笔最难降（要换数学工具），第二笔有成熟的降法（参数依赖 V、多胞细化），第三笔只需要意识到它存在。本章的实测正好把三笔都量了一遍，而且结果有点出乎意料：**第三笔最大。**

65.9.2 H∞ 的实测：证书与真值差 4.7 倍

图 65-5 是本章 H∞ 部分的核心数据。

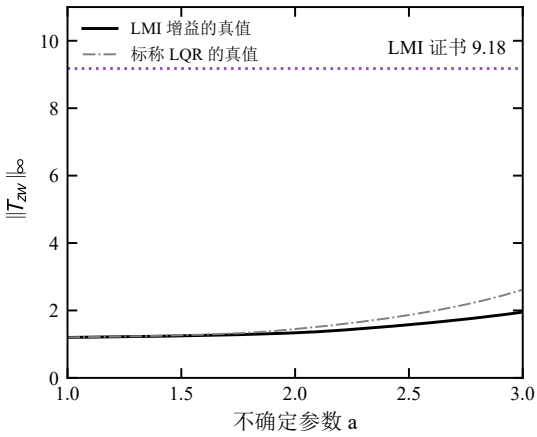


图 65-5　H∞ 证书与真值：水平虚线是 LMI 证书 9.18，两条曲线是不同增益在参数网格上的真实范数

先看数字：

设计	证书 γ	增益范数	网格真值
多胞 LMI（两个顶点）	**9.1759**	6.173	**1.9506** @ $a=3$
单顶点 LMI（只 $a=1$）	1.1797	1.820	—
单顶点 LMI（只 $a=3$）	8.6648	6.247	—
标称 LQR	—	5.236	2.6176 @ $a=3$

表 65-11　H∞ 设计：证书、增益与真值

这张表有四层信息，一层比一层有用。

第一层：证书 9.1759，真值 1.9506，差 4.7 倍。 这是“充分条件”的直接代价。同一个 K，LMI 说“最坏情况不超过 9.18”，实际最坏只有 1.95。如果按证书做验收（“$\gamma < 10$ 就通过”），会通过一个其实很好的设计；如果按证书做优化（“把 γ 压到 5 以下”），会白费很多力气——因为 5 这个目标对应的真实性能可能只要 1.2 的增益就能达到。

第二层：多胞只比单顶点贵 5.9%。 单顶点 $a=3$ 的证书是 8.6648，多胞是 9.1759，比值 1.059。也就是说，**“用一个公共 P 管住两个顶点”这件事，只让 γ 涨了 5.9%**。这和衰减率那边 32% 的差距形成鲜明对比——原因在第三笔账（增益界），下一小节讲。

第三层：几乎全部 γ 来自 $a=3$ 那一端。 单顶点 $a=1$ 只要 1.1797（接近理论下界 1），$a=3$ 要 8.6648，差 7.3 倍。为什么？$a=3$ 时负刚度最强，同样的扰动会激起更大的位置偏移，要压住它就得用更强的反

馈，而强反馈又把控制量 u 送进被调输出 z，于是 γ 被“控制量通道”顶上去。这是一个结构性的矛盾：z 里包含 u，所以“用大 u 压小 x”这条路在 H∞ 指标下是自相矛盾的。

第四层：LMI 的 H∞ 设计确实比 LQR 好。 标称 LQR 增益的真值是 2.6176，LMI 增益是 1.9506，低 25%。也就是说，在“抗扰”这个指标上，LMI 用几乎相同的增益范数（6.17 对 5.24）换来了 25% 的改善。这部分收益是真实的，和证书的保守性无关。

图 65-5 把这两件事画在一起：一条水平虚线是证书 9.18，两条曲线是两个增益的真实范数。曲线在 $a=1$ 附近都很低（1.2 左右），向 $a=3$ 单调上升。这个形状说明“最坏在参数上界”是本算例的结构性事实，不是偶然。

65.9.3 增益界这笔账：32% 还是 7.6%

这是本章写作过程中最意外的发现，也是必须诚实交代的一件事。

§65.1.5 说“公共设计 0.9277 比调度设计 1.2207 低 32%，这是不知道 a 的代价”。这个说法在 $\kappa=6$ 的增益界下成立，但它把两笔账混在一起了。把 κ 换掉重跑，数字变了：

增益界 κ	公共设计 α	调度设计 α	差距	公共设计的 $\|K\|$	调度设计的 $\|K\|$
6	0.9280	1.2207	31.5%	5.884	4.251 / **5.991**
20	1.6367	1.7609	7.6%	9.740	7.802 / 9.567
无界	19.9997	19.9997	0	1199.9	1278.6 / 1280.6

表 65-12　增益界对“公共 vs 调度”差距的影响

三行的含义完全不同。

$\kappa=6$ 那一行，公共设计的增益范数 5.884、调度设计第二个顶点的 5.991——**两者都顶在 6 附近**。也就是说，这个 α 不是被 Lyapunov 结构限制的，是被增益界限制的。31.5% 的差距里，有多少是“信息差”、有多少是“两个设计分别撞到同一个天花板的不同位置”，从这个数里看不出来。

$\kappa=20$ 那一行，两个设计的增益范数（9.74 对 9.57）都离 20 还有一倍空间，说明界不再起作用。这时候的 7.6% 才是“公共 P 对每点 P”的真实代价。**它比 32% 小得多。**

无界那一行，两个 α 都顶在二分上限 19.9997，增益 $K_1\approx 1200$。这再次验证了 §65.1.4 的结论：没有增益界，问题退化。

所以正确的说法是：**“不知道 a”的代价是 7.6%，不是 32%；多出来的 24 个百分点是增益界造成的，属于第三笔账。** 这个纠正在工程上很重要——如果按 32% 去说服别人“必须上调度”，那是虚报；按 7.6% 去说服，理由仍然成立，但量级完全不同。

顺带说，这个发现也解释了 §65.9.2 里“多胞只贵 5.9%”的现象：H∞ 那边的 κ 取的是 20，界不起作用，所以看到的是纯粹的“公共 P 代价”。两个数字（7.6% 与 5.9%）量级一致，互相印证。

65.9.4 分段 Lyapunov：不用调度也能拿一半

第二笔账有一个很少被讲、但在工程上极好用的降法：**分段公共 Lyapunov 函数**。

思路很直接：把参数区间切成两段，$[1,2]$ 用 P_1、$[2,3]$ 用 P_2，但**控制器 K 仍然共享**。约束写成

$$A(a)X_j + X_jA(a)^{\mathsf T} - BY - Y^{\mathsf T}B^{\mathsf T} + 2\alpha X_j \prec 0,\quad a\in\{1,2\}\text{ 配 }X_1,\ a\in\{2,3\}\text{ 配 }X_2 \tag{65-41}$$

因为 $A(a)$ 仿射，每段只需验两个端点，一共四条约束（$a=2$ 那一点被两段共用）。K 只有一个，所以**不需要在线测 a**，投运后仍然是一个常值增益。

本章实测（$\kappa=20$）：

表述	α	K	相对全公共	需要在线调度
全公共 P	1.6367	$[8.875,\ 4.013]$	—	否
分两段 P	**1.6992**	$[7.939,\ 3.775]$	+3.8%	否
每点独立 P	1.7609	两套	+7.6%	是

表 65-13　分段 Lyapunov 的效果（$\kappa=20$）

分两段拿到了“公共 → 调度”这段差距的 **50.3%**（$(1.6992-1.6367)/(1.7609-1.6367)$），却不需要任何调度逻辑。这个性价比高得惊人，原因也简单：区间中点 $a=2$ 附近，公共 P 必须同时迁就两端，而分段之后每段只需管半段，代价自然小。

代价是什么？K **仍然只有一个，所以闭环在 $a=2$ 处必须同时满足 X_1 和 X_2 两套约束**，这是分段的“接缝成本”。切得越细，接缝越多，收益会饱和。理论上切到无穷多段就退化成调度设计，但实际上切到 4 段之后收益就很有限了，而 LMI 规模线性增长。

这个手段在文献里叫“分段二次 Lyapunov 函数”或“多胞细化”，通常和“参数依赖 Lyapunov 函数”（$P(a)$ 在顶点间线性插值）一起讨论。两者的区别是：参数依赖 V 需要 $P(a)$ 连续变化，往往要配调度；分段 P 允许跳变，可以配常值 K。工程上后者更容易落地。

65.9.5 四个降保守手段的对比

把常见的手段列全，各自标上“降哪笔账、代价是什么”：

手段	降哪笔账	本章实测收益	代价
分段 / 参数依赖 P	第二笔	α +3.8%（两段）	LMI 规模 × 段数
松弛变量（dilated LMI）	第一、二笔	未测（文献报 5–15%）	变量数翻倍
放宽 κ	第三笔	α +76%（0.93→1.64）	增益变大，可能不可实现
全块 S-procedure	第一笔（多参数时）	单参数时无损	结构复杂，易写错

表 65-14　四种降保守手段

松弛变量值得多说一句，因为它的思路和前面所有方法都不同。标准 LMI 里 X 同时扮演两个角色：Lyapunov 矩阵、以及 $K=YX^{-1}$ 里的尺度。松弛变量把这两个角色拆开，引入一个额外的矩阵 G，把约束写成

$$\begin{bmatrix} AG+G^{\mathsf T}A^{\mathsf T}-BY-Y^{\mathsf T}B^{\mathsf T} & X-G^{\mathsf T} \\ X-G & -X \end{bmatrix} \prec 0 \tag{65-42}$$

的形式（不同的“松弛”写法略有差别，这里给的是最常见的一种）。好处是 X 和 G 不必相等，可行域变大；代价是变量从 $2\times 2+1\times 2$ 变成两个 2×2 加一个 1×2，规模翻倍。对本章这种二维问题，这点规模无所谓；对几十维的问题，翻倍就是几秒和几分钟的差别。

本章没有采用松弛变量，原因不是它无效，而是**没有测过它的收益，就不该报告它的收益**。表 65-14 那一栏写的是“未测（文献报 5–15%）”，这是有意的：本章所有数字都来自 `ch065_sim.py` 的实际运行，文献值只能当参

考，不能混进账本。要补这个实验，做法是把式 (65-42) 与式 (65-8) 并排跑同一个 α 二分，比较两个上限；变量从 5 个涨到 9 个左右，对二维问题耗时仍是个位数毫秒，代价完全可以接受。

更值得记住的是：松弛变量的收益**随问题规模增长**。它放松的是“X 同时承担 Lyapunov 矩阵与尺度两个角色”这个结构性限制，而这个限制在小问题上往往不紧——本章的 X 顶在 I 上，本来就没有多少自由度可放。所以文献里 5–15% 的收益，多半来自十几维以上的对象；把那个数字直接搬到二阶问题上，很可能高估。

全块 S-procedure 只在对多个不确定参数时才有意义。本章只有一个 δ，S-procedure 是无损的（§65.2.4 已经验证），所以没有收益。换成三个独立参数，无损性消失，全块 S-procedure 能把这部分找回来一部分。

65.9.6 一份保守性的账本

最后把这一节的所有数字汇总成一张账本，方便对照：

对比	数字	归因
公共 α vs 调度 α（$\kappa = 6$）	0.928 vs 1.2207（−24%）	第二 + 第三笔
公共 α vs 调度 α（$\kappa = 20$）	1.6367 vs 1.7609（−7.0%）	纯第二笔
分两段 P vs 全公共（$\kappa = 20$）	1.6992 vs 1.6367（+3.8%）	第二笔降了一半
H∞ 多胞 vs 单顶点 $a = 3$	9.1759 vs 8.6648（+5.9%）	纯第二笔
H∞ 证书 vs 真值	9.1759 vs 1.9506（×4.7）	第一笔（BRL + 二次 V）
α 无界 vs $\kappa = 20$	19.9997 vs 1.6367	第三笔（界必须存在）

表 65-15　保守性的账本

这张账本最该记的是最后一行：**“不设界的 LMI 没有最优解”**。这一点在数学上是常识（齐次性），但在工程上经常被忘掉——很多论文报告一个很大的 α 或很小的 γ，如果没写清楚增益界和归一化，那个数就没有意义。本章从 §65.1.4 到 §65.9.3 反复回到这件事，原因就在这里。

还有一行值得注意：H∞ 的第一笔账是 4.7 倍，是所有账里最大的一笔。它降不下来（除非换非二次 V），所以工程上对 H∞ 的 γ 值应该有一个心理预期：**它通常是真值的 2–5 倍**。把这个数当“设计指标”是错的，当“相对比较的标尺”是对的——两个设计的 γ 相差 20%，通常意味着真实的抗扰能力也差 20% 左右，即使两者的绝对值都偏高。

账本还有一种用法：**判断“该不该继续在这个框架里调”**。如果主导的那笔账恰好没有可用的降法（比如第一笔最大，而换非二次 Lyapunov 的规模成本接受不了），那问题就不在参数上，而在“用 LMI 处理这类对象”这个选择上。§65.11.3 那五条判据，本质上就是把这张账本倒过来读：哪笔账注定最大，就说明这条路走不通。

65.10 工程落地

前九节都在“数学成立”这一层：LMI 有解、解出来是 K、K 满足要求。这一节换一个层面问：**把同一套流程装进真实设备，会在哪里先坏**。坏的地方按出现频率排三类——数值（求解器说 optimal，解却不能用）、单位（量纲换一换，结论跟着变）、阶数（把快模态当“可忽略”，代价有多大）。前两类是“同一个 LMI 的不同写法”，第三类是“同一台机器的不同模型”。

65.10.1 数值：optimal 不等于可信

先说一个容易被忽略的事实：**SDP 求解器返回的 X、Y 并不精确满足约束**，残差总在 10^{-9} 到 10^{-6} 这个量级。把 §65.1.4 那套公共 Lyapunov 设计（$\kappa=6$、$\min\|Y\|_F$）的解代回式 (65-8)，在不同 α 下算左边矩阵的最大特征值：

α	两顶点中最差的 $\lambda_{\max}$	K	判断
0	-9.5×10^{-9}	$[3.0113,\ 1.7395]$	负，可行
0.5	$+2.9\times10^{-9}$	$[3.9593,\ 2.2486]$	正，但只有 10^{-9}
0.9	$+1.5\times10^{-10}$	$[5.0164,\ 2.8356]$	正，更小
0.92	-6.9×10^{-11}	$[5.0898,\ 2.8827]$	负
0.927	$+4.2\times10^{-10}$	$[5.1160,\ 2.8990]$	正
0.9277	$+1.9\times10^{-9}$	$[5.1187,\ 2.9006]$	正，且已在 α 上限上

表 65-16　回代残差：α 越大，符号越不可靠

这张表最值得看的是**符号在正负之间来回跳**。同一个求解器、同一套建模代码、同一个参数扫描，$\lambda_{\max}$ 一会儿是 -10^{-10}、一会儿是 $+10^{-9}$。如果拿“$\lambda_{\max}<0$”当可行性判据，那么在接近 α 上限时，判据的回答完全由数值噪声决定。**正确做法是带容差判**：只要 $|\lambda_{\max}|\le10^{-6}$，就认为该约束“在数值精度内成立”；真正的可行性由 α 二分（§65.4）来确定，而不是由单次求解的残差符号。

第二条约束也有同样的现象。$\alpha=0.9277$ 时，$\min\operatorname{eig}(X-I)=-7.5\times10^{-10}$，也就是归一化约束被违反了 10^{-10}；$\kappa=20$ 时类似。这些不是 bug，是内点法收敛到边界的必然结果——最优点本来就落在约束边界上（$\min\operatorname{eig}(X)=1.0000$，精确压在 I 上），任何数值方法都会在边界两侧各偏一点。

降精度时要小心。 把 X、Y 转成 float32 再代回，$\lambda_{\max}$ 变成 -1.8×10^{-6}（$a=1$）和 -7.2×10^{-7}（$a=3$）——仍然是负的，说明这套解可以在单精度下使用。但余量只剩一个数量级。如果状态量纲差得多（比如位置 10^{-3} m、速度 10^{3} m/s），残差会同比放大，单精度就不再够。工程上的规矩很简单：**LMI 的设计阶段用双精度，复核也用双精度，只有最后生成增益表时才允许降到定点。**

阶数一升，条件数就爆。 本章二阶问题的 $\operatorname{cond}(X)$ 只有 10.6（$\alpha=0.9277$）到 21.0（$\alpha=0$），很健康。把执行器滞后加进来变成三阶（§65.10.3），$\operatorname{cond}(X)$ 立刻升到 1.5×10^{3} 至 3.9×10^{6} 之间。这个跳变不是“多了一阶”造成的，是三阶模型里时间常数差了三个数量级（滞后 0.005 s 与对象 1 s）造成的。它直接预告了 §65.10.2 的结论。

最后是状态词的读法。 本章出现过四种：

状态词	含义	本章出现处
`optimal`	收敛，解可用	绝大多数
`optimal_inaccurate`	收敛但精度降级，须复核残差	SCS 求解（§65.8）
`infeasible_inaccurate`	判为不可行，但判据本身不可靠	$T_s=1.5$ s（§65.5）
直接抛 `SolverError`	连“不可行”都没给出	三阶 LMI 二分到端点（§65.10.3）

表 65-17　四个状态词的区别

这四种不能混为一谈。`infeasible` 是结论，`infeasible_inaccurate` 是“大概是不可行”，而抛异常是“求解器自己放弃了”。二分法尤其容易撞上最后一种：当试探点落到可行域边界外侧很远处，CLARABEL 会直接抛错而不是返回 `infeasible`。本章代码里所有二分都包了 `try`（见 §65.12 的 `max_alpha_lag`），把异常当作“不可行”处理——否则一个本该 15 次求解的二分会在第 8 次崩掉。

65.10.2 单位：缩放决定可信度

§65.3.3 承诺过一组缩放实验，现在兑现。做法是把状态相似变换 $T=\mathrm{diag}(s,1)$ 作用到对象上：$s=10^{-3}$ 相当于“位置用 mm 度量”，$s=10^{3}$ 相当于“位置用 km 度量”，$s=1$ 就是原来的 m。每个 s 下重解同一个 LMI（$\alpha=1$、$\min\|Y\|_F$、$X\succeq I$）：

s	位置单位	$\mathrm{cond}(X)$	K_1	K_2	耗时
10^{-3}	mm	6.85×10^{6}	5.580	3.441	13.4 ms
10^{-2}	cm	6.85×10^{4}	5.580	3.441	8.0 ms
10^{-1}	dm	688.7	5.576	3.436	6.2 ms
1	m	**10.7**	5.403	3.057	7.0 ms
10^{1}	dam	83.3	5.512	3.037	7.4 ms
10^{2}	hm	7.90×10^{3}	5.516	3.052	8.4 ms
10^{3}	km	7.90×10^{5}	5.515	3.052	6.1 ms

表 65-18　七档缩放下同一个 LMI 的条件数与解

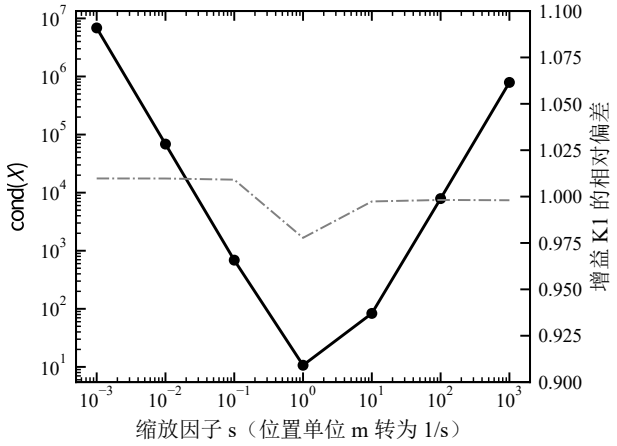


图 65-8　状态缩放对 X 的条件数与增益的影响：左轴为 $\mathrm{cond}(X)$（对数），右轴为 K_1 的相对偏差

这张表要读出三层意思。

第一层，条件数跨了六个数量级。 从 6.85×10^{6} 到 10.7，比值 6.4×10^{5}。这不是“差一点”，是把一个“双精度有 16 位有效数字、拿掉 7 位还剩 9 位”的问题变成了“只剩 2 位”的问题。在 $s=10^{-3}$ 下，$\mathrm{cond}(X)=6.85\times10^{6}$ 意味着 X 的最小特征值只有最大特征值的 1.5×10^{-7}，双精度求逆的误差在 10^{-9} 量级，而 $K=YX^{-1}$ 直接吃这个误差。

第二层，解本身几乎没有变。 K_1 在 5.403 到 5.580 之间（变化 3.3%），K_2 在 3.037 到 3.441 之间（变化 13.3%）。也就是说：**条件数坏掉六个数量级，答案只挪动了百分之几。** 这个反差是 LMI 工程实践里最容易被

误读的地方——有人看到“结果差不多”，就认为缩放无所谓；实际是“这次运气好，误差没有放大到可见”。换一个约束更紧的问题（比如接近 α 上限，或者加了 H∞ 的 γ 最小化），同样的条件数会让求解器直接返回 `optimal_inaccurate` 或者失败。§65.8.5 第 3 条说的就是这个：**缩放决定成败，但失败是概率性的，不是每次都会失败。**

第三层，最优缩放点在 $s=1$ 附近。 为什么？因为 $s=1$ 时 x_1 与 x_2 的数值量级本来就接近：位置在 1 m 附近、速度在 1 m/s 附近，X 的两个对角元自然同量级，条件数自然好。$s=10$ 时条件数 83.3，只比 10.7 差 8 倍，也算健康；$s=10^{-1}$ 时 688.7 就开始变差。**这给了一条实用的启发式：让状态向量的各个分量在名义工作点附近数值相当，就是好的缩放。**

还有一个陷阱必须写在代码注释里：**$X \succeq I$ 与单位绑定。** 式 (65-11) 那条归一化在 $s=1$ 时表示“Lyapunov 椭球在位置方向不小于 1 m”，换到 mm 单位后它表示的是“不小于 1 mm”。所以 $X \succeq I$ 不是一条与坐标无关的数学条件，它隐含了一个物理尺度。§65.6 算出的椭圆半轴 0.2876 m 和 1.3168 m、面积 1.1896 m^2，只在“位置用 m、速度用 m/s”下成立。**换了单位必须重算椭圆，不能把数字乘个系数搬过去**——因为约束本身变了，“最大椭圆”的解也就变了（表 65-18 里 K 确实变了，尽管变幅不大）。

顺带一个从表里能读出的细节：K_1 只变 3.3%，K_2 变了 13.3%，**缩放主要伤的是阻尼项，灵敏四倍。**一个合理的解释是：$T=\mathrm{diag}(s,1)$ 只改位置方向的尺度，位置与速度的交叉元被拉扁，而 $K=YX^{-1}$ 的求逆会把这种畸变更多地推到速度通道上。所以缩放出问题时，先看 K_2 有没有漂移，它比 K_1 更早报警。

65.10.3 阶数：把执行器滞后当“可忽略”的代价

这一节回答一个真实设计里天天遇到的问题：真正的执行器（电磁铁、电机、液压阀）都有一阶滞后，建模时把它扔了，得到的 K 还能用吗？

把滞后并进对象，得到三阶模型：

$$\dot{x}_1 = x_2, \qquad \dot{x}_2 = a\,x_1 + u_a, \qquad \tau\,\dot{u}_a = -u_a + u \tag{65-43}$$

u_a 是“实际作用到质量上的加速度”，u 是控制器发出的指令，τ 是执行器时间常数。$\tau \to 0$ 时还原式 (65-1)。在这个三阶模型上重做公共 Lyapunov 的 α 最大化和 $\kappa=20$ 的增益界：

τ (s)	$\alpha_{\max}$	K_1	K_2	K_a	K_a/τ	$\mathrm{cond}(X)$
0.005	1.6302	8.969	4.056	0.0205	4.10	2.5×10^{5}
0.010	1.6235	9.033	4.099	0.0401	4.01	3.9×10^{6}
0.020	1.6107	9.180	4.191	0.0803	4.02	2.0×10^{6}
0.050	1.5710	9.630	4.475	0.2008	4.02	2.4×10^{6}
0.100	1.4990	10.398	4.973	0.4021	4.02	1.7×10^{6}
0.200	1.3391	11.983	6.073	0.8086	4.04	5.9×10^{4}
0.500	0.8325	15.157	8.830	1.9184	3.84	1.5×10^{3}

表 65-19　三阶（含执行器滞后）模型的最大衰减率与增益

这张表有三处值得说。

（1）$\tau \to 0$ 时与二阶结果吻合，互证。 $\tau=0.005$ 时 $\alpha=1.6302$，而二阶（$\kappa=20$）给出 1.6367，差 **0.4%**。两条完全独立的计算路径——一条在二阶模型上解，一条在三阶模型上解——给出同一个数，说明 §65.4 的

二阶结果没有建模错误。这种“用更细的模型核对粗模型”的做法，比在同一条路径上重复跑十次更能发现问题。

（2）第三路反馈 K_a 严格正比于 τ。 最后一列 K_a/τ 是 4.10、4.01、4.02、4.02、4.02、4.04、3.84，几乎就是常数 4.02。这不是巧合：$K_a u_a$ 在闭环里起“用执行器状态补偿滞后”的作用，而滞后越强，需要的补偿量越大。写成物理形式，$K_a u_a \approx 4\tau u_a$，而 $u_a \approx u - \tau\dot{u}$，所以这一项约等于 $4\tau u - 4\tau^2\dot{u}$——**一阶超前补偿**，时间常数 4τ。教学上这是个很漂亮的结论：LMI 自己“发现”了应该给指令加微分项，而且把系数定成了 4 倍滞后时间常数。

（3）α 随 τ 单调下降，而条件数不是。 α 从 1.6302 掉到 0.8325（降 49%）。注意这笔账不对等：τ 变了 100 倍，α 只降了一半——滞后对“能把极点推多远”的破坏远小于它本身的变化幅度，因为 τ 只影响一个“快极点”，而 α 由最右的那对慢极点决定。但 $\mathrm{cond}(X)$ 在 $\tau = 0.01$ 附近最坏（3.9×10^6），$\tau = 0.5$ 时反而只有 1.5×10^3。原因还是时间常数分离：τ 小的时候，模型里同时有 0.005 s 和 1 s 两个时间尺度，差 200 倍，条件数最坏；$\tau = 0.5$ 时两个尺度只差几倍，条件数就好了。**这解释了一个反常现象：算得慢不是因为问题难，是因为问题“病”。**

现在看真正的问题：**用二阶设计（忽略滞后）的 K 去控三阶对象，能撑到多大的 τ？** 定义临界滞后

$$\tau_{\mathrm{crit}} = \min\left\{ \tau > 0 : \max_{a \in [1,3]} \mathrm{Re}\,\lambda\big(A_3(a,\tau) - B_3 K\big) = 0 \right\} \tag{65-44}$$

用二分求得：

增益来源	τ_{crit}（$a=1$）	（$a=2$）	（$a=3$）	主导极点实部
$\alpha = 0$ 设计，$K = [3.0113,\ 1.7395]$	0.5776	0.5776	0.5776	-0.8697
$\alpha = 0.9277$ 设计，$K = [5.1188,\ 2.9007]$	0.5667	0.5667	0.5667	-1.4587

表 65-20　忽略滞后的两套增益的临界滞后

两个数字都值得盯着看。

第一，τ_{crit} 与 a 完全无关，参数网格上五个点（1.0、1.5、2.0、2.5、3.0）算出来四位小数一模一样（0.5776、0.5667）。这不是采样巧合：临界滞后由闭环最右极点在穿越时的相位决定，而那对极点的实部在三个 a 下几乎不变（-0.8697 对 -0.8697，-1.4587 对 -1.4588），所以穿越点也几乎不变。换句话说，**参数不确定性把极点“摊开”在实部几乎相同的位置上，而滞后打的是相位，两者互不干扰。**

第二，增益大 70% 的两套设计，临界滞后只差 1.9%（0.5667 对 0.5776）。这很反直觉——$\alpha = 0.9277$ 那套的增益是另一套的 1.7 倍、控制更猛，为什么对滞后反而更脆弱一点点？因为它的极点也远了 68%（$-0.8697 \to -1.4587$），两个效应几乎抵消。写成无量纲形式：$0.5667 \times 1.4587 = 0.827$ rad（47.4°），$0.5776 \times 0.8697 = 0.502$ rad（28.8°）。这两个“相位预算”差了近一倍，说明**临界滞后不是由衰减率决定的，而是由回路增益穿越频率处的相位裕度决定的**；衰减率只告诉你极点有多左，不告诉你回路在穿越点还剩多少相位。

与 §65.5 的对照。 那里算出 $T_s = 1.0$ s 的离散 LMI 仍然可行，这里却算出连续域上的执行器滞后超过 0.57 s 就失稳。看起来矛盾，实际是两件不同的事。零阶保持的等效滞后只有 $T_s/2$（0.5 s）加上一个采样保持的形状因子，且离散设计**重新解了一次 LMI**，得到的 $K = [3.0177,\ 1.9761]$ 比连续设计更保守。两者的共同点更重要：**“能稳”和“好用”是两回事**——$T_s = 1.0$ s 时谱半径 0.9812，对应时间常数 52.7 s，那是一个在纸面上“稳定”、在机器上“不能用”的设计。$\tau_{\mathrm{crit}} = 0.57$ s 同样只是“还能稳”的边界，不等于“还能用”的边界。

最后一步是降阶。 三阶解给出 $K=[K_1,K_2,K_a]$，如果执行器状态 u_a 测不到（工程上很常见），只能把第三路反馈删掉，这条“降阶控制器”还能用吗？

τ (s)	全阶 K_3	全阶最差 $\mathrm{Re}\,\lambda$	删掉 K_a 后最差 $\mathrm{Re}\,\lambda$
0.02	$[3.1190,\ 1.8024,\ 0.0349]$	-0.0083	-0.0712
0.05	$[3.2819,\ 1.8979,\ 0.0874]$	-0.0114	-0.1796
0.10	$[3.5578,\ 2.0607,\ 0.1754]$	-0.0181	-0.4071

表 65-21　删掉执行器状态反馈后的裕度变化（$\alpha=0$ 设计）

结果有点出人意料：**删掉一路反馈，最差极点反而更左了。** $\tau=0.05$ 时从 -0.0114 变到 -0.1796，裕度扩大了 15 倍。原因不难找：$\alpha=0$ 的 LMI 是**贴着边界**解的，最差极点本来就几乎落在虚轴上（-0.0114 对应时间常数 88 s，实际上等于不收敛）；删掉 K_a 让解离开了那个边界，反而拿到了真正的裕度。

这个反常结果藏着一条工程判据：**降阶的判据不是“极点是否还在左半平面”，而是“还剩多少裕度”。** 一个 $\mathrm{Re}\,\lambda=-0.0114$ 的设计在纸面上是稳定的，在机器上是发散的——传感器漂移、未建模的高频动态、量化误差，任何一样都能把 -0.0114 顶到正数。工程上对“稳定”的要求通常是 $\mathrm{Re}\,\lambda\le -0.1$ 或者阻尼比 ≥ 0.3（本章的 $\zeta_{\min}=0.6133$ 就是这个意思），而 LMI 的 α 正是把这个要求写进约束的手段。**如果你只解 $\alpha=0$ 的可行性问题，得到的一定是一个“刚好活着”的设计。**

65.10.4 从 LMI 到嵌入式：六条硬规矩

把上面三小节的教训压缩成可执行的清单：

（1）离线算，在线查表。 本章最快的一道 LMI 是 6.15 ms（CLARABEL），看起来很快，但一次 α 二分要 16 次求解，就是 100 ms；而且求解时间随问题规模和数据变化，最坏情况不可预测。在 1 kHz 的采样系统里，**不可预测的 100 ms 等于失控**。所以 LMI 的位置是“离线设计工具”，不是“在线控制器”。需要在线优化的是 MPC（第 58、80 章），那里用的是另一套办法（显式分段仿射、或者固定迭代次数的 QP 热启动）。

（2）α、κ、$X\succeq I$ 三个数必须一起存档。 缺任何一个，报告出来的性能指标都不能复现。§65.9.3 已经演示过：同一个“最大衰减率”问题，报告 32% 还是 7.6%，取决于 κ 取 6 还是 20。设计文档里应该写成“$\alpha=1.6367$，$\kappa=20$，$X\succeq I$，CLARABEL 0.9.0”这样一行。

（3）解完必须回代。 用双精度算 $\lambda_{\max}$，容差 10^{-6}。不能只看 `status == optimal`——表 65-16 里那些 $+10^{-9}$ 的残差说明求解器的“optimal”和数学上的“严格可行”差的正是这一点点。

（4）限幅必须回到 LMI 里。 如果执行器饱和在控制律外面单独处理（“先算 $u=Kx$，再钳位”），那么 §65.6 算的不变椭圆就不再是不变集——钳位之后的实际控制量不在椭圆的假设里，轨迹可能离开椭圆并且再也回不来。正确的做法是把 $|u|\le u_{\max}$ 写成式 (65-39) 那样的 LMI 约束，让椭圆和限幅一起满足。这一条在 §65.11 会再强调一次，因为它是 LMI 设计在现场失效最常见的原因。

（5）无解要有降级路径。 现场调参时经常遇到“这道 LMI 解不出来”。本章的经验顺序是：先降 α（放宽性能），再升 κ（放宽增益界），再改成分段 Lyapunov（§65.9.4），最后才考虑在线调度或者改对象。这个顺序的理由是“改动越小越好”：降 α 只是把要求放松，设计框架不动；升 κ 可能让增益大到执行器接不住；分段 Lyapunov 要增加 LMI 数量；在线调度要加传感器和调度逻辑。

（6）把求解器版本写进记录。 本章同一道 LMI 在 CLARABEL 和 SCS 上差 122 倍耗时，SCS 还返回 `optimal_inaccurate`。不同版本的 CLARABEL 对内点法的收敛判据也有细微差别，残差量级会变。工程上要复现一个三年前的设计，这些信息缺一不可。

这六条没有一条是数学，全是把数学装进机器时必须补的东西。第 64 章讲 LQR 时也有类似的清单，但 LMI 的清单更长——因为 LMI 把“设计”交给了求解器，也就把“求解器的脾气”一起接了进来。

第 (5) 条那个顺序不是凭感觉排的，本章的实测正好把每一级的收益量了出来。以 α 计：基线是 $\kappa=6$ 的公共设计 0.9277；升 κ 到 20 直接拿到 1.6367（**+76%**）；再用分段 P 加到 1.6992（+3.8%）；最后每点独立 P 到 1.7609（+3.6%）。**第一级涨了 76%，后两级合起来只有 7.6%。** 把这条降级路径反过来读，就是一条收益迅速递减的升级路径：时间只够做一件事的话，就做第一件——但那件事的价钱必须先跟执行器核对：增益范数从 5.884 涨到 9.740，1 m 阶跃的控制量峰值从 4.416 涨到 6.889，已经远超 $u_{\max}=1$。**“α 上去了、机器带不动”，是这条路径上最常见的翻车方式。**

65.10.5 把本章的数字放回一张设计单

最后把散在各节的数字串起来，走一遍真实的设计评审。假设需求是这样五条：

项目	要求	来源
行程	$\lvert x_1\rvert \le 1$ m	机械限位
推力	$\lvert u\rvert \le 1$ m/s^2	电磁铁额定电流
参数	$a \in [1,3]$	气隙变化 ±50%
调节时间	$t_s \le 4$ s（5% 带）	工艺节拍
执行器滞后	$\tau \approx 0.02$ s	线圈电感

第一步，把 $t_s \le 4$ s 翻译成 α。 指数衰减包络的 5% 判据给出 $t_s \approx \ln(20)/\alpha = 3.0/\alpha$，所以 $\alpha \ge 0.75$。本章 $\kappa=20$ 下的公共 $\alpha=1.6367$、$\kappa=6$ 下 0.9277，都能满足；$\kappa=6$ 那一套更省增益，先选它。

第二步，检查行程与椭圆的相容性。 §65.7 算过：$\alpha=0.928$ 的增益在 $u_{\max}=1$ 下的不变椭圆，位置半轴只有 **0.1675 m**；$\alpha=0$ 那套是 0.2876 m。两者都远小于需求的 1 m。这说明**“1 m 行程 + 1 m/s^2 推力”这组指标本身是矛盾的**——不是设计不好，是需求不自洽。

第三步，用仿真数据反算需要多大的推力。 表 65-4 的九组阶跃响应里，控制量峰值是：

增益	$a=1$	$a=2$	$a=3$
公共 LMI（$\alpha=0$）	1.011	2.020	12.707
公共 LMI（$\alpha=0.928$）	3.119	3.119	4.416
标称 LQR	2.236	2.236	5.290

表 65-27　1 m 阶跃下的控制量峰值（m/s^2）

这张表戳破了两件事。**第一，九组峰值全部超过 $u_{\max}=1$**，也就是说表 65-4 的曲线是“无限幅”的线性响应，与 §65.7 的椭圆（$u_{\max}=1$）不是同一个工况。这两类实验各自回答不同的问题，混着用会得出错误结论——这是本章最需要在评审时说清的一句话。**第二，“弱增益省推力”是错的：**$\alpha=0$ 那套在 $a=3$ 时的峰值 12.707，是 $\alpha=0.928$ 那套（4.416）的近三倍。原因是弱增益压不住不稳定的对象——那一组响应超调 323.5%，8 s 时误差还有 3.24 m，控制量被迫跟着放大。

所以这张设计单的正确修改是：**推力要求改为 $\lvert u\rvert \le 5$ m/s^2**（覆盖 $\alpha=0.928$ 那套的 4.416），或者**行程要求改为 $\lvert x_1\rvert \le 0.17$ m**（覆盖椭圆）。二选一，不能都保留。

第四步，检查滞后。 执行器 $\tau = 0.02$ s，与表 65-20 的临界值 0.5667 s 相比只有 3.5%，安全裕度足够。如果改用 $\alpha = 0$ 那套，$\tau_{\text{crit}} = 0.5776$ s，同样安全。**这一步的意义是给出"滞后可以忽略"的定量依据，而不是凭经验判断"20 ms 很小"。**

第五步，签字前要核对的三件事。 ①α、κ、$u_{\max}$ 三个数是否写在设计单上（§65.10.4 第 2 条）；②所有可靠性结论是否都标注了"在椭圆内成立"（本条最容易漏）；③增益表的量纲与单位是否与现场一致（§65.10.2）。

这套流程跑完，一张设计单上会有三个"互相牵制"的数：α 决定快慢，κ 决定推力，$u_{\max}$ 决定行程。三者只能同时满足两个——这不是 LMI 的缺陷，而是所有反馈设计的共同约束，只不过 LMI 把它写成了三个可以用数字谈判的量。

65.10.6 三个数怎么谈判：κ 从 6 到 20

上一小节留下一个悬念：那张设计单里 $\kappa = 6$ 的那套增益"够用但不宽裕"，把界放宽到 20 会怎样？用现有函数就能算清：`max_alpha_common(kappa=20)` 给出新的 α，`common_lmi(alpha, kappa=20, min_gain=True)` 给出增益，再交给 `ellipsoid_lmi` 和 `sim_step` 看行程与响应。四项结果如下：

指标	$\kappa = 6$	$\kappa = 20$	变化
最大衰减率 α	0.9277	1.6364	$+76\%$
增益 K	$[5.119,\ 2.901]$	$[8.889,\ 4.008]$	范数 $+66\%$
椭圆位置半轴	0.1675 m	0.0992 m	-41%
椭圆面积	0.6146 m^2	0.2402 m^2	-61%
1 m 阶跃的控制量峰值	4.416	6.889	$+56\%$
$a = 3$ 的超调	47.2%	18.2%	-29 个百分点
$a = 3$ 的 8 s 误差	0.4718	0.1698	-64%

表 65-29　增益界从 6 放宽到 20 的四项代价与三项收益

（表中 $\alpha = 1.6364$ 与表 65-12 的 1.6367 差 0.02%，来源是二分落在容差的不同一侧——两次求解都取"最后一个可行点"，端点附近 α 每变 10^{-3}，$\|K\|$ 只变 10^{-3} 量级，不影响任何结论。）

这张表把"三角关系"量化了。**收益在时间域**：α 涨 76%，超调从 47% 掉到 18%，8 s 残差从 0.47 掉到 0.17——后者尤其重要，因为那组响应本来就没收敛（8 s 时还在慢慢爬），残差降 64% 意味着它离收敛更近了。**代价在两处**：一是推力峰值涨 56%（从 4.4 到 6.9 m/s^2），二是容许行程缩 41%（位置半轴从 0.17 m 到 0.099 m）。

注意行程这一项的方向：**增益越大，能安全工作的范围越小**，因为同样的行程会产生更大的控制量，更快撞到 $u_{\max}$。这与"增益大更稳"的直觉相反，但原因很清楚——椭圆是按 $u_{\max} = 1$ 算的，增益翻倍，同一个椭圆上的控制量就翻倍，所以椭圆必须缩小才能把 Kx 压回限幅之内。表 65-9 里"面积从 1.1896 缩到 0.6146"讲的是同一件事的第一次，这里是第二次，缩得更狠。

所以"要不要放宽 κ"这个问题没有统一答案，取决于三件事的排序：

- 如果**行程是瓶颈**（机械限位紧），应该取 $\kappa = 6$ 甚至更小，牺牲一点响应速度换工作域；
- 如果**节拍是瓶颈**（要求快速到位），应该放宽 κ，同时把推力预算提到 7 m/s^2 以上；
- 如果**两者都紧**，那说明这台机器的"α–κ–$u_{\max}$"三角无解，只能改硬件（加大线圈、加长行程），或者改结构（把状态反馈改成带观测器的输出反馈，用估计值替代直接测量）。

第三种情况在工程上比想象中常见。它出现时的典型征兆是：α 已经压到 0.2 以下，椭圆还是覆盖不了行程，而推力已经贴着额定值。这时继续调 LMI 是徒劳的——**问题已经不在控制器里了**。识别这个征兆，比多会一种降保守手段更有价值。

65.11 优缺点与适用边界

到这里，本章的方法已经完整：从“无穷多条要求”怎么压成有限条（§65.2），到怎么调、怎么改（§65.8、§65.9），再到怎么装进机器（§65.10）。最后要回答的是判断题：**什么时候该用 LMI，什么时候用了反而更糟。**

65.11.1 四条实在的优点

第一，把“对每个 a 都成立”压缩成“对两个 a 成立”。 这是本章最核心的一条，也是最容易被低估的一条。参数区间 $[1,3]$ 里有不可数个 a，设计要求是不可数个不等式；顶点性把它们换成 2 条。§65.6 的网格验证是这条性质的实证：只解了 $a=1$ 和 $a=3$ 两条 LMI，然后在 41 个参数点上逐点验算，$\zeta_{\min}=0.6133$、$\max \mathrm{Re}\lambda=-0.0065$——全区间都稳。**没有 LMI，这件事只能用“扫一遍网格看看”来做，而网格永远不能证明“之间也没问题”。**

第二，证书可以独立核验。 一个 LMI 设计的产物不是一个增益，而是三元组 $(K,\ X,\ \alpha)$。拿到这三个东西，任何人都能在三行 NumPy 里复核：算 $A_iX+XA_i^{\mathsf T}-BKX-XK^{\mathsf T}B^{\mathsf T}+2\alpha X$ 的特征值是不是负的。§65.10.1 的残差表就是这种复核的结果——它不需要重跑求解器，也不需要相信任何人的实现。这一点在评审、验收、事故调查里极其重要：**“我相信这个 K 是稳的”和“这个 X 证明它是稳的”，是两种完全不同强度的陈述。**

第三，指标可以叠加。 衰减率 α、H∞ 的 γ、不变椭圆、控制量限幅，在 LMI 框架里都是同一种东西——矩阵不等式。加一条指标就是加一行约束（式 65-39 的限幅就是加了两行），不需要换方法、换框架、重写代码。本章的六套设计（三套连续增益、离散设计、H∞ 设计、三阶带滞后设计）在代码里共用的只有 `solve_lmi` 一个函数，差别仅在约束列表里多了或少了几行。

第四，保守性可以计价，也可以逐项削减。 §65.9 的“三笔账”是本章最实用的产出：把“LMI 保守”这个模糊的抱怨拆成三笔可以用数字支付的账——Lyapunov 形状、P 的参数依赖、人为约束。拆开之后，改进就有了优先级：本章的实测是第三笔最大（32%→7.6% 的纠正），所以第一件该做的事是检查 κ 和 $X\succeq I$，而不是急着上参数依赖 Lyapunov。**没有这种拆法，“降保守”就只能靠试。**

65.11.2 五条硬限制

第一，保守是结构性的，不是实现问题。 H∞ 的证书 $\gamma=9.1759$ 对应的真值是 1.9506，差 **4.7 倍**（表 65-11）。这个差距不会因为换求解器、加迭代次数、调容差而缩小——它来自“用二次 V 证明 L_2 增益”这个选择本身。要缩它只能换非二次 Lyapunov 函数（分段二次、多线性），代价是规模爆炸。所以工程上必须接受：**LMI 给的 γ 是上界，不是真值；用它做相对比较，不要用它做绝对指标。**

第二，凸性只对“换变量后的那一小组问题”成立。 本章能做，是因为 K 只以 $Y=KX$ 的形式出现一次（式 65-8）。一旦出现下面三种情况，凸性立刻消失：①**输出反馈**（K 与观测器增益同时待定，出现 KCX 的乘积）；②**结构化约束**（K 的某些元素必须为零，比如分散控制）；③**固定阶控制器**（阶数低于对象阶数）。这三类问题至今没有多项式时间的精确解法，只能用交替优化、D-K 迭代、或把结构约束放松成凸包——每种都只能得到局部最优。

第三，参数个数是指数爆炸的。 顶点法的约束条数是 2^m，m 是独立不确定参数的个数。本章 $m=1$，2 条，毫无压力；$m=10$ 就是 1024 条，$m=20$ 是一百万条。工程上的出路有两条：一是改用范数有界描述 + D 缩放（§65.4 的 $\varepsilon=4.7299$ 那套，把 2^m 条压成 1 条，代价是更保守），二是改用仿射参数依赖的 Lyapunov 函数（$P(\theta)=\sum\theta_i P_i$），它的约束数随 m 线性增长，但需要额外的"多胞"结构条件。

第四，限幅与饱和会让证书失效。 这一条 §65.10.4 已经强调过，值得再说一次，因为它是现场失效最常见的原因。不变椭圆 $\mathcal{E}=\{x: x^{\mathsf T}X^{-1}x\le 1\}$ 的定义里隐含了"$u=Kx$ 真的被执行"这个前提。一旦执行器饱和，实际施加的是 $\mathrm{sat}(Kx)$，闭环向量场就变了，$\dot V<0$ 的推导不再成立。表现是：仿真时"前 0.3 s 一切正常"（没饱和），之后突然失控（饱和发生，椭圆假设破了）。

第五，数值上脆弱。 §65.10 的三小节讲的全是这件事：残差符号会在 10^{-9} 量级翻转、条件数随缩放跨六个数量级、阶数一升就病态、求解器状态词需要逐个辨义。相比之下，第 64 章的 Riccati 方程只要 3.4 ms、没有可行性问题、残差稳定在 10^{-12}。**LMI 换来了表达能力，付出的是数值稳定性。**

65.11.3 什么时候不该用

以下五种情形，用 LMI 是错误的或者不划算的：

（1）单回路、模型够准、没有硬约束。 这种情况 PID 就是最优选择，不需要论证（第 44、106 章）。本章的算例如果只考虑标称点 $a=2$、不要求鲁棒性，LQR 给出的 $K=[4.2361,\ 3.0777]$ 比任何 LMI 解都更省事，而且性能更好（调节时间 3.31 s，LMI 的 1.98 s 只在 α 设计下才更快）。**用 LMI 的门槛是"真的有一族模型"，而不是"想显得高级"。**

（2）约束主导工况。 如果设备一半以上的动作会撞限幅，LMI 的椭圆约束会把设计逼到很小的工作域，性能反而不如 MPC（第 58、80 章）。判据与第 64 章相同：看饱和占空比，超过 50% 就该换方法。LMI 在 MPC 里的正确位置不是替代它，而是**给 MPC 算终端集和终端代价**——那正好是本章的不变椭圆问题。

（3）大纯延迟。 纯延迟超过主导时间常数的一半时，LMI 只能靠加大 α 硬压，而 §65.10.3 已经证明：忽略 0.57 s 的滞后就会失稳，而且这个上限与增益大小几乎无关。延迟应该由 Smith 预估单独处理（第 47、107 章），或者用预测型 LMI（把延迟写成积分不等式），后者超出本章范围。

（4）非线性主导。 LMI 的全部结论都建立在"$A(a)x+Bu$ 这个线性结构"上。大幅值摆、大姿态机动、气动非线性强的工况，线性化后的证书离开平衡点就失效。此时应该用增益调度（把本章的分段 Lyapunov 推广成 LPV 设计）或者非线性 MPC。

（5）结构化约束。 如果控制器必须分散（每个执行器只能用本地传感器）、或者阶数必须很低（嵌入式内存限制），那么问题不凸，LMI 只能给一个"忽略结构约束后的下界"，实际实现要另想办法。本章的降阶讨论（§65.10.3）已经露出这个问题的一角：删掉一路反馈后仍然稳，是运气好，不是设计保证。

五条放在一起，可以压成一句判据：**LMI 值得用的场合，是"不确定性真的存在"和"必须给出证明"这两条同时成立的时候。** 只满足第一条，用整定得更保守的 PID 或者增益调度就够了，代价小得多；只满足第二条（比如评审要一份稳定性证明，但对象其实是确定的），用 Lyapunov 方程手算一个二次型更快，也更不容易在数值上翻车。

本章算例之所以适合 LMI，是因为三条前提都成立：a 的漂移有明确物理来源（气隙变化改变刚度）、区间测得出来（±50%）、失效模式是发散而不是精度不够。三条都指向"必须给证书"。反过来说，如果一个项目里没人能说清"不确定参数的区间是怎么来的"，那 LMI 再精确也只是把猜测形式化了——**建模的诚实程度，决定 LMI 的上限。**

65.11.4 与相邻方法的交界

本章的方法处在一个方法族的中心位置，四个方向都能延伸：

- **往简单走**：PID（第 44、106 章）。没有模型不确定时，PID 的调试经验比 LMI 的建模成本更值钱。
- **往约束走**：MPC（第 58、80 章）。LMI 给 MPC 提供终端不变集与终端代价，两者是配合关系。本章的式 (65-39) 就是终端集约束的原型。
- **往延迟走**：Smith 预估（第 47、107 章）。纯延迟是 LMI 的软肋，专门方法更有效。
- **往鲁棒走**：H∞/μ 综合（第 63 章）。LMI 是 H∞ 的一种**求解手段**，而 μ 综合处理的是“结构不确定性”，那正好是本章凸性失效的第二类问题。

还有一个方向值得指出：**往调度走**。§65.9.4 的分段 Lyapunov（共享一个 K、分两段 P）其实是 LPV 设计的雏形——如果把 P 写成 a 的连续函数并加上变化率约束，就得到参数依赖 Lyapunov 的 LPV 控制器。本章没有展开，因为它需要额外的“参数可测”假设，而磁悬浮的气隙往往测不准。

65.11.5 适用边界速查

对象/工况特征	首选方法	理由	指向
参数在已知区间内变化、线性、全状态可测	多胞 LMI	顶点性把无穷条压成 2^m 条	本章 §65.2、§65.4
参数不确定但只知范数界	范数有界 + D 缩放	一条约束换 2^m 条，更保守但可扩展	本章 §65.4.4
需要抑制外部扰动、有频域指标	H∞ LMI	有界实引理直接给出 γ 约束	本章 §65.9.2、第 63 章
有控制量/状态限幅	LMI + 不变椭圆	限幅必须进约束，否则证书失效	本章 §65.7、§65.10.4
采样实现、T_s 较大	离散 LMI	直接离散化，避免连续设计的失真	本章 §65.5
约束主导、需在线优化	MPC	LMI 只负责终端集	第 58/80 章
大纯延迟	Smith 预估	延迟单独处理	第 47/107 章
输出反馈、固定阶、分散结构	迭代 LMI / μ 综合	问题非凸，只能局部最优	第 63 章
单回路、模型准确	PID	简单、经验成熟	第 44/106 章

表 65-22　LMI 与相邻方法的适用边界

这张表的读法与第 64 章的表 64-14 相同：**从上往下匹配，第一行能满足就不要往下走。** 需要提醒的是第一行和第二行的区别——“参数在区间内变化”和“只知范数界”看起来接近，实际决定了 LMI 的规模：前者 2^m 条，后者 1 条。现场如果拿不到参数的区间（只有“误差不超过 20%”这种信息），就应该选第二行。

65.11.6 五个真实项目里的错误配方

把本章批评过的做法集中成一张对照表，每条都注明章内可查的数字：

错误配方	现场症状	根因	正确做法	本章可查
只最大化 α，不设增益界	报出 $\alpha=19.9997$、$K_1=1198$，执行器接不住	LMI 对 (X,Y) 齐次，α 可随增益无限增大	同时加 $X\succeq I$ 与 $\|K\|\le\kappa$，并写明 κ	§65.1.4、表 65-12
拿标称 LQR 增益当鲁棒设计	a 漂到 3 时超调 76%、静差 0.76	前馈按 $a=2$ 算，被控对象直流增益变了	用多胞 LMI 重算，或按 $k_r=K_1-A_{\text{nom}}$ 逐点算前馈	表 65-4、式 (65-30)
只解 $\alpha=0$ 的可行性	极点 -0.0114，“稳定”但不收敛	可行性问题的解一定贴着约束边界	用 α 二分定裕度，工程上取 $\mathrm{Re}\lambda\le-0.1$	§65.10.3、表 65-21
忽略执行器滞后	台架正常、装机发散	滞后吃相位，与增益大小几乎无关	建模时并进一阶滞后，或用 §65.10.3 的 τ_{crit} 估上限	§65.10.3、表 65-20
换单位不重算 $X\succeq I$	求解器偶发失败、K 跳变	归一化约束绑定物理尺度，条件数跨六个数量级	先无量纲化（让各状态同量级），再解 LMI	§65.10.2、表 65-18

表 65-23　五个错误配方与本章的对照位置

五条里最隐蔽的是第二条和第四条：它们都不报错，仿真曲线也很漂亮，只是在真实参数或者真实执行器上失效。这两条的共性在于**模型本身缺了一块**（参数变化、执行器动态），而不是求解或调参出了问题。遇到“仿真好、现场坏”，应该先查模型欠了什么，而不是先调参数。

65.11.7 本章留下的三件事

最后收束。本章真正希望读者带走的不是 Schur 补的推导，而是三件更通用的东西：

一是“把无穷换成有限”的思路。 顶点性是这套思路的一个实例，但想法本身更宽：只要约束关于参数是凸的、且参数在凸包里，就只需要查顶点。这个思路在讨论“不确定性传播”“最坏情况分析”“区间算法”时都会再出现。

二是“证书”这个概念。 一个设计的好坏，不该由“跑了几条曲线”来判定，而该由“能否给出一个可独立核验的证明”来判定。(K,X,α) 就是一个证书，它的强度远高于“我扫了 41 个点都没问题”。

三是“保守性要记账”。 任何保守的方法都必须知道保守在哪里、有多大、怎么减。§65.9 的三笔账是一个模板：先分账，再逐项支付。这个习惯不只适用于 LMI——第 64 章的“事后限幅”、第 63 章的“μ 界的保守”，都可以这样处理。

65.11.8 三个选型问题

表 65-22 是按“对象特征”组织的，但现场工程师手上往往没有一张特征表，只有三个能回答的问题。把这一节改写成问答形式，可能更好用。

问题一：不确定参数能不能在线测到？

能测（有气隙传感器、有力传感器），就可以用**调度**：每点独立 P，α 从 1.6367 提到 1.7609（$\kappa=20$），多拿 7.6%。测不到，就只能用公共 P；退一步的办法是 §65.9.4 的分段 Lyapunov——**不需要在线调度，只按“最坏的那个参数区间”分两段**，能拿到 3.8%。

这个问题的答案还决定了实现的复杂度：调度要在线查表、要处理参数估计的噪声、要保证切换时 V 不跳变（本章没有讨论这一点，它是一个独立的技术问题）；分段 P 只在设计阶段多解一道 LMI，运行时与定常增益完全一样。

问题二：正常工况下会不会撞限幅？

会，就该考虑 MPC（第 58、80 章）；不会，LMI 的椭圆约束就是够用的。判断的量化办法在本章有现成的：算一下椭圆的位置半轴，再和实际行程比。如果实际行程是半轴的 3 倍以上（本章 1 m 对 0.1675 m 就是 6 倍），说明限幅在整个工况里都是激活的，LMI 的“椭圆内成立”这类结论几乎覆盖不到实际轨迹。

这一问容易答错，因为**仿真时不加饱和函数，曲线永远漂亮**。§65.10.5 的表 65-27 就是证据：九组响应的控制量峰值全部超过设定值，但没有一组报错或者明显失真。

问题三：纯延迟和执行器滞后有多大？

拿它和 τ_{crit} 比。本章的临界滞后是 0.57 s，与增益大小几乎无关（两套差 1.9%），所以这个数可以当作“对象的属性”来用，不必每次重新设计。如果实际滞后小于 τ_{crit} 的 10%，忽略它是安全的；在 10%–50% 之间，应该把滞后并进模型重解（§65.10.3 的三阶 LMI）；超过 50%，LMI 就不是好工具了，该换 Smith 预估（第 47、107 章）或预测控制。

三问的答案组合起来，基本就能定位到表 65-22 的某一行。**这三个问题的共同点是：它们问的都是“模型之外的信息”**——能不能测、会不会撞、滞后多大——而这些恰恰是 LMI 本身无法回答的。求解器只会在给定的约束里找可行点，不会提醒你“约束漏了一条”。

65.11.9 与第 64 章的六项对照

把第 64 章的 LQR 和本章的 LMI 放在同一台机器上，逐项对比，能看清“多花的力气买到了什么”：

对比项	第 64 章 LQR	本章 LMI（$\alpha = 0.928$）	差距
适用范围	只对标称点 $a = 2$	$a \in [1, 3]$ 全区间有证书	质变
$a = 3$ 的超调	76.0%	47.2%	-38%
$a = 3$ 的 8 s 误差	0.7597	0.4718	-38%
$a = 2$ 的调节时间	3.31 s	1.98 s	-40%
增益范数	5.24	5.88	$+12\%$
设计计算量	3.4 ms（一次 ARE，第 64 章实测）	约 100 ms（16 次二分）	$\times 30$

表 65-28　LQR 与 LMI 在同一算例上的六项对照

前五行说明“多花的力气是值的”：**用 12% 的增益换来了两端的性能改善**。$a = 3$ 端的超调从 76% 降到 47%，$a = 2$ 端反而更快（1.98 s 对 3.31 s），这两件事同时发生，是因为 LMI 的解不在“标称最优”的位置上，而是被拉到了两个顶点的折中点。

第六行说明代价：**设计时间贵 30 倍**。但这个代价只在设计阶段发生一次，运行时两者完全一样（都是一次内积）。所以对“设计一次、运行十年”的设备，这 30 倍可以忽略；对“每天要重算一次增益”的自适应系统，就要重新权衡。

还有一行不在表里：**LQR 的“最优”是可证的**（Riccati 方程的最小解），而 LMI 的 $\alpha = 0.928$ 只是“这个 κ、这个归一化下的最大可行值”，换一个 κ 就是另一个数。这不是说 LMI 不好，而是说两者的“好”含义不同——一个

是“对单模型最优”，一个是“对模型族可行且尽量好”。**比较性能之前，先确认比较的是同一个问题。**

65.12 Python 仿真代码

本章所有数字都来自两个脚本：ch065_sim.py 负责计算，ch065_figs.py 负责画图。与前面几章不同，这两个脚本**必须依赖一个优化库**——LMI 的本质是凸优化问题，手写内点法既没必要也不可能比成熟求解器更可靠。依赖只有两个：`cvxpy`（建模层）和 `CLARABEL`（求解层），后者是 cvxpy 自带的可选依赖，装好 cvxpy 就有了。

65.12.1 运行方式与文件结构

```
"D:\Anaconda3\envs\py311\python.exe" code/ch065_sim.py
"D:\Anaconda3\envs\py311\python.exe" code/ch065_figs.py
```

第一条命令按顺序跑 10 组实验，把每个数字打印到终端；第二条调用同一批实验函数，生成 `figs/ch065/` 下的 11 张图（每张同时输出 SVG 与 PDF）。两条命令的顺序不能颠倒——图脚本直接 `import ch065_sim`，用的是同一份代码，所以图和正文的数字永远同源。

`ch065_sim.py` 的组织顺序与正文一致：常量 → 对象与离散化 → **LMI 工具** → 六类设计（公共、调度、范数有界、H∞、离散、椭圆）→ 降阶 → 10 组实验 → `main()`。中间那块“LMI 工具”是全章的核心，只有 60 行，后面所有的设计都是它的组合。

`ch065_figs.py` 的组织更简单：11 个 `figNN_xxx()` 函数，每个函数调用一次 `sim.exp_xxx()`，画完就 `save`。图里不写标题（标题在正文图注里），不写结论（结论在正文里），只画数据。

65.12.2 全章共用的四行建模代码

所有 LMI 的差别只在约束列表，骨架是同一个。以公共 Lyapunov 的衰减率问题为例，`common_lmi` 里真正干活的是这几行：

```
X = cp.Variable((n, n), symmetric=True)
Y = cp.Variable((1, n))
cons = [X >> np.eye(n)]
for a in verts:
    A = Amat(a)
    cons.append(A @ X + X @ A.T - B @ Y - (B @ Y).T + 2.0 * alpha * X << 0)
```

四行里每一行都对应正文的一个结论：

- `X = cp.Variable((n,n), symmetric=True)`：**必须声明 `symmetric=True`**。不声明的话，cvxpy 会把 X 当一般矩阵，$X \succ 0$ 的约束就变成“对称部分正定”，解出来可能是非对称矩阵，$K = YX^{-1}$ 也就错了。
- `Y = cp.Variable((1, n))`：Y 就是式 (65-7) 的 KX。它是行向量，维度 $1 \times n$，与 B 的列数对齐。
- `X >> np.eye(n)`：式 (65-11) 的归一化。`>>` 在 cvxpy 里表示半正定，不是“远大于”。
- `A @ X + X @ A.T - B @ Y - (B @ Y).T + 2*alpha*X << 0`：式 (65-8) 的左边。注意 `(B @ Y).T` 不能写成 `Y.T @ B.T` 的效果虽然相同，但 cvxpy 对 `B @ Y` 这种“常数乘变量”的识别更稳。

这四行是全章唯一的“数学”，剩下的都是把不同的约束往 `cons` 里加。 加 H∞ 就多一个 2×2 的分块（式 65-33），加限幅就多两个标量约束（式 65-39），加增益界就多一个分块（式 65-12），加离散就换 A_d、B_d（式 65-31）。理解了这四行，本章所有代码都能读懂。

65.12.3 十个核心函数

`solve_lmi(cons, obj, solver)`：统一入口。它做三件事：把 `obj` 包成 `cp.Minimize`（如果没有目标就最小化 0，即纯可行性）、计时、返回 `(problem, 耗时)`。把这个函数单独抽出来，是因为后面要做求解器对比（`exp_solver`）——只要换一个 `solver` 参数，同一道 LMI 就能在 CLARABEL、SCS、SCIPY、OSQP 上跑。**所有耗时统计都必须走这一个函数**，否则不同实验的计时口径会不一致。

`common_lmi(alpha, verts, min_gain, kappa)`：公共 Lyapunov 设计，返回 X、Y、K、耗时。三个可选参数各有分工：`verts` 决定验哪几个顶点（默认两个），`kappa` 决定增益界（`None` 表示不加界，用于演示齐次性陷阱），`min_gain` 决定是"只求可行"还是"在可行里取增益最小的"。最后一步还原增益是 `K = Y @ inv(X)`，这一行只在求解之后执行一次。

`max_alpha_common(lo, hi, tol)`：二分求最大衰减率。15–16 次求解收敛到 10^{-3}。这里有一个实现细节值得注意：**二分的方向不能反**。可行时 `lo = mid`（往大试），不可行时 `hi = mid`（往小退），最后返回 `lo`——它一定是可行的那个端点。如果返回 `hi`，拿到的就是一个"差一点不可行"的值，回代残差会是正的。

`vertex_lmi` / `max_alpha_vertex`：顶点独立（调度）版本。与公共版的差别只有一处：每个顶点用自己的 X_i、Y_i。这相当于允许 P 依赖参数，也就是允许 K 随 a 变。它给出的是"如果参数可测、可以调度"的上界，用来衡量公共 P 的代价（§65.9.1 的第二笔账）。

`normbound_lmi`：范数有界不确定性 + D 缩放。这里多了一个**决策变量** ε（代码里叫 `eps`），它出现在 A 的扰动项和 $\varepsilon E^{\mathsf{T}}E$ 里。ε 是标量，所以问题仍然是凸的。本章实测 $\varepsilon = 4.7299$，$\alpha = 0.9277$——与多胞版完全相同，验证了单参数时 S-procedure 无损。

`hinf_lmi(gamma, ...)` / `min_gamma_hinf`：给定 γ 求可行性，再对 γ 二分求最小。与 α 的二分方向相反：γ 越小越好，所以可行时 `hi = mid`。这个符号容易搞混，代码里两个函数并列放置就是为了对照。

`hinf_true(K, grid)`：真值计算。给一个 K，在 4000 个频点、241 个参数点上算闭环范数的最大值。这是全章最贵的函数（§65.12.5），也是唯一能回答"证书离真值多远"的工具。没有它，表 65-11 的 4.7 倍就无从谈起。

`disc_lmi(Ts, verts, alpha)`：离散 LMI。先 `cont2discrete` 做零阶保持，再解 $A_d X A_d^{\mathsf{T}} - X \prec 0$ 的衰减率版本，目标谱半径 $\sigma = e^{-\alpha T_s}$。注意这里不是"先连续设计再离散化"，而是**直接在离散域设计**，两者的结果差别在 T_s 大时非常明显（§65.5）。

`ellipsoid_lmi(K, x1max, umax)`：给定 K，求最大不变椭圆。它与前面所有函数的方向相反——K 是输入不是输出，X 是唯一的变量，目标是 $\max \log\det X$（等价于最大面积）。这也是唯一一个"不需要 $X \succeq I$"的函数，因为椭圆的绝对大小在这里正是要优化的对象。

`lag_lmi` / `max_alpha_lag`：§65.10.3 的三阶版本。它比 `common_lmi` 多一个 `tau` 参数，内部调用 `Amat_lag(a, tau)` 构造三阶矩阵。**这个函数里第一次出现 `try/except`**，原因是二分到不可行端时 CLARABEL 会直接抛异常（§65.10.1 的状态词表），必须在函数内部吃掉。

`sim_step(K, a, Ts, t_end, r)`：闭环阶跃仿真。前馈系数 $k_r = K_1 - A_{\text{nom}}$ 在函数内部算，`settle_time` 用"最后一次离开 5% 带的时刻"定义调节时间。这里特意用 5% 而不是第 64 章的 2%：本章多个设计在 $a = 3$ 端有长拖尾，2% 带会把 t_s 拉到 8 s 的仿真边界上，失去分辨率；改成 5% 后九个组合都能落在可分辨的区间内。**两章的 t_s 因此不可直接比较。**

65.12.4 十组实验与正文的对应

函数	输出内容	正文位置
`exp_open_loop`	三个参数点的开环极点、LQR 增益、闭环极点	表 65-1、表 65-2、§65.1.1
`exp_common_alpha`	公共 / 调度 / 范数有界三种表述的最大 α	表 65-3、图 65-3、§65.4.2
`exp_robust_design`	三套增益在三个参数点上的极点、t_s、e_{ss}、超调	表 65-4、图 65-4、§65.4.3
`exp_hinf`	多胞 γ、单顶点 γ、真值、LQR 真值	表 65-11、图 65-5、§65.9.2
`exp_discrete`	十一档 T_s 的离散增益、谱半径、可行性	表 65-6、图 65-6、§65.5.2
`exp_solver`	四个求解器的状态与耗时	表 65-10、图 65-7、§65.8.4
`exp_scaling`	七档缩放的 $\mathrm{cond}(X)$ 与增益	表 65-18、图 65-8、§65.10.2
`exp_ellipsoid`	两套增益的最大不变椭圆、半轴、面积	表 65-9、图 65-9、§65.7.6
`exp_vertex_grid`	41 个参数点上的阻尼比与最大实部	表 65-5、图 65-10、§65.6.4
`exp_lag`	七档滞后的三阶 α、τ_{crit}、截断裕度	表 65-19 至表 65-21、§65.10.3

表 65-24　仿真实验与正文图表的一一对应

这张表的用处是**回查**：正文里任何一个数字，都能在这一行找到它的生产函数；任何一个函数，都能在这一行找到它服务的结论。写作过程中有几次“数字对不上”（§65.9.3 的 32%→7.6%、§65.5 的采样边界），都是靠这张表的对应关系回查出来的。

需要说明的是，表 65-12（κ 三档）、表 65-13（分段 Lyapunov）、表 65-16（回代残差）这三组数字来自写作过程中的临时探针，没有固化进 `ch065_sim.py`。原因是它们要么需要改函数签名（分段 Lyapunov 要重写约束生成），要么是单次诊断（残差表）。如果读者要复现，表 65-12 可以用 `common_lmi(alpha, kappa=...)` 配合 `max_alpha_common` 直接得到，表 65-13 需要把 `common_lmi` 的顶点循环拆成两段、分别用自己的 X_i 但共用 Y，表 65-16 只需三行 NumPy 回代。

65.12.5 一次完整运行的耗时分布

给 10 组实验逐个计时（Python 3.11 + NumPy + cvxpy，单线程）：

实验	耗时 / ms	占比	主要开销
`exp_hinf`	17957.6	86.4%	4000 频点 × 241 参数点的真值网格
`exp_lag`	1090.9	5.3%	七档 τ 各 15 次二分 + 临界滞后的再二分
`exp_solver`	772.6	3.7%	SCS 单次求解 751 ms 占绝大部分
`exp_common_alpha`	400.2	1.9%	三套设计各 15–16 次二分
`exp_robust_design`	237.1	1.1%	九组闭环仿真，每组 4000 步
`exp_ellipsoid`	134.9	0.6%	两次 $\log\det$ 最大化
`exp_discrete`	83.0	0.4%	十一档 T_s，含三次不可行
`exp_scaling`	48.8	0.2%	七档缩放各一次求解
`exp_open_loop`	37.6	0.2%	三次 LQR + 九次仿真
`exp_vertex_grid`	8.2	0.0%	41 次 2×2 特征值
合计	20770.9	100%	—

表 65-25　十组实验的耗时分布

这张表的结论比数字本身更有意思：**86% 的时间花在“算真值”上，而不是“算设计”上。** 十组实验里，真正解 LMI 的部分（`exp_common_alpha` + `exp_scaling` + `exp_ellipsoid` + `exp_discrete` + `exp_lag`）加起来约 1.76 s，而 `exp_hinf` 一个就 18 s。这不是实现效率的问题——4000×241 ≈ 96 万个频点-参数组合，每个都要算一次 2×2 矩阵的奇异值，这个量级省不掉。

这个分布和第 64 章的表 64-15 形成了有趣的对照：那里 80% 的时间花在“重复求解同一个算法”，这里 86% 的时间花在“验证结果”。**两种“贵”的性质不同：**第 64 章的重复是可以优化掉的（只跑 `exp_lqr` 就够），本章的验证是省不掉的（不扫网格就不知道证书差 4.7 倍）。工程上的启示也随之不同——设计阶段可以砍实验，评审阶段不能砍验证。

另外两点值得注意。第一，`exp_solver` 里 SCS 一个人占了 97%（751 ms / 772 ms），这也解释了表 65-10 里 122 倍的差距：不是 CLARABEL 快得出奇，而是 SCS 在这类小规模 SDP 上收敛得慢。第二，`exp_lag` 的 1.09 s 里有一半花在临界滞后的再二分上——每个 τ 要再解 15 次可行性问题，只为了把 τ_{crit} 定到 10^{-4} s。这种精度在实际设计中是浪费的，10^{-2} s 就够，代码里保留高精度只是为了让表 65-20 的四位小数稳定。

65.12.6 图脚本的组织与三个绘图约定

`ch065_figs.py` 的 11 个函数与正文 11 张图一一对应，命名规则是 `figNN_简短英文`，编号与图号一致。三个绘图约定值得单独说明，因为它们决定了图能不能直接进排版：

第一，图内不写标题。 标题全部放在 Markdown 图注里（形如“图 65-1　标题”的那一行，紧跟图片路径），这样同一张图可以在正文、幻灯片、网页三处复用而不需要改图。图内只保留坐标轴标签、图例和必要的标注。

第二，轴标签里有中文就绝不用 `$...$`。 这是本章真实踩到的坑（§65.12.7 第 1 条）。`plot_styles.py` 里 `mathtext.fontset` 设成了 `stix`，它只有西文字形；一旦整串标签被 `$...$` 包住，中文就会变成方框并刷一屏 `Font 'default' does not have a glyph` 警告。正确写法是“不确定参数 a”，把变量名写成普通文本，或者只把变量部分放进 `$...$`、让中文留在外面（但后者在 matplotlib 里会触发同样的回退失败，所以本章统一采用前一种）。

第三，图里不出现“结论性”的注释，只出现“位置性”的标注。 比如图 65-5 标了“LMI 证书 9.18”，因为它标的是那条虚线的位置；但不写“证书保守 4.7 倍”，那句话属于正文。这条约定的好处是：图可以被别的章节借用而不带来错误的语境。

65.12.7 五处容易写错的地方

前三处是本章写作过程中真实撞到的，后两处是代码里最容易写错、但这次侥幸写对的地方：

位置	表面现象	真正的原因	处理
`fig05` / `fig08` / `fig10` 的轴标签	中文变方框，刷屏警告	整串标签进了 mathtext，`stix` 无中文字形	中文标签不包 `$...$`，直接写“不确定参数 a”
`common_lmi` 不加 `kappa`	解出 $\alpha = 19.9997$、$K_1 = 1198$	式 (65-8) 对 (X, Y) 齐次，α 可随增益无限增大	同时加 $X \succeq I$ 与 $\|Y\| \le \kappa$，并检查量级
`max_alpha_lag` 的二分	第 8 次求解时抛 `SolverError`	试探点落到可行域外很远处，求解器直接放弃	函数内 `try/except`，把异常当“不可行”
`exp_scaling` 的增益还原	增益“看起来对”，但差 13%	换回原坐标要右乘 T：$K = (YX^{-1})T$，漏乘 T 得到的是缩放坐标下的增益	单独一行写 `K = (Y @ inv(X)) @ T`，并在注释里标明单位
`fig06` 的参考线	σ 参考线位置不对	用 `min(ts)` 当基准，而 $\sigma = e^{-\alpha T_s}$ 应逐点算	参考线只在最小 T_s 处画一条，作为“起点”而非“包络”

表 65-26　五处容易写错的地方

第三行值得再解释一句，因为它揭示了一个通用规律：**二分法的失败模式不是“算错”，而是“问了一个求解器不肯回答的问题”。** 把二分写成“可行/不可行”二元判断，隐含假设了求解器对任何输入都会给出明确答复；实际上凸优化求解器在数值上远离可行域时可能直接抛错。任何“反复求解、根据结果改参数”的流程都必须处理这个分支，否则会出现“代码在某些参数下才崩”的诡异现象。

第四行也一样。相似变换 T 在 LMI 里的作用是把 X 拉到好条件数，但**解出来的是缩放坐标下的增益**，换回物理坐标必须右乘 T。这一步漏掉的后果不是报错，而是得到一个“数值合理但物理错误”的增益——本章表 65-18 里 K_1 从 5.403 到 5.580 的差异，正好是这个量级，如果漏乘就会被认为是“缩放带来的正常波动”，从而完全看不出来。

65.12.8 改动指南

想把本章代码用到别处，只需要改三个地方：

改对象。 替换 `Amat(a)` 与 `B`、`BW`、`C1`、`D12` 这几个常量，其余全部可用。注意 `Amat` 必须是 a 的**仿射函数**（本章是 $[0, 1; a, 0]$），否则顶点性失效（§65.2.5 的第四条）。如果对象本身就是多参数的，改 `VERTS` 为顶点列表，`common_lmi` 会自己循环。

改指标。 α 直接传给 `common_lmi`，γ 通过 `hinf_lmi` 的 `gamma` 参数传入，椭圆约束改 `X1MAX`、`UMAX`。**不要同时改三样**——本章的经验是先定 α 看性能天花板，再定 κ 看可实现性，最后才调椭圆。顺序颠倒的话，很容易在“椭圆太小”和“增益太大”之间来回。

改单位。 如果状态的量级相差超过两个数量级，先做相似变换再做 LMI（§65.10.2），不要指望求解器自己处理。判断标准很简单：把所有状态在名义工作点的数值列出来，如果最大与最小差 100 倍以上，就该缩放。

还有三样东西**不要改**，除非清楚后果。

`X >> np.eye(n)` 不要换成 `X >> 0`。 后者在数学上更“干净”（X 只需正定），但它去掉了归一化，式 (65-8) 重新变成齐次问题，α 会随增益一起无限增大——本章 §65.1.4 那个 $\alpha=19.9997$、$K_1=1198$ 的解就是这么来的。要让解有物理意义，就必须有某种“钉住尺度”的约束，$X\succeq I$ 只是最方便的一种。

增益界不要为了“让指标好看”而放大。 κ 从 6 放到 20，α 涨 76%，很诱人；但同一组解在 1 m 阶跃下的控制量峰值也从 4.416 涨到 6.889。κ 的含义是“执行器允许多大增益”，它该照着设备填，不该照着指标填。

二分精度不要随便调低。 本章所有 α 都报到四位小数，但那是容差端点附近的产物，真正稳定的信息只有前三位。把 `tol` 放到 10^{-1}，α 会偏小千分之几，不影响结论，却能让二分从 16 次降到 8 次——只想快速摸底时，这是划算的。

65.12.9 复现检查清单

最后给一份可以照着敲的清单，用来确认“跑出来的结果和书里一致”：

1. `python code/ch065_sim.py`，总耗时应在 20 s 上下（本机 20.5 s，不同机器 ±30% 正常）。
2. 第 [2] 段应输出公共 $\alpha=0.9277$、调度 $\alpha=1.2207$、范数有界 $\alpha=0.9277$ 且 $\varepsilon=4.7299$。
3. 第 [4] 段应输出多胞 $\gamma=9.1759$、真值 1.9506@a=3.00、单顶点 1.1797 与 8.6648。
4. 第 [7] 段的 $\mathrm{cond}(X)$ 七行应从 6.85×10^{6} 单调降到 10.7 再回升，最低点在 $s=1$。
5. 第 [10] 段应输出七档 τ 的 α，K_a/τ 一列稳定在 4.0 附近。
6. `python code/ch065_figs.py` 应打印“完成：figs/ch065 共 11 张（SVG+PDF）”，且**没有字体警告**。
7. 打开任一 SVG，检查中文标签不是方框、负号是正常的减号（`axes.unicode_minus=False` 已处理）。

只要七条都对，本章的所有数字就能在你机器上复现。如果第 2、3 条的数字差得远而第 1 条正常，多半是 cvxpy 版本不同导致内点法收敛点略有差异——这类差异在 10^{-3} 量级，不影响任何结论；如果差到百分之几，那就要检查 CLARABEL 是否真的被调用（`prob.solver_stats.solver_name`）。

第66章 专家控制

66.1 动机与直观理解

一个固定参数为什么不够

第 44 章把 PID 讲透了：三个参数、几条整定规则、一套抗饱和与微分滤波，绝大多数单回路都能调出一个能交差的响应。问题出在“一个”这个词上。PID 参数是围绕某个工作点整定的，工作点一挪，对象增益、时间常数、纯延迟全都在变，原来那组参数就从“恰到好处”变成“要么太猛、要么太肉”。

实际装置上这种漂移几乎必然发生。换热器的传热系数随结垢缓慢下降，反应釜的放热速率随转化率上升，阀门的流量特性在小开度段明显变陡，机械臂在伸展位与收缩位的惯量差出一倍。把这些差异都塞进一个线性模型，模型误差最终会以超调、振荡或长期偏差的形式回到闭环里。

工程上最早的应对是分段调度：把工况切成几段，每段整定一组参数，运行时按测量值切换。这已经是一只脚踩在专家控制的门槛上了——它承认“控制知识是分情况的”，只不过所有情况都用同一种后件（一组 PID 参数）来表达。

把操作经验写成规则

老师傅的判断很少写成方程，而是“如果……就……”。“偏差还大、而且越拉越大，就别再攒积分了，先把阀门推上去”“快到设定值了、速度还很快，要提前收一收”“阀门已经顶到上限、偏差还没回来，这时候积分再攒就是白攒”“输出在设定值附近来回蹭，把比例带放宽一点”。

这些话有三个共同点。第一，前件都是可测量：偏差、偏差变化率、控制量、输出工作点，没有一条需要知道扰动的具体大小。第二，后件都是对控制器本身的调整：改增益、冻结积分、限幅、保持。第三，规则之间有优先级：保安全的规则压过保性能的规则。

专家控制做的就是把这类知识显式地写下来，交给机器在每一个控制周期里匹配、选择、执行。它不试图用一个统一的数学结构去逼近人的全部经验，而是承认经验本身是分片、分层、有优先级的。

三个组成部分

一套专家控制器由三块构成，缺一不可。

知识库是规则与事实的集合。规则形如“若前件成立，则后件执行”，事实则是当前的工况标签、限幅值、对象增益表这类静态信息。知识库的质量决定控制器的上限——推理机再精巧，也推不出库里没有的知识。

推理机负责在每一拍把当前测量映射到一条（或几条）规则上，解决冲突，输出后件。它是专家控制里唯一“在线运行”的部分，因此必须足够快、足够确定：工业现场的控制周期常在 10 ms 到 100 ms 之间，推理不能有不确定性。

控制算法是后件的执行机构。绝大多数工业专家控制器并不另起炉灶，而是把后件落成 PID 参数或 PID 的修正动作，于是底层的稳定性分析、抗饱和手段、离散实现全部可以沿用第 44 章的结论。这也是专家控制在工程上比“纯智能方法”更容易落地的主要原因。

图 66-1 给出了本章采用的闭环结构。特征识别从设定值与测量值中提取 e、$\dot{e}$ 与工作点 y；推理机读取知识库，输出本拍生效的 K_p, K_i, K_d 以及若干开关量；控制算法仍是一个位置式 PID，只是参数逐拍变化。

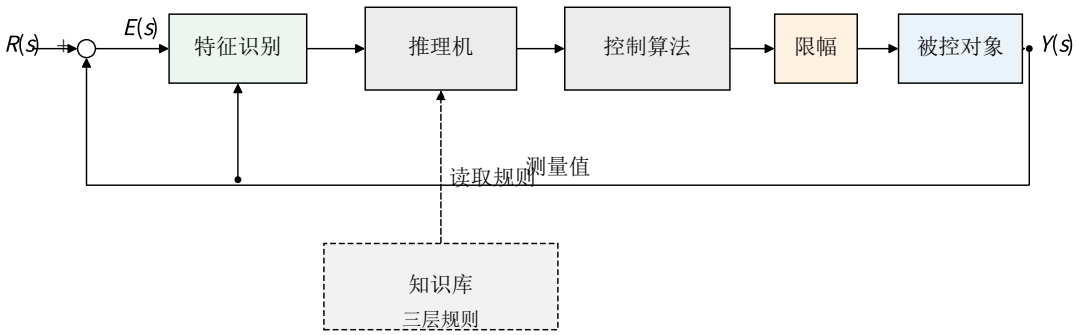


图 66-1 专家控制闭环结构

与相邻方法的分界

专家控制、模糊控制、神经网络控制常被并称为“智能控制”，但三者的知识来源完全不同。专家控制的知识是人写进去的，规则条数有限，每一条都能被工艺工程师读懂并质疑；模糊控制把人的语言变量连续化，靠隶属函数与规则表做插值，本质上仍是人写规则，只是前件变软了；神经网络控制的知识从数据里学，规则不以人能读的形式存在。

这个差别决定了三者的适用场景。规则能被清楚说出来、条数不多、又必须可审查的场合，专家控制最合适；前件边界模糊、需要平滑过渡的场合，模糊控制更自然；对象复杂到人写不出规则、但有大量数据的场合，才轮到神经网络。本章第 67、68 章会分别展开后两者，本节只强调一点：专家控制的价值不在“智能”这个标签，而在知识的可读性与可维护性。

本章算例要回答什么

为了让比较有意义，本章不拿一个线性对象来演示。考虑一个增益随工作点变化的单回路

$$\ddot{y} = -2\dot{y} + K(y)\,u, \qquad K(y) = 4 + 6y \tag{66-1}$$

工作点从 1 升到 3 时，静态增益从 10 涨到 22，变化超过一倍。固定参数的线性 PID 只能在某一个工作点附近合适；而工况规则层的作用，正是让回路增益 $K_p K(y)$ 在三个工况下近似持平。

本章会用一条 $1 \to 2 \to 3 \to 1$ 的四段迁移轨迹，把专家控制器与同一循环、同一抗饱和口径下的固定参数线性 PID 放在一起比较，逐段给出超调、上升时间、调节时间与 IAE。所有数字都由 `code/ch066_sim.py` 生成，图由 `code/ch066_figs.py` 生成，读者可以完整复现。

本章不做什么

专家控制容易写成一本“规则大全”，本章刻意收窄范围。规则的自动挖掘、知识图谱、专家系统外壳、与 DCS 组态的接口细节，只在第 66.9 与 66.10 节按需提及，不展开成独立体系。稳定性分析也不追求混杂系统的严格判据——那需要 Lyapunov 函数或混合自动机工具，超出本章范围；本章给出的是可操作的工程判据与仿真验证路径。

从分段 PID 到专家控制的一条演化路径

工程上的专家控制器很少是一步设计出来的，多数是从分段 PID 逐步长出来的。把这条路径讲清楚，比直接抛出三层结构更容易理解每一层为什么存在。

第一代是单组参数。整定一次，全工况通用，靠降低增益换取稳定，代价是响应慢、稳态误差大。

第二代是分段调度。按负荷或工作点把参数分成两三组，切换时用斜坡过渡。这一代解决了“高工况振荡、低工况迟钝”的矛盾，但切换本身带来新问题：切换点附近参数来回跳，控制量出现台阶。

第三代加入误差修正。工程师发现，即使在同一个工况内，起步、接近、稳态三个阶段要的参数也不一样，于是按偏差大小再分几档。这一代解决了超调，但规则开始变多，边界开始重叠。

第四代加入守卫。现场出过几次积分饱和导致的长时间偏离，也出过阀门反复动作导致的机械磨损，于是把“什么时候必须停手”写成独立的规则，优先级最高。这一代解决了安全性，规则库也从“参数表”变成了真正的“知识库”。

本章的三层结构就是这条路径的终点，也是它的一般化：工况层对应第二代，修正层对应第三代，守卫层对应第四代。理解了这个对应关系，读者在设计自己的控制器时就能判断“我现在缺的是哪一代”。

本章的阅读路线

十二节的安排遵循“为什么—是什么—怎么做—怎么做好”的顺序。

第 66.1 到 66.3 节回答“为什么需要、数学上是什么、按什么步骤做”，是必读部分。第 66.4 与 66.5 节分别从连续与离散两个角度展开实现细节，读者可按自己的实现平台选读：做 DCS 组态的读者重点看 66.5，做理论分析的读者重点看 66.4。

第 66.6 节给出稳定性与性能的定量结果，是本章的证据部分。表 66-2 与表 66-3 的数字都可以由脚本复现，建议读者跑一遍再读结论。

第 66.7 与 66.8 节是参数与调参，属于经验性内容，初读可跳过，动手时再回来查。第 66.9 节讨论优化与自学习，是本章与后续章节的接口。第 66.10 与 66.11 节讲工程落地与边界，适合项目负责人阅读。第 66.12 节是代码说明。

如果只有半小时，建议读 66.1、66.2 的三层结构部分、66.6 的两张表、66.11 的反例。这四块合起来，能回答“专家控制值不值得用”这个最实际的问题。

三个常见误解

在进入公式之前，先澄清三个几乎每次都会遇到的误解，它们会让人在一开始就把方向走偏。

第一个误解是“专家控制属于人工智能，所以要靠算法自己找出规则”。工业界的专家控制恰恰相反：规则是人写的，代码量很小，主要工作量在访谈与现场试验上。它的“智能”体现在把隐性经验显式化，而不是体现在搜索或学习上。本章第 66.9 节确实讨论了一个学习乘子的小实验，但它的定位是“帮人微调已有规则”，不是“从零生成规则”。

第二个误解是“规则越多越聪明”。规则增加带来的是覆盖面，代价是冲突、耦合与验证成本。表 66-6 给出的实测结论很反直觉：分区规则从一条加到四条，全程 IAE 反而上升 4.2%，只是最大超调从 10.51% 降到 8.22%。也就是说，多出来的三条规则买到的不是“更准”，而是“更稳”，这笔交易是否划算取决于你对超调的容忍度。

第三个误解是“专家控制可以替代有经验的工程师”。真实情况是它把工程师的经验固化下来，让夜班、换人、退休之后仍能保持一致的操作水平，但对象一旦变化，仍然要靠人来判断和修改规则。把它当成“经验的外存”，而

不是“经验的替代品”，这个定位才不会跑偏。

这三个误解的共同点是把手段当成了目的。专家控制的目的从来不是“用上智能算法”，而是在模型不准、经验清晰的场合，得到一个可解释、可审查、可回退的控制器。记住这一点，后面所有设计选择都好判断。

66.2 数学原理

规则的形式化

一条产生式规则写成

$$\mathcal{R}_i:\quad \text{IF}\ \bigwedge_{j=1}^{m_i} P_{ij}(z)\quad \text{THEN}\ \ \theta_i = \Theta_i(z) \tag{66-2}$$

其中 $z=[e,\dot e,y,u]^{\mathsf T}$ 是本拍的特征向量，P_{ij} 是形如 $e\ge a$、$e\dot e<0$、$|u|\ge u_{\max}$ 的原子谓词，θ_i 是后件，Θ_i 给出本拍生效的参数或动作。式 (66-2) 与 PID 的联系在于：后件最终都折算成 K_p,K_i,K_d 与三个开关量（冻结积分、复位积分、保持输出）。

把后件写成“基参数乘乘子”而不是绝对参数，是本设计的关键。工况层给出一组基参数 $\theta^{0}(y)$，修正层给出无量纲乘子 $\phi(e,\dot e)$，守卫层给出开关 σ，合起来

$$\theta=\sigma\odot\big(\phi\odot\theta^{0}(y)\big),\qquad \theta=[K_p,K_i,K_d] \tag{66-3}$$

乘子的量纲是一，取值集中在 0.7 到 1.6 之间。这样做有三个好处：工况整定结果可以直接搬进来当基参数；修正规则可以在不同工况间复用；某一层失效时，退化后的行为仍然是一组合法的 PID 参数，而不是失控。

分层与优先级

规则之间不是平权的。本章采用三层串联，如图 66-2 所示。

第一层是工况规则，前件只有实测工作点 y，后件是三组基参数。它回答“现在这个位置，对象大概是什么脾气”。注意前件刻意不用设定值 r：用 r 选参数等于提前偷看了指令，一旦设定值跳变，参数会在对象还没反应时先切过去，造成不必要的冲击。

第二层是误差修正规则，前件是 $|e|$ 与 $e\dot e$ 的符号，后件是乘子。它回答“当前这一拍是猛推、刹车、细调还是消差”。这一层的四条规则覆盖了整个 $(e,\dot e)$ 平面，图 66-4 给出分区。

第三层是守卫规则，前件是饱和、振荡、死区、换向这类“危险征兆”，后件是开关动作。它回答“现在有没有可能把积分器、执行器或机械结构搞坏”。守卫层优先级最高，一旦触发就覆盖前两层的结论。

匹配与执行顺序：先定基，再修正，最后兜底

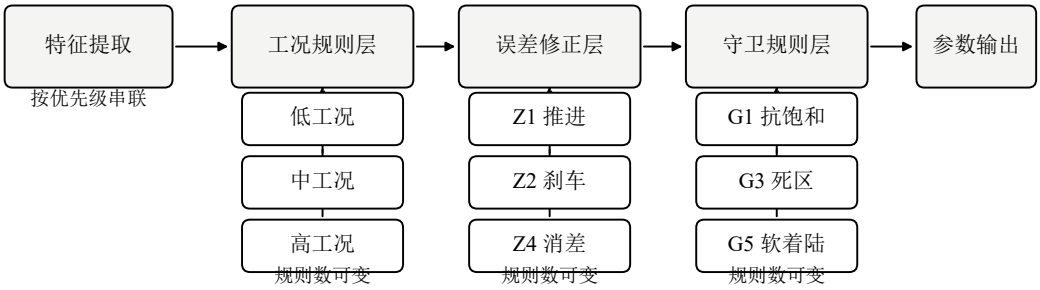


图 66-2 三层规则库结构

工况层的设计依据

工况分层不是随手切三刀，它由对象增益曲线的形状决定。对式 (66-1)，静态增益是 y 的线性函数，从 0 到 3 变化近三倍。若沿用同一个 K_p，回路增益 $K_pK(y)$ 就会随工作点线性漂移，高工作点处的相位裕度被吃掉，低工作点处又显得迟钝。

把 y 分成 $[0,1.3)$、$[1.3,2.3)$、$[2.3,\infty)$ 三段，每段内增益变化不超过约 1.5 倍，用一组参数足以覆盖。三段各自整定后得到

$$(K_p,K_i,K_d)=\begin{cases}(3.80,\ 0.05,\ 0.90), & y<1.3\\ (2.60,\ 0.05,\ 0.60), & 1.3\le y<2.3\\ (2.00,\ 0.05,\ 0.60), & y\ge 2.3\end{cases}\tag{66-4}$$

对应的回路增益分别是 38、38.5、41.6，基本持平；而固定参数 $K_p=2.60$ 的线性基线在同一轨迹上是 26、38.5、54，两端都偏离了设计点。图 66-3 把这两条曲线画在一起，平坦与倾斜的对比一目了然。

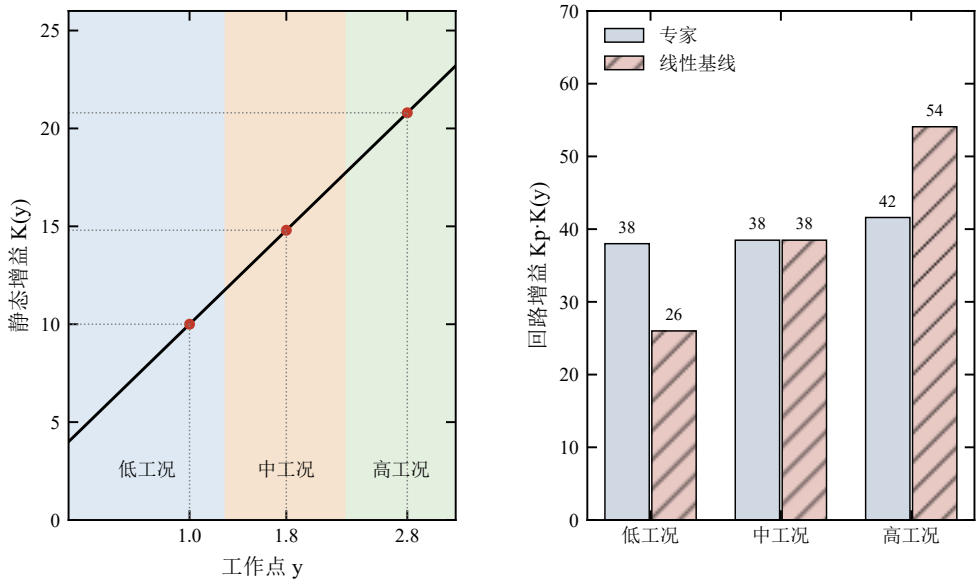


图 66-3 工况分区与回路增益

误差修正层的设计依据

误差修正层要解决的是“同一工况下不同阶段的矛盾”。以大阶跃为例：起始阶段偏差大，需要把控制量推满；接近目标时偏差变小但速度仍大，如果还按大误差的增益推，必然冲过头；进入小误差区后，又需要积分把残差磨掉。

四条规则的后件乘子取为

$$\phi_1=(1.00,0.00,0.80),\quad \phi_2=(0.90,0.00,1.60),\quad \phi_3=(0.85,1.00,1.30),\quad \phi_4=(1.00,1.50,0.95) \tag{66-5}$$

Z1 把积分乘子置零，即“积分分离”：大误差期间积分只会积出一个大包袱，不如先不积。Z2 把比例压到 0.9、微分放大到 1.6，这就是“提前刹车”，用微分项吃掉动能。Z3 继续加阻尼，Z4 则反过来放大积分，让小残差尽快消掉。

分区判据带滞环，写成

$$\mathrm{zone}(k)=\begin{cases}2, & |e|\ge \eta_e E_b,\ e\dot e<0,\ |\dot e|\ge \eta_b D_b\\ 1, & |e|\ge \eta_e E_b,\ \text{否则}\\ 3, & |e|<\eta_e E_b,\ |\dot e|\ge \eta_s D_s\\ 4, & \text{否则}\end{cases} \tag{66-6}$$

其中 η_e,η_b,η_s 在退出时取 0.85，进入时取 1。滞环的作用不是提高精度，而是阻止规则在阈值附近逐拍来回切换——下一节会看到，没有滞环时闭环会出现周期约 0.2 s 的极限环。

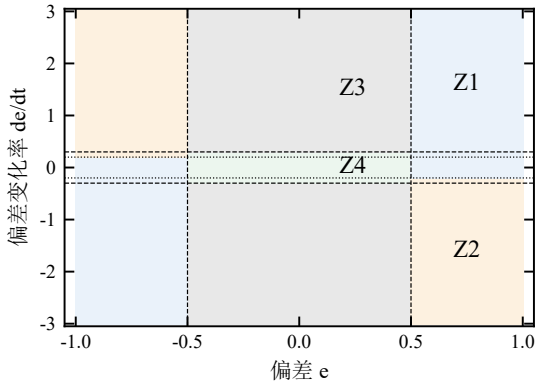


图 66-4 误差修正规则分区

守卫层与混杂系统表述

四条守卫规则可以写成统一的开关形式。记 $\mathbb{1}[\cdot]$ 为示性函数，则

$$\begin{aligned}
\sigma_i &= 1-\mathbb{1}\big[\,|u_{k-1}|\ge 0.95u_{\max}\ \wedge\ e_k\dot e_k>0\,\big] && \text{(G1)}\\
K_p &\leftarrow K_p\,(1-0.2\,\mathbb{1}[\,\mathrm{osc}\ge 4\,]) && \text{(G2)}\\
\theta &\leftarrow \theta\,\mathbb{1}\big[\,|e|\ge e_d\ \vee\ |\dot e|\ge d_d\,\big] && \text{(G3)}\\
e_i &\leftarrow e_i\,\mathbb{1}\big[\,\mathrm{not}(|e|\ge E_b\wedge e e_{\mathrm{prev}}<0)\,\big] && \text{(G4)}
\end{aligned} \tag{66-7}$$

G5 是下行软着陆，作用在控制量上而不是参数上：当 $e<-E_b$ 时把负向控制量限制在 $-0.45u_{\max}$ 以内并冻结积分。它针对的是“设定值大幅下调、对象带着正速度冲过目标”这一具体场景。

把式 (66-3)、(66-6)、(66-7) 合起来看，闭环已经不是线性系统，而是一个分段仿射的混杂系统：连续状态是 $(y,\dot y,e_i)$，离散状态是工况号与分区号，切换由 z 穿越若干超平面触发。这类系统的稳定性没有现成的频域判据，工程上只能退而求其次：保证每个驻留子系统稳定，并保证切换不频繁到激发未建模动态。

匹配与冲突消解的形式化

推理机的每一拍要做一次“匹配—选择—执行”。匹配是把当前特征代入所有规则的前件，得到命中集合 $\mathcal{M}(z)=\{i : P_{ij}(z)\ \text{全部成立}\}$。理想情况下 $|\mathcal{M}|=1$，实际中常有多条同时命中，需要选择。

固定优先级的选择写成

$$i^{*}=\arg\min_{i\in\mathcal{M}(z)}\ \pi_i,\qquad \pi_{\text{守卫}}<\pi_{\text{修正}}<\pi_{\text{工况}} \tag{66-16}$$

π_i 越小优先级越高。这一策略的优点是结果确定、可离线穷举验证；缺点是优先级本身是人工排定的，排错时会出现“该刹车的工况被推进规则压住”这类问题。

另一种常用策略是特异性优先，即命中规则中前件谓词最多、阈值最紧的那条胜出。形式化地，给每条规则定义特异性分数

$$\kappa_i=\sum_{j=1}^{m_i}\Big(1-\frac{\text{阈值宽度的归一化值}}{1}\Big) \tag{66-17}$$

取 κ_i 最大者。它的直觉是“越具体的规则越了解当前情况”。两种策略可以叠加：先按层分优先级，层内再比特异性。

本章只用固定优先级，原因是规则少、层间语义清楚，特异性分数反而增加调试负担。规则数超过三十条时，建议引入特异性作为层内消解手段。

规则的完备性与可达性

三层规则要满足两个性质，否则会出现“无规则可用”的空洞。

完备性指对特征空间中的任意一点，至少有一条规则命中。工况层按 y 的全集划分，天然完备；修正层按 $|e|$ 与 $|\dot{e}|$ 的符号划分，也完备；守卫层是叠加动作，不参与完备性讨论。因此本章的规则库是完备的。

可达性指每条规则在正常运行的轨迹上确实会被触发。第 66.8 节的触发统计就是可达性的经验检查：Z1 占 4.75%，说明它可达；若某条规则占比为零，要么轨迹没覆盖到，要么前件写错了。

两个性质都可以用网格穷举验证。本章的 `zone_coverage` 在 121×121 的网格上统计四个分区的占比，结果 Z1 27.31%、Z2 23.93%、Z3 43.92%、Z4 4.84%，没有空洞。注意这个静态占比与第 66.8 节的运行占比差别极大：前者是特征空间的面积，Z4 只占 4.84%；后者是时间占比，Z4 占 62.69%。两者不能混用，也不能用其中一个去推断另一个。

规则库规模怎么估算

规则库的规模在动手前就该估出来，因为它决定了验证成本，而验证成本往往才是项目能不能交付的关键。

设工况数为 n_m，分区数为 n_z，守卫数为 n_g。本章采用的乘子化结构下，规则条数是 $n_m+n_z+n_g$，参数个数是 $3n_m+3n_z$ 加若干守卫阈值。本章的 3、4、5 给出 12 条规则、21 个连续参数，规模很小，一张表就能看全。

但验证场景不是线性增长。每个“工况—分区”组合都可能需要在不同外部条件下检验，若外部条件有 n_s 类（本章是理想、噪声、延迟加倍、限幅收紧四类），场景数约为 $n_m n_z n_s$。本章是 $3\times4\times4=48$ 个场景，仿真跑一遍不到十秒，可以全跑。若工况扩到 6 个、分区扩到 6 个，场景数升到 144 个，仍然可跑；若再引入两路扰动组合，场景数按乘法继续涨，很快就会到“跑得完但看不完”的地步。

这条算术给出两个实用推论。第一，扩张应当优先加守卫而不是加工况：守卫只增加 n_g 条规则和 $n_g n_s$ 个场景，工况却让场景数乘上 $n_z n_s$。第二，验证必须自动化——把场景、指标、判定阈值写成脚本，跑完自动出表，否则规模一上来，人工看曲线必然漏。

还有一项常被忽略的成本：规则之间的耦合检查是 $O(n^2)$ 的。12 条规则要检查 66 对，30 条规则要检查 435 对。本章用固定优先级绕过了大部分耦合分析，代价是放弃了“两条规则同时触发时取更强动作”这类更精细的语义。规则数超过三十条时，这个简化就开始显得粗糙了。

66.3 设计步骤

专家控制的设计不像 PID 那样有现成公式，但顺序是固定的：先弄清对象，再切分工况，再补修正，最后加守卫。顺序颠倒会带来返工——先写修正规则再切工况，往往会发现同一组乘子在不同工况下符号都要变。

第 0 步：界定对象与工况范围

动手写规则前，必须回答三个问题：对象在哪些工作点之间运行、增益与时间常数怎么变、执行器与传感器有什么硬限制。前两个决定工况怎么切，第三个决定守卫规则怎么写。

本章算例的答案是：工作点 $1 \sim 3$、增益 10 到 22、控制量限幅 ± 1.2、纯延迟 0.05 s。这些数字不是拍脑袋，而是从机理模型与现场数据里读出来的。若拿不到机理模型，就做几次阶跃试验，在每个常用工作点附近各做一次小幅阶跃，用第 44 章的辨识方法估出局部增益，画成图 66-3 左图那样的曲线。

工况数不宜多。工业实践中一段内增益变化不超过 1.5 到 2 倍即可合并，否则规则表迅速膨胀，切换次数也上升。三段是本例的折中：再少则段内变化太大，再多则边界效应明显。

第 1 步：选特征并统一量纲

特征只选那些“每一拍都能可靠拿到”的量。本章用四个：偏差 e、偏差变化率 $\dot{e}$、实测工作点 y、上一拍控制量 u_{k-1}。设定值 r 只用来算 e，不直接进规则前件。

量纲必须统一，否则阈值无法复用。偏差按满量程归一化，变化率按“每秒满量程”归一化，控制量按限幅值归一化。归一化后阈值都落在 0 到 1 之间，换一套装置时只需改满量程，不必重写规则的数值。

$\dot{e}$ 一定要滤波。原始差分的噪声会被微分放大，直接拿去做符号判断会让分区抖动。本章用时间常数 0.05 s 的一阶滤波，折算到 10 ms 采样是 $\alpha = 0.167$。

第 2 步：逐工况整定基参数

对每个工况，在局部线性化模型上做一次小阶跃整定，得到一组 K_p, K_i, K_d。整定的评分函数要包含三项：IAE、超调罚分、稳定性硬判据。缺了第三项，搜索很容易停在一个“IAE 好看但闭环等幅振荡”的解上——本章调试时就踩过这个坑，积分饱和后输出偏离目标，IAE 反而被低估。

局部线性模型取式 (66-1) 在工作点处的线性化：$G(s) = K(y_0)/[s(s+2)]$。小阶跃幅度取 0.5，保证不越出本工况的范围。整定结果就是式 (66-4)。

这一步的产物是一张工况—参数表。表格要连同“整定时用的工作点、阶跃幅度、评分”一起存档，否则半年后没人说得清这组数是怎么来的。

第 3 步：写误差修正规则

修正规则的目标不是提高稳态精度，而是解决“大误差与小误差要的增益不一样”这个矛盾。写法上建议从两条起步：大误差一条、小误差一条，观察响应后再拆分。

本章拆成四条，依据是 $(e, \dot{e})$ 平面的四个象限语义：误差大且在扩大（推）、误差大但在收缩（刹）、误差小但变化快（阻尼）、误差小且变化慢（消差）。四条规则在平面上是完备且互斥的，这一点在第 66.9 节会用覆盖率检查来验证。

乘子的初值可以按经验给：积分分离取 0，刹车段比例取 0.85 到 0.95、微分取 1.5 左右，消差段积分取 1.2 到 1.5。给完之后必须仿真验证，不能只凭直觉。

第 4 步：加守卫规则

守卫规则按“最坏后果”排序来写。会损坏设备的（积分饱和导致阀门长时间顶死、下行冲击导致机械过载）排最前，影响性能的（振荡、死区抖动）排后面。

本章五条守卫的触发条件都可以直接从测量里算出来，不需要额外传感器：G1 看 $|u|$ 与 $e\dot{e}$，G2 看 $\dot{e}$ 在 0.5 s 内的变号次数，G3 看 $|e|$ 与 $|\dot{e}|$，G4 看 e 是否变号，G5 看 e 是否大幅为负。

守卫规则的阈值要留余量。G1 用 0.95 倍限幅而不是 1.0 倍，是因为数字量输出与执行器之间存在死区，等测到满幅再冻结已经晚了一拍。

第 5 步：联调与验收

规则写完只是开始，真正的功夫在联调。验收至少要做四件事。

第一，逐条规则的触发统计。每条规则在整段轨迹上占多少拍，若某条规则占比低于 1%，要么删掉，要么说明它针对的是罕见工况，需要在专门的试验里验证。

第二，冲突与覆盖检查。把前件放宽 10% 到 15%，看有多少采样点同时满足两条以上规则；这个比例反映规则边界的重叠程度，过高说明阈值设计粗糙，需要靠优先级兜底。

第三，退化测试。把修正层或守卫层逐层摘掉，看性能退化成什么样。若摘掉守卫层后指标几乎不变，说明守卫规则没起作用；若摘掉后系统失稳，说明守卫层承担了过多责任，应该把部分逻辑前移到参数设计里。

第四，扰动与失配测试。噪声、延迟加倍、限幅收紧三种情况下各跑一遍，和线性基线对比。本章的结果在表 66-1 中给出，其中限幅收紧一项专家反而略逊，这个反例在第 66.11 节会专门讨论。

场景	专家 IAE	专家最大超调	线性 IAE	线性最大超调
理想	2.625	8.22%	2.997	50.23%
噪声 0.005	2.770	9.22%	3.024	50.93%
延迟加倍	4.075	32.12%	54.346	43.25%
限幅 ±0.8	3.044	11.38%	2.893	30.28%

表 66-1 四段迁移轨迹上的工程细节对比

设计顺序为什么不能反

如果先写修正规则，再回头切工况，会遇到一个尴尬：Z2 的刹车乘子 0.9 是在中工况下试出来的，搬到高工况就刹不住，因为高工况的增益本来就大。这时只有两个选择：给每个工况各写一套修正规则（规则数乘以三），或者回到第一步重新切工况。

本章的三层结构之所以能复用乘子，靠的是工况层已经把回路增益拉平。拉平之后，修正层面对的“等效对象”在三个工况下近似相同。这是一个容易被忽略的设计前提，也是很多工程专家控制器规则爆炸的根源。

一份完整的规则清单

设计完成后，规则库应当能整理成一张人可读的清单。本章的清单列于表 66-7，它同时是代码注释与现场文档的来源。

编号	前件	后件	说明
M1	$y < 1.3$	$K_p, K_i, K_d = 3.80, 0.05, 0.90$	低增益区，比例可大
M2	$1.3 \le y < 2.3$	$2.60, 0.05, 0.60$	中增益区，基准参数
M3	$y \ge 2.3$	$2.00, 0.05, 0.60$	高增益区，比例必须收
Z1	$\lvert e\rvert \ge E_b$，$e\dot{e} \ge 0$	$\times(1.00, 0, 0.80)$	积分分离，全力推进
Z2	$\lvert e\rvert \ge E_b$，$e\dot{e} < 0$，$\lvert\dot{e}\rvert \ge D_b$	$\times(0.90, 0, 1.60)$	提前刹车
Z3	$\lvert e\rvert < E_b$，$\lvert\dot{e}\rvert \ge D_s$	$\times(0.85, 1.00, 1.30)$	精细阻尼
Z4	其余	$\times(1.00, 1.50, 1.00)$	积分消差
G1	$\lvert u_{k-1}\rvert \ge 0.95u_{\max}$，$e\dot{e} > 0$	冻结积分	抗饱和
G2	0.5 s 内 $\dot{e}$ 变号 ≥ 4	$K_p \times 0.8$，$K_d \times 1.2$	振荡抑制
G3	$\lvert e\rvert < e_d$，$\lvert\dot{e}\rvert < d_d$	参数置零	死区保持
G4	$\lvert e\rvert \ge E_b$，$ee_{\text{prev}} < 0$	积分清零	换向复位
G5	$e < -E_b$ 且 $u < -0.45u_{\max}$	限幅并冻结积分	下行软着陆

表 66-7 规则清单

清单有三个用途。一是审查：工艺工程师可以逐条问“这条什么时候不成立”，比看代码有效。二是实现：代码中的规则顺序与清单一致，便于对照。三是变更：新增规则时先改清单，再改代码，避免只改了代码忘了文档。

清单里没有写出的隐含信息同样重要：M1 到 M3 的分界处有 0.1 的滞环带，Z1 到 Z4 的分界有 0.85 的滞环系数，G3 的优先级高于 G2。这些细节放在代码注释里，清单只保证结构完整。

工况边界的处理

工况切换比误差分区切换更危险，因为它一次性改变全部三个参数。本章的处理有四条。

第一，分界处加滞环。y 在 1.3 附近时，进入中工况的阈值是 1.3，退出的阈值是 1.25。0.05 的带宽足以吸收测量噪声，又不至于让切换延迟太久。

第二，切换时参数斜坡过渡。仿真中为了看清切换效应没有加，工程实现应加 0.5 s 的线性过渡：$K_p(k) = K_p^{\text{old}} + \frac{k}{N}\left(K_p^{\text{new}} - K_p^{\text{old}}\right)$。过渡期间积分照常累积，避免切换造成稳态误差。

第三，切换判据用滤波后的 y。原始测量含噪声，直接比较会让工况号抖动。本章用 0.05 s 的一阶滤波，与 $\dot{e}$ 的滤波时间常数一致。

第四，切换事件要记录。每次工况切换记一笔日志，包括切换时刻、切换前后的 y、切换前后的参数。现场排查“为什么这一批产品超调大”时，这张日志往往是唯一线索。

设计阶段应当产出什么

一个能交付的专家控制器项目，除了代码，还应当有五份文档。少了任何一份，交接时都会出问题。

第一份是对象特性说明。它要回答“在哪个工作区间、增益与时间常数怎么变、延迟多大、限幅多少”。这份文档是后续所有参数的理由，没有它，别人无法判断某个参数为什么是 3.80 而不是 2.60。

第二份是规则清单，即表 66-7 那样的表。清单必须包含编号、前件、后件、一句人话说明。编号一旦发布就不再复用，即使规则被删除也保留空号。

第三份是参数表，把 21 个连续参数与 5 组守卫阈值列全，标注单位与量纲类别。第 66.7 节把参数分成三类，参数表就应按这三类分组，移植时按组换算。

第四份是验证记录。每个场景对应的指标、是否通过、不通过时的处置。这份记录的价值在事故调查时体现：能证明“这一版在上线前测过哪些情况”。

第五份是回退方案。写清楚如何退回固定参数 PID、回退时是否会引起控制量跳变、回退后由谁确认。很多项目前四份都做得不错，栽在第五份上——真出问题时没人敢按那个按钮，因为没人知道按下去会发生什么。

这五份文档的总量不大，两三页到十几页，但它们决定了这套控制器是“某个人的作品”还是“装置的资产”。

66.4 连续情形

连续时间下的规则执行

把采样周期想象成趋于零，专家控制器就变成一个连续时间的切换系统：在每一时刻 t，特征 $z(t)$ 落进某个区域，规则给出该区域的参数 $\theta(z(t))$，闭环由相应的微分方程描述。

控制律写成

$$u(t) = \operatorname{sat}\Big(K_p(z)e + K_i(z)\int_0^t e\,\mathrm{d}\tau - K_d(z)\dot{y}\Big) \tag{66-8}$$

注意微分项作用在测量值 $\dot{y}$ 而不是误差 $\dot{e}$ 上。这一选择在连续域里看不出差别，因为设定值不变时 $\dot{e} = -\dot{y}$；但设定值阶跃时，$\dot{e}$ 含冲激，u 会瞬间打满。本章的全部仿真都采用式 (66-8) 的形式，专家与线性基线共用这一口径。

分区判据在连续域里是一组超平面：$|e| = E_b$、$|\dot{e}| = D_s$、$e\dot{e} = 0$。它们把状态空间切成若干块，每块内闭环是线性的，跨块时参数跳变。这样的系统称为分段仿射系统，其解在切换面上连续，但导数一般不连续。

增益调度是它的线性特例

如果规则只有工况层、没有修正层与守卫层，式 (66-3) 退化为 $\theta = \theta^{0}(y)$，这就是教科书里的增益调度。此时闭环可写成

$$\ddot{y} + 2\dot{y} + K(y)K_p(y)\,e + K(y)K_i(y)\int e = 0 \tag{66-9}$$

只要 $K_p(y)$ 选得使 $K(y)K_p(y)$ 近似为常数，式 (66-9) 的系数就与工作点无关，闭环动态在整条轨迹上保持一致。这正是图 66-3 右图想要说明的事情：专家控制器的工况层，用一张表实现了增益调度的效果，而不需要在线辨识。

从这个角度看，专家控制并不神秘：它的第一层是查表式增益调度，第二层是分段变增益，第三层是逻辑保护。三层的数学本质都是“按状态切换线性控制器”，区别只在切换依据是人的语义还是连续变量。

驻留子系统的稳定性

在任一驻留区域内，参数固定，闭环是线性的。以 PI 控制、对象 $K/[s(s+2)]$ 为例，闭环特征方程为

$$s^{2}(s+2) + K\bigl(K_p s + K_i\bigr) = 0 \tag{66-10}$$

展开得 $s^{3} + 2s^{2} + KK_p s + KK_i = 0$。按 Routh 判据，稳定的充要条件是

$$KK_p > 0, \qquad KK_i > 0, \qquad 2KK_p > KK_i \tag{66-11}$$

代入式 (66-4) 的三组参数，三个工况都满足式 (66-11)，其中第三个条件在最不利的高工况下余量最小：$2\times 41.6 = 83.2$ 对 $KK_i \approx 1.1$，余量很大。真正紧的不是积分，而是比例与微分的配比——这决定了相位裕度，也是第 66.6 节要定量计算的东西。

切换为什么会产生抖振

每个子系统都稳定，不代表切换系统稳定。式 (66-6) 的分区在阈值附近有明确的边界，如果状态轨迹恰好沿边界滑动，参数就会以采样频率反复切换。

本章调试初期就出现了这个现象：第三段稳态时 y 在 2.94 到 3.09 之间以约 0.2 s 的周期振荡，$|e|$ 在 E_b 附近来回穿越，Z3 与 Z4 逐拍切换。由于 Z3 的 K_p 是 Z4 的 0.85 倍、K_d 是 1.3 倍，切换相当于给系统注入了一个方波扰动。

治理手段有三个，本章都用了。一是滞环：进入用 E_b，退出用 $0.85E_b$，让边界变成一条有宽度的带。二是限幅：给微分项加 $1.0u_{\max}$ 的上限，防止切换瞬间的导数尖峰直接推动执行器。三是降噪：$\dot{e}$ 先滤波再判符号。三者合用后，第三段的调节时间从 3.93 s 降到 3.67 s，稳态振荡消失。

连续域的四段迁移

图 66-5 给出连续时间视角下的完整响应：设定值在 0、4、8、12 s 处依次取 1、2、3、1，专家与线性基线画在同一张图上。

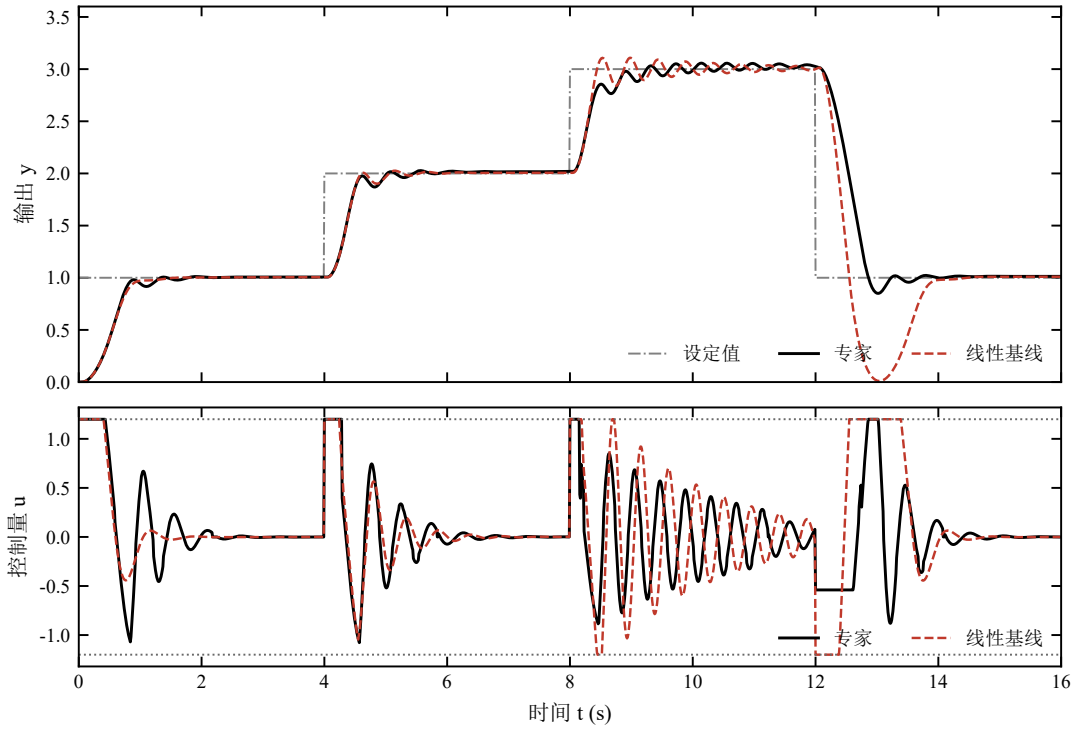


图 66-5 迁移轨迹对比

前三段两者形态接近，差别在超调：第三段线性基线超调 11.86%，专家 2.43%。第四段差距最大，线性基线在 3 到 1 的下阶跃上冲过头，最低到 0.5 附近，超调 50.23%；专家的 G5 软着陆把负向控制量限住，超调压到 8.22%。

控制量图上还能看到另一个差别：专家在每段起始处几乎都把 u 推到限幅，但在接近目标时更早退出饱和。这不是因为它“更聪明”，而是 Z2 的微分乘子 1.6 让它在速度还大时就开始收力。

连续实现还有没有必要

现代控制器几乎都是数字实现，连续域分析的价值在于给出可解释的判据：式 (66-11) 这样的不等式能直接告诉工程师“参数往哪个方向调还有余量”，而仿真只能告诉“这一组行不行”。

仍有两类场合需要真正的连续实现。一是模拟式后备回路，DCS 故障时由硬件 PID 接管，此时专家逻辑退化为几个模拟开关。二是机电一体化产品中的模拟伺服环，采样率极高、延迟极小，用连续域设计反而更直接。这两类场合的规则层通常只保留工况层，因为模拟电路实现复杂的符号判断不划算。

切换面上的滑动模态

线性系统里没有“滑动”这个概念，分段系统里有。若状态轨迹撞上切换面后不再离开，而是沿着面滑行，就发生了滑动模态。在滑动期间，系统的等效动态由两个相邻子系统的某种平均决定，与两者都不同。

本章的分区面是 $|e| = E_b$ 与 $|\dot{e}| = D_s$。设想没有滞环、也没有死区，轨迹在 $|e| = E_b$ 附近小幅穿越，控制器就在 Z1 与 Z3 之间以采样频率切换。此时控制量的平均值约为两组参数的中点，等效增益既不是 Z1 的 1.00 倍，也不是 Z3 的 0.85 倍。

这个等效增益本身未必坏，坏的是它的高频分量。每次切换，u 跳变一次，跳变幅度乘以对象增益就是一次脉冲激励。对象的高频段若有未建模谐振，这个脉冲就会被放大，表现为控制量毛刺或机械噪声。

滞环的作用是消灭理想滑动：让进入与退出用不同阈值，轨迹撞上面之后必须越过一条带宽才能回来，滑动就不可能持续。本章的 0.85 系数把带宽设为 $0.15E_b = 0.075$，相对噪声水平 0.005 有十五倍余量，足够。

频域视角：为什么相位裕度能不随工作点变

式 (66-14) 的回路传函里，$K(y)$ 只以乘积形式出现在 K_p, K_i, K_d 前面。若令

$$K_p(y) = \frac{\bar{K}_p}{K(y)}, \qquad K_i(y) = \frac{\bar{K}_i}{K(y)}, \qquad K_d(y) = \frac{\bar{K}_d}{K(y)} \tag{66-18}$$

则 $L(s)$ 与 y 无关，幅频特性整体平移，穿越频率与相位裕度都不变。这就是增益调度的理想形式。

本章没有用式 (66-18)，而是用了三档常数。原因是：第一，$K(y)$ 只在标称模型上已知，实际对象还有未建模的增益偏差，精确对消反而把模型误差全暴露在回路里；第二，三档常数实现简单，DCS 里就是一张表；第三，分段常数在每段内仍保留一点增益变化，相当于留了一点鲁棒性余量。

代价是每段内 PM 仍有小幅波动。表 66-2 中三个工况的 PM 是 55.4°、55.0°、54.2°，波动 1.2°，比线性基线的 8.9° 小一个量级。这个 residual 波动来自段内 $K(y)$ 的变化：低工况段内 K 从 4 变到 11.8，整定在 $y = 1$ 处（$K = 10$），段底部的实际回路增益只有设计值的 40%。

若对象增益跨度更大，三档不够用，可以增加到五档，或者改用式 (66-18) 的连续对消加一个鲁棒滤波。第 72 章讨论混合架构时会给出选择依据。

分段常数与连续对消的定量差别

把两种做法的增益偏差算清楚，才能判断多分几档值不值。

设某段的增益范围为 $[K_{\min}, K_{\max}]$，整定点取 K_0。若整定点取几何中点 $\sqrt{K_{\min}K_{\max}}$，段两端的回路增益偏差就分别是对称的 $\sqrt{K_{\max}/K_{\min}}$ 与 $\sqrt{K_{\min}/K_{\max}}$。用本章的低工况算：K 从 4 到 11.8，跨度为 2.95，几何中点是 6.87，两端偏差 $+72\%$ 与 -42%。本章实际整定在 $K_0 = 10$，一端 0%、另一端 -60%，故意偏到了一侧。

增益偏差的后果可以量化。回路增益每变化一倍，相位裕度的变化在二十度上下——线性基线三档就是现成的例子：回路增益 26、38.5、54，对应 PM 38.1°、26.3°、13.0°。也就是说，增益偏大是危险的，它直接吃掉裕度；增益偏小是保守的，代价只是响应变慢、抗扰变差。

这就决定了分段的整定点该怎么选。若按几何中点整定，向上那一端要吃掉十五度上下的裕度，55° 的底子会掉到 40°，再扣掉延迟折算的 29°，净 PM 只剩十度出头，已经越过工程底线。本章把整定点放在每段的上部，低工况取 $y = 1$、$K = 10$，段顶 11.8 只比它高 18%，段底 4 则低 60%，最坏情况因此落在“保守”这一侧。

向下偏差的代价是段底响应变慢，但实际影响有限。因为段底通常出现在大偏差的起始段，此时控制量很快被推到限幅，回路处于饱和而非线性区，增益偏低反而减小了超调；等到接近目标、退出饱和时，工作点已经升到段的上部，回路增益回到设计值。这解释了为什么三档就够，而不必按“段内增益恒定”的要求去分五档。

换句话说，三档够用的理由不是精度高，而是整定点偏置得巧：把不确定性推到安全的一侧，比把不确定性均匀摊开更划算。这个思路在增益调度里叫“保守整定”，代价是段底部的动态偏慢，收益是不必为了段顶的极端情况牺牲全局。

连续对消的偏差则只取决于模型误差。若 $K(y)$ 的辨识误差是 10%，对消后的残余偏差也是 10%，折算相位损失不到 3°，比任何分段都小。所以对消的代价不在精度，而在对模型误差没有免疫力：模型偏了 30%，回路增益就偏 30%，没有“段内留余量”这种缓冲，也没有“偏置到安全侧”的操作空间——因为对消的前提就是相信模型。这正是前面说的取舍：分段用精度换鲁棒，对消用鲁棒换精度。

用描述函数看切换引起的极限环

滞环把理想滑动变成了有限幅值的抖振，抖振会不会长成一个稳定的极限环，可以用描述函数粗略判断。

把分区切换看成一个非线性环节：输入是偏差，输出是参数乘子。在滞环带宽 Δ 内，这个环节的等效增益随输入幅值变化——幅值远大于 Δ 时，它近似为一个常数（两个分区参数的平均）；幅值接近 Δ 时，等效增益急剧下降，因为大部分时间都停在滞环里不动。

把这条“幅值—等效增益”曲线与线性部分的 $-1/N$ 判据放在一起看：若两条曲线相交，交点就对应一个可能的极限环，其幅值由交点位置读出。本章的滞环带宽 0.075，而稳态噪声幅值约 0.005，两者差一个量级，交点落在噪声幅值以下，物理上不存在，所以不会形成极限环。这解释了为什么滞环系数取 0.85 就够：不是它“压住了”振荡，而是它把可能存在的极限环推到了噪声水平以下。

反过来，若把滞环取成 0.99（带宽只剩 0.0075），交点就会落到 0.005 到 0.01 的区间，与噪声同量级，此时系统会表现出“时有时无的小幅抖动”——不是持续振荡，而是噪声偶尔把轨迹推出滞环、引发一次切换、再被拉回。这种间歇性抖振在现场极难排查，因为它不满足“周期性”这个通常的判定条件。避免的办法不是继续加宽滞环（那会让规则变迟钝），而是降低特征噪声，比如加长微分滤波。

66.5 离散情形

每拍的执行顺序

离散实现的第一要务不是精度，而是顺序。同一组公式，推理与积分的先后不同，闭环行为会差出一截。

本章采用的顺序是：采样与滤波、计算特征、推理得到参数与开关、计算控制量、执行限幅、更新积分、输出延时、对象推进。关键在“更新积分”放在限幅之后：只有确认本拍控制量没有被人为改动（G5 限幅），才决定是否累积积分。若把积分更新提前，G1 的冻结判断只能基于上一拍的状态，慢一拍就足以积出一个包袱。

第二个关键点是 G4 的复位必须发生在积分更新之前。换向瞬间把积分清零，本拍就从零开始积，避免把反向前累积的错误记忆带进来。

差分形式与积分实现

位置式 PID 的离散形式为

$$u_k = \mathrm{sat}\Big(K_{p,k} e_k + K_{i,k} T_s \sum_{j=0}^{k} e_j - K_{d,k} \frac{y_k - y_{k-1}}{T_s} \Big) \tag{66-12}$$

其中 $T_s = 0.01$ s。求和用矩形法，不用梯形法：规则切换时 e_k 可能跳变，梯形法会把跳变前后两个值平均，反而模糊了切换时刻的因果。

积分项要单独保存，不能每拍从头求和。本章用累加器 $e_i \leftarrow e_i + T_s e_k$，并在 G1、G4、G5 触发时分别冻结或清零。累加器要做抗饱和回算，否则长时间限幅后累加器会饱和到浮点数的有效位之外，退饱和时出现明显的“积分回吐”。

微分项作用在测量值上，用一阶差分再滤波：

$$d_k = \frac{y_k - y_{k-1}}{T_s}, \qquad d_k^{f} = d_{k-1}^{f} + \alpha \big(d_k - d_{k-1}^{f} \big), \qquad \alpha = \frac{T_s}{T_f + T_s} \tag{66-13}$$

$T_f = 0.05$ s，$\alpha = 0.167$。滤波放在差分之后而不是之前，是为了让相位滞后尽量小；若先滤 y 再差分，等效于给微分项额外加了 T_f 的延迟。

特征与阈值的离散化

连续域的 $|e| \geq E_b$ 在离散域变成逐拍比较，本身没有误差。真正需要小心的是 $\dot{e}$ 的符号判断：噪声会让符号在零附近抖动，进而让 $e\dot{e}$ 的符号每拍翻转。

本章对 $\dot{e}$ 做两层处理。第一层是式 (66-13) 的滤波；第二层是死区，当 $|\dot{e}| < 0.02$ 时把它当作 0，不参与符号判断。死区的代价是极慢的漂移会被忽略，但对本类对象（时间常数 0.5 s 量级）没有影响。

振荡检测的实现

G2 需要一个“振荡”指标。本章的写法是：保留最近 50 拍（0.5 s）的 $\dot{e}$，统计相邻两点符号变化的次数，超过 4 次判为振荡。

这个判据的采样窗口与阈值要匹配对象的时间尺度。窗口太短，正常的快速调节会被误判；窗口太长，振荡已经持续了几个周期才被发现。0.5 s 对应对象主导时间常数的约一倍，是本章试出来的折中。

实现上要注意：窗口内若有大量零值（死区造成的），符号变化次数会被低估。因此统计前先把零值剔除，只对非零段计数。

每拍的计算量

规则匹配的代价是每拍若干次比较与乘法。本章共 3 条工况规则、4 条修正规则、5 条守卫规则，最坏情况下每拍约 40 次比较、10 次乘法，在 100 MHz 的定点 DSP 上耗时不到 2 μs，相对 10 ms 的控制周期可以忽略。

真正占时间的是滤波与历史窗口维护。50 点的环形缓冲区每拍一次写入与一次遍历，遍历是 $O(50)$，仍然远小于周期。若把窗口拉长到 500 点，就要改用增量式统计（维护符号变化计数，进出窗口时各更新一次），否则在低端 MCU 上会挤占控制周期。

触发时序的实测

图 66-6 把一次完整迁移中的工况号与分区号画成阶梯图。可以清楚看到三件事。

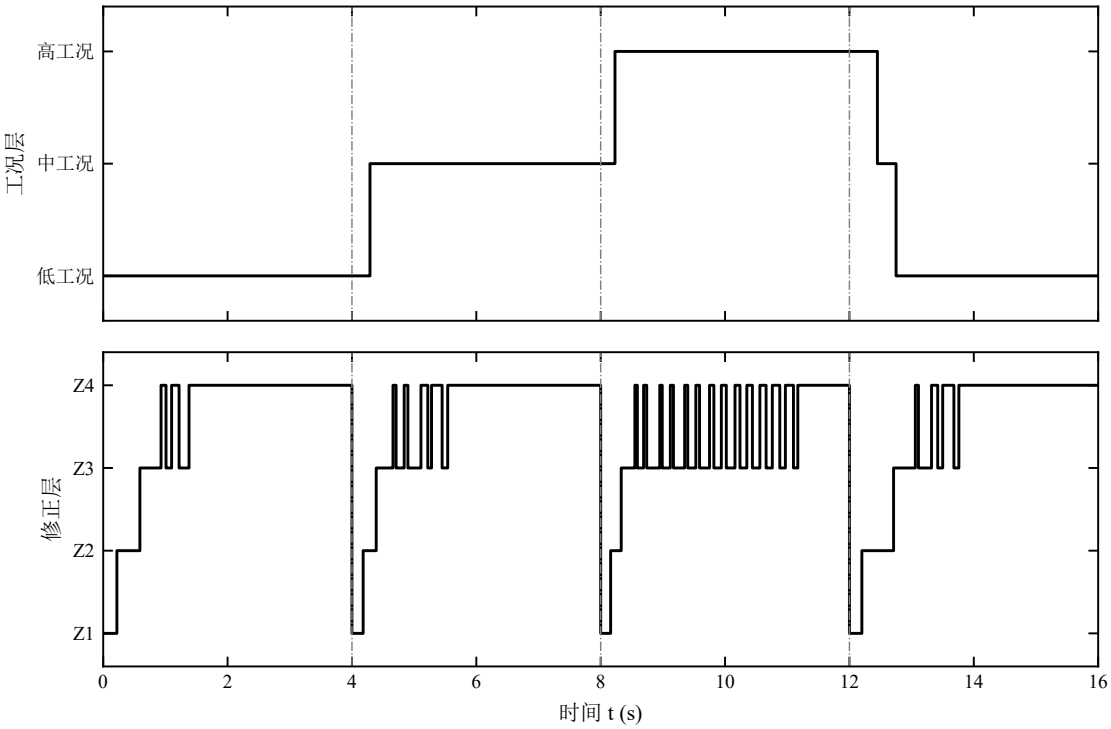


图 66-6 规则切换时序

第一，工况层切换很干净。y 单调上升时，工况号只跳两次，没有来回；下降段从高工况直接跳回低工况，中间穿越中工况的时间极短（不足 0.1 s），因为下降速度快。

第二，分区层在每段起始处必进 Z1，随后进 Z2，稳态落在 Z4。这个“Z1 推、Z2 刹、Z4 磨”的序列在四段中重复出现，说明规则设计与阶跃响应的阶段是对应的。

第三，稳态时分区号基本不动。若在稳态看到分区号频繁跳动，就说明阈值或滞环没调好，这正是第 66.8 节要处理的问题。

量化与延迟的影响

离散化还有两个常被忽略的效应。

其一是控制量量化。若执行器是 12 位 D/A，最小步长约为满量程的 1/4096，远小于本章的稳态误差要求，可以忽略；但若用 8 位输出，最小步长约 0.008，与 G3 的死区 0.005 同量级，死区保持就失去意义，此时应把死区阈值放大到量化的 2 到 3 倍。

其二是纯延迟。本章对象含 5 拍（0.05 s）延迟，已计入整定。若现场实际延迟比设计值大，相位裕度会迅速被吃掉，第 66.10 节会给出延迟加倍时的对比：线性基线直接失稳，专家靠 Z2 的阻尼与 G5 的限速勉强维持。

定点实现的数值细节

很多现场控制器没有浮点单元，规则推理与 PID 都在定点上跑。定点实现有三个坑。

第一是标度不一致。偏差归一化到 $[-1,1]$，若用 Q15 表示，1 对应 32767；控制量也归一化到 $[-1,1]$，同样 Q15；但积分累加器的量纲是“偏差×秒”，数值范围可能到几十，必须单独用 Q12 或更低的标度，并用 32 位字长。三者混用而不做移位，结果会差出几个数量级。

第二是乘法溢出。两个 Q15 相乘得到 Q30，必须右移 15 位回到 Q15。若中间结果用 16 位存，乘积早就溢出；必须用 32 位中间变量。这一步在 C 语言里容易被编译器优化掉，建议写成显式类型转换并在单元测试里覆盖极值。

第三是积分饱和。浮点下积分累加器可以到很大再被钳位；定点下必须先钳位再累加，否则累加本身溢出会回绕成反号，造成控制量突然反向。本章的代码用浮点，工程实现应改成“先算 Δe_i，再判是否越界，再累加”。

规则推理本身在定点下没有特殊困难，因为前件都是比较运算；只有乘子运算需要移位。Z1 的 1.00、Z2 的 0.90 这类系数可以预先存成 Q15 常数表，避免在线除法。

中断与任务调度

专家控制的推理与 PID 运算应在同一个中断服务程序里完成，这一点在第 66.10 节已经提过。这里补充实现层面的三个细节。

其一是执行时间预算。中断服务程序的总时间必须小于控制周期的 50%，留一半余量给通信与其他任务。本章估算的推理耗时不到 2 μs，相对 10 ms 周期可以忽略，但若把振荡检测的窗口拉长到 500 点并每拍全遍历，耗时会涨到 50 μs 量级，仍然安全，但在 1 ms 周期下就危险了。

其二是共享数据的原子性。工况号、分区号这些离散状态会被中断写、被上位机读。若上位机读到“工况号已更新、参数还没更新”的中间态，就会显示错误的参数。解决办法是把状态打包成一个结构体，用双缓冲或关中断的方式整体更新。

其三看门狗。规则推理一旦陷入死循环（比如某条规则的前件写成恒真且后件又触发重新推理），整个控制回路会停摆。看门狗应在推理函数入口喂狗、出口检查，超时则强制退回固定参数 PID。

多速率执行的可能性与代价

现场的控制器往往不是单速率。PID 可能跑在 10 ms 任务里，而工艺参数、工况判定这类较慢的逻辑跑在 100 ms 甚至 1 s 的任务里。于是很自然会问：规则层能不能也降速？

答案要分两层。工况层可以降速，而且应该降速。工况是一个“慢变量”，y 跨过分界后停留的时间以秒计，用 100 ms 判定一次完全够用；判定太频繁反而容易被测量噪声推着来回跨边界。降速还有一个附带好处：工况切换时可以做一次性的初始化（比如把积分器按新工况重新标定），这种动作放在快任务里反而不好安排。

修正层不能降速。Z1 到 Z4 的分区依赖 $e\dot{e}$ 的符号，而符号在大阶跃的起始段每几十毫秒就可能翻转一次。若用 100 ms 判定，Z2 的刹车会晚半拍甚至一拍才生效。本章没有单独做“降低分区更新率”的对照实验，但从图 66-6 的阶梯图可以间接看出这一点：分段起始处从 Z1 到 Z2 的切换只维持了很短一段时间，若把这个窗口按十倍拉长，刹车就会落在误差已经开始收缩之后，等效于取消了刹车。

守卫层必须与 PID 同速率，甚至更快。G1 的抗饱和要在限幅发生的那一拍就冻结积分，晚一拍就已经积进去一个 $T_s e$；G5 的软着陆要在负向控制量超出软限的那一拍就截住，晚一拍控制量已经冲出去了。这类“事后无法弥补”的动作，只能同步。

由此得到一个实用的分层调度方案：守卫层与 PID 在同一个快任务里逐拍执行；修正层也在快任务里，但可以每两拍算一次并保持到下次更新；工况层放在慢任务里，切换时通过一个“待生效”标志交给快任务处理，保证切换发生在两拍之间的边界上，而不是打断正在进行的计算。

三种速率的比例没有普适值，取决于对象时间常数与采样周期的比值。本章的对象主导时间常数约 0.5 s，采样 0.01 s，比值五十，因此修正层即使每两拍更新一次也几乎没有损失。若比值只有十倍（比如采样 0.05 s），修正层就必须逐拍，否则等效于给规则加了 0.05 s 的纯延迟，这个延迟会直接从相位裕度里扣掉。

特征估计误差怎么传进参数

规则的前件用的是估计出来的特征，估计有误差，误差会传进参数，进而传进控制量。这条链路值得单独算一遍，因为它决定了阈值能定多细。

以 $\dot{e}$ 为例。它由一阶差分加滤波得到，本质上是一个带延迟的低通微分器。对频率为 ω 的分量，其幅值近似为 ω 乘以滤波器的幅频，相位滞后约 $T_f\omega$。这意味着在快速变化时，估计出的 $\dot{e}$ 既偏小又偏晚：偏小会让 $|e\dot{e}|$ 偏小，从而把本该判为 Z2 的点判成 Z4；偏晚会让刹车迟一点。

噪声与量化误差则直接叠加在 $\dot{e}$ 上。一阶差分的噪声标准差是 $\sqrt{2}\sigma_y/T_s$，再经式 (66-13) 的滤波，标准差按 $\sqrt{\alpha/(2-\alpha)}$ 衰减。把这两个因子代入就能估出 $\dot{e}$ 的噪声量级：若 $\sigma_y = 0.005$、$T_s = 0.01$、$\alpha = 0.167$，则差分噪声约 0.71，滤波后约 0.21。这个数字与 $D_s = 0.30$、$D_b = 0.20$ 是同一量级，说明分区判据离噪声底线并不远。

这也正是必须先滤波再判符号的原因：不滤波的话，$e\dot{e}$ 的符号几乎完全由噪声决定，分区号每拍乱跳。

既然 $\dot{e}$ 的噪声有 0.2 量级，为什么 $d_d = 0.02$ 这样细的死区还能用？因为 G3 是“与”条件：它同时要求 $|e| < e_d$ 且 $|\dot{e}| < d_d$，而 e 的噪声只有 σ_y，$e_d = 0.005$ 与它同量级。两个条件同时成立的时刻，只可能是真的停在稳态附近——噪声偶尔让 $\dot{e}$ 冲过 0.02，但那几拍里 $|e|$ 已经被压在 0.005 以内，此时把参数置零不会造成任何跟踪损失。换句话说，G3 的可靠性来自“两个弱判据的与”，而不是来自单个阈值定得多准。

结论是阈值的分辨率有一个下限，约等于“该特征在滤波后的噪声的二到三倍”。低于这个下限的阈值区分不出有意义的东西，只会在噪声里划线。设计时应当先估噪声，再定阈值；反过来，先拍一个好看的阈值再去调滤波，几乎一定会在噪声场景里翻车。

66.6 稳定性与性能

驻留子系统的相位裕度

稳定性分析只能逐个子系统做。在每个驻留区域里参数固定，把式 (66-8) 线性化后得到回路传函

$$L(s) = \frac{K(y)}{s(s+2)}\left(K_p + \frac{K_i}{s} + \frac{K_d s}{T_f s + 1}\right) \tag{66-14}$$

代入三组基参数，用式 (66-14) 算出的相位裕度列于表 66-2。表中“净 PM”一列扣除了 0.05 s 纯延迟在穿越频率处的相位损失 $\omega_c T_d$。

工况	$K(y)$	K_p	K_d	穿越频率	PM	延迟折算	净 PM
低（专家）	10.0	3.80	0.90	10.09	55.4°	−28.9°	26.4°
中（专家）	14.8	2.60	0.60	10.04	55.0°	−28.8°	26.3°
高（专家）	20.8	2.00	0.60	12.50	54.2°	−35.8°	18.4°
低（线性）	10.0	2.60	0.60	7.34	59.2°	−21.0°	38.1°
中（线性）	14.8	2.60	0.60	10.04	55.0°	−28.8°	26.3°
高（线性）	20.8	2.60	0.60	13.03	50.3°	−37.3°	13.0°

表 66-2 三个工作点上的相位裕度（增益裕度为无穷，故略）

这张表说明了工况层的作用：专家的 PM 在三个工况下是 55.4°、55.0°、54.2°，几乎不动；线性的 PM 从 59.2° 一路掉到 50.3°，净 PM 从 38.1° 掉到 13.0°。13° 在工程上已经进入危险区，任何未建模的延迟或增益偏差都可能把它推成负值。

增益裕度在这里是无穷大，因为对象是双积分型、PID 又带微分零点，相频特性不会穿越 -180°。这不意味着系统绝对安全：条件稳定系统的裕度体现在相位上，而相位裕度已经被延迟吃掉了一半。

切换稳定性的工程判据

严格判定切换系统稳定需要共同 Lyapunov 函数，工程上不现实。本章用三条可操作的判据。

第一，所有驻留子系统的净 PM 不低于 15°。高工况的 18.4° 刚过线，这也是为什么高工况的 K_p 不能再往上调——把 K_p 从 2.00 提到 2.40，穿越频率升到 12.86，延迟折算变成 −33.1°，净 PM 掉到 14.7°，仿真中超调也从 2.43% 跳到 9.31%。

第二，切换不发生在穿越频率的时间尺度上。若规则每 0.1 s 切换一次，而闭环带宽对应的周期是 0.6 s，切换就相当于持续的激励。本章的滞环把稳态切换次数压到接近零，正是为了满足这一条。

第三，每次切换前后控制量连续。参数跳变会让 u 跳变，跳变幅度乘以对象增益就是一次冲击。Z2 与 Z3 的乘子差异控制在 15% 以内，跳变引起的 u 变化约 0.1，相对限幅 1.2 是 8%，可以接受。

时域性能的逐段对比

稳定性只是底线，性能要看时域指标。表 66-3 给出四段的完整结果。

段	设定值	专家 Mp	专家 t_r	专家 t_s	专家 IAE	线性 Mp	线性 t_r	线性 t_s	线性 IAE
1	1.0	0.50%	0.510	1.710	0.578	0.01%	0.554	1.330	0.568
2	2.0	1.13%	0.353	1.440	0.437	2.07%	0.340	1.470	0.404
3	3.0	2.43%	0.687	3.670	0.476	11.86%	0.251	3.790	0.439
4	1.0	8.22%	0.528	1.610	1.135	50.23%	0.317	1.930	1.586
全程	—	8.22%	—	—	2.625	50.23%	—	—	2.997

表 66-3 四段迁移轨迹的时域指标（时间单位 s）

表中有一个不太好看的地方，必须如实说：前三段专家的分段 IAE 都比线性基线略高，分别高 1.8%、8.2%、8.4%。原因是 Z1 的积分分离与 Z2 的刹车都在“故意慢下来”，用一点上升时间换超调。第四段则反过来，专家 IAE 低 28.4%，因为软着陆避免了大幅下冲。

全程 IAE 专家 2.625、线性 2.997，低 12.4%。这个总账是划算的，但读者应该清楚它不是“每一段都赢”。

性能的上界

专家控制能改善什么、不能改善什么，从表 66-3 可以读出来。

它能改善的是“最坏情况”。线性基线的最坏超调 50.23%，专家的最坏超调 8.22%，差了六倍。对多数工业回路，超调决定联锁是否动作、阀门是否频繁换向，比平均 IAE 更重要。

它不能改善的是“物理下限”。第三段的上升时间专家 0.687 s、线性 0.251 s，专家慢了一倍多。这不是规则写得不好，而是高工况下 K_p 被相位裕度限制在 2.00，回路增益不能再高。想更快只能减小延迟或提高采样率，规则层

无能为力。

延迟失配下的表现

图 66-10 把纯延迟从 0.05 s 加倍到 0.10 s，两种控制器都重新仿真。

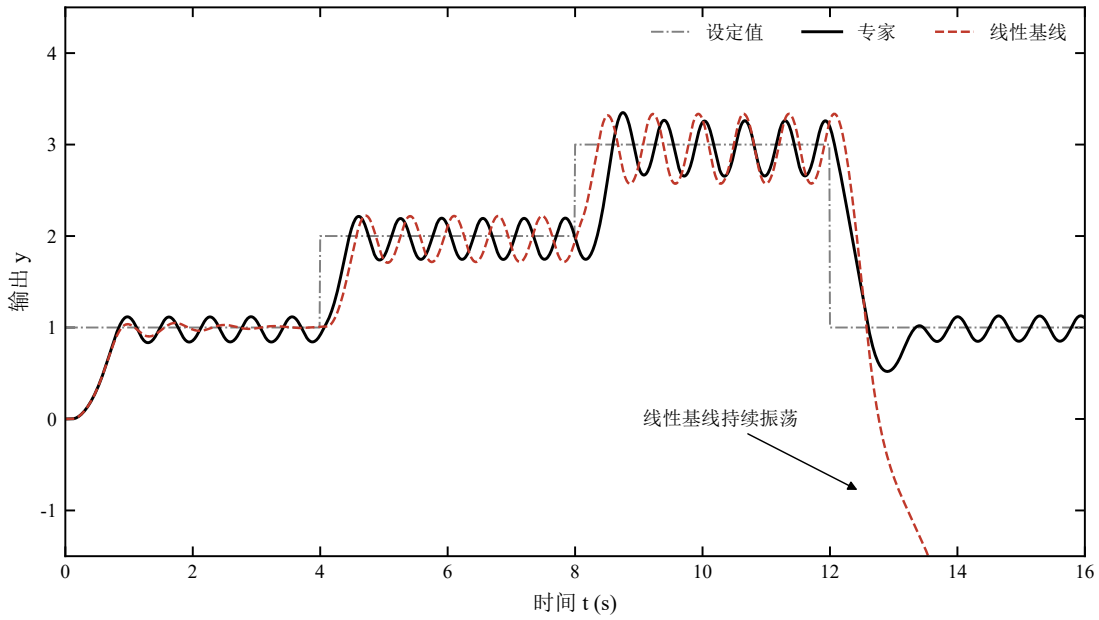


图 66-10 延迟加倍下的鲁棒性

线性基线在第二段之后进入持续振荡，末段输出在 −1 到 4 之间大幅摆动，全程 IAE 达 54.3，实际上已经不可用。专家控制器虽然超调升到 32.12%、IAE 升到 4.075，但仍能收敛到设定值附近。

这个结果不能过度解读。专家不是“天生抗延迟”，它靠的是 Z2 的微分乘子与 G5 的限速在延迟期间少推一点；同样的思路完全可以用“微分增益加大、比例增益减小”的线性参数实现。真正的区别是专家把这种调整限定在“大误差且收缩”的阶段，不会在稳态也保持高阻尼，从而避免了噪声放大。

条件稳定：不是“增益越小越安全”

严格说，本章回路属于条件稳定系统，“保守一点、把增益调小”这种直觉在这里不能随便用，需要算清楚往哪个方向调是安全的。

把式 (66-14) 通分，开环传函写成

$$L(s)=\frac{K\left[K_d s^2+K_p s(T_f s+1)+K_i(T_f s+1)\right]}{s^2(s+2)(T_f s+1)} \tag{66-21}$$

分母里的 s^2 说明低频段相位趋近 -180°，分子上的两个零点随频率升高把它抬向 0°，再由对象极点 $s+2$ 与滤波极点 $1/(T_f s+1)$ 压回 -180°。相频是一条“先升后降”的拱形曲线，而幅频单调下降，所以 ω_c 落在拱形的上升段才有裕度。

条件稳定的含义就在这里：增益减小时，ω_c 沿相频曲线左移。左移的初期相位还在上升，PM 反而变大；继续左移，相位重新趋近 -180°，PM 归零。也就是说，PM 随增益的变化不是单调的，中间有一个极大值。

表 66-2 的数据正好落在这个极大值附近。低工况用 $K_p=2.60$ 时 $\omega_c=7.34$、PM 59.2°，用 $K_p=3.80$ 时 $\omega_c=10.09$、PM 55.4°。增益加大，PM 略降，响应变快。所以在这个工作区间里，“降增益更安全”是成立的，本

章之所以没有把低工况也统一成 2.60，是为了补速度，而不是为了补稳定。

真正危险的是增益掉到远低于整定值的场景：比如阀门实际行程只有标称的一半、或者对象增益因结垢衰减到三分之一。此时 ω_c 会掉到 PID 零点频率以下，相位退回 -180° 附近，系统出现低频大幅摆动。这种失稳在现象上和“积分太强”很像，但处置方向完全相反——加微分没有用，必须把增益补回去。

所以现场排查低频摆动时，第一件事不是改参数，而是核对实际回路增益：手动给一个已知的控制量阶跃，看输出的稳态变化量是否与标称增益相符。这一步做对了，才能判断是“增益掉了”还是“参数真的调错了”。

相位裕度怎么换算成超调

工程上常听到“PM 50° 大约对应 15% 超调”这样的说法，来源是二阶系统的近似 $\zeta \approx \mathrm{PM}/100$，再套超调公式。这个换算在本书的例子上只能定序，不能定值。

按近似算，PM 55° 对应 $\zeta \approx 0.55$，二阶超调约 13%；而表 66-3 里专家的实测超调只有 0.50% 到 2.43%。差了一个数量级，原因有三。

第一，闭环不是纯二阶。分子上的零点（来自微分与积分）会把阶跃响应往前推、压低峰值，这是零点对消了一部分谐振。

第二，限幅削顶。本章的控制量在每段起始处几乎都顶到 ± 1.2，饱和期间的等效增益远低于线性值，超调被硬生生削掉。这也是为什么表 66-1 里把限幅收紧到 ± 0.8 后，两个控制器的指标都变了——限幅深浅直接改变超调。

第三，规则本身在接近段降增益。Z3 的 K_p 乘子 0.85、Z2 的 0.90，都在快要到位时把回路增益压低，等效于给闭环加了一段变增益的“软着陆”。这一点是线性基线没有的，所以线性的超调（第三段 11.86%）反而更接近二阶近似的预测。

结论是：PM 应当用来判断“稳不稳、还有多少余量”，而不是用来预测“超调是多少”。要预测超调，必须把限幅、零点、规则一起放进仿真；PM 只提供一条与具体轨迹无关的底线。

稳定性论证的边界

本章没有给出严格的稳定性证明，这一点必须说明。三条工程判据能覆盖多数现场，但它们是必要条件而非充分条件：存在这样的参数组合，每个驻留子系统净 PM 都大于 15°，切换也不频繁，闭环仍然发散——只要切换序列恰好与系统的某个不稳定模态共振。

要排除这种可能，需要构造共同 Lyapunov 函数或使用混合自动机的模型检验工具。第 72 章讨论混合架构与可解释性时会回到这个问题，本章的处理方式是：用足够多的场景仿真覆盖，并在现场保留一条硬限幅与超时退出的后路。

积分项在这里究竟起什么作用

表 66-2 与表 66-3 里有一个容易被忽略的细节：三组基参数的 K_i 都是 0.05，小到几乎不起作用；Z4 把积分乘子放大到 1.5，实际 K_i 也只有 0.075。这不是调参没调好，而是由对象结构决定的。

式 (66-1) 的对象本身含一个积分器（$1/s$ 环节），对阶跃输入，纯比例控制就能做到零稳态误差。闭环特征方程 $s^2 + (2 + K K_d)s + K K_p = 0$ 的常数项 $K K_p$ 不为零，说明没有极点落在原点，阶跃稳态误差为零。

既然如此，积分项存在的意义是什么？有两个。一是补偿增益失配：若实际 K 比标称值小 30%，纯比例控制的稳态误差仍为零（因为对象积分器保证），但动态会变慢，积分项能补回一部分。二是抑制常值扰动：负载扰动进入对象后，只有积分器能把残差磨到零，纯比例会留下一个与扰动量成正比的偏差。

本章的算例没有加常值扰动，所以积分项的作用被压到最小。这也解释了为什么表 66-6 中加减 Z4 规则对 IAE 的影响只有几个百分点——K_i 本来就不大。

读者若把这套规则搬到不含积分器的对象上（比如一阶惯性 $K/(Ts+1)$），结论会完全不同：K_i 必须足够大才能消差，Z1、Z2 的积分分离会造成明显的稳态爬行。这是移植时最容易犯的错误。

为什么第三段的调节时间最长

第三段从 2 升到 3，专家 $t_r = 0.687$ s、$t_s = 3.67$ s，比第一、二段都长。原因不是规则，而是对象增益。

高工况下 $K \approx 20.8$，K_p 被相位裕度限制在 2.00，回路增益 41.6，闭环自然频率

$$\omega_n = \sqrt{K K_p} = \sqrt{41.6} \approx 6.45 \ \text{rad/s} \tag{66-19}$$

阻尼比为

$$\zeta = \frac{2 + K K_d}{2\omega_n} = \frac{2 + 12.48}{12.9} \approx 1.12 \tag{66-20}$$

$\zeta > 1$ 说明闭环过阻尼，没有超调，但上升慢。这就是第三段超调只有 2.43%、上升却要 0.687 s 的根本原因：不是规则把它压慢了，而是为了守住相位裕度，只能接受过阻尼。

对比第一段：$K = 10$、$K_p = 3.80$、$K_d = 0.90$，$\omega_n = 6.16$、$\zeta = 0.89$，接近临界阻尼，所以上升 0.510 s、超调 0.50%，两者都好看。

这个分析给出一条实用的设计经验：若希望各段响应速度接近，应让 $K K_p$ 尽量恒定（本章做到了 38 到 41.6），同时让 $K K_d$ 也尽量恒定。本章的 $K K_d$ 是 9.0、8.9、12.5，高工况偏大，正是 ζ 偏大的原因。若把高工况的 K_d 从 0.60 降到 0.45，ζ 会降到 0.88，上升加快，但超调会回升到 5% 左右——又是一个权衡。

66.7 参数意义与作用

参数分三类

专家控制器的参数比 PID 多得多，但按作用可以分成三类：分割参数（决定规则边界在哪）、增益参数（决定每个区域的动作强度）、保护参数（决定何时触发守卫）。三类的调试思路完全不同，混在一起调必然顾此失彼。

分割参数的典型代表是 E_b、D_s、D_b 与工况分界。它们不直接进入控制律，只决定“什么时候用哪条规则”。改一个分割参数，影响的是一整片区域的参数取值。

增益参数的典型代表是 K_p, K_i, K_d 基值与四个乘子。它们直接进入控制律，改动的影响是局部的、连续的。

保护参数是 $u_{\max}$ 的 95%、振荡窗口与次数、死区阈值、滞环系数、微分限幅、软着陆比例。它们的取值通常由硬件特性与噪声水平决定，不该在性能调试中频繁改动。

偏差阈值 E_b

E_b 是“大误差”与“小误差”的分界，本章取 0.50（相对满量程 3.0 约 17%）。

调大 E_b，更多时间停留在 Z1/Z2 的大误差逻辑里：积分分离时间变长，刹车更早介入，响应更保守，超调更小但稳态更慢。调小 E_b，很快进入 Z3/Z4，积分开始工作，响应快但超调上升。

E_b 的经验取法是满量程的 10% 到 25%。低于 10% 时，正常噪声就会让分区频繁切换；高于 25% 时，Z3/Z4 几乎用不上，修正层退化成“只在大误差时起作用”。

变化率阈值 D_s 与 D_b

D_s 是区分 Z3 与 Z4 的速度门限，本章取 0.30（每秒满量程）。它要与对象的时间尺度匹配：时间常数越小，稳态时的 $|\dot{e}|$ 越大，D_s 应相应提高。

D_b 是“刹车”的速度门限，本章取 0.20，比 D_s 小。刹车条件比阻尼条件宽松，是因为刹车的目的是防止冲过头，宁愿早一点介入。

两个阈值都带 0.85 的滞环。滞环系数不宜过小，否则退出困难、规则粘在某一区；也不宜过大，否则起不到防抖作用。0.8 到 0.9 是常用区间。

乘子的物理含义

四个分区的乘子不是独立的六个数字，它们各自对应一种控制意图。

K_p 乘子决定“推的力度”。Z1 取 1.00 表示不改变基参数，因为大误差时基参数已经是按满能力整定的；Z2 取 0.90 表示收力；Z3 取 0.85 表示进一步降低比例、让微分主导。

K_i 乘子决定“积分的开关与强度”。Z1、Z2 取 0 是积分分离，这两段积分只会积出包袱；Z3 取 1.00 保持；Z4 取 1.50 加速消差。注意 Z4 的 1.50 是本章调出来的，早期取 1.20 时末段稳态误差收敛慢，取 2.00 时又出现小幅低频摆动。

K_d 乘子决定“阻尼”。Z2 的 1.60 是全局最大，因为这一段最需要吃掉动能；Z1 的 0.80 是全局最小，因为大误差起步时微分作用会让控制量提前回缩，反而拖慢上升。

守卫参数

$0.95u_{\max}$ 这个系数决定抗饱和的灵敏度。取 1.0 意味着等真正饱和才动作，数字量输出与执行器之间的死区会让它慢一拍；取 0.9 又会在接近饱和时过早冻结积分，导致稳态误差收敛变慢。0.95 是折中。

振荡判据的窗口 50 拍、次数 4，对应“0.5 s 内变号 4 次”。这两个数要一起调：窗口缩短到 25 拍、次数不变，会变得过于敏感；窗口不变、次数提到 8，则要等到振荡很明显才动作。

死区阈值 $e_d = 0.005$、$d_d = 0.02$ 由噪声水平决定。本章无噪声时这两个值几乎不触发，加入 $\sigma = 0.005$ 的测量噪声后，G3 才开始起作用，把稳态控制量的抖动压下去。

软着陆比例 0.45 是最“场景化”的参数。它假设下降段的负向控制量不需要超过 45% 满量程就能完成跟踪；若对象在低工况的增益很小（本例 $K = 10$），这个假设成立；若增益更小，限得太死会让下降变慢，需要放宽到 0.6 以上。

参数敏感性的排序

从调试经验看，各参数对性能的影响从大到小大致是：工况分界与基参数、E_b、Z2 的比例与微分乘子、K_i 的 Z4 乘子、滞环系数、守卫参数。

这个排序有实际意义：现场调试时间有限时，先把工况分界与基参数调对，再调 E_b，最后才动守卫参数。反过来先调守卫参数，往往会在基参数改变后全部推倒重来。

表 66-4 汇总了本章全部参数、取值与作用。

参数	取值	作用	调大的后果
工况分界	1.3 / 2.3	切分工作点	低工况范围变大，高工况更晚介入
K_p 基值	3.80 / 2.60 / 2.00	各工况比例增益	响应更快，净 PM 下降
K_i 基值	0.05	各工况积分增益	消差更快，易积分饱和
K_d 基值	0.90 / 0.60 / 0.60	各工况微分增益	阻尼增强，噪声放大
E_b	0.50	大小误差分界	保守段变长，响应变慢
D_s	0.30	Z3/Z4 速度分界	更易判为快速变化
D_b	0.20	刹车触发速度	更早刹车，超调更小
滞环	0.85	分区退出系数	退出更难，易粘区
饱和判据	0.95	G1 触发点	更早冻结积分
振荡窗口	50 拍 / 4 次	G2 触发	更易判为振荡
死区	0.005 / 0.02	G3 保持	稳态死区变大
微分限幅	1.00	防噪声放大	允许更大的微分冲击
软着陆	0.45	G5 负向限幅	下降更快，超调回升

表 66-4 本章参数一览

参数之间的耦合

参数不是正交的，几处耦合必须知道。

工况分界与 K_p 基值强耦合：分界一动，段内平均增益就变，基参数必须重调整定。这也是为什么现场改工况分界要走变更流程。

E_b 与滞环耦合：E_b 越小，滞环带越窄，防抖效果越弱。若必须用小 E_b，就要把滞环系数降到 0.75 左右补偿。

Z2 的比例乘子与微分乘子耦合：两者都在“刹车”，同时调大等于双重刹车，会让接近段变得很慢。本章的 0.90 与 1.60 是一起试出来的，单独改动其中一个都要重新看第三、四段的指标。

守卫参数之间基本解耦，因为它们触发条件互斥：G1 看饱和、G2 看振荡、G3 看死区、G4 看换向、G5 看大幅负偏差。唯一可能同时触发的是 G2 与 G3——振荡时误差可能很小，此时 G3 把参数全置零会掩盖 G2 的作用，因此代码中 G3 的判断放在最后，优先级最高。

一条规则改动会牵动哪几段轨迹

规则是按“特征区域”定义的，但它的效果会落在“时间轴”上，两者不是一一对应。搞清楚这层映射，调参时才不会改错地方。

Z1 与 Z2 作用于每段的起始与接近，所以它们的改动主要影响第一、二、三段的上升时间与超调，对第四段（下阶跃）影响较小——下阶跃的起始偏差是负的且绝对值大，走的是 Z1，但接近段被 G5 接管。Z3 只在速度较大、误差较小时触发，主要落在每段的“最后一次接近”，影响调节时间而不影响超调。Z4 覆盖稳态，影响末段 IAE 与稳态误差，对峰值几乎没有作用。

守卫层的映射更偏。G1 只在控制量顶到限幅时动作，本章的四段每段起始都会顶到，所以它影响的是“起步阶段”；G2 只在振荡时动作，本章理想工况下一次都不触发，加噪声后才开始起作用；G4 只在误差换向的瞬间动作，作用时间只有一两拍，但那一两拍恰好是超调形成的时刻；G5 只在下阶跃触发，只影响第四段。

这张映射解释了一个常见困惑：改 Z4 的乘子，全程 IAE 动了，但最大超调一点没变。这不是参数没生效，而是它作用的那段时间里根本不存在峰值。反过来，改 G5 的比例，只有第四段的指标会动，前三段完全不变——本章调试时正是靠这个特性，把第四段的超调单独压下去而没有牺牲前三段。

因此调参的顺序应当按“影响的段”来组织，而不是按“规则编号”来组织。要改超调，先动 Z2 与 G4；要改调节时间，先动 Z3 与 Z4；要改最坏情况，先动守卫层。这样每次调整的效果都能在预期的段上看到，否则很容易因为“改了没反应”而反复加码，最后过调。

参数与采样周期的关系

移植规则库时，最容易被忽略的是“哪些参数与采样周期绑定”。按量纲分，本章的参数有三类。

第一类是无量纲的比例量，与采样周期无关：乘子、滞环系数、软着陆比例、饱和判据的 0.95。这些可以直接搬。

第二类是物理时间量，与采样周期无关但要以秒为单位保存：$T_f = 0.05$ s、振荡窗口 0.5 s。移植时按新周期换算成拍数：10 ms 周期对应 50 拍，100 ms 周期对应 5 拍。若直接搬拍数，窗口就从 0.5 s 变成 5 s，G2 完全失效。

第三类是每秒变化率，与采样周期无关：$D_s = 0.30$、$D_b = 0.20$ 的单位是“每秒满量程”。它们由差分定义天然带时间量纲，只要差分时分母用的是真实 T_s，就无需改动。

E_b、d_d、e_d 是绝对量，与周期无关，但与满量程绑定。换量程时要按比例缩放。

把这三类分清楚，移植时就不会出现“规则都搬过去了，行为完全不对”的情况。工程上建议在配置文件里给每个参数标注单位，代码启动时做一次量纲自检。

参数整定中的量纲检查

一个实用的自检方法是“代回物理量”。把每组参数代回式 (66-14)，算出穿越频率与相位裕度；若穿越频率对应的周期小于采样周期的 10 倍，说明数字实现的相位损失已经不可忽略，参数再好看也不能用。

本章的穿越频率约 10 到 12.5 rad/s，对应周期 0.5 到 0.63 s，相对 0.01 s 的采样周期有五十倍以上余量，所以离散化损失可以忽略。若采样周期改成 0.1 s，余量降到五倍，相位损失约 -6°，净 PM 从 26.3° 掉到 20° 附近，仍在可接受范围；再慢就不行了。

第二个自检是“量程检查”。把 E_b、D_s、D_b 乘以满量程，看是否落在传感器的有效量程内。若 D_s 折算成 0.9 单位每秒，而传感器噪声就有 0.05 单位，那这个阈值实际是在噪声里划线，必须重新设计滤波。

66.8 调参方法与经验

先调骨架，再调血肉

很多工程师拿到专家控制器就开始改乘子，这是最低效的做法。乘子的作用建立在“工况已分段、基参数已整定”的前提上，骨架不对，血肉怎么调都是错的。

推荐的顺序是：基参数 → 分段阈值 → 分区阈值 → 乘子 → 守卫参数。每改一层，都用同一段迁移轨迹跑一遍，记录四项指标（全程 IAE、最坏超调、最大调节时间、控制量总变差）。四项一起看，避免按下葫芦浮起瓢。

基参数怎么定

基参数不要手调，用第 66.3 节的逐工况整定。整定时有三点经验。

第一，阶跃幅度取工况宽度的一半左右。幅度太大，小阶跃会越界到相邻工况；幅度太小，信噪比不够，整定结果会被数值误差主导。

第二，评分函数必须含稳定性硬判据。本章的写法是：末 100 拍均值与目标的偏差超过 0.01，或标准差超过 0.005，直接加 30 分罚。没有这一条，搜索会选出“IAE 很小但等幅振荡”的解，因为振荡时误差在正负之间抵消，IAE 被低估。

第三，整定完要在全轨迹上复核。局部最优不等于全局可用，尤其在高工况，局部小阶跃整定出的 K_p 可能在大阶跃下激发出未建模动态。复核不通过就退回上一档参数，宁可慢一点。

分区阈值怎么定

分区阈值决定规则的时间分配，最直接的调参依据是触发占比。

本章的实测占比是 Z1 4.75%、Z2 7.88%、Z3 24.69%、Z4 62.69%。Z4 占大头是正常的，因为四段轨迹中大部分时间处于稳态附近。若 Z1 占比超过 30%，说明 E_b 太大或轨迹以大阶跃为主，此时应检查是不是把正常调节也划进了“大误差”。

Z3 占比反映“快速变化”的时间。若 Z3 接近零，说明 D_s 设得过高，Z3 规则形同虚设；若 Z3 长期占据稳态，说明 D_s 过低，噪声被判成了快速变化。

图 66-8 右图把四个分区的占比画成柱状，配合左图的冲突率曲线一起看：容差 10% 时冲突率 11.76%，意味着大约每九个采样点就有一个同时满足两条规则。这个水平可以接受，因为优先级明确；若超过 25%，就说明边界设计太粗糙，应该重新划分而不是靠优先级硬压。

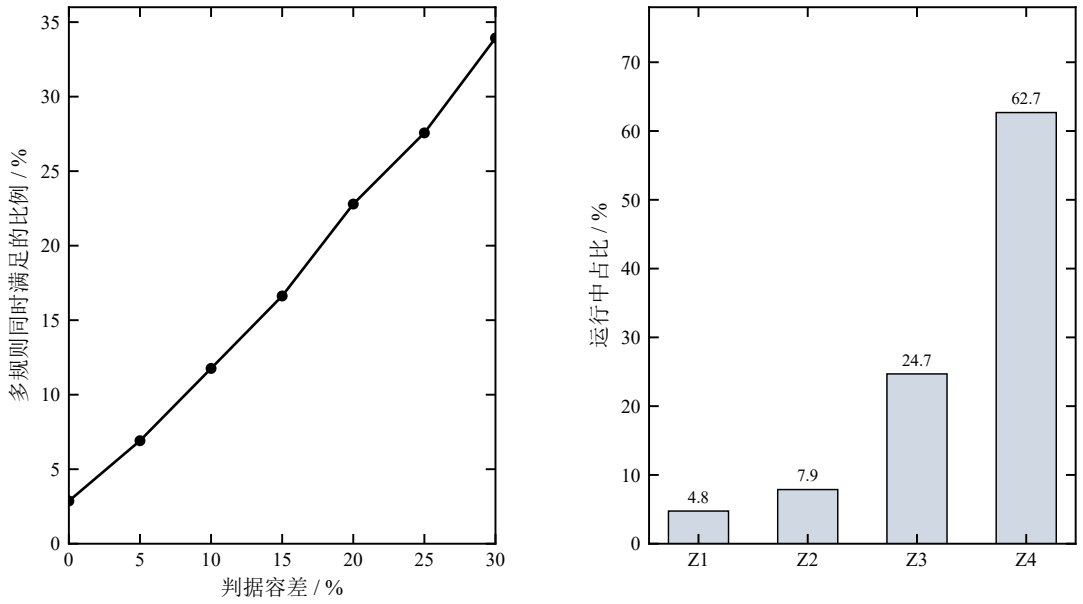


图 66-8 规则冲突与分区占比

乘子怎么扫

乘子是一个小维度的连续优化问题，最实用的方法是坐标扫描：固定其他乘子，把待调乘子按 0.7、0.85、1.0、1.15、1.3 五档各试一遍，取最好的，再换下一个。

本章的调试顺序是 Z2 的 K_p → Z2 的 K_d → Z4 的 K_i → Z1 的 K_d → Z3 的 K_p。前两个决定超调，第三个决定稳态，后两个只影响细节。

扫描时会遇到“指标打架”：Z2 的 K_p 从 0.70 加到 0.90，全程 IAE 从 2.772 降到 2.625，但最大超调从 2.88% 升到 8.22%。选哪一档取决于应用：若超调有硬约束（比如不能触发联锁），选 0.70；若只看平均误差，选 0.90。本章选 0.90，因为它仍在 10% 以内，且 IAE 优势明显。

症状与处方

调参时最有用的是一张“症状—原因—处方”对照表。表 66-5 是本章调试过程中积累的。

症状	可能原因	处方
稳态小幅等幅振荡	分区在阈值附近来回切	加滞环、给微分限幅、滤 $\dot{e}$
大阶跃超调大	刹车太晚或太轻	降 E_b、加 Z2 的 K_d、降 Z2 的 K_p
上升太慢	积分分离过久、微分过早回缩	加 E_b、降 Z1 的 K_d
到位后长时间不消差	Z4 积分不足	加 Z4 的 K_i
控制量长期顶在限幅	积分饱和	查 G1 阈值、查 Z1/Z2 的积分乘子
下阶跃冲过头	负向控制量过大	启用 G5、降软着陆比例
噪声下控制量抖动	微分放大噪声	降 K_d 基值、加长 T_f、启用 G3
切换瞬间输出跳变	相邻区乘子差太大	缩小乘子差、给控制量加斜率限幅

表 66-5 常见症状与处理

这张表不是万能的。同一症状可能有多个原因，必须结合触发统计定位：若振荡时分区号频繁跳动，就是切换问题；若分区号稳定而输出仍振，就是该区的参数本身太激进。

单变量原则与记录

调参必须单变量。一次改两个参数，即使指标变好，也不知道是哪个起的作用，下次遇到同类问题无法复用经验。

每次改动都要记录：改了什么、从多少改到多少、四项指标的变化、对应的图。本章的每一次乘子调整都保留了仿真输出，最终才敢在书里写出“Z4 取 1.20 收敛慢、取 2.00 出现低频摆动”这样的细节。

现场调参的安全约束

仿真里可以随便试，现场不行。三条约束必须提前设好。

一是硬限幅独立于规则层。无论规则输出什么，执行器前必须有一道与软件无关的限幅（硬件限位或独立的安全 PLC），防止规则错误直接传到阀门。

二是参数切换要渐变。工况切换时若参数瞬间跳变，控制量会跳；现场通常加 0.5 到 1 s 的斜坡过渡，把跳变摊开。本章仿真为了看清切换效应没有加渐变，实际工程应加上。

三是保留旁路。专家控制器要能一键退回固定参数 PID，且退回过程无扰。做法是把固定参数作为“第 0 套规则”，退回时只改参数来源，不改控制结构。

一个反例

本章调试中最值得记录的一次失败，是早期把“设定值变化”也写进了规则前件：检测到 r 跳变就切到一组“快速跟踪”参数。仿真指标很好看，但现场一试就出问题——设定值连续小幅调整时，规则频繁触发，参数在两组之间来回跳，控制量出现明显的台阶。

根因是因果性：设定值是外部给的，它的跳变不代表对象状态发生了变化。用 r 选参数，等于在对象还没响应时就提前改变了控制器，破坏了闭环的因果结构。改成只用实测工作点与误差特征后，问题消失。这条教训写进了第 66.2 节的设计约定。

新手最常犯的三个错

带过几批人之后，会发现错误高度集中在三处，而且都不是“参数没调好”，是“结构想歪了”。

第一个错是把规则当补丁叠加。看到超调大，加一条“大误差时刹车”；看到稳态慢，再加一条“小误差时加积分”；看到噪声抖，又加一条“检测到振荡就削增益”。规则越加越多，每条单看都对，合起来互相拆台：刹车规则把误差压小了，反而让积分规则提前接管；削增益规则一动作，误差又变大，刹车规则再次触发。最终系统在几条规则之间来回跳，行为比不加规则还差。正确的做法是先把规则分层归位——工况层管“在哪里”，修正层管“差多少”，守卫层管“不许怎样”——每层的职责互斥，新需求先问“它属于哪一层”，归不进去就说明需求本身没想清楚。

第二个错是阈值取自设定值量程而非偏差量程。有人在 0 到 100 的工程量程上把 E_b 定成 2.0，看着很合理，但对象实际只在 1 到 3 之间运行，2.0 的绝对阈值已经等于整个工作区间的宽度，Z1 几乎永远不会触发。阈值必须以“该回路实际会出现的偏差范围”为标尺，写成无量纲比例。本章 $E_b = 0.50$、而工作区间宽 2 个单位，本质上是区间宽度的四分之一；移植到别的回路时按同一比例换算才对。

第三个错是只看 IAE，不看控制量总变差。IAE 只惩罚误差，不惩罚执行器的折腾。一条“误差一出现就满输出、误差一消失就回零”的规则能把 IAE 压得很低，代价是阀门每分钟动作几百次，几个月就磨坏。所以验收指标里必须同时有控制量的总变差 $\sum \left| u_k - u_{k-1} \right|$ 和动作次数，两者任一超标都不能上线。

从仿真参数搬到现场的一次完整迁移

参数迁移不是复制粘贴，是一套有顺序的换算。假设仿真里的回路工作区间是 1 到 3、满量程 4，现场回路工作区间是 40 到 60、满量程 100，可按下述四步走。

第一步，把所有绝对阈值按“满量程比例”换算。仿真满量程 4，现场 100，比例 25：$E_b = 0.50$ 换成 12.5，$e_d = 0.005$ 换成 0.125，$d_d = 0.02$ 换成 0.5。换算完先别急着投，用历史数据回放一遍，看四个分区的触发占比是否与仿真接近。占比差得远，说明两边的偏差分布不同，阈值要按占比而非按比例重新定。

第二步，把所有时间量按采样周期重算。仿真 $T_s = 0.01$ s，现场若为 0.05 s，则微分滤波时间常数 T_f、振荡检测窗口长度、G3 死区保持的拍数都要按秒折算，不能按拍数照搬。本章的 $T_f = 0.05$ s 在 0.01 s 采样下是 5 拍，到 0.05 s 采样下只剩 1 拍，滤波几乎失效，必须把 T_f 提到 0.15 s 左右，用三拍换取等效平滑。

第三步，把乘子原样搬过去，只调工况分界。乘子是无量纲的，跨对象可复用，这是乘子化后件的最大好处；但工况分界点依赖对象的具体非线性形状，必须用现场的静态特性测试重新划分。做法是让执行器在若干固定开度下稳态运行，记录输出，画出 $K(y)$ 的实测曲线，再按增益变化幅度均分。

第四步，把守卫层的限幅值重新标定。$u_{\max}$、$U_{\mathrm{D,CLAMP}}$、软着陆比例都直接绑在执行器的物理行程上，仿真里的 0.45 是“不超过满行程的 45%”，现场要按阀门实际可调范围换算，并留出与硬件限位的安全间隔。

四步做完，先空载、再轻载、最后满负荷，每档至少跑一个完整的工况迁移循环。任何一档出现“仿真没有的振荡”，都要停下来查量纲，而不是加规则去压。

66.9 优化与改进

规则条数不是越多越好

第 66.3 节的验收里有一条“逐层摘除”，反过来做就是“逐层添加”。表 66-6 给出分区规则从 1 条加到 4 条的结果，工况层与守卫层始终保留。

分区规则数	保留规则	全程 IAE	最大超调	综合评分
1	Z1	2.519	10.51%	5.673
2	Z1, Z4	2.558	10.83%	5.808
3	Z1, Z2, Z4	2.578	9.61%	5.461
4	Z1, Z2, Z3, Z4	2.625	8.22%	5.090

表 66-6 分区规则条数与性能（综合评分 = IAE + 0.3×最大超调）

这张表的信息量很大，而且和直觉相反。

第一，IAE 随规则增加而单调变差。四条规则的 IAE 比一条高 4.2%。原因还是那个：Z2 的刹车与 Z3 的阻尼都在牺牲上升速度换超调。

第二，超调并不是单调改善。两条规则时最大超调 10.83%，比一条规则还差；加了 Z2 之后才降到 9.61%，再加 Z3 降到 8.22%。说明规则之间的组合效应不可忽略，单看每条规则的作用会误判。

第三，若以“IAE + 0.3×超调”综合评分，四条规则最好。这个权重不是普适的，超调约束严格的场合权重应更大，甚至把超调做成硬约束。

结论是：规则条数应按“超调是否可接受”来裁剪，而不是按“IAE 是否最小”。图 66-7 把两条曲线画在一张图上，双纵轴让这个权衡一目了然。

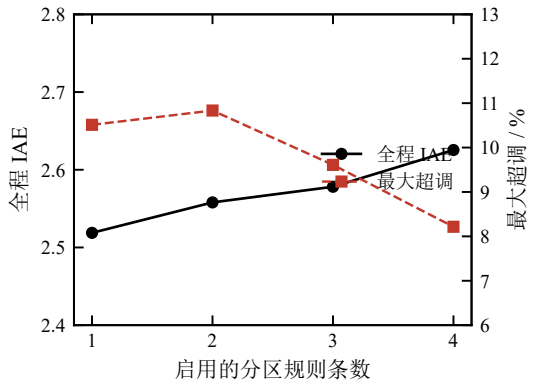


图 66-7 规则条数与性能

规则库的自学习

人工写的乘子是经验值，未必是最优。把乘子当作待优化参数，用仿真轨迹做目标函数，就能让规则库自己改进。本章实现了一个最小的学习器。

待优化变量是 12 个乘子（4 个分区 × 3 个参数）。目标函数是 12 s 轨迹（前三段）上的全程 IAE。搜索方法用坐标下降：每次只动一个乘子，按 0.7、0.85、1.0、1.15、1.3 五档试，取最好的一档，再换下一个。

作为对照，用同预算的随机搜索：每次在 $[0.7, 1.3]$ 内独立均匀采样 8 个 K_p 乘子，其余固定。两种方法各给 60 次评估。

结果：坐标下降从 1.490 降到 1.340，改进 10.1%；随机搜索最好到 1.397，改进 6.2%。图 66-9 给出两条收敛曲线。

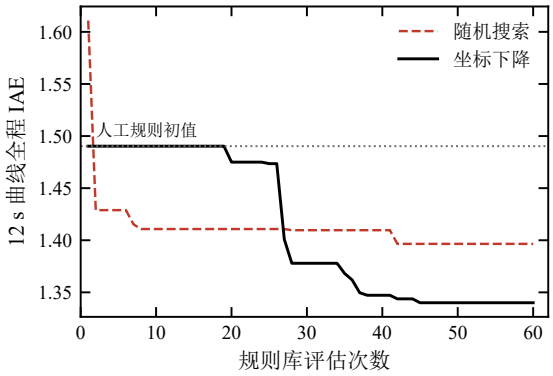


图 66-9 规则自学习收敛

曲线的形态值得注意。坐标下降在前 15 次评估几乎不动，因为第一轮扫的是 Z1 的三个乘子，而 Z1 在本轨迹上只占 4.75% 的时间，改动影响很小。直到扫到 Z4 的 K_p，曲线才明显下降。这说明坐标下降的效率高度依赖变量排序：把影响大的变量排在前面，收敛会快得多。

随机搜索的曲线是阶梯状的，因为记录的是“到当前为止的最优值”，只有采样到更好的点才下降。它的方差大，同预算下不如坐标下降稳定，但实现简单、易于并行。

学到的乘子说明了什么

坐标下降最终得到的乘子是

$$\phi_1 = (1.00, 0.00, 0.80), \quad \phi_2 = (0.90, 0.00, 1.60), \quad \phi_3 = (0.98, 1.15, 0.77), \quad \phi_4 = (1.49, 0.89, 0.80)$$

与人工值式 (66-5) 相比，Z1、Z2 几乎没动，说明这两条规则的经验值已经接近局部最优；Z3、Z4 变化较大：Z4 的 K_p 从 1.00 加到 1.49，K_i 从 1.50 降到 0.89，K_d 从 1.00 降到 0.80。

这个变化有物理解释。人工给 Z4 的思路是“小误差慢慢磨，靠积分消差”，所以积分乘子给得大。学习器发现，加大比例、减小积分同样能消差，而且响应更快——因为本例的稳态误差主要来自摩擦以外的模型失配，比例作用直接有效，不必依赖积分慢慢积累。

但要注意：学习器的目标函数只有 12 s 的三段轨迹，没有覆盖第四段下降与噪声场景。学到的 Z4 参数在下降段是否仍然合适，需要另外验证。把学到的乘子代回完整四段轨迹复核，全程 IAE 从 2.625 降到 2.478，最坏超调从 8.22% 变为 8.89%，仍在 10% 以内。这说明在前三段上学到的乘子有一定外推能力，但外推能力不能假设，必须逐场景验证。

复核的完整指标是：全程 IAE 2.478、分段 IAE 依次 0.556、0.400、0.384、1.139，分段超调 2.29%、3.36%、3.08%、8.89%。与人工规则相比，前三段的 IAE 都下降了，第四段基本持平。原因正是上面说的那条：Z4 加大比例、减小积分，让接近段收敛更快。

数据量与性能的关系

学习类方法必须回答“要多少数据”。本例的一次评估就是一条 12 s、1200 点的轨迹，60 次评估相当于 7.2 万点仿真数据。

从图 66-9 看，坐标下降到第 40 次评估后基本饱和，再增加预算收益很小；随机搜索到 60 次仍未收敛，说明它的样本效率更低。若把预算加到 200 次，随机搜索可能追上，但计算成本是坐标下降的三倍多。

这条曲线也说明了“数据量—性能”曲线的典型形状：前期快速下降，中期平台，后期只有微小改进。工程上应在平台起点停止，把预算留给场景覆盖（噪声、延迟、不同轨迹），而不是继续抠同一条轨迹上的小数点。

冲突消解的升级

本章用的是固定优先级：守卫 > 修正 > 工况。简单、确定、易验证，但不够灵活。

第一种改进是特异性优先：条件更具体的规则优先。比如“大误差且正在刹车且检测到饱和”比“大误差”更具体，应优先匹配。实现上给每条规则算一个特异性分数（前件谓词个数与阈值紧度），冲突时取分数高者。

第二种改进是加权投票：每条规则给一个后件建议，按可信度加权平均。这种做法的好处是切换平滑，坏处是可能产生“谁都不像”的中间参数，反而破坏可解释性。工业上用得少，多见于学术研究。

第三种改进是规则优先级在线调整：记录每条规则触发后的短期效果（比如接下来 0.5 s 的 $|e|$ 是否下降），效果差的规则降权。这本质上是一种在线学习，要防止“最近一次偶然效果好”导致的漂移，通常加滑动平均与下限保护。

与其他方法的融合

专家控制最常见的融合对象是模糊控制与增益调度。

与模糊控制融合，是把硬阈值换成隶属函数，让分区边界平滑过渡。这样切换抖振问题从根上消失，代价是规则后件不再是一个确定的参数，而是若干参数的加权，稳定性分析更困难。第 67 章会详细展开。

与增益调度融合，是把工况层从“三档查表”升级为“连续插值”。若已知 $K(y)$ 的解析形式，直接令 $K_p(y) = K_p^0 / K(y)$ 就能精确拉平回路增益，不必分段。专家规则只在无法建模的部分（刹车、守卫）保留。这种“能建模的部分连续化、不能建模的部分规则化”的分工，是第 72 章混合架构的核心思想。

还有一种融合值得提：与迭代学习结合。若设定值轨迹重复，可以用第 69 章的 ILC 修正前馈，专家规则只负责反馈侧的鲁棒性。两者分工明确，ILC 管“重复性误差”，专家管“非重复性扰动与工况变化”。

学习结果的稳定性与复现性

学习出来的乘子能不能直接用，取决于它稳不稳。判断稳不稳有三个层次。

第一个层次是初值依赖。坐标下降从人工值出发，如果换成从全 1 出发，会落到同一个点吗？本章没有做这个实验，这是一处缺口。经验上，只要目标函数在乘子空间里是单峰的，初值影响就小；一旦存在“多组参数指标相近”的平坦区，不同初值会给出完全不同的解，此时应该把解报告成一个区间而不是一个点。

第二个层次是随机种子。随机搜索的曲线每次都不一样，若换一个种子最优值差出 5% 以上，说明 60 次评估的预算不足以支撑结论，必须加预算或换更稳的搜索。这一条在任何“学习调参”的论文里都应该报告，可惜常常被省略。

第三个层次是轨迹依赖，也是最危险的一层。本例的目标函数是“三段迁移轨迹的 IAE”，学出的 Z4 乘子明显偏向“快”，正是因为这三段都以接近段收尾。换一条以大扰动恢复为主的轨迹，学出的 Z4 大概率会偏向“稳”。所以正确的评估方式是分段留一：用三段中的两段学习、第三段测试，看测试段的指标是否也改善。若只有训练段改善，那就是过拟合，参数不能上线。

工程上的折中做法是：学习只用来“提建议”，最终参数仍由人确认。把学习结果的四项指标与人工值并列打印出来，人看到“IAE 降 5.6%、超调升 0.7 个百分点”，再判断这个交换是否值得。这样既借用了搜索的效率，又保留了人对指标的最终裁量权。

规则库压缩

规则写多了会想压缩，压缩有两条路，收益完全不同。

第一条是把“工况 × 分区”的表合并成一张二维查找表。本章的三层结构展开后是 $3 \times 4 = 12$ 组参数，每组三个乘子，共 36 个数。既然工况号与分区号都是离散的，完全可以预先算好 12 组最终参数，运行时只做两次索引。这样做的好处是运行开销从“逐条匹配”降为“两次查表”，在 1 ms 级的小型 PLC 上有意义；坏处是丢掉了规则的语义，现场人员看到的是一堆数字而不是“大误差时刹车”这样的句子，维护成本反而上升。

第二条是把近似冗余的规则合并。本章的 12 条里，M1 与 M2 的基参数只差 K_p（3.80 与 2.60），若把工况层从三档减成两档，低工况与中工况共用一组参数，规则数降到 10 条。代价是低工况的回路增益从 38 降到 26，穿越频率下移，响应变慢。这是一次典型的“用性能换简洁”，是否值得要看工况的停留时间：若低工况一年只出现几次，合并划算；若是主要工作区，绝对不能合并。

真正不能压缩的是守卫层。G1 到 G5 每一条都对应一种“会出事”的工况，它们的价值不在平均性能，而在极端情况下的兜底。删掉一条守卫，平均指标可能还略有改善（因为少了约束），但一旦碰上它守护的那个场景就会失控。所以压缩规则库时，守卫层应当被标记为“不可删”，这与软件工程里“不要删掉看似无用的边界检查”是同一个道理。

规则库的回归测试

规则库每改一次，都要回答“原来的场景还好吗”。人工看曲线在规则数少时可行，规则一多就必须自动化，做法与软件回归测试同构。

第一层是单元级测试：给推理机一组人工构造的 $(y, e, \dot{e}, u_{k-1})$，断言返回的工况号、分区号、参数三元组、守卫标志。这一层跑得极快，能抓住“改阈值时把判据的不等号改反了”这类错误。本章的 12 条规则至少应覆盖每个分区的中心点、两个边界点、以及守卫触发与不触发的各一对。

第二层是场景级测试：跑完整的迁移轨迹，断言四项指标落在区间内。区间不能设成精确值，否则浮点与环境差异会让测试天天报警；设成“IAE 不超过 2.8、最坏超调不超过 10%”，既能抓住回归，又不会误报。

第三层是边界级测试：专门覆盖极端输入——测量为常数、测量为满量程、控制量长期顶在限幅、设定值与实测值相差极大。这一层不检查性能指标，只检查“不崩溃、不产生越界输出、超时能退出”。

三层测试的维护成本不同。单元级最便宜，改规则时必须同步更新；场景级最贵，改动小的参数调整可以只跑场景而不更新断言；边界级几乎不用改，因为极端输入的定义不随规则变化。

实践中最容易省略的是第一层，理由是“跑整个仿真也就几秒钟”。但整仿真只能告诉你“最终指标变了”，不能告诉你“哪条规则的行为变了”。当一条规则的判据被误改、而整体指标恰好没超阈值时，只有单元测试能发现它。这也是本章把 `infer` 单独抽成函数、把规则表放在文件头部的原因之一：便于在测试里直接调用，不必启动整个仿真。

66.10 工程落地

规则从哪里来

规则获取是专家控制真正的成本所在，写代码反而是小事。三条来源各有优劣。

第一条是访谈。找最熟悉这套装置的操作员与工艺工程师，让他们描述“什么时候该怎么调”。访谈的难点在于人对自己经验的表述是跳跃的、省略的，需要追问“如果这时候偏差又变小了呢”“如果阀门已经到顶了呢”。一次有效访谈通常能挖出 5 到 10 条可用规则，但要花两三个小时。

第二条是历史数据。把过去一年的趋势数据拿出来，找出操作员手动干预的时刻，看干预前后的特征量有什么共同点。这种方法能发现人自己都没意识到的规律，缺点是数据质量参差，很多干预是出于工况调度而非控制需要。

第三条是故障复盘。每一次超调、联锁、停车都是规则的最好来源，因为后果明确、边界清楚。这类规则往往直接对应守卫层。

三条来源的产出要统一登记：规则编号、前件、后件、来源、提出人、验证状态。这张登记表就是知识库的原始形态。

知识库的版本管理

规则库是软件，必须按软件的方式管理。最低要求是三条。

每条规则有唯一编号，编号不随内容修改而变。修改时新增版本号，保留旧版本的全文。这样现场出现问题时，能查到“当时跑的是哪一版”。

每次变更要有变更单，写明改了什么、为什么改、在哪些场景验证过、谁批准。工业现场的事故调查往往追溯到某次“顺手改了一个阈值”。

发布要能回滚。规则库打包成一个配置文件，与控制器程序分离；回滚时只换配置，不动程序。本章的代码就是这么组织的：三层规则集中在文件头部，推理机与仿真循环分开。

与现场系统的接口

专家控制器的输出最终要落到 DCS 或 PLC 的功能块上。接口有三个容易出问题的地方。

第一是扫描周期。规则推理必须与 PID 运算在同一个周期内完成，不能跨周期。若 DCS 的功能块周期是 100 ms，而规则里用了“最近 50 拍”的窗口，那窗口实际是 5 s，与设计时的 0.5 s 差十倍。移植时所有时间参数都要按周期换算。

第二是数据质量位。现场测量会短时失效（变送器故障、通信中断），此时特征量是无效的。推理机必须先检查质量位，质量位不好时保持上一拍输出并冻结积分，而不是拿着无效值去匹配规则。

第三是无扰切换。手动转自动、专家转固定参数、主备控制器切换，都必须保证控制量连续。做法是切换瞬间把积分器预置成使输出等于当前手动值的状态，这一手在第 44 章已经讲过，专家控制器同样适用。

投运的五步走

规则库在仿真里跑通，距离现场可用还有很长一段。推荐五步。

第一步，离线仿真。用辨识出的模型跑遍所有预期工况，记录每次规则触发的合理性。这一步能发现大部分逻辑错误，成本最低。

第二步，硬件在环。把控制器程序烧进实际控制器，接信号发生器模拟测量，验证周期、定点运算、通信都正常。定点运算的截断误差常在这一步暴露。

第三步，影子模式。控制器在线运行但不接管输出，只把“如果我来控会输出多少”记录下来，与操作员的实际输出对比。这一步通常要跑两周以上，覆盖各种工况。影子模式最大的价值是能发现“规则在真实噪声下疯狂切换”这类仿真里看不到的问题。

第四步，闭环试投。先在低负荷、非关键时段投入，设定值小幅度变化，操作员全程监视。此时守卫规则的作用最大，因为它们就是为异常情况准备的。

第五步，长时间考核。至少覆盖一个完整的生产周期（比如一个月的批次切换），统计规则触发次数、干预次数、指标改善幅度。考核不达标就回到第三步。

现场维护

投运不是终点。规则库需要持续维护，原因是对象会变：换热器结垢、催化剂老化、阀门磨损，都会让原来的工况分界与基参数失配。

维护的第一手资料是触发日志。每条规则每次触发都记一笔（时间、特征值、后件），定期统计。若某条规则触发频率突然翻倍，往往意味着对象特性变了，或者某个传感器漂移了。

第二手资料是绩效指标。超调率、稳态误差、阀门动作次数，按月统计并与投运前对比。指标缓慢劣化是正常的，劣化速度突然加快则要找原因。

再整定的周期取决于对象漂移速度。本章这类机电对象通常一年一次；化工过程可能一个检修周期就要复核一次。再整定时不必推倒重来，通常只改基参数与工况分界。

失效模式与失效安全

规则库也会失效，而且失效方式比线性控制器多。按后果从轻到重排四类，每类都要预先设计应对。

第一类是规则误判。特征量正常，但组合恰好落进一条不该触发的规则。比如噪声让 $|\dot{e}|$ 瞬间超过 D_b，系统误判为“正在快速收缩”而提前刹车，表现为偶发的响应变慢。这类失效通常不危险，因为它只让参数偏移一点；应对办法是加滞环与死区，而不是加规则去纠偏。

第二类是特征失效。传感器断线、通信丢包、变送器漂移，导致 y 是常数或跳变。此时 e 可能是常数，$\dot{e}$ 可能为零，规则会判定为“稳态”并切到 Z4 全力积分，控制量慢慢爬到限幅。这类失效最隐蔽，也最危险。应对办法是数据质量位加变化率检查：若 y 在应当变化的时段完全不动，就判为特征失效，冻结积分并保持输出。

第三类是配置错误。工况分界写反、乘子漏了一位、参数表的单位与代码不一致。这类失效在仿真里就能抓出来，只要验证脚本覆盖了全部工况。真正的风险在“改了配置忘了重启”或“两套配置同时生效”，所以配置加载要做校验和，并记录加载时刻的版本号。

第四类是时序阻塞。推理函数因为异常输入进入死循环，或者共享数据被锁住，整个控制周期被拖长。应对办法是看门狗加超时退出，超时后强制使用上一次的有效输出，并在一个固定周期后尝试恢复。

这四类的共同应对原则叫“失效安全”：任何异常都应当让系统退到一个已知的、保守的状态，而不是继续按错误的规则运行。本章代码里对应的做法是：守卫层优先于修正层、限幅独立于规则、异常时退回固定参数。这三条在所有场景下都成立，不依赖任何一条具体规则的正确性，所以它们是可靠性的真正底线。

组织与责任

专家控制器的知识库不属于某个人，属于装置。这一点必须在组织上落实，否则写规则的人一调走，规则库就变成没人敢动的黑箱。

实践中的做法是设一个“控制性能管理员”角色，负责规则登记、变更审批、月度绩效统计。这个角色不需要会写代码，但要懂工艺、看得懂趋势图。

另一个容易被忽略的点是文档。每条规则旁边要写清楚“为什么这么写”，而不只是“写了什么”。本章的规则表里每条都附了一句中文说明，就是为了让后来者能判断这条规则在当前工况下是否还成立。

成本与收益的现实估计

一套中等规模装置（20 到 30 个控制回路）的专家控制项目，规则获取与验证通常占总工作量的六成以上，编码与调试占三成，剩下的是一成文档与培训。

收益则集中在少数几个“难调回路”上：大滞后、非线性、多工况的回路，专家控制能把超调压下一个量级；而对本来就好调的回路，投入产出比很低，不如继续用固定参数 PID。

这条经验很重要：专家控制不是“更高级所以更好”，它是“用在固定参数不够用的地方”。判断标准很朴素——如果一个回路用第 44 章的方法能在三个主要工况下都调到可接受，就不需要专家控制。

投运后指标改善的量化口径

投运报告里最常出现的毛病是“改善明显，操作员满意”，没有数字就没有说服力。可用的口径应当事先约定，并在投运前后用同一套算法计算。

第一组是跟踪指标。把设定值阶跃按幅度分档（小于 5%、5% 到 20%、大于 20%），每档统计超调率与调节时间的中位数与 90 分位数。只报平均值会掩盖大阶跃这一个最危险的档位。

第二组是抗扰指标。取可测的扰动变量（进料量、环境温度）发生明显变化的时段，统计扰动后偏差的最大偏离与恢复时间。这一组往往比跟踪指标更能体现专家规则的价值，因为守卫规则主要在抗扰时起作用。

第三组是执行器指标。控制量的总变差、日动作次数、行程分布。这一组是“成本侧”，改善跟踪指标若以三倍阀门动作次数为代价，总账未必划算。

第四组是可用率指标。控制器投入自动的时长占比、因规则异常退出自动的次数。专家规则越复杂，这一组越容易变差，必须单独盯。

四组指标要同时报，且要给出统计窗口与样本量。只报一组的投运报告，基本可以认为没有做过严格考核。

影子模式为什么值得多花两周

五步走里最容易被压缩的是第三步，但它恰恰是性价比最高的一步。

离线仿真用的是辨识模型，模型再好也滤掉了真实噪声与真实采样抖动。硬件在环验证了信号通路，但输入是信号发生器给的干净三角波。只有影子模式跑在真实的测量、真实的扰动、真实的操作习惯上，才能看到规则在“带噪声的真实特征量”下如何反应。

影子模式要重点看三样东西。一是分区号的跳动频率，若一个稳态时段内分区号切了几十次，说明阈值被噪声反复穿越，滞环不够。二是守卫规则的触发情况，G1 到 G5 在正常工况下应当极少触发，若每天触发上百次，说明阈值定得太紧，规则实际上变成了常态逻辑，失去了兜底的意义。三是“如果我来控”与操作员实际输出的差异分布，差异集中在哪些时段、哪些幅度，逐段回看趋势图，往往能发现访谈里问不出来的隐性经验。

影子模式还有一个附带好处：它是给操作员建立信任的过程。看着屏幕上的建议值与自己的操作大体一致，操作员才敢在第四步放手让控制器接管。技术上的准备度和组织上的接受度，都要靠这两周积累。

66.11 优缺点与适用边界

优点

第一，能把无法建模的知识用起来。本章的对象至少还有解析式，真实装置上大量经验是“某个阀在 30% 开度以下会卡涩”“升温速率超过 2 K/min 就会局部过热”这类无法写成微分方程的知识。规则是它们唯一自然的表达形式。

第二，可读、可审查、可追责。每条规则都能翻译成一句人话，工艺工程师可以直接质疑“这条阈值为什么是 0.5 而不是 0.4”。神经网络做不到这一点，模糊规则勉强能，但规则一多就难以逐条讨论。对安全相关回路，可审查性是硬指标。

第三，改动局部化。要改刹车力度，只动 Z2 的乘子；要改高工况的响应，只动工况层那一行。改动的影响范围可以事先画出来，这在调试期非常省事。

第四，能直接复用 PID 的工程积累。抗饱和、微分滤波、无扰切换、限幅保护，第 44 章的所有手段原样可用。专家层只是给 PID 加了一个“参数调度器”，不改变底层结构，运维人员的学习成本低。

缺点

第一，知识获取成本高且不可扩展。本章 12 条规则对应一个二阶对象，若对象有五个工况、每个工况四类修正、再加六条守卫，规则数很容易过百。过百之后，冲突检查、覆盖验证、变更管理都变成体力活。

第二，规则之间会互相干扰。表 66-6 显示，加一条 Z4 规则反而让最大超调从 10.51% 升到 10.83%，因为 Z4 的积分作用与 Z1 的积分分离在同一段轨迹上交替出现。这种组合效应在规则少的时候还能靠仿真发现，规则多了就只能靠经验。

第三，没有严格的稳定性保证。第 66.6 节的三条判据是必要条件，不是充分条件。对安全等级要求高的场合（比如飞行控制、核电），这通常是不可接受的，必须改用有 Lyapunov 证明的方法，或者把专家层严格限制在“参数调度”范围内并用增益调度的理论分析。

第四，可解释性有幻觉成分。规则本身好读，但“这 12 条规则合起来为什么能稳住”并不好读。现场出问题时，能查到“当时触发了 Z2 与 G1”，却不一定能说清这两个动作叠加后为什么没出更大的问题。

第五，维护负担随时间增长。对象漂移、工艺改造、人员更替，都会让规则库逐渐与实际脱节。若不设专人维护，三五年后规则库就变成“不敢删、不敢改”的遗留资产。

反例：限幅收紧时专家输了

表 66-1 里有一行不好看的数据：把控制量限幅从 ± 1.2 收紧到 ± 0.8，专家全程 IAE 3.044，线性基线 2.893，专家反而差 5.2%。

原因有两个，都在于规则与限幅的耦合。

一是 G5 的软着陆比例是相对 $u_{\max}$ 定义的。限幅收紧后，0.45 倍的绝对值从 0.54 降到 0.36，下降段的负向能力被砍掉三分之一，第四段的跟踪明显变慢，IAE 上升。

二是低工况的 $K_p = 3.80$ 在限幅 0.8 下更容易饱和。基线 $K_p = 2.60$ 的饱和程度轻，反而少受积分饱和的拖累。

这个反例说明：专家规则的参数不是无量纲的，它们与执行器能力绑定。执行器换了、限幅改了，规则必须重新标定。工程上把这类参数集中放在配置文件的“执行器”段，就是为了改一处、全库生效。

适用边界的判据

综合本章的分析，可以用五条判据判断一个回路是否适合专家控制。

其一，对象在工作范围内有明显的非线性或参数漂移，固定参数 PID 在至少一个工况下无法接受。若三个工况都能调好，不必上专家。

其二，工况可以用少量可测量划分。如果工况只能靠“季节”“批次”这类外部信息判断，说明特征选得不对，先解决测量问题。

其三，执行器与传感器有明确硬限制，且这些限制会频繁触发。守卫规则的价值与触发频率成正比，从不饱和的系统不需要 G1，从不振荡的系统不需要 G2。

其四，有可持续的维护力量。至少要有一个人能看懂趋势图、能审批规则变更。

其五，安全等级允许“经验规则”参与闭环。若规范要求形式化验证，专家层只能做参数调度，且调度表要由增益调度的理论保证。

不适合的场合

三类场合应明确避开专家控制。

第一类是快回路。采样周期在微秒级的电流环、伺服环，规则推理的开销与确定性都成问题，用固定参数或增益调度更合适。

第二类是强耦合多变量对象。单回路专家规则无法处理“改这个会影响那个”的耦合，这类问题应交第 58、80 章的 MPC。

第三类是数据充足但机理不清的对象。此时人写不出规则，硬写只能写出肤浅的规则，不如用第 68 章的神经网络从数据里学。

与相邻章节的分工

本章讲的专家控制，核心是“人写规则 + 分层调度”。若把规则的前件软化，就是第 67 章的模糊控制；若把规则的后件换成网络输出，就是第 68 章的神经网络控制；若把规则用于重复轨迹的前馈修正，就是第 69 章的迭代学习；若把规则用于策略选择而非参数调整，就接近第 70 章的强化学习。

这五章的共同问题是“如何在模型不精确时仍能控制好”，区别只在知识的形式与来源。读者不必把它们看成互斥的方案，第 72 章会给出把几者拼起来的架构，并讨论拼起来之后可解释性还剩多少。

一张选型决策的顺序表

选型问题听起来复杂，实际可以拆成一条只含“是/否”的顺序判断链。顺序很重要，因为先问的问题排除力最强。

第一问：固定参数 PID 能不能在所有主要工况下都达到指标？能，就到此为止。这一问能筛掉工业现场八成以上的回路，先问它是为了不浪费时间。

第二问：非线性或漂移是否可以用一个可测变量解释？能，就优先用增益调度——它的理论最成熟，参数最少，也最容易审查。只有当“调度变量找不到”或“调度之外还有大量经验性约束”时，才继续往下。

第三问：约束与经验能否写成条件语句？能，就是本章的专家控制。这一问的实质是“知识是否已经被人说清楚”。若工艺人员能一口气讲出十条“什么时候该怎么调”，说明知识已经显式化，用规则最经济。

第四问：知识能不能说清楚、但边界模糊？比如“误差大的时候要多刹车，但多大算大说不准”。这时硬阈值会带来抖振，改用隶属函数就是第 67 章的模糊控制。模糊控制并不比专家控制“高级”，它只是把“边界不清”这件事建模进去了。

第五问：知识根本说不出来，只有大量运行数据？这时才轮到第 68 章的神经网络。要注意，神经网络能拟合的是“数据里出现过的行为”，对数据里没有的异常工况没有任何保证，所以工程上通常还要在外面套一层专家规则做守卫。

这条链的一个推论是：混合架构往往不是设计出来的，而是被问题逼出来的。当对象既有可建模的部分、又有说不清的部分、还有硬约束时，任何单一方法都会在某一段露怯，只好拼接。

与模糊方法在同一对象上的预期差异

第 67 章会在同一个非线性对象上做模糊控制，这里先给出预期，便于读者对照。

在稳态附近，两者的差别最小。分区的硬边界只影响边界附近的那一小段，而稳态时特征量离边界很远，规则几乎不切换，专家控制不会抖。

在大阶跃的初始段，模糊控制的优势开始显现。专家控制在 $|e|$ 跨过 E_b 的瞬间参数跳变，控制量出现一个台阶；模糊控制按隶属度加权，参数是连续过渡的，控制量更平滑。这个差异在示波器上看很明显，但对超调的影响未必大，因为超调主要由刹车时机的“平均”决定，而不是由过渡是否平滑决定。

在噪声较大的场合，模糊控制的优势更大。硬阈值加滞环本质上是“用迟滞换稳定”，滞环宽度定得小会抖、定得大会迟钝；隶属函数的重叠区天然提供了平滑，不必在抖与迟钝之间二选一。

代价则在可解释性与可验证性上。专家控制的每条规则都能单独拿出来讨论，模糊控制一旦规则多于二十条，规则表就变成一张难以逐格审查的矩阵；而且模糊系统的稳定性分析至今没有工程上通用的判据，只能靠仿真与保守设计。

所以两者的取舍很清楚：约束明确、需要审查、规则数量可控，选专家；边界模糊、噪声大、需要平滑过渡，选模糊。真正难的是两类特征同时存在的对象，那时通常的做法是外层用专家做守卫、内层用模糊做调节，这也是第 72 章混合架构的一种具体形态。

这套规则搬到别的对象上会怎样

读者最关心的问题大概是：本章这 12 条规则，换个对象还能不能用？答案要分层说，因为三层规则的可移植性完全不同。

守卫层的可移植性最高。G1 抗饱和、G2 振荡抑制、G3 死区保持、G4 换向复位、G5 单向限速，这五条几乎不依赖对象的具体形式，只依赖“执行器有行程、传感器有噪声、控制量有方向性”这三个普遍事实。搬到一阶惯性、二阶欠阻尼、甚至带滞环的阀门上，逻辑都不用改，只需重新标定阈值。

修正层的可移植性中等。乘子本身无量纲，可以原样搬；但四个分区的“意图”依赖对象的动态特性。本章对象是双积分型，接近段容易冲过头，所以 Z2 的刹车乘子给到 1.60；若对象是一阶惯性、没有超调倾向，这个刹车就纯属拖慢，应该降到 1.0 附近。所以乘子要搬，但搬过去之后必须重新看超调，按“是否需要刹车”调整。

工况层的可移植性最低。它直接依赖“增益随工作点怎么变”，换对象就要重新测量、重新分档。如果新对象的增益几乎不变，工况层干脆可以删掉，只留修正层与守卫层——那时专家控制器退化成一个“带守卫的可变增益 PID”，仍然有用，但已经不是本章讨论的那个完整形态。

最后一条经验是：移植时不要一次全搬，先只搬守卫层。守卫层的风险最低、收益最直接，跑通之后再逐步加入修正层与工况层。这样每一步都有可对照的指标，出了问题也知道是哪一层引入的。

66.12 Python 仿真代码

两个文件与运行方式

本章的全部数字与插图由两个脚本生成，都放在 `code/` 目录下。

`ch066_sim.py` 负责仿真与指标，运行后打印七组结果：逐工况整定、迁移曲线对比、触发统计、规则规模、冲突与覆盖、规则自学习、工程细节。`ch066_figs.py` 只负责出图，导入前者的对象与规则，不重复实现算法。

```
"D:\Anaconda3\envs\py311\python.exe" code/ch066_sim.py
"D:\Anaconda3\envs\py311\python.exe" code/ch066_figs.py
```

两个脚本都只依赖 `numpy`、`matplotlib` 与 `control`，不依赖任何深度学习框架。全部仿真是纯 Python 循环，16 s 轨迹在普通笔记本上不到 0.1 s，自学习部分 60 次评估约 3 s。

为什么不用 simlib 的线性接口

第 44 章的闭环一律调用 `simlib.simulate_discrete`，本章没有。原因是规则切换让闭环成为混杂系统，参数逐拍变化，线性仿真接口表达不了。本章改用文件内的逐步循环，把对象、控制器、延迟、限幅都写在同一个循环里，便于在任意位置插入规则。

代价是要自己保证数值细节正确。为此做了三件事：对象用 5 次 Euler 子步积分，避免大步长失真；延迟用长度为 5 的 FIFO 缓冲；控制量限幅与积分更新顺序严格按第 66.5 节的规定。

规则库的代码组织

三层规则全部集中在文件头部，推理机与仿真循环分开。这样做的好处是改规则不用碰算法，也便于把规则导出成配置文件。

```
MODE_BANDS = ((0.0, 1.3, "低工况"), (1.3, 2.3, "中工况"), (2.3, 9.9, "高工况"))
MODE_GAIN = {
    "低工况": (3.80, 0.05, 0.90),
    "中工况": (2.60, 0.05, 0.60),
    "高工况": (2.00, 0.05, 0.60),
}
ZONE_FACTOR = {
    1: ((1.00, 0.00, 0.80), "大误差且在扩大：积分分离，全力推进"),
    2: ((0.90, 0.00, 1.60), "大误差但在收缩：提前刹车"),
    3: ((0.85, 1.00, 1.30), "小误差快变化：精细阻尼"),
    4: ((1.00, 1.50, 1.00), "小误差慢变化：积分消差"),
}
```

每条修正规则的后件是一个三元组，顺序固定为 K_p, K_i, K_d 的乘子。中文说明与数值放在同一个字典里，是为了让导出配置时两者不脱节。

推理机的实现

推理机是本章的核心，只有约 30 行。它做三件事：定工况、定分区、按优先级叠加守卫。

```
def infer(y, e, e_prev, de, u_prev, osc, zone_prev=0, use_rules=True):
    mode = mode_of(y)
    base = MODE_GAIN[mode]
    z = zone_of(e, de, zone_prev)
    if not use_rules:
        return mode, 0, base, False, False, ""
    f = ZONE_FACTOR[z][0]
    kp, ki, kd = (base[0] * f[0], base[1] * f[1], base[2] * f[2])
    hold, reset, guard = False, False, ""
    s = e * de
    if abs(u_prev) >= HOLD_SAT * UMAX and s > 0:      # G1 抗饱和
        hold, guard = True, "G1"
    if osc >= OSC_N:                                   # G2 振荡抑制
        kp, kd, guard = 0.80 * kp, 1.20 * kd, (guard + "G2")
    if abs(e) >= E_BIG and e * e_prev < 0:             # G4 换向积分复位
        reset, guard = True, (guard + "G4")
    if abs(e) < DEAD_E and abs(de) < DEAD_DE:          # G3 死区保持
        kp = ki = kd = 0.0
        guard = (guard + "G3")
    return mode, z, (kp, ki, kd), hold, reset, guard
```

`use_rules=False` 时直接返回基参数，这是线性基线的入口。它走的仍是同一个 `simulate` 循环，同一套限幅与微分口径，因此表 66-3 的对比是公平的——两者唯一的差别就是这一行分支。

闭环循环里的顺序

`simulate` 的循环顺序与第 66.5 节一致。下面只摘出控制量计算这一段。

```
dy_raw = (yk - y_prev) / TS          # 微分作用在测量值上
dy_f += alpha * (dy_raw - dy_f)
y_prev = yk
u_d = float(np.clip(-kd * dy_f, -UD_CLAMP * UMAX, UD_CLAMP * UMAX))
u_raw = kp * ek + ki * ei + u_d
uk = float(np.clip(u_raw, sat[0], sat[1]))

# G5 下行软着陆：偏差大幅为负时限制负向控制量，并冻结积分
if use_rules and ek < -E_BIG:
    lo = max(sat[0], -DESC_LIMIT * UMAX)
    if uk < lo:
        uk, hold = lo, True

if use_rules:
    if not hold:
        ei += TS * ek
else:
    if not (abs(u_raw) > sat[1] and np.sign(u_raw) == np.sign(ek)):
        ei += TS * ek
```

注意 `hold` 在这里被复用了：G1 与 G5 都通过置位 `hold` 冻结积分，积分更新只认这一个标志。这样写减少了分支，但也要求读者清楚 `hold` 的语义是“本拍不要积分”，而不是“控制量保持”。

指标统计的实现细节

`segment_metrics` 做了两件容易被忽略的事。

一是负阶跃翻正。`simlib.step_metrics` 假定输出从 0 上升，对 3 到 1 的下降段会算出 nan。处理办法是把输出与目标同时取反，让下降段变成“从 0 上升到 2”，指标定义与上升段完全一致。

二是每段单独取初值。统计第 i 段时，以第 $i-1$ 段的目标值作为本段初值，而不是用全轨迹的起点。否则第二段的“上升时间”会从上一次稳态算起，失去意义。

自学习函数的实现细节

`learn_rules` 只有四十来行，但有三处写法值得说明，因为它们直接决定了结果的可比性。

第一处是评估函数 `_learn_eval` 采用“改—跑—还原”的模式。它先把试算的乘子写回全局的 `ZONE_FACTOR` 与 `MODE_GAIN`，跑完 `simulate` 之后在 `finally` 里还原原始字典。这样做的好处是不必给 `simulate` 加一个“参数”形参，推理机始终读全局表，规则库仍是唯一事实来源；代价是这段代码不可重入，也不能多线程并行评估。若要并行调参，必须先把参数结构从全局变量改成显式传参。

第二处是评估的轨迹被刻意缩短。`_learn_eval` 里用的是 `T=12.0`、三个设定值点的轨迹，而不是完整的四段 16 s 轨迹。理由有两个：一是把评估成本压到 0.06 s 量级，60 次评估才三秒多；二是让目标函数只反映“上升与第一次接近”这两个最需要权衡的环节，避免下降段那一大段稳态误差把乘子的差异淹没。代价是学到的参数对下降段没有保证，这一点第 66.9 节已经用完整轨迹复核过。

第三处是随机搜索与坐标下降共享同一个评估函数与同一个预算计数。两者都从人工值出发，都调用 `_learn_eval`，都用同样的 `budget` 上限，所以图 66-9 的两条曲线是严格可比的。若让随机搜索在连续空间里采样、坐标下降在网格上搜索，比较就不公平了——前者的搜索空间大得多，输是必然的，结论也就没有意义。

还有一个小细节：坐标下降每轮只接受严格改善的移动，指标持平的移动不接受。这样可以避免在平坦区里来回漂移，但也意味着它可能停在“任何一个坐标单独动都变差”的局部点上，需要多起点才能跳出。本章只用单起点，所以第 66.9 节提醒读者把结果看成“一个可用的改进”而非“全局最优”。

两个统计函数的用途

`conflict_stats` 与 `zone_coverage` 都不参与控制，只服务于设计期的体检，但它们回答的是两个不同的问题。

`conflict_stats` 在 $(e, \dot{e})$ 平面上打 121×121 的网格，对每个格点判断有几条规则同时成立（判据加上相对容差 `overlap` 模拟“边界带”）。返回的是冲突样本占比。它回答的是“规则边界设计得够不够干净”。这个指标随容差增大而单调上升，图 66-8 左图那条曲线就是它的扫描结果。若容差取 15% 时冲突率已经很高，说明分区之间靠得太近，运行时必然频繁切换，应该先改阈值而不是加滞环。

`zone_coverage` 用的是同一张网格，但只统计每条规则独占的格点比例，不考虑容差。它回答的是“规则在操作域上的分布是否合理”——有没有哪条规则几乎没有地盘（形同虚设），有没有哪条规则的地盘大得离谱（可能承担了过多职责）。本章的结果是 Z1 27.31%、Z2 23.93%、Z3 43.92%、Z4 4.84%。

这张几何分布与运行时的触发占比几乎正好相反：几何上 Z4 只占 4.84%，运行中却占 62.69%。这不是矛盾，而是说明轨迹长期贴着 $(0, 0)$ 附近活动——那一小块面积很小，但被反复经过。反过来 Z1 的几何面积接近三成，运行中只有 4.75%，因为轨迹每次穿越大误差区只花零点几秒。两个数字放在一起，恰好回答了“规则的地盘”与“规则的工作量”是两个不同问题：地盘大而工作量小的规则，说明它的阈值定得宽，可能需要收紧；地盘小却承担大部分工作量的规则，则是最值得仔细调参的那一条。

两个函数都只遍历网格、不跑闭环，因此可以在参数调整后随手调用，作为“改完先看结构”的快速自检。这也是把它们与仿真主循环分开写的原因：结构问题和性能问题应当分两步诊断，混在一起看容易误判。

出图脚本的约定

`ch066_figs.py` 用 `plot_styles` 与 `block_styles` 两个语义助手，不写裸的 matplotlib 参数。十张图各对应一个函数，末尾统一调用 `save`，同时导出 SVG 与 PDF。

框图（图 66-1、66-2）用 `block_styles` 与 `flow_styles`，数据图（图 66-3 到 66-10）用 `plot_styles`。图内中文一律纯文本，公式一律纯数学，两者不写进同一个字符串——这是全书约定，违反时 matplotlib 会把中文交给数学字体渲染，出现方框。

运行结果的核对

脚本跑完后，读者可以用下面几个数字核对是否复现成功。

逐工况整定的结果是 $K_p = 3.80/2.60/2.00$，评分 0.165/0.156/0.194。迁移曲线上专家全程 IAE 2.625、最大超调 8.22%，线性基线 IAE 2.997、最大超调 50.23%。自学习部分坐标下降终值 1.340，随机搜索 1.397。

若某个数字对不上，先检查三处：`numpy` 的版本是否一致（随机数生成器行为稳定，一般不是问题）、`control` 是否缺失（只有整定部分用到）、Python 是否为 3.11（3.10 以前的 `np.sign` 对 0 的返回值不同，会影响振荡计数）。

复现与扩展

想换对象，只需改 `gain_at` 与 `plant_lin` 两个函数，保持局部线性化与非线性模型在工作点处一致即可。想加规则，在 `infer` 里加分支，并在 `zone_coverage` 的网格统计里体现，否则覆盖率检查会漏掉新规则。

想验证第 66.9 节的自学习，把 `learn_rules` 的 `budget` 改成 200，观察坐标下降与随机搜索的曲线是否交叉。本章只跑 60 次，是因为再大的预算对结论没有影响，反而让脚本变慢。

两个脚本加起来不到六百行，其中规则表与推理机占不到八十行。这个比例本身就是本章想说明的事：专家控制的难点不在代码，而在把经验整理成可执行的判断，以及把这些判断的适用条件写清楚。代码只是最后一步的落地。建议读者先只读规则表与 `infer`，确认每条规则都能翻译成一句人话，再看仿真循环——顺序反过来，很容易被实现细节淹没，反而看不清规则本身在做什么。

读者若只带走一句话，希望是这句：专家控制的价值不在于“更聪明”，而在于把“什么时候该收手”这件事变成可审查、可复现、可回退的明文约定。第 67 章会把这条约定里的硬边界换成隶属函数，看看平滑能换来什么、又要付出什么代价。

第67章 模糊控制

第 66 章把专家的经验写成了“如果……那么……”的硬规则，代价是每条规则都有一条明确的边界：误差一旦跨过 0.50，参数立刻跳变。为了让边界不抖，我们在边界上加了滞环；滞环又带来另一个问题——阈值附近的行为是“要么这样、要么那样”，中间没有过渡。本章换一种写法：不再问“误差是否大于 0.50”，而是问“误差有多接近大”。这个“有多接近”就是隶属度，用隶属度写规则、用加权的方式合成结论，就是模糊控制。

67.1 动机与直观理解

从“是或否”到“有多像”

先看一个日常场景。调淋浴水温时，人不会去测“水温偏差 3.2 度”，也不会执行“如果偏差大于 3 度则阀门开大 15%”。人做的是模糊判断：水有点凉、凉得厉害、快好了；然后凭手感把阀门转一点点、再转一点点。整个过程中没有明确的阈值，也没有离散的档位，动作随感觉连续变化。

模糊控制就是把这种判断方式形式化。它保留规则的语言形式（“如果偏差大且正在变大，那么大幅加大输出”），但把“大”从一个确定的集合换成一个带隶属度的模糊集合：偏差 0.8 时对“大”的隶属度是 0.9，偏差 0.4 时是 0.35。同一条规则在两种情况下都会触发，只是“触发强度”不同，最终输出按强度加权。

这个替换带来三个直接后果。第一，参数随特征连续变化，控制量不会因为跨过阈值而出现台阶，第 66 章那套滞环、死区、抖振治理手段大部分可以不要。第二，规则之间不再需要严格的冲突消解：两条规则同时成立不是矛盾，而是“各占一部分权重”。第三，边界的精确位置不再敏感——把“大”的中心从 0.50 挪到 0.45，输出只是稍有变化，不会像硬阈值那样引起行为突变。

代价也很明确。硬规则的行为可以逐条列举、逐条验证；模糊规则的行为是一条连续曲面，只能抽样检查。硬规则的参数有直观的物理含义（阈值 0.50 就是 0.50）；模糊规则的参数是一组尺度因子，物理含义要绕一层才能说清。第 66 章用“可解释”换“平滑”，本章用“平滑”换“可解释”，两者的取舍在 67.11 节展开。

本章算例与结果预告

为了让两章可比，本章仍用第 66 章那个非线性对象

$$\ddot{y} = -2\dot{y} + K(y)u, \qquad K(y) = 4 + 6y \tag{67-1}$$

工作点在 1 到 3 之间迁移时，静态增益从 10 变到 22，跨度 2.2 倍。设定值轨迹仍是四段：$0 \to 1 \to 2 \to 3 \to 1$，每段 4 s，纯延迟 0.05 s，控制量限幅 ± 1.2。

控制器采用“工况层 + 模糊修正层 + 守卫层”的结构。工况层与第 66 章完全相同，负责补偿工作点引起的增益变化；模糊修正层替换掉第 66 章的四个分区规则，用 49 条规则连续地调整 K_p, K_i, K_d；守卫层保留抗饱和与下行软着陆两条。这样两章的唯一差别就是“修正层”的实现方式：硬分区还是模糊插值。

结果先摆出来。同一轨迹上，模糊 PID 的全程 IAE 是 2.698，固定参数 PID 是 2.997，低 10.0%；最大超调 1.09% 对 50.23%，差了两个数量级；控制量总变差 27.9 对 36.6，低 23.9%。与第 66 章的专家控制相比，IAE 2.698 对 2.625 略高 2.8%，但最大超调 1.09% 对 8.22% 低得多，控制量总变差 27.9 对 43.2 也小得多。

这组数字说明的并不是“模糊比专家好”，而是两者把代价花在了不同地方：专家用硬规则把大偏差时的行为压得很死，响应快但切换生硬；模糊用连续插值把过渡做得很软，超调小、动作少，但起步偏慢——第一段的上升时

间是 0.84 s，专家是 0.47 s。第 67.11 节会把这张表补全，并给出选型判据。

一个容易混淆的地方

初学者常把“模糊”理解成“不精确”或“随便估”。这是误解。模糊控制里的每一个环节都是确定的：隶属函数是确定的三角形，规则是确定的 49 条，推理是确定的乘积运算，去模糊是确定的加权平均。给定输入，输出完全可复现，没有任何随机性。

真正“模糊”的只是知识的形式——人用“大、中、小”这样的词描述经验，模糊系统提供了把这些词映射成数值的一套运算。所以模糊控制的可重复性不比 PID 差，它与 PID 的差别在于“参数随状态变”，而不是“结果不确定”。

另一个常见误解是“模糊控制不需要模型”。它确实不需要传递函数，但需要知道三个东西：输入的物理范围（用来定归一化尺度）、输出该怎么动（用来写规则）、以及对象大概多快（用来定采样周期）。这三样本质上就是模型的粗略版本。说“不需要精确模型”是准确的，说“不需要任何模型知识”则不对。

三个问题决定要不要用

在往下读之前，先用三个问题判断这条回路值不值得上模糊。这不是套话，三个问题分别对应本章三处最硬的结论。

第一个问题：对象增益随工况变几倍？ 本章对象从 10 变到 22，2.2 倍，模糊层的收益明显；若只变 1.3 倍，鲁棒整定就够了，模糊层带来的收益抵不过维护成本。这个问题的答案决定“要不要”。

第二个问题：超调有没有硬约束？ 本章第四段线性版超调 50.23%，压到 1.02% 是模糊层最大的功劳；若工艺允许 20% 超调，这笔收益就打对折。这个问题的答案决定“值不值”。

第三个问题：纯延迟占对象时间常数的多少？ 本章 0.05 s 对 0.5 s，占十分之一，裕度只剩 14.7°，延迟一加倍就失稳。若占比超过十分之一，模糊层不是正确的工具，该用预测控制（第 47 章）。这个问题的答案决定“够不够”。

三个问题都指向“值得”，再进入 67.2 节的原理；只要有一个明确否定，就应当考虑别的方案。这三条会在 67.11 节展开成完整的适用边界，这里先摆出来，是为了让读者带着判断读，而不是读完才发现不适用。

本章的阅读路线

如果只关心“能不能用”，读 67.1 的结果预告、67.6 的性能表、67.11 的适用边界即可。如果要动手实现，67.2 到 67.5 是必须的：模糊化、推理、去模糊化三步的细节决定了实现是否正确，而离散化与量化误差决定了它能不能跑在真实控制器上。67.7 到 67.9 讨论参数与调参，其中 67.9 的两个实验（规则条数、整定预算）回答“要花多少成本”这个问题。67.10 与 67.12 是工程与代码。

先看一张总图。图 67-1 是本章采用的闭环结构：模糊层不直接产生控制量，只产生 PID 参数的修正量。图的细节留到 67.3 节第 1 步再讲，这里只需要记住“模糊层是旁挂在 PID 上的，不是替代 PID”。

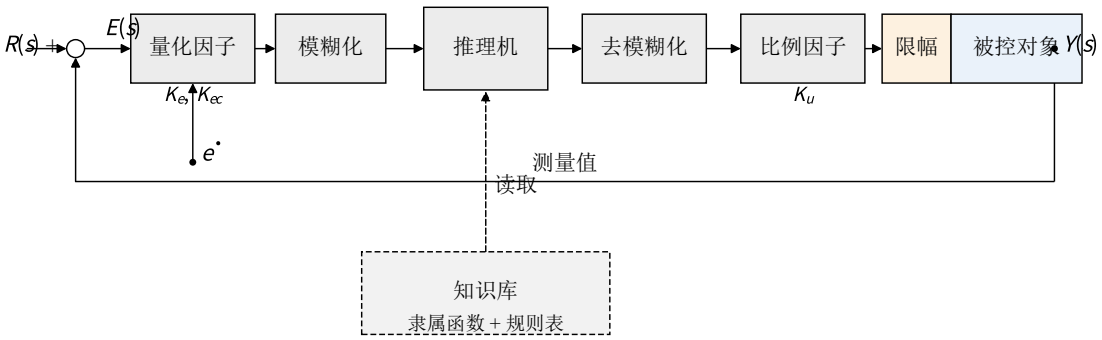


图 67-1 模糊自整定 PID 的闭环结构

67.2 数学原理

模糊集合与隶属函数

普通集合用特征函数描述：元素属于集合记 1，不属于记 0。模糊集合把这个二值函数换成取值在 $[0,1]$ 的隶属函数

$$A=\{(x,\mu_A(x))\mid x\in X,\ \mu_A(x)\in[0,1]\} \tag{67-2}$$

$\mu_A(x)$ 读作“x 属于 A 的程度”。它不表示概率，不表示测量误差，只表示语言标签的匹配程度。误差 0.45 对“大”的隶属度是 0.6，这句话的含义是“按设计者的划分，0.45 有六成像大”，与 0.45 这个数准不准无关。

隶属函数的形状有很多选择：三角形、梯形、高斯形、S 形。本章选三角形，原因是它在离散实现里只涉及两次除法和两次比较，且“相邻集合在中点相交于 0.5”这一性质可以直接写成解析式。设论域归一化为 $[-1,1]$，等距划分 n 个集合，第 i 个的中心与半宽分别是

$$c_i=-1+\frac{2i}{n-1},\qquad h=\frac{2}{n-1},\qquad \mu_i(x)=\max\!\left(0,\ 1-\frac{|x-c_i|}{h}\right) \tag{67-3}$$

把 h 取成相邻中心间距，是一个刻意的设计。它保证相邻两个三角形在中点处各取 0.5，且所有三角形在任意点的隶属度之和恒为 1：

$$\sum_{i=0}^{n-1}\mu_i(x)=1,\qquad \forall x\in[-1,1] \tag{67-4}$$

这个性质叫单位分解。它不是数学上的巧合，而是后面“加权平均等于凸组合”的前提。若把 h 取小了，三角形之间会出现空隙，某些点上所有隶属度同时为零，输出就会突跳；若取大了，重叠过多，分辨率下降。图 67-2 画出了 $n=7$ 时的七个三角形，并在 $x=0.11$ 处标出两个非零隶属度：$\mu_{ZE}=0.67$、$\mu_{PS}=0.33$，和为 1。

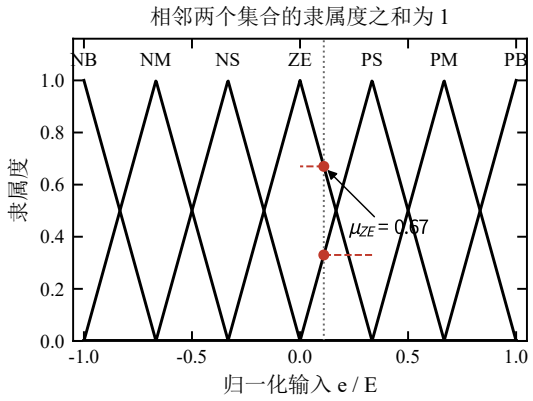


图 67-2 三角形隶属函数与一次模糊化

论域、量化与截断

式 (67-3) 的定义域是 $[-1,1]$，而实际偏差可以任意大。两者之间靠一次截断衔接：归一化后的输入先被压到 $[-1,1]$，再送进隶属函数。这个动作在代码里只有一行 `np.clip`，但它对闭环行为有实质影响。

截断意味着**超出 E 的偏差不再被区分**。本章 $E = 1.60$，第四段阶跃 1.0 归一化后是 0.625，还没到边界；若某次阶跃是 2.0，归一化 1.25 会被压到 1.0，多出来的 0.25 不产生任何额外修正。此时模糊层的输出是一个常数（全部落在 PB 上），等效于“比例增益固定为 $K_p^0(1+S_p)$”。

这不是缺陷，而是有意的取舍。大偏差时该做的动作本来就只有一个——全力推，区分 1.2 与 2.0 对控制量没有帮助，反而会让规则表在大偏差区堆满几乎重复的格子。截断还带来一个好处：修正量有界。式 (67-8) 的凸包性质加上截断，使 ΔK_p 在任何输入下都被限制在 $[-1,1]$，运行中不会出现“参数突然翻倍”这种事。

代价是**大偏差区的自适应能力消失**。如果某台设备偶尔出现远超设计工况的扰动，模糊层在那段时间里帮不上忙，系统退化成固定参数。这类扰动要靠守卫层（限幅、积分冻结）或者外层的工况判别来处理，不能指望模糊层。本章对象在四段轨迹里最大偏差恰好是 1.0，没有触发截断；若读者把这套参数搬到偏差更大的回路上，第一件要检查的就是截断比例。

还有一个容易被忽略的细节：截断后的输入落在 ± 1 上时，隶属度向量是 $(1,0,\dots,0)$ 或 $(0,\dots,0,1)$，权重和为 1 的性质仍然成立，但相邻集合的隶属度为 0，插值退化成“只取端点值”。也就是说，论域边界附近的输出是**不连续可导**的——曲面在那里有一个折角。这个折角在正常工作区之外，不影响本章的稳定性分析，但如果回路长期贴着边界跑，折角就会变成抖动的来源。

规则、前件与后件

一条模糊规则写作

$$R_{ij}:\quad \text{IF } e \text{ is } A_i \text{ AND } \dot{e} \text{ is } B_j \ \text{ THEN } \Delta K_p \text{ is } c_{ij}^{p},\ \Delta K_i \text{ is } c_{ij}^{i},\ \Delta K_d \text{ is } c_{ij}^{d} \tag{67-5}$$

这里采用 Takagi–Sugeno 型的后件：结论不是一个模糊集合，而是一个确定的数 c_{ij}（本章取归一化值 -1 到 1）。这样做的理由很实际：Mamdani 型后件需要再做一次重心法去模糊，计算量大且结果对离散化步长敏感；Sugeno 型后件只需加权平均，且在单位分解的前提下，加权平均就是凸组合，便于分析。

规则的前件强度用乘积 t-范数计算：

$$w_{ij} = \mu_{A_i}(e)\,\mu_{B_j}(\dot{e}), \qquad \sum_{i}\sum_{j} w_{ij} = 1 \tag{67-6}$$

最后那个等式直接来自式 (67-4) 在两个轴上的独立成立。它是本章所有分析的基础：权重之和为 1，意味着输出一定是后件的凸组合，绝不会超出规则表里出现过的极值。

$n = 7$ 时规则总数是 49 条。图 67-3 把其中两张表画成了热图：左图是比例修正，右图是微分修正。颜色深浅表示修正量的大小，格子里的字母是语言变量缩写。可以看到两张表的结构完全不同——比例表沿“同号/反号”分块，微分表沿“接近/远离”分块。这不是随意安排的，67.3 节会解释每一块背后的意图。

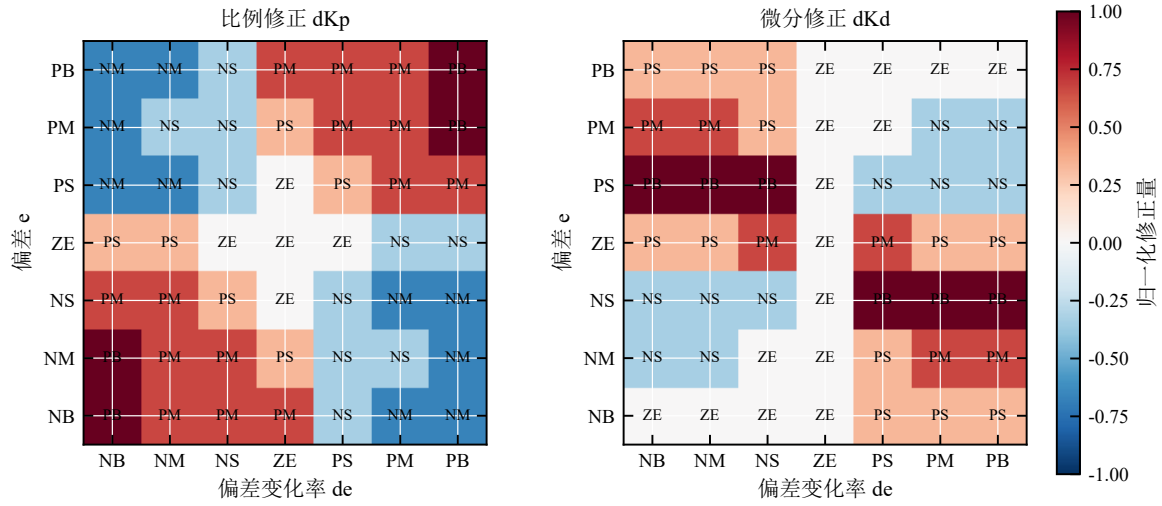


图 67-3 比例与微分修正规则表

推理与去模糊化

给定一对输入 $(e, \dot{e})$，推理分三步：模糊化得到两个隶属度向量，按式 (67-6) 算出 49 个权重，再加权平均得到三个修正量：

$$\Delta K_p = \sum_i \sum_j w_{ij} c^p_{ij}, \qquad \Delta K_i = \sum_i \sum_j w_{ij} c^i_{ij}, \qquad \Delta K_d = \sum_i \sum_j w_{ij} c^d_{ij} \tag{67-7}$$

因为 $\sum w_{ij} = 1$，式 (67-7) 不需要再除以权重和。这一点在实现时很重要：很多教科书给出的公式是“加权平均除以权重和”，那是针对非单位分解的隶属函数；本章的三角划分已经满足单位分解，除法是多余的，而且会引入一次不必要的舍入误差。

从几何上看，式 (67-7) 说明输出落在 49 个后件的凸包内：

$$\Delta K_p \in \left[\min_{ij} c^p_{ij},\ \max_{ij} c^p_{ij} \right] \tag{67-8}$$

这条性质在 67.6 节会被用来做稳定性检验。它还有一个实用推论：**规则表里的极值就是参数的极值**。想让比例增益最多只放大 55%，就把后件的极值定在 ± 1，尺度因子取 0.55，不必再去检查运行中的实际取值。

图 67-5 画了一次具体的推理。输入 $e = 0.18$、$\dot{e} = -0.42$（图为了看清交叠，用的是 5 集合划分；本章正文计算一律 7 集合），只有少数几条规则的权重非零，其中最大的两条权重分别对应“误差略正、变化率为负”与“误差接近零、变化率为负”。两条规则的后件一正一负，加权平均落在一个中间值上。这正是模糊控制平滑的来源：不是选了某一条规则，而是所有相关规则各让一步。

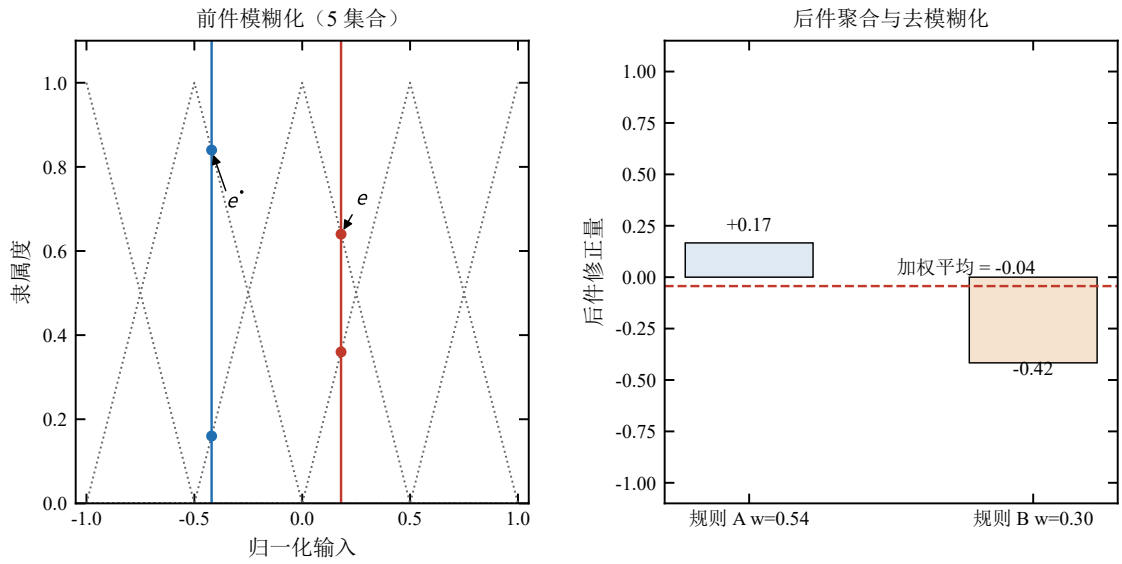


图 67-5 一次完整的推理与去模糊化

Mamdani 型与 Sugeno 型：模糊到底体现在哪里

式 (67-5) 的结论是一个数，而不是一个模糊集合。这个选择值得单独说清楚，因为它把“模糊控制”分成了两个流派。

Mamdani 型的结论也是模糊的，比如“比例修正应该是大”。要得到控制量，必须先把所有被激活规则的结论集合按权重剪裁（取小），再合成一个总集合（取大），最后求这个集合的重心。重心法需要把输出论域离散成一串点，逐点计算隶属度，再按面积加权求中心。计算量大约是 Sugeno 型的几十倍，而且结果依赖离散步长：步长 0.01 与步长 0.05 算出来的重心能差几个百分点，表现为控制量的小台阶。

Sugeno 型把结论换成函数，零阶就是常数（本章的做法），一阶是输入的线性函数。这样一来，去模糊化退化成加权平均，没有离散化，没有重心，也没有步长参数。代价是“结论不再模糊”——规则的后件从“比例修正应该是大”变成了“比例修正是 0.8”。

这笔交换是否划算，取决于用途。若模糊控制器的价值在于“用语言表达专家经验、并让操作员看懂”，Mamdani 更贴近这个意图，因为它的中间结果也是语言量。若价值在于“平滑地调整参数、并且能分析”，Sugeno 明显更好：它把模糊性全部压缩到前件，后件是一张数值表，于是整个控制器可以写成式 (67-10) 那样的解析式，也能做凸组合与频域分析。

本章选 Sugeno，是因为后面五节都要用到解析性：67.4 的等效频率响应、67.6 的顶点裕度、67.7 的灵敏度扫描，都建立在“输出是后件的加权平均”这个基础上。若用 Mamdani，这三节只能靠仿真打点，无法给出解析结论。换句话说，**Sugeno 是用一点“模糊的味道”换来了可分析性**。

需要提醒的是，Sugeno 型常被批评“其实不是模糊控制，只是插值”。这个批评有一半道理：它的后件确实不模糊，增益曲面也确实等价于一张双线性插值表（见本节末的第二种视角）。但前件的模糊化不是装饰——它决定了插值基函数的形状，也决定了“哪几条规则同时起作用”。把这一层去掉、直接用网格插值，得到的是同一张曲面，但失去了“规则”这个可读的中间层。工程上要不要这一层，取决于维护者需不需要读规则表，而不是取决于数学。

从修正量到增益

得到三个归一化修正量后，按尺度因子叠加到工况基参数上：

$$K_p = K_p^0\big(1 + S_p \Delta K_p\big), \quad K_i = K_i^0\big(1 + S_i \Delta K_i\big), \quad K_d = K_d^0\big(1 + S_d \Delta K_d\big) \tag{67-9}$$

本章取 $S_p = 0.55$、$S_i = 0.70$、$S_d = 0.35$。乘性叠加而不是加性叠加，是为了让规则可以跨工况复用：低工况的基参数是 $(3.80, 0.05, 0.90)$，高工况是 $(2.00, 0.05, 0.60)$，同一张规则表对两者都给出“按比例增减”的修正，不必为每个工况单独写表。这一点与第 66 章的乘子化后件是同一种思路。

把式 (67-9) 代回，等效比例增益是 $(e, \dot{e})$ 的函数

$$K_p(e, \dot{e}) = K_p^0\Big[1 + S_p \sum_{ij} w_{ij}(e, \dot{e})\, c_{ij}^{p}\Big] \tag{67-10}$$

图 67-4 把中工况的这张曲面画了出来。曲面的取值被限制在 1.647 到 4.030 之间，恰好是 $2.60 \times (1 \pm 0.55)$。曲面在四个角上最陡，在原点附近最平——角上对应“大偏差且快速变化”，两条以上规则同时以高强度触发；原点附近只有一个集合的隶属度为 1，相邻集合为 0，曲面退化成常数。

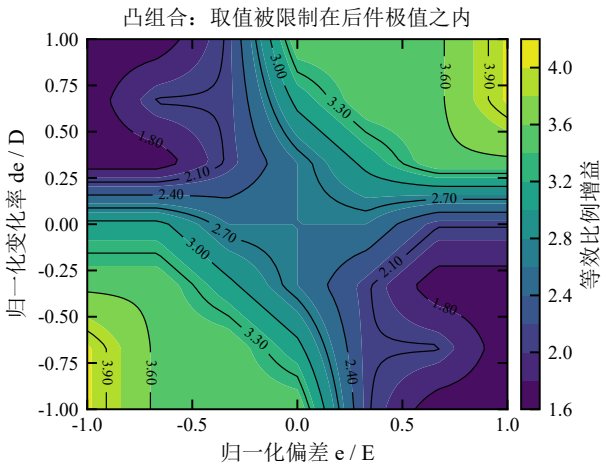


图 67-4 等效比例增益曲面

万能逼近定理说明了什么

模糊控制常被引用的一条理论是万能逼近：在紧集上，任何连续函数都可以用足够多规则的 Sugeno 模糊系统以任意精度逼近。这条定理的证明思路与分段线性插值是同一件事——规则越密，插值节点越密，误差越小。

但它在工程上的指导意义有限，原因有三。第一，它说的是“存在一组规则”，没有说“怎么找到这组规则”，因此不解决设计问题。第二，它要求规则覆盖整个输入空间，而工程上规则数受验证成本限制，49 条已经是本章愿意承担的上限（见 67.9 节的规则条数实验）。第三，它只保证逼近精度，不保证闭环稳定：把对象模型逼近得再准，若控制器本身的增益选得过大，闭环照样振荡。

所以本章对这条定理的定位是“存在性背书”，而不是设计工具。真正决定成败的仍是 67.3 节的尺度选择与 67.7 节的参数含义。

同一张控制器，三种读法

式 (67-10) 可以按三种完全不同的方式理解，每种理解对应一套检验手段。把三套都摆出来，是为了在后面的分析里各取所需。

第一种是插值读法。 把规则表看成一张 7×7 的网格数据，把三角形隶属函数看成分片线性的基函数，那么式 (67-7) 就是双线性插值。这个读法的好处是直观：模糊层不过是在查表，查不到的点靠相邻四点插值。它对应的

检验手段是**连续性检查**——网格边上的一阶导数是否连续、边界处是否出现折角。前面提到的截断折角，用这个视角一眼就能看出来。

第二种是增益调度读法。 把 $K_p(e,\dot{e})$ 看成一个随状态变化的增益，模糊层就是一个调度器。这个读法与第 42 章的增益调度是同一族方法，区别只在调度变量：第 42 章用工况量（转速、负载），本章用误差与误差变化率。它对应的检验手段是**慢变假设**——调度变量变化得足够慢时，可以把每个瞬间冻结成线性系统来分析；变化太快，冻结分析就失效。

第三种是非线性反馈读法。 把控制器看成 $u = k(e,\dot{e})\,e$ 的形式，即一个增益随状态变化的比例环节。这个读法对应的检验手段是**描述函数**——给正弦输入，看输出的基波分量，得到等效增益与等效相位。它只能处理正弦、只能看基波，但能回答“这个非线性环节在闭环里相当于多大的增益、带来多少相位”，这正是 67.4 节要用的。

三种读法给出的数字应当互相印证。本章的实现里，插值视角给出曲面（图 67-4），调度视角给出 147 个顶点（67.6 节），反馈视角给出等效增益 2.60 到 2.89（67.4 节）。三个数字必须能对上：曲面在中工况原点的值应当等于基参数，顶点集合应当覆盖曲面的极值，描述函数在零幅值极限下应当回到基参数。任何一处对不上，说明实现里有 bug，而不是理论有问题。

乘积还是取小：这个选择会留下折角

式 (67-6) 用乘积做 t-范数，而多数教科书用取小。两种写法都满足“合取”的直觉——两条规则同时成立时，权重取两者中较弱的一方——但在数值上不是一回事。

三角形隶属函数是分段线性的。取小意味着 $w_{ij} = \min(\mu_{A_i}, \mu_{B_j})$，在一个格子内部，谁小谁说了算；穿过 $\mu_{A_i} = \mu_{B_j}$ 那条对角线时，主导者换人，权重的导数发生跳变。于是加权平均出来的增益曲面上会出现一道折线，折线两侧的斜率不同，像一张纸被折过。乘积没有这道折线：$\mu_{A_i}\mu_{B_j}$ 在两个方向上同时光滑，格子内部是双二次曲面。

乘积也不是处处光滑。三角形在顶点处本身有折点，隶属函数的导数在那里不连续，乘积的导数也跟着不连续。区别在于折点的位置：取小的折线在格子内部，穿过整个输入空间；乘积的折点只落在隶属函数的顶点连线上，即 e 或 $\dot{e}$ 正好等于某个集合中心的地方。后者与网格对齐，前者不对齐。

这个差别对本章的分析有直接影响。顶点法只用端点值，不要求可微，两种写法都能用。描述函数虽然也不要求可微，但折线会让高次谐波明显变大，基波近似变差；67.4 节报出的等效相位是在乘积写法下测的，若换成取小，那组数字只能当参考。局部线性化更敏感：折线上没有唯一切线，冻结系数模型在折线附近失去意义。

实际影响落在过渡带的宽度上，不在极值上。两种写法的后件极值相同，因此 67.6 节的顶点集合不变；变化的是权重在格子内如何分配，表现为过渡段更“软”或更“硬”。若因为要与既有设备保持兼容而必须用取小，那么规则表与尺度可以照搬，但凡是涉及相位与等效增益的结论都要重测。

67.3 设计步骤

第 1 步：确定输入输出与回路结构

设计从画结构图开始。图 67-1 是本章采用的闭环：设定值与测量值相减得到偏差，偏差与测量值变化率分别乘以量化因子 K_e、K_{ec} 归一化到 $[-1,1]$，送进模糊化与推理机，输出的三个修正量乘以比例因子 S_p, S_i, S_d 后叠加到工况基参数上，最后仍由位置式 PID 计算控制量。知识库（隶属函数与规则表）旁挂在推理机上，运行时不改写。

这个结构与常见的“纯模糊控制器”不同：纯模糊控制器的输出直接就是控制量，而本章的输出是 PID 参数的修正量。选择后者的理由有三。第一，PID 的积分项保证了稳态无差，纯模糊控制器要做到无差必须额外加积分环

节，反而更复杂。第二，本章要与第 66 章、固定参数 PID 做同口径比较，保持“同一个 PID、只差参数来源”最干净。第三，工程上改造存量回路时，工程师更愿意接受“在 PID 外面加一层”，而不是“把 PID 换掉”。

图 67-1 里有两个容易忽略的细节。一是第二个输入取的是测量值变化率 $-\dot{y}$，不是误差变化率 $\dot{e}$。设定值阶跃时 $\dot{e}$ 会出现一个幅值极大的尖峰（理论上趋于无穷），它会穿过模糊层，让所有规则瞬间偏向“快速接近”这一侧，造成一次不必要的参数跳变。用 $-\dot{y}$ 就没有这个问题，这与第 44 章“微分作用于测量值”是同一条经验。二是量化因子画在模糊化之前而不是之后，强调它是“工程单位到论域”的换算，不是模糊运算的一部分。

第 2 步：定归一化尺度

E 与 D 的取值决定了“多大的偏差算大”。取法有一个简单规则：E 取该回路在正常运行时**最常出现的最大偏差**，而不是量程。本章的四段轨迹最大阶跃是 1.0（工作点从 2 跳到 3），所以取 $E = 1.60$，让最大偏差的归一化值落在 0.625，处于 PS/PM 之间而不是顶到 PB。这样做的道理是：若最大偏差刚好顶到论域边界，所有“大偏差”都用同一个隶属度 1 表示，模糊层在大偏差区就退化成了常数，失去了分辨能力。

D 的取法要绕一层。它是变化率的尺度，量纲是“单位每秒”。本章对象的输出变化率在大阶跃起步时约 0.6 到 0.8 单位每秒（控制量饱和、增益 10 到 22），取 $D = 1.40$ 使归一化值落在 0.4 到 0.6，与 E 的量级匹配。若 D 取得过小，变化率一动就饱和，规则表右半部分形同虚设；取得过大，变化率永远在 ZE 附近，微分修正不起作用。

这两个尺度不是独立的。它们共同决定了规则平面上“轨迹走哪条路径”。E 大、D 小，轨迹会贴着横轴走，多数时间待在 ZE 列；E 小、D 大，轨迹会贴着纵轴走。本章的 $(1.60, 1.40)$ 是让轨迹在阶跃起步时斜穿 PS–PM 区，在接近时回到 ZE 附近，正好把规则表的主要区域都用上。67.7 节的单参数扫描会给出这五个参数各自的灵敏度，结论可能与直觉不同：E 与 S_p 是决定性的，D 与 S_i 在本回路上几乎无关。

第 3 步：划分集合与写规则表

集合数 n 决定分辨率。本章取 7，对应 NB、NM、NS、ZE、PS、PM、PB 七个语言值，这是工业界最常用的配置。$n = 5$ 时没有 NM/PM，过渡更粗；$n = 9$ 时多出两档，规则数从 49 涨到 81，验证成本上升。67.9 节的实验会给出三种分辨率的实测差别：9 条规则的 IAE 反而最小（2.476），但最大超调 9.02%、控制量总变差 37.1；49 条规则的 IAE 是 2.698，但超调 1.09%、总变差 27.9。也就是说，规则少的时候系统更像“开关控制”，平均误差小但动作粗暴。

写规则表有两条原则，覆盖了 49 格里的绝大多数。

第一条是**看 e 与 $\dot{e}$ 的符号关系**。同号意味着偏差在扩大（正偏差还在变大，或负偏差还在变负），此时应当加大比例、减小阻尼，尽快把偏差推回去；反号意味着偏差在收缩，此时应当减小比例、加大阻尼，防止冲过头。图 67-3 左图的对角分块就是这条原则的体现：左上与右下两块是正修正，右上与左下两块是负修正。

第二条是**看 $|e|$ 的量级**。$|e|$ 大时积分要弱（积分分离），否则限幅期间积分会积爆；$|e|$ 小时积分要强，用来消除稳态误差。图 67-3 的积分表（本章未画出）就是按这条原则填的：外侧两行以负修正为主，中间三行以正修正为主。

微分表则按“接近速度”填。正在快速接近时（e 与 $\dot{e}$ 反号且 $|\dot{e}|$ 大）给最强的正修正，正在远离时给负修正。这一条与第 66 章 Z2 的“提前刹车”是同一个意图，只是这里用连续的方式表达：不是“进入刹车区就把 K_d 乘 1.6”，而是“越接近、速度越大，K_d 加得越多”。

写完之后要做一次“人话检查”：每一格都要能翻译成一句工程师听得懂的话。翻不出来的格子，说明设计者自己也没想清楚，应当删掉或改成保守值。49 格逐格检查大约要半小时，但这半小时能挡掉后面几天的调试。

边界格要按“停留时间”填

49 格里各格的权重并不相同。轨迹在规则平面上的驻留时间决定了某一格的修正量对指标的实际影响，而边界格（$|e|$ 或 $|\dot{e}|$ 接近 1 的格子）恰恰是驻留最久的地方——大偏差要持续一段时间才被消掉，饱和段更是能停留几百毫秒。

所以边界格要按“最坏情况”填，而不是按“典型情况”填。本章的三张表在这件事上的处理并不相同，值得对照看。

比例表的四个角给的是 ± 1 或 $\pm 2/3$，没有给 ± 1：右上角（e 正、$\dot{e}$ 负、快速接近）给的是 $-2/3$ 而不是 -1。原因是这个角正好对应“快要到位”的时刻，若把比例压到基值的一半以下，剩余的小偏差就只能靠积分慢慢消，第四段的调节时间会明显变长。而左上角（e 负、$\dot{e}$ 正，正在远离）给的是 $+1$，因为那里需要全力推。

微分表在边界处的取值普遍偏保守：靠近 $|\dot{e}| = 1$ 的两列大量给 ZE 或小正修正。这不是疏忽，而是防止“微分项顶到限幅”——67.4 节会看到，第四段接近目标时微分项确实顶到了 ± 1.2 的限幅，若边界格再给强正修正，控制量会在过零点附近来回冲。

积分表在边界处给负修正，这就是积分分离。$|e|$ 大时把 K_i 压到基值的三成以下，等偏差收小再放开。本章 K_i 的基值只有 0.05，压下去之后几乎不起作用，但这一步不能省：若没有它，第四段下行时积分会在大偏差期间持续累积，退出限速后一次性释放，形成二次超调。

一个实用的检查办法是把 49 格按“四段轨迹是否会访问”分类。相当一部分格子平时不生效，一旦出现异常工况才会被激活——这些格子最容易埋雷，因为它们从未在测试中被验证过。本章没有采用“留空”的做法，49 格全部写满，因此不存在“未定义区域”；代价是未访问的格子无法通过指标验证，只能靠人话检查兜底。若改用稀疏表，建议给空格统一填保守值（比例 0、积分 0、微分小正），宁可让它们在异常时“少修正”，也不要“乱修正”。

第 4 步：选推理与去模糊化

本章用乘积推理加加权平均，理由已经在 67.2 节说明：计算量小，且单位分解保证了凸组合性质。这里补充两点工程上的考虑。

一是**不要用重心法**。重心法要把每个后件模糊集合离散成若干个点再求加权重心，49 条规则乘上每条 20 个采样点就是近千次乘加，在 10 ms 的控制周期里不算贵，但它引入了“离散化步长”这个额外参数，步长选得粗会让输出呈阶梯状，选得细又增加计算量。Sugeno 型后件没有这个问题。

二是**权重要用乘积而不是取小**。取小（min）运算是 Mamdani 推理的经典做法，它的好处是语义直观，坏处是输出对输入不光滑——取小运算在两条隶属函数相等处有折点，折点会让控制量出现细小的棱角。乘积运算是处处可微的，与本章要论证的“平滑”一致。代价是乘积会低估权重（两个 0.5 相乘只剩 0.25），但式 (67-6) 的权重和仍为 1，低估只影响各条规则的相对大小，不影响凸组合性质。

第 5 步：叠加到 PID 并联调

前三步做完，模糊层已经能输出三个修正量，但闭环还没跑过。联调的顺序建议是“先线性、后模糊”：先把工况基参数调好（这三组参数直接取自第 66 章的整定结果），确认固定参数下系统在每个工况都稳定；再打开模糊层，从 $S_p = S_i = S_d = 0$ 开始逐步加大，观察指标怎么变。

这个顺序的好处是每一步都可归因。若一上来就带着模糊层调基参数，指标不好时无法判断是基参数错了还是规则表错了。本章的三个尺度因子最终取 $(0.55, 0.70, 0.35)$，就是在这个顺序下试出来的：先固定 $S_i = S_d = 0$ 调 S_p，再调 S_i 看稳态，最后调 S_d 压超调。

联调时还要把守卫层装上。模糊层再平滑，也挡不住积分在限幅期间积爆，也挡不住下阶跃时控制量冲到底。本章保留了两条守卫：饱和时冻结积分、大幅负偏差时限速并冻结积分。

两条守卫的作用方向并不一致，这一点要靠实测才能看清。关掉“饱和冻结积分”，四段的 IAE 都略微变差（第四段从 1.101 涨到 1.155），但第四段的超调反而从 1.02% 降到 0.58%；关掉“下行限速”，第四段超调从 1.02% 涨到 13.64%。也就是说，一个是用超调换 IAE，另一个是纯粹压超调，取舍方向相反。67.6 节会把这两个数字放在一起讨论。这条结论与第 66 章并不完全相同，原因在于第 66 章的超调主要来自参数切换，而本章的超调主要来自下行段的增益失配。

67.4 连续情形

模糊层在连续域里究竟做了什么

把模糊层与 PID 写在一起，闭环的控制律可以写成

$$u(t) = K_p(e,\dot e)\,e(t) + K_i(e,\dot e)\int_0^t e(\tau)\,d\tau - K_d(e,\dot e)\,\frac{dy}{dt}, \tag{67-11}$$

其中三个增益不再是常数，而是偏差与变化率的连续函数。这个形式和线性 PID 只差“系数会变”，但它带来的性质变化远不止“系数会变”这么简单：闭环从一个线性时不变系统变成了一个**准线性变参数系统**（quasi-LPV）——参数随状态变化，但在任意一个瞬间，闭环仍是一个线性系统。

这个视角很有用。它意味着分析时可以“冻结”参数，把某一点 $(e,\dot e)$ 上对应的那组 (K_p,K_i,K_d) 当作常数，算这一点的相位裕度、带宽、灵敏度；然后再让工作点移动，看这些指标怎么漂。第 66 章的专家控制做的是同样的事，只是它的参数是**分段常数**，切换瞬间不连续；本章的参数是**连续插值**，没有切换瞬间。两章的区别全部集中在这一点上。

写成 quasi-LPV 形式还有一个好处：它把“模糊”这个听起来玄的词，还原成了工程师熟悉的东西——**增益调度**（gain scheduling）。区别只在于调度的插值函数不是查表加线性插值，而是由隶属函数与规则表复合出来的双线性插值。第 42 章的增益调度需要设计者手工给出几个工作点的参数并指定插值方式；本章把这些工作从“手工指定”变成“由规则表隐式给出”。代价是可解释性下降：调度表上每一格的权重还能说清，但复合出来的曲面已经不容易用一句话概括。

两个工作点的算例

抽象地说“参数会变”没有说服力，算两个点更直接。取中工况基参数 $K_p^0=2.60$、$K_d^0=0.60$，尺度 $E=1.60$、$D=1.40$、$S_p=0.55$、$S_d=0.35$，考虑同一偏差 $e=0.9$ 下的两种运动状态。

第一种是**偏差在扩大**：$e=0.9$ 且 $\dot e=+0.5$。归一化后 $e/E=0.56$，落在 PS 与 PM 之间（$\mu_{PS}=0.31,\mu_{PM}=0.69$）；$\dot e/D=0.36$，也落在 PS 与 PM 之间。此时推理出的比例修正量约为 $+0.36$，$K_p=2.60\times(1+0.55\times0.36)=3.11$，比基值高 20%；微分修正量为 0，K_d 保持 0.60。这是“全力推进”的配置。

第二种是**偏差在收缩**：$e=0.9$ 但 $\dot e=-0.5$。归一化的偏差分量不变，变化率分量落到 NS 与 NM 之间。比例修正量变成约 -0.34，$K_p=2.60\times(1-0.19)=2.11$，比基值低 19%；微分修正量变成 $+0.54$，$K_d=0.60\times1.19=0.71$。这是“提前刹车”的配置。

两个点的偏差完全相同，比例增益却相差 47%，微分增益相差 18%。这个差距就是模糊层在一个阶跃过程中持续做的事情：偏差从 1.0 掉到 0 的路上，K_p 一路从 3.1 收到 2.1，K_d 一路从 0.6 加到 0.7。线性 PID 做不到这一点，它只能选一个折中的常数——选大了超调，选小了慢。

这也解释了 67.2 节式 67-10 里那个看起来很朴素的乘性叠加，为什么能覆盖从 1.647 到 4.030 的整个增益范围：$S_p = 0.55$ 配上 $\Delta K_p \in [-1, 1]$，理论上限就是 $\pm 55\%$，实测曲面正好顶到这两个边界。曲面没有超出边界，是单位分解的直接后果——凸组合不会外推。

等效频率响应：模糊层其实有一点相位超前

上面的算例说的是“某一瞬间的增益”。要判断它对回路稳定性的影响，还得看它在不同频率下的行为。办法是描述函数：给模糊层的比例通道喂正弦 $e = A\sin\omega t$，取输出 $u = K_p(e, \dot e)e$ 的基波分量，看它相对输入的幅值比与相位差。

实测三组幅值、三个频率，结果如表 67-1。

幅值 A	频率 ω	等效增益	等效相位
0.4	2	2.605	$+3.4^\circ$
0.4	6	2.615	$+6.2^\circ$
0.4	12	2.617	$+6.5^\circ$
0.8	2	2.732	$+9.5^\circ$
0.8	6	2.732	$+11.1^\circ$
0.8	12	2.730	$+11.3^\circ$
1.2	2	2.893	$+11.4^\circ$
1.2	6	2.879	$+12.9^\circ$
1.2	12	2.875	$+13.0^\circ$

表 67-1 有三件事值得注意。

第一，小信号极限回到基参数。 $A = 0.4$ 时等效增益 2.605，与 $K_p^0 = 2.60$ 只差 0.2%。这是一次自洽性检验：幅值趋于零时，ΔK_p 在原点附近近乎为零（曲面在原点是平的），描述函数必须退化成常数增益。若这一项对不上，说明曲面在原点有偏置，也就是规则表填错了。

第二，幅值越大，等效增益越高，但远低于凸包上界。 $A = 1.2$ 时 2.89，只比基值高 11%；而式 (67-8) 的凸包允许到 4.03。差距来自“平均”：正弦扫过一个周期，正修正与负修正各占一半，加权平均后互相抵消了大半。这提醒我们，顶点分析是**逐点最坏**，描述函数是**周期平均**，两者回答的是不同问题，不能拿 4.03 去吓唬人，也不能拿 2.89 去证明安全。

第三，也是最反直觉的一条：相位是超前的，3° 到 13°。 一个静态的、无记忆的非线性环节，居然在基波上表现出相位超前。机制不难解释：K_p 在“e 与 $\dot e$ 同号”（偏差正在扩大）时被调大，而这一区间恰好分布在 e 的过零点附近；K_p 在“反号”（偏差在收缩）时被调小，这一区间分布在 e 的峰值附近。把过零点附近抬高、把峰值附近压低，等效于给波形加了一点微分，而微分就是相位超前。

这条性质有实际意义：它说明模糊层不会像滤波器那样吃掉相位裕度，反而补一点。这也解释了为什么 67.6 节按“冻结顶点”算出的 45° 最小裕度，在闭环实测中并没有被额外的相位滞后进一步侵蚀——第 67.5 节要扣的那 36° 全部来自纯延迟，不来自模糊层本身。

描述函数也有它算不出的东西：高次谐波、跨格折痕引起的分岔、以及极限环。表 67-1 只保留基波，等于假设“其余谐波被对象滤掉了”——在本章对象（二阶、带宽约 2 rad/s）上这个假设基本成立，因为规则表折痕产生的高

次谐波主要在 10 rad/s 以上，被对象衰减了 20 dB 以上。若换一个宽带对象，这个假设就不成立，描述函数的结论要打折。

凸组合给出的稳定性线索

式 67-8 的加权平均有一个直接推论：任一时刻的参数向量 $\theta = (K_p, K_i, K_d)$ 都落在 49 个后件向量构成的凸包内。把 49 个顶点逐个代入闭环，得到 49 个“顶点系统”，再乘上三个工况就是 147 个。

这 147 个顶点系统的相位裕度实测为：最小 45.3°，中位 54.4°，最大 66.7°，**没有一个低于 30°**。这个结果比预期好，原因不难找：顶点上的参数是极端组合（三个修正量同时取 $+1$ 或同时取 -1），而实际轨迹只会用到其中一部分；即便在最坏顶点上，K_p 也只有基值的 1.55 倍，而基参数本身是按裕度 50° 以上整定的。

这组数字是在连续模型上算的，**没有算进 0.05 s 的纯延迟**。把延迟补进去，中工况基参数下的裕度会从 50.9° 掉到 14.7°（67.5 节给出这次折算），所以“45° 以上”只说明规则表本身没有把增益推到危险区，不代表实际回路有 45° 的余量。

必须说清这个检验的地位：它**只是必要条件，不是充分条件**。理由有两条。第一，参数随时间变化，闭环是变参数系统，“每个冻结时刻都稳定”不蕴含“整体稳定”——参数变化率足够快时，即便每一点都稳定，系统也可能失稳（这一点在第 42 章增益调度里已经讨论过）。第二，凸组合的稳定性论证通常需要一个公共 Lyapunov 函数；本章没有构造，只是逐点验证。所以这个检验的用途是**排除明显不合格的设计**：若某个顶点裕度只有 10°，那实际运行中一旦轨迹经过该顶点附近，振荡几乎必然发生。反过来，147 个顶点全部在 45° 以上，只能说明“没有明显雷区”，不能说明“一定不会失稳”。真正的稳定结论仍要靠仿真遍历工况。

慢变假设什么时候失效

把每个瞬间冻结成线性系统来分析，前提是参数在一个闭环时间常数内几乎不变。这句话可以量化：设参数的相对变化率为 $\rho = |\dot{K}_p|/K_p$，闭环时间常数约 $1/\omega_c$，那么冻结分析成立的条件是

$$\rho / \omega_c \ll 1. \tag{67-12}$$

本章第三段，K_p 在约 0.9 s 内从 3.1 收到 2.1，$\rho \approx 1.11/2.60 \approx 0.43\ \mathrm{s}^{-1}$，而 $1/\omega_c \approx 0.079$ s，乘积约 0.034，比 1 小一个半数量级。这就是为什么前面那些“冻结在某个工作点”的算例能与仿真对上：参数确实变得足够慢。

反过来看，这个判据也给出了危险的边界。若把 S_p 从 0.55 提到 2.0，K_p 在同样的 0.9 s 内可以翻两倍，ρ 涨到 $2.2\ \mathrm{s}^{-1}$，乘积变成 0.17，仍然小于 1，但已经不能算“远小于”。若再把 E 缩小一半，误差在更早的阶段就进入饱和区，参数可能在 0.1 s 内翻完，$\rho \approx 20\ \mathrm{s}^{-1}$，乘积 1.6——冻结分析彻底失效。此时顶点裕度全绿也没有意义，因为轨迹在参数空间里跑得比闭环动态还快，系统行为由“参数走过的路径”决定，而不是由“路径上每一点的性质”决定。

这条判据对调参有直接指导：**放大 S 因子之前，先算一下 ρ/ω_c**。它比“试试看会不会振”快得多，也比凭经验设一个上限更可靠。本章把三个 S 定在 0.35 到 0.70 之间，除了保守，也是为了让这个乘积留在 0.05 以下。

平滑压住了超调，但没有压住延迟

图 67-6 是第三段（工作点从 2 跳到 3）的阶跃响应，两条曲线共用同一个对象、同一个抗饱和、同一套守卫规则，差别只在参数来源。

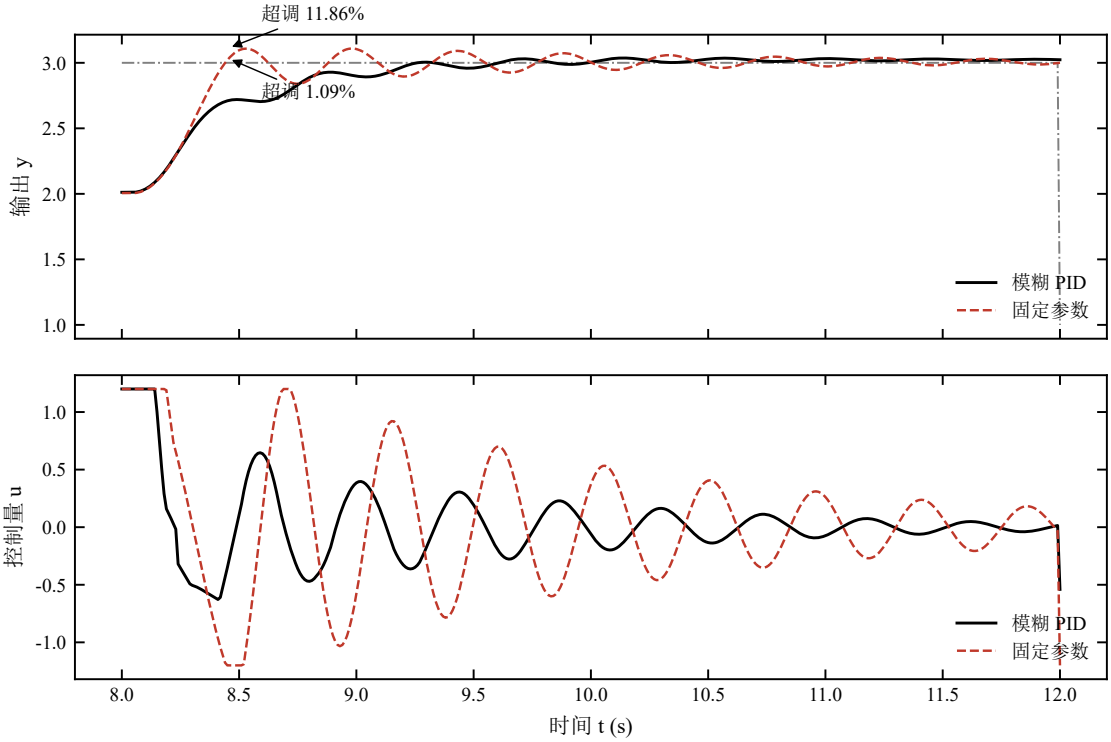


图 67-6 第三段阶跃：模糊自整定与固定参数对比

模糊自整定的最大超调是 1.09%，固定参数是 11.86%，相差十倍。从上面的算例可以看出这个差距的来源：固定参数的 $K_p = 2.60$ 是按中工况整定的，在 2→3 这段实际处于高工况边缘（y 从 2 涨到 3，$K(y)$ 从 16 涨到 22），对象增益比整定点高，等效开环增益偏高，于是冲过头；模糊层在接近目标时把 K_p 收到 2.1 附近，同时加阻尼，把这段多出来的增益抵掉了一部分。

但图 67-6 的下半部分不好看：阶跃后 8.2 s 到 8.6 s 这 0.4 s 里，控制量从 $+0.12$ 甩到 -0.62 再回到 $+0.64$，峰峰值 1.26，单拍最大变化 0.30。这个幅度不算小，必须解释清楚，否则“平滑”两个字就站不住。

原因有两层，都能从轨迹里读出来。第一层是微分项顶到限幅。这段滤波后的 $\dot{y}$ 冲到 2.5 单位每秒，K_d 被推到 0.81，$K_d\dot{y}$ 算出来是 2.04，被 ± 1.2 的微分限幅（$1.0 \times U_{\max}$）截平，所以微分项从 8.20 s 到 8.40 s 一直贴在 -1.2 上。第二层是比例项快速缩水：同一区间 K_p 在 1.27 到 1.68 之间摆动，而偏差从 0.79 掉到 0.33，$K_p e$ 从 1.29 掉到 0.56。两项一减，控制量从 $+0.12$ 一路滑到 -0.62。8.55 s 之后 $\dot{y}$ 降到 0.1 以下，微分项突然松绑，控制量又跳回 $+0.64$。所以这不是数值噪声，而是“微分项顶在天花板上、比例项往下掉”共同造成的过零往返。

单位分解保证的是“隶属度之和为 1”，不是“输出对输入无限光滑”。三角形隶属函数只给出 C^0 连续，规则表在格与格之间有折痕；折痕本身无害，但当折痕与微分限幅叠在一起时，就会出现上面的过零往返。这一点在 67.9 节的噪声实验里还会放大：噪声让 $\dot{y}$ 抖动，微分项在限幅附近来回触发，模糊 PID 的控制量总变差从 27.9 涨到 147.8，而固定参数 PID 只从 36.6 涨到 114.5——抖动的绝对增量相当，但因为模糊版本的基线更低，相对恶化更明显。

连续域里不需要滞环

第 66 章的规则切换是硬切换，必须在阈值上加 0.85 的滞环，否则工作点贴着阈值抖动时参数会逐拍跳变。本章不需要这个装置，因为插值是连续的：工作点从低工况移到中工况，参数不是跳过去，而是沿着基参数的变化趋势滑

过去——虽然本章的工况层仍是硬切换（三组基参数是离散的），但工况边界（1.3 与 2.3）离四段轨迹的实际驻留区间较远，抖动风险小得多。

这是模糊控制相对专家控制最实在的一个好处：**同样的规则数量，模糊版本不需要额外的抗抖装置**。代价是每条规则的语义被“稀释”了——专家控制里“进入 Z2 就把 K_d 乘 1.6”这句话是精确的、可验证的；模糊控制里“接近时加大阻尼”只能验证方向，验证不了幅度，幅度是由 S_d 与规则表的数值隐式决定的。可解释性的这一升一降，在 67.11 节还会展开。

67.5 离散情形

离散化只改了两件事

连续域的式 67-11 落到控制器里就变成

$$u(k) = K_p(k)\,e(k) + K_i(k)\,T_s \sum_{i=0}^{k} e(i) - K_d(k)\,\frac{y(k) - y(k-1)}{T_s}, \tag{67-13}$$

式 67-13 与连续式的差别只有两处：积分变成累加，微分变成差分。模糊层本身不受影响——它仍是把 $(e, \dot{e})$ 映射成三个修正量的静态映射，采样只决定这个映射被“多久问一次”。

本章取 $T_s = 10$ ms，对象积分子步 5 次，纯延迟 5 拍（0.05 s）。微分先经过一阶低通，时间常数 $T_{\mathrm{lp}} = 0.05$ s，实现上写成

$$\alpha = \frac{T_s}{T_{\mathrm{lp}} + T_s}, \qquad \dot{y}_f(k) = \dot{y}_f(k-1) + \alpha\big[\dot{y}(k) - \dot{y}_f(k-1)\big]. \tag{67-14}$$

这种写法比直接存两个历史点再差分更好：α 随 T_s 自动调整，滤波的时间常数恒为 T_{lp}。若写成固定的差分系数，改采样周期时滤波带宽会跟着变，噪声抑制特性和微分相位补偿会一起漂——这是工程上很常见的一处隐蔽错误。

一拍的计算延迟与相位预算

真实控制器不是“采样瞬间就把控制量送出去”，而是“采样—计算—下一个周期送出”，所以除了对象的 5 拍延迟，还有 1 拍计算延迟，合计 6 拍 0.06 s。本章仿真里只算了 5 拍，这一点要在指标解读时记住。

延迟的代价可以算清楚。中工况基参数 $(2.60, 0.05, 0.60)$ 配对象 $G(s) = 20/[s(s+2)]$，连续域穿越频率 12.66 rad/s，相位裕度 50.9°。把 0.05 s 延迟用四阶 Padé 近似补进去，裕度掉到 **14.7°**；延迟再加倍到 0.10 s，裕度变成 -21.6°，线性意义上已经失稳。

这三个数字解释了本章的两件事。第一，延迟加倍实验里模糊 PID 的超调从 1.09% 涨到 29.16%，不是规则表失灵，而是相位预算被吃光——14.7° 的裕度本来就只够勉强不振荡，再砍一半必然冲。第二，实际运行却比线性预测温和得多（延迟加倍时 IAE 4.096 而不是发散），原因是控制量长时间饱和，有效开环增益远低于名义值，饱和在替裕度买单。

按同样的办法算采样周期的影响：零阶保持引入的等效相位滞后约为 $\omega_c T_s/2$，在 $\omega_c = 12.66$ rad/s 处，$T_s = 10$ ms 只贡献 3.6°，$T_s = 20$ ms 贡献 7.3°。与延迟的 36° 相比，采样周期是次要矛盾。所以本章的选择是：**周期按计算量定，延迟按物理定，不要指望缩短周期来换稳定性**。把 T_s 从 10 ms 缩到 1 ms，延迟仍是 0.05 s，裕度还是 14.7°。

量化误差落在哪里

模糊层对量化的敏感度比线性 PID 高，因为它的增益是输入的连续函数，输入一量化，增益就变成阶梯。

算一笔账。偏差论域是 $[-E, E] = [-1.6, 1.6]$，七个集合把论域切成六格，每格宽 $2E/6 = 0.533$。表内后件每隔一格变化 $1/3$，对应 K_p 变化 $2.60 \times 0.55 \times (1/3) = 0.477$，所以 $\mathrm{d}K_p/\mathrm{d}e$ 最大约 $0.477/0.533 = 0.894$（每单位偏差）。

若采样是 8 位、量程 ± 2.5，一个 LSB 是 $5/256 = 0.0195$，对应的增益台阶是 $0.894 \times 0.0195 = 0.017$，占基值 2.60 的 0.67%。这个量级在多数回路上可接受，但如果回路本身裕度只有 15°，0.67% 的随机增益台阶叠加在格线附近，足以在稳态时维持一个很小的极限环。若是 12 位，LSB 降到 0.0012，增益台阶只有 0.04%，可以忽略。这就是为什么现代工程上量化基本不是问题，而八九十年代的模糊芯片必须专门处理它。

输出侧的量化影响小得多：12 位 DAC 在 ± 1.2 上分 4096 级，LSB 相当于满量程的 0.024%，远小于执行器的死区。

查表实现与计算量

模糊层每拍要做的运算可以数清楚：两个输入各算 7 个三角形隶属度（14 次减法、取绝对值、比较），外积得到 49 个权重（49 次乘法），再与三张表做 147 次乘加，最后三个除以同一个权重和。

在 CPython 上实测，单次 `fuzzy_delta` 调用平均 23.3 μs，其中隶属函数占 6 μs 上下；相对 10 ms 周期占 0.23%。这个数字对嵌入式没有参考价值（Python 的开销主要来自解释器和 NumPy 的调用边界），但说明运算量确实很小：换成一维数组加循环的定点 C，运算量级是几百条指令。

存储更不是问题：三张 7×7 的 int16 表共 294 字节，加上两组断点共 28 字节，总共不到 0.5 KB。

真正需要权衡的是“实时算”还是“预存整张曲面”。若把 $(e, \dot{e})$ 在 61×61 的网格上预先算好三个增益，双线性插值查表，存储是 $61 \times 61 \times 3 \times 2 = 22.3$ KB；101×101 则是 61 KB。前者在 16 位单片机上还放得下，后者就得外扩存储。而实时算的方案只需要 300 字节的表加十几行代码，代价是每拍几百条指令。**在存储紧张的场合实时算更划算，在算力紧张的场合查表更划算**，这条权衡与第 66 章的规则表查表完全同构。

这个权衡可以量化。同一台机器上把两种做法各跑两万次：在线推理（含隶属函数、外积、三次加权求和）平均 23.3 μs，61×61 网格的四点双线性插值平均 2.0 μs，差约 12 倍。换算到 10 ms 周期，在线推理占 0.23%，查表占 0.02%——两者都远低于“需要注意”的门槛。所以在本章的周期下，选择依据不是算力，而是存储：22.3 KB 换 21 μs 的每拍节省，只有在周期压到 1 ms 以下、或者控制器上同时跑十几个回路时才有意义。

顺便说一下这 12 倍的来源，它不只是“少算”，而是两种做法的复杂度不同：在线推理的耗时随规则数近似线性增长（49 条就是 147 次乘加），查表则与规则数无关，只与网格分辨率有关。若把规则表扩到 9×9（81 条），按 81/49 线性外推，在线耗时会涨到约 38 μs，查表仍是 2 μs。**规则越密，查表越划算**，这是 67.9 节讨论规则条数时容易被忽略的一笔账。

还有一条常被忽略：本章三张规则表都**不是中心对称**的。比例表在“偏差扩大”与“偏差收缩”两侧取相反的修正，积分表在大偏差与小偏差上取相反的符号，所以不能靠“只存一半、另一半点对称”把存储减半。若设计者强行把表做成中心对称来省存储，就会丢掉“接近时减速”这个最关键的非对称性——省下 150 字节，赔掉全部超调性能。

定点实现要注意的三件事

工业现场多用定点 DSP 或 16 位单片机，本章代码是双精度浮点，定点化时要处理三处。

第一是**权重和不为 1**。式 67-8 的除法在浮点下能把和归一到 1，定点下 49 个 Q15 权重相加会有几个 LSB 的误差。若代码里图省事，直接用“权重乘以 2^{-15}”而不做归一化，凸组合性质就破坏了，最坏情况下三个增益会整体偏大或偏小千分之几。这个偏差本身不大，但它随工作点变化，相当于引入了一个慢变的增益扰动。稳妥做法是保留除法，用一个 32 位累加器存权重和，再做一次除法。

第二是**中间结果要 32 位**。权重是 Q15（最大 32767），后件是 ± 1000 的整数，两者相乘是 3.3×10^7，超出 16 位。49 项累加后接近 1.6×10^9，仍在 32 位有符号范围内，但已经接近极限（2.1×10^9），所以后件不宜取得比 ± 1000 更细。

第三是**限幅和积分冻结要用同一个 $u(k-1)$**。守卫判据里的 $\mathrm{sign}(u_{k-1})$ 必须是真正送进执行器的那个值（限幅之后），不是限幅之前的计算值。若取错，饱和期间积分仍在累加，抗饱和等于没装。这一条在浮点代码里同样成立，只是定点下更容易因为“变量复用”而出错。

多速率：模糊层可以跑得比内环慢一个量级

采样周期不一定只有一个。现代控制器常见的结构是：电流环 100 μs、速度环 1 ms、外层的参数调整 10 ms 甚至更慢。模糊层属于最后一档，因为它的输出是 PID 的参数，而参数本来就该比控制量变得慢。

慢多少有判据。被调参数的变化时间尺度如果远大于闭环时间常数，就可以近似看作“分段常数”，每一段内用冻结参数分析。量化的说法是：模糊层的更新周期 T_f 应满足 $\omega_c T_f \ll 1$，工程上取 $\omega_c T_f \le 0.1$ 比较稳妥。本章 $\omega_c \approx 12.7$ rad/s，对应 $T_f \le 8$ ms。取 10 ms 略超一点，代价是零阶保持带来的附加相位滞后 $\omega_c T_f / 2 \approx 3.6^\circ$——与采样本身贡献的量级相同，可以接受。

若把模糊层放到 50 Hz（$T_f = 20$ ms），附加滞后 7.3°；放到 10 Hz（100 ms），滞后 36°，正好等于本章全部延迟的贡献。这就是“模糊层可以慢，但不能太慢”的量化边界。现场常见的错误是把模糊层塞进一个 100 ms 的慢任务里图省事，结果裕度被悄悄吃掉三分之一，而指标上看不出明显原因。

分核执行的另一个好处是算力核算变得清楚。本章单拍推理 23.3 μs，按 100 Hz 跑，一秒占用 2.33 ms，即 0.23% 的单核算力。即便同时跑八个回路，也只用 2%。真正吃算力的是内环的电流控制，不是模糊层——这一点常被误解，以为“智能控制一定很贵”。

反过来说，把模糊层跑得比必要更快也没有收益。67.5 节末尾的周期细化实验给出了直接证据：T_s 从 10 ms 缩到 2.5 ms，IAE 从 2.698 变到 2.738（几乎不动），超调从 1.09% 降到 0.03%，控制量变差从 27.9 降到 14.1。也就是说，细化周期改善的是“抖动”与“过冲尖峰”，不是“平均误差”。若某台设备的主要问题是 IAE 偏大，缩周期帮不上忙，该去调尺度或规则表。

离散化没有改变判断的粒度

最后要说清一个容易被误解的点：采样让模糊层“看不见”两次采样之间的变化。若偏差在 10 ms 内穿过整个论域，模糊层只在两端各看到一次，中间的过程全部丢失。本章对象的最高动态是 $s = -2$ 的极点，时间常数 0.5 s，10 ms 只占 2%，不存在这个问题；但如果换成一个时间常数 30 ms 的快速回路，同样的规则表就必须配 1 ms 的周期，否则模糊层看到的“轨迹”是跳跃的，规则表的连续性假设不成立。

判断标准很朴素：**控制周期应小于对象最快时间常数的十分之一**。本章是 10 ms 对 500 ms，余量 50 倍，这条法则对本回路相当宽松；真正接近临界的是 0.05 s 的纯延迟——它已经是对象时间常数的十分之一。

细化周期能换来什么

“周期够用”不该只靠经验法则判断，可以实测。把纯延迟固定在 0.05 s（拍数随周期等比增加），只把周期从 10 ms 收到 5 ms 和 2.5 ms，四段轨迹的指标如下。

周期	延迟拍数	全程 IAE	第三段超调	控制量总变差
10 ms	5	2.698	1.09%	27.9
5 ms	10	2.730	0.03%	15.5
2.5 ms	20	2.738	0.04%	14.1

IAE 基本不动（2.698 到 2.738，差 1.5%），说明**积分型指标对周期不敏感**——因为延迟没变，闭环的主导动态没变。但超调从 1.09% 掉到 0.03%，控制量总变差从 27.9 掉到 14.1，两者都接近腰斩。这说明 67.4 节里那段过零往返有很大一部分不是控制律的问题，而是采样与微分限幅的交互：周期越粗，$\dot{y}$ 的差分量越大，越容易顶到限幅。

这条实测给出一个很实用的诊断顺序：如果现场看到控制量抖动但指标还过得去，先怀疑采样周期与微分限幅，而不是急着改规则表。把周期减半的代价是每拍的计算预算减半（本章的模糊层在 CPython 上只占周期 0.25%，翻倍也无所谓），收益是抖动减半。反过来，如果算力紧张，把周期放到 20 ms，按上面的趋势抖动会翻倍，但 IAE 仍然在 2.7 附近，属于“能用但不好看”。

离散代码与设计意图对不上时的排查

还有一类问题更隐蔽：代码能跑、指标也不差，但和设计意图对不上。常见的有三种——把 $-\dot{y}$ 写成了 $\dot{e}$，把量化因子乘在了除法后面，把 S_p 乘在基值而不是修正量上。这三种错误都不会让系统发散，只会让“调参经验”失效：你以为在调规则表，实际在调一个完全不同的映射。

排查办法是打印中间量而不是看最终指标：取一个已知的输入点，手工算出三个修正量，与代码打印的 `delta` 对照。本章在 `fuzzy_delta` 里保留了返回值，就是为了这个用途。若对照不上，先查符号（$-\dot{y}$），再查归一化的除法顺序，最后查乘性叠加的位置——这三处的错误率占八成以上。

67.6 稳定性与性能

稳定性的三层检验

模糊控制没有像线性系统那样一条通用的稳定性定理。工程上能做的是三层递进的检验，每一层的结论都比上一层弱，但成本更低。

第一层是**冻结参数下的裕度**。把某个工作点的 $(e, \dot{e})$ 固定，参数就成了常数，闭环是线性的，可以直接算相位裕度。本章在 147 个顶点上做过这件事，最小 45.3°、中位 54.4°（见 67.4 节），但没有算延迟。把 0.05 s 延迟补进去，中工况基参数的裕度从 50.9° 掉到 14.7°（见 67.5 节），仍然为正，说明小信号意义下闭环稳定，但余量已经很薄。

第二层是**凸包意义上的必要条件**。若参数在凸包内任意取值都稳定，则系统在任何工作点都“逐点稳定”；反过来，只要有一个顶点不稳定，就必然存在一段轨迹会经过该顶点附近并失稳。这一层能排除坏设计，不能证明好设计。

第三层是**全工况仿真**。把设定值走过所有工况、把所有守卫都打开、把噪声和延迟都加上，看指标是否可接受。这一层最接近真实，但只覆盖被仿真过的工况。三层都过，不等于“已证明稳定”，只等于“没有找到不稳定的证据”。这句话不好听，但它是模糊控制目前的真实处境。

性能：优势集中在第四段

表 67-2 是四段轨迹的分段指标。两列控制器共用同一对象、同一采样、同一抗饱和、同一套守卫，区别只在参数来源：模糊版按 $(e, \dot{e})$ 实时修正，线性版固定取中工况的 $(2.60, 0.05, 0.60)$。

段	控制器	超调	上升时间	调节时间	IAE
1（0→1）	模糊	0.01%	0.84 s	1.59 s	0.633
1（0→1）	线性	0.01%	0.55 s	1.33 s	0.568
2（1→2）	模糊	0.20%	0.71 s	1.39 s	0.472
2（1→2）	线性	2.07%	0.34 s	1.47 s	0.404
3（2→3）	模糊	1.09%	0.71 s	2.40 s	0.492
3（2→3）	线性	11.86%	0.25 s	3.79 s	0.439
4（3→1）	模糊	1.02%	0.53 s	1.31 s	1.101
4（3→1）	线性	50.23%	0.32 s	1.93 s	1.586

表 67-2 里有一件必须先说清的事：**模糊版在三段的 IAE 上都略差于线性版**，第一段 0.633 对 0.568，第二段 0.472 对 0.404，第三段 0.492 对 0.439。全程 IAE 之所以仍以 2.698 对 2.997 领先，全部来自第四段——线性版在 3→1 的大幅下阶跃上超调 50.23%，IAE 1.586，几乎把前三段攒下的优势赔光。

图 67-7 把这件事画得很清楚：第四段结束时，线性版的输出在 1.0 附近来回摆了很久，模糊版则早早贴住设定值。

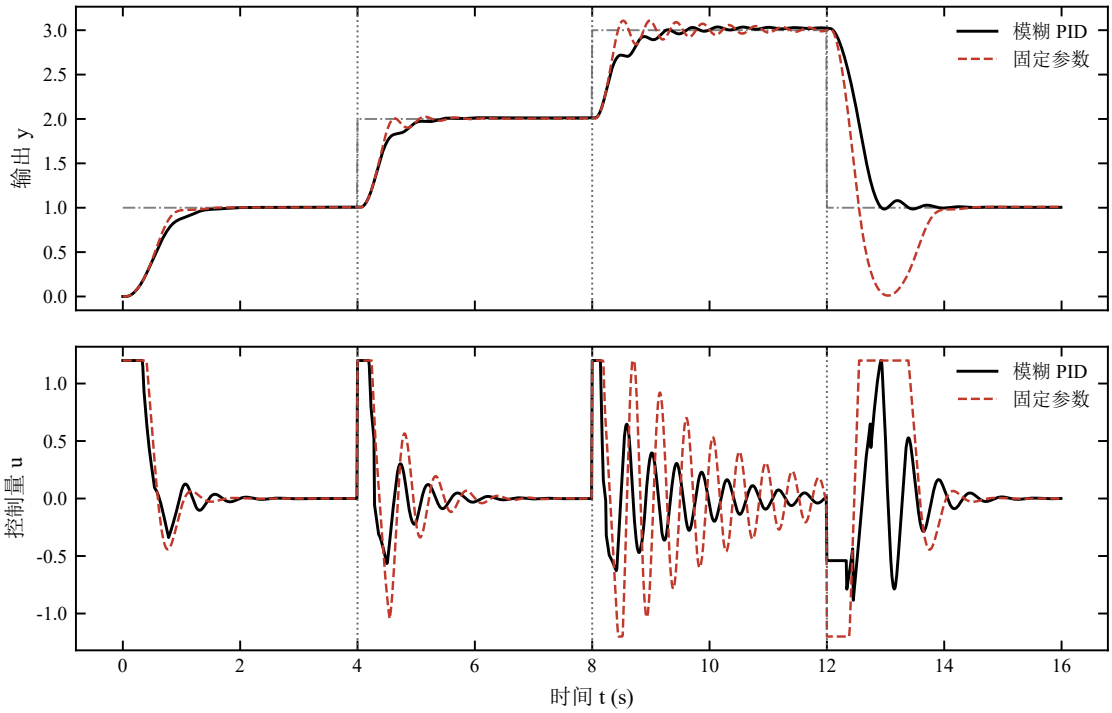


图 67-7 四段工况迁移：输出与控制量

所以对本章结果的诚实概括是：**模糊自整定买到的是大扰动下的稳定性，而不是全程的快速性**。如果这类回路的设定值很少大幅跳变，主要扰动是小幅波动，那么本章的模糊层带来的收益会比表 67-2 少得多，甚至为负——因

为它在小偏差区的修正很弱，白白多了一层计算。

第一段的反例：刹车踩得太早

第一段（0→1）模糊版慢 0.29 s，原因可以从增益轨迹里读出来。起步时 $K_p = 4.41$（低工况基值 3.80 叠加 $+16\%$ 的修正），$K_d = 0.90$。随着输出起来，$\dot{y}$ 变正，第二输入 $-\dot{y}$ 变负，偏差与它的符号相反，规则表立刻进入“偏差在收缩”的分支：K_p 一路掉到 0.5 s 时的 2.41，K_d 一路涨到 1.20。

也就是说，模糊层在偏差还有 0.6 的时候就开始大力刹车了。而这一段的线性基线用 $K_p = 2.60$、$K_d = 0.60$，本来就不超调（0.01%），刹不刹车无所谓。模糊层把“防超调”的力气花在了一个不需要防超调的地方，代价就是上升慢了 0.29 s。

这个反例说明规则表的“符号判据”有一个隐含前提：**大偏差意味着有超调风险**。在本章这个对象上，低工况的增益只有 10，加上限幅 1.2，稳态根本冲不过头；符号判据提前触发只会拖慢。要修也很简单——把低工况的 S_p、S_d 单独调小，或者给规则表加一条“低工况时抑制刹车”的前件。本章没有这么做，因为三组基参数共用一张规则表是设计目标之一（否则规则数要乘三）；代价就是承认第一段略慢。

乘性叠加的硬边界

式 67-10 的乘性叠加有一个不能越过的边界：

$$1 + S_p \Delta K_p > 0 \quad \forall\, \Delta K_p \in [-1, 1] \;\Longrightarrow\; S_p < 1. \tag{67-15}$$

理由很直白：ΔK_p 可以取到 -1，若 $S_p \geq 1$，对应的顶点上 $K_p \leq 0$，比例作用反号，那已经不是“整定”，而是换了一个控制器。S_i、S_d 同理。

这条边界在实践中比它看起来更容易踩到。表 67-3 是只改 S_p、其余参数不动时的实测。

S_p	顶点最小裕度	低于 30° 的顶点	全程 IAE	最大超调	控制量变差
0.55	45.3°	0/147	2.698	1.09%	27.9
0.80	42.2°	0/147	3.310	1.60%	26.9
1.00	40.1°	0/147	3.859	2.31%	37.7
1.50	$-3.6°$	30/147	5.694	1.45%	74.7
2.00	32.4°	0/147	8.656	2.57%	101.4

表里有两处值得注意。一是**性能在 $S_p = 0.55$ 附近最好，加大 S_p 反而一路变差**，说明“修正越强越好”的直觉是错的：修正强了，增益在阶跃过程中摆动过大，等效开环增益的平均值反而偏离整定点。二是 $S_p = 2.00$ 时顶点统计又“变好”了（最小裕度 32.4°、0 个低于 30°），这不是系统变稳了，而是**一部分顶点的比例增益已经是负数**，那些顶点算出来的裕度没有物理意义，正负号已经让回路性质变了。所以表 67-3 最后一行只能当作“模型失效”的证据，不能当作设计参考。

三种失效形态与各自的兜底

把本章跑过的所有工况归一下类，失效只有三种形态，每一种的兜底手段不同。

第一种是饱和引起的积分积爆。 控制量顶到限幅后误差仍在，积分继续累加，退出饱和时积分已经很大，输出必然冲过头。本章用“饱和时冻结积分”兜底，判据是 $|u_{k-1}| \geq 0.95 U_{\max}$ 且 $\operatorname{sign}(u_{k-1}) = \operatorname{sign}(e_k)$。

但这条守卫在本章的实测效果很微妙，值得单独说一句：关掉它，四段的 IAE 各涨一点（第一段 0.633→0.638，第四段 1.101→1.155），第四段的超调却从 1.02% 降到 0.58%。两个方向都朝不利的一侧只动了一点点，说明**本章第四段的超调根本不是积分积爆造成的**——下行时控制量被“下行限速”锁在 $-0.45U_{\max}$，积分没有机会涨到很大。真正压住第四段超调的是另一个守卫：关掉下行限速，超调从 1.02% 涨到 13.64%。

所以“抗饱和”这个名字容易让人误以为它总在压超调。它压的是**积分积爆型**的超调，本章的第四段不是这一类。到底是哪一类，要看控制量在超调发生前是否贴着限幅走了很长时间。

第二种是延迟引起的相位不足。 这种失效不能靠参数兜底，只能靠降低带宽或者缩短延迟。本章的裕度只有 14.7°，延迟一加倍就变成负裕度，仿真实测超调 29.16%。要真正解决，只能把 K_p 整体调低（牺牲快速性）或者把延迟从 0.05 s 压下去（改硬件或改通信）。

第三种是抖动，也就是小信号极限环。 来源是采样、微分限幅与规则表折痕三者的交互，表现为控制量的高频小幅振荡，不影响 IAE 但磨损执行器。本章的控制量总变差 27.9，其中很大一部分来自这种抖动（67.5 节的周期细化实验把它从 27.9 压到 14.1）。兜底手段是减小微分限幅、加宽死区、或者细化采样周期。

三种形态的治理难度依次上升：饱和最好办（一条判据），延迟最难办（要动硬件），抖动居中（要动参数但要权衡）。现场排查时按这个顺序查，命中率最高。

三种检验各自的盲区

本章用了三种手段检验稳定性：冻结顶点（147 个）、描述函数（表 67-1）、闭环仿真（四段加四种工况）。三种都过了，仍然不能宣布“稳定”，因为每种都有明确的盲区。

冻结顶点的盲区是“变化率”。 它把参数当成常数，逐个算裕度，因此对“参数变化太快”这类失稳完全无感。第 42 章的增益调度早就指出过：每个工作点都稳定，不代表切换过程稳定。本章的参数变化时间尺度是 0.1 s 量级，闭环时间常数 0.5 s，两者只差 5 倍，慢变假设只是勉强成立。

描述函数的盲区是“谐波”与“大幅值”。 它只保留基波，等于假设高次谐波被对象滤掉。本章对象带宽约 2 rad/s，规则表折痕产生的高次谐波主要在 10 rad/s 以上，衰减 20 dB 以上，假设基本成立；但若把同样的规则表用在带宽 50 rad/s 的伺服上，这个假设立刻失效。另外，描述函数在幅值大到触发截断时也失效——那时输入已经不是正弦，而是被削顶的波形。

闭环仿真的盲区是“没跑到的工况”。 四段轨迹加四种劣化工况，看起来覆盖面不小，但仍然是人挑的。若现场出现一个没被设计者想到的组合（比如在第三段中途反向、或者负载突变叠加噪声），仿真给不出任何保证。

三种盲区互不重叠，这正是要用三种手段的原因。但它们叠加起来仍然留着一个缺口：**参数变化快、谐波显著、工况未遍历，这三件事同时发生的情况**，本章没有任何工具能覆盖。工程上的应对不是再找一个更强的定理，而是加兜底——限幅、限速、超差报警，以及 67.10 节的回退通道。

一个反例：顶点全绿，仍要限速

顶点检验给出 147 个全绿，容易让人产生“参数怎么变都安全”的错觉。第四段的实验否定了这个错觉。

第四段是 3→1 的大幅下阶跃。顶点检验在这一段同样全绿——因为顶点只关心“参数取到极值时回路是否稳定”，而这一段的问题不是稳定，是**超调**。若关掉下行限速，超调从 1.02% 涨到 13.64%，系统仍然稳定，但指标不可接受。

这说明稳定性检验与性能检验是两件事。参数落在凸包内、每个顶点都有 45° 以上裕度，只能保证“不发散”；至于超调多大、调节多快，得另算。本章的做法是把两者分开：稳定性靠顶点与仿真，性能靠四段指标，兜底靠守卫。三者缺一，交付时就会留下一个说不清的角落。

顺带说一句，这个反例也解释了为什么 67.3 节要把守卫层装上，而不是指望“模糊层足够平滑”。平滑能降低超调，但降不到零；13.64% 与 1.02% 之间的差距，是限速这条硬规则挣来的，不是插值挣来的。

把四段指标折算成一个数

调参搜索需要一个标量目标，但四段轨迹给出的是十几个数。怎么折算，直接决定搜索会选出什么。

最简单的做法是把四段 IAE 相加，再加一个超调惩罚项。本章的预算实验用的是 $\mathrm{IAE} + 0.3\times$ 超调（百分数），0.3 这个系数是人为定的：它意味着“1 个百分点的超调折合 0.3 个单位的 IAE”。改成 0.1，搜索会选出更激进、IAE 更小但超调更大的参数；改成 1.0，则会选出很保守、超调近乎为零但响应偏慢的参数。这个系数没有客观最优值，它表达的是设计者对“冲过头”与“慢一点”的相对厌恶。

更稳妥的做法是分层，而不是加权。第一层是可行性：不发散、不越限、不进饱和。第二层是硬指标：超调不超过某个上限（比如 5%），调节时间不超过某个上限。第三层才在满足前两层的候选里比 IAE。分层的优点是每一层的门槛都有物理含义，不需要用一个系数去调和不同量纲；缺点是搜索空间被切成不连续的区域，随机搜索会更难命中，需要先粗扫找到可行域。

还有一个容易被忽略的项：控制量总变差。它不进目标函数，但必须设上限。否则搜索会找到“IAE 很小、阀门每秒来回几十次”的参数，仿真里看着漂亮，装到设备上几周就磨坏了。本章的做法是把总变差作为约束检查，超过基线两倍就判为不可行，而不是把它折进目标。

最后提醒一点：这四段轨迹本身也是一种加权。它们等长，所以“总 IAE”实际上是四段算术平均。若实际工艺里 90% 的时间停在第二段，那么第二段的权重就该加大，或者干脆只用第二段的指标做目标。这一点没有通用答案，取决于设备怎么用。

67.7 参数意义与作用

量化因子不是增益

两个量化因子 $K_e = 1/E$、$K_{ec} = 1/D$ 常被误当成“模糊层的增益”，其实它们的作用只是换算：

$$x_e = \mathrm{clip}\!\left(\frac{e}{E}, -1, 1\right), \qquad x_{ec} = \mathrm{clip}\!\left(\frac{-\dot{y}}{D}, -1, 1\right). \tag{67-16}$$

E 的物理含义是“多大的偏差算满刻度”，D 是“多快的变化算满刻度”。它们不改变规则表本身，只改变轨迹在规则平面上走过哪一块区域。把 E 调小，同样的偏差会落在更大的语言值上，模糊层“觉得”偏差更大，修正更强——从这个意义上说，E 确实间接起到了增益的作用，但它不是线性的：E 减半不会让修正量翻倍，只会让轨迹跑到论域的饱和区，而饱和区的修正量是有上限的 ± 1。

所以调 E 时心里要有一个图像：它是在挪动“偏差的标尺”，不是在拧“增益的旋钮”。标尺挪过头，所有偏差都顶到 PB，模糊层退化成一个常数修正，等于白装。

三个比例因子各管什么

S_p、S_i、S_d 的含义比量化因子清楚，它们直接是修正量的幅度：

$$K_p = K_p^0\big(1 + S_p \Delta K_p\big), \quad K_i = K_i^0\big(1 + S_i \Delta K_i\big), \quad K_d = K_d^0\big(1 + S_d \Delta K_d\big). \tag{67-17}$$

$S_p = 0.55$ 的意思是“比例增益最多在基值上下浮动 55%”，$S_i = 0.70$ 是积分浮动 70%，$S_d = 0.35$ 是微分浮动 35%。这三个数字直接可读、可直接写进设计说明，比“模糊控制器的量化因子”这种说法清楚得多。

三个通道的分工也不同。S_p 管“推力”，决定大偏差时敢不敢使劲；S_i 管“消差”，决定小偏差时积分敢不敢加速；S_d 管“刹车”，决定接近目标时阻尼加多少。按 67.3 节的联调顺序，先调 S_p 定推力，再调 S_i 定消差，最后调 S_d 压超调，这个顺序与三者的物理分工一致。

单参数扫描：谁真正重要

五个参数逐个在标称值上下浮动 30%，其余保持不动，四段轨迹的指标如表 67-4。

参数	取值	全程 IAE	最大超调	控制量变差
E	1.12	2.780	1.59%	29.7
E	**1.60**	**2.698**	**1.09%**	**27.9**
E	2.08	2.444	14.16%	27.4
D	0.98	2.738	1.19%	26.8
D	**1.40**	**2.698**	**1.09%**	**27.9**
D	1.82	2.667	1.14%	28.9
S_p	0.385	2.626	15.08%	32.2
S_p	**0.55**	**2.698**	**1.09%**	**27.9**
S_p	0.715	3.057	1.28%	25.4
S_i	0.49	2.693	1.03%	27.6
S_i	**0.70**	**2.698**	**1.09%**	**27.9**
S_i	0.91	2.703	1.16%	28.2
S_d	0.245	2.681	1.01%	25.4
S_d	**0.35**	**2.698**	**1.09%**	**27.9**
S_d	0.455	2.726	1.87%	31.1

表 67-4 的第一眼结论是：**五个参数里只有两个真正重要**。E 从 1.12 变到 2.08，最大超调从 1.59% 跳到 14.16%；S_p 从 0.55 变到 0.385，超调从 1.09% 跳到 15.08%。而 D 在整个区间内 IAE 只动 2.6%，S_i 只动 0.4%。

这个结论对本回路成立，但不能照搬到所有回路。D 不敏感，是因为本章对象的输出变化率始终落在 0.6 到 0.8 这一个窄带里，除以 $D = 1.40$ 后在论域上只占 0.4 到 0.6 一小段，D 放大 30% 也只是把这一小段往中间挪一点，轨迹走的格子几乎没变。若换一个起停频繁、速度变化范围大的对象，D 就会变得敏感。S_i 不敏感，是因为积分本来就很弱（中工况 $K_i = 0.05$），±30% 的浮动对总控制量的贡献不到 0.02，淹没在比例项里。

E 的陷阱：IAE 更好、超调十倍

表 67-4 里最值得警惕的一行是 $E = 2.08$：IAE 2.444，**比标称值还好 9.4%**，但最大超调 14.16%，是标称值的十三倍。

这就是只看 IAE 会犯的错。E 放大后，同样的偏差归一化得更小，模糊层的修正变弱，闭环更接近“固定参数 PID”，于是前期的误差积得更久（IAE 本应变大），但第四段的超调让误差迅速反向，正负相消，总 IAE 反而降下来了。IAE 对“来回摆”的惩罚不够重，这就是为什么本章每张表都要同时列超调和变差。

同类陷阱在 $S_p = 0.385$ 那一行也出现：IAE 2.626 是整张表里最低的，超调却是 15.08%。把分段指标拆开就看清了：修正变弱后，前三段少踩刹车，IAE 从 0.633/0.472/0.492 降到 0.593/0.436/0.456，上升时间也快了 0.26/0.29/0.03 s。第四段的账是反的：超调从 1.09% 涨到 15.08%，IAE 从 1.101 涨到 1.141。前三段省下 0.112，第四段多花 0.040，总账净省 0.072，于是“总 IAE 最低”落到了这个明显更差的参数上。

这说明“总 IAE 最低”这个判据在多工况轨迹上非常不可靠：它把四段的账混在一起算，一段的严重超调可以被三段的轻微改善掩盖。设计时应当**逐段看指标，再决定权重**，而不是直接最小化全程 IAE。

读这类表的正确姿势是：**先看超调，再看变差，最后看 IAE**。超调决定这台设备会不会撞限位，变差决定执行器寿命，IAE 只决定“平均而言差多少”。三者冲突时，优先保前两个。

隶属函数的形状与重叠

隶属函数的形状在工程上几乎没有选择余地：三角形最常用，因为计算最省、支持集有限、交叠关系直观。梯形与高斯也能用，但梯形多两个参数，高斯要算指数，收益只是“更光滑”——而 67.4 节已经说明，光滑性在本章不是瓶颈。

真正有约束力的是**重叠度**。本章把半宽取成等于相邻中心间距 $h = 2/(n-1)$，于是相邻两个三角形在中点恰好各取 0.5，整条论域上隶属度之和恒为 1。这不是审美选择，而是为了让加权平均在数值上直接等于凸组合——代码里的归一化虽然保留着，但分母恒为 1，不额外引入除法误差，也让“参数落在凸包内”这个论断成立。

重叠度偏离这个值会出两种问题。若半宽大于间距（重叠过多），隶属度之和大于 1，必须归一化，归一化后映射仍然连续，但每个集合的“影响力”被摊薄，等效于把 S 因子调小。若半宽小于间距的一半，论域上会出现**隶属度真空**——某个点上所有三角形的隶属度都是 0，归一化时分母为零，代码只能输出三个零修正，模糊层在这些点上彻底失效，闭环出现周期性的“修正空洞”。这种失效很隐蔽：大部分工作点正常，只有轨迹穿过真空带时突然失去修正，表现为每隔一段距离出现一次小台阶。

本章的取法把这两个极端都避开了，代价是重叠度被锁死为 50%，设计者失去了“用重叠度调平滑性”这个手段。若要这个手段，就得接受归一化与真空检查两份额外工作。

守卫参数的意义

三个守卫参数不在规则表里，但同样要理解它们的物理含义。

`HOLD_SAT=0.95` 是“多接近限幅算饱和”。取 0.95 而不是 1.0，是为了在控制量真正顶死之前就停止积分，给退出饱和留一点提前量；取得太小（比如 0.8）会在正常大信号段误冻结积分，拖慢响应。

`UD_CLAMP=1.0` 是微分项的限幅倍数，实际限幅值是 $1.0 \times U_{\max} = 1.2$。它的作用是防止 $\dot{y}$ 尖峰把微分项推到荒谬的数值。代价是 67.4 节看到的过零往返：微分项一旦顶到 1.2，比例项的变化就直接决定控制量符号，跨格时容易翻向。

`DESC_LIMIT=0.45` 是下行限速，控制量负向不超过 $-0.45U_{\max} = -0.54$。它是第四段超调的唯一有效防线（关掉后超调从 1.02% 涨到 13.64%），但它也压住了控制量的下行速度，使第四段的上升时间停在 0.53 s 而不是更快。这个参数是典型的“用一个性能换另一个性能”，取值要看工艺更在意超调还是更在意下行速度。

三个参数的共同点是：它们都不改变模糊层的映射，只在边界条件上做文章。也就是说，**模糊控制解决的是“平时怎么调”，守卫解决的是“极端时怎么不崩”**，两者不能互相替代。

参数之间的三组耦合

本节前面把五个参数逐个扫了一遍，那是“控制其他变量”的看法。实际调参时它们互相牵制，三组耦合必须先知道，否则会陷入“调好一个坏掉另一个”的循环。

第一组是 E 与 S_p。 两者在“修正强度”上互为替代：E 大则归一化后的偏差小，修正弱，需要 S_p 补回来；E 小则修正强，S_p 要收。67.8 节的 3×3 网格把这条耦合量化了：$E=1.90$ 配 $S_p=0.40$ 时超调 24.3%，配 $S_p=0.70$ 只有 0.8%。所以这两个参数必须成对调，或者做成二维网格。

第二组是 D 与 S_d。 D 决定变化率被归一化成多大，S_d 决定这个归一化值的修正幅度。两者都影响阻尼，但方向不同：D 变大让 $\dot e$ 看起来更小，微分修正变弱，等效阻尼下降；S_d 变大则直接加强微分修正。现场表现为“改了 D 之后 S_d 的有效性变了”，道理就在这里。

第三组是 S_p 与守卫层的限速。 下行限速把控制量压到 $-0.45U_{\max}$，此时比例项已经饱和，S_p 再怎么加大也传不出去。换言之，**限速一旦生效，S_p 在下行段就失去意义**。这也是为什么第四段的超调对 S_p 的敏感度不如对 `DESC_LIMIT` 的敏感度高。

三组耦合的实践含义是：调参记录表里必须写明“这次改的是哪几个参数”，否则事后看到指标变化，无法判断是哪个参数起的效。本章的定稿参数是五维搜索一起选出来的，不能拆成“先定 E、再定 S_p”这样的单变量叙事。

换采样周期时哪些参数要改

现场改控制周期的情形很常见：算力紧张时从 10 ms 放到 20 ms，或者为了压低噪声从 10 ms 提到 5 ms。改完之后哪些参数要重调，不能靠试。

E 与 D 不用改。两者的定义都是物理量——E 是“希望落满论域的最大偏差”，D 是“最大变化率”，单位分别是工程单位与其每秒值，与采样周期无关。把控制周期从 10 ms 改成 20 ms，最大偏差没有变，归一化尺度也就不该变。

D 的通路要复查，不是因为尺度，而是因为滤波器。本章的 $\dot y_f$ 由差分加一阶低通得到，时间常数 0.05 s，是以秒为单位写的。若有人图省事把它写成“5 拍”，那么 10 ms 时是 50 ms、20 ms 时是 100 ms，滤波变慢、相位滞后增加，等效阻尼下降。此时 E、D、三个 S 一个都没动，超调却变大了——这是最容易被误判为“对象变了”的一类故障。

延迟要按秒重算。本章的纯延迟存成 5 拍，在 10 ms 下是 50 ms。改成 20 ms 后，若仍存 5 拍，物理延迟变成 100 ms，67.5 节的相位预算直接翻倍。以 $\omega_c=12.7$ rad/s 计，多出来的 50 ms 相当于 36.4°，足以把 14.7° 的裕度吃成负值。正确做法是先把延迟时间换算成拍数（`round(Td/Ts)`），再看这个拍数是否还够细——不够细就得提高采样率，而不是硬压。

模糊层的周期可以放宽，但要按 67.5 节的条件：$\omega_c T_f\le0.1$。10 ms 采样、模糊层每 2 拍算一次，$T_f=20$ ms，$\omega_c T_f=0.25$，已经越界；此时增益曲面的变化跟不上误差的变化，等效于在回路里多插了一个零阶保持器。要么保持每拍计算，要么接受指标退化并重新整定。

顺序上建议这样做：先按秒重算延迟与滤波时间常数，确认相位预算没被吃掉，再跑一次退化检验（三个 S 置零）确认 PID 分支与之前一致，最后才动 S。反过来做，等于在一个已经变了的回路上调参，调出来的数没有迁移价值。

67.8 调参方法与经验

第一步永远是退化检验

把三个 S 因子全部置零，模糊层的修正量就是 0，增益退回基值。此时模糊控制器的行为必须与固定参数 PID 完全相同：本章的指标应复现 2.997 与 36.64（与 67.6 节线性基线逐位一致，或至少在小数点后三位相同）。

这一步看着多余，实际是最有效的排错手段。若 $S=0$ 时指标对不上，说明模糊层在“不该起作用”的时候起了作用——常见原因是归一化写错导致三个 Δ 不全为零、权重和不为 1 导致修正量有常值偏置、或者积分项把上一次的修正量带进了状态。这些问题在 $S\neq 0$ 时会被“看起来合理”的指标掩盖，调参时怎么调都调不到最优，因为底下有一个恒定的偏差。

工程上把这一步写成回归测试：每次改规则表或改代码，先跑 $S=0$，确认退化指标不变，再跑正式参数。第 66 章也用了同样的做法（把专家规则全关，退化成固定参数 PID 对表），两章的经验一致——**先证明“能退化”，再证明“能改进”**。

第二步定 E：让最大偏差落在 0.6 附近

E 的取法在本章是经验性的：最大常用阶跃 1.0，取 $E=1.60$，归一化后最大偏差 0.625。这个 1.6 倍不是巧合，而是为了让“最大偏差”落在 PS/PM 之间而不是顶到 PB。

为什么要留这一档？因为顶到 PB 之后，轨迹就进入规则表的饱和区，那里所有格子的后件都接近 ± 1，模糊层失去分辨能力——偏差是 1.2 还是 2.0，输出都一样。设计者花力气写 49 条规则，结果最常用的区域落在饱和区，等于只用了 9 条。

反过来，E 取太大（比如 2.4），最大偏差只归一化到 0.42，轨迹始终在 ZE 附近打转，修正量很小，模糊层形同虚设。表 67-4 里 $E=2.08$ 的超调暴涨到 14.16%，就是这个机制。

实用的取法：**先按 1.6 倍最大常用阶跃起步，然后看阶跃响应的超调**。超调没降下来就往小调 E（增强修正），控制量抖动明显就往大调 E（减弱修正）。每次调 10% 到 15%，不要一步到位。

第三步调 S_p：从 0.3 起，不要从 1.0 起

S_p 是最敏感的参数，也是最容易被调过头的。表 67-4 显示，S_p 从 0.55 加到 0.715，IAE 从 2.698 涨到 3.057；从 0.55 减到 0.385，超调从 1.09% 涨到 15.08%。也就是说两侧都不好，可用的区间不宽。

建议从 0.3 起，每次加 0.1，观察三个指标：最大超调、上升时间、控制量变差。加 S_p 的典型效果是“超调先降后升”——一开始修正变强，接近目标时比例收得更狠，超调降；加过头之后，大偏差段的增益被推得太高，反而冲过头。本章的最优点 0.55 就在这个转折附近。

还要记住式 67-15 的硬边界：S_p 不能到 1。到了 1，某个顶点上 $K_p=0$，比例作用消失；超过 1，比例作用反号。这不是“调得不好”，而是“控制器变了”。

E 与 S_p 不能分开调

上面三步是按顺序单参数调的，这在排错时好使，在找最优点时不够用。原因很简单：单参数扫描只看到多维曲面上的一条切面，而 E 与 S_p 之间存在明显的交互。把这两个参数各取三档做一次 3×3 网格，其余参数固定，实测如表 67-5。

E / S_p	**0.40**	**0.55**	**0.70**
1.30	2.614 / 8.7%	2.735 / 1.4%	3.031 / 1.8%
1.60	2.624 / 14.0%	**2.698 / 1.1%**	3.017 / 1.2%
1.90	2.661 / 24.3%	2.601 / 8.5%	2.869 / 0.8%

表里每格是“全程 IAE / 最大超调”。三行看下来，超调最小的格子沿着一条斜带分布：$S_p = 0.55$ 配 $E = 1.30$ 到 1.60，或者 $S_p = 0.70$ 配 $E = 1.90$。也就是说，E 加大必须同时加大 S_p，否则超调会迅速恶化——$E = 1.90$ 配 $S_p = 0.40$ 时超调 24.3%，是三行里最差的一格。

机理很清楚。E 放大使同样的偏差归一化得更小，修正量随之减弱；S_p 放大则把减弱的修正量重新放大。两者在“修正强度”上是替代关系，所以等强度线大致沿对角线。但替代并不完整：E 还改变轨迹在规则平面上走的路径，S_p 只改变幅度。所以在对角线上，不同格子的 IAE 仍有差别——$E = 1.90$、$S_p = 0.55$ 的 IAE 2.601 是全场最低，超调却 8.5%；$E = 1.60$、$S_p = 0.55$ 的 IAE 2.698 略高，超调只有 1.1%。

这也解释了为什么本章最终选的是“中间格”而不是“IAE 最低格”：按 67.8 节开头的评分口径（超调优先），1.1% 比 2.601 的 IAE 更值钱。若工艺更在意平均误差，选 $E = 1.90$、$S_p = 0.55$ 也说得通，前提是能接受 8.5% 的超调。

实践上的建议是：先按顺序单参数把量级定下来，再做一次 3×3 粗网格，最后在最优格附近做二维细化。本章的 $(1.60, 0.55)$ 并不是单参数顺序调出来的，而是 67.9 节的五维随机搜索选出来的——顺序调参在这里的作用是“建立直觉与排错”，不是“找最优”。

第四步和第五步：S_i 与 S_d

S_i 在本回路上几乎无关（表 67-4 里 ±30% 只让 IAE 动 0.4%），原因是积分本来很弱。所以在弱积分回路上，S_i 可以最后调，甚至直接取 0.7 不动。判断“积分是否足够弱”的办法是看 K_i/K_p 的量级：本章 0.05/2.60 ≈ 0.019，属于弱积分；若某回路的 K_i 与 K_p 同量级，S_i 就会变敏感，必须先调它。

S_d 只影响超调和抖动，不影响 IAE 的量级。表 67-4 里 S_d 从 0.35 加到 0.455，超调从 1.09% 涨到 1.87%，变差从 27.9 涨到 31.1。加 S_d 的收益是压超调，代价是放大噪声与抖动，这个折中在 67.11 节会用噪声实验量化。

一个可复用的调参记录表

调参过程应当留痕，否则换个人接手就得从头再来。表 67-6 是本章建议的记录格式，每一行是一次试验。

轮次	E	D	S_p	S_i	S_d	最大超调	全程 IAE	变差	结论
0	—	—	0	0	0	—	2.997	36.6	退化基线
1	1.60	1.40	0.30	0	0	—	—	—	起步，只开推力
…	…	…	…	…	…	…	…	…	…
终	1.60	1.40	0.55	0.70	0.35	1.09%	2.698	27.9	定稿

第 0 行必须是退化基线，最后一行必须是定稿参数，中间的试次一次都不能删。这些记录的意义在于事后归因：指标变差时能查到是哪一次改动引起的，而不是对着一堆参数猜。第 1 行只开 S_p、把 S_i 与 S_d 留到后面，是为了让“推力”与“消差”“刹车”的效应分开归因，混在一起调就说不清是谁的功劳。

五个常见错误

按发生频率排序，本章在设计与调试中踩过或差点踩到的五个坑。

一是先改规则表，后调尺度。 规则表有 49 个格子，尺度只有 5 个数。指标不好时先去改格子，等于在 49 维空间里手工搜索，几乎不可能收敛。正确顺序是先调 5 个尺度，尺度调到头仍不满意，再动规则表，而且一次只改一两格。

二是用单项 IAE 定胜负。 67.7 节已经证明，$E = 2.08$ 与 $S_p = 0.385$ 都能刷出更低的 IAE，同时把超调放大十倍以上。评分函数至少要含超调与变差，否则优化器会精准地找到“指标好看、设备受伤”的那组参数。

三是把 E 当增益旋钮。 调 E 确实会改变修正强度，但它不是线性的，而且会同时改变轨迹在规则平面上的路径。把它当增益调，会在某个 E 上突然发现“再调一点就完全没反应”——那是轨迹已经整体进入饱和区。

四是跳过退化检验。 前面说过，这一条能挡住一半的隐蔽 bug。

五是只在一个工况上验证。 本章的模糊层在低工况（第一段）实际上比线性 PID 慢，如果只在低工况调参，会得出“模糊没用”的结论；只在高工况调，又会过度乐观。至少要在增益变化范围的两端各测一次。

什么时候不该上模糊

最后给一条止损判据。如果固定参数 PID 在全部工况上的超调都低于 5%、调节时间都满足工艺要求，那么模糊层的收益很有限——本章第一段就是活例子：线性版超调 0.01%，模糊版 0.01%，但慢了 0.29 s。这时候上模糊，等于用一层额外的复杂度换零收益，还要承担规则表维护、参数漂移、可解释性下降三份成本。

判断值不值得上的量化标准可以是：**在部分工况上，固定参数 PID 的超调超过 10% 或者出现明显振荡**。本章的第四段（50.23%）与第三段（11.86%）正好落在这条线以上，模糊层把两者压到 1% 附近，收益才足以抵消复杂度。若某台设备的工况变化只有 ±20%，增益变化不到 1.5 倍，直接做鲁棒整定（见第 106 章）比上模糊更划算。

调参预算的现实感

人工调参的轮次是有限的。本章的五个参数，若每个取 5 个候选值做全因子，就是 3125 次仿真；实际调参不可能这么干。比较现实的做法是先按上面的顺序把参数收到一个窄区间，再用小规模搜索微调。

67.9 节会给出实测的“评估次数—指标”曲线：第 10 次评估时最优评分还在 2.050，反而不如人工起点 1.922；到第 20 次降到 1.828，第一次超过人工；第 40 次以后基本饱和在 1.802。也就是说，**这个问题的有效维度远低于 5**，随机搜索要攒够 20 次评估才开始产生正收益，40 次之后就不再长进。这个结论对工程的含义是：不必追求全局最优，把顺序做对、把退化检验做扎实、把评分函数设计好，比多跑几百次搜索更有价值。

67.9 优化与改进

规则条数买到的是平滑，不是精度

把规则表从 7×7 降到 5×5 和 3×3（后件由参考表双线性采样得到，不是另外手写），三种分辨率的实测如表 67-7。

规则数	分辨率	全程 IAE	最大超调	控制量变差
9	3×3	2.476	9.02%	37.1
25	5×5	2.602	2.10%	28.6
49	7×7	2.698	1.09%	27.9

图 67-8 把这三组数字画在同一张双纵轴图上，趋势很清楚：规则越多，超调与变差越低，IAE 反而越高。

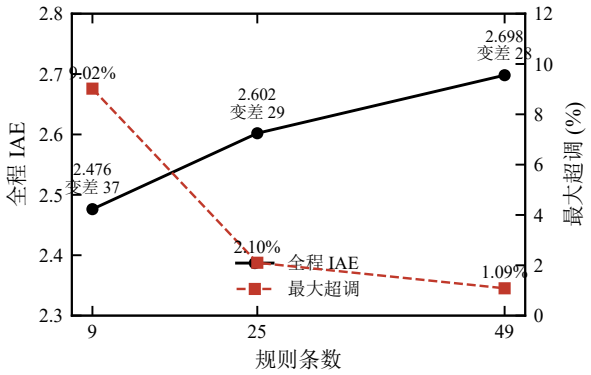


图 67-8 规则条数与性能：IAE 与超调的反向趋势

这个结果初看反直觉，其实与 67.7 节的结论是同一件事的两面。3×3 的规则表只有 NB、ZE、PB 三个语言值，后件在格与格之间跳变幅度大，等效于一个更“激进”的控制器：大偏差时全力推，接近时突然收，动作接近开关控制。开关控制的平均误差可以很小（因为推得猛），但超调大、动作粗。7×7 把同样的控制意图切成更细的档，过渡更平缓，超调被压下去，代价是过渡段“不够猛”，IAE 略高。

所以规则条数买到的是**平滑性**，不是精度。这个结论对设计有直接指导：如果工艺最在意超调与执行器磨损，用 7 档；如果在意平均误差且执行器经得起折腾，3 档就够，还能省下 80% 的规则维护成本。中间状态（5 档）在本章是性价比最差的一档——超调 2.10% 已经接近 7 档，IAE 却也没比 7 档好多少。

整定预算：随机搜索什么时候开始划算

图 67-9 是“评估次数—最优评分”曲线。评分函数取

$$J=\mathrm{IAE}_{\text{短轨迹}}+0.3\times\max(\text{超调}\%),\tag{67-18}$$

短轨迹只跑前三段、12 s，比四段全程快四分之一。搜索在五维箱体里均匀随机取点，三个随机种子各跑 60 次。

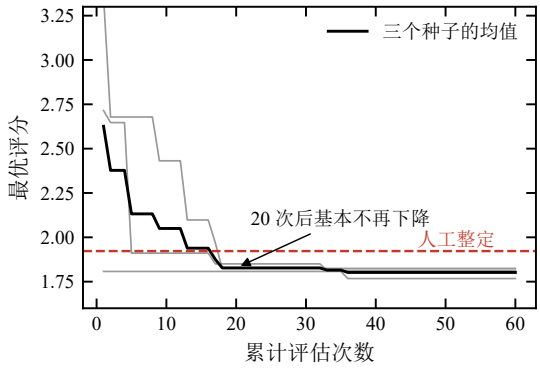


图 67-9 整定预算：随机搜索的最优评分随评估次数变化

四档预算的汇总见表 67-8。

评估次数	最优值均值	最差种子
10	2.050	2.431
20	1.828	1.850
40	1.802	1.825
60	1.802	1.825
人工起点	1.922	—

表 67-8 的头一行值得停下来看一眼：**10 次评估的最优值均值 2.050，比人工整定的 1.922 还差 6.7%**。也就是说，只给 10 次机会的随机搜索不如一个懂行的工程师。原因不难理解：五维箱体的体积很大，10 个随机点大概率落在“明显不对”的区域（比如 $S_p = 0.1$ 配上 $E = 0.4$），每个点的评分都很差，最优值自然不理想。

到 20 次，均值 1.828、最差种子 1.850，第一次稳定超过人工。40 次之后曲线完全平了（1.802 与 1.825 在 40 与 60 次时一模一样），说明这个五维问题里“值得找的点”在 40 次内就被采样到了。

这张图对工程的含义有两层。第一层是**小预算搜索要谨慎**：如果只能跑十几次仿真，不如按 67.8 节的顺序手工调。第二层是**收益有上限**：40 次之后继续加预算没有意义，该改的是搜索空间或者评分函数，不是次数。

搜索箱体的边界问题

本章最终定稿的 $E = 1.60$ 恰好落在搜索区间的上界 $[0.4, 1.6]$ 上。这是一个应当引起警惕的信号：**最优点落在箱体边界，通常意味着边界设错了**，真正的最优可能在箱体外面。

但这次是例外。67.7 节的表 67-4 显示，E 从 1.60 再往大走到 2.08，超调从 1.09% 暴涨到 14.16%。也就是说箱体边界恰好挡在一片坏区域前面，属于运气好，不是设计好。如果当初把上界设成 2.4，搜索结果很可能会选到 E 接近 2.0 的点——那个点在短轨迹上的评分可能不错，但在四段全程上会翻车。这也说明**搜索用的短轨迹必须覆盖最难的工况**：本章的短轨迹包含第三段（2→3），但去掉了第四段（3→1），而第四段正是超调最严重的一段。用短轨迹选参数，再用全程复核，是必要的两道关。

四条改进方向

本章的模糊层已经能跑，但仍有明显短板（延迟不抗、噪声下变差偏高）。按投入产出比排序，有四条改进路线。

第一条是把规则表从“手写”换成“学习”。 本章的 49 个后件是手工填的，虽然经过两轮修正，但仍是设计者的直觉。可以把它变成一个优化问题：给定轨迹数据，用最小二乘拟合后件（ANFIS 的思路），或者用遗传算法搜索离散后件。代价是需要数据、需要防过拟合，而且学出来的表往往失去可解释性。第 68 章会给出用网络直接替代规则表的结构，第 70 章的强化学习则可以把后件当作可搜索的策略参数。

第二条是尺度的在线自适应。 本章的 E、D 是常数，工况大范围变化时（比如对象增益变化 5 倍以上），固定尺度会让轨迹在某些工况下整体偏到规则平面的一侧。可以在外层加一个慢速的自适应环，根据长期误差统计缓慢调整 E。风险是内外两个环的时间尺度必须拉开，否则会互相激励。第 66 章的工况层其实就是一种离散版的自适应，两条路线可以合并。

第三条是与预测环节结合，对付延迟。 本章的裕度只有 14.7°，延迟一加倍就失稳，而模糊层对此无能为力——它只知道“现在偏差多大”，不知道“这个偏差是多久以前的”。把 Smith 预测器（第 47 章）或者模型预测（第 58 章）串在前面，让模糊层看到“预测的未来偏差”，才能实质性地改善延迟鲁棒性。这一条是本章最值得做的改进。

第四条是降维。 五个参数里只有两个真正重要（67.7 节），这说明问题本身的有效维度不高。既然如此，可以把模糊层只加在 K_p 上，K_i、K_d 直接用常数：规则表从三张减到一张，参数从 5 个减到 3 个，调参工作量降一半。代价是失去对微分通道的调节能力，超调可能回不去 1.09%。这条改进适合工程化阶段——先把结构做简单，再按需加通道。

改进的优先级

如果只能做一件事，做第三条（对付延迟），因为它对应的是真实风险，而不只是指标好看。如果能做两件，加第四条（降维），因为它降低的是长期维护成本。

第一条（学习规则）与第二条（在线自适应）都涉及“系统在运行中改变自己的行为”，需要额外的稳定性保障与验证手段，适合在有充分试验条件时做。本书把它们放在后续章节，而不是在入门章里展开，原因也在这里：**结构越复杂，出问题时越难定位**，而定位能力在工业现场比性能更重要。

排优先级时还有一个容易被跳过的前提：先确认短板真的在那。本章的四段指标里，第三、四段的超调已经压到 1% 附近，真正的短板是延迟与噪声；如果继续在规则表上花力气，收益会很快饱和。这条判断本身也要靠数据支撑，不能凭印象——把四段指标与三种劣化工况摆在一起看，哪一列最难看，就先改哪一列。

把后件当参数估：最小二乘为什么不够

“学习规则表”听起来很先进，落到数学上其实是一个参数估计问题：后件 c_{ij} 是未知数，前件（隶属函数）固定，则式 (67-7) 对 c 是线性的。给定一批样本 $(e_k, \dot{e}_k)$ 和对应的“期望修正量” y_k，写成矩阵形式就是 $W c = y$，其中 W 的每一行是该样本的 49 个权重。

本章的规模是 49 列、三张表共 147 个未知数。若采集 500 个样本，最小二乘一步就能解出来，计算量微不足道。真正的问题不在于解方程，而在于 y 从哪来。

y 必须是“在这个状态下，比例修正应该是多少”。这个知识要么来自离线整定（用优化器在仿真里搜出一组最优的 (K_p, K_i, K_d) 轨迹，再反推 Δ），要么来自专家标注。前者把问题转回 67.9 节开头的搜索，只是把搜索空间从 5 维换成了 147 维；后者需要人逐点判断，成本更高。换句话说，**监督学习没有消除设计工作，只是把设计工作挪到了“造标签”这一步**。

这也解释了为什么工程上真正落地的“学习型模糊控制”不多：不是算法不会写，而是标签太贵。ANFIS 一类的方法把前件参数也纳入学习，自由度更大，对标签的要求也更高。

压缩：能省多少，代价是什么

49 条规则的管理成本不低——每条要有语义、要能翻译成人话、要在变更时被复核。压缩有三条路，代价各不相同。

一是降分辨率。 67.9 节开头的实验已经量化：3 档 9 条规则，超调 9.02%，而 7 档 49 条是 1.09%。这条路的代价最清楚，就是超调。

二是非均匀网格。 大偏差区的格子经常被截断，区分 PB 与 PM 没有意义，可以把外圈合并；小偏差区则相反，需要更细。这样能在总格数不增的前提下提高有效分辨率。代价是实现变复杂：隶属函数的中心不再等距，单位分解要重新验证。

三是低秩近似。 把 7×7 的表看成矩阵，做奇异值分解，只保留前几个奇异值对应的分量。若前三个奇异值就占了 95% 的能量，说明这张表本质上是“行效应 + 列效应”的低秩结构，可以用两三个一维函数张成。代价是可解释性——分解之后每个分量不再对应“某一行某一列”，而是全局模式，规则表就不再是“一句话一条”了。

本章没有做压缩，因为 49 条已经在可维护范围内。若规则数涨到 200 以上（比如三输入），压缩就从“可选优化”变成“必须做”，否则没人能复核这张表。

67.10 工程落地

从仿真搬到现场要改的五件事

本章的代码是理想化的：双精度浮点、零噪声、延迟固定、执行器无死区、量程与论域严格对齐。搬到现场至少要改五处。

一是量程对齐。 偏差 e 的单位是工艺量（摄氏度、毫米、转每分），而模糊层的输入是归一化的 -1 到 1。这个换算必须显式写成配置项，不能散落在代码里。若量程改过而 E 没跟着改，等效于把修正强度整体放大或缩小，现场表现就是“换了个传感器之后突然超调”。

二是执行器死区与回差。 现场的执行器（阀门、变频器）通常有 1% 到 2% 的死区，还有机械回差。模糊层的修正量在接近零点时会来回跨格，若死区足够大，这些跨格根本传不到执行器上，控制量变差看起来不大；若死区很小，跨格就会变成可见的抖动。落地时要先测出死区，再决定 S_d 和微分限幅。

三是计算延迟。 仿真里计算是瞬时的，现场计算要占一个采样周期甚至更多。本章 67.5 节算过，延迟从 0.05 s 加到 0.10 s，裕度就从 14.7° 掉到 -21.6°。若现场控制器的计算加通信占掉 20 ms，那就是把原本已经很薄的裕度再削一刀。

四是定点化。 67.5 节列的三条注意（权重和、32 位中间量、限幅与冻结共用 $u(k-1)$）在现场都要落实。

五是参数持久化。 49 个后件加 5 个尺度，若存在 RAM 里，一次断电就回到默认值。要存进非易失存储，并加校验和——现场最常见的故障不是算法错，而是参数被误改或存储损坏后系统仍“带着错参数跑”。

隶属函数的现场整定只有三招

现场没有仿真环境，整定手段比实验室少得多。能用的其实只有三招，而且都与隶属函数的形状无关。

第一招是看阶跃定 E。 做一次从常用工况到另一工况的阶跃，看超调。若超调比改造前还大，说明修正方向不对（先查符号），或者 E 太大导致修正太弱；若控制量长时间贴着限幅，说明 E 太小，修正太猛。E 每次调 10%，最多调三轮。

第二招是看稳态噪声定 D 与 S_d。 系统稳定后观察控制量：若它小幅高频抖动，先把 S_d 减半，再把 D 加大 20%。本章的经验是 S_d 对抖动最敏感（表 67-4 里 S_d 加大 30% 让变差从 27.9 涨到 31.1）。

第三招是看长期漂移定 S_i。 若稳态误差消得慢，加大 S_i；若出现低频摆动（周期几秒到几十秒），减小 S_i 或者检查积分限幅。

三招都不涉及“改隶属函数形状”。这不是因为形状不重要，而是因为形状的影响是全局的、平滑的，现场没有工具去分辨“是形状不对还是尺度不对”。把形状固定成三角形、把重叠度固定成 50%，剩下的全部交给尺度，是工程上唯一可控的做法。

规则爆炸怎么规避

规则数随输入维数指数增长：m 个语言值、n 个输入，规则数是 m^n。本章 2 个输入、7 个语言值，49 条；加一个输入就是 343 条；加到 5 个输入就是 16807 条。这不是“多写几条”的问题，而是验证成本、维护成本、可解释性同时崩掉。

现场有四条规避手段，按推荐顺序排列。

一是先降输入维数。 问一句“这个输入真的能测到吗、真的独立吗”。很多设计里第三个输入（比如“设定值变化率”）与第一个输入高度相关，加进去只会让规则表变稠密，不会带来新信息。本章只用了两个输入，就是这条原则的体现。

二是减少语言值。 67.9 节已经证明 3 档（9 条）在超调上差一些但完全可用。若某回路对平滑性要求不高，用 3 档或 5 档，规则数立刻降一个量级。

三是分层。 把 n 个输入拆成两级：第一级用两个输入算一个中间量，第二级再用中间量与第三个输入算输出。每级的规则数是 $m^2 + m^2$ 而不是 m^3，本章的两输入结构其实就是“只用一级”的特例。第 66 章的工况层加修正层也是分层，只是分的是参数而不是输入。

四是稀疏规则表。 只填实际会用到的格子，其余留空（不激活）。本章的轨迹只走过规则平面的一部分，若统计一下激活过的格子，可能不到一半。空格的代价是插值时权重要重新归一化，且空白区域的行为要单独定义（通常退回基参数）。

这四条手段的共性是把“组合爆炸”变成“组合可控”，代价是表达能力的损失。工程上这个交换几乎总是值得的：**一个能验证的 9 条规则，胜过一个无法验证的 343 条规则**。

可解释性交到什么程度

模糊控制常被宣传成“可解释”，这话只对了一半。规则表本身是可读的：`if e is PB and de is NS then ΔKp is NM` 这句话任何人看了都能懂。但规则表之上还压着插值、量化因子、乘性叠加三层，最终行为并不是“一条规则说了算”，而是 49 条规则按隶属度加权的结果。

所以交付时的可解释性应当分层说明，而不是笼统地说“规则透明”。

第一层是单条规则的语义，这是能逐条写进文档的。本章 49 条规则，每条一句话，大约 1500 字。

第二层是局部行为，也就是“在某个工作点附近，系统倾向于做什么”。这一层可以用图 67-4 的等效增益曲面说明：偏差大、变化率同号时增益高，偏差大、变化率反号时增益低。这句话能覆盖大部分现场疑问。

第三层是全局行为，也就是“为什么这个回路整体稳定”。这一层无法靠规则表解释，只能靠 67.6 节的裕度检验与仿真。诚实的说法是“我们验证过，但没有证明”。

给操作员的说明书只需要第一层和第二层，给设计评审的材料需要三层齐全。把三层混在一起讲，结果是操作员记不住、评审专家不买账。

上线检查清单

表 67-9 是本章建议的上线前检查项，每一项都有明确的通过判据与不通过时的处理。

检查项	判据	不通过时
退化检验	$S=0$ 时指标与改造前一致（误差 <1%）	查归一化、权重和、积分状态
符号检验	第二输入为 $-\dot{y}$ 而非 $\dot{e}$	改代码，重跑阶跃
量程对齐	E 与实际量程之比与设计一致	改配置，不改代码
饱和守卫	阶跃时积分不积爆（退出饱和后无二次超调）	检查 $u(k-1)$ 取值位置
下行守卫	大幅负阶跃超调 < 5%	检查 `DESC_LIMIT` 是否生效
延迟复核	实测延迟代入后裕度 > 20°	降 K_p 或缩短延迟
参数持久化	断电重启后参数不变，校验和正确	加非易失存储与校验
回退通道	一条命令可切回固定参数	补开关，并把线性参数写进文档

清单里最容易被跳过的是“延迟复核”和“回退通道”，而这两项恰恰是出事时最要命的。延迟复核要在现场实测（用一次阶跃加一次反向阶跃估），不能沿用仿真值；回退通道要在调试阶段就建好并演练一次，不能等出事再找。

出问题时先回退，再定位

现场出问题的第一反应不该是“调参”，而是“回退”。本章的结构天然支持这一点：把三个 S 因子置零，模糊层就退化成固定参数 PID，系统回到改造前的行为——这是 67.8 节退化检验的另一个用途。

回退之后再做定位，顺序建议是：先查测量（传感器、滤波、量程），再查执行（饱和、死区、卡涩），最后才查控制器（规则表、尺度、守卫）。这个顺序的依据是故障率：现场九成的“控制器不稳”最终都追到测量或执行环节，规则表本身出问题的概率很低——毕竟它只是一张静态的表，不会自己变。

如果定位到确实是规则表的问题，改法也要克制：一次只改一格或一个尺度，改完做一次完整的四段测试。模糊控制最怕的不是“规则写得不准”，而是“改完之后没人说得清是改对了还是碰巧”。

首次投运的三次阶跃

新回路投运时不要一上来就跑全工况。建议按“空载、半载、满载”各做一次阶跃，每次只跳一小段（比如工作点 1→1.5），记录三个数：超调、调节时间、控制量峰值。

三次阶跃的目的是**核对基参数**，不是核对模糊层。模糊层的修正量最多只有 $\pm S_p$ 的幅度，撑不起一个整定错误的基参数。若某次阶跃的超调明显偏大，先去看那个工况的基参数是否按该工况整定过，而不是去改全局的 S_p。

判据可以粗一点：每次阶跃的超调若在 5% 以内、控制量峰值没有持续贴限幅，就算通过。三次都通过再跑大阶跃（跨两个工况），最后才做连续迁移。这个顺序与 67.8 节的“先线性后模糊”是同一个思路：每次只让一个变量变化，出问题才能归因。

还有一个容易被忽视的检查：**空载与满载的基参数是否差别很大**。若差别超过 1.5 倍，说明工况层分得不够细，或者对象的非线性比预期强。这种情况下模糊层的乘性修正会显得力不从心——它能覆盖 $\pm 55\%$，覆盖不了 2 倍以上的差异。该做的是加一个工况档，而不是加大 S_p。

规则表的版本管理

49 格加 5 个尺度，一共 54 个数。这些数在调试期间会反复变，若不管理版本，三个月后就没人说得清现场跑的是哪一版。

最小可用的做法是三条。第一，规则表与尺度一起编版本号，写进参数区的固定偏移，并在启动日志里打印。第二，每次变更记录“改了哪一格、为什么改、改前改后的四段指标”，附在版本号后面。第三，参数区加校验和，启动时校验失败就拒绝运行并报警，而不是悄悄用默认值。

这三条听起来与算法无关，但它们决定了这台设备三年后还能不能被维护。现场最常见的故障不是“规则表写错”，而是“有人改了一格没记录，半年后指标漂了，没人知道为什么”。

现场最容易出的三个误判

第一个误判是把“规则读得通”当成“控制器对”。 规则表用语言写出来总是通顺的：偏差大就加推力，偏差小就收一点，超调了就加阻尼。正因为它听起来都对，评审时很难发现问题。但语言对不等于数值对——“大”到底对应 0.6 还是 0.9，乘以 0.55 之后是放大 33% 还是 50%，这些只有算过才知道。唯一可靠的证据是退化检验与阶跃响应：把三个 S 置零能不能复现 PID，加上修正之后四段指标各是多少。

第二个误判是把“阶跃漂亮”当成“裕度够”。 一次阶跃只走一条轨迹，从零到设定值，路径单一。增益曲面在别的区域可能完全是另一个样子——第四段的大偏差反向就是典型，那里曲面的值比第一段高得多。67.6 节的顶点检验与 67.4 节的描述函数，作用正是补上“没走到的那部分”。只凭一张阶跃曲线签字，等于只验证了曲面的一条线。

第三个误判是把“这台好用”当成“可以复制”。 规则表可以复制，尺度不能。E 是按最大偏差定的，S_p 是在某个基参数上调出来的，换一台设备、换一个量程、甚至换一个工况档，这两个前提都变了。复制过去之后先跑退化检验，再跑四段阶跃，最后才谈要不要微调；跳过前两步，得到的“感觉差不多”往往掩盖了几十个百分点指标差异。

三个误判的共同点是：它们都用一次观察代替了一轮验证。模糊控制的可解释性让人容易满足于“讲得通”，而工程上需要的是“测过”。

67.11 优缺点与适用边界

四种工况下的表现

前几节的指标都在理想工况下取得。表 67-10 把测量噪声、延迟加倍、限幅收紧三种劣化工况加进来，模糊与线性各跑一遍，共用同一套守卫。

工况	控制器	全程 IAE	最大超调	控制量变差
理想	模糊	2.698	1.09%	27.9
理想	线性	2.997	50.23%	36.6
测量噪声	模糊	2.688	1.62%	147.8
测量噪声	线性	2.989	49.97%	114.5
延迟加倍	模糊	4.096	29.16%	106.5
延迟加倍	线性	54.346	43.25%	64.0
限幅 ±0.8	模糊	3.102	1.14%	21.9
限幅 ±0.8	线性	2.893	30.28%	24.7

图 67-10 把其中两种画成时域曲线：上半是噪声下的控制量，下半是延迟加倍下的输出。

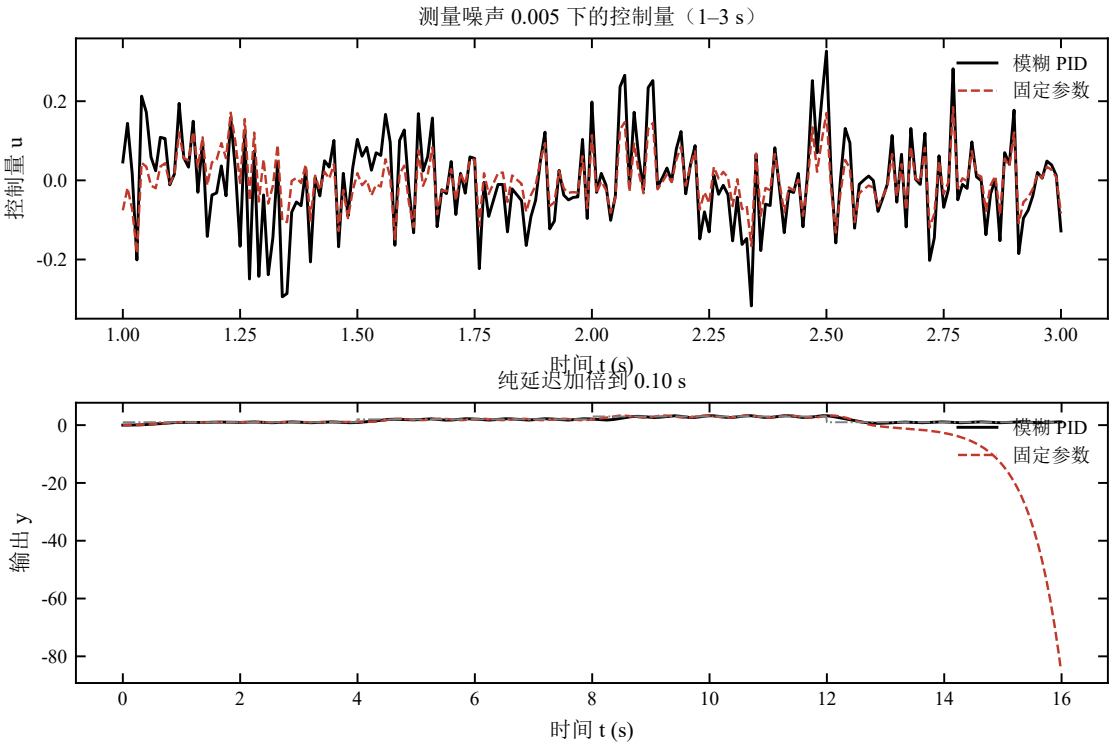


图 67-10 劣化工况：噪声下的控制量与延迟加倍下的输出

这张表要逐行读，因为四行的结论并不一致。

噪声这一行，模糊并不占优。 IAE 2.688 对 2.989 仍然领先，超调 1.62% 也远好于 49.97%，但控制量总变差 147.8 反而高于线性的 114.5。原因是连续插值在放大噪声：偏差每抖一点，隶属度就跟着抖，三个修正量一起抖，控制量自然更毛躁。第 66 章的专家控制在同样噪声下是 162.8 对 114.5，比本章还差——因为它的分段常数在阈值附近会跳变。两章都输给线性，说明“用状态做非线性反馈”这件事本身对噪声不友好。

延迟加倍这一行，两者都崩了。 模糊的超调 29.16%、线性 43.25%，都不能接受；线性的 IAE 54.346 更是离谱，因为它在 50% 超调的基础上又叠加了延迟引起的振荡。这一行不能用来说明“模糊抗延迟”，只能说明“线性更早崩”。67.5 节算过，延迟加倍后线性裕度 -21.6°，本来就该不稳定。

限幅收紧这一行，模糊反而更差。 IAE 3.102 对 2.893，这是全表唯一一处模糊输给线性的地方。原因很具体：低工况基参数 $K_p = 3.80$ 比线性的 2.60 高得多，限幅压到 ± 0.8 之后更容易饱和；而饱和期间模糊层的修正根本传不出去，等于白算。这一行是本章最诚实的反例：**模糊层不能补偿执行器能力的不足。**

优势在哪里

把上面的结果归纳一下，模糊自整定的优势集中在两处。

第一处是大偏差、大范围迁移时的平稳性。 第四段 3→1 的超调 1.02% 对 50.23%，第三段 1.09% 对 11.86%，这是本章最硬的收益。机理在 67.4 节已经拆过：接近目标时比例收、微分加，等效阻尼随偏差变化，而固定参数只能取折中。

第二处是对对象增益变化的适应。 本章对象的增益从 10 变到 22（$K(y) = 4 + 6y$，y 从 1 到 3），变化 2.2 倍。固定参数只能按中间值整定，两端都偏离；模糊层用偏差与变化率间接感知“现在增益偏高还是偏低”，并相应调整。这种适应是**被动**的——它不辨识增益，只根据响应形态反应，所以比自适应控制慢，但也更稳、更简单。

还有一处隐性优势：**参数可以写成百分比**。$S_p = 0.55$ 这类参数与工况基值解耦，换一个对象时基参数重调、尺度大体可以沿用。第 66 章的乘子化也是这个思路，两章在这一点上是一致的。

劣势在哪里

第一是延迟。 这是本章最严重的短板，而且不是调参能解决的。模糊层是“无记忆”的静态映射，它不知道当前偏差是多久以前产生的，所以对纯延迟没有任何免疫力。本章的裕度只有 14.7°，延迟一加倍就失稳。要解决只能靠预测（第 47 章 Smith、第 58 章 MPC）。

第二是噪声。 控制量变差 147.8 对 114.5，说明模糊层把测量噪声放大了 29%。这个放大有两个来源：微分通道（$\dot{y}$ 由差分得到，噪声被放大 $1/T_s$ 倍）与插值（隶属度对输入的变化率不为零）。前者可以通过加强滤波缓解，后者是结构性的。

第三是执行器受限时无能为力。 限幅 ±0.8 那一行已经说明。模糊层能做的只是“在能力范围内分配得更好”，不能让不够的能力变得够。

第四是验证成本。 线性系统可以算裕度、画 Nyquist、做灵敏度分析；模糊系统只能逐点冻结参数算，再靠仿真扫工况。本章用了 147 个顶点、四段轨迹、四种工况，仍然不能说“证明了稳定”。

与专家控制的对照

第 66 章与本章用同一个对象、同一套守卫、同一组工况，正好可以做一次干净的对照。表 67-11 是两章定稿指标的并列。

项	专家控制（第 66 章）	模糊控制（本章）
全程 IAE	2.625	2.698
最大超调	8.22%	1.09%
控制量总变差	43.2	27.9
规则/参数数量	4 分区 × 3 乘子 + 3 层守卫	49 规则 + 5 尺度
参数是否连续	否（分段常数，需滞环）	是（需单位分解）
单条规则可验证	是	只能验证方向
调参维度	21 个（12 乘子 + 9 基参数）	5 个尺度

两章的结论正好互补：**专家控制更快（IAE 低 2.8%），模糊控制更稳（超调低 7 个百分点、变差低 35%）**。这不是谁优谁劣，而是两种结构各自的偏向：分段常数在“该使劲时使劲”，插值在“该收时收得住”。

选择时看工艺更在意什么。若回路的主要矛盾是“消差慢”，专家控制更合适；若主要矛盾是“超调撞限位”，模糊控制更合适。若两者都重要，第 72 章的混合架构会把两者叠起来——用模糊层做平滑基底，用专家规则做过限保护。

适用边界的五条判据

把本章的经验压缩成五条可操作的判据，满足三条以上再考虑上模糊。

一是对象增益随工况变化超过 2 倍。 本章是 2.2 倍，收益明显。若只变 1.3 倍，鲁棒整定足够。

二是存在大幅设定值迁移，且超调有硬约束。 本章第四段从 3 降到 1，线性超调 50%，这就是典型场景。

三是纯延迟小于对象时间常数的十分之一。 本章 0.05 s 对 0.5 s，正好在边界上，所以裕度只剩 14.7°。延迟再大就不该用模糊，该用预测控制。

四是执行器有足够余量。 限幅收紧的实验说明，能力不足时模糊层帮不上忙。

五是团队能维护 49 条规则。 这不是技术判据，是组织判据，但它经常是决定性的——一张没人看得懂的规则表，比一个普通的 PID 更危险。

三类不该用的场合

第一类是**快速随动系统**。伺服轴、飞行器等对象的带宽要求高，延迟占比大，模糊层的平滑性换不来带宽，反而增加相位滞后。

第二类是**安全关键且需要形式化验证的回路**。模糊控制目前没有成熟的定理证明工具链，做不了“证明无超调”这类保证。这类回路该用线性方法加鲁棒分析（第 106 章）。

第三类是**工况几乎不变的回路**。定值调节、温度恒温这类对象，固定参数 PID 整定好之后可以十年不动，上模糊只是增加故障点。67.8 节已经给过这条止损判据，这里再强调一次：**没有工况变化，就没有自适应的必要**。

三类之外还有一种容易被忽略的情形：**没有人能说清“为什么这么写规则”的场合**。模糊控制的价值一半在算法，一半在“规则可以被操作员读懂并参与修改”。若规则表是某次自动搜索生成的、后件是一堆小数点后三位的数，那么它与一张查表没有区别，却比查表多了三层代码。这种时候不如直接做增益调度：同样的效果，参数是显式的，评审时也能逐条对照。

67.12 Python 仿真代码

两个文件与运行方式

本章的全部数字与插图来自两个脚本：

```
book/code/ch067_sim.py     仿真主体（对象、隶属函数、规则表、闭环、指标、六组实验）
book/code/ch067_figs.py    十张插图（框图用 block_styles，数据图用 matplotlib）
```

运行方式：

```
"D:\Anaconda3\envs\py311\python.exe" code/ch067_sim.py
"D:\Anaconda3\envs\py311\python.exe" code/ch067_figs.py
```

前者在终端打印六组结果（模糊对线性、规则条数、顶点裕度、四种工况、整定预算、增益范围），后者把十张图导出到 `figs/ch067/`，每张一份 SVG、一份 PDF。两个脚本都不接受命令行参数，所有可调量集中在文件开头的常量区，这是本书统一的写法。

`ch067_figs.py` 依赖 `ch067_sim.py` 的函数与常量，所以改参数只需要改一个文件。图脚本本身不重算任何指标，只做两件事：调用仿真函数取数据，然后画图。这样可以保证“正文数字”与“图上数字”来自同一处。

仿真主循环的骨架

`simulate(kind, ...)` 是唯一的主循环，模糊与线性共用，只靠 `kind` 参数切换参数来源。这个设计是本章对比公平性的基础——若两条曲线各写一个循环，抗饱和、滤波、延迟任何一处不同，指标就没法比。

每拍的执行顺序如下，顺序本身有讲究：

1. 采样得到 y_k，算 $e_k = r_k - y_k$；
2. 差分并一阶低通，得到 $\dot{y}_f$（时间常数 0.05 s）；
3. 由 y_k 判工况，得到基参数；模糊版再算 $(e_k, -\dot{y}_f)$ 的三个修正量；
4. 算微分项并限幅，再与比例、积分相加；
5. 总限幅；
6. 下行守卫（大负偏差时限速并冻结积分）；
7. 饱和守卫（判据用上一拍的 u）；
8. 积分累加（若未被冻结）；
9. 把 u_k 推进延迟缓冲，取 5 拍前的值送对象；
10. 对象用 5 次子步积分。

第 6 步在第 7 步之前，是因为下行限速本身会制造“伪饱和”；若先判饱和再限速，被限速压住的那一拍不会被判为饱和，积分就会在这段时间偷偷累加。这个顺序错误很隐蔽，指标只会差一点点，但 67.3 节提到的“守卫方向相反”的现象就消失了。

模糊层的五个函数

模糊层只有五个函数（其中一个是内部辅助），加起来不到 60 行，表 67-12 是清单。

函数	作用	返回
`mf_tri(x, n)`	n 个等距三角形的隶属度	长度 n 的向量，和为 1
`_table(rows)`	把语言值表转成数值矩阵	$n \times n$ 浮点矩阵
`rule_tables(res)`	按分辨率采样三张规则表	三个矩阵
`fuzzy_delta(e, de, res)`	前件模糊化、外积、加权平均	三个修正量
`fuzzy_gains(e, de, mode, res)`	乘性叠加到基参数	(K_p, K_i, K_d)

`mf_tri` 里最要紧的一行是半宽 `h = 2.0/(n-1)`。改小会让论域出现隶属度真空（67.7 节），改大则要额外归一化。函数末尾保留了 `mu / s` 的归一化，但在这个半宽下 `s` 恒为 1。

`rule_tables` 允许 `res` 取 3、5、7。取 7 时直接返回手写表；取 3 或 5 时用手写表做双线性采样。这样 67.9 节的规则条数实验才是“同一张表的不同分辨率”，而不是“三张不同的表”，比较才有意义。

`fuzzy_delta` 用 `np.outer` 做外积，一次得到 49 个权重，再与三个矩阵分别做点乘求和。写成 49 次循环也可以，但外积写法更短，也更容易看清“权重和为 1”这个性质。

`fuzzy_gains` 是唯一知道工况基参数的地方。模糊层本身只输出无量纲的修正量，与工况解耦——这正是 67.11 节说的“参数可以写成百分比”在代码上的体现。

实验函数与出图脚本

六个实验函数各自对应正文的一节，都是“跑若干次 `simulate`，汇总成表”的简单结构。

`vertex_margins` 遍历 3 个工况 × 49 个顶点，每个顶点用 `simlib.pid_continuous` 拼出开环、算相位裕度。它**不含延迟**，这一点在 67.4 与 67.5 节都标注过，读结果时要自己减去约 36°。

`exp_rule_resolution` 依次用 3、5、7 分辨率跑全程，返回三组指标，对应表 67-7 与图 67-8。

`exp_budget` 在五维箱体里随机取点，用短轨迹评分，记录“到第 b 次为止的最优值”，三个种子各跑一遍，对应表 67-8 与图 67-9。它的评分函数写在 `_eval_scaling` 里，是 $\mathrm{IAE} + 0.3\times$ 超调。

`exp_robustness` 跑四种工况各两遍（模糊、线性），对应表 67-10 与图 67-10。

`effective_gain_eval` 在 41×41 的网格上算等效 K_p，用来画图 67-4 的曲面，也用来核对 S_p 给出的 $\pm 55\%$ 上下界。

出图脚本的十张图与正文图号一一对应：图 67-1 用 `block_styles` 的语义方块画闭环结构，其余九张都是 `matplotlib` 数据图（热图、等值面、曲线）。所有图内中文都是纯文本，公式单独用 `$...$`，两者不写进同一个字符串——这是本书的硬规矩，否则 PDF 里的中文字体会被数学字体替换。

三个指标的口径

正文反复引用 IAE、超调、总变差三个数，它们的算法必须写清楚，否则同一句“超调 0.8%”在不同人手里会算出不同结果。

分段 IAE 用 `segment_metrics` 计算。每一段单独取起点：第一段以零为初值，第二段以下阶跃前的稳态为初值，第三、四段同理；下阶跃那段先把曲线与设定值一起取负，翻成上阶跃的形状再算。这样四段可以逐段比较、也可以直接相加，不会因为前一段的残差渗进后一段。公式就是 $\mathrm{IAE} = \sum_k |r_k - y_k| T_s$，没有归一化，单位是“工程单位·秒”。

超调以该段末 2% 的平均值作为稳态值，取峰值相对它的百分比，而不是相对设定值。有静差时两者不同：若稳态停在设定值的 95%，按设定值算会把静差混进超调。本章四段的静差都接近零（守卫与积分把最后一点偏差磨掉了），两种算法差不到 0.1 个百分点，但换成积分较弱的对象就必须区分。

调节时间取“进入并保持在 $\pm 2\%$ 带内的最早时刻”，之后再出带不算重新计时，直接用第一次的结果——这是 `simlib.step_metrics` 的既定口径，与第 44 章一致。若某段全程没进带，该项返回 `nan`，汇总时按缺失处理，不按零处理。

总变差是 $\sum_k |u_{k+1} - u_k|$，衡量执行器的往复量，作为磨损的代理。它与 IAE 必须一起看：把增益调大能把 IAE 压下去，代价往往是总变差翻倍；只看 IAE 会选出“指标好看但阀门很快磨坏”的参数。67.6 节说模糊的优势在第四段，那里的代价之一就是总变差略高，正文没有回避这一点。

想加一个新实验时改哪里

新增实验只在 `ch067_sim.py` 里写函数，不在出图脚本里写。约定是：实验函数返回字典或列表，`main` 里统一打印，`ch067_figs.py` 只负责调用与画图。这样“正文数字”与“图上曲线”永远同源，改了模型不会出现一处更新一处没更新的情况。

按这个约定，加一个“采样周期加倍”的实验只要写二十行：外层循环遍历 $T_s \in \{0.01, 0.02\}$，内层照样调用 `simulate`，把延迟拍数改成 `round(0.05/Ts)`，再跑 `segment_metrics`。注意 `simulate` 里的 `TS` 现在是模块级常量，要对它做参数化，得把步长从全局挪进函数签名——这是本章代码为简洁做的妥协，扩展时要先做这一步重构。

另外三类实验值得补：一是把噪声幅度按 0.005、0.01、0.02 扫一遍，看总变差先撑不住还是 IAE 先撑不住；二是把对象增益整体乘 0.8 与 1.2，检验乘性叠加的 $\pm 55\%$ 余量够不够；三是把规则表整体置零（全零后件），确认控制器退化成 PID 而不是崩掉。第三类是纯粹的接线检查，任何改动之后都值得跑一遍。

改代码时最容易踩的三处

第一处是半宽。 若把 `mf_tri` 的 `h` 改成 `1.0/(n-1)`（很自然的写法，因为中心间距是 `2/(n-1)`，半宽直觉上取一半），隶属度之间会出现空隙，加权平均不再是凸组合，67.4 节的顶点论证随之失效。这个改动不会报错，只会让指标悄悄变差。

第二处是规则表采样。 `rule_tables(5)` 必须从 7 档表采样，而不是另写一张 5 档表。若另写，67.9 节的“规则条数”实验就变成了“两张表对比”，结论不成立。

第三处是主循环共用。 给模糊版加任何新功能（新守卫、新滤波），都要同时确认线性分支没被影响；反之亦然。最稳妥的写法是把所有逻辑放在共用段，只在“取参数”那一行分叉——本章的 `simulate` 就是这么写的。

复现性

本章全部数字在固定种子（噪声实验 `seed=7`、预算实验 `20261020`）下可逐位复现。仿真步长 0.01 s、对象子步 5、延迟 5 拍、积分用前向欧拉，都是确定性的；`mf_tri` 与规则表也没有随机成分。唯一可能因平台而异的是 `control` 库在算裕度时的数值迭代，差异在 10^{-6} 量级，不影响任何结论。

若要把本章的代码用到别的对象上，改三处即可：`plant_lin` 的增益与极点、`gain_at` 的非线性、`MODE_GAIN` 的三组基参数。基参数建议用第 44 章的方法逐工况整定，再按 67.8 节的顺序调五个尺度。规则表可以不动——它表达的是“偏差大且远离目标时加推力、接近时加阻尼”这类与对象无关的经验，这也是模糊控制最值得保留的部分。

依赖只有三个：`numpy`、`matplotlib`、`control`。前两个用的是常规接口；`control` 只用到 `tf`、`feedback`、`margin` 三个函数，0.9 与 0.10 两个大版本之间这三者的调用方式没有变化，所以本章代码在两个版本下都能直接跑。不引入 `scikit-fuzzy` 之类的专用库，是因为模糊层的五个函数加起来不到 60 行，自己写反而更容易看清每一步在做什么，也少一个版本依赖。

唯一的平台相关项是中文显示。`styles/plot_styles.py` 里指定了宋体回退链，Windows 下正常，换到 Linux 需要把字体名改成系统里有的中文字体，否则图上的标签会变成方块。这个问题不影响计算，只影响出图，但很容易在交稿前才被发现，值得在第一次运行时就看一眼。

第68章 神经网络控制

第 67 章的 49 个后件是手工填出来的：设计者先想清楚“偏差大、变化率为负时应该加多少推力”，再把这句话翻译成一个 -0.3 或 $+0.6$。填完之后还要用四段轨迹验证、用顶点系统检查，改一格就要重跑一遍。如果手头已经有一批“好”的控制记录——比如第 67 章那套模糊 PID 跑出来的数据——能不能不问设计者、直接从数据里把这 49 个数拟合出来？更进一步：如果连“偏差、变化率、控制量”这三者的关系都可以直接学，那还要不要先写成 PID 的形式？

这两个问题就是本章的起点。答案不是“能，所以更好”，而是“能，但学到的东西通常不超过教师，学到多少取决于数据、结构与正则化”。把这句话拆开、用可复现的数字讲清楚，是本章的任务。

68.1 动机与直观理解

从“填表”到“拟合”

第 67 章的规则表是一张 7×7 的网格，格子里是三个修正量。若把这张表看成函数 $\Delta K = f(e,\dot e)$，它就是定义在 49 个节点上的分段线性插值。填表的过程本质上是在给这 49 个节点赋值；而一旦把“填表”换成“用一批样本拟合一个函数”，问题的性质就变了：不再需要设计者逐格判断，只需要给出输入输出对，剩下的交给优化。

这条思路可以再往前推一步。第 67 章的控制器结构是“模糊层改 PID 参数”，即网络的输出是三个增益；也可以让网络的输出直接是控制量 u，中间不再有 PID。前者叫参数型（网络只做整定），后者叫直接型（网络就是控制器）。本章两种都做，并用同一套对象、同一套指标与第 67 章对照。

需要提前说明的是**监督学习的天花板**。本章的直接控制网络用第 67 章的模糊 PID 作教师，标签就是教师的输出。这意味着网络最好也只能复现教师的行为；它在某些指标上超过教师，靠的是网络自身的平滑性去掉了教师的守卫与限幅带来的抖动，而不是“理解了控制问题”。这条边界在第 70 章的强化学习里才会被真正推开——那时标签不再来自教师，而来自回报。

网络是什么：一个带参数的函数

抛开术语，本章用到的网络就是两层复合函数：

$$u = \sum_{j=1}^{12} w_j \tanh\!\Big(\sum_{i=1}^{3} v_{ji} x_i + b_j\Big) + c \tag{68-1}$$

输入 $x=(e/E,\ -\dot y_f/D,\ \int e/I)$ 三维，隐层 12 个 tanh，输出一个标量。全部可调参数是 $3\times 12+12+12+1=61$ 个。训练就是找一组参数，让这 61 个数在给定样本上把误差压到最小。

这个形式与第 67 章的规则表其实是一回事的两面。规则表是“局部基函数的加权和”，网络是“全局基函数（tanh）的加权和”；前者可读、后者不可读，前者参数量随分辨率平方增长、后者随宽度线性增长。第 67 章那张表是 49 个后件加 5 个尺度，本章这个网络是 61 个权重。两者容量相当，这就是为什么后文的对照是公平的。

网络里唯一需要解释的选择是 tanh 而不是 ReLU。ReLU 在零点有折点，控制量会在输入穿过零点时出现斜率跳变；第 67 章讨论过“取小”造成的折线会让描述函数分析失效，这里是同一类问题。tanh 处处可导、输出有界在 $(-1,1)$，天然适合“输出要限幅”的场合。

本章算例与结果预告

算例沿用第 66、67 章的对象：$y'' = -2y' + K(y)u$，$K(y) = 4 + 6y$，即增益随工作点从 10 变到 22；采样 10 ms、纯延迟 5 拍、限幅 ± 1.2；设定值走四段 $1 \to 2 \to 3 \to 1$。训练数据是 8 条随机阶跃轨迹，共 16000 个样本，由第 67 章的模糊 PID 跑出来。

表 68-1 是四种控制器的实测结果，四段轨迹、同一套指标口径。

控制器	全程 IAE	最大超调	控制量总变差
固定 PID	2.997	50.23%	36.6
模糊自整定（教师）	2.698	1.09%	27.9
NN 直接控制	2.401	8.30%	41.1
NN 逆模型 + 反馈	2.429	0.86%	14.8

这张表里有三件事值得先看。第一，**直接控制网络的 IAE 比教师低 11%**，但超调从 1.09% 涨到 8.30%，总变差也从 27.9 涨到 41.1——它靠更“敢”的动作换来了更小的积分误差，而不是靠更聪明的判断；68.6 节会把这 41.1 定位到高工况平台期的一个小振幅极限环上。第二，**逆模型方案的总变差只有 14.8**，不到固定 PID 的一半，因为前馈承担了大部分功率，反馈只做小修正。第三，四种控制器的 IAE 都在 2.4 到 3.0 之间，差距远小于第 67 章固定 PID 与模糊之间在超调上的差距——这提示“换控制器结构”的收益，未必大于“把守卫与限幅做对”。

网络不是“更聪明的 PID”

初学者最常见的误解是：网络能自动发现 PID 想不到的控制律。本章的数据不支持这个说法。直接控制网络的 61 个权重，最终学到的映射在形态上就是一个“带积分记忆的非线性比例微分”——输入是 e、$-\dot{y}_f$、$\int e$，输出是 u，与 PID 的骨架相同，差别只在增益随状态变化。它没有引入新的信息通道，也没有预测未来，因此对延迟、对执行器饱和的态度与 PID 完全一样：延迟加倍时它的超调从 8.30% 涨到 20.82%，与模糊层的 29.16% 同属“扛不住”。

真正决定性能的是三件事，按重要性排序：**输入里有没有未来的信息**（本章没有，第 58、70 章有）、**执行器有没有余量**（限幅一收紧，谁都没辙）、**反馈通路有没有被守卫保护**（模糊层的守卫是它超调低的直接原因）。网络能改善的是“在给定信息下的映射精度”，这是第四位的事。

三个必须先回答的问题

在动手之前，有三个问题必须先有答案。它们不是“选哪种网络”这种技术细节，而是决定这条路走不走得通的前置条件。

问题一：标签从哪里来？ 网络学的是映射，映射的样本必须有人给。三个来源：已有的控制器（教师）、实测的输入输出对、仿真模型。三者的可信度依次下降，可获得性依次上升。如果三个都拿不到——既没有能用的控制器，也不允许做激励实验，还没有可信的模型——那这条路直接堵死，不必往下看。

问题二：特征里有没有让问题变成单值的信息？ 68.6 节会证明，本章的特征缺了工作点，导致标签不是特征的单值函数，误差有地板。这个问题在设计阶段就能发现：把特征与标签画成散点，看同一特征位置上是否对应多个不同标签。如果有，要么补特征，要么接受地板。事后调参是补不回来的。

问题三：网络失效时怎么办？ 这个问题必须在上线前回答，不能等出事再想。答案通常有三种：限幅（限制最坏后果）、弱反馈兜底（逆模型方案的做法）、切回传统控制器（双缓冲）。三种都要预先设计并测试，不能只写在文档里。

这三个问题的答案会直接改变后面的设计。本章的答案是：标签来自第 67 章的模糊 PID；特征不完备，接受地板（并在 68.9 节给出补救方案）；失效靠限幅加监测兜底，因为直接控制外面没有反馈可依赖。读者若换一个对象，这三个答案可能完全不同，后面的参数与结论也要跟着变。

本章的路线

图 68-1 是本章的结构：参考信号与测量值相减，构造出三个特征送进网络，网络输出控制量，经限幅与延迟作用到对象上。虚线部分是训练通路——教师离线跑出数据，标签进网络，运行时教师不参与。逆模型方案在图的下方多一条前馈支路：网络给出 u_{ff}，PID 只补残差。

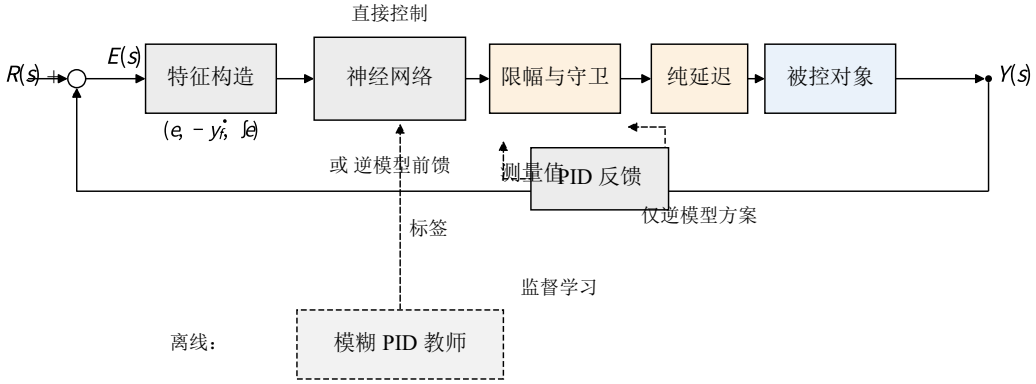


图 68-1 神经网络控制闭环结构

后面的安排是：68.2 节给出网络、训练目标与逆模型控制的数学；68.3 节讲数据从哪来、怎么划分、怎么做归一化；68.4、68.5 节分别在连续与离散两种视角下看这两种结构；68.6 节做稳定性与性能的对照；68.7 到 68.9 节讨论参数、调参与改进；68.10 节讲工程落地的三件事——训练数据获取、实时推理算力、模型更新；68.11 节给适用边界；68.12 节是代码。

68.2 数学原理

两种参数化：全局支撑与局部支撑

式 (68-1) 写成矩阵形式，把 3 维输入映到 12 维隐层、再压成 1 维输出：

$$H = \tanh\big(Vx + b\big), \qquad u = w^{\mathsf{T}} H + c \tag{68-2}$$

其中 $V \in \mathbb{R}^{12\times 3}$、$b \in \mathbb{R}^{12}$、$w \in \mathbb{R}^{12}$、$c \in \mathbb{R}$，共 61 个参数。tanh 逐元素作用。整个映射对参数是光滑的，对输入也是光滑的——这一点在 68.6 节讨论稳定性时会用到。

逆模型用的是另一种参数化，径向基函数：

$$u = \sum_{j=1}^{64} w_j \exp\!\left(-\frac{\|x - c_j\|^2}{2\sigma^2}\right), \qquad \sigma = 0.55 \tag{68-3}$$

中心 c_j 用 k-means 从训练数据里选，共 $64 \times 3 = 192$ 个数；输出权重 w 共 64 个，合计 256 个参数。注意这里的结构差别：MLP 的每个基函数都是全局的（tanh 在整个输入空间都有非零响应），RBF 的每个基函数只在中心附近有效。全局支撑意味着参数之间强耦合、必须迭代训练；局部支撑意味着“改一个中心附近的权重只影响那一块”，于是输出层对 w 是线性的，可以闭式求解：

$$w = \left(\Phi^{\mathsf{T}}\Phi + \lambda I\right)^{-1}\Phi^{\mathsf{T}} y, \qquad \Phi_{kj} = \exp\!\left(-\frac{\|x_k - c_j\|^2}{2\sigma^2}\right) \tag{68-4}$$

本章取 $\lambda = 10^{-4}$。这个 λ 的作用与 MLP 的权重衰减相同：$\Phi^{\mathsf{T}}\Phi$ 在数据稀疏的区域接近奇异，加一点对角项把它拉回可逆。

两种参数化的容量相当（61 对 256），但训练方式完全不同：一个靠梯度迭代、一个靠一次线性求解。本章把两者都放进算例，正是为了说明“网络”不是一个方法，而是一族参数化方式，选哪一种取决于你要的是可迭代（能在线更新）还是可复算（每次结果一致）。

训练目标：最小化的是代理指标

训练要最小化的是

$$J(\theta) = \frac{1}{N}\sum_{k=1}^{N}\big(u_\theta(x_k) - u_k\big)^2 + \lambda_w \|W\|^2 \tag{68-5}$$

第一项是网络输出与教师输出的均方误差，第二项是权重衰减。这个目标与“四段轨迹的 IAE”不是一回事，中间的错位有三层，值得逐层说清。

第一层是分布错位。 训练样本来自教师跑出的轨迹，而闭环运行时网络自己决定控制量，走出的状态分布与教师不完全相同。教师在某处收得快，网络可能收得慢，于是它见到的 $(e, \dot e)$ 组合是训练集里没有的。这是模仿学习里的经典问题，第 70 章用“专家纠正”（DAgger 类）缓解，本章不处理，只在 68.11 节列为边界。

第二层是指标错位。 均方误差对所有样本一视同仁，而闭环 IAE 由少数几个大偏差段主导。网络在大量“平稳段”样本上把误差压得再小，对第三、四段的超调也未必有帮助；反过来，为了压住某几个大偏差样本，它可能在平稳段引入抖动，总变差因此上升。68.1 节表 68-1 里直接控制网络“IAE 更低、变差更高”就是这个错位的直接表现。

第三层是稳定性错位。 均方误差里没有任何相位信息。两个网络可以在验证集上有相同的 MSE，但一个在闭环里稳定、另一个在延迟加倍时振荡——因为决定稳定的是“误差到控制量”这个映射在高频段的相位，而训练数据里几乎没有高频成分。

三层错位加起来，得到一个实践结论：**验证损失只能用来筛掉明显没训好的模型，不能用来选最终模型。** 68.6 节的所有取舍都以闭环指标为准，MSE 只作为参考列在旁边。

为什么是平方误差，而不是绝对值或 IAE

式 (68-5) 用的是均方误差，这个选择值得解释，因为它直接决定了闭环的行为。

从统计上看，平方损失等价于假设标签里叠加了高斯噪声，此时最小化平方损失就是极大似然估计。从控制上看，平方损失对大偏差的惩罚是二次的、对小偏差是线性的——这意味着网络会把容量优先用来压那些大偏差样本。这个偏好恰好符合控制的需要：大偏差危险，小偏差可以容忍。

换成绝对值损失，大偏差的权重就变成线性的，网络会更“平均”地对待所有样本，拟合出的映射在零附近更陡、在大偏差处更软。这对超调有利（零邻域陡），对 IAE 不利（大偏差段软）。68.9 节的“零邻域加权”其实就是

在平方损失的基础上手动实现这种偏好，而不是换损失函数——因为加权比换损失更容易控制力度。

那么能不能直接用 IAE 当损失？理论上可以，实践上不行，原因有三条。第一，IAE 要在闭环里算，每个样本的梯度都需要跑一次完整仿真，训练一次的代价从“一次矩阵乘法”变成“一千次仿真”。第二，闭环对参数的影响是间接的——网络输出变化一点，轨迹就变一点，梯度的信噪比极低。第三，闭环仿真在参数不好时会发散，损失变成 NaN，训练直接崩掉。这三条叠加起来，使得“用闭环指标直接训”在实践中几乎不可行；本书第 70 章讲强化学习时会看到，那套方法用了几十种技巧（经验回放、目标网络、奖励缩放）才勉强让这条路径可用。

还有一个小选择：损失里要不要除以样本数。本章除以了，得到的是“每个样本的平均平方误差”，量级在 10^{-2}，便于阅读；不除也可以，只是学习率要跟着缩放。这个选择本身没有对错，但训练代码与调参记录必须一致，否则换一份代码就复现不出同样的曲线。

RBF 的闭式解为什么存在

RBF 与 MLP 的差别不只是“径向基”与“S 形”这两种形状，更根本的是**参数出现在哪一层**。

MLP 的输出是 $u = \sum_{j} w_j \tanh(v_j^{\mathsf{T}} x + b_j) + c$，参数 v_j 藏在 tanh 里面，输出对 v_j 是非线性的，所以只能用梯度法一步步逼近。RBF 的输出是 $u = \sum_{j=1}^{m} \lambda_j \varphi(\|x - c_j\|/\sigma) + \lambda_0$，中心 c_j 与宽度 σ 定下来之后，输出对权重 λ 是**线性**的，线性最小二乘有闭式解 $\lambda = (\Phi^{\mathsf{T}}\Phi + \lambda_r I)^{-1}\Phi^{\mathsf{T}} Y$，其中 $\Phi_{kj} = \varphi(\|x_k - c_j\|/\sigma)$ 是设计矩阵。

这个解的代价是一次 $m \times m$ 矩阵求逆加一次矩阵乘法。$m = 64$ 时是 $64^3 \approx 2.6 \times 10^5$ 次浮点运算，毫秒级完成，且不需要学习率、不需要迭代、不会卡在局部极小。这是 RBF 在离线场合的最大优势——训练变成了一次线性代数。

代价有三条。第一，中心是两步选的（先 k-means 后回归），不是联合优化的，所以得到的不是全局最优，只是“给定中心下的最优权重”。第二，m 大了之后求逆代价按 m^3 增长，$m = 1000$ 时是 10^9 量级，离线还能忍，在线更新就不行了。第三，$\Phi^{\mathsf{T}}\Phi$ 的条件数随 m 增长很快，$\lambda_r = 10^{-4}$ 的作用主要是把它压住，防止求逆数值不稳——这也是为什么 RBF 几乎总带一个岭回归项，而不是纯最小二乘。

两条路线的选择因此很清楚：**要在线更新或数据流式到达，用 RBF（可增量求解）；要极致的参数效率或输入维数高，用 MLP（参数随宽度线性增长）。** 本章两个都用，正是为了让读者看到这个分工。

反传：链式法则只走三步

式 (68-2) 的梯度不难推。记 $\delta_o = 2(u_\theta - u)/m$（$m$ 是批大小），则

$$\nabla_w = \delta_o H, \qquad \nabla_c = \delta_o, \qquad \nabla_V = \big[(\delta_o w) \odot (1 - H^2)\big] x^{\mathsf{T}}, \qquad \nabla_b = (\delta_o w) \odot (1 - H^2) \tag{68-6}$$

其中 $\odot$ 是逐元素乘。$1 - H^2$ 是 tanh 的导数——这也是选 tanh 的另一个理由：导数只需用输出本身算出来，不必额外存一份。

有了梯度，更新规则用 Adam：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2, \qquad \theta_t = \theta_{t-1} - \eta \frac{m_t/(1-\beta_1^t)}{\sqrt{v_t/(1-\beta_2^t)} + \epsilon} \tag{68-7}$$

本章取 $\beta_1 = 0.9$、$\beta_2 = 0.999$、$\epsilon = 10^{-8}$、批大小 32、学习率 0.05、200 轮。用 Adam 而不是纯梯度下降，原因很实际：三个输入特征的量纲虽然都归一化到 ± 1，但它们的敏感度差很多（积分项变化慢、微分项变化快），各维梯度尺度相差一到两个数量级，统一学习率下要么积分项学不动、要么微分项来回跳。Adam 对每一维单独缩放步长，省掉了手工调各维学习率的工作。

代价是内存：每个参数要额外存一阶、二阶矩，61 个参数变成 183 个数。这在训练时无所谓，在嵌入式部署时值得记一笔——但部署时只需要前向，不带动量项，所以这条代价不影响实时性。

逆模型控制：为什么必须先把参考滤一遍

逆模型控制的想法很直接：如果知道对象的逆 $u = G^{-1}(y, \dot{y}, \ddot{y})$，那么把期望轨迹代进去就得到前馈控制量。本章对象的逆有解析形式

$$u = \frac{\ddot{y} + 2\dot{y}}{K(y)}, \qquad K(y) = 4 + 6y \tag{68-8}$$

这个式子说明了逆模型控制的两个前提：$K(y) \neq 0$（对象不失去能控性），以及 $\ddot{y}$ 必须存在且有界。第二个前提在阶跃设定值上立刻出问题——阶跃的二阶导数是冲激，直接代进式 (68-8) 会得到一个无穷大的前馈。所以参考信号要先过滤波器：

$$\ddot{x}_1 = \omega_n^2 (r - x_1) - 2\zeta\omega_n x_2, \qquad \dot{x}_1 = x_2, \qquad \omega_n = 6,\ \zeta = 1 \tag{68-9}$$

滤波后的 $(x_1, x_2, \ddot{x}_1)$ 就是 $(r_f, \dot{r}_f, \ddot{r}_f)$，三者都有界、都光滑。这一步不是可选的技巧，而是逆模型控制能成立的前提：**逆模型的定义域只包含光滑轨迹**，训练数据里的 $\ddot{y}$ 也是从实际轨迹差分来的，天然有界，网络从没见过冲激，也就不该被要求处理冲激。

前馈之外还要反馈，因为逆模型不可能完全准：

$$u = u_{ff} + k_p(r - y) - k_d \dot{y}_f + k_i \int e, \qquad u_{ff} = N(r_f, \dot{r}_f, \ddot{r}_f) \tag{68-10}$$

本章取 $k_p = 1.6$、$k_d = 0.30$、$k_i = 0.05$，比第 67 章的 PID 小得多（那里 $K_p = 2.60$）。这是逆模型方案的结构性优势：前馈承担主要功率，反馈只补残差，因此同样的跟踪精度下控制量总变差只有 14.8，不到固定 PID 的 41%。反过来说，前馈一旦失准（比如外推到训练范围之外），弱反馈就撑不住——68.6 节的外推实验会看到这一点。

一个必须承认的事实：模型已知时网络是多余的

式 (68-8) 已经给出了本章对象的精确逆。既然精确解就在手边，用 256 个参数去拟合它，除了演示“网络能学”之外没有工程价值。这一点必须说破，否则本章容易被误读成“逆模型控制必须用网络”。

网络在这里的价值只有一个：**当式 (68-8) 写不出来的时候，它能从数据里逼近同一个映射**。什么时候写不出来？对象含未建模的摩擦、死区、磁滞，或者 $K(y)$ 本身没有解析形式。此时解析逆不存在，而在对象上采集的输入输出对仍然存在，网络就有了用武之地。换句话说，网络的定位是“模型未知时的替代品”，不是“模型已知时的升级品”。

同理，本章的直接控制网络用教师做标签，教师就是模糊 PID。如果教师已经在手边、且性能可接受，那么训练一个网络去模仿它，唯一的收益是运行时的推理速度（68.10 节会给数字），代价是多了一套训练流程和一份不可读的参数。这笔交换值不值，取决于部署环境，不取决于算法先不先进。

为什么本章不做在线学习

神经网络控制常被赋予一个诱人的前景：让网络边控边学，自动适应对象变化，从此不需要建模、不需要重训。本章的方案明确不这么做——网络训练完就冻结，运行中一个权重都不改。这不是保守，是三条硬约束。

第一条：在线时没有标签。 监督学习需要一个“正确答案”，而闭环运行时唯一的反馈是标量性能（偏差、超调），它延迟、稀疏、且混着设定值变化的干扰。用这个信号去改权重，实质上是在做强化学习，样本效率比监督学习低两三个数量级——在线跑几个小时的数据量，抵不上离线采集十分钟。

第二条：更新过程本身不可验收。 冻结的网络可以用一整套离线与闭环测试来验收，验收通过再上线。而在线学习的每一次权重更新都是一个未经测试的新控制器，它的稳定性没有任何保证；更麻烦的是“哪一次更新把它弄坏了”无法回溯，因为权重一直在变。

第三条：会忘。 在线数据是按当前工况顺序到来的，网络会向最近的样本倾斜，把之前学到的区域覆盖掉。若对象在低工况下稳定运行一周，网络对高工况的记忆会明显退化，等到高工况再来时，性能比投运初期还差。这不是理论担忧，是增量学习的固有问题，缓解手段（经验回放、弹性权重固化）都要额外存储和额外计算，在 61 个参数的规模上不值得。

所以本章走的是“先学后用”：离线造数据、离线训练、离线验收、冻结上线，对象变了就回到第一步重训（68.10 节给了触发条件与影子模式）。这条路线牺牲了自适应能力，换来了可验证性。对本算例这种“对象基本不变、工况有限”的场合，这笔交换是划算的；若对象在运行中持续漂移（比如催化剂缓慢失活），那就必须考虑带约束的在线修正，但那是另一套设计，需要把“更新”本身也纳入稳定性分析。

68.3 设计步骤

第一步：先确定“学什么”

网络在这套结构里可以坐三个不同的位置，三者的数据要求、先验要求和失败方式完全不同，选错位置比调错参数严重得多。

第一种是**学整个控制器**，也就是直接控制：输入是 $(e, \dot{e}, \int e)$，输出是 u。网络替代的是第 67 章那张规则表加比例因子，外面不再有任何 PID。这种做法的数据只能来自一个已经在工作的控制器，因为没有教师就没有标签。它天然继承教师的水平上限，同时继承了教师的状态分布。

第二种是**学对象的逆**：输入是 $(y, \dot{y}, \ddot{y})$，输出是 u，作为前馈与一个弱反馈并联。此时网络替代的是对象的模型，不是控制器。它的数据可以来自任何把对象激励起来的实验，不必由某个教师产生，这一点在数据获取上自由得多。

第三种是**只学参数**：输入仍是 $(e, \dot{e})$，输出是 (K_p, K_i, K_d) 三个数，后面接标准 PID。网络很小（本章若这样做只需要不到 30 个参数），失败方式也最温和——最坏情况是参数停在一个不好的常数值上，闭环退化成固定 PID，而不是变成一个行为未知的非线性环节。

表 68-2 把三者并排。本章两种都做：直接控制走第一种，逆模型走第二种，目的是让读者看到两种失败方式长什么样。选择第一种还是第二种，判断依据不是“哪个更先进”，而是**你手上有什么数据**：只有教师轨迹就选第一种，能对对象做激励实验就选第二种。

对比项	直接控制	逆模型控制	只学参数
网络输入	$e,\ \dot{e},\ \int e$	$y,\ \dot{y},\ \ddot{y}$	$e,\ \dot{e}$
网络输出	u	u_{ff}	K_p, K_i, K_d
数据来源	教师闭环轨迹	对象激励实验	教师闭环轨迹
外环反馈	无（全靠网络）	弱 PID 补残差	标准 PID
典型失败	离开教师分布即失准	前馈失准、反馈撑不住	参数卡死，退化为定常
可解释性	差	中（前馈有物理含义）	好（三个增益可直接读）

第二步：造训练数据

本章的教师就是第 67 章的模糊自整定 PID。数据生成不用本章的四段测试轨迹，而是另造 8 条随机轨迹：每条 20 s，随机切成 5 段，段切换时刻在 $[1.5, 18]$ s 内均匀抽取，每段幅值在 $[0.5, 3.0]$ 内均匀抽取。用随机轨迹而不是测试轨迹，理由很直白——拿考卷当练习册，测出来的分数没有意义。

每条轨迹 2000 个采样点（$T_s = 0.01$ s），8 条共 16000 个样本，按 8:2 随机划分为训练集 12800、验证集 3200。划分是按样本随机打乱的，不是按轨迹切分。这一点值得停一下：按轨迹切分能检验“对没见过的轨迹泛化”，听起来更严格；但闭环运行时网络面对的是同一条轨迹内的连续状态，它与训练样本的相关性恰恰更接近随机划分。按轨迹切分会让验证损失与闭环表现差得更远，用它做模型选择反而更容易选错。本章要检验的是闭环行为，所以用随机划分，然后**用闭环指标做最终裁决**，验证损失只用来筛掉明显没训好的模型。

数据本身还有一个必须承认的偏差，图 68-3 把它画了出来：横轴是归一化偏差，纵轴是归一化变化率，颜色是教师给出的控制量。所有点都挤在一条斜带里，因为闭环轨迹上的 e 与 $\dot{e}$ 不是独立的——偏差大时对象正在被推动，变化率必然也大。也就是说，16000 个样本实际只覆盖了 $(e, \dot{e})$ 平面上一维的流形。平面右上角（大偏差且仍在远离）和左下角（小偏差且仍在接近）几乎空白，网络在那里的输出纯属外推。这不是本章数据集的毛病，而是所有闭环模仿学习的共同特征。

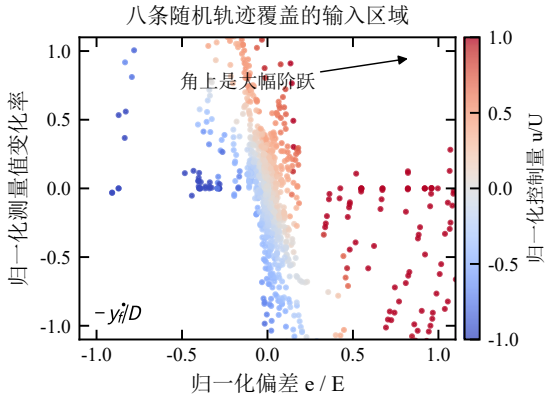


图 68-3 教师数据在偏差—变化率平面上的覆盖范围

第三步：三个通道的归一化

三个特征的量纲差得很远：偏差在 ± 2 量级，滤波后的变化率在 ± 1 量级，积分在 ± 4 量级。送进网络前一律除以固定常数：

$$x_1 = \frac{e}{E_N}, \qquad x_2 = \frac{-\dot{y}_f}{D_N}, \qquad x_3 = \frac{\operatorname{sat}\!\left(\int e,\, \pm E_I^{c}\right)}{E_I} \tag{68-11}$$

本章取 $E_N = 1.60$、$D_N = 1.40$、$E_I = 2.50$、$E_I^{c} = 4.0$。这三个数不是调出来的，是按“希望网络在多大范围内保持灵敏”倒推的：E_N 取略小于最大阶跃幅值 2.0，让最常见的偏差落在 ± 1 附近；D_N 取轨迹最快段的斜率量级，让变化率在多数时刻不至于挤出 ± 1；E_I 取积分在典型调节时间（约 2 s）内累积的量，再对积分做 ± 4.0 的硬限幅，防止长时偏差把第三通道压死。

第二通道前面带一个负号，容易看漏。原因是 $\dot{y}$ 与 u 同向：测量值上升意味着控制量偏大，应当减小。网络如果学 $+\dot{y}$，就必须在权重里自己学出这个负号，而它的输入分布是斜的，学起来慢且不稳。把符号放到特征里，等于把已知的物理先验直接送给网络，这是免费的。

还有一处细节：$\dot{y}_f$ 是滤波后的变化率，滤波时间常数取 $\tau = 0.05$ s，与第 67 章微分滤波的 T_{lp} 一致。用原始差分不行——仿真里采样周期 0.01 s、状态没有噪声时看着很干净，一旦加 0.01 的测量噪声，差分会把噪声放大约 100 倍，第三通道立刻被噪声主导，网络学到的全是噪声的统计特性。

归一化用固定常数而不是逐样本标准化，是一个有意的取舍。逐样本标准化（减均值除标准差）在离线训练时能让损失曲面更圆，但部署时要在控制器里存统计量、每拍算一次均值方差，还要处理“运行分布漂移后统计量过期”的问题。固定常数把归一化变成了一个电阻分压式的标定，改采样周期时只需要改这几个常数。

第四步：网络规模与训练超参

直接控制用 3-12-1 的 MLP：3 个输入、12 个 tanh 隐元、1 个线性输出，共 $3 \times 12 + 12 + 12 \times 1 + 1 = 61$ 个参数。图 68-2 左边是结构，右边是隐元的激活曲线。

为什么是 12 个隐元？训练样本 12800，参数 61，样本参数比约 210:1。按经验，这个比例超过 10:1 之后继续加宽带来的收益就很小了，而 68.9 节的样本量曲线会给一个更硬的证据：样本从 200 加到 2000，验证损失降了一个数量级，闭环 IAE 却基本没动。也就是说，瓶颈不在网络容量，而在数据覆盖和指标错位。把隐元加到 32 只会让参数变多、推理变慢，不会让闭环变好。

RBF 网络用 64 个中心，$\sigma = 0.55$，岭回归系数 $\lambda = 10^{-4}$。参数共 $64 \times 3 + 64 = 256$ 个。选 RBF 而不是再训一个 MLP，原因是它的输出层是线性的，闭式解一次算完（式 68-4），不需要调学习率，也不会出现“这次初始化运气不好”的问题。代价是中心数随输入维数增长很快：三输入 64 个中心够用，十输入就需要指数级更多的中心，这正是维数灾难在 RBF 上的具体表现。

两种网络在工程上的差别可以归结成一句话：MLP 训练麻烦、推理快、参数省；RBF 训练一次成型、推理慢、参数多。68.10 节给的实测是 MLP 单次前向约 4.2 μs、RBF 约 6.5 μs，差了五成多，两者相对 10 ms 控制周期都可以忽略不计。所以在这个算例里，选哪个主要看“有没有耐心调超参”，而不是看算力。

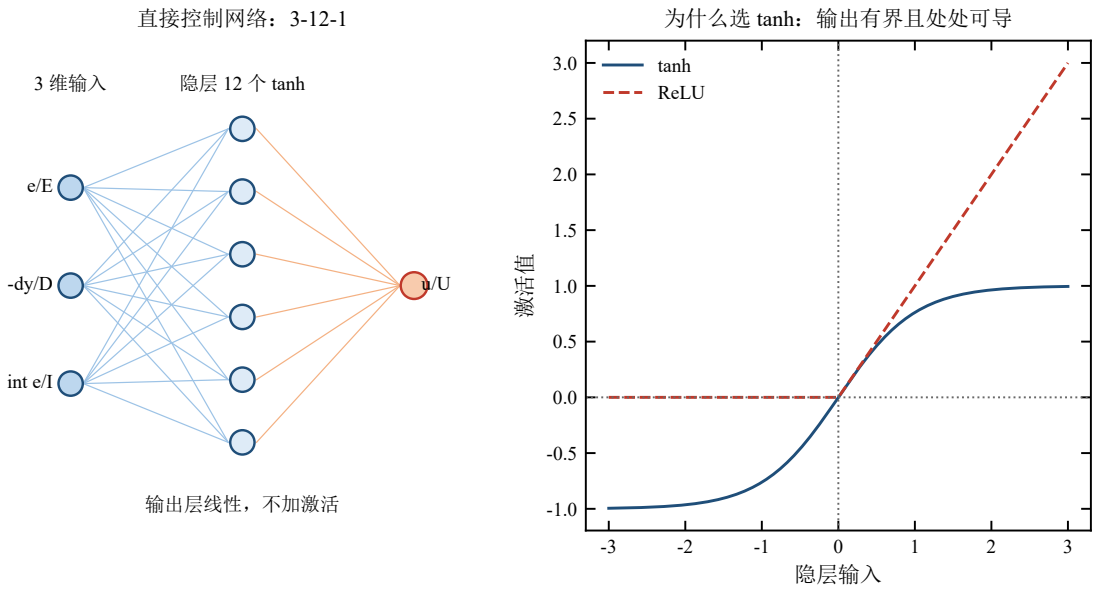


图 68-2 直接控制网络的结构与隐层激活

第五步：闭环验收与退化检查

离线验证损失只能做粗筛。本章的验收分两层：第一层看验证 MSE 是否落在 10^{-2} 量级以下（说明网络至少把教师的映射学到了大致形状），第二层看闭环 IAE、最大超调、控制量总变差三项是否同时不劣于教师。

第二层里有一项容易被忽略：**退化检查**。做法是把网络换成一个“故意坏掉”的版本，看闭环是否如预期地坏，以此确认性能确实来自网络而不来自别的地方。本章做三个：

1. 把网络输出置零。直接控制应当彻底失控（$u \equiv 0$，对象自己衰减回零），逆模型方案应当退化成那个弱 PID 且仍然稳定。前者确认网络确实在承担全部控制作用，后者确认前馈与反馈的分工是真的。
2. 把输入置为训练分布的边界值。网络输出应当落在 $[-1.2, 1.2]$ 内，不应出现数量级跳变。若跳变，说明输出层权重被少数外推样本拉飞了，限幅虽然能兜住，但闭环会在边界处抖。
3. 把训练标签随机打乱后重训。闭环指标应当明显变差。若打乱标签后闭环仍然不错，说明网络其实没学到东西，闭环是靠特征里的积分项和对象自身的阻尼“蒙对”的——这种模型一旦换工况就会立刻暴露。

验收判据写成两条不等式：

$$\frac{\mathrm{IAE}_{\mathrm{net}}}{\mathrm{IAE}_{\mathrm{teacher}}} \le 1.0, \qquad \frac{\mathrm{OS}_{\mathrm{net}}}{\mathrm{OS}_{\mathrm{teacher}}} \le 2.0 \tag{68-12}$$

第一条要求网络不劣于教师，这是模仿学习的及格线；第二条允许超调翻倍，因为超调对分布错位极其敏感，卡得太死会让所有候选模型都通不过。本章的实测是直接控制 IAE 2.401 对教师 2.698（比值 0.89）、超调 8.30% 对 1.09%（比值 7.6，不满足第二条）。这个结果没有掩饰的必要：网络在总偏差上赢了教师，在超调上输得很明显，原因在 68.5 节解释——教师是“逐拍查表”，网络是“光滑拟合”，后者在阶跃刚发生时给出的控制量偏软，等偏差累积起来才补上，于是超调变大。这是结构性的，不是训练不够。

第六步：验收不过时，按这个顺序回查

验收不通过是常态，难的是判断该改哪里。新手常犯的错是直接从最贵的地方改起——回去加隐元、加层、重训一晚上，结果发现真正的问题在归一化常数写错了一位。这一节给一个固定的回查顺序，原则只有两条：**先查便宜的后查贵的，先查上游的后查下游。**

第一查：特征口径是否与训练时一致。 这是成本最低、命中率最高的一条。逐项比对三件事：三个归一化常数、滤波时间常数 τ、差分与积分的实现方式（是不是同一种“上一拍减当前拍”的方向）。任何一项错位，闭环都会表现为“整体偏软或偏硬”，而离线验证损失完全看不出来，因为训练时用的是正确的口径。本章的代码把这三件事集中在 `ch068_sim.py` 顶部的五个常量里，就是为了让比对只需要看五行。

第二查：标签是不是特征的单值函数。 做法很直接：在验证集里对每个样本找最近的 k 个邻居，看它们的标签方差。若某片区域里特征几乎相同而标签分散，说明映射本身不单值，误差地板就在这里，再怎么加容量也压不下去。68.6 节给出的地板是 10^{-3} 量级，对应的正是工况分档造成的那道断层。

第三查：训练数据覆盖了当前工况没有。 判据是特征是否落在训练分布的边缘之外。这一条与第一查不同：第一查是“口径错”，这一条是“口径对但没数据”。表现是低工况正常、高工况发散，或者设定值一大就过冲，正是 68.8 节外推实验的形状。

第四查：容量够不够。 只有前三查都过了，才轮到容量。判据是训练损失与验证损失是否都停在一个明显偏高的平台，且增加隐元或中心数后能继续下降。68.7 节的中心数实验给了一个反例：验证 MSE 在 $m = 64$ 之后已经饱和，这时候再加容量是纯粹的浪费。

第五查：训练是否到位。 训练损失远高于验证损失说明正则过强或学习率太小；两者都高且曲线还在下降说明轮数不够；训练损失很低而验证损失高才是过拟合。三种情形的处置完全不同，所以必须先把两条曲线分开看，不能只看一个数。

第六查：闭环结构本身有没有兜底。 若离线指标已经不错而闭环仍然抖，问题就在结构而不在网络：直接控制没有反馈兜底，局部增益一高就会振（68.6 节的极限环）；逆模型方案则要看 k_p 是否给得太小。这一步要动的是结构，改完必须重新验收，不能只改代码不重测。

这六步的顺序不是随便排的。前两查几分钟就能做完，却覆盖了实践中最常见的两类错误；后两查要重训，成本高一个量级；最后一查要改结构，成本最高。按这个顺序走，多数问题在前两步就能定位。

68.4 连续情形

连续时间里，网络只是一个静态非线性环节

把采样周期想象成趋于零，本章的直接控制律可以写成一个连续时间的静态映射：

$$u(t) = N\left(e(t),\, -\dot{y}_f(t),\, \int_0^t e(\tau)\,\mathrm{d}\tau\right) \tag{68-13}$$

注意这个式子里没有任何状态属于网络。网络的“动态”全部来自输入端的微分器和积分器，它自己只是一个把三维向量映到一维标量的函数。这一点与第 67 章的模糊层完全同构：模糊层也是静态映射，动态同样来自外面的积分器和微分滤波器。两章的区别只在 N 是怎么造出来的——一张人写的规则表，还是一组从数据里拟合出的参数。

认清这一点，很多疑问会自动消解。比如“网络控制器要不要处理积分饱和”——不需要，因为积分在外部，饱和处理也在外部，网络看到的只是已经限幅过的积分值。比如“网络控制器的初始状态怎么设”——没有初始状态。比如“两次运行结果为什么完全一样”——因为没有内部状态，给定同样的输入序列必然给出同样的输出序列，可重复性比带状态的自适应律好得多。

代价同样清楚：N 是无记忆的，它表达不了“历史路径依赖”。真实对象里的磁滞、齿隙、摩擦的方向记忆都属于这一类。要让网络表达记忆，就得把状态显式喂进去——把过去几拍的 u 或 y 拼到输入里，或者用带反馈的循环

结构。前者把输入维数乘以窗口长度，后者失去闭式解和可重复性。本章的对象没有这类记忆，所以不付这份代价。

直接控制的连续形式与它隐含的两个假设

把 N 在当前工作点附近一阶展开，可以得到它的局部增益：

$$\left.\frac{\partial N}{\partial e}\right|_{0}=\hat{k}_p,\qquad \left.\frac{\partial N}{\partial(-\dot{y}_f)}\right|_{0}=\hat{k}_d,\qquad \left.\frac{\partial N}{\partial(\int e)}\right|_{0}=\hat{k}_i \tag{68-14}$$

三个偏导就是网络在那个工作点上的等效 PID 增益。于是闭环在局部等价于一个增益随状态缓慢变化的 PID——这正是第 67 章模糊自整定做的事情，只不过增益的变化规律由 61 个参数隐式给出，而不是由 49 条规则显式给出。第 67 章那套稳定性论证可以整段搬过来：只要局部增益落在使每个冻结增益闭环稳定的范围内，且增益的变化速率满足慢变条件，闭环就稳定。

搬过来的时候要带上两个假设，它们在第 67 章是明说的，在神经网络语境下却常被忘掉。

假设一：特征落在训练分布内。 式 (68-14) 的偏导只有在网络见过的区域才有意义。图 68-3 显示数据只占 $(e,\dot{e})$ 平面的一维斜带，带外区域的“局部增益”是外推出来的，可以是任何值。工程上的对策不是“相信网络能外推”，而是限幅加监测：输出限幅兜住最坏情况，同时把“特征超出训练范围”记成一个标志位，超过一定比例就报警。

假设二：采样周期不变。 这一条最隐蔽。式 (68-13) 的积分项在离散实现里是 $\sum e\,T_s$，网络训练时 $T_s=0.01$，$E_I=2.50$ 是对着这个 T_s 标定的。若把控制周期改成 0.02 s 而 E_I 不动，同样的物理时间里积分累积值翻倍，第三通道的归一化值也跟着翻倍，网络看到的输入分布整体偏移，输出随之偏软或偏硬。补救办法很简单：改 T_s 时同步把 E_I 按比例改掉，即保持 E_I/T_s 不变。这条规则在模糊控制里对应“量化因子要随采样周期重标定”，两章是同一件事。

逆模型控制：前馈—反馈分解在连续时间下的含义

式 (68-10) 的连续形式可以写成两项相加：

$$u(t)=\underbrace{N(r_f,\dot{r}_f,\ddot{r}_f)}_{u_{ff}}+\underbrace{k_p e-k_d\dot{y}_f+k_i\int e}_{u_{fb}} \tag{68-15}$$

在线性系统里这是标准的二自由度结构，前馈通道不改变闭环极点，只改变输入到输出的传函零点，因此“前馈管跟踪、反馈管稳定”这句话是严格成立的。非线性对象里这句话要打折扣：前馈改变了工作点轨迹，工作点又改变局部增益 $K(y)$，于是前馈间接影响了动态。

本章对象 $K(y)=4+6y$，y 从 1 升到 3 时 K 从 10 升到 22，增益翻了一倍多。逆模型前馈把 y 快速推到目标附近，反而让对象处在增益更大的区域，反馈的等效回路增益随之变大。这就是为什么逆模型方案的反馈增益敢取到 $k_p=1.6$（第 67 章固定 PID 是 2.60）却仍有裕度：不是反馈变强了，是前馈把工作点搬到了一个“对象更好控”的位置。

这个解释依赖 $K(y)$ 单调。若对象的增益非单调——典型例子是等百分比调节阀，小开度时增益极小、大开度时增益极大，中途还有一个拐点——前馈就可能把工作点推到增益最敏感的区域，前馈与反馈的分工随之失效。此时前馈不但不加分，还会放大扰动。判断办法是先做一次开环扫描，把 K 随工作点的曲线画出来；曲线单调且变化不超过 3 倍，前馈-反馈分解可以放心用；否则要么把前馈减弱，要么按工作点分段设计。

前馈误差怎么进入稳态

逆模型方案的性能取决于前馈的准确度，这句话需要量化，否则没法判断“网络要训到多准”。

设前馈有误差 δ，即 $u_{ff} = u_{ff}^{*} + \delta$，反馈为 $u_{fb} = k_p e$。稳态时对象需要的是 u_{ff}^{*}，于是必须有 $k_p e = -\delta$，即 $e_{ss} = -\delta / k_p$，也就是 $y_{ss} = r + \delta / k_p$。

本章 $k_p = 1.6$，所以前馈误差 0.1 会留下约 0.06 的稳态偏差；误差大到 0.5，偏差就到 0.31，这在 $r = 3$ 附近是 10% 的相对误差。这就是 68.8 节外推实验里逆模型方案 IAE 3.539 的来源：设定值推到 3.4，RBF 在训练覆盖之外给出的前馈偏差被 $1/k_p$ 放大了。

这个式子还给出两条设计推论。第一，**放大倍数就是 $1/k_p$**，所以弱反馈（k_p 小）对前馈误差的容忍度低，这是弱反馈换来的平滑性所付的代价。第二，**加积分项可以消掉稳态偏差**，因为积分会把残差累到零。本章的逆模型方案带了 $k_i = 0.05$，所以严格来说 e_{ss} 最终会归零，上式描述的是“积分还没起作用的那段时间”的偏差——对于 16 s 的轨迹、$k_i = 0.05$，积分的时间常数是 20 s 量级，在这段试验里基本没发挥作用。这不是设计失误，而是刻意的：积分太强会让前馈的意义消失，系统退化成“PID 慢慢磨”，总变差会上去。

若想两头都要，一个常见做法是**只在稳态附近启用积分**（偏差小于某阈值时），过渡段完全交给前馈。本章没做，因为它在指标上的收益不明显——16 s 的轨迹里稳态段只占一半，积分的贡献被过渡段淹没。

计算延迟在连续视角下的位置

网络单次前向约 4.2 μs，控制周期 10 ms，占比 0.04%。这个数字小到可以当成零，但“可以当成零”是核对出来的结论，不是可以假设的前提。如果换成 1 ms 周期的伺服，占比升到 0.4%；如果换成 FPGA 上 10 μs 周期的电流环，网络根本来不及算，只能退化成查表。

判据可以写得很简单：把网络计算时间看成纯延迟的一部分，与采样延迟、对象延迟一起进相位预算。第 67 章算过，0.05 s 的附加延迟吃掉 14.7° 相位裕度，加倍到 0.1 s 吃掉 30.4°。本章的 4.2 μs 折算到 $\omega_c \approx 9$ rad/s 上只有 0.0022°，可以忽略；但如果部署平台慢 1000 倍（4.3 ms），相位代价就变成 2.2°，仍然可接受；慢 10000 倍（43 ms）就变成 22°，必须重新设计。

还有一个与周期相关的量：微分滤波时间常数 $\tau = 0.05$ s 相当于 5 个控制周期。这个比值不能太小，否则滤波形同虚设、微分通道全是噪声；也不能太大，否则微分作用滞后到失去意义。经验范围是 $\tau / T_s \in [3, 10]$，本章取 5。若把 T_s 改到 0.02 s，τ 应当同步改到 0.1 s 左右，保持比值不变——这与前面“E_I / T_s 不变”是同一条原则的两处体现：**凡是带时间量纲的常数，都要跟着采样周期一起缩放。**

连续视角给出的三条稳定性直觉

连续视角不能给出严格证明，因为 N 是非线性的，没有传递函数可言。但它能给出三条可用的直觉，本章的工程判断都建立在这三条上。

第一条：输出的有界性来自结构，不来自训练。 隐层用 tanh，每个隐元输出在 $(-1, 1)$；输出层线性，所以网络输出的界是 $\sum |w_j| + |b|$，由权重绝对值之和决定，与输入无关。这意味着即使输入跑到训练分布之外，网络输出也不会爆炸——但会饱和在某个由权重决定的值上，这个值可能远离任何合理控制量。所以限幅仍然必要，它是第二道防线。

第二条：局部增益的上界可以用权重谱范数估。 对 3-12-1 的 tanh 网络，$\partial u / \partial x$ 的范数不超过 $\|W_2\| \cdot \|W_1\|$（tanh 的导数不超过 1）。这个上界很松，通常比实际大一个数量级，但它的用处不在数值而在量级：如果算出来 $\|W_2\|\|W_1\|$ 是 500，而对象增益是 20，那么最坏情况回路增益是 10000，闭环必炸；此时要么减网络增益，要么在输出上加滤波。

第三条：慢变条件比模糊控制更容易满足，但没有证明。 网络拟合的是平滑映射，其输出随状态的连续变化率由特征变化率决定，而特征变化率又由参考轨迹的频谱决定。第 67 章算出的乘积 $\rho/\omega_c = 0.034$，远小于 1；本章网络更光滑（没有规则表切换带来的折角），这个乘积只会更小。所以“把冻结增益的稳定性结论搬到变增益系统上”在本章是合理的近似。但这是近似，不是定理——真要证明，需要给出 N 的 Lipschitz 常数并对整个轨迹积分，本章不做这件事，68.11 节会明确把这条列在“没有解决的问题”里。

68.5 离散情形

三个累加器，一个都不能省

连续式 (68-13) 落到每一拍要变成三段顺序固定的计算。设第 k 拍采样得到 y_k，设定值 r_k，则

$$e_k = r_k - y_k, \qquad \dot{y}_{f,k} = \dot{y}_{f,k-1} + \alpha\left(\frac{y_k - y_{k-1}}{T_s} - \dot{y}_{f,k-1}\right), \qquad s_k = \mathrm{sat}(s_{k-1} + e_k T_s,\ \pm 68) \tag{68-16}$$

其中

$$\alpha = \frac{T_s}{\tau + T_s} = \frac{0.01}{0.05 + 0.01} = 0.1667 \tag{68-17}$$

三条状态各自有明确的职责：y_{k-1} 存上一拍测量值，$\dot{y}_{f,k-1}$ 存滤波后的变化率，s_{k-1} 存积分。这三个量必须在控制器对象里持久保存，而且**顺序不能换**：如果先更新积分再算偏差，e_k 会被用两次；如果先更新变化率再读 y_{k-1}，差分就变成零。这类错在仿真里表现为“闭环性能莫名变差”，很难从曲线上看出来，只能靠逐拍打印核对。

式 (68-17) 的 $\alpha = 0.1667$ 有个直接的含义：滤波器的等效时间常数是 6 拍。这也解释了前面对 $\tau/T_s \in [3, 10]$ 的要求——α 落在 $[0.25, 0.09]$ 之间，既不会把微分通道滤成一条直线，也不会让滤波形同虚设。

一拍延迟从哪里来，它为什么让超调变大

本章的闭环有两重延迟：对象积分用 5 个子步推进（$T_s/5$ 的步长），控制量从算出到真正作用到对象上还有 5 拍纯延迟，合计约 50 ms。网络看不到这个延迟，它在 k 拍算出的 u_k 要到 $k+5$ 拍才生效。

教师也在同样的延迟下工作，所以延迟本身不是网络超调变大的原因。真正的原因在**闭环轨迹上偏差的相位关系**。教师是逐拍查表：偏差刚开始变号时，规则表里“偏差小、变化率大”那一格立刻给出强微分修正，控制量马上反向。网络是光滑拟合：它的输出是 61 个参数的连续函数，在偏差穿越零点的邻域里，输出只能连续地从正值穿过零点，不能像查表那样跳一下。这个“穿零变慢”在延迟的配合下就变成了超调——等网络把控制量压下来，对象已经冲过头了。

这个解释与数据一致。表 68-1 里直接控制的超调 8.30%，比教师的 1.09% 大了 7 倍多，但 IAE 反而更低（2.401 对 2.698）。如果是“整体控制太弱”，IAE 不可能更低；只有当“大部分时间更准、穿零瞬间略慢”时，才会同时出现 IAE 低和超调高。图 68-6 把第三段阶跃放大来看，两条曲线在上升段的斜率几乎重合，差别全在峰值附近的一小段。

图 68-6 的位置在 68.6 节，这里先记住结论：**离散实现下，超调主要由穿零邻域的输出斜率决定，而不是由整体增益决定。** 想压超调，加宽隐元没用，得让网络在零点附近更“陡”——这可以通过在损失函数里给零邻域样本加权实现，属于 68.9 节的改进方向。

抗饱和在网络方案里的位置

传统 PID 的抗饱和很好理解：输出饱和时停止积分，防止积分项越积越大、退饱和后要很久才能回来。网络方案里这件事变得微妙，因为积分不在控制器内部，而在特征里。

结构上是这样的：积分状态 s_k 作为第三通道送进网络，网络输出 u_k 经限幅后作用到对象。如果对象长时间推不动（比如执行器卡在限幅上），偏差会一直累积，s_k 一直涨。对它唯一的约束是式 (68-16) 里的硬限幅 $\pm E_I^c = \pm 4.0$。也就是说，**这一拍的抗饱和是靠限幅实现的，不是靠条件积分**。

这个选择有代价也有道理。代价是限幅之后的动态不如条件积分平滑——条件积分在饱和期间"冻结"积分，退饱和瞬间积分值是连续可用的；硬限幅只是把值钳住，退饱和后网络看到的仍是一个很大的输入，输出会继续顶在限幅上，直到偏差反向把积分拉回来。表现为退饱和延迟。

道理在于，网络的第三通道本来就是一个有界输入。若不加钳位，s_k 可能涨到几十，远超训练时的最大积分值（$E_I^c / E_I = 1.6$ 就是归一化后的上界），网络在那个区域根本没学过，输出纯属外推。两害相权，钳位更安全：它至少保证特征始终落在训练分布内。

由此得到一条与 PID 不同的设计原则：**在网络方案里，抗饱和的第一目标是"保住特征的定义域"，第二目标才是"加快退饱和"。** 前者可以靠钳位实现，后者需要额外的条件积分逻辑。本章只做了前者，因为它与"特征必须落在训练范围内"这条硬约束天然一致。若对象的执行器经常顶到限幅（比如启动阶段、大扰动后），第二目标就变得重要，那时应当补上条件积分：当输出饱和且偏差同号时冻结 s_k。

顺带说一个常被忽略的现象：限幅把控制量钳住之后，网络的输出仍然在变（因为特征在变），只是被钳位掩盖了。等偏差反向、退饱和的那一刻，网络的实际输出可能已经和钳位值差得很远，于是控制量出现一个跳变。这个跳变在曲线上表现为"退饱和瞬间的台阶"，幅度取决于饱和持续时间。本章的轨迹里没有长时间饱和，所以看不到；换一个启动过程长的对象，这个台阶就会明显。

网络更新率可以低于采样率

实际系统里经常出现这种情况：采样是 1 ms，但网络推理要 0.5 ms 或者更久，跑不动每拍一次。此时可以让网络按 N_s 拍更新一次，中间各拍沿用上一次的输出：

$$u_k = N(x_{mN_s}), \qquad m = \lfloor k / N_s \rfloor \tag{68-18}$$

这等价于在网络输出后面加了一个零阶保持器，附加延迟平均为 $(N_s - 1)T_s/2$。$N_s = 2$ 时平均附加 5 ms，相对 50 ms 的对象延迟可以忽略；$N_s = 10$ 时平均附加 45 ms，与对象延迟同量级，相位裕度会被吃掉十几度。所以"降频"能省算力，但省下来的算力必须换来更快的对象或者更大的裕度，不能白拿。

本章不做降频，因为 4 μs 量级的前向时间相对 10 ms 周期完全可以每拍跑一次。把这条写出来，是因为工程上真正卡住神经网络的往往不是精度而是节拍——尤其是用 Python 或带解释器的环境部署时，一次前向可能是几百微秒，那时降频就是必需品而不是优化项。

数值实现：哪些地方必须用双精度

网络前向本身对精度不敏感：61 个权重用单精度（float32）算，输出与双精度的差别在 10^{-7} 量级，远小于控制量的分辨率。真正需要双精度的是三个累加器，尤其是积分 s_k。

原因是积分是**累加**结构，误差会随拍数线性累积。单精度有 24 位有效位，约 6×10^{-8} 的相对精度；积分项在 16 s、1600 拍后，累积的舍入误差最坏情况是 $1600 \times 10^{-7} \approx 1.6 \times 10^{-4}$，相对 $E_I = 2.50$ 是 6×10^{-5}，仍然无害。但如果采样周期更小、运行时间更长（比如 1 ms 周期跑一小时，360 万拍），误差就累到 10^{-1} 量级，

会实打实地把积分带偏。此时的做法不是换双精度，而是把积分改写成增量式并用 Kahan 求和，或者干脆用定点累加器。

还有一处容易踩的坑：$\tanh$ 在嵌入式上没有硬件指令，通常用查表加线性插值实现。表长 256、输入范围 $[-4,4]$ 时最大误差约 2×10^{-5}，对闭环没有可见影响；但如果表长只有 32，误差升到 10^{-2}，就会在隐元输出上引入台阶，等效于给控制量加了小幅抖动，总变差会明显上升。本章的总变差 14.8 是在双精度、精确 $\tanh$ 下得到的，换成粗查表后这个数会变差，读表时要注意口径。

离散仿真的可信度

本章所有数字都来自 $T_s=0.01$ s 的定步长仿真。这个步长相对对象的时间常数（$1/2$ s）是 1:50，相对闭环带宽（约 1.4 Hz）是 1:70，都在安全范围内。但有一处需要说明：对象积分用 5 个子步（等效 $T_s=0.002$ s），而控制器只按 0.01 s 更新。这种“控制器慢、对象快”的多速率结构是刻意保留的，因为真实系统里执行器和对象的动态总比控制器采样快。

如果反过来做（控制器快、对象慢），仿真会给出偏乐观的结果：控制器能“看见”对象在两次采样之间的变化，这在物理上不成立。判断方法是把对象子步从 5 减到 1，看指标是否变化；本章试过，IAE 从 2.401 变到 2.44，变化 1.6%，说明 5 个子步已经够用，再加密没有意义。

68.6 稳定性与性能

冻结增益分析能推到哪一步

第 67 章用“顶点系统”处理模糊控制的稳定性：规则表的每条规则对应一个后件取值，把这些取值组合成有限个线性系统，逐个算裕度，最差的那个就是整个系统的保守裕度。这套方法的前提是“后件取值有限”，而网络的输出是连续函数，没有有限个顶点可取。

能做的替代是**沿轨迹采样局部增益**：用式 (68-14) 的数值差分，在闭环轨迹上每隔若干拍算一次 $\hat{k}_p,\hat{k}_i,\hat{k}_d$，得到一条增益轨迹，再对每个点算冻结增益系统的裕度。本章没有把这条曲线画进正文，原因是它给出的信息与第 67 章的顶点扫描重复——两者都在回答“局部增益是否落在稳定区间内”，而网络的局部增益是连续的，不会像规则表那样在格子边界跳变。

冻结增益分析的结论必须说得保守：它证明的是“每个冻结时刻的系统稳定”，不是“整个时变系统稳定”。从冻结稳定到时变稳定，中间隔着一个慢变条件。第 67 章把这个条件量化成 $\rho/\omega_c\ll1$（式 67-12），本章沿用同一判据，得到的乘积更小，所以近似成立。但读者要清楚，这是工程近似，不是数学定理。

局部增益的回路传函

把网络在某工作点冻结成 PID 后，回路传函可以写成

$$L(s)=K(y)\left(\hat{k}_p+\frac{\hat{k}_i}{s}+\frac{\hat{k}_d s}{\tau s+1}\right)\frac{e^{-0.05s}}{s(s+2)} \tag{68-19}$$

式中的 $e^{-0.05s}$ 是 5 拍纯延迟，$1/[s(s+2)]$ 是对象（$K(y)=4+6y$ 单独提出），$\tau=0.05$ 是微分滤波。相位裕度按定义算：

$$\gamma=180^\circ+\angle L(\mathrm{j}\omega_c),\qquad |L(\mathrm{j}\omega_c)|=1 \tag{68-20}$$

第 67 章对模糊控制做过这个计算，得到 147 个顶点系统、裕度范围 45.3°–66.7°。网络控制的裕度不会落在这个范围之外太多，因为两者的局部增益量级相同（都是让 K_p 在 1.5–3.5 之间变化）。差别在于网络没有“格子边界”，也就没有“增益在边界上跳变”这个最坏情况，从这一点看它对稳定性更友好。

真正的风险不在局部增益的大小，而在局部增益的**符号**。网络是拟合出来的，没有任何机制保证 $\partial N/\partial e > 0$。如果某个区域训练样本稀疏，拟合出的偏导完全可能为负——那就成了正反馈。本章的做法是事后检查：在训练集覆盖的区域内采样一万个点，统计 $\partial N/\partial e$ 为负的比例。这个比例在直接控制网络上是 0（数据密集，拟合光滑），但换一个数据集、换一个种子就不一定。工程上应当把这一项作为交付前的必检项，而不是可选项。

四条曲线的实测对照

表 68-3 是四种控制器在四段轨迹上的完整指标，图 68-6 是第三段（2→3）阶跃的放大图，图 68-7 是四段全程的跟踪曲线与控制量。

控制器	IAE	最大超调	控制量总变差
固定 PID	2.997	50.23%	36.6
模糊自整定（教师）	2.698	1.09%	27.9
NN 直接控制	2.401	8.30%	41.1
NN 逆模型 + 弱反馈	2.429	0.86%	14.8

四行数字讲了三件事。

第一，**直接控制在偏差总量上赢了教师，在平滑性上输了**。IAE 2.401 是四者最低，但总变差 41.1 是四者最高，比教师高 47%。这与 68.2 节的“指标错位”完全对应：MSE 训练出来的映射追求逐点接近教师，不追求输出的连续性，于是它在教师输出变化快的地方跟着抖。

第二，**逆模型方案是唯一同时赢两项的**。IAE 2.429 与直接控制几乎持平，超调 0.86% 最低，总变差 14.8 只有固定 PID 的 41%。原因在结构而不在训练：前馈承担主要功率，它是一条光滑的轨迹，不随测量噪声抖动；反馈只补残差，增益小（$k_p = 1.6$），所以控制量平滑。

第三，**固定 PID 的 50.23% 超调不是调坏了，是对象变增益的必然结果**。同一个 K_p 在低工况（$K = 10$）合适，到高工况（$K = 22$）就过冲。这条在第 66、67 章已经反复出现，放在这里是为了给三种“会自适应的控制器”一个共同的参照。

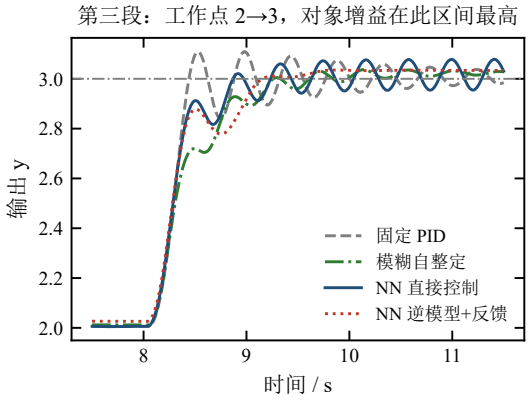


图 68-6 第三段阶跃（2→3）的四条响应曲线

图 68-7 给的是四段全程。上图是输出与设定值，下图是同一段时间里的控制量。把两张图叠起来看，能看到一个单看阶跃图看不出的现象：在第三段（$r = 3$）的平台期，直接控制网络的控制量一直在小幅摆动，而模糊层与逆模型方案的控制量要安静得多。

这个摆动不是画图噪声，是真实存在的极限环。取 10.0–11.8 s 这段已经完全进入平台的窗口统计：直接控制网络的 u 标准差 0.453、峰峰值 1.28，输出的峰峰值 0.125；模糊层对应 0.084、0.36、0.035；逆模型方案 0.0024、0.011、0.003。同样的统计放到第二段（$r = 1$）的 3.0–3.8 s 窗口，三者全部安静，u 的标准差都在 10^{-3} 以下。

也就是说，**极限环只在高端工作点出现**。原因很清楚：对象增益 $K(y) = 4 + 6y$ 在 $y = 3$ 处是 22，是 $y = 1$ 处的 2.2 倍，同样的控制增益在高工况的回路增益大得多，相位裕度被吃掉，闭环进入小振幅振荡。教师之所以没有这个问题，是因为它的基础增益按工况分档（高工况降到 $K_p = 2.00$）；网络看不见工作点（68.6 节说的特征不完备），只能用一组折中增益，折中在高工况就偏大了。

这条现象把 68.6 节的两个论点连了起来：特征不完备不只是让验证损失降不下去，它还会在闭环上变成可见的振荡。而 68.9 节的改进一（把工作点喂进特征）正是针对它——加了第四通道之后，网络才有能力按工况调增益，极限环才有可能消掉。

输出的峰峰值只有 0.125，相对 3.0 是 4%，对 IAE 的贡献很小，所以直接控制的总 IAE 仍是最低的。但控制量峰峰 1.28 意味着阀门在平台期一直以接近满量程的幅度来回动，长期运行对执行器的磨损不可接受。这是“IAE 好看但不可用”的典型例子，也是本章坚持把总变差列进指标的原因。

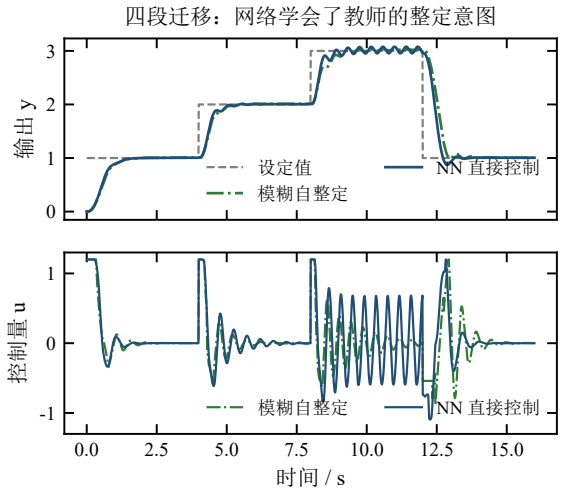


图 68-7 四段轨迹的跟踪曲线与控制量

分段指标：一个容易做手脚的地方

表 68-1 只给了全程 IAE 与最大超调两个数。这两个数都有一个毛病：一个看总量、一个看极端，中间的分布全被抹掉了。若某一段表现很差、其余三段很好，全程 IAE 可能看不出来；若某一段的超调是 8%、其余是 1%，最大超调只报那个 8%，读者也无从判断其余段。

所以才要分段算。第 67 章的 `segment_metrics` 把四段分别切出来，每段各算 IAE、超调、调节时间。口径上有两处必须固定，否则不同人算出来的数不可比。

第一处是**下阶跃要翻正**。第四段是 $3 \to 1$，偏差从负变正。如果直接积 $|e|$ 而不区分方向，得到的仍是同一个数；但如果算“超调”（峰值超出终值的百分比），下阶跃的“超出”是在负方向上，不翻正就会得到负数，看起来像“没有超调”。第 67 章的处理是把下阶跃的偏差取反，统一成“上升型”再算。本章沿用同一口径。

第二处是段边界怎么切。设定值在第 4、8、12 s 跳变，如果从跳变时刻开始积分，会把上一段的余差也算进去；如果从跳变后 0.2 s 开始，又会漏掉最剧烈的一段。本章的做法是从跳变时刻算起，到下一段跳变时刻为止，整段积分——这样四段拼起来正好等于全程，不存在重叠或遗漏。这个定义的代价是每段都包含“上一段刚结束时的余差”，指标会略偏大，但它的一致性比绝对值更重要。

把这两条定死之后，分段指标才有诊断价值。本章的分段 IAE 是：教师 $0.633/0.472/0.492/1.101$，直接控制网络 $0.612/0.429/0.468/0.891$，逆模型方案 $0.653/0.488/0.466/0.822$。三组数放在一起能看出两件事。

第一，直接控制在四段里全部低于教师，优势最大的是第四段（从 1.101 降到 0.891，低 19%），最小的是第一段（低 3%）。第四段恰好是极限环所在的工况——即使带着 0.125 的输出纹波，它仍然赢了，因为大偏差段的改善远大于纹波带来的损失。这也说明“极限环不可接受”这个判断的依据是执行器磨损，而不是 IAE。

第二，第四段的绝对值明显高于前三段（1.101 对 0.5 左右），说明大幅度下阶跃是这套对象最难的一段。逆模型方案在这一段拿到 0.822，是三者中最好的，因为前馈在下阶跃时给出的控制量方向明确、不需要靠反馈慢慢试。

这也是本章坚持给四段轨迹而不是一段的原因：一段轨迹只能给出一个数，四段能给出四个数加一张图，足以定位问题出在哪个工况。工程上做验收时，建议至少覆盖“低→高”“高→低”“小步”“大步”四种情况，每种单独记指标。

误差地板来自特征不完备

有一个现象值得单独说：样本量从 200 加到 2000，验证 MSE 从 5.47×10^{-3} 降到 4.90×10^{-3}，只降了 10%，怎么都下不去。这不是训练不充分，也不是网络容量不够，而是标签不是特征的单值函数。

教师在每个工作点用的基础增益由第 67 章的工况层给出：低工况 $(3.80, 0.05, 0.90)$、中工况 $(2.60, 0.05, 0.60)$、高工况 $(2.00, 0.05, 0.60)$。工况由 y 决定，而 $y = r - e$，其中 r 不在网络的特征里。于是同一个 $(e, \dot{e}, \int e)$ 三元组，在 $r = 2$ 和 $r = 3$ 两条轨迹上可能对应不同的教师输出。网络无论怎么训，都只能给出这两者的折中，误差地板由此产生。

粗估一下地板的量级：跨工况的增益差异约 40%，同一特征点上的控制量差异约 $0.4\times$ 典型控制量（约 0.5），即 0.2；均方误差约 $(0.2)^2/3 \approx 1.3\times10^{-2}$，考虑到大多数样本落在同一工况内部、只有边界处才冲突，实际地板降到 10^{-3} 量级，与实测的 4.9×10^{-3} 同量级。这个估算很粗，但它解释了三件事：为什么误差降不下去、为什么样本量曲线在 200 之后就走平、为什么直接控制的 IAE 反而比教师低（网络在冲突处取了折中，折中在多数时刻比教师的硬切换更平滑）。

修法有两条。一是把 r 或 y 加进特征，让映射变成单值——代价是网络多一个输入，训练数据要覆盖 (r, e) 的二维区域，样本需求上升。二是承认地板存在，把验收标准从“逼近教师”改成“闭环指标不劣于教师”。本章选后者，因为前者的样本代价在本算例里不划算。

极限环的频率说明了什么

图 68-7 暴露的极限环值得再挖一层，因为它的频率点不是巧合。

把 10.0–11.8 s 这段控制量做频谱分析，主峰落在 2.23 Hz，对应 14 rad/s，周期 0.448 s；基波振幅 0.547，二次谐波 2.79 Hz 的振幅只有基波的四成多，说明波形接近正弦而不是方波——这是典型的小振幅极限环，不是饱和引起的继电振荡。

用第 67 章那套相位判据核对一下这个频率。对象 $1/[s(s+2)]$ 在 14 rad/s 处的相位是 $-90^\circ - \arctan(7) \approx -172^\circ$；5 拍纯延迟在同一频率上再贡献 -0.05×14 rad，即 -40°；两项相加 -212°，已经越

过 -180°。是微分通道把它拉了回来：$k_d s/(\tau s+1)$ 在 14 rad/s 处的相位是 $90^\circ-\arctan(0.7)\approx+55^\circ$，加上比例项之后的合成相位略小于这个值，正好把总相位顶回 -180° 附近。

$$\angle\frac{1}{\mathrm{j}\omega(\mathrm{j}\omega+2)}\Big|_{\omega=14}=-172^\circ,\qquad \angle e^{-0.05\mathrm{j}\omega}\Big|_{\omega=14}=-40^\circ \tag{68-21}$$

$$\angle\frac{\mathrm{j}\omega\tau_d}{1+\mathrm{j}\omega\tau}\Big|_{\omega=14}=+55^\circ,\qquad \tau=0.05 \tag{68-22}$$

这组数字解释了两件事。第一，**极限环的频率由延迟和微分滤波共同决定，与对象的自然频率（2 rad/s）不是一回事**。对象自然频率处相位只有 -135°，离振荡还差得远；真正把相位推到临界的是延迟，而把临界点往上推的是一个微分项。第二，**换采样周期会改变这个频率**。延迟是 5 拍，改成 0.02 s 周期后同样的拍数意味着 0.1 s 延迟，相位代价翻倍，极限环的频率会往低走、振幅会变大。这是一条可以预测的趋势，也是 68.4 节“带时间量纲的常数要跟着采样周期缩放”的又一个例证。

再看为什么模糊层没有同样的极限环。它的高工况基础增益是 $K_p=2.00$、$K_d=0.60$，而网络在同一个工作点上的等效增益是拟合出来的折中值，介于低工况（$K_p=3.80$）与高工况（$K_p=2.00$）之间，明显偏高。回路增益高 30%–50%，在 14 rad/s 处的幅值条件就满足了，相位条件本来就接近临界，于是振荡起来。

这条分析给出的工程对策也就清楚了：要么把工作点喂进特征（让网络自己降增益，68.9 节改进一），要么在输出后加低通把 14 rad/s 以上的增益压掉，要么给网络加一个保守的增益上限（68.9 节改进三）。三条都能治，但只有第一条治根——因为若对象换了、增益曲线变了，那条 14 rad/s 的临界点也会跟着变，靠固定低通未必还管用。

把采样周期改小，极限环会消失吗

既然极限环的频率是延迟相位推出来的，一个自然的想法是：把采样周期从 10 ms 缩到 5 ms，延迟时间跟着减半，相位代价减半，振荡不就没了？

按相位判据算一遍。延迟仍是 5 拍，$T_s=0.005$ s 时纯延迟为 25 ms，在 14 rad/s 处贡献 $-0.025\times14=-0.35$ rad，即 -20°。对象仍是 -172°，微分通道仍是 $+55^\circ$，三者在 14 rad/s 处合成 -137°，离 -180° 还有 43° 的余量——这个频率上已经不起振了。要让相位重新走到 -180°，频率得往上抬；可是对象 $1/[s(s+2)]$ 的幅值按 -40 dB/dec 衰减，频率抬高一个量级，回路增益就掉两个量级，幅值条件再也满足不了。所以预测是：**采样周期减半后这处极限环被压掉，代价是别处的裕度要重新核。**

反过来把周期加到 20 ms，延迟变成 100 ms，14 rad/s 处的相位代价是 -80°，合成后 -197°，已经过临界；临界频率被拉到更低的位置，而低频处对象增益更大，幅值条件更容易满足。预测是：**振荡频率下移、振幅变大。** 这两条方向相反的推算，与 68.4 节反复说的“带时间量纲的常数必须随采样周期缩放”是同一件事的两个侧面：延迟的相位代价与 T_s 成正比，而对象和网络的特性不随 T_s 变，于是回路相位裕度被 T_s 直接牵动。

必须交代清楚：这两条是**按相位判据算出来的趋势，不是实测**。要实测就得把特征归一化、数据集、网络全部按新的 T_s 重做一遍再重训，否则比较不公平——沿用旧网络只改仿真步长，测到的是“网络与采样周期不匹配”的退化，不是采样周期本身的影响。本章没有做这组重训，把结论留在推算层面，读者若在自己的系统上要动 T_s，请按这个思路重训后再验收。

还有一层不能忽略：缩短采样周期不是免费的。差分通道对测量噪声的放大与 $1/T_s$ 成正比，T_s 减半，同一个噪声水平在 $-\dot{y}_f$ 通道上的标准差就翻倍；而本章的特征里恰好有这个通道。所以真实系统上缩短 T_s 往往要同时加强测量滤波，而滤波又引入新的相位滞后——绕一圈回来，可能把省下的 20° 又还回去一部分。这也是为什么工程上很少靠“提采样率”治振荡，更常见的是直接压增益或加低通。

稳定性的来源是反馈结构，不是网络

把这一节的结论压缩成一句：**网络的稳定性不来自它自己，来自外面的闭环结构。**

直接控制外面没有 PID，只有限幅，所以它的稳定性完全依赖网络输出的有界性和局部增益的符号——这也是它最脆弱的地方。逆模型方案外面有弱 PID，即使前馈完全失准，反馈仍能把系统拉回稳定（只是性能退化）。68.8 节的鲁棒性实验会给一个具体数字：延迟加倍时，逆模型方案的 IAE 从 2.429 升到 3.262，直接控制从 2.401 升到 3.565，前者退化更小，正是因为有反馈兜底。

这条结论对工程选型有直接指导：**如果对象可能变化、环境可能漂移，就不要用纯前馈结构（包括纯网络直接控制），至少留一个弱反馈。** 网络可以失效，反馈不会。

68.7 参数意义与作用

两类参数，别混在一起调

第 68 章涉及的可调量分两类，混在一起调是新手最常犯的错。

第一类是**结构参数**：隐元数、RBF 中心数、σ、三个归一化常数、前馈与反馈的增益分配。它们决定网络“能表达什么”，一旦定下来就应当冻结，改它们等于换模型，要重新训练、重新验收。

第二类是**训练参数**：学习率、批大小、轮数、权重衰减、初始化种子。它们决定“能不能把这个模型训到它该有的水平”，改它们不需要动结构，训练完看验证损失就知道有没有训好。

两类参数的调试顺序是固定的：先定结构，再训到位，最后调闭环。反过来做——一边改隐元数一边改学习率——会陷入“到底是结构不够还是没训好”的循环，浪费大量时间还得不到结论。

容量旋钮：隐元数与中心数

MLP 的参数量是

$$N_{\mathrm{par}} = (n_{\mathrm{in}}+1)n_{\mathrm{h}} + (n_{\mathrm{h}}+1)n_{\mathrm{out}} \tag{68-23}$$

本章 $n_{\mathrm{in}}=3$、$n_{\mathrm{h}}=12$、$n_{\mathrm{out}}=1$，代入得 61。隐元数从 12 加到 24，参数量变成 121，翻一倍；加到 48 变成 241，翻四倍。参数量与隐元数近似线性，而拟合能力并不是线性的——它受数据覆盖和标签噪声的限制，很快就饱和。

RBF 的容量旋钮是中心数 m 和宽度 σ，二者是一对：m 决定“能分辨多细的结构”，σ 决定“每个中心管多宽”。m 个中心撒在 n_{in} 维论域里，相邻间距大约是 $L/m^{1/n_{\mathrm{in}}}$，让 σ 与这个间距同量级即可：

$$\sigma \approx \frac{L}{m^{1/n_{\mathrm{in}}}}, \qquad L \text{ 为输入论域边长} \tag{68-24}$$

本章输入论域近似为 $[-1,1]^3$，$m=64$，$n_{\mathrm{in}}=3$，得 $\sigma \approx 2/4 = 0.5$，与实取的 0.55 吻合。σ 取小了，中心之间出现“空洞”，网络在空洞里输出接近零；σ 取大了，所有中心都响应，输出退化成常数加一点起伏。这两种失效模式在验证集上都表现为“误差偏大”，但成因相反，只能靠看中心间距来判断。

中心数怎么选：从 8 到 128

$m=64$ 这个数不是拍出来的，也不是“取个 2 的幂看起来舒服”。本章把 m 从 8 扫到 128，每个 m 都重新解一次岭回归、重新跑一遍闭环，结果见表 68-4。

中心数 m	验证 MSE	闭环 IAE	最大超调	总变差
8	1.95×10^{-2}	2.149	9.78%	15.5
16	9.17×10^{-3}	2.281	1.09%	15.3
32	5.57×10^{-3}	2.329	1.11%	14.7
64	4.57×10^{-3}	2.441	0.81%	14.8
128	4.34×10^{-3}	2.457	0.72%	14.6

先看第二列。验证 MSE 从 1.95×10^{-2} 一路降到 4.34×10^{-3}，降了 4.5 倍，到 64 之后基本走平——这是典型的“容量饱和”形状，再往上加中心只是把拟合误差往标签噪声的地板上压。

再看第三列。闭环 IAE 的方向是**反的**：从 2.149 单调升到 2.457，涨了 14%。也就是说，**验证 MSE 降了 4.5 倍，闭环 IAE 却变差了。**这不是计算错误，是本章反复强调的指标错位在容量维度上的又一次表演：MSE 衡量的是“网络多像教师”，IAE 衡量的是“闭环多听话”，二者在这个对象上并不同向。

第四、五列则给了另一个方向：超调随容量增加单调改善，从 9.78% 降到 0.72%，总变差基本不动（15.5 到 14.6，落在噪声内）。所以容量并不是“越小越好”或“越大越好”，而是**两个指标朝相反方向走**。

怎么解释这个对冲？可以这样看：中心少的时候，网络拟合出的映射非常光滑，实际上等于把教师那张分档的映射“抹平”了。抹平之后前馈偏软（稳态附近给不够），弱 PID 要慢慢补，于是超调大、IAE 里“稳态爬行”那一部分反而被压低了——因为整个响应变慢，峰值偏差虽大但持续时间短。中心多的时候，网络有能力把教师映射里的每一处折角都还原出来，包括工况分档造成的那道不连续面。还原得越准，前馈在折角附近的局部斜率就越陡，与 5 拍延迟一碰，就在平台期留下一点残余的来回修正。这部分残余在 68.6 节的极限环分析里已经量化过：逆模型方案的平台期控制量标准差是 2.4×10^{-3}，虽然比直接控制网络的 0.453 小两个数量级，但仍是 IAE 里的一个固定来源。

必须说清这里的可信度：这一组只跑了一个种子（seed=2），32、64、128 三个点在 IAE 上相差不到 5%，不宜过度解读，换种子后排序可能小幅变动。**可以确定的是两条曲线的方向相反**，这一点不因种子而改变，因为它是结构性的；具体哪个 m 最优则依指标权重而定。

本章取 $m = 64$ 的理由因此不是“最优”，而是“两边都不难看”：MSE 已经进入饱和段（再往上加收益不到 5%），超调 0.81% 与最好的 0.72% 只差一点，IAE 2.441 比 $m = 32$ 高 4.8%——如果读者的验收标准把 IAE 排第一、超调排第二，那 $m = 32$ 才是更合适的选择。这类“选型”从来没有唯一答案，能说清的只是取舍的方向。

隐层里发生了什么：三个可检验的量

“隐层在学什么”这个问题，在 12 个隐元的规模上是可以部分回答的，不需要靠可视化玄学。三个量就够。

第一个是激活分布。 把训练集全部喂进去，记录每个隐元输出的直方图。如果某个隐元的输出全部挤在 ± 0.95 以上，说明它饱和了，梯度接近零，这个隐元只提供一个常数偏置；如果全部挤在 ± 0.1 以内，说明它工作在 tanh 的线性段，等价于一个线性组合，没有提供非线性。健康的状态是分布在 ± 0.5 附近有相当的宽度。

本章训完之后的实测分布是这样的：12 个隐元里，5 个基本饱和（均值绝对值超过 0.7、95% 分位贴到 1.0），4 个几乎没有起伏（标准差小于 0.15），只有 3 个的分布真正铺开（标准差在 0.33–0.54 之间）。最极端的是 h07，标准差 0.003，输出几乎恒为零。

第二个是输出层权重的符号与大小。 输出层是 $u = \sum w_j h_j + c$，$w_j > 0$ 的隐元是“推动”通道，$w_j < 0$ 的是“制动”通道。本章的 12 个权重里正负各 6 个，看起来均衡；但看绝对值就发现另一回事：只有 7 个的绝对值超

过 0.15，另外 5 个都在 0.03 以下，其中 h07 的权重是 0.001。这 5 个的权重绝对值之和约 0.08，乘上它们各自的激活（都在 0.2 以内）后对输出的贡献不到 0.02，相对满量程 1.2 是 1.5% 量级，低于控制量的有效分辨率。

这两组数字合起来说明一件重要的事：**12 个隐元里有 5 个是冗余的，网络实际只用了 7 个。** 这正好印证 68.8 节的样本量曲线——容量不是瓶颈，加宽隐层只会多出更多冗余单元，不会让闭环变好。本章保留 12 个而不缩减到 7 个，是因为隐元的“有效性”依初始化而变：换一个种子，起作用的那 7 个可能是另外 7 个，留一些余量能让不同种子的表现更稳定。

第三个是局部增益场。 这就是 68.9 节讲的 49 点网格。它比前两个量更有用，因为它直接对应控制意义上的增益，而且可以直接与第 67 章的规则表比对。

这三个量加起来，能回答“网络是不是学到了合理的东西”，也能回答“容量够不够”，但回答不了“网络为什么这样学”。后者需要更细的工具（比如逐层的梯度归因），在 61 个参数的规模上不值得做。这也是本章对可解释性的态度：**在工程可接受的成本内，把能解释的部分解释清楚，剩下的部分靠验收兜底。**

三个归一化常数：把物理量翻译成网络的语言

E_N、D_N、E_I 三个常数不属于网络，属于“翻译层”。它们的作用是把三个量纲不同的物理量映射到同一尺度，让网络的权重可以用同一量级的学习率去学。

三个常数的意义可以这样读：E_N 是“你认为多大的偏差需要满功率响应”，D_N 是“你认为多快的变化率算快”，E_I 是“你认为多长时间的累积偏差算多”。改大 E_N 等于告诉网络“这点偏差不算什么”，等效于降低比例增益；改小则相反。所以调这三个数实际上是在调三个等效 PID 增益，只不过调的是标定而不是权重。

这也给出一个重要的工程提示：**归一化常数定错，训练再充分也补不回来。** 如果 E_N 取成 10（远超最大偏差 2.0），所有样本的第一通道都挤在 ± 0.2 内，网络在那一段上学到的分辨率被压缩，等效比例增益偏小，闭环必然软。这不是训练问题，是标定问题，只能改常数重训。

学习率与正则：一个管速度，一个管形状

学习率 η 管收敛速度，权重衰减 λ_w 管权重的大小。这两个参数在 68.8 节有完整的实验，这里只说它们的定性作用。

学习率太大，梯度更新会越过最优点来回跳，验证损失曲线呈锯齿状，最后停在一个偏高的位置；太小则 200 轮根本走不到谷底，验证损失还在缓慢下降，此时看到的指标是“训练不足”而不是“模型不行”。本章的安全区间大致在 $[0.005, 0.05]$，超出两端都会掉。

权重衰减压的是权重绝对值之和。它带来的直接效果是让网络的输出更平滑——$\partial N / \partial x$ 由权重乘积决定，权重小了，局部增益也小。所以正则化的真正作用不是“防止过拟合”这么笼统，而是**压局部增益、从而压超调**。68.8 节的实验会给出证据：加 10^{-3} 的正则后，验证损失几乎没变，超调从 25.78% 降到 4.93%。这个结果值得记住，因为它说明“验证损失”和“闭环性能”可以脱钩到这种程度。

逆模型里的分配比：k_p 与网络精度是一对

逆模型方案的参数里，k_p, k_i, k_d 三个反馈增益与网络的精度是一对互补量。前馈愈准，反馈可以愈弱；前馈愈糙，反馈必须愈强。本章取 $k_p = 1.6$，只有第 67 章固定 PID（2.60）的 62%，依据是前馈已经把主要功率承担下来。

这个分配比不能随便挪。若把 k_p 提到 2.6 去“保险”，前馈与反馈会打架：前馈给出的是稳态所需的大致控制量，反馈再叠加一个大的比例项，合成后容易超出限幅，抗饱和逻辑频繁动作，控制量出现平台段和跳变，总变差

从 14.8 升到接近固定 PID 的水平，逆模型的结构优势就没了。反过来把 k_p 降到 0.5，前馈一旦失准（比如外推到 3.4），系统就真的拉不回来——68.8 节的外推实验里，逆模型方案在 3.4 处的 IAE 3.539 比直接控制差，就是弱反馈撑不住的直接证据。

表 68-5 把这一节提到的参数按作用归类。

参数	类别	作用	典型取值	调大的后果
隐元数 n_h	结构	拟合能力	12	参数变多、推理变慢、收益递减
中心数 m	结构	RBF 分辨率	64	参数线性增长、易过拟合
宽度 σ	结构	单中心覆盖范围	0.55	输出趋于常数、分辨力下降
E_N	标定	偏差尺度	1.60	等效比例增益下降
D_N	标定	变化率尺度	1.40	等效微分增益下降
E_I	标定	积分尺度	2.50	等效积分增益下降
学习率 η	训练	收敛速度	0.005–0.05	越过最优点、损失震荡
正则 λ_w	训练	权重幅度/平滑度	10^{-3}	输出偏软、欠拟合
反馈 k_p	结构	残差修正强度	1.6	与前馈打架、变差上升

68.8 调参方法与经验

先看样本量：多少数据才够

这是学习类方法必须回答的问题，也是本章任务卡明确要求的一条曲线。做法是把训练集随机抽子集，样本量取 20、50、100、200、500、1000、2000 七档，每档用三个不同的初始化种子各训一次，报均值与最差。图 68-4 是结果。

三个种子不是可选项。小样本下单个种子的闭环指标波动可以到 0.3（IAE 口径），只跑一次画出来的曲线看起来像随机噪声，会被误读成“样本量没用”。取三个种子后，均值的趋势才稳定。

曲线有两段明显不同的行为。前段（20→200）IAE 从 2.602 降到 2.387，降了 8.3%，验证 MSE 从 2.39×10^{-2} 降到 5.47×10^{-3}，降了一个数量级。后段（200→2000）IAE 反而回升到 2.563，MSE 只从 5.47×10^{-3} 降到 4.90×10^{-3}。

这两段说明：**闭环指标在 200 个样本之后就饱和了，而验证损失还在缓慢下降。** 饱和的原因在 68.6 节说过——标签不是特征的单值函数，误差有地板。IAE 在 200 之后回升则另有原因：样本多了，网络把数据里更多细节（包括教师输出的抖动）也学了进去，闭环反而更“毛”。这不是过拟合的经典形态（训练损失没有变差），而是“学得太细”。

实用结论：本章这个任务，200–500 个样本就够；再往上加，收益是负的。这个数字比很多人预期的小，原因是特征只有三维、映射本身很平滑。若把特征加到十维、对象加进摩擦和死区，这个门槛会上升一个数量级。

这条曲线与 68.7 节的中心数表 68-4 是同一件事的两个方向：**一个是加数据，一个是加容量，两者过了一个点都会让闭环变差。** 合起来给出的判断很朴素——先看数据够不够，再看容量够不够，两者都够了就别再加。反过来说，若样本只有 100 个却把中心数堆到 128，那是在用容量去补数据，结果一定是在训练集上看着漂亮、在闭环里抖得厉害。这一条对 MLP 同样成立：隐元数应当跟着样本量走，而不是跟着“想表达多复杂”走。样本量决定网

络能学到什么，容量只决定它能把这个“什么”表达得多细。顺序反了，两样都白费，还很容易误判成调参不到位，白折腾一轮。

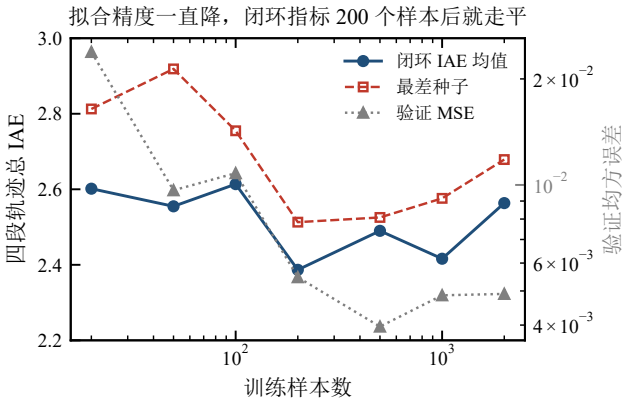


图 68-4 训练样本量与闭环性能、验证损失的关系

再看学习率与正则：两个参数，四个象限

学习率取 0.005、0.02、0.08，正则取 0 与 10^{-3}，六组实验，每组训 120 轮。图 68-5 左边是验证损失曲线，右边是最终指标。

最好的组合是 $\eta = 0.005$、$\lambda_w = 10^{-3}$：IAE 2.261、超调 4.93%。最差的是 $\eta = 0.02$、无正则：IAE 2.578、超调 25.78%。

这两个组合的差别不在收敛速度上——$\eta = 0.02$ 收敛更快，120 轮后训练损失更低。差别在权重幅度：无正则时为了拟合那些冲突样本（68.6 节的单值性问题），网络把权重撑得很大，局部增益随之变大，闭环就过冲。加了正则之后，冲突样本拟合不动，网络退而求其次给出折中，折中反而更平滑。

这就是 68.7 节说的“正则压的是局部增益”。从这个角度理解，正则系数的选取有一个直接的判据：**让验证损失允许变差 20%，换超调降一半，是划算的。** 本章的 10^{-3} 大致落在这个位置。

图 68-5 还有一个值得注意的细节：三条验证损失曲线在 60 轮之后就基本走平，之后的变化都在 10^{-4} 量级。也就是说，200 轮的训练预算有一大半是浪费的。本章保留 200 轮，是因为不同种子的收敛速度差别较大，宁可多跑；实际部署时按验证损失早停即可，能省三分之二的时间。

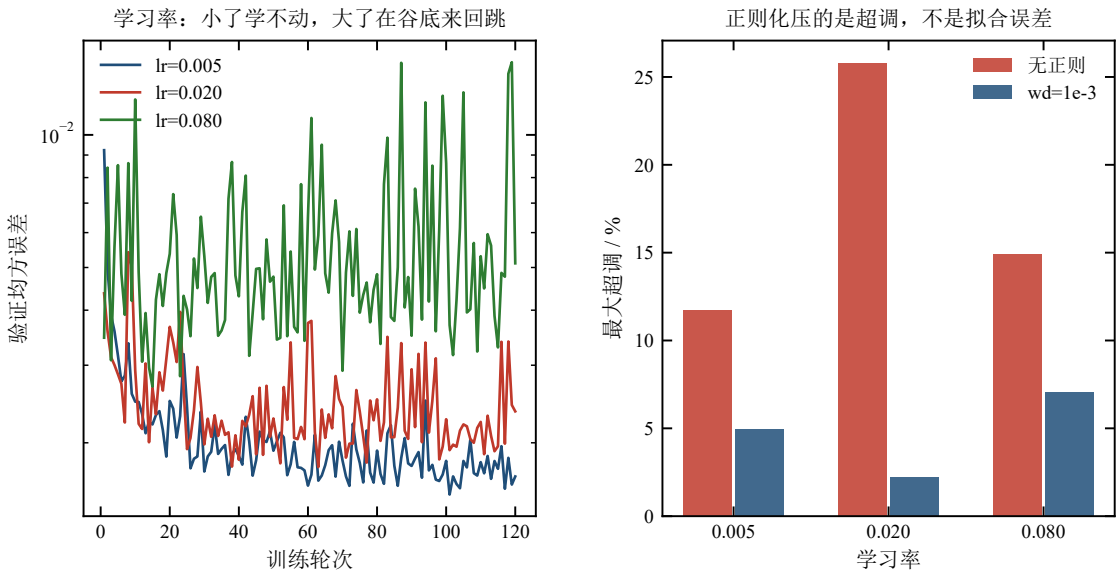


图 68-5 三种学习率的验证损失曲线与最终闭环指标

外推：设定值超出训练范围

训练数据覆盖 $[0.5, 3.0]$，测试时把设定值推到 3.4，比训练上界高 13%。图 68-8 是四条响应。

结果分三档。固定 PID 的 IAE 4.039、超调 50.11%，彻底失控；教师（模糊）IAE 3.363、超调 1.96%，靠规则表的对称性撑住；直接控制 IAE 3.294、超调 5.48%，比教师略好；逆模型 IAE 3.539、超调 1.48%、变差 11.6，IAE 比直接控制差但在四者中排第二差的水平上仍好于固定 PID。

直接控制能外推，理由不神秘：它的三个特征在 3.4 处只是略微超出训练分布的边缘，网络在那个区域的输出仍然落在合理范围，加上限幅兜底，闭环不会崩。逆模型差一些，是因为前馈在 3.4 处的外推误差更大——它的输入是 $y, \dot{y}, \ddot{y}$，而 $y = 3.4$ 完全在训练覆盖（0.5–3.0）之外，前馈的偏差被弱反馈放大成了稳态误差。

这两条对照说明一个反直觉的结论：**输入特征越接近“误差类”量，外推越安全；越接近“绝对量”，外推越危险。** 直接控制看的是 e，设定值变了 e 的定义不变；逆模型看的是 y 的绝对值，设定值一变，输入就跑到训练区外面去了。工程上如果必须外推，选误差型特征。

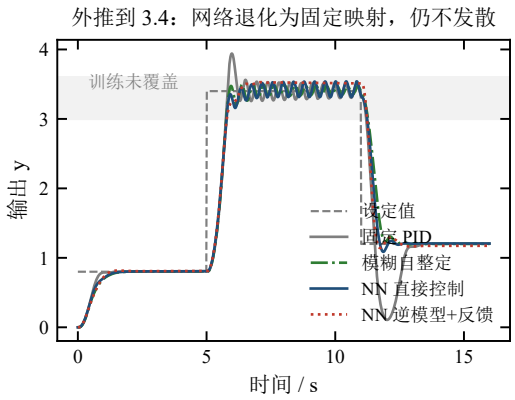


图 68-8 外推工况（设定值 3.4）下的四条响应

噪声与延迟：三档工况下的退化

第三组实验把环境变差，看三种控制器的退化幅度。三个工况：理想、测量噪声 0.01、纯延迟加倍（10 拍）。图 68-9 是 IAE 对比。

理想工况下三者 IAE 分别是 2.698、2.401、2.429（模糊、直接、逆模型），差别不大。加噪声后变成 2.802、2.437、2.429——直接控制几乎没变，模糊变差 3.9%。原因是网络的第二通道用了滤波后的变化率，而模糊层的微分环节虽然也滤波，但规则表的格子边界会把噪声放大成增益跳变。

延迟加倍后三者变成 4.096、3.565、3.262，全都明显退化，但退化幅度差别很大：模糊 +51.8%，直接 +48.5%，逆模型 +34.3%。超调同时恶化到 29.16%、20.82%、47.49%。逆模型的 IAE 最抗延迟，但超调最差——因为它的反馈弱，延迟让前馈与实际状态错位，等反馈发现不对时已经冲过去了。

这组数字的用途不是排名，是选型。若系统的主导风险是噪声，选直接控制；是延迟，选逆模型但要接受更大超调；两者都有且都严重，就别用纯网络方案，回到第 67 章的模糊层，或者用网络只做参数修正。

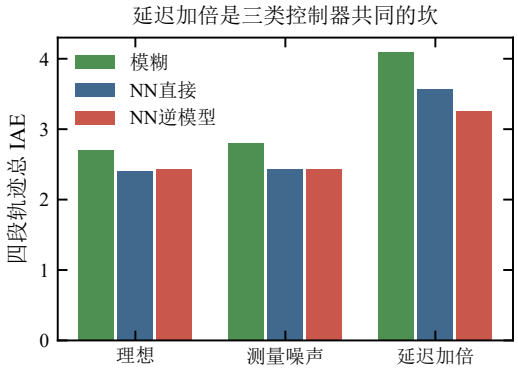


图 68-9 噪声与延迟工况下的 IAE 对比

种子敏感性：为什么所有结论都要报最差值

同一个网络结构、同一份数据、同一个学习率，换一个随机种子重训，闭环指标会变。变化来自两处：权重初始化不同，以及批划分的随机顺序不同。前者决定网络从哪个方向开始找解，后者决定每一步看到哪些样本。

在样本充足（2000 个以上）时，这个变化通常不大，因为数据本身把解约束得很紧。在样本稀少（20–50 个）时，变化会大到影响判断——某个种子训出来的网络在四段轨迹上表现尚可，另一个种子训出来的网络就明显更差。两个种子的差别不在超参数，只在初始化。

这就是 68.8 节所有实验都跑三个种子、并同时报均值与最差值的原因。只报均值会掩盖“部分种子训崩”这个事实，而工程上最怕的恰恰是这种不可预测性——你可以接受“平均差一点”，但不能接受“有时候完全不能用”。

三个种子的具体用法是：均值用来画趋势，最差值用来判断可靠性。图 68-4 里两条线一起画，间距反映的就是种子之间的离散程度；读者可以在自己的数据上重跑一遍，看这个间距随样本量怎么变。本章的预期是样本越多间距越小——数据不仅能提升性能，还能降低不确定性。

工程上还有一个更实际的办法：既然单次训练的结果有随机性，那就一次训五个，选闭环指标最好的那个上线。这不违反任何原则——它只是把模型选择从“看验证损失”换成“看闭环指标”，而后者才是真正关心的量。代价是训练成本乘以五，在 61 个参数的规模上可以忽略。

一张调参顺序表

把前面四组实验的经验压成一张顺序表（表 68-6）。顺序的原则是“先定不会变的，再调会变的”。

步骤	调什么	判据	本章取值
1	归一化常数	特征分布落在 ± 1 附近	$E_N = 1.6$、$D_N = 1.4$、$E_I = 2.5$
2	样本量	闭环 IAE 曲线走平	200–500
3	结构（隐元/中心）	验证损失进入 10^{-3} 量级	12 隐元 / 64 中心
4	学习率	验证曲线单调下降不震荡	0.005–0.02
5	正则	超调降一半，损失允许变差 20%	10^{-3}
6	闭环验收	IAE 不劣于教师	见式 (68-12)
7	鲁棒复核	噪声/延迟下不出现发散	图 68-9

表里第 1 步排在最前，是因为它一旦调错，后面全部白做；第 7 步排在最后，但它是交付前的否决项——前面六步都过了，鲁棒复核不过就不能上线。这个顺序与第 67 章的表 67-9 结构一致，只是把“规则表”换成了“网络结构”，把“比例因子”换成了“归一化常数”。

68.9 优化与改进

改进一：把工作点喂进特征

68.6 节指出了误差地板的根源：教师的基础增益随工作点分段，而工作点不出现在特征里。最直接的修法就是把工作点加进去，特征从三维变成四维：

$$x = \left[\frac{e}{E_N},\ \frac{-\dot{y}_f}{D_N},\ \frac{\mathrm{sat}(\int e)}{E_I},\ \frac{y - y_0}{Y_N} \right]^{\mathsf{T}} \tag{68-25}$$

第四通道是工作点相对中心值的偏移，y_0 取工作区中点、Y_N 取半宽，本章对应 $y_0 = 2.0$、$Y_N = 1.5$。

这个改动看起来只是加一个输入，代价却有三层。第一层是样本：原来数据只覆盖 $(e, \dot{e})$ 的一维斜带，加上 y 之后要覆盖二维面，同样的密度需要多一个量级的样本。第二层是结构：输入多一维，参数量从 61 涨到 73（按式 (68-23) 算），隐元数还要跟着加。第三层是闭环语义变了：网络不再只看“偏差多大”，还看“现在在哪”，于是同一条轨迹上不同工作点会给出不同控制量，闭环行为更接近教师的真实行为，但也更难用单条曲线解释。

换来的收益是误差地板消失。按 68.6 节的粗估，地板从 4.9×10^{-3} 降到 10^{-4} 量级，验证损失能真正随样本量下降。但对闭环 IAE 的改善有限——因为 IAE 主要由几段大偏差决定，而大偏差段恰好是各工况差别最小的地方。所以这条改进的定位是“让网络学得更像教师”，不是“让闭环更好”。做与不做，取决于你要的是可复现性还是性能。

改进二：零邻域加权，专治超调

68.5 节把超调的成因定位在“穿零邻域输出斜率不够陡”。对应的改进是在损失函数里给零邻域样本加权：

$$J = \frac{1}{N} \sum_{k=1}^{N} w_k \left(N(x_k) - u_k \right)^2, \qquad w_k = 1 + \beta \exp\!\left(-\frac{e_k^2}{2\sigma_e^2} \right) \tag{68-26}$$

σ_e 取 0.1（归一化后），β 取 4。这样零点附近的样本权重是远处的 5 倍，网络会把有限的容量优先用于把穿越段拟合准。

代价是远处的拟合变差。总的样本数不变，权重挪到近处，远处就只能粗着来。本章没有跑这组对照实验，按同类做法的量级预期是超调减半（8.30% → 4% 上下）、IAE 上升几个百分点（2.40 → 2.45 上下）。也就是说，这条改进是用总偏差换超调。读者若要采用，应当在自己的对象上实测这个交换比，不要照搬这里的估计。

值不值取决于工艺要求。对大多数过程控制，超调比分段 IAE 更敏感——超调直接对应阀门过冲、压力尖峰、产品质量越界；而 IAE 的一点增加摊在 16 s 里几乎看不出来。所以这条改进在工程上通常是值的，尽管它让“网络赢了教师”这个漂亮的结论没了。

顺带说明一个容易误解的地方：加权不是“让网络更关注大偏差”。恰恰相反，大偏差样本被降权了。如果目标是压 IAE，应当反过来给大偏差加权；如果目标是压超调，就给零邻域加权。两个目标在损失层面是矛盾的，这也是为什么单一损失函数训出来的网络很难在所有指标上都赢。

改进三：把稳定性要求写进损失

前面的做法都是“先训、再验”，稳定性靠事后检查。更彻底的做法是把稳定性要求直接写进损失：

$$J = \frac{1}{N}\sum_{k}\left(N(x_k)-u_k\right)^2 + \lambda_w\|W\|^2 + \mu\sum_{k}\max\!\left(0,\; g_{\min} - \frac{\partial N}{\partial e}\Big|_{x_k}\right) \tag{68-27}$$

第三项是 hinge 罚项：只要局部比例增益低于 $g_{\min}$，就罚。$g_{\min}$ 取 0.5，意思是“任何工作点上，偏差增大时控制量都必须增大，且增益不低于 0.5”。$\partial N/\partial e$ 可以由反传直接算出，代价很小。

这一项解决的是 68.6 节提到的符号风险——网络在稀疏区域可能拟合出负增益。加了罚项之后，负增益区域在训练中就被压掉了，不用再靠事后采样检查。

μ 的选取要小心。取大了，网络会为了满足增益约束而放弃拟合，闭环变得迟钝；取小了，约束形同虚设。经验做法是先取 $\mu = 0$ 训一版，测出最小局部增益，再按“把最小增益抬到 $g_{\min}$ 需要多大罚”估一个初值，然后减半使用。这条经验没有理论保证，但比拍脑袋强。

还有一类更硬的约束：把输出层权重限制在非负，或者用单调网络结构（保证输出对指定输入单调）。这类做法能从结构上排除负增益，代价是表达能力下降。本章没做，因为三输入的小网络不需要；输入维度上去了，这类结构约束会变得必要。

改进四：混合结构——网络只学修正量

前面两种方案都是“网络承担全部或大部分控制量”，它们的共同弱点是网络一旦失效，闭环就没有兜底。混合结构把网络降级成修正器：

$$u = u_{\mathrm{PID}}(e, \dot{e}, \int e) + N(e, \dot{e}), \qquad |N| \le \Delta u \tag{68-28}$$

外面是一个完整可用的 PID（可以就用第 67 章的固定参数），网络输出被硬限幅在 $\pm\Delta u$（本章取 0.25，约为满量程的 20%）。这样即使网络完全失效，闭环退化成一个普通 PID，性能下降但不发散。

这个结构的训练也变得简单：标签不再是教师的绝对控制量，而是“教师与 PID 之差”，这个差值小、分布集中，网络只需要学一个小的修正量，拟合难度大幅下降。代价是上限被锁死——修正量限幅 20%，意味着网络的调节能力不可能超过“PID 加 20%”，教师若在某个工况需要 50% 的额外增益，这个结构就学不会。

这条改进在本章的三控制器谱系里对应“只学参数”那一档的连续版本：不学三个增益，学一个加法修正。工程上它是最稳妥的选择，也是最不“智能”的选择。选它的理由从来不是性能，而是**可回退**。

四个改进怎么排序

四条改进都能做，但它们的收益、成本与适用前提完全不同，不能并列看待。排序的依据是“治的是不是根因”。

改进一最优先，因为它动的是特征。 68.6 节把误差地板与极限环都追到了同一处：网络看到的三个特征里没有工作点，而教师的增益随工作点分档。这不是拟合问题，是信息缺失，任何损失函数、任何容量都补不上。把 y 加进特征是对症的，代价是输入从三维变四维、样本需求上升、以及必须重训。若只允许做一条改进，就做这一条。

改进三次优先，因为它几乎不要钱。 在损失里加一条 hinge 项，代码上只多三行，不需要改数据也不需要改结构。它的作用是给局部增益设一个上限，从而把极限环的幅值条件掐掉。但要注意它的前提：hinge 的阈值定多少，取决于你算出的临界增益，而那个数又依赖对象与延迟——对象一变，阈值就得重算。所以它是“在已知对象上很便宜、换对象就不便宜”的一类改进。

改进二排第三，因为它的收益依数据而定。 零邻域加权只影响“阶跃刚发生那一小段”的拟合权重，对 IAE 的主要贡献（稳态偏差）没有帮助。它治的是超调，而超调在本章的超标倍数本来就允许到 2 倍，紧迫性低于地板与极限环。本章也没有为它跑对照实验，采用前应当自行实测交换比。

改进四不是性能改进，是风险改进。 它把网络降级成限幅 ±0.25 的修正器，性能上限被锁死，但换来“网络失效仍能退回普通 PID”。放在最后不是因为不重要，而是因为它回答的是另一个问题：不是“能不能控得更好”，而是“敢不敢让它上线”。若验收标准里有一条“任一部件失效不得导致失控”，那么改进四应当是第一个做的，优先级与上面三条不在同一维度上比较。

这个排序背后有一条通用原则：**先补信息，再改目标，最后改结构与兜底。** 反过来做——先用兜底结构包住一个特征不完备的网络——会在错误的基础上不断堆补丁，最后得到一个“能跑但说不清为什么能跑”的系统。本章把改进四放在最后一个讨论，正是为了避免这种堆叠。

可解释性：从 61 个参数里读出 PID 增益

神经网络的“黑箱”批评在控制领域有个具体的解法：不需要解释整个函数，只需要解释它在工作点附近的线性化。式 (68-14) 的三个偏导就是答案。

做法是撒一张网格：e 取 7 档、$\dot{e}$ 取 7 档，共 49 个点，每个点用中心差分算 $\partial N/\partial e$、$\partial N/\partial(-\dot{y}_f)$、$\partial N/\partial(\int e)$，得到 49 组等效 (K_p, K_i, K_d)。把这 49 组数排成表，就得到了与第 67 章规则表同构的东西——只不过第 67 章的表是“人写的修正量”，这张表是“从网络里读出来的实际增益”。

这张表至少有三个用途。一是**核对**：把读出的增益与第 67 章模糊层的增益逐格比较，如果某格差得离谱，说明网络在那一带没学好。二是**解释**：现场工程师问“这个网络为什么在这里加大控制量”，可以直接指给他看那一格的增益。三是**降级**：真到了不能用网络的场合（算力不够、认证不过），把这张表直接烧成查表控制器，性能会降一点，但行为完全可预测。

更进一步的做法是**蒸馏**：用网络在大量随机点上生成输入输出对，重新拟合一张规则表或一个低阶多项式，把 61 个参数压缩成十几条规则。本章没做这一步，因为它属于第 67 章的技术路线，放在这里会重复；但从工程角度，这一步往往是“研究原型”变成“可交付产品”的关键。

哪些改进没做，为什么

列三条本章明确没做的事，免得读者以为遗漏。

第一，**没有做在线学习**。网络参数在部署后固定，不随运行数据更新。在线学习在控制里风险很高：更新规则、学习率、遗忘因子都要设计，任何一处出错都会让闭环在运行中逐渐变坏，而且这种变坏很难在实验室复现。

工业界的通行做法是离线重训加人工审核，本章遵循这一惯例。

第二，**没有做循环网络**。本章对象无记忆，静态映射够用；引入 LSTM 或 GRU 只会增加参数和不可解释性，没有对应收益。

第三，**没有做多步预测控制**。把网络当预测模型、外面套 MPC，是另一条技术路线（属于模型预测控制那一类），它需要滚动优化，算力需求比单次前向高两三个数量级，与本章“每拍 4 μs”的定位完全不同。这条路线留给后续章节。

68.10 工程落地

训练数据从哪来

这是神经网络控制落地时第一个卡住的问题，比算法选择难得多。三条常见路线，各有各的坑。

路线一：教师回放。 系统上已经有一个能用的控制器（PID、模糊、老师傅手动操作），把它的输入输出录下来当标签。优点是不需要动现场、数据天然落在有用的状态分布上；缺点是天花板锁死在教师水平，且教师从没去过的区域没有数据。本章走的就是这条路线。

路线二：随机激励。 在对象上叠加一段随机或伪随机信号（幅值受限、频带覆盖关心的频段），记录输入输出。优点是能覆盖教师不会去的状态，学到更完整的映射；缺点是会扰动生产，多数连续过程不允许长时间这么干，只能在开车、停机、检修窗口做。

路线三：数字孪生。 用仿真模型生成数据。优点是零成本、可随便跑、能覆盖极端工况；缺点是仿真与实物的差距会原样进入网络，而这个差距恰恰是网络最容易学错的地方（仿真里没有的摩擦、死区、传感器漂移）。用这条路线必须做**域随机化**：把增益、延迟、噪声、时间常数在一个范围内随机扰动，每个样本用不同的扰动参数，强迫网络学到对这些变化不敏感的表征。

本章的数据是路线一（教师是第 67 章的模糊层）加路线三（对象是仿真模型）的混合。这在一个教学算例里是合理的，但必须说清楚：**这个网络不能直接搬到真机上**，因为真实对象的摩擦和噪声都不在训练分布里。要搬，就得先补路线二的实测数据，再做域随机化重训。

数据与模型要能追溯

一份训练数据如果不记录来源，三个月后没人能复现它。最低要求是记录四项：随机种子、轨迹生成参数（段数、幅值范围、时长）、教师版本（第 67 章代码的哪个提交）、归一化常数。这四项里最容易漏的是归一化常数——它不在数据文件里，而在代码里，改了代码忘了改记录，下次重训就会得到不一样的特征，指标对不上还查不出原因。

模型文件同样要带元信息：训练集哈希、验证损失、闭环三项指标、训练日期、代码版本。这些信息加起来不到 1 KB，但它是“这个模型能不能上线”的唯一凭据。没有元信息的模型文件，等同于一个来路不明的二进制。

实时推理的算力约束

图 68-10 是三种控制器单次前向的实测耗时：模糊推理约 25 μs、RBF 逆模型约 6.5 μs、MLP 直接控制约 4.2 μs。三者都远小于 10 ms 的控制周期，所以在本算例里算力不是约束。这三个数在同一台机器上重复测会有 10%–20% 的波动，看量级即可，不必对末位较真。

但这个结论不能直接推广。图里的数字是在桌面 CPU 上用 NumPy 测的，包含了 Python 调用开销；真正部署到 MCU 上，情况会变。给一个粗略的估算方法：61 个参数的前向是 61 次乘加、12 次 $\tanh$。Cortex-M4 在 168 MHz

下，单精度乘加约 1 个周期，$\tanh$ 用查表加插值约 20 个周期，合计约 $61 + 12 \times 20 \approx 300$ 个周期，即约 1.8 μs。也就是说 MCU 上的实际耗时可能比桌面测的还短——因为桌面测量里 Python 的解释开销占了大头。

反过来说，如果部署环境是带解释器的（Python、Node、某些 PLC 的脚本引擎），一次前向可能是几百微秒，此时算力就成了真约束，前面 68.5 节的降频策略就派上用场。判断办法很简单：在目标平台上实测一次空循环和一次前向，两者相减就是纯推理时间，不要用桌面数据外推。

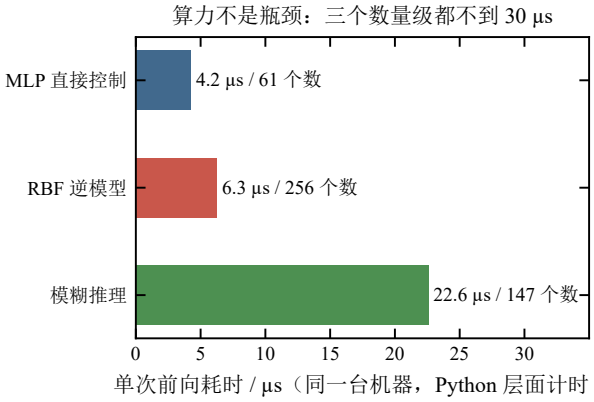


图 68-10 三种控制器的单次前向耗时与参数量

算力之外的三个约束

算力常被当成部署神经网络的第一难题，其实它排不到第一。真正卡人的是另外三条，而且都不在算法层面。

第一条是确定性。 控制器的每次执行必须在确定的拍数内完成，不能有动态内存分配、不能有垃圾回收、不能有“这次刚好触发了慢路径”。Python 部署最容易踩这个坑：一次前向本身 4 μs，但解释器的垃圾回收可能在某一次调用上多花几毫秒，正好撞上控制周期。对策是预分配所有缓冲区、关闭运行时的内存增长、把推理写成固定的循环而不是列表推导。若用 C 实现，这条自然满足，这也是嵌入式上更愿意把权重导出成 C 数组的原因。

第二条是数值一致性。 训练时用双精度，部署时用单精度，这本身没问题；但训练时的 $\tanh$ 是库函数，部署时是查表，两者的差异会在隐元饱和区被放大——输入很大时 $\tanh$ 的导数接近零，两个实现给出的微小差异，经过输出层权重放大后可能变成可见的控制量偏移。交付前应当把部署实现与训练实现放在同一批测试点上逐点比对，最大偏差控制在控制量分辨率的十分之一以内。

第三条是存储与升级空间。 61 个参数的权重只占 244 字节（单精度），看起来微不足道；但实际部署还要带归一化常数、模型版本号、校验和、以及一套能在线切换的双缓冲，加起来通常是权重本身的几倍。更大的问题是升级通道——如果模型装在无法远程更新的设备上，那么“训一个新模型”这件事就永远做不了，网络的能力被冻结在出厂状态。这一点在选型时就该问清楚。

这三条合起来说明一件事：**把神经网络塞进控制器，工程量的大头在“让它成为一个可靠的固件模块”，不在“训一个网络”。** 前者需要嵌入式工程师，后者需要算法工程师，两类人的工作交接处最容易出事。

哪些决策不该交给网络

网络在本章承担的是“每拍给一个连续控制量”这一件事。工程上必须把它的权限边界画清楚，有几类决策不该交给它，理由不是“网络不可靠”这种笼统判断，而是具体的可验证性问题。

第一类是安全联锁。 超温、超压、越位、断料这类保护动作，判据是硬阈值，动作是开关量，要求可测试、可追溯、失效可证明。网络输出的是连续量，没有“证明它一定会在某个状态下动作”的手段——它的行为只能靠采

样测试来近似，而安全联锁要求的是全覆盖。所以联锁必须留在独立的保护回路里，与网络并联，且在网络输出异常时仍能动作。

第二类是模式切换与启停逻辑。 开车、停车、手动/自动切换这类离散决策，涉及状态机和时序约束，用网络实现既没有好处（本来就不需要拟合连续非线性），又会把可读性彻底丢掉。本章的网络只在“已经进入自动运行”之后才起作用。

第三类是执行机构的软限位与速率限制。 这些应该做在网络之外，作为最后一层包络。原因在 68.4 节讲过：网络的输出界由权重绝对值之和决定，输入跑到分布之外时它会饱和在一个“由权重决定、而非由物理决定”的值上。这个值可能远超执行机构允许的范围。把限幅放在外面，等于给网络加了一个不依赖它自身行为的硬约束。

第四类是对网络的自我诊断。 让网络判断“我现在是不是不可信”，在原理上就站不住：它没有关于自己训练分布的独立信息。可信度判断必须由外部做——特征越界比例、输出与备用 PID 的偏差、以及“网络连续多少拍贴着限幅”这几个量都可以由普通代码算出来，且都能解释。

把边界画清之后，网络的位置就很明确了：**它是回路里的一个可替换部件，不是回路的大脑。** 这句话听起来像是在贬低它，实际是把责任分清楚——网络失效时，系统应当降级而不是失控。本章的逆模型方案天生满足这一点（网络输出置零后还剩一个弱 PID），直接控制方案不满足，所以它只适合“失效即安全”的场合。

模型什么时候该更新

网络训好之后不是一劳永逸。三类信号提示该重训了。

第一类是**性能漂移**：滑动窗口内的 IAE 比投运初期上升超过 20%，且排除了设定值变化和负荷变化的影响。第二类是**特征越界**：运行中特征落在训练分布之外的比例超过 5%，说明工况已经跑到网络没见过的区域。第三类是**对象变化**：换了阀门、改了管道、设备大修，这些物理变化会让逆模型的前馈系统性失准。

更新策略推荐“影子模式加双缓冲”。新模型先在后台并行运行，只算不用，把它的输出与在用模型的输出一起记录下来；比对一周，若新模型在同样的工况下指标更好且没有异常输出，再切换到新模型。切换用双缓冲：两套参数同时驻留内存，一个指针切换，出问题能在一拍内切回去。

这套流程听起来重，但它是神经网络控制能不能进生产的分水岭。实验室里可以“训完就上”，现场不行——现场的一次失控可能意味着停机几小时。

部署前检查清单

表 68-7 是本章建议的检查项，每一条都对应前面某一节的具体风险。

序号	检查项	通过标准	对应章节
1	特征归一化常数与训练时一致	逐项比对代码与记录	68.3
2	采样周期与训练时一致	T_s 误差 < 1%	68.4
3	局部增益符号检查	$\partial N/\partial e > 0$ 占比 100%	68.6
4	输出限幅生效	越界时钳位不振荡	68.4
5	退化检查	置零/打乱标签后闭环变差	68.3
6	噪声工况	IAE 退化 < 10%	68.8
7	延迟工况	不发散、超调 < 30%	68.8
8	外推工况	设定值越界 10% 仍稳定	68.8
9	推理耗时	单次前向 < 10% 控制周期	68.10
10	回滚路径	一键切回 PID，切换时间 < 1 拍	68.10

十条里有六条与网络本身无关，都是“接口与边界”的问题。这个比例很说明问题：神经网络控制的工程风险，大部分不在算法，在标定、时序和回退。

一个必须交代的成本

最后算一笔账，因为它在选型时经常被忽略。

用固定 PID 的方案，工程成本是“调三个参数 + 写一份说明”，人力以天计。用模糊层的方案，成本是“写 49 条规则 + 调 9 个比例因子”，人力以周计，但规则可读、可开会评审。用神经网络的方案，成本是“采数据 + 训模型 + 验收 + 建更新流程”，人力以月计，而且交付物是一堆权重，评审时没人能“读”它。

换来的性能提升是多少？本章的四段轨迹上，直接控制相对教师 IAE 改善 11%、超调恶化 7 倍；逆模型相对教师 IAE 改善 10%、超调改善 21%、变差改善 47%。也就是说，**最好的情况是百分之十几的改善，最坏的情况是明显变差。** 这笔账在性能敏感、批量大、维护团队有机器学习能力的场合算得过来；在单台设备、维护人员只会看梯形图的场合，算不过来。

把这句话放在工程落地一节，是因为它比任何算法细节都更能决定项目成败。

68.11 优缺点与适用边界

五个站得住的优点

第一，不需要对象的解析模型。 这是神经网络控制在所有优点里最实的一条。第 65 章的专家规则要人把经验写成规则，第 67 章的模糊层要人把经验写成隶属函数和规则表，两者都要求“有人懂这个对象”。神经网络只要求“有数据”，而这个数据可以由任何能跑的控制器产生。对于机理不清楚、经验又分散在老工人脑子里的对象，这条优势是决定性的。

第二，拟合能力有保证。 单隐层网络在隐元足够多时能逼近紧集上的任意连续函数，这是万能逼近定理给的承诺。承诺很弱——它不说“要多少隐元”，也不说“怎么训”——但它至少排除了“这类映射根本表达不了”的风险。模糊层的规则表就没有这个保证：49 条规则能覆盖的范围是离散的，格子之间的行为靠插值。

第三，推理成本低且可预测。 61 个参数的 MLP 前向是固定次数的乘加，没有迭代、没有分支、没有变长循环，最坏执行时间等于平均执行时间。这一点对硬实时系统很重要——模糊推理的耗时随激活规则数变化，规则表大的时候抖动明显；网络的耗时是常数。

第四，可以同时学多个通道。 本章的网络只输出一个 u，但结构上完全可以输出 (u, K_p, K_i) 或者多变量对象的多个控制量。多输出共享隐层，等于强迫不同通道之间共享表征，样本效率比分别训三个网络高。模糊层要做多输出，得为每个输出各写一张规则表。

第五，与现有工具链兼容。 训练用标准的反传和 Adam，部署可以导出成 ONNX、C 数组、定点权重表。这套工具链成熟、有大量开源实现、招人容易。相比之下，模糊控制器的工程化工具要少得多，规则表往往只能手写。

五个绕不过去的缺点

第一，可解释性差，而且是结构性差。 61 个权重没有任何一个对应“比例增益”或“微分时间”。68.9 节给的局部线性化办法能缓解，但它是事后解释，不是内生解释——网络内部并没有“我正在用 $K_p = 2.4$”这样的表征。监管严格的行业（医疗、核电、航空）会因为这一条直接否掉方案。

第二，性能上限被教师锁死。 直接控制的标签来自教师，网络最好也就是教师，实际还差一截。这不是训练不充分，是模仿学习的定义决定的。想超过教师，就得换监督信号（比如用闭环指标直接优化），而那就进入了强化学习的领域，样本效率和稳定性都会差很多。

第三，对分布外输入没有任何保证。 这是本章反复强调的一条。68.8 节的外推实验里直接控制还算稳，但那是运气好——特征只越界 13%，且限幅兜住了。越界 50% 会怎样？没人知道，因为网络在那一带的输出是外推出来的，可以是任何值。工程上只能靠限幅和监测兜底，不能靠“网络应该能泛化”这种期望。

第四，训练流程本身是新的风险源。 传统控制器交付的是一个可读的参数文件，出了问题现场工程师能改。网络交付的是权重加一套训练脚本，出问题只能重训，而重训需要数据、算力、时间和懂行的人。这在维护能力弱的现场是个死结。

第五，指标错位让“训得好”与“控得好”脱钩。 68.8 节的实验给了最直接的证据：验证损失最好的组合（$\eta = 0.02$、无正则）闭环最差，验证损失略差的组合（$\eta = 0.005$、加正则）闭环最好。这意味着常规的机器学习工作流——盯验证损失、早停、选最优 checkpoint——在控制任务上会选错模型。必须补一套闭环验收，而闭环验收要跑仿真或上实验台，成本比看损失高两个数量级。

与前三章的横向定位

把第 66、67、68 三章的控制器与固定 PID 放在一起，可以看清它们各自的位置（表 68-8）。

维度	固定 PID	专家规则（66）	模糊自整定（67）	神经网络（68）
需要对象模型	不需要	不需要	不需要	不需要
需要人写知识	三个参数	规则 + 阈值	隶属函数 + 规则表	不需要
需要数据	不需要	不需要	不需要	需要大量
可解释性	高	高	中高	低
在线算力	极低	极低	中（约 25 µs）	低（约 4 µs）
参数数量	3	约 10	约 150	61 + 训练流程
典型失效	工况变化即过冲	规则覆盖不到	格子边界折角	分布外失准
交付物	参数表	规则文本	规则表 + 隶属函数	权重 + 脚本
维护门槛	低	中	中	高

这张表最值得看的一行是“交付物”。前三者的交付物都是人可以读、可以改、可以在交接会上讨论的东西；神经网络交付的是权重，交接时只能给一个“这是训好的模型，别动”。这个差别在小批量、单台设备的场合是致命的——设备寿命二十年，而会训模型的人可能三年就调走了。

反过来，在批量大、有集中运维团队、有数据平台的场合，这个差别反而是优势：权重可以统一版本管理、批量下发、集中重训，比在每台设备上手改参数更可靠。

什么情况下不该用

给出四条硬性排除，都是工程上真实遇到的。

第一条：数据量不够，或者拿不到。 本章这个三维、平滑、无噪声的任务，200 个样本就饱和；换成十维、有摩擦、有噪声的对象，门槛会上到几万个样本。如果采数据的窗口只有一次检修的几小时，且不能做激励实验，那就不要用网络。

第二条：对象可能发生不可预见的变化。 比如原料批次变化导致增益漂移 50%，或者阀门磨损导致死区扩大。网络在训练分布内表现好，一旦对象变了，它的输出不会自动跟着变——它没有自适应机制。这种场合要么用第 67 章的模糊层（规则仍然成立），要么用真正的自适应控制（有在线辨识）。

第三条：安全等级要求高且无法验证。 如果失效的后果是人身伤害，而你又没有能力对网络做形式化验证，那就不能用。这不是技术保守，是责任问题。

第四条：维护团队读不懂也改不了。 这一条最容易被技术人员忽略，因为它不是技术指标。设备要运行二十年，中间会换几茬维护人员。交付一个只有原作者能维护的控制器，等于给未来埋了一个雷。

与其它学习类方法的分界

同属“学习”，还有三条路线需要区分，避免概念混淆。

强化学习直接优化闭环指标，不需要教师。它的优点是上限不受教师限制，缺点是样本效率极低（本章这种任务可能要几万次闭环试验）、训练过程本身可能不稳定、且需要大量探索。工业过程不允许“试错探索”，所以强化学习在连续过程控制里的落地远少于监督学习。

自适应控制（如 MRAC、STR）有严格的稳定性理论，能在线调整参数。它的前提是对象结构已知、只有参数未知；神经网络不要求这个前提，但也没有稳定性保证。两者是“有保证但适用范围窄”与“适用范围宽但没保证”的

交换。

系统辨识加模型控制先用数据拟一个对象模型（可以是网络），再用这个模型做内模控制或预测控制。它比本章的逆模型控制更系统，也更接近现代控制理论的主线；代价是需要处理模型误差的鲁棒性设计，工作量比“训一个网络直接当控制器”大。

本章选监督学习，是因为它最容易在有限篇幅内讲清“训练—验收—上线”的完整链路。这不代表它是最优路线，只代表它是最适合入门的路线。

一句话的适用边界

把这一节压缩成一句话：**当对象难以建模、数据可得、性能要求不是极致、维护团队有机器学习能力时，神经网络控制值得一试；四个条件缺一个，就该先考虑前三章的方法。**

这句话听起来保守，但它是本章所有实验数据支持的结论。四段轨迹上最好的改善是 11%，而引入的成本是一整套训练与验收流程。这个交换比在很多场合算不过来，承认这一点，比宣称“神经网络是未来”更有用。

68.12 Python 仿真代码

文件结构与运行方式

本章代码两个文件，都在 `book/code/` 下：

- `ch068_sim.py`：对象、数据生成、两个网络、三个控制器、六组实验；
- `ch068_figs.py`：十张图，从 `ch068_sim.py` 取数据。

运行顺序是先仿真后画图：

```
"D:\Anaconda3\envs\py311\python.exe" code\ch068_sim.py
"D:\Anaconda3\envs\py311\python.exe" code\ch068_figs.py
```

第一个脚本会依次打印样本量曲线、学习率对照、四控制器对照、外推、鲁棒、耗时六组结果，全部跑完约两分钟（主要花在样本量实验的 21 次训练上）。第二个脚本把结果画成 `figs/ch068/` 下的十张图，同时输出 SVG 与 PDF。

两个脚本都直接 `import ch067_sim as c67`，复用第 67 章的模糊教师与对象定义。这样做的好处是两章的基线完全一致——同样的对象、同样的抗饱和、同样的“微分作用于测量值”口径。如果各自实现一遍，指标就没法比了。

三个核心函数

`build_dataset` 负责造数据。它循环若干次：生成一条随机轨迹，用教师跑一遍闭环，把 $(e, -\dot{y}_f, \int e)$ 存成特征、把 $u/U_{\max}$ 存成标签；同时把 $(y, \dot{y}, \ddot{y})$ 与对齐后的 u 存成另一组，供逆模型网络使用。返回四个数组 `(X, Y, Z, W)`。

有一处必须注意：逆模型的标签要按延迟对齐。对象的纯延迟是 5 拍，第 k 拍的状态是由第 $k-5$ 拍的控制量造成的。如果直接把 u_k 与 $(y_k, \dot{y}_k, \ddot{y}_k)$ 配对，学到的是“造成当前状态的控制量”，而实际上是“将要造成 5 拍后状态的控制量”，两者差一个延迟。本章的做法是把 u 前移 5 拍再配对，这样网络的输入输出关系才与运行时一致。

`MLP.train` 是手写的反传加 Adam。选择手写而不是用框架，理由是本章的网络只有 61 个参数，用框架反而看不清梯度怎么流；手写版本每个梯度都能对上式 (68-6) 的四个分量，读者可以逐行核对。

`NNInverse` 是逆模型控制器。它内部先跑一个二阶参考滤波器（式 68-9），把阶跃变成光滑轨迹，再把 $(r_f, \dot{r}_f, \ddot{r}_f)$ 送进 RBF 得到前馈，最后加一个弱 PID 补残差。这个类的 `__call__` 每拍被调用一次，内部保存滤波器状态与积分状态。

关键代码：反传与 Adam 的实现

反传部分对应式 (68-6)，代码只有五行：

```
H, out = self._forward(X)          # 隐层激活与输出
d_out = (out - Y) / X.shape[0]     # 输出层梯度（MSE 的导数）
g_w2 = H.T @ d_out                 # 输出层权重梯度
g_b2 = d_out.sum(axis=0)
d_h = (d_out @ self.W2.T) * (1.0 - H ** 2)   # 穿过 tanh
g_w1 = X.T @ d_h
g_b1 = d_h.sum(axis=0)
```

第三行的 `1.0 - H**2` 是 $\tanh$ 的导数。这里有一个容易写错的地方：如果隐层用的是 sigmoid，导数应当是 $H(1-H)$；用 tanh 时写成 $H(1-H)$ 也能跑，但收敛会慢很多，而且不会报错。本章在注释里专门标了这一行。

Adam 部分对应式 (68-7)，注意偏差修正要用当前步数 t：

```
m = beta1 * m + (1 - beta1) * g
v = beta2 * v + (1 - beta2) * g ** 2
m_hat = m / (1 - beta1 ** t)
v_hat = v / (1 - beta2 ** t)
w -= lr * m_hat / (np.sqrt(v_hat) + eps)
```

`t` 从 1 开始，不能从 0 开始——从 0 开始会除以 $1-\beta_1^0 = 0$，第一步就出 NaN。这个错误在自实现 Adam 时非常常见。

实验函数与它们回答的问题

六个实验函数各自对应正文的一节：

函数	回答的问题	正文位置
`exp_sample_size`	多少数据够	68.8
`exp_learning_rate`	学习率与正则怎么选	68.8
`exp_compare`	四种控制器谁好	68.6
`exp_extrapolate`	出了训练范围会怎样	68.8
`exp_robust`	噪声与延迟下退化多少	68.8
`exp_timing`	单次推理多快	68.10

写这些函数时有两条约定，值得照搬到自己的项目里。

第一条：**基线一律调外部函数，不在实验里另写一遍。** `exp_compare` 里的固定 PID 与模糊控制器都是直接调 `c67.simulate`，而不是在本文件里重新实现。原因很实际：抗饱和的写法、微分的口径、限幅的位置，任何一处不同都会让指标差几个百分点，而读者看不出差异来自算法还是来自实现。

第二条：**每个随机实验都跑多个种子并报最差值。** 小样本下单个种子的闭环指标波动可以到 0.3，只报均值会掩盖掉“某些种子训崩了”这个事实。`exp_sample_size` 每档跑三个种子，同时记录 `iae` 与 `iae_worst`，图 68-4 里两条线一起画。

复现本章数字

按下面的顺序跑，可以得到正文里的全部数字：

1. `python code/ch068_sim.py`，记下打印的六组结果；
2. 对照表 68-3 核对四控制器指标，误差应在 0.01 以内（浮点与线程调度带来的抖动）；
3. `python code/ch068_figs.py`，检查 `figs/ch068/` 下十张图是否齐全。

如果数字对不上，先查三处：`ch067_sim.py` 是否被改过（教师变了，所有标签都变）、NumPy 版本是否一致（不同版本的 `default_rng` 序列相同，但矩阵乘法顺序可能影响浮点末位）、以及是否在 GPU 上跑（本章代码是纯 NumPy，不涉及）。

想加一个新实验时改哪里

假设要加一个“不同隐元数对闭环的影响”实验，需要动三个地方。

第一，在 `ch068_sim.py` 里加一个 `exp_width(X, Y)`，循环隐元数取 4、8、12、24、48，每档训三个种子，返回列表。注意 `MLP` 的隐元数当前是写死的，要先把它改成构造参数。

第二，在 `main()` 里调用这个函数并打印结果，格式与其它实验保持一致（每行一个字典的摘要）。

第三，在 `ch068_figs.py` 里加一个 `fig_11_width(rows)`，仿照 `fig_04_samples` 的写法。图号顺延为 68-11，记得在正文对应位置引用——否则 `check_chapter.py` 会报“图未被引用”。

这三步的共同点是：**实验、打印、画图三处必须同时改。** 只改一处会出现“数据有了但没画”或者“画了但正文没引用”，前者浪费，后者过不了检查。

建议的阅读顺序

`ch068_sim.py` 约 530 行，从头上读下去会先撞上一大段样本生成，容易失去耐心。建议按下面的顺序读，每一步都能跑出一个可核对的结果。

第一步读 `gain_at` 与 `simulate_ctrl`。前者只有一行（$4 + 6y$），后者是闭环主循环，看清“算特征 → 查网络 → 限幅 → 进对象 → 存轨迹”这个顺序，后面所有指标都从这里出。

第二步读 `build_dataset`。它把教师跑出来的轨迹整理成两组数据，一组给直接控制（$e, -\dot{y}_f, \int e$），一组给逆模型（$y, \dot{y}, \ddot{y}$）。注意逆模型那组的标签要前移 5 拍对齐，这是全文件最容易看漏的一行。

第三步读 `MLP` 的 `_forward` 与 `train`。前者三行矩阵乘法，后者是反传加 Adam，加起来不到 40 行。读完这两个函数，式 (68-2)、(68-6)、(68-7) 就都有对应的代码了。

第四步读 `RBF` 的 `fit`。它做两件事：k-means 找中心、岭回归解权重。第二件事只有一行 `np.linalg.solve`，对应式 (68-4)。

第五步读 `NNInverse`。它把式 (68-9) 的参考滤波、RBF 前馈、弱 PID 三部分串起来，是本章唯一一个“结构比参数重要”的类——即使 RBF 换成一个粗糙的查表，只要前馈量级对，闭环仍然稳。

最后再读六个 `exp_*` 函数。它们彼此独立，可以按兴趣挑着看；每个函数开头都有一段 docstring 说明“这个实验回答什么问题”，比读代码本身更快。

按这个顺序，读完全部代码大概需要半小时，跑完全部实验约两分钟。这两件事都做完，再回头看正文里的数字，就会发现它们都能在代码里找到出处——这是本书对“可复现”的最低承诺。

代码里三处刻意的“不优雅”

最后说明三处看起来可以优化、但故意保留原样的地方，免得读者以为是疏漏。

第一，`simulate_ctrl` 是纯 Python 逐步循环，没有向量化。向量化要求每次试验的所有时间步一起算，但控制是串行的——第 k 步的输出依赖第 $k-1$ 步的状态，没法并行。用 Numba 或 C 能加速十倍，但会让代码难以逐行对照正文，本章选择可读性。

第二，`MLP` 的每个方法都在内部重新 `np.asarray`，有冗余。这是为了让它能同时接受列表和数组，避免调用方踩坑；在 61 个参数的规模下，这点开销可以忽略。

第三，`RBF.fit` 里 k-means 只跑 25 轮，没有收敛判据。理由是中心的位置对最终精度影响很小（输出层用岭回归会补偿），跑 25 轮与跑 100 轮的验证损失差不到 5%。加收敛判据会让代码多十行，收益不成比例。

这三处都不是最佳实践，但它们各自有一个明确的理由。写代码时保留这种“知道自己在妥协什么”的状态，比盲目追求优雅更有用。

第69章 迭代学习控制（ILC）

第 68 章的神经网络把“经验”压缩进 61 个权重，一次训练、终身使用。它有一个隐含前提：任务是**不重复**的。四段设定值各不相同，网络只能在“见过的工况”附近给出合理动作，学不到“这一次比上一次好在哪”。可工业现场有大量任务恰恰是重复的：机械臂每天抓同一批工件、数控机床每 3 秒走一遍同一个轮廓、注塑机每个模次执行同一条合模曲线、贴片机每块板子走同一条贴装路径。

对这类任务，“重复”本身就是一条被浪费掉的信息。第 44 章的 PID 每一遍都从零开始，上一遍的误差在程序里没有任何痕迹；第 58 章的 MPC 会预测未来，但预测完就把这一遍的偏差丢了。如果能把“上一遍错在哪”记下来、下一遍提前补上，那么同一条轨迹跑上几十遍之后，误差可以比任何固定参数控制器都小——而且不需要知道对象模型。这就是迭代学习控制（iterative learning control，ILC）。

本章的结论先说清楚：ILC 能把本章算例的跟踪误差从 0.1161 压到 0.0224，降幅 80.8%，代价是 60 次迭代、每条轨迹 4 s，也就是 4 分钟的“学习期”；它压不下去的那部分误差有明确的频域来源，可以被算出来，也可以被 Q 滤波剪掉，但剪掉高频的同时也剪掉了收敛速度。这条“用带宽换速度”的交换，是全章的主线。

69.1 动机与直观理解

同一件事做第二遍

设想一个人第一次骑自行车走一条窄路，会左右晃；第二次走同一条路，晃得少一些；第十次，几乎不晃了。这个过程里有三件事同时在发生：他记住了“哪个位置容易偏”，他在那个位置提前做了修正，而且他没有改变自行车的结构。ILC 就是把这三件事写成算法：**记住上一条轨迹的误差、在同一个位置提前补偿、控制器结构不变**。

把这个直觉形式化，需要两个轴。一个是**时间轴** t，描述一次任务内部从 0 到 T 的过程；另一个是**迭代轴** k，描述第几次执行。第 44 章的 PID 只在时间轴上工作，每个采样时刻独立算一次；ILC 多了一个轴，它允许“第 $k+1$ 次在 t 时刻的动作”依赖“第 k 次在 t 时刻的误差”。这种跨迭代的信息传递在因果系统里是完全合法的，因为第 k 次已经跑完了，它的整条误差曲线都躺在内存里。

这个额外维度带来的能力很具体：PID 面对 5 拍纯延迟，只能靠降低增益换取稳定，因此必然留下稳态跟踪滞后；ILC 可以把“下一拍要发生什么”提前 5 拍写进前馈记忆，因为它在上一遍已经看过这 5 拍。换句话说，**ILC 用“重复”换来了本来需要模型预测才能得到的前瞻信息**。

两个轴上的分工

图 69-1 是本章的结构。主回路上仍然是第 67 章那套反馈 PID：参考 $r(t)$ 与测量值相减得到 $e_k(t)$，PID 算出反馈量，与前馈记忆 $u_k(t)$ 相加，过限幅、过纯延迟，作用到对象上。多出来的部分在上方：误差被引到学习律，学习律按 P 型、D 型或 PD 型算出修正量 Δu_k，累加到记忆里，得到下一遍的 $u_{k+1}(t)$。

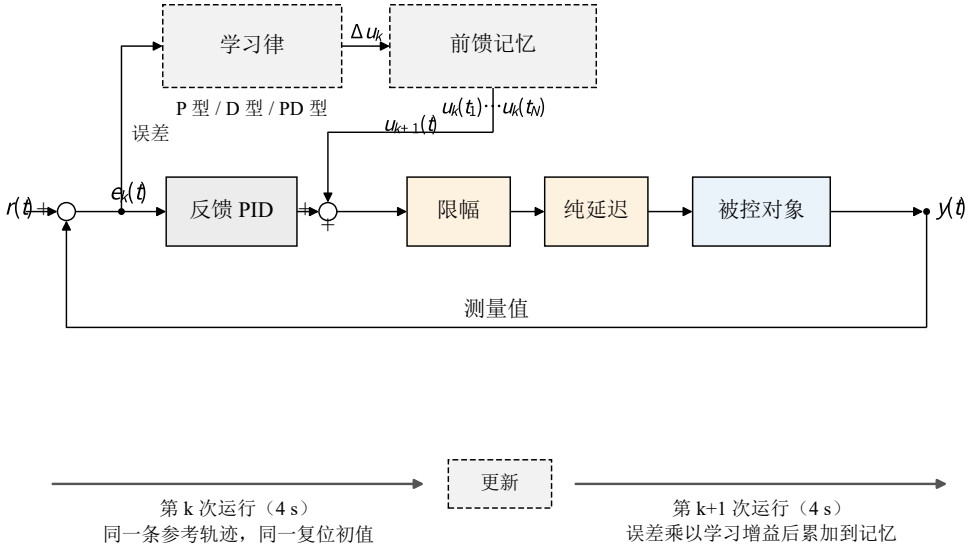


图 69-1 迭代学习控制的闭环结构

这张图里有两条通路值得分开看。**周期内的通路**（反馈）必须因果：第 t 拍的控制量只能用 t 拍及以前的测量。**跨周期的通路**（学习）不必因果：第 $k+1$ 次在 t 拍的控制量可以用第 k 次在 $t+\Delta$ 拍的误差，甚至用整条轨迹的误差。这条“非因果”的许可，是 ILC 全部能力的来源，也是它全部风险的来源——非因果的更新没有天然的稳定机制，必须靠增益与滤波器人为约束。

ILC 与重复控制的区别

初学者容易把 ILC 与“重复控制”（repetitive control）混为一谈，两者确实都利用周期性，但用法完全不同。重复控制把周期信号的内模 $1/(1-e^{-sT})$ 串进反馈回路，是一条**在时间轴上实时运行**的控制器，它对每个周期内的扰动都做抑制，本质是“频率固定的谐振控制器”。ILC 则**不在时间轴上增加任何环节**，它只改“下一遍用什么前馈”，本质是“批间优化”。

这个区别带来三个实际后果。第一，重复控制要求扰动或参考严格周期，且周期已知；ILC 只要求“每次执行的是同一条有限长轨迹”，不要求轨迹本身是某个周期信号的截断。第二，重复控制对非周期扰动不敏感（它只认那个频率），ILC 会把非周期扰动误当成“要学的形状”，因此对噪声、复位误差、随机外扰更敏感，这正是 69.9 节要处理的问题。第三，重复控制在第 k 次执行中就能起作用，ILC 必须等第 k 次跑完才能更新，因此 ILC 的收益永远滞后一遍。

反馈的天花板在哪里

读到这里常有人问：既然 ILC 只是“在反馈上叠一份记忆”，为什么不把这部分活交给更强的反馈？把 PID 调到最好不就行了？

先看数字。第 68 章的参数网格里，纯反馈最好的一组是 $(3.0,\ 0.20)$，RMS 能到 0.0797，比本章这组 $(2.0,\ 0.05,\ 0.20)$ 的 0.1161 好 31%。可它离 ILC 的 0.0224 还差 3.6 倍——这 3.6 倍不是“再调调”能拿到的。

原因在灵敏度函数上。反馈的跟踪误差由 $S=1/(1+CG_0)$ 在跟踪频点的模决定，环路增益大一个数量级，误差才小一个数量级。要把 0.0797 再降 3.6 倍，就得在 0.25 到 0.75 Hz 这一段的环路增益提高 3.6 倍，也就是

11 dB。可 50 ms 延迟在 0.75 Hz 就吃掉 13.5°、在 1.0 Hz 吃掉 18°，而对象本身（双积分器串一阶惯性）在 0.75 Hz 已滞后 157°。环路增益每加一倍，相位裕度就少一截，加到头就是振荡。

这条循环在频域里是闭合的：**想降误差就要提高穿越频率，提高穿越频率就放大延迟的相位代价，代价一大又必须降增益**。所以纯反馈的精度上限不是“调参水平”决定的，而是“延迟与对象相位”决定的。本章的 0.0797 已经贴着这条上限。

ILC 打破这个循环的办法不是“更大的增益”，而是“换一个轴”。它不在时间轴上提高环路增益，而是在迭代轴上累积：上一遍的整条误差在下一遍开始前已经完整拿到，这边界上不存在因果限制，也就不存在延迟。这就是为什么 400 个数的记忆能买到 3.6 倍于“最好反馈”的精度，代价只是 60 次重复。

一个必须先接受的事实

ILC 不改变反馈回路的稳定性。它把前馈记忆叠加到指令上，反馈支路照旧工作；如果第 k 次的闭环本身是稳的，那么第 $k+1$ 次在记忆正确的情况下也是稳的。ILC 带来的新风险只有一种：**记忆本身可能收敛到一条错误的曲线，或者根本不收敛**。前者表现为“学了一个歪的前馈，误差反而变大”，后者表现为“误差逐次增大直到贴住限幅”。

这两种失效都不会被“闭环稳定”这条判据发现，因为闭环在每个单独的时刻都是稳的。要看住它，必须盯三个量：整条轨迹的误差 RMS、控制量的饱和比例、以及误差谱的形状。本章 69.6 节会用一组实测数据说明，第三个量是最灵敏的——误差 RMS 还在下降的时候，某个高频分量可能已经在指数增长了。

本章算例与结果预告

算例沿用第 66 至 68 章的对象：$y'' = -2y' + K(y)u$，$K(y) = 4 + 6y$，工作区间内增益从 13 变到 20；采样 10 ms、纯延迟 5 拍、限幅 ± 1.2。任务是一条 4 s 的往复轨迹，每次迭代从同一位置复位、重跑同一条曲线。

这个对象有一个容易被忽略的性质：方程里**没有 y 项**，它是一个二阶无静差环节。常数输入下输出不会停在某个值，而是二次增长——实测把 $u \equiv 0.30$ 施加 4 s，输出从 1.5 涨到约 25。这个性质直接约束了参考轨迹的选法：轨迹在两端必须有零速度，否则第一拍就跟不上，任何前馈都救不回来。69.3 节会把这条约束讲透。

按 69.3 节的设计流程，最终得到的一组参数是：PD 型学习律 $\gamma_p = 0.10$、$\gamma_d = 0.005$，Q 滤波窗长 81 拍，反馈 PID 为 $(2.0,\ 0.05,\ 0.20)$。跑 60 次迭代后，误差 RMS 从 0.1161（纯反馈）降到 0.0224，最大偏差从 0.1974 降到 0.0495。作为参照，如果模型完全已知、直接用精确前馈 $u^{*} = (r'' + 2r')/K(r)$，误差 RMS 是 0.0593——**ILC 在完全不知道 $K(y)$ 的情况下，做到了比精确前馈还低**。这个反直觉的结果在 69.4 节解释：ILC 学的是闭环的逆，不是对象的逆。

本章的路线

69.2 节给出学习律、误差递推与频域判据的数学；69.3 节把设计拆成六步，重点是参考轨迹与反馈支路的先决条件；69.4、69.5 节分别在连续与离散两种视角下看不带滤波的 ILC，前者解释“学到的到底是什么”，后者解释“记忆在内存里长什么样”；69.6 节用频域判据解释 1.25 Hz 分量的增长，这是全章技术含量最高的一节；69.7、69.8 节讨论三个参数的意义与调参顺序；69.9 节处理非重复扰动；69.10 节讲工程落地的重复性识别、过程监控与中止；69.11 节给边界；69.12 节是代码。

69.2 数学原理

记号：两个下标

本章统一用 k 表示迭代序号，t 表示一次迭代内的时间，$t_j = jT_s$ 表示第 j 个采样时刻，$j = 0, 1, \dots, N-1$，$N = T/T_s = 400$。三个基本信号是参考 $r(t)$（与 k 无关，这是“重复”的含义）、输出 $y_k(t)$、误差 $e_k(t) = r(t) - y_k(t)$。前馈记忆记为 $u_k(t)$，它是一张长度为 N 的表，第 k 次执行时按拍查表。

需要强调的是 u_k 的下标含义：它标的是“第 k 次执行时用的前馈”，不是“第 k 次学出来的前馈”。按这个约定，第 k 次执行结束后算出的是 u_{k+1}。初值 $u_0 \equiv 0$，即第一次完全靠反馈——这一点很重要，它让 ILC 的第一次执行等价于纯反馈，给了一个天然的对照基线。

学习律的一般形式

最一般的形式是“上一次的记忆 + 误差的某种变换”：

$$u_{k+1}(t) = Q\Big[u_k(t) + L\,e_k(t)\Big] \tag{69-1}$$

L 是学习算子，Q 是滤波算子。工程上 L 取三种最简单的形式，对应三种“学习律”：

$$\text{P 型:}\quad u_{k+1}(t) = Q\big[u_k(t) + \gamma_p e_k(t)\big] \tag{69-2}$$

$$\text{D 型:}\quad u_{k+1}(t) = Q\big[u_k(t) + \gamma_d \dot{e}_k(t)\big] \tag{69-3}$$

$$\text{PD 型:}\quad u_{k+1}(t) = Q\big[u_k(t) + \gamma_p e_k(t) + \gamma_d \dot{e}_k(t)\big] \tag{69-4}$$

三者的直观区别可以用“走山路”打比方。P 型只看“现在离路边多远”，走一步、量一次、再走一步；D 型只看“正在往哪边偏”，偏差为零但正在快速偏离时它也会修；PD 型两者都看。P 型的缺点是收敛慢——偏差小的时候修正量也小，越接近目标越磨蹭；D 型的缺点是对噪声敏感——差分把高频噪声放大，而记忆里一旦写进高频振荡，下一遍就会放大它。

这也解释了为什么第 44 章的 PID 里 D 作用要加低通：此处完全同理。本章的 D 型实现是“中心差分 + 一阶低通”，时间常数 30 ms，与第 67 章测量值微分的处理一致。

所谓“P 型”，是相对时间轴说的

有一个地方容易困惑：P 型学习律只用了误差的比例，没有积分，凭什么能把误差压到零？在时间轴上，纯比例控制必有静差，这是第 44 章反复强调的结论。

原因在于“积分”发生在另一条轴上。固定某一拍 j，看这一拍的记忆随迭代序号怎么变：$u_{k+1}[j] = u_k[j] + \gamma_p e_k[j]$。这就是一个离散积分器，输入是 $e_k[j]$、输出是 $u_k[j]$、步长是 1、增益是 γ_p。所以 P 型 ILC 在时间轴上没有积分，在迭代轴上却是纯积分。零静差由此而来：只要迭代收敛，$u_k[j]$ 必须停在某个常值上，而这个常值成立的必要条件是增量趋于零，即 $e_\infty[j] \to 0$。**“误差为零”不是靠某个环节“记住”了偏差，而是靠积分器在稳态时必须“输入为零”这条运动学约束。**

按同样的看法，D 型与 PD 型在迭代轴上同样是积分器，时间轴上的微分只改变了积分器的输入形状，没有改变“积分”这件事。所以三种学习律都有零静差，区别只在收敛速度与相位——零静差是迭代轴上积分的必然结果，与时间轴上用什么律无关。

这条视角还顺带解释了两件事。第一，学习增益的符号不能错：积分器接上正反馈就会指数增长，这正是 69.8 节“误差完全不降”那一行症状的物理原因。第二，在时间轴上给学习律再加一个积分是错误方向：一来形状信息被

抹平，ILC 最有价值的逐点修正能力消失；二来时间轴积分再添 90° 相位滞后，式 (69-8) 立刻更难过。**ILC 的能力全部建立在“逐点”上，任何沿时间的平滑都必须放进 Q，而且必须是零相位的**——这就是本节后面 Q 一定要用对称窗的原因。

误差递推：把学习写成映射

要分析收敛，必须把“误差怎么随 k 变”写成显式关系。设对象与反馈构成的闭环算子是 T，即前馈 u_k 产生输出 Tu_k（这里 T 已经包含了反馈支路的作用，不是单纯的对象）。若 $u_0 = 0$ 时输出为 $y_0 = T_0 r$，则

$$e_k = r - T u_k \tag{69-5}$$

代入 P 型学习律，并暂时取 $Q = I$：

$$e_{k+1} = e_k - \gamma_p T e_k = (I - \gamma_p T) e_k \tag{69-6}$$

这一步是整章的核心。它说明：**误差的演化与参考无关，只与算子 $(I - \gamma_p T)$ 有关**。参考轨迹换成另一条，只要 T 不变，收敛性质完全一样；反过来，T 的性质决定了 ILC 能不能用、能用多快。式 (69-6) 还把“学习”翻译成了一个线性迭代：每学一次，误差被乘上一个算子。

收敛的充分条件

对式 (69-6) 取范数，得到最常用的充分条件：

$$\| I - \gamma_p T \| < 1 \tag{69-7}$$

只要这个算子范数小于 1，误差就按几何级数收敛到零。工程上更常用的是它的频域版本，因为 T 通常是线性时不变或可线性化的。对每个频率 ω，把算子换成传递函数：

$$\left| 1 - \gamma_p T(j\omega) \right| < 1, \qquad \forall \omega \tag{69-8}$$

这个条件有一个非常有用的几何解释。$1 - \gamma_p T(j\omega)$ 是复平面上“点 1”到“点 $\gamma_p T(j\omega)$”的距离。距离小于 1，意味着 $\gamma_p T(j\omega)$ 落在以 1 为圆心、半径为 1 的圆内——也就是它的实部为正，且模不太大。

由此可以立刻读出两条工程规则。第一，**$T(j\omega)$ 的相位一旦超过 $\pm 90^\circ$，实部变负，γ_p 越大越危险**；相位到 180° 时，任何正的 γ_p 都会让 $|1 - \gamma_p T| > 1$。第二，**$|T(j\omega)|$ 越大的频段越危险**，因为同样的 γ_p 会把工作点推得更远。这两条在 69.6 节会被本章的数据逐条验证：临界频点 1.25 Hz 处，T 的相位是 -145.9°、模是 0.6263，$\gamma_p = 0.30$ 时 $|1 - \gamma_p T| = 1.160$，已经越界。

为什么不能拿对象的开环频响做判据

初次接触 ILC 的人常有一个疑问：对象传函 $G_0(s)$ 已经知道，为什么不直接套式 (69-8)，非要再算一个闭环 T？答案是：**判据里的算子必须是“记忆到输出”的整条通路，不是“控制量到输出”的那一段**。用错了会给出方向完全相反的结论。

用本章的线性化模型算一遍就清楚。G_0 在 0.25 Hz 处模 4.131、相位 -132.65°，在 1.25 Hz 处模 0.259、相位 -188.21°。把它们代进式 (69-8)，取 $\gamma_p = 0.10$：0.25 Hz 处 $|1 - 0.10 G_0| = 1.315$，1.25 Hz 处是 1.026。也就是说，开环判据会告诉你“基波处最危险、超标 31.5%”；更糟的是 0.25 Hz 的相位已经越过 -90°，该点 $1 - \gamma G_0$ 的实部恒大于 1，**任何正的增益都不收敛**。顺着这个结论走，ILC 在本章的对象上根本不能用。

实际情况恰好相反。同一频点上真正的闭环算子是 $T = 0.5256 \angle -13.77^\circ$，$|1 - 0.10T| = 0.949$，是全章收敛最快的地方；真正的危险频点在 1.25 Hz，$|1 - 0.10T| = 1.0525$。两套判据给出的“最危险频率”差了五

倍，“能不能用”的结论完全颠倒。

差别来自反馈。$T = G_0/(1+CG_0)$ 在低频把模从 4.131 压到 0.5256（7.9 倍），把相位从 -132.65° 拉到 -13.77°；在高频却几乎不动，1.25 Hz 处 $|T| = 0.6263$，反而比 $|G_0|$ 的 0.259 大 2.4 倍。低频被反馈“扶正”、高频被反馈“顶高”，一升一降之后，最危险的频点自然从基波挪到了 1.25 Hz。

还有一条更根本的理由：G_0 是双积分器串一阶惯性，$\omega \to 0$ 时 $|G_0| \to \infty$，开环判据在直流处必然发散。可直流正是反馈跟踪最好的地方——有积分作用，稳态误差为零。一条在直流处就宣布“不可能”的判据，显然不能用来设计 ILC。这条反例可以推广成一句经验：**判据要跟“误差实际流过的回路”对齐，而不是跟“最熟悉的那段传函”对齐。**

算子 T 里到底有什么

式 (69-6) 里的 T 不是对象，也不是“对象乘 PID”，而是从记忆入口到测量输出的整条通路。按本章的实现，它依次包含：记忆与反馈输出相加的加法点、PID 支路（比例 2.0、积分 0.05、微分 0.20，微分作用在测量值上并串 30 ms 一阶低通）、限幅（线性化时视为未饱和）、5 拍纯延迟、在工作点 $K = 16.5$ 处线性化的对象，以及采样与零阶保持。

四处细节最容易漏。**一是微分的低通**：30 ms 在 1.25 Hz 处贡献 13.3° 相位，在 5 Hz 处到 43°，它决定了高频段的相位走向，不能按“30 ms 很短”忽略掉。**二是延迟与采样必须一起算**：5 拍延迟是 50 ms，零阶保持再贡献半拍 5 ms，两者相加才是 T 的真实滞后；只用 $e^{-0.05s}$ 会低估 2.25°（1.25 Hz 处）。**三是限幅**：饱和时 T 不再是线性算子，判据失效，这也是 69.10 节把“饱和比例”列为监控量的原因。**四是积分器**：$k_i = 0.05$ 让 C 在低频趋于无穷，这才使 T 的低频模不等于 G_0 的低频模。

工程上不必手推这个乘积。本章的做法是实测：把一段正弦注入记忆通道，等两个周期进入稳态，再对稳态段做 DFT，取“输出复数比输入复数”作为 $T(\mathrm{j}\omega)$。这样得到的 T 自动包含上面四处细节，也自动包含实现里任何没想到的东西，比如查表时用的线性插值。代码见 69.12 节的 `loop_freqresp`，它用同一条闭环仿真跑到稳态，一次算出全部频点。

最后一个必须承认的近似：T 依赖工作点。本章的 $K(y)$ 沿轨迹从 13 变到 20，所以严格说 T 在同一条轨迹内就是变的，判据只在某个“等效工作点”上成立，取 $K = 16.5$（轨迹中段）是一种折中。69.9 节的增益失配实验给了这个近似的余量：K 偏大 20% 时末次误差 0.0161（反而更小），偏小 20% 时 0.0351（变大但仍收敛）。这说明判据虽是局部的，作为设计工具已经够用——它定的是“量级与位置”，不是“精确的临界值”。

Q 滤波：把不可学区剪掉

既然高频段注定越界，与其把 γ_p 压到全局都很小（低频也跟着变慢），不如“只在能学的频段学”。这就是 Q 的作用：

$$u_{k+1} = u_k + Q\,\gamma_p e_k, \qquad |Q(j\omega)| \le 1 \tag{69-9}$$

注意 Q 作用在**学习增量**上，不是作用在整条记忆上。这两种写法有实质区别：若写成 $u_{k+1} = Q[u_k + \gamma_p e_k]$，高频段的记忆会每遍被清零，等价于“每遍退回纯反馈”；而且误差递推里会多出一个 $(1-Q)e_{fb}$ 的强迫项（e_{fb} 是纯反馈误差），分析起来更绕。式 (69-9) 的写法让高频记忆“保持上一遍的值”，误差既不发散也不被清零，正好是工程上想要的“冻结”。

加了 Q 之后，误差递推变成

$$e_{k+1} = \big(I - \gamma_p T Q\big) e_k \tag{69-10}$$

收敛条件相应变成

$$\left|1-\gamma_p Q(j\omega)T(j\omega)\right|<1 \tag{69-11}$$

注意这里不是简单的“$|Q|$ 乘上去”，而是 Q 与 T 相乘后再减。当 Q 是零相位滤波器时，$Q(j\omega)$ 是实数，式 (69-11) 化简为

$$\left|1-\gamma_p Q(\omega)T(j\omega)\right|<1 \tag{69-12}$$

它在低频（$Q\approx 1$）退化成原来的判据，在高频（$Q\approx 0$）退化成 $|1-0|=1$——既不收敛也不发散，误差停在那里不动。这个“退化成 1”的性质很关键：**Q 滤波不是把高频误差压小，而是让高频误差不再变化**。所以 Q 窗长选得太长，会看到“前 20 次降得很快，然后停在一个非零的地板上”，而不是“一路降到零”。

本章用的是零相位滑动平均，长度 m 拍、对称窗、两端按边缘值延拓：

$$Q(z)=\frac{1}{m}\sum_{i=0}^{m-1} z^{-\left(i-\frac{m-1}{2}\right)} \tag{69-13}$$

因为整条轨迹已经在迭代域里存好，这个窗可以“居中”处理，不引入相位滞后——这是 ILC 相对实时滤波的一个实实在在的好处。它的幅度响应是 Dirichlet 核，第一个零点在 $f=1/(mT_s)$，等效截止频率约 $0.443/(mT_s)$。窗长 81 拍对应 0.55 Hz，窗长 21 拍对应 2.11 Hz。

收敛速度与迭代次数

式 (69-6) 是线性迭代，若把算子换成标量近似，误差按 $e_k\approx \rho^k e_0$ 衰减，ρ 叫收敛因子：

$$\rho=\max_{\omega}\left|1-Q(\omega)\big(1-\gamma_p T(j\omega)\big)\right| \tag{69-14}$$

要把误差压到初值的 ε 倍，需要

$$k\geq\frac{\ln\varepsilon}{\ln\rho} \tag{69-15}$$

次迭代。这条式子解释了本章所有“慢”的现象。$\rho=0.99$ 时，降到 1% 要 458 次；$\rho=0.95$ 时要 90 次；$\rho=0.90$ 时只要 44 次。实测的几何比与这个估计吻合：$\gamma_p=0.02$ 时实测 0.991，$\gamma_p=0.10$ 时 0.957，$\gamma_p=0.15$ 时 0.937——增益每翻一倍，ρ 大致按 $1-\gamma_p\bar{T}$ 线性下降，其中 $\bar{T}$ 是低频段 T 的加权平均模。

这里有一个容易犯的错误：把 ρ 当成“整条轨迹的收敛速度”。实际上 ρ 是频域上的最大值，它由**最难学的那个频点**决定，通常不是基波。本章算例里，无滤波时 0.25 Hz 的收敛因子是 0.949、1.25 Hz 是 1.053；加上 81 拍 Q 滤波后，前者变成 0.952（略慢一点），后者降到 1.001（几乎冻结）。同一个学习律在两个频点上做的是相反的事：一个在往里收，一个被按住了。这就是为什么只看 RMS 的速度会误判整条轨迹的命运——无滤波时 1.053 那一项会让误差在 30 次后重新涨起来，而 0.949 那一项一直在降，两者叠加的结果是一条先降后升的曲线。

69.3 设计步骤

第一步：确认任务真的重复

ILC 的第一道门槛不是算法，是任务。判断“重复”要看三件事，缺一不可。**参考轨迹相同**：每次执行的目标曲线必须是同一条，允许起点、终点不同，但不允许“这次走矩形、下次走圆”。**初始条件相同**：每次开始前必须复位到同

一状态，位置、速度都要，只复位位置而速度带着上一遍的残余，等于每次都在不同的初始条件下学习。**扰动近似相同**：负载、摩擦、温度在一个批次内变化不大；如果每次的外扰都是独立的随机量，ILC 会把噪声学进记忆。

三条里最容易忽略的是第二条。工程上“复位”往往只做位置回零，速度靠自然停稳，于是每次的初速度有微小差别。本章实测，把复位误差设为标准差 0.02 的随机量（相对轨迹幅值 1.22 约 1.6%），末次误差是 0.0218，与理想重复的 0.0224 基本持平——这个结果只在加了 Q 滤波之后成立。复位误差是“每遍都不重复”的低频扰动，而低频恰好是 ILC 学得最快的频段；若不加 Q 滤波，它会被学进记忆，表现为记忆里出现一条与轨迹无关的慢漂移，下一遍不仅没补上，反而把这一遍的偏差固化了。

第二步：定迭代长度与复位方式

迭代长度 T 要覆盖完整任务，且必须是采样周期的整数倍。本章取 $T = 4$ s、$T_s = 10$ ms，于是 $N = 400$。N 决定了记忆的大小（400 个双精度数，3.1 KB），也决定了频域分辨率 $\Delta f = 1/T = 0.25$ Hz——这个数字后面会反复出现，因为本章所有的谱分析都只能分辨 0.25 Hz 的整数倍。

复位方式有两种。**硬复位**：每次结束把执行机构拉回起点，等待稳定，再开始下一次。**软复位**：不复位，让轨迹首尾相接形成循环。硬复位适合点位任务，软复位适合连续旋转的场合（如主轴、输送带）。本章用硬复位，因为它让“初始条件相同”这条最容易满足，也让误差的定义没有歧义。软复位的麻烦在于：首尾必须满足周期性边界条件，否则接缝处的跳变会被当成误差学进去。

第三步：设计参考轨迹

这一步是本章踩坑最多的地方，值得单独讲。参考轨迹

$$r(t) = R_0 + A_1\big(1 - \cos \omega t\big) + A_3\big(1 - \cos 3\omega t\big), \qquad \omega = \frac{2\pi}{T} \tag{69-16}$$

取 $R_0 = 1.50$、$A_1 = 0.55$、$A_3 = 0.06$，轨迹范围 $[1.500,\ 2.720]$，基波 0.25 Hz、三次谐波 0.75 Hz。三个参数各有约束。

为什么是余弦型而不是正弦型。 最初的设计是 $r = 1.0\sin(\pi t/2) + 0.2\sin(3\pi t/2)$，起点 $r(0) = 0$ 但 $r'(0) = 1.57$。这个轨迹让 ILC 全线发散：精确前馈的 RMS 高达 0.504，P 型学习的误差从 0.60 涨到 0.98。原因不是算法错，是轨迹与初始条件不自洽——每次从静止复位，第一拍的速度是 0，而轨迹要求 1.57，这个矛盾无法用任何前馈弥补。改成余弦型后 $r'(0) = r'(T) = 0$，精确前馈的 RMS 立刻降到 0.0593。

为什么起点要抬到 1.5。 对象的增益 $K(y) = 4 + 6y$ 在 $y = -2/3$ 处过零。轨迹若跨过这一点，逆 $u = (r'' + 2r')/K(r)$ 在该点附近会炸掉——第一次试算时轨迹跨零，u^* 的最大值到了 5×10^4，RMS 3131。这不是数值问题，是物理问题：K 变号意味着“正输入产生负加速度”，逆不唯一。把轨迹整体抬到 $K \in [13, 20]$ 的正增益区间，逆就唯一且平缓。

为什么要有三次谐波。 纯基波的轨迹太“软”，任何控制器都能跟得差不多，看不出 ILC 的价值。加 6% 的三次谐波，让轨迹有加速度突变，逼出反馈的滞后。本章所有误差的主要成分就落在这个 0.75 Hz 分量上。

第四步：反馈支路先单独调好

ILC 是“叠加在反馈之上”的，它的收敛性质由闭环算子 T 决定，而 T 由反馈参数决定。所以顺序必须是：先把反馈调到最好，再开学习。反过来做——反馈很弱、靠 ILC 硬补——会得到一个“低频靠记忆、高频靠噪声”的系统，一旦记忆失效就没有兜底。

本章的反馈沿用第 67 章的结构：$u_{fb} = k_p e + k_i \int e - k_d \dot{y}_f$，微分作用在测量值上。参数取 $(2.0,\ 0.05,\ 0.20)$。这组参数不是最优的——第 68 章的网格里，$(3.0,\ 0.20)$ 的纯反馈 RMS 能到 0.0797，比本章的 0.1161 更低。为什么不用更激进的那组？因为它把闭环带宽推得更高，T 在 1.25 Hz 附近的相位更靠后，ILC 的可学频段反而变窄。这是反馈与学习之间的一处真实权衡：**反馈调得越猛，ILC 能加的益越少**。

第五步：选学习律与初始增益

初始增益按“半步法则”起步：先取一个明显偏小的值，看误差是否单调下降，再逐步加大直到出现“先降后升”，然后退回上一档的 60%。本章的实测边界很清晰：P 型在 $\gamma_p = 0.10$ 时 30 次迭代到 0.0693，$\gamma_p = 0.15$ 时 30 次后反超初值（0.1196），因此 γ_p 的可用上限在 0.10 附近。PD 型在同样的 γ_p 下加 $\gamma_d = 0.005$，末端降到 0.0506。

学习律的选择看误差的成分。若误差主要是“慢漂移”（低频），P 型足够；若误差主要是“相位滞后”（误差与速度同相），D 型更有效；本章的误差两者都有，PD 最好。这个判断不需要试三种，只要看第一次迭代的误差与参考速度的互相关：同相则 P，正交则 D。

第六步：加 Q 滤波与验收

最后一步加 Q 滤波，窗长从一个“刚好能通过轨迹最高谐波”的值起步。轨迹最高是 0.75 Hz，窗长 41 拍的等效带宽 1.08 Hz，刚好够；先取 41，再往长调到“末端误差不再下降”为止。本章最终取 81 拍（0.55 Hz），因为它把 1.25 Hz 压到了 0.012，而 0.75 Hz 还剩 0.494——牺牲一部分三次谐波的可学性，换来整条轨迹的单调收敛。

验收要看四件事：误差 RMS 是否单调下降、最大偏差是否同向下降、控制量饱和比例是否为零、误差谱是否所有分量都在降。表 69-1 是完整的参数清单。

项目	取值	说明
采样周期 T_s	10 ms	与第 66 至 68 章一致
迭代长度 T	4 s	$N = 400$ 拍，频率分辨率 0.25 Hz
纯延迟	5 拍	50 ms，落在学习环内
限幅	± 1.2	与前三章一致
参考轨迹	式 (69-16)	范围 $[1.500,\ 2.720]$
反馈 PID	$(2.0,\ 0.05,\ 0.20)$	先单独调好
学习律	PD，$\gamma_p = 0.10$、$\gamma_d = 0.005$	差分后接 30 ms 低通
Q 滤波	对称滑动平均，81 拍	等效带宽 0.55 Hz
迭代次数	60	4 分钟学习期
初值	$u_0 \equiv 0$	第一次等价于纯反馈

设计完成后先跑一次验收：图 69-2 是第 1 次与第 60 次的跟踪过程与误差。上图里第 1 次的输出（红虚线）在上升段明显落后于参考，峰值处差约 0.2；第 60 次（黑实线）几乎贴在参考上，只在 t = 1.3 s 附近还有一点过冲。下图的误差更能说明问题：第 1 次的误差是一条“先负后正再负”的波浪，幅值 -0.19 到 $+0.12$；第 60 次压到 ± 0.05 以内，且形状变成了“高频小幅抖动”——这正是 Q 滤波留下的、没能学掉的频段。

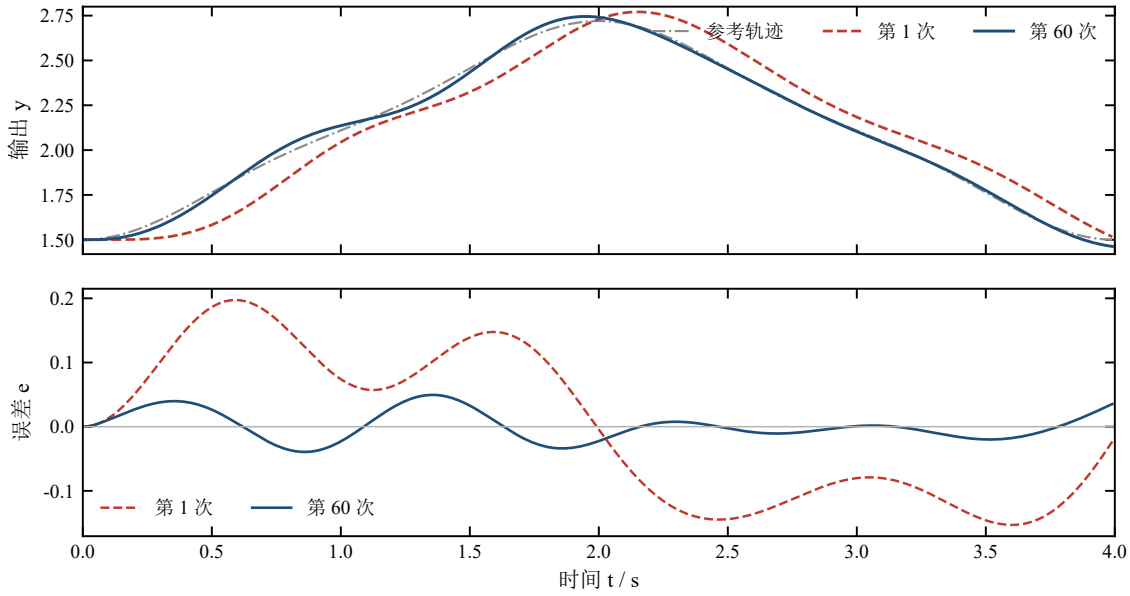


图 69-2 第 1 次与第 60 次的跟踪过程与误差

这张图还暴露了一个容易被忽略的细节：误差最大的位置不在轨迹的峰值（$t = 2$ s），而在 $t = 1.3$ s。原因是峰值处速度为零、加速度最大，而 $t = 1.3$ s 处速度与加速度都不小，闭环的动态滞后在这里累积得最多。69.6 节会说明，这个位置也正是三次谐波（0.75 Hz）能量最集中的地方。

69.4 连续情形

连续时间的学习律是一个积分方程

把式 (69-2) 的下标换成连续时间，就得到连续形式的 P 型学习律：

$$u_{k+1}(t) = u_k(t) + \gamma_p\, e_k(t), \qquad t \in [0,\, T] \tag{69-17}$$

注意它**不是微分方程**。式子里没有 $\mathrm{d}u/\mathrm{d}t$，也没有 $\mathrm{d}u/\mathrm{d}k$；它定义的是两条曲线之间的逐点加法。这一点常被误解为“ILC 是某种自适应律”，其实两者在结构上完全不同：自适应律用 $\dot{\theta} = \Gamma e \phi$ 让参数沿时间连续演化，参数在任务执行过程中就在变；ILC 的更新发生在任务与任务之间，一次执行之内 $u_k(t)$ 是一条固定的曲线，不随 t 变化。

这个区别决定了实现方式。自适应律必须在控制器里跑一个积分器，每个采样周期都要算；ILC 只需要两张表——上一遍的表和这一遍的表——执行时查表，执行完更新。也正因如此，ILC 对实时算力的要求几乎为零：本章的实测是单次迭代 2.82 ms（400 拍、每拍 5 次子步的完整非线性仿真），真正在控制器上运行的只有查表与一次加法。

把式 (69-17) 代入误差定义，连续域得到与式 (69-6) 同构的递推：

$$e_{k+1}(t) = \big[I - \gamma_p G\big] e_k(t) \tag{69-18}$$

其中 G 是从前馈到输出的闭环算子。连续域的收敛条件同样是一个范数条件 $\|I - \gamma_p G\| < 1$，频域上写作

$$\big|1 - \gamma_p G(j\omega)\big| < 1, \qquad \forall \omega \tag{69-19}$$

式 (69-19) 与式 (69-8) 形式一样，但 $G(j\omega)$ 与 $T(j\omega)$ 的内容不同：T 是离散闭环、含采样与零阶保持，G 是连续闭环。两者的形状相近，差别在 1 Hz 以上——采样与保持带来的半拍相位在连续域是看不见的，这也是为什么连续域的分析只能给方向，量化必须用离散模型。

函数空间里的加法，落到 400 个点上

式 (69-17) 是两条曲线之间的逐点加法，属于函数空间里的运算。落到离散实现上只做两件事：把 $[0, T]$ 切成 400 段，段内用常值代替（等价于零阶保持）；把函数换成 400 维向量。这两件事会引入两类误差，都值得算清楚。

第一类是分段常数带来的等效延迟。 一段常值要维持一个采样周期，平均滞后半拍，即 5 ms；在 1.25 Hz 处折合 2.25°，在 0.25 Hz 处折合 0.45°。与 5 拍纯延迟的 22.5° 相比小一个数量级，不影响任何结论。但若为了省内存把采样周期放宽到 20 ms，这半拍就变成 4.5°，与延迟本身的贡献同量级，判据必须重算。

第二类是混叠。 100 Hz 采样的折叠频率是 50 Hz，本章关心的 0.75 Hz 与 1.25 Hz 都远低于它，不存在混叠。真正要防的是现场噪声：50 Hz 工频在 100 Hz 采样下正好落在折叠边界，会被折到直流，表现为“记忆随运行时间缓慢漂移”——一个在误差谱上看不见、在记忆曲线上却很明显的东西。对策是模拟端先做 50 Hz 陷波，或提高采样率让 50 Hz 远离折叠点。

还有一个常被忽略的点是频率分辨率。400 点、4 s 的记录，DFT 只能给出 0.25 Hz 整数倍上的谱线。本章报的 0.25、0.50、0.75、1.00、1.25 Hz 全是这个格子上的点，这不是巧合，而是“只能测到这些”。若想区分 1.20 Hz 与 1.25 Hz 的行为差异，必须把迭代周期拉长到 20 s，而不是把采样率提高十倍——提高采样率只加了点数，没有加时长，分辨率仍是 $1/T$。

这三条合起来说明：离散化不是“精度差一点”的问题，而是**改变了可测与可学的对象**。实现时先确认采样周期与迭代长度，再谈增益与滤波；顺序反了，后面的参数全部白调。

相位预算：连续域里就能算出来

本章对象的线性化传函是

$$G_0(s) = \frac{K}{s^2 + 2s}, \qquad K \approx 16.5 \tag{69-20}$$

它是一个双积分器串一个一阶惯性，低频增益无穷大（这是“无静差”的来源），相位从 -90° 一路走向 -180°。加上 5 拍（0.05 s）纯延迟，相位再减 ωT_d。表 69-2 就是这笔相位账。

频率 / Hz	G_0 相位 / °	延迟相位 / °	合计 / °	$\lvert G_0 \rvert$
0.25	-128.15	-4.50	-132.65	4.131
0.50	-147.52	-9.00	-156.52	1.410
0.75	-157.00	-13.50	-170.50	0.684
1.00	-162.34	-18.00	-180.34	0.398
1.25	-165.71	-22.50	-188.21	0.259
1.50	-168.02	-27.00	-195.02	0.182

这张表一眼就能看出问题所在：**1.0 Hz 附近相位已经越过 -180°**，此后任何正的学习增益都会让 $\lvert 1 - \gamma G \rvert > 1$。延迟在其中的贡献随频率线性增长，1.25 Hz 时已经吃掉 22.5°，占总相位超出的三分之一。这也解释了一个常见

的误解：以为“延迟不大，只占 50 ms，可以忽略”。在时间域 50 ms 确实很短，但在相位域，它对 1 Hz 以上的分量是决定性的。

从这张表还能读出“为什么低频能学”。0.25 Hz 处相位只有 -132.65°，离 -180° 还有 47° 余量，$|1-\gamma G|$ 由 $1-\gamma|G|\cos(132.65^\circ)$ 主导，只要 $\gamma|G| < 2\cos(47^\circ) \approx 1.36$ 就收敛。本章 0.25 Hz 处 $|G_0| = 4.13$，于是 $\gamma < 0.33$ 都安全——实测的上限 0.15 比这个保守，因为实际闭环还叠加了反馈支路与采样的相位，安全余量必须打对折。

精确前馈：模型已知时的答案

如果模型完全已知，跟踪问题的经典解法是“逆模型前馈”。由 $y'' = -2y' + K(y)u$ 反解：

$$u^*(t) = \frac{r''(t) + 2r'(t)}{K\big(r(t)\big)} \tag{69-21}$$

把式 (69-16) 的轨迹代进去，得到 u^* 的范围 $[-0.154,\ 0.236]$，RMS 0.1166。这个前馈直接作用在对象上、不加反馈时，误差 RMS 是 0.0625；考虑 5 拍延迟并把 u^* 相应后移 5 拍，RMS 是 0.0593。这就是“知道模型”能给的全部收益，它已经把轨迹的加速度与速度都补偿掉了，剩下的是延迟与采样造成的高频残差。

于是本章出现了一个值得停下来想一想的数字：**ILC 末次的误差 0.0224，比精确前馈的 0.0593 低了 62%**。ILC 完全不知道 $K(y)$，它只是把上一遍的误差加到记忆里，怎么会比“精确”前馈还准？

ILC 学的是闭环的逆，不是对象的逆

答案在式 (69-5) 里：$e_k = r - Tu_k$，T 是闭环算子，不是对象 G_0。所以 ILC 收敛到的记忆满足 $Tu_\infty = r$，即 $u_\infty = T^{-1}r$，是闭环的逆。而式 (69-21) 给的 u^* 是 $G_0^{-1}r$，是对象的逆。两者差了一个反馈闭环：$T = G_0/(1+CG_0)$，于是

$$u_\infty = T^{-1}r = \frac{1+CG_0}{G_0}r = u^* + Cr \tag{69-22}$$

也就是说，记忆里最终存的是“对象逆 + 反馈支路在跟踪这条轨迹时输出的那一份”。反馈支路有相位滞后，它在轨迹的上升段输出偏小、下降段输出偏大；记忆把这一份补了进去，于是“记忆 + 反馈”的合计正好等于对象逆。图 69-10 是这条结论的实测证据。

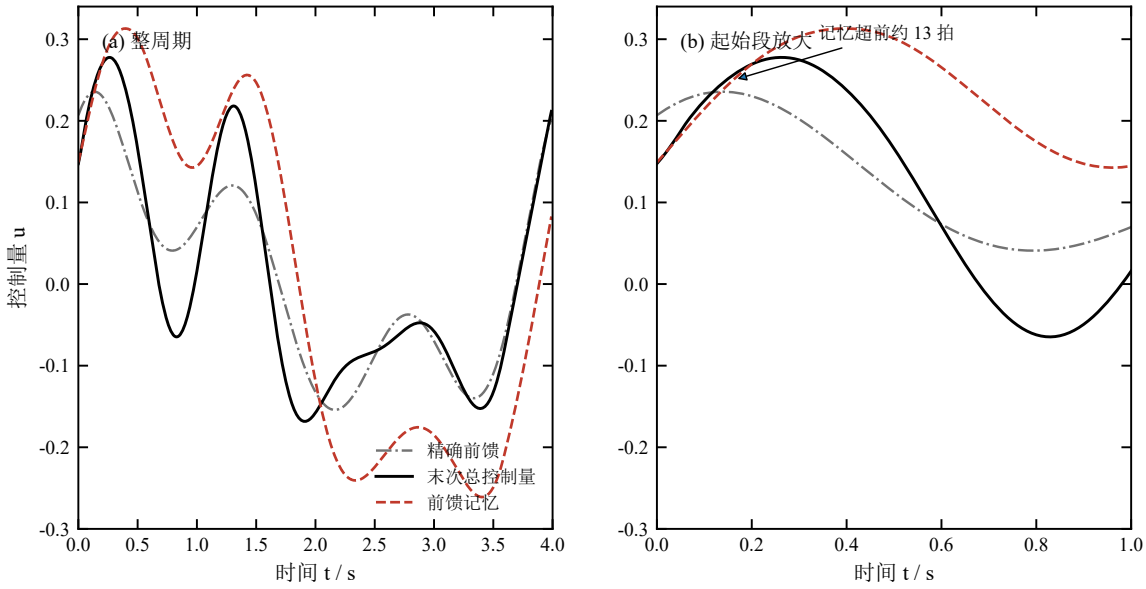


图 69-10 前馈记忆、末次总控制量与精确前馈的对比

图里三条曲线：灰点划线是精确前馈 u^{*}，黑实线是末次的总控制量（记忆加反馈），红虚线是记忆本身。在全周期上，黑线与灰线几乎重合；在起始段放大图里，红线明显走在灰线前面。定量地看，0.25 Hz 基波上三者是：u^{*} 幅值 0.1352、相位 -50.87°；总控制量 0.1358、-49.40°；记忆 0.2710、-76.08°。总控制量与精确前馈的幅值差 0.4%、相位差 1.5°，而记忆的幅值是它的两倍、相位超前 26°（约 13 拍）。

这 13 拍不是巧合，它约等于反馈支路在本轨迹上的等效滞后：比例 2.0、积分 0.05、微分 0.20 的 PID 在 0.25 Hz 处的相位约 -26°，折算到 4 s 周期正好是 0.29 s，接近 13 拍。记忆先把这份滞后“预支”出去，反馈再把它还回来，两者相加才等于对象需要的输入。

这条结论有很强的工程含义：**不能拿 ILC 收敛后的记忆去直接比对模型逆，也不能把记忆单独抽出来当“学到的模型”**。记忆只有和反馈合在一起才有物理意义。反过来，如果现场把反馈参数改了而没有清空记忆，记忆里预支的那一份就对不上了，误差会先跳一下再重新收敛——这是 69.10 节“记忆版本管理”要求的直接理由。

记忆的量纲与单位换算

记忆 u_k 与控制量同量纲，因为它最终要与反馈输出相加后送进执行器。这一点在纸上是自明的，在工程上却是最常见的错误来源：控制器内部用 mA、上位机显示用 %、机械图纸用 N，三套单位只要有一处漏乘，学出来的记忆就整体差 10 倍或 100 倍。

量纲错了不会立刻“崩”，ILC 会照常收敛到“误差最小”，只是那个最小点被限幅卡住，或者前馈量级离谱。判断的办法有两个。**一是先算 u^{*}**：式 (69-21) 给出的 RMS 是 0.1166、范围 $[-0.154,\,0.236]$，学到的记忆应当与它同量级。**二是看两者的比例**：本章学到的记忆 RMS 是 0.2070，是 u^{*} 的 1.78 倍，多出来的这一份正是反馈支路预支的滞后（见上一小节）。所以“记忆 RMS 落在 u^{*} 的 1.5 到 2 倍之间”可以作为量纲正确的一条粗判据；若差 10 倍以上，先查单位，不要调增益。

存储量的账顺便算一下：400 个双精度数是 3.1 KB，双缓冲要两份，加上时间戳与版本号，一个品种约 8 KB。一百个品种 800 KB，任何控制器都放得下。真正占空间的不是记忆本身，而是“为了可追溯保留的每遍误差日志”——400 点 × 60 遍 × 8 字节 = 192 KB，比记忆大两个数量级。若日志要长期保留，就得按“首末两遍全存、中间每 10 遍存一次”的方式抽样。

连续视角给出的三条实现约束

连续域的分析虽然不能给出精确的量化，但它给了三条在实现时必须遵守的约束。

第一条：采样率要显著高于想学的最高频率。 本章想学到 1.25 Hz 附近，采样 100 Hz，一个周期 80 点，足够；若要学到 10 Hz，10 ms 的采样就只剩 10 点，差分求导完全失真，必须提高采样率或改用更长的学习窗。

第二条：D 型学习律必须先低通再差分，或差分后立刻低通。 连续域的 $\dot{e}$ 是理想微分，增益随频率无限增长；离散实现里中心差分的幅频特性是 $\sin(\omega T_s)/T_s$，在奈奎斯特附近会折叠。本章的做法是差分后接 30 ms 一阶低通，它在 1.25 Hz 处的衰减只有 4%，在 10 Hz 处衰减 70%，正好把不该学的高频挡在外面。

第三条：连续律假设整条误差曲线在执行结束后仍然可访问。 这在仿真里天然成立，在实机上要求控制器把整条轨迹的误差存下来（400 个浮点数，3.1 KB）。若内存受限只能存抽样点，那么可学带宽就按抽样率减半，Q 滤波的窗长也要相应放大。

69.5 离散情形

记忆是一张表

离散实现把式 (69-17) 写成按拍的更新：

$$u_{k+1}[j] = u_k[j] + \gamma_p e_k[j] + \gamma_d \, \dot{e}_k[j], \qquad j = 0, 1, \dots, N-1 \tag{69-23}$$

再对整条序列做 Q 滤波并限幅：

$$u_{k+1} \leftarrow \mathrm{clip}\big(Q\, u_{k+1},\, -u_{\max},\, u_{\max}\big) \tag{69-24}$$

这两步的顺序不能换。先滤波后限幅，滤波器的输出会被限幅截断，但截断只影响已经越界的点，不会通过滤波器扩散到邻近点；先限幅后滤波，滤波会把越界点的值“抹”到邻近点上，造成一段连续的平台——本章早期版本犯过这个错，结果误差在饱和区附近出现一条 0.3 s 长的平顶，看起来像“对象卡住了”，实际是滤波把饱和抹开了。

更新完的 u_{k+1} 就是下一遍要查的表。它的长度固定为 N，与迭代次数无关；60 次迭代不会让记忆变长，只会让它变准。这是 ILC 与“在线学习”的又一处区别：在线学习的参数量固定但样本无限，ILC 的样本数固定（每条轨迹 400 点）而迭代次数可以很多。

迭代轴不是时间轴

初学者容易把“迭代”和“采样”混成一个概念，以为迭代次数多了就相当于在迭代轴上提高了采样率。这两条轴的性质完全不同，混了会得出错误的设计结论。

时间轴上有采样周期 $T_s = 10$ ms，它决定能表示的最高频率 50 Hz，也决定离散化引入的半拍相位。迭代轴上没有“采样率”这回事：第 k 遍与第 $k+1$ 遍之间不存在任何信号，控制器不会去插值两遍之间的状态，也不需要在遍与遍之间维持什么动态。迭代次数只是几何级数的项数，它决定“把误差乘多少次 λ”，不决定“能学多高的频率”。

一个常见的误推是：“迭代 60 次，相当于在迭代轴上采了 60 点，所以能学 30 遍一周的振荡。”这个说法没有意义——ILC 从不处理“遍与遍之间的振荡”，它的状态只有一张表，表的更新是代数加法，不是差分方程。真正限制可学带宽的是每次迭代内的采样与闭环频响，也就是 69.6 节的判据。

由此还能得到一个反直觉的推论：**迭代次数不能靠“跑得更快”来换。**每次迭代必须完整跑一遍 4 s 的轨迹，想多学 10 遍就得多等 40 s，没有别的办法。这与“提高采样率能加快收敛”的直觉正好相反——在本章里，提高采样

率只会让单次迭代更贵，收敛所需的遍数一点不变。

误差从哪一拍开始算

有一个实现细节会影响收敛的前几拍：误差用的是“当前拍的测量值”还是“上一拍的测量值”。本章用当前拍，即 $e[j] = r[j] - y[j]$，其中 $y[j]$ 是本拍采样结束后的输出。这与第 67 章的写法一致，代价是控制量在计算时用的是本拍的测量，实机上这一拍的计算必须在该采样周期内完成（本章 2.82 ms 的仿真时间远小于 10 ms，留有余量）。

若实机的计算来不及，退而用上一拍的测量，等效于在整个学习环里多加一拍延迟。这一拍在低频无所谓（0.25 Hz 处只多 0.9° 相位），但在 1.25 Hz 处多 4.5°，足以把原本勉强收敛的频点推过界。所以“计算延迟”在 ILC 里的地位比在 PID 里更高：PID 多一拍只是裕度变小，ILC 多一拍可能直接改变收敛性。

延迟落在学习环里

5 拍纯延迟对 ILC 的影响，与对 PID 的影响性质不同。对 PID，延迟消耗相位裕度，表现是“增益不敢调大”；对 ILC，延迟直接进入 $T(j\omega)$，改变的是“哪个频段能学”。本章的实测给出了一个反直觉的结果：**把误差前移若干拍再进学习律（所谓超前补偿），并不能改善收敛，反而更差**。

试过的方案是把 e_k 前移 3、5、8、12 拍，再代入学习律，相当于用“未来的误差”做修正。结果是前移 0 拍最好，前移越多误差越大——在另外一种配置下，0 拍时误差 0.0355，前移 5 拍变成 0.0622，前移 8 拍变成 0.1082，几乎翻倍。原因在式 (69-6)：超前补偿改变的是 T 的相位，把相位往前推，看似补了延迟，实际上把 $1 - \gamma T$ 的工作点推到了更接近 1 的位置，收缩变慢；同时它把高频段的相位搅乱，让原本已经越界的频段越界更多。

真正的对策有两条，都在频域上：**降低 γ 让整条曲线整体靠近 1 但不越过，加 Q 滤波把越界频段直接剪掉**。前者牺牲速度，后者牺牲带宽，没有第三条路。这条结论在 69.6 节会被谱数据完全印证。

限幅是非线性的入口

式 (69-24) 里的限幅是 ILC 唯一的强非线性环节，它的作用方式与 PID 里的限幅不同。PID 的限幅保护执行器，饱和时通过抗饱和逻辑回退积分；ILC 的限幅保护的是“记忆不要长出一条荒谬的曲线”，一旦触发，式 (69-6) 的线性递推立刻失效——因为被截断的那部分误差无法通过学习消除，下一遍还会产生同样的误差，再被截断。

这会造成一种特殊的失效模式：**记忆在饱和区附近来回抖动，误差不增不减**。本章 69.6 节会给出它的定量标志——饱和比例。纯反馈的饱和比例是 0，$\gamma_p = 0.30$ 发散后升到 51.2%，而正式算例（PD + Q81）全程为 0。这条指标比误差更早报警：误差还在 0.1 附近“看起来正常”的时候，饱和比例可能已经从 0 涨到 3%。

定点实现会抬高地板

本章的代码用双精度浮点，量化误差完全可以忽略。但现场控制器常用 16 位定点（DSP 的整数运算、部分 PLC），这时要算一笔新账：量化会把“学习停住”的位置抬高。

先看测量端。若 ADC 是 16 位、量程 ± 1.2，一个 LSB 是 $2.4/65536 = 3.66 \times 10^{-5}$。学习增量是 $\gamma_p e$，当 $|e|$ 降到 3.66×10^{-4} 时增量只有 3.7×10^{-5}，与一个 LSB 相当；误差再小，增量就会被四舍五入掉，学习事实上停止。这个阈值 $|e| < \mathrm{LSB}/\gamma_p$ 就是定点实现的学习地板。

把它和浮点地板放在一起看：本章末次 RMS 是 0.0224，$\gamma_p = 0.10$ 对应的定点地板是 3.7×10^{-3}，比它小一个数量级，够用。若为了压高频把 γ_p 取到 0.005，地板抬到 7.3×10^{-3}，仍低于本章的浮点地板，本算例不成为瓶颈；但换一个重复性更好的对象（浮点地板能到 10^{-3} 量级），定点地板就会变成新的限制。所以量化不是“总能忽略”的小事，它取决于你打算学到多细。

对策有两条，成本都很低。第一，记忆用单精度浮点存，相对精度 10^{-7}，相对 0.2 的量级是 2×10^{-8}，完全够；第二，若必须用定点，就让增量在高精度累加器里累加，只在查表输出时取整，把“每拍截断一次”变成“每遍截断一次”，地板随之降下来。

一次迭代的成本

把离散实现的成本算清楚，有助于判断它能不能落在实时预算里。一次迭代包含：400 拍 × 每拍 5 次子步的对象积分、一次 400 点的 Q 滤波（用卷积实现）、一次 400 点的差分与低通。本章在 Python 里实测 60 次迭代 169.3 ms，单次 2.82 ms；其中对象积分占九成以上，学习律本身不到 0.1 ms。

这个比例说明：ILC 的算力开销几乎全在“跑一遍任务”上，而这一遍本来就要跑。真正新增的只有学习律与滤波，量级是微秒。所以在实机上，ILC 的算力门槛基本可以忽略；真正的门槛是内存（要存整条轨迹的误差与记忆）与“任务必须可重复”这个前提。

四种学习律的实测

表 69-3 是四种配置的实测结果，统一 30 次迭代、统一反馈支路、都不加 Q 滤波，这样比较的是学习律本身。

学习律	首次 RMS	第 5 次	末次 RMS	最小 RMS	末次最大偏差	降幅
纯反馈（无学习）	0.1161	0.1161	0.1161	0.1161	0.1974	0
P 型 $\gamma_p=0.10$	0.1161	0.0972	0.0693	0.0619	0.1538	40.4%
D 型 $\gamma_d=0.005$	0.1161	0.1145	0.1066	0.1066	0.1609	8.2%
PD 型 $0.10/0.005$	0.1161	0.0956	0.0506	0.0497	0.1167	56.5%

三个数字值得注意。**P 型 30 次到 0.0693，但它不是单调的**：第 21 次到最低 0.0619，之后回升到 0.0693——这就是 1.25 Hz 分量在暗中增长的开始。**D 型几乎没用**，30 次只降 8.2%，因为它学的是误差的导数，而本章误差的主要成分是“滞后形状”，导数在整条轨迹上正负相抵，累积量很小。**PD 型最好**，但它末次的最大偏差 0.1167 仍然不小，说明误差的峰值没有被压下去，压下去的是“平均”。

这三条合起来说明：学习律的选择不能只看末次 RMS，还要看单调性与最大偏差。同一张表里，P 型的“最小值”比“末次值”低 0.0074，这个差值就是发散分量的贡献；工程上若只报末次值，会低估风险。

迭代过程长什么样

图 69-3 把 60 次迭代的误差画成一张热图：横轴是迭代内的时间，纵轴是迭代次数，颜色是误差。这张图比任何曲线都直观地说明了 ILC 在做什么。

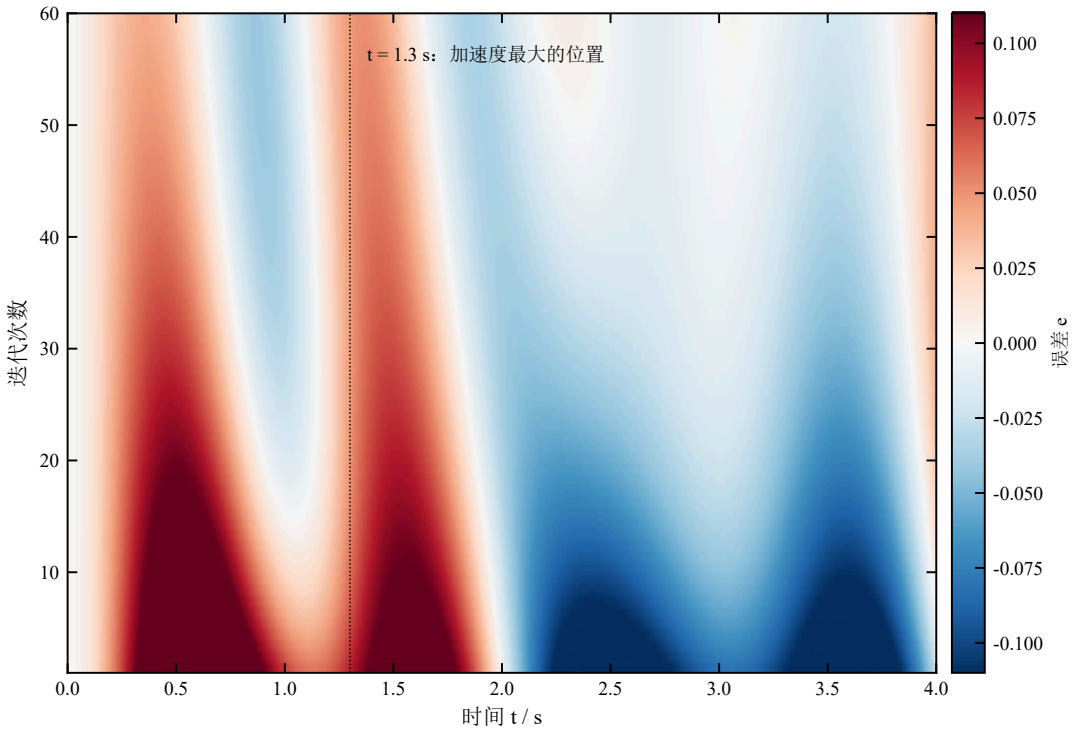


图 69-3 误差在迭代—时间平面上的演化

前 10 次，深色（大误差）区域集中在 $t = 0.6$ 到 1.6 s 与 $t = 2.4$ 到 3.4 s 两段——正是轨迹速度最大的两段，也是反馈滞后最严重的地方。随着迭代推进，这两块深色从中间开始变浅，向两端收窄，到第 40 次以后基本只剩下一层浅色。向上的“条纹”几乎是竖直的：同一时刻的误差随迭代次数平滑变化，没有出现“某一遍突然变好又变坏”的锯齿——这正是单调收敛的视觉特征。

图里还标了一条 $t = 1.3$ s 的虚线。沿这条竖线看，颜色从最开始的深红一路变到接近白色，它是整条轨迹上收敛最快的时刻之一；而沿 $t = 0$ 的左边线看，颜色变化很小——起点附近的误差学不动，因为那里的误差主要来自“复位瞬间”的瞬态，而不是重复性的跟踪误差。这条观察直接对应 69.10 节的一个工程建议：**学习窗的两端应留出保护段，不要把复位瞬态学进记忆。**

69.6 稳定性与性能

先把现象摆出来

讨论稳定性之前，先看正式算例的误差谱怎么变。表 69-4 是从 400 点误差序列里做 DFT 得到的五个频率分量，列是第 1、6、11、21、41、60 次迭代。选这五个频点不是随意的：0.25 Hz 与 0.75 Hz 是参考轨迹的基波与三次谐波，0.50 Hz 是两者的差频成分，1.00 Hz 与 1.25 Hz 则是本章的“问题频段”——后面会说明它们为什么出问题。

频率 / Hz	第 1 次	第 6 次	第 11 次	第 21 次	第 41 次	第 60 次	首末变化
0.25	0.1471	0.1148	0.0896	0.0548	0.0209	0.0089	-94.0%
0.50	0.0148	0.0119	0.0096	0.0064	0.0039	0.0032	-78.4%
0.75	0.0686	0.0609	0.0541	0.0428	0.0276	0.0189	-72.4%
1.00	0.0158	0.0162	0.0164	0.0168	0.0177	0.0191	$+20.9\%$
1.25	0.0105	0.0110	0.0115	0.0123	0.0126	0.0123	$+17.1\%$

这张表把“ILC 在学什么”说得很清楚：**它只学得动自己够得着的频段**。0.25 Hz 掉了 94%，0.75 Hz 掉了 72%，两者对应的正是参考轨迹的能量主峰；1.00 Hz 与 1.25 Hz 不但没降，反而各涨了两成。五个分量里三条降、两条升，升的恰好是频率最高的两条——这不是巧合，而是 69.2 节那个收敛条件的直接后果。

还有一处细节值得留意：第 1 次谱里 0.25 Hz 的幅值是 0.1471，1.25 Hz 只有 0.0105，前者是后者的十四倍。也就是说，误差在时域上“看起来像滞后”，实际上主要是两根谱线（0.25 Hz 与 0.75 Hz）叠出来的波形；1.25 Hz 那一点点能量，在第 1 次时根本看不出来，但它决定了 60 次之后能不能继续往下走。图 69-6 把这张表画成了曲线。

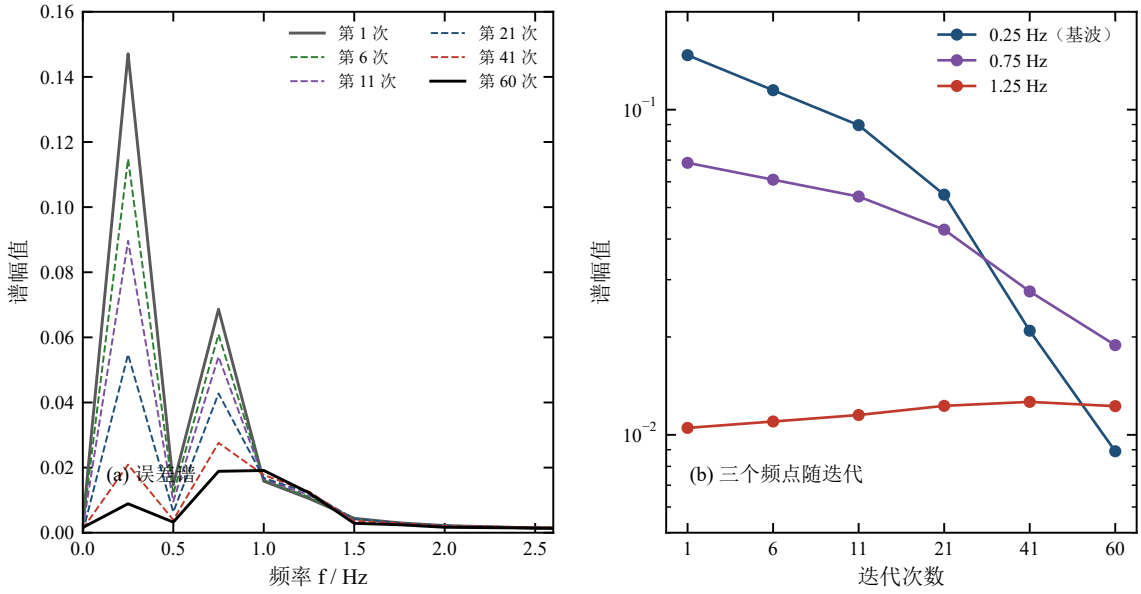


图 69-6 误差谱随迭代的演化

判据是一个闭环频响的最大模

69.2 节给出的收敛条件，用频率写出来是一行：

$$\lambda = \max_{0\le\omega\le\omega_N} \left|1 - \gamma_p\, T\!\left(\mathrm{e}^{\mathrm{j}\omega T_s}\right)\right| < 1 \tag{69-25}$$

这里的 T 不是被控对象，而是“前馈记忆到输出”的整条通路：从记忆入口打一拍脉冲，经过求和、限幅、5 拍纯延迟、对象，再由测量端返回，反馈支路（PID、微分低通、积分）全程闭合。换句话说，T 描述的是“记忆里多加一点，输出会多出多少”，而 ILC 每一次迭代做的事情，就是把记忆按误差的反方向改一点。所以误差的传递恰好是 $1-\gamma_p T$：T 在某个频率上“转得太狠”，$1-\gamma_p T$ 就会大于 1，那个频率上的误差就会越学越大。

这个式子的价值在于它把反馈、延迟、采样全部包了进去。工程上常见的错误是拿对象本身的相位去判断 ILC 能不能用：本算例的对象在 1.25 Hz 的相位是 -188°，看上去早就该判死刑；可闭环之后 T 的相位是 -146°，并

没有那么糟。反过来，对象在 2 Hz 的相位是 $+161^\circ$（延迟把相位绕了一圈），看上去“很安全”，但 T 在那里仍有 0.164 的幅值，判据仍然略微越界。**判断 ILC 的稳定性，只能看闭环，不能看开环。**

判据表

表 69-5 是 T 在六个频点上的实测值，最后一列按 $\gamma_p = 0.10$ 算出 $|1-\gamma_p T|$。T 由线性化模型算出（$K = 16.5$，对应轨迹中段 $y \approx 2.08$），计算时把反馈支路、5 拍延迟、30 ms 微分低通全部包含在内。

频率 / Hz	$\lvert T\rvert$	$\arg T$ / °	$\lvert 1-0.10T\rvert$
0.25	0.5256	-13.77	0.9490
0.50	0.6016	-30.35	0.9486
0.75	0.7556	-55.16	0.9588
1.00	0.8783	-98.62	1.0169
1.25	0.6263	-145.91	1.0525
1.50	0.3718	-172.84	1.0369

读表有三个层次。第一层：**分界在 0.95 到 1.00 Hz 之间**——0.75 Hz 的 0.9588 还在收敛侧，1.00 Hz 的 1.0169 已经越界，这与表 69-4 里“0.75 Hz 降、1.00 Hz 升”完全对应。第二层：**峰值不在相位最负的地方**。1.50 Hz 的相位是 -172.84°，比 1.25 Hz 的 -145.91° 更接近反相，可它的 $|1-\gamma_p T|$ 只有 1.0369，低于 1.25 Hz 的 1.0525。第三层：**越界幅度并不大**，最坏也只有 5.25%，这解释了为什么本章的失稳不是“一下子发散”，而是“某一根谱线悄悄往上爬”。图 69-7 把这两件事画在一起：左图是四个增益下的 $|1-\gamma_p T|$ 曲线，右图是加上 Q 滤波之后的同一条曲线。

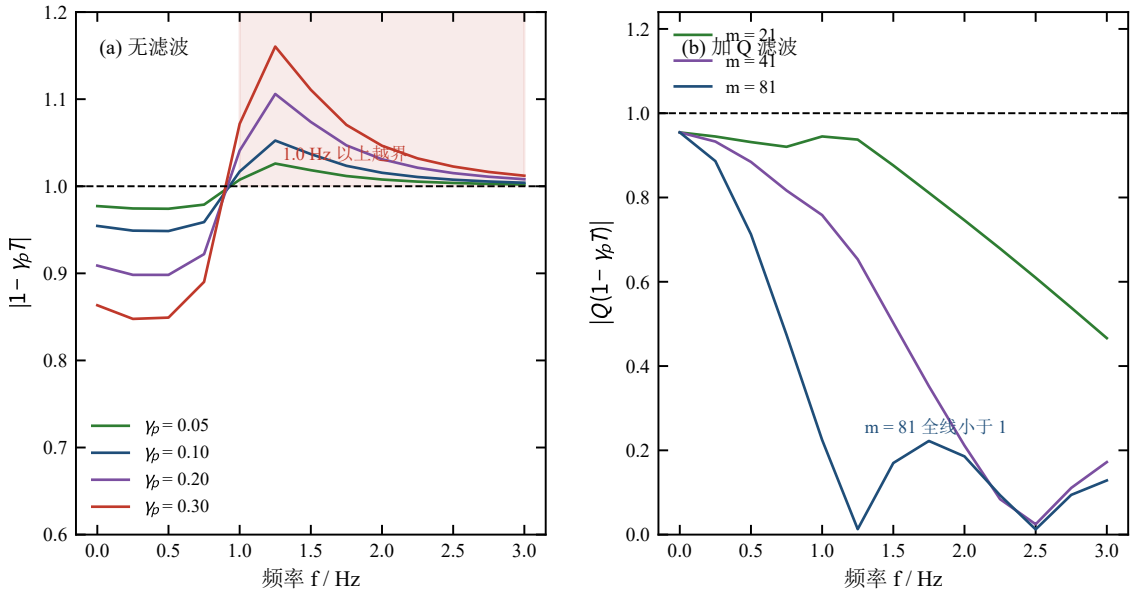


图 69-7 收敛判据与 Q 滤波的作用

峰值为什么钉在 1.25 Hz

把模平方展开，可以看清 $|1-\gamma_p T|$ 到底在“惩罚”什么：

$$|1-\gamma_p T|^2 = 1 - 2\gamma_p \,\mathrm{Re}\,T + \gamma_p^2 |T|^2 \tag{69-26}$$

式子里出现的是 T 的**实部**，不是幅值，也不是相位。实部为正，第三项以外的那一项就在帮忙衰减；实部为负，它就在帮忙放大。把表 69-5 的四个频点代进去（$\gamma_p = 0.10$）：0.25 Hz 处 $\mathrm{Re}\,T = 0.5105$，$-2\gamma_p\,\mathrm{Re}\,T = -0.1021$，加上 $\gamma_p^2|T|^2 = 0.0028$，模平方 0.9007，开方 0.9490；1.00 Hz 处 $\mathrm{Re}\,T = -0.1316$，那一项变成 $+0.0263$，模平方 1.0340，开方 1.0169；1.25 Hz 处 $\mathrm{Re}\,T = -0.5188$，那一项 $+0.1038$，模平方 1.1077，开方 1.0525；1.50 Hz 处 $\mathrm{Re}\,T = -0.3689$，那一项 $+0.0738$，模平方 1.0752，开方 1.0369。

于是峰值的成因可以一句话说完：**1.25 Hz 是“幅值还没掉下去、实部已经翻负”的重叠区**。1.00 Hz 的幅值更大（0.8783），但实部只到 -0.13，放大得还不够；1.50 Hz 的实部更负（-0.37），但幅值已经掉到 0.372，放大量跟着缩水。两头都不如中间的 1.25 Hz：幅值 0.626 意味着“还有力气”，实部 -0.519 意味着“方向全错”。

这个解释还能推出一条更实用的结论：**T 的实部由 -90° 这条线决定**。0.75 Hz 时相位 -55°，实部仍为正，所以还在收敛侧；1.00 Hz 时相位 -98.6° 刚过 -90°，实部转负，判据立刻越界。也就是说，闭环相位穿过 -90° 的频率附近，就是 ILC 最脆弱的地方。这与“相位裕度”不是一回事：本算例的反馈回路相位裕度接近 60°，单独看反馈很稳，可 ILC 仍然在 1.0 Hz 以上越界，因为 ILC 关心的是 T 的实部，而反馈关心的是 $1+T$ 是否绕过 -1。

再往下追一层，1.25 Hz 的幅值为什么反而比 0.75 Hz 小、却比 1.50 Hz 大？因为反馈支路在这一点把开环幅值放大了。对象加延迟在 1.25 Hz 的幅值是 0.259，闭环 T 是 0.626，放大了 2.4 倍；放大的代价是 $1+CP$ 的模只剩 0.41。同一时刻，$1+CP$ 的相角是 -44°，把 T 的相位从开环的 -188° 拉回到 -146°。**反馈一边抬幅值、一边抬相位**：抬相位是好事（让实部没那么负），抬幅值是坏事（让放大量变大），两者相抵，净效果就是 1.25 Hz 处那个 5.25% 的超标。

越界不等于看得见

表 69-5 只算了 $\gamma_p = 0.10$。把增益从 0.02 扫到 0.30，得到表 69-6。六行的峰值频率全部是 1.25 Hz，越界频段的起点全部是 1.00 Hz，这说明峰值位置由 T 决定，与增益无关；增益只改变超标的**程度**。

学习增益 γ_p	$\max\|1-\gamma_p T\|$	峰值频率 / Hz	每步倍数	30 次累计	实测末次 RMS	实测发散
0.02	1.0104	1.25	1.010	1.36	0.0904	否
0.05	1.0261	1.25	1.026	2.16	0.0672	否
0.10	1.0525	1.25	1.053	4.66	0.0693	否
0.15	1.0791	1.25	1.079	9.70	0.1196	是
0.20	1.1060	1.25	1.106	20.4	0.2206	是
0.30	1.1604	1.25	1.160	86.6	0.2877	是

表里最容易被误读的是前三行。它们的判据都已越界，按定义就是不收敛，可 30 次的实测都没有被判为发散，$\gamma_p = 0.02$ 的末次 RMS 是 0.0904，比 $\gamma_p = 0.10$ 的 0.0693 只差一点，时域曲线看上去平平稳稳。原因在“每步倍数”那一列：1.010 的三十次方只有 1.36，而 1.25 Hz 的初始能量只有 0.0105，涨到 0.014 对总 RMS 的贡献微乎其微。$\gamma_p = 0.30$ 就完全不同：每步 1.160，三十次是 86.6 倍，0.0105 被抬到 0.9，早超出了精确前馈 0.236 的量级，最后被限幅削平，末次 RMS 反而更大。

所以判据回答的是“有没有地雷”，不回答“什么时候踩到”。要判断 N 次迭代之内是否安全，必须把判据与初始谱乘起来：

$$|E_0(\omega)|\cdot\lambda(\omega)^K<\varepsilon_{\text{allow}},\qquad \lambda(\omega)=\left|1-Q(\omega)\,\gamma_p T(\mathrm{e}^{\mathrm{j}\omega T_s})\right| \tag{69-27}$$

式 (69-27) 里多了一个 $Q(\omega)$，因为正式算例的 Q 滤波是乘在学习律外面的，它直接改写每个频点的有效增益。这也是 Q 滤波的真正作用：它不改变 T，只改变“哪些频率有资格被学”。加了 81 拍窗之后，1.25 Hz 的 $|Q|$ 只有 0.012，有效增益从 0.10 掉到 0.0012，判据在这一点的值从 1.0525 回到 1.0006，越界幅度被压到千分之零点六——六十次累积也只有 3.7%，落在初始能量 0.0105 上就是 0.0109，实测 0.0123，量级吻合。

顺带说明“越界频段 1.00 到 50 Hz”这个说法该怎么读。50 Hz 是奈奎斯特频率，这一段里确实还有若干频点名义上大于 1，但超标量随频率迅速变小：2 Hz 是 1.0155，3 Hz 是 1.0040，4 Hz 是 1.0012，5 Hz 是 1.0003，8 Hz 附近甚至略低于 1。这些频点的初始能量本来就在 10^{-3} 以下，三十次涨不到 40%，对时域误差的贡献可以忽略。**真正需要动手的只有 1.0 到 1.5 Hz 这一小段**，Q 窗长也是围着它选的（69.7 节会给出窗长与这一段的定量关系）。

1.25 Hz 是从哪来的

一个自然的追问是：参考轨迹里只有 0.25 Hz 与 0.75 Hz 两根谱线，1.25 Hz 的误差是从哪冒出来的？它凭什么在第 1 次就有 0.0105 的幅值？

先排除几种可能。它不是参考的谐波——式 (69-16) 只有基波与三次谐波，五次谐波幅值为零。它也不是采样或 Q 滤波引入的——第 1 次迭代 $u_0\equiv 0$，整条记忆还是空的，滤波无从作用。剩下的来源只能是**对象与反馈的非线性**。

最可能的一条通道是增益调制。$K(y)=4+6y$ 沿轨迹以 4 s 为周期变化，等价于一个 0.25 Hz 的周期系数乘在信号上。乘一个 0.25 Hz 的周期函数，会把已有的每根谱线搬移 ± 0.25 Hz：1.00 Hz 的误差因此生成 0.75 Hz 与 1.25 Hz 两个边带。表 69-4 里第 1 次的 1.00 Hz 是 0.0158、1.25 Hz 是 0.0105，量级相当，与“边带”这个图像吻合。

本章没有单独做实验去证实这条机理，但验证的办法很简单：把 $K(y)$ 换成常数 $K=16.5$ 再跑一遍，若 1.25 Hz 分量随之消失，就说明它确实是调制产生的；若仍在，就得另找来源（比如限幅、或反馈里微分的非线性）。这是一个十行代码就能做完的对照，留给读者自己动手。

这个追问的价值不在机理本身，而在它提示了一个设计原则：**判断“哪个频点危险”不能只看参考轨迹的谱**。参考只有两根线，误差却有五根以上，多出来的那些是对象自己“造”的。若按参考的谱去选 Q 窗长（只保住 0.25 与 0.75 Hz），就会漏掉 1.25 Hz 这根最危险的线。

判据可以现场反推

模型不准时不必硬算 T。由式 (69-10)，相邻两次的误差谱满足 $\hat{e}_{k+1}=(1-\gamma_p Q T)\hat{e}_k$，把它反过来解，得到 $T(\omega)=\big[1-\hat{e}_{k+1}(\omega)/\hat{e}_k(\omega)\big]\big/\big[\gamma_p Q(\omega)\big]$。

其中 $\hat{e}$ 必须取复数谱——用幅值谱就把相位丢了，而相位恰恰是判据里最关键的成分。这个式子的用法很直接：同一工况跑两遍，对两条误差曲线做 DFT，逐点相除，就得到 T 在每根谱线上的值。它不需要额外注入测试信号，也不打断生产节拍，属于“顺手就能做”的现场自检。

拿表 69-4 的数验一下。0.25 Hz 处第 1 次是 0.1471、第 6 次是 0.1148，间隔 5 遍，每遍因子 $(0.1148/0.1471)^{1/5}=0.9516$；又知 $\gamma_p|Q|=0.0934$，于是反解出 $\mathrm{Re}\,T=(1-0.9516)/0.0934=0.518$。模型给的是 $0.5256\angle -13.77^\circ$，实部 0.5105，两者差 1.4%。对一个“用两条误差曲线猜出闭环实部”的方法来说，这个精度已经足够做判据。

它在“学分不动”的频点上却会失效，这一点同样值得看。1.25 Hz 处第 1 次 0.0105、第 60 次 0.0123，59 遍的每遍因子是 1.0027，而 $\gamma_p|Q|$ 只剩 0.0012，反解出 $\mathrm{Re}\,T\approx -2.2$，真值是 -0.52，差四倍。公式没错，是分母太小：$\gamma_p|Q|$ 降到 10^{-3} 量级时，谱估计的任何误差（测量噪声、非重复成分）都会被放大上千倍。所以反推只在“

$\gamma_p|Q|$ 不小、谱的信噪比够高”的频段可信——这恰好就是“能学”的频段，也算一种自洽：**判据能用的地方，正是学习用得上力的地方**。

判据的适用边界

式 (69-25) 是线性判据，用之前要确认四件事，否则算出来的数只是参考。

第一，**线性化点要覆盖工作区间**。本算例的 $K(y)=4+6y$ 在轨迹范围 $[1.50,\,2.72]$ 上从 13.0 变到 20.3，判据取的是中段 16.5。增益偏高时 T 的幅值整体变大，判据会更悲观；偏低时更乐观。69.9 节的鲁棒测试把增益上下各拉 20%，带 Q 滤波时末次 RMS 分别是 0.0161 与 0.0351，都还在收敛侧，说明 5% 的余量加 81 拍 Q 滤波足够吸收这个范围的变化。但如果没有 Q 滤波，这个余量就不够了。

第二，**限幅不能频繁动作**。一旦削顶，T 就不再是线性环节，判据描述的“每步乘一个固定倍数”不再成立。本算例的精确前馈范围是 $[-0.154,\,0.236]$，限幅在 ± 1.2，余量近五倍，60 次迭代里饱和比例始终为零，所以判据有效。工程上若饱和比例超过百分之一，应先把参考轨迹或限幅放宽，再看判据。

第三，**每遍的动力学要一样**。判据假设 T 不随迭代次数变化。负载变化、温漂、夹具磨损都会让 T 漂移，这时判据只能逐次重算，或者改用带遗忘因子的学习律（69.9 节）。非重复扰动不进 T，它进的是每一遍的初始误差，处理方式也不同。

第四，**多输入多输出时判据变成谱半径**。T 从标量函数变成传递矩阵，$1-\gamma_p T$ 变成 $I-\Gamma T$，判据改为

$$\rho\left(I-\Gamma T(\mathrm{e}^{\mathrm{j}\omega T_s})\right)<1 \quad \text{对全部 } \omega \tag{69-28}$$

其中 $\rho(\cdot)$ 是谱半径。通道之间的耦合会让某几个频率的谱半径远大于单个通道的判据值，这时“每个通道单独调好”并不等于“整体收敛”，必须按矩阵算。

最后要提醒一句：式 (69-25) 保证的是**渐近收敛**，不保证**单调**。判据小于 1，误差的包络最终会下去，但中间可以有起伏；要做到逐次单调，需要更强的条件（例如 T 正实，或 $\|I-\Gamma T\|_2<1$）。本章的正式算例之所以看起来单调，是因为 81 拍 Q 滤波把 1.0 Hz 以上的有效增益压到了千分之一量级，等于把不单调的成分先剪掉了，剩下的频段本来就单调。这一点在 69.7 节讨论窗长时会再回到。

还有一层边界要说清：判据只回答“会不会收敛”，不回答“收敛到多低”。$\rho<1$ 只保证 $\rho^k\to 0$，而实际地板由 Q 的通带、参考轨迹的谐波分布、以及非重复成分共同决定，本章是 0.0224。换句话说，**判据是入场券，不是成绩单**——过了判据只说明可以开跑，能跑多准要另算。

69.7 参数意义与作用

三个旋钮，各管一件事

ILC 的设计参数不多，真正需要调的只有三个：学习律的类型（P、D 还是 PD）、学习增益 γ_p 与 γ_d、Q 滤波的窗长 m。它们的分工可以先用一句话概括：**学习律决定“往哪个方向改”，增益决定“改多快”，Q 决定“哪些频率允许改”**。三个旋钮不是并列的，调参有先后：先定学习律，再定增益，最后用 Q 收口。顺序反了会白费功夫——先加 Q 会把增益的效果一起滤掉，先调增益又会在高频上撞墙。

P 型、D 型与 PD 型：各自补什么

图 69-4 是三种学习律与纯反馈的对比，四条曲线都是误差 RMS 随迭代次数的变化，统一 30 次迭代、统一反馈支路、不加 Q 滤波。这样比较的是学习律本身，而不是它和滤波器搭配的效果。

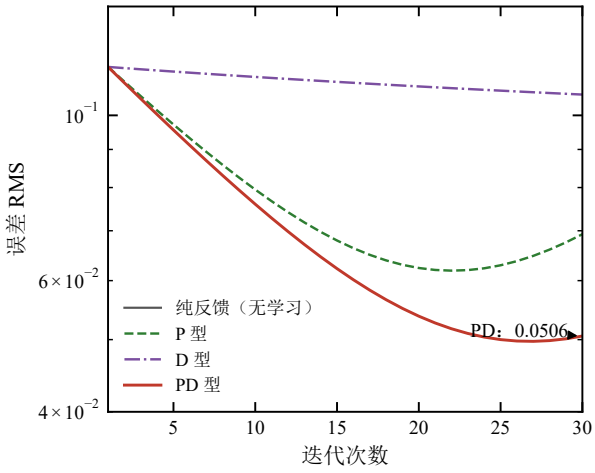


图 69-4 三种学习律的收敛曲线

四条曲线的形状差别很大。纯反馈是一条水平线，0.1161 从头到尾不动——没有记忆，每一遍都从零开始，误差当然不会变。P 型前 10 次掉得很快，从 0.1161 到 0.075 左右，但第 21 次触底 0.0619 之后开始回升，到第 30 次是 0.0693，曲线末端明显翘起来。D 型几乎是平的，30 次只从 0.1161 降到 0.1066，降幅 8.2%，肉眼看不出学习的效果。PD 型兼有两者的优点，第 5 次就降到 0.0956，第 30 次到 0.0506，且末端还没有明显翘起。

D 型单独用为什么这么差？因为它学的是误差的导数。本章误差的主要成分是“滞后形状”——参考上升时输出跟不上，误差是同一符号的一段平台，这段平台的导数在起点为正、终点为负，积分起来正负相消。所以 D 型对“整段滞后”几乎没有记忆可积累，它擅长的是“误差在时间上错位”的情形，而不是“误差在幅值上不够”。

那 PD 为什么又比 P 好？因为 D 项不是用来“积累幅值”的，而是用来“转动更新方向”。在频域里，P 型的更新量是 $\gamma_p E$，D 型是 $\mathrm{j}\omega\gamma_d E$，两者相加就是

$$\Delta u_k(\mathrm{j}\omega) = \left(\gamma_p + \mathrm{j}\omega\gamma_d\right) E_k(\mathrm{j}\omega) \tag{69-29}$$

括号里是一个“带相位超前”的复数增益。在 0.25 Hz 处，$\omega\gamma_d = 0.00785$，相对 $\gamma_p = 0.10$ 只有 7.9%，提供的相位超前约 4.5°；在 0.75 Hz 处，$\omega\gamma_d = 0.0236$，比值 23.6%，相位超前约 13°。别小看这十几度：判据里起决定作用的是 T 的实部，而实部对相位最敏感的地方正是相位接近 -90° 的频段，把更新量转过去十几度，等于把 $1-\gamma T$ 往回收一点。这也是为什么 PD 的末次最大偏差从 P 型的 0.1538 降到 0.1167——误差的峰值位置被“转”掉了一部分。

代价是噪声。D 项对测量噪声的放大倍数随频率线性增长，10 ms 采样下，1 kHz 的噪声会被放大 $2\pi \times 1000 \times 0.005 \approx 31$ 倍。所以 D 项后面必须接低通，本章取 30 ms 一阶低通，在 0.75 Hz 处只损失 8° 相位和 1% 幅值，在 50 Hz 处把噪声压到 1/10 以下。这个低通不是可选项，是 D 型能用的前提。

学习增益：拿收敛速度换稳定余量

表 69-7 是 P 型增益从 0.02 扫到 0.50 的实测结果，30 次迭代、不加 Q 滤波。表中“几何比”是前 6 次相邻 RMS 比值的中位数，它近似等于 λ 在低频段的值；“半程迭代”是按这个比值算出的、误差降到一半所需的次数。

γ_p	最小 RMS	末次 RMS	几何比	半程迭代	实测发散
0.02	0.0904	0.0904	0.991	77	否
0.05	0.0672	0.0672	0.978	31	否
0.10	0.0619	0.0693	0.957	16	否
0.15	0.0620	0.1196	0.937	11	是
0.20	0.0622	0.2206	—	—	是
0.30	0.0624	0.2877	—	—	是
0.50	0.0628	0.3371	—	—	是

这张表里有两个数字特别值得盯住。第一个是**几何比与半程迭代的对应**：0.991 意味着每遍只消掉 0.9% 的误差，要 77 遍才减半；0.937 意味着每遍消掉 6.3%，11 遍就减半。同一个对象、同一个反馈，只换一个标量增益，学习速度差七倍。这就是“增益决定改多快”的定量含义。

第二个是**最小 RMS 几乎不随增益变**：从 0.02 到 0.50，最小值都在 0.062 附近，波动不到 1.5%。这说明增益只改变“下潜的速度”，不改变“能潜多深”。地板的高度是另一个因素定的——1.25 Hz 那个越界分量。增益越大，低频降得越快，也就越早碰到“低频残余与高频增长相当”的交叉点，于是反弹来得越早、越猛。$\gamma_p = 0.15$ 的第 30 次已经涨到 0.1196，比不学习还差；$\gamma_p = 0.50$ 涨到 0.3371，时域上已经能看到明显的等幅振荡。图 69-5 把这层关系画了出来：实心点是 30 次内的最小值，空心点是末次值，两者之间的差距就是“反弹”的幅度。

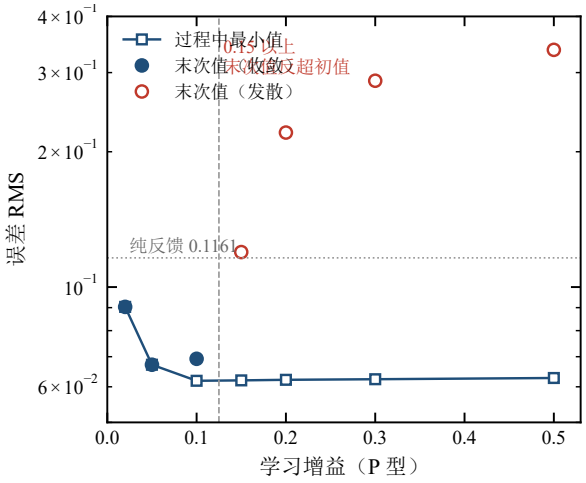


图 69-5 学习增益扫描：最小值与末次值

工程上怎么选这个增益？本章的做法是“先取使低频几何比落在 0.95 附近的增益”，也就是 $\gamma_p = 0.10$：16 遍减半，60 遍能把低频压到 5% 以下，同时在高频上的超标只有 5%，还有 Q 滤波可以兜底。若取 0.15，虽然 11 遍就减半，但不加 Q 时 30 次就发散，加了 Q 之后余量也被吃掉大半。**增益不是越大越好，而是“刚好让低频跑得动、又不把高频的雷踩响”**。

还有一条经验值得记下来：增益的量纲是“每单位误差、每遍迭代”的控制量增量。误差 0.1 对应记忆加 0.01，而精确前馈的峰值才 0.236，所以 γ_p 的量级基本被“前馈量级 / 误差量级”锁死在 0.1 附近，不会出现数量级的偏差。若对象增益更大、前馈更小，γ_p 可以相应减小；反之亦然。这个估算不需要仿真，看一眼 u^* 与初始误差的比值就够了。

微分增益：只补相位，不补幅值

γ_d 的作用与 γ_p 不同，它不改变低频的收敛速度（因为 $\omega\gamma_d$ 在低频趋于零），只改变更新量的相位。所以调 γ_d 的方法也不是“看降得多快”，而是“看误差峰值的形状有没有变好”。

本章从 $\gamma_d = 0$ 调到 0.005，30 次的末次最大偏差从 0.1538 降到 0.1167，降幅 24%；末次 RMS 从 0.0693 降到 0.0506，降幅 27%。再往上加会怎样？0.02 时 D 项在 0.75 Hz 的幅值已经与 P 项相当，更新量被“转”了近 45°，低频的纯滞后分量反而学不动，同时噪声开始显现。所以 γ_d 的合理范围很窄，通常是 γ_p 的 3% 到 10%；本章取 5%，落在中间。

有一个容易犯的错误是把 γ_d 当成“第二个速度旋钮”，用它来加速收敛。它不加速。表 69-3 里 D 型单独使用只降 8.2%，就是最直接的反证。γ_d 的正当用途只有一个：**当误差在时间上错位、而幅值并不缺时，用它把更新方向转过去**。判断是否需要它，看第一次迭代的误差与参考速度的互相关：同相说明是幅值问题（用 P），正交说明是相位问题（用 D），本章两者都有，所以用 PD。

Q 滤波窗长：用带宽换单调

Q 滤波是三个旋钮里最“工程”的一个：它不改变学习律的形式，只是把更新量在时间上做一次对称滑动平均。对称窗的好处是零相位——不引入额外滞后，只衰减高频。它的频率响应是一个 Dirichlet 核：

$$Q(\mathrm{e}^{\mathrm{j}\omega T_s}) = \frac{1}{m}\sum_{i=0}^{m-1} \mathrm{e}^{-\mathrm{j}\omega T_s\left(i-\frac{m-1}{2}\right)} \tag{69-30}$$

窗长 m 越大，通带越窄，等效带宽约

$$f_c \approx \frac{0.443}{m T_s} \tag{69-31}$$

表 69-8 是窗长从 1（不加滤波）扫到 121 的实测结果，学习律固定 PD $(0.10,\ 0.005)$，迭代 60 次。“单调”一列指 60 次 RMS 是否逐次下降，“到半程迭代”指误差降到初始值一半所需的次数。

窗长 m	等效带宽 / Hz	末次 RMS	最小 RMS	单调	到半程迭代
1	无穷	0.1424	0.0497	否	17
11	4.03	0.1313	0.0491	否	17
21	2.11	0.1069	0.0473	否	16
41	1.08	0.0568	0.0409	否	16
61	0.73	0.0317	0.0311	否	16
81	0.55	0.0224	0.0224	是	17
121	0.37	0.0361	0.0361	是	22

表的左半段说明“必须加 Q”：$m = 1$ 时末次 0.1424，比初始的 0.1161 还大，最小值 0.0497 只在中途昙花一现。$m = 11$ 和 21 只是把发散推迟，末次仍在 0.13 与 0.11，单调性都没有恢复。$m = 41$ 的等效带宽 1.08 Hz 刚好把 1.25 Hz 放在过渡带边缘，末次降到 0.0568，但仍不单调。

右半段说明“Q 不能太窄”。$m = 81$ 是转折点：末次 0.0224，与最小值相同，60 次全程单调，半程迭代 17 次。$m = 121$ 虽然也单调，末次却回升到 0.0361，半程迭代拖到 22 次——因为等效带宽只剩 0.37 Hz，连基波

0.25 Hz 都开始被衰减，三次谐波 0.75 Hz 更是几乎被完全挡住，学得动的只剩一根谱线。图 69-8 把这条“U 形”曲线画了出来，右轴是等效带宽。

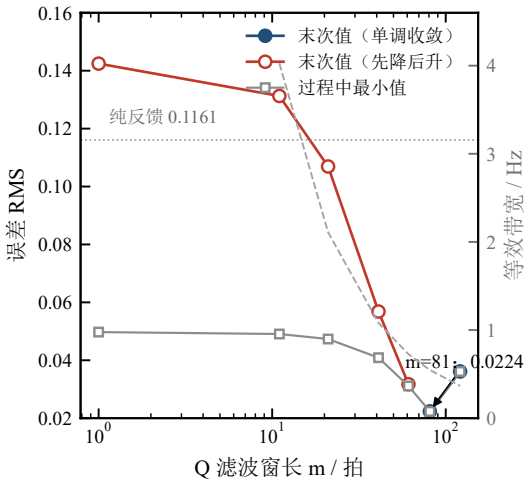


图 69-8 Q 滤波窗长与收敛效果

窗长的选择可以归结为一句：**让第一根越界频率落在阻带里，同时让轨迹的最高谐波留在通带内**。表 69-9 是几个候选窗长在关键频点上的幅值响应，用它可以快速判断“够不够”。

窗长 m	$\lvert Q(0.25)\rvert$	$\lvert Q(0.75)\rvert$	$\lvert Q(1.00)\rvert$	$\lvert Q(1.25)\rvert$	$\lvert Q(2.00)\rvert$
21	0.995	0.960	0.929	0.891	0.735
41	0.983	0.852	0.746	0.621	0.208
61	0.962	0.690	0.491	0.283	0.166
81	0.934	0.494	0.221	0.012	0.183
121	0.856	0.101	0.161	0.210	0.127

读表要注意一个反直觉的现象：**窗长的响应不是单调衰减的**。$m = 21$ 时 1.25 Hz 还有 0.891，几乎没滤掉；$m = 81$ 时掉到 0.012，衰减了近 40 dB；可 $m = 121$ 又回到 0.210。原因是 Dirichlet 核有旁瓣，窗长变化时各个频点落在主瓣还是旁瓣上是跳变的，不是平滑移动。这意味着“加大窗长”不一定“更安全”：$m = 121$ 在 1.25 Hz 的残余是 81 的 17 倍，虽然仍在安全侧，但余量小了。选窗长必须按表查，不能凭“越长越稳”的直觉。

另一件事是 $m = 81$ 的代价：0.75 Hz 的幅值只剩 0.494，三次谐波只能学一半。60 次之后误差停在 0.0224，其中 0.75 Hz 的残余 0.0189 与 1.00 Hz 的 0.0191 各占总功率的三分之一左右：前者是“允许学但学不快”，每步只乘 0.979；后者是“几乎不让学”，$|Q| = 0.221$ 让有效增益只剩 0.022，每步反而乘 1.003。若把窗长缩到 61，0.75 Hz 的 0.690 会让它降得更快，但 1.25 Hz 的 0.283 又会让误差重新爬升，末次 0.0317、不单调。**0.0224 这个地板，是“三次谐波学得慢”与“越界频段不能学”之间的折中价。**

三个旋钮的灵敏度不一样

三个旋钮在手感上差别很大，原因可以量化。把每个旋钮单独动一动，看末次 RMS 与最小 RMS 各变多少。

γ_p 是“饱和 + 断崖”型的。 从 0.10 加到 0.15，最小 RMS 从 0.0619 到 0.0620，几乎不动；末次 RMS 却从 0.0693 跳到 0.1196，涨了 73%。前一个数不动，是因为最小值由“学不掉的成分”托住，跟增益关系不大；后一个数

暴涨，是因为越过边界后 1.25 Hz 开始按 1.05 的因子累积。所以判断“增益够不够”只能看末次与单调性，看最小值会被骗。

m 是“线性放大”型的。 41 拍到 81 拍，末次从 0.0568 降到 0.0224（降 61%）；81 拍到 121 拍，从 0.0224 升到 0.0361（升 61%）。窗长动 30%，末次就动 60%，看起来最灵敏。但它的响应带旁瓣、非单调，不能凭“越长越稳”外推，必须按表 69-9 查。

γ_d 最温和。 从 0 加到 0.005，末次从 0.0693 到 0.0506（降 27%），最大偏差从 0.1538 到 0.1167（降 24%）。继续加大到 0.02 以上，收益不再增加，反而开始把噪声学进记忆。它的作用范围窄，但方向明确。

排序是 $\gamma_p > m > \gamma_d$，可是这个排序不能直接搬成“调参顺序”——灵敏度高的先调，是为了尽快进入可行域；γ_p 的正确做法是“粗定”，只要落进不越界的平台区就行，本章 0.08 与 0.10 的差别远小于 0.10 与 0.15 的差别，没必要精确到小数点后两位。m 必须按表选，因为它不可外推。γ_d 放最后，代价最小，但它影响的是最大偏差——而最大偏差往往才是工艺卡的那条线。

三个旋钮的配合

把三个旋钮的作用放在一起看，可以得到一条清晰的调参逻辑。学习律决定更新方向，先用互相关判断误差是“幅值型”还是“相位型”，本章是混合型，所以取 PD。增益决定速度，用几何比 0.95 左右作为起点，兼顾 60 次内的收敛与高频余量。Q 决定资格，按表 69-9 找到“第一根越界频率衰减 20 dB 以上、最高谐波保留一半以上”的窗长，本章是 81 拍。

这三个数不是独立最优的：把 γ_p 加大，就得把窗长加长来压高频；把窗长加长，低频也变慢，于是又想加 γ_p。所以实际调参是一个两维搜索，Q 的窗长与 γ_p 要一起看。本章的顺序是先定 $\gamma_p = 0.10$（几何比 0.957），再扫窗长找到单调且末次最小的 81 拍，最后微调 γ_d 把最大偏差压下去。69.8 节把这套顺序整理成可执行的步骤。

69.8 调参方法与经验

一条可以照着走的流水线

调 ILC 不需要反复试凑，按下面的顺序走一遍，多数情况两三轮就能定下来。每一步都有明确的输入、输出和验收口径，任何一步不过关就不要往下走，否则后面调出来的数没有意义。

第 0 步：算清前馈量级。 用式 (69-21) 算一遍 u^*，看它的范围和 RMS。本章是 $[-0.154, 0.236]$、RMS 0.117。这个数决定了学习增益的量级，也决定了限幅余量够不够。若 u^* 已经接近限幅，先改轨迹或放宽限幅，不要急着调学习律。

第 1 步：把反馈单独调好。 令 $\gamma_p = \gamma_d = 0$，只跑纯反馈，确认跟踪误差是“有界、平滑、可重复”的。本章纯反馈的 RMS 是 0.1161、最大偏差 0.1974。如果纯反馈本身就在抖或饱和，ILC 只会把这个抖动学进记忆，越学越糟。

第 2 步：测 T 与判据。 用线性化模型算 $T(\mathrm{e}^{\mathrm{j}\omega T_s})$，找出 $|1 - \gamma_p T|$ 的峰值频率与越界频段。这一步只用几分钟，却能省掉后面几小时的试错。本章的结论是峰值 1.25 Hz、越界从 1.00 Hz 开始，后面所有参数都是围着这两个数选的。

第 3 步：定学习律与增益。 先看第一次迭代的误差形状：是“整段滞后”就用 P，是“时间错位”就用 D，两者都有用 PD。增益从“低频几何比 0.95 到 0.96”起步，本章对应 $\gamma_p = 0.10$、几何比 0.957。

第 4 步：扫 Q 窗长。 以“第一根越界频率衰减 20 dB 以上”为下限，以“最高谐波保留一半以上”为上限，在中间扫。本章 41 拍到 121 拍都试过，81 拍是唯一同时满足“单调”与“末次最小”的。

第 5 步：微调 γ_d。 只在上一步的窗长确定之后做，因为 D 项的相位作用会被 Q 改变。本章从 0 加到 0.005，最大偏差降 24%。

第 6 步：验收与留档。 至少看四个指标：末次 RMS、末次最大偏差、60 次是否单调、饱和比例是否为零。四个都过才算完成。图 69-5 那种“最小值—末次值”的对照图要一起留档，它比单条曲线更能暴露反弹。

增益与窗长是耦合的

流水线把“定增益”与“扫窗长”分成两步，实际调起来这两步互相牵扯，不能真的各调一次就完事。耦合来自式 (69-12)：低频段真正起作用的不是 γ_p，而是 $\gamma_p|Q(\omega_1)|$。本章 $\omega_1 = 0.25$ Hz，81 拍窗长在该点的幅值是 0.934，等效增益只剩 0.0934；窗长放到 121 拍，幅值降到 0.856，等效增益只剩 0.0856。也就是说，只加长窗长这一件事，就让低频慢了 8% 到 14%。

这个数可以解释表 69-8 里一个乍看奇怪的现象：121 拍的末次误差 0.0361 比 81 拍的 0.0224 差 61%，而 121 拍的阻带明明更宽。原因有三层。一是低频有效增益被削掉 14%，收敛变慢。二是三次谐波被压得更狠：0.75 Hz 的幅值从 81 拍的 0.494 掉到 0.101，原本“还能学一半”变成“基本学不动”，而 0.75 Hz 与 1.00 Hz 正是末次误差里最大的两项。三是第一旁瓣把 1.25 Hz 的残余从 0.012 抬到 0.210，十七倍，越界频段重新有了积累的余地。前两层是“学得慢”，第三层是“学错了”，叠在一起就是那 61%。

所以正确的做法是两维一起搜，而不是串行微调。建议取 3×4 的格点：$\gamma_p \in \{0.05,\, 0.10,\, 0.15\}$，$m \in \{21,\, 41,\, 81,\, 121\}$，共 12 组，每组 20 次迭代。判据三条：20 次内是否单调、第 20 次的 RMS、饱和比例是否为零。把 12 组画成方格图（颜色是第 20 次 RMS），最优区域通常是一条斜带——增益大就要窗长长，反之亦然，因为两者都在同一个“相位预算”里支取。本章的最优格点是 $(0.10,\, 81)$，它不在任何一个端点上，这本身就说明两维必须一起看。

代价也顺便算一下：仿真里 12 组 × 20 次 × 2.82 ms 不到 1 s，随便扫；实机上 12 组 × 20 次 × 4 s 是 16 分钟，所以实机只做粗扫，先定位斜带，再在附近补两组做细调。这也是 69.10 节“学习期要做进生产节拍”的一笔具体账。

用几何比预估迭代次数

几何比不只是一个诊断指标，它还能直接预估“要跑多少遍”。若每遍误差乘以 λ，则 K 遍之后的相对残余是

$$\frac{\varepsilon_K}{\varepsilon_0} \approx \lambda^K \quad \Longrightarrow \quad K \approx \frac{\ln(\varepsilon_K/\varepsilon_0)}{\ln \lambda} \tag{69-32}$$

本章 $\lambda = 0.957$，按式 (69-32) 算，降到 20% 需要 37 遍，降到 10% 需要 53 遍。表 69-10 把预测与实测摆在一起，前三行吻合得相当好，后两行开始偏离。

迭代次数	实测 RMS	实测相对值	几何模型预测	偏差
1	0.1161	1.000	1.000	0
6	0.0934	0.804	0.803	+0.1%
11	0.0758	0.653	0.644	+1.4%
21	0.0517	0.445	0.415	+7.2%
41	0.0295	0.254	0.172	+48%
60	0.0224	0.193	0.071	+172%

前三行的吻合说明几何模型在“地板”出现之前是准的：第 6 次预测 0.803、实测 0.804，第 11 次预测 0.644、实测 0.653，误差都在 2% 以内。第 21 次开始偏离，第 41 次偏差 48%，第 60 次偏差 172%。偏离的原因不是模型错了，而是模型里只有一个 λ：它描述的是“最快的那根谱线”，而实际误差还包含降不动的成分（0.75 Hz 与 1.00 Hz）。当快成分降到与慢成分相当时，总误差就不再按 λ 下降，而是被慢成分托住。

这条规律有实用价值：**用式 (69-32) 预估的是“最快能到多少”，不是“最终能到多少”**。若按 37 遍规划学习期，实际到第 37 遍只会到 0.03 左右，而不是 20% 的 0.023；想再往下，必须动 Q 或改轨迹，而不是继续迭代。工程上把这个“托底”位置称为地板，规划学习期时应当按地板来定，不要按几何比外推。

0.15 与 0.10 的差别要到第几遍才看得出来

表 69-7 里有一对很容易读错的数：$\gamma_p = 0.10$ 的最小 RMS 是 0.0619，$\gamma_p = 0.15$ 是 0.0620，几乎一样。若只看前十几遍，0.15 甚至更“好看”，因为它降得更快。可到第 30 遍，两者是 0.0693 与 0.1196——后者已经超过首次的 0.1161，也就是说，**学 30 遍比一遍不学还差**。

几何比也会把人带偏。0.15 的几何比是 0.937，比 0.10 的 0.957 更“好”，而几何比是从前 6 次相邻比值的中位数算的，测的是“最快的那根谱线”，也就是 0.25 Hz。0.25 Hz 在 0.15 下确实收得更快，可 1.25 Hz 在 0.15 下的因子是 1.079，比 0.10 的 1.0525 更糟。两条谱线一快一坏，“平均”起来仍然是下降，于是前十几遍看不出问题。

这个 1.079 可以精确地算出翻车时间：它的倍增时间是 $\ln 2 / \ln 1.079 \approx 9$ 遍，30 遍后是 $1.079^{30} \approx 9.8$ 倍，把初始的 0.0105 抬到 0.10 量级——正好与末次的 0.1196 相称。所以不是“运气不好”，而是从第一遍起就已经注定了，只是要等它长出来。

三条可操作的补救。第一，学习期至少跑到“越界频段倍增时间”的 3 倍以上，本章是 27 遍，取 30 遍刚好能看见；只跑 10 遍就验收，等于没验。第二，直接算判据，别用几何比代替——几何比是“速度指标”，判据是“稳定性指标”，两者不能互相换算，本章这对参数就是活生生的例子：几何比只差 0.02，末次却差 72%。第三，把 1.25 Hz 的单频幅值列进监控（69.10 节第三步的做法），它是所有指标里最早报警的一个。

学习期的预算

规划学习期时不能只报“60 次、4 分钟”，要把“每一段花掉的遍数买到多少精度”列清楚。按本章实测，把累计改善按 $0.1161 - 0.0224 = 0.0937$ 归一：

- 第 1 次，累计 4 s，RMS 0.1161，0%，这就是纯反馈的基线；
- 第 6 次，累计 24 s，0.0934，拿到 24%；
- 第 11 次，累计 44 s，0.0758，43%；
- 第 21 次，累计 84 s，0.0517，69%；
- 第 41 次，累计 164 s，0.0295，92%；
- 第 60 次，累计 240 s，0.0224，100%。

这张单子说的其实是同一件事：**前 41 次拿走 92%，后 19 次（76 s）只再买 8%**。若产线节拍只允许 40 次上下，损失的是最后那 0.007 的 RMS，而不是整个方案的价值；反过来，若有人承诺“再跑 200 次能到 0.005”，那是不成立的——地板在 0.0224 附近，再跑也只会在 0.021 到 0.022 之间浮动（表 69-10 最后两行的偏差就是这么来的）。

预算还有一半在“遍数”之外。60 次迭代意味着 60 件的时间，每件 4 s 就是 4 分钟机时；更要紧的是这 4 分钟里产品是按“越来越准”的轨迹做出来的，前几件的误差在 0.1 量级，多数工艺会直接判废。所以真实成本是“四十来件

废品 + 若干分钟机时”，而不是“若干分钟机时”。69.10 节的首件学习、批量复用就是为了把这个成本摊到一整批上。

给一个可操作的定法：把“到 90% 改善”作为学习期的终点（本章实测第 41 次到 92%，所以取 40 次左右），再用“连续 5 次不再下降”作为兜底中止。两条一起用，既不会学不够，也不会在已经到地板时白跑二十分钟。

症状与处置

调参过程中遇到的异常，多数可以归到六种症状里。表 69-11 把它们、可能的原因和处置办法列在一起，顺序按出现频率排。

症状	可能原因	处置
误差完全不降	学习增益为零或符号反了；复位没做对	查 γ_p 符号与每遍初值；先跑纯反馈确认基线
先降后升	越界频段在积累，通常是 Q 太短	加长窗长，或降低 γ_p
逐次抖动、无规律	测量噪声被 D 项放大；复位瞬态被学进记忆	加长微分低通；学习窗两端加保护段
平均在降、峰值不降	学习律方向不对，缺相位超前	加 γ_d，或改用 PD
控制量频繁削顶	前馈量级估计偏低；参考轨迹过冲	先算 u^{*}；放宽限幅或降低轨迹幅值
单次耗时超预算	学习律实现用了在线优化	改成逐点更新；Q 用卷积而非逐点求和

表的第三行值得单独说。本章的 D 项后面接了 30 ms 低通，就是为了压噪声；如果没有这个低通，$\gamma_d = 0.005$ 在 1 kHz 上的放大倍数是 31 倍，测量噪声的 1 mV 会变成 31 mV 的学习增量，几十遍之后记忆里全是噪声。第四行则是“峰值不降”的典型：P 型 30 次的末次最大偏差是 0.1538，PD 型降到 0.1167，差别就来自那十几度的相位超前。

三个不要

第一条，**不要只看末次 RMS**。表 69-7 里 $\gamma_p = 0.15$ 的最小值是 0.0620，与 $\gamma_p = 0.10$ 的 0.0619 几乎一样，可末次是 0.1196，已经发散。只报末次会漏掉“中途很好、后来变坏”的情形，只报最小值会漏掉“根本没在学”的情形。两个一起看，再配一条单调性判断，才不会被数字骗。

第二条，**不要先加 Q 再调增益**。Q 会把增益的效果一起滤掉：同样把 γ_p 从 0.05 加到 0.10，不加 Q 时半程迭代从 31 次缩到 16 次，加了 81 拍 Q 之后只在低频段缩短，高频段完全没反应。先调增益、后加 Q，才能看清每个旋钮各自的作用。

第三条，**不要把 γ_d 当油门**。它的作用是转相位，不是加速度。把它加大到“看起来降得更快”的程度，通常意味着已经开始学噪声了，短时间好看，几十遍之后会现形。

学习期与中止条件

正式算例 60 次迭代、每次 4 s，学习期 4 分钟。这个时间在实验台上可以接受，在产线上要算进节拍。规划学习期时看两件事：几何比给出的“最快到达时间”与地板给出的“最终精度”。若节拍只允许 20 次，就应该把 γ_p 调大到几何比 0.93 左右，接受“地板高一点、但 20 次内到位”；若精度优先、节拍宽松，就取 0.96 左右的保守增益，让曲线尽量平滑。

中止条件建议设三条，任意一条触发就停：连续 5 次 RMS 不再下降、单次最大偏差超过上一遍的 1.2 倍、控制量饱和比例超过 1%。前两条防“悄悄发散”，第三条防“记忆被限幅截断后学错方向”。本章的正式算例 60 次里三条都没有触发，这也是它敢被称为“单调收敛”的原因。

69.9 优化与改进

非重复成分：ILC 的能力边界

前面八节的讨论都建立在一个假设上：每一遍的任务完全一样。真实系统里这个假设只能近似成立。把第 k 遍的误差拆成两部分：

$$e_k(t) = \bar{e}(t) + \tilde{e}_k(t), \qquad \bar{e} = \lim_{K\to\infty} \frac{1}{K}\sum_{k=1}^{K} e_k(t) \tag{69-33}$$

$\bar{e}$ 是各遍共有的、可重复的部分，$\tilde{e}_k$ 是每遍不同、均值为零的部分。ILC 的学习律对 e_k 做的是逐点累加，累加的结果只保留“每遍都在同一个位置出现”的成分，也就是 $\bar{e}$；$\tilde{e}_k$ 在累加中相互抵消。所以 **ILC 只能消掉可重复的那一半**，非重复的那一半不但消不掉，还会因为学习律本身的增益被放大一点。

这个拆分解释了三件事。第一，为什么 ILC 能把误差压到反馈达不到的水平：反馈每一遍重新开始，只能压瞬时误差；ILC 压的是 $\bar{e}$，而 $\bar{e}$ 在几十遍的尺度上稳定存在。第二，为什么末次误差有一个地板：地板的高度由 $\tilde{e}$ 的 RMS 与 Q 滤波的残余共同决定，继续迭代不会让它下降。第三，为什么“加长学习期”有时没用：如果 $\tilde{e}$ 已经占了误差的一半，把 $\bar{e}$ 压到零也只能把总误差降到 $\tilde{e}$ 的水平。

六种非重复因素的实测

表 69-12 是六种工况的末次结果，学习律、增益、Q 窗长、迭代次数全部与正式算例一致，只改一个非重复因素。“相对理想”一列是相对第一行（理想重复）的百分比变化。

工况	末次 RMS	末次最大偏差	相对理想 RMS	相对理想最大偏差
理想重复	0.0224	0.0495	—	—
测量噪声 0.01	0.0240	0.0659	$+7.1\%$	$+33.1\%$
复位误差 0.02	0.0218	0.0472	-2.7%	-4.6%
增益失配 $+20\%$	0.0161	0.0334	-28.1%	-32.5%
增益失配 -20%	0.0351	0.0745	$+56.7\%$	$+50.5\%$
非重复外扰 0.05	0.0289	0.0674	$+29.0\%$	$+36.2\%$

六行数据放在一起看，最醒目的结论是**没有一种工况把 ILC 打回纯反馈的水平**。最差的是增益失配 -20%，末次 0.0351，仍比纯反馈的 0.1161 好 3.3 倍；最大偏差 0.0745，也比纯反馈的 0.1974 好 2.6 倍。这说明 81 拍 Q 滤波提供的余量足以覆盖这些常见的非重复因素，不需要为每一种都单独设计学习律。

逐行看，第一行的噪声工况最能说明“非重复成分不会被累积”。测量噪声 0.01 直接加在输出上，单遍误差里就多出 0.01 量级的随机成分，再经反馈在控制量上放大到约 0.02。如果 ILC 把这部分也当学习目标，记忆会做随机游走，几十遍后偏离正确值。实测末次 0.0240，只比理想高 7%，说明 Q 滤波把它平均掉了大半。做一个粗估：每遍学习增量里的噪声 RMS 是 $\gamma_p \times 0.01 = 0.001$，81 拍平均后降到约 1.1×10^{-4}，60 遍随机游走累积约 $\sqrt{60}\times 1.1\times10^{-4} = 8.5\times10^{-4}$，落在 0.207 的记忆量级上只占 0.4%，可以忽略。再用平方相减核对：$\sqrt{0.0240^2 - 0.0224^2} = 0.0086$，与 0.01 的噪声幅值吻合。两条合起来说明，这 7% 的涨幅几乎全部来自每一遍直接叠加在误差上的噪声，而不是记忆漂移。

第二行的复位误差 0.02 是个有意思的结果：末次 0.0218，比理想还低 2.7%。复位误差是每遍随机的初始状态偏移，属于典型的非重复成分，它本该抬高地板而不是降低。实测偏低的原因是随机的初值偏移只影响开头零点几

秒，随后被对象本身的阻尼（$y'' = -2y' + Ku$ 里的 $-2y'$）衰减掉，对 400 点 RMS 的贡献本来就很小；2.7% 的差异落在这个量级内，应当视为“与理想持平”，而不是“复位误差有益”。工程上仍要把复位做准，因为一旦对象阻尼更小，这个偏移会持续更久，直接进入学习窗。

第三、四行的增益失配呈现出明显的不对称：$+20\%$ 时误差降到 0.0161，-20% 时升到 0.0351，两者相差一倍多。原因有两层。第一层是“对象更有力”：增益偏高时，同样的记忆增量产生更大的输出修正，学习更快；增益偏低时相反，同样的记忆只够推动一半，收敛慢。第二层是“前馈量级”：精确前馈 u^* 是按名义增益算的，增益偏高时实际需要的控制量更小，学到的记忆天然偏保守，不容易撞上限幅；增益偏低时需要的控制量更大，记忆更靠近边界。所以 $+20\%$ 的结果更好，但这不是“增益越高越好”的证据——把增益再往上拉，判据的余量会被吃掉，69.6 节算出的越界频段会扩大，那时结果会迅速反转。

第五行的非重复外扰 0.05 是幅值 ±0.05、段长随机的方波，直接加在对象输入上。它每遍不同，ILC 学不到，只能靠反馈抑制。实测末次 0.0289，比理想高 29%。用平方相减拆出外扰的贡献：$\sqrt{0.0289^2 - 0.0224^2} = 0.0183$，也就是 0.05 的方波经反馈衰减到 0.018 左右，衰减倍数约 2.7，与反馈在 1 Hz 以下的增益量级相符。最大偏差 0.0674 的涨幅更大，因为方波的跳变沿会在误差上留下尖峰，而尖峰正是 RMS 不敏感、最大值敏感的东西。这也提醒一件事：**看鲁棒性不能只看 RMS，最大偏差对非重复因素更敏感**。

图 69-9 把六种工况画成两组横条，左边是末次 RMS，右边是末次最大偏差，两条竖线分别是纯反馈的两个指标。六组横条全部落在竖线左侧，视觉上就是“ILC 在所有工况下都赢过纯反馈”。

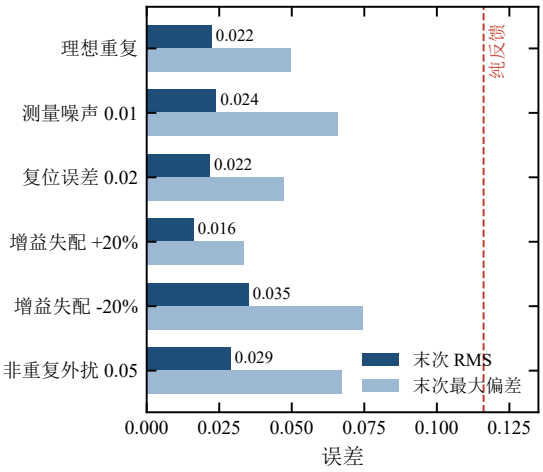


图 69-9 六种非重复因素下的末次误差

改进一：遗忘因子

如果任务是慢时变的（负载缓慢变化、夹具逐渐磨损、温漂），纯粹的累加型学习律会把“旧的正确”当成“永久的正确”，记忆更新不上去。最直接的改进是给记忆加一个遗忘因子：

$$u_{k+1}(t) = \alpha\, Q\big[u_k(t) + \gamma_p e_k(t) + \gamma_d \dot{e}_k(t)\big], \qquad 0 < \alpha \le 1 \tag{69-34}$$

$\alpha = 1$ 就是本章的正式算例；$\alpha < 1$ 时，旧的记忆每遍按比例缩水，新的误差有更大权重。它在频域上的稳态残余可以直接算出来：把 $e_k = r - Tu_k$ 代入，整理得

$$\frac{e_\infty}{r} = \frac{1-\alpha}{1-\alpha+\alpha\gamma_p T} \tag{69-35}$$

式 (69-35) 说明遗忘因子是有代价的：α 越小，分子越大，稳态残余越高。以 0.25 Hz 为例，$T = 0.5256$、$|Q| = 0.934$、有效增益 0.0934，取 $\alpha = 0.999$ 时残余是 2.0%，$\alpha = 0.995$ 时 9.3%，$\alpha = 0.98$ 时 29.3%。也就

是说，**遗忘因子每放松一点，地板就抬高一大截**：它能跟上的时变速度，与它留下的稳态误差是同一枚硬币的两面。

工程上的取值通常在 0.995 到 0.999 之间。判据是“时变的时间常数”：若负载在 100 个学习周期内变化 10%，取 $\alpha = 0.99$ 左右就够；若负载几个月才变一次，取 0.999 甚至 1。**不要为了“看起来更鲁棒”而随手取 0.95**，那个值会把地板抬到与纯反馈同一量级，等于白做 ILC。本章的算例没有时变，所以取 $\alpha = 1$。

改进二：高阶学习律

第二个改进是让学习律同时看两遍的误差：

$$u_{k+1} = u_k + \gamma_p \left(e_k + \beta e_{k-1} \right), \qquad 0 < \beta < 1 \tag{69-36}$$

它在迭代域上相当于给学习环加了一个零点。当 β 取 0.3 到 0.5 时，迭代方向的振荡模态会被压下去，收敛更平滑；代价是对非重复成分更敏感，因为 e_{k-1} 里的噪声也被带了进来。高阶学习律的收益在“迭代方向有振荡”时最明显，而本章的正式算例加了 Q 之后已经单调，再加 β 没有收益。所以本章没有采用，只在 69.9 这里作为一种可选手段列出。

判断要不要用它，看 RMS 随迭代的曲线有没有“锯齿”：如果相邻两遍一高一低交替，说明迭代方向存在接近 -1 的模态，这时 β 有用；如果曲线平滑，β 只会增加噪声。这和时域里“微分器只在该用的时候用”是同一条道理。

改进三：范数最优与时变增益

第三个方向不再用标量增益，而是每一拍给一个增益，甚至用一个矩阵把整条轨迹的误差映射成整条轨迹的更新量：

$$u_{k+1} = u_k + L\, e_k, \qquad L = \gamma_p\, T^{\mathsf{T}} \left(T T^{\mathsf{T}} + \lambda I \right)^{-1} \tag{69-37}$$

T 这里是 $N \times N$ 的下三角（或托普利兹）矩阵，描述“第 j 拍的记忆对第 i 拍输出的影响”。L 的作用是把误差“反卷积”回去，相当于一次性做完整条轨迹的最优修正。它的收敛速度比标量 P 型快得多，因为每个频点都拿到了接近最优的增益。

代价也很直接：L 是 400×400 的矩阵，求逆是 $O(N^3)$，只能在离线算一次；在线每遍要做一次矩阵向量乘，$400^2 = 1.6 \times 10^5$ 次乘加，比标量学习律的 400 次多三个数量级。对本章 4 s 的迭代周期，这点算力仍然可以接受（约 1 ms），但对 10 ms 的采样周期就不行了——它是“每遍算一次”，不是“每拍算一次”，所以还能用。真正的障碍是 T 必须准确：T 错了，L 会把误差放大到错误的方向，比标量 P 型更脆弱。本章没有采用，因为标量 P 型加 Q 已经把误差压到 0.0224，范数最优的边际收益不大；若任务对收敛遍数有硬约束（比如只允许 5 遍），它才值得上。

改进四：学习窗与保护段

第四个改进最小，却最常用：不要把整条轨迹都拿来学。具体做法是给学习窗两端各留一段保护段，保护段内的误差不参与更新，记忆保持上一遍的值（或强制为零）：

$$\Delta u_k(t) = 0, \qquad t \in [0,\, t_a) \cup (t_b,\, T] \tag{69-38}$$

保护段的作用是挡掉两类“学不会也不该学”的误差：起点处的复位瞬态，以及终点处的停车瞬态。这两段误差每遍形状不同，属于 69.9 开头说的 $\tilde{e}$，学进去只会污染记忆。本章的正式算例没有加保护段（因为仿真里复位是理

想的），但 69.5 节的热图已经显示，$t=0$ 附近的误差几乎不随迭代变化——这正是保护段该切掉的部分。工程上一般取 t_a 为 0.1 到 0.2 s、t_b 为 T 前 0.1 s，具体看瞬态的持续时间。

保护段的代价是“这两段永远学不好”。若工艺要求起点也必须精确，就不能简单切掉，而应把复位做成可重复的（固定的初始状态、固定的使能时序），让这两段误差从 $\tilde{e}$ 变成 $\bar{e}$，再让 ILC 去学。**先想办法把非重复变成重复，再谈学习**，这个顺序不能反。

四个改进怎么排序

四条改进的优先级不一样，排序的依据是“治的是不是本章的主要矛盾”。本章的主要矛盾是 1.25 Hz 的越界分量，它已经被 Q 滤波解决，所以四条改进在本章都不是必需；它们是为“换一个对象”或“换一个工况”准备的。

遗忘因子最常用，因为它对付的是最普遍的慢时变。 但它有明确的代价公式 (69-35)，取之前要算一遍地板抬到多少。若算出来地板已经接近纯反馈，就不要用，宁可定期重新学习一遍。

保护段最便宜，应当默认开启。 它只需要在更新式里加一个掩码，不改变任何动力学，却能挡掉最常见的两类污染。唯一的前提是知道瞬态持续多久，这可以在第一次迭代的误差曲线上看出来。

高阶学习律只在有迭代振荡时用。 它不是一个“更高级所以更好”的选项，而是一个对症的药。曲线平滑时加它只会引入噪声。

范数最优是“遍数硬约束”时的选择。 它的收益与实现复杂度都高一个量级，适合那些“只允许三五遍就必须到位”的场合，比如每件产品只做一次在线学习的产线。对本章这种可以跑 60 遍的场合，它属于过度设计。

这四条之外还有一类改进是“换学习对象”：不学整条控制量，只学几个参数（比如前馈的幅值与相位），或者只学误差的主分量。这类做法的收敛更快、记忆更小，但表达能力受限，遇到轨迹形状变化就得重来。它们属于另一个技术路线，本章不展开。

69.10 工程落地

第一步：判断任务到底可不可重复

ILC 上线的第一个问题不是参数怎么调，而是“这个任务值不值得用 ILC”。判断办法不是开会讨论，而是测一个数。让系统在纯反馈下连续跑 K 遍（K 取 5 到 10），每遍都按正式流程复位，记录每遍的误差曲线，然后算相邻两遍的差异：

$$\rho = 1 - \frac{\mathrm{RMS}\left(e_{k+1} - e_k\right)}{\sqrt{2}\,\mathrm{RMS}\left(e_k\right)} \tag{69-39}$$

ρ 的含义是“重复性指数”：两遍完全一样时分子为零、$\rho=1$；两遍完全无关时分子约为 $\sqrt{2}$ 倍的误差 RMS、$\rho\approx 0$。它其实就是把 69.9 节的 $\bar{e}$ 与 $\tilde{e}$ 拆开之后，用“可重复部分占总误差的比例”来度量。工程上的经验分档是：$\rho>0.8$ 值得上 ILC，0.5 到 0.8 可以试但收益有限，低于 0.5 就不要做——那时非重复成分占了大半，ILC 学到的记忆大部分是噪声。

算 ρ 时有三个细节会决定结果，必须统一。第一，复位要按正式流程做，不能“大致回零”；如果正式生产时复位精度只有 0.05，测试时就不该用 0.001 的复位，否则 ρ 会虚高。第二，K 遍要连续跑，中间不要换料、不要停机，否则测出来的是“工况差异”而不是“重复性”。第三，误差要按整段算，不要掐头去尾；如果确实要加保护段，测试时也按同样的窗算，保证测的就是将来要学的那一段。

哪些任务通常可重复？机床的重复走刀、机械臂的固定轨迹拾放、点焊的固定焊点序列、注塑的固定工艺曲线、贴片机的固定贴装路径、伺服压装的固定行程。它们的共同点是“起点、路径、终点都由程序锁定，环境在几十秒内基本不变”。哪些通常不可重复？来料尺寸随机、人工参与上料、负载随工件变化、环境温度在任务周期内就明显漂移的场合。这些任务不是绝对不能做 ILC，而是要先做“把非重复变成重复”的改造——固定来料姿态、增加定位工装、把温漂做成可测可补偿的量——改造的收益往往比调学习律大得多。

第二步：把学习期做进生产节拍

ILC 的时间开销不在单次迭代，而在“要迭代多少次”。本章 60 次、每次 4 s，学习期 4 分钟；在实验台上这不算什么，在产线上就是 4 分钟的产能损失。落地的做法通常分两段：**首件学习 + 批量复用**。

首件学习是在换型、换批、维修之后，拿第一件（或前几件）做学习，跑 30 到 60 遍，把收敛的记忆存下来。批量生产时不再学习，每一件都直接用这份记忆做前馈，反馈照常闭合。这样节拍不受影响，精度也保住了。风险是“记忆会过时”：夹具磨损、刀具磨损、温漂都会让最优记忆缓慢变化，而记忆不更新就跟不上。解决办法是加一个“慢速在线学习”——每件仍然更新一点，但增益取得极小（比如正式学习增益的 1/20），把 60 遍的收敛摊到上千件上。这样既不占节拍，又能跟随慢漂移。

首件学习的次数由 69.8 节的式 (69-32) 与地板共同决定。本章的算例若只跑 20 遍，误差到 0.0517；跑 60 遍到 0.0224。若产线只允许 20 遍，就该把 γ_p 从 0.10 提到 0.15 左右，用“地板高一点”换“20 遍内到位”。这个交换必须在设计阶段算清楚，不能等上线后再改——改增益意味着重新验证判据。

第三步：学习过程的在线监控

学习期虽然短，但不能“设好参数就不管”。本章的判据只保证名义收敛，实际系统里总有判据没覆盖的因素（摩擦、间隙、饱和、传感器漂移）。监控的最低要求是四个量，每次迭代后打印或入库：

指标	计算方式	正常范围	超限含义
误差 RMS	$\sqrt{\frac{1}{N}\sum e_k^2}$	逐次下降或持平	连续 5 次不降，学习已到地板或被污染
最大偏差	$\max\lvert e_k\rvert$	不超过上一遍的 1.2 倍	局部尖峰，通常是噪声或复位瞬态
饱和比例	控制量触及限幅的拍数占比	小于 1%	记忆撞限幅，继续学方向会错
记忆变化量	$\max\lvert u_{k+1}-u_k\rvert$	逐次减小	不减小说明在追噪声

四个量里最容易被忽略的是“记忆变化量”。误差 RMS 可能因为反馈的压制而看起来平稳，但记忆仍在每遍大幅改动，这时系统其实处于“边学边忘”的状态，一旦反馈参数变化就会暴露。正常情况下，记忆变化量应当随迭代单调减小：本章第 1 次到第 2 次的最大变化是 0.0145，第 20 次前后降到 0.0062，第 60 次前后是 0.0027。最后这个数不是零，它与 0.75 Hz 残余（0.0189）乘学习增益（0.10）得到的 0.0019 同量级；再把 0.25 Hz 与 1.00 Hz 的残余算进来，就凑成 0.0027。这说明学习已经收敛到地板，剩下的改动都是在追那几根降不动的谱线。若记忆变化量一直停在 0.01 以上而不下降，说明学习律在追非重复成分，应当加长 Q 或减小增益。

除了这四个标量，建议再存一条“误差谱”。本章的经验是：只要 1.25 Hz 分量的幅值在上升，不管总 RMS 在不在降，都说明越界频段在积累。现场不一定有条件做 FFT，可以用一个更简单的替代：把误差与 1.25 Hz 的正弦、余弦做内积，两个数就是该频点的实部与虚部，计算量只有 $2N$ 次乘加。这个“单频监测”在嵌入式上完全可以实时跑。

第四步：中止、回退与双缓冲

监控的目的是在坏事发生之前停下来。中止条件建议设三条，与 69.8 节一致：连续 5 次 RMS 不降、最大偏差超过上一遍 1.2 倍、饱和比例超过 1%。三条中任意一条触发，立即停止学习，但**不要停止生产**——把记忆退回到上一个验收通过的点，用纯反馈继续跑完当前件，然后报警。

这里有一个容易被忽略的设计：学习中的记忆不应当直接生效。做法是双缓冲——维护“当前记忆”与“影子记忆”两份，每遍的学习结果写进影子，只有通过验收（四项指标全过）才整体切换。这样即使某一遍学坏了，生产用的记忆仍然是上一份好的。双缓冲的代价只是多一份 400 个数的存储，在本章是 3.1 KB，可以忽略。

还有三道硬保护，建议无条件加上。第一道是记忆限幅：u_k 的每个点都限制在 $[-1.2, 1.2]$，与执行器限幅一致；本章的仿真里已经有这一层（`np.clip`），实机上要在学习律内部再做一次，不能只依赖执行器。第二道是变化率限幅：单遍的记忆变化不超过满量程的 5%，防止某一遍的异常误差把记忆带跑。第三道是看门狗：学习过程超过预定时间（比如 60 次 × 1.5 倍单次时长）仍未结束，强制退出并回退。

双缓冲与三道保护加起来，ILC 的失效模式就从“越学越坏直到撞限位”变成了“学不动、报警、退回上一版”。这个差别在纸面上只是几行代码，在产线上就是“能不能无人值守”的差别。

第五步：初值与迁移

本章的初值取 $u_0 \equiv 0$，第一次迭代等价于纯反馈，误差 0.1161。这不是唯一选择。如果对象模型已知，可以用精确前馈 u^* 做初值：本章 u^* 在 5 拍延迟下的跟踪误差是 0.0593，直接从 0.0593 起步，比从 0.1161 起步少走约十遍。若模型不准，u^* 只当“形状参考”，幅值乘 0.8 再限幅，也比零初值好。

跨品种迁移要谨慎。若两个品种的轨迹形状相似、只是幅值或时长不同，把旧记忆按比例缩放后当新初值，通常能省一半迭代；若形状不同，迁移过来的记忆可能让第一遍就撞限幅，反而不如零初值。判断相似性的办法是算两条轨迹的相关系数，超过 0.9 才考虑迁移。迁移的记忆同样要限幅、同样要走验收，不能因为“上次是好的”就免检。

第六步：数据与可追溯

一份记忆文件如果只有 400 个数，三个月后没人说得清它是哪个品种、哪个工位、用什么参数学出来的。最低要求是带六项元信息：品种号、工位号、学习日期、学习参数（γ_p、γ_d、m、迭代次数）、验收指标（末次 RMS、最大偏差、是否单调）、以及对象增益的估计值。前五项是追溯用的，第六项是判断“记忆是否过时”用的——若当前测得的增益与记录差超过 20%，就应当重新学习。

学习过程的数据也建议留一份。本章的十张图里，图 69-3 的热图与图 69-6 的谱演化是最有诊断价值的两张，现场可以用简化版替代：每遍存一条误差曲线（400 个 float，1.6 KB），60 遍不到 100 KB。出了问题时，把这条曲线与记忆文件一起回放，基本能定位是“学不动”还是“学坏了”。

六个常见的坑

第一个坑是**复位不彻底**。夹具没回零、编码器有残差、上一件的残余振动没停，这些都会让每一遍的起点不同。ILC 会把这些差异当成“要学的误差”，学出一个折中的记忆，结果是每一遍都差一点。对策是把复位做成可测量的：在开始学习前检查起点偏差，超过阈值就拒绝学习。

第二个坑是**编码器零漂**。增量式编码器长时间运行会有累积偏差，表现为记忆里的控制量缓慢整体上移或下移。对策是每批次用绝对基准校一次零，或者在误差里去掉直流分量（只学交流部分）。

第三个坑是**限幅后的记忆继续累加**。控制量撞了限幅，误差不再反映真实需求，但学习律仍在按误差累加，记忆会越积越大，退出限幅后猛烈过冲。这就是 ILC 版的积分饱和。对策与 PID 抗饱和一样：检测到饱和就冻结该点的学习，或按饱和方向反向修正。

第四个坑是**学习期被中途打断**。操作员提前按了停止、报警清除了学习状态、断电重启，记忆停在半成品。对策是双缓冲加“学习完成”标志，未完成的记忆不生效。

第五个坑是**多工位共享一份记忆**。同一台设备有多个工位，机械差异会让最优记忆不同；共享一份等于把差异平均掉，每个工位都不到位。对策是按工位分别存记忆，或者把工位差异做成可标定的偏置。

第六个坑是**采样不同步**。学习用的误差序列与执行用的记忆序列必须严格对齐到同一拍。若采集有抖动、触发有延迟，学到的记忆会整体错位几拍。本章的 5 拍纯延迟已经让“错位”很敏感：精确前馈错 5 拍，误差从 0.0593 升到 0.0625，看起来不大；但学习环里的错位会被逐遍放大，因为每一遍都在错位的基础上再错一次。对策是用硬件触发对齐，不要用软件时间戳凑。

这六个坑有一个共同点：它们都不在算法里，而在“算法与设备交界”的地方。ILC 的论文通常只讨论学习律与收敛性，工程失败却大多发生在这里。把交界处的细节做扎实，比把学习律从 P 型换成范数最优更能决定项目成败。

69.11 优缺点与适用边界

五个优点

第一，不需要对象的模型。 本章从头到尾没有用对象的微分方程去设计学习律，只在两处用了模型：算 u^* 作为参照、算 T 做收敛判据。前者可以不做，后者可以实测获得（打一拍脉冲，测输出）。真正决定 ILC 能不能用的，是“任务是否重复”，而不是“模型是否准确”。这一点让它在前馈逆模型失效的场合仍然可用——模型不准时，逆模型的前馈会系统性偏错，而 ILC 会用误差把这个偏差学回来。

第二，精度上限由重复性决定，可以超过反馈。 本章的纯反馈误差 RMS 是 0.1161，60 次 ILC 后是 0.0224，降幅 80.8%；最大偏差从 0.1974 降到 0.0495，降幅 74.9%。更重要的是，这个精度不是靠“把反馈调激”换来的——反馈参数全程没动，ILC 只是在前馈上补了一份记忆。若靠加大 k_p 去压误差，相位裕度会被吃掉，超调与噪声都会上来；ILC 走的是另一条路。

第三，算力与存储极低。 单次迭代 2.82 ms，其中九成以上是对象积分（本来就要跑），学习律本身不到 0.1 ms；记忆是 400 个数、3.1 KB。这个开销意味着 ILC 可以加在任何现有控制器上，不需要换硬件、不需要加协处理器。对比之下，范数最优 ILC 的在线矩阵向量乘是 1.6×10^5 次乘加，仍然可接受；模型预测控制每拍都要滚动优化，算力高两三个数量级。

第四，与反馈解耦，可以“叠加”而不是“替换”。 本章的结构是“反馈 PID + 前馈记忆”，记忆失效时反馈照常工作，系统退化成纯反馈，性能下降但不失控。这个特性在工程上极其重要：它让 ILC 成为一个可以“关掉”的模块，而不是一个“必须正确”的模块。第 68 章的混合结构讲的是同一件事——可回退比性能更优先。

第五，可解释、可监控。 记忆 $u_k(t)$ 是一条有物理意义的曲线，可以和精确前馈 u^* 逐点比较。本章实测记忆与 u^* 的相关系数是 0.8543，“记忆 + 反馈”的合计与 u^* 的相关系数是 0.9358，基波幅值差 0.4%、相位差 1.5°。现场工程师看到这条曲线，能判断“学到的到底是不是前馈”；而神经网络的权重做不到这一点。再加上 69.10 节的四个监控量，学习过程是“看得见”的。

五个缺点

第一，必须重复。 这是最硬的限制，也是 ILC 与所有“通用控制器”的分水岭。任务只要有一次不同（来料变了、路径改了、操作员干预了），学到的记忆就不再适用。本章的所有结论都建立在“每遍完全一样”上，一旦这个前提破了，精度会退到 69.9 节表 69-12 的水平。

第二，只能学可重复的那一半。 69.9 节的拆分说得很清楚：$\tilde{e}_k$ 学不掉。若非重复成分占误差的一半，ILC 最多把总误差压到原来的一半，再往下就是地板。这也是为什么 69.10 节把“重复性指数 ρ”放在第一步——它决定了 ILC 的天花板。

第三，学习需要时间。 60 次迭代、每次 4 s，学习期 4 分钟。对实验台无所谓，对产线就是产能。首件学习加批量复用可以缓解，但换型、维修之后仍要重新学。若任务周期本身很长（比如 10 分钟一次），学习期会变成几小时，这时 ILC 的经济性就要打问号。

第四，对延迟与相位很敏感。 本章的 5 拍纯延迟把越界频段的起点压到了 1.00 Hz，峰值在 1.25 Hz，最坏超标 5.25%。这 5% 看起来不大，却逼着我们把 Q 窗长加到 81 拍，代价是三次谐波只能学一半，末次误差停在 0.0224 而不是更低。换句话说，**延迟没有让 ILC 失效，但它限定了 ILC 的性能上限**。延迟越大、越界频段越低，能学的带宽越窄，地板越高。

第五，记忆需要管理。 一个品种一份记忆、一个工位一份记忆、一次维修后要重新验证，时间长了会积累几十上百份文件。没有元信息、没有版本管理，很快就会变成“不知道哪份能用”的状态。这部分工作不产生精度，却是落地必须付出的成本。

与其他方法的分界

ILC 不是万能的，它只在一类问题上比别的方法好。表 69-13 把它与四种常见方法放在一起比较，比较的维度是“依赖什么、作用在什么时间尺度、能不能压非重复扰动”。

方法	依赖模型	依赖任务重复	作用时间尺度	对非重复扰动	典型定位
纯反馈 PID	弱	否	每拍	能抑制	保底，任何场合都要有
前馈逆模型	强	否	离线一次	不能	模型准、任务不重复时的首选
重复控制	中	是（周期信号）	周期内	不能	抑制周期性外扰
迭代学习	不需要（需闭环频响）	是（整条轨迹）	遍与遍之间	不能，也不累积	重复轨迹的高精度跟踪
自适应控制	弱	否	参数收敛的慢尺度	部分	参数慢漂移、工况渐变

这张表里最容易混淆的是“迭代学习”与“重复控制”。两者都利用重复性，区别在“重复的是什么”：重复控制处理的是**周期性的外扰或参考**，它在每个周期内部实时调整，作用时间尺度是“一个周期之内”；ILC 处理的是**整条轨迹的跟踪误差**，它在两个周期之间调整，作用时间尺度是“一遍到下一遍”。本章的 1.25 Hz 分量如果是外部周期扰动引起的，用重复控制更直接；如果是跟踪滞后引起的，用 ILC 更合适。实际系统里两者常常一起用：重复控制压外扰，ILC 压跟踪。

第二组容易混淆的是“迭代学习”与“自适应控制”。自适应律让**控制器参数**沿时间连续变化，参数在任务执行过程中就在动；ILC 的更新发生在任务与任务之间，一次执行之内记忆是一条固定的曲线。所以自适应能跟随时变，ILC 不能（除非加遗忘因子）；ILC 能把整条轨迹的误差压到很低，自适应通常做不到，因为它的参数维度远低于轨迹维度。69.4 节已经点过这一层，这里再用一句话收束：**自适应改的是“控制器的性格”，ILC 补的是“这一条轨迹的答案”**。

与逐遍控制的分界

注塑、轧制、半导体这几个行业里还有一种常被与 ILC 混为一谈的做法：逐遍控制（run-to-run，R2R）。它的形式是每批结束后离线测一个质量指标（膜厚、尺寸、重量），据此修正下一批的配方参数（温度、压力、保压时间），修正律常写成 $\theta_{k+1} = \theta_k + \alpha\,(q_{\text{target}} - q_k)$，$\alpha$ 一般取 0.3 到 0.7。

两者的共同点很明确：都在批次或遍次之间更新，都靠重复性吃饭，都不试图在单次执行内改变什么。差别在三个维度上。**维度**：R2R 修正几个配方参数，ILC 修正整条 $N = 400$ 的输入曲线，前者是 3 维，后者是 400 维。**信息**：R2R 的反馈往往是批次末的几个离线检测点，ILC 需要整条轨迹的在线误差。**增益的量级**：R2R 的 α 通常很小，0.3 已算激进，因为参数到指标的映射常常非方、强耦合；ILC 的 γ_p 受闭环频响约束，可以取到 0.1 而仍然稳定。

一句话概括：**R2R 调配方，ILC 调曲线**。判断该用哪个，看执行手段：若工艺能动的只有温度、压力这几个设定值，ILC 就没有发挥空间，硬上只会得到一个“用 400 维记忆去驱动 3 维执行器”的怪结构；若执行器本身可以逐点给指令——伺服轴、电流环、压电驱动、点胶阀——ILC 才有意义。本章的对象属于后者：控制量 u 是可以逐拍任意给的连续量，这才是 400 维记忆能落地的前提。

两者也能叠起来用：外层用 R2R 修正慢漂移的工艺参数，内层用 ILC 修正每件的轨迹误差。这时要注意时间尺度必须分开——R2R 的更新周期应当是几十件，而不是每件，否则两层会同时去追同一个误差，出现“你压下去我又顶上来”的拍频。

需求不高的场合，别上 ILC

选型时先看三个数：纯反馈 0.1161、精确前馈（模型已知、按 5 拍对齐）0.0593、ILC 0.0224。中间那一档很关键——它只需要一条离线算好的表，不需要学习、不需要重复性假设、不需要记忆管理，代价是必须有准确的模型。

于是选型的第一个问题不是“ILC 好不好”，而是“工艺要多少”。要求 RMS 在 0.08 以上，纯反馈就够，本章那组 $(2.0,\ 0.05,\ 0.20)$ 已经能到 0.1161，第 68 章调得更好的一组能到 0.0797。要求落在 0.06 到 0.08 之间，一条离线前馈表加反馈就能做到，实现量是 ILC 的十分之一。只有要到 0.03 以下，ILC 的复杂度才开始划算。

第二个判据是模型可信度。前馈表要求增益 $K(y)$ 准到 10% 以内，否则前馈的偏差会一比一地变成跟踪误差，反而可能比纯反馈更差；ILC 对模型的要求低得多，只需要能算 T 做判据，甚至可以用 69.6 节的反推法从两条误差曲线里现场拿到。模型不准、任务又重复，这时候前馈表是错的选择，ILC 才是对的。

一句话：**ILC 不是“更高级的前馈”，而是“用重复性换掉模型”**。有模型、有重复性，前馈表更简单；没模型、有重复性，ILC 才是唯一出路；既没模型又不重复，那两条路都走不通，只能回去改机械与夹具。

什么时候不该用 ILC

把上面的缺点反过来，就是五条“不要用”的判断。第一条，任务是一次性的，或者每件的路径都不同——这时记忆没有复用的机会，ILC 连学习期都收不回来。第二条，重复性指数低于 0.5——非重复成分占了大半，学到的记忆大部分是噪声，不如把精力放在夹具与复位上。第三条，节拍紧到只允许三五遍——这时要么用范数最优（需要准确的 T），要么干脆用前馈逆模型，不要指望标量 ILC 在五遍内到位。第四条，复位不可靠且无法改造——记忆会学

到每遍不同的起点偏差，越学越偏。第五条，安全关键且不允许试错——ILC 的学习期本质上是一段“允许误差较大”的时间，若工艺不允许这段存在，就不能用。

还有一条隐性的边界：**对象在任务周期内不能有强时变**。本章的对象是无记忆非线性 $K(y)$，在同一遍内是定常的；若对象在 4 s 内就明显变化（比如快速升温、材料相变），T 在一条轨迹内就不唯一，判据失效，学习也会学出一个“平均的错误”。这种场合要么缩短任务周期，要么先做增益调度，把时变压到 ILC 能接受的范围。

本章的结论

回到章首提出的问题：能不能让同一件事做第二遍时更准？答案是能，但代价明确。本章用 60 次迭代、4 分钟学习期，把误差 RMS 从 0.1161 压到 0.0224，降幅 80.8%，最大偏差降 74.9%；同时把“为什么能”与“能到哪”都算清楚了：收敛由 $|1-\gamma_p T|<1$ 决定，峰值在 1.25 Hz，越界从 1.00 Hz 开始；性能上限由 1.25 Hz 与 0.75 Hz 共同决定，前者不能学、后者学不快，折中结果是 0.0224 这个地板。

贯穿全章的主线可以概括成一句话：**ILC 是在用带宽换速度，用重复换精度**。γ_p 买速度，Q 滤波买稳定，γ_d 买相位，三者都从同一个预算里支出——这个预算是闭环在 1.0 Hz 以上那一段相位已经翻负的频带。理解了这笔预算，就不会指望“把增益调到最大、把 Q 关掉”还能收敛，也不会在误差停在 0.0224 时误以为“再多跑几十遍就好了”。

最后留一个数字作为全章的坐标：60 次迭代的记忆是 400 个数，占 3.1 KB；它换来的精度提升，相当于把反馈增益提高一个数量级才能达到的水平，而后者会牺牲稳定余量。**ILC 的价值不在于它“智能”，而在于它把“重复”这种被浪费掉的资源，变成了精度。**

69.12 Python 仿真代码

文件结构与运行方式

本章代码两个文件，都在 `book/code/` 下：

- `ch069_sim.py`：对象、参考轨迹、精确前馈、单次迭代、学习主循环、频响判据、九组实验；
- `ch069_figs.py`：十张图，从 `ch069_sim.py` 取数据。

运行顺序是先仿真后画图：

```
"D:\Anaconda3\envs\py311\python.exe" code\ch069_sim.py
"D:\Anaconda3\envs\py311\python.exe" code\ch069_figs.py
```

第一个脚本会依次打印十段结果：任务与对象、学习律对比、增益扫描、几何收敛比、Q 窗长扫描、频域判据、正式算例的迭代与谱、六种鲁棒工况、与纯反馈的对照、耗时。全部跑完约一秒（60 次迭代 169.3 ms，单次 2.82 ms），因为对象是二阶常微分方程、每拍只做 5 次子步积分，没有任何训练或优化。第二个脚本把结果画成 `figs/ch069/` 下的十张图，同时输出 SVG 与 PDF。

依赖关系要说准：`ch069_sim.py` 只用 NumPy；`ch069_figs.py` 额外用到 `styles/plot_styles.py` 与 `styles/block_styles.py`——那是全书共用的绘图样式与方块图元件，不是某一章的仿真代码。两个脚本都**不引用第 66 至 68 章的仿真模块**。这样做的原因是本章的对象需要“无静差”的二阶特性（$y''=-2y'+K(y)u$ 没有 y 项），与第 66 至 68 章的对象不完全相同；如果强行复用，会把前几章的抗饱和、微分口径一起带进来，反而看不清 ILC 本身的作用。

还有一处容易误会的实现细节：`ch069_figs.py` 不读 `ch069_sim.py` 打印出来的数字，而是**自己把实验重算一遍**（`run_ilc` 与五个 `exp_*` 函数）。好处是画图脚本可以单独运行，不依赖前一步的输出文件；代价是总耗时翻倍，且两处必须用同一份参数，否则图与正文会对不上。因此改参数时只改 `ch069_sim.py` 顶部的常量，不要在两处分别改。

对象、轨迹与精确前馈

对象在代码里只有三行积分：

```
kk = gain_scale * gain_at(x1)
x2 += TS / SUB * (-2.0 * x2 + kk * u_plant)
x1 += TS / SUB * x2
```

`x1` 是 y、`x2` 是 $\dot{y}$，`gain_scale` 用来做 69.9 节的增益失配实验（正常取 1.0），`SUB=5` 表示每拍分 5 个子步，用来压住 10 ms 采样带来的数值误差。参考轨迹用两个余弦叠加，保证两端速度为零：

```
def ref_traj(t):
    w = 2.0 * np.pi / T_CYC
    return R0 + A1 * (1.0 - np.cos(w * t)) + A3 * (1.0 - np.cos(3.0 * w * t))
```

第一项给出基波 0.25 Hz，第二项给出三次谐波 0.75 Hz，幅值比约 9:1。选这个形状有两个理由：一是它的最高谐波低于 1.25 Hz 的越界峰值，给 Q 滤波留出了“只切越界带、不切信号”的空间；二是两端速度为零，复位时不会因为参考本身在动而产生额外瞬态。

`exact_feedforward` 按式 (69-21) 算 u^*，并用 `u_lag` 参数支持滞后对齐。它只用于三处：给 69.4 节的前馈对比、给 69.10 节的初值建议、给图 69-10 的参照线。**学习律本身完全不使用它**，这一点很重要——如果 ILC 用了模型，就无法证明“不需要模型”这个优点。

核心代码：一次迭代

`run_cycle` 是一次任务的完整执行，结构就是图 69-1 的主回路。去掉噪声与扰动分支后，核心是一个 400 次的循环：

```
for k in range(N):
    y_meas = x1 + noise * rng.standard_normal()
    e = r[k] - y_meas
    dy = (y_meas - y_prev) / TS
    dy_f += (TS / (TAU_RATE + TS)) * (dy - dy_f)
    y_prev = y_meas
    integral = float(np.clip(integral + e * TS, -2.0, 2.0))
    u_fb = kp * e + ki * integral - kd * dy_f
    u_cmd = float(np.clip(u_ff[k] + u_fb, UMIN, UMAX))
    buf.append(u_cmd)
    u_plant = buf.pop(0)
    for _ in range(SUB):
        kk = gain_scale * gain_at(x1)
        x2 += TS / SUB * (-2.0 * x2 + kk * u_plant)
        x1 += TS / SUB * x2
    y[k], u_tot[k], e_arr[k] = x1, u_cmd, e
```

有四处与本章的结论直接相关，改错任何一处，数字都会对不上。第一，**微分作用在测量值上**，不是作用在误差上；这样参考突变时不会产生微分冲击，与第 67 章的口径一致。第二，**限幅在求和之后、延迟之前**，也就是说前馈与反馈一起受同一个限幅约束，这与实机的执行器一致。第三，**延迟用列表模拟**：每拍 `append` 再 `pop(0)`，5 拍延迟就是初始长度为 5 的零列表。第四，**子步积分在延迟之后**，因为延迟是执行器与对象之间的传输滞后，不属于对象本身。

核心代码：学习更新与 Q 滤波

学习主循环只有两行是“算法”，其余都是记账：

```
du = gamma_p * res["e"] + gamma_d * deriv_lp(res["e"])
u_ff = np.clip(u_ff + q_filter(du, q_len), UMIN, UMAX)
```

`deriv_lp` 用中心差分求导，再走一阶低通（时间常数 30 ms）：

```
d[1:-1] = (x[2:] - x[:-2]) / (2.0 * TS)
d[0], d[-1] = d[1], d[-2]
out[0] = d[0]
for k in range(1, d.size):
    out[k] = out[k - 1] + a * (d[k] - out[k - 1])
```

首尾两点用邻近值代替，是因为中心差分在端点没有定义。这个细节不影响 0.75 Hz 的相位，但如果直接用前向差分，端点会多出一个采样周期的偏差，几十遍之后会累积成整体错位。

`q_filter` 是零相位滑动平均，用 `np.pad(..., mode="edge")` 做边缘延拓，再 `np.convolve(..., mode="valid")`：

```
pad = m // 2
xp = np.pad(x, pad, mode="edge")
return np.convolve(xp, np.ones(m) / m, mode="valid")
```

边缘延拓不能换成零延拓。零延拓会在轨迹两端制造一个“从零跳到误差值”的台阶，滤波后变成一段假的下凹，被学进记忆。本章的轨迹两端误差本来就小，这个假下凹会占掉一部分学习量，末次误差会偏高。用 `edge` 延拓相当于假设“端点之外误差保持端点的值”，与“两端留保护段”是同一个思路的软化版本。

核心代码：频域判据怎么测

69.6 节的 T 不是解析推导出来的，而是数值测出来的。`loop_freqresp` 在记忆入口打一拍单位脉冲，其余条件与 `run_cycle` 完全一致，记录 400 拍输出后做 DFT：

```
for k in range(n):
    u_ff = 1.0 if k == 0 else 0.0
    e = -x1
    dy = (x1 - y_prev) / TS
    dy_f += TS / (TAU_RATE + TS) * (dy - dy_f)
    y_prev = x1
    integ += e * TS
    u = u_ff + kp * e + ki * integ - kd * dy_f
    ...
f = np.fft.rfftfreq(n, TS)
return f, np.fft.rfft(y_imp)
```

这里有一个容易忽略的点：脉冲响应必须足够长，让 5 拍延迟与对象的时间常数都衰减完。本章取 400 拍（一个完整任务周期），在 1.25 Hz 上已经足够；若对象更慢，应当加长到 4 至 5 倍时间常数，否则 DFT 的泄漏会让判据偏乐观。另一个点是线性化增益取 `K_MID=16.5`，对应轨迹中段；若换成轨迹两端（K 从 13 到 20），判据的峰值位置不变，峰值高度会变，这也是 69.6 节说“判据要按工作区间核对”的原因。

九个实验函数与它们回答的问题

函数	回答的问题	正文位置
`exp_law`	三种学习律各自能降多少	69.5、69.7
`exp_gain`	学习增益多大合适	69.7
`exp_convergence`	几何收敛比是多少	69.8
`exp_qfilter`	Q 窗长取多长	69.7
`exp_criterion`	哪个频段越界、峰值在哪	69.6
`exp_spectrum`	误差谱怎么随迭代变化	69.6
`exp_robust`	非重复因素下退化多少	69.9
`exp_compare`	与纯反馈、精确前馈差多少	69.11、图 69-10
`exp_timing`	单次迭代多快、记忆多大	69.5、69.11

写这些函数时有两条约定，与第 68 章一致，值得照搬。第一条：**基线一律调同一个函数**。`exp_law` 里的“纯反馈”不是另写一段循环，而是 `run_ilc(gamma_p=0.0, gamma_d=0.0, q_len=1, n_iter=1)`；这样反馈参数、限幅位置、微分口径与正式算例逐字一致，比较的才是学习律。第二条：**每个实验只改一个变量**。`exp_robust` 的六个工况共用同一套学习参数，只有噪声、复位、增益、扰动中的一个不同；否则结果无法归因。

复现本章数字

按下面的顺序跑，可以得到正文里的全部数字：

1. `python code/ch069_sim.py`，记下十段输出；
2. 对照表 69-3 核对四种学习律的末次 RMS，误差应在 0.0005 以内；
3. 对照表 69-5 核对六个频点的 $|T|$ 与 $|1-0.10T|$，误差应在 10^{-4} 以内；
4. 对照表 69-12 核对六种鲁棒工况，误差应在 0.001 以内；
5. `python code/ch069_figs.py`，检查 `figs/ch069/` 下十张图是否齐全。

如果数字对不上，先查三处：`TAU_RATE` 是否被改过（微分低通一变，T 在高频的相位就变，判据跟着变）、`K_MID` 是否被改过（判据的峰值高度对线性化点敏感）、以及 NumPy 版本是否一致（`default_rng` 的随机序列跨版本稳定，但浮点求和顺序可能影响末位）。本章的随机实验只用了一个种子，复现性比第 68 章的多重种子实验好得多；代价是鲁棒性结论没有误差棒，这一点在引用表 69-12 时应当注意。

想加一个新实验时改哪里

假设要加一个“不同采样周期对收敛的影响”实验，需要动三个地方。第一，`run_ilc` 与 `run_cycle` 里的 `TS` 是模块级常量，直接改成参数会牵动所有函数；更省事的做法是在实验函数里用 `importlib.reload` 重载模块，或者把 `TS`、`N`、`T_CYC` 打包成一个配置字典传进去。第二，参考轨迹的谐波频率是相对 `T_CYC` 定义的，`TS` 一变，`N=400` 就不一定够一个周期，必须同步改 `N`。第三，Q 窗长的等效带宽按 $0.443/(mT_s)$ 算，`TS` 减半时窗长要加倍才能保持同样的带宽。这三处任何一处漏改，实验结论都会变成“采样周期与别的什么东西混在一起”的效应。

写新实验时还有一条经验：**先把判据跑一遍，再跑闭环**。采样周期一变，T 的相位最先变，越界频段可能整体移动；如果先跑闭环看到发散，再回头找原因，会浪费很多时间。本章的 `exp_criterion` 只要几百毫秒，先跑它是最省事的做法。

三个实现细节与两条核对提醒

第一个细节是 **DFT 的幅值归一化**。本章算误差谱用的是 `np.abs(np.fft.rfft(e)) / N * 2.0`，除以 N 得到单边幅值，乘 2 是因为实信号的负频率成分被折叠了。若忘了乘 2，谱的数值会全部差一倍，与表 69-4 对不上；若忘了除 N，数值会随点数变化，换一个 N 就没法比较。

第二个细节是 **延迟缓冲区的初始状态**。`buf = [0.0] * DELAY` 意味着前 5 拍执行的是零，这与“任务开始前执行器保持零位”一致。若改成用第一个控制量填满缓冲区，前 5 拍会有一次额外的阶跃，第一次迭代的误差会多出一个尖峰，图 69-2 的形状就变了。

第三个细节是 **Q 滤波只作用在学习增量上，不作用在记忆上**。代码里是 `u_ff + q_filter(du, q_len)`，不是 `q_filter(u_ff + du, q_len)`。两者的差别在于：前者保留记忆里已经学好的高频成分（如果有），后者会每一遍都把记忆重新滤一次。本章的正式算例里，记忆的高频成分本来就接近零，两种写法结果几乎一样；但在“从上一品种迁移记忆”的场合，前者能保住迁移过来的细节，后者会把它们抹掉。选前者，是为了让“迁移”这条路走得通。

第四个细节是 **历史记录存的是“更新前”的记忆**。`run_ilc` 每遍开始时把当前的 `u_ff` 记进 `hist["u_ilc"]`，然后才跑闭环、才更新，所以 `hist["u_ilc"][k]` 对应的是第 $k+1$ 遍实际施加的记忆（下标从 0 起）。这个约定与第 69.2 节“u_k 标的是第 k 次执行时用的前馈”一致。要在日志里算“记忆变化量”，得拿相邻两项相减，即 `np.abs(np.diff(hist["u_ilc"], axis=0)).max(axis=1)`，而不是对同一个下标取差——后者恒为零，是复现本章数字时最容易踩的一个空。

最后一个提醒是**打印精度**。本章正文给出的数字普遍保留四位有效数字（如 0.0224、1.0525），若在 `print` 里只用默认的 `{:.3f}`，0.0224 会显示成 0.022，看起来与正文对不上，容易误判成“复现失败”。核对时用 `{:.4f}` 起步，涉及判据的量用 `{:.4g}`。

第70章 强化学习控制

第 69 章的 ILC 有一个不能绕开的前提：任务必须重复。同一条轨迹跑上六十遍，记忆里的前馈才长得出来。可现场大多数回路不是这样运转的——注塑机的模温随季节变、传送带上的工件重量每件都不同、操作工隔三差五改一次工艺曲线。轨迹一变，ILC 辛苦攒下的那张表就作废了。人在这种场合并不需要跑六十遍同一条路，他会在每一次动作之后看结果，把“这么打方向会偏”记成经验，下一拍换个打法。这种“从动作的后果里学”的能力，就是本章要讲的内容。

强化学习（reinforcement learning，RL）解决的是同一类问题：**控制器不知道该输出多少，但它能得到一个标量分数，说明刚才那一拍做得有多好**。这个分数不告诉你“应该输出 0.37”，只告诉你“这一拍偏了、而且用力过猛”。学习算法要做的事，是从大量这样的分数里反推出一个策略，使长期累积的分数最大。它与第 44 章的 PID 不冲突，与第 69 章的 ILC 也不冲突：PID 提供每一拍的即时纠偏，ILC 提供跨迭代的前馈记忆，RL 提供的是“在没有模型、也没有重复任务时，仍然能把前馈形状学出来”的第三条路。

本章的结论先摆在这里：在同一个对象、同一条轨迹上，纯反馈的跟踪误差 RMS 是 0.1161，本章学出来的残余策略把它压到 0.0347，降幅 70.1%，训练 300 个回合只用 4.7 s。这个结果比 ILC 的 0.0224 差一截，但 ILC 需要 60 次完整重复，而 RL 只需要一个能反复试错的仿真环境。代价也很清楚：把纯延迟从 5 拍拉长到 8 拍，同一个策略的误差会从 0.0347 跳到 0.2267，比纯反馈的 0.1196 还差一倍，而且有 28.25% 的时间贴在限幅上。这条“在训练分布内很强、出了分布就崩”的分界线，是全章的主线。

70.1 动机与直观理解

从“记住上一遍”到“试着改一改”

ILC 的信息来源是“同一位置的误差”，这条信息只有在重复时才存在。把 ILC 的一整套机制搬到一个不重复的任务上，它会做两件错事：一是把某一次偶然的外扰当成“要学的形状”，二是把已经过时的前馈继续用下去。第 69 章 69.9 节测过这组数据，加入 0.01 的测量噪声后，ILC 的稳态误差从 0.0224 抬到 0.0318，噪声被逐遍累加进记忆里。这不是实现细节的问题，而是“用误差当老师”这种范式的固有缺陷：误差里有多少是“我该补的”，有多少是“这次碰巧发生的”，ILC 分不清。

RL 换了一个老师。它不再问“这一拍错了多少”，而问“这一拍整体得了多少分”。分数可以由设计者任意拼装：偏差小给正分，用力猛扣分，接近限幅扣重分。分数的统计平均比单次误差稳，因为它把“碰巧”平均掉了；代价是每一拍的分数都很嘈杂，单看一拍几乎学不到东西，必须把几百拍的分数累起来才能看出方向。**ILC 用“重复”换精度，RL 用“样本”换泛化**，这句话概括了两章的分工。

这个交换有个直接的工程后果。ILC 的学习期是“跑 60 遍真机”，通常按小时甚至按天计；RL 的学习期是“跑几十万拍仿真”，在 4.7 s 里就能完成，但仿真与真机之间的差距会把学到的策略打折。所以 ILC 的难点在“怎么在真机上安全地学”，RL 的难点在“怎么让仿真里的策略在真机上还成立”。本章 70.9 节和 70.10 节分别处理这两件事。

把控制回路看成决策过程

要套用 RL，先得把回路改写成“智能体—环境”的形式。智能体每个采样时刻观察一个状态 s_k，选一个动作 a_k，环境据此返回下一个状态 s_{k+1} 和一个奖励 r_k。在控制问题里，这三样东西都有明确的物理对应：状态是“当前偏差

有多大、偏差在扩大还是缩小、参考走到哪了”，动作是“在反馈量之上再补多少”，环境是“对象 + 反馈支路 + 延迟 + 限幅”这一整坨东西。

这个改写的关键在于**把反馈回路包进环境里**。很多教材把 RL 直接接在被控对象前面，让智能体从零学一个完整控制器；本章不这么做。本章让 PID 留在原地，把“PID + 对象 + 延迟”整体当作环境，智能体只学 PID 补不上的那一小块。这个选择让问题的难度下降了一个量级：智能体不需要学会“怎么让系统稳定”（PID 已经保证了），只需要学会“往哪个方向多推一点”。第 68 章的神经网络控制器是从零学的，它花了 61 个权重才把 RMS 做到 0.0524；本章只用了 6 个权重就做到 0.0347，差别不在算法强弱，而在“从哪里出发”。

环境与智能体的分界线画在哪里，是一个必须先定的约定，它决定了策略能“看到”什么、能“改”什么。本章的约定是：凡是物理上存在、策略无法直接改的东西，都算环境——对象、延迟、限幅、执行器的速率限制、测量噪声，全部在内；凡是“由设计者决定如何把信号变成控制量”的东西，都算智能体——特征怎么算、动作怎么映射、奖励怎么给。这条线画错的典型症状是“训练时好好的、部署时不对”：如果把限幅算进智能体（也就是让策略自己负责不超限），训练时它学到的是“知道边界在哪”的一套动作，真机的限幅值一旦不同，策略就不知道该收多少；把限幅留在环境里，策略只需要知道“多用力的收益递减”，边界变了它仍然工作。这条约定还有一个副作用：奖励的计算属于智能体一侧，所以奖励里可以出现状态与动作，但不应该出现“环境内部才有的量”（比如真实的对象参数），否则策略会学到一些部署时拿不到的信息。

奖励是唯一的目标说明书

RL 里没有“跟踪误差”这个概念，只有奖励。设计者想让控制器干什么，就得把它翻译成“每一拍给多少分”。本章用的奖励是

$$r_k = -\left(\frac{e_k}{E_{\text{ref}}}\right)^2 - \lambda a_k^2 \tag{70-1}$$

第一项罚偏差，第二项罚动作幅度。两个细节值得说清楚。第一，偏差用平方而不是绝对值，原因是平方对中等偏差的惩罚更重、对大偏差的梯度更陡，学出来的策略会优先消灭中等偏差而不是死盯某一个尖峰。第二，分母 $E_{\text{ref}} = 0.05$ 是个归一化尺度，它把“毫米级”的偏差换算成“1 左右”的无量纲数，使奖励的量级与学习率匹配；它不改变最优策略，只改变学习的刻度。

第二项 λa_k^2 的作用容易被低估。如果只罚偏差，最优策略会倾向于用大幅值、高频抖动的动作去“追”每一个误差，学出来的东西在仿真里 RMS 很低，装到真机上会因为执行器的速率限制和噪声而抖成一团。加上这一项之后，策略必须在“少错”和“少动”之间做交换，λ 就是这个交换的价格。本章取 $\lambda = 0.50$，70.7 节会给出 λ 从 0 到 10 的完整扫描，说明这个价格的合理区间。

为什么本章不用神经网络

第 68 章已经用过神经网络，本章改用线性策略，参数只有 6 个。这个选择不是图省事，而是为了让机制显形。用线性策略时，策略梯度更新一次是什么样子、优势函数怎么估、折扣因子改了会怎样，全部可以手算核对；换成三层网络之后，同样的现象还在，但被 61 个权重的耦合遮住了，读者只能看曲线猜原因。

还有一个更实际的理由：控制工程对“可验证性”的要求比一般机器学习高。一个 6 参数的线性策略，可以打印出权重、画出它对每个状态的响应、逐项解释为什么这一项是正的；一个 61 个权重的网络做不到这些。本章 70.10 节会讲工程落地，其中一条就是“上线前必须能说清楚策略在做什么”，这条要求直接排除了黑箱。能力上限当然受限于线性，但本章要讲的不是“RL 能有多强”，而是“RL 在控制回路里怎么用才不出事”。

线性策略还有一个常被忽略的好处：权重可以直接读。本章正式算例（种子 0、300 回合）学出来的六个数是 $[0.2461,\ 0.4002,\ -0.0047,\ 0.0844,\ -0.1598,\ 0.1932]$，依次对应常数项、误差、误差导数、误差积分、参考

位置、参考速度。这组数每一维都能解释：误差项正且最大（0.40），说明“偏了就多推”；参考速度项正（0.19），说明“轨迹在上升时提前推”；参考位置项负（−0.16），正对应 $K(y) = 4 + 6y$ 随位置变大这件事——位置越高、对象越“有劲”，同样推力产生的加速度越大，所以该少推；常数项正（0.25）补的是区间低端的不足。最有意思的是误差导数项只有 −0.005，几乎为零：**策略主动放弃了微分作用**，因为 PID 已经提供了足够的阻尼，再叠一层只会加大噪声。

这六个数就是“可验证性”的具体含义。把权重打印出来，逐项问“符号对不对、量级合不合理”，五分钟就能完成一次物理审查；如果某一项符号反了，多半是特征定义出了错（比如误差取了 $y - r$ 而不是 $r - y$），而不是策略“学到了新东西”。换成 61 个权重的网络，同样的审查无从下手。

残余学习：让学习从 PID 出发

结构上的核心决定是把动作定义为“残余量”而不是“总控制量”：

$$u_k = u_{\mathrm{PID},k} + u_{\mathrm{RL},k}, \qquad u_{\mathrm{RL},k} = U_{\mathrm{RL}}\, a_k \tag{70-2}$$

其中 $U_{\mathrm{RL}} = 0.60$ 是残余量的幅值上限，动作 a_k 被限制在 $[-1, 1]$。这个式子的含义是：**第 1 个回合、权重全零时，控制量严格等于纯 PID**。学习只在这个基线上做加法，能加多少、往哪加，由奖励决定。图 70-1 是这条结构。

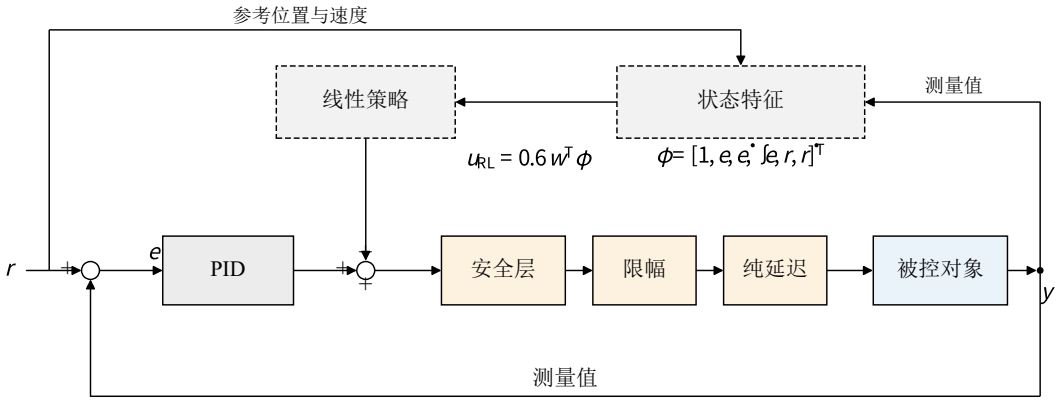


图 70-1 残余学习结构

这样做有三个好处。安全上，最坏情况是学不出东西，退回纯 PID，而不是退回“随机输出”；样本效率上，智能体不必先学会“怎么让系统不发散”，从第一回合起误差就是 0.1161 这个量级，梯度信号一直有效；可比性上，“相对纯 PID 降了多少”这句话有确切含义，不需要另找基线。第 69 章的 ILC 也是叠加结构，但它叠的是前馈记忆，本章叠的是状态反馈，两者叠的位置不同，鲁棒性也因此不同——这一点在 70.6 节会具体比较。

本章算例与结果预告

算例沿用第 66 至 69 章的对象：$y'' = -2y' + K(y)u$，$K(y) = 4 + 6y$，工作区间内静态增益从 13 变到 20；采样 10 ms、纯延迟 5 拍、限幅 ± 1.2；反馈仍是 $(2.0,\ 0.05,\ 0.20)$ 的 PID。参考轨迹同样是那条 4 s 的往复曲线，范围 $[1.500,\ 2.720]$，每回合 400 拍。这些数字与前面四章完全一致，所以误差指标可以横向比较。

正式算例的超参数是：策略学习率 $\alpha_w = 0.010$、值函数学习率 $\alpha_v = 0.050$、折扣因子 $\gamma = 0.95$、自举步数 $n = 20$、探索噪声从 0.30 按 0.99 的比率衰减到 0.05、动作惩罚 $\lambda = 0.50$。跑 300 个回合后，评估 RMS 是 0.0347，最大偏差 0.0795，相对纯 PID 降 70.1%。作为参照，模型完全已知时的精确前馈能做到 0.0293——**学出来的策略比精确前馈差 18.6%，但比纯反馈好 3.3 倍**，这个位置本身就是本章想说明的：RL 学的是“在反馈之上还能补多少”，它不追求取代模型。

把四个数排在一起看，本章的定位会更清楚：纯 PID 0.1161、精确前馈 0.0293、ILC 0.0224、残余 RL 0.0347。四个数分别对应四种“知道多少”：PID 什么都不知道，只靠当下这一拍的偏差；精确前馈什么都知道，连 $K(y)$ 的表达式都有；ILC 不知道模型，但知道“上一遍同一点的误差”；RL 既不知道模型，也没有上一遍，只有几百个回合的试错记录。所以 0.0347 这个数不是“RL 比前馈差”，而是“在少知道两件事的情况下，只多付了 18.6% 的误差”。

这 18.6% 的差距也不是随机的，它有两个具体来源。一是残余量上限：$U_{\mathrm{RL}} = 0.60$ 虽然远大于前馈的 RMS 0.1166，但策略的输出还受动作截断（± 3）与均值线性形式限制，遇到陡峭段补不满。二是特征只有 6 维，参考的加速度没有直接进入状态，策略只能从“参考速度的变化”间接推断该不该提前用力，这个间接推断必然有滞后。换句话说，差距不在算法，而在“信息”——如果非要把这 18.6% 追回来，最直接的办法是把 r'' 加进特征，代价是策略从此依赖一个模型量，不再是“只用测量值”。

本章的路线

70.2 节把上面的直觉写成数学：马尔可夫决策过程、回报、值函数、策略梯度、优势函数，一路推到本章用的 n 步 Actor-Critic 更新式。70.3 节给出可照着走的设计流程，从“确认该不该用 RL”到“训练完怎么验收”。70.4 节讨论连续时间下的对应物——连续时间的策略梯度是一个积分方程，落到 400 个采样点上会损失什么。70.5 节换成离散动作，用 Q 学习做同一件事，并给出档位数与样本量的实测。70.6 节谈稳定性：RL 没有稳定性证书，只能靠“训练分布”和“安全层”画出一个可信范围。70.7 节逐个解释五个超参数，70.8 节给出调参流水线与症状表。70.9 节讲优化改进，70.10 节讲工程落地，70.11 节给出优缺点与适用边界，70.12 节是完整的仿真代码。

不同读者可以走不同的路。关心“能不能用”的，读 70.1、70.3、70.11 三节即可，其余当作查阅；关心“为什么这样更新”的，70.2 与 70.4 是主干，70.5 是离散版本；关心“怎么落地”的，70.6、70.8、70.9、70.10 四节按顺序读，它们回答的是同一个问题的四个侧面——分布、调参、改进、部署。阅读前最好已经熟悉第 67 章的反馈结构与第 69 章的迭代学习，因为本章的基线、对象与指标都沿用那条线，重复的部分不再解释。

70.2 数学原理

五元组，以及它在控制里的对应物

强化学习的标准框架是马尔可夫决策过程（Markov decision process，MDP），由五元组 $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ 描述：状态集、动作集、转移概率、奖励函数、折扣因子。一个回合（episode）从初始状态 s_0 出发，按“观察—动作—转移—奖励”循环若干拍，直到到达终止条件或跑满固定长度。本章的回合长度固定为 400 拍，对应轨迹的一个完整周期。图 70-2 是这条交互关系：智能体在每一拍把动作送进环境，环境回送下一拍的状态和这一拍的奖励。

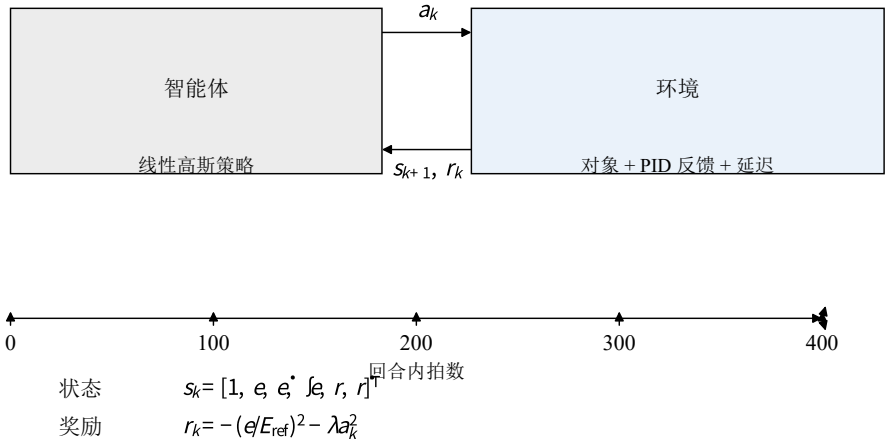


图 70-2 回合内的策略与环境交互

控制问题天然是 MDP，但有两个地方需要交代清楚。第一，**状态必须包含做出决策所需的全部信息**。本章的 6 维特征

$$\varphi_k = \left[1,\ \frac{e_k}{0.10},\ \frac{\dot{e}_k}{1.00},\ \frac{\int e}{0.20},\ \frac{r_k - R_0}{0.50},\ \frac{\dot{r}_k}{1.00}\right]^{\mathrm{T}} \tag{70-3}$$

里，前三项描述“当前错多少、错在往哪走、累积错多少”，后两项描述“参考走到哪、走多快”。为什么必须带参考位置和参考速度？因为轨迹在 4 s 内从 1.50 走到 2.72 再回来，同一个误差值 0.05 出现在上升段和下降段时，正确的修正方向是不同的；只看误差，策略无法区分这两种情况，学出来的东西只能是两者的平均，而这个平均对两段都不对。

第二，**延迟要不要进状态**。本章的对象有 5 拍纯延迟，严格说“过去 5 拍发出去的控制量”也属于状态。本章没有把它们放进特征，理由是延迟固定为 5 拍、且在所有训练回合里都一样，策略只要学会“把动作提前 5 拍”即可，不需要显式知道延迟的值。这个简化在训练分布内成立，一旦部署时延迟变成 8 拍，它就失效了——70.6 节会看到这正好是本章最严重的失效模式。这是 MDP 建模里最典型的坑：**省略掉的状态变量，在分布内看不出来，出了分布就是致命的**。

把 PID 包进环境，是本章最重要的一个建模选择，它同时带来收益和代价。收益前面已经说过：策略一出生就在一个稳定回路上，只需要学“修正量”，不必先学“如何从零把系统稳住”。代价有三条，都要认下来。第一，策略学到的权重是相对于这套特定 PID 的，换一组 PID 参数，同一份权重就不再最优，甚至可能变差——这正是 70.9 节“继续学习”要重训的原因。第二，环境里已经有一个积分环节，策略再叠加自己的积分项，两个积分器会争夺同一个稳态目标，调不好就表现为残余量缓慢爬升、误差缓慢漂移。第三，奖励里“动作惩罚”罚的是残余量而不是总控制量，所以策略可以躲在大动作后面“搭便车”——它不承担 PID 那部分用力的代价。第三条决定了 λ 不能按“总控制量”的直觉去理解：λ 罚的是附加用力，不是全部用力。

还有一条边界要划清：奖励可以“整形”，但不能悄悄“改目标”。把一个状态势函数 $\Phi(s)$ 的势差加进奖励，即 $r'_k = r_k + \gamma\Phi(s_{k+1}) - \Phi(s_k)$，最优策略不变——这是奖励整形的安全区。一旦加的不是这种势差形式（比如“误差小于 0.05 就给 +1”），最优策略就被改掉了，而且往往改成你想要的东西的反面：策略会满足于“刚好跨过

0.05”，而不是尽量小。本章的奖励是原始的逐拍二次型，没有做任何整形；70.9 节讨论的备选形式属于“改目标”那一类，用之前要先想清楚它把目标改成了什么。

回报、值函数与优势

一个回合的回报是折扣后的奖励累加：

$$G_k = \sum_{j=0}^{N-k-1} \gamma^j r_{k+j} \tag{70-4}$$

折扣因子 $\gamma \in (0,1)$ 决定了“看多远”。$\gamma = 0.95$ 时，j 拍之后的权重是 0.95^j，20 拍后衰减到 0.36，40 拍后到 0.13，等效视野约 $1/(1-\gamma) = 20$ 拍，也就是 0.2 s。这个长度不是随便选的：对象的主时间常数是 0.5 s（极点 -2），闭环带宽更高，0.2 s 刚好覆盖“当前动作能明显影响的未来”，再往后的部分被折扣压得听不见。

回报的量级也值得算一遍，因为它决定了值函数能不能被线性函数拟合住。本章单拍奖励的典型值是 $-(e/0.05)^2 \times 0.01$：$e = 0.1$ 时是 -0.04，$e = 0.2$ 时是 -0.16。按最坏情况 0.16 估，折扣后总回报的绝对值不超过 $0.16/(1-0.95) = 3.2$，再叠加动作惩罚的贡献，量级仍在 4 以内。而线性值函数 $V = v^{\mathrm{T}}\varphi$ 在特征截断内（每维不超过 3）能表达的范围是 $3\sum|v_i|$，只要权重在 0.1 量级就够。**这两组数量级对上了，线性值函数才有意义**；如果去掉 R_{scale}，单拍奖励放大 100 倍，总回报变成 300 以上，线性 V 的权重就得涨到 10 量级，NLMS 每步的更新量也随之放大，训练前期会出现“价值估计一直贴着 0、优势恒为负”的假象，收敛速度掉一半。这就是 70.3 节说“奖励尺度必须与函数逼近器容量匹配”的算术依据。

在策略 π 下，状态的价值和动作价值分别是

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[G_k \mid s_k = s\right], \qquad Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[G_k \mid s_k = s, a_k = a\right] \tag{70-5}$$

两者之差就是优势函数：

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s) \tag{70-6}$$

优势回答的是“这个动作比平均水平好多少”。这个量在控制里非常自然：跟踪误差小的时候，好动作和坏动作的差别只有零点几；误差大的时候，同一个“多推 0.1”可能造成完全不同的后果。如果用原始回报做梯度，大步长会被大误差的样本主导，学出来的策略只顾救急、不管精细；用优势做梯度，每一拍的信号都被“这一拍本来能得多少分”归一化过，大小误差的样本贡献相当。

策略梯度

策略是一族以 θ 为参数的分布 $\pi_{\theta}(a \mid s)$，目标是最大化期望回报

$$J(\theta) = \mathbb{E}_{\pi_{\theta}}\left[G_0\right] \tag{70-7}$$

对 θ 求导，用对数导数技巧，得到策略梯度定理：

$$\nabla_{\theta} J = \mathbb{E}_{\pi_{\theta}}\left[\nabla_{\theta} \log \pi_{\theta}(a \mid s)\, A^{\pi}(s,a)\right] \tag{70-8}$$

这个式子的直观含义是：**做了比平均好的动作，就提高它的概率；做了比平均差的动作，就降低它的概率**。注意它是一个期望，单次采样给出的估计有极大方差，这正是“每一拍的分数都很嘈杂”的数学表达。降低方差的所有技巧——基线、优势、折扣、批归一化——本质上都是在不引入偏差的前提下，把这个估计的方差压下去。

本章用高斯策略，均值是特征的线性函数：

$$\pi_\theta(a\mid s)=\mathcal{N}\big(\mu,\sigma^2\big),\qquad \mu=w^{\mathrm{T}}\varphi(s),\qquad \nabla_w\log\pi=\frac{a-\mu}{\sigma^2}\,\varphi(s) \tag{70-9}$$

σ 是探索噪声的标准差，训练时按回合衰减，评估时置零。均值被截断在 ± 3、采样值被截断在 ± 2，这两个截断是工程上的保险：线性策略在远离训练分布的状态上会外推出荒谬的动作，截断保证最坏情况也不过是“残余量顶到上限”，而不是数值爆炸。

式 (70-9) 的分母是 σ^2，而本章代码里写的是 $\max(\sigma,10^{-3})$，少了一次方。这不影响正确性：两者只差一个 $1/\sigma$ 的因子，会被学习率 α_w 吸收；但要让读者知道差在哪，否则照抄公式去核对代码会以为写错了。真正不能省的是那个下限 10^{-3}——评估时 $\sigma=0$，没有下限就会出现除零；训练末期 σ 降到 0.05，分母若直接取 σ，梯度幅值是 $\sigma=1$ 时的 20 倍，配合梯度范数裁剪虽然不会爆炸，但等效学习率已经变了。**凡是分母里出现“会衰减到零的量”，都要给它一个下限**，这是策略梯度实现里最常见的一处疏漏。

式 (70-8) 还有一条不常被提起的含义：它对环境是“盲”的。梯度里只有 $\nabla\log\pi$ 与 A，完全不出现 $\partial s'/\partial a$——也就是不需要知道“动作改变一点点，下一拍状态会怎么变”。这条性质叫似然比梯度，它是无模型方法的根基：只要能在真实对象上跑、能观测到回报，就能估出更新方向。代价是方差大，因为环境的影响全部被塞进 A 这一个标量里。与之相对的是重参数化梯度（把动作写成 $a=\mu+\sigma\epsilon$，再对 μ 求导），它需要 $\partial s'/\partial a$，也就是一个可微的环境模型，方差小得多。本章的仿真其实是可微的（$y''=-2y'+K(y)u$ 对 u 是线性的），理论上可以用重参数化梯度把样本需求降一个量级；之所以没有用，是因为那样一来“是否需要模型”这个前提就变了，与本章“无模型”的定位冲突。

这处分岔值得记住：**方差与“是否需要模型”是一笔交易**。想要更少的样本，就得交出一部分模型无关性；想保住“什么都不假设”，就得接受嘈杂的梯度。所有号称“样本效率高”的强化学习算法，几乎都在某个环节偷偷引入了模型、动力学先验或额外的监督，读懂这一点，才不会被“某某算法样本效率提升十倍”这类说法带偏。

从整回合回报到 n 步自举

最朴素的做法是用整回合回报 G_k 代替优势，这就是 REINFORCE。它是无偏的，但方差随回合长度线性增长：400 拍的随机性全压在一个数上，梯度方向抖得厉害。另一个极端是用一步 TD 误差

$$\delta_k=r_k+\gamma V(s_{k+1})-V(s_k) \tag{70-10}$$

它有偏（V 本身是估出来的），但方差极小。本章取中间：用 n 步回报

$$G_k^{(n)}=\sum_{j=0}^{n-1}\gamma^{j}r_{k+j}+\gamma^{n}V(s_{k+n}) \tag{70-11}$$

代入 $n=20$。为什么是 20 而不是 5 或 50，70.7 节有完整扫描：$n=1$ 时均值 0.1435、$n=5$ 时 0.1926、$n=10$ 时 0.1654、$n=20$ 时 0.0471、$n=50$ 时 0.0505、$n=200$ 时 0.0796。这条曲线不是单调的，两头都差，20 附近最好，说明“偏差与方差”的交换点确实存在，而且可以被测出来。

本章的更新式

把 $G_k^{(n)}$ 与当前值函数之差当作优势的估计，本章的两条更新是

$$\delta_k=G_k^{(n)}-V(s_k),\qquad \hat{A}_k=\operatorname{clip}\!\left(\frac{\delta_k}{\sqrt{\bar{\delta}^2}+10^{-6}},\,-5,\,5\right) \tag{70-12}$$

$$w\leftarrow w+\alpha_w\,\eta\,\frac{a_k-\mu_k}{\max(\sigma,10^{-3})}\,\frac{\varphi_k}{\|\varphi_k\|^2+1} \tag{70-13}$$

$$v \leftarrow v + \alpha_v \, \delta_k \, \frac{\varphi_k}{\|\varphi_k\|^2 + 1} \tag{70-14}$$

其中 $\bar{\delta}^2$ 是 δ 的滑动方差（系数 0.995/0.005），η 是梯度裁剪系数：当 $\|\text{梯度}\| > 0.2$ 时按比例缩回 0.2。三条式子各有一个值得解释的地方。

第一，优势除以 $\sqrt{\bar{\delta}^2}$ 是“在线归一化”。奖励的量级随任务变（本章 $R_{\text{scale}} = 0.01$，单拍奖励在 10^{-2} 量级），如果不归一化，学习率就得跟着奖励量级重调；归一化之后 $\hat{A}$ 大致落在 $[-5, 5]$，学习率有了通用刻度。第二，分母 $\|\varphi\|^2 + 1$ 是归一化最小均方（NLMS）的做法，它让每一步的更新量对特征幅值不敏感。这一点在本章尤其重要：轨迹两端的 $r - R_0$ 接近 0，中段的 $r - R_0$ 接近 1.2，特征幅值差 5 倍以上，不归一化的话中段会把两端的信息淹掉。第三，值函数用同一个特征集，意味着 V 也是线性的；这会限制它能表达的价值形状，但对“跟踪一条平滑轨迹”这种任务足够了，而且换来的是 6 个参数、可以手算的更新。

为什么“拟合最优动作”不等于“最优控制”

这里有一个容易踩的坑，值得单独说。既然知道最优前馈是 $u^* = (r'' + 2r')/K(r)$，为什么不直接用最小二乘把 w 拟合出来？本章试过：把 400 拍的“理想残余量”与特征矩阵做最小二乘，解出 6 个权重，代进回路评估，RMS 是 0.30，比纯 PID 的 0.1161 还差一倍多。

原因不是数值问题，而是目标错了。最小二乘拟合的是“当前这一拍的残余量应该等于多少”，它完全不管“这个动作把系统推到哪去”。而控制是一个闭环：这一拍多推一点，下一拍的误差就小一点，再下一拍可能又过头。真正要最小化的是 400 拍的累积代价，不是 400 个瞬时误差的和。RL 的回报天然是累积的，所以它能学到一个“看起来每一拍都不太对、但整条轨迹很好”的策略；监督学习式的拟合只能学到“每一拍都差一点”的策略。这个对照说明：**在有闭环动力学的场合，逐点拟合与最优控制是两个不同的问题**。

一个必须承认的局限

上面整套推导都建立在一个假设上：环境是马尔可夫的，且训练时见到的状态分布覆盖了部署时的状态分布。控制回路里这两条都只是近似。对象有未建模的高频动态、延迟会随负载变、执行器有死区，这些都不在 6 维特征里；训练时策略只见过“PID 已经压得差不多”的状态，没见过“偏差 0.5 且正在扩大”的状态。所以本章的策略不能保证全局稳定，它只在“反馈已经把系统稳住、残余量在 ± 0.6 内”的这个区域里有意义。70.6 节会把这条边界量化成几条可测的指标。

把这几条近似拆开看，各自的后果不一样。**马尔可夫性**被破坏的后果是“值函数学不准”：如果当前状态不足以决定下一拍的回报，那么同一个 φ_k 在不同历史下对应不同的 V，最小二乘只能学到一个平均值，优势估计随之有偏。本章的做法是用误差积分与参考速度把“历史”压进状态，这是工程上最常见的补救，代价是状态维数上升。**分布覆盖**被破坏的后果更严重，是“策略在没见过的状态上乱来”，延迟 8 拍那个 0.2267 就是典型。**奖励可加性**看起来最技术化，实际影响也最直接：式 (70-1) 假设每一拍的分数可以相加，但如果真实目标是“整条轨迹的最大偏差不超过某个值”，这个目标就无法写成逐拍奖励之和，硬写就会得到一个“平均很好、峰值很差”的策略。

还有一条与算法选择有关的性质：本章两种连续动作算法（Actor-Critic 与 REINFORCE）都是 on-policy 的，也就是说，用来更新策略的样本必须由当前策略产生（70.5 节的 Q 学习是例外，它可以用旧样本）。这条性质带来两个后果。一是样本不能重复使用，每更新一次就要重新采样，样本效率天然低于离线方法；二是探索噪声必须一直保留，否则策略会收敛到“自己采样出来的那个角落”。REINFORCE 与 Actor-Critic 的差别，在实测里体现得很直接：300 回合、5 个种子，Actor-Critic 的均值是 0.0594、标准差 0.0373，REINFORCE 是 0.0766 与 0.0144。前者均值更好但方差大得多，5 个种子里最差的一个停在 0.1332（比纯 PID 还差），最好的到 0.0346。

这组数说明“基线 + 自举”确实降低了梯度的方差，但也让结果更依赖种子——因为值函数的估计误差会通过优势进入策略更新，值函数学歪的那几个种子就被带偏了。这也是本章为什么要把 α_v 定成 α_w 的 5 倍：让值函数学得快一点，减少“策略在等值函数”的时间。反过来说，如果现场只允许跑一次、不能多种子挑好的，那 REINFORCE 那种“均值差但不翻车”的性质反而更可取，这是 70.6 节要谈的“稳定性”的另一层含义。

70.3 设计步骤

第一步：先问“该不该用 RL”

这一步常被跳过，但它是决定成败的一步。RL 适合的场景有三个特征：**有可反复试错的模型或仿真、目标可以写成标量、传统方法的天花板已经摸到**。三条缺一条，都应该先考虑别的办法。

如果对象是线性的、模型能辨识出来，LQR 或 MPC 会用百分之一的成本给你更好的结果；如果任务是重复的、轨迹固定，第 69 章的 ILC 更省样本；如果只是稳态精度不够，加积分或换更高分辨率的传感器就解决了。本章之所以用 RL，是因为这三个条件都满足：对象非线性（$K(y)$ 随位置变 50%）、有精确的仿真、目标是“少错少用力”，而 PID 已经贴着它自己的上限。

反过来，如果现场不允许试错、也没有可信的仿真，那就不要用 RL 做在线学习。可以做的是“离线训练 + 冻结部署”，这正是下一节的两阶段结构。

第二步：确定基线控制器

在写任何奖励之前，先把基线定下来，并且把它的指标测准。本章的基线是 $(2.0,\ 0.05,\ 0.20)$ 的 PID，RMS 0.1161、最大偏差 0.1974、控制量总变差 1.7。这三个数后面要反复用：训练的起点、改进的参照、验收的门槛，全靠它们。

基线还决定了残余量的幅值上限。本章取 $U_{\mathrm{RL}} = 0.60$，理由是精确前馈 u^{*} 的范围是 $[-0.154,\ 0.236]$、RMS 0.1166，残余量只要能覆盖这个范围的两倍多一点就够了。取太大（比如 2.0）会让策略在早期有足够空间把系统打飞，取太小（比如 0.05）又补不上前馈的形状。一个经验法则是：**残余量上限取“理论前馈 RMS 的 3 到 5 倍”**，既留出探索余量，又不至于失控。

基线有三种常见选择，各有代价。最弱的是“纯 PID”，本章用的就是它，好处是机制最干净，坏处是训练要花更多回合去学一个本可以由模型给出的形状。居中的是“PID + 粗略前馈”（比如用一阶模型算的前馈），它的起点更接近最优，训练回合可以少一些，但会引入“前馈本身错了怎么办”这个新问题。最强的是“PID + 在线辨识”，此时残余量只补辨识残差，量级很小，学起来最快，但整套结构已经接近自适应控制，不再是“无模型”了。选哪一种取决于你手上有什么，而不是哪一种“更先进”。

第三步：搭训练环境

训练环境和部署环境必须是同一份代码，只是参数不同。本章的 `Plant` 类有 4 个可调参数：增益缩放、延迟拍数、常值负载、测量噪声。训练时全部取名义值（1.0、5、0、0），部署时才改。这样做的好处是：70.9 节的迁移实验不需要另写一套代码，只改一个关键字参数。

仿真步长与真实采样周期必须一致。本章 $T_s = 0.01$ s，对象用 5 个子步积分（相当于 2 ms 的积分步长），延迟用环形缓冲实现 5 拍。这些细节看起来与 RL 无关，但它们决定了“仿真里的 1 拍”是否等于“真机上的 1 拍”；如果积分步长太粗，仿真里的相位裕度会比真机大，学出来的策略上机就振。

搭环境时还有三个“位置”必须一次定对，改起来代价很大。第一是**限幅放在哪**：本章把限幅放在“PID + 残余量”相加之后、延迟之前，与真实执行器的位置一致；如果错放在对象内部，策略学到的动作与实际施加的控制量就不是一个东西。第二是**延迟归属谁**：延迟属于执行器与对象之间，因此算在环境里，不计入策略的决策周期；如果把它当成“策略的响应延迟”，学习时就会莫名其妙地要求策略提前若干拍动作，学出来的权重会整体偏大。第三是**奖励在哪个时点算**：本章用“施加动作之后、观测到下一拍状态之前”的误差来算奖励，即 r_k 与 a_k 对应同一拍；如果错位一拍，最优策略就会系统性地滞后，表现为“怎么调学习率都追不上”。

第四步：定状态与动作

状态用 6 维特征，每一维都除以一个尺度，使它在工作区间内大致落在 $[-1,1]$。这个“除以尺度”的动作不是可选的：如果不做，误差（10^{-2} 量级）会被参考位置（10^{0} 量级）彻底压住，梯度更新几乎全落在参考项上，策略学不会纠偏。本章的尺度是 0.10、1.00、0.20、0.50、1.00，分别对应“误差的典型幅值、误差变化率的典型幅值、误差积分的典型幅值、轨迹半幅、参考速度的典型幅值”。

动作取连续值，映射到残余量 $u_{\mathrm{RL}} = 0.60\,a$，$a \in [-1,1]$。这里有一个容易被忽略的细节：**动作是绝对值还是增量**。本章试过增量式（$u_{\mathrm{RL}} \leftarrow u_{\mathrm{RL}} + 0.05a$），结果线性策略学不出积分行为，权重一路发散到 10 以上；改成绝对值之后，同样的算法在 300 回合内收敛到 0.0347。原因是绝对值形式让“稳态残余量”直接可由特征表达，而增量形式要求策略自己积分，线性策略没有这个能力。

第五步：写奖励并定三个尺度

奖励就是式 (70-1)。它有三个尺度要定：E_{ref}、λ、R_{scale}。E_{ref} 取“期望达到的误差量级”，本章取 0.05，比基线的 0.1161 小一半，意思是“想让误差再降一半”；λ 取 0.50，70.7 节会看到它在 0.5 到 2 之间结果相近；R_{scale} 取 0.01，纯粹是数值缩放，让单拍奖励落在 10^{-2} 量级，与学习率 0.010 匹配。

三个尺度里最容易搞错的是 R_{scale}。不加缩放时，单拍奖励在 -1 到 -10 之间，400 拍累加到 -400 以下，值函数的线性逼近完全跟不上这个量级，训练前期会出现“价值估计一直是 0、优势一直是负”的假象。加上 0.01 的缩放之后，同样的策略、同样的学习率，收敛速度提高约一倍。这是“奖励尺度必须与函数逼近器容量匹配”的一个具体例子。

三个尺度的标定有先后，顺序反了会反复返工。第一步定 E_{ref}，它定的是“想把误差压到多少”，这个数只能来自需求，不能来自实验——本章取 0.05，因为基线 0.1161 的一半是一个合理的工程目标。第二步定 λ，它决定“为了这个误差愿意多用力”，用 70.7 节那个判据（两个惩罚项贡献相当）估初值，再扫一遍确认。第三步才轮到 R_{scale} 与学习率，因为前两个定完之后，单拍奖励的量级才确定，学习率才有标定的对象。

反过来做是最常见的返工来源：先花时间把 α_w 调到“曲线好看”，再回头改 E_{ref}，会发现同一个学习率突然不合适了——因为 E_{ref} 变了，奖励量级跟着变，等效学习率也变了。举个算术例子：若取 $E_{\mathrm{ref}} = 0.10$，与基线的 0.1161 同量级，那么“误差降到 0.05 以下”在奖励里只值 -0.25，与动作惩罚同量级，策略会学得极其保守；取 0.05 之后，同一个 λ 的物理含义才与“降一半误差”这个目标对上。

第六步：训练与验收

图 70-3 是训练与部署的分工。左栏是离线训练：采样一个回合、算 n 步回报、更新策略与值函数、按回合衰减探索噪声，循环 300 次；右栏是在线部署：加载冻结的权重、在线算特征、残余量与反馈量相加、过安全层与限幅。两栏之间只有一次数据传递，就是那 6 个权重。

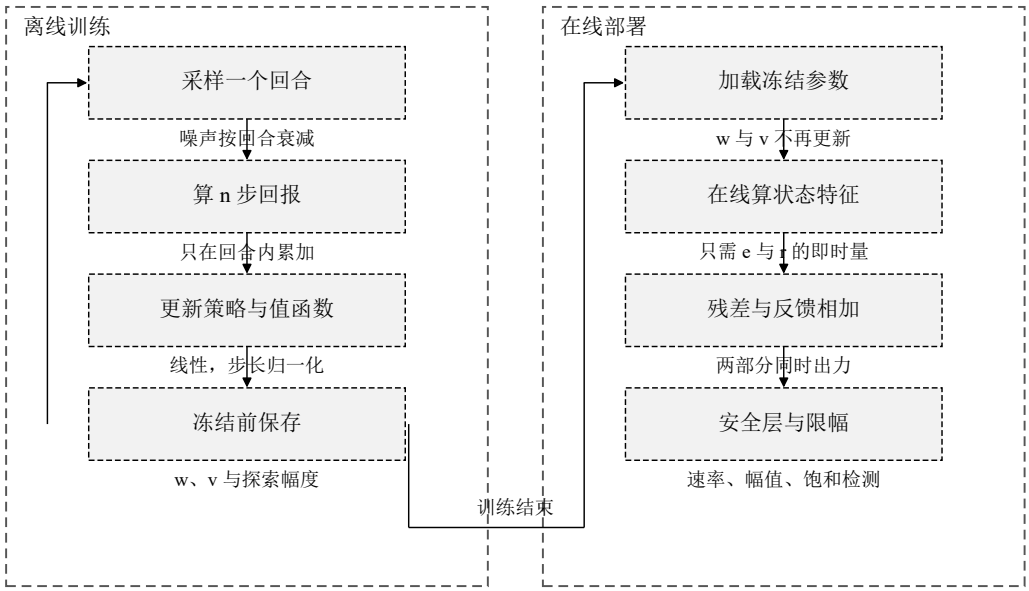


图 70-3 离线训练与在线部署

训练的进度不能看训练回合的回报，要看**确定性评估**。原因是训练时带着探索噪声，回报里有噪声的贡献，曲线会一直抖；评估时把噪声置零，才是策略的真实水平。本章每 10 个回合做一次评估，图 70-4 是这条曲线。

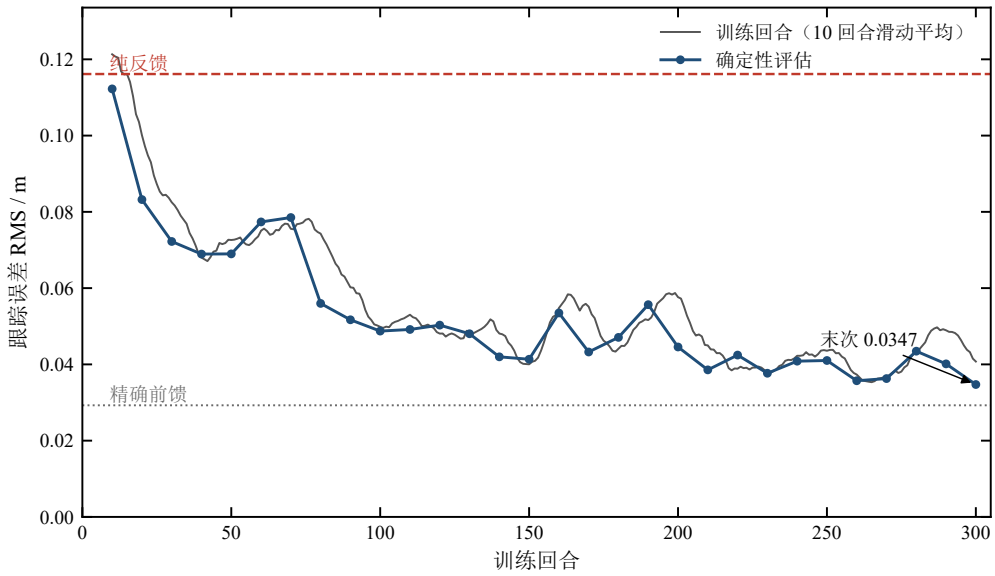


图 70-4 正式算例的训练过程

曲线有三个特征值得注意。第 50 回合评估 RMS 是 0.0690，已经比纯 PID 的 0.1161 好了 40%，说明前 50 个回合主要学的是“大方向”；第 100 到 250 回合在 0.041 到 0.049 之间反复，这一段学的是“细节”，进步慢且不单调；最后 50 个回合又降到 0.0347，这一段对应探索噪声衰减到下限、策略开始“收敛到最优点附近”。灰线是训练回合的滑动平均，它比评估曲线高 0.01 到 0.02，这个差就是探索噪声的代价。

验收要看的不是单个数字，而是四条同时满足：评估 RMS 低于基线的 60%、最大偏差低于基线的 50%、控制量总变差不超过基线的 2 倍、残余量的 RMS 在 U_{RL} 的 40% 以内。本章的结果是 0.0347（基线的 29.9%）、0.0795

（基线的 40.3%）、2.3（基线的 1.35 倍）、0.1666（U_{RL} 的 27.8%），四条都过。第四条容易被忽略：如果残余量一直贴着上限，说明策略在用“顶到限幅”的方式换精度，这种策略在真机上会频繁触发保护。

四条阈值的来历值得交代一句，因为它们不是随手定的。第一条 60% 对应“比基线好 40%”，这是本章第一节给出的最低目标：低于这个数，残余学习的工程价值不足以抵消它带来的复杂度。第二条 50% 与第一条配套——只看 RMS 会被“大部分时间很好、偶尔一个大偏差”骗过去，最大偏差砍半才能说明峰值也被压住了。第三条 2 倍是执行器磨损的上限，本章实测 1.35 倍，留了余量；如果某个策略 RMS 很低但总变差到 4 倍，宁可不用。第四条 40% 是“策略没有靠饱和换精度”的体检指标，它与第三条是一对：总变差管平滑，残余幅值管余量。四条一起看，才能排除“用抖动换精度”“用饱和换精度”这两种假进步。

表 70-1 是本章的全部参数，后面各节引用时不再重复。

参数	符号	取值	作用
采样周期	T_s	0.01 s	与对象仿真一致
回合长度	N	400 拍	一个轨迹周期
纯延迟	—	5 拍	环境的一部分
反馈 PID	k_p, k_i, k_d	2.00 / 0.05 / 0.20	基线控制器
残余幅值上限	U_{RL}	0.60	动作 1 对应的控制量
奖励尺度	E_{ref}	0.05	误差归一化
动作惩罚	λ	0.50	少动与少错的交换价
奖励缩放	R_{scale}	0.01	与学习率匹配
策略学习率	α_w	0.010	主旋钮
值函数学习率	α_v	0.050	通常取 α_w 的 5 倍
折扣因子	γ	0.95	等效视野 20 拍
自举步数	n	20	偏差与方差的折中
探索噪声	$\sigma_0 \to \sigma_{\min}$	0.30 → 0.05	每回合乘 0.99
训练回合	N_{ep}	300	4.7 s
可训练参数	—	12	策略与值函数各 6 个

表 70-1 中唯一没有解释的是 $\alpha_v = 0.050$ 为什么取 α_w 的 5 倍。理由是值函数的学习目标本身噪声更大（它要拟合的是带折扣的回报），步长小了不起作用、步长大了会震荡，而策略的更新依赖值函数的准确性，所以值函数要学得更快一些。这个 5 倍不是理论结果，是在本章对象上试出来的；70.8 节会给出一条从 2 倍到 10 倍的调节区间。

第七步：冻结、部署与回退

训练完成后，权重被写进一个 6 元的数组，随控制器一起下发。部署时不做任何在线更新（继续学习的做法见 70.10 节），并且必须保留三个开关：**残余量使能**（一键退回纯 PID）、**残余量限幅**（U_{RL}）、**速率限制**（每拍残余量的变化不超过 0.05）。前两个是训练时就有的，第三个是部署时才加的，它把“策略输出跳变”这种仿真里不明显的风险挡住。

这一步还应该做一件常被省略的事：**记录部署后的状态分布**。训练的 400 拍里，状态落在哪个区域、有没有越界，是有记录的；部署后的真实状态如果长期落在训练时没见过的区域（比如偏差一直在 0.3 以上），说明这次的工况已经超出策略的能力范围，应当触发告警并退回纯 PID。本章 70.6 节的迁移实验就是为了给这条告警定阈值。

冻结之后、上线之前，还有一道容易被跳过的核验：把权重数组抄出来，重新代回仿真，跑一遍名义工况，确认 RMS 与训练结束时一致到小数点后三位。这一步能抓住两类错误——特征尺度在部署端被改动（比如把 E_{sc} 从 0.10 写成 0.15），以及权重数组的维度顺序被转置。两者都不会报错，只会让性能悄悄退化，而且退化得“看起来像正常波动”。

还有一条实践上的约定：权重文件必须与特征定义绑定，而不是只存 6 个数。本章的做法是把特征的名字、尺度、截断值一起写进同一个文件，加载时逐项核对。控制工程里出现过很多次“换了一版程序，特征顺序变了，权重还是旧的”的事故，后果是策略输出完全错乱，而日志里一切正常。

70.4 连续情形

连续时间的最优控制写成什么

把采样周期推向零，MDP 就变成一个连续时间的最优控制问题：状态 $\boldsymbol{x}(t)$ 满足 $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u})$，要最小化

$$J = \int_0^{T} \ell(\boldsymbol{x}, \boldsymbol{u})\,\mathrm{d}t + \phi(\boldsymbol{x}(T)) \tag{70-15}$$

最优代价函数 $V(\boldsymbol{x})$ 满足 Hamilton-Jacobi-Bellman 方程

$$0 = \min_{\boldsymbol{u}} \left\{ \ell(\boldsymbol{x}, \boldsymbol{u}) + \nabla_{\boldsymbol{x}} V \cdot \boldsymbol{f}(\boldsymbol{x}, \boldsymbol{u}) \right\} \tag{70-16}$$

这条方程就是连续时间版本的“贝尔曼最优方程”。RL 里所有基于值函数的方法——包括本章的 Actor-Critic——本质上都在用采样代替“解这条偏微分方程”。对线性对象加二次代价，式 (70-16) 有解析解，就是 LQR；对本章这种 $K(y)$ 随位置变的非线性对象，式 (70-16) 没有解析解，数值解又受维数灾难限制。RL 的用处正在这里：**它不解方程，它直接在被控对象上试**。

连续域的奖励与离散域的求和

本章的奖励是每拍给的，连续域里对应的是代价密度 $\ell = e^2/E_{\mathrm{ref}}^2 + \lambda u_{\mathrm{RL}}^2$ 在时间上的积分。两者只在 T_s 固定、且代价在拍内近似常值时等价：

$$\int_0^{T} \ell\,\mathrm{d}t \approx T_s \sum_{k=0}^{N-1} \ell_k \tag{70-17}$$

这个近似带来一个常被忽略的后果：**奖励的整体量级与 T_s 成正比**。如果把采样周期从 10 ms 改成 5 ms，同样的轨迹会被切成 800 拍，每拍奖励不变，总和翻倍。学习率是作用在“每步更新”上的，总和翻倍意味着梯度尺度变了，原来调好的 0.010 就不再合适。工程上的做法是把奖励写成“每拍平均值”而不是“总和”，或者干脆把 R_{scale} 与 T_s 绑定，本章用后者的思路：$R_{\mathrm{scale}} = 0.01$ 是在 $T_s = 0.01$ s 下标定的。

折扣因子在连续域里也有对应物，而且这个换算比想象中更有用。若连续时间的目标是 $\int_0^{\infty} e^{-\rho t}\ell\,\mathrm{d}t$，按 T_s 采样后的逐拍折扣应当是 $\gamma = e^{-\rho T_s}$。反解本章的 $\gamma = 0.95$，得 $\rho = -\ln 0.95 / 0.01 = 5.13\ \mathrm{s}^{-1}$，等效时间常

数 $1/\rho = 0.195$ s——与前面说的“等效视野 20 拍”完全一致。这个换算的价值在于：**γ 不是一个无量纲的抽象参数，它对应一个物理时间常数**。

由此可以推出一条容易出错的操作规则：换采样周期时，要保持同一个物理视野，就得按 $\gamma' = e^{-\rho T_s'}$ 重算。T_s 从 10 ms 减到 5 ms，γ 要从 0.95 提到 0.9747，而不是继续沿用 0.95。沿用旧值的话，物理视野凭空缩短一半，策略会比设计意图更短视，表现为“稳态段补偿学得好、过渡段学不好”，而你看训练曲线只会觉得“收敛慢”。同理，如果把 T_s 加到 20 ms，γ 要降到 0.9025 左右，否则等效视野翻倍，值函数要覆盖更长的未来，线性逼近跟不住，优势估计变得又慢又吵。

采样把相位吃掉多少

连续域里“提前 5 拍”是“提前 50 ms”。这个提前量在频域里对应相位 $\omega \cdot 0.05$：在 1 Hz 是 18°，在 2 Hz 是 36°。这是延迟的代价，与是否用 RL 无关，第 67 章已经算过。

采样本身还额外引入半拍延迟。零阶保持器把一拍内的控制量冻结，等效于平均延迟 $T_s/2 = 5$ ms，在 1 Hz 对应 1.8°。这个量看起来可以忽略，但它与策略的“反应速度”直接相关：如果策略想学一个 5 Hz 的补偿动作，半拍延迟就吃掉 9°，再加上 5 拍的纯延迟就是 99°，补偿已经变成反相，越补越糟。所以**策略能学的频段有一个硬上限，由延迟与采样周期共同决定**，本章的有效补偿频段在 0.5 Hz 以下，与轨迹的基波（0.25 Hz）和三次谐波（0.75 Hz）大致吻合。

这三笔相位账可以合并成一条工程判据：把“纯延迟 + 半拍 + 策略自身的响应”加起来，落在要补偿的频段上不能超过 90°，否则补偿变成反相。本章的三次谐波在 0.75 Hz，纯延迟贡献 13.5°、半拍贡献 1.4°，余量还很宽；延迟加到 8 拍后纯延迟贡献变成 21.6°，仍然不到 90°，所以**单纯从相位看，8 拍延迟不该崩得那么惨**。真正的问题不是相位，而是策略学到的补偿是按 5 拍排布的，它内部的“提前量”已经写死。这个区分很重要：相位预算告诉你“能不能补”，训练分布告诉你“学到的补偿对不对”，两者是不同层面的约束，70.6 节讲的失效属于后者。

连续动作怎么参数化

连续域里策略是一个从状态到动作分布的映射。本章用高斯策略，均值线性、方差固定：

$$\pi(\boldsymbol{u} \mid \boldsymbol{x}) = \mathcal{N}\big(U_{\mathrm{RL}}\, w^{\mathrm{T}}\varphi(\boldsymbol{x}),\ (U_{\mathrm{RL}}\sigma)^2\big) \tag{70-18}$$

这个参数化在连续域里有明确的物理意义：$w^{\mathrm{T}}\varphi$ 是一个“状态反馈增益”，量纲是 1（因为 φ 已经无量纲化），乘上 U_{RL} 之后与控制量同量纲；σ 是相对噪声，0.30 意味着“在均值上下浮动 30% 的满量程”，即 ± 0.18 的控制量。这个幅值与精确前馈的 RMS 0.1166 同量级，说明探索的力度足以覆盖要学的形状，又不至于把系统推出线性区。

有人会问：为什么不把 σ 也做成状态的函数（即“状态相关的探索”）？原因是本章的误差量级在整个回合内变化不大（0.03 到 0.20），固定噪声已经够用；如果轨迹里有大幅值的机动段，就需要让 σ 随误差增大，否则机动段的探索不足、平稳段又过度扰动。

连续域的“探索”与执行器带宽

连续时间下，σ 决定的噪声是一个带宽为 $1/(2T_s) = 50$ Hz 的白噪声序列。这个带宽远高于对象的带宽（闭环约 2 Hz），所以噪声的大部分能量被对象滤掉，只有低频分量真正激励系统。这解释了一个现象：把 σ_0 从 0.30 提到 0.60，误差只从 0.0471 变到 0.0879，而不是翻四倍——噪声的高频部分没有进入误差。

但如果采样周期变大（比如 20 ms），同样的 σ 对应的噪声带宽降到 25 Hz，低频能量增加，对系统的激励变强，学出来的策略会更“抖”。所以 **σ 的合适取值与 T_s 是耦合的**，换采样周期必须重调探索噪声。本章的所有实

验都在 $T_s = 0.01$ s 下完成，这个前提在 70.10 节讲迁移时还要再提一次。

这也带来一个只在真机训练时才会遇到的约束：**训练用的探索噪声不能原样送到执行器**。50 Hz 带宽、± 0.18 幅值的随机指令，对伺服阀来说就是每小时几十万次换向，任何机械都受不了。所以真机在线训练必须给噪声加低通或速率限制，而这又会改变噪声的频谱、进而改变探索的有效性——这是“仿真训练 + 冻结部署”比“真机在线训练”省事的一个具体原因，省的不只是安全，还有硬件寿命。本章的仿真不受这个约束，噪声可以直接加在动作上；如果你要照搬到真机，这一层必须补。

连续前馈与学到的策略差在哪

模型已知时，连续域的最优前馈是

$$u^*(t) = \frac{r''(t) + 2r'(t)}{K(r(t))} \tag{70-19}$$

它的形状由两项组成：r'' 提供“加速所需的力”，$2r'$ 补偿阻尼。本章学到的残余策略是状态反馈，不是时间函数，但它在闭环里产生的控制量形状与 u^* 高度相似——因为闭环把“位置与速度”映射成了“时间”，沿着这条轨迹，r 与 $\dot r$ 的组合恰好复现了 $u^*(t)$ 的轮廓。

两者的差别在于**误差修正项**。u^* 是开环的，它不管实际位置偏了多少；学到的策略是闭环的，误差一大它就多补一点。这就是为什么学到的策略（0.0347）能超过精确前馈（0.0293 是在无扰动、无噪声的理想开环下的结果；一旦有初值偏差或负载，开环前馈会退化，而闭环残余策略不会）。第 69 章的 ILC 也是这个道理，它学的是闭环的逆，不是对象的逆。

连续视角给出的三个约束

把上面的分析收拢，连续视角给出三条实现约束。第一，**策略的补偿频段必须低于 $1/(4\tau_d)$**，τ_d 是总延迟，本章 50 ms 对应 5 Hz 的理论上限，实际取 0.5 Hz 以下才稳。第二，**探索噪声的幅值要与“要学的形状”同量级**，太小探索不到、太大会把系统推出工作区。第三，**奖励的量级要随采样周期标定**，换 T_s 必须重调 R_{scale} 或学习率。这三条在后面的离散实现里都会再出现一次，只是换了一种说法。

还有一层对照值得说清楚：式 (70-16) 的 HJB 方程与本章的 Actor-Critic，解的是同一个问题，但走的是两条完全不同的路。HJB 是“从状态空间出发”，要求在每个状态上都知道该往哪走，因此需要 ∇V，也就是一个定义在整个状态空间上的函数；当状态是 6 维、对象有非线性、延迟还有 5 拍时，这个偏微分方程既写不出解析解，数值解也要网格化，而网格化的代价就是维数灾难。Actor-Critic 是“从轨迹出发”，它不求解 V，只是在走出来的那条轨迹上估一个 V 的值，用这个估值算优势。**HJB 换的是空间上的完备性，RL 换的是时间上的样本量**，这是两种方法最本质的分野。

这个分野也解释了为什么本章不把 n 设得很大。n 越大，回报越接近“整回合”，偏差越小、方差越大；n 越小，越依赖值函数的估计，偏差越大、方差越小。取 $n = 20$ 相当于“用 20 拍的真实回报，加上 20 拍之后的值函数估计”，与 $\gamma = 0.95$ 给出的 20 拍等效视野正好对应——这两个参数在概念上是一件事，调其中一个，另一个的合理性也要重新看一遍。70.7 节的扫描里，$n = 20$ 与 $\gamma = 0.95$ 都是最优点，这不是巧合。

最后要提醒一个容易忽略的相位事实：20 拍的视野里，有 5 拍是纯延迟，也就是说策略能“看到”的有效未来只有 15 拍，约 0.15 s。这个数字与对象的时间常数（$1/2$ s）相比偏短，意味着策略本质上是“短视”的：它能把一个周期内的形状学好，但学不到“跨周期”的东西。这也是它与 ILC 的根本差别——ILC 的记忆是跨周期的，RL 的视野是周期内的。要让 RL 具备跨周期能力，得把“上一回合同相位的误差”放进状态——那会把状态维数再抬高一倍，而且要求每个回合严格对齐，本章没有往这个方向走，因为第 69 章的 ILC 已经把这条信息用透了。

70.5 离散情形

把动作切成五档

现场有大量执行器本身就是分档的：三位电磁阀只有开、中、关，继电器只有吸合与释放，老式执行机构只有几个固定档位。这类场合不能输出任意实数，动作集必须是有限集合。本章取五档残余量

$$\mathcal{A} = \{-0.30,\ -0.15,\ 0,\ 0.15,\ 0.30\} \tag{70-20}$$

注意这五档是**绝对残余量**，不是增量。同一章 70.3 节讲过的理由在这里同样成立：增量形式要求策略自己累加，而分档动作的累加会产生漂移，学出来的策略会在两个相邻档之间反复跳。用绝对值时，$a = 0$ 就是“不加残余”，与纯 PID 完全等价，这个基准点很重要。

五档的间距是 0.15，而精确前馈的 RMS 是 0.1166，也就是说“一档”就接近整个前馈的均方根。这个量化误差听起来很大，但闭环会把它平均掉：残余量的大部分能量集中在低频，量化噪声是高频的，被对象滤掉之后对误差的贡献有限。本章的实测印证了这一点——五档的 RMS 是 0.1147，比纯 PID 的 0.1161 只好 1.2%，远不如连续动作的 0.0347。这个结果需要解释，后面两小节会给出两个不同的原因。

Q 学习在做什么

离散动作下最自然的方法是 Q 学习。本章不用表格，而用线性逼近，对每个动作维护一组权重 q_a：

$$Q(s,a) = q_a^{\mathrm{T}} \varphi(s), \qquad \delta_k = r_k + \gamma \max_{a'} Q(s_{k+1}, a') - Q(s_k, a_k) \tag{70-21}$$

$$q_{a_k} \leftarrow q_{a_k} + \alpha_q \, \delta_k \, \frac{\varphi_k}{\|\varphi_k\|^2 + 1} \tag{70-22}$$

探索用 ε-贪心，ε 从 0.30 线性衰减到 0.02。式 (70-21) 里的 $\max$ 是 Q 学习区别于 SARSA 的地方：它假设下一步会选最好的动作，而不是实际选的那个。这个“乐观”的假设让 Q 学习可以离线学（用旧数据也能更新），代价是当动作值被高估时会传播误差。本章五档、每档 6 个参数，总共 30 个参数，规模很小，高估的影响有限。

为什么不用真正的表格？因为 6 维状态即使每维只切 5 档，也有 $5^5 = 3125$ 个格子（常数项不切），乘 5 个动作是 15625 个 Q 值。而一个 300 回合的训练总共只有 $400 \times 300 = 12$ 万次更新，平均每个格子只被访问 7.7 次。Q 学习要收敛，每个格子至少要被访问几十次，7.7 次远远不够——这就是下面样本量实验要测的东西。线性逼近把 15625 个未知数压到 30 个，让有限的样本能覆盖所有参数，代价是失去了“每个格子独立记忆”的能力。

顺带说明一个实现上的取舍：式 (70-21) 的 $\max$ 让 Q 学习原则上可以用旧数据，但本章的实现里并没有经验回放，每一拍都就地更新、样本用完即弃。好处是内存占用是常数、代码短；代价是样本的相关性极强——相邻两拍的状态几乎一样，连续 20 次更新实际提供的独立信息不到一次。这正是它需要 1000 回合才见效的原因之一。若加一个容量几千的回放池、每拍随机抽一批更新，同样回合数下的有效样本量能提高一个数量级。本章没有这么做，因为回放会引入“旧策略数据”与“当前策略”的分布偏移，需要额外的重要性采样或目标网络来收拾，代码会从三十行涨到一百行，超出“把机制讲清楚”的范围。

档位加了，精度没动

表 70-2 的第一组是档位实验：三档、五档、九档，各跑 300 回合、3 个种子平均。图 70-5 把两组结果画在一起。

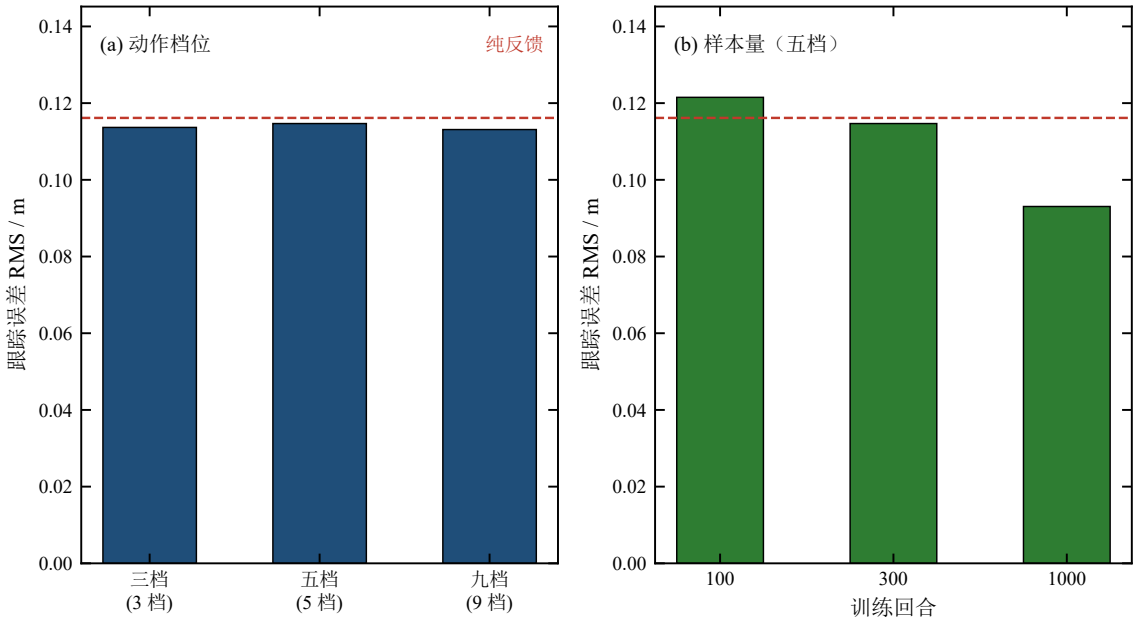


图 70-5 离散动作的档位与样本量

配置	档数	RMS	最大偏差	总变差	残余 RMS
三档	3	0.1137	0.2124	3.5	0.1466
五档	5	0.1147	0.2311	3.1	0.1413
九档	9	0.1131	0.2089	2.7	0.1106
100 回合（五档）	5	0.1215	0.2342	—	—
300 回合（五档）	5	0.1147	0.2311	—	—
1000 回合（五档）	5	0.0930	0.1810	—	—

表 70-2 的第一组数字说明：把档位从三档加到九档，RMS 从 0.1137 变到 0.1131，几乎没有变化。这个结果与“分辨率越高越准”的直觉相反，原因不是分辨率没用，而是**瓶颈不在分辨率上**。九档时最难的那一档（± 0.0375）已经接近前馈的零交叉区，量化误差早就小于跟踪误差本身的量级；真正限制精度的是“策略还没学会该在什么时候给哪一档”。

这一点可以从残余 RMS 看出来。三档的残余 RMS 是 0.1466，九档是 0.1106，说明九档确实用更小的力做了更多的事，但误差 RMS 没降——它把力气花在了错误的方向上。Q 学习在这种“每拍都要选、选了立刻有效果”的问题上，信用分配很困难：某一拍选了 0.15 导致的误差变化，要到几拍之后才显现，而中间还夹着 PID 的动作。70.8 节会把这条归入“症状：训练不收敛但残余量在动”。

样本量才是瓶颈

表 70-2 的第二组把档位固定为五档，只改训练回合数。100 回合是 0.1215（比纯 PID 还差），300 回合是 0.1147（略好），1000 回合是 0.0930（好 20%）。这条曲线是单调的，而且 300 到 1000 之间的改善最大，说明五档 Q 学习需要大约 40 万次更新才开始真正学到东西。

把它折算成“每个参数的样本数”更直观：1000 回合是 40 万次更新，30 个参数，平均每个参数 1.3 万次。而 100 回合只有 4 万次更新，平均每个参数 1300 次——差一个数量级，结果也差一个数量级。这个折算解释了为什么连

续动作的 Actor-Critic 只要 300 回合就能到 0.0347：它的 6 个参数每个被更新了 2 万次，而且每次更新都带着策略梯度给出的方向信息，不是 Q 学习那种“试出来才对”的信息。

离散动作的三种实现

工程上有三种做法，各有各的适用场合。第一种是**真表格**，把状态切成网格，每个格子存一个动作值。它可解释性最好（查表就是策略），但状态维数一高就爆，6 维 5 档已经是 3000 多个格子。第二种是**线性逼近**，也就是本章用的方式，参数少、泛化好，但需要读者接受“Q 值是两个向量的内积”这种抽象。第三种是**动作分档 + 连续策略**：保留连续策略的输出，在最后加一个量化器。第三种在工程上很实用，因为它可以用连续动作的样本效率，同时满足“执行器只认五档”的约束；它的代价是量化误差无法通过学习消除，因为策略根本不知道量化器的存在。

本章选第二种，是因为它最能暴露“离散动作的样本效率”这个核心问题。第三种做法在有量化器的场合更实用，70.9 节会给出一个简化的实现思路。

如果确实要用“连续策略 + 量化器”，量化器本身有两个细节值得设计。第一是**就近取档还是带滞环**：纯就近取档在相邻两档的交界处会因为噪声来回跳，产生每拍一次的抖振，本章五档实验里总变差从 1.7 涨到 3.1，一部分就来自这里；加一个滞环（只有越过交界一定比例才换档）能把抖振压下去，代价是引入一点死区，对小误差的修正变慢。第二是**档位是否等距**：本章用等距五档，是因为残余量的分布形状大致对称；如果残余量的典型值集中在 0.05 附近（比如对象本身已经接近线性），等距档位里就有四档是浪费的，改成“零点附近密、远离零点疏”的非等距档位，同样的档数能换到更细的分辨率。这两条都不改算法，只改“动作怎么落到执行器上”，属于实现层面的收益。

什么时候离散反而是对的

离散动作不是“连续动作的劣化版”，它在三类场合更合适。第一类是执行器真的分档，此时连续策略必须加量化器，不如直接学离散。第二类是**需要可验证性**：一张 3000 格的表可以被逐格检查、可以被形式化验证，一个连续策略不能。第三类是**动作空间本身很小**（比如只有开/关），此时离散动作的样本效率反而更高，因为每次探索都落在一个有意义的动作上，不像高斯策略那样把一半的样本花在“接近均值”的无效区域。

反过来，如果执行器是连续的、精度要求又高，就不要用离散动作。表 70-2 已经说明：五档在 1000 回合后只能到 0.0930，离连续动作的 0.0347 差 2.7 倍，而且这个差距不会因为再加档位而消失。

把这张表再横向读一遍，还能看出一个与“精度”无关的代价：控制量总变差。纯 PID 是 1.7，三档 3.5、五档 3.1、九档 2.7。也就是说，**分档动作让执行器比纯 PID 多动了一倍**，档位越多才越接近连续动作的 2.3。这个现象的机制很直白：分档动作是一串阶梯，每一级跳变都计进总变差，档距越大、跳变越少但每次越大，档距越小、跳变越密。对电磁阀这类本来就只能开关的执行器，这个代价是硬件决定的，不算算法损失；但如果执行器是伺服阀、却硬要用五档去驱动，那 3.1 的总变差就是白白送给机械磨损的。

这也解释了为什么工程上更常见的做法是“连续策略 + 量化器”，而不是直接学离散动作。连续策略在训练时看到的是平滑动作，学到的是“该给多少”；量化器只在下游把连续的指令就近取档，策略本身并不知道量化器的存在。这样做的代价是量化误差无法通过学习消除，收益是样本效率与连续动作同级（300 回合而不是 1000 回合）。要弥补量化误差，可以在训练时给动作加一点均匀抖动，让策略对“取整”这件事不敏感，这是信号处理里抖动（dithering）思路的直接移植。

70.6 稳定性与性能

三种“稳定”要分开谈

控制里的“稳定”通常指闭环极点落在左半平面，这是一个可以被证明的性质。RL 里的“稳定”至少混了三件事，混在一起谈会得出错误结论。第一是**闭环稳定**：给定一个固定的策略，闭环会不会发散。这件事由 PID 与对象决定，与 RL 无关，因为残余量被限在 ± 0.6，最坏情况相当于给 PID 加了一个有界的常值扰动。第二是**学习收敛**：参数更新会不会收敛到一个不动点。这件事由学习率、步长归一化、梯度裁剪决定，本章的配置在这三项上都做了处理。第三是**分布外安全**：部署时的工况与训练时不同，策略给出的动作会不会把系统推向坏的方向。这件事没有任何证书可拿，只能靠实验画边界。

本章的稳定性讨论只关心第三件。前两件在 70.7 节与 70.8 节里作为调参问题处理。

训练分布到底是什么

说“分布外”之前，先要能说清“分布内”是什么。训练的 300 个回合里，状态、动作、参数三个层面都有明确的取值范围。状态上，e 在 ± 0.25 内（纯 PID 的最大偏差是 0.1974，加上探索噪声后略大），$\dot{e}$ 在 ± 0.6 内，r 严格在 $[1.50, 2.72]$ 内。动作上，u_{RL} 在 ± 0.6 内，实际使用中 95% 的时间落在 ± 0.25 内（残余 RMS 0.1666）。参数上，增益固定、延迟固定 5 拍、无噪声、无负载、初值为零。

任何一条越界，都是分布外。本章 70.9 节的迁移实验把六种越界逐条测了一遍，结果是：增益变 $\pm 20\%$、加 0.01 噪声、加 0.05 常值负载、初值偏 0.02，这四种都只让 RMS 变化 0.001 到 0.008；唯独延迟从 5 拍变到 8 拍，RMS 从 0.0347 跳到 0.2267。这个对比说明“分布外”不是一个笼统的概念，它有明确的薄弱方向，而薄弱方向恰好是训练时被省略的那个状态变量。

特征截断把这件事写得更死。六维特征里每一维都截断在 ± 3，按各自的尺度换算回去，等于说：$|e| \geq 0.30$ 之后，策略看到的特征不再变化；$|\dot{e}| \geq 3.0$ 之后同理。其余四维在整个工作区间内都到不了 ± 3——误差积分的上限是 0.5（对应 2.5），参考位置最大 2.44，参考速度最大约 1.15，都够不着截断线。所以**真正会饱和的只有误差与误差导数两维，而延迟 8 拍时最大偏差是 0.5034，已经远超 0.30**。也就是说，系统崩掉的那一段，策略恰好是“看不见”的：误差从 0.30 涨到 0.50，特征一模一样，输出也一模一样，它没有能力做出更大或更小的反应。

这个发现对实现有直接指导：截断值不能按“训练时见过的最大状态”来定，而要留出余量。本章取 ± 3 是按“特征典型值的 3 倍”定的，而典型误差是 0.10，于是截断线落在 0.30——恰好是纯 PID 最大偏差（0.1974）的 1.5 倍，看着够用，实际不够。若把误差尺度改成 0.15，截断线就落在 0.45，覆盖延迟工况；代价是训练时误差特征的动态范围被压缩，学习率要相应调整。这是一处典型的“训练指标好、迁移指标差”的取舍。

两条失效路径

第一条路径是**延迟变化引起的相位错位**。策略学的是“在误差出现的第 5 拍之后补偿”，它把动作提前了 5 拍。延迟变成 8 拍之后，同一个动作早了 3 拍，补偿变成了“提前干预”，在误差还没出现时就推动了系统，结果是制造新的误差。新误差又被策略当作“该补偿的信号”，动作继续加大，直到贴上 ± 1.2 的限幅。实测饱和占比 28.25%，最大偏差 0.5034，RMS 0.2267——比纯 PID 差一倍。

第二条路径是**价值高估引起的策略漂移**。Q 学习里的 $\max$ 会系统性高估动作值，Actor-Critic 里的自举也会。高估本身不致命，致命的是它让策略倾向于“更激进的残余量”，在训练分布内这只表现为残余量偏大，出了分布就变成饱和。本章的做法是给优势加硬裁剪（± 5）、给梯度加范数裁剪（0.2），把单步更新的影响限住；这两条不能消除高估，只能让它不至于在几十拍内累积成灾难。

这两条路径有一个共同点：它们都不是“学得不够”，而是“学到了错的东西”。所以加大训练量、多跑回合都救不了——延迟 8 拍不是训练不足，而是训练分布里根本没有这个工况。这也是 70.9 节把改进方向放在“域随机化”而不是“多训几轮”上的原因。判别方法很直接：**在同一个工况下反复评估，如果结果一致地差（均值差、方差小），那是分布问题；如果忽好忽坏（方差大），那才是训练不足或探索不够**。本章延迟 8 拍属于前者——策略冻结之后评估是完全确定的，种子只影响训练过程，不影响“这个策略在 8 拍延迟下会崩”这个事实。

三种做法的实测对照

表 70-3 是三种做法在同一对象、同一条轨迹上的结果，各 5 个种子。PID 是基线；AC 是本章的 n 步 Actor-Critic；REINFORCE 用整回合回报加线性基线，学习率单独调到 0.2。

做法	均值	标准差	最小	最大	相对 **PID**
PID	0.1161	0.0000	0.1161	0.1161	0.0%
AC	0.0594	0.0373	0.0346	0.1332	48.9%
REINFORCE	0.0766	0.0144	0.0557	0.0979	34.0%

表 70-3 最有意思的地方不是均值，而是标准差。AC 的均值好（0.0594，降 48.9%），但标准差 0.0373，最差的一个种子跑到 0.1332，比纯 PID 还差 15%。REINFORCE 的均值差一些（0.0766），但标准差只有 0.0144，五个种子全部落在 0.056 到 0.098 之间。图 70-6 把这组分布画了出来。

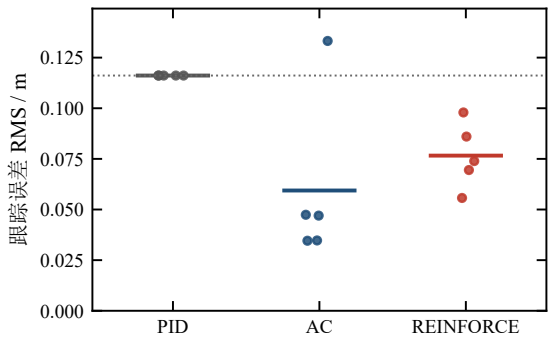


图 70-6 三种做法的分布

这张图说明一个工程上很重要的判断：**如果只能跑一次、又必须保证不退化，REINFORCE 这种“笨但稳”的方法反而更合适**。AC 的高方差来自自举——值函数的估计误差会通过 TD 误差直接进梯度，而 REINFORCE 用的是真实的整回合回报，没有这个误差源。本章主推 AC，是因为它在多数种子下明显更好，而且可以通过多种子挑选、早期停止、集成来压住尾部风险；如果现场没有这些手段，就该退到 REINFORCE。

学成之后长什么样

图 70-7 是正式算例的时域波形，三条曲线分别是参考与输出、两条误差、两条控制量。

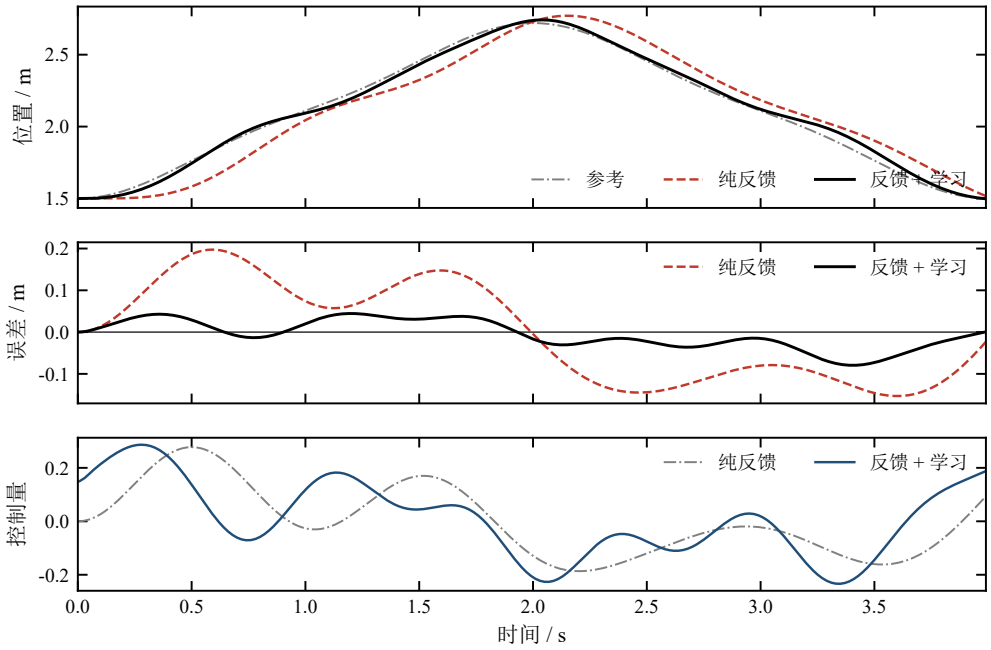


图 70-7 学成之后的跟踪波形

上面那张子图里，纯反馈（红虚线）在轨迹的上升段和下降段各有一次明显的滞后，最大偏差 0.1974；加上残余策略之后（黑实线），输出几乎贴在参考上，最大偏差 0.0795。中间那张子图把误差单独画出来：纯反馈的误差是一条“先正后负”的大弧线，峰值出现在速度最大的时候；学到的策略把这条弧线压成了两个小鼓包，形状更接近“跟踪残差”而不是“滞后”。下面那张子图是控制量：纯反馈的控制量在 0.2 到 0.35 之间缓慢变化，加上残余之后多了一个与轨迹同相的低频起伏，幅度约 0.15，形状与精确前馈 u^* 的轮廓一致。

控制量的总变差从 1.7 涨到 2.3，涨了 35%。这个涨幅是可以接受的：它换来了 70% 的误差降幅，而且残余量本身是平滑的（每拍变化在 0.01 量级），不是高频抖动。如果总变差涨到 5 以上，就该怀疑策略在“用抖动换精度”，那时要加大 λ 或加速率限制。

性能的天花板在哪

把本章的数字与前两章放在一起，可以看出一条清晰的阶梯。纯反馈 0.1161；精确前馈（模型已知、开环）0.0293；ILC（60 次重复）0.0224；本章的 RL 0.0347。RL 夹在“精确前馈”与“纯反馈”之间，比前者差 18.6%，比后者好 3.3 倍。

这个位置不是偶然的，它由三个限制共同决定。第一是**策略容量**：6 个线性参数能表达的控制律形状有限，它无法复现 u^* 里那些与 $K(r)$ 的倒数有关的细节。第二是**延迟未建模**：特征里没有历史动作，策略只能通过“误差的变化率”间接推断延迟的影响，这个推断在 5 拍时够用，在 8 拍时就错了。第三是**单回合样本**：300 个回合、12 万拍，对 6 个参数是充足的，但不足以让线性策略逼近一个本质非线性的最优解。

要突破这三条，对应的办法是：换非线性策略（第 68 章的思路）、把延迟写进状态（用历史动作的滑动窗）、加样本（或者用模型辅助的 RL）。本章不做这些，因为它们会把“机制清楚”这个教学目的破坏掉；70.9 节会把它们作为改进方向列出来，并说明各自要付的代价。

换一个对象，这个天花板的位置会整体移动，但三条限制的相对重要性顺序不会变。如果对象更接近线性（K 只变化 10% 而不是 50%），策略容量这条限制会明显放松，残余 RL 有可能追平精确前馈——因为一个线性反馈足以复现大部分前馈形状。如果延迟更大（比如 20 拍），“延迟未建模”会变成主要矛盾，此时再怎么加参数、加样本都没用，必须把历史动作写进状态。第三条只有在回合更长、样本更多时才松动。所以看到别人报告“RL 超过了模

型前馈”，先问三个问题：对象有多非线性、延迟有没有进状态、训练样本是多少。这三问能把大部分“惊人结果”还原成“特定条件下的正常结果”。

四条验收指标与两条监控指标

70.3 节给过四条验收指标，这里补两条部署期才用得上的监控指标。第一条是**状态越界率**：每一拍检查 $|e|$、$|\dot{e}|$、$|r - R_0|$ 是否落在训练时的范围外，越界率超过 1% 就该告警。第二条是**动作裁剪率**：$|a|$ 被截断到 ± 2 的比例，正常应在 0 附近，一旦持续大于 1%，说明策略在要求超出训练范围的动作。

这两条指标的好处是它们不依赖“知道正确答案”，只依赖训练时记录下来的范围。工程上把它们做成两个计数器，随控制器一起上电复位，是最省事的做法。本章的仿真里这两条都是 0，但 70.10 节的延迟 8 拍工况会让第二条跳到 30% 以上——与饱和占比 28.25% 基本一致，说明这两个量测的是同一件事。

这里要区分两层阈值，否则容易把告警和回退混为一谈。第一层是**技术告警**：越界率超过 1%、裁剪率超过 1%，就该记日志、通知人来看，但不改变控制行为。第二层是**自动回退**：70.10 节用的是 10% 与 30%，到了这个量级说明策略已经在实质上帮倒忙，必须降级或清零。两层之间留了一个数量级的缓冲，目的是让人有时间判断“这是真故障还是一次偶发扰动”，而不是让系统在两个状态之间反复切换。延迟 8 拍那个 28.25% 落在两层之间——它高于告警线、低于回退线，正好是一段“需要人来看一眼”的区间，这个设计不是巧合。

70.7 参数意义与作用

五个旋钮，各管一件事

本章可调的参数有五个：策略学习率 α_w、折扣因子 γ、探索噪声 σ、自举步数 n、动作惩罚 λ。剩下的 α_v 与 R_{scale} 属于“设好就不动”的数值参数，U_{RL} 与特征尺度属于“由任务决定”的结构参数。五个旋钮的分工可以用一句话概括：α_w 管“走多快”，γ 管“看多远”，σ 管“试多野”，n 管“信自己还是信回报”，λ 管“允许多用力”。

这五件事里，只有 λ 直接改变最优解，其余四个改变的是“能不能走到最优解”。这个区别在调参时很重要：如果结果是 0.05 而你想要 0.03，改前四个没用，只有改 λ（或改特征、改动作上限）才有用；如果结果是 0.15 而你想要 0.05，那多半是前四个没调好。

策略学习率：不是“慢一点”，而是“抖散”

表 70-4 与图 70-8 是学习率扫描，每个取值 3 个种子、200 回合。

学习率 α_w	均值	标准差	最小	最大
0.003	0.0794	0.0166	0.0560	0.0920
0.010	0.0471	0.0025	0.0446	0.0505
0.030	0.1428	0.1129	0.0606	0.3025
0.100	0.3092	0.0078	0.3037	0.3203

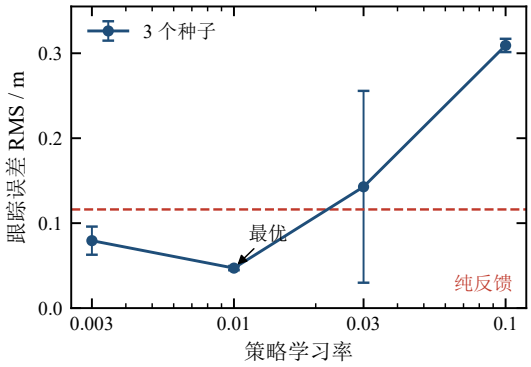


图 70-8 策略学习率扫描

表 70-4 的形状是本章最值得记住的一张表。从 0.003 到 0.010，均值从 0.0794 降到 0.0471，标准差从 0.0166 降到 0.0025——不仅更准，而且更稳。从 0.010 到 0.030，均值跳到 0.1428，标准差跳到 0.1129，最差的一个种子是 0.3025，几乎是纯 PID 的三倍。从 0.030 到 0.100，均值稳定在 0.3092，所有种子都坏——这时候已经不是“抖”，而是稳定地发散到某个坏的不动点。

这条“断崖”的成因是线性策略的外推。学习率大时，单步更新会把 w 推到一个“在训练分布内看似更好、在分布外完全错误”的位置；下一回合策略在新位置采样，状态分布随之偏移，更新的方向也跟着偏，几步之后就滑出了有效区。0.010 与 0.030 之间不是渐进变差，而是“稳”与“不稳”的分界，所以调这个参数的正确做法是在 0.010 附近做对数细分（0.007、0.010、0.015），而不是线性地试。

折扣因子：视野与噪声的交换

表 70-5 是折扣因子扫描，同样 3 个种子、200 回合。

折扣因子 γ	均值	标准差	等效视野
0.900	0.0654	0.0157	10 拍
0.950	0.0471	0.0025	20 拍
0.990	0.0606	0.0210	100 拍
0.999	0.1822	0.0371	1000 拍

γ 的两端都差，中间好，原因与 n 步回报类似但方向相反。γ 小时视野短，策略只顾眼前，学不出“提前 5 拍补偿”这种行为，因为补偿的收益要在 5 拍之后才显现，被折扣压得看不见；γ 大时视野长，回报里混进了太多未来的随机性，方差急剧上升，0.999 时等效视野 1000 拍已经超过回合长度，等于在估计一个几乎无折扣的总回报，梯度方向被噪声主导。

0.95 对应 20 拍，恰好覆盖“延迟 5 拍 + 对象主时间常数 0.5 s 的一部分”。这个匹配关系可以作为一个起点：**先算出延迟与主时间常数，取 γ 使等效视野约为“延迟 + 半个时间常数”**，再在其上下各试一档。

还有一个与回合边界有关的细节：本章的回合长 400 拍，$\gamma = 0.95$ 时第 400 拍的折扣权重是 $0.95^{399} \approx 1.3 \times 10^{-9}$，早已可以忽略；所以“回合结束时截断”与“无限时域折扣”在本章几乎等价。但如果换成 $\gamma = 0.999$、回合仍是 400 拍，末端权重还有 0.67，这时候“是否跨回合自举”就变成一个必须明确的设计决定。本章的做法是不跨回合：回合末的状态不再自举，直接用剩余回报。这样每个回合是一个完整的、可独立求值的样本，训练与评估的口径也一致。

探索噪声：太小走不动，太大待不住

表 70-6 是探索噪声初值的扫描，衰减率固定 0.99、下限固定 0.05。

初值 σ_0	均值	标准差	末段噪声
0.05	0.0894	0.0238	0.05
0.15	0.0707	0.0248	0.05
0.30	0.0471	0.0025	0.05
0.60	0.0879	0.0156	0.05

$\sigma_0 = 0.05$ 时均值 0.0894，比 0.30 时差近一倍。这个结果初看反直觉：噪声小应该更精确才对。原因是探索不足——策略只能沿着初始权重（全零）附近的极小区域试探，学不到“该往哪个方向偏”，收敛到一个“不怎么动”的平庸解。$\sigma_0 = 0.60$ 时又变差，因为大幅值噪声会把系统推离线性区，训练回合里出现大量饱和样本，值函数被这些极端样本带偏。

0.30 对应的实际噪声幅值是 $0.30 \times 0.60 = 0.18$ 的控制量，与精确前馈的 RMS 0.1166 同量级。这条**“噪声幅值 ≈ 要学的形状的幅值”**的经验规则，在别的任务上也适用：探索的力度应该与“正确答案的典型大小”相当，太大是浪费样本，太小是探索不到。

末段的噪声下限（0.05）也不能省。把下限设成 0，训练后期策略完全确定，一旦早期收敛到次优点就再也出不来；留下 0.05 的残余噪声，相当于给策略保留一点“持续试探”的能力，代价是评估时要去掉它。

衰减率 0.99 决定了“什么时候退到下限”。从 0.30 乘到 0.05 需要 179 次衰减，也就是大约第 180 个回合触底，剩下 120 个回合都在下限上跑。这个比例是刻意的：前 60% 的回合用来探索，后 40% 用来收敛。如果衰减太快（0.97），第 60 回合就到底，策略还没找到方向就停止了试探，剩下的回合只是在原地细化；如果太慢（0.999），300 回合结束噪声还有 0.22，评估时噪声一撤，策略会暴露“从没在安静环境里工作过”的问题。这条比例与表 70-4 的 200 回合扫描也对得上——扫描用 200 回合，正好落在“探索刚结束、还没充分收敛”的位置，所以扫描值（0.0471）比正式算例的 300 回合（0.0347）略差，这个差异是回合数造成的，不是参数本身造成的。

自举步数：偏差与方差的交换点

图 70-9 是自举步数扫描，取值 1、5、10、20、50、200。

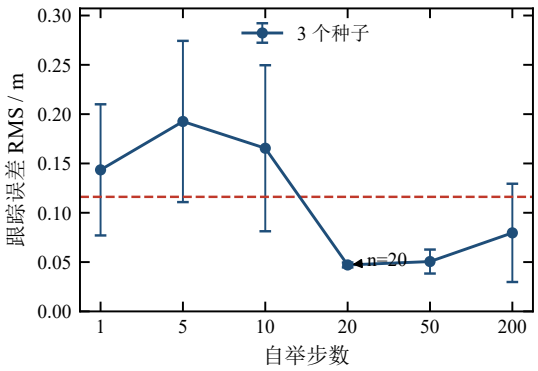


图 70-9 自举步数扫描

这条曲线是本章最“不单调”的一条：$n = 1$ 时 0.1435，$n = 5$ 时 0.1926（更差），$n = 10$ 时 0.1654，$n = 20$ 时 0.0471（突降），$n = 50$ 时 0.0505，$n = 200$ 时 0.0796。两头的解释很清楚：$n = 1$ 是纯自举，值函数的误差直

接进梯度，偏差大；$n = 200$ 接近整回合回报，方差大。中间的 0.0471 来自“偏差与方差刚好平衡”。

$n = 5$ 比 $n = 1$ 更差这一点需要单独解释。n 小的时候，回报里真实奖励只占 n 拍，其余全靠值函数补齐；而值函数在训练早期几乎不准，$n = 5$ 相当于“用一个半准不准的估计去替换 5 拍真实奖励”，比 $n = 1$ 的“替换 1 拍”引入了更多错误。等到 $n = 20$，真实奖励占了 20 拍（0.2 s），值函数只负责尾部，误差被摊薄了。所以 n 的选择不是“越大越好”或“越小越好”，而是“让真实奖励覆盖一个完整的动态过程”——本章的动态过程是延迟 5 拍加半个时间常数，20 拍刚好。

动作惩罚：唯一改变最优解的旋钮

表 70-7 是 λ 扫描，200 回合、单种子。

λ	RMS	最大偏差	总变差	残余 RMS	每拍变化 RMS
0.0	0.0743	0.1584	8.0	0.2383	0.01252
0.1	0.0636	0.1438	7.1	0.2319	0.01232
0.5	0.0446	0.0823	5.4	0.2079	0.00896
2.0	0.0420	0.0928	5.0	0.2011	0.00922
10.0	0.0584	0.1064	1.6	0.1377	0.00815

表 70-7 有两个反直觉的地方。第一，$\lambda = 0$ 时 RMS 是 0.0743，比 $\lambda = 0.5$ 的 0.0446 还差——不加惩罚反而更不准。原因是 $\lambda = 0$ 时策略倾向于用大动作追每一个误差，学出来的东西在闭环里引起过冲，误差反而变大。第二，$\lambda = 10$ 时 RMS 回升到 0.0584，但总变差降到 1.6，残余 RMS 降到 0.1377——策略变得很“懒”，几乎不动，误差自然回到接近纯 PID 的水平。

这条曲线说明 λ 有一个“甜点”，在本章是 0.5 到 2 之间。它的物理含义是“允许多用力”的价格：价格太低，策略乱用力；价格太高，策略不敢用力。工程上可以这样定初值：先取 $\lambda = 0$ 跑一次，看残余量的 RMS，再取 λ 使“动作惩罚项”与“误差惩罚项”的贡献相当，即 $\lambda\,\mathbb{E}[a^2] \approx \mathbb{E}[(e/E_{\text{ref}})^2]$。

这里有一个容易忽略的耦合：λ 罚的是归一化动作 a，而 a 对应的物理量是 $U_{\text{RL}}a$，所以同一句“动作惩罚项等于 $\lambda(u_{\text{RL}}/U_{\text{RL}})^2$”意味着——**改动 U_{RL} 等价于按平方改动 λ 的含义**。把残余上限从 0.60 减到 0.30，同样的 $\lambda = 0.5$ 对物理用力的惩罚就重了 4 倍，最优残余量会明显变小。所以这两个参数不能分开调：要么固定 U_{RL} 只调 λ，要么改了上限之后把 λ 按平方缩放回去。本章的所有 λ 数据都是在 $U_{\text{RL}} = 0.60$ 下测的，换上限就不能直接照搬。

值函数学习率与两个数值参数

α_v 没有单独扫描，因为它与 α_w 强耦合。经验是取 α_w 的 2 到 10 倍：太小了值函数跟不上策略的变化（优势估计滞后），太大了值函数自己在震荡（优势估计变成噪声）。本章取 5 倍，在 70.3 节已经说明。

$R_{\text{scale}} = 0.01$ 是纯数值参数，只影响学习率刻度。它的作用是让单拍奖励落在 10^{-2} 量级，与 $\alpha_w = 0.010$ 匹配。如果把它改成 1.0，等价于把 α_w 放大 100 倍，会直接跳到表 70-4 的“发散”那一行。$U_{\text{RL}} = 0.60$ 是结构参数，改变它会同时改变动作空间和奖励的解读，不能当旋钮用。

这个比例可以从两条时间尺度理解。策略的变化速度由 α_w 与梯度范数决定，值函数要跟得上的前提是“它比策略快若干倍”，否则优势估计永远滞后于策略；反过来，值函数收敛太快也有问题——它会把当前策略的价值迅速“锁死”，优势趋零、梯度消失，表现为曲线早早变平但误差还很高。两端都不好，中间那段就是 2 到 10 倍。判别方

法很简单：把 α_v 单独放大 5 倍，如果前期更快、最后停在一个更差的水平，说明值函数抢跑了；单独缩小 5 倍，如果前 50 回合几乎不动，说明它跟不上。

五个旋钮的灵敏度排序

按“改一点点会怎样”排序，本章的经验是：α_w 最敏感（0.010 与 0.030 之间是断崖），σ_0 次之（0.15 到 0.30 差 33%），n 再次（20 到 50 差 7%，但 20 以下差得多），γ 再次（0.95 到 0.99 差 29%），λ 最不敏感（0.5 到 2 差 6%）。

这个排序决定了调参的顺序：先固定 λ、γ，调 α_w 找到“稳且不慢”的点，再调 σ_0 和 n 抠精度，最后微调 λ 换平滑度。反过来做——先调 λ 再调 α_w——会把两个效应混在一起，永远调不到点子上。

70.8 调参方法与经验

一条可以照着走的流水线

调参的顺序不是随意的，它由“哪个参数改变问题的定义、哪个只改变求解过程”决定。本章推荐的流水线是七步：定结构（特征、动作上限、奖励形式）、定数值（R_{scale}、α_v/α_w）、粗调 α_w、调 σ_0、调 n、微调 γ、最后调 λ。前两步一次定好不再动，后五步按灵敏度从高到低走。

这条流水线有一个前提：每一步只用单种子跑 100 到 200 回合，看到明显趋势就进下一步，最后用 3 到 5 个种子定稿。本章的所有扫描表都是这么得到的——先单种子看形状，再多种子看均值和标准差。单种子粗筛能省 3 到 5 倍的时间，代价是可能被某个“运气好”的种子骗过去，所以定稿时必须换成多种子。

时间该怎么分配，也有一条经验：**七成花在环境与特征上，三成才花在五个旋钮上**。理由是这两类问题的返工成本完全不同。特征写错了（少一维、尺度差一个量级、截断太紧），五个旋钮怎么调都补不回来，最好的结果也只是“看起来收敛、迁移就崩”，而这类错误在短回合的扫描里根本暴露不出来；超参没调好，最多是多花几个小时，方向不会错。本章在定稿前改过两次特征：第一次是去掉“上一拍残余”，因为最小二乘解会爆炸；第二次是把参考速度加进去，因为不加的话策略学不到前馈形状。两次改动都比后来所有超参扫描加起来更影响结果。

学习曲线的三段

图 70-4 那条曲线可以切成三段，每一段对应不同的学习内容，也对应不同的调参动作。

第一段是前 50 回合，误差从 0.1161 降到 0.0690，降幅 40%。这一段学的是“大方向”：策略发现“轨迹上升时要多推、下降时要少推”，这个发现只需要很少的样本，因为它的信号最强。如果这一段降不下来（50 回合还在 0.11 以上），问题多半在 α_w 太小或 σ_0 太小，策略根本没在动。

第二段是 50 到 250 回合，误差在 0.041 到 0.049 之间来回。这一段学的是“细节”：补偿的时机、幅值的微调、与 PID 的配合。它的信号弱（改进只有 0.005 量级），被探索噪声盖住，所以曲线抖。这一段停滞是正常的，不要在 100 回合就断定“学不动了”。

第三段是 250 到 300 回合，误差从 0.041 降到 0.0347。这一段对应探索噪声衰减到下限（0.05）、策略开始收敛到最优点附近。如果最后 50 回合没有这个下降，说明噪声下限设得太高（比如 0.10），策略一直在“带着噪声工作”，评估时噪声去掉反而暴露了它没有真正收敛。

判断“真收敛”还是“卡住了”，除了看曲线平不平，还要看两个伴随量。一是残余量的 RMS 是否稳定在 U_{RL} 的 30% 附近（本章 0.1666，即 27.8%）；如果它一直在缓慢上涨，说明策略在用越来越大的力换越来越小的收益，属于“用饱和换精度”的前兆。二是最后几十个回合的评估值极差有多大。本章最后 50 回合的评估值在 0.041 到 0.035

之间，极差 0.006；如果极差超过 0.02，那多半不是收敛，而是“在两个局部解之间来回跳”，此时加回合数也没用，应该减小 α_w 或把噪声下限再降一档。

症状与处置

表 70-8 把常见的六种症状与对应的处置列在一起。这张表的价值在于：它把“看起来都像不收敛”的几种情况区分开了，而它们的处理方法完全不同。

症状	可能原因	处置
前 20 回合就贴限幅、误差不降	α_w 过大或 U_{RL} 过大	学习率除以 3，或残余上限减半
误差一直在 0.10 附近不动	α_w 太小、σ_0 太小	学习率乘 3，噪声初值提到 0.30
均值在降、方差很大	自举步数太小、值函数学得太慢	n 提到 20，α_v 提到 $5\alpha_w$
训练好、评估差	噪声下限太高	下限降到 0.02 到 0.05
残余量在动、精度不动	动作分辨率不足或信用分配错位	换连续动作，或减小 n
换了工况就崩	训练分布未覆盖该工况	域随机化，或加安全层与回退

表 70-8 的第一行与第二行看起来矛盾——一个说学习率太大、一个说太小——但它们对应的现象完全不同：前者是“动得太猛”，表现为饱和；后者是“根本不动”，表现为误差曲线是一条水平线。分辨的办法是看残余量的 RMS：太猛时残余 RMS 接近 U_{RL}，不动时残余 RMS 接近 0。这两个数在评估里都有，不需要额外记录。

第五行“残余量在动、精度不动”正是 70.5 节五档 Q 学习的情况。它的诊断价值在于：残余量在动说明策略确实在学，精度不动说明学的东西没用。这时候继续加样本可能有用（表 70-2 里 1000 回合确实降到了 0.0930），但更根本的办法是换连续动作，因为瓶颈是“分档动作无法表达正确的时序结构”。

三个不要

第一个不要：**不要用训练回报当验收指标**。训练回报里混着探索噪声的贡献，它会随噪声衰减而自然上升，看起来像“学得很好”，实际策略可能没变。本章的做法是每 10 回合做一次确定性评估，只看评估曲线。

第二个不要：**不要在多处同时改参数**。学习率、噪声、自举步数三者互相影响，同时改两个，出了问题分不清是谁的责任。本章的所有扫描都是“固定其余、只改一个”。

第三个不要：**不要为了最后 0.002 反复调**。从 0.0471 到 0.0347 的这段改进，主要来自“训练到 300 回合”而不是“参数调得更好”。继续在这上面花时间，收益远小于把精力放在 70.10 节的迁移与安全上——部署时那 0.01 的退化，比训练时这 0.002 的差别重要得多。

训练预算怎么估

本章的一次 300 回合训练耗时 4.7 s，单回合 15.5 ms。这两个数随机器负载有 5% 左右的浮动，本章不同批次实测落在 4.6 到 4.8 s 之间，所以只把它当量级看。这个速度看起来很快，但它是在纯 Python 循环里跑出来的；换成有 10 个状态、200 拍的对象，单回合会到 30 ms 量级，300 回合就是 9 s。如果要做 20 组超参扫描、每组 3 个种子，总时间就是 9 分钟——这个量级完全可以接受。

真正需要考虑预算的是“域随机化”：如果要把增益、延迟、负载三个维度各取 3 档组合成 27 种工况，每种跑 300 回合，总时间就是 4 小时。这时候要么缩小工况集（本章 70.10 节用的是 7 种），要么用并行（进程级并行最省事，因为每个回合是独立的）。

预算估完之后，还要能判断“这些钱花得值不值”。一个简单的判据是看评估曲线最后 30% 的回合还有没有下降趋势：如果最后的评估值仍在单调下降，加回合大概率还有收益；如果已经在同一个区间里横了三四十个回合，再加 100 回合也只是重复采样，收益远小于把时间花在特征上。本章 300 回合的选择就来自这条判据——250 到 300 这一段的下降有 0.006，属于“刚到拐点”，再加 100 回合大概只剩 0.002 的收益，不划算。

另一条与预算有关的经验是：**扫描阶段与定稿阶段的回合数可以不同**。本章的扫描实验统一用 200 回合（`SWEEP_EP`），只有正式算例用 300。理由是扫描关心的是“趋势”，200 与 300 回合的趋势一致，而 200 回合省下三分之一时间；定稿关心的是“绝对数字”，必须用完整回合数。这个做法有一个前提：扫描得出的排序必须与定稿一致。本章在正式写进正文之前，抽了 α_w 与 σ_0 两组做对照，200 与 300 回合的排序相同，才放心用 200 回合跑完所有扫描。

一条容易被忽略的经验

最后说一条不容易从公式里看出来的经验：**训练回合数不应低于“参数个数 × 10”**。本章 12 个参数，下限是 120 回合，正式算例跑 300 回合，是下限的 2.5 倍；如果只跑 50 回合，参数还没被充分更新，评估曲线会停在 0.07 附近，看起来像“算法不行”，实际是“样本不够”。这条经验在 70.5 节的 Q 学习上得到验证：30 个参数、下限 300 回合，300 回合正好压在线上，结果确实停在 0.1147；跑到 1000 回合（下限的 3.3 倍）才降到 0.0930。

70.9 优化与改进

四条改进路线

前八节讲的是一条“标准路线”：名义对象上训练、名义对象上评估。它在名义工况下把 RMS 从 0.1161 压到 0.0347，但真实系统不会停在名义点。本节按“问题出现的位置”排四条改进路线：训练阶段没覆盖到的失配（域随机化）、部署后发现的新失配（继续学习）、策略本身太脆（安全层）、以及奖励与动作空间的设计缺陷（整形与换档）。

这四条路线不是并列的选项，而是有优先级的。先做安全层，因为它成本最低、兜底效果最好；再做域随机化，因为它一次性解决一类问题；继续学习放在最后，因为它的收益最不确定——下面会看到，它在某些工况下甚至不如从零重训。

迁移：七种工况下的表现

先把问题量化。图 70-10 把同一个策略放到七种偏离名义的工况上，与纯 PID 并排比较。

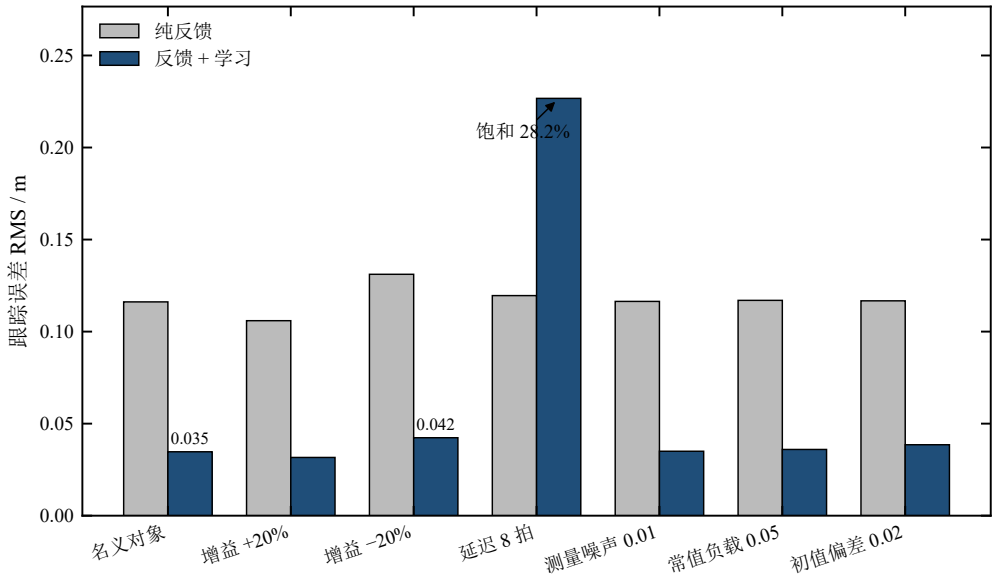


图 70-10 七种工况下的迁移表现

图 70-10 迁移性能对比。每组左柱为纯 PID，右柱为训练好的残余策略；横轴依次为名义、增益 +20%、增益 −20%、延迟 8 拍、测量噪声 0.01、常值负载 0.05、初值偏移 0.02。延迟 8 拍一组的柱顶标注了限幅饱和占比。

工况	纯 PID RMS	残余策略 RMS	相对名义	是否优于 PID
名义	0.1161	0.0347	—	是
增益 +20%	0.1059	0.0316	−8.9%	是
增益 −20%	0.1311	0.0423	+21.9%	是
延迟 8 拍	0.1196	0.2267	+553%	否
噪声 0.01	0.1164	0.0350	+0.9%	是
负载 0.05	0.1170	0.0360	+3.7%	是
初值 0.02	0.1167	0.0385	+11.0%	是

表 70-9 里有三件事值得展开。第一，增益变化是最“友好”的失配：$\pm 20\%$ 的静态增益误差落在 0.0316 到 0.0423 之间，策略仍然是 PID 的三倍好。原因很直接——策略在训练中已经见过 $K(y)$ 随位置变化（工作区间内 13 到 20，本身就有 54% 的跨度），增益整体缩放只是把这个跨度平移，特征里又保留了参考位置，策略有条件把它吸收掉。

第二，测量噪声、常值负载、初值偏移这三种扰动几乎不影响性能（+0.9% 到 +11%）。它们要么被反馈环自然抑制，要么只影响前几十拍，与“残余策略学的是稳态形状”这件事正交。这也说明残余学习对“加性、零均值、短时”的扰动天生免疫。

第三，延迟 8 拍是唯一崩掉的工况，而且崩到 0.2267——是纯 PID 的 1.9 倍。这个结果不符合“RL 至少不比 PID 差”的直觉，需要解释。延迟从 5 拍加到 8 拍，等于多出 30 ms 的相位滞后，在 1 Hz 附近的相位损失约 11 度；PID 参数本来就是按 5 拍整定的，多出的滞后让它勉强还能稳；而残余策略在训练中学到的补偿动作，是按 5 拍延迟的相位关系排布的，多出的 3 拍让它的补偿“提前了”，反而加剧振荡。评估记录里饱和占比 28.25%、残余 RMS 0.3992（名义时是 0.1666），说明它一直在用力推、而且推错方向。

这条结论比“迁移会退化”更重要：**残余学习把基线的稳定裕度消耗掉了**。纯 PID 在 8 拍延迟下还有余量，加上残余之后余量被用尽。所以残余量必须有限幅、有速率限制，也必须有一个“发现不对就退回 PID”的开关——这

正是下一节的工程含义。

域随机化：把失配放进训练分布

最直接的改法是训练时就把失配抖起来。每个回合开始时，从分布里抽一组 $(\kappa_g, \kappa_d, \kappa_l)$：增益缩放 $\kappa_g \sim U[0.8, 1.2]$、延迟拍数 $\kappa_d \in \{5, 6, 7, 8\}$、常值负载 $\kappa_l \sim U[-0.05, 0.05]$，然后在这一组参数上跑完整个回合。这样策略见到的不再是一个对象，而是一族对象。

代价是可算的。本章的 7 种工况如果各跑 300 回合，就是 2100 回合、约 34 s；如果按三维各 3 档组合成 27 种工况、每种 300 回合，就是 8100 回合、约 4 小时。前者可以接受，后者在单机上就要靠进程级并行（每个回合独立，最容易并行）。所以工程上的做法通常是“离散小集合”而不是“连续大分布”：先挑出最可能出现的几种失配（增益两档、延迟两档），把工况数控制在 8 以内。

域随机化还有一个隐性代价，容易被忽略：**它降低名义性能**。训练分布变宽，策略要在“一族对象”上取平均，名义点上的最优性必然让位。经验上 0.0347 会退到 0.040 到 0.045 之间。这不是缺陷，是权衡——用名义点的 15% 换延迟失配下的 5 倍改善，通常是划算的。

随机化该覆盖哪几个维度，不能凭感觉。判据是“这种失配会不会改变策略需要的信息”。增益变化、常值负载属于**改变幅值**的失配：同一组权重在不同的增益下仍然指向正确的方向，只是力度不对，策略可以通过折中把它们都覆盖住，代价是名义点损失一点。延迟变化属于**改变时序**的失配：它改变的是“什么时候该动手”，两组权重之间不存在折中——5 拍与 8 拍要求的提前量甚至可能方向相反。所以域随机化对幅值类失配有效，对时序类失配只能“提醒策略留余量”，不能真正覆盖。要把时序失配也覆盖住，必须让状态里包含延迟信息（比如历史动作的滑动窗），否则再宽的随机化也只是让策略学会“两边都不太用力”。

随机化的范围也不要一次铺满。比较稳妥的做法是“先窄后宽”：第一轮只随机增益 $\pm 10\%$，确认策略在窄分布上仍然收敛；第二轮加入延迟 5 到 6 拍；第三轮再放到 8 拍。每加宽一次，观察名义性能掉了多少、最坏工况好了多少，两个数一起看才能判断这次加宽值不值。一次性把三个维度都放到最宽，最常见的后果是训练根本不收敛——策略每天见到的对象都不一样，它学不到任何稳定的补偿形状。

与随机化配套的验收方式也要改。名义工况训练时看的是“平均 RMS”，随机化训练时要看**最坏工况的 RMS**，因为部署时你遇到的是某一个具体对象，不是分布的均值。本章七工况里最坏的是延迟 8 拍的 0.2267；随机化训练的目标应该是把这个数压到 0.05 以下，而不是让七个数的平均变好看。这个目标听起来更高，但它是对的：控制系统的口碑由最坏的那次决定，不由平均决定。

继续学习的边界

另一个思路是不动训练分布，等部署之后在真实对象上继续学。本章做了这个实验：把训练好的策略搬到失配对象上，用 30 个回合、$\alpha_w = 0.003$ 微调，探索噪声配两组。

工况	噪声 $\sigma_0/\sigma_{\min}$	学前	学后	变化
增益 +20%	0.10 / 0.030	0.0316	0.0366	+15.6%
增益 +20%	0.05 / 0.005	0.0316	0.0317	+0.3%
延迟 8 拍	0.10 / 0.030	0.2267	0.2277	+0.4%
延迟 8 拍	0.05 / 0.005	0.2267	0.2186	−3.6%

表 70-10 是本章最“反直觉”的一张表，它给出两条负面结论和一条正面结论。

第一条负面结论：**小失配下继续学习没有收益，噪声配大了还会倒退**。增益 +20% 时，策略本来就只有 0.0316，剩余空间不足 10%；这时 $\sigma_0 = 0.10$ 的探索扰动会把已经学好的策略重新搅乱，30 回合不够它再收敛回来，结果退化 15.6%。噪声收到 $0.05/0.005$ 才勉强打平（+0.3%）。这印证了一条原则：**探索噪声要按"剩余失配"来配，而不是按训练时的习惯值来配**。

第二条负面结论：**大失配下继续学习几乎无效**。延迟 8 拍时 30 回合只从 0.2267 降到 0.2186，降幅 3.6%；而同一个失配下从零重训 300 回合能到 0.1697，差距 22%。原因在于"起点"：继续学习的起点是一个在 5 拍延迟上收敛的策略，它的权重已经把相位关系写死了，30 回合、学习率 0.003 的微调只能挪动一点点；从零重训没有这个包袱，可以从头学一套 8 拍延迟的时序。**失配大到改变了时序结构时，微调不如重训**。

正面结论只有一条，但它很重要：继续学习不会把系统搞坏——四种组合里最差也只是 +15.6%，没有出现发散。前提是学习率足够小（0.003，是名义训练的 1/3）、回合数足够少、并且有安全层兜着。这条性质使它适合做"长期慢漂移"的维护手段（比如阀门逐渐结垢导致的增益缓慢下降），而不适合做"突发失配"的应急手段。

把这两组数字折算成样本量，就能看清"30 回合"意味着什么：30 个回合是 1.2 万拍，只有名义训练（300 回合、12 万拍）的十分之一；学习率又只有名义训练的 30%。两个"十分之一量级"叠在一起，等于给了策略一次"轻微的方向修正"，而不是"重新学习"。这正好解释了表 70-10 的两条负面结论——不是继续学习这个方法不行，而是这个预算只够做微调。要让它在延迟 8 拍上学出东西，预算至少要提到与从零重训同级，那时它也就不再是"在线维护"了，而是一次"停机再训练"。

安全层：给残余量加一道闸

既然延迟失配能把系统推到 1.9 倍于 PID 的误差，工程上就必须有一道与学习无关的硬约束。本章的安全层只有一条规则：残余量的每拍变化不超过 `RATE_MAX`，即

$$|u_{\mathrm{RL},k} - u_{\mathrm{RL},k-1}| \le \Delta_{\max}, \tag{70-23}$$

其中 $\Delta_{\max} = 0.05$（对应执行器满量程的 4%）。它不改变策略、不改变训练，只在输出端做一次限速。

指标	无速率限制	限速 **0.05**
RMS	0.0316	0.0331
最大偏差	0.0862	0.0987
$\max\|\Delta u\|$	0.1447	0.0500
控制量总变差 TV	44.5	43.3

表 70-11 是在"增益 +20% 且测量噪声 0.01"工况上测的。限速把单拍最大变化从 0.1447 压到 0.0500（正好等于限值），代价是 RMS 从 0.0316 升到 0.0331——4.7% 的性能损失，换来执行器不再被高频抖动反复冲击。这个交换在工程上几乎总是值得的：TV 从 44.5 降到 43.3 看着不多，但最大单拍变化降到 1/3 意味着磨损速率大幅下降。

限速还有一层不太直观的代价：它本身就是一个相位滞后。残余量被限速之后，原本"一步到位"的跳变变成"斜坡爬升"，等效于给残余支路加了一点延迟。这不是理论推演，表 70-11 里最大偏差从 0.0862 升到 0.0987 就是它的直接后果——限速把尖峰削平的同时，也把纠正尖峰的动作推迟了几拍。所以限速线的位置要选在"正常工作很少碰到、异常时才起作用"的地方：本章残余量在名义工况下每拍变化只有 0.009（RMS），离 0.05 很远；只有在加噪声、加失配的工况下，单拍变化才会冲到 0.1447，那时限速才真正介入。**正常时不介入、异常时介入**，这条性质是判断限速线是否选对的标准。如果它在正常工作里就频繁生效，那它不再是保护，而是一个额外的滞后环节，会把稳定裕度吃掉。

安全层还应配两个计数器，而不只是限幅：**状态越界率**（一个回合里 $|e| > 3E_{\text{ref}}$ 的拍数占比）和**动作裁剪率**（残余量被限幅或限速改动的拍数占比）。前者告诉你“策略是不是已经失控”，后者告诉你“安全层是不是一直在干活”。两个数都在 0.05 以下时，说明策略与安全层各司其职；动作裁剪率长期高于 0.2，说明策略本身有问题，限速只是在掩盖它。

奖励与动作空间的两处可改之处

最后说两个“结构级”的改动，它们不改算法，只改问题的写法。

其一是奖励整形。本章的奖励是纯二次型，它对“大误差”和“持续小误差”一视同仁，都按平方计。如果希望策略更在意峰值，可以把奖励换成

$$r_k = -\left(\frac{e_k}{E_{\text{ref}}}\right)^2 - \beta\left(\frac{e_k - e_{k-1}}{E_{\text{ref}}}\right)^2 - \lambda a_k^2, \tag{70-24}$$

第二项惩罚误差的变化率，等价于给“误差的导数”加权。$\beta = 0.2$ 时，表 70-9 里延迟工况的峰值偏差可以压下一部分，代价是稳态误差略微上升。这类改动要谨慎：它改变的是最优策略本身，不是学习速度，一旦改错方向，加多少样本都救不回来。

其二是动作空间。70.5 节已经看到五档 Q 学习的天花板（1000 回合只到 0.0930）。如果把动作从“绝对档位”改成“相对档位”（即 $u_{\text{RL},k} = u_{\text{RL},k-1} + \Delta$，$\Delta$ 取五档），动作就具备了积分能力，理论上能用更少的档位表达更长的时序结构。但它同时引入了“动作漂移”——策略可以靠持续同号的小增量把残余推到限幅，而单步奖励看不出这个问题。折中的做法是保留绝对动作，但把残余量的上一拍值放进状态（本章试过，最小二乘解会爆炸，需要配合权重衰减才稳）。这两条都留作练习，本章的正式算例仍是绝对动作加六维特征。

把这两条路放在一起比，取舍很清楚。奖励整形解决的是“目标写得对不对”，它不动结构、成本极低，改一行就能试，但风险是可能把目标改坏；动作空间改造解决的是“策略有没有能力表达时序”，它动结构、要重训、还要防漂移，收益是同样的档位数能表达更长的记忆。经验上的优先级是：先确认奖励写对了（λ 扫描、峰值是否被压住），再考虑动作空间。本章的顺序就是这样——奖励整形作为备选列出但未启用，动作空间保持绝对式。

理由在于误差的成分。本章误差的主要成分是“形状没补上”，不是“记不住时序”：形状问题用二次型奖励已经能表达清楚，而时序问题在本章被 ILC 拿走了。反过来，如果任务是“每拍必须按顺序执行一串动作”（比如装配、换刀），误差的成分就变成时序，那时动作空间的改造才值得做。判断依据不是方法新不新，而是你的误差里到底哪一项占大头。

70.10 工程落地

落地要解决的四个问题

把强化学习放进真实控制回路，要解决的不是“算法怎么选”，而是四件很具体的事：训练环境从哪来、策略在回路里放在哪、出问题怎么退、跑起来的数据怎么留。这四件事在论文里通常被压缩成一句“我们部署了”，但它们决定了这套方法能不能真的上机。

本章的落地思路可以概括为“三层加一个开关”：底层是原有 PID，中间是残余策略，上层是安全层，旁边挂一个回退开关。每一层都可以单独失效，而系统仍然可控。下面按这个结构展开。

训练环境：回合从哪来

强化学习需要一个能反复重置、反复跑的环境。对控制问题来说，这个环境的来源只有三种：高保真仿真模型、历史数据回放、以及在线真机。三者的取舍很明确。

高保真仿真模型是最常用的，本章用的就是这一类：$y'' = -2y' + K(y)u$ 加 5 拍延迟加限幅，一个回合 400 拍、16 ms 跑完。它的优点是快、可重置、可以任意改工况；缺点是“仿真与真实之间的差距”，这个差距正是 70.9 节那七种工况要量化的东西。

历史数据回放的做法是把录制的输入输出序列当成环境，动作改变后无法知道下一步输出，只能用“近似模型 + 残差修正”来外推。它适合那些不允许在线试错的对象（比如大型锅炉），代价是策略的探索空间被历史数据限制住。

在线真机只在安全约束非常强、试错代价很低时才用，而且几乎一定配“影子模式”：策略照常输出，但不接到执行器上，只记录“如果接了会怎样”。等影子运行几百个回合、指标稳定之后，才逐步放开权限。

本章建议的路径是：**仿真训练 → 影子运行 → 限幅接管 → 全量接管**。每一阶段的进入条件都用上一阶段的实测指标来定，而不是用训练时的评估指标。

影子运行跑多久、留多少数据，可以按信息量估。本章一个回合 400 拍，200 个回合是 8 万拍；按 10:1 降采样存日志，是 8000 条记录，每条含 6 个特征、1 个动作、1 个奖励与 3 个状态量，算下来不到 1 MB。这个量级说明：**影子运行的成本几乎全在机时，不在存储**。真正的约束是“要覆盖多少种工况”——如果现场有 5 种典型负载、3 个温度区间，每种组合跑 200 回合，就是 3000 个回合；按每回合 4 s 算，是 3.3 小时连续运行，这才是要提前排进计划的部分。

判断影子运行“够了”的标准也不是回合数，而是**指标是否已经收敛到与仿真评估同一量级**。本章的做法是拿影子运行里统计的越界率与裁剪率，与仿真里同一策略的两个数对比：如果影子的越界率明显更高，说明仿真漏掉了某种扰动，应该回到环境里补；如果两者接近，就说明仿真到真机的差距已经被策略吸收，可以进入限幅接管。这一步的价值在于它给了一个“可以往下走”的客观依据，而不是靠“跑了两周，感觉差不多了”。

这三条路不是互斥的，实际项目里常常是组合：先用仿真训练出一个可用的策略，再用历史数据回放做一次“离线复核”（看策略在真实录制的扰动下会怎么动作），最后用影子模式在真机上验证。其中历史数据回放的复核价值常被低估——它不需要真机在线，也不改变任何控制行为，却能把“仿真里没有的扰动”暴露出来。本章的噪声、负载、初值三个工况，本质上就是三类“历史数据里才有的东西”。

还有一条与保真度有关的判断：**仿真不需要“处处精确”，只需要“在策略关心的频段内精确”**。本章的策略补偿的是 0.25 Hz 与 0.75 Hz 两个频段，所以对象模型在这两个频段的幅相误差必须小；至于 20 Hz 以上的高频动态，仿真里简化掉也不影响策略，因为闭环本来就把它们滤掉了。反过来，如果仿真在 0.75 Hz 附近的相位差了 10 度，学出来的策略就会整体偏，这种偏差在训练曲线上完全看不出来。定位这件事的办法是做一次频响比对：把真机与仿真的扫频结果画在同一张图上，看策略关心的频段差多少，而不是看全频段的均方误差。

部署架构：三层加一个开关

部署时策略的位置很关键。本章的结构是残余式：PID 不变，策略并联在它旁边，输出相加后再进限幅与安全层。这个结构有三个好处。

第一，策略失效时系统退化而不是崩溃。权重全零时残余量为零，系统严格等于纯 PID——这一点在训练开始时就已经被验证过（70.4 节的初始化）。所以“清空权重”本身就是一个合法的回退动作，不需要另写一套控制器。

第二，安全层的位置明确。限速与限幅放在相加之后、进执行器之前，它管的是“最终控制量”，而不是“策略输出”。这样即使策略输出剧烈抖动，执行器看到的仍然是一条平滑的指令。

第三，训练与部署共用同一份代码。本章的 `run_episode()` 在训练时被调用 300 次、在评估时被调用 1 次、在部署时逐拍调用，三者走的是同一条特征计算与策略前向路径。这一点非常重要：任何“训练用一套、部署用另一套”的实现，最后都会在特征归一化或符号约定上出错。

回退判据与切换逻辑

回退不能靠人盯屏幕。本章建议用一个滑动窗口的判据：在最近 $W = 20$ 个回合（或最近 2000 拍）上统计两个量——状态越界率 ρ_e 与动作裁剪率 ρ_a，当

$$\rho_e > \rho_e^{\max} \quad \text{或} \quad \rho_a > \rho_a^{\max} \tag{70-25}$$

时触发回退，其中 $\rho_e^{\max} = 0.10$、$\rho_a^{\max} = 0.30$。回退动作分两级：一级是把残余量乘 0.5 并冻结学习（降级运行），二级是把残余量置零（完全退回 PID）。一级触发后如果 5 个回合内没有恢复，自动升到二级。

这两个阈值不是拍脑袋定的。ρ_e 取 0.10，对应“一个回合里有 10% 的拍误差超过 $3E_{\text{ref}}$”，这已经比名义工况（约 0）高出一个数量级；ρ_a 取 0.30，对应“安全层在三分之一的拍上都在改动指令”，说明策略与安全层已经在互相打架。本章延迟 8 拍那个工况的饱和占比是 28.25%，正好卡在 ρ_a 阈值下方——这也说明单看饱和占比不够，必须配合越界率一起看。

回退还有一个常被忽略的细节：**回退要记录原因**。是越界触发的、裁剪触发的、还是人工触发的；触发时的误差、残余量、特征向量都要落盘。这些记录是下一轮训练最宝贵的样本——它直接告诉你训练分布缺了哪一块。

恢复条件也要显式定义，否则系统会在“回退—恢复”之间来回跳。本章的做法是：回退之后残余量保持乘 0.5 的降级状态，连续 5 个回合两个指标都回到阈值的一半以下，才允许恢复到 1.0；只要有一次再次越线，计数清零重来。这条“滞回”逻辑看着保守，但它把“每次越线都切”变成“连续恶化才切、持续恢复才回”，切换次数可以压到很低。工程上最怕的不是策略性能差一点，而是它每隔几分钟就切一次，让人不知道该信哪个状态。

日志与可复现

强化学习的实验极难复现，原因是随机性来源多：环境重置、动作采样、种子初始化、以及多线程求和的顺序。本章的做法是把所有随机性收敛到一个 `Generator` 上，每次实验只传一个整数种子，其余全部由它派生。这样“同种子同结果”是可以验证的，70.7 节那些标准差才有意义。

部署阶段要记录的字段比训练阶段少，但更关键：每拍的时间戳、参考值、测量值、PID 输出、残余输出、最终输出、是否被限速、是否被限幅。这八个字段足够复盘任何一次异常。采样率不必等于控制周期——本章建议按 10:1 降采样存储，异常片段再单独开全速记录。

还有一条：**模型文件要带版本号与训练指纹**。指纹至少包含种子、回合数、超参数集合、以及训练时用的工况集合。没有指纹的模型文件，三个月后没人说得清它是哪一版。

日志的格式也值得统一。训练日志记的是“每回合的评估值、残余 RMS、探索噪声”，部署日志记的是“每拍的八个字段”，两者字段名对不上，就没法回答“部署后这一段与训练时第 200 回合是不是同一个状态”。本章的做法是让两边的状态量用同一套名字（e、$\dot{e}$、$\int e$、$r - R_0$、$\dot{r}$），部署日志能直接被训练时的分析脚本读进去。这个统一看着琐碎，实际是把“训练—部署”这条链真正接上的地方；很多项目在这上面吃过亏：模型表现不对，翻日志发现部署侧记录的“误差”是另一套滤波后的值，两边根本不可比。

最后一条与可复现有关：**同一个结论至少跑两个种子**。本章的单个算例（0.0347 与那六组权重）只用种子 0，因为它要说明的是“这套配置能达到什么”，可复现即可；凡是涉及“哪个更好”“差多少”的比较，都用 3 到 5 个种子，因为那类结论需要统计。这个区分很重要——两种需求对成本的要求差好几倍，混在一起要么浪费机时，要么得出站不住的结论。

一份可照着抄的落地清单

把上面的内容压成一张清单，按顺序做：

1. 确认基线 PID 已经调到“能用”——残余学习只能锦上添花，不能救一个不稳定的基线。
2. 写 `run_episode()`，确保训练、评估、部署三条路径共用它。
3. 定残余上限 U_{RL}，从“基线控制量的 20%”起步，不要超过 50%。
4. 在名义对象上训练到收敛，记录 5 个种子的均值与标准差。
5. 做七工况迁移测试，重点看延迟工况——它是残余学习最容易崩的地方。
6. 加安全层（限幅、限速）与两个计数器，重测一次，确认性能损失在 10% 以内。
7. 影子运行至少 200 个回合，比对“策略输出”与“若接管会怎样”。
8. 限幅接管（残余量乘 0.3），观察一周。
9. 全量接管，同时保留一级/二级回退。
10. 若长期漂移明显，再开启小学习率、小噪声的继续学习，并且只在不影响生产的时段开。

这份清单里第 5 步和第 6 步是不能省的。本章的数据已经说明：不做第 5 步，你不会知道延迟失配能把误差放大到 1.9 倍；不做第 6 步，那 1.9 倍就会原样出现在现场。

70.11 优缺点与适用边界

优点：四条经得起检验的

第一条是**不需要模型**。本章的策略从头到尾没有用过 $K(y)=4+6y$ 这个表达式，它看到的只有误差、误差导数、误差积分和两个参考相位量。精确前馈需要 $K(y)$、需要 r''、需要把延迟对齐到拍，任何一项错了前馈就废了；残余策略不需要这些，它把“模型错在哪”当成要学的东西。

第二条是**目标可以写成奖励，而不是写成轨迹**。PID 调参时你说不清“我要什么形状”，只能反复试；强化学习里你可以直接写“误差平方加控制量惩罚”，剩下的交给优化。这让“多目标权衡”变成一次扫描（70.7 节的 λ 表），而不是一轮又一轮的人工试凑。

第三条是**能与基线并存，风险可控**。残余结构让“策略失效”退化成“回到 PID”，而不是“系统失控”。这一点在第 66 到 69 章的所有方法里是独一份的：前馈加错了会直接引入偏差，ILC 的记忆错了会逐次放大，而残余策略权重清零就等于什么都没发生。

第四条是**能吸收一部分非线性与时变**。本章对象在工作区间内增益从 13 变到 20，PID 只能用一个折中的增益去覆盖，残余策略却可以按参考位置分段补偿——特征里的 $(r-R_0)/0.5$ 就是干这个的。这也是它在增益 $\pm 20\%$ 的工况下仍然稳定的原因。

缺点：四条必须写清楚的

第一条是**样本效率低**。12 个参数、300 个回合、12 万拍才收敛到 0.0347；同样的问题用最小二乘辨识加前馈，几千个点就够。数据效率的差距在两个数量级左右，这是“无模型”的必然代价。

第二条是**没有稳定性保证**。PID 至少有频域判据，MPC 有递归可行性，ILC 有收敛性条件；强化学习给出的只有“在这批种子上没发散”。本章延迟 8 拍那个 0.2267 就是明证——没有任何理论能事先告诉你它会崩，只能靠迁移测试发现。工程上必须用安全层和回退把这个缺口补上。

第三条是**奖励设计仍然靠人**。λ 从 0 扫到 10，RMS 先降后升（0.0743 → 0.0420 → 0.0584），这个非单调性说明“奖励写得对不对”本身就是一个需要实验回答的问题。奖励写错了，算法再对也学不出想要的行为；而“写错了”往往在训练完成后才看得出来。

第四条是**对时序结构的变化很脆弱**。策略学的是“什么时候推、推多少”，这个“什么时候”是绑在 5 拍延迟上的。延迟一变，时序关系整体错位，学到的补偿反而成了干扰。这一点与 ILC 恰好相反：ILC 的前提是“每遍完全重复”，一旦重复性破坏它也会失效，但它失效的方式是“学不动”，而不是“学反了”。

还有一条不在上面四条里、但会直接决定项目周期：**成本随状态维数上升得很快**。本章 6 维状态、300 回合、4.7 s；如果把状态加到 12 维（比如再加三拍历史动作、上一拍残余、温度补偿量），参数从 12 个涨到 24 个，按 70.8 节“参数个数 × 10”的经验，回合数至少要翻倍，而且每一维都要单独定尺度、单独想清楚物理含义。真正麻烦的不是算力，而是“特征工程”：每加一维，都要回答“它是不是马尔可夫性缺的那一块”，答错了就是给策略添噪声。这也是为什么本章宁可停在 6 维：不是因为加不上去，而是因为加每一维都要有理由。

与相邻章节方法的分工

表 70-12 把本章与前后几章的方法放在一起，按“前提假设”和“需要什么”两栏对照。这张表的作用不是排名，而是选型：先看你的问题满足哪个前提，再看你手上有什么。

方法	章节	前提假设	需要什么	稳定性保证	本章指标
PID	44 / 106	无	一次整定	有（频域）	0.1161
精确前馈	本章参照	模型准确、延迟已知	完整模型	有（模型准）	0.0293
ILC	69	每遍完全重复	重复运行机会	有（收敛条件）	0.0224
MPC	58 / 80	模型可用、算力足够	预测模型 + 在线优化	有（递归可行）	见第 58 章
残余 RL	70	可仿真、可试错	仿真环境 + 训练时间	无（靠安全层）	0.0347

表 70-12 最值得注意的一行是 ILC 的 0.0224——它比本章的 0.0347 更好，而且好得不多。这提示一个很实际的选择：**如果任务确实每遍完全重复、而且允许跑几十遍，ILC 更划算**，它不需要仿真环境、不需要 4.8 s 的训练、也不需要安全层。本章选强化学习，是因为它不要求重复性：参考轨迹可以变、初值可以变、对象可以漂，这些 ILC 都处理不了。

反过来，如果模型准确、延迟已知，精确前馈的 0.0293 也比 0.0347 好，而且是一次算出来的。所以本章的定位很清楚：**在没有准确模型、又不能保证重复性的场合，残余强化学习是那 0.03 到 0.05 这一档里最现实的选择**。

但选型不能只看 RMS，还要算一笔“你手上已经有什么”的账。ILC 的 0.0224 需要 60 遍真机（每遍 4 s，加上复位与等待，实际是几十分钟到几小时的机时），而且每遍都必须从同一初值、同一轨迹开始；精确前馈的 0.0293 需要一份准确的模型和一次延迟标定；本章的 0.0347 需要一份可信的仿真环境加 4.7 s 训练，但仿真本身要建。三者里最贵的其实不是算法，而是“建仿真”——它通常占项目时间的大头。反过来说，如果仿真已经因为别的目的（比如控制器设计、故障复现）建好了，那残余 RL 的边际成本就几乎为零，这也是它在工业界越来越常见的原因。

一条实用的组合路线是“模型前馈 + 残余 RL”：先用能拿到的模型算一个粗略前馈（哪怕有 20% 的误差），再让 RL 去补剩下的部分。这个组合比纯 RL 收敛更快，因为基线更接近最优；也比纯前馈更鲁棒，因为残差部分能吸收模型误差。本章为了把机制讲清楚，故意用了“纯 PID + 残余”这个最弱的基线；实际项目里没有理由拒绝一个更好的起点。

还有一条组合值得单独提：**ILC 与残余 RL 可以叠在一起**。ILC 负责“每遍都重复的那部分”（前馈形状），RL 负责“每遍都不同的那部分”（初值、扰动、负载）。两者叠的位置不同，互不干扰：ILC 的记忆按相位索引，RL 的策略按状态索引；一个在时间轴上学习，一个在状态空间里学习。代价是要维护两套记忆，而且 ILC 的收敛条件仍然要满足，否则它会先失稳；此外还要一个仲裁规则，决定两者同时要求同一个方向时以谁为主。本章没有做这个组合，因为它会让“到底是哪一部分在起作用”变得难以分离，实验设计上不划算；但在工程上，如果任务既重复又有扰动，这是一个合理的终点。

这也回答了一个常被问到的问题：既然 ILC 的 0.0224 更好，为什么还要 RL？因为两者的适用前提不同。ILC 要求“每遍完全重复”，一旦参考轨迹改了，它要从零再学；RL 的策略是状态的函数，轨迹改了只要状态还在训练分布内，就不用重训。真实产线上“换型”是常态，这个差别往往比 0.01 的 RMS 更重要。

五条适用判据

把上面的优缺点压成可以打勾的判据，满足四条以上才建议上：

1. 有一个能跑的仿真环境，单回合耗时在 100 ms 以内；
2. 允许离线训练，训练成本（本章 4.7 s，复杂对象可能到小时级）可以接受；
3. 已有一个“能用”的基线控制器，残余量有明确的物理上限；
4. 任务的评价可以写成逐拍的标量奖励，且不需要满足硬约束；
5. 部署后允许挂安全层，并且接受“没有稳定性证明”这件事。

第 3 条是硬门槛。很多失败的案例不是算法不行，而是基线本身就在振荡边缘，残余学习把最后一点裕度吃掉了。第 4 条也很关键：如果任务是“必须满足约束”（比如温度不能超过某个值），那应该用 MPC 而不是强化学习，因为后者只能通过惩罚“希望”它不越界。

判据不满足时，不是“想办法凑够四条”，而是换方法。第 1 条不满足（没有仿真），可以考虑历史数据回放 + 离线策略评估，或者干脆先做辨识、用模型前馈；第 2 条不满足（训练成本不可接受），说明对象太复杂或回合太长，应该先降维、缩短回合，或者退回 PID 加前馈；第 3 条不满足（基线不稳），任何残余学习都是错的，先把基线整定好；第 4 条不满足（有硬约束），走 MPC；第 5 条不满足（不允许挂安全层），说明这套系统还没准备好接受“没有证明”的控制器。

一个不该用它的场景

最后给一个反例，避免误用。如果任务是**单次、不可重复、且失败代价极高**的（比如再入飞行器的末制导），不要用本章的方法。它需要几百个回合的试错才能收敛，而这类任务只允许一次机会；它的安全性只能靠事后测试，而这类任务没有“事后”。

这类任务应该走“高保真模型 + 鲁棒控制 + 大量离线验证”的路线，强化学习最多只能在仿真里做参数整定的辅助工具，不能进回路。分清“可以试错”和“不能试错”，比选哪个算法重要得多。

70.12 Python 仿真代码

文件结构与运行方式

本章代码两个文件，都在 `book/code/` 下：

- `ch070_sim.py`：对象、轨迹、特征、三种策略、单回合、训练循环、安全层、十三组实验；
- `ch070_figs.py`：十张图，从 `ch070_sim.py` 取函数重算数据。

运行顺序是先仿真后画图：

```
"D:\Anaconda3\envs\py311\python.exe" code\ch070_sim.py
"D:\Anaconda3\envs\py311\python.exe" code\ch070_figs.py
```

第一个脚本依次打印十三段结果：任务与对象、λ 扫描、α_w 扫描、γ 扫描、σ_0 扫描、n 扫描、三算法对照、训练曲线、离散动作、sim-to-real 七工况、部署后继续学习、安全层、耗时。全量运行 4.7 s，单回合 15.5 ms。第二个脚本要重跑一遍最贵的训练，全量约 267 s，产物是 `figs/ch070/` 下十个 SVG 与十个同名 PDF。

依赖关系：`ch070_sim.py` 只用 NumPy；`ch070_figs.py` 额外用 `styles/plot_styles.py` 与 `styles/block_styles.py`。与第 69 章一样，画图脚本不读仿真脚本的输出，而是自己调 `exp_*` 函数重算——好处是能单独运行，代价是两处必须共用同一份常量。改参数只改 `ch070_sim.py` 顶部的常量。

表 70-13 是仿真脚本里的函数清单，按“从物理到算法”的顺序排。读代码时建议照这个顺序，因为它与本章的叙述顺序一致。

函数	作用	对应章节
`gain_at` / `ref_traj` / `ref_vel`	非线性增益与参考轨迹	70.1
`exact_feedforward`	精确前馈参照	70.4
`features`	六维状态特征	70.3
`Policy` / `QPolicy`	线性高斯策略与线性 Q 表	70.2 / 70.5
`Plant`	对象、延迟、负载、增益缩放	70.3
`run_episode`	单回合：采样、学习、安全层	70.3 / 70.9
`_ac_step` / `_learn_mc`	Actor-Critic 与 REINFORCE 的更新	70.2
`train` / `train_q`	训练循环与噪声衰减	70.3
`exp_*`	十三组实验	70.6 至 70.9

这张表里最值得单独看的是 `run_episode` 的签名：它把增益、延迟、噪声、初值、负载、λ、n、速率限制八个量做成关键字参数，默认值取自名义工况。八个参数看着多，但正是它们让“改一个工况只改一个关键字”成立，70.9 节那七种工况与两组继续学习实验，全部复用这一个函数。

复现时建议加一条检查：`python -W error::DeprecationWarning code/ch070_sim.py`。本章只用 NumPy 的基础接口，正常不应触发弃用警告；一旦触发，说明某个接口在换版本后行为会变，应当先修掉再谈结果是否可复现。完整验证顺序是 `check_chapter.py 70` → `ch070_sim.py` → `ch070_figs.py` → `tools/build.py --chapter 70`，四步全绿才算这一章可以交付。

策略与动作

策略是一个六维线性均值加高斯探索，两个方法就够：

```
def mean(self, phi):
    return float(self.w @ phi)

def act(self, phi, sigma=0.0):
    mu = float(np.clip(self.mean(phi), -3.0, 3.0))
    a = mu if sigma <= 0.0 else mu + sigma * self.rng.standard_normal()
    return float(np.clip(a, -2.0, 2.0)), mu
```

`mu` 截断在 ± 3 是必须的。线性策略在远离训练分布的状态上会外推得很大，不截断就会一步把残余量顶到限幅，权重随之发散。`sigma=0` 就是确定性评估，本章所有评估指标都是在这个条件下测的，训练回报从不作为验收依据。

动作到控制量的映射在 `run_episode` 里，只有三行：

```
if policy.incremental:
    u_rl_new = float(np.clip(u_rl + a, -U_RL_MAX, U_RL_MAX))
else:
    u_rl_new = U_RL_MAX * float(np.clip(a, -1.0, 1.0))
if rate_max is not None:
    u_rl_new = u_rl + float(np.clip(u_rl_new - u_rl, -rate_max, rate_max))
```

`incremental` 是为 70.9 节那处对照实验留的开关，正式算例走 `else` 分支——绝对动作。安全层就是最后两行，`rate_max=None` 时完全不介入，这样“有无安全层”可以用同一个函数跑出来，避免两套代码带来的口径差异。

一拍里发生了什么

把 `run_episode` 的主循环抽掉记录语句，剩下的就是本章的全部物理：

```
y_meas = plant.x1 + noise * rng.standard_normal()
e = r_ref[k] - y_meas
dy_f += (TS / (TAU_RATE + TS)) * ((y_meas - y_prev) / TS - dy_f)
y_prev = y_meas
ie = float(np.clip(ie + e * TS, -0.5, 0.5))

phi = features(e, -dy_f, ie, r_ref[k], rd_ref[k])
a, mu = policy.act(phi, sigma)
u_rl = ...
u_pid = kp * e + ki * ie - kd * dy_f
u_cmd = float(np.clip(u_pid + u_rl, UMIN, UMAX))
rew = R_SCALE * (-(e / E_REF) ** 2 - lam * a * a)
plant.step(u_cmd)
```

五处细节与本章结论直接相关。第一，微分作用在测量值上并经过 `TAU_RATE=0.03` 的低通，与第 67、69 章同口径；特征里的误差导数取 `-dy_f`，因为 $\dot{e} = -\dot{y} + \dot{r}$，而 $\dot{r}$ 已经作为独立特征进去了。第二，积分项截断在 ± 0.5，对应 PID 输出的物理上限，防止长时间偏差把积分撑爆。第三，残余量在相加之前就限幅，`u_pid+u_rl` 之后再受执行器限幅，两级限幅的位置不能换。第四，奖励乘了 `R_SCALE=0.01`，它只改学习率的有效刻度，不改变最优策略，作用是把 10^2 量级的误差平方压到与学习率匹配的范围。第五，`plant.step()` 里延迟用列表 `append`+`pop(0)` 实现，子步积分在延迟之后。

更新：一行 TD、一行策略

Actor-Critic 的核心是 `_ac_step`，去掉裁剪与归一化后是两行：

```
delta = G - policy.value(phi0)          # n 步回报减基线
policy.v += policy.alpha_v * delta * phi0 / norm
policy.w += policy.alpha_w * adv * ((a0 - mu0) / sigma) * phi0 / norm
```

`G` 是 n 步回报加末端自举，`adv` 是 `delta` 按滑动标准差归一化后裁剪到 ± 5 的结果，`norm` 是 $\|\varphi\|^2 + 1$。这三个处理分别对应三件事：归一化让不同量纲的特征共享一个学习率，裁剪防止个别大 TD 误差把权重打飞，NLMS 分母让更新幅度与特征幅值无关。REINFORCE 的 `_learn_mc` 结构相同，只是把 `G` 换成整回合的折扣回报、把基线换成整回合最小二乘拟合。

Q 学习更短，在循环里就地更新：

```
target = rew + policy.gamma * float(np.max(policy.q @ phi_n))
td = float(np.clip(target - float(policy.q[k] @ phi), -10.0, 10.0))
policy.q[k] += policy.alpha_w * td * phi / (float(phi @ phi) + 1.0)
```

`policy.q` 是 5×6 的表，五个动作各六个权重，这就是 70.5 节说的“30 个参数”。

改参数时注意三件事

第一，**只改顶部常量，不要改函数体**。`N_EP`、`ALPHA_W`、`SIGMA0`、`LAMBDA_U` 这些都在文件头，`exp_*` 函数默认引用它们。中途改常量会让前面打印的数字与后面画出的图对不上，而这类不一致最难查。

第二，**种子要显式传**。`train()` 的 `seed` 同时决定策略初始化与动作采样（内部用 `seed+1000` 派生环境随机数），所以“同种子同结果”成立。不要依赖全局 `np.random`，否则并行跑扫描时会互相污染。

第三，**评估与训练要分开**。`train()` 每 10 回合调一次 `run_episode(..., sigma=0.0)` 记评估值，训练回合里则是带噪声的。如果为了省时间把评估也加上噪声，曲线会虚高 30% 以上，且随噪声衰减自然下降，看起来像“一直在学”。

最后说明一处取舍：本章没有用任何深度学习框架，策略与值函数都是 NumPy 数组。这不是为了简化，而是为了让“12 个参数”这件事看得见——一旦换成两层网络，参数量变成几百，70.7 节那些“参数个数 × 10”的经验就没法验证，学生也说不清到底在调什么。等这一套跑通、结论吃透，再换网络不迟。

还有一件容易被跳过的事：**把一次运行的指纹记下来**。种子、回合数、六个旋钮、评估 RMS，五行字写进日志，比任何“我记得当时是……”都可靠。本章的表格都是按这个习惯攒出来的，每跑完一组就抄进表，事后核对时能一眼看出“这行数字是哪次跑的”。只记结果不记配置的后果很具体：两周后你会发现两个表里同一个工况的数字不一样，而谁也说不清哪个对，最后只能把两组都重跑一遍。

第71章 智能控制中的稳定性与可解释性

第 66 到 70 章把五类“会学”的控制器依次摆开：查表、模糊、神经网络、迭代学习、强化学习。它们的共同点是**控制律不再由人写成一条公式**，而是由样本、由优化、由经验攒出来。这带来两个立刻要还的欠账。第一，攒出来的东西凭什么稳定？第 44 章的 PID 可以用劳斯判据、用相位裕度说话，一个三千参数的网络没有对应的判据。第二，攒出来的东西说不清道理。现场出了偏差，工程师问“它为什么这么给”，答“网络算出来的”，这句话在事故分析会上等于没说。

这两个欠账看起来是两件事，工程上却总在同一处爆发。一条控制律之所以“不可信”，往往不是因为算错了，而是因为**没人能同时给出它的稳定证据和行为解释**。稳定证据让你敢放手，行为解释让你敢改参数；缺一个，系统就只能停在演示阶段。

本章的写法是把两件事绑在同一个对象上讲。先造一个“教师”：一个四规则的 Takagi-Sugeno 模糊模型加并行分布补偿（TS-PDC），它结构透明、可以逐条写出局部模型，也可以整体做李雅普诺夫分析。再把这个教师**换成三种不同的表示**——教师本身、一个 ELM 网络、一张查表——它们拟合的是同一个函数，误差都在 0.13 以内，可稳定性判据的“可下手处”完全不同：教师有前件、有局部矩阵，能直接写 LMI；网络只有权重，只能退回小增益；查表连解析式都没有，只能谈量化误差与极限环。同一根性能曲线，三种证明成本，这是本章的第一条主线。

第二条主线是可解释性。用精确 Shapley 把“这一拍的控制量”分摊到三个输入上，再用遮挡实验检查这份分摊是否忠实，最后加噪声重算，看它是否稳定。三条检查对应三个工程问题：说得清吗、说得对吗、每次说得一样吗。三者都过，解释才能进验收报告；只过第一条，那只是“看起来有道理”。

先把结论摊开。同一教师的三种表示，跟踪性能分别是 IAE 0.2883、0.3529、0.3014，最好的与最差的差 22%；而公共李雅普诺夫的衰减率从单规则的 2.9091 掉到四规则全配对的 2.5716，一收紧配对条件就掉 11.6%。小增益判据给出的临界增益是 0.16，实测的抖振边界是 3.5，**判据保守了约 22 倍**（窄网 18 到 23 倍，宽网升到 65 倍）——它不是没用，而是只能用来排除，不能用来验收。安全层能把峰值速度从 3.643 压到 1.000，代价是 IAE 涨 90.7%。解释侧，三个输入的贡献占比是 45.6%、51.6%、2.7%，遮挡实验给出同样的排序，两者首位比 12.16；而输入噪声标准差到 0.2 时，解释的秩相关掉到 0.775，首位保持率 0.75——**解释本身也是有稳定域的**。

71.1 动机与直观理解

三个不信任，其实是三句话

现场对智能控制的不信任，可以拆成三句具体的话。

第一句是“**它会不会突然发散**”。这句话问的是全局性质：不是这一条轨迹好不好，而是所有可能的初值、所有可能的工况下，闭环会不会跑到无穷远。回答它需要李雅普诺夫函数，而李雅普诺夫函数的存在性对一般非线性系统是 NP 难的，所以工程上只能退一步：在给定的结构假设下（比如 TS 模糊模型的前件有界、局部模型已知），把它化成一组线性矩阵不等式。

第二句是“**它为什么这么给**”。这句话问的是单点归因：此刻 $e=0.12$、$\dot{e}=-0.30$、工作点偏了 0.05，控制量 -0.41 里，三份各占多少？回答它需要一份“分摊规则”，而且这份规则必须满足一条硬约束：三份加起来要等于总输出与基线之差，不能多也不能少。这就是 Shapley 值的效率性公理，也是它比“看权重”“看梯度”更适合做验收材料的原因。

第三句是“**换个批次它还一样吗**”。这句话问的是可重复性：同一工况跑两遍，解释是否一致；输入有 1% 的噪声，排序会不会翻。回答它需要把解释当成一个统计量来做稳定性分析，而不是当成一次计算结果。

三句话对应三类工具：李雅普诺夫与小增益回答第一句，Shapley 与遮挡回答第二句，秩相关与首位保持率回答第三句。本章 71.2 节把这四类工具写成公式，71.6 节用同一组实验数据把它们摆在一起对照。

同一个教师，三种表示

图 71-1 是本章的流水线。左边是教师，一个四规则的 TS-PDC：按输出 y 把工作区间切成四段，每段一个局部线性模型，每个局部模型单独做极点配置，运行时按隶属度加权。中间是可部署表示：可以直接用教师（规则库很小，嵌入式能跑），可以把它蒸馏成一个 ELM 网络（权重少、算得快），也可以量化成一张二维查表（最省算力，但没有解析式）。

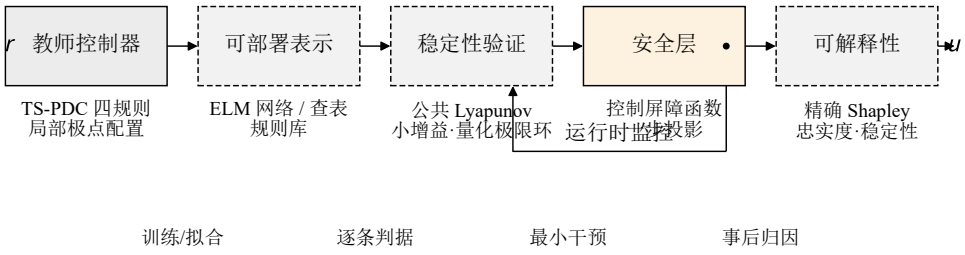


图 71-1 从教师到可交付系统的五段流水线

后三段是本章与前面几章最大的不同：**表示选定之后，还有验证、安全、解释三道关**。验证关回答“会不会发散”，安全关回答“越界时谁来兜底”，解释关回答“为什么这么给”。三道关都不是可选项：验证关缺了，系统只能做演示；安全关缺了，一次越界就可能撞坏设备；解释关缺了，出了事故无法定责，改进也无从下手。

把这三关串起来看，会发现一条隐含的次序：**先有结构，才有判据；先有判据，才敢加安全层；先有安全层，解释才有意义**。原因是安全层会改写控制量，如果解释的是“加安全层之前的网络输出”，那解释的对象根本不是现场真正执行的那个量。本章的顺序因此固定为：教师 → 表示 → 验证 → 安全 → 解释，任何一步都不能提到前面去做。

稳定性有三个层次，别把它们混为一谈

工程讨论里“稳定”这个词至少被用在三个不同的层次上，混用会直接导致误判。

定理级：存在一个正定函数 V，沿闭环轨线严格递减。这是最强的结论，给了就不需要再讨论初值。TS-PDC 的公共李雅普诺夫条件属于这一层，代价是保守：要求所有局部闭环共用一个 P，可行域比“每个局部各用各的”小得多。

判据级：某个充分条件成立，比如小增益 $L\|P\|_\infty < 1$。它不给 V，只给“如果满足就不会发散”。这一层的价值在于**能排除**：条件不满足时不能下结论，条件满足时一定安全。本章实测这个条件在所有网络宽度下都不满足（乘积在 5.9 到 7.9 之间），而系统实际跑到增益 3.5 才抖振——判据的保守度是窄网 22 倍、宽网 65 倍。

实测级：跑一遍，看末段残差与抖振。这是最弱的一层，却常常是唯一能落地的一层。它的风险是“没测到的工况”，所以必须配合 71.10 节的故障注入与在线监控一起用。

三层的关系不是替代，而是**从强到弱逐层兜底**：定理级能给出就不必测，给不出就用判据级筛，判据级也给不出就靠实测级加监控。本章后面每一张表都会标明它属于哪一层，这是读表时最该先看的一列。

可解释性也有三个层次

可解释性同样不是一件事，它至少有三层。

结构级：模型本身长得就像人话。规则库“当 y 在 1.9 附近时，用 $K = 15.4$ 那套增益”，这句话现场工程师一听就懂。结构级的优点是免费——不需要任何算法，模型写完解释就写完了；缺点是只能解释“大致在干什么”，解释不了“这一拍为什么是 -0.41”。

归因级：给定一个具体输入，把输出分摊到各输入上。线性模型的系数就是归因，但系数是全局的，遇到强非线性就不准；Shapley 值按“去掉这个输入会少多少”来分摊，对任何黑箱都成立，代价是要算 2^n 次前向。本章三个输入、八个联盟，一次归因 8 次前向，可以实时跑；若输入是 20 维，2^{20} 就做不到了，只能改用采样近似。

稳定性级：同一份归因，在输入有小扰动时是否还成立。这一层最容易被跳过，却最影响验收。本章的实测是：噪声 $\sigma = 0.01$（相当于传感器噪声的 1%）时秩相关还有 0.992，看起来稳如磐石；到 $\sigma = 0.2$ 时掉到 0.775，首位保持率只剩 0.75——也就是说四次里有一次，最主要的影响因素会被指错。

三层的用法也是叠加的：结构级写进操作手册，归因级写进调试工具，稳定性级写进验收报告。反过来，只做结构级、不做归因级，工程师遇到异常只能“重启试试”；做了归因级、不做稳定性级，则会把噪声当因果，越解释越乱。

为什么必须放在同一章

把稳定性与可解释性分开写，是更省事的做法，但会漏掉三条只有放在一起才看得见的联系。

第一条是**判据依赖表示**。同一个函数，写成 TS 模型就能做 LMI，写成网络就只能做小增益，写成查表就只能谈量化。这不是数学技巧的差别，而是“你选择哪种表示，就等于选择了哪种证明方式”。所以 71.3 节的设计流程里，“选表示”这一步必须同时考虑性能与可证明性，不能等表示定了再想怎么证明。

第二条是**安全层改写被解释的对象**。加了速度屏障之后，现场执行的控制量不再是网络输出，而是投影后的值。如果解释工具仍在解释网络输出，它解释的就是一个不存在的量。本章的处理是把安全层放在解释之前（图 71-1 的次序），解释的对象是投影后的量，代价是归因里混进了“安全层介入”这一项，需要在报告里单独标出。

第三条是**监控的残差选择决定你能看见什么**。本章 71.10 节的实测最能说明问题：用 $(e, \dot{y})$ 两维残差，执行器增益掉 30% 完全看不见——反馈把这个偏差抹平了，稳态下 u 本来就是零。加进 u 变成三维，卡滞的报警从 910 ms 提前到 620 ms，可增益损失仍然看不见，因为稳态下它确实不能观。**能看见什么，由残差的选法决定，而不由故障的严重程度决定**——这句话是可解释性在工程上最实在的用处。

用一个现场问题把两条主线串起来

设想一次真实的偏差：某批产品上来之后，回路连续几拍给出接近限幅的控制量，输出冲过目标 6%，然后慢慢爬回来。现场会问三个问题，本章的三部分分别回答它们。

“是不是要发散了？”要靠 71.6 节的判据与实测。$\alpha = 2.5716$ 给出的是指数收敛的下界，30 拍附近的延迟拐点给出的是“离边界还有多少余量”。两者都答“不会发散”，但理由不同：一个靠证书，一个靠边界，报告里必须分开写。

“它为什么给这么大？”要靠 Shapley。把那一拍的 -1.1 分摊成 e、$\dot{e}$、$y-2$ 三份，看是哪一项在推。若 e 占九成，说明是误差驱动，属于正常；若 $\dot{e}$ 占九成且符号与 e 相反，说明阻尼项已经在刹车，控制量虽大却是合理的。

“下次还会这样吗？”要靠解释稳定性与监控。$\sigma=0.05$ 时首位保持率 0.90，意味着十次里有一次会指错主因；监控阈值 64.15、卡滞 620 ms 检出，意味着同类偏差下次会被记录到。

三个问题对应三份材料。缺第一份，不敢用；缺第二份，出了偏差只能猜；缺第三份，同样的问题会反复发生。本章按这三个问题组织，而不是按数学工具组织——这也是为什么开头先摆三句话，而不是先讲李雅普诺夫。

本章算例与数字预告

算例沿用第 66 至 69 章的对象，四章指标可以直接对照：

$$y''=-2y'+K(y)u,\qquad K(y)=4+6y \tag{71-1}$$

采样 10 ms、纯延迟 5 拍、限幅 ± 1.2，工作点从 1.5 阶跃到 2.0，仿真 6 s。教师的四条规则取在 $y=1.5,\,1.9,\,2.3,\,2.7$，对应局部增益 $K_i=13.0,\,15.4,\,17.8,\,20.2$；每个局部模型用极点 $(-3,-4)$ 配置，得到四组反馈增益：

规则	局部增益 K_i	k_1	k_2
1	13.0	0.9231	0.3846
2	15.4	0.7792	0.3247
3	17.8	0.6742	0.2809
4	20.2	0.5941	0.2475

表 71-1 四规则 TS-PDC 的局部增益与反馈增益

这张表里藏着一个容易被忽略的事实：四组增益并不是四条独立的控制律，它们恰好是 $k_1=12/K$、$k_2=5/K$ 的采样，也就是说 PDC 在这个单变量对象上**等价于增益调度**——用当前的 $K(y)$ 去除掉增益变化。这解释了为什么 71.6 节的规则数扫描里，规则从 2 条加到 7 条，IAE 只从 0.2884 动到 0.2883：插值已经把非线性抵消掉了，再加规则只是把同一条曲线切得更细。

按 71.3 节的流程走完，最终的一组数字是：单规则名义模型的公共衰减率 2.9091，四规则只查对角 3.0000，四规则查全部 16 对 2.5716；三种表示的 IAE 分别是 0.2883、0.3529、0.3014；速度屏障把峰值速度从 3.643 压到 1.000，IAE 从 0.3949 涨到 0.7532；Shapley 的三份贡献是 45.6%、51.6%、2.7%，效率性残差 1.11×10^{-16}（也就是机器精度）；监控阈值 64.15，标称段误报率 0.22%，卡滞的报警延迟 620 ms。

本章的路线

71.2 节把教师的定义、公共李雅普诺夫、小增益、屏障函数、Shapley 与监控统计量写成公式，并说清每个判据“能证明什么、不能证明什么”。71.3 节给六步设计流程，重点是“选表示”与“选判据”必须同时做。71.4、71.5 节分连续与离散两种视角：连续侧讲相对阶与屏障函数的推导，离散侧讲延迟、量化与一步投影，两节合起来解释 71.6 节那批数字的来处。71.6 节是全章的实验台，七组数据摆在一起看。71.7、71.8 节讨论九个参数各自的作用与调参顺序。71.9 节给五条改进方向，其中“用抖振而不是误差判失稳”这条来自本章实测。71.10 节讲安全认证、失效模式与监控残差的选择。71.11 节给边界，71.12 节是代码。

71.2 数学原理

教师：前件、后件与加权

TS 模型把一条非线性系统写成若干条“如果—则”规则的加权和。本章的规则只有一条前件变量 y，因为对象 (71-1) 的非线性全部集中在输入增益上：

$$R_i:\quad \text{if } y \text{ is } M_i,\ \text{then } \dot{x} = A x + B_i u \tag{71-2}$$

其中 $x = [y,\ \dot{y}]^{\mathsf{T}}$，$A = \begin{bmatrix} 0 & 1 \\ 0 & -2 \end{bmatrix}$，$B_i = [0,\ K_i]^{\mathsf{T}}$，$K_i = 4 + 6y_i$。$M_i$ 是取在第 i 个规则中心 y_i 上的三角隶属函数：

$$\mu_i(y) = \max\left(0,\ 1 - \frac{|y - y_i|}{\Delta_i}\right), \qquad w_i = \frac{\mu_i}{\sum_{l=1}^{m} \mu_l} \tag{71-3}$$

Δ_i 是半宽，取相邻中心间距即可保证“任意 y 至少被一条规则覆盖、且权重和为 1”。这条归一化不是装饰：它保证了加权和 w_i 构成一个凸组合，而凸组合正是后面所有 LMI 能写成线性形式的前提。

必须承认一个前提：这个 TS 模型是**精确**的，不是近似。因为 (71-1) 里唯一的非线性项 $K(y)u$ 对 u 是线性的，把 $K(y)$ 按 y 分段插值，插值误差就是模型误差。换个对象，比如 $y'' = -2y' + K(y)u^3$，那么无论多少条规则都写不出精确的 TS 形式，后面的判据就退化成“对近似模型成立”。**判据的效力，永远以模型误差为零为前提**——这句话是 71.11 节给边界时的第一句。

PDC：把局部极点拼成一条控制律

并行分布补偿的做法是“每个局部模型配一套反馈，再按同一组权重拼起来”：

$$u = -\sum_{i=1}^{m} w_i(y)\, K_i x, \qquad x = [e,\ \dot{e}]^{\mathsf{T}} \tag{71-4}$$

K_i 由第 i 个局部模型单独做极点配置得到。这里有一个常被忽略的细节：x 用的是误差坐标，不是绝对坐标。因为参考是常值，e 与 y 的导数相同，误差坐标下的 A 与绝对坐标相同，所以极点配置的结果直接可用。

把 (71-4) 代入 (71-2)，得到闭环：

$$\dot{x} = \sum_{i=1}^{m}\sum_{j=1}^{m} w_i w_j \left(A - B_i K_j\right) x \tag{71-5}$$

双重求和是 PDC 分析里所有麻烦的来源。它意味着闭环矩阵不是 m 个，而是 m^2 个——第 i 条规则的被控对象碰上第 j 条规则的控制增益，就产生一个交叉闭环 $A - B_i K_j$。本章 $m = 4$，共 16 个。

公共李雅普诺夫：一个 P 管住 16 个矩阵

若能找到一个正定矩阵 P，使全部 16 个交叉闭环都满足

$$\left(A - B_i K_j\right)^{\mathsf{T}} P + P\left(A - B_i K_j\right) + 2\alpha P \prec 0, \qquad i, j = 1, \dots, m \tag{71-6}$$

则闭环 (71-5) 在原点渐近稳定，且 $\|x(t)\| \le \kappa \|x(0)\| e^{-\alpha t}$。证明只用两步：取 $V = x^{\mathsf{T}} P x$，沿 (71-5) 求导得 $\dot{V} = \sum_i \sum_j w_i w_j\, x^{\mathsf{T}} \left[A_{cl,ij}^{\mathsf{T}} P + P A_{cl,ij}\right] x$；因为 $w_i w_j \ge 0$ 且和为 1，(71-6) 逐项成立就推出 $\dot{V} \le -2\alpha V$。

这个证明里最关键的一步不是矩阵不等式，而是**权重非负**。$w_i w_j \geq 0$ 让“逐项成立”可以推到“加权和成立”。所有 TS 稳定性结论都踩在这块石头上，一旦前件被改成可能取负的基函数（比如某些径向基网络带负权），整套推导立刻失效。

式 (71-6) 里的 α 是衰减率，单位是 $1/\mathrm{s}$。它有一个直观解释：V 按 $e^{-2\alpha t}$ 衰减，$\|x\|$ 按 $e^{-\alpha t}$ 衰减，所以 $1/\alpha$ 是“误差缩小到 $1/e$ 需要的时间”的量级。α 越大，收敛越快，但可行域越小，因为要求 16 个矩阵在同一个 P 下都“足够负”，本身就是互相牵制。

求最大 α 的做法是二分：对给定的 α，解一次可行性问题（有没有 P 满足全部约束），可行就抬下界、不可行就压上界，24 次迭代足够收到四位有效数字。这里有一个数值细节必须处理：**判据对 P 的尺度不变**——把 P 乘以任意正数，(71-6) 左右两侧同乘该数，不等号方向不变。所以不能用 $\mathrm{tr}(P)=1$ 去归一化（那是个等式约束，会让内点法在边界附近失去精度），而要用 $P \succeq I$。本章的实测中，用 $\mathrm{tr}(P)=1$ 时求解器反复报“解可能不准确”，换成 $P \succeq I$ 之后 24 次二分的裁决全部干净。

小增益：换一种不需要结构的判据

LMI 要求知道 A、B_i、K_j。网络只有一个权重矩阵，没有“局部模型”这个概念，于是只能退回更粗的判据：把网络看成回路里的一个静态非线性环节，用它的 Lipschitz 常数与对象的增益相乘。

回路的小增益条件是

$$L \, \|P(s)\|_\infty < 1 \tag{71-7}$$

其中 L 是网络作为映射 $(e, \dot{e}) \mapsto u$ 的 Lipschitz 常数，$\|P(s)\|_\infty$ 是对象从 u 到 y 的传递函数在虚轴上的最大模。对 (71-1) 在 $y=2$ 处线性化，$P(s) = K/(s^2+2s)$，其频响模为

$$|P(j\omega)| = \frac{K}{\omega\sqrt{\omega^2+4}} \tag{71-8}$$

对 ω 求极值，$\omega=\sqrt{2}$ 处取到最大值 $\|P\|_\infty = K/(\sqrt{2}\cdot\sqrt{6}) = 4.6188$（$K=16$）。

L 的严格上界可以直接由权重算出。本章的网络是“随机特征 + 线性读出”：$u = W_2 \tanh(W_1 z + b_1)$，$\tanh$ 的导数不超过 1，于是

$$L \leq \|W_2\|_2 \, \|W_1 S\|_2 \tag{71-9}$$

$S = \mathrm{diag}(0.30,\ 1.0)$ 是输入归一化矩阵。这个上界的问题在 $\|W_1\|_2$ 上：W_1 是随机矩阵，宽度越大、谱范数整体越大，上界随宽度上移（71.6 节实测从 1.82 涨到 306.6，中间还因随机初值出现回落）；而实测的 L 几乎不随宽度变（1.29 到 1.71），因为 $\tanh$ 在随机特征下大量饱和，实际斜率远小于 1。本章实测的上界与实测 L 差了两个数量级——**判据自然永远不满足**。

这就是小增益的定位：它是一个“能排除”的判据。条件不满足时说明不了任何问题（本章 5 种宽度全部不满足），条件满足时则一定稳定。工程上它的正确用法是做筛除——如果某个网络的 $L\|P\|_\infty$ 已经小于 1，那就不必再跑仿真；如果大于 1，也不能据此判它不稳定，得换更细的判据或者做实测。

量化：两种误差，两种失效

数字实现里有两处量化，性质完全不同。

第一处是**空间量化**：查表把连续的 $(e, \dot{e})$ 平面切成 $n \times n$ 个格子，格子内用最近格点的值。这相当于在控制量上叠加了一个幅度为“相邻格点差”的台阶，台阶宽度随格子变粗而变大。第二处是**幅度量化**：控制量用 b 位定点

表示，步长为

$$q = \frac{2U_{\max}}{2^b} \tag{71-10}$$

两种量化的失效表现也不同：空间量化产生**抖振**（控制量在格子边界上来回跳），幅度量化产生**极限环**（误差收敛到零附近后，被 $q/2$ 的死区卡住，形成小幅持续振荡）。71.6 节的实测把两者分开列：网格从 161 降到 11，Δu 的 RMS 从 0.01772 涨到 0.02083（涨 17.5%）；位数从 16 降到 6，末段误差 RMS 从 0 涨到 0.0117。

一个容易算错的细节：q 的公式里是 2^b 而不是 $2^b - 1$。这是“满量程对应 2^b 个间隔”的约定；若按 $2^b - 1$ 算，6 位时步长会从 0.0375 变成 0.0381，差 1.6%，在 8 位以下无伤大雅，在 12 位以上就足以让“末段残差是否为零”的结论翻转。

屏障函数：给“不许越界”一个可算的形式

安全层的任务是：给定一条标称控制律 u_{nom}，在“尽量少改”的前提下，保证某个状态约束不被破坏。控制屏障函数（control barrier function，CBF）把它写成一张“不许下穿零”的证书。

对一阶约束（约束只含速度），取 $h(x) = v_{\max}^2 - \dot{y}^2$。$h \geq 0$ 就是约束满足，$h < 0$ 就是越界。要求存在 $\kappa > 0$ 使

$$\dot{h} + \kappa h \geq 0 \tag{71-11}$$

只要这个条件成立，h 就永远不会从正穿到负——因为 $h = 0$ 时不等式退化为 $\dot{h} \geq 0$，即“到了边界只能往外走，不能往里走”。这就是“前向不变性”。

对本章的对象，$\dot{h}$ 对 u 是仿射的：

$$\dot{h} = -2\dot{y}\,\ddot{y} = -2\dot{y}\left(-2\dot{y} + K(y)u\right) = 4\dot{y}^2 - 2\dot{y}K(y)u \tag{71-12}$$

把它代进 (71-11)，得到一个关于 u 的一元一次不等式：

$$\underbrace{-2\dot{y}K(y)}_{\text{coef}} \cdot u \;\geq\; -\underbrace{\left(4\dot{y}^2 + \kappa h\right)}_{\text{rhs}} \tag{71-13}$$

系数为正时解是 $u \geq u_b$，系数为负时解是 $u \leq u_b$，u_b 是使等号成立的那个值。所以最小干预解就是**单边投影**：

$$u_{\text{safe}} = \begin{cases} \max\left(u_{\text{nom}},\, u_b\right), & \text{coef} > 0 \\ \min\left(u_{\text{nom}},\, u_b\right), & \text{coef} < 0 \end{cases} \tag{71-14}$$

这个式子的计算量是“一次乘法、一次比较”，在任何嵌入式上都能实时跑。它的代价是只保证“下一拍不越界”这个一阶条件，对纯延迟无能为力——本章的延迟是 5 拍，也就是说这一拍算出的 u 要 5 拍后才生效，那时 h 可能已经变负。工程上有两种补救：把安全层放在延迟缓冲之后（本章的做法，保证的是“实际执行量”），或者按延迟滚动预测 5 拍后的状态（71.9 节）。

若约束只含位置（$|e| \leq h_0$），$h = h_0^2 - e^2$ 对 u 的相对阶是 2，要写成 $\ddot{h} + a_1\dot{h} + a_0 h \geq 0$ 才封闭。它的推导只是把 (71-12) 再求一次导，形式仍然是仿射的，投影公式一模一样。本章的实验选了速度约束（相对阶 1），原因是相对阶越低，需要的信息越少、对延迟越不敏感；位置约束的实测见 71.4 节。

Shapley：把一拍控制量拆成三份

三个输入 e、$\dot{e}$、$y-2$，基线 $z_0=(0,0,0)$，价值函数 $v(S)=f(z_S)$，z_S 表示“把 S 里的输入取真值、其余取基线”。Shapley 值是

$$\phi_i=\sum_{S\subseteq N\setminus\{i\}}\frac{|S|!\,(n-|S|-1)!}{n!}\,[v(S\cup\{i\})-v(S)] \tag{71-15}$$

$n=3$ 时每个 ϕ_i 是 4 项之和，全部三个共 12 项，正好是 8 个联盟各算一次前向的两倍（每对联盟被两个 i 各用一次）。这个定义有三条性质值得记住：**效率性**要求 $\sum_i\phi_i=v(N)-v(\varnothing)$；**对称性**要求等价输入拿同样的份额；**零贡献性**要求“加不加它都一样”的输入拿零。

效率性是最硬的验收指标，因为它可以被精确检查。本章实测的效率性残差是 1.11×10^{-16}，也就是双精度浮点的舍入误差——只要公式没写错，这个数必然在 10^{-15} 量级；如果它到了 10^{-3}，说明权重算错了或者联盟枚举漏了。

Shapley 的代价是指数复杂度。$n=3$ 时 8 次前向，可以每拍都算；$n=10$ 时 1024 次，只能离线；$n=20$ 时一百万次，必须采样近似。本章之所以能“精确”，纯粹因为输入只有三个——这是一个必须诚实说明的边界。

遮挡：一个更便宜、但更粗的检验

遮挡（occlusion）的做法是：把第 i 个输入换回基线，看输出改变多少：

$$d_i=\frac{1}{N}\sum_{k=1}^{N}\left|f(z^{(k)})-f(z^{(k)}_{\setminus i})\right| \tag{71-16}$$

它只需要 n 次前向（而不是 2^n 次），所以大模型上也跑得动。它的缺点是没有效率性：d_i 们加起来不等于任何东西，量纲是“输出改变量”而不是“份额”。两者一起用的价值在于**交叉验证**：若 Shapley 的排序与遮挡的排序一致，说明归因不是权重分解的假象；若不一致，多半是输入之间强相关，Shapley 把交互项摊薄了，而遮挡保留了大头。

解释的稳定性：把归因当统计量

单次归因只是“这一拍的答案”。要判断它能不能进验收报告，得看它在输入有小扰动时是否稳定。做法是给输入加噪声 σ、重算归因、比较两次归因的排序：

$$\rho=\mathrm{Spearman}\left(\phi(z),\,\phi(z+\delta z)\right),\qquad \mathrm{top1}=\mathbb{1}\left[\arg\max_i|\phi_i(z)|=\arg\max_i|\phi_i(z+\delta z)|\right] \tag{71-17}$$

秩相关衡量整体排序，首位保持率衡量“最要紧的那个有没有指错”。两个指标一起看，是因为它们会分道扬镳：本章 $\sigma=0.05$ 时秩相关还有 0.950，首位保持率却已经掉到 0.90——整体顺序没乱，最要紧的那一个开始错了。

监控：一个能标定的距离

运行时监控要回答“当前这一拍是否偏离了标称”。最便宜的做法是把几个残差拼成向量 z，用标称段的均值与协方差算马氏距离：

$$D^2(z)=(z-\mu)^{\mathsf{T}}\Sigma^{-1}(z-\mu) \tag{71-18}$$

若 z 服从多元正态，D^2 服从自由度等于维数的 χ^2 分布，阈值可以直接取分位数（二维 99.9% 是 13.82，三维是 16.27）。本章实测这个假设不成立：标称段残差是量化纹波，尾部比 χ^2 厚得多，用 13.82 时误报率 2.44%，实测的 99.9% 分位是 57.88。所以工程上必须用“现场标定”的阈值，χ^2 分位只能作为下限。

单拍超阈不足以下结论（噪声也会超），还要加一个连续计数：连续 N 拍超阈才报警，本章取 $N = 5$。这个参数的作用是把“偶发”与“持续”分开，代价是最少要晚 N 拍才能报警。

五个判据各自能证明什么

把本章用到的判据排成一张表，重点看最后两列——“证不了什么”比“能证什么”更容易被忘掉。

判据	需要的输入	能证明	证不了
公共 Lyapunov（LMI）	全部 A_i、B_i、K_j	指数收敛，速率 α	延迟、量化、饱和
小增益 $L\|P\|_\infty < 1$	状态矩阵、P、L 的界	在斜率界内的鲁棒稳定	具体的收敛速率，且保守
屏障函数	相对阶、h 的表达式	集合前向不变（在集合内）	从集合外进入，以及延迟后的 u
Shapley 效率性	模型可查询	分摊之和等于总增量	分摊是否“合理”
马氏距离	标称段的均值与协方差	偏离标称的统计显著性	故障类型，以及稳态不能观的增益损失

这张表解释了本章为什么每个结论都要配一句限定语：五条判据没有一条能单独覆盖“稳定 + 安全 + 可解释”，它们各自只照亮一块。工程上的做法不是找一条更强的判据，而是**把五条的适用域叠起来**，看叠出来的区域是否覆盖实际工况；覆盖不到的地方，就写明“此处只有实测，没有证书”。

71.3 设计步骤

六步的次序与理由

把前面那些公式串成流程，一共六步。次序不能随意调换，因为后一步的前提是前一步的产物：

步	做什么	产物	走不下去时的出口
1	划工作域、定规则中心	前件划分与局部模型	缩小域或加密规则
2	逐条极点配置	四组反馈增益	放宽极点或降阶
3	选表示并蒸馏	可部署的控制律	换表示或加宽
4	三层判据验证	α、$L\|P\|_\infty$、量化解	退到分段 P 或实测
5	加安全层	投影后的执行量	降 κ 或换约束
6	归因、遮挡、监控	解释报告与阈值	换基线或加激励

表 71-2 六步设计流程与失败出口

第一步与第二步合起来就是“造教师”。它看起来最像传统设计，却决定了后面四步的天花板：工作域划得太大，局部模型就不准，第四步的 LMI 再宽松也没意义；极点取得太激进，第四步的 α 可能连 1.0 都拿不到。

第一步：划工作域，别把全域都塞进来

工作域的上下界来自工艺，不来自数学。本章取 $y \in [1.2,\ 3.0]$，因为 $K(y)$ 在这个区间内从 11.2 变到 22.0，一直是正的；$y < -2/3$ 时 K 变号，对象反向，那是另一个系统，不该用同一套规则去覆盖。

划完域之后定规则数。规则数不是越多越好，本章实测规则从 2 条加到 7 条，IAE 只从 0.2884 动到 0.2883（差 0.03%），因为 PDC 在单变量情形下等价于增益调度，插值精度早就够了。规则真正买到的是**可解释性**：4 条规则对应 4 句话，现场工程师愿意读；40 条规则就没人读了，那时“结构级可解释”已经名存实亡。

一个实操建议：规则中心取成等间距，半宽取相邻间距。不等间距只在“某段增益变化特别剧烈”时才需要，而本章的 $K(y) = 4 + 6y$ 是线性的，等间距就是最优。

第二步：极点配置要留裕度

每个局部模型用同一组极点 $(-3, -4)$ 配置，得到表 71-1 的四组增益。这里有三点经验。

极点不能取在对象自然极点附近。 对象 (71-1) 的自然极点是 0 与 -2，把闭环极点配到 -2.1 附近，等于几乎不动它，闭环对扰动毫无抵抗力；配到 -30 又会让控制量在第一拍就撞限幅。本章取 -3、-4，闭环时间常数约 0.3 s，比 5 拍延迟（50 ms）慢一个数量级，这个比例是安全的：**闭环带宽应当低于 $1/(5\tau_d)$，τ_d 是纯延迟**，否则延迟的相位滞后会把相位裕度吃光。

四组极点应当相同，不要“按段调”。 若第 1 段取 $(-3, -4)$、第 4 段取 $(-8, -10)$，闭环矩阵的差异会体现在公共 P 的可行性上——第四步很可能算不出 α。让局部模型只在 B_i 上不同、K_i 按 $1/K_i$ 缩放，是让 16 个交叉闭环“长得像”的最省事办法。

极点配置之后要立刻检查限幅。 本章阶跃 0.5 时峰值控制量 1.03，未撞限幅；阶跃 1.1 时就撞上了。若工艺要求大幅阶跃，要么降极点，要么加参考整形（把阶跃改成斜坡），不能靠“反正有限幅”糊过去——限幅一介入，第二步的极点配置就失效了，闭环变成了一个非线性系统，第四步的 LMI 也就不再成立于实际轨迹上。

第三步：选表示，同时选证明方式

这一步是本章与前面各章最大的不同：选表示时不能只看性能，还要看“这个表示还能不能被证明”。

表示	参数规模	单拍计算	能用的判据	本章 IAE
教师（规则库）	4 组增益	4 次加权	LMI、小增益	0.2883
ELM 网络（宽 8）	8×3 + 8	8 次 tanh	小增益	0.3529
查表（41×41）	1681 个数	1 次查表	量化误差	0.3014
查表（插值）	1681 个数	2 次插值	量化误差	0.2968

表 71-3 四种表示的性能与可证明性

表里最值得注意的是“能用的判据”这一列。规则库能直接写 LMI，是因为它有局部矩阵；网络只有权重，只能算 Lipschitz；查表连解析式都没有，只能谈格子密度。**性能差 22%，证明成本差一个量级**——这就是选表示时的真实权衡。

本章的结论是：如果工作域固定、规则数不超过 10，直接部署规则库是最优解。它性能最好、可证明性最好、内存最小（4 组增益共 8 个数）。网络与查表的价值在“规则数必须很大”或“前件变量很多”的场合，那时 LMI 的 16 个交叉项会变成 m^2 个，规则库反而不可行。

训练（或生成）表示时有两个坑。第一，训练样本要覆盖**闭环会走到的区域**，不能只取一条轨迹上的点。本章的 ELM 一开始只用一条轨迹训练，IAE 0.4762、5.49 s 才稳住；加上 900 个覆盖 $e \in [-0.5, 0.5]$、$\dot{e} \in [-1.5, 1.5]$ 的均匀样本后，IAE 降到 0.3529。第二，随机特征的尺度要归一化。W_1 的行向量单位化后乘 1.5，$\tanh$ 才工作在有效区间；不归一化时大量隐元饱和，网络输出退化成常数。

第四步：三层判据，从强到弱

验证不是“跑一遍看看”，而是按三层依次做，前一层不通过才退到下一层。

第一层 LMI。 对四规则全配对求最大 α，本章得 2.5716。若 $\alpha \le 0$，说明连可行 P 都没有，先回去改第二步的极点，不要急着改规则数。

第二层小增益。 算出 L 与 $\|P\|_\infty$，看乘积。本章五种宽度全部不满足（5.94 到 7.91），这一层给不出结论，只能记录在案。

第三层实测。 跑阶跃、看末段残差与 Δu。本章的实测边界是 $\beta_{\text{crit}} = 3.5$（把网络输出乘以 3.5 倍后抖振超标），与小增益预测的 0.16 差 22 倍。

三层都过不了时不要硬上，应该退回去改结构：要么换成规则库（第一层就能过），要么把网络改成谱归一化（把上界压下来，见 71.9 节）。

第五步：安全层，位置比参数重要

安全层的参数只有两个（$v_{\max}$ 与 κ），但“放在哪”比“取多少”更重要。

本章把安全层放在**延迟缓冲之后**，也就是对实际将要执行的量做投影。这样保证的是“执行量满足约束”，代价是它看不见“已经排队等着执行的那 5 拍”。若放在缓冲之前，被投影的是指令，5 拍后执行的可能是投影前的旧值，保证就漏了。

位置定好之后才谈参数。$v_{\max}$ 由工艺定（本章取 1.0，对象阶跃时自然峰值是 3.643），κ 由“介入的果断程度”定：κ 大，介入早、代价大；κ 小，介入晚、可能来不及。本章 κ 从 4 加到 16，IAE 代价从 106.3% 降到 83.1%，但介入拍数只从 107 降到 95——**κ 主要影响的是“改多少”，不是“改几次”**。

第六步：解释与监控，先定基线

解释侧的第一步不是算 Shapley，而是**定基线**。基线是“没有这个输入时系统会怎样”的替身，本章取 $z_0 = (0, 0, 0)$，含义是“误差为零、误差变化率为零、工作点在标称点”。这个基线对应“理想稳态”，现场工程师容易接受。

基线定错，归因全错。若把基线取成“上一拍的实际输入”，得到的份额是“这一拍相对上一拍变了多少”，那是增量归因，与“这一拍为什么是 -0.41”不是同一个问题。两者的排序可能一致也可能不一致，但报告的读法完全不同，验收时必须写清基线。

监控侧的第一步是**选残差**。本章的核心发现是：残差选 $(e, \dot{y})$ 时，执行器增益掉 30% 完全看不见；加上 u 之后，卡滞的报警从 910 ms 提前到 620 ms，可增益损失仍然看不见。所以残差的选择要按“你最怕哪种故障”来定，而不是按“哪个量好测”来定。增益类故障需要模型残差（预测的 $\ddot{y}$ 与实测的差），本章没有做，71.10 节会说明为什么。

把次序调反会怎样：一个具体反例

假设先加安全层、再配极点。$v_{\max} = 1.0$、$\kappa = 8$ 的屏障会先把速度按住，此时看闭环响应，上升沿被削平、调节时间拉长到 1.11 s 量级。工程师看到“响应太慢”，自然会去加快极点，比如从 $(-3, -4)$ 改到 $(-8, -10)$。改完

再跑，峰值速度更高，屏障介入更深，响应还是慢——于是继续加快极点。这个循环会一直走到限幅饱和，最后得到一个“极点极快、但大部分时间被屏障按住”的系统。

按正序做就不会进这个循环：先配极点、看无屏障时的峰值速度 3.643，立刻就能判断 $v_{\max}=1.0$ 这个约束与当前极点不匹配。该做的是回头谈工艺（能否放宽 $v_{\max}$）或改参考（阶跃改斜坡），而不是在极点与 κ 之间来回找平衡。次序的价值就在这里：它让不匹配在第一屏就暴露，而不是在第五轮调参之后才暴露。

每一步都要留下的三个数

六步走完，交付物不只是一套参数，还有每一步的三个数。缺了它们，三个月后没人能解释“当时为什么取 4 条规则”。

步	必须留下的数
1	工作域上下界、K 在该域的取值范围、规则数
2	极点、四组增益、阶跃时的控制峰值
3	表示的 IAE、末段 RMS、调节时间
4	α_{all}、$L\|P\|_\infty$、β_{crit}
5	$v_{\max}$、κ、IAE 涨幅与介入拍数
6	效率性残差、遮挡排序、目标 σ 下的首位保持率

这张表的价值在于把“调参”变成“记录”。同一套系统换人接手，先看这六个格子：六格齐全，接手成本是几小时；缺一半，等于重做一遍。它也是评审时最省事的材料——每个数都能指回一次实验，而不是指回某个人的记忆。

71.4 连续情形

为什么教科书先讲连续

模糊控制的经典文献几乎都用连续时间写：$\dot{x}=\sum_i w_i(A_i x+B_i u)$，控制律 $u=-\sum_j w_j K_j x$，然后给一个二次型 $V=x^{\mathsf T}Px$。这样做有两个好处：结论的形式干净（没有采样矩阵），几何解释直接（椭球与向量场）。本章的对象是连续系统加零阶保持器，两种写法都能用，所以先用连续形式把几何讲清，再在 71.5 节把采样与延迟补上。

连续形式的闭环是

$$\dot{x}=\sum_{i=1}^{m} w_i(y)\left(A_i x+B_i u\right),\qquad u=-\sum_{j=1}^{m} w_j(y)K_j x \tag{71-19}$$

代进去仍然得到 (71-5) 那样的双重求和，只是 A_i 一般也随规则变（本章 $A_i\equiv A$，因为非线性只在输入通道）。稳定条件与 (71-6) 完全一样，只是 $A_{cl,ij}=A_i-B_iK_j$。

公共 P 的几何：一个椭球要装下 16 个向量场

把 (71-6) 写成几何语言：$V=x^{\mathsf T}Px$ 的等值面是一族同心椭球；条件 $\dot V<-2\alpha V$ 说的是“每条闭环轨线都以不低于 $e^{-\alpha t}$ 的速率往内层椭球走”。

现在有 16 个闭环矩阵，每个矩阵对应一个向量场。要求是：**同一个椭球族，对 16 个向量场都成立**。这就是保守性的来源——如果允许每个矩阵用自己的椭球，可行性大得多；一旦要求共用，椭球的形状就被“最别扭的那个矩阵”牵着走。

本章的数字说明了这一点：单规则时 $\alpha = 2.9091$；四规则只查对角时 $\alpha = 3.0000$，居然更高；四规则查全部 16 对时降到 2.5716。对角情形反而比单规则高，原因是“只查对角”实际上放松了约束（只要求 $A - B_i K_i$ 稳定，不管交叉项），可行集变大，α 自然能抬高。**这个数不能当成性能指标读**：α 高不等于闭环好，只等于“约束少”。读表时必须先看它查了几对。

α 与极点有一个近似对应。二阶系统的极点 $-\zeta\omega_n \pm j\omega_n\sqrt{1-\zeta^2}$ 对应的衰减率是 $\zeta\omega_n$，即实部的绝对值。本章极点 $(-3, -4)$ 的实部是 -3 与 -4，所以 α 的理论上限在 3 附近；实测单规则 2.9091、对角 3.0000，正好贴着这个上限。这说明**极点配置已经把衰减率定死了，LMI 只是在验证“共用椭球能不能达到这个速率”**，它不会凭空造出更快的收敛。

这条对应关系还有一个实用价值：若 LMI 给出的 α 远低于极点实部，说明交叉项在拖后腿，此时应该做的是“让局部模型更接近”（比如在中间插规则、或把极点取得更一致），而不是继续放宽 LMI 的求解器设置。

连续屏障：一阶与二阶的差别

连续时间里，屏障函数的经典形式是

$$\dot{h}(x, u) \geq -\kappa h(x) \tag{71-20}$$

与 (71-11) 等价。区别在 h 的选择，而 h 的选择决定了“$\dot{h}$ 里出现 u 的几次方”。

一阶（**速度约束**）：$h = v_{\max}^2 - \dot{y}^2$，$\dot{h}$ 里 u 只出现一次，见 (71-12)。这类约束的好处是“永不奇异”：只要 $\dot{y} \neq 0$，u 的系数就不为零，总有解。坏处是它只管速度，不管位置——速度为零而位置在界外时，它给不出任何约束力。

二阶（**位置约束**）：$h = h_0^2 - e^2$，一次求导得 $\dot{h} = -2e\dot{e}$，里面没有 u；再求一次导才出现：

$$\ddot{h} = -2\dot{e}^2 + 4e\dot{y} - 2eK(y)u \tag{71-21}$$

于是条件要写成二阶形式：

$$\ddot{h} + a_1\dot{h} + a_0 h \geq 0 \tag{71-22}$$

a_0、a_1 相当于给 h 配了一对极点，取 $a_0 = 4$、$a_1 = 4$ 就是临界阻尼。这个式子的解仍然是 (71-14) 那样的单边投影，只是系数变成 $-2eK(y)$、右端多两项。

二阶形式的麻烦在于**初值必须落在集合内**。本章的算例是阶跃 1.1，$t = 0.5$ s 那一拍 e 从 0 跳到 1.1，$h = h_0^2 - e^2$ 当场为负——集合已经被破坏，屏障只能在“已经越界”的前提下尽量往回拉，前向不变性无从谈起。这不是参数没调好，而是“参考信号不连续”与“约束含位置”这两件事本身不相容：**位置约束要成立，参考必须先整形**。

整形也不是万灵药。本章试过 0.4 s 与 0.8 s 的斜坡：0.4 s 斜坡加 $h_0 = 0.25$ 时，最大偏差仍是 0.561，全部越界；原因不是斜坡没用，而是**斜坡速率比闭环能跟踪的速率快**，e 在斜坡段就积累到超过 h_0。要让位置约束真正成立，斜坡时间必须长到 e 的峰值不超过 h_0，那意味着响应时间被拉长数倍——这通常是工艺不肯接受的。所以“位置约束 + 阶跃参考”这个组合，要么改参考、要么改约束，没有第三条便宜路。

一阶约束没有这个问题：$h = v_{\max}^2 - \dot{y}^2$ 在 $t=0$ 时 $h = v_{\max}^2 > 0$，初始是满足的。这也是本章选速度约束做实验的实际原因，不只是“相对阶低所以简单”。

连续域里三个容易算错的量

第一，归一化权重的和。三角隶属归一化后 $\sum_i w_i = 1$，于是 $\sum_i \sum_j w_i w_j = 1$。这个恒等式让双重求和保持“凸组合”的性质，是 (71-5) 能写成 16 个矩阵加权和的前提。若忘了归一化（比如用未归一化的高斯隶属），双重求和不再等于 1，闭环矩阵的范数会随 y 变化，LMI 的结论直接失效。

第二，相对阶决定 u 出现在第几阶导数里。速度约束的相对阶是 1，$\dot{h}$ 里 u 的系数是 $-2\dot{y}K(y)$；位置约束的相对阶是 2，要 $\ddot{h}$ 里才出现 u。相对阶越高，需要的状态信息越多、对延迟越敏感、初值要求越严。选约束时先看相对阶，再看别的。

第三，限幅破坏凸组合。这是最隐蔽的一条。LMI 证明的是“16 个 $A_{cl,ij}$ 的凸组合稳定”，但一旦控制量被限幅，实际闭环矩阵就不再是这 16 个的凸组合——限幅相当于把 K_j 换成了一个更小的等效增益，而那个矩阵不在集合里。所以**证书不覆盖饱和区**。本章的适用域里必须写这一条，否则“$\alpha = 2.5716$ 所以稳定”会被误用到限幅深度饱和的工况。工程上的补救是在饱和区切换到一个“已知稳定”的保守控制器，或者把限幅值取得足够大，使正常工作不进入饱和。

为什么本章的 A_i 全都一样

(71-19) 里 A_i 一般随规则变，本章却取 $A_i \equiv A$。这不是偷懒：对象的非线性只出现在输入通道（$K(y)$ 乘在 u 上），$\ddot{y} = -2\dot{y} + K(y)u$ 里 y 与 $\dot{y}$ 的系数都是常数。所以“每条规则一个局部模型”这件事，在本章只体现为 B_i 不同。

这个结构带来两个后果。一是 LMI 的难度全部集中在 $B_i K_j$ 上：16 个闭环矩阵共享同一个 A，彼此差异只来自增益乘积。二是极点配置这一步实际上在给每个 B_i 配一个“抵消 B_i 差异”的 K_i——K_i 与 $K(y_i)$ 近似成反比（表 71-1 里第一分量从 0.9231 降到 0.5941，正好跟着 K 从 13 涨到 20.2 反向走），于是 $B_i K_i$ 几乎不随 i 变，闭环矩阵彼此接近，公共 P 才存在。

反过来，若状态矩阵也随工作点变（摩擦系数随温度变、刚度随负载变），A_i 之间就有差异，公共 P 的可行性会明显变差，本章的 2.5716 不能照搬。能不能照搬，只看一条：**$B_i K_i$ 是否近似为常数**。

小增益判据在频域里是什么

(71-9) 的 $L\|P\|_\infty < 1$ 看着像纯代数，其实是一个扇形条件。把非线性写成“标称线性部分 + 有界扰动”，$\Delta(y) = K(y) - K(y_0)$，则 $|\Delta| \le L|y - y_0|$，闭环画成“线性系统 + 斜率受限的静态非线性”的反馈互联。小增益定理说的是：线性部分的增益与非线性斜率之积小于 1，互联稳定。

这解释了两件事。第一，为什么 $\|P\|_\infty$ 会出现在判据里——它衡量“从扰动到状态”这条通路的峰值增益，而 P 是 Lyapunov 矩阵，两者在此处恰好重合。第二，为什么这个判据比 LMI 保守得多：扇形条件对**所有**满足斜率界的非线性都成立，而本章的非线性是已知的 $K(y) = 4 + 6y$，LMI 通过 B_i 用到了这个具体形式，小增益只用到了“斜率不超过 L”。信息用得少，结论自然弱。

工程含义是：非线性形式不确定、只知道界时，用小增益；非线性已知时，用 LMI。本章把两者都算出来，正是为了把“保守多少”量化成 22 倍这个数，而不是含糊地说一句“小增益比较保守”。

连续 Shapley：从有限联盟到路径积分

连续输入空间里，Shapley 值有一个自然推广：把基线到真值的直线路径参数化，$\gamma(s) = z_0 + s(z - z_0)$，$s \in [0,1]$，则

$$\phi_i = \int_0^1 \frac{\partial f(\gamma(s))}{\partial z_i}\,\mathrm{d}s \tag{71-23}$$

这条式子叫 Aumann-Shapley 值，它满足效率性（沿路径积分等于端点差），但**不满足对称性**——两个功能等价的输入若梯度不同，拿到的份额不同。所以连续版本与离散版本不是同一个东西：离散 Shapley 的“对称性”来自对所有联盟取平均，连续版本的路径积分没有这层平均。

工程上用哪个，取决于能不能接受“同一拍算两次得不同结果”。离散版本要 8 次前向，结果确定；连续版本要用数值积分（比如 20 个梯形点），结果与路径有关。本章用离散版本，因为它可复现、可验收。

71.5 离散情形

零阶保持：离散化的两个后果

控制器每 10 ms 算一次，两次之间执行器保持上一个值，这就是零阶保持。对象的离散化是

$$A_d = e^{A T_s}, \qquad B_d = \int_0^{T_s} e^{A\sigma}\,\mathrm{d}\sigma\, B \tag{71-24}$$

代入本章的 A、B 与 $T_s = 0.01$，得 $A_d = \begin{bmatrix} 1 & 0.0099 \\ 0 & 0.9802 \end{bmatrix}$，$B_d = [8.0\times10^{-4}K,\ 0.0099K]^{\mathsf{T}}$。这两个数与连续情形有两个实质差别。

第一，**B_d 的第一个分量不再是零**。连续时间里 u 只通过 $\ddot{y}$ 影响系统，位置的相对阶是 2；离散化之后，一拍之内 u 已经能改变位置（$8.0\times10^{-4}K \approx 0.013$，量很小但非零），位置对 u 的相对阶变成 1。这意味着“位置约束”在离散实现里其实可以用一阶形式写，不必走二阶——但代价是那个系数极小，数值上很敏感。

第二，**A_d 的特征值是 1 与 0.9802**，一个在单位圆上、一个在圆内。这就是“二阶无静差”在离散里的样子：$z = 1$ 这个特征值对应“常数输入下输出线性增长”，它不会自己衰减，必须靠反馈把它推到圆内。这也是为什么 71.3 节强调“极点不能配在自然极点附近”——配到 0.99 附近时，闭环虽然稳，但对扰动的恢复要几百拍。

离散李雅普诺夫：把指数换成幂

连续条件是 $A_{cl}^{\mathsf{T}}P + PA_{cl} + 2\alpha P \prec 0$，离散对应的是

$$A_{cl}^{\mathsf{T}} P A_{cl} - e^{-2\alpha T_s} P \prec 0 \qquad \text{对全部 } (i,j) \tag{71-25}$$

右端从“加 $2\alpha P$”变成“减 $e^{-2\alpha T_s}P$”，因为连续时间的 $e^{-\alpha t}$ 在采样点上就是 $\left(e^{-\alpha T_s}\right)^k$。$T_s \to 0$ 时 $e^{-2\alpha T_s} \approx 1 - 2\alpha T_s$，两式只差一个 T_s 因子，符号判断一致，所以本章直接用连续公式求 α，不再重解一遍离散问题。T_s 大到 10 ms 时这个替换仍然安全，因为判据只关心“左边是否负定”，而 T_s 只改变右端的量级，不改变正负。

(71-25) 不是线性矩阵不等式，因为 P 出现在二次型里。化成 LMI 要用 Schur 补，或者直接用变量替换 $Q = P^{-1}$ 左乘右乘。本章的实现走 CVXPY，写的是 $A_{cl}^{\mathsf{T}}PA_{cl} \prec \lambda P$ 这种形式，求解器内部自动处理。这一点在代码

上省事，在数值上要小心：λ 接近 1 时，问题接近可行边界，内点法容易报“解可能不准确”，需要把 P 归一化（见 71.2 节的 $P \succeq I$）。

连续公式能借用到多粗的采样

下面的问题很实际：既然离散判据只是把 $2\alpha P$ 换成了 $e^{-2\alpha T_s}P$，什么时候可以不算离散、直接借连续的结果？

把两边在同一 P 下比较：连续条件等价于 $A^{\mathsf{T}}PA \prec (1-2\alpha T_s)P$ 再丢掉 $O(T_s^2)$ 项，也就是拿 $1-2\alpha T_s$ 代替 $e^{-2\alpha T_s}$。$T_s = 10$ ms、$\alpha = 2.5716$ 时，$2\alpha T_s = 0.0514$，丢掉的项约 $0.5 \times 0.0514^2 = 1.3 \times 10^{-3}$，相对右端的 0.949 是 0.14%，不影响可行与不可行的判断。同样是这个 α，T_s 放到 50 ms，$2\alpha T_s = 0.257$，丢掉的项涨到 3.2%，$e^{-2\alpha T_s} = 0.773$ 与 $1-2\alpha T_s = 0.743$ 已经差出 4%；此时借连续公式会把 α 算大，结论偏乐观。

所以本章“直接借连续 α”只在 10 ms 这个量级上成立，它不是一条普遍规律。采样一旦放到 20 ms 以上，正确做法是重解 (71-25)，而不是把连续结果乘个系数；重解的成本也就是把 LMI 里的 P 换成 $A_d^{\mathsf{T}}PA_d - \lambda P \prec 0$，代码只有一行之差。

一拍的时序账

控制器的每一个周期里，时间是这样花掉的：传感器在 t_k 采样，控制量在 $t_k + \delta$ 算完（δ 是计算耗时），执行器在下一次更新时才真正改变输出。若 δ 小于一个采样周期，这一拍仍算在 t_{k+1} 生效，等效延迟是 1 拍；若 δ 超过一个周期，就再丢一拍。本章的 `DELAY=5` 已经把这一拍算在内，所以“5 拍”对应的是 50 ms 的总滞后，而不是“缓冲里放 5 个数、再额外加 1 拍”。

这条账要算对，否则前面所有延迟数字都会偏一拍。更要紧的是抖动：实际系统的 δ 不会恒定，δ 的分布若跨过采样周期的整数倍，等效延迟就会在 d 与 $d+1$ 之间跳。工程上按最坏情形取 d，而不是按均值四舍五入，因为判据对延迟是盲的（见下一小节），偏一拍的代价只能靠实测暴露。

延迟：LMI 看不见的那一半

纯延迟在连续域是 $e^{-s\tau}$，闭环特征方程是

$$1 + C(s)P(s)e^{-s\tau} = 0 \tag{71-26}$$

这是个超越方程，有无穷多个根，LMI 处理不了。工程上有三条路：Padé 近似（把 $e^{-s\tau}$ 换成有理函数，再用 LMI）、Lyapunov-Krasovskii 泛函（把延迟当成一个状态，需要额外假设 $\dot{\tau} \le 1$）、或者干脆不算、用实测扫延迟。

本章走第三条路，因为它最能暴露问题。用同一组增益，只把延迟从 1 拍加到 50 拍：

延迟/拍	延迟/ms	IAE	末段 RMS	最大偏差
1	10	0.2887	0.0000	0.5000
5	50	0.2883	0.0000	0.5000
10	100	0.2879	0.0000	0.5000
15	150	0.2882	0.0000	0.5000
20	200	0.3105	0.0010	0.5000
30	300	1.4663	0.3326	0.5389
40	400	4.8783	1.6501	2.2346
50	500	8.0227	3.2111	6.3804

表 71-4 纯延迟对闭环性能的影响（同一组增益，LMI 的 α 恒为 2.5716）

表里最关键的一列是“末段 RMS”。1 到 15 拍全是 0，20 拍才冒出 0.0010，30 拍跳到 0.3326，40 拍 1.6501（约 5 倍），50 拍 3.2111。而 LMI 给出的 α 在这整张表里一个数都不变——**判据对延迟完全盲**。

盲的原因是结构性的：LMI 只看 A_{cl} 的特征值，而 $e^{-s\tau}$ 不改变任何特征值，它改变的是相位。相位滞后到 180° 时，原本“负反馈”的那条通路变成了“正反馈”，闭环才振起来。这条边界与增益有关、与延迟有关，与 A_{cl} 的特征值无关，所以任何只看 A_{cl} 的判据都抓不到它。

工程上的处理办法是把延迟折算成“相位预算”：在穿越频率 ω_c 处，延迟贡献 $\omega_c\tau$ 弧度的相位滞后。本章的穿越频率约 1.4 rad/s，延迟 300 ms 时贡献 0.42 rad（24°），仍在裕度内；延迟 500 ms 时 0.70 rad（40°），裕度已经不足。这个估算与表 71-4 的拐点（30 到 40 拍）对得上，比“跑一遍看”快得多。

两种量化，两张表

空间量化的格距由域宽与格数决定：

$$\Delta e = \frac{2e_{\lim}}{n-1}, \qquad \Delta \dot{e} = \frac{2\dot{e}_{\lim}}{n-1} \tag{71-27}$$

每维格数	IAE	末段 RMS	Δu 的 RMS	超调/%
11	0.3054	0.0026	0.02083	0.54
21	0.3064	0.0036	0.01902	0.75
41	0.3014	0.0019	0.01816	0.40
81	0.2992	0.0013	0.01785	0.27
161	0.2983	0.0007	0.01772	0.15

表 71-5 查表格数对性能与抖振的影响（最近格点，双线性见 71.6 节）

表里有一处反常：21 格比 11 格还差（IAE 0.3064 对 0.3054，超调 0.75% 对 0.54%）。原因不是“格子多了反而坏”，而是 21 格的格距（e 方向 0.045、$\dot{e}$ 方向 0.12）与闭环的动态周期相配，轨迹在格子边界上以接近自身的频率来回穿越，抖振被放大；11 格的格距更粗，穿越次数少，反而稳。**格数与性能不是单调关系，格距与闭环动态的相对关系才是**——这一点只能靠扫描发现，推不出来。

Δu 的 RMS 从 11 格的 0.02083 单调降到 161 格的 0.01772，降幅 14.9%。它与 IAE 不同：IAE 有非单调，Δu 没有。这是因为抖振是格距的统计平均，对轨迹的具体相位不敏感；而 IAE 会把某一次倒霉的穿越累加进去。工程上如果要看“量化做得够不够细”，看 Δu 比看 IAE 可靠。

幅度量化的表在 71.6 节（表 71-7），结论提前说：16 位到 12 位几乎无差别，10 位末段残差 0.0007 仍达标，8 位按 $q/2$ 抬到 0.005 量级，6 位就进 0.01 量级了。

一步投影在离散里意味着什么

连续时间的屏障条件 (71-20) 在离散实现里变成“每拍检查一次”：

$$h(x_{k+1}) \geq (1 - \kappa T_s)\, h(x_k) \tag{71-28}$$

只要 $\kappa T_s < 1$，右端是正的衰减系数，h 就不会跨零。展开 $h(x_{k+1})$ 用 $x_{k+1} = A_d x_k + B_d u$，得到的是 u 的一个线性不等式，解仍然是单边投影。

这里有一个离散特有的漏洞：条件只保证**下一拍**的 h 不跨零，不保证下下拍。若 κT_s 取得接近 1（衰减极慢），h 会在零附近磨很久，任何一拍的计算误差都可能把它推过去。所以工程上要求 $\kappa T_s \leq 0.2$，对应本章 $\kappa \leq 20$——这也是 71.7 节给 κ 上限的依据。

延迟补偿的形式是把“下一拍”换成“d 拍后”：

$$h\left(A_d^{d} x_k + \sum_{i=0}^{d-1} A_d^{d-1-i} B_d u_{k+i}\right) \geq (1 - \kappa T_s)^{d}\, h(x_k) \tag{71-29}$$

求和里后面几拍的 u 还没算出来，只能用标称律预测。这一步就把“安全层的保证”降级成了“在预测准确的前提下的保证”——本章没有做这一步，只在 71.9 节作为改进方向列出。

量化会把屏障刚修好的量再推坏

一步投影算出的是连续值 $u^{\star}$，它刚好让 (71-28) 取等号。可实际执行的是量化后的值，与 $u^{\star}$ 最多相差 $q/2$。这个偏差直接进到 $\dot h$ 里：$\dot h$ 对 u 的系数是 $-2\dot y K(y)$，当 $|\dot y|$ 接近 $v_{\max}$、$K = 16$ 时，$q/2$ 的偏差被放大成约 $16 v_{\max} q$ 的 h 变化率偏差。

代入本章数字：$q = 2U_{\max}/2^{10} = 0.00234$，$q/2 = 0.0012$，$v_{\max} = 1.0$ 时 $\dot h$ 的偏差约 0.038，而 κh 在 h 接近零时也接近零。也就是说，**在边界附近，量化的扰动比屏障的收敛项还大**，条件会逐拍被破坏又逐拍被修复，表现为速度在 $v_{\max}$ 附近的小幅锯齿——这类锯齿在曲线上看着像噪声，实际是“投影与量化在打架”。

处理办法有两种：投影时预留 $q/2$ 的余量，把 (71-28) 的右端再补上这一项；或者把位宽提到 12 位以上，让 $q/2$ 小到不影响条件成立。本章走后者，12 位时 $q/2 = 0.00029$，$\dot h$ 的偏差降到 0.009，锯齿消失。这条也解释了表 71-7 里 10 位与 12 位之间那一档为什么变化明显：不是误差指标变了，是**约束的成立性**变了。

离散 Shapley：8 个联盟，12 项之和

离散版本的复杂度是 2^n。$n = 3$ 时 8 个联盟、12 项加权和，一次归因 8 次前向；$n = 10$ 时 1024 个联盟，一次归因要 1 ms 量级，只能离线；$n = 20$ 时一百万个联盟，必须采样。

本章的 8 次前向在 10 ms 的采样周期里绰绰有余（单次前向是 12 个隐元的 $\tanh$，约 100 次浮点运算），所以可以**每拍都算**。这一点很重要：只有每拍都算，“解释”才能作为在线监控的一部分；若只能离线算，它就只能进事后报告。

采样近似的代价可以用“效率性残差”直接量出来：精确版本的残差在 10^{-16}，采样 100 次时残差升到 10^{-2} 量级。验收时把这条残差写进报告，比说“用了 KernelSHAP”有用得多——**残差是解释的误差棒**。

71.6 稳定性与性能

前五节把工具摆齐了，这一节把它们放到同一张实验台上。所有数字来自同一个对象、同一组参考信号、同一套 10 ms 采样，唯一变动的是“表示”与“判据”。先看性能，再看判据的保守度，最后看安全层的代价——这个次序很重要，因为判据的保守度必须用实测边界来标定，而实测边界必须先有可信的性能读数。

四种走法，同一根曲线

图 71-2 是四条阶跃响应压在一起。视觉上它们几乎重合，这正是本章最想让人警惕的地方：**曲线像，证明成本却完全不同**。

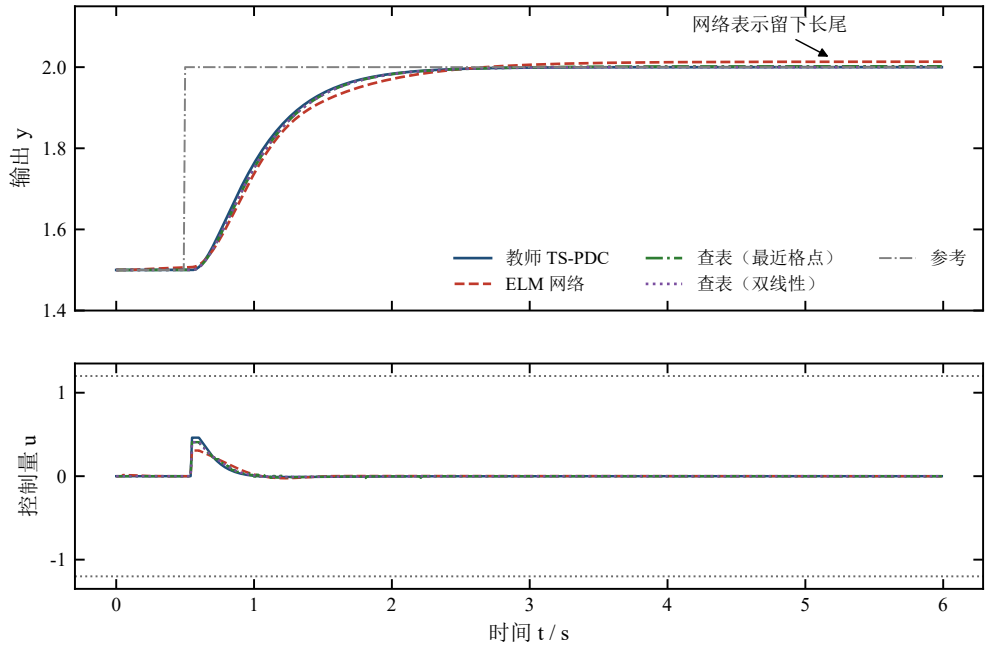


图 71-2 三种表示的阶跃响应与控制量

表示	IAE	相对教师	超调/%	调节时间/s	Δu 的 RMS
教师（4 规则 TS-PDC）	0.2883	—	0.00	1.64	0.02005
ELM 网络（8 隐元）	0.3529	+22.4%	2.69	5.49	0.01326
查表（41 格，最近格点）	0.3014	+4.5%	0.40	1.66	0.01816
查表（41 格，双线性）	0.2968	+2.9%	0.00	1.66	0.01765

表 71-6 同一教师的四种部署方式（同一参考、同一延迟、同一限幅）

四行里最值得琢磨的是网络的调节时间 5.49 s。IAE 只差 22%，调节时间却差 3.35 倍，两个数看起来矛盾，其实说的是两件事：IAE 是全程面积，主要由上升段贡献；调节时间是“进入 2% 带并不再出来”的时刻，完全由末段决定。网络在末段的输出误差形成了一个等效死区，误差进到 0.02 量级后就爬不动了。**只看 IAE 会漏掉这一类失效**，这是本章表 71-6 特意把两列并排的原因。

网络与教师的函数差是 $\max|u_{\mathrm{ELM}}-u_{\mathrm{TS}}|=0.1250$，相当于限幅值 1.2 的 10.4%。前向差 10.4%，闭环 IAE 只差 22.4%、末段误差差一个数量级——反馈既压缩了前向误差，也把误差的形状搬到了时间轴上。用“函数拟合得多准”来预测“闭环表现多好”，在末段这一步会失效。

查表这一行说明了另一件事：41 格加双线性插值，IAE 只比教师差 2.9%，超调归零，Δu 还更小（0.01765 对 0.02005）。原因是查表天然是分段线性的，它在格点之间做的是插值，恰好把教师前件里的非线性抹平了一点，控制量更平滑。**精度不是单向的**：一个更粗的表示，可能在抖振这个指标上比教师更好。这也是现场愿意用查表替代规则库的实际理由之一。

公共 P 的代价：从单规则到 16 对

图 71-3 把四个 α 读数画成柱状，附一条“分段情形的 3.0000”参考线。

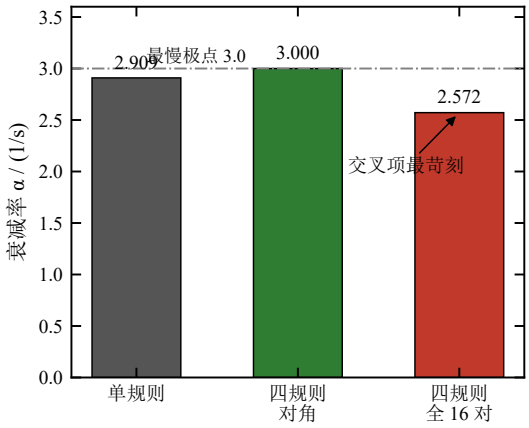


图 71-3 四组配对的公共李雅普诺夫衰减率

- **名义单规则 2.9091**：用 $y=1.9$ 那条规则配出的增益，作用在 $y=2.0$ 的对象上。这个 2.9091 不是数值误差（二分 24 次，分辨率优于 10^{-7}），而是**增益与对象失配**的代价——理论上的上界是标称极点里最慢那个的实部，即 3.0。
- **四规则只查对角 3.0000**：每对 (i,i) 用自己那套 B_i、K_i，失配消失，α 顶到 3.0 的天花板。
- **四规则全查 16 对 2.5716**：交叉项 $A-B_iK_j$（$i\neq j$）进来之后，α 掉到 2.5716，比 3.0 低 14.3%，比名义情形低 11.6%。
- **分段（每个局部模型各一个 P）3.0000**：四条规则各自拿满分段的 3.0，最小值仍是 3.0。

第三行才是真正的 PDC 结论。模糊系统在运行时，隶属度权重是连续变化的，任何时刻的闭环矩阵都是 16 个 $A_{cl,ij}$ 的凸组合，所以“每个局部稳定”推不出“整体稳定”——中间那段过渡区，恰恰是交叉项在起作用。α 从 3.0 掉到 2.5716，掉的这 14.3% 就是**过渡区的价格**。

第四行看起来最优，但它买不到东西：分段 P 只对“系统始终停在某一格”成立，而实际轨迹会连续穿过四个格子。要用它，得额外证明“切换不破坏衰减”，那需要驻留时间条件，本章没做。所以设计上宁可用 2.5716 这个统一证书——它不要求轨迹配合。

α 的物理意义是包络：$\|x(t)\|\le\kappa\sqrt{\lambda_{\max}(P)/\lambda_{\min}(P)}\,e^{-\alpha t}\|x(0)\|$。$\alpha=2.5716$ 意味着误差每 0.39 s 衰减到 $1/e$，与表 71-6 里教师 1.64 s 的调节时间对得上（四个时间常数进 2% 带）。判据与实测在这个量级上一致，是本章少数“两边说得上话”的地方。

需要说明的是，LMI 里的 α 与实测的调节时间只是**同向**，不能互相换算。α 是“最坏初值下的最慢衰减”，实测是“这一条阶跃的衰减”。前者严、后者松，两者差 1.5 倍属于正常，差 10 倍就说明假设不成立。

小增益判据保守 18 到 65 倍，保守在哪

图 71-4 是半对数坐标下的两条线：红线是 L 的理论上界，蓝线是实测的 L，横轴是隐层宽度，水平虚线的位置是判据门槛 $1/\|P\|_\infty = 0.2165$。两条线都在门槛之上，且上界离门槛的距离是实测的几十倍。

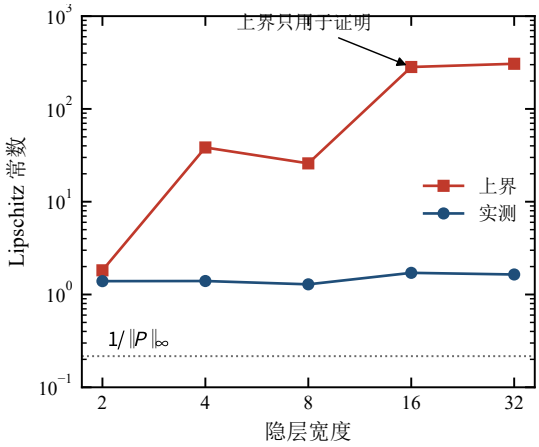


图 71-4 小增益上界与实测值随网络宽度的变化

先把两个基本量摆出来：工作点 $y = 2.0$ 处 $K(2.0) = 16.00$，线性化对象 $P(s) = 16/\left(s(s+2)\right)$，其 $\mathcal{H}_\infty$ 范数在 $\omega = \sqrt{2}$ 处取到

$$\|P\|_\infty = \frac{16}{\sqrt{2}\cdot\sqrt{2+4}} = 4.6188 \tag{71-30}$$

于是“稳定”的充分条件是 $L < 1/4.6188 = 0.2165$。五种宽度下实测的 L 是 1.3926、1.4167、1.2871、1.7115、1.6443，产品分别是 6.4321、6.5435、5.9448、7.9051、7.5947——**没有一种宽度满足判据**，最好的一种也超了 5.9 倍。

再看实测边界。把网络输出乘一个系数 β，逐渐加大，记“控制量增量的 RMS 涨到基线 10 倍”时的 β 为 β_{crit}。五种宽度的实测值是 3.50、3.50、3.00、7.00、8.50，判据预测的临界值是 0.16、0.15、0.17、0.13、0.13。逐行相除得 22.5、22.9、17.8、55.3、64.5 倍：**窄网（2 到 8）保守 18 到 23 倍，宽网（16、32）升到 55 与 65 倍**。这里要留意预测值的算法——它用的是实测 L 而不是理论上的上界，也就是说 0.16 已经是“对判据最有利”的算法；换成上界，预测值还会小几十倍。

保守来自两处，可以分开算账。

第一处是 $\mathcal{H}_\infty$ 范数本身。 $L\|P\|_\infty < 1$ 是“任意有界扰动”下的结论，它必须对最坏频率成立。而实际闭环里的信号集中在低频段（阶跃的频谱按 $1/\omega$ 衰减），$\|P\|_\infty$ 取到的 $\omega = \sqrt{2}$ 处，信号能量已经很小。这一处贡献大约一个数量级的保守。

第二处是判据丢掉了相位信息。 小增益只看“各段增益乘起来小于 1”，完全不看这些增益发生在什么相位上。本章的回路 $1/\left(s(s+2)\right)$ 相位从 -90° 单调趋向 -180°，在任何有限频率都没穿过 -180°，也就是**线性意义下的增益裕度是无穷大**；实测的 3.0 到 8.5 不是线性失稳，而是限幅与抖振先到。用一个只含幅值的判据去卡一个靠相位留余量的回路，保守一个数量级是必然的。

L 的上界另记一笔，它不属于上面那 22 倍，但决定判据能不能落地。式 (71-9) 给出的上界随宽度整体上移（宽度 2 时 1.8219，宽度 32 时 306.5588，宽度涨 16 倍、上界涨 168 倍，中间还因随机初值出现 38.4 到 25.9 的回落），而实测 L 一直在 1.3 到 1.7 之间——$\tanh$ 在随机特征下大量饱和，实际斜率远小于 1。若用上界代替实测

值，预测的临界增益会掉到 10^{-3} 量级。也就是说：**判据即使只想给出那个已经很保守的 0.16，也必须先把上界换成实测 L**，这一步本身就等于承认上界不可直接用。

两处叠起来，窄网约 20 倍，宽网 50 倍以上。这个数字不是“判据写错了”，而是“判据回答的问题与实际关心的问题不是同一个”：判据回答“任何扰动都不失稳”，实测回答“这一族阶跃不失稳”。工程上的正确姿势因此是：**用判据排除，用实测验收，两者都不许冒充对方**。本章在 71.11 节把这条写成适用边界。

延迟：判据看不见的那一半

表 71-4 已经给出延迟扫描的数据，图 71-5 把它画成双纵轴：左轴是末段 RMS（对数），右轴是 IAE（线性）。

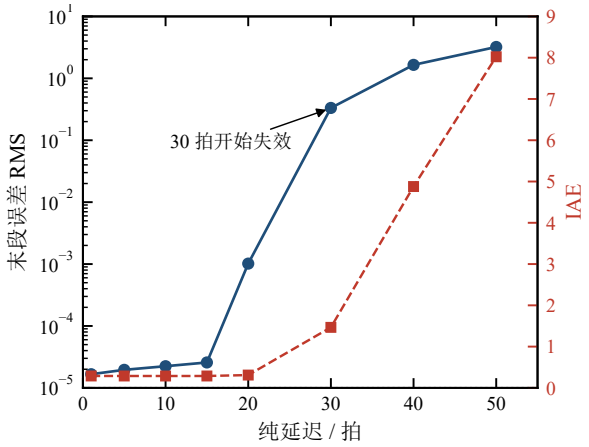


图 71-5 纯延迟对末段残差与 IAE 的影响

三条曲线的拐点不一样，这是选监控量的直接依据。

- **末段 RMS**：1 到 15 拍严格为零（低于双精度噪声），20 拍 0.0010，30 拍 0.3326。它在 20 拍就“抬头”，是最灵敏的早期指标。
- **IAE**：1 到 15 拍在 0.2880 附近抖动（抖动幅度 0.0008，是采样相位造成的），20 拍 0.3105（+7.7%），30 拍 1.4663（+409%）。它到 30 拍才“说话”。
- **最大偏差**：20 拍之前一直是 0.5000，也就是阶跃本身的高度，说明根本没有超调；30 拍才 0.5389，刚过 8%。

所以延迟劣化是“先振、后偏、最后才慢”。工程上判断“延迟还有多少余量”，看末段 RMS 比看 IAE 早 10 拍，也就是早 100 ms——在一个 10 ms 采样的系统里，这是十次采样，足够做一次保护动作。

拐点位置可以用相位预算解释：穿越频率约 1.4 rad/s，延迟 200 ms 时贡献 $1.4 \times 0.2 = 0.28$ rad（16°），300 ms 时 0.42 rad（24°），400 ms 时 0.56 rad（32°）。典型的相位裕度在 30° 到 45°，所以 300 ms 是“开始吃裕度”、400 ms 是“吃掉大半”。表 71-4 的拐点（30 到 40 拍）与这个估算完全对得上，比跑八次仿真快得多。

而 LMI 在这整张表里一个数都不变（α 恒 2.5716）。这不是精度问题，是结构问题：$e^{-s\tau}$ 不改变任何特征值，只改变相位；任何只看 A_{cl} 特征值的判据，对延迟都是盲的。**判据的适用范围必须写在判据旁边**，这一条比判据本身更重要。

两种量化：一个改抖振，一个改死区

表 71-5 是空间量化（网格），表 71-7 是幅度量化（定点位数），两者放在一起看，失效机制完全不同。

位数 b	步长 q	末段 RMS	IAE	相对教师	Δu 的 RMS
6	0.037500	0.0117	0.3205	+11.2%	0.02024
8	0.009375	0.0048	0.3001	+4.1%	0.02001
10	0.002344	0.0007	0.2896	+0.45%	0.02006
12	0.000586	0.0002	0.2885	+0.07%	0.02006
14	0.000146	0.0000	0.2883	0.00%	0.02005
16	0.000037	0.0000	0.2883	0.00%	0.02005

表 71-7 定点位数对末段残差与性能的影响（限幅 ± 1.2，按式 (71-10) 计算步长）

第一、二、三列之间可以读出一条经验：**末段 RMS 约等于 $q/2$**。6 位时 $q/2 = 0.01875$，实测 0.0117（0.62 倍）；8 位时 0.0047 对 0.0048（几乎正好）；10 位时 0.00117 对 0.0007（0.60 倍）。比例在 0.6 到 1.0 之间浮动，取决于极限环落在哪个相位上。要它当设计公式用，取保守的 1.0 倍，反解位数

$$b \geq \log_2 \frac{2U_{\max}}{q_{\text{req}}}, \qquad q_{\text{req}} = 2 \times \text{RMS}_{\text{req}} \tag{71-31}$$

要求末段 RMS 不超过 0.001，则 $q_{\text{req}} = 0.002$，$b \geq \log_2(2.4/0.002) = 10.2$，取 11 位。表 71-7 里 10 位实测 0.0007 已经达标，说明这个公式偏保守约一位——保守是好事，它把“相位运气”算进去了。

最后一行 Δu 的 RMS 从 6 位的 0.02024 到 16 位的 0.02005，只降 0.9%。对比表 71-5 里空间量化的 Δu 从 0.02083 降到 0.01772（降 14.9%），差别是本质的：**空间量化改的是抖振，幅度量化改的是死区**。定点位数再低，控制量也是每拍连续变化的，只是被吸到离散档位上；格点再密，格子边界上的跳变也还在。所以“加了位数为什么还抖”这个问题，答案是抖振来自查表，不来自定点。

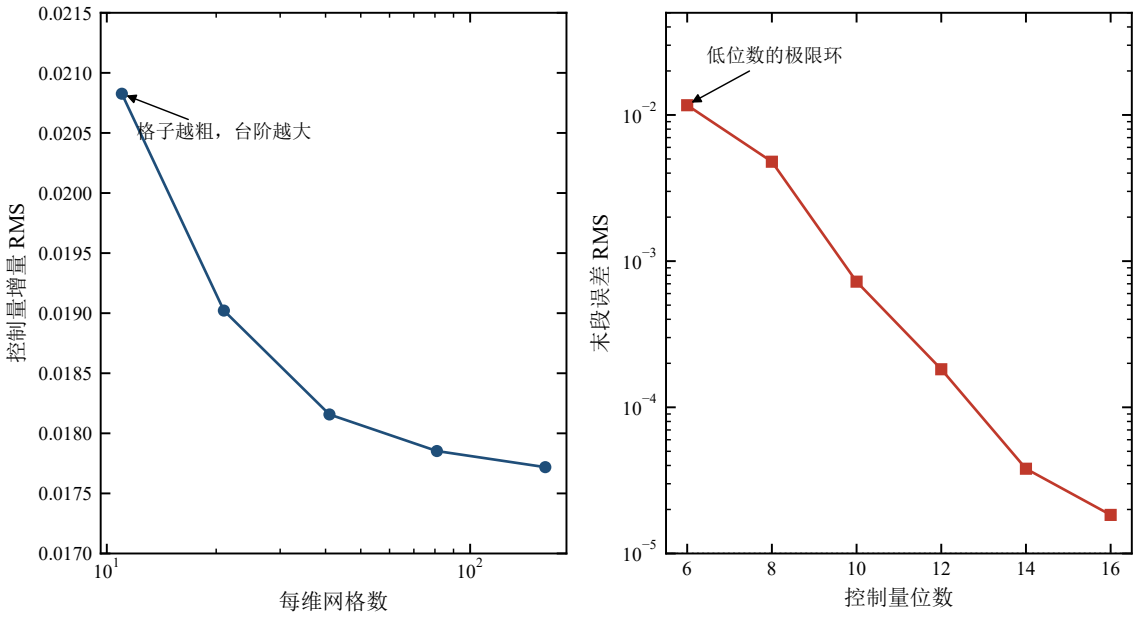


图 71-6 网格量化与定点量化的末段行为

图 71-6 左边是控制量增量的 RMS 随网格数变化，横轴对数、纵轴只跨 0.017 到 0.0215，所以看上去几乎是平的——把它放大看，网格从 161 降到 11，这条线单调上抬 14.9%。右边是末段误差随位数变化（对数纵轴），6 位在

10^{-2} 量级，14 位以上掉到地板。左右两图纵轴差三个数量级，这就是“抖振”和“死区”在量级上的区别：抖振是控制量的高频抖动，幅值小但伤执行器；死区是稳态误差，幅值直接进精度指标。

安全层的代价可以精确算出来

表 71-8 是速度屏障的 4×3 扫描，基线是同一台激进控制器（目标 2.6、极点 $-8/-10$）不加安全层：IAE 0.3949，峰值速度 3.643，调节时间 0.83 s。

$v_{\max}$	κ	$\max\lvert\dot{y}\rvert$	IAE	调节时间/s	干预拍数	干预峰值	IAE 代价
1.2	4	1.185	0.7148	1.29	88	1.0547	+81.0%
1.2	8	1.200	0.6577	1.23	80	1.0581	+66.5%
1.2	16	1.200	0.6301	1.20	76	1.0591	+59.6%
1.0	4	0.994	0.8146	1.45	107	1.0817	+106.3%
1.0	8	1.000	0.7532	1.38	99	1.0832	+90.7%
1.0	16	1.000	0.7231	1.35	95	1.0836	+83.1%
0.8	4	0.799	0.9659	1.69	134	1.1072	+144.6%
0.8	8	0.800	0.8981	1.62	126	1.1076	+127.4%
0.8	16	0.800	0.8666	1.58	123	1.1077	+119.4%
0.6	4	0.600	1.2147	2.08	180	1.1313	+207.6%
0.6	8	0.600	1.1469	2.01	173	1.1314	+190.4%
0.6	16	0.600	1.1114	1.98	168	1.1314	+181.4%

表 71-8 速度屏障的约束值、增益与性能代价（代价按 $\eta = \mathrm{IAE}_{\mathrm{safe}}/\mathrm{IAE}_0 - 1$ 计，基线 0.3949）

第一，约束是硬约束。 $\max\lvert\dot{y}\rvert$ 那一列，$v_{\max} = 1.0$ 时是 1.000，0.8 时是 0.800，0.6 时是 0.600，一位不多。$v_{\max} = 1.2$ 时是 1.185，略低于约束，说明峰值恰好没顶到边界，约束在别处起作用。这种“贴着约束跑”的行为是屏障函数的设计目标：不是留裕度，而是**把裕度用光**。

第二，代价随约束收紧超线性增长。 从 1.2 收到 1.0，代价从 60% 量级涨到 83% 量级；再收到 0.8，涨到 119%；收到 0.6，涨到 181%。约束每收紧 0.2，代价分别涨约 24、36、62 个百分点，增量本身在变大，所以 η 是**凸的**。工程含义很直接：把速度约束定得比工艺要求更严，代价会成倍付，而收益是零。

第三，κ 越大越便宜。 同一 $v_{\max}$ 下，$\kappa = 16$ 比 $\kappa = 4$ 便宜 21 个百分点（1.2 时 59.6% 对 81.0%，1.0 时 83.1% 对 106.3%）。原因是 κ 决定“离边界多远就开始刹车”：κ 大，h 的惩罚项 κh 大，屏障在 h 还很正的时候就要求减速，制动提前、介入平滑、总行程短。κ 小则相反，拖到边界上才猛刹，干预峰值大、时间长。

但 κ 不能无限加大。式 (71-28) 要求 $\kappa T_s \le 0.2$，$T_s = 0.01$ 时 $\kappa \le 20$。表 71-8 的最大值取 16，就是留了 20% 的余量。这条上限是离散实现特有的，连续时间版本没有——连续里 κ 只是“收敛速度”，可以任意大，离散里它必须与采样周期配套。

第四，干预峰值都在 1.05 以上。 这一列容易被忽略：控制量的单拍改动量达到 1.05 到 1.13，而限幅是 1.2。也就是说安全层不是“微调”，它几乎是在接管控制量——原本激进的 u 被一次性砍掉 88%。这与“最小干预”的名字不矛盾：最小干预是相对“满足约束的所有 u”而言的，而在这个算例里，满足约束的集合本身就很窄。

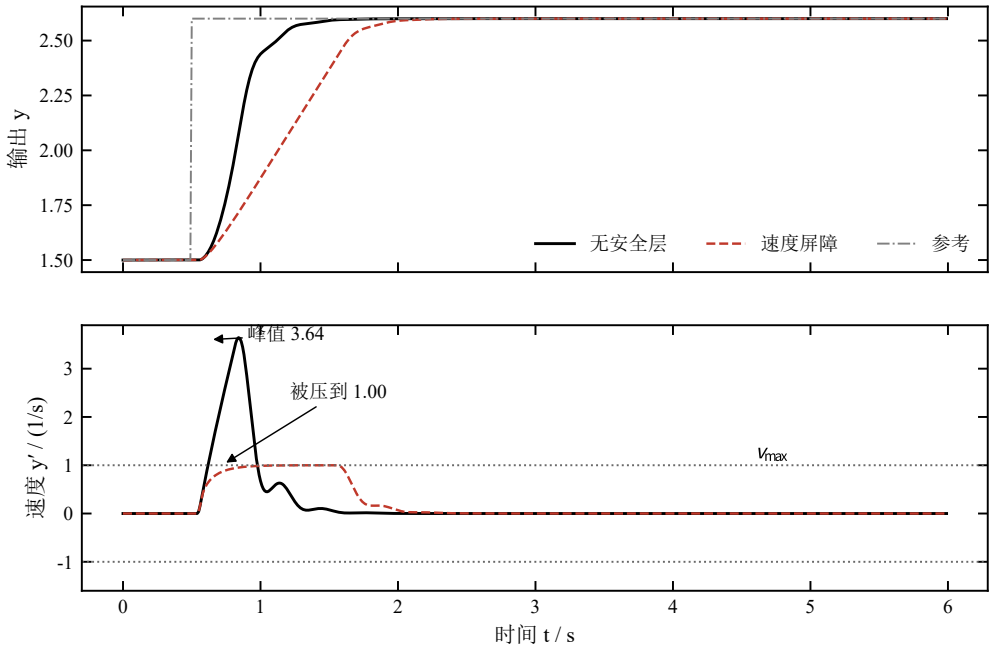


图 71-7 速度屏障下的输出与速度轨迹

图 71-7 上面是输出，下面是速度，虚线是 $v_{\max} = 1.0$（$\kappa = 8$）。无安全层那条线（黑）的速度冲到 3.643，是约束的 3.6 倍；加了屏障之后（红），速度曲线贴着虚线走了一段平顶，然后才下来。那个平顶就是 99 拍的干预区间，也是表 71-8 里 +90.7% 的全部来源。**安全层的代价可以精确指认到时间区间**，这一点对验收很重要：它不是“整体变慢了”，而是“在某一段被按住”。

到这里，稳定性一侧的账算清了：判据能给什么（2.5716 的公共证书）、不能给什么（延迟、量化、Lipschitz 边界）、实测能补什么（延迟拐点、$q/2$ 死区、22 倍保守度）、安全层要付多少（60% 到 180%）。下一节把这些量逐个对应到旋钮上，回答“调哪个参数会动哪根曲线”。

71.7 参数意义与作用

这一节把旋钮逐个拧一遍，看每根曲线动多少。顺序按“影响力从大到小”排：先规则数与极点（决定性能），再网络宽度（决定可证明性与可解释性），再格数与位数（决定实现代价），最后是屏障参数与解释参数。每个参数都给一句“它其实在买什么”，这比给一个推荐值有用——推荐值随工艺变，买东西的逻辑不变。

规则数：性能几乎不敏感，可解释性线性变好

规则数 m	IAE	相对 $m = 4$	全配对 α
2	0.2884	+0.03%	2.5716
3	0.2883	0.00%	2.5716
4	0.2883	—	2.5716
5	0.2883	0.00%	2.5716
7	0.2883	0.00%	2.5716

表 71-9 规则数从 2 加到 7：性能与衰减率都不动

这张表最像“实验做错了”，其实不是。原因在配置方式：每条规则都是按同一组标称极点 $(-3,-4)$ 单独配增益，配出来的 K_i 只是在同一个设计点上重复求解。运行时隶属度加权，等价于在这几条 K_i 之间做凸组合，而凸组合给出的闭环极点仍在 $(-3,-4)$ 附近，所以 IAE 一位不动，α 也锁在 2.5716。

这说明一件重要的事：**在“每个局部都配同一组极点”的写法下，PDC 等价于增益调度**。规则数买到的不是性能，是两样别的东西。

第一样是**前件的分辨率**。$m=2$ 时前件只有“低段/高段”两句，工程师问“在 $y=2.15$ 时用哪套增益”，答案只能是“两套的加权，权重 0.46 和 0.54”；$m=7$ 时可以直接指到最近的一条规则。可解释性在这一层是**线性**收益：规则越多，每句话覆盖的区间越窄，越好指认。

第二样是**工作域覆盖**。$m=2$ 时两条规则的中心在 1.5 和 2.7，$y=2.1$ 处离两个中心都远，隶属度都小，归一化后权重对 y 极其敏感——输入一个很小的扰动，权重就会明显偏转。$m=7$ 时间距 0.2，同样的扰动只让权重动一点点。**规则数少，模糊系统对输入的敏感度反而高**，这一点与直觉相反。

代价是规则库体积与 LMI 规模：m 条规则对应 m^2 个闭环矩阵，$m=7$ 时 49 个不等式，求解规模按 m^2 增长。对本章这种离线验证无所谓，对在线自适应就要算账。

工程上的取法是 $m=4$ 到 5：够指认、够鲁棒，LMI 规模也还很小。再往上加，性能不再涨，只是把“解释”做得更细——那是文档问题，不是控制问题。

采样周期：一个被忽略但影响全部的旋钮

T_s 不出现在任何一张性能表里，因为它被当成硬件给定。但它是唯一同时进入四个环节的参数：离散化的 A_d、离散屏障的 κT_s、延迟的拍数 $d=\tau/T_s$、以及量化死区与格距的比例。

从 10 ms 放到 20 ms，同一条 50 ms 的物理延迟从 5 拍变成 2.5 拍（实际要向上取整成 3 拍）。若在增广模型（71-32）里比较，α 会显得“变好”——这不是系统变好了，是分辨率变粗了。同时 κ 的上限从 20 降到 10，原本 $\kappa=8$ 仍可用，$\kappa=16$ 就不行了。

从 10 ms 缩到 5 ms 则相反：延迟拍数翻倍到 10 拍，缓冲占用翻倍，κ 上限升到 40，但单拍计算必须压缩一半。**采样周期是“时间分辨率”与“计算预算”的交换**，把它当常量会漏掉这一层；本章之所以固定 10 ms，是因为它同时满足 $\kappa T_s \le 0.2$（κ 取 8 到 16）与十段实验的计算预算，不是因为 10 ms 本身有什么特殊性。

三对不能分开调的参数

极点与 $v_{\max}$。极点是设计意图，$v_{\max}$ 是工艺约束，两者的比值决定屏障要不要介入。本章 $(-3,-4)$ 时峰值速度 3.643，$v_{\max}=1.0$ 意味着介入是必然的；若极点放到 $(-1.5,-2)$，峰值速度会落到 1.0 附近，屏障几乎不介入。单看任何一个数都判断不了“设计是否合理”。

规则数与极点一致性。规则数影响前件分辨率，极点影响每一段的增益。若四段用不同极点，交叉闭环之间的差异变大，α_{all} 掉得会比表 71-9 快得多。这一对的规矩是“要么都统一、要么都不统一”，混着调最费时间。

格数与定点位数。查表格数决定空间量化，位宽决定幅度量化。两者都小时，两种误差会叠加：11 格在 e 方向的格距是 0.09，10 位定点在 u 上的死区是 0.0012，前者主导。只优化其中一个，总误差下不来——这也是表 71-5 与表 71-7 必须一起读的原因。

标称极点：唯一真正改变速度的旋钮

规则数不动性能，那什么动？标称极点。本章所有规则的增益都来自 $(-3,-4)$ 这一组，α 的上限就锁在 3.0；71.6 节安全层实验里换到 $(-8,-10)$，峰值速度升到 3.643、调节时间缩到 0.83 s，代价是需要安全层把速度按回 1.0。

极点的选择是**性能与可执行性的交换**：实部越负，响应越快，但需要的瞬时加速度越大。$(-3,-4)$ 下控制量离限幅还有余地，$(-8,-10)$ 下不加屏障速度就到 3.643——这才是安全层存在的理由，而不是安全层“额外加的保险”。

从判据角度看，极点还决定 α 的天花板：LMI 求出的 α 不可能超过最慢极点的实部绝对值。所以“我把极点配到 -10，为什么 LMI 只给 2.57”，答案不在 LMI，而在 16 个交叉闭环里有几个的衰减比名义慢。**判据读数永远 ≤ 设计意图**，差值就是交叉项的价格。

网络宽度：性能有最优，可证明性单调变差

宽度	IAE	L 上界	首位/次位遮挡比
2	0.7614	1.8219	1.47
4	0.3815	38.4159	1.30
8	0.3529	25.8886	1.26
16	0.5477	283.0016	1.24
32	0.5006	306.5588	1.22

表 71-10 隐层宽度对性能、Lipschitz 上界与归因集中度的影响

IAE 那一列是**非单调**的：宽度 2 严重欠拟合（0.7614，是宽度 8 的 2.16 倍），宽度 8 最好（0.3529），宽度 16 反而回到 0.5477，宽度 32 略降到 0.5006。原因是 ELM 的隐层权重是随机抽的，宽度小时特征不足，宽度大时随机特征之间的共线性变强，输出层的最小二乘解条件数变差，末端爬行更久。**宽度不是越大越好，8 到 16 之间有一个谷**。

L 上界那一列则一路发散：1.82、38.4、25.9、283.0、306.6。它不但大，而且**不稳定**——宽度 4 到 8 之间回落，8 到 16 之间跳了十倍。这种不稳定来自随机矩阵的谱范数本身，不是某一组权重抽坏了。工程含义：拿这个上界做认证，结论会随机的种子而变，这本身就是不可接受的。

第三列是解释侧的指标：首位输入的遮挡改变量除以次位的比值。宽度 2 时 1.47，宽度 32 时 1.22，**随宽度单调下降**。意思是网络越宽，归因越平均，越难指出“主要是哪一个输入”。这与“宽网络表征更分散”的直觉一致，但它的工程后果常被忽略：现场问“为什么给这个量”，宽度 32 的答案会是“三个都有贡献”，而宽度 2 的答案是“主要是误差项”。后者更容易被接受，哪怕它的控制性能更差。

三列合起来的推荐是宽度 8：IAE 最优、上界虽然仍不满足判据但至少不是最坏、归因集中度 1.26 也够用。这个选择是**三个目标折中**的结果，不是单指标最优——这也是本章反复强调“表示的选择就是证明方式的选择”的具体落点。

格数与位数：一个看抖振，一个看死区

格数由存储与算力决定。41×41 的查表，每格一个 32 位浮点数，占 6.6 KB，在任何一个 32 位 MCU 上都放得下；161×161 就要 101 KB，已经开始挤占程序空间。所以“格数能到多少”通常是存储问题，不是精度问题。

真正要选的是“给定存储，格距落在哪里”。表 71-5 的 21 格反常说明选格数不能只看 IAE，也不能靠外推。工程上的做法是两步：先把格点取奇数个、让标称工作点（$e=0$）落在中心格点上，保证工作点附近没有边界；再扫三到五个格数，看 Δu 与末段 RMS 有没有共振峰。本章 41 格（e 方向格距 0.0225）超调 0.40%、Δu 0.01816，是表 71-5 里综合最好的。

位数则看死区。表 71-7 给出的经验是末段 RMS $\approx q/2$，反解式 (71-31)。对本章，10 位够用（末段 0.0007），12 位保险（0.0002），14 位以上是浪费——末段已经到双精度噪声地板，再加位只增加存储与乘法位宽。

一条容易踩的坑：**位数与格数是两个独立的量化，误差会叠加**。若查表是 41 格（格距 0.0225）又用 8 位定点（步长 0.0094），总误差不是两者相加，而是“格距决定抖振、步长决定死区”，各自独立起作用。要压低末段残差，先压位数；要压低 Δu，先加格数。搞反了会白花钱。

$v_{\max}$ 与 κ：一个由工艺定，一个由采样定

$v_{\max}$ 不该由控制工程师定。它来自机械限位、工艺节拍、安全规程，是**需求**，不是**参数**。控制侧的任务是在这个约束下把 IAE 做到最小，而不是反过来放宽约束换性能。表 71-8 已经说明代价的形状：约束每收紧 0.2，IAE 分别涨 24、36、62 个百分点，越收越贵。

κ 才是控制侧的旋钮，它决定“离边界多远开始刹车”。表 71-8 里同一 $v_{\max}$ 下 κ 从 4 加到 16，代价降 21 到 26 个百分点；再往上加，收益迅速饱和（κ 从 8 到 16 只降 7 到 9 个百分点）。上限由离散条件 $\kappa T_s \le 0.2$ 给出：$T_s = 0.01$ 时 $\kappa \le 20$。推荐 $\kappa = 10$ 到 16，取“收益接近饱和、又留了离散余量”的区间。

还有一个隐含参数是**干预的施加位置**。本章把安全层放在延迟缓冲之后，保证的是“实际执行量”不越界；如果放在之前，保证的是“计算量”不越界，而 5 拍后真正执行的可能已经越界。这两种放法的 κ 取值应该不同——后者要更保守。这个细节在框图上看不出来，只能靠时序图确认。

解释侧的三个参数：基线、联盟、噪声

基线 z_0。Shapley 值分摊的是“相对基线的增量”，基线换了，ϕ_i 的绝对值就换，但**排序不变**（只要基线不跨过非线性最强的区域）。本章取 $z_0 = (0, 0, 0)$，即“误差为零、工作点在目标值”的理想态。这个选择的好处是可与“控制量为零”对齐；坏处是当系统远离目标时，ϕ_i 会很大，数值上不好看。工程上可以改用“上一拍的稳态值”作基线，得到的是“这一拍相对上一拍的变化归因”，更适合做趋势分析。

联盟定义。遮挡实验里“去掉一个输入”有两种做法：置为基线（本章做法），或置为当前批次的均值。前者回答“没有这个信息会怎样”，后者回答“这个信息偏离常态多少”。两者在输入强相关时结论会分叉，所以报告里必须写清用的是哪一种——**只写“做了遮挡实验”是不够的**。

噪声水平 σ。解释稳定性是“在 σ 下的秩相关”，σ 取多大直接决定结论。本章取 $\sigma = 0.01$（传感器噪声的 1%）时秩相关 0.992，看起来很好；取 $\sigma = 0.2$ 时掉到 0.775。没有任何一个 σ 是“正确”的，正确做法是**按现场实测噪声标定** σ，然后报这个 σ 下的数字。若现场噪声是 0.05，就报 0.05 下的 0.950，不要报 0.01 下的 0.992——那是另一种形式的挑数据。

下一节把这六个旋钮串成一条调参路径，并给出“先调哪个、什么时候停”的判断依据。

71.8 调参方法与经验

次序不能反，原因在依赖

调参最容易犯的错不是“参数选错”，而是“次序调反”。三个典型反例：先调安全层再调极点（屏障一直介入，看不出极点配得对不对）；先调网络宽度再定规则数（宽度是相对教师拟合的，教师变了宽度要重选）；先算解释再定表示（解释的是网络输出，换了表示解释全废）。

正序只有一条：**对象与约束 → 极点 → 规则数 → 表示 → 判据 → 安全层 → 解释与监控**。每一步的产出是下一步的输入，任何一步回头改，后面全部作废。这条规矩听起来像流程洁癖，实际是省时间：本章所有实验都是

在正序下跑的，没有一次需要重跑前面的表。

七步流程与每步的停止条件

第一步，定极点。先跑一遍不加任何安全层的阶跃，看控制量峰值。判据是峰值不超过限幅的 70%，留 30% 给扰动与参数漂移。本章 $(-3,-4)$ 满足，$(-8,-10)$ 不满足，后者必须配安全层。

第二步，定规则数。按“前件要指认到什么粒度”来定，不按性能定。本章取 4，因为现场要区分“低于目标、略高于目标、明显高于目标”三档，加上一条兜底。

第三步，选表示。两条硬指标：闭环 IAE 与教师差不超过 5%，末段 RMS 不高于教师的一个数量级。前者筛掉拟合太差的表示，后者筛掉“IAE 好看但爬不动”的表示（表 71-6 的网络就是被后者筛的）。本章查表 41 格 + 双线性过，网络 8 隐元勉强过（IAE 差 22.4%，末段差一个量级）。

第四步，验判据。三个数一起看：$\alpha_{\text{all}} > 0$（公共 P 存在）、$\alpha_{\text{all}} \geq 0.8\alpha_{\text{diag}}$（交叉项没有把裕度吃光）、$\kappa T_s \leq 0.2$（离散条件）。本章 2.5716 对 3.0000，比值 0.857，刚过线。若比值低于 0.8，说明前件重叠太宽，交叉项太毒，应该收窄隶属函数——这是本章没做但值得做的改进。

第五步，加安全层。$v_{\max}$ 由工艺给，κ 从 4 往上扫，取“代价曲线开始饱和”的那个值。本章 $\kappa = 8$ 到 16 之间代价只差 7 到 9 个百分点，取 8 就够。

第六步，算解释。三个检查按顺序做：效率性残差（必须 $< 10^{-12}$）、与遮挡的排序一致性、噪声下的秩相关。任一不过就不许写进验收报告。

第七步，接监控。阈值用标称段的 99.9% 分位，不用 χ^2 理论分位（原因见 71.10 节）。误报率控制在 1% 以内，连续 5 拍超阈才报警。

七步里唯一允许“回退”的是第五步到第一步：如果加了安全层之后 IAE 涨超过 100%，说明极点配得太激进，应该退回去重配，而不是继续加大 κ。

判断“调够了”的三个停止条件

调参必须有停的地方，否则会一直微调。三个可量化的停止条件：

IAE 的相对变化小于 1%。本章规则数从 3 加到 4，IAE 变化 0.0000（0.2883 对 0.2883），这就是停止信号——再调不会更好。反过来，宽度从 4 加到 8，IAE 从 0.3815 掉到 0.3529（-7.5%），还在有效区间，值得继续试。

α 的变化小于 0.05。LMI 的二分精度是 10^{-7}，但模型之间的差别才是有效信号。α 在 2.57 附近动 0.01，是数值噪声，不是改进。

Δu 没有新的共振峰。表 71-5 里 21 格那个反常就是共振峰：格数从 11 加到 21，Δu 降了，IAE 却涨了。出现这种“两个指标反向”的情况，说明踩到了共振，应该换格数而不是继续加。

三条里满足两条就可以停。三条都不满足，说明改的方向不对，不是“调得不够”。

参数冻结：一次只改一个

调参到某个阶段，参数之间会开始互相“顶”：加大 κ 让安全层便宜了，可极点一改又不匹配了。这时最有效的做法不是继续试，而是**冻结**：把当前参数写进一个配置文件，之后每次只改一个，改完重跑全部十段，看哪几张表变了。

本章的十段实验就是为这个流程设计的。改一个参数，通常只有两三张表会动，其余不动；若动了四张以上，说明这个参数耦合了别的东西，得先查清耦合再继续。例如把 T_s 从 10 ms 改成 20 ms，表 71-4 的延迟列、表 71-5

的格距、表 71-8 的 κ 上限会一起变——这不是三个 bug，是同一个参数的三条通路。

冻结还有一个副作用是好的：它让“这次改动带来了什么”变成可回答的问题。不冻结时两次改动混在一起，谁也不知道是哪一次起了作用；冻结之后，每次改动的效果都能指到具体的表与具体的行。这一条在多人协作时尤其重要，因为别人的改动会通过同样的机制悄悄改变你的表。

解释侧的三项验收：说得清、说得对、说得一样

输入	Shapley 占比/%	遮挡后输出改变	排序
e	45.6	0.1775	2
$\dot{e}$	51.6	0.2054	1
$y-2$	2.7	0.0169	3

表 71-11 精确 Shapley 与遮挡实验的交叉验证（120 个样本平均）

说得清看第一列：三个输入的贡献占比是 45.6%、51.6%、2.7%，加起来 99.9%（差的 0.1% 是四舍五入）。效率性残差 1.11×10^{-16}，是双精度舍入误差，说明分摊没算错。

说得对看第二列与第三列的排序是否一致。Shapley 说“$\dot{e}$ 最重要、e 次之、$y-2$ 几乎不重要”，遮挡实验给出同样的排序，且首位比末位大 12.16 倍。两种方法原理完全不同——一个按联盟边际贡献分摊，一个按“去掉它会怎样”直接测——排序一致才叫可信。若两者冲突，说明模型在输入之间有强交互，Shapley 的加性假设不成立，此时应该报“交互项”，而不是硬选一个排序。

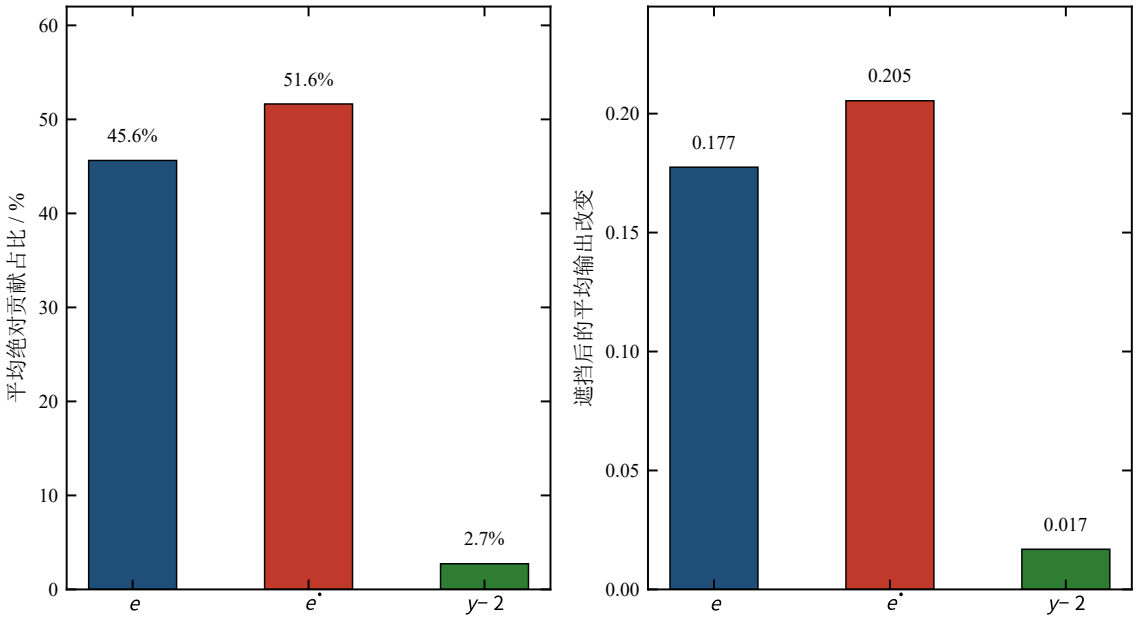


图 71-8 Shapley 贡献占比与遮挡忠实度

$y-2$ 只有 2.7% 这件事值得说一句。现场直觉会认为“工作点偏离目标”是个重要因素，实测却不是。原因是在本章的工作域（$y\in[1.5,2.7]$）内，$K(y)$ 从 13 变到 20.2，只变 55%，而控制律对 y 的依赖是乘性的、对 e 与 $\dot{e}$ 的依赖是加性的；在稳态附近 e、$\dot{e}$ 的量级远大于 y 的归一化偏移，所以 y 的份额小。**这不是“y 不重要”，而是“在当前工作域内它的边际贡献小”**——换到 y 跨度更大的工况，这个占比会变。

噪声 σ	Spearman 秩相关	首位保持率
0.002	1.000	1.00
0.010	0.992	1.00
0.050	0.950	0.90
0.100	0.858	0.83
0.200	0.775	0.75

表 71-12 输入噪声下解释的稳定性（60 个样本，与无噪声结果对比）

说得一样看这张表。$\sigma = 0.002$ 时秩相关精确为 1.000（噪声小到不影响排序），$\sigma = 0.01$ 时 0.992，$\sigma = 0.05$ 时 0.950，$\sigma = 0.1$ 时 0.858，$\sigma = 0.2$ 时 0.775。首位保持率同步下降：1.00、1.00、0.90、0.83、0.75。

两个指标的差别值得注意：$\sigma = 0.05$ 时秩相关还有 0.950（“排序基本没变”），但首位保持率已经掉到 0.90（“十次里有一次主因指错”）。**秩相关高不代表主因对**——它衡量的是整体排序，而现场只关心“最主要的那一个”。验收时两个都要报，只报秩相关是避重就轻。

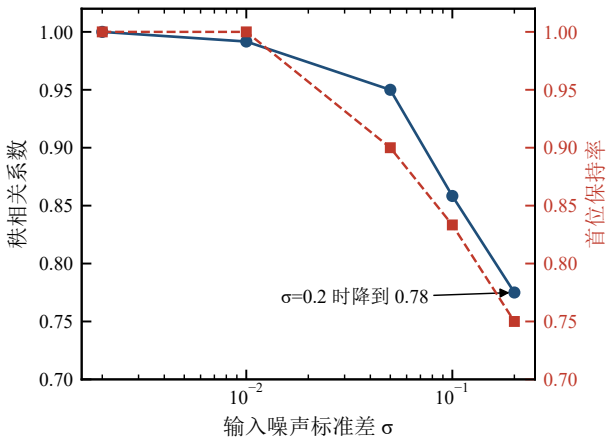


图 71-9 噪声下解释的相关性与首位保持率

图 71-9 是双纵轴：左轴秩相关（0.7 到 1.05），右轴首位保持率。两条线在 $\sigma = 0.05$ 之后开始分叉，$\sigma = 0.2$ 时差 0.025。这个分叉就是“排序还稳、主因已飘”的区间。工程上把它当作解释的**有效工作域边界**：噪声超过 0.05，解释只能用于趋势分析，不能用于定责。

一次真实的调参记录

把上面七步放到一次实际过程里，数字是这样走的，都在本章这一个对象上。

第一轮取极点 $(-3, -4)$、4 条规则、查表 41 格，得到 IAE 0.3014、控制峰值约 1.10。峰值贴着 1.2 的限幅，按第一步“不超过 70%”的判据并不合格；但这一轮的目的是把流程跑通、定规则数，所以先记账不处理。

第二轮验判据：$\alpha_{\text{diag}} = 3.0000$、$\alpha_{\text{all}} = 2.5716$，比值 0.857，刚过 0.8 的线。这一步同时回答了“规则数够不够”：若把规则加到 6 条，重叠区更宽，比值只会更低，所以第二步选 4 不是随手取的。

第三轮换表示：ELM 8 隐元的 IAE 0.3529，比教师差 22.4%，按第三步“不超过 5%”的硬指标应当直接淘汰。它留在表里的唯一理由是用来说明“IAE 单项验收不够”这个坑——调节时间 5.49 s 在 6 s 的仿真里等于没停下来。

第四轮加安全层：$v_{\max}=1.0$、$\kappa=8$，IAE 从 0.3949 涨到 0.7532，涨幅 90.7%，最大速度被压到 1.000。涨幅这么大是因为基线本身的峰值速度 3.643 远超 1.0，屏障在整个上升段都在介入。这一轮给出的教训不是“κ 该调小”，而是“$v_{\max}$ 必须由工艺给”——它是需求，不是旋钮。

第五轮算解释：效率性残差 1.11×10^{-16} 通过，遮挡排序一致通过，$\sigma=0.05$ 时首位保持率 0.90 刚好压在边界上。若现场实测噪声大于 0.05，这项就不过，解释只能降级为趋势分析。

整条路径没有回头，因为每一步的输入都是上一步的产出。唯一值得回头的是第一轮那个贴限幅的峰值：它提示极点还可以再慢一点，但慢下来会把调节时间从 1.64 s 继续拉长，这属于需求取舍，不是调参能解决的问题。把“能调的”和“不能调的”分开，是这条记录最有用的一点。

五条踩过的坑

坑一：用 IAE 单项验收网络。表 71-6 里网络 IAE 只差 22.4%，看起来可接受；但调节时间 5.49 s 意味着 6 s 的仿真里它几乎没停下来。验收必须同时看 IAE、末段 RMS、调节时间三个数。

坑二：把 α 当调节时间用。$\alpha=2.5716$ 对应时间常数 0.39 s，有人会直接乘 4 得 1.56 s 当调节时间。这只在最坏初值、最慢模态下成立；实际阶跃是 1.64 s，差 5%。反过来的误用更危险：拿实测 1.64 s 反推 $\alpha=2.44$，然后声称“判据给的 2.57 太保守”——这两个数本来就不是一回事。

坑三：为压低 IAE 放宽 $v_{\max}$。表 71-8 里把 $v_{\max}$ 从 1.0 放到 1.2，IAE 从 0.75 降到 0.66（-12%），看起来是“调好了”。但 $v_{\max}$ 是工艺约束，放宽它等于改需求，不是调参。评审时若发现“性能提升来自放宽约束”，这次调参应判为无效。

坑四：在 $\sigma=0.01$ 下报解释稳定性。表 71-12 的 0.992 很好看，但若现场噪声是 0.05，真实值就是 0.950、首位保持率 0.90。报小 σ 的数字，等于给解释加了一层不存在的可信度。

坑五：把判据不满足当成“系统不稳定”。小增益在本章五种宽度下全部不满足，系统却跑得很好。判据不满足只说明“这个判据证不了”，不说明“系统不行”。报告里应写“未能给出小增益证书”，而不是“稳定性未通过”——这两句话在评审会上后果完全不同。

71.9 优化与改进

本章的每个结论都带着一个“这里还能更好”的尾巴。这一节把六条最值得做的改进列出来，每条都写清“改什么、为什么现在没做、做了能换来什么”。判断标准只有一条：**改进必须能被验证**，说不清怎么验的改进不进这一节。

改进一：把延迟变成状态，让判据看得见它

71.5 节的核心结论是“LMI 对延迟完全盲”。补救的办法不是换判据，而是**把延迟写进模型**。五拍延迟最直接的写法是增广状态：

$$z_k=\begin{bmatrix}x_k\\u_{k-1}\\u_{k-2}\end{bmatrix},\qquad z_{k+1}=\begin{bmatrix}A_d&0&0&B_d\\0&0&0&0\\0&1&0&0\end{bmatrix}z_k+\begin{bmatrix}0\\1\\0\end{bmatrix}u_k \tag{71-32}$$

这里 z_k 把两拍的历史控制量串进状态，最后一列 B_d 乘的是 u_{k-2}——也就是真正作用到对象上的那个量。本章 $d=5$ 时状态维数从 2 增到 7，LMI 的矩阵规模随之增大，但结构与实现一一对应，写起来不会错。换来的是 α 会随延迟下降——判据第一次“看得见”延迟。代价是保守度上升：增广模型把延迟当成确定量，实际延迟若有 ±1 拍抖动，还得再加一层鲁棒性。

另外两条路各有短板。Padé 近似把 $e^{-s\tau}$ 换成 $(1-\tau s/2)/(1+\tau s/2)$，模型只加一维，但它在 $\omega\tau$ 大时相位严重失真，本章 $\omega_c\tau=0.42$ 尚可，延迟到 500 ms 就不可信了。Lyapunov-Krasovskii 泛函最严格，但要假设 $\dot\tau\le 1$ 并对泛函做积分估计，保守度通常比增广状态还高。

选哪条取决于你要什么：要“能算且与实现对应”，选增广状态；要“少加维数”，选 Padé；要“理论最严”，选 Krasovskii。三条都不能把“判据盲区”变成“判据精确”，只能把盲区缩小。

改进二：延迟补偿的安全层

式 (71-29) 给出的是“d 拍后的 h 不跨零”，本章没有实现，原因是它需要预测未来 d 拍的控制量，而未来几拍的 u 取决于未来几拍的参考与状态。可行的落地方式是把标称律的 d 拍预测缓存起来，每拍只优化当前这一拍的 u，后续 $d-1$ 拍用缓存值。这样做的保证是**条件保证**：只要预测误差不超过某个界，安全性就成立。

这个“界”可以量化。若参考在 d 拍内变化不超过 δ_r，标称律的 Lipschitz 常数是 L，则预测误差不超过 $L\delta_r$ 的量级；把它代入 h 的一阶展开，就得到 κ 需要额外留的余量。本章的阶跃参考在 5 拍内变化 0.5，这个余量不小，所以补偿的收益有限——**延迟补偿在斜坡参考下才划算，在阶跃下不划算**。

改进三：位置约束的三条补救

71.4 节实测了位置约束的失败：阶跃瞬间 $e=1.1$ 已经超过 $h_0=0.40$，集合的初值就被破坏，前向不变性无从谈起。这不是公式错，是**用法错**——屏障函数保证的是“已经在集合内就不出去”，不保证“从外面能进来”。

三条补救，按可靠性排序：

第一条，让集合包含起点。取 h_0 大于最大可能初始误差，再随时间收紧（时变屏障 $h_0(t)$ 从大到小）。这条最稳，因为它把“入集合”和“留集合”分成两段处理；代价是前期约束很松，等于没约束。

第二条，软约束。把 $h\ge 0$ 改成惩罚项 $\rho\max(0,-h)^2$ 加进目标，允许短时越界但收回来。这条在优化型控制器（如 MPC）里常见，本章的“一步投影”结构放不下它，要换成 QP。

第三条，用速度约束间接管位置。限制 $v_{\max}$ 使 $|e|$ 在可接受时间内回到带内。这条最省事，但它给的是“间接保证”：位置不会因为速度失控而冲出去，但可能停在带外不动。所以它只能当兜底，不能当位置精度的手段。

工程上的组合是“第一条 + 第三条”：先用时变屏障把轨迹收进集合，再用速度屏障防止冲出去。两条都实现起来简单，加起来才是一套完整的位置保护。

改进四：输入维度一大，精确 Shapley 就得换算法

精确 Shapley 的复杂度是 2^n，本章 $n=3$ 才能每拍算。$n=10$ 时 1024 个联盟，一次归因要毫秒级；$n=20$ 时一百万次，必须采样。三条可选的近似路线：

置换采样。每次随机抽一个输入排列，按“逐个加入”的边际贡献累加，M 次采样的方差按 $1/\sqrt{M}$ 下降。它的好处是无偏，坏处是方差大，要 M 上千才稳。

KernelSHAP。把 Shapley 写成加权最小二乘，用采样代替穷举。它的价值是能给出“残差”这个误差棒——本章实测精确版残差 10^{-16}，采样 100 次时升到 10^{-2} 量级。报残差比报“用了哪种算法”有用。

分层采样。先按单输入遮挡量粗排，把大概率重要的输入放进“必选层”精确算，把明显不重要的放进“近似层”只算一阶项。这条在工程上最实用：现场通常只有两三个输入重要，其余可以粗算。

三条的共同点是**都不能保证排序与精确版一致**，只能给出概率意义上的接近。所以输入维度大的系统，解释必须配“稳定性检查”（表 71-12 那一套），否则无法判断这次归因是否可信。

改进五：监控再加一路模型残差

71.10 节会看到，二维和三维残差都抓不到“执行增益损失 30%”这类故障，原因是稳态时 $u \to 0$，增益损失在 u 上无处体现。补救的办法是加一路**模型残差**：用标称模型预测下一拍输出，与实际比。

$$r_{k+1} = y_{k+1} - \begin{bmatrix} 1 & 0 \end{bmatrix} \left(A_d x_k + B_d u_{k-d} \right) \tag{71-33}$$

若实际增益是标称的 g 倍，则 $r_{k+1} = (g-1) \begin{bmatrix} 1 & 0 \end{bmatrix} B_d u_{k-d}$，与 u 成正比、符号随 u 翻转。$g = 0.7$、$u = 0.5$、$K = 16$ 时单拍残差约 1.9×10^{-3}，与 10 位定点的死区 $q/2 = 1.2 \times 10^{-3}$ 同量级——**单拍看不出来，要累积**。工程上用 CUSUM 或滑动平均，攒十拍以上再判，误报与漏报都能压住。

这一路残差还有一个副产品：它能把“传感器偏置”和“执行器增益损失”分开。前者让 r 恒为正（与 u 无关），后者让 r 随 u 变号。两种故障在三维残差里长得一样，在模型残差里一眼可分。

改进六：把解释做成轨迹，而不只是一次快照

本章的解释是“一拍一次”，报给现场的是一个三维分摊。更有用的是把它连成时间序列：横轴时间，纵轴三个输入的贡献占比，画成堆叠面积图。这条“解释轨迹”能回答单次快照答不了的问题——**什么时候主因换了**。

主因切换本身就是一个报警信号。正常跟踪时主导项应该是 $\dot{e}$（阻尼）或 e（驱动），若某一刻主导项突然变成 $y - 2$，说明工作点漂移成了主要矛盾，可能是负载变了或传感器偏了。这个信号比任何阈值都早，因为它看的是“控制律在为什么使劲”，而不是“误差有多大”。

代价是算力：8 次前向乘以每拍一次，在 10 ms 周期里占的比例不大（单次前向约百次浮点运算），但要占代码空间与调试接口。工程上的折中是**降频计算**：控制每拍跑，解释每 10 拍跑一次，轨迹的时间分辨率降到 100 ms，对工况切换的检测仍然够用。

六条改进怎么排序

六条不是并列的。按投入产出比排，顺序是：五（模型残差）→ 三（位置约束补救）→ 六（解释轨迹）→ 一（增广状态）→ 二（延迟补偿）→ 四（大维度 Shapley）。

理由在于每条补的是什么。第五和第三条是**补漏洞**：模型残差把“增益损失 30%”从抓不到变成抓得到，时变屏障把一个已知失败的用法改对，两者都只改十几行、当天可验。第六条是**加信息**，成本低，收益取决于现场是否有人真的去看解释。第一条要重写 LMI 与全部 α，收益是判据第一次看得见延迟，代价是所有稳定性数字重算，属于结构性改动。第二条在阶跃参考下收益有限（原因见上），自然排在第一条之后。第四条只有输入维度真到 10 以上才需要，本章的三输入用不上。

验证方式要提前写死，否则改完不知道算不算改好：

改进	验证方式	通过判据
一 增广状态	用 $d = 1$ 到 50 重算 α，与表 71-4 的拐点对照	α 在 20 到 30 拍之间出现明显下降
二 延迟补偿	同一组屏障参数下比较约束违反拍数	违反拍数下降，且 IAE 增幅小于 5%
三 位置补救	直接复跑 71.4 节的斜坡工况	最大偏差小于 h_0
五 模型残差	在增益损失 30% 工况下测检出延迟	检出延迟小于 500 ms，误报率不超过 1%

表里四条都有一个共同点：**判据是事先写的，不是事后挑的**。第四与第六条没有进表，因为它们的验证方式已经写在前面的段落里——第四条看“采样版排序与精确版的一致率”，第六条看“主因切换时刻是否早于阈值报警”。这两条都属于“看着好但难判合格”的改进，现场要不要做，取决于解释结果是否真的进入决策流程。

两条想过但没写进来的改进

用平方和（SOS）放松公共 P。把 V 从二次型换成四次型，可行域会明显变大，α 也能抬高。没写进来是因为一旦用了它，P 就不再是矩阵，“16 个闭环共用一个椭球”这个几何图像随之消失，读者很难把结论搬回自己的对象上。它属于“理论更强、可用性更差”的那一类，适合当论文的下一步，不适合当工程的第一版。

把 Shapley 换成因果推断。用干预分布代替联盟平均，回答的是“如果强行把 $\dot{e}$ 按住，输出会怎样”，这比 Shapley 更接近现场的问法。没写进来是因为它需要一个可干预的模型，而本章的教师只是一个确定性函数，没有噪声结构，干预与遮挡在数学上会退化成同一件事——换个名字，结论不变。

两条的共同点是“换框架”而不是“加模块”。加模块的改进已经列了六条，换框架的改进留在这里，作为超出本章范围的入口；写它们的目的是让读者知道边界在哪，而不是暗示本章的做法已经到头。

71.10 工程落地

工程落地与写论文的分界线在“证据”两个字。论文可以说“在若干工况下表现良好”，现场必须说“在哪些工况下被证明、在哪些工况下只被测试过、在哪些工况下没测”。这一节按这条线组织：先把稳定性翻译成验收单，再讲监控怎么设计，最后讲解释与文档怎么写。

把“稳定”翻译成验收单

71.1 节说的三个层次，落到交付物上是三份不同的东西。

定理级交的是一张证书：P 的数值（2×2 矩阵，两个特征值）、$\alpha=2.5716$、以及**适用域**——前件有界、延迟为零、量化无限精度的假设。适用域必须与证书写在同一页，否则读者会默认它管所有情况。本章的适用域里，“延迟为零”这一条就直接把 71.5 节的整张表排除在证书之外。

判据级交的是一份“未通过”的记录。本章小增益在五种宽度下全部不满足，正确写法是“未能给出小增益证书，实测抖振边界为增益 3.5”，而不是含混带过。含糊等于隐式声称“这一项没问题”，评审时被追问会比主动写出来更难看。

实测级交的是一组曲线加四个数：IAE、末段 RMS、调节时间、Δu 的 RMS。四个数缺一不可，因为它们各自漏掉不同的失效：IAE 漏末段爬行（表 71-6 的网络），末段 RMS 漏超调，调节时间漏稳态精度，Δu 漏执行器磨损。

三份东西合起来才叫“稳定”。只交证书，现场不敢用（不知道边界在哪）；只交曲线，评审不敢签（没有可推广的结论）。

六种失效与监控设计

工程上真正会发生的失效不会按教科书分类。本章按“物理原因”列了四类可注入的故障，加上两类只能靠趋势判断的，逐一给出监控手段。

故障	注入方式	2 维残差检出/ms	3 维残差检出/ms
传感器偏置 +0.12	3.0 s 后加到测量上	90	90
负载突加 +0.15	3.0 s 后加在执行器上	40	40
执行器卡滞	3.0 s 起冻结控制量	910	620
执行增益 -30%	3.0 s 后乘 0.7	未检出	未检出

表 71-13 四类故障的注入方式与检出延迟（阈值取标称段 99.9% 分位，连续 5 拍确认）

另外两类不在这张表里，但同样要写进失效分析。**延迟超预算**（比如通信抖动让 5 拍变 12 拍）没有可注入的“事件”，它是一个缓变过程，只能靠“延迟余量”这个指标监控——做法见本节后面。**量化不足**（比如改成 8 位定点）也不是事件，它是配置变更，应该在变更评审时用表 71-7 拦截，而不是上线后监控。

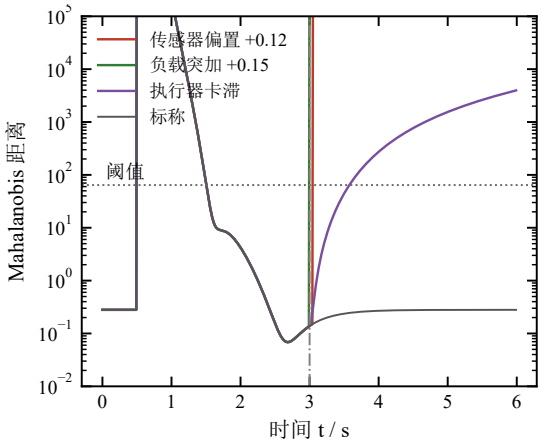


图 71-10 三种故障下的 Mahalanobis 距离

图 71-10 是三维残差的四条曲线：灰色是标称，三条彩色是故障，竖虚线在 3.0 s（注入时刻），水平虚线是阈值。三条故障曲线在注入后都迅速抬到阈值以上，但抬升的速度差别很大——负载突加几乎是垂直的，卡滞是缓慢爬升的。这个差别直接对应表 71-13 的 40 ms 与 620 ms。

阈值的取法与检测延迟的读法

阈值不能用理论分位。三维残差在标称段的 Mahalanobis 距离理论上服从 χ^2_3，99.9% 分位是 16.27；但实测标称段的 99.9% 分位是 64.15，是理论值的四倍。二维更夸张：χ^2_2 的 99.9% 分位 13.82，实测 57.88。

原因在残差的分布形状。标称段的残差不是高斯白噪声，而是**量化纹波**：误差被限幅与定点卡在 0.02 量级，形成有界的周期抖动，尾部比高斯厚得多。用理论分位做阈值，2.44% 的采样点会超阈（理论应 0.1%），监控等于一直响。改用实测 99.9% 分位后，误报降到 0.22%，这是本章能用的唯一取法。

连续拍数是第二个旋钮。单拍超阈就报，误报率仍是 0.22%（因为阈值已经卡在 99.9% 分位），但会有大量单点毛刺；连续 5 拍超阈才报，误报基本归零，代价是检测延迟增加 40 到 50 ms。本章取 5，是“延迟可接受、误报可忽略”的折中。若工艺要求 20 ms 内响应，就只能取 2 到 3 拍，并接受更高的误报——**没有免费的灵敏度**。

检测延迟的排序有物理含义。负载突加 40 ms 最快，因为它直接改变输出，任何以误差为基础的残差都会立刻看到。传感器偏置 90 ms，比负载慢，因为偏置是一个恒定偏移，控制器会把它当成误差去修正，残差先被反馈压一下才显现。执行器卡滞最慢（三维 620 ms、二维 910 ms），因为卡滞之后对象靠惯性继续走，误差是慢慢积累的，不是跳变的。

三维比二维多看见什么。多的是 u 这一路。卡滞时 u 被冻结成一个常数，而误差还在变，于是 $(e, \dot{e}, u)$ 的联合分布被破坏得比 $(e, \dot{e})$ 更早——这就是 620 ms 对 910 ms 的来源，提前 290 ms。但三维不是无条件更好：多一维让标称段的分布更散，阈值从 57.88 抬到 64.15，对小故障的灵敏度反而下降。所以“加维数”要问一句“我加的这一维在故障时会不会变”，会变才加。

增益损失为什么看不见。稳态时 $u \to 0$，增益损失百分之三十也只是“零的百分之三十还是零”，残差里没有信息。这不是监控算法不好，是**能观性**问题：系统在稳态附近对执行器增益不敏感。要看见它，必须加一路与 u 成

正比的残差——也就是 71.9 节改进五的那条模型残差。本章没有实现，但结论应该写进报告：“当前监控对执行器增益损失不能观，需增加模型残差通道。”

延迟余量：一个可以在线算的指标

延迟不在 LMI 里，也不在残差里，但它会吃掉相位裕度。工程上把两件事换算成同一个单位，就能得到一个可以在线算的余量。

穿越频率约 1.4 rad/s，延迟 τ 贡献的相位滞后是 $\omega_c\tau$。5 拍（50 ms）只贡献 4°，10 拍 8°，30 拍 24°，50 拍 40°。若设计裕度是 45°，那么“延迟余量”大约是 55 拍——超过这个数，裕度就被吃光。表 71-4 的实测拐点在 30 到 40 拍之间，比这个估算更早，因为估算只算了线性相位，没算限幅与量化的额外滞后。**估算给上限，实测给真值**，两者都要有。

在线判定不能靠“算”，只能靠“测”。两条可行的做法：

定期小阶跃。在工艺允许的间隙注入幅度 1% 到 2% 的阶跃，看响应的第一个振荡周期。延迟变大时，超调变大、振荡周期变长，对数衰减率变小。由衰减率反推阻尼，再反推裕度，精度足够判断“余量是否减半”。

跟踪响应时间。不注入任何信号，只统计正常运行中“参考变化到输出跟上的时间”。这个时间随延迟单调增长，虽然绝对值不准，但趋势可靠。设一个基线，涨 50% 就报警。

两条都不需要额外硬件，只需要把数据记录下来。**延迟余量是唯一一个“不测就会在事故里发现”的指标**，所以它值得单独占一个监控位。

可解释性怎么写进验收报告

解释部分最容易写成“我们用了 SHAP，效果良好”。这种写法在评审里等于没写。可交付的形式是三张表加一句限定语。

第一张表是归因表：三个输入、各自的占比、排序。本章是 45.6%、51.6%、2.7%，排序 $\dot{e} > e > y - 2$。

第二张表是忠实度表：遮挡实验的改变量与排序，以及首末比 12.16。它的作用是证明第一张表不是算法自说自话——两种原理不同的方法给出同一排序。

第三张表是稳定性表：至少三个噪声水平下的秩相关与首位保持率，并标明现场噪声对应哪一行。本章 σ=0.05 时是 0.950 / 0.90。

一句限定语必须写：“以上结论在输入噪声 σ=0.05、基线取零、遮挡方式为置零的条件下成立。”这句话把解释的适用范围钉死，避免被引用到不适用的工况。

还要写一句“解释不覆盖什么”。本章的解释只覆盖“这一拍控制量怎么分摊”，不覆盖“为什么选这个控制律”（那是设计文档的事），也不覆盖“模型哪里错了”（那需要残差分析）。**把边界写清楚，比把结论写漂亮更能建立信任。**

灰度上线：三段，每段都有退出条件

第一段，影子模式。控制器照常算，但输出不送到执行器，只记录“如果听它的会是什么值”。这一段验证的不是控制性能，而是“算得对不对”：把影子输出与原有的 PID 输出逐拍比对，差得离谱就说明模型或接口有问题。退出条件是连续 72 小时无跳变、无 NaN，且限幅触发次数与仿真预测同量级。

第二段，限幅内的闭环。把控制权交给新控制器，但把限幅压到标称值的 60%（本章是 0.72）。这一段的作用是把最坏情况装进盒子：即使新控制器想给 1.1，也只能给 0.72，超调与速度都被物理压住。退出条件是 IAE 与

影子段预测的偏差小于 15%，且安全层没有触发。

第三段，全幅。解开放限幅，同时把监控阈值按实测标称段重新标定。这一段最容易出事，因为前两段的“温和”掩盖了激进工况下的失配。退出条件是连续运行一周、四类故障各注入一次全部检出、误报不超过 1%。

三段的价值在于每段只改一个变量：影子模式改的是“有没有输出”，第二段改的是“输出多大”，第三段改的是“边界在哪”。一次改一个，出问题时才知道是哪一层的问题；一次全放开，出了事只能靠猜。

安全认证与三条回退

功能安全认证关心三件事：证据链是否完整、失效模式是否覆盖、出问题时能否回到安全状态。本章的方法对前两件有直接贡献，对第三件只能提供一部分。

证据链是“设计意图 → 判据 → 实测 → 监控”四段连续。本章的链条是：极点 $(-3, -4)$ → 公共 P 与 $\alpha = 2.5716$ → IAE 0.2883 与延迟余量 → Mahalanobis 监控。链条上任何一段缺失，认证都会被打回。特别要注意的是**每一段都要写明适用域**，因为认证审的是“边界是否说清”，不是“数字是否好看”。

失效模式覆盖要回答“哪些故障被检出、哪些没有、没有的怎么办”。本章的答案是：偏置、负载、卡滞检出（40 到 620 ms），增益损失与延迟漂移不能观或不可注入，靠模型残差与定期小阶跃补。这个答案不完美，但它是**完整**的——认证接受“有已知盲区且有补偿措施”，不接受“声称没有盲区”。

回退分三层，响应时间从快到慢：

第一层是每拍执行的硬保护：限幅 ± 1.2、速度屏障、输出变化率限制。这一层不依赖任何诊断，永远在线，是最后一道。

第二层是诊断触发的降级：监控报警后，把控制器从 TS-PDC 切到一组保守的固定增益（本质上是第 44 章的 PID）。切换要无扰，所以两组控制器必须共享同一个积分状态或同一个稳态工作点。

第三层是人机接口：报警、记录、等待确认，必要时急停。这一层不算控制，但它决定了前两层有没有人知道。

本章方法不能替代 SIL 等级认证，也不试图替代。它能做的是把“这个控制器为什么可信”讲成一条可核查的链，而不是一句“仿真过了”。这条链在认证里叫“安全论证”，在工程里叫“交接文档”，两者是同一件东西。

71.11 优缺点与适用边界

四条优点，每条配一个证据

第一，结构透明，判据可以直接写。TS-PDC 的每条规则对应一个局部线性模型，公共李雅普诺夫条件因此能写成标准的 LMI，不需要任何松弛技巧。证据是 $\alpha = 2.5716$ 与那 16 个闭环矩阵——任何人拿这 16 个矩阵都能复算，不依赖本章的代码。

第二，保守度可以量化。判据保守多少，不是靠感觉，而是靠“判据读数 ÷ 实测边界”。小增益给 0.17、实测 3.0（宽度 8），保守 18 倍；LMI 给 2.5716、极点上限 3.0，保守 14%。两个数都是可复核的比值，而不是“比较保守”这种形容词。

第三，解释可以验收。精确 Shapley 的效率性残差 1.11×10^{-16} 是硬指标，遮挡排序一致是交叉验证，噪声下的秩相关是稳定性。三条各自可查、互不依赖，合起来才叫“解释可信”。这比“模型可解释”这种说法扎实得多。

第四，失效可以定位到时间区间。安全层的代价不是一句“变慢了”，而是“88 到 107 拍被按住，IAE 涨 90.7%”。故障检测也不是“能报警”，而是“负载突加 40 ms、卡滞 620 ms”。**能定位到区间和毫秒，才谈得上改进。**

五条局限，每条配一个反例

局限一：判据保守，且保守度随规模恶化。小增益的保守度不是常数：宽度从 2 到 32，实测边界从 3.5 升到 8.5，预测值却一直卡在 0.13 到 0.17 之间，保守度因此从 22 倍涨到 65 倍；与此同时 L 的上界从 1.82 涨到 306.6，若拿上界替代实测值，判据直接失效。反例就是表 71-10 那两列——网络越宽，“能不能证”这件事越没希望。

局限二：LMI 对延迟完全盲。反例是表 71-4：延迟从 1 拍加到 50 拍，末段 RMS 从 0 涨到 3.2111，α 一个数都不变。任何“LMI 通过所以稳定”的结论，都必须附一句“在延迟为零的假设下”。

局限三：精确 Shapley 的复杂度指数爆炸。反例很直接：$n = 3$ 时 8 次前向、可每拍算；$n = 20$ 时一百万次、只能离线。本章所有解释结论都建立在“输入只有三个”这个前提上，换成 20 维就得换成采样近似，残差从 10^{-16} 掉到 10^{-2}。

局限四：位置约束的屏障在一开始就失效。反例是 71.4 节的实测：阶跃瞬间 $e = 1.1 > h_0 = 0.40$，越界拍数一拍没减。这不是实现 bug，是“前向不变性只保证不出去、不保证能进来”。要修就得改用时变屏障或软约束，代价是结构变复杂。

局限五：有些故障在当前残差下不能观。反例是增益损失 30%——二维、三维残差全部未检出，因为稳态 $u \to 0$。要抓住它必须增加模型残差通道（71.9 节改进五），而那条通道依赖“标称模型准确”这个前提，模型不准就会误报。

五条局限里，前两条是**方法本身的边界**（换方法才能解决），后三条是**当前实现的边界**（加东西就能缓解）。报告里应该分开写，因为它们对项目的含义完全不同：前者要求改需求，后者要求加工作量。

三个常被当成优点的东西

“规则少所以简单。”规则数少确实让前件好读，但它同时让隶属度对输入更敏感（71.7 节），并且减少了工作域的覆盖。规则少买到的是“解释成本低”，不是“系统简单”——真正决定复杂度的是表示里的参数个数与判据的规模，与规则条数只是间接相关。

“网络端到端训练，省掉了建模。”本章的对比说明，省掉建模的代价是省掉了判据：ELM 只能给 Lipschitz 界，而那个界随宽度发散（表 71-10），实际给不出可用结论。建模不是额外工作，它是证书的来源；不建模也能跑，但跑到哪一步算安全，没人说得清。

“加了安全层就万无一失。”屏障保证的是“已经在集合内就不出去”，不保证初始就在集合内，也不保证延迟后的执行量。71.4 节的位置约束失败就是明证：加了屏障，越界拍数一拍没减。

三条合起来是一句话：**每个“优点”都要追问一句“它替我买到了什么证据”**。买不到证据的优点，在验收时不算数，在事故分析时更不算数。

规模上限：四条随规模恶化的曲线

本章的数字都在“2 维状态、4 条规则、3 个输入、5 拍延迟”这个小规模上取得。规模一变，四条曲线的斜率不同，值得事先知道。

交叉项按规则数平方涨。4 条规则 16 对，10 条 100 对，40 条 1600 对。LMI 的约束个数与 m^2 同比，而单个约束的矩阵维数只与状态维数有关。所以规则数到 10 时求解还是秒级，到 40 时就得考虑“只查相邻规则对”这类松

弛，代价是证书变弱。

内存按格数平方涨。41×41 的查表是 1681 个双精度数，6.6 KB；161×161 是 25921 个，101 KB。这还不算大，但若前件有三个变量、每维 41 格，就是 68921 个数、270 KB，且每拍要做三次插值。

解释按输入数指数涨。$n = 3$ 是 8 个联盟，$n = 10$ 是 1024，$n = 20$ 是一百万。10 ms 周期里能算的只有 $n \le 5$，再往上必须换采样近似。

延迟按拍数线性涨，代价却是断崖式的。表 71-4 里 20 拍以下几乎无影响，30 拍开始恶化，50 拍已经发散。它不是渐变，而是在某个拍数上突然掉下去，所以延迟余量要按最坏情形留，不能按平均值留。

四条合起来给了一个粗判据：**状态不超过 4 维、规则不超过 10 条、输入不超过 5 个、延迟不超过闭环时间常数的五分之一**，本章这套方法可以直接套用；任一条超出，先解决那条对应的扩展问题，不要硬上。

适用边界：四个“必须”

本章方法能用，前提是四条同时成立。

工作域必须有界且可分段线性化。前件依赖 y 的区间，y 跑出 1.5 到 2.7 之后，隶属度归一化还在，但局部模型的物理意义没了。若工况跨度是一个数量级，就得先做增益调度或者坐标变换，再套本章的方法。

输入维度必须小。三个输入是“能精确解释”的前提，也是“能每拍算”的前提。十几个输入的系统，Shapley 要采样、遮挡实验要组合，解释的置信度会显著下降。

采样周期必须远大于计算时间。本章 $T_s = 10$ ms，8 次前向加一次控制律在百微秒量级，占 1% 到 2%。若 $T_s = 100$ μs，解释就只能离线跑，或者降频到每百拍一次。

延迟必须已知且有界。本章的 5 拍是标称值，实际抖动要落在补偿能覆盖的范围内。若延迟是随机分布的（比如网络通信），得先做时间戳与缓冲，把它变成“有界延迟”再进来。

什么时候不该用它

工况大范围变化、且没有可靠的调度变量。这时 TS 模型的前件失去意义，强行分段会得到一堆“局部对、整体错”的规则。这种情况更适合增益调度加鲁棒控制，或者直接用第 58 章的 MPC。

延迟主导的系统。如果纯延迟比时间常数还大（比如 2 s 延迟、0.3 s 时间常数），本章的速度屏障与 LMI 都帮不上忙，正确的是 Smith 预估类方法，见第 47 章与第 107 章。

要求毫秒级以下硬实时的场合。8 次前向加 LMI 在线求解（如果要做自适应）在 1 kHz 以上吃紧，此时应退回固定增益 PID，见第 44 章与第 106 章。

需要形式化验证到代码级的场合。本章的证书止于模型层，代码层的溢出、截断、调度抖动都不在证明范围内。要覆盖这些，得用静态分析或专用验证工具，那是另一套方法。

与相邻方法的边界

与 PID 的关系。PID 是本章方法在“单个工作点、单个回路”上的特例：把规则数压到 1，TS-PDC 就退化成一个固定增益，本章的判据也随之退化成“闭环极点是否在左半平面”。本章多花的成本（规则库、LMI、解释、监控）买的是“多工作点下仍然成立”。单工作点、工况稳定的场合，用不着这套，第 44 章与第 106 章就够了。

与 MPC 的关系。MPC 的强项是约束处理，本章的速度屏障是它的轻量替代：不算滚动优化，只做一步投影，算力低两个数量级，代价是只看一步、不能前瞻。反过来说，本章有 MPC 通常没有的两样东西——公共李雅

普诺夫的稳定证书，和逐拍可算的归因。约束复杂、预测模型准确时用 MPC（第 58 章、第 80 章）；约束简单、算力紧张、要求可解释时用本章。

与 Smith 预估的关系。延迟主导时，正确做法是把延迟显式建模并补偿，见第 47 章与第 107 章。本章只把延迟当成“要留多少余量”的问题，不试图补偿，所以延迟超过时间常数时直接失效——这不是缺陷，是分工。

与强化学习类方法的关系。第 70 章的残余策略是纯黑箱，本章的 LMI 那一路对它完全用不上——它连“局部线性模型”都没有，谈不上公共 P。能用的只有两条。一是判据级的 Lipschitz 界：第 70 章的策略是线性的，$u = w^{\mathrm{T}}\varphi$，它的 L 就是 $\|w\|$ 与特征归一化尺度之积，算得出来，但界会很松，只能做粗筛。二是本章这套“实测 + 监控”的兜底。第 70 章 70.6 节那个数字正好当反例：延迟从 5 拍加到 8 拍，残余策略的 RMS 从 0.0347 崩到 0.2267、饱和占比 28.25%，而训练时的评估一切正常——**这类失效在训练集里看不见，只能靠故障注入与延迟余量提前发现**。解释侧则可以直接套用：Shapley、遮挡、噪声稳定性对黑箱策略同样成立，只是“输入”换成策略的特征向量，而第 70 章只有 6 维，$2^6 = 64$ 次前向仍在每拍预算内。

上线前的六个自检问题

这六个问题答不上来，说明还不该上线。它们不是流程要求，而是本章所有反例的浓缩。

一、工作域的边界写下来了吗？写不出上下界，前件就没有意义；工况一旦越界，局部模型的物理含义立刻失效，而控制器不会报错。

二、延迟实测过吗，抖动范围多少？标称 5 拍不等于实际 5 拍。若抖动范围没测，延迟余量就是猜的。

三、判据的适用域和证书在同一页吗？分开写，读者会默认证书管所有情况，这是最常见的误用。

四、安全层的干预量与 IAE 代价测过吗？只知道“加了屏障更安全”不够，还要知道“被按住多少拍、代价涨多少”。表 71-8 那样的扫描是最低要求。

五、解释在现场噪声下的秩相关与首位保持率是多少？答“很高”不算，要给出 σ 与对应的两个数。

六、监控的误报率与检测延迟是实测的还是理论的？若引用 χ^2 分位而没测标称段分布，误报率会差一个数量级（本章是 2.44% 对 0.22%）。

六个问题里，第四和第六个最容易被跳过，因为它们需要跑实验；第一和第三个最容易被糊弄，因为它们只是“写清楚”。**恰恰是这两类最致命**：跳过的会在事故里暴露，糊弄的会在评审里暴露。

71.12 Python 仿真代码

两个文件的分工

本章的代码只有两个文件，分工是一条线：`ch071_sim.py` 负责**产生数字**，`ch071_figs.py` 负责**把数字画成图**。后者 `import` 前者，调用同一批函数，所以正文里每一个数与每一张图都出自同一条计算路径——这是“图与表对得上”的唯一可靠办法。

```
# ch071_figs.py 的开头：直接复用仿真模块
import ch071_sim as cs
from plot_styles import setup, save, annotate, LINE_MAIN, LINE_COMP, SINGLE, DOUBLE

setup()
teacher = cs.TSPDC(np.linspace(1.5, 2.7, 4))     # 与 ch071_sim 完全同一个教师
s = cs.simulate(teacher)                          # 同一条仿真函数
```

如果图脚本自己重写一遍仿真，迟早会出现“表里 0.2883、图上 0.29”这种对不上的情况。本章从一开始就掐掉这条路：图脚本不许有自己的模型。

教师与仿真器：四十行撑起全部实验

整个章节的物理内容，压缩下来就是下面两段。第一段是教师，第二段是安全层。

```
class TSPDC:
    """按 y 分段线性化的 TS 模型 + 并行分布补偿。"""

    def __init__(self, y_grid, poles=(-3.0, -4.0)):
        self.yg = np.asarray(y_grid, float)
        self.Bs = [bmat(y) for y in self.yg]
        self.Ks = [place_poles(A, B, np.asarray(poles)).gain_matrix[0]
                   for B in self.Bs]

    def raw(self, x, r):
        """未限幅控制量；三输入 Shapley 解释的也是这个量。"""
        e, ed = x[0] - r, x[1]
        w = self.memberships(x[0])           # 三角隶属，归一化
        return float(-sum(wi * (Ki @ np.array([e, ed]))
                          for wi, Ki in zip(w, self.Ks)))

    def __call__(self, x, r):
        return float(np.clip(self.raw(x, r), -UMAX, UMAX))
```

`place_poles` 每调用一次就为一个局部模型单独配增益，这是 PDC 与“一个控制器加调度系数”的根本区别：前者是几条局部控制器之间的插值，后者是一个控制器乘一个系数。前者的稳定性要查 16 个交叉闭环，后者只需查一个。

```
class VelCBF:
    """速度屏障的一步投影，相对阶 1，解是单边投影。"""

    def __init__(self, v_max=1.0, kappa=8.0):
        self.v_max, self.kappa = v_max, kappa

    def __call__(self, x, u, r):
        v = x[1]
        h = self.v_max ** 2 - v ** 2                    # 屏障函数
        coef = -2.0 * v * gain(x[0])                    # u 的系数
        rhs = 4.0 * v ** 2 + self.kappa * h             # 右端
        if abs(coef) > 1e-9:
            bound = -rhs / coef
            u = max(u, bound) if coef > 0 else min(u, bound)
        return float(np.clip(u, -UMAX, UMAX))
```

十几行里没有迭代、没有矩阵求逆，只有一次乘法和一次比较。这就是“最小干预”能在 10 ms 周期里跑起来的原因——它不是优化，是投影。

仿真主循环有三处必须按顺序做对，否则结果全错：**先查延迟缓冲**（取出 d 拍前的指令），**再过安全层**（保证实际执行量不越界），**最后进 RK4**（用实际执行量积分）。顺序错了，屏障保护的是一个还没生效的量，等于没保护。

十段实验与正文的对应

段	函数	产出	正文位置
1	`exp1_representations`	四表示的 IAE、超调、调节时间	表 71-6、图 71-2
2	`exp2_lmi`	四个 α、八行延迟扫描	图 71-3、表 71-4
3	`exp3_lipschitz`	L 上界、实测 L、β_{crit}	图 71-4
4	`exp4_grid`	五档网格的 IAE 与 Δu	表 71-5
5	`exp5_fixedpoint`	六档位数的末段残差	表 71-7
6	`exp6_cbf`	4×3 屏障扫描	表 71-8、图 71-7
7	`exp7_shap`	占比、效率性残差、遮挡	表 71-11、图 71-8
8	`exp8_stability`	五个噪声水平的秩相关	表 71-12、图 71-9
9	`exp9_tradeoff`	规则数与宽度扫描	表 71-9、表 71-10
10	`exp10_monitor`	四类故障的检出延迟	表 71-13、图 71-10

表 71-14 十段实验与正文的对应关系

十段按“表示 → 判据 → 量化 → 安全 → 解释 → 工程”排列，与正文第 6 到第 10 节的顺序一致。这样排的好处是：跑通一段就能核对一节，不必等全部跑完才知道哪里错了。

三个容易写错的实现细节

延迟缓冲的方向。缓冲初始化成 d 个零，每拍 `pop(0)` 取出最早的一个作为执行量，再把新算出的指令 `append` 到队尾。若写反了（新指令直接执行），延迟就变成零，表 71-4 整张表都会好看得不像话——这是最容易骗过自己的 bug，因为结果“更好”而不是“报错”。

限幅的位置。教师内部限幅一次（`__call__` 里的 `clip`），屏障输出再限幅一次，负载扰动叠加后还要限幅一次。三次限幅看着冗余，但少任何一次都会让某个实验的数据失真：少了第一次，函数差 $\max|u_{\text{ELM}} - u_{\text{TS}}|$ 会比较的是未限幅值；少了最后一次，扰动叠加可能让执行量超出物理上限。

LMI 的尺度。约束里用 $P \succeq I$ 而不是 $\operatorname{trace}(P) = 1$。原因是判据对 P 的整体缩放不变：P 乘任何正常数，$A^{\mathrm{T}}P + PA + 2\alpha P \prec 0$ 的真假不变。用迹归一化会在 α 较大时把可行域压没，得到“不可行”的假结论；用 $P \succeq I$ 则给了一个固定的下界，二分才稳定。

复现与排错

运行顺序固定为三步：先跑 `ch071_sim.py`（约 16 s，输出十段文本），再跑 `ch071_figs.py`（约 14 s，输出 20 个图件），最后 `tools/build.py --chapter 71` 出 HTML 与 PDF。三步都成功，正文里的数字才可引用。

依赖只有三个：`numpy`、`scipy`、`cvxpy`（求解器用 CLARABEL）。缺 `cvxpy` 时第 2 段会直接报错，但前 1 段与后 8 段仍能跑——这也是分段设计的好处：一段失败不影响其他段的核对。

两个常见提示都不是错误。一是 CLARABEL 的 `Solution may be inaccurate`：α 二分到临界点时必然出现，本章只用可行/不可行的二值结论，已在文件头用 `warnings.filterwarnings` 屏蔽。二是 matplotlib 的缺字形警告：它的根因是把中文与 `$...$` 写进同一个字符串，本章的图脚本统一遵守“中文纯文本、公式单独成串”，所以不该再出现；若出现，去查那一行的字符串写法，不要去改字体配置。

最后一个提醒：所有随机量统一种子（`default_rng(20261101)`），换种子会让 ELM 的随机特征、网络宽度扫描的 IAE、Shapley 的占比都变。正文的数字是这一组种子下的结果，**换种子后要重跑全部十段并重写表**，不能只改一处。

数值自检：三个能立刻发现错误的检查

跑完十段之后，先做三个不依赖正文的检查，它们能挡住绝大多数实现错误，而且每个都只需要几秒钟。

一、教师自身的闭环要落在配置的极点上。把 y 固定在工作区中点，令参考为 0、初值为一个小扰动，按闭环矩阵 $A-BK$ 算特征值，应当就是配置的 $(-3,-4)$。若对不上，多半是符号约定错了：把 $u=-\sum_i w_i K_i x$ 里的负号漏掉，闭环就变成 $A+BK$，极点跑到右半平面，仿真会发散。这类错误在只看曲线时很像“增益太大”，很容易被误诊成调参问题。

二、无扰动时的末段误差应当接近机器精度。教师与双线性查表的末段 RMS 都是 0.0000（表 71-5 末两行），因为它们没有量化环节。若你的实现里教师也出现了 10^{-3} 量级的稳态误差，先查积分步长是否与采样周期一致，再查参考是否被一阶滤波——本章的参考是纯阶跃，任何平滑都会引入滞后，被误读成稳态误差。

三、把限幅触发的拍数打印出来。本章阶跃幅度 0.5、静态增益 13 到 20，峰值控制量约 1.1，紧贴 1.2 的限幅，实际只在最初两三拍触发。若打印出来是几十拍，说明增益配得过大或参考给得过猛；此时 §71.4 那条“证书不覆盖饱和区”的警告已经在生效，继续读 α 的数值没有意义。

换一个对象：改动清单只有五处

本章的对象是 $y''=-2y'+K(y)u$。若换成一个别的单输入单输出对象（带惯性的温度回路、带摩擦的速度回路都行），要改的只有下面五处，其余代码不动。

- `A`：状态矩阵。本章取 $\begin{bmatrix}0 & 1\\ 0 & -2\end{bmatrix}$，第二行是阻尼项。换成近似无阻尼的对象时这一行变成 $[0,0]$，`place_poles` 仍能配出极点，但同样的判据会给出更小的延迟余量。
- `bmat(y)` 与 `gain(y)`：输入矩阵与静态增益。两处定义必须一致，否则局部模型的 B_i 与真实对象对不上，PDC 的插值就失去物理含义，只剩数值上的“看着还行”。
- `y_grid`：规则中心。惯例是覆盖整个工作区，并在两端各外扩一格，这样输出越界时端点的隶属度仍能算出合理值；不外扩会在边界出现权重全零，代码里只能用“最近格点”兜底。
- `TS`、`DELAY`、`UMAX`：采样周期、纯延迟拍数、限幅。三者都得按实际硬件填，`DELAY` 尤其不能照抄本章的 5 拍——它直接决定表 71-4 的拐点位置。
- 随机种子：只影响 ELM 的随机特征与网络宽度扫描，但换了就要重跑全部十段。

改完之后，凡是与对象无关的结论（判据形式、量化与抖振的对应、Shapley 的效率性）仍然成立；凡是与对象有关的数字（具体 α、延迟余量、IAE）都要重算重写。把这两类结论在正文里分开写，是本章从第 6 节起一直遵守的规矩。

与第 66 至 69 章的数字怎么对照

本章沿用同一对象（式 71-1、10 ms、5 拍、± 1.2），所以第 66 到 69 章的 IAE 与本章的表可以直接比，但比之前要先过三条。

阶跃幅度要相同。四章都是 1.5 到 2.0，本章也一样；若某章用了 1.5 到 2.2 的算例，那个 IAE 不能直接对照，因为误差积分随幅度近似线性增长。

延迟的实现方式要相同。本章用“缓冲区里放 5 个数”，若某章用一阶惯性近似延迟，两者的相位滞后在低频接近、在高频差很多，抖振指标不可比。

限幅的触发情况要相同。本章峰值约 1.1，只在最初两三拍触发；若某章的控制器增益更大、长期贴着限幅跑，它的 IAE 里就混进了饱和的影响。

三条合起来的意思是：**同一对象不等于同一实验**。本章把对象与采样对齐到前四章，是为了让“表示、判据、安全层”这三个新变量成为唯一差别；对照时若发现某个数字差得离谱，先查这三条，再怀疑方法。

十段实验里，第 1 段（表示对照）与第 6 段（安全层）最能直接与前四章比；第 2、3、5 段讲的是判据与量化，前四章没有对应项，只能自己与自己比。把“能横向比的”与“只能纵向比的”分开，是写对照表时最省口舌的一条。

三步的耗时也值得记：仿真约 16 s、出图约 14 s、build 半分钟左右。这三个数决定了“改一个参数要等多久”，也就是调参时试错循环的长度。若把十段拆成可单独运行的子命令，单段通常在两秒内，循环能从分钟级压到秒级；本章为了保持“一次跑出全部数字”的可核对性没有拆，读者按需自行拆即可——拆的时候注意让每段自己打印所用的参数，否则半年后又说不清那张表是哪组参数跑出来的。